Н. Секунов

САМОУЧИТЕЛЬ VISUAL C++ 6

Книга посвящена методам обьектно-ориентированного программирования для 32-разрядных операционных систем Windows. Рассмотрен широкий круг вопросов создания приложений: консольного, диалогового и многооконного. Тексты соответствующих примеров, которые можно найти на прилагаемой к данной книге дискете, помогут пользователю, не имеющему опыта работы в среде Visual C++, за короткий срок создать собственное работоспособное приложение. Рассмотрены вопросы построения справочной системы и русификации приложения. Кроме того, большое внимание уделено таким особенностям программирования, как применение механизма исключений для обработки ошибок, работа с шаблонами, многозадачность и взаимодействие потоков. Приведен обширный справочный материал по функциям и классам Visual C++. Кроме новичков, книга представит несомненный интерес для программистов, имеющих опыт работы с DOS и осваивающих программирование в среде Windows.

Содержание

Введение 1
- Для кого предназначена эта книга? 2
- Структура данной книги 4
- Соглашения, принятые в данной книге 7
- Требования к аппаратным средствам и программному обеспечению 9

Часть I. Создание простейших приложений 11

Глава 1. Использование мастера создания приложений 13

Консольное приложение 13

Создание консольного приложения 14

Использование консольного приложения 21

Диалоговое приложение 21

Многооконное приложение Windows 27

Однооконное приложение 37

Глава 2. Классы приложений, документов и представлений 39

Класс документа 39

Класс представления 44

Класс приложения 50

Дизайн элементов управления и системный реестр 55

Шаблон документа 56

Создание окон 57

Глава 3. Диалоговые окна и простейшие элементы управления 59

Диалоговое окно 59

Формирование ресурсов диалогового окна 60

Создание класса диалога 71

Вкладки и мастера 85

Создание вкладок диалогового окна 85

Создание мастера 92
- Некоторые модификации окна мастера 97

Использование Component Gallery 101
Часть II. Программирование интерфейса пользователя 113
Глава 4. Классы элементов управления 115
Класс списка 115
Классы линейного регулятора и линейного индикатора 123
Создание пользовательского линейного индикатора 133
Класс просмотрового окна списка 138
Копирование демонстрационного приложения 139
Анализ демонстрационного приложения 143
Глава 5. Сообщения и команды 171
Обработка сообщений 171
Карта сообщений 173
Сообщения в Windows 3.x 176
Сообщения в Win32 176
Сообщения, посылаемые всеми новыми элементами управления Windows 177
Обработка извещений 178
Обработка отраженных сообщений 180
Макросы карты сообщений и заготовки функций обработки отраженных сообщений 181
Пример создания элемента управления, обрабатывающего отраженные сообщения 187
Использование карты сообщений приложением 195
Использование мастера ClassWizard для создания функций обработки сообщений 196
Диалоговое окно MFC ClassWizard 196
Добавление обработчиков сообщений Windows 200
Список сообщений 201
Обновление команд 202
Глава 6. Вывод информации на экран 205
Интерфейс графических устройств (GDI) 205
Контекст устройства 206
Отображение текста 209
Шрифты 209
Работа с текстом 216
- Форматирование текста 216
- Вывод текста 219
- Программа вывода текста 220

Перерисовка окна 222
Использование перьев 224
Работа с кистью 229
Настройка параметров в диалоговом окне 232

Работа с битовыми образами 236
Аппаратно-зависимые битовые образы 237
Глава 7. Работа с файлами документов 245
Работа с архивом 245
Непосредственное чтение и запись файлов 254
Использование объектов класса CFile при работе с классом CArchive 255
Автономное использование класса CFile 259
Работа с системным реестром 267
Глава 8. Работа с текстовыми документами 271
Создание простейшего текстового редактора 271
Создание более сложного редактора 273
Форматирование документов 278
Задание пользовательского шрифта 284
Глава 9. Панели инструментов и строка состояния 288
Работа с панелью инструментов 288
Удаление кнопок из панели инструментов 289
Добавление кнопок в панель инструментов 291
Работа со строкой состояния 305
Глава 10. Печать документов и организация прокрутки в окне 311
Организация прокрутки в окне 311
Режимы отображения 317
Распечатка и предварительный просмотр 319
Работа с окном предварительного просмотра печати 319
Распечатка многостраничного документа 324
Использование функций библиотеки MFC при печати 333
Часть III. Особенности программирования в среде Visual C++ 339
Глава 11. Исключения, шаблоны и новые возможности Visual C++ 341
Исключения 341
Аргументы исключений 342
Механизмы исключений Visual C++ 344
- В каких случаях следует вызывать исключения 345
- Перехват и уничтожение исключений 347
- Уничтожение объектов в исключениях 349
- Вызов исключений из функций пользователя 351
- Преобразование макросов исключений в операторы C++ 352
- Совместное использование макросов и операторов C++ 356

Шаблоны 358
Понятие шаблона 359
Шаблоны функций 362
Шаблоны классов 363
Классы коллекций 366

Виды классов коллекций 367
Классы коллекций, использующие шаблоны 370
Классы коллекций, не использующие шаблоны 375
Доступ к элементам классов коллекций 376
Удаление элементов классов коллекций 378
Использование классов коллекций 380
Глава 12. Многозадачность на основе потоков Windows 383
Работа с одним потоком 384
Создание рабочего потока 384
Создание интерфейсных потоков 386
Прекращение работы потока 388
Взаимодействие между потоками 389
Взаимодействие между потоком и приложением 390
Использование классов синхронизации 390
Примеры приложений, работающих с потоками 392
Приложение MTGDI 393
Приложение MUTEXES 409
Глава 13. Справка в приложении 423
Описание справочной системы приложения 423
Способы доступа к справочной системе 424
Способы представления справочной информации 425
Формы представления справочной информации 436
Программирование справочной системы 437
Компоненты справочной системы 438
Обработка сообщений справочной системы 439
Русификация файла ресурсов 444
Создание системы командной справки 462
Создание системы контекстной справки 465
Подготовка справочных текстов 469
Русификация справочной системы приложения 470
Добавление новых тем 493
Внесение изменений в оглавление справочной системы 497
Диалоговое окно О программе 499
Глава 14. Отладка приложения 501
Средства отладки, предоставляемые интерфейсом пользователя 501
Точки останова 503
Анализ значений переменных 504
Настройка уровня предупреждений транслятора 506
Программные средства отладки 507
Макросы ASSERT и TRACE 508
Отладочные функции 510
Устранение утечки памяти 511
Основные причины утечек памяти 511
Отладочные версии операторов new и delete 515

Особенности отладки приложений, использующих шаблоны 516
Часть IV. Приложения 517
Приложение A. Объектно-ориентированное программирование и классы 519
Обзор объектно-ориентированных методов программирования 519
- Инкапсуляция 521
- Наследование 522
- Полиморфизм 522

Классы 523
- Классы как типы данных 523
- Файлы заголовков и файлы реализации 524
- Когда следует использовать классы 532
- Перегрузка функций и операторов 532
- Использование виртуальных функций 534
- Область действия класса 535

Приложение B. Описание классов и функций Visual C++ 537
Глобальные функции Windows 537
- AddFontResource 537
- AfxBeginThread 538
- AfxEnableControlContainer 539
- AfxEndThread 540
- AfxGetApp 540
- AfxGetInstanceHandle 540
- AfxGetResourceHandle 541
- AfxGetAppName 541
- AfxRegisterWndClass 541
- AfxWinInit 542
- CFHookProc 544
- ChooseFont 546
- CloseHandle 547
- CreateEvent 547
- CreateFontIndirect 549
- DeleteCriticalSection 550
- EnterCriticalSection 550
- EnumFontFamilies 551
- EnumFontFamiliesEx 552
- EnumFontFamProc 554
- GdiFlush 555
- GetCommandLine 556
- GetCurrentDirectory 556
- GetCurrentProcess 557
- GetDlgItem 558
- GetExitCodeThread 558
- GetLastError 559

- GetMenuContextHelpId 560
- GetMessage 560
- GetModuleHandle 562
- GetWindowContextHelpId 563
- GetWindowLong 563
- GetWindowRect 565
- InitializeCriticalSection 565
- LeaveCriticalSection 566
- MessageBox 567
- OpenEvent 570
- PulseEvent 571
- RegisterClass 572
- ReleaseCapture 573
- RemoveFontResource 573
- ResetEvent 574
- SendMessage 574
- SetCurrentDirectory 575
- SetEvent 576
- SetMenuContextHelpId 577
- SetPriorityClass 577
- SetThreadPriority 578
- SetWindowContextHelpId 580
- SetWindowLong 581
- Sleep 583
- WaitForSingleObject 583
- WindowProc 585
- WinHelp 585

Класс CArchive 589

- CArchive::operator >> 590
- CArchive::operator << 591
- CArchive::Flush 591
- CArchive::GetFile 592
- CArchive::IsLoading 592
- CArchive::IsStoring 592
- CArchive::Read 592
- CArchive::Write 593

Класс CArray 593

- CArray::Add 594
- CArray::GetSize 595
- CArray::GetUpperBound 595
- CArray::operator [ ] 595
- CArray::RemoveAll 596
- CArray::SetSize 596

Класс CBitmap 597

- CBitmap::CreateCompatibleBitmap 597
- CBitmap::GetBitmapBits 598
- CBitmap::SetBitmapBits 598

Класс CButton 599

- CButton::GetState 600
- CButton::SetCheck 600

Класс CCmdUI 601

- CCmdUI::ContinueRouting 602
- CCmdUI::Enable 602
- CCmdUI::SetCheck 603

Класс CCommandLineInfo 603

- CCommandLineInfo::ParseParam 603

Класс CControlBar 604

- CControlBar::EnableDocking 605

Структура CCreateContext 606

Класс CDC 607

- :Attach 608
- :BitBlt 609
- :CreateCompatibleDC 611
- :Detach 612
- :EndDoc 612
- :EndPage 613
- :FillRect 613
- :GetDeviceCaps 614
- :GetSafeHdc 618
- :GetTextExtent 619
- :IsPrinting 619
- :Rectangle 619
- :SelectClipRgn 620
- :SelectObject 622
- :SetMapMode 623
- :SetViewportOrg 624
- :StartDoc 625
- :StartPage 626

Класс CDialog 626

- CDialog::CreateIndirect 628
- CDialog::InitModalIndirect 629
- CDialog::DoModal 630
- CDialog::OnInitDialog 631

Класс CDocTemplate 631

- CDocTemplate::GetDocString 632

Класс CDocument 634

- CDocument::GetFile 635
- CDocument::GetPathName 636

- CDocument::OnNewDocument 636
- CDocument::ReleaseFile 636
- CDocument::SetModifiedFlag 637
- CDocument::UpdateAllViews 637

Класс CEdit 638
- CEdit::CharFromPos 640

Класс CEditView 640
- CEditView::FindText 641
- CEditView::GetBufferLength 642
- CEditView::GetEditCtrl 642
- CEditView::OnFindNext 643
- CEditView::OnTextNotFound 643
- CEditView::SerializeRaw 643

Класс CFile 644
- CFile::CFile 644
- CFile::Close 646
- CFile::GetFilePath 647
- CFile::GetLength 647
- CFile::Read 647
- CDocument::ReleaseFile 648
- CFile::Seek 648
- CFile::SeekToBegin 649
- CFile::Write 649

Класс CFont 650
- CFont::CreateFontIndirect 650

Класс CFrameWnd 651
- CFrameWnd::Create 653
- CFrameWnd::DockControlBar 654
- CFrameWnd::EnableDocking 655
- CFrameWnd::LoadBarState 656
- CFrameWnd::LoadFrame 656
- CFrameWnd::SaveBarState 657
- CFrameWnd::ShowControlBar 657

Класс CGdiObject 658
- CGdiObject::DeleteObject 658

Класс CHeaderCtrl 658
- CHeaderCtrl::GetItem 659
- CHeaderCtrl::SetImageList 659
- CHeaderCtrl::SetItem 660

Структура CHOOSEFONT 660

Класс CImageList 665
- CImageList::Add 666
- CImageList::BeginDrag 667
- CImageList::Create 667

- CImageList::DragEnter 669
- CImageList::DragLeave 670
- CImageList::DragMove 670
- CImageList::DragShowNolock 670
- CImageList::EndDrag 671
- CImageList::SetDragCursorImage 671

Класс CList 671

- CList::AddHead 672
- CList::AddTail 672
- CList::GetAt 673
- CList::GetHead 673
- CList::GetHeadPosition 674
- CList::GetNext 674
- CList::GetPrev 675
- CList::GetTail 676
- CList::GetTailPosition 676
- CList::IsEmpty 677
- CList::RemoveHead 677
- CList::RemoveTail 677
- CList::RemoveAll 678
- CList::RemoveAt 678
- CList::SetAt 678

Класс CListBox 678

- CListBox::AddString 680
- CListBox::DeleteString 681
- CListBox::GetCurSel 681
- CListBox::GetText 681
- CListBox::InsertString 682
- CListBox::SetCurSel 683
- CListBox::SetSel 683

Класс CListCtrl 684

- CListCtrl::ApproximateViewRect 687
- CListCtrl::Create 688
- CListCtrl::CreateDragImage 689
- CListCtrl::GetHeaderCtrl 690
- CListCtrl::GetHotCursor 690
- CListCtrl::GetItemPosition 691
- CListCtrl::GetItemText 691
- CListCtrl::GetOrigin 692
- CListCtrl::HitTest 692
- CListCtrl::InsertColumn 693
- CListCtrl::InsertItem 694
- CListCtrl::SetExtendedStyle 695
- CListCtrl::SetItem 695

- CListCtrl::SetItemPosition 696
- CListCtrl::SetItemText 697
- CListCtrl::SetWorkAreas 697

Класс CMap 697

- CMap::GetNextAssoc 698
- CMap::GetStartPosition 699
- CMap::IsEmpty 699
- CMap::Lookup 699
- CMap::operator [ ] 700
- CMap::RemoveAll 700

Класс CMenu 700

- CMenu::AppendMenu 701

Класс CMutex 703

- CMutex::CMutex 704

Класс CObArray 705

- CObArray::Add 705
- CObArray::GetSize 706
- CObArray::GetUpperBound 707
- CObArray::operator [ ] 707
- CObArray::RemoveAll 708
- CObArray::SetSize 708

Класс CObject 709

- CObject::AssertValid 709
- CObject::Dump 710
- CObject::Serialize 711

Класс CPoint 712

Класс CPrintInfo 712

- CPrintInfo::SetMaxPage 713

Класс CProgressCtrl 713

- CProgressCtrl::Create 714
- CProgressCtrl::SetPos 715
- CProgressCtrl::SetRange 715

Класс CPropertyPage 716

- CPropertyPage::OnSetActive 716
- CPropertyPage::OnWizardBack 716
- CPropertyPage::OnWizardFinish 717
- CPropertyPage::OnWizardNext 717

Класс CPropertySheet 717

- CPropertySheet::AddPage 718
- CPropertySheet::Construct 719
- CPropertySheet::CPropertySheet 720
- CPropertySheet::DoModal 720
- CPropertySheet::SetFinishText 721
- CPropertySheet::SetWizardButtons 721

- CPropertySheet::SetWizardMode 722

Класс CRect 722

- CRect::NormalizeRect 723
- CRect::OffsetRect 724
- CRect::PtInRect 724
- CRect::SetRect 725

Класс CRichEditView 725

- CRichEditView::OnCharEffect 726
- CRichEditView::GetCharFormatSelection 726
- CRichEditView::OnParaAlign 727
- CRichEditView::SetCharFormat 727

Класс CRgn 727

- CRgn::CreateRectRgnIndirect 728

Класс CScrollView 728

- CScrollView::SetScrollSizes 730

Класс CSemaphore 732

- CSemaphore::CSemaphore 732

Класс CSingleLock 733

- CSingleLock::IsLocked 734
- CSingleLock::Lock 734
- CSingleLock::CSingleLock 734
- CSingleLock::Unlock 735

Класс CSize 735

Класс CSliderCtrl 736

- CSliderCtrl::ClearTics 736
- CSliderCtrl::Create 736
- CSliderCtrl::GetPos 738
- CSliderCtrl::SetLineSize 738
- CSliderCtrl::SetPageSize 738
- CSliderCtrl::SetPos 739
- CSliderCtrl::SetRange 739
- CSliderCtrl::SetTic 739
- CSliderCtrl::SetTicFreq 740

Класс CStatusBar 740

- CStatusBar::CommandToIndex 741
- CStatusBar::Create 741
- CStatusBar::SetIndicators 742
- CStatusBar::SetPaneInfo 742
- CStatusBar::SetPaneText 743

Класс CString 743

- CString::Empty 745
- CString::Format 745
- CString::GetBuffer 745
- CString::IsEmpty 747

- CString::LoadString 747
- CString::MakeReverse 747

Класс CTime 748

- CTime::GetCurrentTime 748

Класс CTimeSpan 749

- CToolBar 749
- CToolBar::CreateEx 751
- CToolbar::LoadToolBar 752

Класс CView 752

- CView::DoPreparePrinting 753
- CView::GetDocument 754
- CView::OnBeginPrinting 754
- CView::OnDraw 755
- CView::OnInitialUpdate 755
- CView::OnPrepareDC 756
- CView::OnPreparePrinting 757
- CView::OnPrint 758
- CView::OnUpdate 759

Класс CWinApp 760

- CWinApp::AddDocTemplate 761
- CWinApp::Enable3dControls 761
- CWinApp::ExitInstance 762
- CWinApp::GetProfileString 763
- CWinApp::InitInstance 764
- CWinApp::LoadCursor 764
- CWinApp::LoadIcon 765
- CWinApp::LoadStdProfileSettings 765
- CWinApp::OnHelp 766
- CWinApp::ParseCommandLine 766
- CWinApp::ProcessShellCommand 767
- CWinApp::RegisterShellFileTypes 768
- CWinApp::SetRegistryKey 768
- CWinApp::WinHelp 769
- CWinApp::WriteProfileString 770

Класс CWinThread 771

- CWinThread::CreateThread 772
- CWinThread::ResumeThread 773
- CWinThread::Run 773
- CWinThread::SetThreadPriority 774
- CWinThread::SuspendThread 774

Класс CWnd 775

- CWnd::BeginPaint 776
- CWnd::DestroyWindow 776
- CWnd::DoDataExchange 777

- CWnd::EnableWindow 779
- CWnd::EndPaint 779
- CWnd::GetClientRect 780
- CWnd::GetDC 780
- CWnd::GetDlgItem 781
- CWnd::GetParent 781
- CWnd::GetParentFrame 782
- CWnd::GetParentOwner 782
- CWnd::GetSystemMenu 783
- CWnd::GetStyle 783
- CWnd::GetWindowRect 784
- CWnd::Invalidate 784
- CWnd::InvalidateRect 785
- CWnd::IsDlgButtonChecked 785
- CWnd::ModifyStyle 786
- CWnd::ModifyStyleEx 786
- CWnd::OnCommandHelp 787
- CWnd::OnContextMenu 788
- CWnd::OnHelpHitTest 788
- CWnd::OnHelpInfo 789
- CWnd::OnHScroll 789
- CWnd::OnMouseMove 790
- CWnd::OnPaint 791
- CWnd::PreCreateWindow 792
- CWnd::RedrawWindow 792
- CWnd::ReleaseDC 794
- CWnd::ScreenToClient 795
- CWnd::SetDlgItemText 795
- CWnd::SetScrollPos 795
- CWnd::SetWindowText 796
- CWnd::ShowWindow 796
- CWnd::UpdateData 797
- CWnd::UpdateWindow 798

Структура CHARFORMAT 798
Структура ENUMLOGFONT 800
Структура HDITEM 800
Структура HELPINFO 803
Структура LOGFONT 803
Структура LVITEM 809
Структура NEWTEXTMETRIC 811
Структура NMHDR 814
Структура OUTLINETEXTMETRIC 815
Структура PAINTSTRUCT 818
Структура POINT 819

Структура RECT 819
Структура SIZE 820
Структура WNDCLASS 820
Приложение С. Средства для работы с классами Visual C++ 824
Работа с базой данных мастера ClassWizard 825
Обновление базы данных при изменении текста программы 825
- Уничтожение классов 825
- Переименование и перемещение классов 826

Создание нового файла базы данных мастера ClassWizard 827
Добавление нового класса 828
Создание классов, имеющих прототипы в библиотеке MFC 829
Подключение ресурса к классу 832
Добавление поддержки библиотеки ATL в приложение MFC 833
Создание пользовательского класса, не имеющего прототипов в библиотеках 834
Создание функций обработки сообщений 835
Создание функций обработки сообщения с использованием панели ClassView окна Workspace или панели инструментов WizardBar 836
- Быстрый способ создания функций обработки сообщения для элементов управления диалогового окна, определенных для него по умолчанию 838
- Быстрый способ создания функций обработки сообщения для элементов управления диалогового окна 840

Создание функции обработки сообщения при помощи мастера ClassWizard 841
- Быстрый способ создания функций обработки сообщения для кнопок диалогового окна 843

Создание функций обработки для отраженного сообщения 844
- Создание функций обработки для отраженного сообщения при помощи мастера ClassWizard 844
- Создание функций обработки для отраженного сообщения при помощи панели инструментов WizardBar 845

Уничтожение функций обработки сообщений 846
Редактирование функций обработки сообщений 847
Работа с переменными класса диалогового окна 848
Использование мастера ClassWizard для включения переменных в класс диалогового окна 849
Функции DDX_ и DDV_ 851
- Принцип работы динамического обмена и проверки данных 852

- Расширение пользователем возможностей данного механизма 853
- Использование мастера ClassWizard 854

Приложение D. Интерфейс пользователя Visual C++ 857
Панели инструментов 858
Панель инструментов Standard 862
Панель инструментов Build 863
Система меню 864
Меню File 865

- Команда File, New (<Ctrl>+<N>) 866
- Команда File, Open (<Ctrl>+<O>) 866
- Команда File, Close 867
- Команда File, Open Workspace 867
- Команда File, Save Workspace 868
- Команда File, Close Workspace 868
- Команда File, Save (<Ctrl>+<S>) 868
- Команда File, Save As 868
- Команда File, Save All 869
- Команда File, Page Setup 869
- Команда File, Print (<Ctrl+P>) 870
- Команда File, Recent Files 870
- Команда File, Recent Workspaces 871
- Команда File, Exit 871

Меню Edit 871

- Команда Edit, Undo (<Ctrl>+<Z>) 871
- Команда Edit, Redo (<Ctrl>+<Y>) 872
- Команда Edit, Cut (<Ctrl>+<X>) 873
- Команда Edit, Copy (<Ctrl>+<C>) 873
- Команда Edit, Paste (<Ctrl>+<V>) 873
- Команда Edit, Delete (<Del>) 873
- Команда Edit, Select All (<Ctrl+A>) 873
- Команда Edit, Find (<Ctrl>+<F>) 873
- Команда Edit, Find in Files 875
- Команда Edit, Replace (<Ctrl>+<H>) 878
- Команда Edit, Go To (<Ctrl>+<G>) 879
- Команда Edit, Bookmarks (<Alt>+<F2>) 880
- Команда Edit, Advanced 881
- Команда Edit, Breakpoints (<Alt>+<F9>) 882

Меню View 882

- Команда View, ClassWizard (<Ctrl>+<W>) 883
- Команда View, Resource Symbols 883
- Команда View, Resource Includes 884
- Команда View, Full Screen 885
- Команда View, Workspace (<Alt>+<0>) 886

- Команда View, Output (<Alt>+<2>) 886
- Команда View, Debug Windows 886
- Команда View, Properties (<Alt>+<Enter>) 887

Меню Insert 887

- Команда Insert, New Class 887
- Команда Insert, New Form 889
- Команда Insert, Resource (<Ctrl>+<R>) 889
- Команда Insert, Resource Copy 890
- Команда Insert, New ATL Object 890

Меню Project 890

- Команда меню Project, Set Active Project 890
- Команда меню Project, Add to Project 891
- Команда Project, Dependencies 893
- Команда Project, Settings (<Alt>+<F7>) 893

Меню Build 896

- Команда Build, Compile (<Alt>+<F7>) 896
- Команда Build, Build (<F7>) 897
- Команда Build, Rebuild All 897
- Команда Build, Batch Build 898
- Команда Build, Clean 898
- Команда Build, Start Debug 898
- Команда Build, Debugger Remote Connection 899
- Команда Build, Execute (<Ctrl>+<F5>) 900
- Команда Build, Set Active Configuration 900
- Команда Build, Configurations 900
- Команда Build, Profile 901

Меню Debug 901

- Команда Debug, Go (<F5>) 901
- Команда Debug, Restart (<Ctrl>+<Shift>+<F5>) 902
- Команда Debug, Stop Debugging (<Shift>+<F5>) 902
- Команда Debug, Step Into (<F11>) 902
- Команда Debug, Step Over (<F10>) 902
- Команда Debug, Step Out (<Shift>+<F11>) 902
- Команда Debug, Run to Cursor (<Ctrl>+<F10>) 902
- Команда Debug, Exceptions 902
- Команда Debug, Threads 903
- Команда Debug, Modules 904
- Команда Debug, QuickWatch (<Shift>+<F9>) 904

Меню Tools 905

- Команда Tools, Customize 906
- Команда Tools, Source Browser (<Alt>+<F12>) 906
- Команда Tools, Options 906
- Команда Tools, Macro 908

Меню Window 908
- Команда Window, New Window 908
- Команда Window, Split 908
- Команда Window, Docking View (<Alt>+<F6>) 910
- Команда Window, Close 910
- Команда Window, Close All 910
- Команда Window, Next 910
- Команда Window, Previous 910
- Команда Window, Cascade 910
- Команда Window, Tile Horizontally 910
- Команда Window, Tile Vertically 912
- Список открытых окон 912
- Команда Window, Windows 913

Меню Help 914
- Команда Help, Contents 914
- Команда Help, Search 914
- Команда Help, Index 914
- Команда Help, Keyboard Map 916
- Команда Help, Tip of the Day 917
- Команда Help, Technical Support 917
- Команда Help, Microsoft on the Web 919
- Команда Help, About Visual C++ 919

Окна Visual C++ 919
Окно Workspace 919
- Вкладка ClassView 919
- Вкладка ResourceView 924
- Вкладка FileView 925

Окно Output 926
Редактирование текстов программ 927
Предметный указатель **929**
Описание дискеты 942

Предметный указатель

А

Аргументы по умолчанию 533
Архив 245
Атрибуты
  прав доступа 527
  устройств. 206

Б

Битовый образ 236, 237
Блок
  catch 341, 348, 721
  try 341
Бордюр 861

Г

Галерея компонентов 101
Глобальные функции Windows 537

Д

Демонстрационное приложение
  BrushApp 229
  Console 14
  DDBApp 237
  DialogApplication 22, 60
  EditApp 271

LineApp 225
Gallery 103
ListApp 115
MFC ClassWizard 841
MTGDI 393
PrintApp 311
ProgressApp 123
Reflect 187
RichApp 279
SheetApplication 85
TextApp 220, 245
ToolApp 290
Дескриптор HINSTANCE 540
Деструктор класса 528
Диалоговое окно
Add Member Function 75, 841
Add Member Variable 72, 117, 126, 190, 193, 237, 312, 849
Adding a Class 86, 189
Advanced Options 32
Batch Build 898
Bookmarks 880
Breakpoints 503, 882
Browse Files 274
Change Files 274
Check Box Properties 125
Components and Controls Gallery 103
Configurations 900
Customize 906
Dialog Properties 61, 86, 274
Edit Properties 274
Exceptions 902
Find 873
Find In Files 875, 886, 926
Go To 879
Group Box Properties 234
Insert Files into Project 102
Insert Resource 87, 292, 889
Insert Resource Copy 890
Keyboard Map 916
Macro 908
Menu Item Properties 108, 273, 281, 285
MFC AppWizard 272, 288, 311, 499
MFC ClassWizard 72, 196
Module List 904
New 14, 43, 464, 866
New Class 190, 830, 887
New Form 889
New Project Information 16, 37
New Symbol 306
New Windows Message and Event Handlers for class CDialogApplicationDlg 200
New Windows Message and Event Handlers for class CMyListCtrl 183
Open 249, 252, 264, 866
Open Workspace 102, 143, 867
Options 906
Page Setup 869
Paste Special 279
Picture Properties 125
Print 870
Progress Properties 124
Project Dependencies 893
Project Settings 344, 506, 893
Push Button Properties 117
QuickWatch 505, 904
Repair Class Information 825
Replace 878
Resource Includes 884
Resource Symbols 306, 883
Save As 248, 252, 264, 868
Select Class 294, 833
Select Source File 827
Set Active Configuration 900
Set Active Project Configuration 142, 396
Slider Properties 124
String Properties 250, 306
Text Properties 274
Threads 903
Tip of the Day 917
Toolbar Button Properties 293
Toolbar Properties 293

Windows 913
Запуск программы 268
Найденные разделы 433
Шрифт 213, 544-546, 661
Динамическая проверка корректности данных 848
Динамический обмен данными в диалоговом окне 848
Документы
составные 29
разбиение на страницы 324
Дюйм логический 219
И
Иерархия классов 522
Изменение положения элемента управления 63
Инкапсуляция 520, 521
Интерфейс
графических устройств 49, 205, 607
многооконный 27
однооконный 37, 652
Исключение 341
CFileException 648
STATUS_NO_MEMORY 566
К
Карта
дескрипторов 195
сообщений 78, 173
отображений 367
Кисть 229
Класс
CArchive 44, 245, 589
CArchive Exception 346
CArray 371, 593
CBitmap 242, 597
CBitmapButton 599
CBrush 136
CButton 599
CByteArray 706
CClientDC 207, 607
CCmdTarget 173, 601, 836
CCmdUI 601
CComboBox 417
CCommandLineInfo 57, 603, 766
CControlBar 604
CCriticalSection 392
CDaoException 346
CDataExchange 852
CDBException 346
CDC 242, 607
CDialog 626
CDocTemplate 631, 652
CDocument 39, 634
CDumpContext 510, 594
CDWordArray 706
CEdit271, 638
CEditView 640
CEvent 392
CFile 255, 589, 644
CFileException 346
CFont 332, 650
CFrameWnd 57, 651
CGdiObject 658
CHeaderCtrl 161, 658
CImageList 665
CList 371, 380, 671
CListBox 678
CListCtri 138, 684
CMap 371, 380, 697
CMapPtrToPtr 373
CMapPtrToWord 373
CMapStringToOb 373
CMapStringToPtr 373
CMapWordToPtr 373
CMemoryException 346
CMenu 700
CMetaFileDC 207
CMeteFileDC 607
CMultiDocTemplate 56, 632
CMultiLock 392, 704
CMutex 392, 409, 703
CNotSupportedException 346
CObArray 372, 705
CObject 370, 709
CObList 372, 375
COleDispatchException 346
COleException 346

CPaintDC 207, 607
CPoint 402, 712
CPrintInfo331, 712
CProgressCtrl 713
CPropertyPage 716
CPropertySheet 717
CPtrArray-370, 372, 706
CPtrList 370, 372
CRect613, 722
CResourceException 346
CRgn 137, 727
CRichEditView271, 725
CScrollBar 132
CScrollView 728
CSemaphore 392, 732
CSingleDocTemplate 56, 632
CSingleLock 392, 409, 704, 733
CSize 317, 619, 735
CSliderCtrl 736
CStatusBar 740
CString 743
CStringArray 706
CTime 748
CTimeSpan 749
CToolbar 302
CTypedPtrArray 372
CTypedPtrList 372
CTypedPtrMap 373
CUIntArray 706
CUserException 346
CView 39, 46, 752
CWinApp 20, 54, 195, 384, 760
CWindowDC 207, 607
CWinThread 54, 384, 539, 771
CWnd 775
CWordArray 706
приложения 50
приоритета процесса 578
синхронизации
доступа 392
работы 392
Кнопка
<Назад 718
About 289
Build 863
Compile 863
Copy 289, 863
Cut 289, 863
Execute Program 864
Find in Files 863
Finish 272
Go 864
Help 569
Insert/Remove Breakpoint 503, 864
Keep Visible 61
New 289
New Text File 862
Open 249, 289, 862
Output 863
Paste 289, 863
Print 289
Redo 863
Save 289, 862
Save All 863
Search 863
Stop Build 863
Undo 863
Window List 863
Workspace 863
Готово 718
Далее> 718
Закрыть 247
Открыть 249
Применить 718
Сохранить 248
Команда
HELP_COMMAND 586
HELP_CONTENTS 431, 586
HELP_CONTEXT 587
HELP_CONTEXTMENU 431, 587
HELP_CONTEXTPOPUP 432, 587
HELP_FINDER 430, 587
HELP_FORCEFILE 587
HELP_HELPONHELP 587
HELP_INDEX431, 587
HELP_KEY 433, 588
HELP_MULTIKEY 433, 588

HELP_PARTIALKEY 433, 588
HELP_QUIT 436, 588
HELP_SETCONTENTS 588
HELP_SETPOPUP_POS 588
HELP_SETWINPOS 435, 442, 589
HELP_TCARD 435, 589
HELP_WM_HELP 431, 440, 589
Команда меню
Build, Batch Build 898
Build, Build 897
Build, Clean 898
Build, Compile 896
Build, Configurations 900
Build, Debugger Remote Connection 899
Build, Execute 900
Build, Profile 901
Build, Rebuild All 897
Build, Set Active Configuration 900
Debug, Exceptions 902
Debug, Go 901
Debug, QuickWatch 505, 904
Debug, Restart 501, 902
Debug, Run to Cursor 502, 902
Debug, Step Into 502, 902
Debug, Step Out 502, 902
Debug, Step Over 502, 902
Debug, Stop Debugging 501, 902
Debug, Threads 903
Debug, Go 501
Debug, Modules 904
Edit, Advanced 881
Edit, Bookmarks 880
Edit, Breakpoints 503, 882
Edit, Copy 873
Edit, Cut 873
Edit, Delete 873
Edit, Find 873
Edit, Find in Files 875
Edit, Go To 879
Edit, Paste 873
Edit, Redo 872
Edit, Replace 878
Edit, Select All 873
Edit, Undo 871
File, Close 867
File, Close Workspace 868
File, Exit 871
File, New 636, 866
File, Open 249, 866
File, Open Workspace 143, 246. 867
File, Page Setup 869
File, Print 870
File, Print Preview 319
File, Recent Files 870
File, Recent Workspaces 246, 871
File, Save 868
File, Save All 869
File, Save As 868
File, Save Workspace 868
Help, About Visual C++ 919
Help, Contents 914
Help, Index 139, 914
Help, Keyboard Map 916
Help, Microsoft on the Web 919
Help, Search 914
Help, Technical Support 917
Help, Tip of the Day 917
Insert Dialog 273
Insert, New ATL Object 890
Insert, New Class 829, 887
Insert, New Form 889
Insert, Resource 889
Insert, Resource Copy 890
Project, Add to Project 891
Project, Add to Project, Components and Controls 103
Project, Add To Project, Files 102
Project, Dependencies 893
Project, Set Active Project 890
Project, Settings 344, 506, 893, 901
Start Debug 898
Tools, Customize 906
Tools, Macro 908
Tools, Options 906
Tools. Source Browser 906

View, Class Wizard 883
View, Debug Windows 886
View, Full Screen 885
View, Output 886
View, Properties 887
View, Resource Includes 884
View, Resource Symbols 306, 883
View, Workspace 886
Window, Cascade 910
Window, Close 910
Window, Close All 910
Window, Docking View 910
Window, New Window 652, 908
Window, Next 910
Window, Previous 910
Window, Split 908
Window, Tile Horizontally 910
Window, Tile Vertically 912
Window, Windows 913
Консольное приложение 13
Конструктор класса 528
Контекст устройства 206, 607
атрибуты 318
Контекстная справка 436
Концепция Документ/Представление 39
Координаты
логические 319
физические 318
M
Макрос
AND_CATCH 355
ASSERT 508
BEGIN_MESSAGE_MAP 173
CATCH 352, 354, 355
DECLARE_DYNACREATE 57, 386, 652
DECLARE_DYNAMIC 709
DECLARE_DYNCREATE 528
DECLARE_MESSAGE_MAP 173
DECLARE_SERIAL 590, 709
END_CATCH 355
END_MESSAGE_MAP 173
IMPLEMENT_DYNACREATE 57, 386
IMPLEMENT_DYNAMIC 709
IMPLEMENT_DYNCREATE 530
IMPLEMENT_SERIAL 375, 590, 698, 705, 709
MAKEINTRESOURCE 654
ON_COMMAND 174
ON_COMMAND_EX 296
ON_COMMAND_RANGE 174
ON_CONTROL 175
ON_CONTROL_RANGE 175
ON_CONTROL_REFLECT 186
ON_CONTROL_REFLECT_EX 187
ON_EDIT_CHANGE 639
ON_EN_ERRSPACE 639
ON_EN_HSCROLL 639
ON_EN_KILLFOCUS 639
ON_EN_SETFOCUS 639
ON_EN_UPDATE 639
ON_EN_VSCROLL 639
ON_ENMAXTEXT639
ON_MESSAGE 175
ON_NOTIFY 151, 178
ON_NOTIFY_REFLECT 186
ON_NOTIFY_REFLECT_EX 187
ON_NOTIFY_REFLECTION 151
ON_UPDATE_COMMAND_UI 175, 203
ON_UPDATE_COMMAND_UI_RANGE 175
ON_UPDATE_COMMAND_UI_REFLECT 186
ON_WM_CHARTOITEM_REFLECT 187
ON_WM_COMPAREITEM_REFLECT 187
ON_WM_CTLCOLOR_REFLECT 186
ON_WM_DELETEITEM_REFLECT 186
ON_WM_DRAWITEM_REFLECT 186
ON_WM_HSCROLL_REFLECT

187
ON_WM_MEASUREITEM_REFLECT 186
ON_WM_PARENTNOTIFY_REFLECT 187
ON_WM_VKEYTOITEM_REFLECT 187
ON_WM_VSCROLL_REFLECT 187
RUNTIME_CLASS 56
TCard 435
THROW 355
THROW_LAST 355
TRACE 302, 508 TRY 355
UPDATE_COMMAND_UI 652
карты сообщений 599
исключений 344
Маска обратного вызова 686
Массивы 367
Мастер
ClassWizard 71
MFC AppWizard 13
Меню
Build 896
Debug 901
Edit 871
File 865
Help 914
Insert 887
Project 890
Tools 905
View 882
Window 908
Многозадачность
кооперативная 383
вытесняющая 383
Н
Набор символов
ASCII 211
Unicode 212
Наследование 520
Начертание 209
О
Обработка сообщений 151, 179, 836, 844
Объект 524
графический 206
интерфейса графических устройств 658
Объектно-ориентированное программирование 519
Окно
MFC AppWizard 437
MSDN Library Visual Studio 6.0 139, 393
Output 886
Output 926
Variables 504
Watch 504
Workspace 886, 919
Вкладка Class View 919
Вкладка FileView 925
Вкладка Resource View 924
Редактор реестра 268
Справка 426
Справочная система 426
Оператор throw 341
Определение координат графических объектов 317
Отображение
значков 684
отчета 684
списка 684
Очередь сообщений 172
П
Панель инструментов 288
Build 863
Build MiniBar 863
Controls 62, 116, 124
Dialog 70
Standard 862, 872
WizardBar 183
WizardBar 824, 829, 836
плавающая 861
Перегрузка операторов и функций 532—534
Полиморфизм 520, 522
Полоса прокрутки

Бегунок 314
Страница 317
Строка 317
Пользовательская система координат 319
Пользовательский линейный индикатор 133
Последовательный доступ к файлу 254, 589
Потоки 383
прекращение работы 388
синхронизация 384, 390
функция исполняющая 385
Предварительный просмотр печати 319
Приложение
Help Workshop 497
RegEdit.exe 268
Приоритет
потока 578
базовый уровень 578—580
Произвольный доступ к файлу 255
Прокрутка в окне 311
Процесс 383
P
Режим EVENT_ALL_ACCESS 548
Режим отображения 623, 730
MM_LOENGLISH 723
MM_TEXT 804
C
Система координат оконная 318
Скрытие данных 521
Создание
колонтитулов 331
очереди 381
стека 381
Сообщение
BN_CLICKED 75
CBN_SELENDOK 76
EM_GETCHARFORMAT 800
EM_SETCHARFORMAT 800
EN_PROTECTED 726
EN_UPDATE 639
HDN_GETDISPINFO 801, 802
HELPMSGSTRING 664
ID_EDIT_FIND 641
ID_EDIT_PRINT 641
ID_EDIT_REPEAT 641
ID_EDIT_REPLACE 641
ID_EDIT_SELECT_ALL 641
LBN_DBLCLK 119
LBS_SELCHANGE 680
LVM_FINDITEM 811
LVM_GETITEM 810
LVM_SORTITEMS811
LVN_ENDLABELEDIT 685
LVN_GETDISPINFO 810
NM_CLICK 177
NM_DBLCLK 177
NM_KILLFOCUS 178
NM_OUTOFMEMORY 178
NM_RCLICK 177
NM_RDBLCLK 178
NM_RETURN 178
NM_SETFOCUS 178
ON_BN_CLICKED 600
ON_BN_DOUBLECKICKED 600
ON_COMMAND_EX 602
ON_UPDATE_COMMAND_UI 601
TB_LINEDOWN 738
TB_LINEUP 738
TB_PAGEDOWN 739
TB_PAGEUP 739
Translate Message 172
TTN_POP 177
TTN_SHOW 177
UPDATE_COMMAND_UI 259, 294, 295, 303
WM_CAPTURECHANGED 573
WM_CHAR 680
WM_CHOOSEFONT_GETLOGFONT 546, 661
WM_CHOOSEFONT_SETFLAGS 546
WM_CHOOSEFONT_SETLOGFONT 546
WM_CLOSE 296, 303, 420

WM_COMMAND 176, 202, 424, 439, 545, 661
WM_COMMANDHELP 441
WM_CONTEXTMENU 425, 439
WM_CTLCOLORDLG 545
WM_DESTROY 436, 777
WM_DRAWITEM 685, 702
WM_ENABLE 779
WM_ERASEBKGND 542, 793
WM_ERASEBKGND 823
WM_FONTCHANGE 214, 538, 574
WM_HELP 425, 440, 789
WM_HELPHITTEST 443
WM_HSCROLL 132, 730
WM_INITDIALOG 545, 631, 664
WM_INITMENU 783
WM_KEYDOWN 680
WM_MEASUREITEM 702
WM_MOUSEMOVE 667, 791
WM_MOUSEWHEEL 728
WM_NCDESTROY 777
WM_NCPAINT 793
WM_NOTIFY 176, 815
WM_ONERASEBKGND 776
WM_PAINT 58, 136, 208, 561, 760, 776, 784, 791
WM_PARENTNOTIFY 777
WM_QUIT 172, 560, 773
WM_SETTEXT 795
WM_SYSCOMMAND 783
WM_TCARD 435
WM_VSCROLL 730
Списки 367
недавно использованных файлов 55
панели инструментов Find 863
Способы запроса справочной информации 424
Справка
командная 437
по ключевому слову 436
по методике решения задачи 436
Стиль 209
CS_CLASSDC 542
CS_DBLCLKS 541
FWS_ADDTOTITLE 654, 656
IDC_ARROW 541
LBS_NOTIFY 679
LBS_SORT 680
LBS_WANTKEYBOARD1NPUT 680
LVS_ALIGNLEFT 684
LVS_ALIGNTOP 684
LVS_AUTOARRANGE 684, 696
LVS_EDITLABELS 684
LVS_ICON 685
LVS_LIST 685
LVS_NOCOLUMNHEADER 685
LVS_NOLABELWRAP 685
LVS_NOSCROLL 685
LVS_NOSORTHEADER 685
LVS_OWNERDRAWFIXED 685
LVS_REPORT 685
LVS_SHOWSELALWAYS 685
LVS_SINGLESEL 685
LVS_SMALLICON 685
LVS_SORTASCENDING 685
LVS_SORTDESCENDING 685
LWS_SHAREIMAGELISTS 685
PBS_SMOOTH 714
PBS_VERTICAL 714
WS_BORDER 714
WS_CHILD 714, 782
WS_CLIPCHILDREN 714, 794
WS_CLIPSIBLINGS 714
WS_DISABLED 714, 779
WS_EX_NOPARENTNOTIFY 777
WS_EX_TOPMOST 568
WS_HSCROLL 730
WS_TABSTOP 714
WS_VISIBLE 629, 714
WS_VSCROLL 730
Строка состояния 288
Строковый ресурс
редактирование 306
шаблона документа 633

Структура
CCreateContext 606, 654
CHARFORMAT 286, 726, 798
CHOOSEFONT 213, 545, 546, 660
CREATESTRUCT 792
DLGTEMPLATE 627
DOCINFO 626
ENUMLOGFONT 554, 800
HDITEM 161, 659, 800
HELPINFO 425, 789, 803
HELPWININFO 435, 442
LOGFONT 214, 549, 552, 650, 803
LVITEM 809
MSG 560
NEWTEXTMETRIC 554, 811
NMHDR 151, 176, 814
OUTLINETEXTMETRIC 215, 815
PAINTSTRUCT 776, 818
POINT 819
RECT565, 722, 819
SECURITY_ATTRIBUTES 547
SIZE 820
TEXTMETRIC 554
WNDCLASS 572, 781, 820
WNDCLASSEX 564, 582

Т

Таблица
акселераторов 653
ключевых слов 434
Типы сносок в файле текста справки 470
Точка останова 501
Трансляция условная 526

У

Уровень доступа
EVENT_MODIFY_STATE 572, 574, 576
PROCESS_ALL_ACCESS 557
PROCESS_SET_INFORMATION 577
SYNCHRONIZE 584
THREAD_QUERY_IN FORMATION 558
THREAD_SET_INFORMATION 578
Утечки памяти 514

Ф

Файл
afx.h 590, 644, 709, 744, 748, 749
afxcmn.h 659, 666, 687, 713, 736
aficcoll.h 705
afxcore.rtf 439, 473
afxdlgs.h 716, 718
afxexth 605, 607, 641, 713, 741, 751
afxmt.h 704, 733
afxprint.rc 60, 445
afxprint.rtf 439, 489
afxres.rc 60, 445
afxrich.h 725
afxtempl.h 594, 672, 698
afxwin.h 597, 600, 602, 603, 608, 628, 632, 635, 640, 650, 653, 658, 680, 701, 712, 723, 727,730,735,753,761,776
BrushAppView.cpp 230, 235
DDBAppView.cpp 238
DemoDlg.cpp 468
DemoDlg.h 467
DialogApplication.rc 69
DialogApplicationDlg.cpp 78, 110
DialogApplicationDlg.h 76
EditAppView.cpp 276
GalleryView.cpp 110
HelpApp.cnt 492
HelpApp.hpj 465, 493
HelpApp.rc 463
helpapp.rtf 494
HelpAppView.cpp 467
ListAppDlg.cpp 120
ListAppView.cpp 226
MDIApplication.cpp 50
MDIApplicationDoc.cpp 41
MDIApplicationDoc.h 40
MDIApplicationView.cpp 46
MDIApplicationView.h 44
NewHelp.hm 464, 469
OnSetActive 716

Pagel.cpp 92
PrintAppDoc.cpp 313
PrintAppView.cpp 313, 321, 324
ProgressAppDlg.cpp 128, 133
Reflectdlg.h 194
Resource.h 60, 885
RichAppView.cpp 282, 285
Sheet.h 90
SheetApplication.cpp 91
StdAfx.cpp 17
StdAfx.h 17
TextApp.cpp 250, 263
TextApp.Doc 250
TextAppDoc.cpp 222, 247, 262
TextAppView.cpp 220, 223, 247, 260
Threads.cpp 393, 396, 410
ToolAppView.cpp 298, 308
WinGdi.h 210
Winhelp.exe 653
YellowEdit.cpp 192
mlistctrl.cpp 145
Resource.h 172
адресации справок 438
группового списка поиска 438
заголовка 524
заголовка справки 438
инициализации 267
конфигурации 438
поиска по всему тексту 438
проекта справочной системы 438
реализации 524
текста справки 438
Флаг
CF_ENABLEHOOK 545, 546
CREATE_SUSPENDED 773
CS_BYTEALIGNCLIENT 610
CS_BYTEALIGNWINDOW 610
CS_CLASSDC781
CS_OWNDC 781
CS_PARENTDC 781
DEVICE_FONTTYPE 554
HDI_IMAGE 161
PSF_WIZARD 722
PSWIZB_BACK 97
PSWIZB_DISABLEFINISH 97
PSWIZB_FINISH 97
PSWIZB_NEXT97
RASTER_FONTTYPE 554
RDW_ALLCHILDREN 794
RDW_ERASE 793
RDW_ERASENOW 794
RDW_FRAME 793
RDW_INTERNALPAINT 793
RDW_INVALIDATE 793
RDW_NOCHILDREN 794
RDW_NOERASE 793
RDW_NOFRAME 793
RDW_NOINTERNALPAINT 793
RDW_UPDATENOW 794
RDW_VALIDATE 793
TRUETYPE_FONTTYPE 554
HDI_FORMAT 161
Фокус ввода 569
Форматирование текста справки в MS Word 495
Функция
Abort 258, 648
Add 157, 371, 594, 666, 705
AddDocTemplate 57, 761
AddFontResource 214, 537, 538
AddHead 672
Add Page 100, 718
AddString 123, 417, 680
AddTail 381, 405, 672
AddToListBox 414
AfxBeginThread 384, 418, 538, 772
AfxEnableControlContainer 55, 539
AfxEndThread 388, 539, 540
AfxGetApp 156, 254, 540, 761
AfxGetAppName 541, 761
AfxGetInstanceHandle 540, 761
AfxGetResourceHandle 541, 761
AfxMessageBox 440
AfxRegisterWndClass 541, 651
AfxThrowMemoiyException 351

AfxWinInit 20, 542
Append Menu 417, 701
Approximate View Rect 169, 687
AssertValid 510, 709
Attach 390, 398, 608
BeginDrag 152, 667
BeginPaint 207, 319, 776
BitBlt 242, 609
CallWindowProc 582
CDumpContext 672
CFHookProc 544, 546, 664
Char From Pos 278, 640
Choose Font 213, 545, 546
ClearTics 736
Close 258, 267, 591, 646
CloseHandle 402, 547, 549. 557, 571
CommandToIndex 310, 741
CommDlgExtended Error 546
Construct 719
ConstructElement 373
ContinueRouting 204, 304, 602
Create 57, 157, 310, 541, 653, 667, 688, 714, 736, 741
CreateCompatibleBitmap 242, 597
CreateCompatibleDC 611
CreateDC 318
CreateDrag Image 152, 689
CreateEvent 396, 547, 571
CreateEx 302, 751
CreateFile 547
CreateFont 214, 650
CreateFontIndirect 214, 549, 650
CreateIndirect 628
CreatePointFont 650
CreatePointFontIndirect 650
Create Process 549, 571
CreateRcctRgnIndirect 137, 728
CreateScalableFontResource 214
CreateThrcad 387, 539, 772
CreateWindow 558, 572
CreateWindowEx 425, 558
CToolBar 749
DDX_Control 679
DefWindowProc 424
Delete 348
DeleteCriticalSection 402, 550, 551, 566
DelcteObjcct 550, 658
DeleteString 123, 681
Destroy Window 163, 629, 776
DestructElement 373
Detach 398, 400, 612
DispatchMessage 173, 195
DockControlBar 303, 654
DoDataExchange 85, 777, 850
DoDialogExchange 853
DoModal 278, 630, 720
DoPreparePrinting 335, 753
DoPromptFileName 264
DPtoLP 723
DragEnter 153. 669
DragLeavc 153, 670
DragMove 153, 670
DragShowNolock 152, 670
DrawText 219
Dump 510, 710
DuplicateHandle 389, 549, 557, 571
ElementAt 372
Empty 414, 745
Enable 204, 259, 304, 601, 602
EnableSdControls 55, 761
Enable3dControlsStatic 55
EnableDocking 303, 605, 655
Enable Window 158, 779
EndDialog 545
EndDoc 333, 334, 612
EndDrag 154, 671
EndPage 333, 334, 613
EndPaint 207, 779
EnterCriticalSection 400, 550, 565
EnumFontFamilies 213, 551, 618
EnumFontFamiliesEx 552
EnumFontFamProc 552, 554
EnumFonts 618
Escape 333
ExitInstance 387, 762

ExitThread 559
ExtTextOut 219, 220
FillRect 137, 241,613
FillSolidRect 614
FindClose 547
FindFirstFile 547
FindResource 541
FindText 278, 641
Flush 591
Format 403, 745
Format Message 560
Free Library 562
FromHandle 390, 622
GdiFIush 402, 555
GdiSetBatchLimit 556
GetAt 372, 377, 673
GetBitmapBits 242, 598
GetBkColor2l8
GetBkMode 218
GetBuffer 160, 745
GetBufferLength 278, 642
GetButtonStyle 750
GetButtonText 750
GetCharABCWidths 218
GetCharABCWidthsFloat 218
GetCharFormatSelection 287, 726
GetCharWidth32 218
GetCharWidthFloat 218
GetClientRect 241, 243, 317, 397, 780
GetCommandLine 20, 556
GetCount 403
GetCurrentDirectory 254, 556
GetCurrentProcess 420, 557
GetCurrentTime 748
GetCurSel 123, 681
GetDC 207, 319, 780
GetDeviceCaps 219, 330, 331, 611, 612, 614
GetDlgItem 137, 158, 417, 419, 558, 781
GetDocString 253, 632
GetDocument 49, 317, 754
GetEditCtrl 278, 642
GetExitCodeThread 388, 419, 558
GetFile 257, 592, 635
GetFileName 266, 647
GetFilePath 266, 647
GetFileTitle 647
GetFontData 216
GetFromPage 335
GetHead 372, 673
GetHeaderCtrl 161, 690
GetHeadPosition 377, 379, 407, 674
GetHotCursor 157, 690
GetItem 161, 659
GetItemPosition 151, 691
Get_ItemText 154, 691
GetLastError 538, 547-549, 556, 558-560, 562, 563, 565, 570-578, 580, 581, 584, 585
GetLength 265, 647
GetMaxPage 335
GetMenuContextHelpId 432, 560
GetMessage 172, 195, 560
GetMinPage 335
GetModuleHandle 20, 562
GetNext 372, 377, 379, 407, 674
GetNextAssoc 373, 377, 698
GetObject 598
GetOrigin 152, 692
GetOutlineTextMetrics 215, 219
GetParent 781
GetParentFrame 403, 782
GetParentOwner 310, 782
GetPathName 257, 636
GetPos 132, 738
GetPrev 372, 675
GetPriorityClass 580
GetProcAddress 562
GetProfileInt 270
GetProfileString 254, 270, 763
GetSafeHdc 405, 618
GetSize 379, 595, 706
GetStartPosition 377, 699
GetState 419, 422, 600
GetStockObject 613

GetStyle 304, 783
GetSystemMenu 417, 783
GetTabbedTextExtent 218
GetTail 372, 676
GetTail Position 676
GetText 123, 681
GetTextAlign 217
GetTextColor 218
GetTextExtent 310, 619
GetTextExtentEx Point 218
GetTextExtentPoint32 217, 218
GetTextMetrics 218
GetThreadPriority 580
GetToPage 335
GetUpperBound 595, 707
GetWindowContextHelpId 432, 563
GetWindowDC319
GetWindowLong 153, 163, 165, 563
GetWindowRect 137, 159, 565, 784
GetWindowText 638 HitTest 153, 692
InitializeCriticalSection 551, 565
InitializeCriticalSectionAndSpinCount 551, 565
InitInstance 54, 254, 263, 387, 397, 539, 603, 764
InitModalIndirect 629
InsertColumn 159, 693
InsertItem 160, 694
InsertString 682
Invalidate 258, 317, 784
InvalidateRect 164, 785
IsChild 561
IsDialogMessage 443
IsDlgButtonChecked 413, 785
IsEmpty 417, 677, 699, 747
IsLoading 592
IsLocked 734
IsPrinting 330, 619, 755
IsStoring 44, 249, 592
IsWindowUnicode 573
LeaveCriticalSection 400, 550, 566
LoadBarState 303, 656
LoadCursor 157, 764
LoadFrame 57, 656
LoadIcon 157, 765
LoadLibrary 562
LoadResource 562
LoadStdProfileSettings 56, 270, 765
LoadString 343, 403, 417, 747
LoadToolbar 302, 752
Lock 391, 413, 734
Lookup 373, 378, 699
LPtoDP 723
Main 20
MakeReverse 414, 747
MessageBox 257, 343, 567
ModifyStyle 164, 786
ModifyStyleEx 163, 786
MsgWaitForMultipleObjects 583
NormalizeRect 723
OffsetRect401, 724
OnActivateView 652
OnBeginPrinting 324, 328, 334, 754
OnCancel 630
OnCharEffect 283, 726
OnClose 297, 418
OnCommandHelp 787
OnContextHelp 443
OnContextMenu 469, 788
OnCreate 296, 302, 307, 310, 408
OnDblclkList 123
OnDeltaposSpin 85
OnDraw 49, 207, 239, 314, 321, 755
OnEndPrinting 334
OnFindNext 278, 643
OnHelp440, 441, 766
OnHelpHitTest 788
OnHelpIndex441, 444
OnHelpInfo 469, 789
OnHScroll 132, 789
OnIdle 443
OnInitDialog 131,415, 631

OnInitialUpdate 755
OnMouseMove 790
OnMouseWheel 728
OnMyToolbarCheck 298, 304
OnNewDocument 43, 313, 636
OnNotify 178
OnOK 630
OnPaint 136, 208, 791
OnParaAlign 283, 727
OnPrepareDC 207, 325, 330, 332, 334, 756
OnPreparePrinting 333, 757
OnPrint327, 331, 334, 758
OnQueryEndSession 653
OnRegisteredMouseWheel 728
OnSetActive 101
OnTextNotFound 643
OnUpdate 208, 759
OnUpdate MyToolbar 297
OnWizardBack 100, 716
OnWizardFinish 100, 717
OnWizardNext 100, 717
OnWndMessage 195
OpenEvent 549, 570
OpenProcess 557
ParseCommandLine 57, 603, 766
ParseParam 603
PeekMessage 172
PostMessage 390, 545, 575
PostNcDestroy 628
PostQuitMessage 388
PostThreadMessage 172, 561
PreCreateWindow 49, 792
PreTranslateMessage 387, 443
ProcessShellCommand 57, 603, 767
ProcessWndProcException 387
PtInRect 242, 724
Pulse Event 571, 574
Read 258, 265, 592, 647
ReadObject 590
Rectangle 317, 619
RectVisible 755
RedrawWindow 792
RegisterClassEx 564, 582
RegisterClass 572
RegisterShellFileTypes 768
RegisterWindowMessage 575
ReleaseCapture 155, 573
ReleaseDC 207, 794
Release File 258, 636, 648
Remove 267
RemoveAll 379, 596, 678, 700, 708
RemoveAt 379
RemoveFontResource 214, 538, 573
RemoveHead 372, 381, 407, 677
RemoveKey 380
RemoveTail 372, 677
ResetDC 626
ResetEvent 548, 549, 574, 576
ResumeThread 389, 405, 418, 773
Run 195, 387, 773
SaveBarState 303, 657
ScreenToClient 137, 319, 795
Seek 648
SeekTo Begin 265, 649
SelectClipRgn 137, 620
SelectObject 215, 221, 242, 550, 622
SendMessage 214, 538, 574, 844
Serialize 44, 247, 249, 250, 313, 316, 711
SerializeElement 374
SerializeRaw 643
SetActivePage 99, 100
SetAt 678
SetBitmapBits 243, 598
SetBkColor 217
SetBkMode 217
SetButtonStyle 750
SetButtonText 750
SetCapture 573
SetCharFormat 287, 727
SetCheck 158, 204, 303, 419, 600-602
SetCurrentDirectory 254, 575
SetCurSel 417, 683

SetDialogBkColor 628
SetDlgItemText 85, 123, 795
SetDragCursorImage 152, 671
SetEvent 407, 549, 574, 576
SetExtendedStyle 163, 167, 695
SetFileAttributes 266
SetFinishText 100, 721
SetForegroundWindow 569
SetHotCursor 167
SetImageList 161, 659
SetIndicators 310, 742
SetItem 160, 161, 660, 695
SetItemPosition 155, 170, 696
SetItemText 154, 697
SetLastError 559, 581
SetLineSize 132, 738
SetMapMode 331, 623
SetMapperFlags 215
SetMaxPage 330, 335, 713
SetMenu 700
SetMenuContextHelpId 432, 577
SetModifiedFlag 208, 249, 258, 637, 638
SetOwner 418
SetPageSize 132, 738
SetPaneInfo 310, 742
SetPaneText 310, 743
SetParent 583
SetPos 132, 715, 739
SetPriorityClass 420, 577
SetRadio204, 601, 602
SetRange 132, 715, 739
SetRect 169, 725
SetRegistryKey 55, 269, 768
SetScaleToFitSLze 729
SetScrollPos 795
SetScrollSizes317, 730
SetSel 123, 683
SetSize 594, 596, 708
SetTabStops 642
SetText 204, 601, 602
SetTextAlign 217
SetTextCharacterExtra 217
SetTextColor 217
SetTexUustification 217
SetThreadPriority 385, 387, 399, 405, 422, 578, 774
SetTic 739
SetTickFreq 132, 740
SetViewportExt 623
SetViewPortOrg 331, 624
SetWindowContextHelpld 432, 580
SetWindowExt 623
SetWindowLong 165, 302, 581
SetWindowPos 163, 582
SetWindowText 403, 638, 796
SetWizardButtons 101, 721
SetWizardMode 722
SetWorkAreas 163, 169, 697
ShowControlBar 304, 657
ShowCursor 671
ShowWindow 58, 629, 796
Sleep 414, 583
StartDoc 333, 334, 612, 625
StartPage 332, 333, 334, 626
StretchBlt 611
SuspendThread 422, 774
TabbedTextOut 219
TerminateThread 559
TextOut 219
Translate Message 195
TryEnterCriticalSection 550, 565
Unlock 391,735
UpdateAllViews 207, 390, 637
UpdateData 85, 797, 849
UpdateWindow 58, 798
WaitForSingleObject 398, 407, 583
WindowProc 195, 585
WinHelp 425, 429, 441, 585, 769
WinMain 20, 172, 384, 543
WndProc 195
Write 264, 265, 593, 649
WriteObject 591
WriteProfileInt 270
WriteProfileString 253, 270, 770
виртуальная 523, 534
встраиваемая 50
дружественная 534

обработки сообщения 171, 599
оператор [] 372, 377, 595, 700, 707
оператор << 249, 591
оператор >> 249, 590
Ц
Цикл обработки сообщений 172, 384
Ш
Шаблон
документа 39, 56, 631
класса 364
функции 362
Шрифт 209, 215
Э
Экранные координаты 318
Элемент управления
Инкрементный регулятор 63
Переключатель 67
Рамка группы 67
Раскрывающийся список 65
Статический текст 62
Текстовое поле 63
Флажок 67
Элементы данных обратного вызова 686

Введение

Еще совсем недавно программирование в среде Windows можно было сравнить с плаваньем в финских шхерах. Стоя на верхней палубе пассажирского парома, можно наслаждаться открывающимися перед тобой видами и не задумываться о работе капитана. Однако стоит самому встать на его место и попробовать провести через этот лабиринт пускай даже намного меньшее судно, как после очередной мели, преградившей выход на казалось бы такой близкий фарватер, ты дашь торжественное обещание никогда в жизни не пускаться в подобные авантюры, не заручившись согласием опытного лоцмана составить тебе компанию или не достав подробнейших карт всего маршрута следования.

Роль такого лоцмана в море Windows отводится среде программирования Visual C++ версии 6.0, являющейся мощным средством для разработки 32-разрядных приложений в операционных системах Windows 95, Windows 98 и Windows NT. Этот достаточно сложный продукт позволяет создавать намного более объемные и более сложные приложения, чем его предшественники, разработанные для 16-разрядных Windows 3.x или для DOS.

Windows изначально разрабатывалась как графическая среда "для домохозяек", которая впоследствии превратилась в операционную систему, сохранив при этом основной подход, характерный для данной категории программных продуктов, отличительной чертой которых является то, что они устанавливаются на компьютер до его передачи пользователю специалистом очень высокой квалификации, способным устранить все конфликты, возникающие в установленной программной конфигурации. На компьютер, переданный пользователю, не может быть установлена ни одна новая программа, поскольку это может привести к краху всей системы. И действительно, о какой надежности может идти речь в операционной системе, при установке в которую нового приложения может быть замещена добрая половина системных библиотек. Следствием данного подхода является недопустимость создания в данной операционной системе новых программ.

В принципе программу любой сложности можно написать на любом языке программирования и с использованием любой среды программирования.

Это, конечно, так, но хорошая среда программирования отличается от плохой тем, что в ней на программирование той же самой задачи будет затрачено меньше времени, а получившаяся в результате программа будет работать быстрее и надежнее. Для создания подобной среды программирования необходимо досконально изучить окружение, в котором будет работать скомпилированная программа, а кто изучил ОС Windows лучше ее разработчиков? Кроме того, фирма Microsoft не опубликовала подробного описания своего творения. Поэтому пользователь, скорее всего, оценит возможности, предоставляемые *библиотекой базовых классов* (Microsoft Foundation Classes, MFC) Visual C++.

Учитывая вышеизложенное, можно предположить, что внимание пользователя привлекут мастера, такие как AppWizard, ClassWizard и ActiveX ControlWizard, позволяющие получать заготовки больших фрагментов исходного кода программы путем выбора соответствующих опций в панелях специальным образом организованных диалоговых окон.

Поскольку программирование в операционной системе Windows, строго говоря, "не предусмотрено", то, разрабатывая программы, необходимо соблюдать особую осторожность и максимально использовать стандартные подходы. Поэтому, содержащая их книга нужна не только начинающему, но и достаточно опытному программисту.

Для кого предназначена эта книга?

В настоящее время остается еще достаточно много программистов, имеющих обширный опыт программирования на языке C++ в среде DOS. Как правило, они использовали для программирования продукты фирмы Borland, позволявшие даже при работе под управлением Windows создавать полноценные продукты для работы в среде DOS.

Фирма Inprise, подразделением которой стала фирма Borland, сосредоточила свое внимание на разработке инструментария для работы с базами данных. Выпущенный ею продукт C++ Builder 4 ориентирован на проектирование клиентских и серверных компонентов распределенных программ, основанных на объектных архитектурах COM (Component Object Model) и CORBA (Common Object Request Broker Architecture). Такая специализация предлагаемого программного обеспечения заставляет прежних пользователей продуктов фирмы Borland искать среды программирования более приспособленные для создания небольших приложений, способных решать ограниченный круг задач. В качестве такой среды может выступить Microsoft Visual C++. Однако для такого перехода требуется иметь достаточно простое описание способов программирования в данной среде. Описание этих способов содержится в данной книге.

Другими потенциальными читателями этой книги являются программисты, имеющие опыт программирования в среде Windows 3.x и продолжающие его

использовать при работе в Visual C++, не желая изучать особенности данной среды программирования. Конкуренция и требования рынка уже заставляют их не просто использовать некоторые заготовки, предоставляемые им мастером AppWizard, но и более полно задействовать возможности, предоставляемые библиотекой MFC, также описанные в данной книге.

Эта книга позволяет изучить основные приемы использования среды программирования Visual C++ для разработки 32-разрядных приложений Windows. В настоящее время имеется большое количество книг, посвященных данной теме, но все они в основном рассказывают о написании приложений для работы с базами данных и Internet. Однако для работы в данной области имеется специализированное программное обеспечение, например уже упомянутый выше C++ Builder 4, объединяющий в себе возможности Visual C++ по созданию базовой системы обработки информации и возможности Visual Basic по созданию пользовательских интерфейсов и прототипов приложений. Причем, все это сделано на более высоком уровне и с возможностью использования обширного инструментария, недоступного в продуктах фирмы Microsoft.

В данной книге среда программирования Visual C++ рассматривается исключительно как средство создания простейших пользовательских приложений, позволяющих решать текущие задачи пользователя. К таким задачам можно отнести создание оболочки для системы, позволяющей протестировать некоторый алгоритм или создать простую панель управления некоторым устройством, подключенным к компьютеру, если его драйверы могут вызываться операторами языка C++.

У каждого пользователя среды программирования Visual C++, пытавшегося разобраться в ней самостоятельно, накопилось множество вопросов, и он надеется получить в одной книге ответы на большинство из них. Как показывает опыт, уровень подготовки, даже у достаточно опытных программистов, существенно отличается: некоторые до сих пор писали исключительно в среде DOS и теперь, перейдя к работе в среде Windows 95, нуждаются в простейшем компиляторе для своих задач, не использующих и не требующих сложного графического интерфейса, другие работали в среде Windows 3.x и не привыкли следовать при написании программ требованиям, предъявляемым 32-разрядными приложениями, но хотели бы это делать, и, наконец, любому программисту нужен достаточно подробный справочник, в котором он мог бы найти решение стандартных проблем, возникающих при программировании.

Данная книга не является учебником по алгоритмическому языку C++ и операционной системе Windows. Предполагается, что читатель уже написал по крайней мере пару программ на этом языке. Однако, как выяснилось, многие программисты, работавшие под DOS и даже использовавшие библиотеку классов Turbo Vision фирмы Borland, имеют о классах достаточно

смутное представление. Поэтому в данную книгу включен раздел, посвященный концепции объектного программирования и классам (Приложение А). Те, кто считает, что достаточно хорошо знакомы с этим вопросом, могут пропустить его.

Структура данной книги

Конечно, читатель может не иметь опыта программирования в среде Windows, поэтому в книгу включен материал, позволяющий начать создание приложений практически с нуля. Ниже приведено краткое описание разделов и глав, позволяющее пользователю лучше ориентироваться в структуре книги:

- Часть I. *"Создание простейших приложений"*

  Рассмотрено создание простейших приложений, использующих основные элементы интерфейса Windows. Здесь же рассмотрена концепция Документ/Представление, лежащая в основе программирования с использованием библиотеки MFC.

  - Глава 1. *"Использование мастера создания приложений"*

    Описывается мастер создания приложений, позволяющий автоматически создать заготовку приложения. Обсуждается также создание консольных приложений.

  - Глава 2. *"Классы приложений, документов и представлений"*

    Посвящена концепции Документ/Представление. Кроме того, в ней описаны классы шаблонов документов.

  - Глава 3. *"Диалоговые окна и простейшие элементы управления"*

    Рассмотрено создание диалоговых окон с использованием редактора ресурсов и описано включение в них простейших элементов управления.

- Часть II. *"Программирование интерфейса пользователя"*

  Описывается, каким образом пользователь может выводить на экран необходимую информацию и управлять работой приложения.

  - Глава 4. *"Классы элементов управления"*

    Изучаются сложные элементы управления, работа с которыми требует создания объектов классов данных элементов управления и вызова содержащихся в них методов обработки информации.

  - Глава 5. *"Сообщения и команды"*

    Рассмотрен механизм обработки сообщений, используемый в операционной системе Windows для обеспечения взаимодействия различных процессов и потоков в приложении.

- Глава 6. *"Вывод информации на экран"*

  Описаны принципы работы интерфейса графических устройств Windows (GDI).

- Глава 7. *"Работа с файлами документов"*

  Обсуждается работа с файловой системой Windows.

- Глава 8. *"Работа с текстовыми документами"*

  Операционная система Windows создавалась преимущественно для работы с текстовыми документами и оптимизирована исключительно для выполнения данной задачи. Поэтому вопросы создания приложения, работающего с текстовой информацией, вынесены в отдельную главу.

- Глава 9. *"Панели инструментов и строка состояния"*

  Панели инструментов и строка состояния являются непременным атрибутом любого законченного приложения Windows. Здесь описаны способы создания и программирования работы с этими элементами интерфейса.

- Глава 10. *"Печать документов и организация прокрутки в окне"*

  Печать документов и прокрутка изображения в окне большей частью реализуются при помощи стандартных средств среды программирования Visual C++, однако для обеспечения работы данных функций в приложении необходимо произвести его настройку. Этому вопросу и посвящена данная глава.

❒ Часть III. *"Особенности программирования в среде Visual C++"*

Средств, описанных в первых двух частях данной книги, достаточно для создания работоспособного приложения. Однако среда программирования Visual C++ предоставляет пользователю возможности, позволяющие уменьшить размеры исходных текстов программ, повысить их надежность и рационально распределить предоставляемые приложению вычислительные ресурсы. Здесь и содержится описание этих возможностей.

- Глава 11. *"Исключения, шаблоны и новые возможности Visual C++"*

  Рассматриваются новые возможности, предоставляемые средой программирования Visual C++. Исключения представляют собой механизм, существенно упрощающий процесс обработки ошибок, без использования которого практически невозможно создать надежно работающее приложение. Шаблоны представляют собой метод задания функций обработки для переменных абстрактных классов. Это позволяет существенно сократить количество практически идентичных по тексту функций, выполняющих одну и ту же операцию для переменных разных типов, что уменьшает размер исходного текста программ и

обеспечивает гарантированное внесение изменений во все функции обработки различных типов данных, выполняющих над ними одну и ту же операцию.

- Глава 12. *"Многозадачность на основе потоков Windows"*

  Поток Windows представляет собой основную программную единицу, которой операционная система Windows предоставляет квант процессорного времени. Создание нескольких потоков в одном приложении позволяет более рационально использовать выделяемое данному приложению процессорное время. Создание фоновых потоков и правильная расстановка приоритетов позволяет обеспечить быструю реакцию приложения на запрос пользователя при максимально возможной загрузке процессора.

- Глава 13. *"Справка в приложении"*

  Любое серьезное приложение независимо от того, создается оно для внутреннего использования или для продажи, должно содержать обширную справочную систему, позволяющую получить справку по любому вопросу, связанному с его работой. Для коммерческих приложений, а также для приложений, рассчитанных на пользователя со слабым знанием английского языка, актуален вопрос русификации приложения и содержащейся в нем справочной системы. Поэтому в данной главе рассмотрены вопросы не только связанные с созданием справочной системы, но и с ее русификацией. Также обсуждается русификация всего приложения.

- Глава 14. *"Отладка приложения"*

  Отладка приложения является непременным этапом его создания. Практически ни одно приложение не работает безошибочно с первого раза. В данной главе рассмотрены возможности, предоставляемые средой программирования Visual C++ для отладки приложения.

❒ Приложения

В приложениях изложены вопросы, не связанные напрямую с программированием приложений, но изучение которых может способствовать успешной работе в Visual C++.

- Приложение А. *"Объектно-ориентированное программирование и классы"*

  Как отмечалось выше, опыт показывает, что не все программисты, использующие классы, хорошо понимают их назначение. Работа с библиотекой MFC построена на использовании классов, поэтому пользователь должен хорошо разбираться в основах ООП. В данном приложении приводится краткий обзор этой концепции и связанного с ней понятия класса.

- Приложение B. *"Описание классов и функций Visual C++"*

  Данное приложение представляет собой выборочный перевод информации, содержащийся в библиотеке MSDN. Необходимость включения в книгу этого материала обусловлена тем, что далеко не все пользователи настолько хорошо знают английский язык, чтобы гарантированно понять все тонкости использования функций, описанных в данной библиотеке. Здесь приводится полное описание большинства функций, используемых в программах, приведенных в качестве примеров.

  Все функции, классы и структуры в данном приложении расположены в алфавитном порядке. Сначала описываются глобальные функции, а затем — классы и структуры.

- Приложение C. *"Средства для работы с классами Visual C++"*

  Рассмотрены основные средства, предоставляемые средой программирования Visual C++ для работы с классами. Работа с классами обсуждается практически в любой главе данной книги, поэтому наличие раздела, суммирующего эту информацию, позволяет читателю получить целостное представление по этому вопросу до того, как он приступит к практической работе.

- Приложение D. *"Интерфейс пользователя Visual C++"*

  Visual C++ представляет собой достаточно сложную среду программирования. Хотя ее интерфейс и задумывался как "интуитивно ясный", но многие функции данной среды таковыми не являются. Кроме того, пользователь может и не догадываться о необходимости той или иной функции, пока не прочтет о ней и не попробует ею воспользоваться. Данное приложение содержит описание интерфейса пользователя, достаточное для начала работы с ним абсолютно неподготовленного пользователя.

Соглашения, принятые в данной книге

В данной книге использовалось специальное форматирование текста для выделения некоторых текстов или фрагментов текста. Ниже приведены основные принципы выделения текста.

- Исходные тексты фрагментов программ, представляющие собой одну или более строк текста, выделяются шрифтом `Courier`, как показано ниже:

```
void CMainFrame::Dump(CDumpContext& dc) const
{
  CMDIFrameWnd::Dump(dc);
}
```

- Когда на эти фрагменты программ имеются ссылки из различных фрагментов текста, они оформляются листингом, как показано ниже:

Листинг 1. Заголовок листинга

```
int CMainFrame::OnCreate(LPCREATESTRUCT lpCreateStruct)
{
  if (CMDIFrameWnd::OnCreate(lpCreateStruct) == -1)
    return -1;

  if (!m_wndToolBar.CreateEx(this, TBSTYLE_FLAT, WS_CHILD | WS_VISIBLE
                                                  | CBRS_TOP
    | CBRS_GRIPPER | CBRS_TOOLTIPS | CBRS_FLYBY | CBRS_SIZE_DYNAMIC) ||
    !m_wndToolBar.LoadToolBar(IDR_MAINFRAME))
  {
    TRACE0("Failed to create toolbar\n");
      //Не удалось создать панель инструментов
    return -1;      // fail to create
  }

  if (!m_wndStatusBar.Create(this) ||
    !m_wndStatusBar.SetIndicators(indicators,
      sizeof(indicators)/sizeof(UINT)))
  {
    TRACE0("Failed to create status bar\n");
      //Не удалось создать строку состояния
    return -1;      // fail to create
  }

  // TODO: Delete these three lines if you don't want the toolbar to
  // be dockable
  // Удалите следующие три строки, если вы не хотите, чтобы панель
  // инструментов была переносимой (фиксируемой)
m_wndToolBar.EnableDocking(CBRS_ALIGN_ANY);
  EnableDocking(CBRS_ALIGN_ANY);
  DockControlBar(&m_wndToolBar);

  return 0;
}
```

- Если в тексте встречается имя класса, функции, имя типа переменной или фрагмент текста длиной менее строки, он выделяется шрифтом Courier. Например: `int`, `CMainFrame`, `DockControlBar` и т. д.
- Имена функций обычно даются в полной форме. Это значит, что если функция является членом класса, то перед ней указывается имя класса, членом которого она является. При этом указывается имя того класса

в иерархии, в котором впервые появилась данная функция. Например: `CObject::Serialize`. Если перед именем функции не стоит имя класса, то это означает, что это либо глобальная функция, либо функция пользовательского класса, либо часто встречающаяся функция, полное описание которой было только что приведено. Это позволяет сравнительно просто отыскать описание данной функции в Приложении В

- Заголовок диалогового окна, имя кнопки или команды меню выделяются полужирным шрифтом. Например: команда **Open**, команда меню **File**, **New** и т. д. Если в имени команды меню, кнопки или заголовка элемента управления содержится подчеркивание, оно воспроизводится в тексте
- Имя клавиши заключается в угловые скобки (<>). Например: <Ctrl>, <F5> и т. д.
- Если требуется одновременно нажать несколько клавиш, они объединяются знаком (+). Например: сочетание клавиш <Ctrl>+<Alt>+<Del>
- При первом появлении термин выделяется курсивом. Например: *новый термин*
- Ссылка на другую главу содержит ее номер. Например:

  См. также главу 5

Требования к аппаратным средствам и программному обеспечению

Для работы в среде программирования Visual C++ версии 6.0 требуется следующее аппаратное и программное обеспечение:

- IBM-совместимый PC с процессором Intel 80486 или лучшим
- Не менее 16 Мб оперативной памяти
- Жесткий диск объемом не менее 500 Мб
- Дисплей SVGA и соответствующий видеоадаптер, обеспечивающие разрешение не менее 800×600 точек
- Дисковод CD-ROM
- Совместимая с Microsoft мышь
- Операционная система Windows 95, Windows 98 или Windows NT
- Инсталлированная среда программирования Visual C++ 6.0

Естественно, эти требования являются минимальными, и работа в такой конфигурации не доставит вам большого удовольствия.

http://www.nevalink.ru
info@nevalink.ru

nevalink
невалинк

Подключение ко всем ресурсам Интернет частных лиц и корпоративных клиентов.

Ваше электронное представительство.

World Wide Web.

Реклама в Интернете.

Выделенные каналы связи.

Скидки постоянным клиентам.

"Горячая линия" поддержки пользователей 24 часа, без выходных.

Информационно-торговый отдел.

Quake flat-rate.

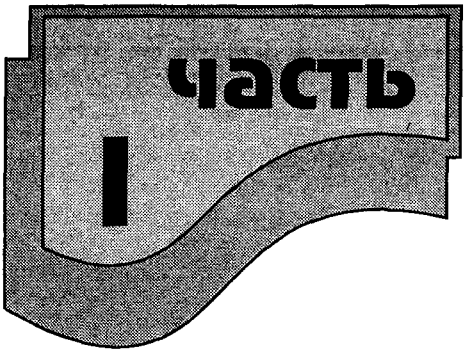
Лицензия № 5369 МС РФ

Сервис-центры по доступу в Интернет
На Московском вокзале и в Центральных ж/д кассах
т.т.: (812) 277-25-17, (812) 168-67-34

невалинк

пять лет на рынке Санкт-Петербурга. Более 5000 абонентов.
Среди разработок: http://www.spb.ru
старейший и один из самых популярных серверов России.

Санкт-Петербург, Канал Грибоедова, 36, тел.: 113-47-12, факс: 110-57-64



Создание простейших приложений

1. Использование мастера создания приложений
2. Классы приложений, документов и представлений
3. Диалоговые окна и простейшие элементы управления

Использование мастера создания приложений

Среда программирования Visual C++ используется не только для редактирования, компиляции и отладки программ, но и для генерации заготовок программ. Эти заготовки представляют собой работоспособные программы, реализующие основные элементы интерфейса Windows, необходимые в любой пользовательской программе, и, естественно, не выполняющие никакой обработки. Мастер приложений позволяет создать такую заготовку за несколько минут, однако, чтобы получить на ее основе полноценное коммерческое приложение, у вас уйдут недели и месяцы работы. Программисту, начинающему свое ознакомление с Visual C++, необходимо, прежде всего, ознакомиться с простейшими типами приложений, генерируемых мастерами. К ним следует отнести консольные приложения и приложения, сгенерированные мастером MFC AppWizard (exe) (мастер AppWizard, генерирующий исполнимые программы с использованием библиотеки MFC). Название последнего типа приложений следует признать крайне неудачным, поскольку библиотеку MFC используют практически все типы приложений, компилируемых Visual C++ (даже некоторые консольные приложения). Кроме того, использование имени мастера AppWizard только в трех мастерах, генерирующих программные коды для исполнительных файлов, динамических библиотек и управляющих элементов ActiveX, вносит ненужную неоднозначность в терминологию. Поэтому целесообразнее было бы использовать это имя для всех мастеров приложений. Для определенности будем называть приложения, сгенерированные мастером MFC AppWizard (exe), приложением Windows. Особым классом приложений Windows является диалоговое приложение.

Консольное приложение

Предположим, что вы до сих пор остались ярым приверженцем программирования в среде DOS, и, что единственной причиной, побудившей вас взять

в руки эту книгу, явилось практически полное исчезновение хороших средств программирования под DOS и желание начальства, чтобы все его подчиненные работали в "современной операционной системе" Windows.

Если разобраться, то с точки зрения программиста, пишущего программы исключительно для внутреннего пользования, и имеющего большой задел программ под DOS, эта операционная система имеет только один недостаток: она не позволяет использовать более 640 Кб оперативной памяти (а в действительности намного меньше). "Идя навстречу пожеланиям трудящихся", Microsoft разработала специальный вид приложений: консольные приложения. С точки зрения программиста, консольное приложение является привычной программой DOS, использующей все стандартные операторы, но позволяющей, однако, использовать всю доступную оперативную память (при условии, что каждый элемент данных программы не будет превышать по размеру 1 Мб). Этот тип приложения запускается в особом типе окна, называемом "Окно MS-DOS", в такое окно в среде Windows 95 можно выйти по команде **Пуск**, **Программы**, **Сеанс MS-DOS**.

Преобразуя программы, написанные под DOS, в консольное приложение необходимо помнить, что MS-DOS является 16-разрядной операционной системой, а Windows 95, Windows 98 и Windows NT являются 32-разрядными операционными системами. С точки зрения программы это означает, что в MS-DOS и Windows 3.x используются 16-разрядные указатели, а в Win32 — 32-разрядные указатели. Для упрощения переноса программ, написанных для 16-разрядных ОС, в 32-разрядные ОС размер переменной типа `int` устанавливается равным размеру указателя. Поэтому в консольном приложении переменная типа `int` занимает 4 байта, а не два, как это имеет место в программах, написанных под DOS. В большинстве случаев это приводит лишь к незначительному росту объема памяти, занимаемого программой, но в некоторых случаях, например при работе с файлами, необходимо помнить о различии форматов целых чисел в различных операционных системах.

Не стоит рассматривать консольные приложения как некоторый рудимент, оставшийся от DOS, который вскоре исчезнет из Visual C++. Этим приложениям разработчики Visual C++ уделяют постоянное внимание, и именно на примере консольных приложений и возможностей мастера, создающего для них заготовки, наиболее ярко прослеживаются этапы развития Visual C++ при переходе от одной его версии к другой.

Создание консольного приложения

Чтобы создать заготовку консольного приложения с использованием мастера:

1. Закройте все открытые проекты и файлы и выберите команду **File**, **New** (Файл, Создать). Появится диалоговое окно **New** (Создать), раскрытое на вкладке **Projects** (Проекты).

2. Выделите пункт **Win32 Console Application** (консольное приложение для 32-разрядного Windows) и введите в текстовое окно **Project name** (Имя проекта) имя проекта Console. В результате этих действий диалоговое окно **New** примет вид, изображенный на рис. 1.1.

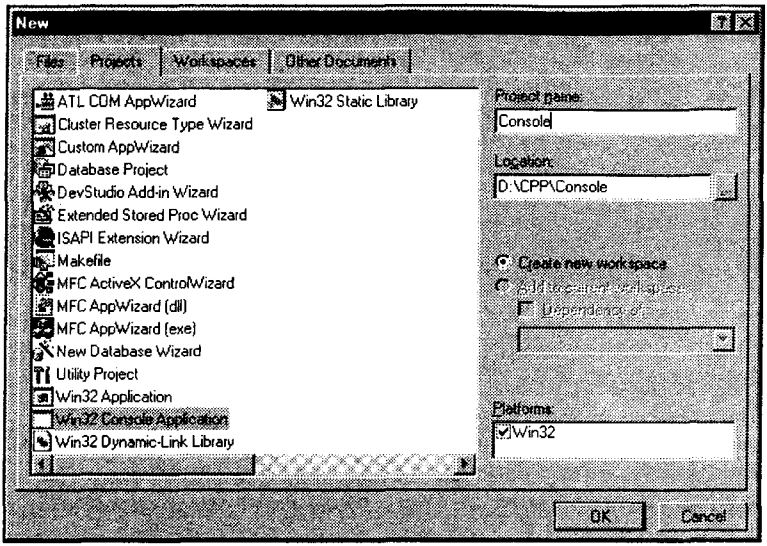


Рис. 1.1. Диалоговое окно **New**

3. Нажмите кнопку **OK**. Появится диалоговое окно **Win32 Console Application — Step 1 of 1** (Консольное приложение для 32-разрядного Windows. — Шаг 1 из 1), изображенное на рис. 1.2.

   Группа переключателей **What kind of Console Application do you want to create?** (Какое консольное приложение следует создать?) содержит следующие позиции:

- **An empty project** (Пустой проект) — мастер создает приложение, в котором отсутствуют файл реализации, файл заголовка и файл ресурсов. Возможность создания всех этих файлов полностью предоставляется вам
- **A simple application** (Простое приложение) — мастер создает приложение, состоящее из файла реализации и файла заголовка. В файле реализации отсутствуют исполнительные операторы
- **A "Hello World!" application** (Приложение "Hello World!") — мастер создает приложение, аналогичное предыдущему, но в нем файл реализации содержит оператор вывода на экран строки "Hello World!"
- **An application that supports MFC** (Приложение, использующее функции из библиотеки MFC) — мастер создает приложение, выводящее на экран строку, оформленную, как строковый ресурс

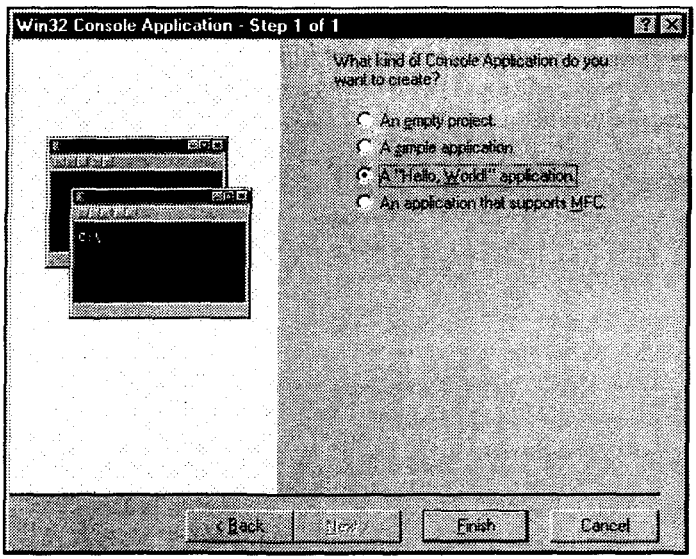


Рис. 1.2. Диалоговое окно **Win32 Console Application — Step 1 of 1**

4. После установки переключателя в соответствующее положение нажмите кнопку **Finish** (Завершение работы). Появится диалоговое окно **New Project Information** (Информация о создаваемом проекте), изображенное на рис. 1.3.

5. Нажмите кнопку **OK**. Мастер создаст новое приложение.

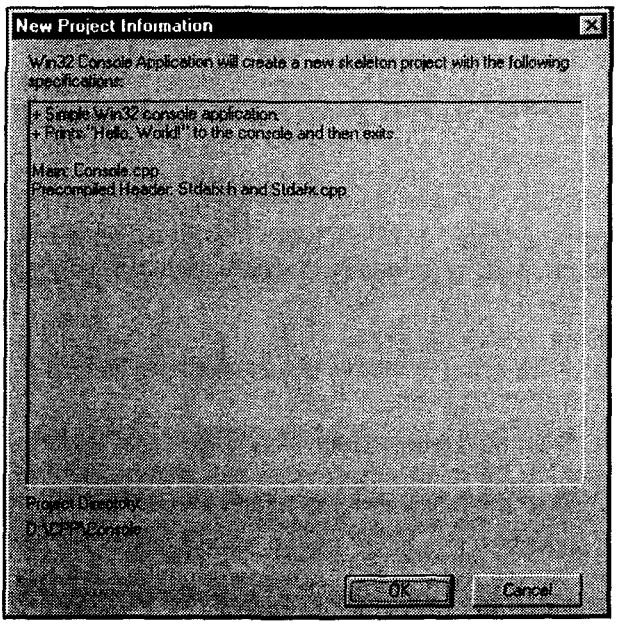


Рис. 1.3. Диалоговое окно **New Project Information**

Если вы установите переключатель выбора генерируемого приложения в положение **An empty project**, то вам придется писать все файлы программ проекта вручную, что нельзя назвать удачным выбором при генерации заготовки программы.

При установке переключателя выбора генерируемого приложения в положение **A simple application** в результате работы мастера будет создана простейшая заготовка файла реализации и файл заголовка StdAfx.h, относящийся к генерируемому мастером файлу StdAfx.cpp, ответственному за создание перекомпилированных заголовков. Текст заготовки файла реализации, получаемый при данных установках элементов управления, приведен в листинге 1.1.

Листинг 1.1. Заготовка файла реализации консольного приложения

```
// Console.cpp : Defines the entry point for the console application.

#include "stdafx.h"

int main(int argc, char* argv[])
{
  return 0;
}
```

Поскольку в данном проекте в функции `main` отсутствуют исполняемые операторы, то результатом запуска данного проекта на исполнение явится появление пустого окна MS-DOS, которое необходимо тут же закрыть. Как следует из текста данного листинга, эта установка является оптимальной для пользователя, не собирающегося пользоваться классами из библиотеки MFC.

Положения переключателя выбора генерируемого приложения **A "Hello World!" application** и **An application that supports MFC** являются чисто демонстрационными и разными способами реализуют одно и то же действие — вывод на экран строки символов. Это действие осуществлялось уже многократно и описано практически во всех учебниках и справочниках по C и C++. Как следует из листинга 1.2, первое демонстрационное приложение осуществляет это стандартным способом с использованием функции `printf`.

Листинг 1.2. Консольное приложение "Hello World!"

```
// Console.cpp : Defines the entry point for the console application.

#include "stdafx.h"

int main(int argc, char* argv[])
{
  printf("Hello World!\n");
  return 0;
}
```

Чтобы запустить данный проект на исполнение:

1. Выберите команду меню **Build**, **Execute Console.exe** (Компиляция, Запустить файл Console.exe на исполнение) или нажмите кнопку **Execute Program** (Запустить программу на исполнение) в панели инструментов **Build** (Компиляция) или в панели инструментов **Build MiniBar** (Уменьшенная панель инструментов Компиляция). Появится окно сообщения **Microsoft Visual C++**, изображенное на рис. 1.4. В этом окне сообщается, что исполняемый файл данного приложения отсутствует, и спрашивается о необходимости его создать.

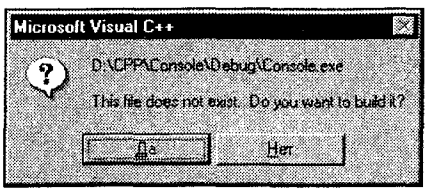


Рис. 1.4. Диалоговое окно **Microsoft Visual C++**

2. Нажмите на кнопку **Да**, подтверждая необходимость создания файла. Начнется компиляция программы, по окончании которой полученный исполнительный файл будет запущен, и появится окно MS-DOS, в котором будут отображены результаты работы программы. Это окно приведено на рис. 1.5.

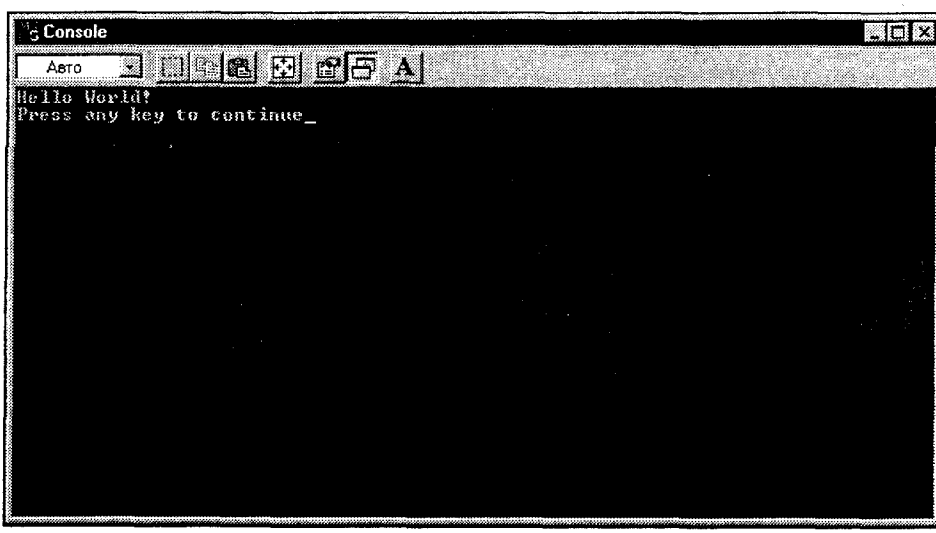


Рис. 1.5. Окно MS-DOS с результатом выполнения программы

Строка "Press any key to continue" (Нажмите любую клавишу для продолжения работы), расположенная после строки "Hello World!", выведенной программой, сообщает пользователю о том, что программа завершила работу,

и окно остается на экране только для того, чтобы пользователь мог прочитать результаты ее работы. После того как пользователь убедится, что программа корректно завершила свою работу, ему достаточно нажать на любую клавишу, чтобы закрыть окно MS-DOS и перейти к выполнению других задач.

Последнее, четвертое положение переключателя выбора генерируемого приложения — **An application that supports MFC** — служит для демонстрации того, что консольные приложения могут использовать функции из библиотеки MFC, однако, поскольку данная библиотека ориентирована, в основном, на работу с окнами и документами, большая ее часть остается невостребованной консольным приложением. Для того чтобы продемонстрировать возможности этой библиотеки в данном типе приложения, выводимая на экран строка оформлена в виде строкового ресурса. Таким образом, в данном приложении генерируются файлы реализации, файлы заголовков и файл описания ресурсов. Текст файла реализации данного приложения приведен в листинге 1.3.

Листинг 1.3. Консольное приложение, использующее библиотеку MFC

```
// Console.cpp : Defines the entry point for the console application.

#include "stdafx.h"
#include "Console.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

//////////////////////////////////////////////////////////////////////
// The one and only application object

CWinApp theApp;

using namespace std;

int _tmain(int argc, TCHAR* argv[], TCHAR* envp[])
{
  int nRetCode = 0;

  // initialize MFC and print and error on failure
  if (!AfxWinInit(::GetModuleHandle(NULL), NULL, ::GetCommandLine(), 0))
  {
    // TODO: change error code to suit your needs
    cerr << _T("Fatal Error: MFC initialization failed") << endl;
    nRetCode = 1;
  }
```

```
  else
  {
    // TODO: code your application's behavior here.
    CString strHello;
    strHello.LoadString(IDS_HELLO);
    cout << (LPCTSTR)strHello << endl;
  }

  return nRetCode;
}
```

Сравнивая листинги 1.2 и 1.3, следует обратить внимание, что в них обработка начинается с вызова функции `AfxWinInit` и проверки возвращаемой ею величины. Обычно эта функция вызывается функцией `WinMain`, являющейся частью библиотеки MFC, и этот вызов происходит в процессе инициализации объекта класса `CWinApp`. Поскольку консольные приложения не используют функцию `WinMain`, то в них необходимо непосредственно вызывать функцию `AfxWinInit` для инициализации библиотеки MFC.

При непосредственном вызове пользователем функции `AfxWinInit` в качестве первого ее параметра необходимо указать дескриптор текущего модуля, в котором хранится объект класса `CWinApp`. В случае консольного приложения пользователь может выбрать в качестве текущего модуля объект, производный от класса `CWinApp`, или непосредственно использовать объект класса `CWinApp`. Последняя стратегия применяется в том случае, когда вся обработка информации сосредоточена в функции `main` данного приложения.

Функция `GetModuleHandle`, используемая в качестве первого аргумента функции `AfxWinInit`, возвращает дескриптор модуля по имени файла (.exe или .dll), в котором содержится исполняемый код данного модуля. Если, как в данном случае, параметром данной функции является `NULL`, то функция `GetModuleHandle` возвращает дескриптор файла, использованного для создания данного процесса.

Функция `GetCommandLine`, используемая в качестве третьего аргумента функции `AfxWinInit`, возвращает указатель на командную строку текущего процесса. Этот указатель может быть использован консольным процессом, оформленным в соответствии со стандартом ANSI языка программирования C, для доступа к аргументам командной строки при помощи параметров `argc` и `argv`, передаваемых функции `main`.

Если функция `AfxWinInit` возвращает значение `FALSE`, то программа выдает сообщение об ошибке с использованием стандартного потока для вывода ошибок и прекращает свою работу. В противном случае формируется объект класса `CString`, описанного в библиотеке MFC. В этот объект, имеющий имя `strHello`, загружается строковый ресурс `IDS_HELLO`, а затем объект выводится на экран с использованием стандартного потока вывода `cout`.

Процедура запуска программы на исполнение и результаты ее работы в основном аналогичны процедуре запуска и результатам работы описанной до этого программы, не использующей классы MFC, поэтому нет никакого смысла описывать эту процедуру еще раз.

Данная программа может быть использована в качестве заготовки консольного приложения, использующего библиотеку MFC. Для этого вместо фрагмента программы, осуществляющего вывод строки на экран, должен размещаться текст пользовательской программы.

Использование консольного приложения

Как уже говорилось, консольное приложение является настоящим подарком для истинных приверженцев программирования под DOS. Оно позволяет им с минимальными потерями перейти к программированию под Windows и продолжать использовать привычный надежный и простой пользовательский интерфейс.

Другой областью использования консольного приложения является тестирование новых алгоритмов. Если эти тестируемые алгоритмы не связаны с выводом на экран графической информации, использование консольного приложения является лучшим выходом по созданию среды для проверки их функционирования. Единственное неудобство, связанное с необходимостью включения в файл ресурсов консольного приложения строковых ресурсов, к которым обращается пользовательская библиотека программ, может быть легко устранено использованием заголовочных файлов, включаемых в этот файл ресурсов.

Диалоговое приложение

Диалоговое приложение не может иметь никаких меню, кроме системного, а также не может открывать или сохранять информацию в файле, что накладывает существенные ограничения на область его использования. Это связано с тем, что диалоговое приложение не работает с документами, а стандартные диалоговые окна открытия и сохранения файла построены на основе этой концепции. Таким образом, диалоговое приложение или должно получать извне всю информацию об используемых в нем файлах, или выполнять задачи, не связанные с файлами. Примером такого приложения служит таблица символов Windows.

Однако не следует забывать, что диалоговое приложение является полноценным приложением Windows, в котором может быть использована поддержка технологии ActiveX, что позволяет решить вопрос с передачей данных в приложение и из приложения. С другой стороны, это приложение может быть оформлено в виде самостоятельного модуля, что позволит задать ему при выполнении уровень приоритета, недоступный для потока основ-

ного приложения. Поэтому диалоговое приложение может применяться для реализации критичных ко времени исполнения фрагментов программы.

Чтобы создать заготовку диалогового приложения с использованием мастера AppWizard:

1. Закройте все открытые проекты и файлы и выберите команду **File**, **New**. Появится диалоговое окно **New**, раскрытое на вкладке **Projects**, как это показано на рис. 1.1.
2. Выберите пункт **MFC AppWizard (exe)** и введите в текстовое окно **Project name** имя проекта "DialogApplication". Эти действия аналогичны действиям, предпринятым на первом шаге создания консольного приложения.
3. Нажмите кнопку **OK**. Появится диалоговое окно **MFC AppWizard — Step 1** (Мастер создания заготовки проекта, использующего библиотеку MFC. — Шаг 1).
   - Переключатель **What type of application would you like to create?** (Какой тип приложения следует создавать?) имеет следующие положения:

     Single document (Однооконное приложение)

     Multiple documents (Многооконное приложение)

     Dialog based (Диалоговое приложение)
   - Кроме того, при установке переключателя в положение **Single document** или **Multiple documents** появляется возможность установки флажка **Document/View architecture support?** (Использовать архитектуру Документ/Представление?)
4. Установите переключатель **Dialog Based**. При этом диалоговое окно примет вид, изображенный на рис. 1.6.

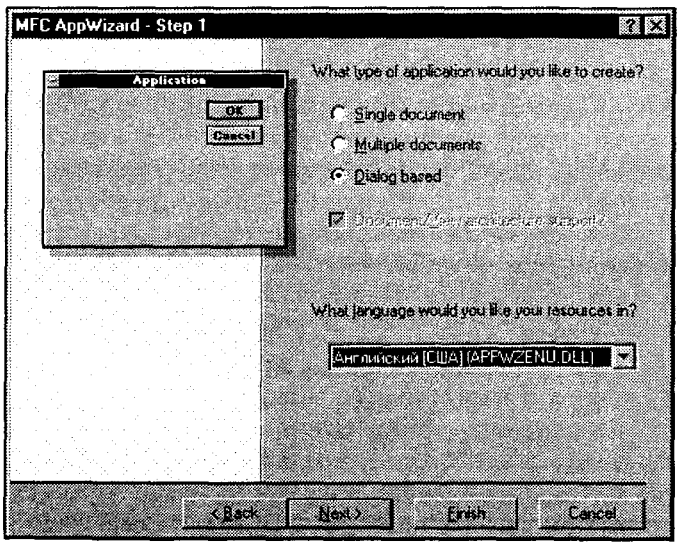


Рис. 1.6. Диалоговое окно **MFC AppWizard — Step 1**

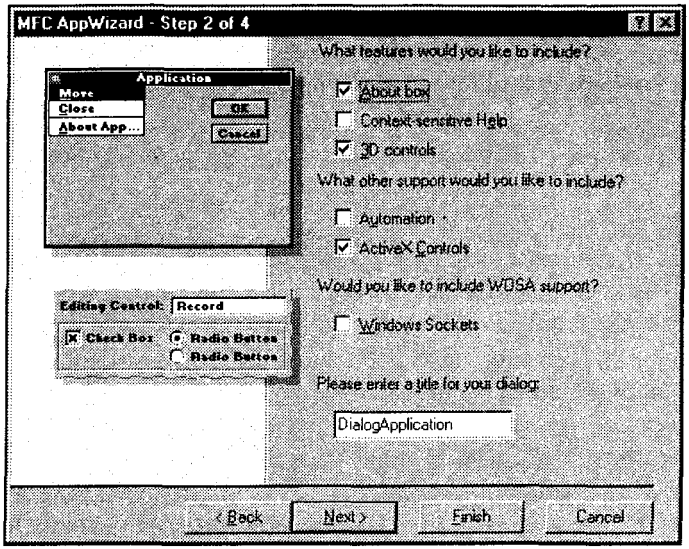


Рис. 1.7. Диалоговое окно **MFC AppWizard — Step 2 of 4**

5. Нажмите кнопку **Next>** (Далее). Появится диалоговое окно **MFC AppWizard — Step 2 of 4** (Второй шаг из четырех мастера AppWizard по генерации заготовки программы, использующей MFC), изображенное на рис. 1.7.

- В группу **What features would you like to include?** (Какое свойства должно иметь создаваемое окно?) объединены следующие флажки:

  About box (Диалоговое окно свойств) — определяет возможность вызова из создаваемого диалогового окна другого диалогового окна, в котором будет содержаться информация о версии программы и ее разработчике и которое будет вызываться по умолчанию

  Context-sensitive Help (Контекстная справка) — определяет необходимость включения в создаваемое приложение справочной системы. По умолчанию в приложении отсутствует справочная система

  См. также главу 13

  3D controls (Объемные элементы управления) — определяет внешний вид элементов управления диалогового окна. Объемные элементы управления используются по умолчанию

- В группе **What other support would you like to include?** (Какие дополнительные свойства должно иметь создаваемое окно?) объединены следующие флажки:

  Automation (Автоматизация) — определяет способность данного приложения отдать управление другому приложению через механизм Automation. По умолчанию такая возможность не предусмотрена

ActiveX Controls (Элементы управления ActiveX) — определяет возможность использования в создаваемом окне элементы управления ActiveX. Возможность предусмотрена по умолчанию

В данной книге вопросы, связанные с автоматизацией и элементами управления ActiveX, не рассматриваются. Поэтому во всех примерах используются установки этих флажков по умолчанию

- В группе **Would you like to include WOSA support?** (Следует ли включить в приложение поддержку Internet?), имеется всего один флажок — **Windows Sockets**, назначение которого ясно без перевода. По умолчанию поддержка Internet отсутствует

  В данной книге вопросы, связанные с работой в Internet не рассматриваются. Поэтому во всех приложениях используется установка этого флажка по умолчанию

- В текстовом поле **Please enter a title for your dialog:** (Введите заголовок диалогового окна) содержится предлагаемый мастером AppWizard заголовок создаваемого диалогового окна. Пользователь может ввести в данное текстовое поле любой заголовок

6. Оставьте все установки без изменений и нажмите кнопку **Next>**. Появится диалоговое окно **MFC AppWizard — Step 3 of 4** (Третий шаг из четырех мастера AppWizard по генерации заготовки программы, использующей MFC), изображенное на рис. 1.8.

   - Группа переключателей **What style of project would you like?** (Выбор стиля проекта) имеет следующие положения:

     MFC Standard (Стандартное приложение MFC). Это установка по умолчанию

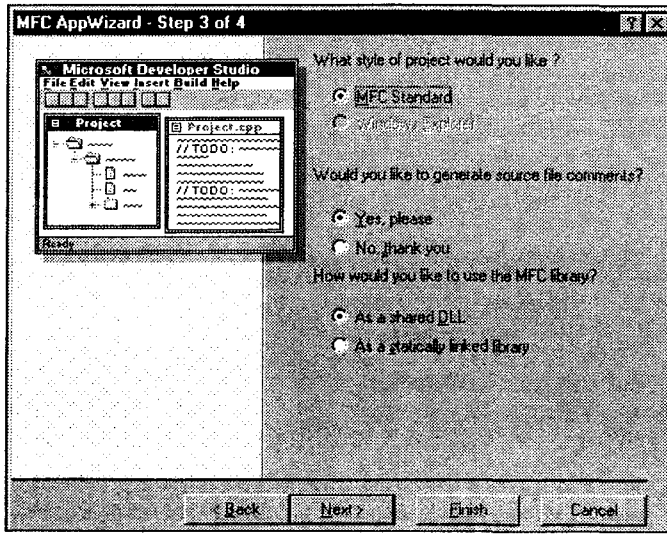


Рис. 1.8. Диалоговое окно **MFC AppWizard — Step 3 of 4**

Windows Explorer (Стиль браузера Internet). Возможность переключения в это положение в данном случае отсутствует

- Группа переключателей **Would you like to generate source file comments?** (Включать ли комментарии в тексты исходных файлов?) имеет следующие положения:

  Yes, please (Включите, пожалуйста). Это установка по умолчанию

  No, thank you (Спасибо, не надо)

- Группа переключателей **How would you like to use the MFC library?** (В каком виде использовать библиотеку MFC?) имеет следующие положения:

  As a shared DLL (Как разделяемые библиотеки динамической компоновки). Такой выбор уменьшает размер исполняемого файла и снижает объем используемой оперативной памяти. Это установка по умолчанию

  As a statically linked library (Как статические библиотеки). Такой выбор позволяет программе работать на компьютере, где не установлены необходимые библиотеки динамической компоновки

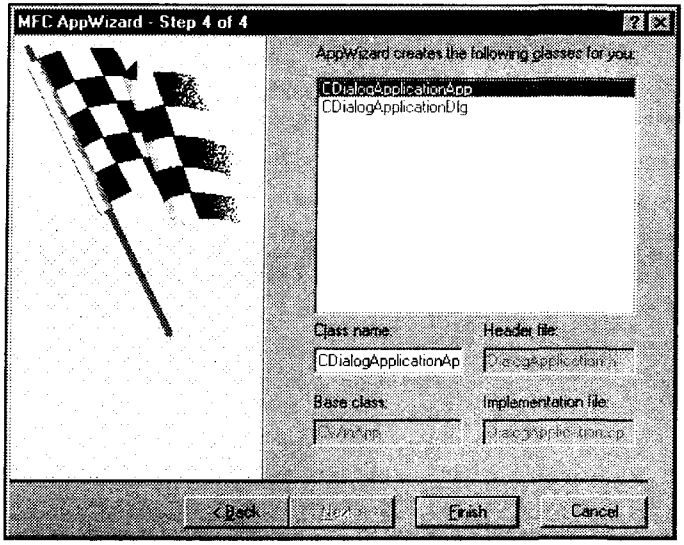


Рис. 1.9. Диалоговое окно **MFC AppWizard — Step 4 of 4**

7. Оставьте все установки данного диалогового окна без изменений и нажмите кнопку **Next>**. Появится диалоговое окно **MFC AppWizard — Step 4 of 4** (Четвертый шаг из четырех мастера AppWizard по генерации заготовки программы, использующей MFC), изображенное на рис. 1.9.

   - В этом диалоговом окне перечислены создаваемые мастером AppWizard классы. При выделении класса в верхнем окне в соответствующих

текстовых полях, расположенных ниже, появляется имя этого класса, имя базового класса, имя файла заголовка, в котором содержится заголовок данного класса, и имя файла реализации, в котором содержится описание методов данного класса. Если соответствующая информация расположена на белом фоне, то она может быть изменена, если она расположена на сером фоне, то она является справочной информацией и не подлежит редактированию

- Это окно является последним окном мастера. Если вы согласны с содержащейся в данном окне информацией, переходите к п. 11. Если вас не устраивает какой-либо параметр, заданный в одном из предыдущих диалоговых окон:

8. Нажмите кнопку **<Back** (Возврат к предыдущему диалоговому окну) и вернитесь в предыдущее окно.
9. Если установка изменяемого параметра производится не в этом окне, повторите п. 8.
10. Измените требуемый параметр и нажмите кнопку **Next>** столько раз, сколько необходимо для возврата в диалоговое окно **MFC AppWizard — Step 4 of 4**.
11. Нажмите кнопку **Finish**. Появится диалоговое окно **New Project Information**, изображенное на рис. 1.10, в котором будут перечислены значения всех установленных параметров.
12. Если вы согласны с ними, нажмите кнопку **OK**, если же не согласны — нажмите кнопку **Cancel** и полностью повторите процедуру создания данного приложения.

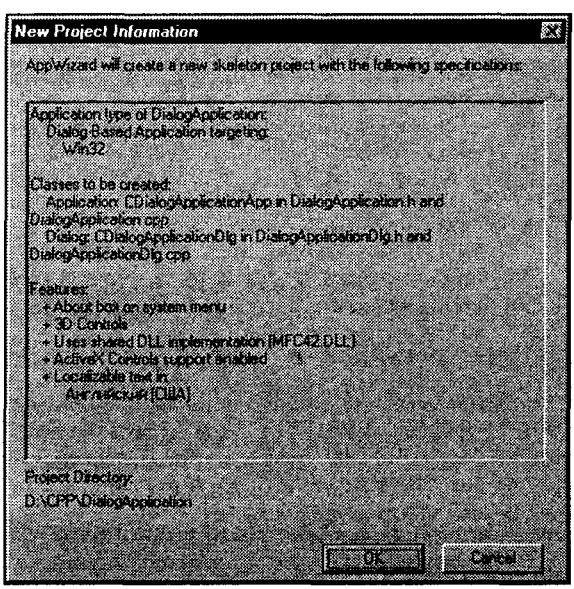


Рис. 1.10. Диалоговое окно **New Project Information**

Если вы нажали кнопку **OK**, то система генерирует необходимые файлы и открывает заготовку диалогового окна, изображенную на рис. 1.11.

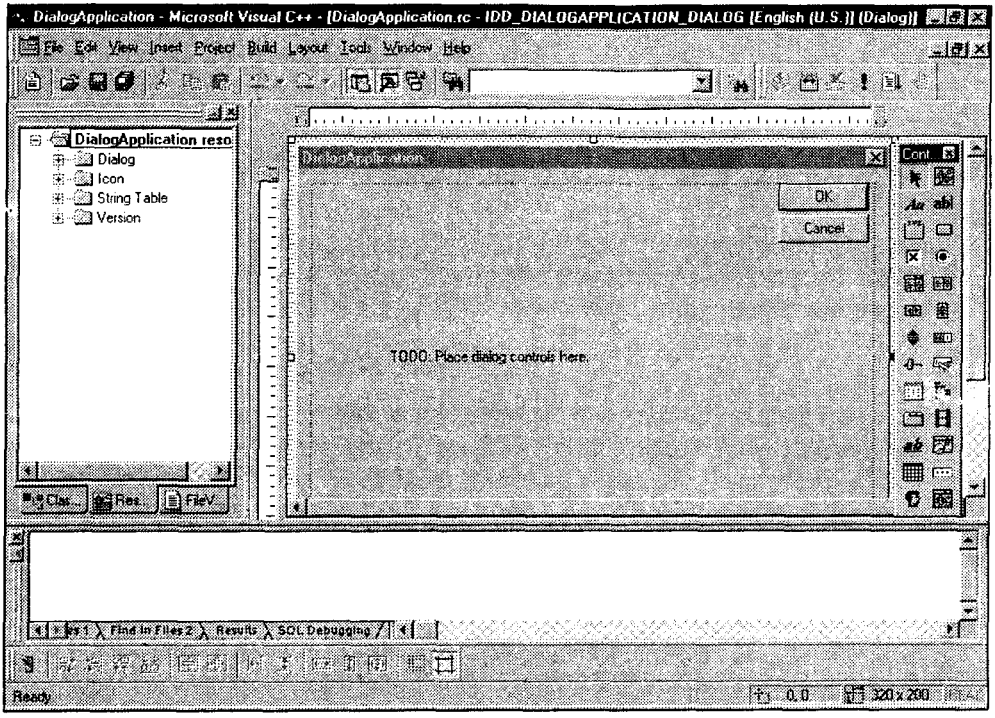


Рис. 1.11. Заготовка диалогового окна

Рядом с диалоговым окном расположена панель инструментов **Controls** (Элементы управления), позволяющая автоматизировать процесс создания диалогового окна. Элементы управления диалогового окна будут более подробно описаны в главе 3.

См. также главу 3

Многооконное приложение Windows

Под многооконным приложением Windows мы будем понимать приложение, использующее *многооконный интерфейс приложения* (Multiple Document Interface, MDI). Подобные приложения являются основными приложениями Windows, поэтому в дальнейшем мы будем в основном рассматривать именно их.

Чтобы создать заготовку оконного приложения Windows с использованием мастера AppWizard:

1. Закройте все открытые проекты и файлы и выберите команду меню **File**, **New**. Появится диалоговое окно **New**, раскрытое на вкладке **Projects**, как это показано на рис. 1.1.
2. Выделите пункт **MFC AppWizard (exe)** и введите в текстовое поле **Project name** имя проекта "MDIApplication". Эти действия, за исключением выбранного имени проекта, тождественны действиям, предпринятым на первом шаге создания диалогового приложения.
3. Нажмите кнопку **OK**. Появится диалоговое окно **MFC AppWizard — Step 1**, изображенное на рис. 1.6.
4. Оставьте переключатель типа генерируемого приложения в положении **Multiple documents** (Многооконное приложение) и не меняйте установку флажка **Document/View architecture support?** (Использовать архитектуру Документ/Представление?).
5. Если используемая вами версия Visual C++ не поддерживает русский язык при генерации проектов, оставьте в текстовом поле **What language would you like your resources in?** (Какой язык следует использовать при создании ресурсов?) значение **Английский (США) (APPWZENU.DLL)**.

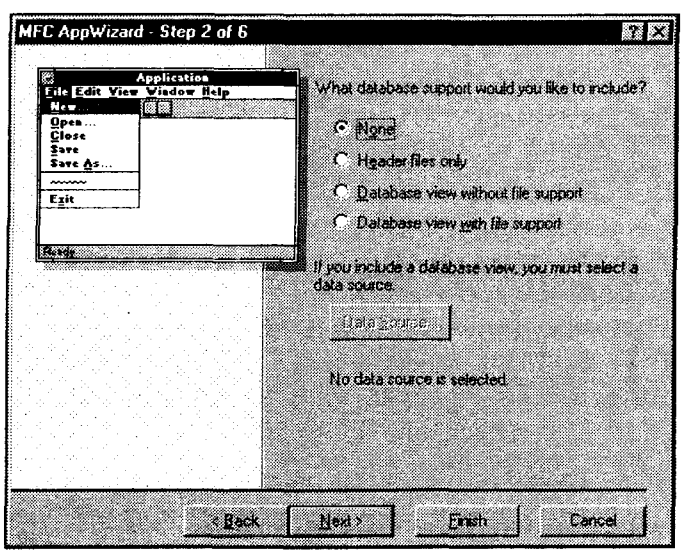


Рис. 1.12. Диалоговое окно **MFC AppWizard — Step 2 of 6**

6. Нажмите кнопку **Next>**. Появится диалоговое окно **MFC AppWizard — Step 2 of 6**, изображенное на рис. 1.12.
   - Группа переключателей **What database support would you like to include?** (Каким образом в проекте будут использованы базы данных) имеет следующие положения:

     None (Базы данных не используются). Это установка по умолчанию

Header files only (Только файлы заголовков). Переключатель следует устанавливать в это положение, если вы хотите иметь доступ к базе данных, но не хотите, чтобы ваш класс представления являлся производным от класса `CFormView`, или не хотите иметь меню **Record**

Database view without file support (Просмотр баз данных без файловой поддержки). Переключатель следует устанавливать в это положение, если вы хотите, чтобы ваш класс представления являлся производным от класса `CFormView`, или хотите иметь меню **Record**, но не нуждаетесь в использовании встроенных в документ методов работы с архивом (использовать метод `Serialize`)

Database view with file support (Базы данных при наличии файловой поддержки). Переключатель следует устанавливать в это положение, если вы нуждаетесь во всех возможностях, предоставляемых предыдущей установкой переключателя, и хотите использовать встроенные в документ методы работы с архивом

- В случае, если вы установили переключатель в последние два положения, вам придется нажать кнопку **Data Source...** (Источник данных) и выбрать исходную базу данных

7. Нажмите кнопку **Next>**. Появится диалоговое окно **MFC AppWizard — Step 3 of 6**, изображенное на рис. 1.13.

   - Группа переключателей **What compound document support would you like to include?** (Каким образом в проекте будут использованы составные документы?) имеет следующие положения:

     None (Составные документы не используются). Это установка по умолчанию

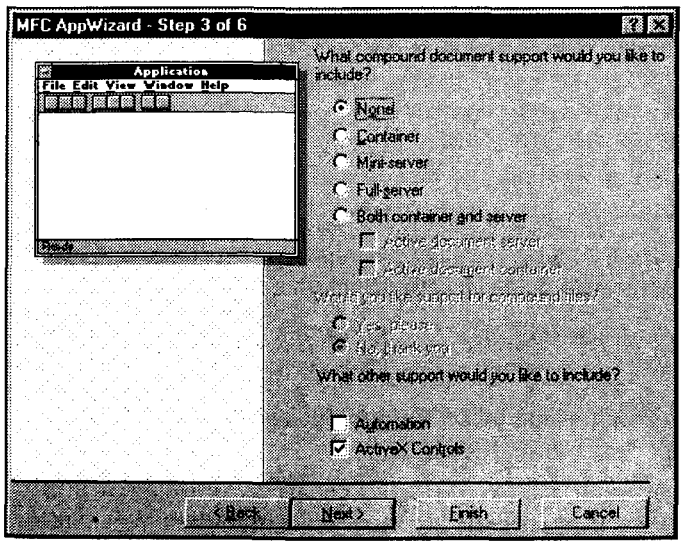


Рис. 1.13. Диалоговое окно **MFC AppWizard — Step 3 of 6**

Container (Контейнер). При данной установке переключателя ваше приложение сможет содержать связанные или внедренные объекты ActiveX, такие как документы Word или Excel

Mini-server (Мини-сервер). При данной установке переключателя ваше приложение будет способно создавать документы, которые могут внедряться в другие приложения, но которые не предполагается запускать как самостоятельные приложения

Full-server (Полноценный сервер). При данной установке переключателя ваше приложение будет создавать документы, которые не только могут внедряться в другие приложения, но и запускаться как самостоятельные приложения

Both container and server (Одновременно сервер и контейнер). При данной установке переключателя ваше приложение будет создавать документы, которые не только могут внедряться в другие приложения, но и документы других приложений будут иметь возможность внедряться в ваше приложение

- Если ваше приложение будет являться сервером, то у вас появится возможность установить флажок **Active document server** (Сервер активного документа), а если ваше приложение является контейнером, то у вас появится возможность установить флажок **Active document container** (Контейнер активного документа). Оба этих флажка по умолчанию сброшены. Если вы установите хотя бы один из них, то на следующем этапе вам придется нажать кнопку **Advanced** и определить расширение для обрабатываемого документа. Если вы этого не сделаете, то не сможете не только перейти к следующему окну, но и вернуться к данному и снять этот флажок

- Если вы выбрали поддержку составных документов, то вам необходимо выбрать и поддержку *составных файлов* (compound files). Составные файлы содержат один или несколько объектов ActiveX и должны храниться особым образом, чтобы любой объект мог быть изменен без необходимости внесения изменений в весь файл, что позволяет сэкономить массу времени

- Группа переключателей **Would you like support for compound files?** (Использовать ли составные файлы?) имеет следующие положения:

  Yes, please (Включите, пожалуйста). Это установка по умолчанию

  No, thank you (Спасибо, не надо)

- Группа **What other support would you like to include?** (Что еще включить в проект?) содержит следующие флажки:

  Automation (Автоматизация). Этот флажок следует установить в том случае, если вы хотите, чтобы ваше приложение передавало управле-

ние собой другому приложению посредством механизма Automation. По умолчанию такая возможность не предоставляется

ActiveX Controls (Элементы управления ActiveX). Этот флажок следует установить, если в приложении предполагается использование элементов управления ActiveX. Использование подобных элементов управления в проекте предполагается по умолчанию

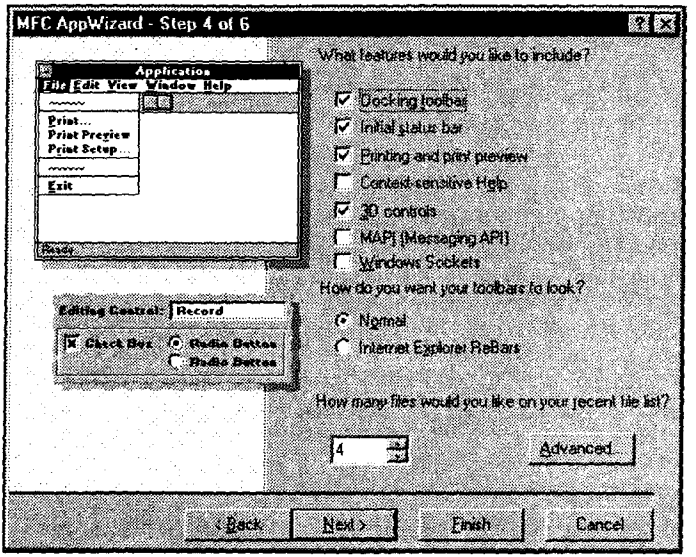


Рис. 1.14. Диалоговое окно **MFC AppWizard — Step 4 of 6**

8. Оставьте все элементы управления в их положении по умолчанию и нажмите кнопку **Next>**. Появится диалоговое окно **MFC AppWizard — Step 4 of 6**, изображенное на рис. 1.14.

- В группу **What features would you like to include?** (Какие свойства должно иметь создаваемое окно?) объединены следующие флажки:

  Docking toolbar (Перемещаемая панель инструментов). При установке данного флажка главное окно программы будет содержать панель инструментов, которая может быть расположена вдоль любой из его сторон. По умолчанию этот флажок установлен

  См. также главу 9

  Initial status bar (Панель состояния). При установке данного флажка главное окно программы будет содержать панель состояния, отображающую справочную информацию по пунктам меню, кнопкам панели инструментов, а также иную справочную информацию. По умолчанию этот флажок установлен

  См. также главу 9

Printing and print preview (Печать и предварительный просмотр печати). При установке этого флажка в меню **File** (Файл) создаваемого приложения будут находиться пункты **Print** (Печать) и **Print preview** (Предварительный просмотр печати), а мастер включит в приложение большую часть программного кода, связанного с реализацией этих задач. По умолчанию этот флажок установлен

См. также главу 10

Context-sensitive Help (Контекстная справка). При установке этого флажка в создаваемое приложение включается справочная система. По умолчанию этот флажок снят

См. также главу 13

3D controls (Объемные элементы управления). При установке этого флажка в создаваемом приложении будут использоваться объемные элементы управления. По умолчанию этот флажок установлен

MAPI (Messaging API) (Почтовый интерфейс MAPI). При установке этого флажка создаваемое приложение сможет обмениваться сообщениями по электронной почте и отправлять факсы. По умолчанию этот флажок снят

Windows Sockets (Поддержка Internet). При установке этого флажка создаваемое приложение сможет иметь непосредственный доступ к Internet, используя такие протоколы, как FTP и HTTP. По умолчанию этот флажок снят

- Группа переключателей **How do you want your toolbars to look?** (Как должна выглядеть панель инструментов?) имеет следующие положения:

  Normal (Нормальная). При данной установке переключателя ваше приложение будет иметь стандартную панель инструментов. Это установка по умолчанию

  Internet Explorer ReBars (Панели инструментов Internet Explorer). При данной установке переключателя ваше приложение будет иметь панель инструментов, выполненную в стандарте браузера Internet Explorer

- В текстовом поле с инкрементным регулятором **How many files would you like on your recent file list?** (Сколько имен файлов должен содержать список последних закрытых файлов?) устанавливается размерность списка последних закрытых файлов

- Нажатие кнопки **Advanced...** (Дополнительные установки) позволяет произвести дополнительные установки. При ее нажатии появляется диалоговое окно **Advanced Options** (Дополнительные установки), изображенное на рис. 1.15

- Из всех текстовых полей данного окна нас больше всего интересует текстовое поле **File extension:** (Расширение файла), поскольку именно

в него должна быть введена информация о расширении файла обрабатываемого документа, если на третьем шаге мастера вы установили флажок **Active document server** или флажок **Active document container**

- Как уже говорилось ранее, при установке указанных флажков без заполнения этого текстового поля пользователь не сможет завершить четвертый шаг мастера AppWizard

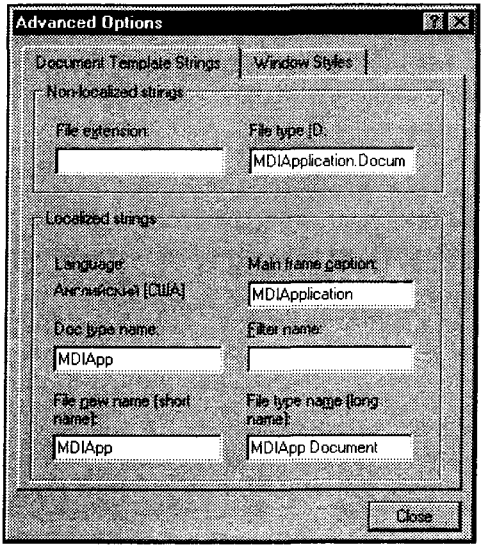


Рис. 1.15. Диалоговое окно **Advanced Options**

9. Закройте, если вы его открыли, диалоговое окно **Advanced Options**, оставьте без изменения все установки по умолчанию и нажмите кнопку **Next>** в диалоговом окне **MFC AppWizard — Step 4 of 6**. Появится диалоговое окно **MFC AppWizard — Step 5 of 6**, изображенное на рис. 1.16.

- Группа переключателей **What style of project would you like?** (Какой стиль следует избрать для создаваемого проекта?) имеет следующие положения:

  MFC Standard (В стандарте MFC). При данной установке переключателя будет создано стандартное приложение, использующее библиотеку MFC. Его внешний вид приведен на рис. 1.17. Это установка по умолчанию

  Windows Explorer (Стиль браузера Internet). Внешний вид полученного при этой установке приложения с именем ExplorerApplication приведен на рис. 1.18

- Группа переключателей **Would you like to generate source file comments?** (Включать ли комментарии в тексты исходных файлов?) имеет следующие положения:

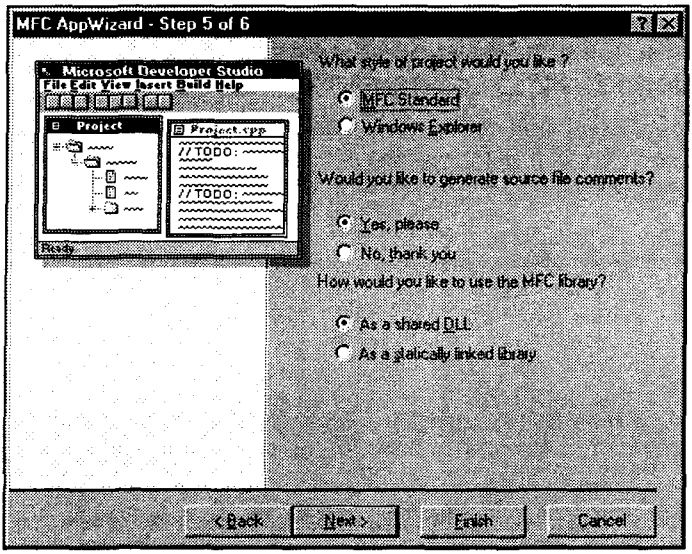


Рис. 1.16. *Диалоговое окно* **MFC AppWizard — Step 5 of 6**

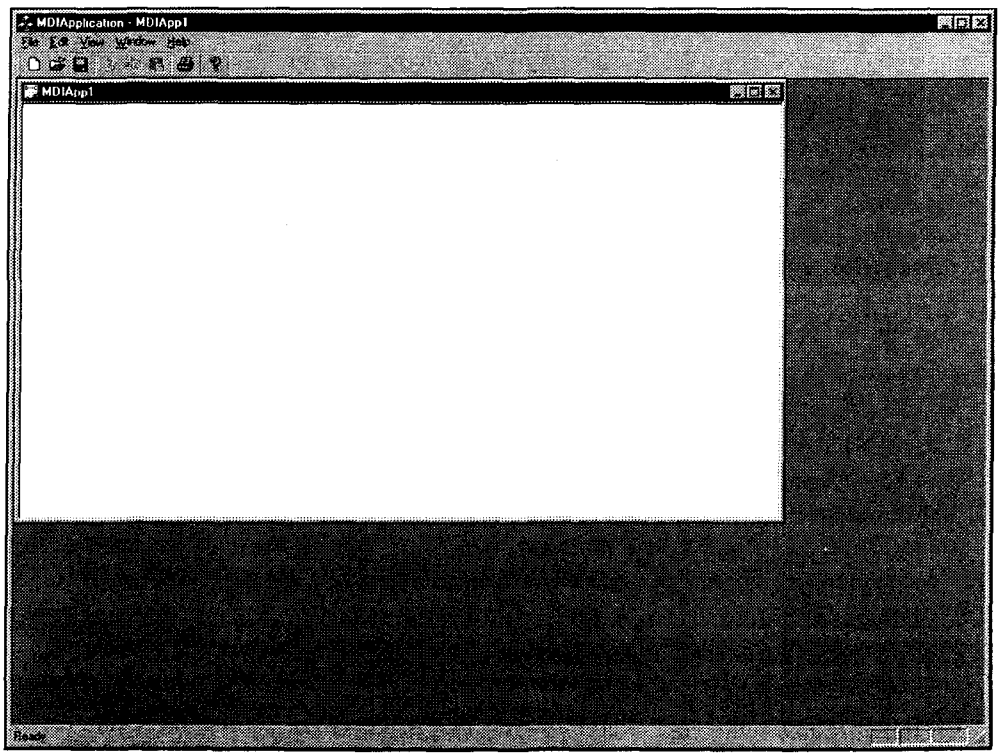


Рис. 1.17. Внешний вид стандартного приложения MFC

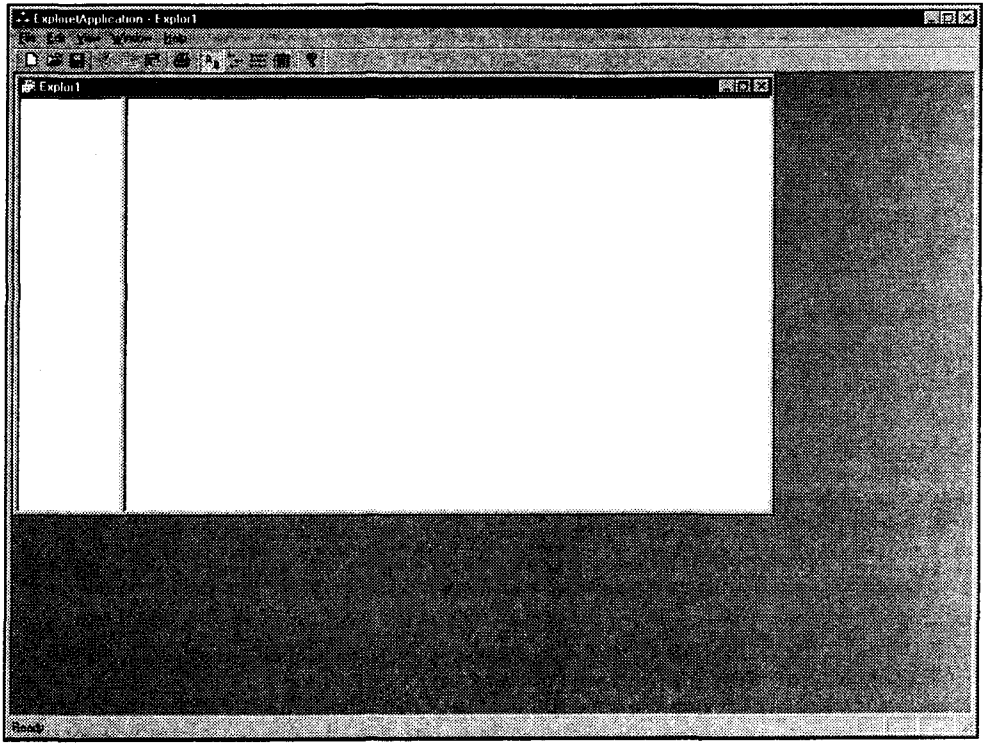


Рис. 1.18. Внешний вид приложения в стандарте Internet Explorer

Yes, please (Включите, пожалуйста). Это установка по умолчанию

No, thank you (Спасибо, не надо)

- Группа переключателей **How would you like to use the MFC library?** (В каком виде использовать библиотеку MFC?) имеет следующие положения:

  As a shared DLL (Как разделяемые библиотеки динамической компоновки). Такой выбор уменьшает размер исполняемого файла и снижает объем используемой оперативной памяти. Это установка по умолчанию

  As a statically linked library (Как статические библиотеки). Такой выбор позволяет программе работать на компьютере, где не установлены необходимые библиотеки динамической компоновки

  Эти переключатели аналогичны установкам на шаге 3 создания диалогового приложения

10. Оставьте без изменения все установки по умолчанию и нажмите кнопку **Next>**. Появится диалоговое окно **MFC AppWizard — Step 6 of 6**, изображенное на рис. 1.19.

Это окно и его поля аналогичны диалоговому окну, получаемому на шаге 4 создания диалогового приложения. Однако при создании многооконного приложения возникает один вопрос, который целесообразно осветить именно в этом месте

Как видно из рис. 1.19, мастер AppWizard собирается создать классы с именами `CMDIApplicetionView` и `CMDIApplicetionDoc`, которые будут подробно рассмотрены в главе 2. Это классы представления и документа. Их имена связаны с именем приложения. Этот подход целесообразен в случае, когда в приложении будет только один класс представления и связанный с ним класс документа. Однако в большинстве случаев в многооконном приложении приходится работать с несколькими классами документов и соответствующими им классами представления. В этом случае, если не внести изменения в имена этих классов, одна пара классов получает "слепые" имена, не связанные с особенностями обрабатываемых ими документов и представлений. Конечно, можно уничтожить созданные мастером классы, но целесообразнее просто переименовать эти классы по имени одного из видов обрабатываемых документов. В данном случае создается демонстрационное приложение, поэтому имена классов оставлены без изменений

См. также главу 2

Раскрывающийся список **Base class:** позволяет выбрать базовый класс для класса представления данного документа. Если приложение создается для вывода графической информации, то нет необходимости изменять выбранный по умолчанию базовый класс `CView`. Однако если приложение предполагается использовать как текстовый редактор, то в качестве базового класса следует выбрать класс `CEditView` или класс

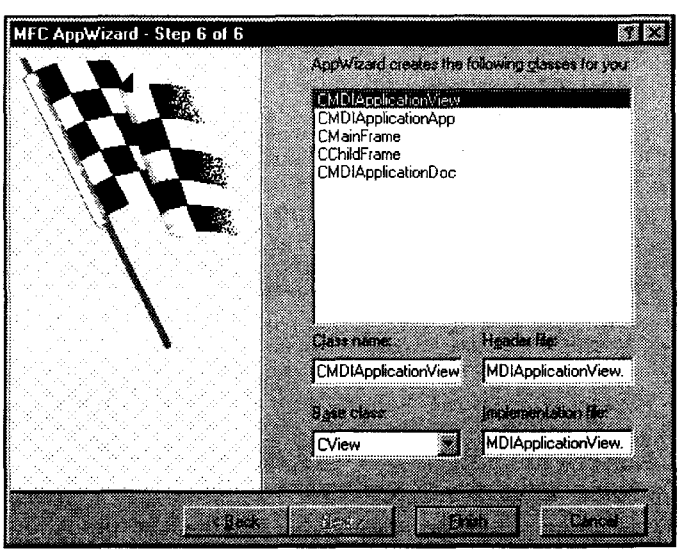


Рис. 1.19. Диалоговое окно **MFC AppWizard — Step 6 of 6**

CRichEditView, как это описано в главе 8. Существуют и другие случаи, в которых необходимо изменять содержимое текстового поля раскрывающегося списка **Base class:**

См. также главу 8

Если вы согласны с приведенным в данном окне перечнем создаваемых файлов и классов, перейдите к п. 12.

11. При необходимости внесите необходимые изменения в текстовые поля данного окна или нажмите кнопку **<Back** и внесите изменения в содержимое предыдущих окон, как это было описано в пп. 8—10 методики создания диалогового приложения.
12. Нажмите кнопку **Finish**. Появляется диалоговое окно **New Project Information**, изображенное на рис. 1.20.

    Это окно уже рассматривалось при описании создания предыдущих проектов

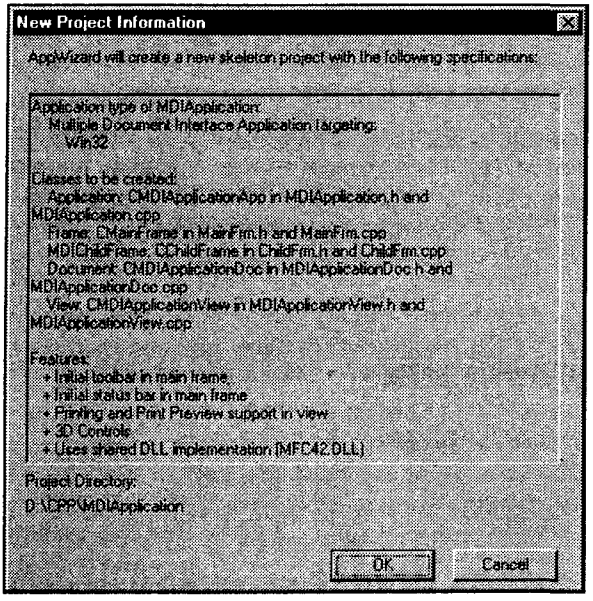


Рис. 1.20. Диалоговое окно **New Project Information**

13. Если вы согласны создать описанный в этом окне проект, нажмите кнопку **OK**. Будет создан проект, подробно рассмотренный в главе 2.

Однооконное приложение

В диалоговом окне **MFC AppWizard — Step 1**, изображенном на рис. 1.6, осталось одно до сих пор не рассмотренное положение переключателя **What type of application would you like to create?**, а именно, **Single Document**. При

установке переключателя в это положение будет создано однооконное приложение, которое иногда называют еще приложением, использующим один документ. Это приложение, по своей сути, занимает промежуточное положение между диалоговым и многооконным приложениями.

Данный тип приложения может использоваться для обработки конкретного типа документа. При этом предполагается, что при обработке данного документа не требуется использования других документов. Подобные задачи, но без использования документов, решает диалоговое приложение. Однако такие задачи, как правило, требуют обеспечения дружественного интерфейса пользователя. С этой точки зрения диалоговые приложения имеют большое преимущество по сравнению с однооконными приложениями, единственным достоинством которых является возможность работы с документом, за которую приходится платить сведением всего пользовательского интерфейса к работе с меню и панелями инструментов.

Обычно обработка одного документа предполагает возможность одновременной работы со связанными с ним документами, например, для внесения в них соответствующих изменений по ходу работы или получения из них необходимой информации. Поэтому при любой работе с документами лучше сразу предусматривать использование многооконного интерфейса.

В силу указанных выше причин здесь не будет подробно описана процедура создания однооконного приложения, тем более что она во многом аналогична процедуре создания многооконного приложения. Вы можете создать подобное приложение самостоятельно в качестве проверки своих знаний по работе с мастером AppWizard.

Классы приложений, документов и представлений

Как уже говорилось в предыдущей главе, в основе концепции программирования в среде Windows с использованием библиотеки классов MFC лежит концепция Документ/Представление. Эта концепция подразумевает, что в каждом приложении должны быть четко разделены функции манипулирования информацией и функции ее отображения.

См. также главу 1

Поэтому каждое приложение, построенное в соответствии с данной концепцией, должно включать в себя описание класса, производного от класса `CDocument`, на который возлагаются задачи хранения обрабатываемой информации в оперативной памяти, предоставление этой информации по внешним запросам, запись и чтение этой информации с диска. Это же приложение должно включать в себя описание класса, производного от класса `CView`, на который возлагаются задачи отображения этой информации на экране и реализация интерфейса пользователя. Кроме этих двух классов, приложение может содержать и другие полезные классы, но именно взаимодействие этих двух классов в рамках *шаблона документа* позволяет существенно упростить программирование многих стандартных операций по взаимодействию создаваемого приложения с операционной системой Windows.

Другим неотъемлемым компонентом любой программы, использующей библиотеку MFC, является класс приложения, производный от класса `CWinApp`. Методы данного класса отвечают за инициализацию, работу и правильное завершение всего приложения.

Класс документа

Рассмотрим класс документа в многооконном приложении, созданном в главе 1. Полный текст данного приложения содержится в директории

MDIApplication на прилагаемой к данной книге дискете. Порядок работы с данной дискетой и ее содержимое описаны в главе 15.

Если вы еще не создали это приложение, обратитесь к главе 1 и выполните все описанные в ней действия по созданию многооконного приложения.

В листинге 2.1 представлен файл MDIApplicationDoc.h, содержащий заголовок класса.

Листинг 2.1. Файл заголовка класса CMDIApplicationDoc

```
CMDIApplicationDoc.
// MDIApplicationDoc.h : interface of the CMDIApplicationDoc class
////////////////////////////////////////////////////////////////////////

#if !defined(AFX_MDIAPPLICATIONDOC_H__F06A8B2F_855E_11D2_82AE_
                                                    9FE355108C6E__INCLUDED_)
#define AFX_MDIAPPLICATIONDOC_H__F06A8B2F_855E_11D2_82AE_9FE355108C6E
                                                              __INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CMDIApplicationDoc : public CDocument
{
protected: // create from serialization only
  CMDIApplicationDoc();
  DECLARE_DYNCREATE(CMDIApplicationDoc)

// Attributes
public:

// Operations
public:

// Overrides
  // ClassWizard generated virtual function overrides
  // {{AFX_VIRTUAL(CMDIApplicationDoc)
  public:
  virtual BOOL OnNewDocument();
  virtual void Serialize(CArchive& ar);
  //}}AFX_VIRTUAL

// Implementation
public:
  virtual ~CMDIApplicationDoc();
#ifdef _DEBUG
  virtual void AssertValid() const;
```

```
  virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
  //{{AFX_MSG(CMDIApplicationDoc)
    // NOTE - the ClassWizard will add and remove member functions here.
    // DO NOT EDIT what you see in these blocks of generated code !
  //}}AFX_MSG
  DECLARE_MESSAGE_MAP()
};

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately
before the previous line.

#endif      // !defined(AFX_MDIAPPLICATIONDOC_H__F06A8B2F_855E_11D2_82AE_
                                                    9FE355108C6E__INCLUDED_)
```

Структура файла заголовка и структура описания класса рассмотрены в Приложении А, поэтому сразу перейдем к рассмотрению элементов описания данного класса. Класс `CMDIApplicationDoc` является непосредственным потомком класса `CDocument`, динамически создаваемым в процессе исполнения программы. Он имеет единственный защищенный конструктор, из чего следует, что непосредственное создание объекта данного класса программистом не предусматривается. Мастер AppWizard создает для данного класса всего два метода для обработки информации и два метода для отладки. Кроме того, данный класс имеет пустую карту сообщений. Обычно эта карта так и остается незаполненной, поскольку в соответствии с концепцией Документ/Представление все сообщения должен принимать и обрабатывать объект класса представления, а объект класса документа должен предоставлять ему для этого необходимые методы по прямым запросам.

См. также главу 5

Чтобы подробнее ознакомиться с этими методами, рассмотрим файл реализации данного класса MDIApplicationDoc.cpp, приведенный в листинге 2.2.

Листинг 2.2. Файл реализации класса `CMDIApplicationDoc`

```
// MDIApplicationDoc.cpp : implementation of the CMDIApplicationDoc class

#include "stdafx.h"
#include "MDIApplication.h"
```

```
#include "MDIApplicationDoc.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationDoc

IMPLEMENT_DYNCREATE(CMDIApplicationDoc, CDocument)

BEGIN_MESSAGE_MAP(CMDIApplicationDoc, CDocument)
  //{{AFX_MSG_MAP(CMDIApplicationDoc)
    // NOTE - the ClassWizard will add and remove mapping macros here.
    // DO NOT EDIT what you see in these blocks of generated code!
  //}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationDoc construction/destruction

CMDIApplicationDoc::CMDIApplicationDoc()
{
  // TODO: add one-time construction code here

}

CMDIApplicationDoc::~CMDIApplicationDoc()
{
}

BOOL CMDIApplicationDoc::OnNewDocument()
{
  if (!CDocument::OnNewDocument())
    return FALSE;

  // TODO: add reinitialization code here
  // (SDI documents will reuse this document)

  return TRUE;
}

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationDoc serialization

void CMDIApplicationDoc::Serialize(CArchive& ar)
{
  if (ar.IsStoring())
```

```
    {
      // TODO: add storing code here
    }
    else
    {
      // TODO: add loading code here
    }
}

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationDoc diagnostics

#ifdef _DEBUG
void CMDIApplicationDoc::AssertValid() const
{
  CDocument::AssertValid();
}

void CMDIApplicationDoc::Dump(CDumpContext& dc) const
{
  CDocument::Dump(dc);
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationDoc commands
```

Как видно из листинга 2.2, созданный класс имеет пустые конструктор и деструктор. Это связано с тем, что стандартные операции по созданию и уничтожению элементов данного класса производятся в конструкторе и деструкторе базового класса, а данных, которые необходимо инициализировать или уничтожать в конструкторе и деструкторе, этот класс пока не содержит. Отсутствие данных в классе объясняется тем, что приведенный листинг представляет собой только заготовку пользовательского класса.

Функция `CDocument::OnNewDocument` будет вызываться всякий раз, когда понадобится создать пустой документ, например, в результате выбора команды меню **File**, **New** (или **Файл**, **Создать**, если вы русифицировали меню вашего приложения). По умолчанию эта функция просто вызывает соответствующий метод базового класса. Поскольку созданное нами приложение использует только один тип документов, то в результате выполнения данной функции в приложении появится пустое окно документа. Если бы данное приложение использовало несколько типов документов, то в результате выполнения данной функции появилось бы диалоговое окно **New**, изображенное на рис. 2.1.

Окно списка **New** данного диалогового окна содержит краткие имена документов, используемых в рассматриваемом приложении. Чтобы выбрать нуж-

ный тип документа, выделите его имя в окне списка **New** и нажмите кнопку **OK**. Появится пустое окно документа выбранного типа.

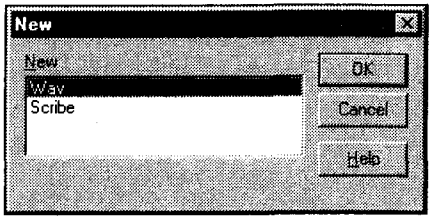


Рис. 2.1. Диалоговое окно **New**

Функция `CObject::Serialize` осуществляет чтение объекта из архива и запись его в архив. Единственным аргументом данной функции является объект класса `CArchive`, осуществляющий запись информации, содержащейся в объекте, в файл архива и последующее чтение этой информации из файла архива в создаваемый объект. Данная функция и передаваемый ей в качестве аргумента объект класса `CArchive` используются как для записи информации в файл, так и для чтения ее из файла, поэтому чтобы определить, какая операция будет выполняться в настоящий момент, используется функция `CArchive::IsStoring`, возвращающая `TRUE`, если объект сохраняется, и — `FALSE`, если производится его инициализация. Поскольку данный класс документа не содержит данных, то в нем отсутствуют операции по их сохранению в архиве и извлечению из него, располагаемые в различных ветвях условного оператора.

Функция `CObject::AssertValid` служит для целей отладки и проверяет данный объект на допустимость значений хранимых в нем величин.

Функция `CObject::Dump` также служит для целей отладки и передает содержимое вашего объекта в объект `CDumpContext`.

См. также главу 14

Поскольку функции `AssertValid` и `Dump` используются только в процессе отладки, их объявление в файле заголовка и их текст в файле реализации помещаются между операторами условной трансляции `#ifdef _DEBUG` и `#endif`. Завершает данную программу строка комментариев, указывающая на то, что за ней должны располагаться методы, обрабатывающие данные, хранящиеся в данном классе.

Класс представления

Рассмотрим класс представления в данном многооконном приложении. В листинге 2.3 представлен файл MDIApplicationView.h, содержащий заголовок данного класса.

Листинг 2.3. Файл заголовка класса CMDIApplicationView

```
// MDIApplicationView.h : interface of the CMDIApplicationView class
//////////////////////////////////////////////////////////////////////////

#if !defined(AFX_MDIAPPLICATIONVIEW_H__F06A8B31_855E_11D2_82AE_
                                                    9FE355108C6E__INCLUDED_)
#define AFX_MDIAPPLICATIONVIEW_H__F06A8B31_855E_11D2_82AE_
                                                    9FE355108C6E__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000


class CMDIApplicationView : public CView
{
protected:    // create from serialization only
  CMDIApplicationView();
  DECLARE_DYNCREATE(CMDIApplicationView)

// Attributes
public:
  CMDIApplicationDoc* GetDocument();

// Operations
public:

// Overrides
  // ClassWizard generated virtual function overrides
  // {{AFX_VIRTUAL(CMDIApplicationView)
  public:
  virtual void OnDraw(CDC* pDC);       // overridden to draw this view
  virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
  protected:
  virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
  virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
  virtual void OnEndPrinting(CDC* pDC, CPrintInfo* pInfo);
  //}}AFX_VIRTUAL

// Implementation
public:
  virtual ~CMDIApplicationView();
#ifdef _DEBUG
  virtual void AssertValid() const;
  virtual void Dump(CDumpContext& dc) const;
#endif
```

```
protected:

// Generated message map functions
protected:
  //{{AFX_MSG(CMDIApplicationView)
    // NOTE - the ClassWizard will add and remove member functions here.
    // DO NOT EDIT what you see in these blocks of generated code!
  //}}AFX_MSG
  DECLARE_MESSAGE_MAP()
};

#ifndef _DEBUG  // debug version in MDIApplicationView.cpp
inline CMDIApplicationDoc* CMDIApplicationView::GetDocument()
   { return (CMDIApplicationDoc*)m_pDocument; }
#endif

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately
before the previous line.

#endif // !defined(AFX_MDIAPPLICATIONVIEW_H__F06A8B31_855E_11D2_82AE_
                                                    9FE355108C6E__INCLUDED_)
```

Класс CMDIApplicationView является непосредственным потомком класса CView, динамически создаваемым в процессе исполнения программы. Так же, как и класс CMDIApplicationDoc, он имеет единственный защищенный конструктор, из чего следует, что непосредственное создание объекта данного класса программистом не предусматривается. Заголовок данного класса, как и заголовок рассмотренного выше класса документа, имеет пустую карту сообщений. Однако в реальном приложении эта карта практически никогда не остается пустой.

См. также главу 5

Чтобы подробнее ознакомиться с методами данного класса, рассмотрим файл его реализации MDIApplicationView.cpp, приведенный в листинге 2.4.

Листинг 2.4. Файл реализации класса CMDIApplicationView

```
// MDIApplicationView.cpp : implementation
//                          of the CMDIApplicationView class

#include "stdafx.h"
#include "MDIApplication.h"

#include "MDIApplicationDoc.h"
#include "MDIApplicationView.h"
```

```
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationView

IMPLEMENT_DYNCREATE(CMDIApplicationView, CView)

BEGIN_MESSAGE_MAP(CMDIApplicationView, CView)
  //{{AFX_MSG_MAP(CMDIApplicationView)
    // NOTE - the ClassWizard will add and remove mapping macros here.
    // DO NOT EDIT what you see in these blocks of generated code!
  //}}AFX_MSG_MAP
  // Standard printing commands
  ON_COMMAND(ID_FILE_PRINT, CView::OnFilePrint)
  ON_COMMAND(ID_FILE_PRINT_DIRECT, CView::OnFilePrint)
  ON_COMMAND(ID_FILE_PRINT_PREVIEW, CView::OnFilePrintPreview)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationView construction/destruction

CMDIApplicationView::CMDIApplicationView()
{
  // TODO: add construction code here

}

CMDIApplicationView::~CMDIApplicationView()
{
}

BOOL CMDIApplicationView::PreCreateWindow(CREATESTRUCT& cs)
{
  // TODO: Modify the Window class or styles here by modifying
  // the CREATESTRUCT cs

  return CView::PreCreateWindow(cs);
}

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationView drawing

void CMDIApplicationView::OnDraw(CDC* pDC)
{
  CMDIApplicationDoc* pDoc = GetDocument();
```

```
  ASSERT_VALID(pDoc);
  // TODO: add draw code for native data here
}

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationView printing

BOOL CMDIApplicationView::OnPreparePrinting(CPrintInfo* pInfo)
{
  // default preparation
  return DoPreparePrinting(pInfo);
}

void CMDIApplicationView::OnBeginPrinting(CDC* /*pDC*/,
                                          CPrintInfo* /*pInfo*/)
{
  // TODO: add extra initialization before printing
}

void CMDIApplicationView::OnEndPrinting(CDC* /*pDC*/,
                                        CPrintInfo* /*pInfo*/)
{
  // TODO: add cleanup after printing
}

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationView diagnostics

#ifdef _DEBUG
void CMDIApplicationView::AssertValid() const
{
  CView::AssertValid();
}

void CMDIApplicationView::Dump(CDumpContext& dc) const
{
  CView::Dump(dc);
}

CMDIApplicationDoc* CMDIApplicationView::GetDocument() // non-debug
                                                       // version is inline
{
  ASSERT(m_pDocument->IsKindOf(RUNTIME_CLASS(CMDIApplicationDoc)));
  return (CMDIApplicationDoc*)m_pDocument;
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationView message handlers
```

Как видно из листинга 2.4, созданный класс имеет пустые конструктор и деструктор. Это связано с тем, что стандартные операции создания и уничтожения элементов данного класса выполняются в конструкторе и деструкторе базового класса, а никаких данных, характерных только для данного окна представления, заготовка пользовательского класса не содержит.

Карта сообщений файла реализации класса `CMDIApplicationView` содержит макросы обработки стандартных сообщений печати `ID_FILE_PRINT`, `ID_FILE_PRINT_DIRECT` и `ID_FILE_PRINT_PREVIEW`. Для их обработки используются методы базового класса `CView::OnFilePrint` и `CView::OnFilePrintPreview`. Эти функции являются защищенными членами класса, поэтому информация о них отсутствует в справочной системе.

Функция `CWnd::PreCreateWindow` вызывается приложением перед созданием окна, связанного с данным объектом класса `CWnd`. В данном случае единственным оператором перегруженной функции `PreCreateWindow` является вызов *метода базового класса*.

Функция `CView::OnDraw` вызывается приложением для отображения документа на экране, при печати документа и при его предварительном просмотре перед печатью. Для доступа к *интерфейсу графических устройств* (GDI) функция `OnDraw` использует указатель на объект класса `CDC`. Работа с данной функцией в каждом из упомянутых выше режимов описана в соответствующей главе данной книги.

См. также главы 6 и 10

Текст функции `OnDraw`, содержащийся в листинге 2.4, состоит из двух операторов. В первом операторе создается указатель на объект класса `CMDIApplicationDoc`, для чего используется функция `CView::GetDocument`. Эта функция занимает центральное место в концепции Документ/Представление, поскольку именно она связывает эти два понятия в единое целое. Эта функция возвращает указатель на объект класса `CDocument`, ассоциированный с данным представлением. Если с данным представлением не ассоциирован никакой документ, данная функция возвращает ноль. Полученный указатель позволяет объекту представления получить доступ к данным и методам объекта документа.

Второй оператор функции, `OnDraw`, является макросом `ASSERT_VALID`, осуществляющим проверку допустимости значений параметров в полученном объекте документа. При компиляции окончательной версии приложения, использующего библиотеку MFC, данный макрос не производит никаких операций. Однако в отладочной версии этой библиотеки данный макрос проверяет свой аргумент на ненулевое значение и вызывает функцию `AssertValid` передаваемого объекта. Если хотя бы одна из этих проверок дает отрицательный результат, то на экране появляется сообщение, аналогичное сообщению, выдаваемому макросом `ASSERT` и функцией `AssertValid` класса документа.

См. также главу 14

Функции `CView::OnPreparePrinting`, `CView::OnBeginPrinting` и `CView::OnEndPrinting` используются при печати документа и не представляют особого интереса на данном этапе рассмотрения. Работе с этими функциями посвящена отдельная глава данной книги.

См. также главу 10

Класс `CMDIApplicationView`, так же, как и класс `CMDIApplicationDoc`, имеет отладочные функции `AssertValid` и `Dump`, подробное описание которых содержится в Приложении В. Кроме них отладочная секция программы содержит реализацию функции `GetDocument`. Эта функция не является отладочной, просто для нее предусмотрены две версии: отладочная, содержащая необходимые проверки, и окончательная, которая с целью повышения быстродействия программы сделана *встраиваемой* (inline).

Первый оператор отладочной версии функции `GetDocument` проверяет, является ли объект, на который указывает хранящийся в данном классе указатель, объектом класса документа, ассоциированного с данным классом представления. В случае положительного результата проверки второй оператор возвращает этот указатель на объект документа.

Завершает данную программу строка комментариев, указывающая, что за ней должны располагаться функции обработки пользовательских сообщений, поступающих в объект данного класса. Поскольку данный класс является заготовкой, он не содержит функций обработки пользовательских сообщений.

Класс приложения

При описании класса представления несколько раз упоминалось об ассоциации класса представления и класса документа. Где и каким образом происходит эта ассоциация? Чтобы ответить на этот и некоторые другие вопросы, необходимо рассмотреть класс приложения.

Как видно из листингов 2.1 и 2.3, файлы заголовков не несут информации, необходимой для понимания сути проблемы. В большинстве случаев они заполняются автоматически при помощи мастера ClassWizard. Если в демонстрационном приложении возникает необходимость ввести переменную в обход мастера ClassWizard, в тексте его описания содержится вся необходимая для этого информация. Кроме того, полные тексты приложений, включающие тексты файлов заголовков, содержатся на прилагаемой к данной книге дискете. Поэтому в дальнейшем при рассмотрении приложений будут приводиться только тексты файлов реализации.

Файл MDIApplication.cpp, являющийся файлом реализации класса приложения `CMDIApplicationApp`, приведен в листинге 2.5.

Листинг 2.5. Файл реализации класса CMDIApplicationApp

```
// MDIApplication.cpp : Defines the class behaviors for the application.

#include "stdafx.h"
#include "MDIApplication.h"

#include "MainFrm.h"
#include "ChildFrm.h"
#include "MDIApplicationDoc.h"
#include "MDIApplicationView.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationApp

BEGIN_MESSAGE_MAP(CMDIApplicationApp, CWinApp)
  //{{AFX_MSG_MAP(CMDIApplicationApp)
  ON_COMMAND(ID_APP_ABOUT, OnAppAbout)
    // NOTE - the ClassWizard will add and remove mapping macros here.
    // DO NOT EDIT what you see in these blocks of generated code!
  //}}AFX_MSG_MAP
  // Standard file based document commands
  ON_COMMAND(ID_FILE_NEW, CWinApp::OnFileNew)
  ON_COMMAND(ID_FILE_OPEN, CWinApp::OnFileOpen)
  // Standard print setup command
  ON_COMMAND(ID_FILE_PRINT_SETUP, CWinApp::OnFilePrintSetup)
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationApp construction

CMDIApplicationApp::CMDIApplicationApp()
{
  // TODO: add construction code here,
  // Place all significant initialization in InitInstance
}

/////////////////////////////////////////////////////////////////////////////
// The one and only CMDIApplicationApp object

CMDIApplicationApp theApp;
```

```
////////////////////////////////////////////////////////////////////////
// CMDIApplicationApp initialization

BOOL CMDIApplicationApp::InitInstance()
{
  AfxEnableControlContainer();

  // Standard initialization
  // If you are not using these features and wish to reduce the size
  //  of your final executable, you should remove from the following
  //  the specific initialization routines you do not need.

#ifdef _AFXDLL
  Enable3dControls();          // Call this when using MFC in a shared DLL
#else
  Enable3dControlsStatic();  // Call this when linking to MFC statically
#endif

  // Change the registry key under which our settings are stored.
  // TODO: You should modify this string to be something appropriate
  // such as the name of your company or organization.
  SetRegistryKey(_T("Local AppWizard-Generated Applications"));

  LoadStdProfileSettings();  // Load standard INI file options
                             // (including MRU)

  // Register the application's document templates.  Document templates
  //  serve as the connection between documents, frame windows and views.

  CMultiDocTemplate* pDocTemplate;
  pDocTemplate = new CMultiDocTemplate(
    IDR_MDIAPPTYPE,
    RUNTIME_CLASS(CMDIApplicationDoc),
    RUNTIME_CLASS(CChildFrame),    // custom MDI child frame
    RUNTIME_CLASS(CMDIApplicationView));
  AddDocTemplate(pDocTemplate);

  // create main MDI Frame window
  CMainFrame* pMainFrame = new CMainFrame;
  if (!pMainFrame->LoadFrame(IDR_MAINFRAME))
    return FALSE;
  m_pMainWnd = pMainFrame;

  // Parse command line for standard shell commands, DDE, file open
  CCommandLineInfo cmdInfo;
  ParseCommandLine(cmdInfo);

  // Dispatch commands specified on the command line
  if (!ProcessShellCommand(cmdInfo))
    return FALSE;
```

```
  // The main window has been initialized, so show and update it.
  pMainFrame->ShowWindow(m_nCmdShow);
  pMainFrame->UpdateWindow();

  return TRUE;
}

/////////////////////////////////////////////////////////////////////////////
// CAboutDlg dialog used for App About

class CAboutDlg : public CDialog
{
public:
  CAboutDlg();

// Dialog Data
  //{{AFX_DATA(CAboutDlg)
  enum { IDD = IDD_ABOUTBOX };
  //}}AFX_DATA

  // ClassWizard generated virtual function overrides
  //{{AFX_VIRTUAL(CAboutDlg)
  protected:
  virtual void DoDataExchange(CDataExchange* pDX);    // DDX/DDV support
  //}}AFX_VIRTUAL

// Implementation
protected:
  //{{AFX_MSG(CAboutDlg)
    // No message handlers
  //}}AFX_MSG
  DECLARE_MESSAGE_MAP()
};

CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
  //{{AFX_DATA_INIT(CAboutDlg)
  //}}AFX_DATA_INIT
}

void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
  CDialog::DoDataExchange(pDX);
  //{{AFX_DATA_MAP(CAboutDlg)
  //}}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
  //{{AFX_MSG_MAP(CAboutDlg)
```

```
    // No message handlers
  //}}AFX_MSG_MAP
END_MESSAGE_MAP()

// App command to run the dialog
void CMDIApplicationApp::OnAppAbout()
{
  CAboutDlg aboutDlg;
  aboutDlg.DoModal();
}

/////////////////////////////////////////////////////////////////////////////
// CMDIApplicationApp message handlers
```

Класс `CMDIApplicationApp` — потомок класса `CWinApp`, который, в свою очередь, является потомком класса `CWinThread`. Поэтому можно сказать, что данный класс является классом главного потока программы. Наличие потоков — главная отличительная особенность 32-разрядных систем Windows от их 16-разрядных предшественников. 16-разрядные системы Windows являлись "согласующей" многозадачной средой, без квантования времени. Это означает, что по истечении определенного кванта времени управление у задачи не отнимается. Для обеспечения возможности параллельной работы нескольких приложений в 16-разрядных системах Windows приходилось генерировать особое прерывание от таймера, которое прерывало выполнение одного приложения и передавало управление другому. Работа с таймером полностью возлагалась на пользователя.

В "вытесняющих" средах, к которым относятся и 32-разрядные системы Windows, операционная система выделяет каждому потоку квант времени по истечении которого его выполнение прерывается и операционная система решает, какому потоку следует передать управление. Поток представляет собой некоторый неделимый процесс, выполняемый в оперативной памяти компьютера и имеющий определенный уровень приоритета, определяющий порядок предоставления ему квантов времени. Поток может взаимодействовать с другими потоками и порождать их.

См. также главу 12

Каждое приложение, использующее MFC, может содержать только один объект класса, производного от `CWinApp`. Этот объект создается вместе с глобальными объектами и уже существует к моменту вызова функции `WinMain`, содержащейся в библиотеке MFC.

Функция `InitInstance`, принадлежащая данному классу, служит для создания объекта класса главного окна приложения.

Дизайн элементов управления и системный реестр

Первым оператором функции `InitInstance` является вызов функции `AfxEnableControlContainer`, обеспечивающей данному приложению поддержку элементов управления OLE. Далее, в зависимости от того, собираетесь ли вы подключать библиотеку MFC как библиотеку динамической компоновки или как статическую библиотеку, вызывается функция `Enable3dControls` или функция `Enable3dControlsStatic`.

Вызов функции `SetRegistryKey` позволяет сохранять параметры начальной установки вашего приложения в системном реестре, а не в файлах с расширением .ini, как это имело место в старых версиях Windows. Кроме того, вызов данной функции приводит к сохранению в системном реестре списка недавно использованных файлов, о котором уже говорилось в главе 1 в связи с установкой его размера. Окно приложения с раскрытым меню **File**, в котором содержится список недавно использованных файлов, приведен на рис. 2.2.

См. также главу 1

Рис. 2.2. Список недавно использованных файлов

Функция LoadStdProfileSettings служит для чтения из системного реестра параметров начальной установки данного приложения.

См. также главу 7

В следующем разделе рассматриваются операции описания шаблона документа.

Шаблон документа

Ассоциация класса представления и класса документа осуществляется в шаблоне документа, объединяющем следующие объекты:

- Идентификатор ресурса меню — в данном случае IDR_MDIAPPTYPE
- Класс документа — в данном случае CMDIApplicationDoc
- Класс окна в случае многооконного приложения — это CChildFrame, а в случае однооконного приложения — CMainFraim
- Класс представления — в данном случае CChildFrame

Описание шаблона документа начинается с получения указателя на объект класса CMultiDocTemplate (в случае однооконного приложения — CSingleDocTemplate). По адресу, содержащемуся в полученном указателе, записывается создаваемый объект шаблона документа. Обратите внимание, что в качестве параметра в макросе RUNTIME_CLASS выступает не объект класса или указатель на него, а чистое имя класса. Это позволяет создавать объекты данных классов в качестве внутренних переменных класса шаблона документа. Именно объединение классов документа и представления в одном объекте класса шаблона позволяет установить между ними однозначное соответствие.

Кроме имен классов документа и представления шаблон документа содержит идентификатор ресурса меню и класс окна. Включение информации о меню в класс шаблона документа представляется абсолютно логичным, поскольку каждый документ имеет характерную только для него структуру данных и методов их обработки. Поэтому ясно, что меню, оптимизированное для целей обработки текстовой информации, будет достаточно сложно использовать для редактирования изображений. Создание же универсального меню приведет к неизбежной путанице и существенно усложнит работу с приложением. Поэтому при работе с каждым типом документов в главное окно загружается соответствующее меню, оптимизированное для работы именно с этим типом документов.

Необходимость внесения в шаблон документа информации об окне данного приложения не кажется столь очевидной, поскольку каждому из классов шаблонов (CMultiDocTemplate или CSingleDocTemplate) четко соответствует тип окна (CChildFrame или CMainFraim). Но... пути Microsoft неисповедимы.

Чтобы макрос RUNTIME_CLASS нормально работал, нужно соблюдать некоторые соглашения относительно объявления включенных в него классов. В заголовке класса один из его конструкторов (обычно закрытый) объявляется с использованием макроса DECLARE_DYNACREATE, а в файле реализации данного класса должен присутствовать макрос IMPLEMENT_DYNACREATE для данного конструктора.

Завершает процедуру создания шаблона документа вызов функции CWinApp::AddDocTemplate, единственным параметром которой является указатель на только что созданный объект класса шаблона документа.

В данном приложении был создан всего один шаблон документа. При необходимости работы с несколькими типами документов программист должен создать соответствующие классы документа и представления, разработать соответствующее меню, имеющее свой идентификатор, и самостоятельно написать процедуру создания шаблона документа, аналогичную описанной выше. Естественно, каждый объект класса шаблона документа должен иметь уникальный идентификатор.

Создание окон

После создания шаблона документа получается указатель на объект класса главного окна приложения CMainFrame. Создание объекта класса, производного от CFrameWnd, выполняется в два этапа. Сначала вызывается конструктор, создающий объект класса CFrameWnd, а затем — функция CFrameWnd::LoadFrame, загружающая главное окно приложения и связанные с ним ресурсы и связывающая загруженное окно с объектом класса, производного от класса CMainFrame.

В ресурсе, идентификатор которого передается функции в качестве параметра, определены меню, таблица назначения клавиш, значки и строковый ресурс заголовка окна. Для задания всех параметров создания окна вместо функции LoadFrame следует использовать функцию CFrameWnd::Create. Приложение вызывает функцию LoadFrame при создании главного окна программы с использованием объекта класса шаблона документа.

Полученный указатель на объект главного окна приложения присваивается идентификатору m_pMainWnd, являющемуся членом класса приложения.

Следующим шагом по инициализации приложения является создание объекта класса CCommandLineInfo, используемого для выделения параметров инициализации приложения из командной строки. Созданный объект передается в качестве аргумента функции ParseCommandLine, вызываемой функцией ParseParam для заполнения объекта класса CCommandLineInfo. Этот объект затем передается функции ProcessShellCommand, использующей значения аргументов и флагов командной строки. Эта функция возвращает ненулевое значение, если процесс завершился нормально, и ноль — в противном случае.

На вызове этой функции следует остановиться особо. Разработчики из Microsoft, наверное, не предполагали, что с помощью Visual C++ будут разрабатываться какие-либо иные программы, кроме тестовых редакторов, поэтому предусмотрели при запуске многооконного приложения вывод на экран пустого окна. Целесообразность данного подхода сомнительна даже в текстовом редакторе, поскольку в большинстве случаев пользователь не начинает создание нового документа, а занимается редактированием старого. Однако в данном случае еще можно понять, зачем на экране появилось пустое окно. Но представьте себе, что пустое окно появляется в приложении, созданном для обработки и анализа различных типов сигналов, каждый из которых представлен своим типом документа. Окно какого типа документа должно выводиться при запуске приложения? И что должен делать пользователь с пустым окном? Не рисовать же в нем вручную спектр сигнала. Поэтому хорошим тоном в серьезном приложении считается не выводить никаких окон при запуске его на исполнение.

Но мастер AppWizard создает приложение, выводящее пустое окно на экран. Что же делать? Наиболее простым решением данного вопроса является уничтожение условного оператора, содержащего вызов функции `ProcessShellCommand` (строки, содержащие этот оператор, можно закомментировать, если вам не хочется удалять операторы, поставленные приложением AppWizard). При этом не следует забывать, что условный оператор включает в себя не только сам оператор, но и оператор `return`, выполнение которого зависит от величины, возвращаемой функцией `ProcessShellCommand`.

Функция `CWnd::ShowWindow` устанавливает режим отображения окна, например, стандартный размер отображаемого окна, развернутое на весь экран или отображаемое как пиктограмма. При вызове этой функции в процедуре `InitInstance` ее аргументом является переменная `CWinApp::m_nCmdShow`. При последующих ее вызовах в качестве аргумента необходимо непосредственно указывать режимы отображения окна, для обозначения которых существуют предопределенные величины.

Последним исполняемым оператором процедуры `InitInstance` является вызов функции `CWnd::UpdateWindow`, выполняющей перерисовку рабочей области окна путем посылки сообщения `WM_PAINT` в том случае, если эта область не пуста. Данная функция осуществляет непосредственную посылку сообщения `WM_PAINT`, не помещая его в очередь приложений.

Если программа дошла до этой точки, то инициализация приложения считается успешно завершенной, поэтому функция возвращает значение `TRUE`.

Кроме описания класса приложения данный файл содержит также описание класса диалогового окна справки о программе. Поскольку диалоговые окна будут подробно рассмотрены в главе 13, здесь мы оставим данный фрагмент файла без комментариев.

Диалоговые окна и простейшие элементы управления

Рассмотрев заготовки приложений, создаваемых мастером AppWizard, и концепции Документ/Представление, сделаем отступление и перейдем к диалоговым окнам.

Дело в том, что без диалоговых окон практически невозможно написать какую-либо программу в среде Windows. Каждая программа создается для обработки некоторой информации, поэтому ей необходимо в какой-либо форме передать сведения о том, где взять эту информацию, что с ней делать и куда ее поместить. При отладке программ эту информацию можно, конечно, помещать непосредственно в текст программы и изменять ее путем редактирования файла. Но данный подход нельзя признать изящным, и он абсолютно не применим к программам, с которыми должен работать кто-нибудь еще, кроме их создателя. Другим вариантом передачи информации в программу является меню, но работа с ним тоже нуждается в описании и, кроме того, при работе с меню так же, как и при работе с диалоговыми окнами, невозможно обойти вопрос о сообщениях и их обработке.

Диалоговое окно

Полноценная программа в среде Windows будет иметь множество диалоговых окон, каждое из которых предназначено для обмена или вывода информации. Описание каждого диалогового окна состоит из двух частей, расположенных в различных файлах: описания ресурсов диалогового окна, для хранения которых используются специальные файлы ресурсов, имеющие, как правило, расширение .rc, и описания класса диалогового окна, расположенного в файле реализации. Каждому из этих файлов соответствует свой файл заголовка.

Формирование ресурсов диалогового окна

Каждое приложение в среде Visual C++ содержит единственный файл ресурсов, имеющий расширение .rc, имя которого совпадает с именем приложения. В этом файле содержится описание всех ресурсов данного приложения, к которым, помимо диалоговых окон, относятся также таблица назначения клавиш, значки, меню, таблица строковых ресурсов, панели инструментов и сведения о версии приложения. Данный файл использует файл заголовка Resource.h, содержащий таблицу соответствия идентификаторов ресурсов числовым величинам.

Использование программой единственного файла ресурсов, имя которого совпадает с именем приложения, не означает, что пользователь не может создать собственный файл ресурсов. Просто этот файл необходимо включить в главный файл ресурсов приложения с использованием директивы `#include`, как это сделано для некоторых файлов стандартных ресурсов, таких как afxres.rc и afxprint.rc. Однако ресурсы, описанные в данных файлах, не могут редактироваться в стандартных окнах редактирования ресурсов, а сами эти ресурсы не отображаются в панели **ResourceView** окна **Workspace**. Это обстоятельство заставляет пользователя три раза подумать прежде, чем он создаст свой файл ресурсов. Использование этих файлов оправдано только в том случае, когда пользователь имеет большую библиотеку ресурсов, которые он хочет использовать в данном приложении. Но и этот вопрос может быть решен намного проще при помощи галереи компонентов, описанной в данной главе.

В качестве заготовки для исследования диалоговых окон выберем заготовку диалогового приложения, созданную в главе 1. Внешний вид интегрированной оболочки пользователя по завершении создания диалогового приложения при помощи мастера AppWizard приведен на рис. 3.1.

Файл с текстом демонстрационного приложения DialogApplication, содержащего основные элементы управления диалогового окна, находится в директории DialogApplication на прилагаемой к данной книге дискете. Чтобы самостоятельно создать данное приложение:

1. Щелкните левой кнопкой мыши по статическому тексту **TODO: Place dialog controls here**, расположенному в центре заготовки диалогового окна, и нажмите кнопку **Del**. Статический текст исчезнет.
2. В окне **Workspace** (Проект), обычно расположенном слева от окна редактора, раскройте вкладку **ResourceView** (Просмотр ресурсов). Поскольку эта вкладка имеет достаточно длинное имя, а область экрана, выделенная под окно **Workspace**, ограничена, то название этой вкладки может быть сокращено, например, до **Res...**.
3. Раскройте папку \DialogApplication resources, а в ней — папку \Dialog. Щелкните правой кнопкой мыши на идентификаторе ресурса `IDD_`

DIALOGAPPLICATION_DIALOG и в появившемся контекстном меню выберите команду **Properties** (Свойства). Появится диалоговое окно **Dialog Properties**, изображенное на рис. 3.2.

4. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна. (Считается, что на этой кнопке изображена канцелярская кнопка.) После этого данное диалоговое окно будет находиться на экране до тех пор, пока вы его сами не закроете. В противном случае оно будет закрываться при активизации следующего элемента управления.
5. В раскрывающемся списке **Language** (Язык) выберите строку **Russian** (Русский).

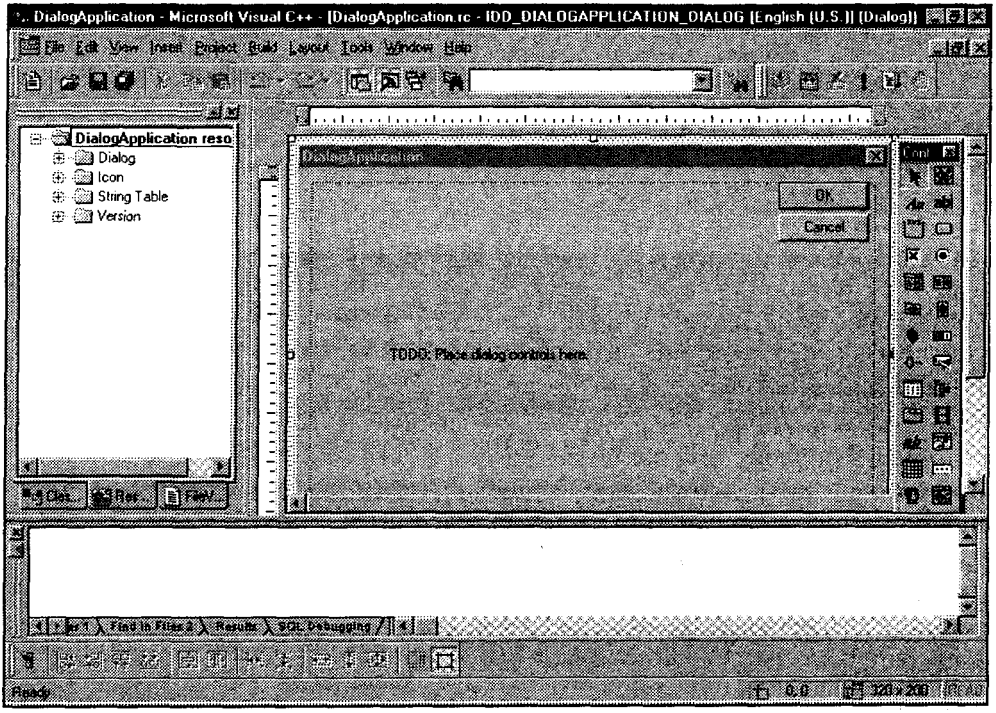


Рис. 3.1. Окно редактирования ресурса диалогового окна

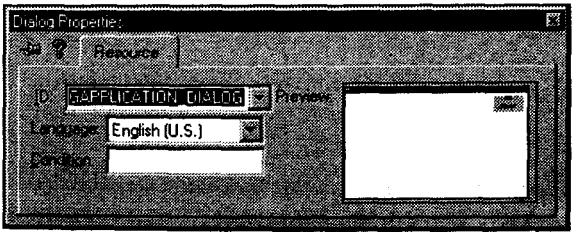


Рис. 3.2. Диалоговое окно **Dialog Properties**

6. Щелкните левой кнопкой мыши в области диалогового окна. Появится диалоговое окно **Dialog Properties**, изображенное на рис. 3.3.

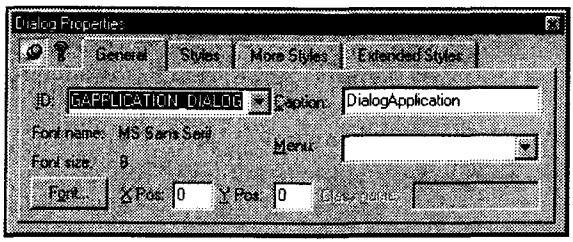


Рис. 3.3. Диалоговое окно **Dialog Properties**

7. В текстовое окно **Caption** (Заголовок) введите текст "Тестовое диалоговое приложение".
8. В панели инструментов **Controls** (Элементы управления) поместите курсор мыши на кнопку **Static Text** (Статический текст), нажмите левую кнопку мыши и, удерживая ее, перетащите рамку статического текста в левый верхний угол диалогового окна. Заготовка диалогового окна примет вид, изображенный на рис. 3.4.

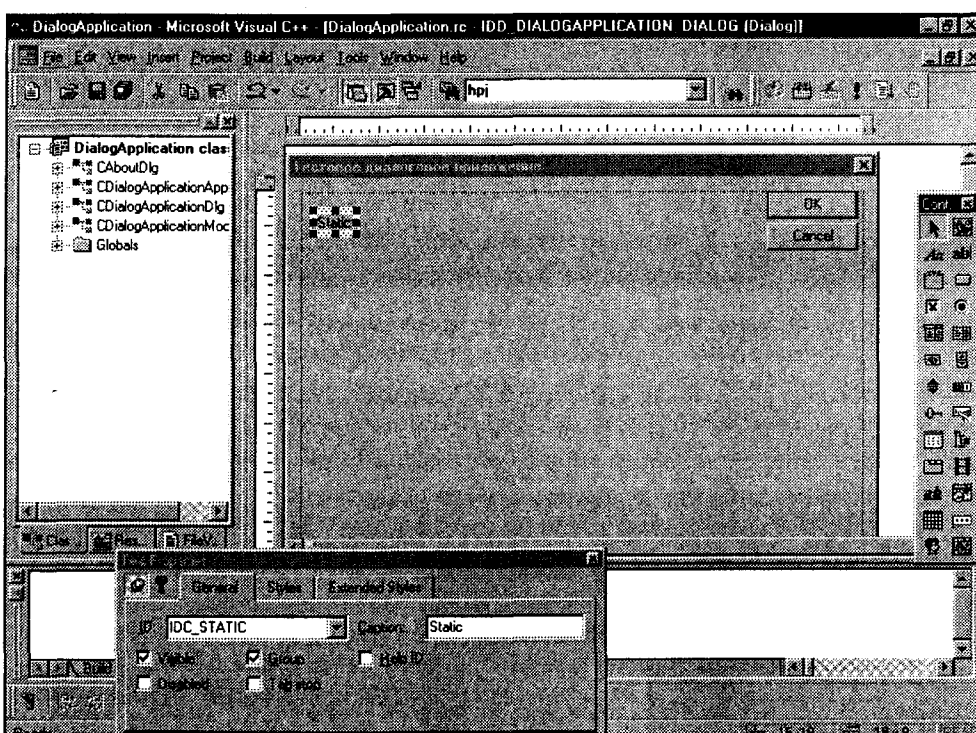


Рис. 3.4. Заготовка диалогового окна с рамкой статического текста

9. В текстовое поле **Caption** диалогового окна **Text Properties** введите текст "Текстовое поле".
10. Поместите курсор мыши на кнопку **Edit Box** (Текстовое поле) в панели инструментов **Controls**, перетащите рамку текстового поля ниже соответствующего ей статического текста и отрегулируйте ее размеры и положение.
11. Чтобы изменить положение элемента управления, нужно его выделить, щелкнув на нем левой кнопкой мыши. Вокруг выделенного элемента управления появится толстая рамка с восьмью синими квадратиками (по одному в каждом углу рамки и по одному в середине каждой стороны рамки).
12. Указатель мыши в пределах рамки приобретает вид крестика, образованного стрелками, направленными наружу. При нажатой левой кнопке мыши подобный указатель позволяет перетаскивать выделенный элемент управления без изменения его геометрических размеров.
13. Если указатель мыши помещается на один из восьми синих квадратов, он приобретает форму двунаправленной стрелки, указывающей на возможное направление перемещения границы рамки. При нажатии левой кнопки мыши подобный указатель позволяет перемещать границу рамки в указанном стрелками направлении. При установке указателя мыши на угловой квадрат перемещаются обе стороны рамки, образующие этот угол. При помещении указателя мыши на квадрат, расположенный в центре одной из границ рамки, перемещается только эта граница рамки.
14. После регулировки размеров и положения текстового поля заготовка диалогового окна примет вид, изображенный на рис. 3.5.
15. В текстовое поле раскрывающегося списка **ID:** (Идентификатор ресурса) вместо идентификатора `IDC_EDIT1` введите идентификатор `IDC_EDIT_BOX`.
16. Повторите эти операции и создайте второе текстовое поле с идентификатором `IDC_BUDDY` и статическим текстом "Инкрементный регулятор", причем последним из этих двух элементов управления должно формироваться текстовое поле.
17. Выберите в панели инструментов **Controls** элемент управления **Spin** (Инкрементный регулятор) и поместите его справа от сформированного в предыдущем пункте текстового поля. Заготовка диалогового окна примет вид, изображенный на рис. 3.6.
18. В диалоговом окне **Spin Properties** измените установленный по умолчанию идентификатор на `IDC_SPIN`.
19. Раскройте вкладку **Styles** (Стили). В раскрывающемся списке **Alignment:** (Выравнивание) выберите опцию **Right** (Справа) и установите флажки **Auto buddy** (Автоматическое подключение связанного элемента) и **Set**

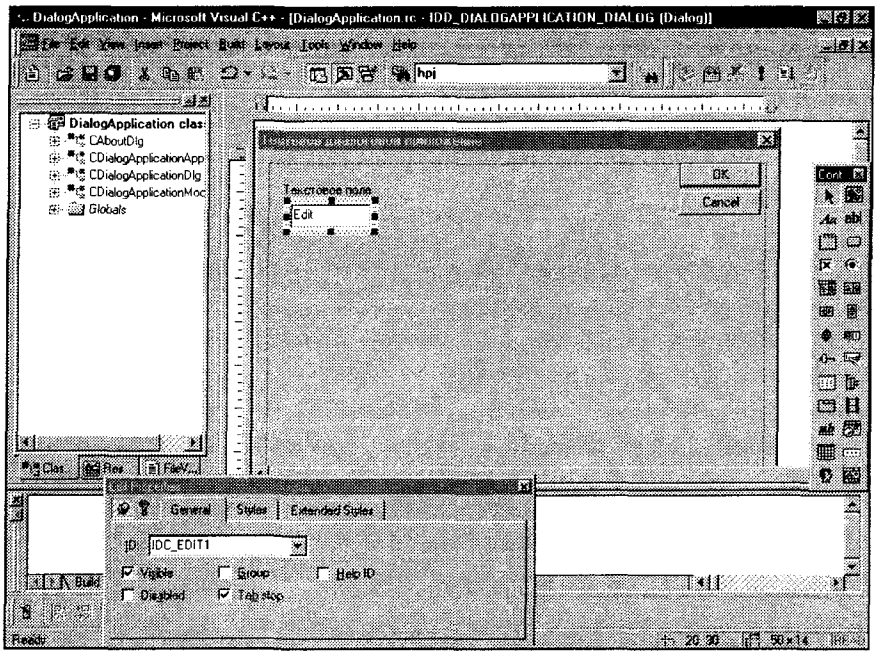


Рис. 3.5. Заготовка диалогового окна с текстовым полем

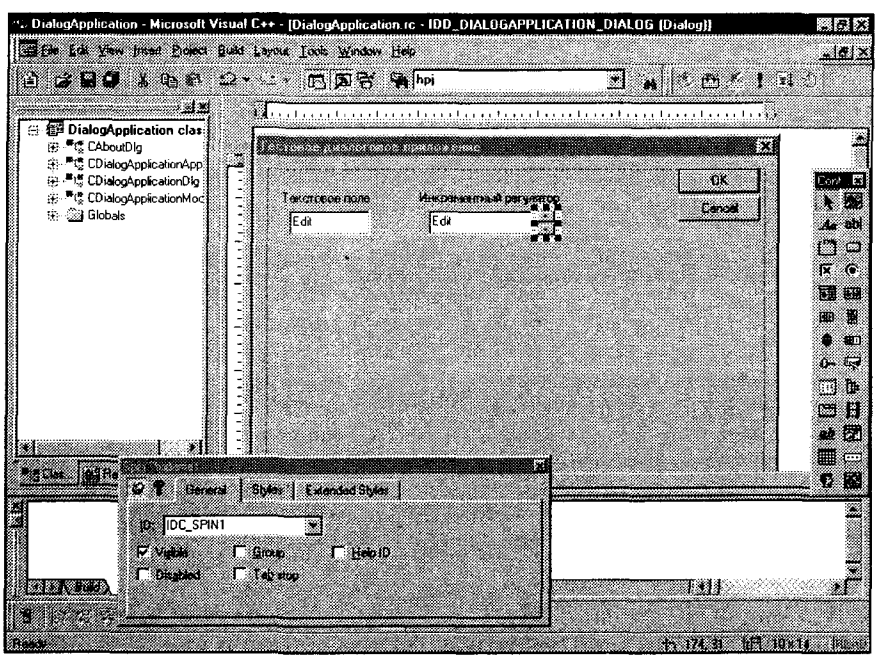


Рис. 3.6. Заготовка диалогового окна с инкрементным регулятором

buddy integer (Рассматривать содержимое связанного элемента как целое число без знака). Под отростком понимается элемент управления, содержимым которого будет управлять инкрементный регулятор. При установке флажка **Auto buddy** в качестве связанного выбирается последний созданный перед регулятором элемент управления. При установке в текстовом поле раскрывающегося списка **Alignment:** значения **Right** инкрементный индикатор становится составной частью поля связанного элемента и располагается в его правой части. Если установлен флажок **Wrap** (Цикл), а величина в связанном элементе достигла максимального значения, то при следующем ее увеличении она сбрасывается в ноль, а попытка уменьшения обнуленного значения приводит к записи в отросток максимально возможного значения. Установка флажка **Arrow keys** (Управление с клавиатуры) позволяет изменять значения в отростке с помощью соответствующих клавиш, если фокус ввода установлен на данном элементе управления. Установка флажка **No thousands** (Отсутствие разбиения по триадам) удаляет разделитель между триадами цифр. Диалоговое окно **Spin Properties** примет вид, изображенный на рис. 3.7.

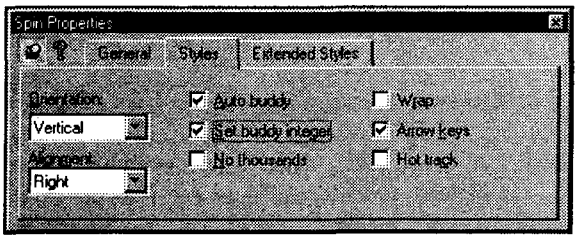


Рис. 3.7. Диалоговое окно **Spin Properties**

20. Щелкните левой кнопкой мыши на кнопке **Cancel** в заготовке диалогового окна и введите в текстовое поле **Caption** диалогового окна **Push Button Properties**, расположенного на вкладке **General**, заголовок "Отмена".

21. Расположите под кнопками **OK** и **Отмена** статический текст "Раскрывающийся список". Выберите в панели инструментов **Controls** элемент управления **Combo Box** (Раскрывающийся список) и поместите его в заготовку диалогового окна. Щелкните левой кнопкой мыши на кнопке раскрытия списка в раскрывающемся списке, поместите указатель мыши на синий квадратик на нижней границе рамки элемента управления и переместите его вниз, как это показано на рис. 3.8.

22. В диалоговом окне **Combo Box Properties** измените установленный по умолчанию идентификатор на `IDC_COMBO` и раскройте вкладку **Data** (Данные). Введите в окно списка величину "4" и нажмите комбинацию клавиш <Ctrl>+<Enter>. Затем введите в него величину "16" и опять нажмите комбинацию клавиш <Ctrl>+<Enter>. После чего введите в окно списка "256". Диалоговое окно **Combo Box Properties** примет вид, изображенный на рис. 3.9.

23. Раскройте вкладку **Styles** данного диалогового окна и снимите флажок **Sort** (Сортировка). Диалоговое окно **Combo Box Properties** примет вид, изображенный на рис. 3.10.

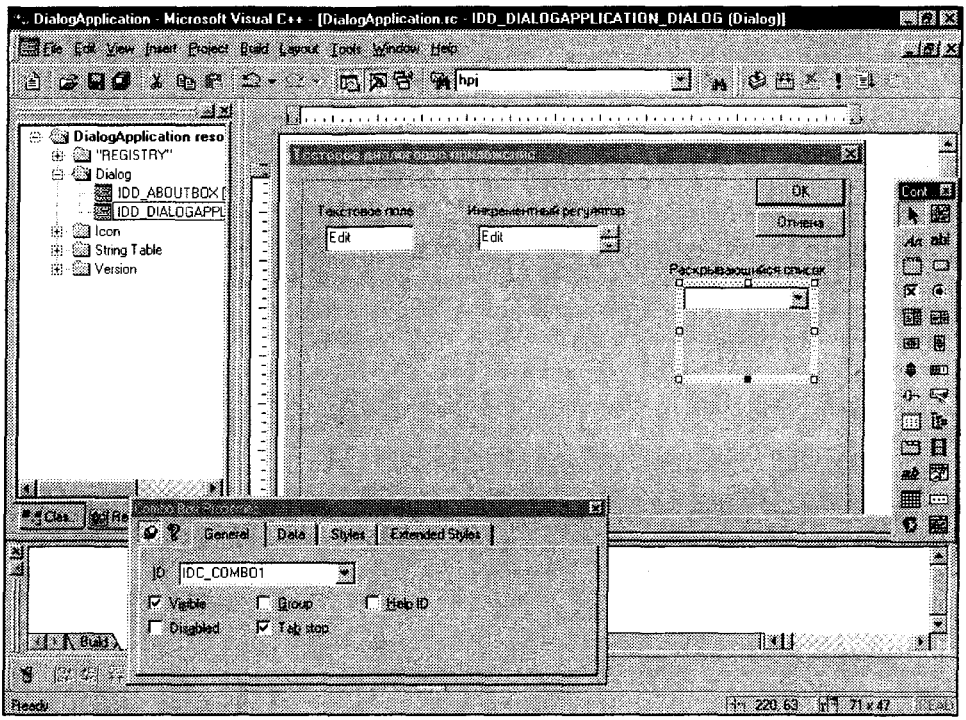


Рис. 3.8. Заготовка диалогового окна с раскрывающимся списком

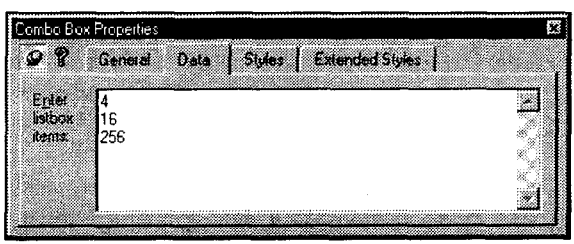


Рис. 3.9. Вкладка **Data** диалогового окна **Combo Box Properties**

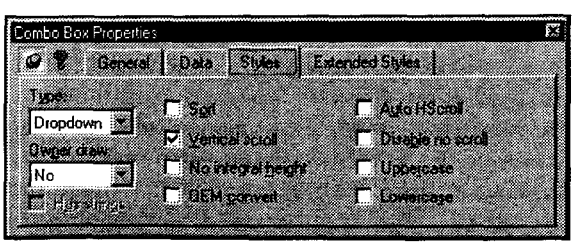


Рис. 3.10. Диалоговое окно **Combo Box Properties**, вкладка **Styles**

24. Раскройте вкладку **Extended Styles** данного диалогового окна и установите флажок **Right aligned text** (Выравнивание текста по правому краю). Диалоговое окно **Combo Box Properties** примет вид, изображенный на рис. 3.11.

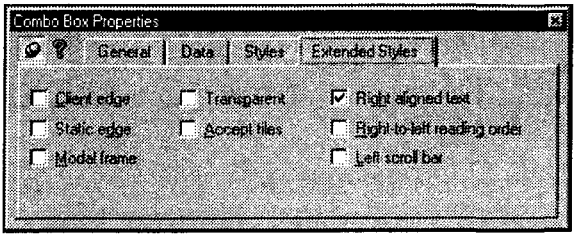


Рис. 3.11. Диалоговое окно **Combo Box Properties**, вкладка **Extended Styles**

25. Выберите в панели инструментов **Controls** элемент управления **Radio Button** (Переключатель) и поместите его в заготовку диалогового окна под текстовым полем и инкрементным регулятором. В диалоговом окне **Radio Button Properties** измените идентификатор ресурса на `IDC_EDIT_SWITCH`, измените содержимое текстового поля **Caption:** на "Текстовое поле" и установите флажок **Group** (Группа). Диалоговое окно **Radio Button Properties** примет вид, изображенный на рис. 3.12.

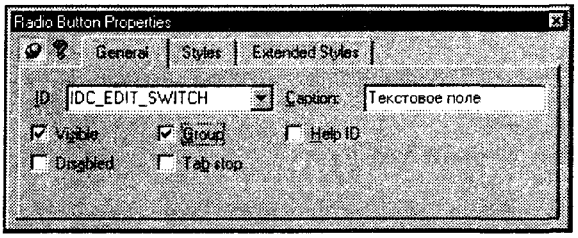


Рис. 3.12. Диалоговое окно **Radio Button Properties**

26. Повторите эту операцию для второго переключателя, расположив его под первым. Используйте для него идентификатор ресурса `IDC_SPIN_SWITCH` и заголовок "Инкрементный регулятор". В этом диалоговом окне не нужно устанавливать флажок **Group**.
27. Выберите в панели инструментов **Controls** элемент управления **Group Box** (Рамка группы) и обведите этой рамкой два переключателя. В текстовое поле **Caption:** диалогового окна **Group Box Properties** введите заголовок "Выбор поля для сохранения". Заготовка диалогового окна примет вид, изображенный на рис. 3.13.
28. Выберите в панели инструментов **Controls** элемент управления **Check Box** (Флажок) и поместите его в заготовку диалогового окна. В диалоговом окне **Check Box Properties** измените идентификатор ресурса на `IDC_SAVE` и измените содержимое текстового поля **Caption:** на "Сохранять?". Заготовка диалогового окна примет вид, изображенный на рис. 3.14.

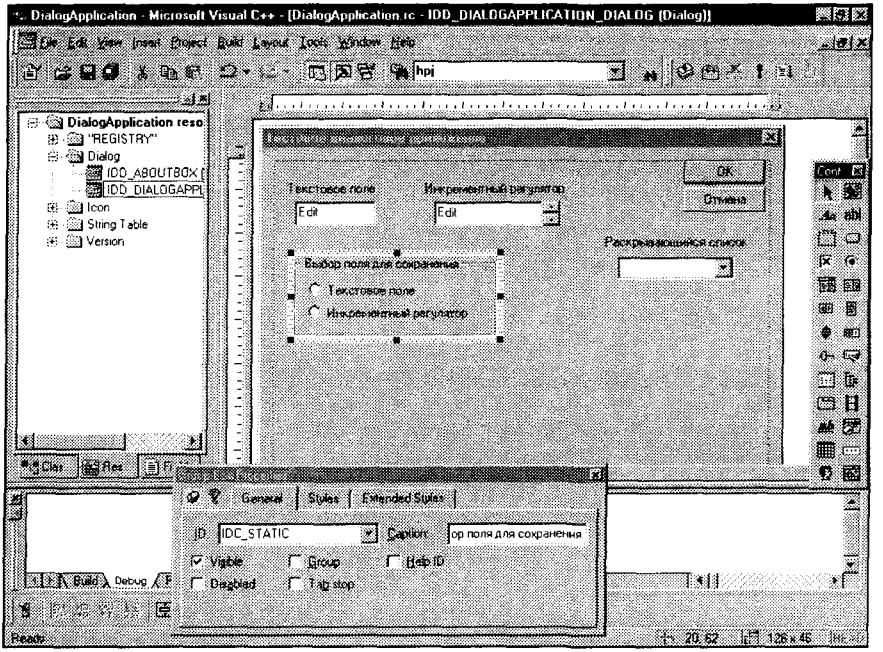


Рис. 3.13. Заготовка диалогового окна с группой переключателей

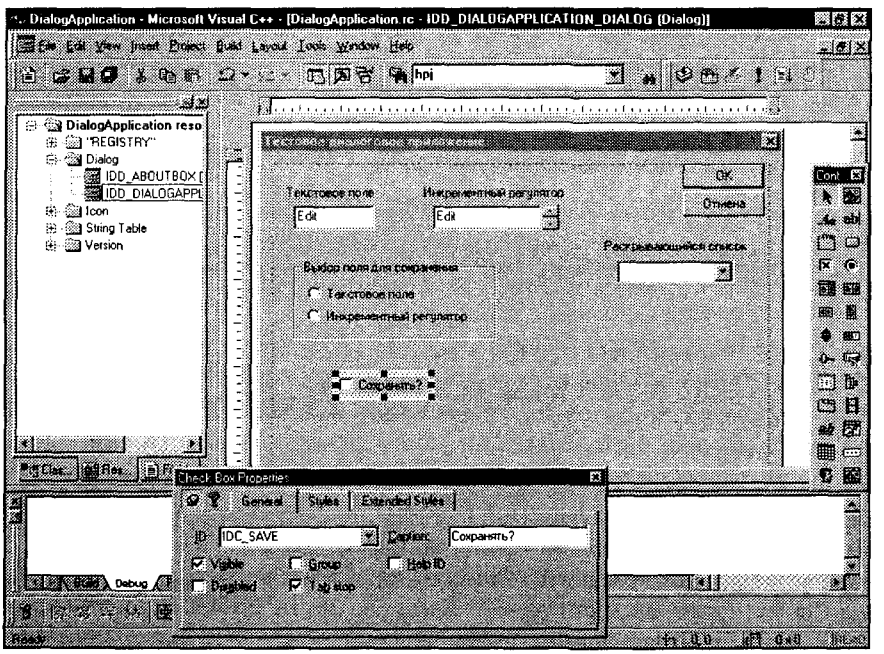


Рис. 3.14. Окончательный вид диалогового окна

На этом этап формирования ресурсов диалогового окна можно считать законченным. Дело в том, что одновременно с производимыми нами действиями программная оболочка Visual C++ фиксировала все вносимые изменения в файлах DialogApplication.rc и resource.h. В результате наших действий в файле DialogApplication.rc появилась, среди прочего, структура диалогового окна, приведенная в листинге 3.1.

Листинг 3.1. Описание ресурса диалогового окна в файле ресурса приложения

```
//////////////////////////////////////////////////////////////////////
// Dialog

IDD_DIALOGAPPLICATION_DIALOG DIALOGEX 0, 0, 320, 200
STYLE DS_MODALFRAME | WS_POPUP | WS_VISIBLE | WS_CAPTION | WS_SYSMENU
EXSTYLE WS_EX_APPWINDOW
CAPTION "Тестовое диалоговое приложение"
FONT 8, "MS Sans Serif"
BEGIN
    DEFPUSHBUTTON   "OK",IDOK,260,7,50,14
    PUSHBUTTON      "Отмена",IDCANCEL,260,23,50,14
    LTEXT           "Текстовое поле",IDC_STATIC,15,19,55,8
    EDITTEXT        IDC_EDIT1,20,31,50,14,ES_AUTOHSCROLL
    EDITTEXT        IDC_EDIT2,106,31,68,14,ES_AUTOHSCROLL
    LTEXT           "Инкрементный регулятор",IDC_STATIC,100,19,88,8
    CONTROL         "Spin1",IDC_SPIN,"msctls_updown32",UDS_SETBUDDYINT |
                    UDS_ALIGNRIGHT | UDS_AUTOBUDDY | UDS_ARROWKEYS,174,
                    31,10,14
    LTEXT           "Раскрывающийся список",IDC_STATIC,212,50,88,8
    COMBOBOX        IDC_COMBO,220,63,71,47,CBS_DROPDOWN | WS_VSCROLL |
                    WS_TABSTOP,WS_EX_RIGHT
    CONTROL         "Текстовое поле",IDC_EDIT_SWITCH,"Button",
                    BS_AUTORADIOBUTTON | WS_GROUP,28,77,70,10
    CONTROL         "Инкрементный регулятор",IDC_SPIN_SWITCH,"Button",
                    BS_AUTORADIOBUTTON,28,90,103,10
    GROUPBOX        "Выбор поля для сохранения",IDC_STATIC,20,62,126,46
    CONTROL         "Сохранять?",IDC_SAVE,"Button",BS_AUTOCHECKBOX |
                    WS_TABSTOP,48,130,55,10
END
```

В первой строке этой структуры указан идентификатор ресурса диалогового окна и его размеры в оконных координатах, поэтому верхний левый угол данного окна всегда имеет координаты 0, 0. После этого следует описание стиля диалогового окна, его заголовок и описание используемого в нем шрифта. Далее, между программными скобками BEGIN и END, располагается описание элементов управления данного окна. Описание элемента управле-

ния начинается с указания на то, что это за элемент управления, затем следует его заголовок, если он есть, идентификатор и другая специфическая информация. В конце описания элемента управления расположены координаты его рамки.

В случае необходимости пользователь может вносить изменения в описание элемента управления или любого другого ресурса непосредственно в файле описания ресурсов, однако намного удобнее и надежнее делать это с использованием интегрированной оболочки разработчика.

Если вас не устраивает дизайн диалогового окна на рис. 3.14, вы можете внести в него изменения. Чтобы перетащить любой элемент управления на новое место, достаточно выделить его щелчком левой кнопки мыши, что приведет к появлению рамки элемента управления и превращению курсора мыши в крестик со стрелками. После этого вы можете его перемещать с клавиатуры клавишами управления курсором или мышью, для чего достаточно нажать левую кнопку мыши, когда курсор имеет форму крестика со стрелками и, удерживая кнопку нажатой, переместить элемент управления в новое место. При этом перемещаться будет не сам элемент управления, а его тонкая штриховая рамка.

При необходимости изменить размер элемента управления достаточно выделить его, как описано выше, и поместить курсор на один из синих квадратов в рамке элемента управления. При этом курсор примет форму отрезка линии с двумя стрелками. Направление стрелок указывает, в каком направлении будет перемещаться соответствующая сторона рамки.

Для выравнивания размеров и положения элементов управления можно воспользоваться услугами панели инструментов **Dialog**, обычно расположенной под окнами редактирования ресурса, и **Workspace**. Внешний вид панели инструментов **Dialog** приведен на рис. 3.15.

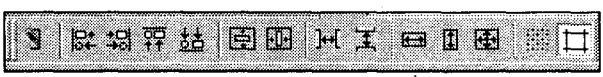
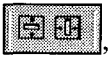


Рис. 3.15. Панель инструментов **Dialog**

Кнопка (Test, <Ctrl>+<T>) панели инструментов **Dialog**, расположенная на панели первой слева, позволяет, не запуская программы, проверить функционирование элементов управления диалога. Выход из этого режима возможен при помощи кнопок закрытия диалогового окна в запущенном диалоге. В том случае, когда эти кнопки в диалоге отсутствуют, например, при создании вкладок диалогового окна, выход из этого режима осуществляется нажатием кнопки **<Esc>**.

Следующая группа кнопок управляет выравниванием положения элементов управления. Она включает в себя четыре кнопки: **Align Left** (<Ctrl>+<Left Arrow>), **Align Right** (<Ctrl>+<Right Arrow>), **Align Top**

(<Ctrl>+<Up Arrow>) и **Align Bottom** (<Ctrl>+<Down Arrow>) (Выровнять по левому краю, по правому краю, по верхней границе и по нижней границе). Чтобы выровнять элементы управления с использованием этих кнопок, необходимо выделить все элементы управления, к которым должна быть применена эта операция. Для этого достаточно последовательно щелкнуть на них левой кнопкой мыши, удерживая нажатой клавишу <Shift> или <Ctrl>. После чего нажать соответствующую кнопку.

За этой группой расположена группа кнопок  , управляющая размещением элементов управления в пределах диалогового окна. Эта группа включает в себя две кнопки: **Center Vertical** (<Ctrl>+<F9>) и **Center Horizontal** (<Ctrl>+<Shift>+<F9>) (Центрирование по вертикали и горизонтали). Работа с этими кнопками в пояснениях не нуждается.

Следующие две кнопки управляют взаимным положением элементов управления. Это кнопки **Space Across** (<Alt>+<Right Arrow>) и **Space Down** (<Alt>+<Up Arrow>) (Выровнять по горизонтали и выровнять по вертикали). В результате нажатия этих кнопок крайние из выделенных элементов управления остаются на своих местах, а находящиеся между ними элементы управления перемещаются таким образом, чтобы выровнять расстояние между соседними элементами в выбранном направлении.

Предпоследняя группа кнопок используется для выравнивания геометрических размеров рамок выделенных элементов управления. В эту группу входят кнопки **Make Same Width**, **Make Same Height** и **Make Same Size** (Установить одинаковую ширину, установить одинаковую высоту и установить одинаковые размеры). Как понятно из названия этих кнопок, в результате нажатия на первую кнопку рамки всех элементов управления будут иметь одну и ту же ширину, в результате нажатия на вторую кнопку — одну и ту же высоту, а при нажатии на третью кнопку — как одинаковую ширину, так и одинаковую высоту.

Две последние кнопки представляют собой переключатель, при установке которого в положение **Toggle Grid** на поверхности заготовки диалогового окна появляется сетка, к которой привязываются размеры каждого элемента управления. При установке этого переключателя в положение **Toggle Guides** сетка исчезает, и каждый элемент управления может иметь произвольные размеры и занимать произвольное положение.

Создание класса диалога

Сформировав ресурсы диалогового окна, приступим к созданию класса диалога. Конечно, это сильно сказано, поскольку львиную долю работы по созданию этого класса уже выполнил мастер AppWizard, а еще некоторую часть работы готов взять на себя мастер ClassWizard. Чтобы связать идентифика-

торы созданных нами элементов управления с классом диалогового окна `CDialogApplicationDlg`, созданным мастером AppWizard:

1. Оставаясь в окне редактора ресурсов диалогового окна выберите команду **View, ClassWizard** (Вид, ClassWizard) или щелкните в окне редактора ресурсов правой кнопкой мыши и выберите в появившемся контекстном меню команду **ClassWizard**. Появится диалоговое окно **MFC ClassWizard**.
2. Раскройте вкладку **Member Variables** (Переменные, члены класса). Диалоговое окно **MFC ClassWizard** примет вид, приведенный на рис. 3.16.

   В окне списка **Control IDs:** данной вкладки перечислены все идентификаторы ресурсов, используемые в созданном нами диалоговом окне. Теперь необходимо сопоставить этим идентификаторам ресурсов идентификаторы переменных в классе, обрабатывающем сообщения данного диалогового окна.

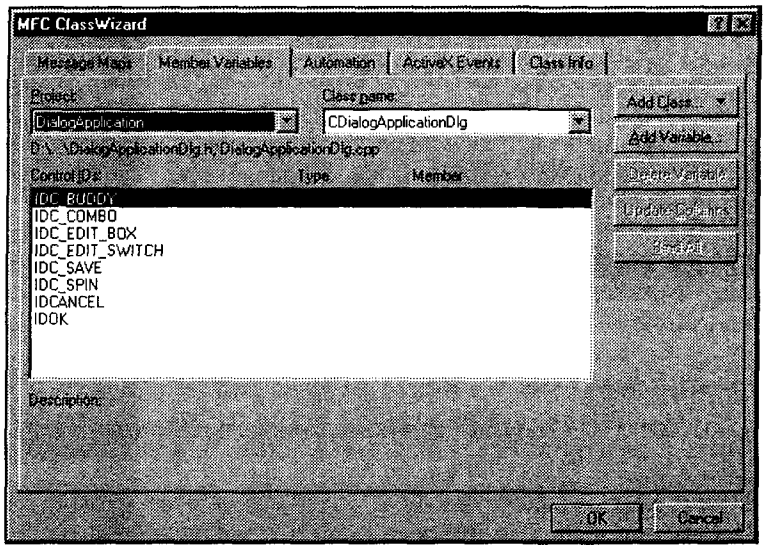


Рис. 3.16. Диалоговое окно **MFC ClassWizard**

3. Как видно из рис. 3.16, в нем уже выделен идентификатор ресурса `IDC_BUDDY`. Оставив это выделение, нажмите кнопку **Add Variable** (Добавить переменную). Появится диалоговое окно **Add Member Variable**, изображенное на рис. 3.17.

   В соответствии с венгерской нотацией имя переменной, являющейся членом класса, должно начинаться с `m_`, что отражено в текстовом поле **Member variable name:** (Идентификатор переменной, члена класса).
4. Введите в текстовое поле **Member variable name:** идентификатор `m_Buddy` и нажмите кнопку **OK**. Диалоговое окно **MFC ClassWizard** примет вид, приведенный на рис. 3.18.

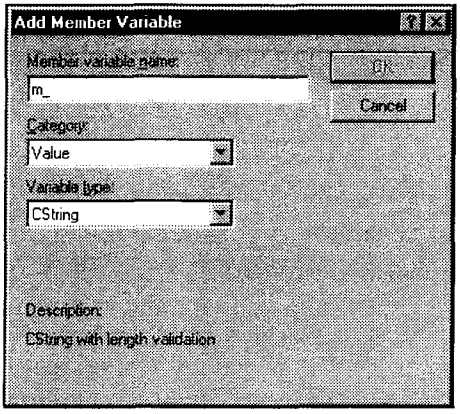


Рис. 3.17. Диалоговое окно **Add Member Variable**

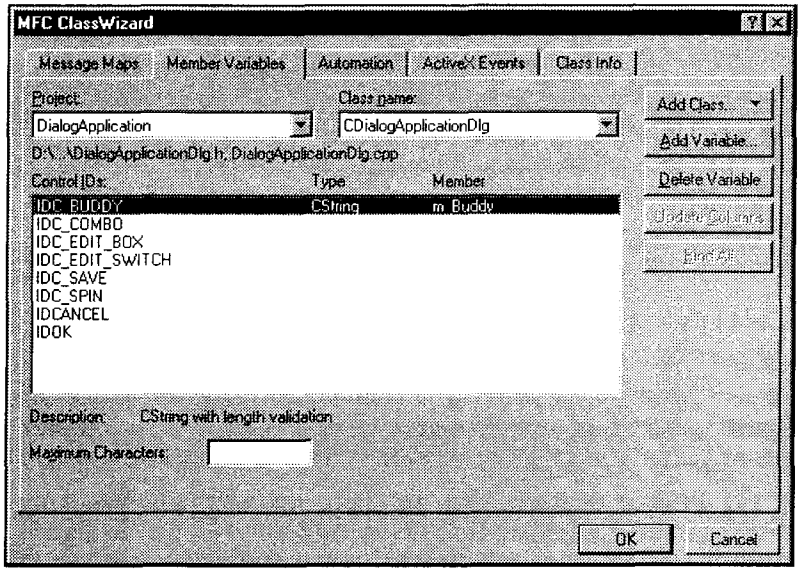


Рис. 3.18. Диалоговое окно **MFC ClassWizard** после добавления первой переменной

5. Введите в текстовое поле **Maximum Characters:** (Максимальное количество символов) цифру 2.
6. Выделите идентификатор ресурса `IDC_COMBO` и повторите для него пп. 3—5, сопоставив ему идентификатор переменной `m_Combo` и установив максимальное количество символов равным трем.
7. Сопоставьте идентификатору ресурса `IDC_EDIT_BOX` идентификатор члена класса `m_Edit` и ничего не вносите в текстовое поле **Maximum Characters:**.
8. Идентификатору ресурса `ID_EDIT_SWITCH` сопоставьте целочисленный идентификатор `m_Switch`.

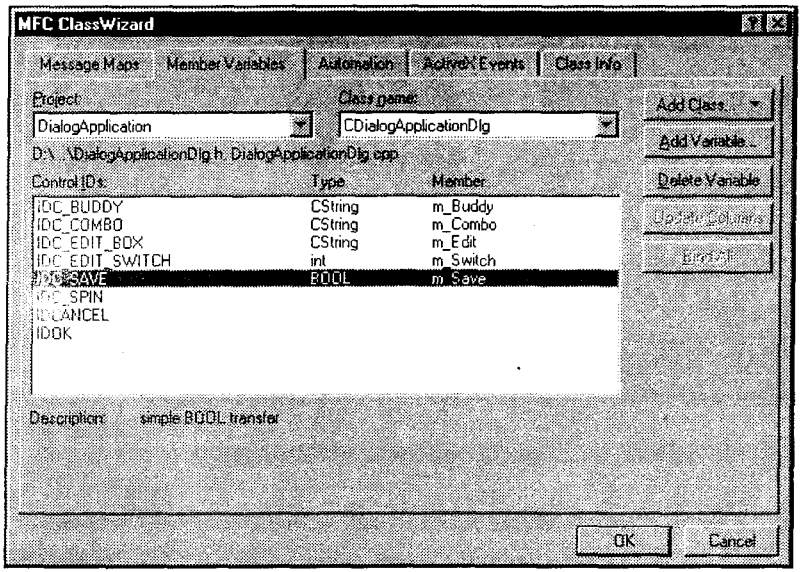


Рис. 3.19. Окончательный вид вкладки **Member Variables** диалогового окна **MFC ClassWizard**

9. Идентификатору ресурса `IDC_SAVE` сопоставьте идентификатор логической переменной логического типа `m_Save`. В результате описанных выше действий диалоговое окно **MFC ClassWizard** примет вид, приведенный на рис. 3.19.

10. Раскройте вкладку **Message Maps** (Карты сообщений). Диалоговое окно **MFC ClassWizard** примет вид, приведенный на рис. 3.20.

    Данная вкладка имеет несколько окон и текстовых полей. В раскрывающемся списке **Project:** указано имя текущего проекта. Мы работаем только с одним проектом, поэтому у нас есть только один выбор в данном списке. В раскрывающемся списке **Class name:** (Имя класса) указано имя текущего класса. Поскольку мы запустили мастер ClassWizard из окна редактирования ресурса диалога, то текущим классом в настоящее время является класс обработки сообщений данного диалогового окна. Из этого раскрывающегося списка можно выбрать любой зарегистрированный в данном мастере класс. Под этими двумя раскрывающимися списками расположено текстовое поле, предназначенное только для вывода информации и не имеющее внешней рамки. В это поле выводится информация об именах файла заголовка и файла реализации данного класса.

    В окне списка **Object IDs:** (Идентификаторы объектов) перечислены идентификаторы ресурсов, доступные в данном классе, и имя самого класса, имеющего свой набор сообщений и функций их обработки. В ок-

не списка **Messages:** (Сообщения) перечислены все сообщения и функции их обработки, связанные с выделенным в предыдущем окне идентификатором объекта. В нижнем окне списка **Member functions:** (Функции-члены класса) перечислены все функции обработки сообщений, включенные в данный класс и связанные с ними сообщения.

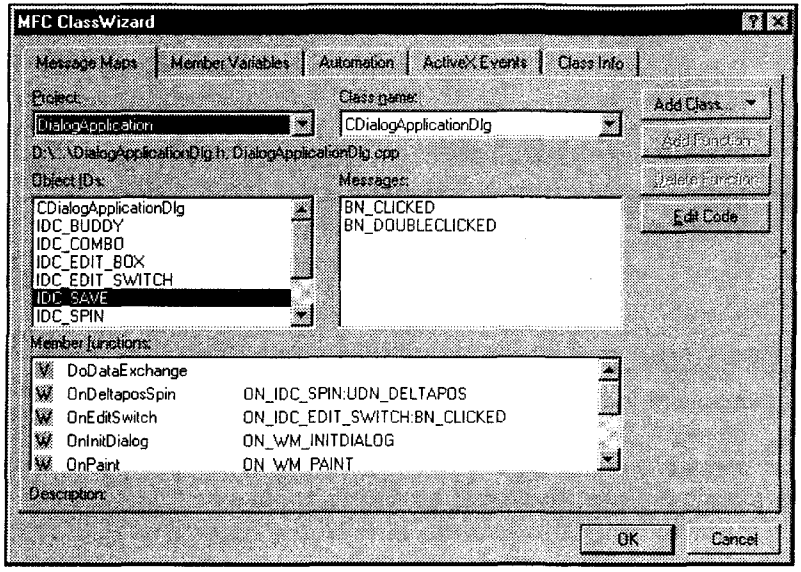


Рис. 3.20. Диалоговое окно **MFC ClassWizard**, вкладка **Message Maps**

11. Выделите в окне списка **Object IDs:** идентификатор ресурса `IDC_EDIT_SWITCH`, в окне списка **Messages:** выделите сообщение `BN_CLICKED` и нажмите кнопку **Add function**. Появится диалоговое окно **Add Member Function** (добавить функцию-член класса), изображенное на рис. 3.21.

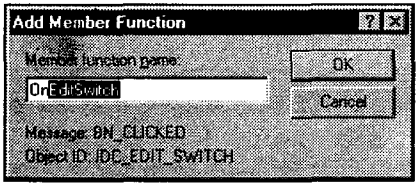


Рис. 3.21. Диалоговое окно **Add Member Function**

12. Оставьте без изменения предложенное в диалоговом окне имя функции и нажмите кнопку **OK**. Диалоговое окно **MFC ClassWizard** примет вид, приведенный на рис. 3.22.
13. Выделите в окне списка **Object IDs:** идентификатор ресурса `IDC_SPIN`, а в окне списка **Messages:** выделите сообщение `UDN_DELTAPOS` и повторите процедуру, описанную в пп. 11 и 12.

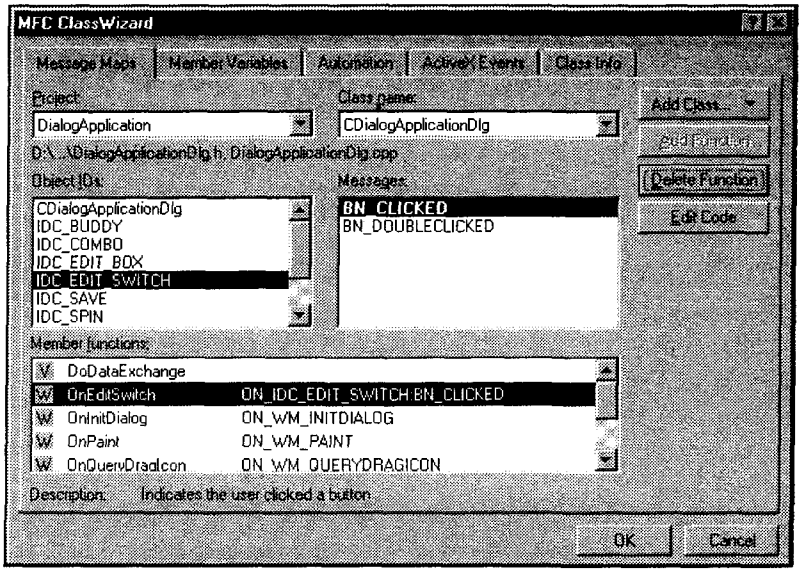


Рис. 3.22. Диалоговое окно **MFC ClassWizard**

14. Выделите в окне списка **Object IDs:** идентификатор ресурса `IDC_SPIN_SWITCH`, а в окне списка **Messages:** выделите сообщение `BN_CLICKED` и повторите процедуру, описанную в пп. 11 и 12.
15. Повторите эту операцию для идентификатора ресурса `IDC_COMBO` и сообщения `CBN_SELENDOK`.
16. Нажмите кнопку **OK** в диалоговом окне **MFC ClassWizard**.

По завершении описанной выше процедуры все операции, которые могли быть автоматизированы, уже выполнены. Осталось внести некоторые коррективы вручную.

Текст файла заголовка класса диалогового окна приведен в листинге 3.2.

Листинг 3.2. Файл заголовка класса диалогового окна

```
// DialogApplicationDlg.h : header file

#if !defined(AFX_DIALOGAPPLICATIONDLG_H__7BE57575_8222_11D2_9C0A_
                                                    9C8505994670__INCLUDED_)
#define AFX_DIALOGAPPLICATIONDLG_H__7BE57575_8222_11D2_9C0A_
                                                    9C8505994670__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000
```

```
/////////////////////////////////////////////////////////////////////////////
// CDialogApplicationDlg dialog

class CDialogApplicationDlg : public CDialog
{
// Construction
public:
  CDialogApplicationDlg(CWnd* pParent = NULL);  // standard constructor

// Dialog Data
  //{{AFX_DATA(CDialogApplicationDlg)
  enum { IDD = IDD_DIALOGAPPLICATION_DIALOG };
  CString  m_Buddy;
  CString  m_Combo;
  CString  m_Edit;
  int    m_Switch;
  BOOL  m_Save;
  //}}AFX_DATA

  // ClassWizard generated virtual function overrides
  //{{AFX_VIRTUAL(CDialogApplicationDlg)
  protected:
  virtual void DoDataExchange(CDataExchange* pDX);  // DDX/DDV support
  //}}AFX_VIRTUAL

// Implementation
protected:
  HICON m_hIcon;

  // Generated message map functions
  //{{AFX_MSG(CDialogApplicationDlg)
  virtual BOOL OnInitDialog();
  afx_msg void OnSysCommand(UINT nID, LPARAM lParam);
  afx_msg void OnPaint();
  afx_msg HCURSOR OnQueryDragIcon();
  afx_msg void OnEditSwitch();
  afx_msg void OnSpinSwitch();
  afx_msg void OnDeltaposSpin(NMHDR* pNMHDR, LRESULT* pResult);
  afx_msg void OnSelendokCombo();
  //}}AFX_MSG
  DECLARE_MESSAGE_MAP()
};

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately
before the previous line.

#endif // !defined(AFX_DIALOGAPPLICATIONDLG_H__7BE57575_8222_11D2_
                                                9C0A_9C8505994670__INCLUDED_)
```

Как следует из листинга 3.2, все функции обработки сообщений, как включенные в класс мастером AppWizard, так и созданные по нашему требованию мастером ClassWizard, помещены в карту сообщений файла заголовка. Это не значит, что все пользовательские программы нужно помещать туда же. Карта сообщений поддерживается соответствующими мастерами, и запись и удаление функций из нее лучше всего производить при помощи этих мастеров. Все пользовательские функции, не связанные с обработкой сообщений, включаются в класс диалогового окна таким же образом, как и в любой другой класс. Однако наша программа не будет содержать таких функций.

Файл реализации DialogApplicationDlg.cpp содержит не только реализацию класса обработки сообщений созданного нами диалогового окна, но и описание класса обработки сообщений диалогового окна **About**, которое нас в настоящий момент не интересует. Поэтому его реализация в листинге 3.3 отсутствует.

Листинг 3.3. Реализация класса диалогового окна

```
/////////////////////////////////////////////////////////////////////////////
// CDialogApplicationDlg dialog

CDialogApplicationDlg::CDialogApplicationDlg(CWnd* pParent /*=NULL*/)
  : CDialog(CDialogApplicationDlg::IDD, pParent)
{
  //{{AFX_DATA_INIT(CDialogApplicationDlg)
  m_Buddy = _T("");
  m_Combo = _T("");
  m_Edit = _T("");
  m_Switch = -1;
  m_Save = FALSE;
  //}}AFX_DATA_INIT
  // Note that LoadIcon does not require a subsequent DestroyIcon in Win32
  m_hIcon = AfxGetApp()->LoadIcon(IDR_MAINFRAME);
}

void CDialogApplicationDlg::DoDataExchange(CDataExchange* pDX)
{
  CDialog::DoDataExchange(pDX);
  //{{AFX_DATA_MAP(CDialogApplicationDlg)
  DDX_Text(pDX, IDC_BUDDY, m_Buddy);
  DDV_MaxChars(pDX, m_Buddy, 2);
  DDX_CBString(pDX, IDC_COMBO, m_Combo);
  DDV_MaxChars(pDX, m_Combo, 3);
  DDX_Text(pDX, IDC_EDIT_BOX, m_Edit);
  DDX_Radio(pDX, IDC_EDIT_SWITCH, m_Switch);
  DDX_Check(pDX, IDC_SAVE, m_Save);
  //}}AFX_DATA_MAP
}
```

```
BEGIN_MESSAGE_MAP(CDialogApplicationDlg, CDialog)
  //{{AFX_MSG_MAP(CDialogApplicationDlg)
  ON_WM_SYSCOMMAND()
  ON_WM_PAINT()
  ON_WM_QUERYDRAGICON()
  ON_BN_CLICKED(IDC_EDIT_SWITCH, OnEditSwitch)
  ON_BN_CLICKED(IDC_SPIN_SWITCH, OnSpinSwitch)
  ON_NOTIFY(UDN_DELTAPOS, IDC_SPIN, OnDeltaposSpin)
  ON_CBN_SELENDOK(IDC_COMBO, OnSelendokCombo)
  //}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CDialogApplicationDlg message handlers

BOOL CDialogApplicationDlg::OnInitDialog()
{
  CDialog::OnInitDialog();

  // Add "About..." menu item to system menu.

  // IDM_ABOUTBOX must be in the system command range.
  ASSERT((IDM_ABOUTBOX & 0xFFF0) == IDM_ABOUTBOX);
  ASSERT(IDM_ABOUTBOX < 0xF000);

  CMenu* pSysMenu = GetSystemMenu(FALSE);
  if (pSysMenu != NULL)
  {
    CString strAboutMenu;
    strAboutMenu.LoadString(IDS_ABOUTBOX);
    if (!strAboutMenu.IsEmpty())
    {
      pSysMenu->AppendMenu(MF_SEPARATOR);
      pSysMenu->AppendMenu(MF_STRING, IDM_ABOUTBOX, strAboutMenu);
    }
  }

  // Set the icon for this dialog.  The framework does this automatically
  //  when the application's main window is not a dialog
  SetIcon(m_hIcon, TRUE);      // Set big icon
  SetIcon(m_hIcon, FALSE);     // Set small icon

  // TODO: Add extra initialization here

  return TRUE;  // return TRUE  unless you set the focus to a control
}

void CDialogApplicationDlg::OnSysCommand(UINT nID, LPARAM lParam)
{
  if ((nID & 0xFFF0) == IDM_ABOUTBOX)
```

```
    {
      CAboutDlg dlgAbout;
      dlgAbout.DoModal();
    }
    else
    {
      CDialog::OnSysCommand(nID, lParam);
    }
}

// If you add a Minimize button to your dialog, you will need the code
// below to draw the icon.  For MFC applications using the document/view
// model, this is automatically done for you by the framework.

void CDialogApplicationDlg::OnPaint()
{
  if (IsIconic())
  {
    CPaintDC dc(this); // device context for painting

    SendMessage(WM_ICONERASEBKGND, (WPARAM) dc.GetSafeHdc(), 0);

    // Center icon in client rectangle
    int cxIcon = GetSystemMetrics(SM_CXICON);
    int cyIcon = GetSystemMetrics(SM_CYICON);
    CRect rect;
    GetClientRect(&rect);
    int x = (rect.Width() - cxIcon + 1) / 2;
    int y = (rect.Height() - cyIcon + 1) / 2;

    // Draw the icon
    dc.DrawIcon(x, y, m_hIcon);
  }
  else
  {
    CDialog::OnPaint();
  }
}

// The system calls this to obtain the cursor to display while
// the user drags the minimized window.
HCURSOR CDialogApplicationDlg::OnQueryDragIcon()
{
  return (HCURSOR) m_hIcon;
}

void CDialogApplicationDlg::OnEditSwitch()
{
  // TODO: Add your control notification handler code here
```

```
  UpdateData();
}

void CDialogApplicationDlg::OnDeltaposSpin(NMHDR* pNMHDR, LRESULT* pResult)
{
  NM_UPDOWN* pNMUpDown = (NM_UPDOWN*)pNMHDR;
  // TODO: Add your control notification handler code here

  *pResult = 0;
}

void CDialogApplicationDlg::OnSpinSwitch()
{
  // TODO: Add your control notification handler code here

}

void CDialogApplicationDlg::OnSelendokCombo()
{
  // TODO: Add your control notification handler code here

}
```

Вставьте в тело функций `OnEditSwitch`, `OnSpinSwitch` и `OnSelendokCombo` вызов функции `CWnd::UpdateData`. А в тело функции `OnDeltaposSpin` перед последним оператором вставьте оператор `pNMUpDown-> iDelta = - pNMUpDown-> iDelta;`, после чего функции обработки указанных нами сообщений примут вид, приведенный в листинге 3.4.

Листинг 3.4. Функции обработки сообщений класса диалогового окна

```
void CDialogApplicationDlg::OnEditSwitch()
{
  // TODO: Add your control notification handler code here

  UpdateData();
}

void CDialogApplicationDlg::OnDeltaposSpin(NMHDR* pNMHDR, LRESULT* pResult)
{
  NM_UPDOWN* pNMUpDown = (NM_UPDOWN*)pNMHDR;
  // TODO: Add your control notification handler code here

  pNMUpDown-> iDelta = - pNMUpDown-> iDelta;

  *pResult = 0;
}
```

```
void CDialogApplicationDlg::OnSpinSwitch()
{
  // TODO: Add your control notification handler code here

  UpdateData();
}

void CDialogApplicationDlg::OnSelendokCombo()
{
  // TODO: Add your control notification handler code here

  UpdateData();
}
```

Измените тексты тела функций конструктора класса и функции `DoDataExchange` таким образом, чтобы они соответствовали содержимому листинга 3.5.

Листинг 3.5. Конструктор класса диалогового окна и функция `DoDataExchange`

```
CDialogApplicationDlg::CDialogApplicationDlg(CWnd* pParent /*=NULL*/)
  : CDialog(CDialogApplicationDlg::IDD, pParent)
{
  //{{AFX_DATA_INIT(CDialogApplicationDlg)
  m_Buddy = _T("24");
  m_Combo = _T("64");
  m_Edit = _T("32");
  m_Switch = 0;
  m_Save = FALSE;
  //}}AFX_DATA_INIT
  // Note that LoadIcon does not require a subsequent DestroyIcon in Win32
  m_hIcon = AfxGetApp()->LoadIcon(IDR_MAINFRAME);
}

void CDialogApplicationDlg::DoDataExchange(CDataExchange* pDX)
{
  CDialog::DoDataExchange(pDX);
  //{{AFX_DATA_MAP(CDialogApplicationDlg)
  DDX_Text(pDX, IDC_BUDDY, m_Buddy);
  DDV_MaxChars(pDX, m_Buddy, 2);
  DDX_CBString(pDX, IDC_COMBO, m_Combo);
  DDV_MaxChars(pDX, m_Combo, 3);
  DDX_Text(pDX, IDC_EDIT_BOX, m_Edit);
  DDX_Radio(pDX, IDC_EDIT_SWITCH, m_Switch);
  DDX_Check(pDX, IDC_SAVE, m_Save);
  //}}AFX_DATA_MAP
```

```
  if( m_Save){

    switch( m_Switch){
      case 0: SetDlgItemText(IDC_EDIT_BOX, m_Combo);
          break;
      case 1:  SetDlgItemText(IDC_BUDDY, m_Combo);
    }
  }
}
```

Нажмите кнопку **Execute Program** на панели инструментов **Build MiniBar** или нажмите клавишу <F5>. Появится диалоговое окно **Microsoft Visual C++**, изображенное на рис. 3.23.

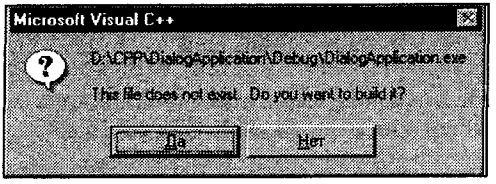


Рис. 3.23. Диалоговое окно **Microsoft Visual C++**

Нажмите кнопку **Да**. Произойдет трансляция программы, и появится диалоговое окно, изображенное на рис. 3.24.

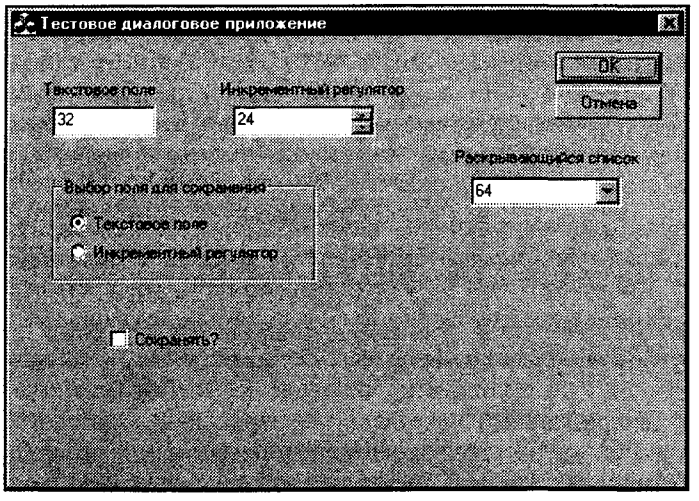


Рис. 3.24. Диалоговое окно программы

Как следует из анализа листинга 3.5 и рис. 3.24, первый оператор в конструкторе класса присваивания задает начальное значение содержимого элемента, связанного с инкрементным регулятором. Второй оператор производит начальную установку содержимого текстового поля раскрывающегося списка, а третий оператор устанавливает начальное значение для простого

текстового поля. Обнуление переменной `m_Switch` инициализирует соответствующий переключатель и устанавливает его в положение, соответствующее первому идентификатору ресурса в группе (если эту переменную приравнять −1, то переключатель не будет установлен ни в одно из возможных положений). Если переменной `m_Save` присвоить значение `FALSE`, то соответствующий флажок будет сброшен.

Поработайте с диалоговым окном приложения. Попробуйте ввести в связанный элемент трехзначное число. У вас ничего не получится, поскольку вы указали, что строка в этом текстовом поле должна содержать не более двух символов. Попробуйте ввести четырехзначное число в текстовое поле раскрывающегося списка. Раскройте список и выделите любое из содержащихся в нем чисел. Появится сообщение о том, что нужно ввести не более трех символов и, после нажатия кнопки **OK**, в текстовом поле появится выбранное вами число, содержащее не более трех цифр. Такое странное поведение диалогового окна объясняется тем, что диалоговое окно не предназначено для интенсивной работы с данными. Предполагается, что это окно должно быть заполнено при создании, а затем пользователь может вносить в него данные и менять установки, причем контроль над пользователем на этой стадии считается вмешательством в его личную жизнь. После этого окно закрывается, из него считывается занесенная пользователем информация, и только на этой стадии производится ее контроль.

Но так получилось, что в Windows имеется только два типа окон: обычное окно и диалоговое. Поэтому, все, что нельзя сделать в обычном окне, приходится делать в диалоговом. Вследствие этого на диалоговые окна часто приходится возлагать функции, которые не могут быть реализованы в схеме работы с единственной проверкой данных при закрытии окна. Во многих случаях необходимо, чтобы изменения, внесенные в один элемент управления, немедленно отражались на состоянии других элементов управления. Примером реализации такой возможности является данное приложение.

При установленном флажке **Сохранять?** предыдущее содержимое текстового поля раскрывающегося списка переписывается в простое текстовое поле или в элемент, связанный с инкрементным регулятором, причем конкретный адрес определяется положением переключателя **Выбор поля для сохранения**.

Раскройте список и выберите в нем величину 256. Установите флажок **Сохранять?** Ничего не произошло. Установите переключатель **Выбор поля для сохранения** в положение **Инкрементный регулятор**. Содержимое текстового поля раскрывающегося списка копируется в элемент, связанный с инкрементным регулятором. Установите переключатель **Выбор поля для сохранения** в положение **Текстовое поле**. Появится окно сообщения, требующее вводить не более двух символов. Оно относится к содержимому элемента, связанного с инкрементным регулятором, хотя ничто в сообщении на это не указывает. Используя кнопку управления инкрементным регулятором или соответствующую клавишу, уменьшите величину в связанном элементе на единицу. Вместо числа 256 в нем появится 99. Установите в текстовом поле

раскрывающегося списка вместо числа 256, число 16. Содержимое простого текстового поля меняется на 256, то есть на прежнее содержимое текстового поля раскрывающегося списка.

Такой сложный алгоритм работы определяется тем, что для обновления полей элементов управления приходится использовать обходные пути. Введенные нами в тела функций обработки сообщений вызовы функции `CWnd::UpdateData` служат единственной цели — внеочередному вызову функции `CWnd::DoDataExchange`, осуществляющей чтение данных из полей элементов управления и обновления информации, хранящейся в соответствующих величинах. После обновления информации нами производится ее оценка и обновление текстовых полей с использованием функции `CWnd::SetDlgItemText`.

Как следует из листинга 3.4, обновление содержимого полей производится при нажатии на кнопку любого из переключателей или при проверке допустимости введенной пользователем величины в текстовом поле раскрывающегося списка.

Функция `OnDeltaposSpin` используется для установления естественного соответствия клавиш управления величиной в отростке направлению изменения этой величины. Как можно убедиться в режиме моделирования диалогового окна, разработчики библиотеки MFC полагали, что стрелке вниз должно соответствовать увеличение значения, а стрелке вверх — его уменьшение, в то время как все остальные разработчики Microsoft, например разработчики Word, считали естественным обратное соответствие. Данная функция устанавливает более распространенное соответствие клавиш направлению изменения величины. Попутно следует отметить еще одну странность инкрементного регулятора: определив из установленного пользователем количества используемых символов, что максимальная величина в связанном элементе должна составлять 99, разработчики зачем-то установили ее равной 100. При попытке использовать эту величину, появляется сообщение об ошибке.

Вкладки и мастера

Работая с продуктами фирмы Microsoft, вы постоянно встречались с диалоговыми окнами, содержащими несколько вкладок, и с мастерами, в которых содержимое диалогового окна меняется при нажатии кнопок **Далее>** (Next) и **<Назад** (Back). Подобные возможности существенно расширяют функции диалога пользователя, предоставляемого приложениями, разработанными в среде программирования Visual C++.

Создание вкладок диалогового окна

Текст диалогового приложения, использующего вкладки в диалоговом окне, располагается в директории SheetApplication на демонстрационной дискете, поставляемой с данной книгой.

Чтобы самостоятельно создать данное демонстрационное приложение:

1. Создайте новое диалоговое приложение с именем SheetApplication, следуя указаниям по созданию подобного приложения, изложенным в главе 1.
2. Уничтожьте все элементы управления, расположенные на заготовке диалогового окна, включая кнопки.
3. Щелкните правой кнопкой мыши по заготовке диалогового окна и в появившемся контекстном меню выберите команду **Properties**, открывающую одноименное диалоговое окно.
4. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна **Dialog Properties**.
5. Раскройте вкладку **General** (Общие свойства) диалогового окна **Dialog Properties**, измените в текстовое поле **ID:** идентификатор ресурса на `IDD_PAGE1` и введите в текстовое поле **Caption** текст "Панель1".
6. Раскройте вкладку **Styles** (Стили) этого диалогового окна. Из раскрывающегося списка **Style:** (Стиль) выберите **Child** (Дочернее окно), из раскрывающегося списка **Border:** (Рамка) выберите **Thin** (Тонкая). Снимите флажок **System menu** (Системное меню). Диалоговое окно **Dialog Properties** примет вид, изображенный на рис. 3.25.

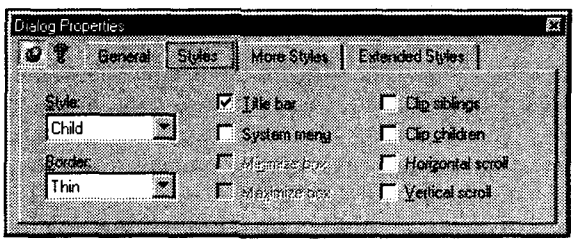


Рис. 3.25. Диалоговое окно **Dialog Properties**

7. В панели инструментов **Controls** выберите элемент управления **Check Box** и поместите его на заготовку диалогового окна.
8. Откройте диалоговое окно **Check Box Properties**, введите в текстовое поле **Caption** текст "Разрешить переход?" и отцентрируйте его с помощью кнопок панели инструментов **Dialog**. В результате получится окно, изображенное на рис. 3.26.
9. Щелкните правой кнопкой мыши на идентификаторе ресурса `IDD_PAGE1`, расположенного в папке \Dialog, во вкладке **ResourceView** окна **Workspace**. И выберите значение **Russian** в раскрывающемся списке **Language:** появившегося диалогового окна **Dialog Properties**, как это показано на рис. 3.27.
10. Вызовите мастер ClassWizard. На фоне диалогового окна **MFC ClassWizard** появится диалоговое окно **Adding a Class** (Добавление класса), изображен-

ное на рис. 3.28. В этом окне сообщается, что идентификатор ресурса `IDD_PAGE1` соответствует новому ресурсу и спрашивается, создавать ли для него новый класс.

11. Оставьте установку переключателя в положении по умолчанию **Create a new class** (Создать новый класс) и нажмите кнопку **OK**. Появится диалоговое окно **New Class** (Новый класс), изображенное на рис. 3.29.

12. Введите в текстовое поле **Name:** идентификатор `CPage1`. В раскрывающемся списке **Base class:** (Базовый класс) выберите класс `CPropertyPage` и нажмите кнопку **OK**. Диалоговое окно **MFC ClassWizard** станет активным.

13. Раскройте панель **Member Variables** диалогового окна **MFC ClassWizard**.

14. Сопоставьте идентификатору ресурса `IDC_CHECK1` логическую переменную `m_Check` и нажмите кнопку **OK**.

15. Во вкладке **ResourceView** окна **Workspace** щелкните правой кнопкой мыши на папке \SheetApplication Resources (Ресурсы приложения SheetApplication) и в появившемся контекстном меню выберите команду **Insert** (Вставить). Появится диалоговое окно **Insert Resource**, изображенное на рис. 3.30.

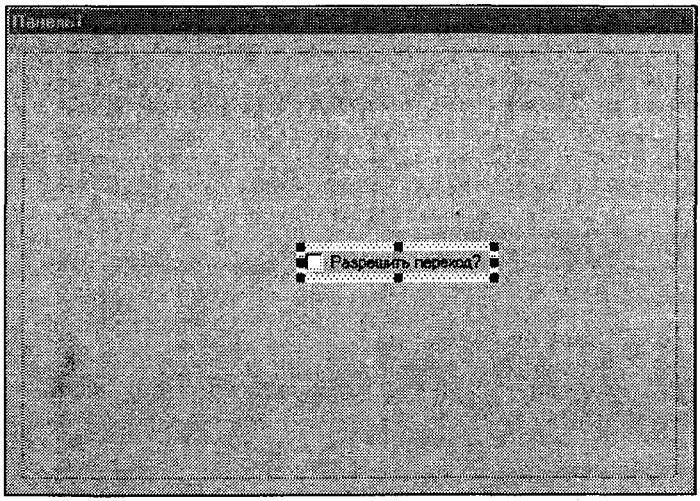


Рис. 3.26. Заготовка первой вкладки диалогового окна

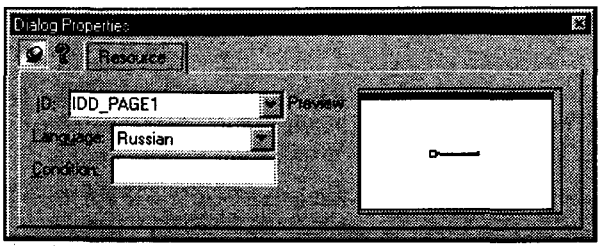


Рис. 3.27. Диалоговое окно **Dialog Properties**

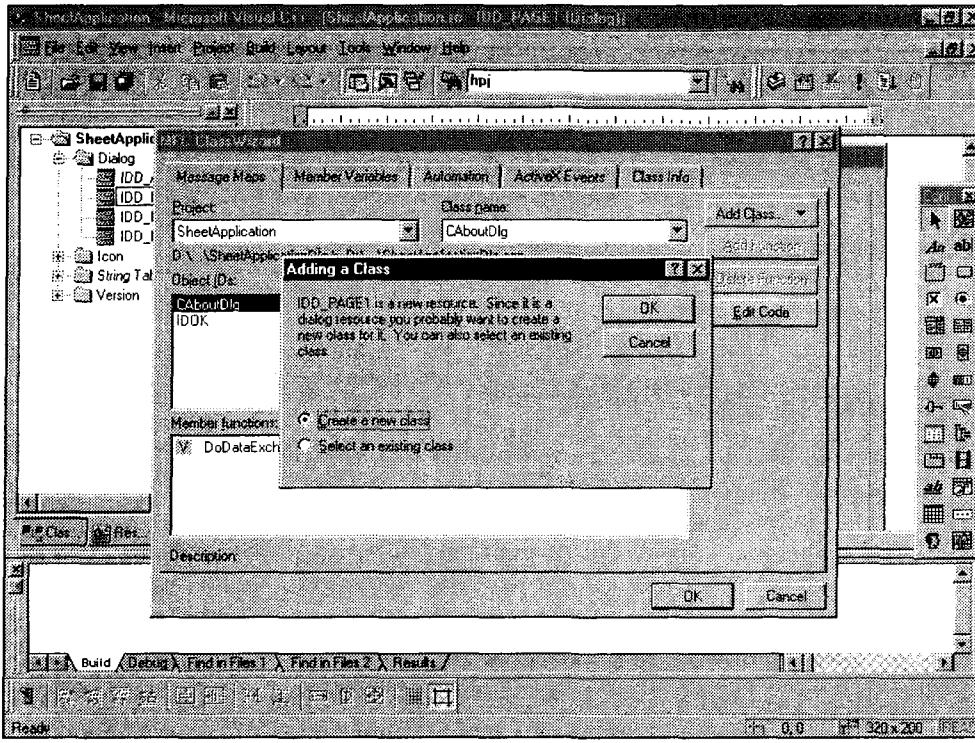


Рис. 3.28. Диалоговое окно **Adding a Class**

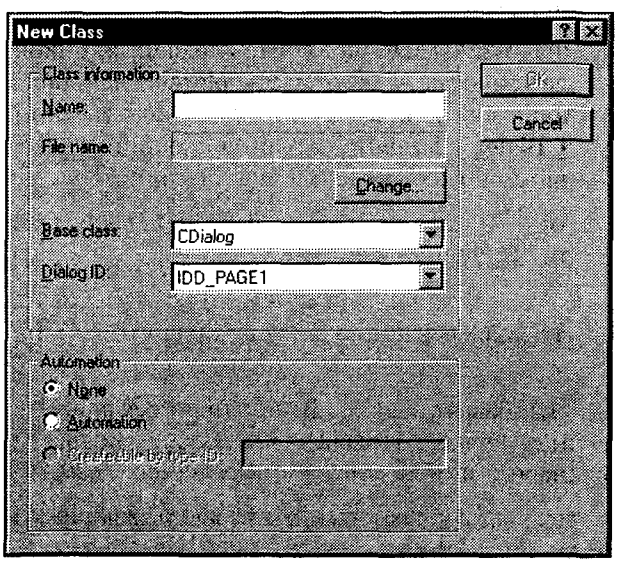


Рис. 3.29. Диалоговое окно **New Class**

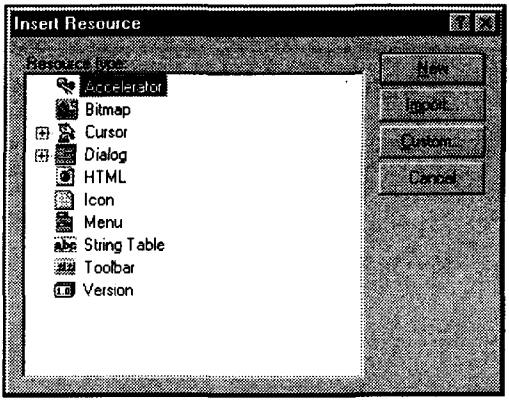


Рис. 3.30. Диалоговое окно **Insert Resource**

16. Выберите тип ресурса **Dialog** и нажмите кнопку **New**. В окне редактирования ресурсов появится новая заготовка диалогового окна.
17. Щелкните на этой заготовке и, "зацепившись" за синий квадрат в правом нижнем углу рамки, измените размер диалогового окна таким образом, чтобы он соответствовал размеру диалогового окна первой панели. Размер и положение выделенного элемента управления или диалогового окна указываются в панели состояния.
18. Замените идентификатор ресурса диалогового окна на `IDD_PAGE2`, а его заголовок на "Панель2". После этого повторите все операции, произведенные над панелью 1.
19. По методике, описанной при создании класса `CPage1`, создайте для нее класс `CPage2` и добавьте в него переменную `m_Check`.
20. Во вкладке **ResourceView** окна **Workspace** щелкните правой кнопкой мыши на папке \Dialog, расположенной в папке \SheetApplication Resources (Ресурсы приложения SheetApplication), и в появившемся контекстном меню выберите команду **Insert Dialog** (Вставить диалоговое окно). В окне редактирования ресурсов появится новая заготовка диалогового окна.
21. Создайте на основе этой заготовки третью панель с идентификатором ресурса `IDD_PAGE3` и заголовком "Панель3".
22. Вызовите мастер ClassWizard, создайте класс с именем `CPage3` и добавьте в него переменную `m_Check`. Не закрывайте после этого диалоговое окно **MFC ClassWizard**.
23. Нажмите кнопку **Add Class...** (Добавить класс). В появившемся контекстном меню выберите команду **New...** (Новый). На экране появится диалоговое окно **New Class**, аналогичное изображенному на рис. 3.29.
24. В текстовое поле **Name:** введите имя класса `CSheet`, в раскрывающемся списке **Base class:** выберите `CPropretySheet`.

25. Нажмите кнопку **OK** в диалоговом окне **New Class**. Мастер ClassWizard создаст новый класс с указанным именем.
26. Нажмите кнопку **OK** в диалоговом окне **MFC ClassWizard**.

На этом процесс создания заготовок программ закончен. Для завершения приложения SheetApplication:

1. Раскройте вкладку **FileView** (Просмотр файлов) окна **Workspace**. В ней раскройте папку \SheetApplication Files (Файлы приложения SheetApplication), а в ней откройте папку \Header Files (Файлы заголовков), как это показано на рис. 3.31.
2. Дважды щелкните левой кнопкой мыши по имени файла Sheet.h. Откроется окно редактирования соответствующего файла.
3. Перед объявлением класса `CSheet` вставьте операторы включения файлов заголовков классов вкладок диалогового окна:

```
#include "Page1.h"
#include "Page2.h"
#include "Page3.h"
```

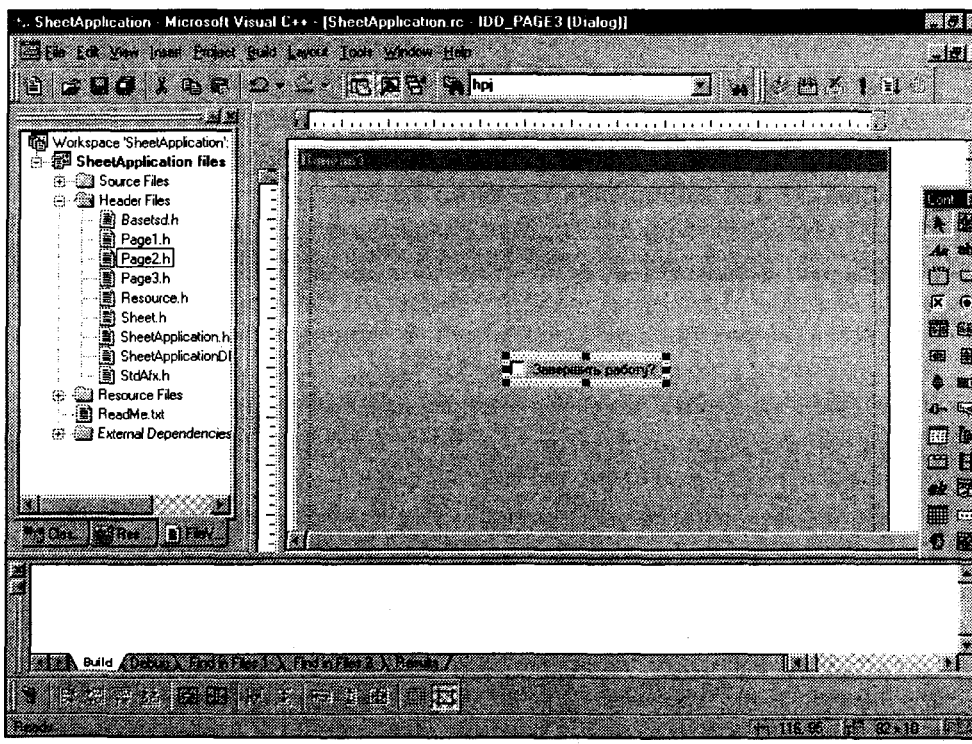


Рис. 3.31. Окно **Workspace**

4. В секцию `// Attributes` описания класса `CSheet` вставьте операторы объявления данных-членов класса, представляющих собой объекты классов вкладок:

```
CPage1  m_page1;
CPage2  m_page2;
CPage3  m_page3;
```

5. Во вкладке **FileView** окна **Workspace** откройте папку \Source Files (Файлы реализации) и дважды щелкните левой кнопкой мыши на имени файла Sheet.cpp.
6. Добавьте в тело функций обоих конструкторов класса `CSheet` операторы включения вкладок в диалоговое окно:

```
AddPage(&m_page1);
AddPage(&m_page2);
AddPage(&m_page3);
```

7. В файле SheetApplication.cpp в теле функции `InitInstance` замените оператор `CSheetApplicationDlg dlg;` оператором

```
CSheet dlg("Диалоговое окно с вкладками");
```

8. Нажмите клавишу <F5> и запустите проект на исполнение. Появится диалоговое окно, изображенное на рис. 3.32.

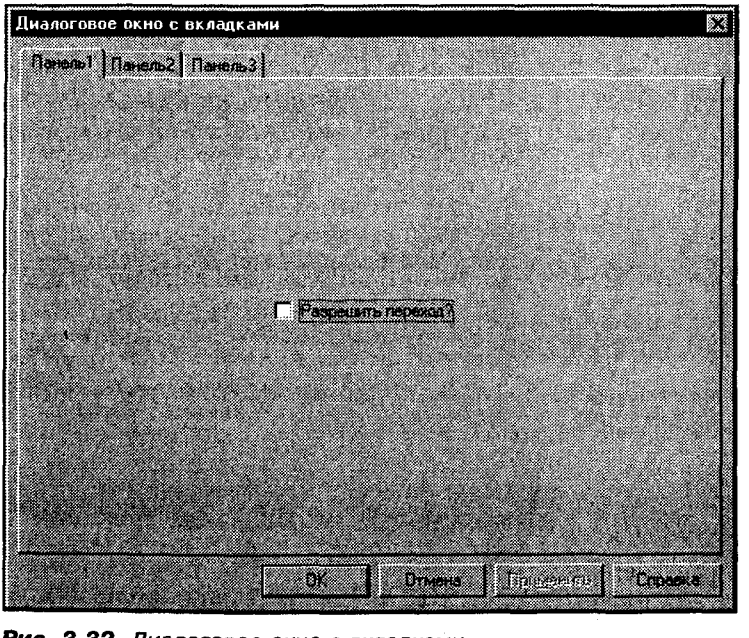


Рис. 3.32. Диалоговое окно с вкладками

Вы можете открыть любую вкладку, но ее содержимое не изменится, поскольку мы ввели во все вкладки идентичную информацию.

Создание мастера

Диалоговое окно мастера так же, как и диалоговое окно с вкладками, позволяет практически полностью менять набор элементов управления диалогового окна, но, в отличие от окна с вкладками, последовательность перехода от одного окна к другому в нем четко зафиксирована, а сам переход происходит посредством нажатия кнопок **Next>** и **<Back**.

Окно мастера можно рассматривать как разновидность диалогового окна с вкладками. Для демонстрации возможностей окна мастера рассмотрим описанное выше приложение `SheetApplication`.

В приложение `SheetApplication` внесите следующие изменения:

1. В файле SheetApplication.cpp в теле функции `InitInstance` после оператора `CSheet dlg("Диалоговое окно с вкладками");` вставьте оператор

   ```
   dlg.SetWizardMode();
   ```

2. Откройте окно редактирования файла Page1.cpp и вызовите из него мастер ClassWizard.
3. Раскройте вкладку **Message Maps**. Выделите в окне списка **Object IDs:** имя класса `CPage1`, выделите в окне списка **Messages:** имя функции `OnSetActive` и нажмите кнопку **Add Function** (Добавить функцию). Эта функция появится в окне списка **Member functions:**.
4. Таким же образом добавьте в класс `CPage1` функцию-член класса `OnWizardNext`. Диалоговое окно **MFC ClassWizard** будет выглядеть, как изображено на рис. 3.33.
5. Нажмите кнопку **OK** и выполните аналогичные операции для класса `CPage2`, но добавьте в нее дополнительно функцию-член класса `OnWizardBack`.
6. Нажмите кнопку **OK** и выполните аналогичные операции для класса `CPage3`, включив в него функции-члены класса `OnWizardBack` и `OnWizardNext`.
7. Откройте окно редактирования файла Page1.cpp и внесите в его функции изменения, зафиксированные в листинге 3.6.

Листинг 3.6. Функции обработки сообщений первой панели мастера

```
//////////////////////////////////////////////////////////////////////////
// CPage1 message handlers

BOOL CPage1::OnSetActive()
```

```
{
  // TODO: Add your specialized code here and/or call the base class
  CPropertySheet*  parent = (CPropertySheet*) GetParent();
  parent-> SetWizardButtons(PSWIZB_NEXT);
  return CPropertyPage::OnSetActive();
}

LRESULT CPage1::OnWizardNext()
{
  // TODO: Add your specialized code here and/or call the base class
  CButton* checkBox = (CButton*) GetDlgItem( IDC_CHECK1);

  if( !checkBox->GetCheck())
  {
    MessageBox("Установите флажок");
    return -1;
  }

  return CPropertyPage::OnWizardNext();
}
```

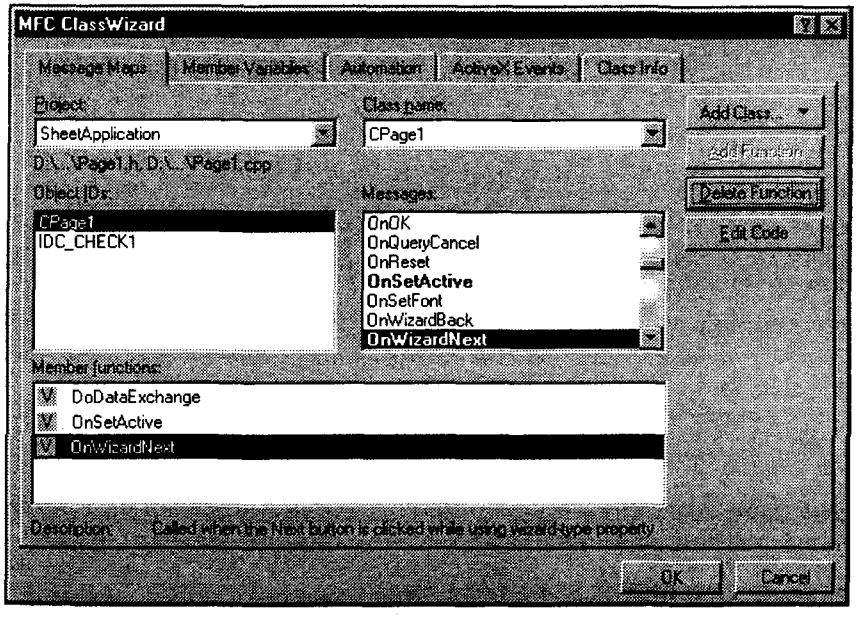


Рис. 3.33. Диалоговое окно **MFC ClassWizard**

8. Откройте файл Page2.cpp и внесите в его функции изменения, зафиксированные в листинге 3.7.

Листинг 3.7. Функции обработки сообщений второй панели мастера

```
//////////////////////////////////////////////////////////////////////
// CPage2 message handlers

BOOL CPage2::OnSetActive()
{
  // TODO: Add your specialized code here and/or call the base class
  CPropertySheet*  parent = (CPropertySheet*) GetParent();
  parent-> SetWizardButtons( PSWIZB_NEXT | PSWIZB_BACK);
  return CPropertyPage::OnSetActive();
}

LRESULT CPage2::OnWizardBack()
{
  // TODO: Add your specialized code here and/or call the base class

  UpdateData();
  if( !m_Check)
  {
    MessageBox("Установите флажок");
    return -1;
  }
  return CPropertyPage::OnWizardBack();
}

LRESULT CPage2::OnWizardNext()
{
  // TODO: Add your specialized code here and/or call the base class

  CButton* checkBox = (CButton*) GetDlgItem( IDC_CHECK1);
  if( !checkBox->GetCheck())
  {
    MessageBox("Установите флажок");
    return -1;
  }
  return CPropertyPage::OnWizardNext();
}
```

9. Откройте файл Page3.cpp и внесите в его функции изменения, зафиксированные в листинге 3.8.

Листинг 3.8. Функции обработки сообщений третьей панели мастера

```
//////////////////////////////////////////////////////////////////////
// CPage3 message handlers

BOOL CPage3::OnSetActive()
```

```
{
  // TODO: Add your specialized code here and/or call the base class
  CPropertySheet*  parent = (CPropertySheet*) GetParent();
  parent-> SetWizardButtons(PSWIZB_BACK);
  return CPropertyPage::OnSetActive();
}

LRESULT CPage3::OnWizardBack()
{
  // TODO: Add your specialized code here and/or call the base class
  UpdateData();
  if( !m_Check)
  {
    MessageBox("Установите флажок");
    return -1;
  }
  return CPropertyPage::OnWizardBack();
}
```

10. Запустите приложение на исполнение. Появится диалоговое окно мастера, изображенное на рис. 3.34.

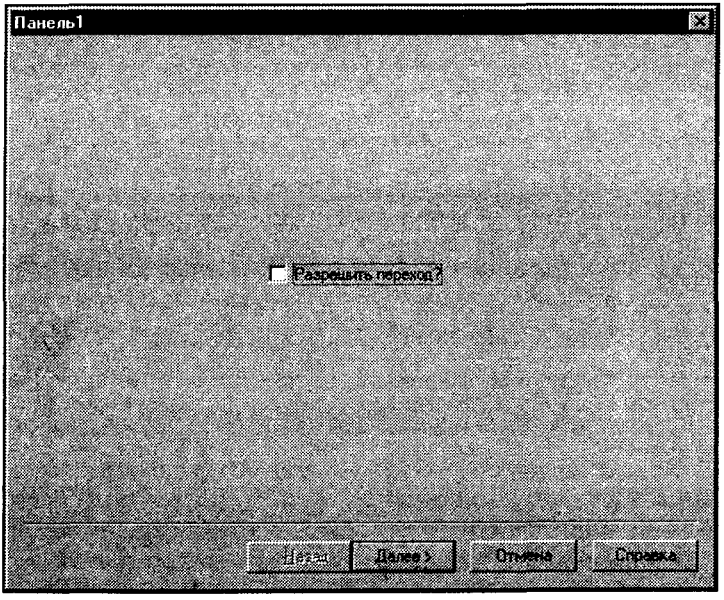


Рис. 3.34. Диалоговое окно мастера. **Панель1**

11. Нажмите на кнопку **Далее>**. Появится окно сообщения, изображенное на рис. 3.35.

Рис. 3.35. Окно сообщения

12. Нажмите кнопку **OK**, установите флажок **Разрешить переход?** и снова нажмите кнопку **Далее>**. Появится диалоговое окно мастера, изображенное на рис. 3.36.

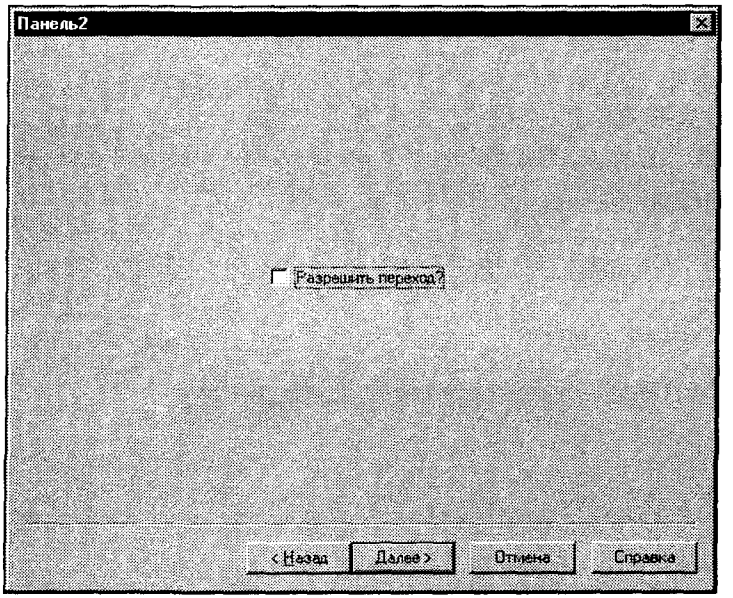


Рис. 3.36. Диалоговое окно мастера. **Панель2**

13. Установите флажок **Разрешить переход?** и нажмите кнопку **Далее>**. Появится диалоговое окно мастера, изображенное на рис. 3.37.
14. Установите флажок **Разрешить переход?** и нажмите кнопку **<Назад**. Появится диалоговое окно мастера, изображенное на рис. 3.36, но с установленным флажком.

Как видно из сравнения рисунков, в диалоговом окне **Панель1** недоступна кнопка **<Назад**, в диалоговом окне **Панель3** — кнопка **Далее>**, а в диалоговом окне **Панель2** доступны обе эти кнопки. Доступность этих кнопок определяется функцией `CPropertySheet::SetWizardButtons`, обычно вызываемой из функции `CPropertyPage::OnSetActive`, поскольку данная функция может вызываться только после открытия диалога, то есть после вызова функции `DoModal`. Функция `SetWizardButtons` определяет функцию и внешний вид кнопок диалогового окна мастера. В качестве ее параметра выступает набор

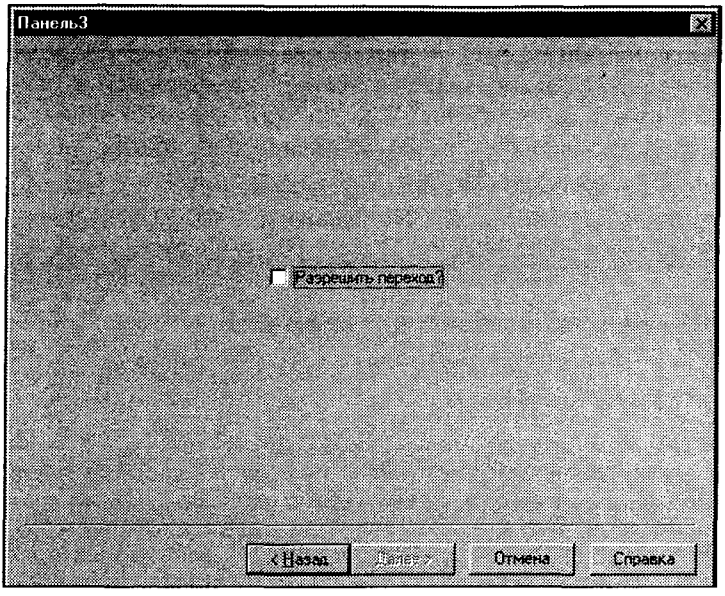


Рис. 3.37. Диалоговое окно мастера. **Панель3**

флагов, объединенных функцией логического ИЛИ, этот набор включает в себя следующие флаги:

- `PSWIZB_BACK` — делает доступной кнопку **<Назад**
- `PSWIZB_NEXT` — отображает и делает доступной кнопку **Далее>**
- `PSWIZB_FINISH` — отображает и делает доступной кнопку **Готово (Finish)**
- `PSWIZB_DISABLEFINISH` — делает недоступной кнопку **Готово**

Следует отметить, что флаги `PSWIZB_NEXT` и `PSWIZB_FINISH` являются несовместимыми, поскольку относятся к одной и той же физической кнопке. Поэтому панель может иметь или кнопку **Далее>** или кнопку **Готово**, но не обе сразу. Флаг `PSWIZB_DISABLEFINISH` может появиться только в комбинации с флагом `PSWIZB_FINISH`.

Некоторые модификации окна мастера

Окно мастера является очень удобной модификацией диалогового окна, но оно нацелено на использование в одном, достаточно конкретном случае, когда последовательность смены окон определена заранее и не может меняться ни при каких обстоятельствах. Однако на практике часто возникает необходимость определять последовательность перехода между окнами, например при реализации диалогового окна, управляющего обработкой данных, когда в первой панели стоит переключатель, определяющий тип обработки, и в зависимости от его положения нажатие кнопки **Далее>** приводит к появлению панели, устанавливающей параметры выбранного типа обработки.

Изменим наше приложение таким образом, чтобы из первой панели переход осуществлялся сразу в третью панель, и уже из нее — во вторую. Кроме того, сделаем так, чтобы при установке флага в третьей панели работа мастера считалась завершенной, и пользователь уже не мог бы переходить к другим панелям, а мог только завершить работу с диалоговым окном мастера. Чтобы внести все необходимые для этого изменения в проект:

1. Раскройте вкладку **ResourceView** окна **Workspace** и раскройте папку \SheetApllication resources.
2. Раскройте папку \Dialog и дважды щелкните левой кнопкой мыши на значке **IDD_PAGE3**. Появится окно редактирования ресурса.
3. Откройте диалоговое окно **CheckBox Properties** и замените текст в текстовом поле **Caption** на "Завершить работу?".
4. Закройте окно редактирования ресурсов и откройте вкладку **FileView** окна **Workspace**. Дважды щелкните левой кнопкой мыши на значке **Page1.cpp**. Появится окно редактирования файла реализации.
5. Вставьте в раздел включения файлов заголовка строку

```
#include "Sheet.h"
```

6. Внесите изменения в функцию `OnWizardNext` в соответствии с приведенным ниже текстом:

```
LRESULT CPage1::OnWizardNext()
{
  // TODO: Add your specialized code here and/or call the base class
  CSheet*  parent = (CSheet*) GetParent();
  CButton* checkBox = (CButton*) GetDlgItem( IDC_CHECK1);
  if( !checkBox->GetCheck())
  {
    MessageBox("Установите флажок");
    return -1;
  }
  parent-> SetActivePage(parent-> GetPageIndex(&parent-> m_page3) - 1);
  return CPropertyPage::OnWizardNext();
}
```

7. Откройте окно редактирования файла Page3.cpp и вызовите из него мастер ClassWizard.
8. В окне списка **Object IDs:** выделите идентификатор ресурса `IDC_CHECK1`, в окне списка **Messages:** выделите сообщение `BN_CLICKED` и нажмите кнопку **Add Function**.
9. В появившемся диалоговом окне **Add Member Function** нажмите кнопку **OK**, согласившись с предложенным именем функции. В окне списка **Member functions:** появится имя новой функции `OnCheck1`.

10. Нажмите кнопку **OK** и закройте диалоговое окно **MFC ClassWizard**.
11. В окно редактирования файла Page3.cpp перед описанием класса `CPage3` вставьте строку

```
#include "Sheet.h"
```

12. В текст программы внесите изменения, содержащиеся в листинге 3.9.

Листинг 3.9. Функции обработки сообщений класса CPage3

```
LRESULT CPage3::OnWizardBack()
{
  // TODO: Add your specialized code here and/or call the base class
  CSheet*  parent = (CSheet*) GetParent();
  parent-> SetActivePage( parent-> GetPageIndex( &parent-> m_page1) + 1);
  return CPropertyPage::OnWizardBack();
}

LRESULT CPage3::OnWizardNext()
{
  // TODO: Add your specialized code here and/or call the base class

  CSheet*  parent = (CSheet*) GetParent();
  parent-> SetActivePage(parent-> GetPageIndex(&parent-> m_page2) - 1);
  return CPropertyPage::OnWizardNext();
}

void CPage3::OnCheck1()
{
  // TODO: Add your control notification handler code here
  CSheet*  parent = (CSheet*) GetParent();
  parent->SetFinishText("Готово");
}
```

13. Запустите приложение на исполнение. Появится диалоговое окно мастера, изображенное на рис. 3.34.

На первый взгляд ничего не изменилось, но при нажатии кнопки **Далее>** вы переходите к панели 3, а не к панели 2, как в предыдущем случае. Обратите внимание, что в этой панели разблокирована кнопка **Далее>**, при нажатии которой переходите в панель 2. Порядок перехода из панели 2 остался неизменным. Если в панели 3 вы установите флаг **Завершить работу?**, то эта панель примет вид, изображенный на рис. 3.38.

Произвольный переход между панелями достигается использованием функции `CPropertySheet::SetActivePage`. Данная функция возвращает логическую величину, принимающую ненулевое значение в случае успешной активизации вкладки и нулевое значение — в противном случае. В качестве аргумента

этой функции может выступать индекс устанавливаемой вкладки или непосредственно указатель на объект класса данной вкладки. Индекс вкладки может принимать любое положительное значение, включая 0, не превышающее количества вкладок, включенных в данный мастер. Порядок нумерации вкладок соответствует порядку их включения в класс мастера функциями `AddPage`. В нашем случае следует учитывать, что функция `SetActivePage` вызывается в функциях `OnWizardNext` и `OnWizardBack`, которые сами осуществляют переход между вкладками, поэтому при указании индекса вкладки необходимо учесть результат выполнения функции, из которой производится вызов.

Приведенная программа также демонстрирует результат вызова функции `CPropertySheet::SetFinishText`. Как видно из ее названия, эта функция служит для определения текста, отображаемого в кнопке **Готово** (Finish). Однако основное назначение этой функции заключается в том, что она убирает из окна мастера кнопки **Далее>** и **<Назад** и заменяет их одной кнопкой **Готово**, в которой будет отображаться текст, введенный в функцию в качестве параметра. Данная функция имеет необратимый эффект и после установки флажка **Завершить работу?** его сброс или повторная установка не приведут к появлению кнопок **Далее>** и **<Назад**. Обычно эта функция вызывается пользователем после завершения всех процедур, предусмотренных в мастере. Действия, которые будут предприняты при нажатии кнопки **Готово**, определяются функцией `OnWizardFinish`, вызываемой с использованием мастера ClassWizard.

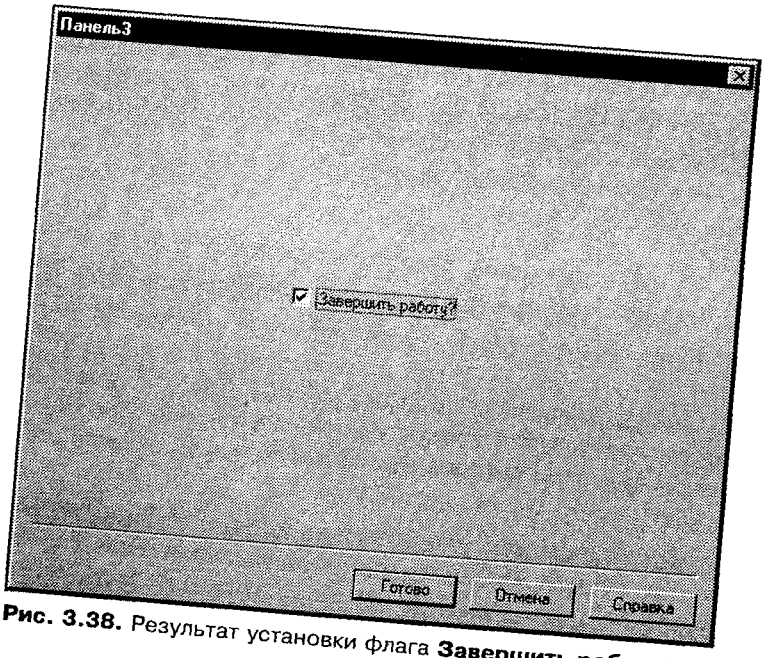


Рис. 3.38. Результат установки флага **Завершить работу?**

На функции `OnWizardFinish` следует остановиться подробнее. Основной вопрос заключается в том, где ее вызывать. Одним из возможных мест вызова данной функции является функция `OnSetActive` последней вкладки мастера, где она может заменить функцию `SetWizardButtons`. После такого вызова окно мастера, в котором отображается последняя вкладка, будет содержать только кнопки **Готово** и **Отмена**. Но в этом случае пользователь лишается возможности внести изменения в установки, произведенные им на предыдущих стадиях настройки мастера. Поскольку на последней странице мастера часто помещается сводка произведенных пользователем установок, то отсутствие на ней кнопки **<Назад** можно расценить как тонкую форму издевательства над пользователем.

Поэтому кнопку **Готово** лучше всего устанавливать в том случае, когда установки, произведенные им на предыдущих стадиях настройки мастера, используются как параметры некоторой достаточно длительной процедуры, которая может быть в любой момент прервана пользователем с сохранением результатов обработки. В этом случае кнопка **Готово** появляется после запуска этой обработки и служит для ее завершения без потери результата или выхода из диалогового окна после завершения обработки, если в результате проведенной обработки в диалоговое окно выводится информация, которая должна быть оценена пользователем.

Использование Component Gallery

В начале данной главы рассматривался вопрос о создании пользовательского файла ресурсов и было сказано, что одной из причин, по которой нецелесообразно создавать такой файл, является наличие в Visual C++ библиотеки стандартных компонентов, которая может пополняться пользователем.

Действительно, после долгих и упорных трудов по созданию некоторого законченного приложения у пользователя возникает естественное желание максимально использовать компоненты данного приложения в своих дальнейших разработках. Действительно, в процессе создания приложения пользователь создает множество различных классов, многие из которых являются достаточно универсальными, и нет необходимости разрабатывать их заново для каждого приложения.

Component Gallery (галерея компонентов) представляет собой один из способов поддержки идеи повторного использования классов в среде Visual Studio. Однако следует задуматься, в каких случаях и в какой форме следует повторно использовать уже разработанные классы.

Если пользователь разработал достаточно сложную и взаимосвязанную систему классов, позволяющих достаточно просто осуществлять обработку некоторого класса данных, то ему нет никакого смысла использовать Component Gallery, т. к. в ней каждый класс хранится отдельно, поскольку это

приведет к разбиению созданной им библиотеки на отдельные классы, тесная связь которых, все равно, не позволит включать их в приложение по частям. В данном случае целесообразно оформить их в виде библиотеки динамической компоновки или, если пользователь не считает эту библиотеку завершенной и собирается вносить в нее изменения, создать несколько файлов заголовков и связанных с ними файлов реализации и включать их в проект с использованием команды меню **Project**, **Add to Project**, **Files...**. Данная команда меню открывает диалоговое окно **Insert Files into Project** (Добавление файлов в проект), изображенное на рис. 3.39.

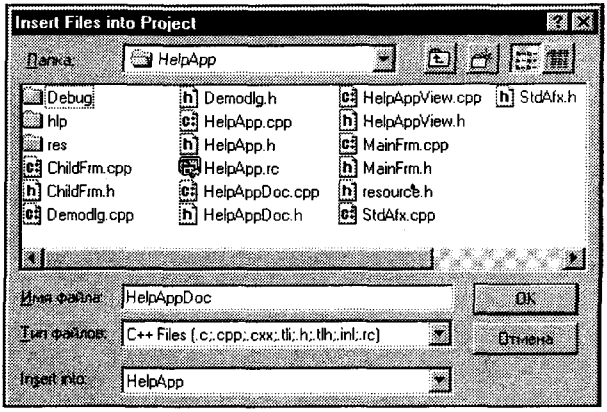


Рис. 3.39. Диалоговое окно **Insert Files into Project**

В данном диалоговом окне нужно выделить все файлы, добавляемые в проект, включая файлы заголовков и файлы реализации, и нажать кнопку **OK**. Чтобы одновременно выделить несколько файлов в окне списка, необходимо при выделении первого файла щелкнуть по его имени левой кнопкой мыши, а для добавления к выделенным файлам нового файла щелкнуть по его имени левой кнопкой мыши, удерживая нажатой клавишу <Ctrl>.

Поэтому, основным применением Component Gallery является хранение классов диалоговых окон и аналогичных им классам. Основным достоинством галереи компонентов является то, что она позволяет сохранять не только сам класс, но и связанные с ним ресурсы. При добавлении класса из галереи компонентов Visual Studio автоматически разрешает все конфликты имен, которые могут при этом возникнуть.

В качестве примера использования Component Gallery рассмотрим класс диалогового окна из приложения DialogApplication.

Чтобы добавить класс диалогового окна в галерею компонентов:

1. Выберите команду **File**, **Open Workspace...** (Файл, Открыть проект). Появится диалоговое окно **Open Workspace**.
2. В окне списка диалогового окна **Open Workspace** раскройте папку \DialogApplication, в которой хранятся файлы данного проекта. Диалоговое окно **Open Workspace** примет вид, изображенный на рис. 3.40.

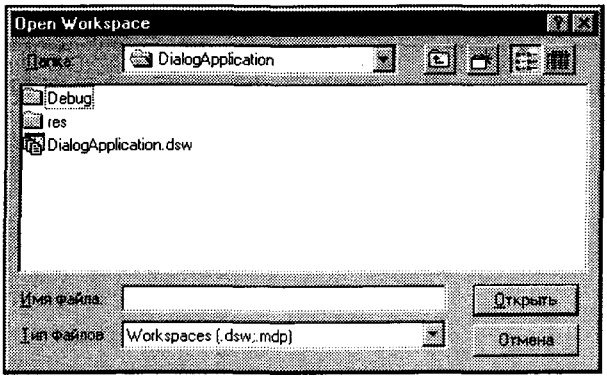


Рис. 3.40. Диалоговое окно **Open Workspace**

3. Дважды щелкните левой кнопкой мыши на значке **DialogApplication.dsw**. Откроется соответствующий проект, содержащий исходные файлы данного приложения.
4. Раскройте вкладку **ClassView** окна **Workspace**, раскройте папку \DialogApplication classes и щелкните правой кнопкой мыши на значке **CDialogApplicationDlg**.
5. В раскрывшемся контекстном меню выберите команду **Add to Gallery** (Добавить к галерее компонентов). Данный класс будет добавлен к галерее компонентов.

Убедитесь, что в результате предпринятых действий описание данного класса действительно было помещено в галерею компонентов, для чего выберите команду **Пуск**, **Программы**, **Проводник** в панели задач и убедитесь, что в каталоге D:\Visual Studio\Common\MSDev98\Gallery располагается папка DialogApplication, содержащая файл DialogApplicationDlg.ogx. При этом окно **Проводник** будет иметь вид, изображенный на рис. 3.41. Если вы установили Visual Studio в другой каталог, то первые каталоги данного пути могут не совпадать.

Чтобы продемонстрировать добавление в проект класса из галереи компонентов:

1. Создайте многооконное приложение с именем Gallery по методике, описанной в главе 1, оставив без изменения все установки параметров, принятые по умолчанию. Полный текст этого приложения можно найти в каталоге \Gallery на дискете, прилагаемой к данной книге.
2. Выберите команду меню **Project**, **Add to Project**, **Components and Controls...** (Проект, Добавить в проект, Компоненты и элементы управления). Появится диалоговое окно **Components and Controls Gallery** (Галерея компонентов и элементов управления), изображенное на рис. 3.42.
3. В окне списка данного диалогового окна раскройте папку \DialogApplication и дважды щелкните на значке **DialogApplicationDlg.ogx**. Появится окно сообщения **Microsoft Visual C++**, изображенное на рис. 3.43.

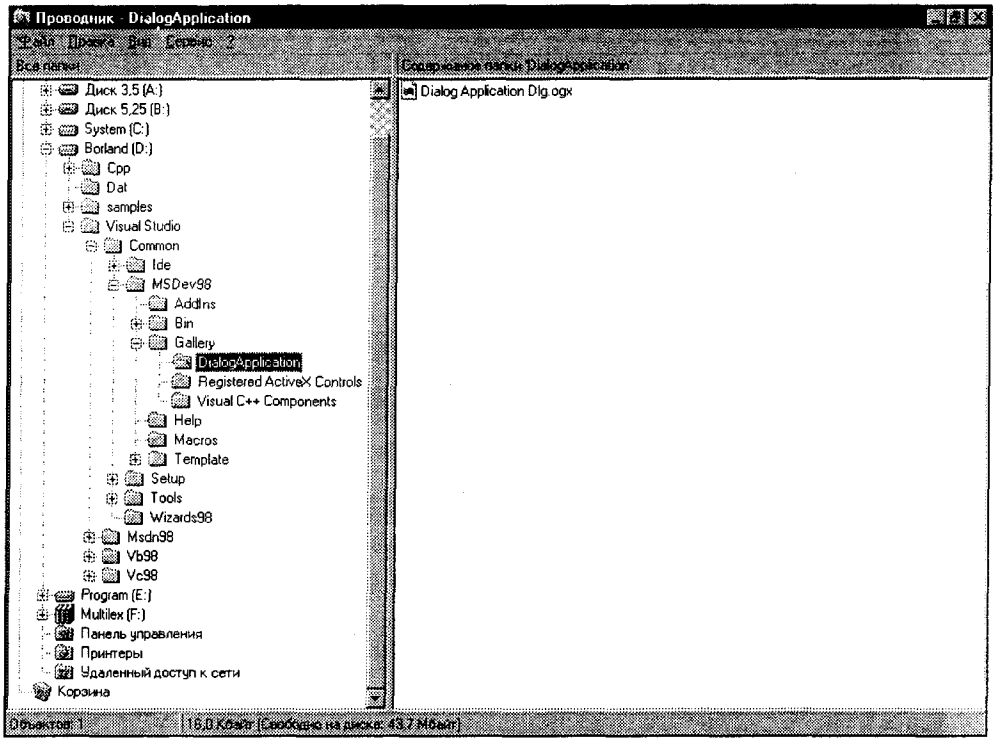


Рис. 3.41. Окно **Проводник**

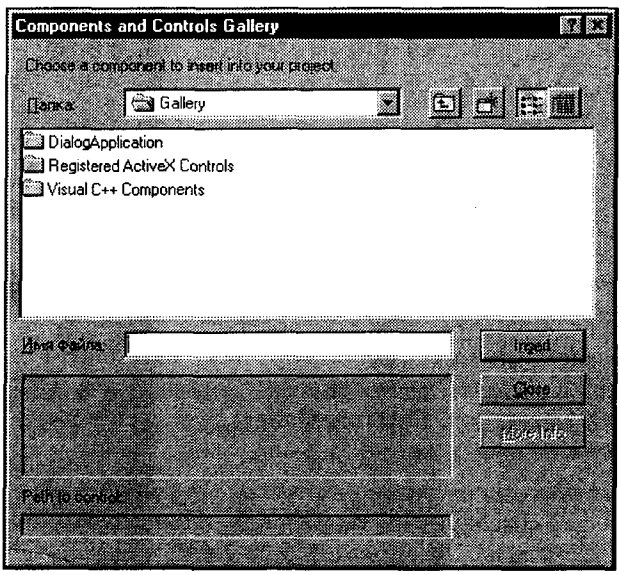


Рис. 3.42. Диалоговое окно **Components and Controls Gallery**

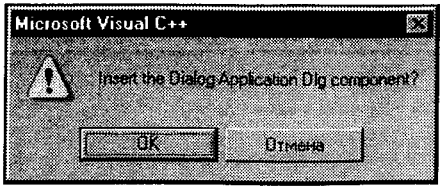


Рис. 3.43. Окно сообщения **Microsoft Visual C++**

4. Нажмите кнопку **OK**, подтвердив необходимость включения в текущий проект компонента DialogApplicationDlg.
5. Нажмите кнопку **Close** и закройте диалоговое окно **Components and Controls Gallery**.
6. Раскройте вкладку **ClassView** окна **Workspace**, как показано на рис. 3.44, и убедитесь, что в приложение включен класс `CDialogApplicationDlg`.
7. Перейдите на вкладку **ResourceView** окна **Workspace**, раскройте папку \Gallery resources, а в ней папку \Dialog и дважды щелкните на значке **IDD_DIALOGAPPLICATION_DIALOG**. В окне редактирования ресурса появится созданное нами диалоговое окно, как это показано на рис. 3.45.

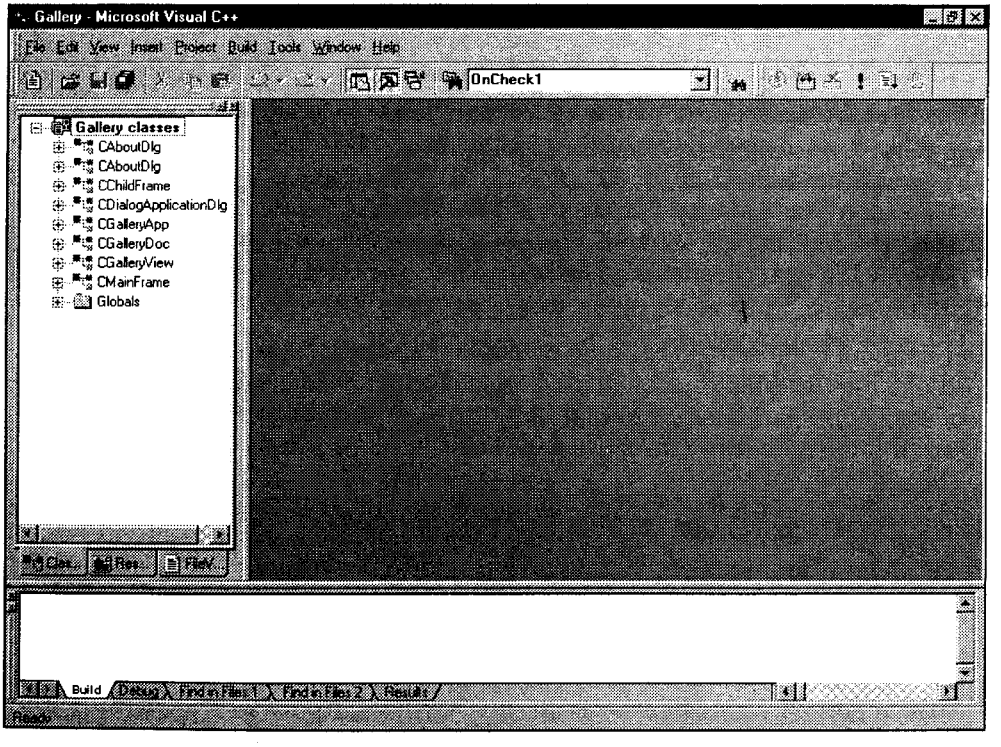


Рис. 3.44. Вкладка **ClassView** окна **Workspace**

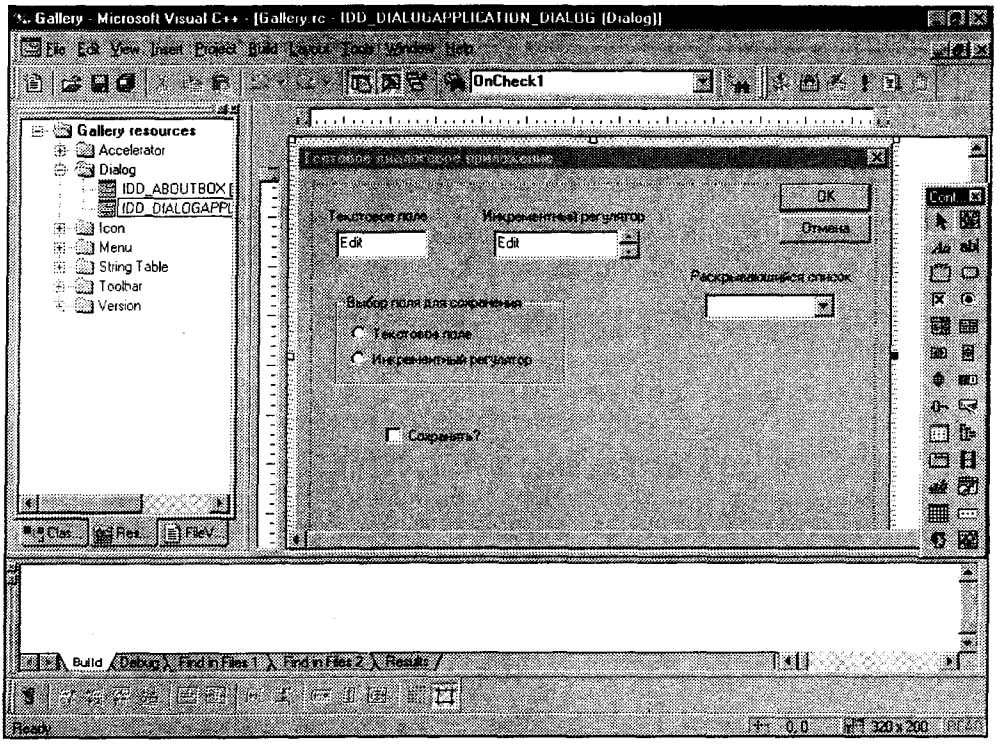


Рис. 3.45. Вкладка **ResourceView** окна **Workspace**

8. Перейдите на вкладку **FileView** окна **Workspace**, раскройте папку \Gallery files, а в ней папки \Source Files и \Header Files и дважды щелкните на значке **DialogApplicationDlg.cpp** в папке \Source Files. Откроется окно соответствующего файла реализации, как это показано на рис. 3.46.

Как видно из приведенных выше рисунков, при добавлении класса диалогового окна из галереи компонентов в приложение добавляются все элементы, необходимые для использования данного окна в приложении. Для использования в приложении Gallery диалогового окна необходимо добавить лишь его вызов.

Чтобы вызвать диалоговое окно в приложении Gallery:

1. Перейдите на вкладку **ResourceView** окна **Workspace**, раскройте папку \Menu и дважды щелкните на значке **IDR_GALLERYTYPE**. В окне редактирования ресурса появится меню, как это показано на рис. 3.47.
2. Поместите указатель мыши на пунктирный прямоугольник, расположенный в правой части панели меню, нажмите левую кнопку мыши. Указатель мыши примет форму вертикальной линии, на концах которой расположены стрелки, направленные вовнутрь.

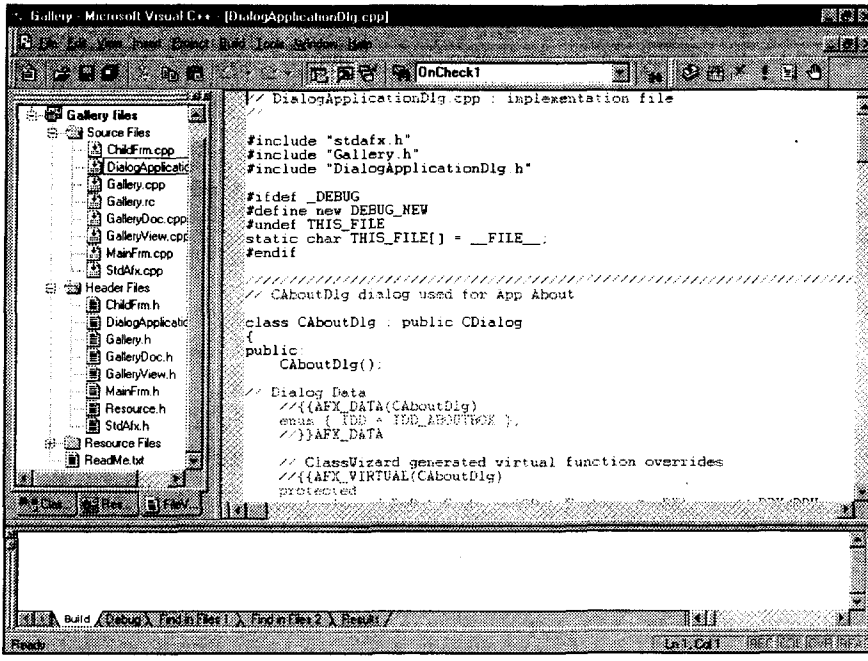


Рис. 3.46. Вкладка **FileView** окна **Workspace**

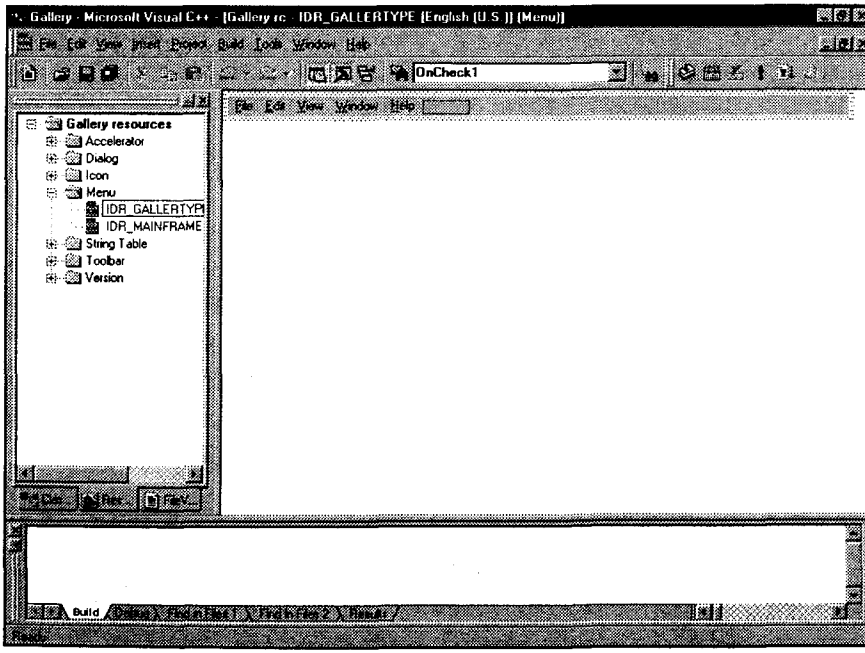


Рис. 3.47. Окно редактирования ресурса меню

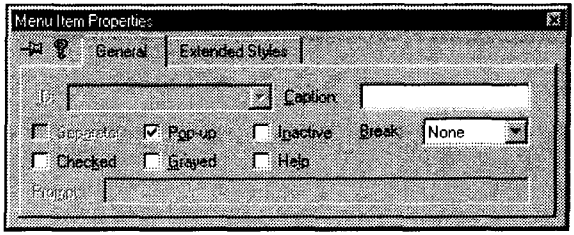


Рис. 3.48. Диалоговое окно **Menu Item Properties**

3. Не отпуская левой кнопки мыши, установите указатель мыши между пунктами меню **View** и **Window**.
4. Отпустите левую кнопку мыши. Пунктирный прямоугольник установится между пунктами меню **View** и **Window**.
5. Щелкните в прямоугольнике правой кнопкой мыши и в появившемся контекстном меню выберите команду **Properties**. Появится диалоговое окно **Menu Item Properties**, изображенное на рис. 3.48.
6. Нажмите кнопку **Keep Visible**, расположенную в правом верхнем углу диалогового окна.
7. Введите в текстовое поле **Caption** заголовок "&Dialog". Под данным пунктом меню раскроется пустое меню, содержащее только пустой пунктирный прямоугольник, как показано на рис. 3.49.

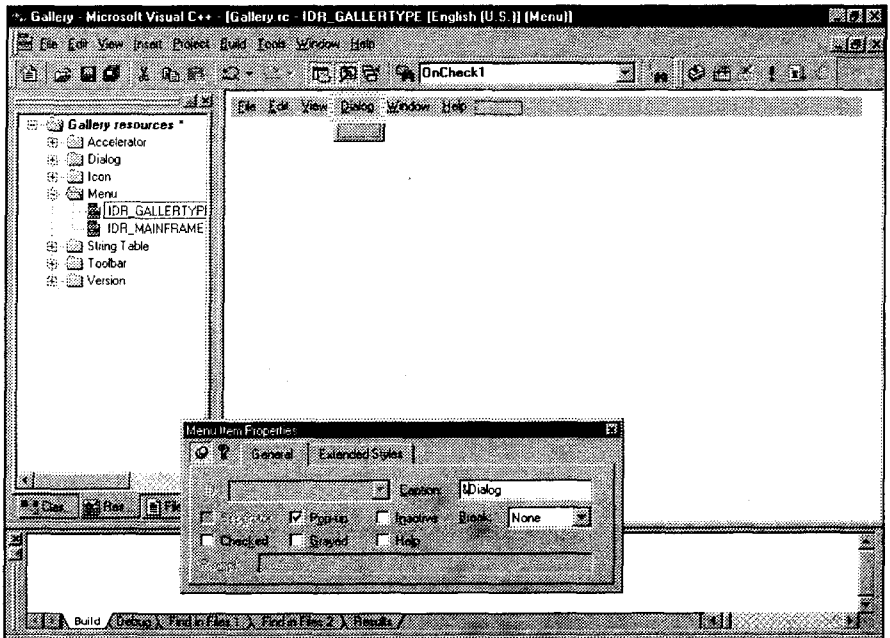


Рис. 3.49. Раскрытое пустое меню

8. Щелкните левой кнопкой мыши на пунктирном прямоугольнике в раскрытом меню. Поля диалогового окна **Menu Item Properties** очистятся.

9. Введите в текстовое поле раскрывающегося списка **ID:** идентификатор ресурса `ID_OPEN`, в текстовое поле **Caption:** — заголовок "&Open", а в текстовое поле **Prompt:** введите справочную информацию по данной команде меню — "Open dialog\nOpen".

10. Закройте диалоговое окно **Menu Item Properties** и вызовите мастер ClassWizard.

11. Раскройте вкладку **Message Maps**, в окне списка **Object IDs:** выделите идентификатор ресурса `ID_OPEN`, в окне списка **Messages:** выделите сообщение `COMMAND` и нажмите кнопку **Add Function**. Появится диалоговое окно **Add Member Function**.

12. Нажмите кнопку **OK**, согласившись с предложенным именем функции. В окне списка **Member functions:** появится новое имя функции `OnOpen`, как показано на рис. 3.50.

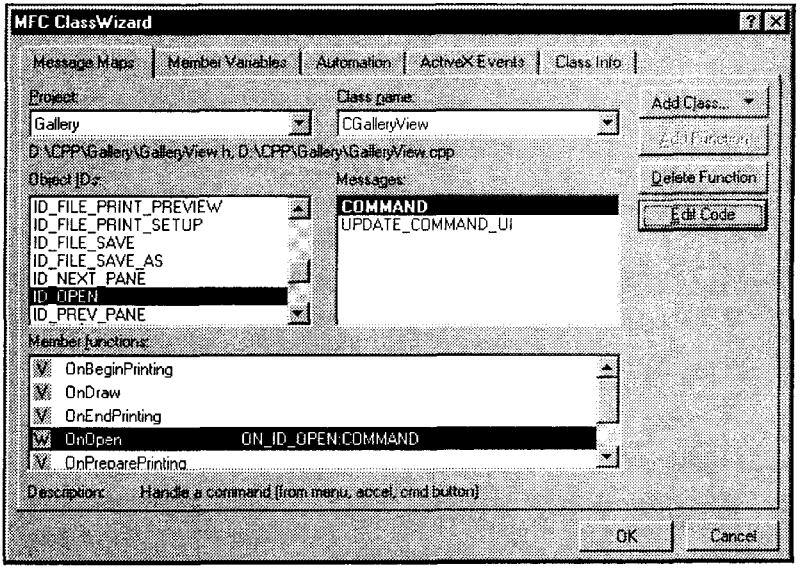


Рис. 3.50. Диалоговое окно **MFC ClassWizard**

13. Нажмите кнопку **Edit Code**. Диалоговое окно **MFC ClassWizard** закроется, откроется окно редактирования соответствующего файла реализации, а текстовый курсор будет помещен в текст функции `OnOpen`.

14. Измените содержимое функции `OnOpen` в соответствии с приведенным ниже текстом:

```
void CGalleryView::OnOpen()
{
  // TODO: Add your command handler code here

  CDialogApplicationDlg  dlg(this);

  dlg.DoModal();
}
```

15. В файле GalleryView.cpp после строки `#include "GalleryView.h"` введите директиву

```
#include "DialogApplicationDlg.h"
```

16. Откройте на редактирование файл DialogApplicationDlg.cpp и уничтожьте в нем текст

```
/////////////////////////////////////////////////////////////////////////
// CAboutDlg dialog used for App About

class CAboutDlg : public CDialog
{
public:
  CAboutDlg();

// Dialog Data
  //{{AFX_DATA(CAboutDlg)
  enum { IDD = IDD_ABOUTBOX };
  //}}AFX_DATA

  // ClassWizard generated virtual function overrides
  //{{AFX_VIRTUAL(CAboutDlg)
  protected:
  virtual void DoDataExchange(CDataExchange* pDX); // DDX/DDV support
  //}}AFX_VIRTUAL

// Implementation
protected:
  //{{AFX_MSG(CAboutDlg)
  //}}AFX_MSG
  DECLARE_MESSAGE_MAP()
};

CAboutDlg::CAboutDlg() : CDialog(CAboutDlg::IDD)
{
  //{{AFX_DATA_INIT(CAboutDlg)
  //}}AFX_DATA_INIT
}
```

```
void CAboutDlg::DoDataExchange(CDataExchange* pDX)
{
  CDialog::DoDataExchange(pDX);
  //{{AFX_DATA_MAP(CAboutDlg)
  //}}AFX_DATA_MAP
}

BEGIN_MESSAGE_MAP(CAboutDlg, CDialog)
  //{{AFX_MSG_MAP(CAboutDlg)
    // No message handlers
  //}}AFX_MSG_MAP
END_MESSAGE_MAP()
```

17. Измените содержимое функции `OnInitDialog` в соответствии с приведенным ниже текстом:

```
BOOL CDialogApplicationDlg::OnInitDialog()
{
  CDialog::OnInitDialog();

  // Set the icon for this dialog.  The framework does this
  // automatically when the application's main window is not a dialog
  SetIcon(m_hIcon, TRUE);      // Set big icon
  SetIcon(m_hIcon, FALSE);    // Set small icon

  // TODO: Add extra initialization here

  return TRUE;  // return TRUE  unless you set the focus to a control
}
```

18. Измените содержимое функции `OnSysCommand` в соответствии с приведенным ниже текстом:

```
void CDialogApplicationDlg::OnSysCommand(UINT nID, LPARAM lParam)
{
    CDialog::OnSysCommand(nID, lParam);
}
```

19. Нажмите клавишу <F5> и запустите приложение на исполнение.
20. В приложении выберите команду **Dialog**, **Open**. Откроется диалоговое окно **Тестовое диалоговое приложение**, как показано на рис. 3.51.

Работа с командами меню будет подробно описана в главе 5, поэтому здесь она останется без комментариев.

См. также главу 5

Включение файла заголовка класса диалогового окна в файл реализации класса приложения обусловлено тем, что в данном классе производится вы-

зов данного диалогового окна. Поскольку мастер ClassWizard не может по имени команды меню определить ее назначение и классы, которые будут использоваться при обработке данной команды, то включение соответствующих файлов заголовков в файлы реализации полностью возлагается на пользователя.

Изменения, внесенные в файл DialogApplicationDlg.cpp, обусловлены тем, что в качестве источника для класса диалогового окна, внесенного в галерею компонентов, использовалось диалоговое приложение. В этом приложении главное окно программы может вызывать диалоговое окно **About** (О программе), содержащее информацию о разработчике данной программы, ее версии и правах собственности на нее. Это окно является атрибутом законченного приложения. Приложение Gallery также имеет подобное окно, класс которого имеет стандартное имя, используемое для этих окон, что приводит к конфликту имен, который, вопреки заверениям фирмы Microsoft, не устраняется галереей компонентов. Для устранения этого конфликта пришлось уничтожить заголовок и реализацию класса диалогового окна `CAboutDlg`, содержавшуюся в файле DialogApplicationDlg.cpp, и все ссылки на него в классе `CDialogApplicationDlg`.

См. также главу 13

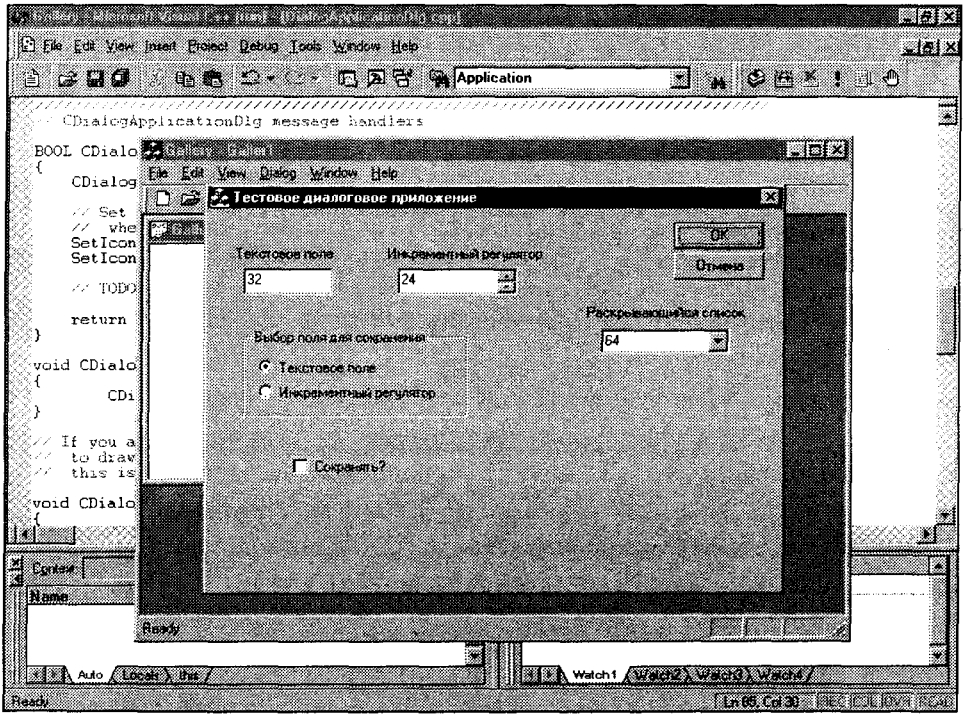


Рис. 3.51. Диалоговое окно **Тестовое диалоговое приложение**

Программирование интерфейса пользователя

4. Классы элементов управления
5. Сообщения и команды
6. Вывод информации на экран
7. Работа с файлами документов
8. Работа с текстовыми документами
9. Панели инструментов и строка состояния
10. Печать документов и организация прокрутки в окне

Классы элементов управления

В главе 3 было описано создание класса диалога и рассмотрены простейшие элементы управления. Эти элементы управления, как правило, представляют собой кнопку или текстовое поле, и работа с ними понятна на интуитивном уровне. Однако существуют элементы управления, реализация которых описывается достаточно сложными классами, каждый из которых нуждается в отдельном описании. Ниже будут рассмотрены основные классы элементов управления.

См. также главу 3

Основным отличием классов элементов управления от всех остальных является то, что объект данного класса может быть сформирован только после того, как соответствующее диалоговое окно будет выведено на экран. Нельзя вызывать функции-члены класса до того, как будет порожден объект класса диалогового окна. Поэтому все операции с объектами данных классов должны выполняться в функциях-членах класса родительского диалогового окна или в вызываемых ими функциях. Это гарантирует, что объект класса диалогового окна будет существовать на момент подобного вызова.

Класс списка

Текст демонстрационного приложения, предназначенного для исследования возможностей класса списка содержится в директории \ListApp на дискете, прилагаемой к данной книге. Чтобы самостоятельно создать это приложение:

1. Создайте заготовку диалогового приложения по методике, описанной в главе 1, и назовите ее "ListApp".
2. Раскройте папку \Dialog во вкладке **ResourceView** окна **Workspace**.

3. Щелкните правой кнопкой мыши на идентификаторе ресурса `IDD_LISTAPP_DIALOG` и выберите команду **Properties** в раскрывшемся при этом контекстном меню. Появится диалоговое окно **Dialog Properties**.
4. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна.
5. Выберите в списке **Language:** опцию **Russian**.
6. Нажмите кнопку **Cancel** в заготовке диалогового окна. Появится диалоговое окно **Push Button Properties**.
7. Измените текст в текстовом поле **Caption:** на "Отмена".
8. Удалите с заготовки диалогового окна статический текст, выберите в панели инструментов **Controls** элемент управления **List Box** (Окно списка) и поместите его в заготовку диалогового окна.
9. Щелкните левой кнопкой мыши, поместив курсор на новое окно списка. Появится диалоговое окно **List Box Properties**.
10. Введите в текстовое поле **ID:** идентификатор ресурса `IDC_LIST`.
11. Поместите над ним статический текст "Список". Окно редактирования ресурса примет вид, изображенный на рис. 4.1.

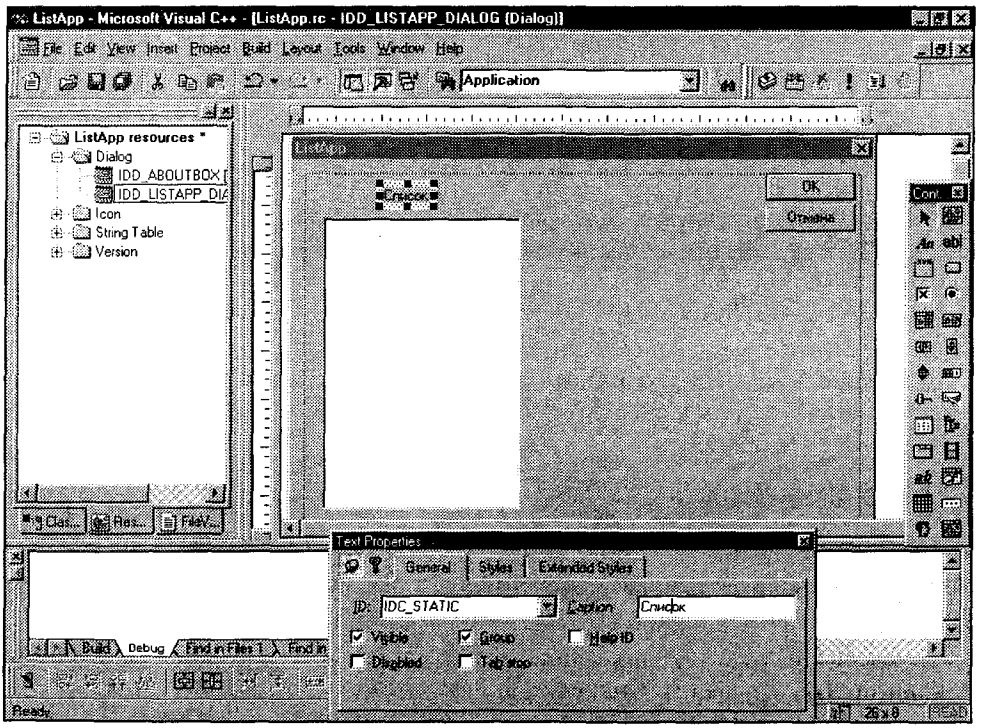


Рис. 4.1. Окно редактирования ресурса с окном списка

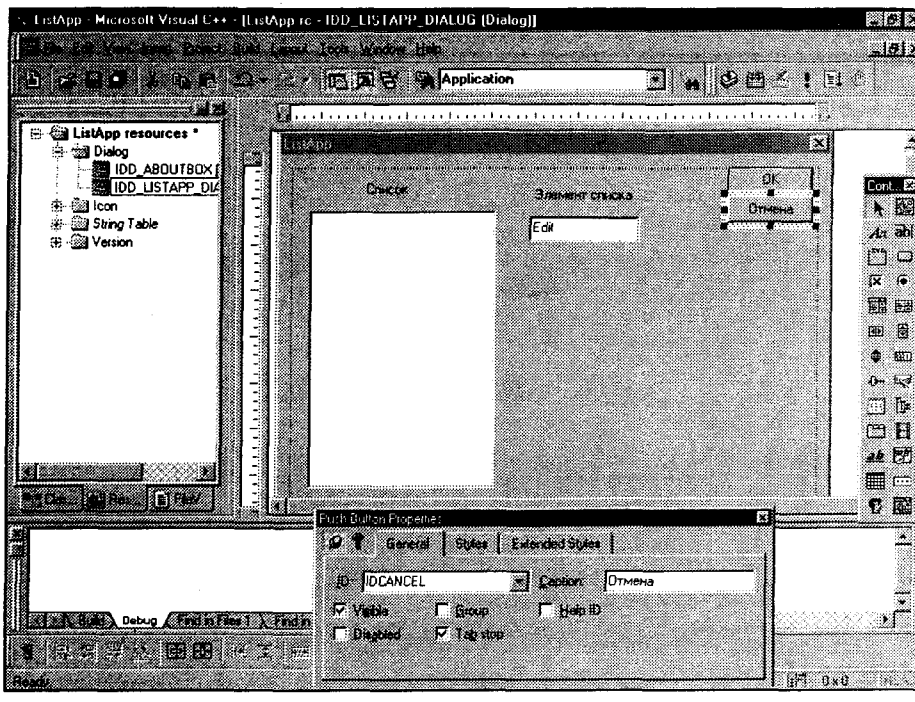


Рис. 4.2. Окно редактирования ресурса с окном списка и текстовым полем

12. Выберите в панели инструментов **Controls** элемент управления **Edit Box** и поместите его в заготовку диалогового окна.
13. Поместите над ним статический текст "Элемент списка". Нажмите кнопку **Отмена**. Окно редактирования ресурса примет вид, изображенный на рис. 4.2.
14. Выберите в панели инструментов **Controls** элемент управления **Button** и поместите его в заготовку диалогового окна.
15. Введите в текстовое поле **ID:** диалогового окна **Push Button Properties** идентификатор ресурса `IDC_ADD`, а в текстовое поле **Caption:** введите заголовок "&Добавить".
16. Повторите операции, описанные в пп. 14 и 15, использовав идентификатор ресурса `IDC_DELETE` и заголовок "&Удалить". В результате этих действий заготовка диалогового окна примет вид, изображенный на рис. 4.3.
17. Закройте диалоговое окно **Push Button Properties**.
18. Не выходя из окна редактирования ресурса диалогового окна, вызовите мастер ClassWizard и раскройте вкладку **Member Variables**. В списке **Control IDs:** выделите идентификатор ресурса `IDC_EDIT1` и нажмите кнопку **Add Variable**. Появится диалоговое окно **Add Member Variable**, как это показано на рис. 4.4.

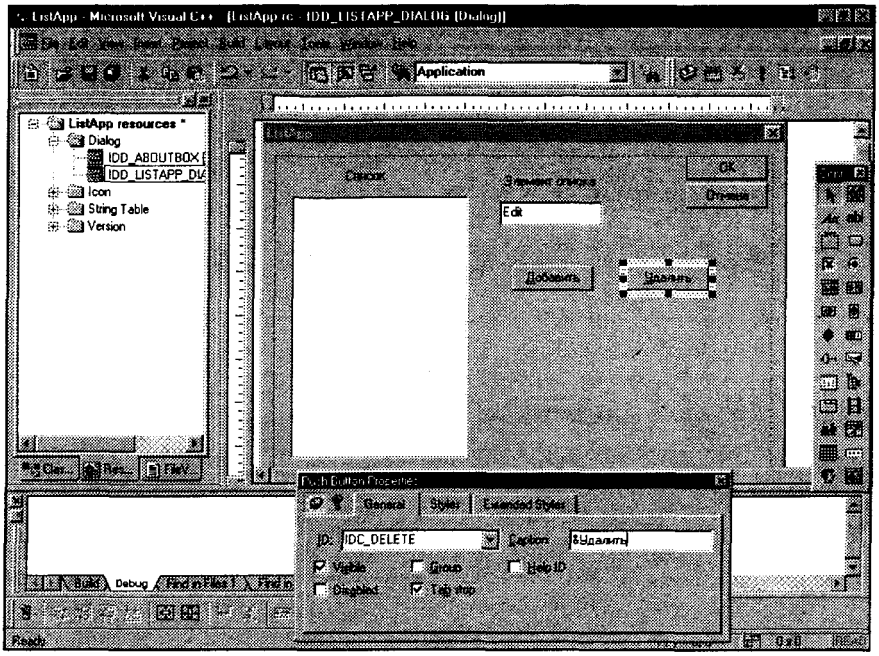


Рис. 4.3. Заготовка диалогового окна с окном списка

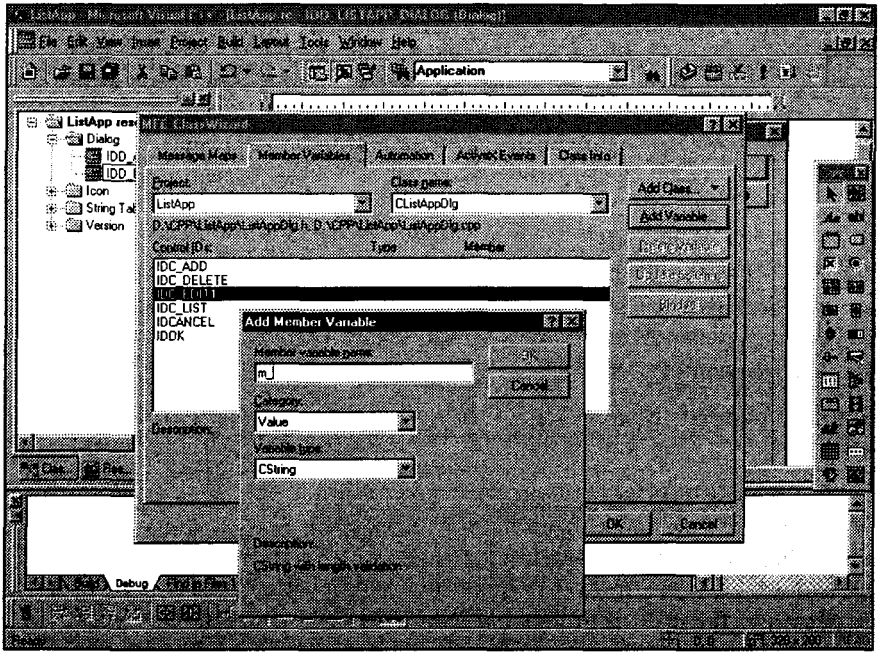


Рис. 4.4. Добавление переменной в класс

19. Введите в текстовое поле **Member variable name:** идентификатор `m_Edit` и нажмите кнопку **OK**. Диалоговое окно **Add Member Variable** закроется, и станет доступным диалоговое окно **MFC ClassWizard**.

20. В списке **Control IDs:** выделите идентификатор ресурса `IDC_LIST` и нажмите кнопку **Add Variable**.

21. В появившемся диалоговом окне **Add Member Variable** введите в текстовое поле **Member variable name:** идентификатор переменной-члена класса `m_List`, а в списке **Category:** (Категория) выберите опцию **Control** (Элемент управления). Диалоговое окно **Add Member Variable** примет вид, изображенный на рис. 4.5.

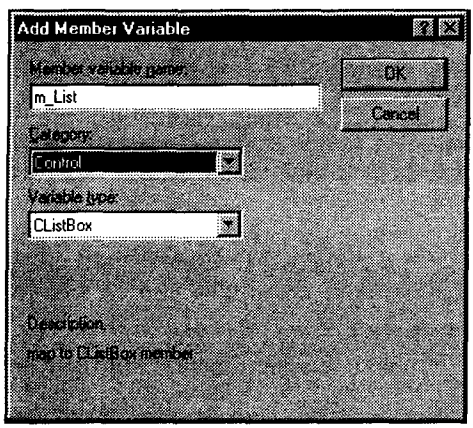


Рис. 4.5. Диалоговое окно **Add Member Variable**

22. Нажмите кнопку **OK** в диалоговом окне **Add Member Variable**.

23. Раскройте вкладку **Message Maps** диалогового окна **MFC ClassWizard**.

24. В окне списка **Object IDs:** выделите идентификатор ресурса `IDC_ADD`, в списке **Messages:** выделите сообщение "BN_CLICKED" и нажмите кнопку **Add Function**. В появившемся диалоговом окне оставьте без изменений предложенное по умолчанию имя функции и нажмите кнопку **OK**. В окне списка **Member functions:** появится имя новой функции `OnAdd`.

25. Проделайте аналогичную операцию для идентификатора ресурса `IDC_DELETE`.

26. В окне списка **Object IDs:** выделите идентификатор ресурса `IDC_LIST`, в списке **Messages:** выделите сообщение "LBN_DBLCLK" и нажмите кнопку **Add Function**. В появившемся диалоговом окне оставьте без изменений предложенное по умолчанию имя функции и нажмите кнопку **OK**. В окне списка **Member functions:** появится имя новой функции `OnDblclkList`, а диалоговое окно **MFC ClassWizard** примет вид, изображенный на рис. 4.6.

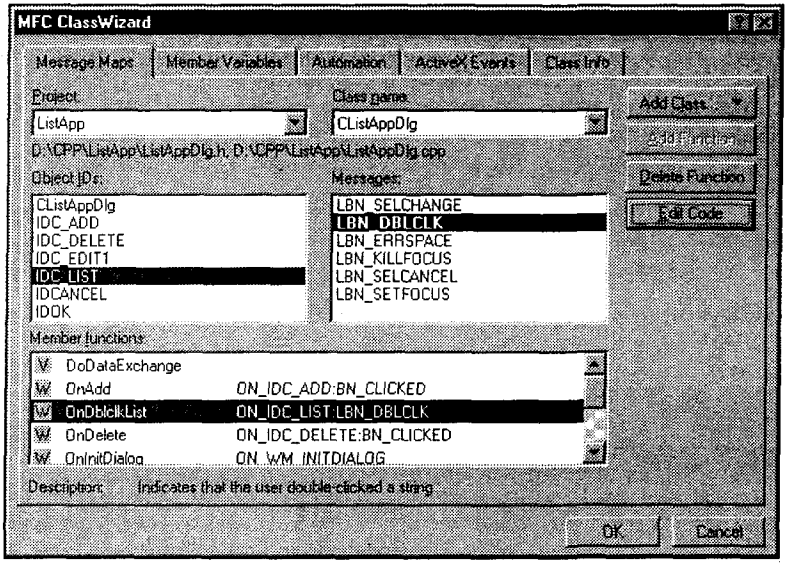


Рис. 4.6. Диалоговое окно **MFC ClassWizard**

27. Нажмите кнопку **Edit Code**. Диалоговое окно **MFC ClassWizard** закроется, откроется окно редактирования файла ListAppDlg.cpp, а текстовый курсор будет установлен в тексте функции `OnDblclkList`.
28. Внесите изменения в файл ListAppDlg.cpp в соответствии с листингом 4.1.

Листинг 4.1. Функции обработки сообщений класса `CListAppDlg`

```
void CListAppDlg::OnAdd()
{
  // TODO: Add your control notification handler code here
  UpdateData();
  m_List.AddString(m_Edit);
}

void CListAppDlg::OnDelete()
{
  // TODO: Add your control notification handler code here
  if( m_List.GetCurSel() != LB_ERR)
    m_List.DeleteString(m_List.GetCurSel());
  UpdateData();
}

void CListAppDlg::OnDblclkList()
{
  // TODO: Add your control notification handler code here
  CString  Temp;
```

```
  if( m_List.GetCurSel() != LB_ERR)
  {
  m_List.GetText(m_List.GetCurSel(), Temp);
  SetDlgItemText(IDC_EDIT1, Temp);
  }
}
```

29. Выберите команду **Build**, **Execute ProgressApp.exe** (Компиляция, Запустить на исполнение файл ProgressApp.exe) и ответьте утвердительно на вопрос о необходимости создания исполняемого файла в появившемся после этого диалоговом окне. Начнется компиляция программы, по завершении которой она будет запущена на исполнение. Появится диалоговое окно, изображенное на рис. 4.7.

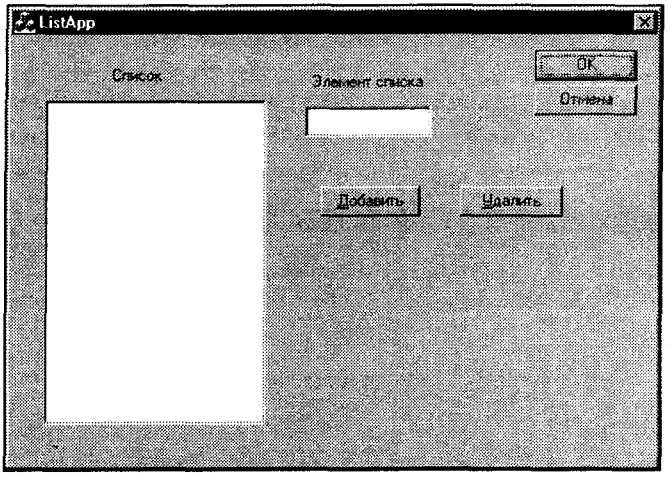


Рис. 4.7. Диалоговое окно списка

30. Введите в текстовое поле **Элемент списка** текст "Первый" и нажмите кнопку **Добавить**. Этот текст появится в списке.

31. Введите в текстовое поле **Элемент списка** текст "Второй" и нажмите кнопку **Добавить**. Этот текст появится в списке на первом месте.

32. Введите в текстовое поле **Элемент списка** текст "Третий" и нажмите кнопку **Добавить**. Этот текст появится в списке на последнем месте.

33. Дело в том, что при создании элемента управления **List Box** во вкладке **Styles** диалогового окна **List Box Properties** был установлен флажок **Sort** (Сортировка), поэтому содержимое списка сортируется по алфавиту.

34. Дважды щелкните левой кнопкой мыши на второй строке списка. В текстовом поле **Элемент списка** появится строка "Первый", как показано на рис. 4.8.

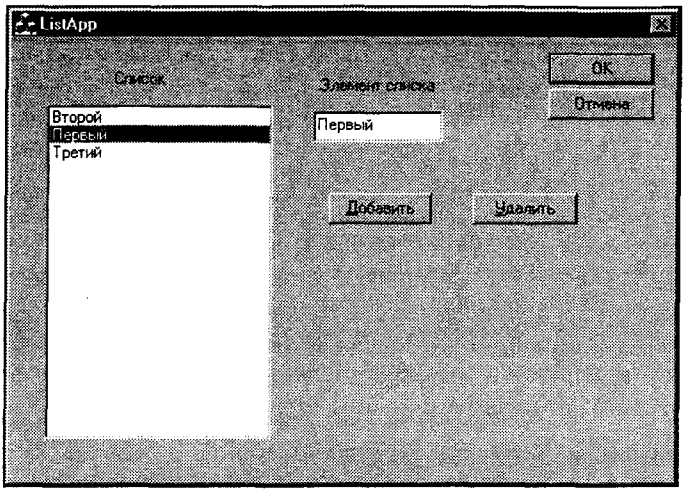


Рис. 4.8. Заполненное диалоговое окно списка

35. Оставив выделенной вторую строку списка, нажмите кнопку **Удалить**. Эта строка будет удалена из списка, но останется в текстовом поле, как показано на рис. 4.9.

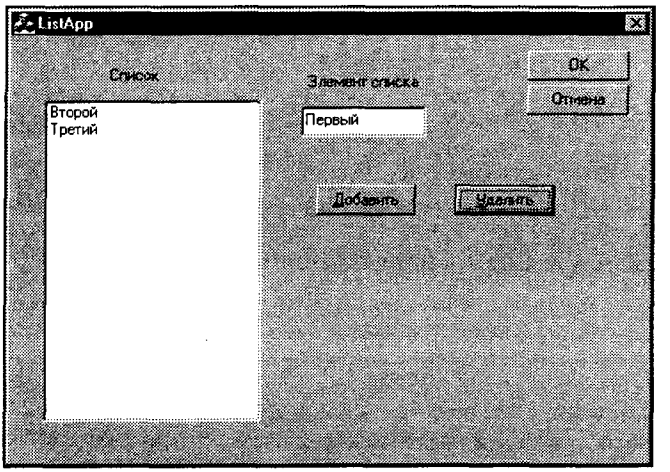


Рис. 4.9. Диалоговое окно списка с удаленным элементом

В данном примере продемонстрировано применение основных функций работы со списком. Полный набор этих функций достаточно обширен и включает в себя, помимо прочих, функции для работы с множественным выделением элементов. Подробное их описание можно найти в справочной системе Visual C++. Здесь мы остановимся только на использованных нами функциях.

Функция `OnAdd` включает в себя вызовы функции `UpdateData`, выполняющей запись содержимого текстового поля в соответствующую переменную, и

функции CListBox::AddString, которая добавляет значение этой переменной в список. Функция OnDelete проверяет наличие в списке выделенной переменной, для чего вызывает функцию CListBox::GetCurSel, возвращающую индекс выделенной строки, и проверяет, не содержит ли возвращаемая величина значение ошибки. Если ошибка не обнаружена, то эта функция вызывается еще раз в качестве аргумента функции CListBox::DeleteString, уничтожающей строку, индекс которой указан ей в качестве аргумента. Функция CListBox::OnDblclkList создает объект класса CString с именем Temp и при помощи функции CListBox::GetText записывает в него содержимое выделенной строки списка. После чего функция SetDlgItemText записывает содержимое переменной Temp в текстовое поле. Если двойной щелчок был произведен на пустой строке списка, и ни один элемент при этом не был выделен, то чтение элемента списка и запись его в текстовое поле не выполняются.

Если необходимо установить или снять выделение элемента списка из программы, можно воспользоваться функцией CListBox::SetSel, первый аргумент которой представляет собой индекс строки, а второй является логической переменной, которой присваивается значение TRUE, если необходимо выделить данную строку списка, и — FALSE, если необходимо снять выделение.

Классы линейного регулятора и линейного индикатора

Текст демонстрационного приложения, предназначенного для исследования возможностей классов линейного регулятора и линейного индикатора, содержится в директории \ProgressApp на дискете, прилагаемой к данной книге. Чтобы самостоятельно создать это приложение:

1. Создайте заготовку диалогового приложения по методике, описанной в главе 1 и назовите ее ProgressApp.
2. Раскройте папку \Dialog во вкладке **ResourceView** окна **Workspace**. Щелкните правой кнопкой мыши на идентификаторе ресурса IDD_PROGRESSAPP_DIALOG и выберите команду **Properties** в раскрывшемся при этом контекстном меню. Появится диалоговое окно **Dialog Properties**.
3. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна.
4. Выберите в списке **Language:** значение **Russian**.
5. Нажмите кнопку **Cancel**. Откроется диалоговое окно **Push Button Properties**.
6. Измените текст в текстовом окне **Caption:** на "Отмена".
7. Удалите с заготовки диалогового окна статический текст.

8. В панели инструментов **Controls** выберите элемент управления **Progress** (Линейный индикатор) и поместите его в заготовку диалогового окна. Измените его горизонтальный размер таким образом, чтобы он занимал большую часть диалогового окна.

   См. также главу 3

9. Щелкните левой кнопкой мыши на элементе управления **Линейный индикатор**. Откроется диалоговое окно **Progress Properties**.
10. Введите в текстовое поле списка **ID:** идентификатор ресурса `IDC_PROGRESS`. Окно редактирования ресурса примет вид, изображенный на рис. 4.10.
11. В панели инструментов **Controls** выберите элемент управления **Slider** (Линейный регулятор) и поместите его в заготовку диалогового окна. Откроется диалоговое окно **Slider Properties**.
12. Введите в текстовое поле списка **ID:** идентификатор ресурса `IDC_SLIDER`.
13. Раскройте вкладку **Styles** (Стили) и установите флажки **Tick marks** (Использовать метки на шкале регулятора) и **Auto ticks** (Устанавливать автоматически).

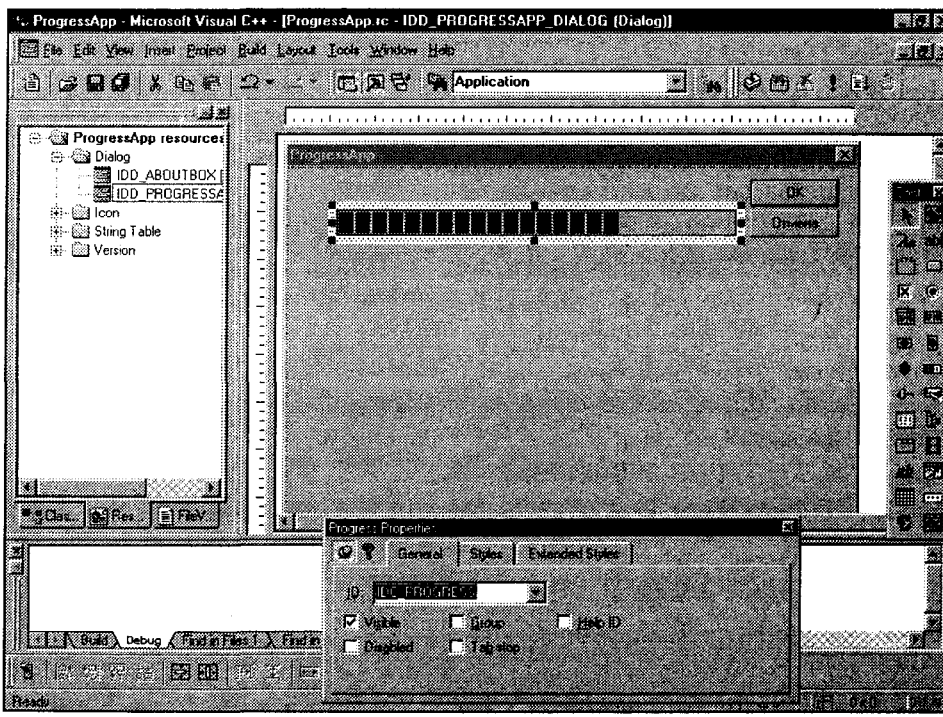


Рис. 4.10. Окно редактирования ресурса с линейным индикатором

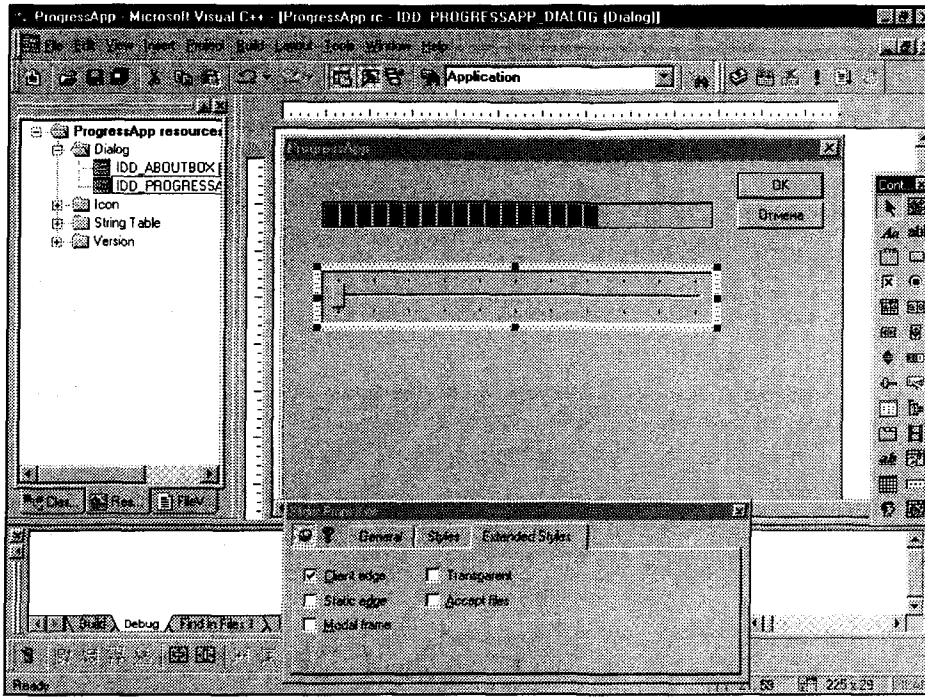


Рис. 4.11. Окно редактирования ресурса с линейным индикатором и линейным регулятором

14. Раскройте вкладку **Extended Styles** (Дополнительные стили) и установите флажок **Client edge** (Рамка).
15. Измените вертикальный размер линейного регулятора таким образом, чтобы метки отображались полностью, а его горизонтальный размер и положение согласуйте с горизонтальным размером и положением линейного индикатора. Окно редактирования ресурса примет вид, изображенный на рис. 4.11.
16. В панели инструментов **Controls** выберите элемент управления **Picture** (Изображение) и поместите его в заготовку диалогового окна. Откроется диалоговое окно **Picture Properties**.
17. Введите в текстовое поле списка **ID:** идентификатор ресурса `IDC_PICTURE`.
18. Раскройте панель **Styles** и установите флажок **Sunken** (Утопленный).
19. Отрегулируйте размеры и положение данного элемента управления в соответствии с рис. 4.12.
20. В панели инструментов **Controls** выберите элемент управления **Check Box** и поместите его в заготовку диалогового окна. Откроется диалоговое окно **Check Box Properties**.

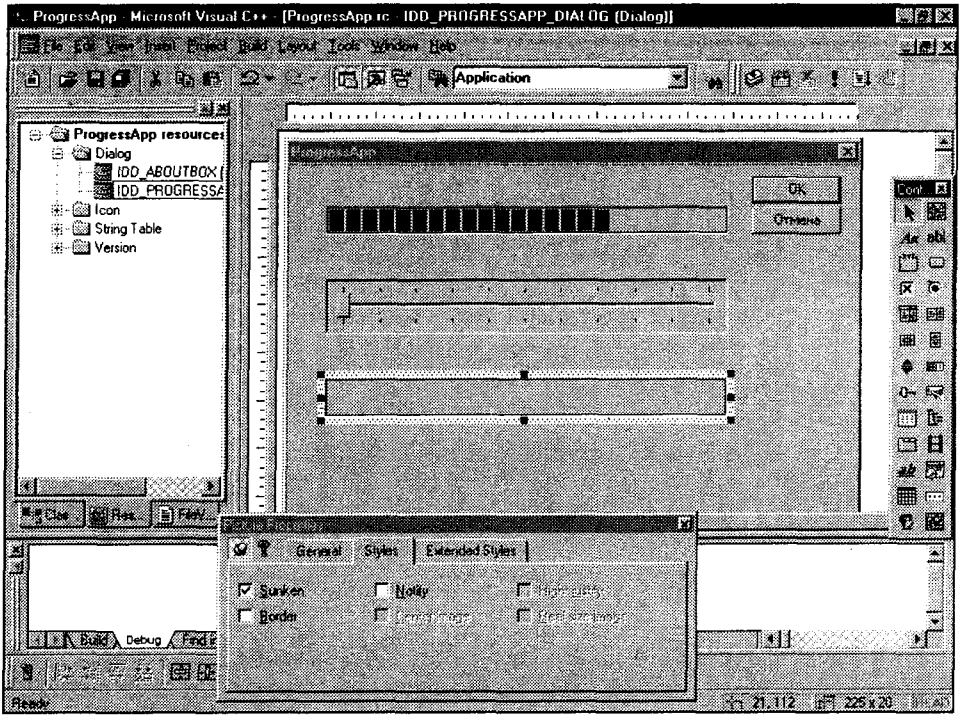


Рис. 4.12. Окно редактирования ресурса

21. Введите в текстовое поле списка **ID:** идентификатор ресурса `IDC_RESET`, а в текстовое поле **Caption:** введите "&Сброс". Заготовка диалогового окна примет вид, изображенный на рис. 4.13.
22. Закройте диалоговое окно **Check Box Properties**.
23. Не выходя из окна редактирования ресурса диалогового окна, вызовите мастер ClassWizard и раскройте в нем вкладку **Member Variables**.
24. В окне **Control IDs:** выделите идентификатор ресурса `IDC_PROGRESS` и нажмите кнопку **Add Variable**.
25. В текстовое поле **Member variable name:** появившегося диалогового окна **Add Member Variable** введите идентификатор переменной `m_Progress` и нажмите кнопку **OK**.
26. Повторите пп. 24 и 25 для идентификатора ресурса `IDC_RESET`, сопоставив ему переменную `m_Reset`.
27. В окне **Control IDs:** выделите идентификатор ресурса `IDC_SLIDER` и нажмите кнопку **Add Variable**.
28. В текстовое поле **Member variable name:** появившегося диалогового окна **Add Member Variable** введите идентификатор переменной `m_Slider`, а в

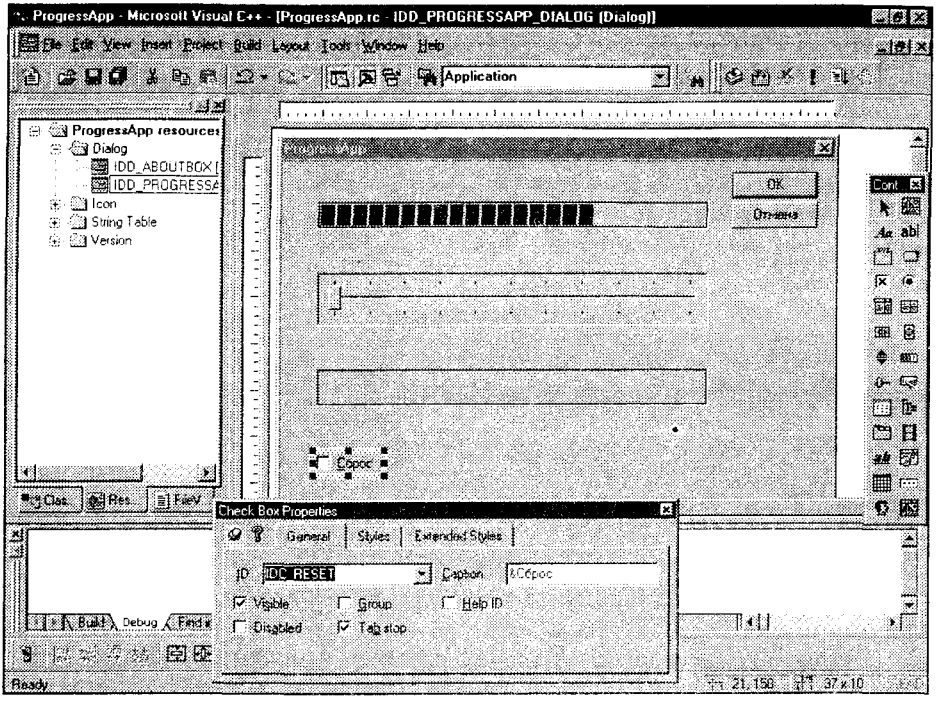


Рис. 4.13. Заготовка диалогового окна

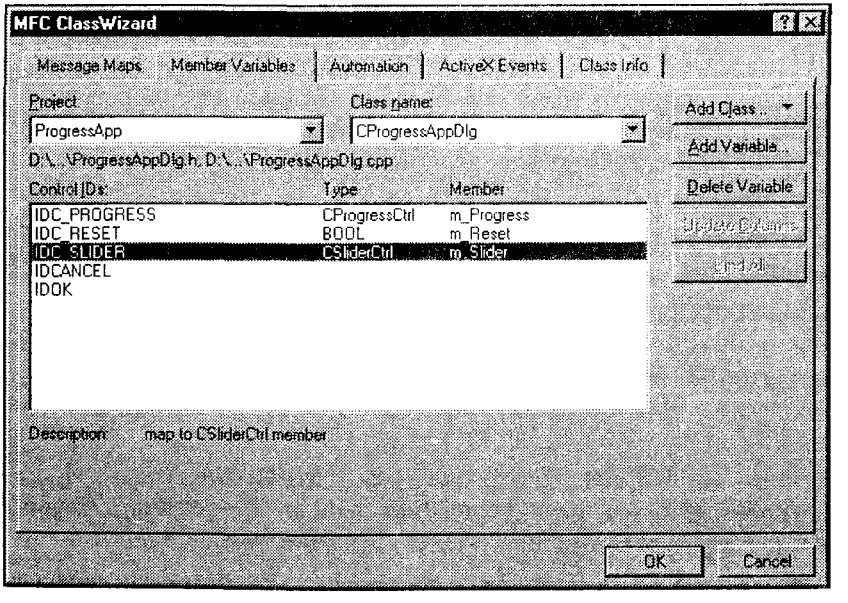


Рис. 4.14. Вкладка **Member Variables** диалогового окна **MFC ClassWizard**

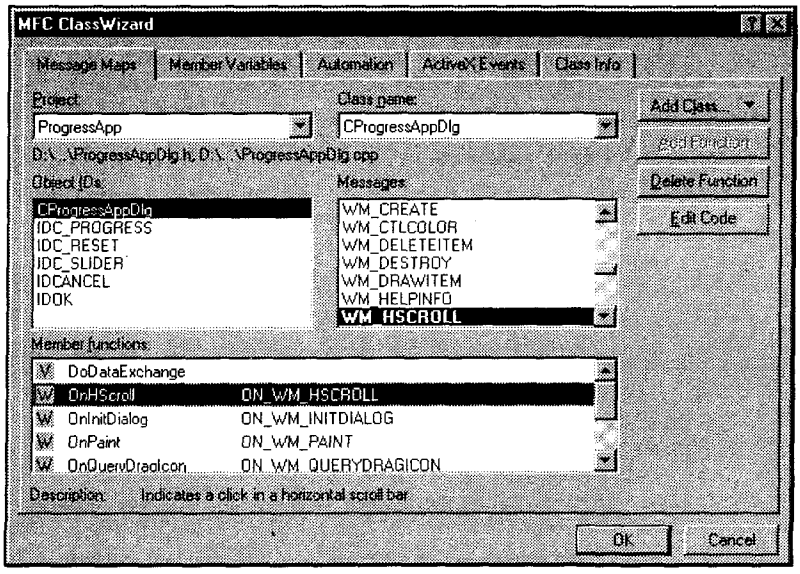


Рис. 4.15. Вкладка **Message Maps** диалогового окна **MFC ClassWizard**

списке **Category:** выберите значение **Control** и нажмите кнопку **OK**. Вкладка **Member Variables** диалогового окна **MFC ClassWizard** примет вид, изображенный на рис. 4.14.

29. Раскройте вкладку **Message Maps**. В окне списка **Object IDs:** выделите идентификатор ресурса `IDC_RESET`, в окне списка **Messages:** выделите сообщение "BN_CLICKED" и нажмите кнопку **Add Function**.

30. В появившемся диалоговом окне **Add Member Function** оставьте без изменений предложенное по умолчанию имя функции и нажмите кнопку **OK**. В окне списка **Member functions:** появится имя новой функции `OnReset`.

31. В окне списка **Object IDs:** выделите имя класса `CProgressAppDlg`, в окне списка **Messages:** выделите сообщение "WM_HSCROLL" и нажмите кнопку **Add Function**.

32. В появившемся диалоговом окне **Add Member Function** оставьте без изменений предложенное по умолчанию имя функции и нажмите кнопку **OK**. В окне списка **Member functions:** появится имя новой функции `OnHScroll`. Вкладка **Message Maps** диалогового окна **MFC ClassWizard** примет вид, изображенный на рис. 4.15.

33. Нажмите кнопку **Edit Code**. Диалоговое окно **MFC ClassWizard** будет закрыто, откроется окно редактирования файла ProgressAppDlg.cpp, и текстовый курсор будет помещен в текст функции `OnHScroll`.

34. Внесите изменения в функции обработки сообщений класса `CProgressAppDlg` в соответствии с листингом 4.2.

Листинг 4.2. Функции обработки сообщения класса CProgressAppDlg

```
////////////////////////////////////////////////////////////////////////////
// CProgressAppDlg message handlers

BOOL CProgressAppDlg::OnInitDialog()
{
  CDialog::OnInitDialog();

  // Add "About..." menu item to system menu.

  // IDM_ABOUTBOX must be in the system command range.
  ASSERT((IDM_ABOUTBOX & 0xFFF0) == IDM_ABOUTBOX);
  ASSERT(IDM_ABOUTBOX < 0xF000);

  CMenu* pSysMenu = GetSystemMenu(FALSE);
  if (pSysMenu != NULL)
  {
    CString strAboutMenu;
    strAboutMenu.LoadString(IDS_ABOUTBOX);
    if (!strAboutMenu.IsEmpty())
    {
      pSysMenu->AppendMenu(MF_SEPARATOR);
      pSysMenu->AppendMenu(MF_STRING, IDM_ABOUTBOX, strAboutMenu);
    }
  }

  // Set the icon for this dialog.  The framework does this automatically
  //  when the application's main window is not a dialog
  SetIcon(m_hIcon, TRUE);       // Set big icon
  SetIcon(m_hIcon, FALSE);      // Set small icon

  // TODO: Add extra initialization here

  m_Slider.SetRange(0, 1024, TRUE);
  m_Slider.SetTicFreq( 128);
  m_Slider.SetLineSize( 32);
  m_Slider.SetPageSize( 256);
  m_Progress.SetRange(0, 1024);

  return TRUE;  // return TRUE  unless you set the focus to a control
}

void CProgressAppDlg::OnReset()
{
  // TODO: Add your control notification handler code here
  UpdateData();
  if(m_Reset)
```

```
    m_Progress.SetPos( 0);
}

void CProgressAppDlg::OnHScroll(UINT nSBCode, UINT nPos,
                                CScrollBar* pScrollBar)
{
  // TODO: Add your message handler code here and/or call default
  CSliderCtrl*  slider = (CSliderCtrl*) pScrollBar;

  if( !m_Reset && (m_Progress.GetPos() < slider-> GetPos()))
    m_Progress.SetPos( slider-> GetPos());
  CDialog::OnHScroll(nSBCode, nPos, pScrollBar);
}
```

35. Нажмите клавишу <F5> и запустите приложение. Появится диалоговое окно, изображенное на рис. 4.16.

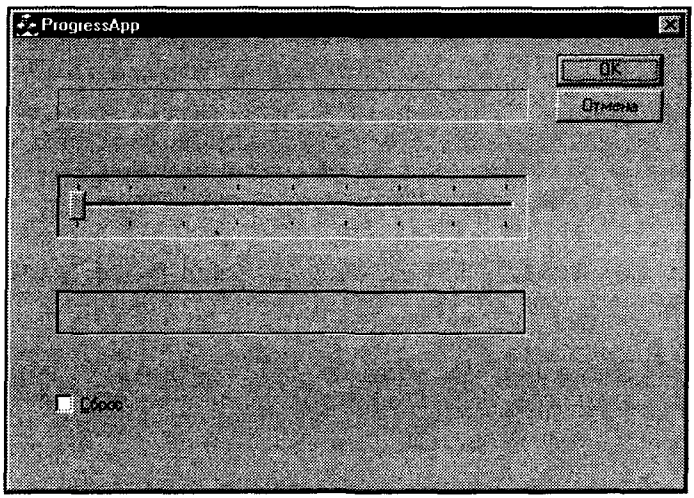


Рис. 4.16. Диалоговое окно приложения

36. Установите указатель мыши на бегунок линейного регулятора, нажмите левую кнопку мыши и, удерживая ее, переместите бегунок вправо. Одновременно с перемещением бегунка вправо начнет заполняться линейный индикатор.
37. Переместите бегунок влево. Линейный индикатор останется в прежнем положении, как это показано на рис. 4.17.
38. Установите флажок **Сброс**. Линейный регулятор очистится, и перестанет реагировать на перемещения бегунка линейного регулятора, как показано на рис. 4.18.
39. Сбросьте флажок и переместите бегунок линейного регулятора. Линейный индикатор отобразит крайнее правое положение линейного регулятора.

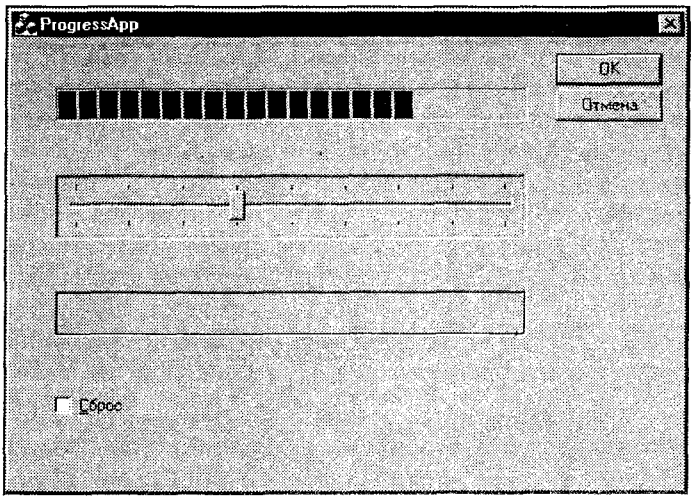


Рис. 4.17. Диалоговое окно приложения в работе

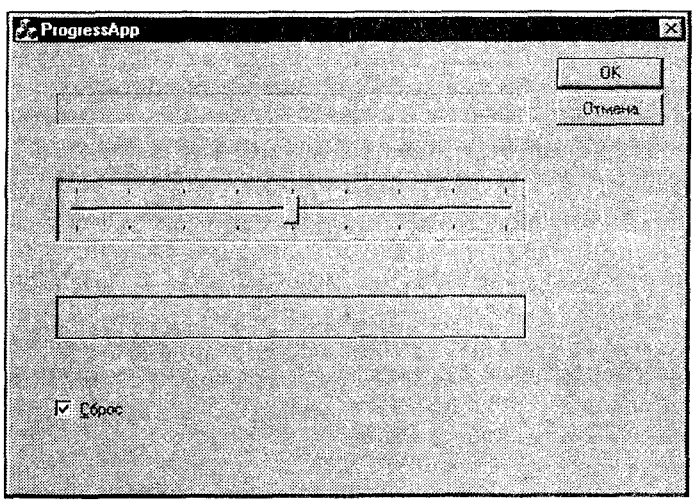


Рис. 4.18. Установленный флажок **Сброс**

40. Нажмите на клавиши управления курсором. После того как фокус ввода, пройдя другие элементы управления, установится на инкрементном регуляторе, он начнет управляться этими клавишами. При этом стрелка вправо и стрелка вниз будут передвигать движок линейного регулятора вправо, а стрелка влево и стрелка вверх — влево. Нажатие клавиши <Home> установит линейный регулятор в крайнюю левую позицию, а нажатие клавиши <End> — в крайнюю правую. Клавиша <PgUp> предназначена для ускоренного перемещения регулятора влево, а клавиша <PgDn> — вправо.

Чтобы добиться подобного эффекта при управлении с клавиатуры, в функции `OnInitDialog` вызываются функции настройки параметров линейного

регулятора. Функция `CSliderCtrl::SetRange` имеет три аргумента, первый из которых определяет нижнюю границу диапазона регулирования, второй — верхнюю границу этого диапазона, а третий — необходимость отрисовки элемента управления после установки заданных параметров. Функция `CSliderCtrl::SetTicFreq` устанавливает расстояние, измеряемое в тех же единицах, что и диапазон регулирования, через который будут устанавливаться метки на шкале регулятора. Функция `CSliderCtrl::SetLineSize` устанавливает перемещение бегунка линейного регулятора при нажатии клавиш управления курсором. Функция `CSliderCtrl::SetPageSize` устанавливает перемещение бегунка линейного регулятора при нажатии клавиш <PgUp> и <PgDn>. Функция `CProgressCtrl::SetRange` действует аналогично подобной функции в классе `CSliderCtrl` и используется для синхронизации диапазонов объектов этих двух классов.

Линейный регулятор представляет собой одну из разновидностей полосы прокрутки. При перемещении бегунка линейный регулятор формирует сообщение `WM_HSCROLL`, которое перехватывается функцией-членом класса диалогового окна `OnHScroll`. Первый аргумент этой функции содержит код операции, выполняемой с полосой прокрутки. Полный перечень этих кодов и более полное описание методики работы с полосой прокрутки содержится в главе 10. Второй аргумент этой функции содержит положение бегунка, однако в данном случае он не используется. Дело в том, что этот аргумент используется только в двух случаях: для отслеживания перемещения бегунка мышью и для установления бегунка в определенную позицию. При работе с клавиатурой содержимое данного аргумента игнорируется. Третьим аргументом данной функции является указатель на объект класса `CScrollBar`. Именно этот указатель и будет использоваться для получения текущей позиции бегунка. Прежде всего, мы преобразуем указатель на объект класса `CScrollBar` в указатель на объект класса `CSliderCtrl`. Что мы, строго говоря, не имеем права делать, поскольку родительским классом для `CSliderCtrl` является класс `CWnd`, а не `CScrollBar`, но в Windows возможно и не такое. После получения указателя на объект класса мы используем функцию `CSliderCtrl::GetPos` для получения текущей позиции бегунка. Аналогичная функция вызывается и для класса `CProgressCtrl`. Эта функция возвращает текущее положение линейного индикатора. Если текущее положение линейного регулятора превышает по величине текущее положение линейного индикатора (бегунок линейного регулятора расположен правее текущей позиции линейного индикатора) и не установлен флажок **Сброс**, то вызывается функция `CProgressCtrl::SetPos`, устанавливающая положение линейного индикатора, а в качестве ее аргумента используется текущая позиция линейного регулятора.

См. также главу 10

Функция `OnReset` служит для обеспечения немедленной реакции на установку или снятие флажка **Сброс**. Первым оператором данной функции является

вызов функции `UpdateData`, считывающей текущее состояние флажка. Если флажок установлен, то вызовом функции `CProgressCtrl::SetPos` с нулевым значением аргумента линейный индикатор сбрасывается в исходное положение.

Создание пользовательского линейного индикатора

Вы, наверное, уже заметили, что в данном приложении существует элемент управления, который до сих пор не принимал никакого участия в его работе. Это элемент управления Picture. Дело в том, что точность линейного индикатора, создаваемого мастером ClassWizard, невелика, и, как вы уже успели убедиться, составляет "плюс-минус кирпич".

Если данный индикатор используется для отображения процесса загрузки файла с диска, то нет ничего страшного в том, что индикатор покажет начало процесса только после того, как значительная часть его будет уже завершена. Однако представьте себе, что весь процесс занимает несколько часов, и только через десять минут после его начала вы получаете возможность убедиться, что он действительно начался. Или, что еще хуже, вы испытываете новый алгоритм и не знаете, сколько времени займет его реализация. Знаете только, что достаточно много, и ждете первых признаков его начала.

В этом случае вам хотелось бы получить подтверждение о начале работы, выражающееся в начале заполнения линейного индикатора, как можно раньше. При использовании соответствующих стилей в функции `CProgressCtrl::Create` может быть создан линейный индикатор, не имеющий разбиения полосы на блоки, но мастер ClassWizard не предоставляет пользователю непосредственный доступ к этой функции при создании объекта класса данного элемента управления.

Ниже будет представлен способ самостоятельного формирования линейного индикатора с использованием элемента управления Picture. Этот простой пример поможет понять основные принципы работы с данным элементом управления.

Чтобы самостоятельно создать линейный индикатор, внесите в созданный ранее проект следующие изменения:

1. Откройте окно редактирования файла ProgressAppDlg.cpp.
2. Добавьте перед описанием класса `CAboutDlg` после оператора условной трансляции `#endif` строку

```
CRect  P_Rect;
```

3. Внесите изменения в соответствующий фрагмент файла реализации ProgressAppDlg.cpp в соответствии с листингом 4.3.

Листинг 4.3. Функции, реализующие пользовательский линейный индикатор

```
// If you add a minimize button to your dialog, you will need the code
// below to draw the icon.  For MFC applications using
// the document/view model, this is automatically done for you by
// the framework.

void CProgressAppDlg::OnPaint()
{
  if (IsIconic())
  {
    CPaintDC dc(this); // device context for painting

    SendMessage(WM_ICONERASEBKGND, (WPARAM) dc.GetSafeHdc(), 0);

    // Center icon in client rectangle
    int cxIcon = GetSystemMetrics(SM_CXICON);
    int cyIcon = GetSystemMetrics(SM_CYICON);
    CRect rect;
    GetClientRect(&rect);
    int x = (rect.Width() - cxIcon + 1) / 2;
    int y = (rect.Height() - cyIcon + 1) / 2;

    // Draw the icon
    dc.DrawIcon(x, y, m_hIcon);
  }
  else
  {
    CDialog::OnPaint();
  }

  CClientDC  clientDC(this);
  CBrush    brBack( RGB(255, 255, 255));

  ::GetWindowRect( ::GetDlgItem( m_hWnd, IDC_PICTURE), &P_Rect);
  ScreenToClient( &P_Rect);

  P_Rect.left   += 2;
  P_Rect.right  -= 2;
  P_Rect.top    += 2;
  P_Rect.bottom -= 2;

  clientDC.FillRect( P_Rect, &brBack);
}

void CProgressAppDlg::OnReset()
{
  // TODO: Add your control notification handler code here
  UpdateData();
```

```
  if(m_Reset){
    m_Progress.SetPos( 0);

    CClientDC clientDC(this);
    CRgn      pRgn;
    CBrush    brBack( RGB(255, 255, 255));

    pRgn.CreateRectRgnIndirect( P_Rect);
    clientDC.SelectClipRgn( &pRgn);
    clientDC.FillRect( P_Rect, &brBack);
  }
}

void CProgressAppDlg::OnHScroll(UINT nSBCode, UINT nPos,
                                CScrollBar* pScrollBar)
{
  // TODO: Add your message handler code here and/or call default
  CSliderCtrl*  slider = (CSliderCtrl*) pScrollBar;

  if( !m_Reset && (m_Progress.GetPos() < slider-> GetPos())){
    CClientDC  clientDC(this);
    CRgn    pRgn;
    CBrush  brDone( RGB(255, 0, 0));
    CRect   D_Rect = P_Rect;

    pRgn.CreateRectRgnIndirect( P_Rect);
    clientDC.SelectClipRgn( &pRgn);

    D_Rect.right = P_Rect.left + (int)(P_Rect.Width()*
                                slider-> GetPos())/slider->GetRangeMax();
    clientDC.FillRect( D_Rect, &brDone);

    m_Progress.SetPos( slider-> GetPos());
  }

  CDialog::OnHScroll(nSBCode, nPos, pScrollBar);
}
```

4. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится диалоговое окно, изображенное на рис. 4.19.

   В данном окне пользовательский линейный индикатор имеет белый фон.

5. Переместите бегунок линейного регулятора вправо. Диалоговое окно примет вид, изображенный на рис. 4.20.

В приложение были введены элементы интерфейса графических устройств Windows, подробно описанного в главе 6. Поэтому основное внимание будет сосредоточено на элементе управления Picture и способах работы с ним.

См. также главу 6

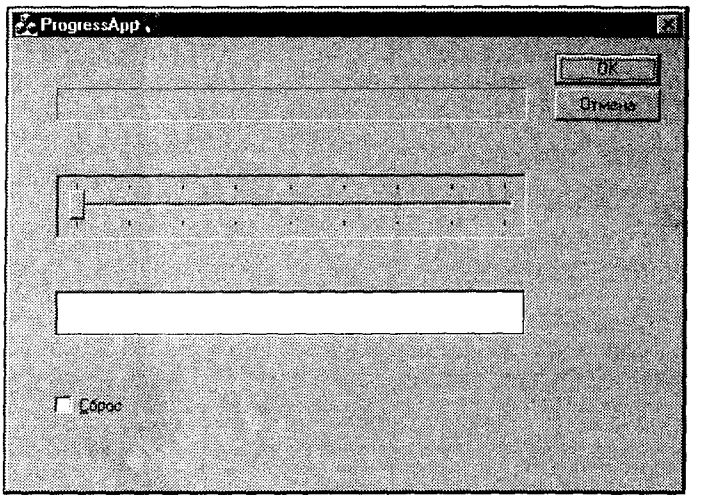


Рис. 4.19. Диалоговое окно с пользовательским линейным индикатором

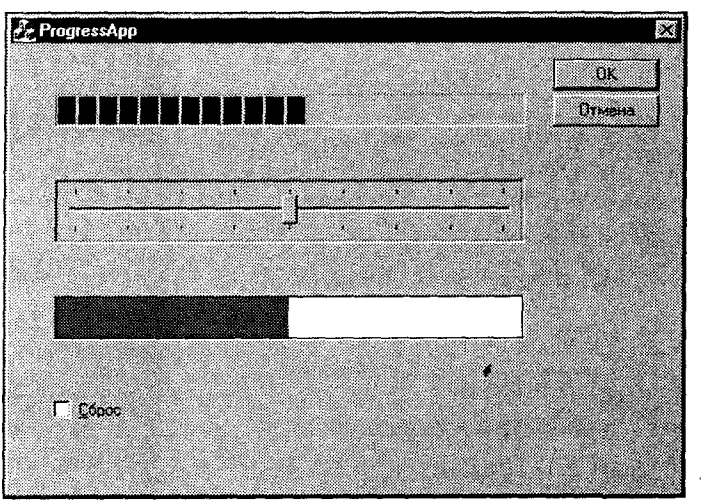


Рис. 4.20. Диалоговое окно с пользовательским линейным индикатором в работе

Как следует из текста программы, для работы с пользовательским линейным индикатором была введена всего одна глобальная переменная. Ее можно было бы оформить переменной-членом класса, но это потребовало бы внесения изменений также и в файл заголовка. Нельзя объявлять объекты класса `CBrush` как глобальные переменные или переменные — члены класса, поскольку все ресурсы должны создаваться непосредственно перед их использованием и освобождаться сразу, как только в них отпала необходимость.

Первое изменение было внесено в функцию `OnPaint`. Приложение вызывает эту функцию всякий раз, как только операционная система Windows или само приложение выдают заявку на перерисовку фрагмента окна приложения. Эта функция является обработчиком сообщения `WM_PAINT`. Если окно

не минимизировано, то данная функция вызывает функцию-член родительского класса `CDialog::OnPaint`, после чего создается контекст устройства и кисть для рисования фона пользовательского элемента управления. Затем следует вызов глобальной функции `GetWindowRect`, позволяющей получить экранные координаты окна элемента управления. Для получения указателя на объект окна элемента управления используется глобальная функция `GetDlgItem`, в качестве первого аргумента которой выступает дескриптор диалогового окна, являющийся членом класса `CWnd`, а второго аргумента — идентификатор ресурса элемента управления.

Поскольку в результате вызова функции `GetWindowRect` были получены экранные координаты окна, то следующим оператором является вызов функции `CWnd::ScreenToClient`, преобразующей экранные координаты в координаты рабочей области окна. В качестве аргумента этой функции могут выступать указатели на объекты структур `POINT` и `RECT`, причем входная и выходная величины располагаются в том же самом объекте.

Полученные координаты относятся ко всему элементу управления и соответствуют внешним его границам, но при его создании в окне редактирования ресурсов в панели **Styles** диалогового окна **Picture Properties** был установлен флажок **Sunken**. В результате данный элемент управления имеет рамку, имитирующую его заглубление в поверхность диалогового окна. Ширина рамки равна двум элементам изображения с каждой стороны рамки. Поэтому для корректного вывода информации необходимо внести изменения в координаты окна, чтобы они соответствовали его рабочей области.

Последним оператором функции `OnPaint` является вызов функции `CDC::FillRect`, осуществляющей заливку области, ограниченной прямоугольником, заданным в ее первом аргументе указателем на объект структуры `RECT`, цветом, определяемым в ее втором аргументе указателем на класс `CBrush`. Эта функция заполняет всю рабочую область элемента управления цветом фона. В качестве цвета фона в конструкторе соответствующей кисти установлен белый цвет (яркости всех основных цветов присваивается максимальное значение).

В функции `OnReset`, кроме всего прочего, производится заливка цветом фона всей рабочей области окна пользовательского элемента управления. Эта операция реализуется тем же набором операторов, что и в функции `OnPaint`, но в эту последовательность добавлены дополнительные операторы. Прежде всего, в данной функции создается объект класса `CRgn`. Данный класс служит для задания эллиптической или многоугольной области для вывода изображения. Функция `CRgn::CreateRectRgnIndirect` позволяет создать прямоугольную область вывода на экран и сохранить ее в данном объекте класса `CRgn`. Указатель на созданный объект используется в качестве аргумента функции `CDC::SelectClipRgn`, задающей способ добавления изображения в данную область и указывающий, что любая часть выводимого изображения,

выходящая за пределы данной области, не будет отображаться. Если в функции указан только один аргумент, то изображение просто копируется в эту область.

Функция `OnHScroll` создает новую кисть и объект класса `CRect` для отображения полосы линейного индикатора. Объекту `CRect` первоначально присваивается значение объекта всей рабочей области элемента управления. После этого вычисляется и выводится на экран правая граница этой полосы.

Поскольку в данном случае (как и в большинстве реальных случаев) размер полосы линейного индикатора может только увеличиваться, то нет необходимости при каждом изменении ее размера заново перерисовывать участок, которому соответствует цвет фона. Если такая необходимость возникнет, то, во избежание мерцания полосы при ее перерисовке, следует ограничиться изменением цвета только на том участке, где он действительно меняется.

Поскольку в функции `OnDraw` производится только закрашивание рабочей области элемента управления цветом фона, то при восстановлении окна из пиктограммы пользовательский линейный индикатор останется незаполненным до следующего вызова функции `OnHScroll`. В реальных задачах необходимость полного восстановления пользовательского линейного индикатора в функции `OnPaint` зависит от частоты вызова функции его обновления. Если это обновление происходит через доли секунды, то нет никакой необходимости в полном восстановлении пользовательского линейного индикатора в функции `OnPaint`.

Класс просмотрового окна списка

Элемент управления этого типа упрощает разработку приложений, которые работают со списками объектов и облегчают поиск их атрибутов. Типичной областью применения данного элемента управления является работа с именами файлов.

Каждый файл имеет множество атрибутов, среди которых прежде всего следует назвать его имя, дату создания и размер. Windows позволяет отображать информацию о файлах в различных форматах. Это может быть набор пиктограмм с соответствующими именами файлов, причем некоторым типам файла соответствуют уникальные пиктограммы. Другим способом представления информации о файлах является таблица, где каждому атрибуту соответствует свой столбец. Существуют и другие способы представления этой информации. Большинство этих способов инкапсулированы в классе `CListCtrl`.

Класс `CListCtrl` и связанное с ним просмотровое окно списка позволяют отображать набор элементов данных, каждый из которых состоит из значка (хранящегося в списке изображений) и текстовой метки. Помимо значка и метки каждый элемент данных может содержать информацию, которая будет отображаться в столбцах, расположенных справа от значка и метки. Этот

элемент управления (а, следовательно, и сам класс `CListCtrl`) доступен только в программах, выполняемых под управлением операционных систем Windows 95 или Windows NT версии 3.51 и выше.

Копирование демонстрационного приложения

Класс `CListCtrl` предназначен для работы со сложными объектами, поэтому для демонстрации его возможностей лучше не создавать приложения с нуля, а воспользоваться примером из библиотеки MSDN.

Чтобы переписать текст проекта с примером на диск вашего компьютера:

1. Выберите команду меню **Help**, **Index** (Справка, Индекс). Появится окно **MSDN Library Visual Studio 6.0**, изображенное на рис. 4.21.
2. Раскройте вкладку **Указатель**. В текстовое поле **Введите ключевое слово для поиска:** введите имя класса `CListCtrl` и в появившемся ниже этого имени списке дважды щелкните левой кнопкой мыши на строке **examples** (примеры). Окно **MSDN Library Visual Studio 6.0**, примет вид, изображенный на рис. 4.22.

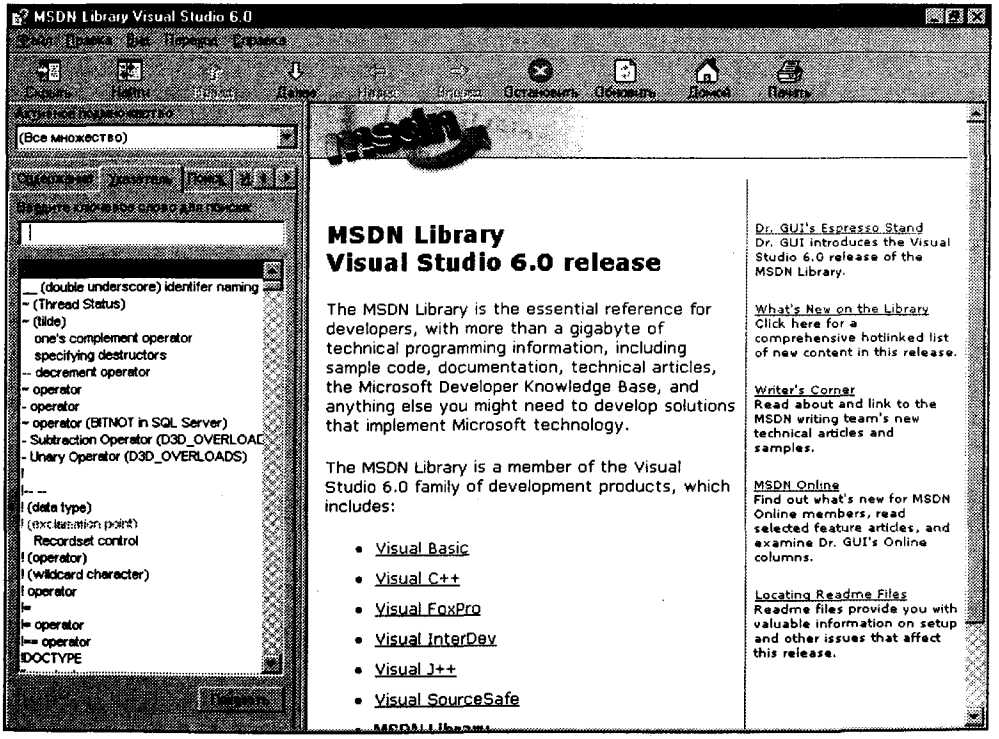


Рис. 4.21. Окно **MSDN Library Visual Studio 6.0**

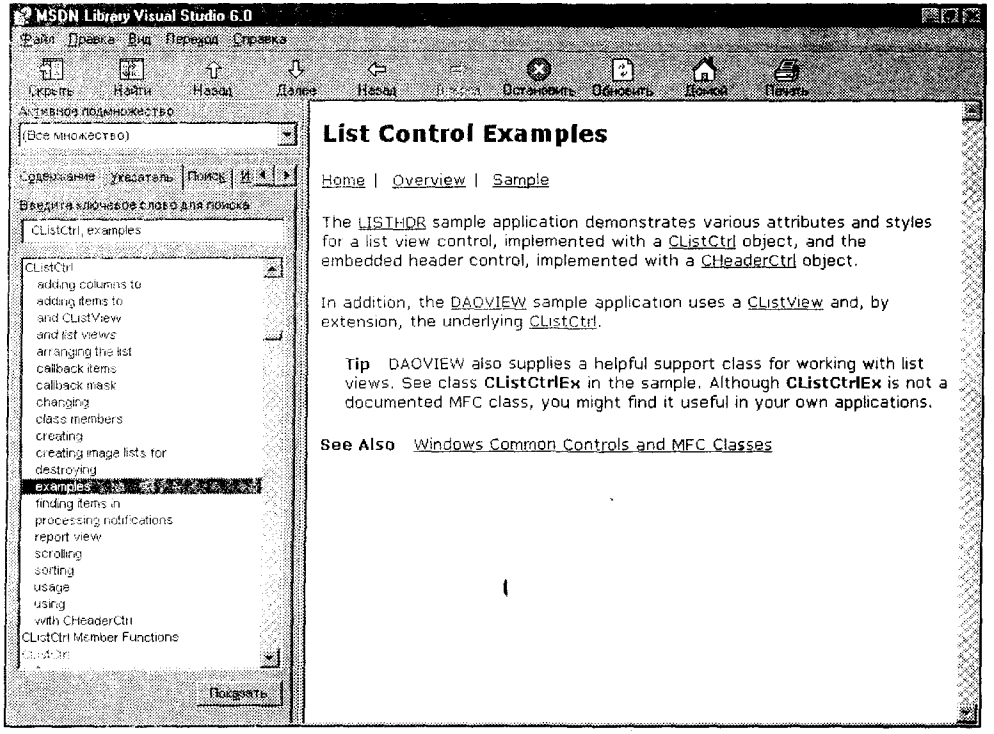


Рис. 4.22. *Окно* **MSDN Library Visual Studio 6.0** *с панелью примеров*

3. Щелкните левой кнопкой мыши на синем тексте "LISTHDR" в первой строке текста справки. Окно **MSDN Library Visual Studio 6.0** примет вид, изображенный на рис. 4.23.
4. Щелкните на синей строчке "Click to open or copy the LISTHDR project files" (Щелкните левой кнопкой мыши, чтобы открыть или скопировать файлы приложения LISTHDR). Если библиотека MSDN не установлена на диске вашего компьютера, появится диалоговое окно **Справка**, изображенное на рис. 4.24.
5. Вставьте компакт-диск, помеченный **"MSDN VS 6.0 CD1"**, и нажмите кнопку **OK**. Появится диалоговое окно **Visual C++ Samples** (Примеры Visual C++), изображенное на рис. 4.25.
6. Нажмите кнопку **Копировать все...**. Появится диалоговое окно **Копирование**, изображенное на рис. 4.26.
7. Выберите каталог, в который будет производиться копирование, и нажмите кнопку **OK**.
8. Согласитесь с созданием нового каталога, если вы собираетесь копировать туда файлы проекта, и нажмите кнопку **OK** после того, как появится окно сообщения об успешном завершении копирования.

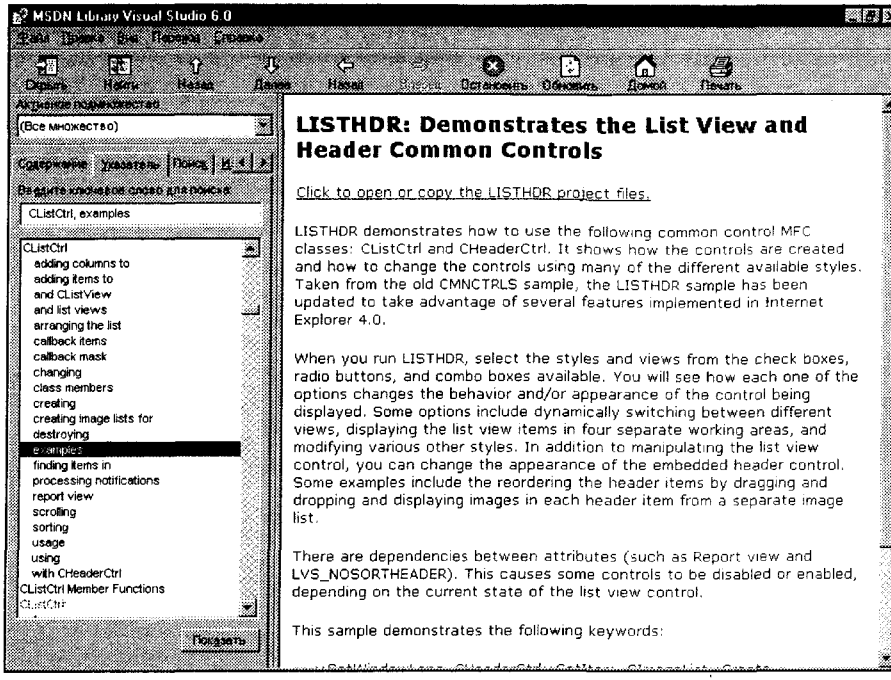


Рис. 4.23. Окно **MSDN Library Visual Studio 6.0** с описанием демонстрационного приложения

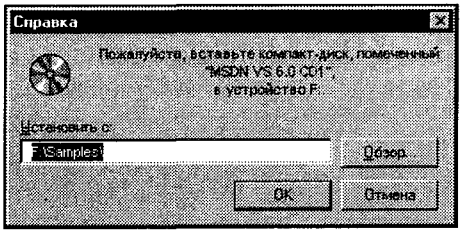


Рис. 4.24. Диалоговое окно **Справка**

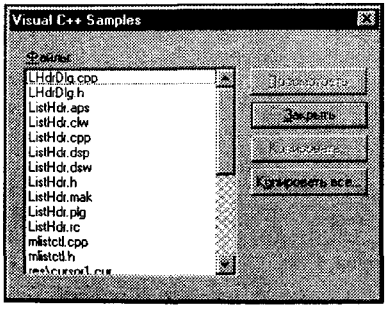


Рис. 4.25. Диалоговое окно **Visual C++ Samples**

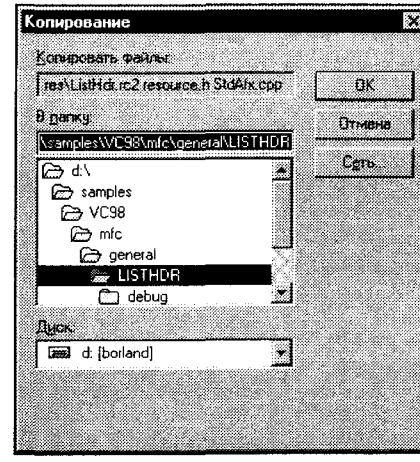


Рис. 4.26. Диалоговое окно **Копирование**

9. Нажмите кнопку **Закрыть** в диалоговом окне **Visual C++ Samples** и закройте окно **MSDN Library Visual Studio 6.0**.
10. Если библиотека Unicode не установлена, выберите команду **Build**, **Set Active Configuration**. Появится диалоговое окно **Set Active Project Configuration** (Установка активной конфигурации проекта). В окне списка **Project configurations:** выделите строку **ListHdr — Win32 Debug**, как показано на рис. 4.27, и нажмите кнопку **OK**.

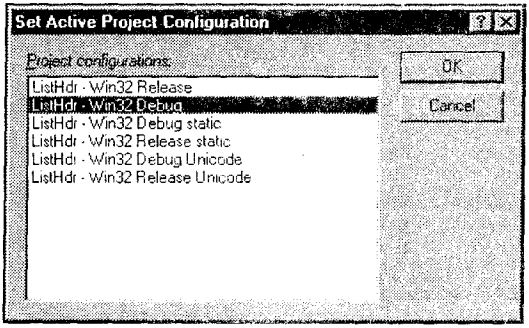


Рис. 4.27. Диалоговое окно **Set Active Project Configuration**

11. Нажмите клавишу <F5>, выберите команду **Build**, **Execute ListHdr.exe**, нажмите кнопку **Execute Program** в панели инструментов **Build MiniBar** или запустите приложение на исполнение другим способом. Появится диалоговое окно приложения, изображенное на рис. 4.28.

Поработайте с этим диалоговым окном, установите различные комбинации параметров и обратите внимание на совместимость различных параметров просмотрового окна списка. Эта работа лучше всякого обучающего курса поможет вам разобраться во всех особенностях работы с данным видом элемента управления.

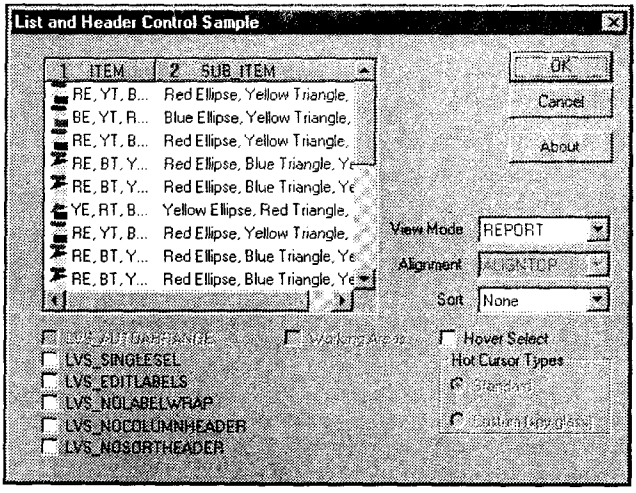


Рис. 4.28. Диалоговое окно тестового приложения ListHdr

Анализ демонстрационного приложения

Ознакомившись с основными параметрами просмотрового окна списка, выясним, как эти параметры передаются объекту класса, производного от класса `CListCtrl`. Чтобы подробнее ознакомиться с файлами проекта:

1. Выберите команду меню **File**, **Open Workspace** (Файл, Открыть проект). Появится диалоговое окно **Open Workspace**, изображенное на рис. 4.29.

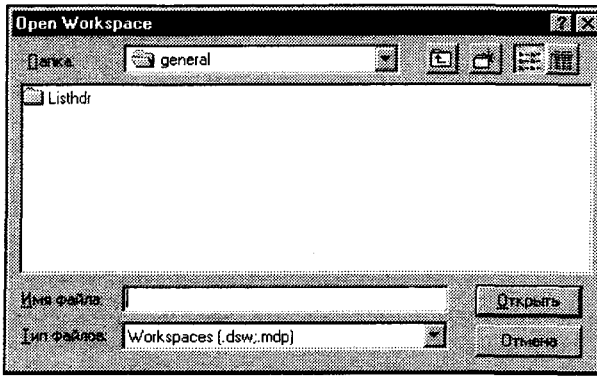


Рис. 4.29. Диалоговое окно **Open Workspace**

2. Раскройте в нем папку, в которую были скопированы файлы проекта ListHdr, выберите в ней файл ListHdr.dsw и нажмите кнопку **Открыть**.
3. Раскройте панель **ResourceView** окна **Workspace**, раскройте папку \ListHdr Resources, в ней раскройте папку \Dialog и дважды щелкните левой кнопкой мыши на идентификаторе ресурса `IDD_LISTHDR_DIALOG`. В окне редактирования ресурса появится ресурс диалогового окна проекта, как показано на рис. 4.30.
4. Щелкните правой кнопкой мыши на флажке **LVS_AUTOARRANGE** и в появившемся контекстном меню выберите команду **Properties**. Откроется диалоговое окно **Check Box Properties**.
5. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна. Окно редактирования ресурса диалогового окна примет вид, изображенный на рис. 4.31.
6. Убедитесь, что флажку константы стиля **LVS_AUTOARRANGE** соответствует идентификатор `IDC_AUTOARRANGE` в текстовом поле списка **ID:**.
7. Щелкните левой кнопкой мыши на флажке константы стиля **LVS_SINGLESEL**. В текстовом поле списка **ID:** диалогового окна **Check Box Properties** появится идентификатор ресурса `IDC_SINGLESEL`.
8. Аналогичным образом убедитесь, что флажкам констант стиля соответствуют идентификаторы ресурса, в которых префикс `LVS` заменен на префикс `IDC`.

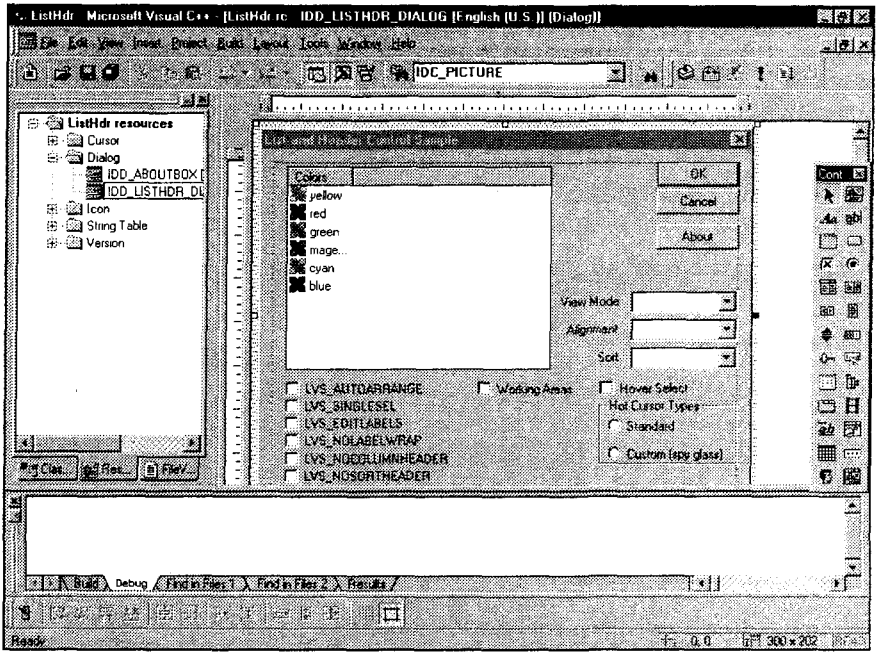


Рис. 4.30. Окно редактирования ресурса диалогового окна

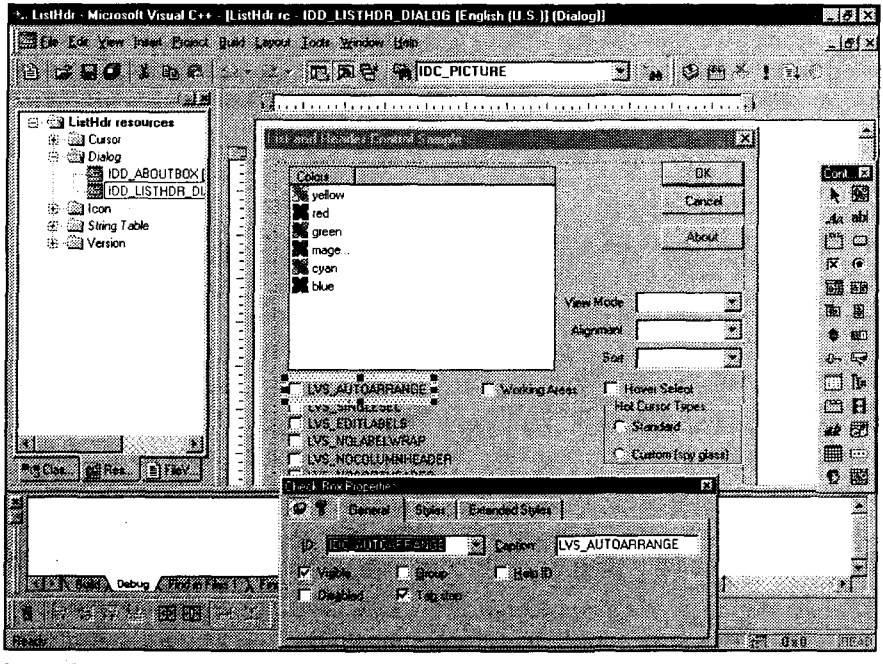


Рис. 4.31. Диалоговое окно **Check Box Properties**

9. Щелкните левой кнопкой мыши на флажке **Working Areas**. В текстовом поле списка **ID:** диалогового окна **Check Box Properties** появится идентификатор ресурса `IDC_WORKAREAS`.
10. Щелкните левой кнопкой мыши на флажке **Hover Select**. В текстовом поле списка **ID:** диалогового окна **Check Box Properties** появится идентификатор ресурса `IDC_HOVER`.
11. Щелкните левой кнопкой мыши в поле списка **View Mode**. В текстовом поле списка **ID:** диалогового окна **Combo Box Properties** появится идентификатор ресурса `IDC_VIEWMODE`.
12. Щелкните левой кнопкой мыши в поле списка **Alignment**. В текстовом поле списка **ID:** диалогового окна **Combo Box Properties** появится идентификатор ресурса `IDC_ALIGN`.
13. Щелкните левой кнопкой мыши на списке **Sort**. В текстовом поле списка **ID:** диалогового окна **Combo Box Properties** появится идентификатор ресурса `IDC_SORT`.
14. Установите переключатель **Standard** в группе **Hot Cursor Types**. В текстовом поле списка **ID:** диалогового окна **Radio Box Properties** появится идентификатор ресурса `IDC_STDHOTCUR`.
15. Установите переключатель **Custom (spy glass)** в группе **Hot Cursor Types**. В текстовом поле списка **ID:** диалогового окна **Radio Box Properties** появится идентификатор ресурса `IDC_MYHOTCUR`.
16. Закройте диалоговое окно **Radio Box Properties** и вызовите мастер ClassWizard.
17. В диалоговом окне **MFC ClassWizard** раскройте вкладку **Message Maps**.
18. Переместите бегунок линейки прокрутки окна списка **Member functions:** таким образом, чтобы в нем отображались только функции обработки сообщений. Диалоговое окно **MFC ClassWizard** примет вид, изображенный на рис. 4.32.

    Из содержимого вкладки **Message Maps** диалогового окна **MFC ClassWizard** можно сделать вывод о том, что каждому флажку в данном диалоговом окне сопоставлена своя функция обработки сообщения об изменении его состояния. Каждому списку в окне сопоставлена своя функция обработки сообщения об изменении содержимого его текстового поля. Аналогичная функция сопоставлена и каждому положению переключателя типа курсора. Самому классу диалогового окна сопоставлен только минимальный набор функций обработки сообщений.
19. Нажмите кнопку **OK** и закройте диалоговое окно **MFC ClassWizard**.
20. Раскройте панель **FileView** окна **Workspace**, выберите папку \ListHdr Files, в ней раскройте папку \Source Files и дважды щелкните левой кнопкой мыши на файле mlistctrl.cpp. В окне редактирования появится текст файла реализации класса `CMyListCtrl`, приведенный в листинге 4.4.

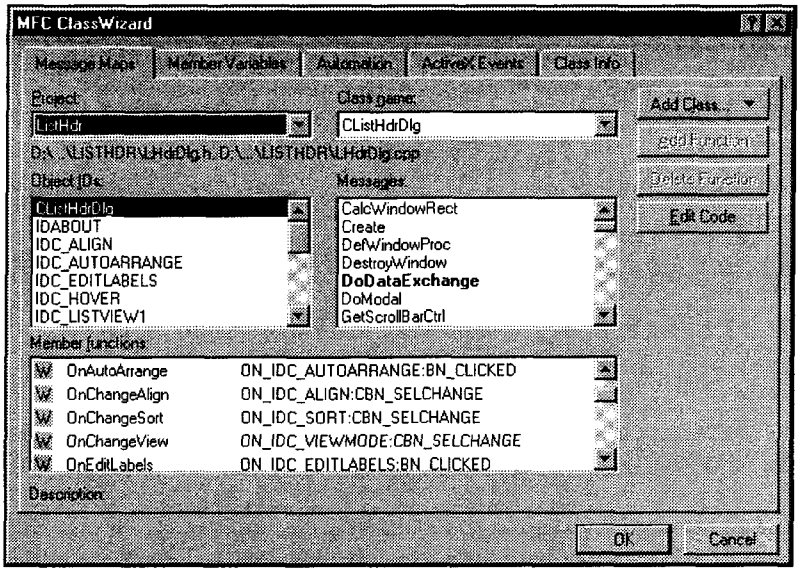


Рис. 4.32. *Диалоговое окно* **MFC ClassWizard**

Листинг 4.4. Файл реализации класса `CMyListCtrl`

```
// MyListCtrl.cpp : implementation file

// This is a part of the Microsoft Foundation Classes C++ library.
// Copyright (C) 1992-1998 Microsoft Corporation
// All rights reserved.
//
// This source code is only intended as a supplement
// to the Microsoft Foundation Classes Reference and related
// electronic documentation provided with the library.
// See these sources for detailed information regarding
// the Microsoft Foundation Classes product.

#include "stdafx.h"
#include "listhdr.h"
#include "mlistctl.h"
//#include "listcpg.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif
```

```
//////////////////////////////////////////////////////////////////////////
// CMyListCtrl

CMyListCtrl::CMyListCtrl()
{
  m_bDragging = FALSE;
  m_pimageListDrag = NULL;
}

CMyListCtrl::~CMyListCtrl()
{
}

BEGIN_MESSAGE_MAP(CMyListCtrl, CListCtrl)
  //{{AFX_MSG_MAP(CMyListCtrl)
  ON_NOTIFY_REFLECT(LVN_BEGINDRAG, OnBeginDrag)
  ON_NOTIFY_REFLECT(LVN_BEGINRDRAG, OnBeginDrag)
  ON_NOTIFY_REFLECT(LVN_ENDLABELEDIT, OnEndLabelEdit)
  ON_WM_MOUSEMOVE()
  ON_WM_LBUTTONUP()
  ON_WM_RBUTTONUP()
  //}}AFX_MSG_MAP
END_MESSAGE_MAP()

//////////////////////////////////////////////////////////////////////////
// CMyListCtrl message handlers
void CMyListCtrl::OnBeginDrag(LPNMHDR pnmhdr, LRESULT *pResult)
{
  CPoint          ptItem, ptAction, ptImage;
  NM_LISTVIEW     *pnmListView = (NM_LISTVIEW *)pnmhdr;

  ASSERT(!m_bDragging);
  m_bDragging = TRUE;
  m_iItemDrag = pnmListView->iItem;
  ptAction = pnmListView->ptAction;
  GetItemPosition(m_iItemDrag, &ptItem);  // ptItem is relative to (0,0)
                                          // and not the view origin
  GetOrigin(&m_ptOrigin);

  ASSERT(m_pimageListDrag == NULL);
  m_pimageListDrag = CreateDragImage(m_iItemDrag, &ptImage);
  m_sizeDelta = ptAction - ptImage;    // difference between cursor pos
                                       // and image pos
  m_ptHotSpot = ptAction - ptItem + m_ptOrigin;  // calculate hotspot for
                                                 // the cursor
  m_pimageListDrag->DragShowNolock(TRUE);  // lock updates and show drag
                                           // image
```

```
  m_pimageListDrag->SetDragCursorImage(0, m_ptHotSpot);  // define the
                                        // hot spot for the new cursor image
  m_pimageListDrag->BeginDrag(0, CPoint(0, 0));
  ptAction -= m_sizeDelta;
  m_pimageListDrag->DragEnter(this, ptAction);
  m_pimageListDrag->DragMove(ptAction);  // move image to overlap
                                         // original icon
  SetCapture();
}

void CMyListCtrl::OnMouseMove(UINT nFlags, CPoint point)
{
  long        lStyle;
  int         iItem;
  LV_ITEM     lvitem;

  lStyle = GetWindowLong(m_hWnd, GWL_STYLE);
  lStyle &= LVS_TYPEMASK;  // drag will do different things in list and
                           // report mode
  if (m_bDragging)
  {
    ASSERT(m_pimageListDrag != NULL);
    m_pimageListDrag->DragMove(point - m_sizeDelta);  // move the image
    if ((iItem = HitTest(point)) != -1)
    {
      m_iItemDrop = iItem;
      m_pimageListDrag->DragLeave(this); // unlock the window and
                                         // hide drag image
      if (lStyle == LVS_REPORT || lStyle == LVS_LIST)
      {
        lvitem.iItem = iItem;
        lvitem.iSubItem = 0;
        lvitem.mask = LVIF_STATE;
        lvitem.stateMask = LVIS_DROPHILITED;  // highlight the
                                              // drop target
        SetItem(&lvitem);
      }

      point -= m_sizeDelta;
      m_pimageListDrag->DragEnter(this, point);  // lock updates
                                                 // and show drag image
    }
  }

  CListCtrl::OnMouseMove(nFlags, point);
}
```

```
void CMyListCtrl::OnButtonUp(CPoint point)
{
  if (m_bDragging)  // end of the drag operation
  {
    long        lStyle;
    CString     cstr;

    lStyle = GetWindowLong(m_hWnd, GWL_STYLE) & LVS_TYPEMASK;
    m_bDragging = FALSE;

    ASSERT(m_pimageListDrag != NULL);
    m_pimageListDrag->DragLeave(this);
    m_pimageListDrag->EndDrag();
    delete m_pimageListDrag;
    m_pimageListDrag = NULL;

    // The drop target's sub-item text is replaced by the dragged item's
    // main text
    if (lStyle == LVS_REPORT && m_iItemDrop != m_iItemDrag)
    {
      cstr = GetItemText(m_iItemDrag, 0);
      SetItemText(m_iItemDrop, 1, cstr);
    }

    // the character string "**" is added to the end of the drop target's
    // main text
    if (lStyle == LVS_LIST && m_iItemDrop != m_iItemDrag)
    {
      cstr = GetItemText(m_iItemDrop, 0);
      cstr += _T("**");
      SetItemText(m_iItemDrop, 0, cstr);
    }

      // move the icon
    if (lStyle == LVS_ICON || lStyle == LVS_SMALLICON)
    {
      point -= m_ptHotSpot;  // the icon should be drawn exactly
                             // where the image is
      point += m_ptOrigin;
      SetItemPosition(m_iItemDrag, point);  // just move the dragged item
    }

    ::ReleaseCapture();
  }
}

void CMyListCtrl::OnLButtonUp(UINT nFlags, CPoint point)
{
  OnButtonUp(point);
```

```
  CListCtrl::OnLButtonUp(nFlags, point);
}

void CMyListCtrl::OnRButtonUp(UINT nFlags, CPoint point)
{
  OnButtonUp(point);
  CListCtrl::OnRButtonUp(nFlags, point);
}

void CMyListCtrl::OnEndLabelEdit(LPNMHDR pnmhdr, LRESULT *pLResult)
{
  LV_DISPINFO  *plvDispInfo = (LV_DISPINFO *)pnmhdr;
  LV_ITEM      *plvItem = &plvDispInfo->item;

  if (plvItem->pszText != NULL)
    SetItemText(plvItem->iItem, plvItem->iSubItem, plvItem->pszText);
}
```

Как следует из листинга 4.4, конструктор данного класса задает начальные значения двум переменным: присваивает значение `FALSE` переменной `m_bDragging` и значение `NULL` — переменной `m_pimageListDrag`. Данный класс имеет пустой деструктор.

21. Вызовите мастер ClassWizard из окна редактирования файла и раскройте вкладку **Message Maps**, диалоговое окно **MFC ClassWizard** будет иметь вид, изображенный на рис. 4.33.

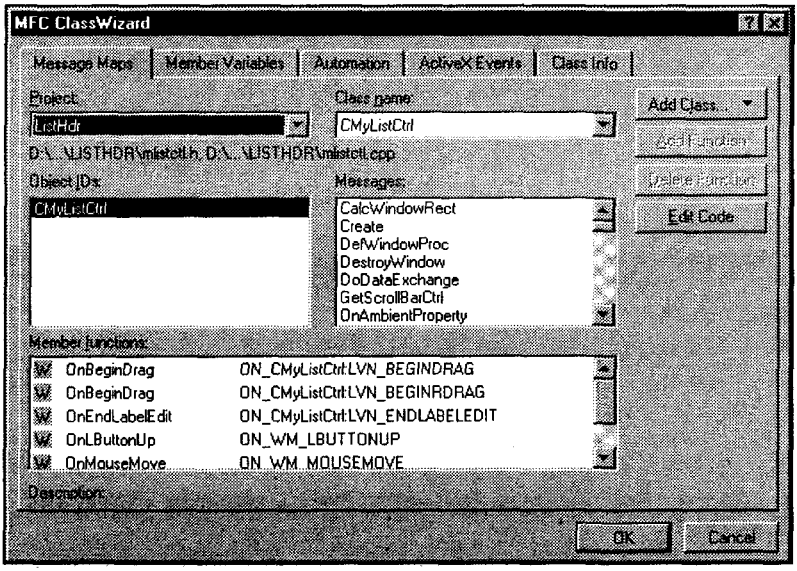


Рис. 4.33. Диалоговое окно **MFC ClassWizard**

Как следует из рис. 4.33, все обрабатываемые сообщения относятся к самому классу, т. е. он не содержит внутренних элементов управления.

Карта сообщений данного класса содержит три макроса передачи обработки заданного кода извещения от элемента управления функции-члену класса и три макроса вызова функций обработки сообщений мыши. В макросах передачи обработки заданного кода извещения от элемента управления функции-члену класса вместо стандартного макроса ON_NOTIFY использован макрос *обработки отраженного сообщения* (message reflection) ON_NOTIFY_REFLECTION. Необходимость в подобных сообщениях связана с тем, что при работе с элементами управления часто возникает необходимость передать обработку какого-либо сообщения родительскому окну. Таким образом, каждое родительское окно, в котором используется данный элемент управления, должно включать в себя функцию обработки данного сообщения. Однако во многих случаях все эти функции выполняют одни и те же операции и их целесообразно было бы поместить в сам класс элемента управления. Именно такую операцию и выполняет механизм отражения сообщений.

Если в родительском окне отсутствует функция обработки данного сообщения, то вызывается функция, указанная в макросе обработки отраженного сообщения. Синтаксис макроса обработки отраженного сообщения отличается от макроса обработки сообщения тем, что в нем к имени макроса добавляется окончание _REFLECTION.

См. также главу 5

Первая функция обработки сообщения вызывается в ответ на сообщение LVN_BEGINDRAG. Это сообщение служит для извещения родительского окна о том, что началась операция перетаскивания, инициализированная нажатием левой кнопки мыши. Данное сообщение посылается в формате сообщения WM_NOTIFY. Функция обработки сообщения WM_NOTIFY имеет два аргумента, первый из которых представляет собой идентификатор элемента управления, пославшего сообщение, а второй — указатель на объект структуры NMHDR, содержащей код сообщения и дополнительную информацию (в некоторых сообщениях этот указатель относится к более сложной структуре, первым элементом которой является объект структуры NMHDR).

Прежде всего в функции OnBeginDrag при помощи макроса ASSERT проверяется значение переменной m_bDragging. Если эта логическая переменная имеет значение TRUE, то перетаскивание не начинается, а уже давно идет. В этом случае макрос ASSERT выдает сообщение об ошибке. После данной проверки переменной m_bDragging присваивается значение TRUE, указывающее, что перетаскивание началось. К сожалению, о назначении следующих двух строк можно только строить предположения, поскольку в них используется объект структуры NM_LISTVIEW, которая не описывается ни в библиотеке MSDN, ни в данном приложении. Исходя из использования переменной m_iItemDrag в функции CListCtrl::GetItemPosition, первым аргументом

которой является индекс элемента данных, а вторым — указатель на объект структуры POINT, в который будет записана информация о положении этого элемента данных, можно сделать вывод о том, что в переменной m_iItemDrag хранится индекс элемента данных, а в переменной ptItem — ее координаты.

Функция CListCtrl::GetOrigin позволяет сохранить в переменной m_ptOrigin, являющейся объектом класса CPoint, координаты рабочей области просмотрового окна списка.

В следующем разделе функции OnBeginDrag проверяется нулевое значение указателя m_pimageListDrag на объект класса CImageList, т. е. отсутствие такого объекта. Если этот объект существует, то макрос ASSERT выдаст сообщение об ошибке.

Следующий оператор данной функции создает объект класса CImageList при помощи функции CListCtrl::CreateDragImage. Как можно понять из комментария к следующему оператору функции, в переменной ptAction хранится текущее положение курсора. Поэтому в данном операторе вычисляется разность между текущим положением курсора и исходным положением изображения, которая сохраняется в переменной m_sizeDelta.

После этого в функции OnBeginDrag вычисляется разность между текущим положением курсора и координатами элемента данных. Из этого оператора можно заключить, что положение курсора определяется относительно верхнего левого угла просмотрового окна списка.

Функция CImageList::DragShowNolock управляет отображением перетаскиваемого изображения во время операции перетаскивания без блокировки окна. Если ее аргумент имеет значение FALSE, то перетаскиваемое изображение скрывается. Это делается для ускорения операций на медленных компьютерах.

Функция CImageList::SetDragCursorImage создает новое изображение, обычно являющееся комбинацией указателя мыши и перетаскиваемого изображения. После вызова функции SetDragCursorImage следует вызвать стандартную функцию Windows ShowCursor, в противном случае при выполнении операции перетаскивания на экране могут появиться два курсора: стандартный и созданный функцией CImageList::SetDragCursorImage. (В данном примере это соображение не учтено.)

Функция CImageList::BeginDrag создает временный список изображений, используемый в процессе перетаскивания. Эта функция вызывается для инициализации процесса перетаскивания. Первый аргумент данной функции представляет собой индекс перетаскиваемого изображения, совпадающий с индексом комбинированного изображения, полученного в предыдущей функции. Второй аргумент данной функции представляет собой координаты начальной точки перетаскивания (обычно это позиция курсора). В данном случае началом системы отсчета является верхний левый угол изображения.

После этого изображение перемещается таким образом, чтобы закрыть собой исходное изображение. Для этого соответствующим образом смещается позиция курсора и вызываются функции `CImageList::DragEnter` и `CImageList::DragMove`. Функция `CImageList::DragEnter` запрещает все операции перерисовки окна на время выполнения операции перетаскивания, а функция `CImageList::DragMove` перемещает перетаскиваемое изображение в точку с координатами, указанными в ее аргументе.

За ней следует функция `OnMouseMove`, являющаяся стандартной функцией обработки сообщения о перемещении мыши. Вызываемая в ней функция `GetWindowLong` позволяет получить информацию об указанном окне. Отличительной особенностью функции `GetWindowLong` является то, что смещение задается 32-разрядной величиной, что позволяет использовать дополнительную память для окна. В данном случае эта функция нужна для получения стиля просмотрового окна списка, который сразу же модифицируется, поскольку операция перетаскивания в режиме отображения списков отличается от той же операции в режиме отображения отчетов. Затем проверяется, какая из операций инициализирована в данный момент.

Если в данный момент инициализирована операция перетаскивания, то проверяется корректность указателя на объект класса `CImageList`, в котором должна храниться информация о перетаскиваемом изображении. Если указатель на данный объект имеет нулевое значение, то макрос `ASSERT` выдает сообщение об ошибке. В противном случае при помощи описанной выше функции `CImageList::DragMove` производится перемещение изображения в точку, получаемую из текущей позиции курсора путем введения в нее поправки на относительное положение курсора и базовой точки изображения.

Функция `CListCtrl::HitTest` проверяет, не находится ли в новой позиции какой-нибудь элемент данных просмотрового окна списка, и нет ли необходимости прокрутить окно списка, чтобы получить к нему доступ. В случае, если на данной позиции уже находится элемент данных, вызывается функция `CImageList::DragLeave`, снимающая блокировку окна, определяемого ее единственным аргументом, и скрывает перетаскиваемое изображение, позволяя внести в окно необходимые изменения.

Если просмотровое окно списка находится в режиме отображения отчета или в режиме отображения списка, то у элемента данных, который оказался на текущей позиции перемещаемого изображения, изменяются атрибуты. Механизм этого изменения остается неясным, поскольку в библиотеке MSDN очень сложно понять различие между структурами `LV_ITEM` и `LVITEM`, а также отсутствует описание константы `LVIS_DROPHILITED`.

После этого, независимо от того в каком режиме находится просмотровое окно списка, текущая позиция курсора перемещается вниз и используется функция `CImageList::DragEnter` для отрисовки изображения на новом месте.

Последним оператором функции OnMouseMove всегда является вызов соответствующего метода базового класса.

Поскольку при перетаскивании могут использоваться как левая, так и правая кнопки мыши, то в данном приложении была создана единая функция обработки сообщения об отпускании кнопки мыши OnButtonUp, вызов которой, помимо последующего вызова соответствующей функции базового класса, является единственным оператором функций OnLButtonUp и OnRButtonUp.

В функции OnButtonUp производится проверка, выполняется ли в данное время операция перетаскивания, и если нет, то данная функция не выполняет никаких действий. Если же данная операция выполнялась, то в данной функции определяется стиль окна, скорректированный маской текущего режима отображения, и переменной m_bDragging присваивается значение FALSE, говорящее о том, что данная операция завершена.

После стандартной проверки указателя на объект класса CImageList вызывается уже описанная нами функция CImageList::DragLeave, снимающая блокировку окна. После чего вызывается функция CImageList::EndDrag, завершающая операцию перетаскивания. После завершения работы с объектом класса CImageList он уничтожается и указателю на него присваивается значение NULL.

Если просмотровое окно списка находится в режиме отображения отчета и перетаскиваемый элемент данных располагается в "чужой" строке (то есть был перемещен), то в столбце, содержащем элементы данных более низкого уровня, отображается метка перемещаемого элемента данных, а содержимое первого столбца остается без изменений. Для этого при помощи функции CListCtrl::GetItemText в строковую переменную копируется текст метки основного элемента данных. При этом в первом аргументе данной функции указывается индекс основного элемента данных, а во втором — определяется элемент данных более низкого уровня, откуда следует копировать текст. Нулевое значение второго аргумента указывает на то, что в данном случае копируется метка основного элемента данных.

После этого содержимое данной строковой переменной используется в качестве третьего аргумента при вызове функции CListCtrl::SetItemText, первым аргументом которой является индекс основного элемента данных, вторым аргументом — индекс элемента данных более низкого уровня, а третьим — указатель на строку, содержащую текст, который необходимо поместить в этот элемент данных более низкого уровня.

Если просмотровое окно списка находится в режиме отображения значков или маленьких значков, то текущая позиция курсора корректируется таким образом, чтобы значок отображался точно на том месте, где в настоящее время расположено комбинированное изображение. Скорректированная позиция курсора преобразуется в оконные координаты родительского окна и используется в качестве второго аргумента при вызове функции CListCtrl::

SetItemPosition, перемещающей в режиме отображения значков или маленьких значков элемент данных на определенную позицию в пределах просмотрового окна списка. Если в просмотровом окне списка установлен стиль LVS_AUTOARRANGE, то выравнивание производится после установки элемента данных в новую позицию.

Функция ReleaseCapture освобождает мышь от ее захвата окном в текущем потоке и восстанавливает нормальную процедуру обработки сообщений мыши. Если окно захватывает мышь, то все ее сообщения направляются в него независимо от положения курсора мыши. Единственным исключением является обработка щелчка кнопкой мыши, если ее курсор находится в рабочей области другого окна.

Последней функцией обработки сообщений данного класса является функция OnEndLabelEdit, обрабатывающая сообщение LVN_ENDLABELEDIT. Данное сообщение извещает родительское окно о том, что в элементе данных закончено редактирование метки. Функция OnEndLabelEdit имеет формат функции обработки сообщения WM_NOTIFY, описанного при рассмотрении функции OnBeginDrag. Поскольку в библиотеке MSDN отсутствует описание структуры LV_DISPINFO, то можно только предполагать, что данная функция сначала проверяет, не пуста ли метка, а затем при помощи функции SetItemText, описанной нами ранее, заносит этот текст в соответствующий элемент данных.

Чтобы вывести в окно редактирования следующий рассматриваемый файл, дважды щелкните левой кнопкой мыши на имени файла ListHdrDlg.cpp, расположенного в папке \Source Files панели **FileView** окна **Workspace**. В окне редактирования появится текст файла реализации класса CListHdrDlg.

Начнем рассмотрение функций-членов данного класса с функции OnInitDialog, которая вызывается в том случае, если приложение устанавливает фокус ввода на одном из элементов управления данного диалогового окна. Текст этой функции приведен в листинге 4.5.

Листинг 4.5. Функция OnInitDialog класса CListHdrDlg

```
BOOL CListHdrDlg::OnInitDialog()
{
  CListHdrApp     *pApp;
  CRect           rect;

  CDialog::OnInitDialog();  // let the base class do the default work
  UpdateData(TRUE);         // bring the information from the dialog.
  pApp = (CListHdrApp *)AfxGetApp();
  srand((unsigned) time(NULL));     // start the random number generator

  // create image list for header items
  m_pImageHdrSmall = new CImageList();
```

```
    ASSERT(m_pImageHdrSmall != NULL); // serious allocation failure checking
    m_pImageHdrSmall->Create(16, 16, ILC_MASK, 2, 2);
    m_pImageHdrSmall->Add(pApp->LoadIcon(IDI_HDRICON1));
    m_pImageHdrSmall->Add(pApp->LoadIcon(IDI_HDRICON2));

    // fill in image lists
    m_pImageList = new CImageList();
    m_pImageListSmall = new CImageList();
    ASSERT(m_pImageList != NULL && m_pImageListSmall != NULL);  // serious
                                                    // allocation failure checking
    m_pImageList->Create(32, 32, TRUE,  4, 4);
    m_pImageListSmall->Create(16, 16, TRUE, 4, 4);
    m_pImageList->Add(pApp->LoadIcon(IDI_ICONLIST1));
    m_pImageList->Add(pApp->LoadIcon(IDI_ICONLIST2));
    m_pImageList->Add(pApp->LoadIcon(IDI_ICONLIST3));
    m_pImageList->Add(pApp->LoadIcon(IDI_ICONLIST4));
    m_pImageListSmall->Add(pApp->LoadIcon(IDI_ICONLIST1));
    m_pImageListSmall->Add(pApp->LoadIcon(IDI_ICONLIST2));
    m_pImageListSmall->Add(pApp->LoadIcon(IDI_ICONLIST3));
    m_pImageListSmall->Add(pApp->LoadIcon(IDI_ICONLIST4));

    // initialize the standard and custom hot cursors
    m_hMyHotCursor= pApp->LoadCursor(IDC_SPYGLASS);
    m_hStdHotCursor= m_listctrl.GetHotCursor();

    FillListCtrl();

    // Enable/disable inter-dependent list control styles
    ((CButton*)GetDlgItem(IDC_STDHOTCUR))->SetCheck(m_bHotCursor);
    GetDlgItem(IDC_STDHOTCUR)->EnableWindow(m_bHoverSelect);
    GetDlgItem(IDC_MYHOTCUR)->EnableWindow(m_bHoverSelect);
    GetDlgItem(IDC_WORKAREAS)->EnableWindow(FALSE);
    GetDlgItem(IDC_ALIGN)->EnableWindow(FALSE);  // default is  report mode
                                                 // Does not support this
    GetDlgItem(IDC_AUTOARRANGE)->EnableWindow(FALSE);

    return FALSE;  // there is no change in any control focus stuff here.
}
```

Первым оператором данной функции является вызов соответствующего метода базового класса. После этого вызывается функция `UpdateData`, извлекающая информацию из элементов управления диалогового окна. Функция `AfxGetApp` возвращает указатель на объект класса `CWinApp`. Этот указатель может быть использован для доступа к хранящейся в данном классе информации, например информации о главном окне программы.

После этого происходит инициализация генератора псевдослучайных чисел с использованием функции `srand`, единственным аргументом которой явля-

ется целое число без знака, используемое в дальнейшем как начальное значение для рекуррентного генератора случайных чисел. В качестве аргумента данной функции используется значение, возвращаемое функцией `time` и представляющее собой количество секунд, прошедшее до момента вызова данной функции с полуночи 1 января 1970 года. Нулевой аргумент данной функции указывает на то, что данное значение не должно сохраняться, в противном случае аргументом этой функции должен быть указатель на структуру `time_t`, в которой и должен храниться результат.

В следующей секции функции `OnInitDialog` создается список изображений для заголовков столбцов. В переменную `m_pImageHdrSmall` записывается указатель на объект класса `CImageList` и проверяется, что этот указатель не равен нулю, то есть при создании указателя на объект не возникло грубых ошибок. После чего с использованием функции `CImageList::Create` создается список изображений и связывается с соответствующим объектом. В данном случае первый аргумент функции содержит горизонтальный размер каждого элемента данных в элементах изображения, а второй ее аргумент содержит его вертикальный размер в тех же единицах измерения. Третий аргумент определяет тип создаваемого списка изображений. Четвертый аргумент указывает, сколько изображений должен изначально содержать список, а пятый аргумент определяет приращение списка изображений системой при помещении в него новых изображений.

Функция `CImageList::Add` добавляет изображения в список. В данном случае изображение задается дескриптором значка, содержащим его битовый образ и маску. Этот дескриптор является возвращаемым значением функции `CWinApp::LoadIcon`, аргументом которой в данном случае является идентификатор ресурса значка, описанный в ресурсах данного приложения. Как следует из листинга 4.5, в данный список добавляются два значка для первого и для второго столбцов.

После этого создаются списки изображений для значков и маленьких значков. Процедура их создания аналогична описанной выше для списка изображений значков столбцов, только в данном случае в каждый список помещается по четыре значка.

Затем следует создание пользовательского и стандартного курсоров. При создании пользовательского курсора используется функция `CWinApp::LoadCursor`, в качестве аргумента которой в данном случае выступает идентификатор ресурса курсора, описанный в ресурсах данного приложения. При создании стандартного курсора используется функция `CListCtrl::GetHotCursor`. Этот курсор будет виден только в том случае, если разрешено выделение "на лету", и будет появляться в момент прохождения курсора мыши над элементом данных просмотрового окна списка.

После этого вызывается функция `FillListCtrl`, которая будет рассмотрена ниже. Последние операторы функции `OnInitDialog` устанавливают началь-

ное состояние элементов управления диалогового окна. Для этого с помощью функции `CWnd::GetDlgItem` получается указатель на дочернее окно или элемент управления, идентификатор которого указан в качестве аргумента данной функции, а затем полученный указатель используется для доступа к методу соответствующего класса.

Функция `CButton::SetCheck` устанавливает состояние соответствующего флажка или переключателя: если ее аргумент имеет значение 0, то данный флажок сброшен, а переключатель не отмечен, если аргумент имеет значение 1, то данный флажок установлен, а переключатель отмечен, если аргумент имеет значение 2, то положение кнопки не определено. Функция `CWnd::EnableWindow` разрешает или запрещает реакцию окна на нажатие клавиш или на сообщения от мыши. Значение `TRUE` для аргумента данной функции разрешает обработку сообщений, а значение `FALSE` — запрещает. Элемент управления, которому запрещена обработка сообщений, не может получить фокус ввода, и пользователь не имеет к нему доступа.

Функция `OnInitDialog` возвращает значение `FALSE`, поскольку в ней не изменился фокус ввода ни для одного из элементов управления.

Рассмотрим теперь функцию `FillListCtrl`, вызываемую из функции `OnInitDialog`. Текст этой функции приведен в листинге 4.6.

Листинг 4.6. Функция `FillListCtrl` класса `CListHdrDlg`

```
void CListHdrDlg::FillListCtrl()
{
  CRect           rect;
  int             iIcon, iItem, iSubItem, iActualItem;
  LV_ITEM         lvitem;
  CString         strItem1= _T("ITEM");
  CString         strItem2= _T("SUB_ITEM");
  CString         strIconDesc[4], strIconShortDesc[4];
  LPTSTR          pStrTemp1, pStrTemp2;
  CListHdrApp     *pApp;

  strIconDesc[0]= _T("Blue Ellipse, Yellow Triangle, Red Rectangle");
  strIconDesc[1]= _T("Yellow Ellipse, Red Triangle, Blue Rectangle");
  strIconDesc[2]= _T("Red Ellipse, Blue Triangle, Yellow Rectangle");
  strIconDesc[3]= _T("Red Ellipse, Yellow Triangle, Blue Rectangle");
  strIconShortDesc[0]= _T("BE, YT, RR (Title));
  strIconShortDesc[1]= _T("YE, RT, BR (Title));
  strIconShortDesc[2]= _T("RE, BT, YR (Title));
  strIconShortDesc[3]= _T("RE, YT, BR (Title));

  pApp = (CListHdrApp *)AfxGetApp();
  m_listctrl.SetImageList(m_pImageList, LVSIL_NORMAL);
  m_listctrl.SetImageList(m_pImageListSmall, LVSIL_SMALL);
```

```
  // insert two columns (REPORT mode) and modify the new header items
  m_listctrl.GetWindowRect(&rect);
  m_listctrl.InsertColumn(0, strItem1, LVCFMT_LEFT,
    rect.Width() * 1/3, 0);
  m_listctrl.InsertColumn(1, strItem2, LVCFMT_LEFT,
    rect.Width() * 2/3, 1);
  ModifyHeaderItems();

  for (iItem = 0; iItem < 20; iItem++)  // insert the items and subitems
                                        // into the list view.
    for (iSubItem = 0; iSubItem < 2; iSubItem++)
    {
      if (iSubItem == 0)
        iIcon = rand() % 4;  // choose the icon and legend for the entry

      lvitem.mask = LVIF_TEXT | (iSubItem == 0? LVIF_IMAGE : 0);
      lvitem.iItem = (iSubItem == 0)? iItem : iActualItem;
      lvitem.iSubItem = iSubItem;

      // calculate the main and sub-item strings for the current item
      pStrTemp1= strIconShort-
Desc[iIcon].GetBuffer(strIconShortDesc[iIcon].GetLength());
      pStrTemp2= strIconDesc[iIcon].GetBuffer(strIconDesc[iIcon].
                                                GetLength());
      lvitem.pszText = iSubItem == 0? pStrTemp1 : pStrTemp2;

      lvitem.iImage = iIcon;
      if (iSubItem == 0)
        // Insert new item
        iActualItem = m_listctrl.InsertItem(&lvitem);
      else
        m_listctrl.SetItem(&lvitem); // modify existing item
                                     // (the sub-item text)
    }
}
```

Первые операторы данной функции формируют и заполняют два строковых массива из четырех элементов каждый. После чего создаются два списка изображений для значков и маленьких значков. Функция `CWnd::GetWindowRect` записывает в объект структуры `RECT`, на который указывает ее аргумент, экранные координаты левого верхнего и правого нижнего углов прямоугольника, охватывающего объект класса `CWnd`, включая заголовок, рамку и полосы прокрутки, если таковые имеются.

Функция `CListCtrl::InsertColumn` создает новый столбец. В данном случае ее первый аргумент содержит номер формируемого столбца, второй — указатель на строковую переменную, содержащую заголовок столбца, третий

аргумент определяет выравнивание текста в столбце, четвертый определяет ширину столбца в элементах изображения, а пятый аргумент — индекс элемента данных более низкого уровня, ассоциированного с данным столбцом. Как следует из листинга 4.6, в просмотровом окне списка создается два столбца, с первым из которых ассоциируется основной элемент данных, а со вторым — первый элемент данных более низкого уровня. После этого вызывается функция `ModifyHeaderItems`, которая будет рассмотрена позднее.

Последним оператором функции `FillListCtrl` является оператор цикла, в котором производится заполнение просмотрового окна списка элементами данных. В данный цикл вложен второй цикл, заполняющий объекты данных различных уровней. Как следует из структуры программы, заполнение основного элемента данных практически ничем не отличается от заполнения элемента данных более низкого уровня.

Первый оператор данного цикла случайным образом выбирает значок для основного элемента данных. В переменную `LV_ITEM::mask` записывается значение маски. Если маска создается для основного элемента данных, то в данную переменную записывается величина `LVIF_TEXT|LVIF_IMAGE`, если же маска создается для элемента данных более низкого уровня, то записывается величина `LVIF_TEXT`. В переменную `LV_ITEM::iItem` основного элемента данных записывается номер текущего элемента данных, а в ту же переменную элемента данных более низкого уровня — индекс существующего основного элемента данных, к которому относится данный элемент. Переменной `LV_ITEM::iSubItem` присваивается номер элемента данных в иерархии.

Использование функции `CString::GetBuffer` при передаче содержимого объекта класса `CString` в текстовый буфер по указателю типа `LPTSTR` позволяет редактировать полученный текст, что невозможно при простом присваивании. Далее, в зависимости от положения элемента данных в иерархии, данная строка передается в переменную `LV_ITEM::pszText`. В переменную `LV_ITEM::iImage` записывается индекс значка элемента данных. Если в данный момент формируется основной элемент данных, то с использованием функции `CListCtrl::InsertItem` в просмотровое окно списка добавляется новый элемент данных, а его индекс сохраняется для дальнейшего использования. В случае элемента данных более низкого уровня при помощи функции `CListCtrl::SetItem` производится добавление атрибутов в существующий элемент данных. На этом функция `FillListCtrl` завершает свою работу.

Функция `ModifyHeaderItems` вызывается из функции `FillListCtrl`, а ее текст приведен ниже:

```
void CListHdrDlg::ModifyHeaderItems()
{
  HD_ITEM curItem;
```

```
  // retrieve embedded header control
  CHeaderCtrl* pHdrCtrl= NULL;
  pHdrCtrl= m_listctrl.GetHeaderCtrl();

  pHdrCtrl->SetImageList(m_pImageHdrSmall);
  // add bmaps to each header item
  pHdrCtrl->GetItem(0, &curItem);
  curItem.mask= HDI_IMAGE | HDI_FORMAT;
  curItem.iImage= 0;
  curItem.fmt= HDF_LEFT | HDF_IMAGE | HDF_STRING;
  pHdrCtrl->SetItem(0, &curItem);

  pHdrCtrl->GetItem(1, &curItem);
  curItem.mask= HDI_IMAGE | HDI_FORMAT;
  curItem.iImage= 1;
  curItem.fmt= HDF_LEFT | HDF_IMAGE | HDF_STRING;
  pHdrCtrl->SetItem(1, &curItem);
}
```

В первых операторах данной функции с помощью вызова функции `CListCtrl::GetHeaderCtrl` создается указатель на объект класса `CHeaderCtrl`. Для этого объекта вызывается функция `CHeaderCtrl::SetImageList`, связывающая с данным объектом список изображений, созданный и инициализированный в функции `OnInitDialog`.

После этого в данный класс добавляются соответствующие битовые образы. В результате вызова функции `CHeaderCtrl::GetItem` программа получает информацию о текущем состоянии заголовка конкретного столбца. Эта информация хранится в объекте структуры `HDITEM`. В полученном объекте структуры устанавливается маска. Установка флага в этой переменной указывает на то, что в соответствующем элементе данной структуры содержатся корректные данные. Если в переменной `mask` содержит ноль, то величины в других полях структуры теряют смысл.

Установка флага `HDI_IMAGE` в маске указывает на то, что в данной структуре используется содержимое переменной `iImage`, определяющее выводимое на экран изображение. Установка флага `HDI_FORMAT` в маске указывает, что в данной структуре используется содержимое переменной `fmt`. В переменную `iImage` заносится номер элемента данных в иерархии, а в переменную `fmt` — комбинация битовых флагов, указывающая, что текст в данном элементе управления выравнивается по левому краю, изображение выводится из списка изображений, и что в данном элементе управления будет выводиться строка. После этого заполненная структура `HDITEM` используется как второй аргумент при вызове функции `CHeaderCtrl::SetItem`, устанавливающей атрибуты элемента данных, индекс которого указан в ее первом аргументе.

Данная операция выполняется для основного элемента данных и элемента данных более низкого уровня.

В классе CListHdrDlg осталось рассмотреть только функции обработки сообщений, созданные с использованием мастера ClassWizard. Функции OnAutoArrange, OnEditLabels, OnNoColHdr, OnNoSortHdr, OnSingleSel и OnNoLabelWrap имеют очень простую структуру: вызывается функция UpdateData с аргументом, имеющим значение TRUE, выполняющая чтение состояния элементов управления диалогового окна, а затем вызывается соответствующая функция модификации просмотрового окна списка. Функции OnAutoArrange, OnEditLabels и OnSingleSel вызывают функцию ChangeListCtrlStyle, а функции OnNoColHdr, OnNoSortHdr и OnNoLabelWrap — функцию RenewListCtrl. Обе эти функции имеют одинаковый набор аргументов: первый аргумент представляет собой константу стиля просмотрового окна, а второй — логическую величину, указывающую на состояние соответствующего флажка. Текст этих функций приведен ниже:

```
// This function is used when a requested style
// LVS_NOLABELWRAP, LVS_NOCOLUMNHEADER, and LVS_NOSORTHEADER
// forces the current control to be re-created with the new style.
void CListHdrDlg::RenewListCtrl(DWORD dwStyle, BOOL bSetBits)
{
  DWORD   dwStyleOld, dwExtStyles;
  CRect   rect;

  dwStyleOld = GetWindowLong(m_listctrl.m_hWnd, GWL_STYLE);
  if (bSetBits)
    dwStyleOld |= dwStyle;    // turn on bits specified by caller.
  else
    dwStyleOld &= ~dwStyle;  // turn off bits specified by caller.

  m_listctrl.GetWindowRect(&rect);
  ScreenToClient(&rect);
  dwExtStyles= m_listctrl.GetExtendedStyle(); //save extended styles
  m_listctrl.DestroyWindow();
  m_listctrl.Create(dwStyleOld, rect, this, IDC_LISTVIEW1);
  m_listctrl.ModifyStyleEx(0,WS_EX_CLIENTEDGE); // renew the 3D border
                                                // of the control
  m_listctrl.SetExtendedStyle(dwExtStyles);

  FillListCtrl(); // repopulate with a new item group

  // work areas are not carried over with the new control
  m_listctrl.SetWorkAreas(0, NULL);
  m_bWorkAreas= FALSE;
  UpdateData(FALSE);  // update the dialog

}

// This function is used when a requested style
// can be applied to the existing control, and demonstrated.
```

```
void CListHdrDlg::ChangeListCtrlStyle(DWORD dwStyle, BOOL bSetBits)
{
  CRect   rect;

  m_listctrl.GetWindowRect(&rect);
  ScreenToClient(&rect);
  ASSERT(dwStyle != 0); // watch out for LVS_foo DEFINITIONS which are 0.
  if (bSetBits)
    m_listctrl.ModifyStyle(0, dwStyle);
  else
    m_listctrl.ModifyStyle(dwStyle, 0);

  InvalidateRect(rect);
  UpdateData(FALSE);  // send information back to the dialog
}
```

Функция `RenewListCtrl` при помощи функции `GetWindowLong` получает текущий стиль окна и, в зависимости от состояния соответствующего флага, устанавливает или отменяет действие константы стиля, передаваемой в первом аргументе. После этого функция получает оконные координаты просмотрового окна списка и сохраняет его расширенный стиль.

Затем вызывается функция `CWnd::DestroyWindow`, уничтожающая окно, которое ассоциировано с объектом класса `CWnd`. Данная функция посылает окну соответствующие сообщения о его закрытии, перемещает из него фокус ввода, уничтожает оконное меню, очищает очередь приложений данного окна, уничтожает связанные с ним таймеры и отменяет право собственности на буфер обмена. Данная функция не уничтожает сам объект класса `CWnd`.

После этого при помощи функции `CListCtrl::Create` снова создается объект просмотрового окна списка. Первый аргумент данной функции определяет набор стилей, используемых при создании просмотрового окна списка, второй — определяет размер и положение данного элемента управления, третий является указателем на объект класса родительского окна, а четвертый — идентификатором создаваемого ресурса. Функция `CWnd::ModifyStyleEx` позволяет модифицировать расширенные стили окна. Первый ее аргумент определяет стили, которые необходимо удалить, второй — стили, которые необходимо добавить, а третий аргумент определяет флаги, передаваемые функции `SetWindowPos`.

При помощи вызова этой функции в данном элементе управления устанавливается объемная рамка. После этого вызывается функция `SetExtendedStyle`, восстанавливающая остальные расширенные стили. Затем вызывается уже описанная нами функция `FillListCtrl`, являющаяся членом данного класса и заполняющая окно элементами данных.

Функция `CListCtrl::SetWorkAreas` устанавливает рабочие области элемента управления. Первый аргумент данной функции указывает на размерность

второго аргумента, представляющего собой указатель на массив объектов структуры RECT, содержащих координаты рабочих областей просмотрового окна списка.

Последним оператором функции RenewListCtrl является вызов функции UpdateData с аргументом, имеющим значение FALSE, обновляющей содержимое элементов управления, но не считывающей его.

Функция ChangeListCtrlStyle получает оконные координаты просмотрового окна списка, проверяет, что передаваемая константа стиля не равна нулю, что, впрочем, излишне, и, в зависимости от состояния соответствующего флага, устанавливает или отменяет действие константы стиля, переданной в первом аргументе функции. Для этого используется функция CWnd::ModifyStyle, работающая аналогично уже рассмотренной нами функции CWnd::ModifyStyleEx, только в данном случае модифицируются не расширенные, а основные стили. Затем вызывается функция CWnd::InvalidateRect, обновляющая информацию в указанном ей в первом аргументе прямоугольника, расположенном в рабочей области окна. Эта функция будет подробно рассмотрена в главе 6. Последним оператором функции ChangeListCtrlStyle является вызов функции UpdateData с аргументом, имеющим значение FALSE.

См. также главу 6

Функция OnChangeView обрабатывает сообщение об изменении содержимого текстового поля раскрывающегося списка **View Mode**. Текст этой функции приведен ниже.

```
void CListHdrDlg::OnChangeView()
{

  long        lStyle, lStyleOld;
  BOOL        bReport, bIconic;

  // retrieve view mode selection
  UpdateData(TRUE);
  if (m_strViewMode == _T("ICON"))
    lStyle = LVS_ICON;
  else if (m_strViewMode == _T("SMALL ICON"))
    lStyle = LVS_SMALLICON;
  else if (m_strViewMode == _T("REPORT"))
    lStyle = LVS_REPORT;
  else
  {
    ASSERT(m_strViewMode == _T("LIST"));
    lStyle = LVS_LIST;
  }
```

```
  // update interdependent styles, depending on view mode choice
  bReport = lStyle == LVS_REPORT;
  bIconic = lStyle == LVS_ICON || lStyle == LVS_SMALLICON;
  GetDlgItem(IDC_NOSORTHEADER)->EnableWindow(bReport);
  GetDlgItem(IDC_NOCOLUMNHEADER)->EnableWindow(bReport);
  GetDlgItem(IDC_WORKAREAS)->EnableWindow(bIconic);
  GetDlgItem(IDC_ALIGN)->EnableWindow(bIconic);
  GetDlgItem(IDC_AUTOARRANGE)->EnableWindow(bIconic);

  lStyleOld = GetWindowLong(m_listctrl.m_hWnd, GWL_STYLE);
  lStyleOld &= ~(LVS_TYPEMASK); // turn off all the style (view mode) bits
  lStyleOld |= lStyle;          // Set the new style for the control
  SetWindowLong(m_listctrl.m_hWnd, GWL_STYLE, lStyleOld);

  // In order to change the alignment, the control must be recreated.
  // However, changing the view mode of a list control does not force
  // a re-creation.
  // Therefore, if the list view is in iconic mode, the control must
  // be recreated to carry over the current alignment mode.
  if(bIconic)    // need to keep alignment also
    RenewListCtrl(0, 0);
}
```

Данная функция начинает свою работу с обновления информации о содержимом элементов управления диалогового окна. Далее следует группа условных операторов, проверяющих допустимость содержимого переменной `m_strViewMode` и, в случае положительного ответа, устанавливающих значение стиля, соответствующее содержимому этой переменной.

Затем устанавливаются значения двух логических переменных, первая из которых определяет, является ли текущий режим режимом отображения отчета, а вторая — является ли текущий режим одним из режимов отображения значков. Эти переменные используются для установки режима доступа к флажкам диалогового окна. Способ установки этого режима уже был описан при рассмотрении функции `OnInitDialog`.

После этого функция `GetWindowLong` возвращает старый стиль окна, в нем сбрасываются все биты стиля и устанавливаются биты нового стиля. Далее при помощи функции `SetWindowLong` устанавливается новый стиль элемента управления. Эта функция изменяет атрибуты окна, дескриптор которого указан в качестве ее первого аргумента. Второй аргумент данной функции имеет то же назначение, что и второй аргумент функции `GetWindowLong`. В третьем ее аргументе содержится величина, которую следует поместить в определенную вторым аргументом позицию.

Чтобы изменить выравнивание элементов данных в просмотровом окне списка, данный элемент управления следует создать заново. Однако изменение режима просмотра в данном окне к этому не приводит. Новые пара-

метры выравнивания вступят в силу, если изменяемый элемент управления, находящийся в режиме отображения значков, будет создан заново, для чего следует вызвать функцию `RenewListCtrl` — член данного класса, описанную нами ранее.

Функция `OnChangeAlign` имеет простейшую структуру. Вызывая функцию `UpdateData`, она обновляет информацию, содержащуюся в элементах управления окна, проверяет корректность содержимого переменной `m_strAlignMode` и вызывает функцию `RenewListCtrl` — член данного класса, описанную нами выше. Текст функции `OnChangeAlign` приведен ниже:

```
void CListHdrDlg::OnChangeAlign()
{
  // This function takes into consideration that LVS_ALIGNTOP
  //is defined as zero.
  UpdateData(TRUE);
  ASSERT(m_strAlignMode == _T("ALIGNTOP") || m_strAlignMode ==
                                                  _T("ALIGNLEFT"));
  RenewListCtrl(LVS_ALIGNLEFT, (m_strAlignMode == _T("ALIGNLEFT")));
}
```

Функция `OnHover`, текст которой приведен ниже, обрабатывает сообщение об изменении состояния флажка **Hover Select**.

```
void CListHdrDlg::OnHover()
{
  UpdateData(TRUE);

  if(m_bHoverSelect) // "hover selection" is enabled
  {
  // LVS_EX_ONECLICKACTIVATE (or LVS_EX_TWOCLICKACTIVATE must also be
  // set before  "hover selection" is enabled)
    m_listctrl.SetExtendedStyle(LVS_EX_TRACKSELECT |
      LVS_EX_ONECLICKACTIVATE);
    GetDlgItem(IDC_STDHOTCUR)->EnableWindow(m_bHoverSelect);
    GetDlgItem(IDC_MYHOTCUR)->EnableWindow(m_bHoverSelect);
  }
  else // "hover selection" is disabled
  {
    m_listctrl.SetExtendedStyle(0);
    GetDlgItem(IDC_STDHOTCUR)->EnableWindow(m_bHoverSelect);
    GetDlgItem(IDC_MYHOTCUR)->EnableWindow(m_bHoverSelect);
  }
}
```

Данная функция обновляет информацию о состоянии элементов управления диалогового окна и, в зависимости от содержимого переменной `m_bHoverSelect`, устанавливает или отменяет режим выделения "на лету" (Hover Selection).

Чтобы установить режим выделения "на лету" с помощью функции CListCtrl::SetExtendedStyle, описанной выше, для просмотрового окна списка устанавливается комбинация расширенных стилей LVS_EX_TRACKSELECT и LVS_EX_ONECLICKACTIVATE, первый из которых разрешает использование выделения "на лету", а второй указывает, что при щелчке пользователя по элементу данных родительскому окну посылается сообщение LVN_ITEMACTIVATE, а при перемещении курсора мыши над значком он подсвечивается, но не выделяется. Для отмены режима выделения "на лету", вызывается та же самая функция с нулевым значением аргумента.

За вызовом функции CListCtrl::SetExtendedStyle в обеих ветвях условного оператора следует набор из двух операторов, который спокойно можно было бы вынести за его пределы. Эти операторы устанавливают состояние соответствующих элементов управления.

Программа установки пользовательского курсора OnStdHotCur состоит всего из двух операторов:

- Вызова функции CListCtrl::SetHotCursor, единственным аргументом которой является объект структуры HCURSOR, сформированный в функции OnInitDialog
- Оператора присваивания переменной m_bHotCursor значения FALSE

Функция OnChangeSort, текст которой представлен ниже, предназначена для обработки сообщения о внесении изменений в текстовое поле списка **Sort**.

```
void CListHdrDlg::OnChangeSort()
{
  long    lStyle;

  UpdateData(TRUE);
  lStyle = GetWindowLong(m_listctrl.m_hWnd, GWL_STYLE);
  lStyle &= ~(LVS_SORTASCENDING | LVS_SORTDESCENDING);
  SetWindowLong(m_listctrl.m_hWnd, GWL_STYLE, lStyle); // set style
                                                       // without sorting
  if (m_strSortMode == _T("None"))
    RenewListCtrl(LVS_SORTASCENDING | LVS_SORTDESCENDING, FALSE);
  else if (m_strSortMode == _T("ASCENDING"))
    RenewListCtrl(LVS_SORTASCENDING, TRUE);
  else
  {
    ASSERT(m_strSortMode == _T("DESCENDING"));
    RenewListCtrl(LVS_SORTDESCENDING, TRUE);
  }
}
```

После обновления информации о состоянии элементов управления диалогового окна, удаления из него информации о сортировке и применения об-

новленного стиля к элементу управления производится проверка содержимого соответствующего текстового поля на допустимость и, в случае благополучного исхода проверки, производится соответствующее обновление элемента управления с использованием функции `RenewListCtrl` — члена данного класса. Необходимость очистки стиля вызвана тем, что несовместимые флаги сортировки по возрастанию и убыванию могут быть одновременно установлены в стиле. Данная операция позволяет избежать подобной ситуации.

Функция `OnWorkAreas`, текст которой представлен ниже, предназначена для обработки сообщения об изменении состояния флажка **Working Areas**.

```
void CListHdrDlg::OnWorkAreas()
{
  UpdateData(TRUE);

  if(m_bWorkAreas)
  {
    CRect curRect;
    CSize size;

    size= m_listctrl.ApproximateViewRect();
    size.cx+= 100;
    size.cy+= 100;

/*      Work area layout

    ____________________
    |          |         |
    |          |         |
    |   0      |    1    |
    |          |         |
    |----------|---------|
    |          |         |
    |   2      |    3    |
    |          |         |
    |__________|_________|*/

    CRect rcWorkAreas[4];

    rcWorkAreas[0].SetRect(0, 0, (size.cx / 2) - 5, (size.cy / 2) - 5);
    rcWorkAreas[1].SetRect((size.cx / 2) + 5, 0, size.cx,
                                                 (size.cy / 2) - 5);
    rcWorkAreas[2].SetRect(0, (size.cy / 2) + 5, (size.cx / 2) - 5,
                                                  size.cy);
    rcWorkAreas[3].SetRect((size.cx / 2) + 5, (size.cy / 2) + 5,
                                                  size.cx, size.cy);

    //set work areas
    m_listctrl.SetWorkAreas(4, rcWorkAreas);
```

```
    // set insertion points for each work area
    CPoint  rgptWork[4];
    for (int i = 0; i < 4; i++)
    {
      rgptWork[i].x = rcWorkAreas[i].left + 10;
      rgptWork[i].y = rcWorkAreas[i].top + 10;
    }
    // now move all the items to the different quadrants
    for (i = 0; i < 20; i++)
      m_listctrl.SetItemPosition(i, rgptWork[i % 4]);
  }
  else    //delete all work areas and reorganize items
  {
    m_listctrl.SetWorkAreas(0, NULL);

    CPoint pt(0, 0);
    for (int i = 0; i < 20; i++)
      m_listctrl.SetItemPosition(i, pt);  // move all the items
                                          // to the origin
  }
  m_listctrl.Arrange(LVA_DEFAULT); // force the control to re-arrange the
                                   // shuffled items
}
```

После обновления информации о состоянии элементов управления данная функция проверяет состояние соответствующего флажка и, если он установлен, вызывает функцию `CListCtrl::ApproximateViewRect`, определяющую приблизительные размеры рабочей области элемента управления, необходимые для размещения заданного числа элементов данных. Первым аргументом данной функции являются предполагаемые размеры рабочей области, если они не заданы, функция использует текущие размеры рабочей области элемента управления. Вторым ее аргументом является число размещаемых элементов данных. Если оно не задано, то функция использует общее число элементов данных, включенное на данный момент в элемент управления. После этого полученные размеры увеличиваются на 100 элементов изображения по каждой координате.

Затем создается массив из четырех объектов класса `CRect` и с помощью функции `CRect::SetRect` устанавливаются размеры четырех прямоугольных областей, расположенных в соответствии с приведенной в тексте функции диаграммой. После чего вызывается функция `CListCtrl::SetWorkAreas`, устанавливающая рабочие области просмотрового окна списка и рассмотренная нами вместе с функцией `RenewListCtrl`.

После установки рабочих областей создается массив из четырех объектов класса `CPoint`, в который заносятся точки вставки элементов данных, расположенные на десять элементов изображения ниже верхней границы рабочей

области и на столько же элементов изображения левее ее левой границы. Далее в эти точки помещаются элементы данных просмотрового окна списка, причем заполнение идет циклически по одному элементу в каждую рабочую область. При этом используется функция `CListCtrl::SetItemPosition`, описанная нами при рассмотрении функции `CMyListCtrl::OnButtonUp`.

Если флажок **Working Areas** сброшен, то выполняется очистка всех рабочих областей с использованием функции `CListCtrl::SetWorkAreas`, после чего всем элементам данных сопоставляется одна и та же позиция в элементе управления — его левый верхний угол.

Функция `OnStdHotCur` в основном аналогична функции `OnMyHotCur`, только вместо идентификатора пользовательского курсора используется идентификатор стандартного курсора, а переменной `m_bHotCursor` присваивается значение `TRUE`.

На этом можно считать, что описание тестового приложения, демонстрирующего возможности класса просмотрового окна списка, закончено. Однако, если вы считаете, что прочитав это описание, вы имеете достаточно информации, чтобы использовать его в серьезных программах, запустите приложение на исполнение, выделите в режиме отображения отчетов любую метку элемента данных и медленно переместите ее вниз. Если у вас получится что-нибудь более достойное, чем диалоговое окно, изображенное на рис. 4.34, я вас поздравляю. В противном случае задайте себе вопрос: "Если разработчики Visual C++ не смогли написать на нем простейшего тестового приложения без грубейших ошибок, то чего можно требовать от простых программистов?"

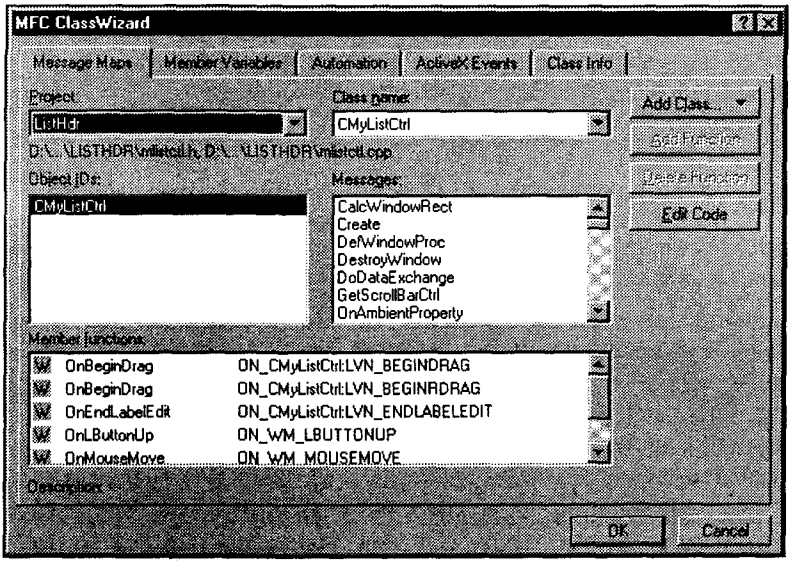


Рис. 4.34. Демонстрационное приложение в работе

Сообщения и команды

В предыдущих главах вам часто встречалось выражение "функция обработки сообщения". Что же понимается под сообщением, и почему его нужно обрабатывать? Сообщения являются основной отличительной особенностью операционной системы Windows от операционной системы DOS.

Для того чтобы иметь возможность работать с каким-либо устройством, например, с клавиатурой или мышью, программам DOS приходилось отслеживать состояние этих устройств и ожидать их реакции на посланные им сообщения. В Windows картина кардинально меняется. Данная система управляется сообщениями, и уже не программа ожидает реакции от устройства, а сообщение о реакции устройства запускает ту или иную программу. Та часть программы, которая запускается в ответ на конкретное сообщение, называется функцией его обработки.

Большую часть работы по передаче сообщений и вызову соответствующих функций обработки берут на себя внутренние процедуры Windows и функции библиотеки MFC. Однако поскольку в большинстве случаев посылка сообщения является единственной возможностью обеспечить интерфейс между отдельными частями программы пользователя, прежде чем приступить к детальному рассмотрению различных программ, необходимо глубже познакомиться с механизмом работы с сообщениями.

Обработка сообщений

Для идентификации сообщений операционная система использует целые числа. Однако для удобства пользователей вместо этих чисел используются символьные идентификаторы. Для установления соответствия между идентификатором и соответствующей ему числовой величиной используется огромное количество директив `#define`. Незначительное количество этих директив, а именно те, которые относятся к сообщениям и ресурсам поль-

зователя, расположены в заголовочном файле Resource.h, имеющемся в каждом приложении. Сообщения в Windows имеют префикс `WM_`, что означает сообщение Windows (Windows Message).

В любом сообщении содержится информация о том, какому окну оно предназначено. Кроме того, любое сообщение может иметь до двух аргументов, в которых могут передаваться различные данные (в том числе и указатели на большие структуры данных).

Обработка различных сообщений выполняется различными компонентами операционной системы, причем некоторые из них включают в себя собственные функции обработки данного сообщения. Например, при перемещении мыши необходимо не только сообщить об этом пользователю и передать ее новые координаты, но и нарисовать курсор на новом месте. Поэтому многие функции обработки сообщений включают в себя вызов соответствующего метода базового класса.

Библиотека MFC берет на себя обработку большинства сообщений низкого уровня, таких как `WM_CHAR`, `WM_LBUTTONDOWN` и других, и позволяет пользователю получать сообщения о том, что в данное текстовое поле введен текст или что нажата кнопка **OK**. До появления этой библиотеки пользователям приходилось самостоятельно отслеживать большинство сообщений низкого уровня, что негативно сказывалось на объеме программы и ее читаемости.

Каждая программа Windows имеет свой *цикл обработки сообщений* (Message Loop), который практически всегда находится в функции `WinMain`. Библиотека MFC скрыла эту функцию в своих недрах, и теперь она не появляется в текстах пользовательских приложений.

Обычно цикл обработки сообщений реализуется с использованием оператора `while`, внутри которого вызывается функция `GetMessage`, отыскивающая сообщение в очереди сообщений данного потока и помещающая его в объект соответствующей структуры. Эта функция позволяет как отыскивать сообщения, направленные данному окну, так и сообщения от потоков направленные функцией `PostThreadMessage`. При поиске сообщений производится проверка на нахождение их в определенном диапазоне значений. Данная функция не может отыскивать сообщения, направленные другим окнам или приложениям. Если найдено любое другое сообщение, кроме `WM_QUIT`, данная функция возвращает ненулевое значение. В противном случае она возвращает ноль. В случае ошибки возвращаемое значение равно −1, поэтому при построении цикла обработки сообщений необходимо проверять выходную величину не только на ненулевое значение, но и на отсутствие ошибки.

Следующей функцией, вызываемой в цикле обработки сообщений, является `TranslateMessage`. Она переводит сообщения, поступающие от клавиатуры, в сообщения о введенных символах. Эти сообщения помещаются в *очередь сообщений* и впоследствии считываются функциями `GetMessage` и `PeekMessage`. Единственным аргументом данной функции является указатель на структуру, содержащую сообщение.

Последней функцией, вызываемой в стандартном цикле обработки сообщений, является функция DispatchMessage, передающая сообщение, содержащееся в структуре, на которую указывает ее единственный аргумент, процедуре окна.

Функция процедуры окна обычно представляет собой огромный оператор switch, перечисляющий все возможные сообщения, каждому из которых сопоставлен вызов соответствующей функции обработки. Естественно, что работать с таким оператором было бы очень сложно, поэтому в библиотеке MFC предусмотрено использование карты сообщений.

Карта сообщений

Использование *карты сообщений* (Message Map) является основным подходом библиотеки MFC к вопросу обработки сообщений. Программист не должен писать функцию процедуры окна и указывать в ней все возможные сообщения, библиотека MFC использует стандартный набор макросов, позволяющий определить для каждого класса его вклад в данную функцию, формируемую самой библиотекой на основе предоставленной пользователем информации. Кроме того, мастер ClassWizard освобождает пользователя даже от необходимости составления карты сообщений, — она формируется автоматически.

Каждый класс, производный от класса CCmdTarget, должен включать в себя карту сообщений для обработки поступающих ему сообщений. Карта сообщений состоит из двух частей, одна из которых расположена в файле заголовка класса, а вторая — в файле реализации. Первая часть карты сообщений, располагаемая в файле заголовка, должна заканчиваться макросом DECLARE_MESSAGE_MAP, не имеющим аргументов. Вторая часть карты сообщений, располагаемая в файле реализации, должна начинаться макросом BEGIN_MESSAGE_MAP, первым аргументом которого является имя класса, сообщения которого будут обрабатываться данной картой, а вторым аргументом — имя базового класса. Вторая часть карты сообщений должна заканчиваться макросом END_MESSAGE_MAP, не имеющим аргументов. В листинге 5.1 представлена карта сообщений заголовка класса диалогового окна тестового приложения, демонстрирующего реализацию простейших элементов управления, описанного в главе 4.

См. также главу 4

Листинг 5.1. Карта сообщений, располагаемая в заголовке класса

```
// Generated message map functions
//{{AFX_MSG(CDialogApplicationDlg)
virtual BOOL OnInitDialog();
```

```
afx_msg void OnSysCommand(UINT nID, LPARAM lParam);
afx_msg void OnPaint();
afx_msg HCURSOR OnQueryDragIcon();
afx_msg void OnEditSwitch();
afx_msg void OnSpinSwitch();
afx_msg void OnDeltaposSpin(NMHDR* pNMHDR, LRESULT* pResult);
afx_msg void OnSelendokCombo();
//}}AFX_MSG
DECLARE_MESSAGE_MAP()
```

Специальным образом оформленный комментарий позволяет мастеру ClassWizard определить местоположение карты сообщений в описании класса и вносить в него соответствующие изменения при выборе дополнительных функций обработки сообщений в панели **Message Maps** диалогового окна **MFC ClassWizard**.

Вторая часть карты сообщений расположена в файле реализации данного класса. В листинге 5.2 представлена вторая часть карты сообщений из файла реализации класса, первая часть которой приведена в листинге 5.1.

Листинг 5.2. Карта сообщений, располагаемая в файле реализации класса

```
BEGIN_MESSAGE_MAP(CDialogApplicationDlg, CDialog)
  //{{AFX_MSG_MAP(CDialogApplicationDlg)
  ON_WM_SYSCOMMAND()
  ON_WM_PAINT()
  ON_WM_QUERYDRAGICON()
  ON_BN_CLICKED(IDC_EDIT_SWITCH, OnEditSwitch)
  ON_BN_CLICKED(IDC_SPIN_SWITCH, OnSpinSwitch)
  ON_NOTIFY(UDN_DELTAPOS, IDC_SPIN, OnDeltaposSpin)
  ON_CBN_SELENDOK(IDC_COMBO, OnSelendokCombo)
  //}}AFX_MSG_MAP
END_MESSAGE_MAP()
```

Поскольку карту сообщений формирует мастер ClassWizard, пользователю нет особой необходимости разбираться в ее макросах, поэтому ниже приводится далеко не полный список макросов, используемых в карте сообщений:

- `ON_COMMAND` — определяет функцию, которая будет обрабатывать сообщение `WM_COMMAND`, обычно посылаемое из меню или из панели инструментов. Данный макрос имеет два аргумента, первым из которых является идентификатор команды, а вторым — имя функции, обрабатывающей данное сообщение
- `ON_COMMAND_RANGE` — работает аналогично макросу `ON_COMMAND`, однако сопоставляет функции обработки не одно сообщение, а целый диапазон

сообщений. Данный макрос имеет три аргумента, первый из которых определяет нижнюю границу диапазона идентификаторов команд, второй — верхнюю границу диапазона, а третий — функции обработки команд данного диапазона. Данный макрос не поддерживается мастером ClassWizard, поэтому все необходимые действия по включению его в программу пользователю придется произвести самостоятельно. Одному сообщению, независимо от того, входит ли оно в заданный диапазон или для него предусмотрен отдельный макрос, должна соответствовать только одна функция его обработки

- ON_UPDATE_COMMAND_UI — определяет функцию, которая будет обрабатывать сообщение об обновлении команды интерфейса пользователя. Данное сообщение будет выдаваться каждый раз, когда приложению понадобиться определить состояние элементов пользовательского интерфейса при их отображении на экране. Данный макрос имеет два аргумента, первым из которых является идентификатор сообщения, а вторым — имя функции, обрабатывающей данное сообщение
- ON_UPDATE_COMMAND_UI_RANGE — работает аналогично макросу ON_UPDATE_COMMAND_UI, однако определяет состояние не одного, а целого диапазона элементов интерфейса пользователя. Так же, как и ON_COMMAND_RANGE, данный макрос имеет три аргумента аналогичного назначения
- ON_CONTROL — определяет функцию обработки извещения от пользовательского элемента управления. Извещения от пользовательского элемента управления направляются его родительскому окну. Каждому подобному извещению должен соответствовать один макрос ON_CONTROL. Данный макрос имеет три аргумента, первый из которых определяет код извещения элемента управления, второй — идентификатор команды, а третий — функцию обработки сообщения для данной команды
- ON_CONTROL_RANGE — используется для задания функции обработки сообщений диапазону идентификаторов команд при едином для всех идентификаторов диапазона коде извещения элемента управления, например, BN_CLICKED. Данный макрос имеет четыре аргумента, первый и последний из которых совпадают с соответствующими аргументами макроса ON_CONTROL, второй определяет нижнюю границу диапазона идентификаторов, а третий — верхнюю. Данный макрос не поддерживается мастером ClassWizard, поэтому все необходимые действия по включению его в программу пользователю придется произвести самостоятельно
- ON_MESSAGE — определяет функцию обработки сообщения пользователя. Обычно сообщения пользователи находятся в диапазоне от WM_USER до 0x7FFF, и под ними понимаются любые сообщения, кроме стандартного сообщения Windows WM_MESSAGE. Каждому сообщению пользователя должен соответствовать свой макрос ON_MESSAGE. Данный макрос имеет два аргумента, первым из которых является идентификатор сообщения, а вторым — имя функции, обрабатывающей данное сообщение

Сообщения в Windows 3.x

В Windows 3.x элементы управления посылали своим родительским окнам извещения о таких событиях, как щелчки кнопки мыши, изменение содержимого полей и о выделении объектов. С помощью механизма сообщений осуществлялась также фоновая перерисовка содержимого окон. Простейшие извещения посылались как специальные сообщения формата WM_COMMAND, содержащего код извещения (например, BN_CLICKED), а также идентификатор элемента управления, передаваемый в аргументе wParam данного сообщения, и дескриптор элемента управления, передаваемый в аргументе lParam сообщения. Поскольку в данном формате сообщения используются оба его аргумента, то в нем не может быть передана никакая дополнительная информация. Поэтому данное сообщение может служить только для извещения о событии. Например, при передаче кода извещения BN_CLICKED нет никакой возможности передать информацию о положении курсора мыши в момент щелчка.

При необходимости передачи дополнительной информации в извещении при работе в Windows 3.x приходилось использовать множество специальных сообщений, таких как WM_CTLCOLOR, WM_VSCROLL, WM_HSCROLL, WM_DRAWITEM, WM_MEASUREITEM, WM_COMPAREITEM, WM_DELETEITEM, WM_CHARTOITEM, WM_VKEYTOITEM и других подобных сообщений. Эти сообщения могли возвращаться родительским окном тому сообщению, которое их послало.

Сообщения в Win32

Для элементов управления, унаследованных от Windows 3.1, Win32 API в большинстве случаев использует те же специальные сообщения, что и Windows 3.x. Однако в Win32 определены также достаточно сложные элементы управления, которых не было в Windows 3.x. В большинстве случаев при работе с данными элементами управления возникает необходимость в передаче дополнительных данных вместе с извещением о событии. Вместо того, чтобы создавать для каждого нового типа данных, который необходимо передать вместе с извещением о событии, новое сообщение формата WM_*, разработчики Win32 API ввели только одно новое сообщение данного формата, а именно WM_NOTIFY, позволяющее передавать любое количество дополнительной информации стандартным образом.

Сообщение WM_NOTIFY содержит идентификатор элемента управления, посылающего данное сообщение, в аргументе wParam и указатель на объект некоторой структуры данных в аргументе lParam. Этот объект может являться или объектом структуры NMHDR или объектом другой структуры, первым элементом которой является объект структуры NMHDR. Поскольку объект структуры NMHDR является первым элементом структуры, то указатель на объект любой структуры, передаваемый в аргументе lParam сообщения, может рассматриваться как указатель на структуру NMHDR.

В большинстве случаев объект структуры NMHDR является только первым элементом большей структуры, при обращении к которой необходимо применять квалифицированную ссылку (то есть ссылку с указанием типа переменной). К немногим исключениям из данного правила относятся простейшие извещения (имена которых начинаются с NM_) и извещения о вызове справки по элементу управления TTN_SHOW и TTN_POP, использующие указатель на объект структуры NMHDR.

Структура NMHDR содержит дескриптор и идентификатор элемента управления, пославшего данное сообщение, и код извещения (например, TTN_SHOW). Данная структура имеет следующий формат:

```
typedef struct tagNMHDR {
    HWND hwndFrom;
    UINT idFrom;
    UINT code;
} NMHDR;
```

Для сообщения TTN_SHOW переменная code должна содержать код извещения TTN_SHOW.

Рассмотрим в качестве примера структуры, первым элементом которой является объект структуры NMHDR, структуру, используемую сообщением LVN_KEYDOWN просмотрового окна списка. Структура, на которую ссылается указатель в сообщении LV_KEYDOWN имеет следующий формат:

```
typedef struct tagLV_KEYDOWN {
    NMHDR hdr;
    WORD wVKey;
    UINT flags;
} LV_KEYDOWN;
```

Сообщения, посылаемые всеми новыми элементами управления Windows

Имеется ряд сообщений, посылаемых всеми новыми элементами управления Windows. Их отличительной особенностью является то, что все они используют указатель непосредственно на объект структуры NMHDR. К подобным сообщениям относятся:

- NM_CLICK — сообщение о том, что пользователь щелкнул левой кнопкой мыши в рабочей области элемента управления
- NM_DBLCLK — сообщение о том, что пользователь произвел двойной щелчок левой кнопкой мыши в рабочей области элемента управления
- NM_RCLICK — сообщение о том, что пользователь щелкнул правой кнопкой мыши в рабочей области элемента управления

- NM_RDBLCLK — сообщение о том, что пользователь дважды щелкнул правой кнопкой мыши в рабочей области элемента управления
- NM_RETURN — сообщение о том, что пользователь нажал клавишу <Return>, когда фокус ввода принадлежал данному элементу управления
- NM_SETFOCUS — сообщение о том, что данный элемент управления получил фокус ввода
- NM_KILLFOCUS — сообщение о том, что данный элемент управления потерял фокус ввода
- NM_OUTOFMEMORY — сообщение о том, что данный элемент управления не может завершить операцию вследствие недостатка оперативной памяти

Обработка извещений

Обработку извещений осуществляет функция CWnd::OnNotify. По умолчанию она просматривает карты сообщений для определения функций обработки поступивших в нее извещений. Обычно эта функция не перегружается. Вместо этого в карту сообщений класса окна добавляются функции обработки новых извещений.

Использование мастера ClassWizard позволяет не только ввести соответствующие макросы ON_NOTIFY для обработки сообщения, но и создать для пользователя заготовки функций обработки этих сообщений. Макрос ON_NOTIFY имеет три аргумента, первым из которых является код извещения, которое следует обработать, например LVN_KEYDOWN, вторым — идентификатор элемента управления, пославшего данное извещение, а третьим — функция обработки данного сообщения.

Функция обработки сообщения должна иметь следующий формат:

```
afx_msg void memberFxn( NMHDR * pNotifyStruct, LRESULT * result),
```

где:

- memberFxn — имя функции обработки сообщения
- pNotifyStruct — указатель на структуру извещения, описанную выше
- result — указатель на код результата, устанавливаемый пользователем перед выходом из процедуры

Например, чтобы указать, что функция OnKeydownList1 должна обрабатывать сообщение LVN_KEYDOWN, поступающее от объекта класса CListCtrl с идентификатором IDC_LIST1, мастер ClassWizard добавляет в карту сообщений следующий макрос:

```
ON_NOTIFY( LVN_KEYDOWN, IDC_LIST1, OnKeydownList1)
```

а в раздел реализации соответствующего класса — следующую заготовку функции реализации данного сообщения:

```
void CMessageReflectionDlg::OnKeydownList1(NMHDR* pNMHDR, LRESULT* pResult)
{
    LV_KEYDOWN* pLVKeyDow = (LV_KEYDOWN*)pNMHDR;
    // TODO: Add your control notification handler code here

    *pResult = 0;
}
```

Обратите внимание, что мастер ClassWizard автоматически обеспечивает соответствие классов используемых переменных. Для доступа к объекту класса LV_KEYDOWN используется указатель на этот класс pLVKeyDow, получаемый из указателя pNMHDR с использованием непосредственного преобразования типов указателей.

В файле заголовка данная функция должна быть объявлена следующим образом:

```
afx_msg void memberFxn(UINT id, NMHDR * pNotifyStruct, LRESULT * result)
```

Если необходимо, чтобы данное сообщение могло обрабатываться не одним, а несколькими объектами, участвующими в обработке сообщений, используйте вместо макроса ON_NOTIFY (или ON_NOTIFY_RANGE) макрос ON_NOTIFY_EX (или ON_NOTIFY_EX_RANGE). Единственным отличием расширенной версии макроса является то, что макросы с окончанием EX возвращают величину типа BOOL, указывающую, нужно ли продолжать обработку данного сообщения на следующих уровнях его обработки. Если возвращаемая величина имеет значение FALSE, то данное сообщение может быть обработано на следующих уровнях.

Мастер ClassWizard не поддерживает работу с макросами ON_NOTIFY_EX или ON_NOTIFY_EX_RANGE, поэтому вся работа по заполнению обеих частей карты сообщений ложится в данном случае на пользователя.

Макросы, помещаемые в карту сообщений, имеют следующий вид:

```
ON_NOTIFY_EX( nCode, id, memberFxn)
ON_NOTIFY_EX_RANGE( wNotifyCode, id, idLast, memberFxn)
```

Назначение их аргументов полностью совпадает с назначением аргументов обычных версий данных макросов. В файле соответствующая функция обработки сообщения должна быть объявлена следующим образом:

```
afx_msg BOOL memberFxn( UINT id, NMHDR * pNotifyStruct, LRESULT * result);
```

В обоих случаях аргумент id содержит идентификатор элемента управления, пославшего данное сообщение. Данная функция должна возвращать значение TRUE, если обработка данного сообщения полностью завершена, или FALSE, если предполагается дальнейшая обработка данного сообщения на следующих уровнях.

Обработка отраженных сообщений

В главе 4 мы уже кратко рассматривали вопрос об отраженных сообщениях. Остановимся на нем подробнее. Отражение сообщений является новой особенностью библиотеки MFC 4.0. Необходимость в подобных сообщениях связана с тем, что при работе с элементами управления часто возникает необходимость послать извещение родительскому окну. Например, многие элементы управления посылают своему родительскому окну извещение WM_CTLCOLOR или одну из его разновидностей, чтобы передать ему свой запрос на кисть для заполнения фона рабочей области данного элемента управления.

В Windows и в библиотеках MFC более ранних версий, чем версия 4.0, каждое родительское окно, в котором используется данный элемент управления, должно включать в себя функцию обработки данного сообщения. Таких классов родительских окон, содержащих практически идентичные по тексту функции обработки, могло набраться очень много. Так, в предыдущем примере, каждое диалоговое окно, использующее элементы управления с цветом фона, определенным пользователем, должно было содержать функцию обработки запроса на кисть для заполнения фона. Намного проще было бы создать класс элемента управления, который сам мог бы работать с цветом фона рабочей области своего окна.

В библиотеке MFC 4.0 и более поздних ее версиях продолжает использоваться старый механизм обработки извещений, по которому обязанность их обработки возложена на класс родительского окна. Но кроме того, данная библиотека допускает повторное использование программного кода за счет применения механизма, называемого *обработкой отраженного сообщения* (message reflection), позволяющего обрабатывать извещения в классе элемента управления, в классе родительского окна или одновременно в обоих классах. В нашем примере с цветом фона рабочей области окна элемента управления обработка сообщения WM_CTLCOLOR может быть реализована методами самого класса элемента управления, без внесения каких-либо изменений в класс его родительского окна. (Следует, однако, отметить, что поскольку механизм отражения сообщений реализован средствами библиотеки MFC, а не средствами Windows, класс родительского окна элемента управления, в котором реализован механизм обработки отраженного сообщения, должен быть производным от класса CWnd.)

Для достижения сходного эффекта в ранних версиях библиотеки MFC использовались виртуальные функции для обработки некоторых сообщений, таких как сообщения для созданных пользователем окон списков (WM_DRAWITEM и другие). Однако механизм обработки отраженного сообщения представляет собой единую и стройную систему. Процедуры, реализующие этот механизм, могут быть использованы и в программах, написанных с применением ранних версий библиотеки MFC.

Если в классе родительского окна предусмотрена обработка данного извещения или диапазона извещений, то эти функции обработки сообщений перегружают функции класса элемента управления и механизм отражения сообщений перестает работать для данного класса родительского окна.

При обработке сообщения или диапазона сообщений WM_NOTIFY применяется диаметрально противоположные принципы перегрузки функций обработки отраженного сообщения. В этом случае функция обработки сообщения, расположенная в классе родительского окна, будет вызвана, только если в классе элемента управления отсутствует соответствующая функция обработки отраженного сообщения, и в карте сообщений данного класса отсутствует макрос ON_NOTIFY_REFLECT. При наличии в карте сообщений класса элемента управления макроса ON_NOTIFY_REFLECT_EX, обработка соответствующего сообщения может продолжаться, а может и не продолжаться в классе родительского окна. Если функция обработки данного сообщения, расположенная в классе элемента управления, возвращает TRUE, то обработка данного сообщения будет продолжена функцией-членом класса родительского окна, если же функция-член класса элемента управления возвращает FALSE, то обработка сообщения считается законченной.

После того как элемент управления послал сообщение WM_NOTIFY, право первой обработки данного сообщения предоставляется функции-члену класса данного элемента управления. Если посылается любое другое отраженное сообщение, то право первой обработки данного сообщения предоставляется функции-члену класса родительского окна данного элемента управления. Чтобы вызвать нужную функцию обработки, класс должен включать в себя соответствующую функцию, а в карте сообщений должен находиться соответствующий макрос.

Макросы карты сообщений, обрабатывающие отраженные сообщения, имеют особый синтаксис: к обычному имени макроса добавляется окончание _REFLECT. Например, для обработки сообщения WM_NOTIFY в родительском окне в карту сообщений соответствующего класса окна добавляется макрос ON_NOTIFY, а для обработки того же отраженного сообщения в карту сообщений соответствующего класса элемента управления добавляется макрос ON_NOTIFY_REFLECT. В некоторых случаях данные макросы имеют даже различные аргументы. Однако в большинстве случаев за правильностью составления списка формальных параметров функций и макросов следит мастер ClassWizard, предоставляющий также заготовки функций обработки сообщений.

Макросы карты сообщений и заготовки функций обработки отраженных сообщений

Для обработки отраженных извещений, посылаемых классами элементов управления, используются макросы карты сообщений и заготовки функций

обработки отраженных сообщений. После создания нового класса элемента управления, использующего библиотеку MFC, можно с помощью мастера ClassWizard задать функции обработки посылаемых им сообщений. Для этого:

1. Вызовите диалоговое окно **MFC ClassWizard** и раскройте в нем вкладку **Message Maps**.
2. Раскройте раскрывающийся список **Class name:** и выберите в нем имя класса вашего элемента управления.
3. В окне списка **Object IDs:** выделите имя класса вашего элемента управления.
4. В окне списка **Messages:** выделите сообщение, для которого вы хотите задать функцию обработки. Отраженные сообщения данного класса отмечены знаком равенства (=). Сообщения, для которых уже заданы функции обработки, выделяются жирным шрифтом. При выделении сообщения в нижней части диалогового окна **MFC ClassWizard** появляется краткая справка по данному сообщению.
5. Нажмите кнопку **Add Function** (или дважды щелкните на идентификаторе сообщения). Если для данного сообщения нет стандартного имени функции обработки, появляется диалоговое окно **Add Member Function**, содержащее текстовое поле с именем функции, предлагаемым мастером ClassWizard.
6. Если вы согласны с предложенным именем функции обработки сообщения, нажмите кнопку **OK** в диалоговом окне **Add Member Function**. Появится диалоговое окно **MFC ClassWizard**, в котором идентификатор сообщения будет выделен жирным шрифтом, а имя соответствующей функции обработки будет включено в список **Member functions:**. Диалоговое окно **MFC ClassWizard** примет вид, аналогичный изображенному на рис. 5.1.
7. Выполните одно из возможных действий:
   - Повторите пункты 2—6 для задания функций обработки других сообщений
   - Нажмите кнопку **Cancel** и выйдите из диалогового окна **MFC ClassWizard** без сохранения изменений, внесенных в него в данном сеансе
   - Нажмите кнопку **OK** и выйдите из диалогового окна **MFC ClassWizard** с сохранением всех изменений, внесенных в него в данном сеансе
   - Нажмите кнопку **Edit Code** для закрытия диалогового окна **MFC ClassWizard** с сохранением всех изменений, внесенных в него в данном сеансе, и выхода в окно редактирования, содержащее выделенную в окне списка **Member functions:** функцию. При этом текстовый курсор будет установлен на строку комментария, указывающую место вставки пользовательского текста в данную функцию

При нажатии кнопок **OK** или **Edit Code** мастер ClassWizard производит следующие действия:

- Заносит в файл заголовка описания новых функций
- Заносит в файл реализации корректно функционирующие заготовки новых функций
- Заносит в карту сообщений класса новые макросы обработки сообщений

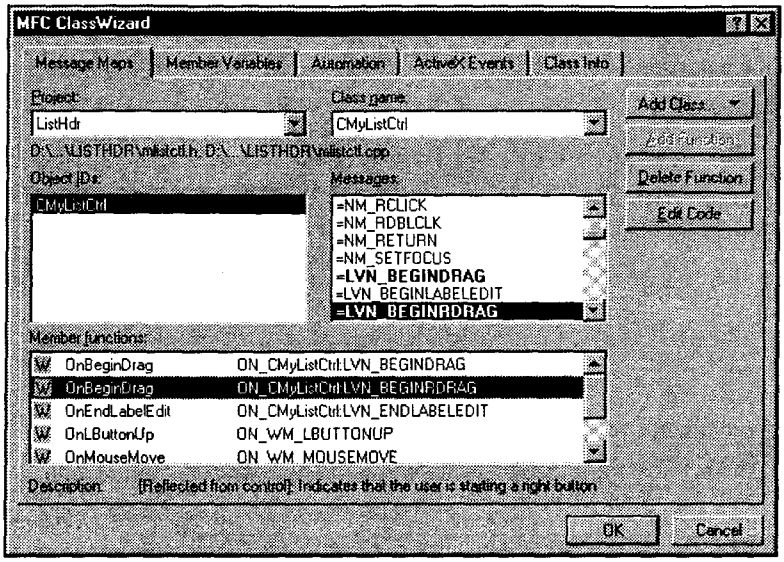


Рис. 5.1. Диалоговое окно **MFC ClassWizard**

Чтобы задать функции обработки отраженных сообщений с помощью панели инструментов **WizardBar**:

1. В панели инструментов **WizardBar** раскройте список **WizardBar C++ Class** и выберите в нем имя класса вашего элемента управления. Нажмите кнопку **WizardBar Actions (Default: Go To Function Declaration)**. Раскроется окно (рис. 5.2) редактирования файла реализации данного класса и текстовый курсор установится на первую функцию данного класса (как правило, его конструктор).
2. Раскройте контекстное меню **WizardBar Actions**, как это показано на рис. 5.3.
3. Выберите в нем команду **Add Windows Message Handler...**. Появится диалоговое окно **New Windows Message and Event Handlers for class CMyListCtrl** (имя класса будет другим), изображенное на рис. 5.4.
4. В окне списка **Class or object to handle:** будет выделено имя класса вашего элемента управления.
5. В окне списка **New windows messages/events:** выделите сообщение, для которого вы хотите задать функцию обработки. Отраженные сообщения данного класса отмечены знаком равенства (=).

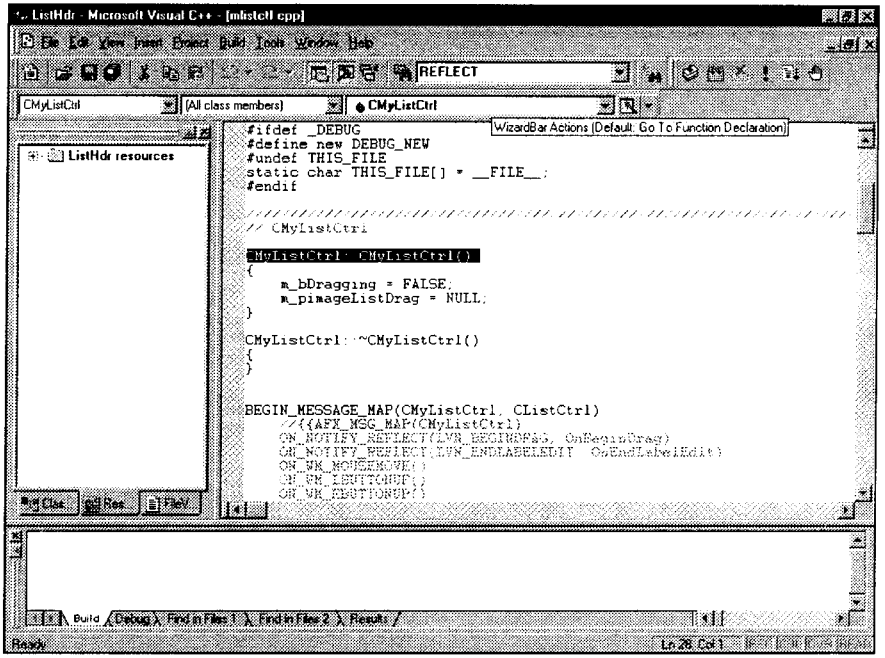


Рис. 5.2. Панель инструментов **WizardBar**

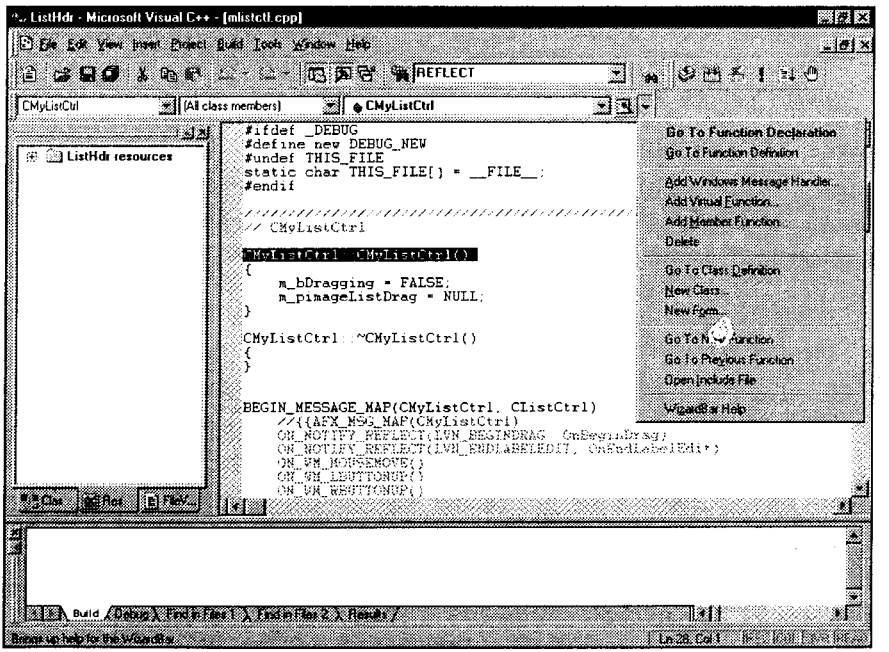


Рис. 5.3. Контекстное меню **WizardBar Actions**

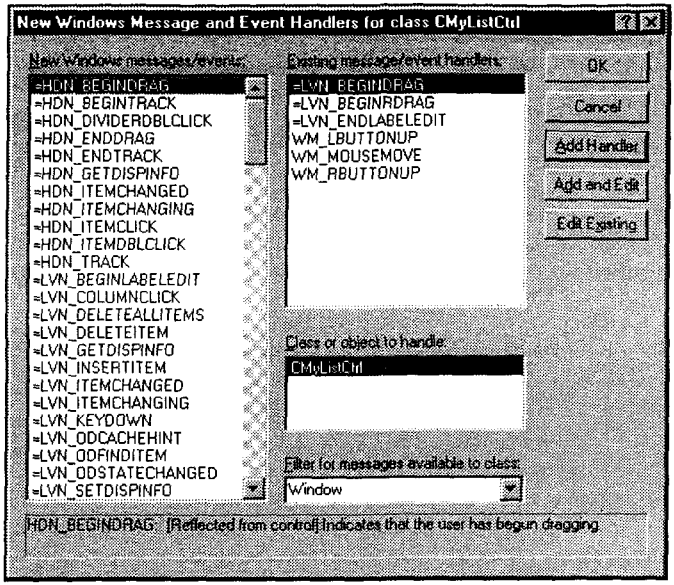


Рис. 5.4. Диалоговое окно **New Windows Message and Event Handlers for class CMyListCtrl**

6. Нажмите кнопку **Add Handler** или **Add and Edit**. Если для данного сообщения нет стандартного имени функции обработки, появляется диалоговое окно **Add Member Function**.
7. Если вы согласны с предложенным именем функции обработки сообщения, нажмите кнопку **OK** в диалоговом окне **Add Member Function**.
8. Панель инструментов **WizardBar** выполнит следующие действия:
   - Занесет в файл заголовка описания новых функций
   - Занесет в файл реализации корректно функционирующие заготовки новых функций
   - Занесет в карту сообщений класса новые макросы обработки сообщений
9. Если вы нажали кнопку **Add and Edit**, диалоговое окно **New Windows Message and Event Handlers for class CMyListCtrl** закроется, а текстовый курсор в окне редактирования соответствующего файла будет установлен на строку комментария функции обработки выделенного сообщения, указывая место вставки пользовательского текста в данную функцию.

Между именами макросов карты сообщений и идентификаторами отраженных сообщений, как правило, существует взаимно однозначное соответствие: для того чтобы преобразовать идентификатор отраженного сообщения в обрабатывающий его макрос, необходимо добавить к нему префикс `ON_` и окончание `_REFLECT`. Например, сообщение `WM_CTLCOLOR` обрабатывает макрос `ON_WM_CTLCOLOR_REFLECT`. (Чтобы определить, какое сообщение обрабатывает данный макрос, необходимо произвести обратное преобразование.)

Из этого правила существуют три исключения:

- Сообщению WM_COMMAND соответствует макрос ON_CONTROL_REFLECT
- Сообщению WM_NOTIFY соответствует макрос ON_NOTIFY_REFLECT
- Сообщению ON_UPDATE_COMMAND_UI соответствует макрос ON_UPDATE_COMMAND_UI_REFLECT

В перечисленных выше особых случаях пользователю предлагается самому выбрать имя функции обработки сообщения или утвердить предложенное системой. В остальных случаях используется стандартное имя функции обработки данного сообщения.

Подробную справку по аргументам и возвращаемой величине функции обработки сообщения можно получить в разделе справки по данной функции или по функции, имя которой получено добавлением префикса On к имени данной функции. Например, справка по функции CtlColor содержится в описании функции OnCtlColor. Некоторые функции обработки отраженных сообщений имеют меньше аргументов, чем соответствующие им функции обработки данного сообщения в классе родительского окна. Для установления соответствия аргументов данных функций необходимо воспользоваться информацией о типах и стандартных именах аргументов, приведенных в документации по этим функциям.

В табл. 5.1 приведены макросы карты сообщений и соответствующие им стандартные имена функций обработки данных сообщений.

Таблица 5.1. *Макросы карты сообщений и соответствующие им функции обработки*

Макрос карты сообщений	Функция обработки сообщения
`ON_CONTROL_REFLECT (wNotifyCode, memberFxn)`	`afx_msg void memberFxn ();`
`ON_NOTIFY_REFLECT (wNotifyCode, memberFxn)`	`afx_msg void memberFxn ( NMHDR * pNotifyStruct, LRESULT* result);`
`ON_UPDATE_COMMAND_UI_REFLECT (memberFxn)`	`afx_msg void memberFxn (CCmdUI* pCmdUI);`
`ON_WM_CTLCOLOR_REFLECT()`	`afx_msg HBRUSH CtlColor (CDC* pDC, UINT nCtlColor);`
`ON_WM_DRAWITEM_REFLECT()`	`afx_msg void DrawItem (LPDRAWITEMSTRUCT lpDrawItemStruct);`
`ON_WM_MEASUREITEM_REFLECT()`	`afx_msg void MeasureItem (LPMEASUREITEMSTRUCT lpMeasureItemStruct);`
`ON_WM_DELETEITEM_REFLECT()`	`afx_msg void DeleteItem (LPDELETEITEMSTRUCT lpDeleteItemStruct);`

Таблица 5.1 *(окончание)*

Макрос карты сообщений	Функция обработки сообщения
ON_WM_COMPAREITEM_REFLECT()	afx_msg int CompareItem (LPCOMPAREITEMSTRUCT lpCompareItemStruct);
ON_WM_CHARTOITEM_REFLECT()	afx_msg int CharToItem (UINT nKey, UINT nIndex);
ON_WM_VKEYTOITEM_REFLECT()	afx_msg int VKeyToItem (UINT nKey, UINT nIndex);
ON_WM_HSCROLL_REFLECT()	afx_msg void HScroll (UINT nSBCode, UINT nPos);
ON_WM_VSCROLL_REFLECT()	afx_msg void VScroll (UINT nSBCode, UINT nPos);
ON_WM_PARENTNOTIFY_REFLECT()	afx_msg void ParentNotify (UINT message, LPARAM lParam);

Макросы ON_NOTIFY_REFLECT и ON_CONTROL_REFLECT обеспечивают возможность обработки одного и того же сообщения несколькими объектами (класса элемента управления и класса его родительского окна). В табл. 5.2 приведен синтаксис соответствующих функций обработки сообщения.

Таблица 5.2. *Функции обработки для макросов ON_NOTIFY_REFLECT и ON_CONTROL_REFLECT*

Макрос карты сообщений	Функция обработки сообщения
ON_NOTIFY_REFLECT_EX(wNotifyCode, memberFxn)	afx_msg BOOL memberFxn (NMHDR * pNotifyStruct, LRESULT* result);
ON_CONTROL_REFLECT_EX(wNotifyCode, memberFxn)	afx_msg BOOL memberFxn ();

Пример создания элемента управления, обрабатывающего отраженные сообщения

В качестве примера элемента управления, обрабатывающего отраженные сообщения, создадим элемент управления CYellowEdit, представляющий собой текстовое поле, в котором черный текст отображается на желтом фоне. Текст этого приложения содержится в каталоге \Reflect дискеты, поставляемой с данной книгой. Чтобы самостоятельно создать это приложение:

1. Создайте заготовку диалогового приложения по методике, описанной в главе 1, и назовите его Reflect.

2. Раскройте папку \Dialog во вкладке **ResourceView** окна **Workspace**. Щелкните правой кнопкой мыши на идентификаторе ресурса `IDD_REFLECT_DIALOG` и выберите команду **Properties** в раскрывшемся при этом контекстном меню. Появится диалоговое окно **Dialog Properties**.
3. Нажмите на кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна.
4. Выберите из раскрывающегося списка **Language:** значение **Russian**.
5. Удалите из заготовки диалогового окна статический текст.
6. Выберите в панели инструментов **Controls** элемент управления **Edit Box** и поместите его в заготовку диалогового окна.
7. Введите в текстовое поле **ID:** диалогового окна **Edit Properties** идентификатор ресурса `IDC_EDIT`.
8. Нажмите кнопку **Cancel**. Появится диалоговое окно **Push Button Properties**.
9. Измените текст в текстовом поле **Caption:** на **Отмена**.
10. Закройте диалоговое окно **Edit Properties**.
11. Щелкните правой кнопкой мыши в окне редактирования ресурса. Появится контекстное меню, изображенное на рис. 5.5.

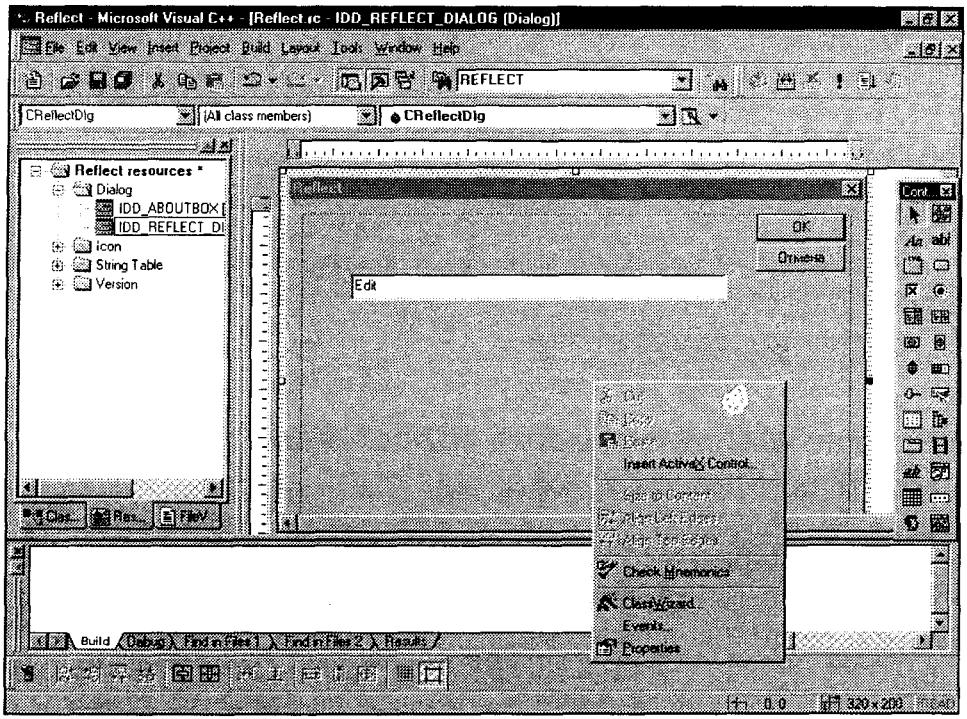


Рис. 5.5. Раскрытое контекстное меню

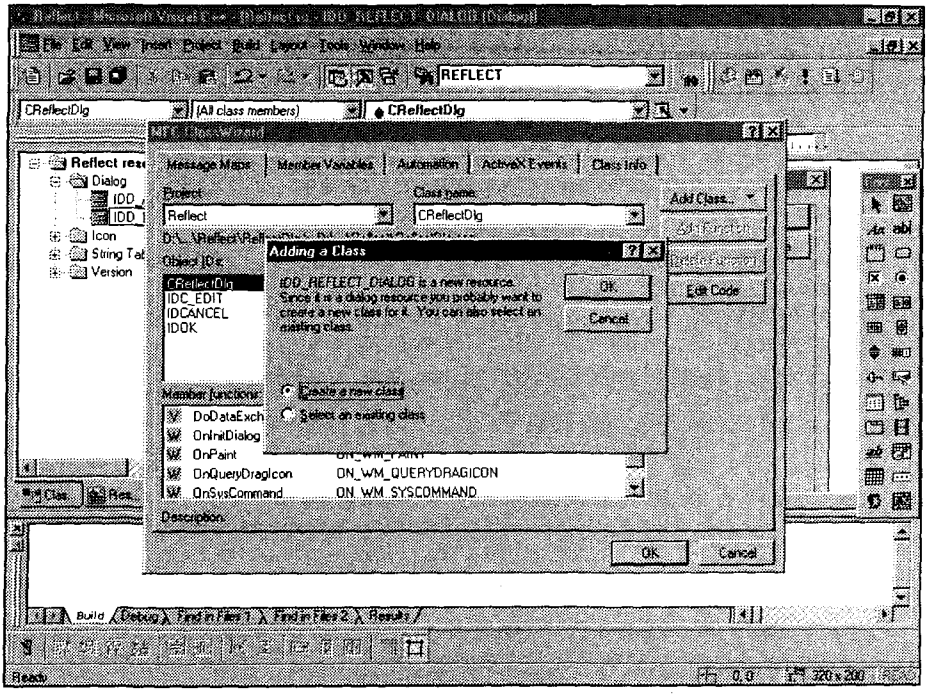


Рис. 5.6. Диалоговое окно **Adding a Class**

12. Выберите команду **ClassWizard**. Появится диалоговое окно **MFC ClassWizard**, на фоне которого будет открыто диалоговое окно **Adding a Class**, как это показано на рис. 5.6.
13. Установите переключатель в положение **Select an existing class** (Выберите существующий класс) и нажмите кнопку **OK**. Появится диалоговое окно **Select Class**, изображенное на рис. 5.7.

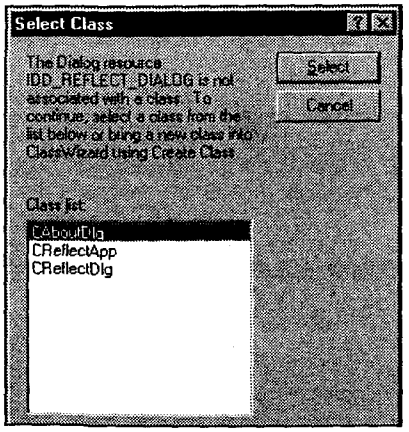


Рис. 5.7. Диалоговое окно **Select Class**

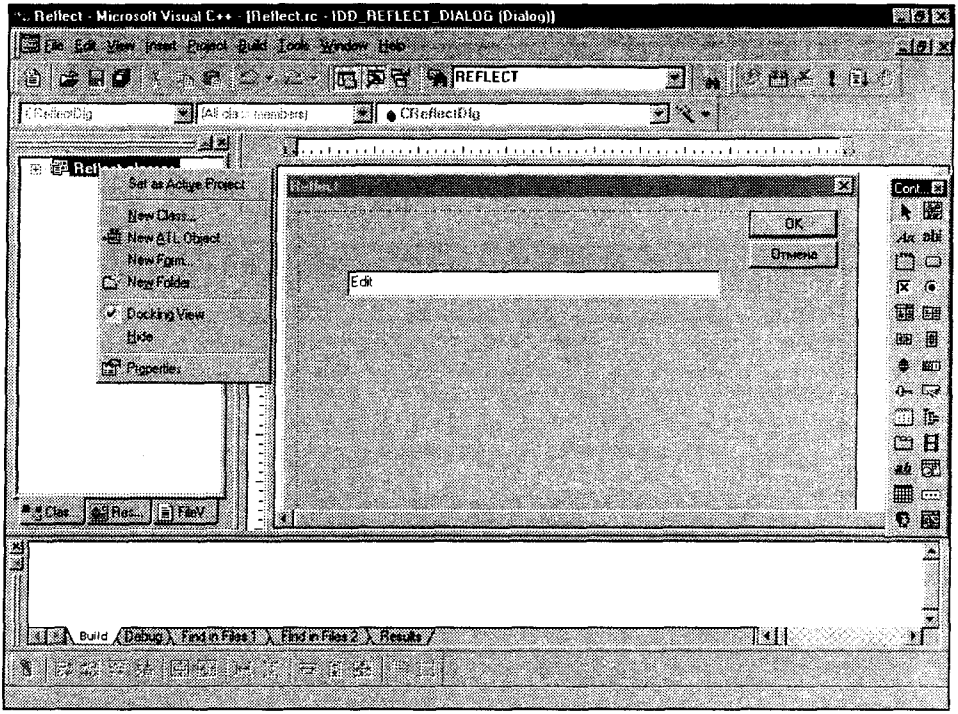


Рис. 5.8. Контекстное меню вкладки **ClassView** окна **Workspace**

14. В окне списка **Class list:** выделите имя класса `CReflectDlg` и нажмите кнопку **Select**. Диалоговое окно **MFC ClassWizard** станет доступным, а в текстовом поле раскрывающегося списка **Class name:** появится имя класса `CReflectDlg`.
15. Нажмите кнопку **OK** и закройте диалоговое окно **MFC ClassWizard**.
16. Раскройте вкладку **ClassView** окна **Workspace**, щелкните правой кнопкой мыши на папке \Reflect Classes. Появится контекстное меню, как это показано на рис. 5.8.
17. Выберите команду **New Class...**. Появится диалоговое окно **New Class**, изображенное на рис. 5.9.
18. Введите в текстовое поле **Name:** имя класса `CYellowEdit`, выберите в раскрывающемся списке **Base class:** имя базового класса `CEdit` и нажмите кнопку **OK**.
19. Раскройте папку \Reflect Classes во вкладке **ClassView** окна **Workspace** и щелкните правой кнопкой мыши на значке **CYellowEdit**. Появится контекстное меню, как это показано на рис. 5.10.
20. Выберите команду **Add Member Variable...**. Появится диалоговое окно **Add Member Variable**, изображенное на рис. 5.11.

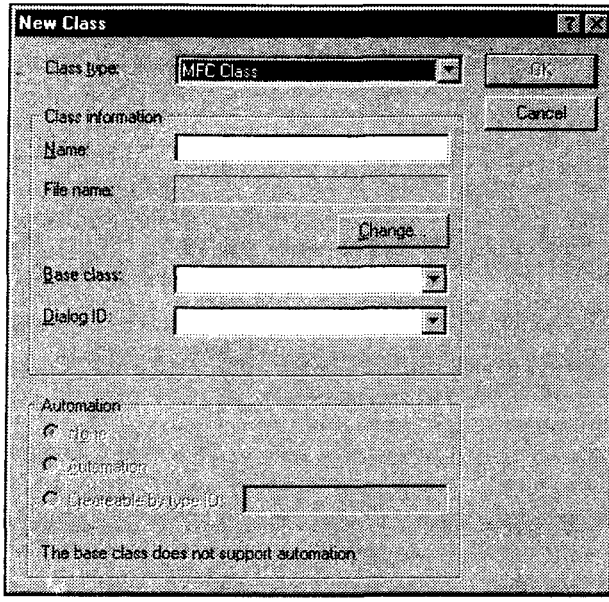


Рис. 5.9. Диалоговое окно **New Class**

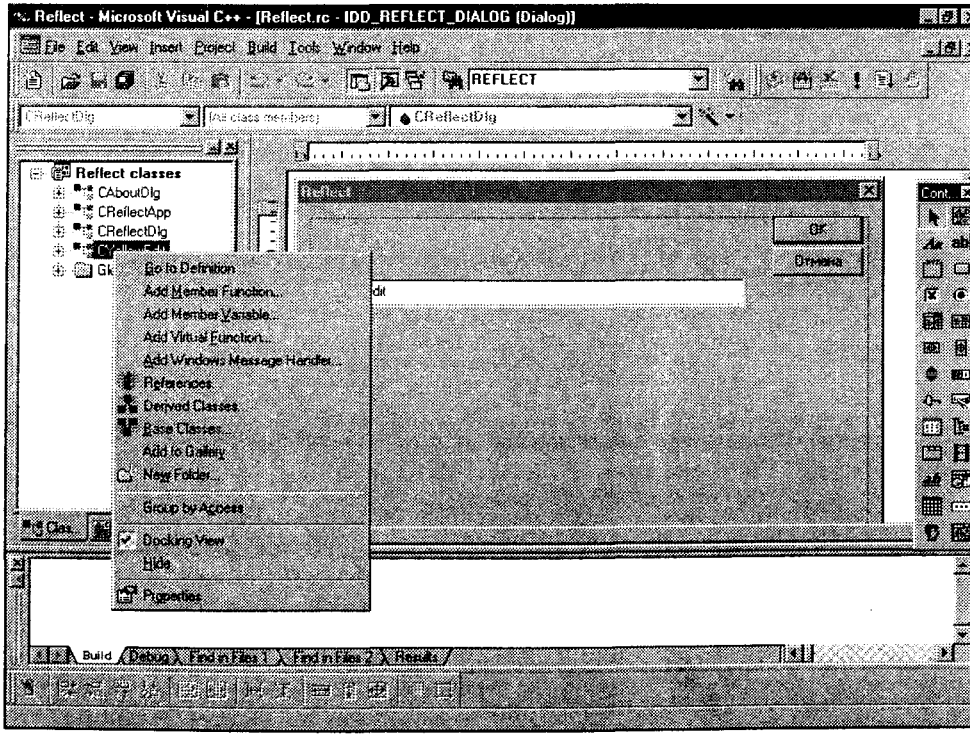


Рис. 5.10. Контекстное меню класса

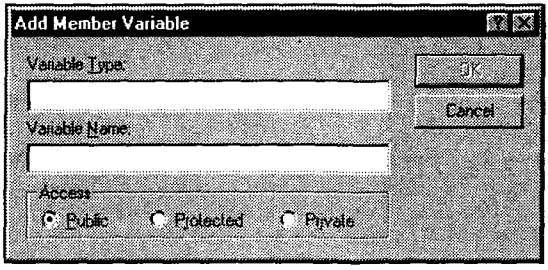


Рис. 5.11. Диалоговое окно **Add Member Variable**

21. Введите в текстовое поле **Variable Type** имя структуры `COLORREF`, введите в текстовое поле **Variable Name** идентификатор переменной `m_clrText`, оставьте переключатель **Access** в положении **Public** и нажмите кнопку **OK**. В этой переменной будет храниться цвет текста.

22. Повторите пп. 19—21 для переменной `m_clrBkgnd`, имеющей тип `COLORREF`. В этой переменной будет храниться цвет фона.

23. Повторите пп. 19—21 для переменной `m_brBkgnd`, являющейся объектом класса `CBrush`. Эта переменная будет использоваться для хранения кисти, используемой для заполнения фона.

24. Выберите панель **FileView** окна **Workspace**, раскройте папку \Reflect files, затем папку \Source Files и щелкните левой кнопкой мыши на значке **YellowEdit.cpp**. Откроется окно редактирования файла YellowEdit.cpp.

25. Измените содержимое конструктора класса `CYellowEdit` в соответствии со следующим текстом:

```
CYellowEdit::CYellowEdit()
{
    m_clrText = RGB( 0, 0, 0);
    m_clrBkgnd = RGB( 255, 255, 0);
    m_brBkgnd.CreateSolidBrush( m_clrBkgnd);
}
```

26. Щелкните правой кнопкой мыши в окне редактирования файла и в появившемся контекстном меню выберите команду **ClassWizard**. Откроется диалоговое окно **MFC ClassWizard**.

27. В окне списка **Object IDs:** выделите имя класса `CYellowEdit`, в окне списка **Messages:** выделите сообщение "=WM_CTLCOLOR" и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится имя новой функции обработки сообщения `CtlColor`, как это показано на рис. 5.12.

28. Нажмите кнопку **Edit Code**. Окно **MFC ClassWizard** закроется, откроется окно редактирования класса `CYellowEdit`, и текстовый курсор установится на первой строке комментария функции `CtlColor`.

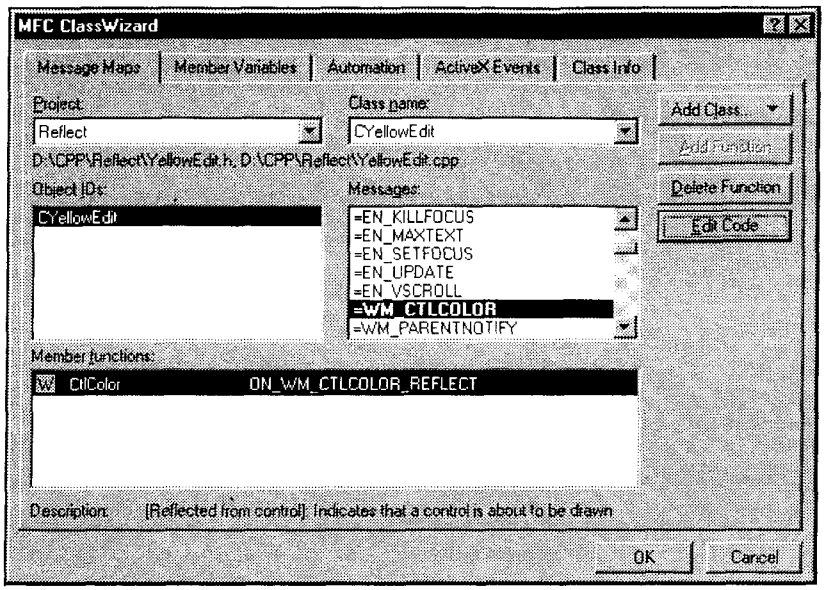


Рис. 5.12. Диалоговое окно **MFC ClassWizard**

29. Измените содержимое данной функции в соответствии с приведенным ниже текстом:

```
HBRUSH CYellowEdit::CtlColor(CDC* pDC, UINT nCtlColor)
{
// TODO: Change any attributes of the DC here
pDC->SetTextColor( m_clrText);
pDC->SetBkColor( m_clrBkgnd);
return m_brBkgnd;
}
```

30. Убедитесь, что в карте сообщений присутствует макрос

```
ON_WM_CTLCOLOR_REFLECT()
```

31. Откройте окно редактирования диалогового окна и дважды щелкните левой кнопкой мыши в текстовом поле, удерживая клавишу <Ctrl>. Появится диалоговое окно **Add Member Variable**, изображенное на рис. 5.13.

32. В текстовое поле **Member variable name:** введите идентификатор переменной `m_Edit`, в раскрывающемся списке **Category:** выберите категорию **Control**, а в списке **Variable type:** выберите класс элемента управления `CYellowEdit`. Нажмите кнопку **OK**. Появится окно сообщения **Microsoft Visual C++**, изображенное на рис. 5.14, предупреждающее о необходимости ввести в файл заголовка Reflectdlg.h директиву `#include "YellowEdit.h"`.

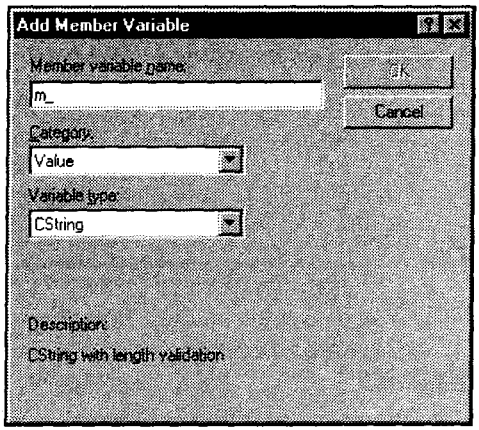


Рис. 5.13. Диалоговое окно **Add Member Variable**

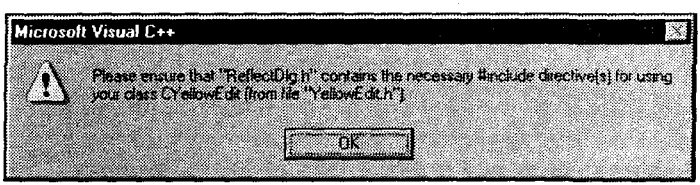


Рис. 5.14. Окно сообщения **Microsoft Visual C++**

33. Выберите панель **FileView** окна **Workspace**, раскройте папку \Header Files, расположенную в папке \Reflect files, и щелкните левой кнопкой мыши на значке **Reflectdlg.h**. Откроется окно редактирования файла Reflectdlg.h.

34. После директивы условной трансляции `#endif // _MSC_VER > 1000` вставьте следующую строку:

```
#include "YellowEdit.h"
```

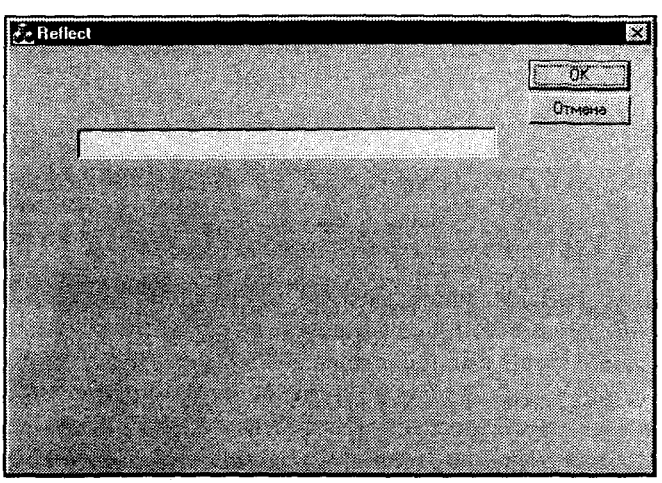


Рис. 5.15. Диалоговое окно с желтым текстовым полем

35. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится диалоговое окно, изображенное на рис. 5.15, в котором текстовое поле имеет желтый фон.

Использование карты сообщений приложением

Каждое приложение содержит объект класса, производного от класса `CWinApp`, содержащего функцию-член класса `Run`. Эта функция вызывает функцию `CWinThread::Run`, содержащую собственный цикл сообщений, состоящий из вызовов функций `GetMessage`, `TranslateMessage` и `DispatchMessage`. Практически все объекты оконных классов используют в качестве базового класс `CWnd` и функцию `AfxWndProc`, практически ничем не отличающуюся от функции `WndProc`, используемой в старых программах.

Функция `WndProc` позволяет определить дескриптор окна, которому предназначено данное сообщение, а в библиотеке MFC содержится *карта дескрипторов* (handle map), содержащую таблицу дескрипторов окон и указателей на соответствующие им объекты. Совместное использование полученной информации позволяет определить указатель на объект класса `CWnd`, который должен осуществлять обработку данного сообщения. После этого в данном объекте вызывается виртуальная функция `CWnd::WindowProc`. В каждом классе, производном от класса `CWnd`, производится перегрузка данной функции, позволяющая отразить все особенности обработки сообщений в данном конкретном классе окна.

Функция `WindowProc`, в свою очередь, вызывает функцию C++ `OnWndMessage`, которая, собственно, и обрабатывает сообщения. Сначала данная функция проверяет тип поступившего сообщения: является ли это собственно сообщением, командой или кодом извещения. Если поступило сообщение, то функция просматривает карту сообщений своего класса и находит в ней функцию обработки данного сообщения. Если в карте сообщения данного класса отсутствует макрос обработки данного сообщения, то просматривается карта сообщений родительского класса и так до тех пор, пока не будет найдена функция обработки данного сообщения. Механизм наследования карты сообщений работает аналогично механизму наследования C++, но полностью независим от него, что позволяет избежать многих проблем, возникающих при использовании виртуальных функций.

Библиотека MFC позволяет существенно упростить для пользователя работу с картой сообщений, поскольку она берет на себя обработку многих стандартных сообщений и не отражает их в карте сообщений. Например, при выборе пользователем команды **File**, **Open** классы библиотеки MFC сами производят обработку посылаемого по этой команде сообщения и в результате данной обработки вызывают функцию `Serialize` соответствующего

класса документа. При этом карта сообщений данного класса документа не содержит макроса обработки данного сообщения.

Использование мастера ClassWizard для создания функций обработки сообщений

Мы уже много раз использовали мастер ClassWizard для задания функций обработки сообщений. Однако все сведения, касающиеся работы с ним, оказались разбросанными по разным главам. В данном разделе будет собрана основная информация, касающаяся работы с этим мастером.

Диалоговое окно *MFC ClassWizard*

Чтобы вызвать на экран диалоговое окно **MFC ClassWizard**, выполните команду **View, ClassWizard**, нажмите комбинацию клавиш <Ctrl>+<W> или щелкните правой кнопкой мыши в окне редактирования. В последнем случае на экране появится контекстное меню, показанное на рис. 5.16, в котором нужно выбрать команду **ClassWizard**.

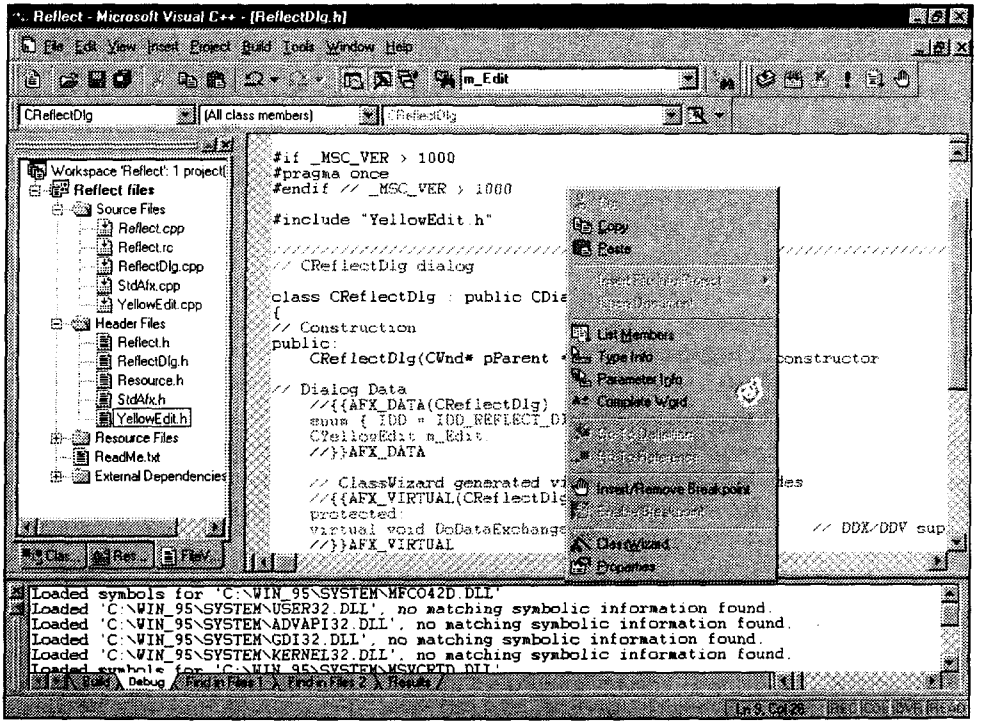


Рис. 5.16. Контекстное меню вызова мастера ClassWizard

Диалоговое окно **MFC ClassWizard** состоит из пяти вкладок. На первом этапе работы достаточно описания первых двух из них. На рис. 5.17 представлено диалоговое окно **MFC ClassWizard**, раскрытое на вкладке **Message Maps**.

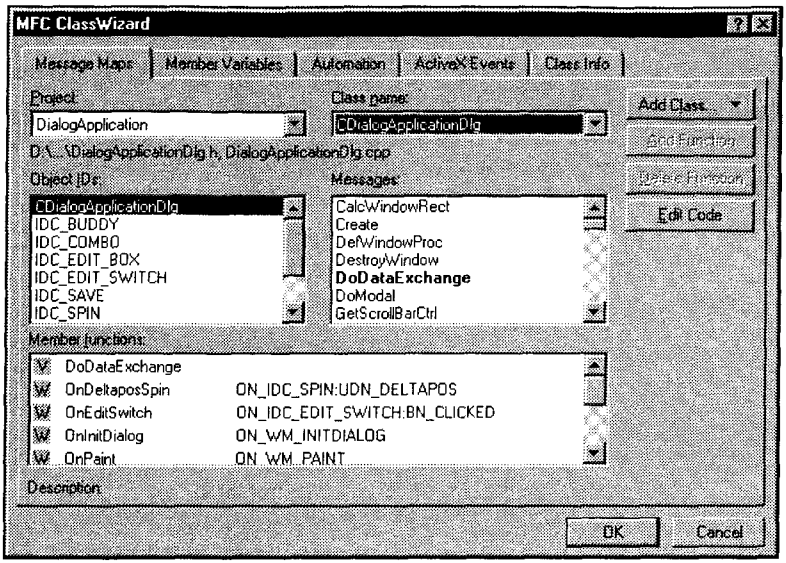


Рис. 5.17. Диалоговое окно **MFC ClassWizard**, вкладка **MessageMaps**

В верхней части вкладки расположены два раскрывающихся списка. В текстовом поле списка **Project:** указано имя текущего проекта, а в текстовом поле списка **Class name:** — имя класса, информация о котором содержится в окнах списка данной вкладки.

В окне списка **Object IDs:** приведено имя данного класса и идентификаторы объектов классов элементов управления, включенных в данный класс.

В окне списка **Messages:** содержится полный список сообщений и имен стандартных функций обработки сообщений, доступных для объекта класса, выделенного в окне списка **Object IDs:**. Имена стандартных функций обработки сообщений, включенных в данный класс, и имена идентификаторов сообщений, функции обработки которых включены в данный класс, выделяются в окне списка **Messages:** полужирным шрифтом. Для функции или идентификатора сообщения, выделенного в окне списка **Messages:**, в нижней части вкладки появляется краткое описание.

В окне **Member functions:** содержится список функций обработки сообщений, уже включенных в данный класс, а для функций, включенных в карту сообщений, указано также соответствующее им сообщение.

Кнопка **Add Class...** служит для создания нового класса или выбора его из библиотеки типов. При нажатии этой кнопки раскрывается контекстное меню, изображенное на рис. 5.18.

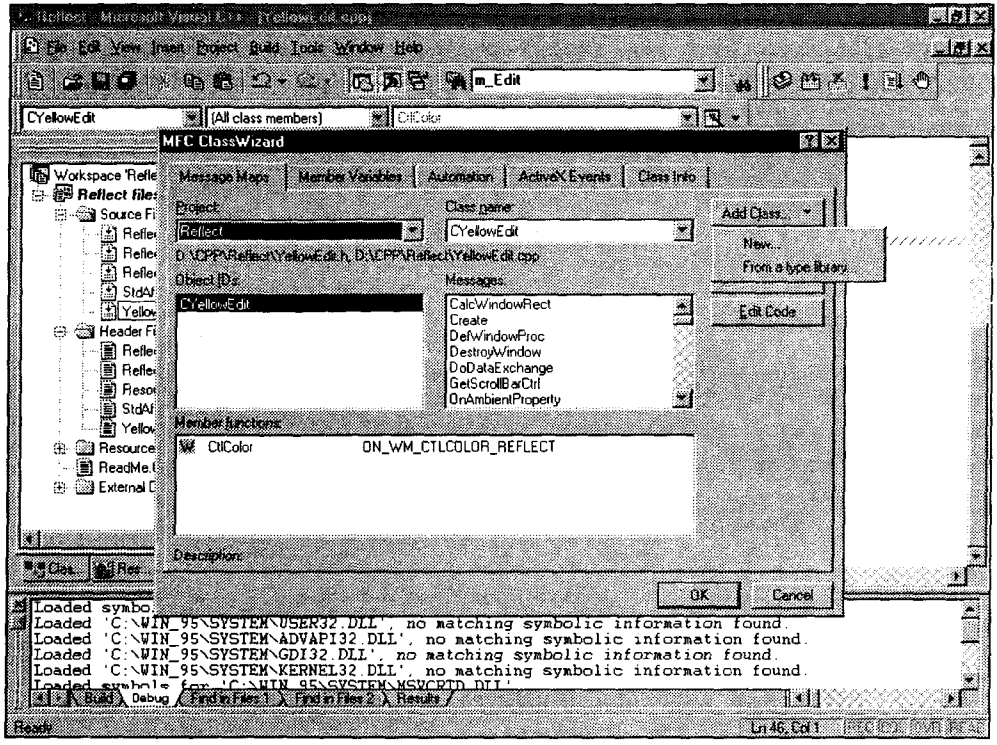


Рис. 5.18. Контекстное меню создания класса

Работа с этой кнопкой описана в главе 3 в разделе, посвященном созданию вкладок диалогового окна. Кнопка **Add Function** служит для добавления в класс функции обработки сообщения. В главе 4 имеется множество примеров использования данной кнопки. Кнопка **Delete Function** служит для уничтожения соответствующего макроса в карте сообщений файла реализации класса и описания функции обработки сообщения в карте сообщений файла заголовка. К сожалению, нажатие данной кнопки не приводит к уничтожению текста, реализующего данную функцию, о чем сообщает соответствующее диалоговое окно, приведенное на рис. 5.19.

См. также главы 3 и 4

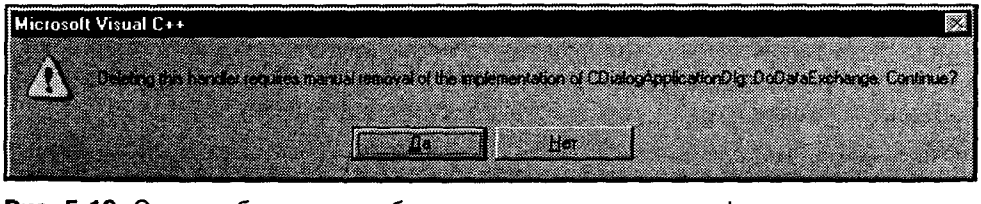


Рис. 5.19. Окно сообщения о необходимости удаления текста функции вручную

Поэтому пользователю придется в случае положительного ответа на поставленный вопрос войти в соответствующий файл реализации и самому удалить текст данной функции. В случае отрицательного ответа на вопрос никаких изменений в приложение внесено не будет. Назначение кнопок **Edit Code**, **OK** и **Cancel** описано ранее в данной главе.

Вторая вкладка диалогового окна **MFC ClassWizard** называется **Member Variables**. Диалоговое окно **MFC ClassWizard**, раскрытое на вкладке **Member Variables**, приведено на рис. 5.20.

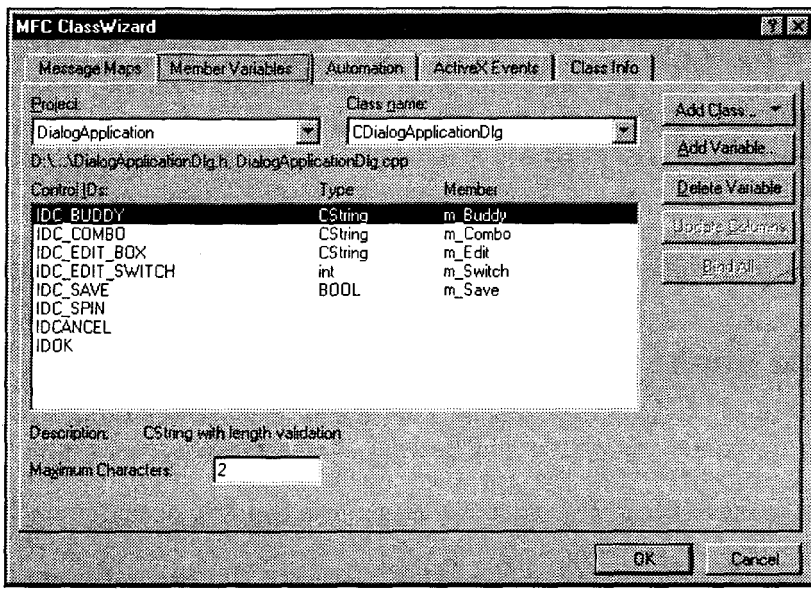


Рис. 5.20. Диалоговое окно **MFC ClassWizard**, вкладка **Member Variables**

В верхней части данной вкладки расположены те же раскрывающиеся списки, что и во вкладке **Message Maps**. Ниже расположено окно списка, содержащее идентификаторы элементов управления, соответствующие им переменные и тип этих переменных. Под этим окном списка расположено текстовое поле, позволяющее задавать максимальное количество символов в переменной типа `CString`, выделенной в окне списка.

Кнопка **Add Class...** данной вкладки имеет то же назначение, что и в предыдущей вкладке. Кнопка **Add Variable...** служит для сопоставления идентификатору элемента управления переменной, являющейся членом класса, имя которого указано в текстовом поле соответствующего раскрывающегося списка. При нажатии этой кнопки появляется диалоговое окно **Add Member Variable**, изображенное на рис. 5.21.

В главе 4 имеется множество примеров использования данной кнопки. Кнопка **Delete Variable** служит для уничтожения выделенной в окне списка

переменной. Последние две кнопки данной вкладки используются только при работе с базами данных.

См. также главу 4

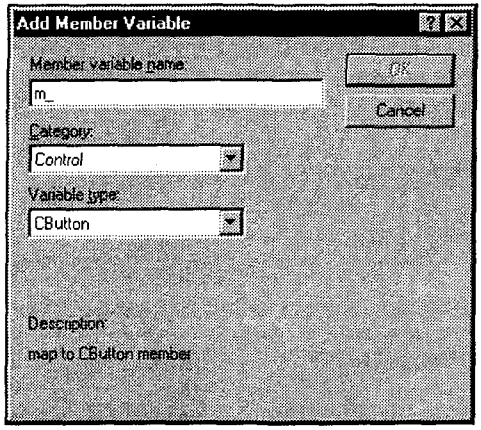


Рис. 5.21. Диалоговое окно **Add Member Variable**

Добавление обработчиков сообщений Windows

В последних версиях Visual C++, начиная с версии 5.0, добавлен новый способ задания функций обработки сообщений. Вместо того чтобы открывать диалоговое окно **MFC ClassWizard** и искать в нем имя класса, можно просто открыть вкладку **ClassView** окна **Workspace** и щелкнуть правой кнопкой мыши на значке соответствующего класса. Появится контекстное меню, в котором нужно выбрать команду **Add Windows Message Handler...**. Появится диалоговое окно **New Windows Message and Event Handlers for class CDialogApplicationDlg** (имя класса соответствует имени класса, выделенного во вкладке **ClassView**), показанное на рис. 5.22.

В данном диалоговом окне не показаны виртуальные функции, приведенные в списке **Messages:** вкладки **Message Maps** диалогового окна **MFC ClassWizard**.

В окне списка **New Windows messages/events:** диалогового окна **New Windows Message and Event Handlers for class CDialogApplicationDlg** приведены идентификаторы всех сообщений и событий, обработчики которых могут быть включены в объект класса, выделенного в окне списка **Class or object to handle:**.

В окне списка **Existing message/event handlers:** содержатся идентификаторы всех сообщений и событий, обработчики которых уже включены в объект данного класса.

Содержимое текстового поля раскрывающегося списка **Filter for messages available for class:** позволяет выбрать класс, список сообщений которого бу-

дет выводиться в окне списка **New Windows messages/events:**. В данном списке может быть выбран любой класс из иерархии родительских классов данного класса.

Чтобы добавить функцию обработки для сообщения в конкретный класс, выделите имя класса в окне списка **Class or object to handle:**, выделите соответствующее сообщение в окне списка **New Windows messages/events:** и нажмите кнопку **Add Handler**. Если вы хотите сразу же приступить к редактированию созданного прототипа функции, то вместо кнопки **Add Handler** нажмите кнопку **Add and Edit**. В этом случае диалоговое окно **New Windows Message and Event Handlers for class CDialogApplicationDlg** закроется, откроется окно редактирования соответствующего файла реализации, и текстовый курсор будет помещен в текст созданной функции.

Чтобы отредактировать существующую функцию обработки сообщения, выделите соответствующее сообщение в окне списка **Existing message/event handlers:** и нажмите кнопку **Edit Existing**.

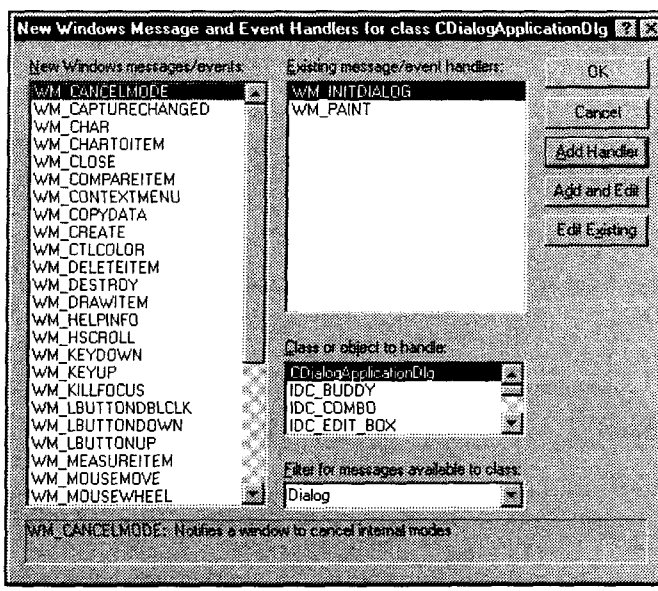


Рис. 5.22. Диалоговое окно **New Windows Message and Event Handlers for class CDialogApplicationDlg**

Список сообщений

В настоящее время в операционной системе Windows используется около 1000 различных сообщений. Конечно, в каждом классе имеется своя карта сообщений и количество обрабатываемых им сообщений обычно на несколько порядков меньше. Даже потенциально ни один класс не может обработать все имеющиеся сообщения, поскольку они разделяются на классы,

и, например, только потомок класса CListBox может получить сообщение LB_SETSEL, выделяющее в списке указанный в нем элемент.

По префиксу имени сообщения можно определить класс, объект которого послал данное сообщение, или объект класса окна, которому оно было направлено. В табл. 5.3 приведены префиксы некоторых сообщений и указано, какому окну они принадлежат.

Таблица 5.3. *Таблица соответствия префиксов сообщений послившему их окну*

Префикс	Описание
BM	Сообщение от кнопки
BN	Извещение от кнопки
CB	Сообщение от комбинированного списка
CBN	Извещение от комбинированного списка
CDM	Сообщение от стандартного диалогового окна
CDN	Извещение от стандартного диалогового окна
CPL	Сообщение от панели управления
DM	Сообщение о нажатии кнопки по умолчанию в диалоговом окне

Нет никакой необходимости продолжать этот список, поскольку за правильностью синтаксиса идентификаторов элементов управления следит мастер ClassWizard, а какое окно породило данное сообщение, ясно из контекста программы.

Обновление команд

Мы уже ознакомились с сообщениями и извещениями, но до сих пор не выяснили, что же представляют собою команды. Команды представляют собой извещения формата WM_COMMAND, посылаемые объектами классов пользовательского интерфейса, такими как меню, кнопки панели инструментов и клавиши акселераторов. В Windows 3.x сообщение WM_COMMAND формировалось как в ответ на выбор команды меню, так и в ответ на нажатие кнопки. В Win32 сообщение WM_COMMAND формируется только в ответ на выбор команды меню, а нажатие кнопки или выделение элемента в списке порождает сообщение WM_NOTIFY с кодом извещения элемента управления.

Отличительной чертой команд является необходимость задания состояния связанных с ними элементов управления. Обычно команды меню и кнопки панели инструментов могут находиться в различных состояниях. Например, команда меню в данной конкретной ситуации может быть недоступной.

Команды меню могут также отмечаться флажками и переключателями. Все вышеперечисленное полностью относится и к кнопкам панели инструментов.

Для определения текущего состояния подобных элементов управления приложение должно при каждом обновлении своего состояния иметь информацию о том, в каком состоянии при этом должен находиться каждый из его элементов управления. Эту информацию может дать только сам программист. Логично предположить, что если команда меню посылает сообщение, обрабатываемое некоторым документом, то именно этот документ и должен определять состояние команды в меню, поскольку в данном документе должна содержаться вся необходимая для этого информация.

Если одной и той же команде соответствуют различные элементы пользовательского интерфейса (например, пункт меню и кнопка панели инструментов), то их состояние должно определяться одной и той же функцией обработки сообщения.

В библиотеке MFC предусмотрен удобный интерфейс управления состоянием подобных элементов управления. Пользователь может применять для этого свои функции и методы, но обычно бывает достаточно стандартных методов.

При раскрытии меню приложение запускает механизм проверки состояния команд в данном меню. Для этого приложение посылает сообщение об обновлении всем элементам управления, содержащимся в данном меню. Каждой команде обновления меню соответствует макрос в карте сообщений, имеющий форму `ON_UPDATE_COMMAND_UI`. В нем и указана вызываемая функция обработки сообщения. Таким образом, для меню, содержащего шесть команд, вызывается шесть функций обработки сообщения. Если пользователем указана функция обработки сообщения об обновлении состояния команды с данным идентификатором, то вызывается данная функция. В противном случае приложение проверяет, имеется ли в классе функция обработки для команды с данным идентификатором, и, в соответствии с результатами проверки, делает данную команду доступной или недоступной.

Если приложение не находит соответствующий макрос `ON_UPDATE_COMMAND_UI`, то оно делает команду доступной, если в карте сообщений активного класса данного приложения имеется макрос `ON_COMMAND` для идентификатора данной команды. Если же такой макрос отсутствует, то команда делается недоступной. Поэтому, чтобы сделать команду меню доступной, для нее нужно написать функцию обработки.

Типичная функция обработки сообщения об обновлении команды выглядит следующим образом:

```
afx_msg void OnUpdateEditClearAll( CCmdUI* pCmdUI);
```

Ее аргументом является объект класса `CCmdUI`. Функции-члены данного класса перечислены в табл. 5.4.

Таблица 5.4. *Функции-члены класса* `CCmdUI`

Функция	Описание
`Enable`	Делает элемент управления доступным или недоступным
`SetCheck`	Отмечает элемент управления флажком
`SetRadio`	Отмечает элемент управления переключателем
`SetText`	Устанавливает текст в элементе управления
`ContinueRouting`	Указывает механизму обработки сообщений, что данная функция не является конечной и что необходимо продолжить обработку данного сообщения

Подробное описание этих функций содержится в Приложении В.

В табл. 5.5 содержатся сведения о реакции отдельных элементов управления на вызов функций-членов класса `CCmdUI`.

Таблица 5.5. *Реакция элементов управления на вызов функций-членов класса* `CCmdUI`

Элемент управления	Функция-член класса `CCmdUI`			
	`Enable`	`SetCheck`	`SetRadio`	`SetText`
Команда меню	Делает доступной или недоступной	Устанавливает или снимает флажок (v)	Устанавливает или снимает переключатель (.)	Устанавливает текст элемента управления
Кнопка панели инструментов	Делает доступной или недоступной	Выделяет, снимает выделение или переводит в неопределенное состояние	Работает аналогично функции `SetCheck`	Не используется
Панель строки состояния	Делает текст видимым или невидимым	Устанавливает выступающую или нормальную рамку	Работает аналогично функции `SetCheck`	Устанавливает текст в панели
Обычная кнопка в `CDialogBar`	Делает доступной или недоступной	Устанавливает или снимает флажок	Работает аналогично функции `SetCheck`	Устанавливает текст в кнопке
Обычный элемент управления в панели `CDialogBar`	Делает доступной или недоступной	Не используется	Не используется	Устанавливает текст окна

Вывод информации на экран

Заготовки многооконных и однооконных приложений, создаваемые мастером AppWizard, при запуске на исполнение выводят на экран пустое окно документа. В данной главе рассказывается, каким образом это окно может быть заполнено полезной информацией.

Существуют четыре принципиально разные возможности заполнить окно документа в приложении:

- Использование линии для рисования фигур
- Использование кисти для заполнения замкнутых геометрических фигур
- Вывод тестовой информации
- Вывод битовых образов

Прежде чем перейти к рассмотрению вопросов, связанных с каждой из этих возможностей, необходимо рассмотреть интерфейс графических устройств Windows (GDI), используемый приложением для вывода любой информации на экран.

Интерфейс графических устройств (GDI)

Программы, работающие в среде MS DOS, должны самостоятельно управлять устройствами отображения информации. Так, любой текстовый редактор в MS DOS, обеспечивающий вывод на печать, должен включать в себя драйверы всех принтеров, на которые предполагается выводить информацию.

Одной из основополагающих концепций Windows является независимость текстов программ от используемых аппаратных средств. Поэтому для работы с устройствами отображения информации приложения Windows используют *интерфейс графических устройств* (Graphic Device Interface, GDI) Windows, не обращаясь непосредственно к устройствам.

Эта особенность программ Windows, с одной стороны, усложняет методику программирования операций отображения информации, но с другой — избавляет программиста от необходимости использования в программе драйверов всевозможных устройств и гарантирует, что программа будет работать даже с теми устройствами, которые еще не созданы на момент написания программы. Управление устройствами осуществляется с использованием драйверов устройств Windows, которые устанавливаются во время инсталляции операционной системы на конкретный компьютер или поставляются вместе с новым оборудованием. Эти драйверы получают информацию из приложения, обрабатывают ее и формируют на основе результатов обработки сигналы управления для конкретного устройства.

Ясно, что цветной принтер и цветной монитор решают, в принципе, одну и ту же задачу: выводят двумерную графическую информацию в растровой форме. Поэтому для их управления могут быть использованы одни и те же команды, но, с другой стороны, никакой цветной принтер не обеспечит 24-битной глубины цвета, и пока что никакой дисплей не имеет разрешающей способности 600×600 точек на дюйм. Поэтому, получив от приложения задание на вывод изображения драйвер принтера должен преобразовать его цвета в цвета, имеющиеся в палитре данного принтера, а драйвер монитора должен объединить соседние точки изображения, чтобы результат преобразования соответствовал разрешающей способности установленного на компьютере дисплея.

Поскольку программист при работе с устройствами отображения информации не должен отвлекаться на особенности конкретного устройства, Windows предоставляет ему возможность написания программ, работающих с некоторым виртуальным устройством, имеющим превосходные характеристики, которых не имеет ни одно реальное устройство. В тот момент, когда возникнет необходимость вывести это "идеальное" изображение на некоторое реальное устройство, приложение запросит его контекст, определит возможности данного устройства отображения информации и с минимальными потерями преобразует изображение таким образом, чтобы его параметры полностью соответствовали возможностям реального устройства.

Контекст устройства

Контекст устройства представляет собой структуру данных, определенную в операционной системе Windows и содержащую информацию о наборе *графических объектов* и связанных с ними атрибутах таких устройств, а также о графических режимах вывода информации. Под графическими объектами подразумеваются перья для рисования линий, кисти для закрашивания замкнутых контуров, битовые образы для вывода на экран или прокрутки изображения в окне, палитры, определяющие набор доступных цветов, регионы для усечения изображения и подобных операций, а также пути для операций рисования.

Информация в Windows может отображаться только с использованием объектов классов контекста устройств, содержащих методы Windows API, реализующие рисование линий, форм и вывод текста. Использование контекста устройства в Windows позволяет осуществлять вывод графической информации, независимый от устройства. С его помощью графическая информация может быть выведена на экран, на принтер или в метафайл.

Основные принципы работы с контекстом устройств Windows реализованы в классе `CPaintDC`. Конструктор данного класса вызывает функцию `CWnd::BeginPaint`, а его деструктор — функцию `CWnd::EndPaint`. При вызове функции `CView::OnDraw` приложение само вызывает функцию `CView::OnPrepareDC` и передает пользователю указатель на объект класса `CDC`. Уничтожение этого объекта приложение также берет на себя.

Класс `CClientDC` содержит методы, позволяющие работать только в рабочей области окна. Конструктор данного класса вызывает функцию `CWnd::GetDC`, а его деструктор — функцию `CWnd::ReleaseDC`. Большинство процедур рисования располагается в функциях `OnDraw`, вызываемых при создании окна и при необходимости его перерисовки. Однако иногда требуется вывести в окно некоторый графический объект, не прибегая к полной или частичной перерисовке окна. Именно в этом случае и следует использовать для задания контекста устройства объект класса `CClientDC`.

Класс `CWindowDC` содержит методы, позволяющие работать со всем окном, включая его рамку. Класс `CMetaFileDC` содержит методы, позволяющие сохранять графические объекты в метафайлах. В отличие от указателя на класс `CPaintDC`, передаваемого в функции `OnDraw`, при работе с данным классом пользователь должен сам вызывать функцию `OnPrepareDC`.

Как уже отмечалось, почти все операции рисования в приложении осуществляются функцией `OnDraw`, перегружаемой в каждом производном от нее классе. В перегруженной функции `OnDraw` необходимо выполнить следующие действия:

- Получить данные из связанного с этим представлением документа
- Вывести эти данные в графическом виде с использованием объекта класса контекста устройства, указатель на который передается данной функции в качестве параметра

В случае внесения каких-либо изменений в данные, содержащиеся в документе, эти изменения должны быть немедленно отображены на экране. Обычно все изменения вносятся в документ связанным с ним объектом класса представления. Если с данным документом связаны несколько объектов представления, то для внесения изменений во все объекты класс представления, через который вносятся изменения, получает указатель на связанный с ним объект и вызывает функцию `CDocument::UpdateAllViews`, имеющую три аргумента, первым из которых является указатель на объект

класса CView, в котором произошли изменения, вызвавшие модификацию документа. Если первый аргумент данной функции равен нулю, то необходимо внести изменения во все объекты представления. Второй параметр функции UpdateAllViews имеет тип LPARAM и содержит информацию о необходимых изменениях. Третий параметр данной функции представляет собой указатель на объект класса CObject, в котором также хранится информация о необходимых изменениях.

Данная функция обычно вызывается после внесения изменений в документ и после вызова функции CDocument::SetModifiedFlag. Функция UpdateAllViews посылает сообщения всем объектам класса представления, за исключением объекта, на который указывает первый аргумент данной функции, о внесении изменений в связанный с ним документ. Для этого она вызывает функцию CView::OnUpdate, имеющую тот же набор аргументов, что и вызывающая функция.

Как только представление получает сообщение о необходимости перерисовки, Windows посылает сообщение WM_PAINT. В классе представления имеется функция обработки данного сообщения CWnd::OnPaint, создающая объект класса контекста устройств CPaintDC и вызывающая функцию OnDraw данного класса представления. Обычно пользователь не должен перегружать функцию OnPaint.

Типичный пример перегруженной функции OnDraw приведен ниже.

```
void CMyView::OnDraw( CDC* pDC)
{
    CMyDoc* pDoc = GetDocument();
    CString s = pDoc-> szDemoString;
    CRect rect;

    GetClientRect( &rect);
    pDC->SetTextAlign( TA_BASELINE | TA_CENTER);
    pDC->TextOut( rect.right / 2, rect.bottom / 2,
                  s, s.GetLength());
}
```

В данном примере приложение получает указатель на объект документа, связанного с данным представлением. Затем, используя полученный указатель на объект класса документа, данная функция получает доступ к переменной-члену класса документа szDemoString. После этого полученная из документа строка печатается в центре экрана. Если данная функция осуществляет вывод на экран, то передаваемый ей указатель на объект класса CDC в действительности является указателем на объект класса CPaintDC, для которого уже вызвана функция BeginPaint. Для вызова функций работы с графическими объектами используются методы соответствующего класса контекста устройства.

Отображение текста

Человеку, знакомому с типографскими терминами, может показаться странным их трактовка в системе Windows, однако большинство терминологических различий между шрифтами Win32 и стандартными типографскими шрифтами отражают различие используемых технологий печати. Стандартные типографские термины связаны с технологическим процессом отливки матриц, в то время как используемые в Win32 термины связаны с компьютерной версткой текстов и печатью их на лазерных принтерах.

Шрифты

Под *шрифтом* понимается набор букв и символов, объединенных единым дизайном. Основными элементами, определяющими дизайн шрифта, являются начертание, стиль и размер.

Под термином *начертание* понимаются отличительные черты букв и символов данного шрифта, такие как ширина жирных и тонких штрихов, составляющих символы, а также наличие или отсутствие засечек. Засечка представляет собой короткую поперечную линию на открытых концах штрихов. Шрифт или начертание без засечек обычно называется "сансериф".

Термин *стиль* определяет толщину линии знака и наклон знаков. Шрифты Win32 могут иметь следующие значения толщины линии (начиная с самой тонкой и кончая самой толстой):

- Thin (самая тонкая)
- Extralight (очень тонкая)
- Light (тонкая)
- Normal (нормальная)
- Medium (средняя)
- Semibold (полужирная)
- Bold (толстая)
- Extrabold (очень толстая)
- Heavy (самая толстая)

Параметр наклона шрифта может принимать одно из трех значений: roman (без наклона), oblique (наклонный), и italic (курсив). Символы со значением этого параметра roman характеризуются отсутствием наклона. Параметр oblique означает искусственный наклон шрифта. Наклон достигается специальным начертанием шрифтов roman. Шрифты с параметром italic действительно наклонены.

Размер шрифта не может быть определен точно. Обычно он определяется как расстояние между нижней точкой строчной буквы g и верхней точкой заглавной буквы M. Размер шрифта измеряется в пиках. Одна пика составляет 0.013837 дюйма. В соответствии с системой пик, разработанной Пьером Симоном Фурнье, под одной пикой обычно понимают 1/72 дюйма.

Под термином *семейство шрифтов* (font family) понимается совокупность шрифтов, имеющих единую ширину штрихов и засечек. Всего имеется пять

семейств шрифтов. Шестое семейство предполагает использование в приложении шрифтов по умолчанию. Ниже перечислены семейства шрифтов и даны краткие их характеристики.

- Decorative — определяет нестандартный шрифт. Например, Old English
- Dontcare — является общим именем семейства шрифтов. Это имя присваивается шрифту, если информация о нем отсутствует или используемый шрифт не имеет значения. В этом случае загружается шрифт по умолчанию
- Modern — определяет шрифт с одинаковой шириной букв, имеющий или не имеющий засечек. К таким шрифтам, как правило, относятся современные шрифты, например, Pica, Elite и Courier New
- Roman — определяет пропорциональный шрифт с засечками. Примером такого шрифта является Times New Roman
- Script — определяет шрифт, который выглядит как рукописный. Например, Script
- Swiss — определяет пропорциональный шрифт без засечек. Примером такого шрифта является Arial

Данным шрифтам в файле заголовка WinGdi.h соответствуют предопределенные константы `FF_DECORATIVE`, `FF_DONTCARE`, `FF_MODERN`, `FF_ROMAN`, `FF_SCRIPT` и `FF_SWISS`. Приложение использует эти константы при создании и выборе шрифта, а также при получении информации о шрифте.

Приложения Win32 могут использовать четыре различных способа вывода шрифтов на экран или принтер: растровый, векторный, TrueType и OpenType. Различия между этими шрифтами состоят в способе хранения символа в памяти компьютера или в соответствующем файле ресурса шрифта:

- Символы растровых шрифтов хранятся в виде битовых изображений, которые копируются приложением в ту позицию экрана или распечатки, в которую требуется поместить изображение символа
- В векторных шрифтах символ представлен конечными точками отрезков линий, которые рисует приложение при выводе данного символа
- В шрифтах TrueType и OpenType символ представляется набором команд рисования отрезков прямых линий и сегментов кривых, а также набором подсказок. Команды рисования отрезков линий используются для грубого задания начертания символа в шрифтах TrueType или OpenType. Система подсказок позволяет отредактировать положения начальных и конечных точек отрезков, используемых при рисовании символов. Эти подсказки и соответствующие им корректировки учитывают текущий масштаб отображения символа. Шрифт OpenType эквивалентен шрифту TrueType за исключением того, что шрифт OpenType допускает использование команд Postscript при формировании начертания шрифта

Поскольку битовые образы каждого символа растрового шрифта разрабатывались с учетом разрешающей способности устройства, на котором они должны отображаться, то растровые шрифты считаются шрифтами, зависящими от устройства. Векторные шрифты не зависят от устройства, на которое они будут выводиться, поскольку каждый их символ представлен легко масштабируемым набором конечных точек отрезков. Однако вывод векторных шрифтов требует больших вычислительных затрат, чем вывод растровых шрифтов или шрифтов TrueType и OpenType. Шрифты TrueType и OpenType обеспечивают достаточно высокую скорость вывода символов и полную независимость от устройства. Использование подсказок, содержащихся в описании символов шрифта, позволяет легко масштабировать символы шрифтов TrueType и OpenType без потери качества их отображения.

Как уже говорилось, описание символов шрифта хранится в файле ресурса шрифта. Файл ресурса шрифта фактически представляет собой библиотеку динамической компоновки, которая содержит данные, но не содержит методов их обработки. Для растровых и векторных шрифтов данная библиотека состоит из двух разделов: заголовка, описывающего метрику шрифта, и данных, описывающих символы данного шрифта. Файл ресурса шрифта для растрового или векторного шрифта имеет расширение .fon. Каждому из шрифтов TrueType или OpenType соответствует два файла: первый содержит относительно короткий заголовок, а второй — информацию о символах данного шрифта. Первый файл имеет расширение .fot, а второй — расширение .ttf.

Каждый шрифт характеризуется набором включенных в него символов. Набор символов включает в себя знаки препинания, символы верхнего и нижнего регистра, а также все другие печатаемые символы. Каждому элементу символьного набора соответствует свой номер.

Большинство символьных наборов использует набор символов ASCII, определенный стандартом США, содержащий 96 символов, которым соответствуют номера от 32 до 127. Имеются пять основных групп символьных наборов:

- Windows
- Unicode
- OEM
- Symbol
- Специальные наборы

В приложениях Win32 обычно используется набор символов Windows. Этот набор практически полностью эквивалентен набору символов ANSI. Первым символом в наборе символов Windows является знак пробела. Он имеет шестнадцатеричное значение 0x20 (десятичное число 32). Последний символ в наборе символов Windows имеет шестнадцатеричное значение 0xFF (десятичное число 255).

Во многих шрифтах определяется символ по умолчанию. Всякий раз, когда данный шрифт получает код запроса, которому не соответствует ни один из

его символов, в ответ на этот запрос шрифт посылает символ по умолчанию. Многие шрифты, использующие набор символов Windows, определяют в качестве символа по умолчанию точку (.). Шрифты TrueType и OpenType обычно используют в этом качестве символ открытого поля.

Многие шрифты используют символ абзаца для выделения слов и выравнивания текста. Большинство шрифтов, использующих набор символов Windows, определяют в качестве знака абзаца символ пробела.

В Microsoft Windows версии 3.1 к кодовой странице Windows добавлено 24 символа, обозначающие символы валют, крестики, кавычки, тире и другие служебные символы.

Символьный набор Windows использует для идентификации каждого символа байт или 8 бит, следовательно, максимальное количество символов, которое может содержать данный набор, составляет 256. Этого обычно бывает достаточно для западных языков, включая представление диакритических знаков, используемых во французском, немецком, испанском и других языках. Однако восточные языки используют тысячи различных символов, которые не могут быть закодированы одним байтом. Стремительный рост компьютерного бизнеса привел к необходимости создания двухбайтовых схем кодирования, позволяющих идентифицировать символы по 8-, 16-, 24- или 32-разрядным последовательностям. Эти схемы требуют сложных алгоритмов дешифрации, но даже они не обеспечивают одинакового воспроизведения одного и того же текста на двух различных компьютерах.

Для решения этой задачи был разработан стандарт Unicode, позволяющий кодировать символы 16-разрядным кодом. В результате стандарт Unicode может кодировать до 65 536 символов, что должно быть достаточно для работы со всеми шрифтами, используемыми в настоящее время, со всеми знаками препинания, математическими символами, и в нем еще осталось достаточно свободных кодов для дальнейшего расширения. Стандарт Unicode устанавливает уникальный код для каждого символа, что гарантирует правильное декодирование любого символа любого языка.

Набор символов OEM обычно используется программами, работающими в операционной системе MS DOS. Символам с кодами от 32 до 127 обычно соответствуют те же символы, что и в стандарте ASCII и в наборе символов Windows. Другим кодам в наборе символов OEM (от 0 до 31 и от 128 до 255) соответствуют символы, используемые при отображении текста в программах MS DOS. Эти символы отличаются от символов Windows.

Набор символов Symbol содержит специальные символы, обычно используемые в математических и научных формулах.

Многие принтеры и другие устройства вывода работают со шрифтами, использующими символьные наборы, отличающиеся от символьных наборов Windows и OEM. Примером такого символьного набора может служить расширенный набор двоично-десятичных символов (EBCDIC). Чтобы исполь-

зовать подобный символьный набор, драйвер принтера должен обеспечивать трансляцию набора символов Windows в данный специализированный символьный набор.

Шрифт может быть уже установлен на используемом приложением устройстве вывода информации. В противном случае для вывода текста необходимо загрузить данный шрифт в системную таблицу шрифтов. Таблица шрифтов представляет собой внутренний массив, идентифицирующий загружаемые в устройство шрифты, используемые данным приложением Win32. Приложение может получать имена шрифтов, загруженных на данный момент в устройство и хранящихся во внутренней таблице шрифтов, используя функции `EnumFontFamilies` и `ChooseFont`.

Функция `EnumFontFamilies`, первым аргументом которой является дескриптор контекста устройства, позволяет получить информацию для каждого шрифта семейства, указанного во втором ее аргументе, и передать эту информацию функции обратного вызова, указатель на которую является ее третьим аргументом, а указатель на передаваемые функции обратного вызова данных содержится в четвертом аргументе функции `EnumFontFamilies`. Эта функция возвращает последнее значение, возвращенное функцией обратного вызова.

Функция `ChooseFont` вызывает диалоговое окно **Шрифт**, изображенное на рис. 6.1. Данное окно позволяет пользователю выбрать атрибуты логического шрифта. Эти атрибуты включают в себя имя, начертание, стиль (полужирный, наклонный или обычный), размер шрифта в пиках и эффекты (подчеркивание, зачеркивание и цвет текста). Единственным параметром данной функции является указатель на структуру `CHOOSEFONT`.

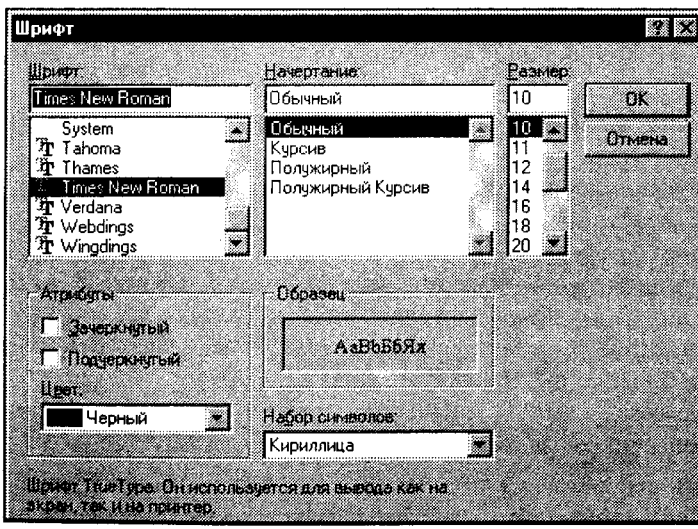


Рис. 6.1. Диалоговое окно **Шрифт**

Приложение может устанавливать шрифт, используя вызов функции AddFontResource, единственным параметром которой является имя файла, содержащего ресурс данного шрифта. Этот файл может иметь расширение .fon, .fnt, .ttf или .fot.

При использовании шрифтов TrueType и OpenType иногда необходим дополнительный шаг, прежде чем шрифт может быть установлен в таблицу шрифтов. Некоторые разработчики шрифтов поставляют для этих шрифтов только файлы данных (имеющие расширение .ttf). Чтобы система могла загрузить подобный файл, требуется создать соответствующий файл заголовка (имеющие расширение .fot). Чтобы создать этот файл заголовка, приложение должно вызвать функцию CreateScalableFontResource и передать ей имя файла данных шрифта в качестве третьего аргумента. Первый аргумент данной функции определяет, будет ли этот шрифт впоследствии модифицироваться. Второй содержит указатель на имя создаваемого файла заголовка, а четвертый параметр — путь к этому файлу. Когда файл заголовка создан, приложение может устанавливать шрифт, для чего необходимо вызвать функцию AddFontResource и передать ей в качестве аргумента имя нового файла заголовка.

Когда приложение заканчивает использовать установленный шрифт, оно должно удалить его, вызывая функцию RemoveFontResource, указав ей в качестве аргумента то же имя файла, что было указано в качестве аргумента функции AddFontResource.

Всякий раз, когда приложение вызывает функции, добавляющие или удаляющие ресурсы шрифта, оно должно также вызвать функцию SendMessage и послать сообщение WM_FONTCHANGE всем главным окнам приложений в системе. Это сообщение извещает другие приложения о том, что внутренняя таблица шрифтов была изменена приложением, которое добавило или удалило шрифт.

После того, как пользователь выбрал шрифт в диалоговом окне **Шрифт** и нажал кнопку **ОК**, функция ChooseFont инициализирует элементы структуры LOGFONT с атрибутами запрошенного шрифта. Чтобы использовать этот шрифт для действий с текстовым выводом, приложение должно сначала создать логический шрифт, а затем выбрать этот шрифт в контекст устройства. Под логическим шрифтом понимается содержащееся в приложении описание идеального шрифта. Пользователь может создать логический шрифт при помощи функций CreateFont или CreateFontIndirect. В большинстве случаев целесообразнее вызывать функцию CreateFontIndirect, поскольку функция CreateFont имеет несколько аргументов, а функция CreateFontIndirect использует указатель на объект структуры LOGFONT.

Прежде чем приложение сможет выводить текст с использованием логического шрифта, оно должно найти ему наиболее близкое соответствие среди шрифтов, хранящихся во внутренней памяти устройств, и шрифтов, ресурсы

которых были загружены в операционную систему. Шрифты, хранящиеся в устройстве или в операционной системе, называются *физическими шрифтами*. Процесс нахождения физического шрифта, максимально соответствующего определенному логическому шрифту, называется *подстановкой шрифтов*. Это происходит при вызове из прикладной программы функции `CDC::SelectObject`. Подстановка шрифтов выполняется внутренней процедурой операционной системы, сравнивающей атрибуты запрошенного логического шрифта и атрибуты доступных физических шрифтов.

Функция `CDC::SetMapperFlags` определяет необходимость учета при сопоставлении логического и физических шрифтов коэффициента сжатия, характерного для данного физического устройства. Коэффициент сжатия устройства определяется как отношение ширины и высоты элемента изображения в данном устройстве.

При передаче документов на другой компьютер часто используется технология внедрения шрифтов в документ. Внедрение шрифта гарантирует, что шрифт, определенный в передаваемом файле документа, будет присутствовать на компьютере, получающем данный файл. Не все шрифты могут перемещаться с одного компьютера на другой, поскольку большинство шрифтов имеют лицензию на использование только на одном компьютере. Внедряться могут только шрифты TrueType и OpenType.

Приложения могут внедрять шрифты в документ только по требованию пользователя. Дистрибутив приложения не может содержать документы с внедренными в них шрифтами, и само приложение не может содержать внедренный шрифт. Если совместно с приложением поставляются шрифты в любом формате, при этом должны быть подтверждены права собственности владельца шрифта.

Несоблюдение приведенных выше правил может быть расценено как нарушение прав собственности поставщика шрифта или лицензионного соглашения пользователя. Лицензия шрифта может содержать разрешение на чтение и запись шрифта, который будет установлен и будет использоваться на компьютере адресата. Лицензия может содержать разрешение только на чтения. Подобное разрешение допускает просмотр и печать (но не изменение) документа на компьютере адресата. Документы с внедренными шрифтами, имеющими лицензию только для чтения, сами являются документами только для чтения. Внедренные шрифты, имеющие лицензию только для чтения, не могут быть извлечены из документа и установлены на компьютере адресата.

Приложение может определять состояние лицензии путем вызова функции `CDC::GetOutlineTextMetrics` и анализа переменной `otmfsType`, являющейся членом структуры `OUTLINETEXTMETRIC`, объект которой является третьим аргументом данной функции. Если младший бит переменной `otmfsType` установлен, то для данного шрифта внедрение запрещено. В противном случае

этот шрифт может быть внедрен. Если в данной переменной установлен старший бит, то данный шрифт имеет лицензию только на чтение.

Для внедрения шрифта приложение может использовать функцию `CDC::GetFontData`, производящую чтение файла шрифта. Чтобы приложение прочитало данный файл полностью, необходимо установить значения аргументов данной функции `dwTable` и `dwOffset` в 0, а аргумента `cbData` — в −1.

Получив всю необходимую информацию о шрифте, приложение может сохранить ее вместе с документом, используя любой подходящий формат. Большинство приложений формируют в документе специальный каталог для шрифтов, перечисляя в нем все внедренные шрифты и режимы работы с ними. Приложение может использовать для идентификации шрифта переменные `otmpStyleName` и `otmFamilyName`, являющиеся объектами структуры `OUTLINETEXTMETRIC`.

Если для внедренного шрифта установлен бит только для чтения, приложение должно зашифровать данный шрифт перед его сохранением в документе. Метод шифрования не должен быть слишком сложным. Для этого можно, например, использовать операцию `XOR` между данными шрифта и некоторой константой, определенной в приложении.

Работа с текстом

Вывод текста является наиболее распространенным типом вывода графической информации в рабочую область окон приложений Win32. Текстовая информация используется в системах подготовки документов, позволяющих задать достаточно сложные параметры форматирования текста, при работе с электронными таблицами, базами данных, системами автоматизированного проектирования и другими приложениями.

Win32 API обеспечивает функционально полный набор методов, позволяющих форматировать и отображать текст в рабочей области окна приложения и выводить его на принтер. Эти функции могут быть разделены на две категории: функции форматирования текста (или подготовки его к выводу) и функции фактического вывода текста на устройство. *Функции форматирования* осуществляют выравнивание текста, устанавливают межсимвольные интервалы, устанавливают цвет текста и цвет фона и выравнивают текст. *Функции вывода* текста осуществляют отображение отдельных его символов или всей строки текста.

Форматирование текста

Функции форматирования текста могут быть условно разделены на следующие группы:

1. Функции установки атрибутов форматирования текста в контексте устройства.

2. Функции определения ширины символа.
3. Функции определения ширины и высоты строки текста.

Группа установки атрибутов форматирования текста в контексте устройства состоит из шести функций: `CDC::SetTextAlign`, `CDC::SetTextCharacterExtra`, `CDC::SetTextJustification`, `CDC::SetTextColor`, `CDC::SetBkColor` и `CDC::SetBkMode`. Эти функции устанавливают параметры выравнивания текста, межсимвольный интервал, параметры текста, а также цвет текста и фона. Кроме того, существуют шесть функций, используемых для извлечения текущих установок форматирования текста из любого контекста устройства: `CDC::GetBkColor`, `CDC::GetBkMode`, `CDC::GetTextAlign`, `CDC::GetTextCharacterExtra`, `CDC::GetTextColor` и `CDC::GetTextExtentPoint32`.

Функция `CDC::SetTextAlign` позволяет определить, каким образом приложение должно выравнивать строку при выводе текста на устройство. Эта функция может использоваться для выравнивания заголовков, номеров страниц, ссылок и т. д. Система выравнивает строку в зависимости от значения второго аргумента данной функции. По умолчанию выравнивание выполняется по левому краю текста. Приложение может получить информацию о текущих установках выравнивания текста в контексте устройства с помощью вызова функции `CDC::GetTextAlign`.

Функция `CDC::SetTextCharacterExtra` позволяет изменить межсимвольный интервал при выполнении всех операций вывода текста с использованием данного контекста устройства. По умолчанию для любого контекста устройства устанавливается нулевое значение межсимвольного интервала. Приложение может получить информацию о текущем значении межсимвольного интервала для данного контекста устройства с помощью вызова функции `CDC::GetTextCharacterExtra`.

Функции `CDC::GetTextExtentPoint32` и `CDC::SetTextJustification` позволяют установить параметры выравнивания строки текста по формату. Эта операция широко применяется во всех настольных издательских системах и в большинстве приложений подготовки текстов. Функция `CDC::GetTextExtentPoint32` вычисляет ширину и высоту строки текста. На основании информации о ширине и высоте строки текста функция `CDC::SetTextJustification` определяет размеры дополнительных интервалов между словами текста в строке, позволяющими выровнять строку по ширине.

Для установки цвета текста, выводимого в рабочую область окна приложения или на цветной принтер, используется функция `CDC::SetTextColor`. Для установки цвета фона, на котором выводится отдельный символ, используется функция `CDC::SetBkColor`, а для установки цвета фона всей рабочей области окна используется функция `CDC::SetBkMode`.

По умолчанию для вывода текста в контексте устройства выбирается черный цвет, а для отображения фона — белый. При этом выбирается режим непро-

зрачного фона. Приложение может получить информацию о текущих установках цвета текста в контексте устройства с помощью вызова функции `CDC::GetTextColor`. Соответствующая информация о цвете фона символа и рабочей области окна возвращается функциями `CDC::GetBkColor` и `CDC::GetBkMode`.

Приложения должны получать информацию о ширине символов при размещении строк текста на странице или в окне. Для этой цели приложение может использовать одну из четырех функций. Две из этих функций позволяют определить ширину символа с межсимвольным интервалом, а две другие — фактическую ширину символа.

Функции `CDC::GetCharWidth32` и `CDC::GetCharWidthFloat` позволяют определить ширину символа с межсимвольным интервалом. Под шириной символа с межсимвольным интервалом понимается расстояние, на которое курсор на дисплее или печатающая головка принтера должны переместиться перед печатью следующего символа в строке текста. Функция `CDC::GetCharWidth32` возвращает ширину символа с межсимвольным интервалом в формате целого числа. Если требуется большая точность, приложение может использовать функцию `CDC::GetCharWidthFloat`, позволяющую получить эту величину в формате числа с плавающей запятой.

Функции `CDC::GetCharABCWidths` и `CDC::GetCharABCWidthsFloat` позволяют определить ширину символа. Функция `CDC::GetCharABCWidthsFloat` работает со всеми шрифтами, а функция `CDC::GetCharABCWidths` — только со шрифтами TrueType и OpenType.

Ширина с межсимвольным интервалом может быть больше или меньше фактической ширины символа вследствие того, что символы в строке могут нависать друг над другом.

Кроме информации о ширине каждого символа в строке, приложение также нуждается в информации о ширине и высоте всех строк. Эта информация может быть получена при вызове одной из двух функций: `CDC::GetTextExtentPoint32` или `CDC::GetTabbedTextExtent`. Если строка не содержит символов табуляции, то для определения высоты и ширины строки приложение может использовать функцию `CDC::GetTextExtentPoint32`. В противном случае следует вызвать функцию `CDC::GetTabbedTextExtent`.

Кроме того, Win32 API содержит функцию `GetTextExtentExPoint`, способную работать с приложениями, обеспечивающими автоматический перенос слов. Эта функция позволяет получить количество символов в определенной строке, попадающее в определенный диапазон.

Некоторые приложения устанавливают межстрочный интервал в зависимости от максимального надстрочного элемента символа и минимального подстрочного элемента используемого в них шрифта. Для этого приложение должно вызвать функцию `CDC::GetTextMetrics`, а затем использовать значе-

ния переменных tmAscent и tmDescent, являющихся членами структуры TEXTMETRIC.

В некоторых приложениях величины максимального надстрочного и минимального подстрочного элементов отличаются от соответствующих типографских величин данных параметров. В шрифтах TrueType и OpenType типографские величины надстрочного и подстрочного элементов обычно определяются как верхняя точка символа "f" и нижняя точка символа "g", соответственно. Приложение может получить величину типографских надстрочного и подстрочного элементов для шрифтов TrueType или OpenType с помощью вызова функции CDC::GetOutlineTextMetrics и проверки значений переменных otmMacAscent и otmMacDescent, являющихся членами структуры OUTLINETEXTMETRIC.

Для определения физических размеров шрифтов TrueType или OpenType приложение должно вызвать функцию CDC::GetOutlineTextMetrics. Для получения подобной информации о любом другом шрифте необходимо вызвать функцию CDC::GetTextMetrics. Для определения возможностей вывода текста на конкретном устройстве приложение может вызвать функцию CDC::GetDeviceCaps, возвращающую как физические, так и логические размеры шрифтов.

Логический дюйм представляет собой меру длины, используемую системой для формирования на экране удобочитаемых шрифтов на экране. Он примерно на 30—40% больше физического дюйма. Использование логических дюймов приводит к отсутствию точного соответствия между текстом, выведенным на экран и на принтер. Разработчики приложений должны понимать, что при одновременном выводе на экран и на принтер текстовые и графические объекты имеют различный масштаб.

Вывод текста

После того как приложение выбрало соответствующий шрифт, установило для него требуемые параметры форматирования и вычислило ширину символов и размеры строки текста, оно может приступить к выводу данного текста на устройство отображения информации. Для этого может быть использована одна из четырех функций. Функции CDC::DrawText и CDC::TabbedTextOut представляют собой методы Windows Manager и хранятся в библиотеке User.dll, а остальные две являются частью GDI и содержатся в библиотеке Gdi.dll.

При вызове из прикладной программы одной из этих функций операционная система передает запрос графической подсистеме, которая, в свою очередь, передает этот запрос соответствующему драйверу устройства. На уровне драйвера устройства эти обращения преобразуются в одно или несколько обращений к функциям-членам класса драйвера: CDC::ExtTextOut или CDC::TextOut. Наиболее быстро задача отображения текста решается при по-

мощи функции CDC::ExtTextOut, которая сразу же преобразуется в функцию ExtTextOut запрашиваемого устройства. Однако в некоторых случаях приложению выгоднее вызвать одну из оставшихся трех функций. Например, вывод многострочного текста в прямоугольную область на экране наиболее эффективно выполняется функцией CDC::DrawText. Для создания таблицы, имеющей несколько столбцов форматированного текста, лучше всего воспользоваться функцией CDC::TabbedTextOut.

Программа вывода текста

Текст демонстрационного приложения, выводящего текст на экран, расположен в каталоге \TextApp дискеты, прилагаемой к данной книге. Чтобы самостоятельно создать данное приложение:

1. Создайте заготовку многооконного приложения Windows по методике, описанной в главе 1, и назовите его TextApp.
2. Во вкладке **FileView** окна **Workspace** последовательно раскройте папки \TextApp Files и \Source Files и дважды щелкните левой кнопкой мыши на значке **TextAppView.cpp**. В окне редактирования появится содержимое этого файла.
3. Вызовите мастер ClassWizard и раскройте вкладку **Message Maps**.
4. В окне списка **Object IDs:** выделите имя класса CTextAppView, в окне списка **Messages:** выделите идентификатор сообщения WM_LBUTTONDOWN и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится функция обработки данного сообщения OnLButtonDown.
5. Нажмите кнопку **Edit Code**. Диалоговое окно **MFC ClassWizard** закроется, а текстовый курсор в окне редактирования файла переместится на текст заготовки данной функции.
6. Отредактируйте функцию OnLButtonDown в соответствии с приведенным ниже текстом:

```
void CTextAppView::OnLButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  CClientDC  clientDC(this);
  LOGFONT    logFont;
  CFont    font;
  CString    Out;

  logFont.lfHeight            = 12;
    logFont.lfWidth           = 0;
    logFont.lfEscapement      = 0;
    logFont.lfOrientation     = 0;
    logFont.lfWeight          = FW_NORMAL;
```

```
    logFont.lfItalic          = 0;
    logFont.lfUnderline       = 0;
    logFont.lfStrikeOut       = 0;
    logFont.lfCharSet         = ANSI_CHARSET;
    logFont.lfOutPrecision    = OUT_DEFAULT_PRECIS;
    logFont.lfClipPrecision   = CLIP_DEFAULT_PRECIS;
    logFont.lfQuality         = PROOF_QUALITY;
    logFont.lfPitchAndFamily = VARIABLE_PITCH | FF_ROMAN;
  strcpy(logFont.lfFaceName, "MS Sans Serif");

  sprintf( Out.GetBuffer(16),"[%d,%d]", point.x, point.y);

  Out.ReleaseBuffer();
  font.CreateFontIndirect( &logFont);

  CFont*  oldFont = clientDC.SelectObject( &font);

  clientDC.TextOut( point.x, point.y, Out);
  clientDC.SelectObject( oldFont);

  CView::OnLButtonDown(nFlags, point);
}
```

7. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится пустое окно.
8. Нажмите кнопку **Развернуть** в заголовке окна приложения. Окно приложения развернется на весь экран.
9. Нажмите кнопку **Развернуть** в заголовке окна документа.
10. Несколько раз щелкните левой кнопкой мыши в рабочей области окна. Оно примет вид, изображенный на рис. 6.2.

Рассмотрим текст функции обработки сообщения `OnLButtonDown`. В данной функции создаются объекты классов контекста устройства, структуры логического шрифта, класса шрифта и строкового класса. После этого заполняются переменные в объекте структуры логического шрифта, а в объект строкового класса записывается информация о текущем положении курсора мыши.

На основе объекта структуры логического шрифта создается объект класса шрифта, который затем выбирается в контекст устройства при помощи функции `CDC::SelectObject`, при этом указатель на объект класса старого шрифта сохраняется в переменной `oldFont`. Функция `CDC::TextOut` выводит информацию, содержащуюся в объекте строкового класса, на экран. Последние два оператора данной функции возвращают в контекст устройства старый шрифт и вызывают соответствующую функцию обработки сообщения базового класса.

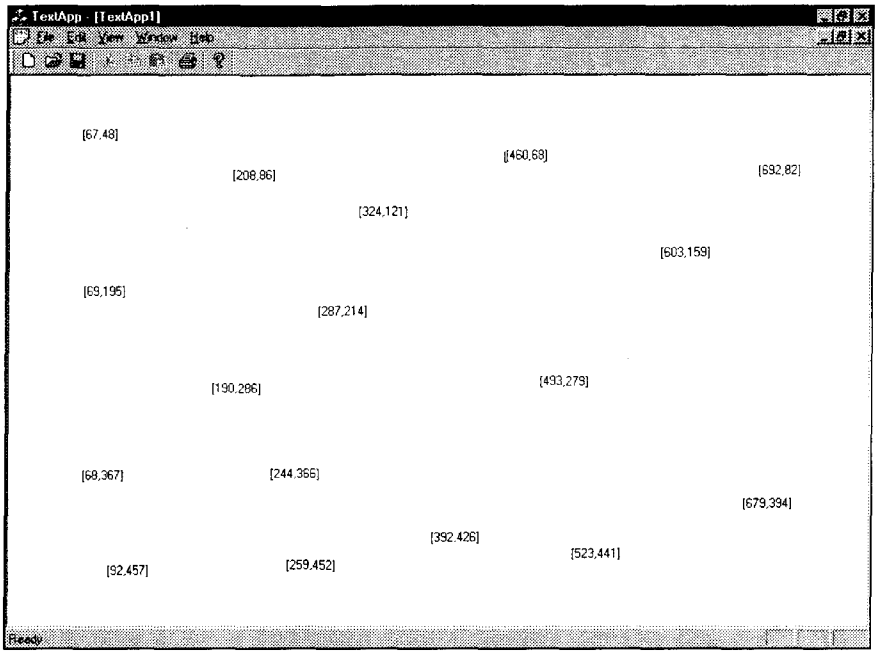


Рис. 6.2. Результат работы приложения TextApp

Перерисовка окна

Попробуйте изменить размер окна в описанном выше приложении. Вся отображавшаяся в нем информация исчезнет. Дело в том, что, как указывалось выше, при изменении размеров или положения окна вызывается функция `CView::OnDraw`, выполняющая все необходимые для этого операции. Поскольку мы не внесли никаких изменений в текст этой функции, то при ее вызове выполняется процедура по умолчанию, состоящая в очистке содержимого окна.

Чтобы внести необходимые изменения в приложение TextApp:

1. Откройте окно редактирования файла TextAppDoc.cpp.
2. Щелкните правой кнопкой мыши на строке `#include "TextAppDoc.h"`. Появится контекстное меню, изображенное на рис. 6.3.
3. Выберите команду **Open Document "TextAppDoc.h"**. Откроется окно редактирования заголовочного файла TextAppDoc.h.
4. В тексте описания класса `CTextAppDoc` в раздел `// Attributes` после модификатора прав доступа `public:` вставьте следующий текст:

```
CUIntArray  aX;
CUIntArray  aY;
```

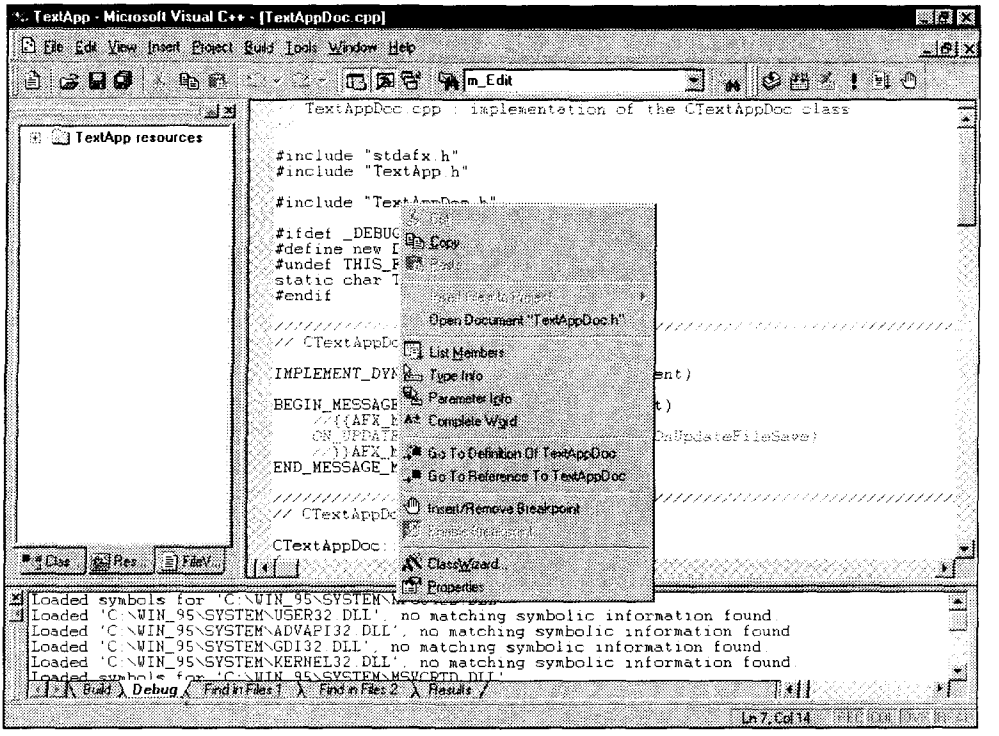


Рис. 6.3. Контекстное меню окна редактирования файла реализации

5. Откройте окно редактирования файла TextAppView.cpp и вставьте в текст функции `OnLButtonDown` после объявления всех объектов класса `CString` следующий текст:

```
CTextAppDoc*  lpDoc = GetDocument();

lpDoc-> aX.Add( point.x);
lpDoc-> aY.Add( point.y);
```

6. Отредактируйте функцию `OnDraw` в соответствии с приведенным ниже текстом:

```
void CTextAppView::OnDraw(CDC* pDC)
{
  CTextAppDoc* pDoc = GetDocument();
  ASSERT_VALID(pDoc);
  // TODO: add draw code for native data here

  LOGFONT    logFont;
  CFont     font;
  CString    Out;
```

```
    logFont.lfHeight          = 12;
      logFont.lfWidth         = 0;
      logFont.lfEscapement    = 0;
      logFont.lfOrientation   = 0;
      logFont.lfWeight        = FW_NORMAL;
      logFont.lfItalic        = 0;
      logFont.lfUnderline     = 0;
      logFont.lfStrikeOut     = 0;
      logFont.lfCharSet       = ANSI_CHARSET;
      logFont.lfOutPrecision  = OUT_DEFAULT_PRECIS;
      logFont.lfClipPrecision = CLIP_DEFAULT_PRECIS;
      logFont.lfQuality       = PROOF_QUALITY;
      logFont.lfPitchAndFamily = VARIABLE_PITCH | FF_ROMAN;
    strcpy(logFont.lfFaceName, "MS Sans Serif");

    font.CreateFontIndirect( &logFont);

    CFont*  oldFont = pDC-> SelectObject( &font);

    for(int i=0; i < pDoc-> aX.GetSize(); i++){

      sprintf( Out.GetBuffer(16),"[%d,%d]", pDoc-> aX[i], pDoc-> aY[i]);

      Out.ReleaseBuffer();

      pDC-> TextOut( pDoc-> aX[i], pDoc-> aY[i], Out);
    }

    pDC-> SelectObject( &oldFont);
  }
```

7. Нажмите клавишу <F5> и запустите приложение на исполнение.
8. Несколько раз щелкните левой кнопкой мыши в рабочей области окна.
9. Переместите окно документа или измените его размер. Информация останется в окне.

Сравнение текстов функций `OnLButtonDown` и `OnDraw` выявляет почти полную их идентичность, за исключением того, что в них используются различные классы контекста устройств, а в функции `OnDraw` используются записанные в специальные массивы координаты предыдущих нажатий левой кнопки мыши.

Использование перьев

Работа с перьями намного проще работы со шрифтами, поскольку при этом не нужно использовать сложные структуры данных, подобные структуре `LOGFONT`. Перо имеет всего три параметра: стиль начертания линии, толщину и цвет.

Текст демонстрационного приложения, выводящего текст на экран, расположен в каталоге \LineApp дискеты, прилагаемой к данной книге. Чтобы самостоятельно создать данное приложение:

1. Создайте заготовку многооконного приложения Windows по методике, описанной в главе 1, и назовите его LineApp.
2. Во вкладке **FileView** окна **Workspace** последовательно раскройте папки \LineApp Files и \Source Files и дважды щелкните левой кнопкой мыши на значке **LineAppView.cpp**. В окне редактирования появится содержимое данного файла.
3. Вызовите мастер ClassWizard и раскройте вкладку **Message Maps**.
4. В окне списка **Object IDs:** выделите имя класса `CLineAppView`, в окне списка **Messages:** выделите идентификатор сообщения `WM_LBUTTONDOWN` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится функция обработки данного сообщения `OnLButtonDown`.
5. Нажмите кнопку **Edit Code**. Диалоговое окно **MFC ClassWizard** закроется, а текстовый курсор в окне редактирования файла переместится на текст заготовки данной функции.
6. Повторите эту операцию для идентификаторов сообщений `WM_LBUTTONUP`, `WM_RBUTTONDOWN`, `WM_RBUTTONDBLCLK` и `WM_MOUSEMOVE`.
7. Нажмите кнопку **OK**. Диалоговое окно **MFC ClassWizard** закроется.
8. Перейдите на вкладку **ClassView** окна **Workspace**. Раскройте папку \LineApp Classes и щелкните правой кнопкой мыши на значке **CLineAppView**.
9. В появившемся контекстном меню выберите команду **Add Member Variable...**. Появится диалоговое окно **Add Member Variable**, изображенное на рис. 6.4.

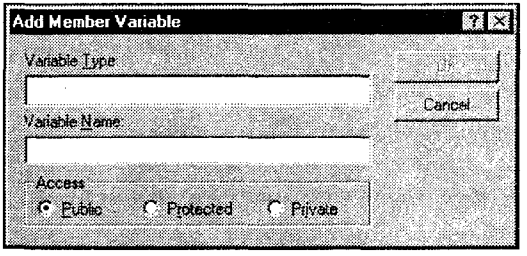


Рис. 6.4. Диалоговое окно **Add Member Variable**

10. В текстовое поле **Variable Type:** введите тип указателя на объект класса `CClientDC*`, в текстовое поле **Variable Name:** введите идентификатор переменной `lpDC`, установите переключатель **Access** в положение **Protected** и нажмите кнопку **OK**.
11. Повторите пп. 8—10 для переменной `lpPen`, имеющей тип `CPen*`.

12. Повторите пп. 8—10 для переменной `oldPen`, имеющей тип `CPen*`.
13. Повторите пп. 8—10 для переменной `nWidth`, имеющей тип `int`.
14. Повторите пп. 8—10 для переменной `isDown`, имеющей тип `bool`.
15. Откройте окно редактирования файла ListAppView.cpp и отредактируйте текст его конструктора в соответствии с приведенным ниже листингом:

```
CLineAppView::CLineAppView()
{
  // TODO: add construction code here

  lpDC   = NULL;
  lpPen  = NULL;
  oldPen  = NULL
  nWidth  = 1;
  isDown  = false;
}
```

16. Отредактируйте текст функций обработки сообщений в соответствии с приведенным ниже листингом:

```
////////////////////////////////////////////////////////////////////////
// CLineAppView message handlers

void CLineAppView::OnLButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  lpDC   = new CClientDC( this);
  lpPen  = new CPen( PS_SOLID, nWidth, RGB(0,0,0));
  oldPen  = lpDC-> SelectObject( lpPen);
  isDown  = true;

  lpDC-> MoveTo( point.x, point.y);

  CView::OnLButtonDown(nFlags, point);
}

void CLineAppView::OnLButtonUp(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  lpDC-> SelectObject( oldPen);

  delete  lpPen;
  delete  lpDC;

  isDown  = false;

  CView::OnLButtonUp(nFlags, point);
}
```

```
void CLineAppView::OnRButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  nWidth++;

  CView::OnRButtonDown(nFlags, point);
}

void CLineAppView::OnRButtonDblClk(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  if( nWidth > 2)
    nWidth -= 2;
  else
    nWidth = 1;

  CView::OnRButtonDblClk(nFlags, point);
}

void CLineAppView::OnMouseMove(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  if( isDown)
    lpDC->LineTo( point.x, point.y);

  CView::OnMouseMove(nFlags, point);
}
```

17. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится пустое окно документа.
18. Нажмите кнопку **Развернуть** в заголовке окна приложения. Окно приложения развернется на весь экран.
19. Нажмите кнопку **Развернуть** в заголовке окна документа.
20. Нажмите левую кнопку мыши в рабочей области окна и, удерживая ее нажатой, переместите мышь. В окне появится линия, соответствующая траектории перемещения указателя мыши.
21. Отпустите левую кнопку мыши, нажмите ее правую кнопку и повторите операцию. Линия станет толще.
22. Дважды щелкните правой кнопкой мыши и нарисуйте еще одну линию. Она станет тоньше. Результат подобных операций приведен на рис. 6.5.

При нажатии левой кнопки мыши в рабочей области окна вызывается функция обработки сообщения `OnLButtonDown`, в которой создаются новые объекты классов контекста устройства и пера, и указатели на них присваи-

ваются переменным-членам класса представления. Объект класса пера выбирается в контекст устройства, а старый объект данного класса сохраняется в переменной-члене класса представления. После этого логической переменной присваивается значение, указывающее на то, что нажата левая кнопка мыши, и текущая позиция перемещается в точку, соответствующую текущему положению курсора мыши.

При перемещении мыши в рабочей области окна вызывается функция обработки сообщения `OnMouseMove`, в которой проверяется, нажата ли в данный момент левая кнопка мыши, и в случае положительного исхода проверки на экран выводится линия от текущей позиции до текущего положения курсора, при этом текущая позиция перемещается в конец нарисованной линии.

При отпускании левой кнопки мыши в рабочей области окна вызывается функция обработки сообщения `OnLButtonUp`, которая выбирает в контекст устройства сохраненный объект класса пера и уничтожает объекты класса контекста устройства и класса пера. Логической переменной присваивается значение, указывающее на то, что левая кнопка мыши отпущена.

При нажатии правой кнопки мыши в рабочей области окна вызывается функция обработки сообщения `OnRButtonDown`, увеличивающая параметр ширины линии на одну дискрету.

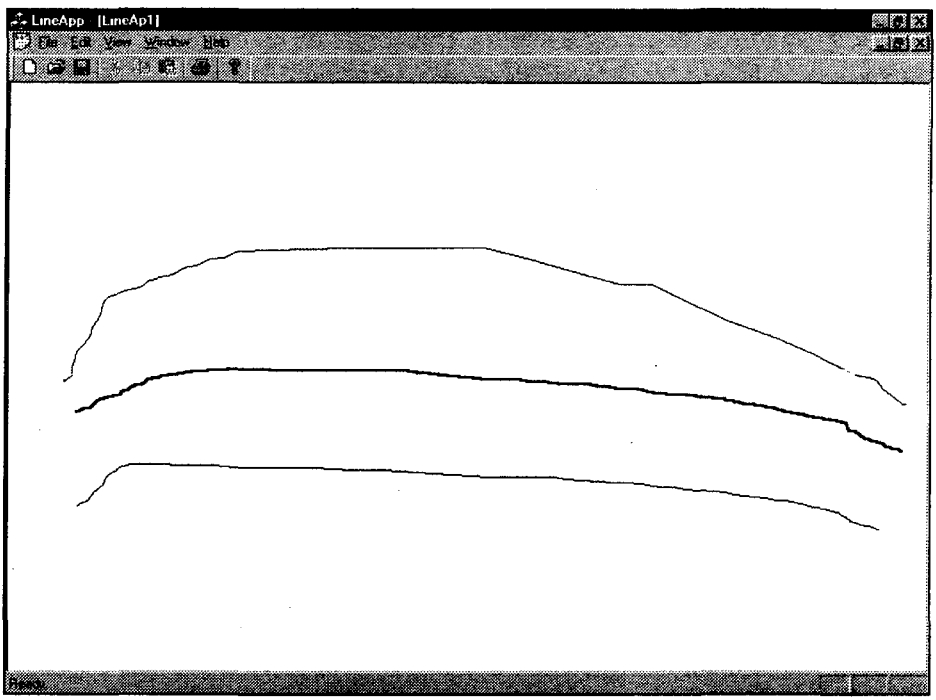


Рис. 6.5. Результат работы приложения LineApp

При двойном нажатии правой кнопки мыши в рабочей области окна вызывается функция обработки сообщения `OnRButtonDblClk`. Данная функция имеет более сложную структуру, чем функция `OnRButtonDown`. Это связано прежде всего с тем, что значение ширины линии должно быть положительным, поэтому, прежде чем ее уменьшать, следует проверить, не находится ли она уже на нижнем пределе.

С другой стороны, двойной щелчок кнопкой мыши состоит из двух одиночных щелчков, и система не может не послать сообщение о первом нажатии кнопки на соответствующую обработку, поскольку она не обладает телепатическими способностями и не может знать заранее, какой именно щелчок собирается выполнить пользователь — одиночный или двойной. Кроме того, может поступить сообщение о перемещении мыши с нажатой кнопкой. Поэтому при двойном щелчке правой кнопкой мыши сначала вызывается функция обработки сообщения `OnRButtonDown`, увеличивающая ширину линии на одну дискрету, и только потом — функция обработки сообщения `OnRButtonDblClk`, которая должна уменьшить эту величину на одну дискрету. Поэтому в данной функции ширина линии уменьшается на две дискреты. Если ширина линии уже находилась на нижнем пределе, то эта функция должна только восстановить исходное значение.

Работа с кистью

Если перья используются для рисования линий и фигур, то кисти предназначены для заполнения замкнутых пространств определенными цветами или текстурами. *Кисть* может заполнять область равномерным фоном любого цвета, использовать стандартные трафареты (pattern) или определенные пользователем битовые поля.

Текст приложения, демонстрирующего принципы работы с кистью, расположен в каталоге \BrushApp дискеты, прилагаемой к данной книге. Чтобы самостоятельно создать данное приложение:

1. Создайте заготовку многооконного приложения Windows по методике, описанной в главе 1, и назовите ее BrushApp.
2. Во вкладке **FileView** окна **Workspace** последовательно раскройте папки \BrushApp Files и \Source Files и дважды щелкните левой кнопкой мыши на значке **BrushAppView.cpp**. В окне редактирования появится содержимое данного файла.
3. Вызовите мастер ClassWizard и раскройте вкладку **Message Maps**.
4. В окне списка **Object IDs:** выделите имя класса `CBrushAppView`, в окне списка **Messages:** выделите идентификатор сообщения `WM_LBUTTONDOWN` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится функция обработки данного сообщения `OnLButtonDown`.

5. Повторите эту операцию для идентификатора сообщения WM_RBUTTONDOWN.
6. Нажмите кнопку **OK**. Диалоговое окно **MFC ClassWizard** закроется.
7. Перейдите на вкладку **ClassView** окна **Workspace**. Раскройте папку \BrushApp Classes и щелкните правой кнопкой мыши на значке **CBrushAppView**.
8. В появившемся контекстном меню выберите команду **Add Member Variable...**. Появится диалоговое окно **Add Member Variable**.
9. В текстовое поле **Variable Type:** введите тип int, в текстовое поле **Variable Name:** введите идентификатор переменной nBrushStyle, установите переключатель **Access** в положение **Protected** и нажмите кнопку **OK**.
10. Откройте окно редактирования файла BrushAppView.cpp и вставьте в его конструктор следующую строку:

```
nBrushStyle  = 0;
```

11. Отредактируйте текст функций обработки сообщений в соответствии с приведенным ниже листингом:

```
//////////////////////////////////////////////////////////////////////
// CBrushAppView message handlers

void CBrushAppView::OnLButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  CClientDC  dc(this);
  CBrush*    lpBrush;
  CPen    pen( PS_SOLID, 0, RGB(0,0,255));
  CPen*    oldPen = dc.SelectObject( &pen);

  if( nBrushStyle)
    lpBrush = new CBrush( nBrushStyle - 1, RGB(0,0,0));
  else
    lpBrush = new CBrush( RGB(255,0,0));

  CBrush*    oldBrush = dc.SelectObject( lpBrush);

  dc.Rectangle( point.x, point.y, point.x + 40, point.y + 40);
  dc.SelectObject( oldBrush);
  dc.SelectObject( oldPen);

  delete  lpBrush;

  CView::OnLButtonDown(nFlags, point);
}

void CBrushAppView::OnRButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default
```

```
    nBrushStyle++;
    nBrushStyle  %= 7;

    CView::OnRButtonDown(nFlags, point);
}
```

12. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится пустое окно.
13. Нажмите кнопку **Развернуть** в заголовке окна приложения. Окно приложения развернется на весь экран.
14. Нажмите кнопку **Развернуть** в заголовке окна документа. Окно документа развернется на все окно приложения.
15. Попеременно нажимайте левую и правую кнопки мыши в рабочей области окна. Окно заполнится квадратами, содержащими образцы стандартных трафаретов и заполнения сплошным фоном, как это показано на рис. 6.6.

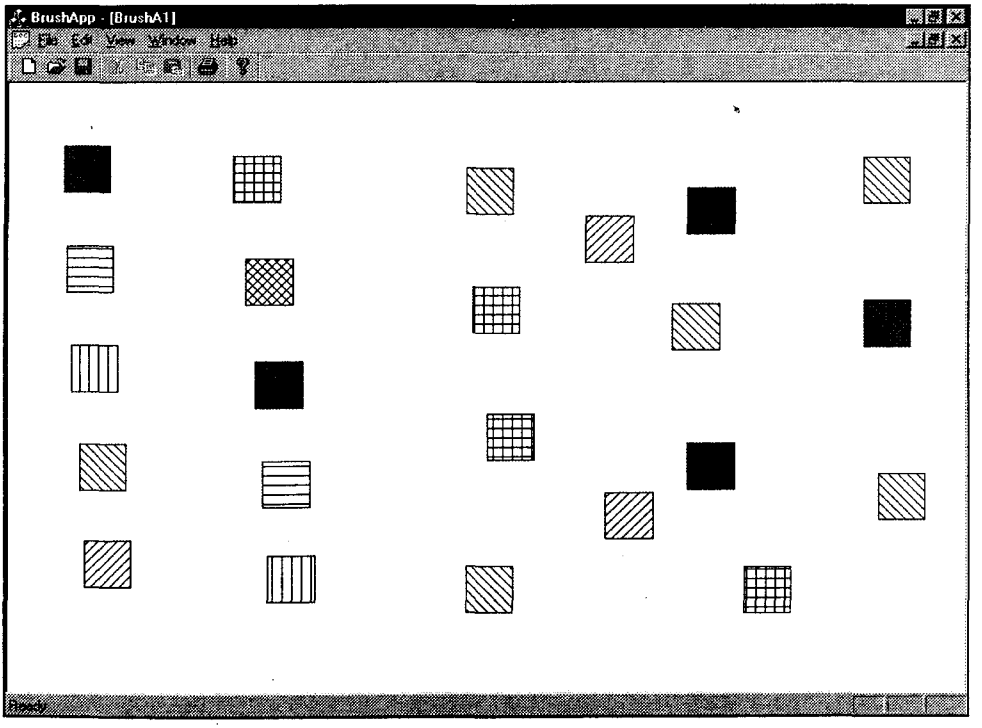


Рис. 6.6. Результат работы приложения BrushApp

При нажатии левой кнопки мыши в рабочей области окна вызывается функция обработки сообщения `OnLButtonDown`, в которой создаются новые

объекты классов контекста устройства и пера. Объект класса пера выбирается в контекст устройства, а старый объект данного класса сохраняется. После этого в зависимости от значения переменной, характеризующей стиль кисти, создается новый объект кисти с соответствующим стилем. Если стиль равен нулю, то создается кисть с равномерным красным фоном. В противном случае создается кисть с одним из стандартных трафаретов. Поскольку нумерация трафаретов начинается с нуля, то при передаче его номера в конструктор кисти из него вычитается единица. Созданная кисть выбирается в контекст устройства, а старый объект кисти сохраняется. После этого в рабочем поле окна вычерчивается прямоугольник, границы которого нарисованы выбранным в контекст пером, а внутренняя поверхность заполнена выбранной в контекст устройства кистью. Координаты верхнего левого угла данного прямоугольника соответствуют текущему положению курсора мыши.

При нажатии правой кнопки мыши в рабочей области окна вызывается функция обработки сообщения `OnRButtonDown`, выполняющая циклическое изменение параметра стиля кисти.

Настройка параметров в диалоговом окне

В предыдущих примерах изменение параметров отображения информации на экране производилось путем нажатия правой кнопки мыши. В реальном приложении такой способ задачи параметров неудобен, поскольку в нем имеется слишком много параметров, меняющихся в широком диапазоне.

В реальных приложениях для изменения значений практически всех параметров приложения используются диалоговые окна. В главе 3 мы уже рассмотрели этот вид окон, но для его исследования использовались специальные диалоговые приложения, в которых диалоговое окно вызывалось непосредственно при вызове приложения и являлось его главным окном. В большинстве случаев диалоговые окна вызываются из функций обработки соответствующих сообщений. Подробно процесс создания диалогового окна рассмотрен в Приложении В в разделе описания класса `CDialog`. Ниже рассмотрим простейший пример использования диалогового окна для изменения параметров кисти на базе приложения BrushApp.

См. также главу 3

Чтобы внести в приложение BrushApp изменения, позволяющие производить настройку параметров кисти в диалоговом окне:

1. Выберите команду меню **File**, **Open Workspace...** (Файл, Открыть проект). Появится диалоговое окно **Open Workspace**, изображенное на рис. 6.7.
2. Раскройте папку \BrushApp и дважды щелкните левой кнопкой мыши на значке **BrushApp.dsw**. Откроется соответствующий проект.
3. Перейдите на вкладку **ResourceView** окна **Workspace**. Раскройте папку \BrushApp resources и щелкните правой кнопкой мыши на папке \Dialog.

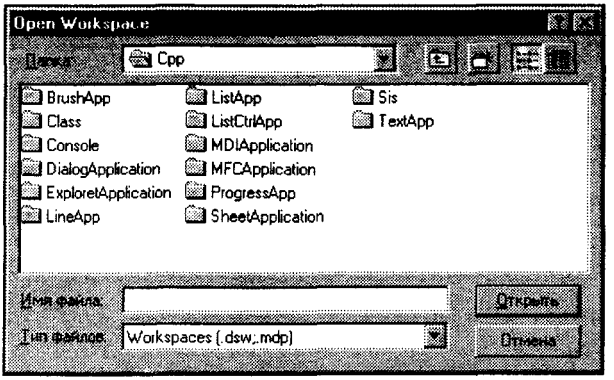


Рис. 6.7. Диалоговое окно **Open Workspace**

4. В появившемся контекстном меню выберите команду **Insert Dialog**. В окне редактирования ресурса появится заготовка диалогового окна, а в раскрывшейся папке \Dialog появится идентификатор ресурса `IDD_DIALOG1`.
5. *Щелкните правой кнопкой мыши на идентификаторе ресурса* `IDD_DIALOG1` и в раскрывшемся меню выберите команду **Properties**. Появится диалоговое окно **Dialog Properties**.
6. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна.
7. В списке **Language:** выберите опцию **Russian** (если она еще не выбрана).
8. В текстовое поле раскрывающегося списка **ID:** введите идентификатор ресурса "IDD_BRUSH_DIALOG".
9. Щелкните левой кнопкой мыши в заготовке диалогового окна. В текстовое поле **Caption:** диалогового окна введите заголовок "Настройка кисти".
10. В панели инструментов **Controls** поместите указатель мыши на кнопку **Radio Button**, нажмите левую кнопку мыши и, удерживая ее, перетащите рамку переключателя в левый верхний угол диалогового окна.
11. В текстовое поле **Caption:** диалогового окна **Radio Button Properties** введите текст "Сплошной фон" и установите флажок **Group**.
12. Повторите пункты 10 и 11 для шести других переключателей, сопоставляя им заголовки: "Горизонтальный", "Вертикальный", "Диагональный", "Наклон влево", "Диагональный", "Наклон вправо", "Прямая клетка" и "Косая клетка". При этом не нужно устанавливать флажок **Group**. При необходимости увеличьте горизонтальный и вертикальный размеры заготовки диалогового окна.
13. В панели инструментов **Controls** поместите курсор мыши на кнопку **Group Box**, нажмите левую кнопку мыши и, удерживая ее, перетащите групповую рамку в левый верхний угол диалогового окна.

14. Отрегулируйте размеры групповой рамки таким образом, чтобы она охватывала все переключатели.
15. В текстовое поле **Caption:** диалогового окна **Group Box Properties** введите заголовок "Трафареты кисти".
16. Выровняйте переключатели внутри групповой рамки, а саму групповую рамку — относительно окна. Окно редактирования ресурса диалогового окна примет вид, изображенный на рис. 6.8.
17. Закройте диалоговое окно **Group Box Properties**.
18. Вызовите мастер ClassWizard из окна редактирования ресурса диалогового окна.
19. В диалоговом окне **Adding a Class** оставьте переключатель в положении **Create a new class** и нажмите кнопку **OK**. Появится диалоговое окно **New Class**.
20. В текстовое поле **Name:** введите имя нового класса `CBrushDialog` и нажмите кнопку **OK**.
21. Раскройте вкладку **Member Variables** в диалоговом окне **MFC ClassWizard**.

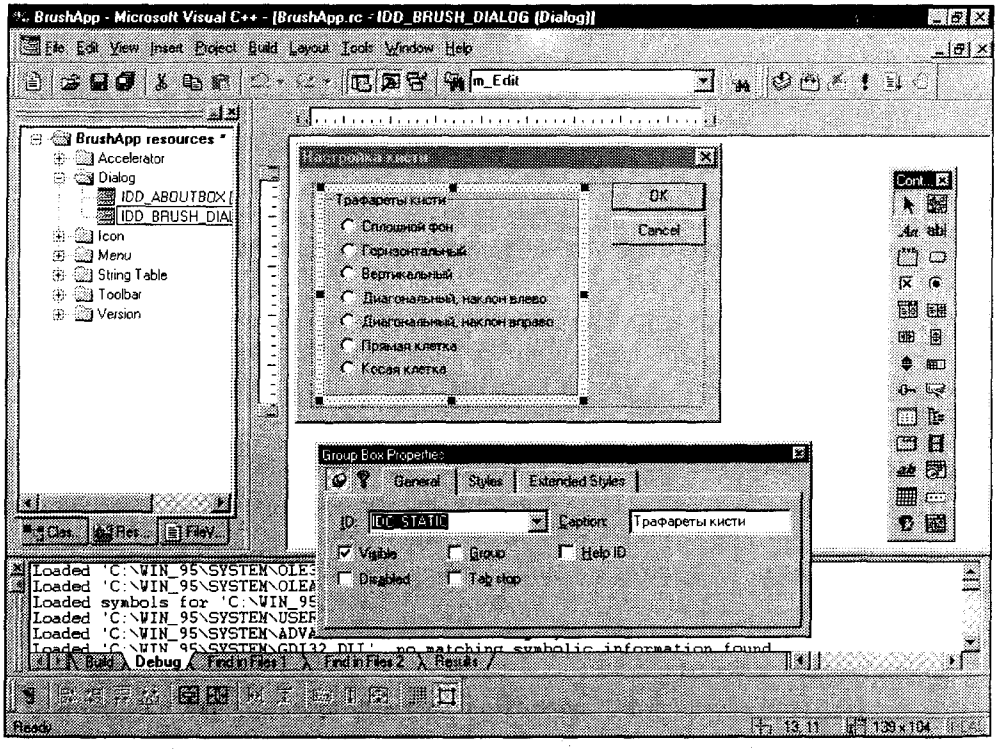


Рис. 6.8. Заготовка диалогового окна

22. В окне списка **Control IDs:** выделите идентификатор ресурса `IDC_RADIO1` и нажмите кнопку **Add Variable...**. Появится диалоговое окно **Add Member Variable**.
23. Введите в текстовое поле **Member variable name:** идентификатор целочисленной переменной `m_nBrush` и нажмите кнопку **OK**.
24. Нажмите кнопку **OK** и закройте диалоговое окно **MFC ClassWizard**.
25. Откройте окно редактирования файла BrushAppView.cpp и вставьте в него после строки `#include "BrushAppView.h"` директиву:

```
#include "BrushDialog.h".
```

26. Измените текст функции `OnRButtonDown` в соответствии с приведенным ниже листингом:

```
void CBrushAppView::OnRButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default
  CBrushDialog  dlg;

  dlg.m_nBrush  = nBrushStyle;

  if( dlg.DoModal() == IDOK)
    nBrushStyle  = dlg.m_nBrush;

  CView::OnRButtonDown(nFlags, point);
}
```

27. Нажмите клавишу <F5> и запустите приложение на исполнение.
28. Щелкните правой кнопкой мыши в рабочей области окна. Появится диалоговое окно **Настройка кисти**, как это показано на рис. 6.9.
29. Установите переключатель **Трафареты кисти** в положение **Косая клетка** и нажмите кнопку **OK**. Диалоговое окно **Настройка кисти** закроется.
30. Щелкните левой кнопкой мыши в рабочей области окна. На месте щелчка появится квадрат, заполненный косой клеткой.
31. Повторите пп. 29—31 для других трафаретов кисти.

Первый оператор функции `OnRButtonDown` создает объект класса `CBrushDialog`. Переменной данного объекта `m_nBrush` присваивается текущее значение переменной `nBrushStyle` и вызывается функция `CDialog::DoModal`, осуществляющая весь обмен информацией с объектом диалогового окна и возвращающая функции `OnRButtonDown` управление только после закрытия диалогового окна.

Если диалоговое окно было закрыто нажатием кнопки **OK**, переменной `nBrushStyle` присваивается значение переменной-члена класса диалога `m_nBrush`. Присваивание переменной `m_nBrush` значения переменной `nBrushStyle` по-

зволяет установить переключатель диалогового окна в положение, соответствующее текущей установке трафарета кисти. Обратное присваивание позволяет передать и сохранить в классе представления новую установку трафарета кисти.

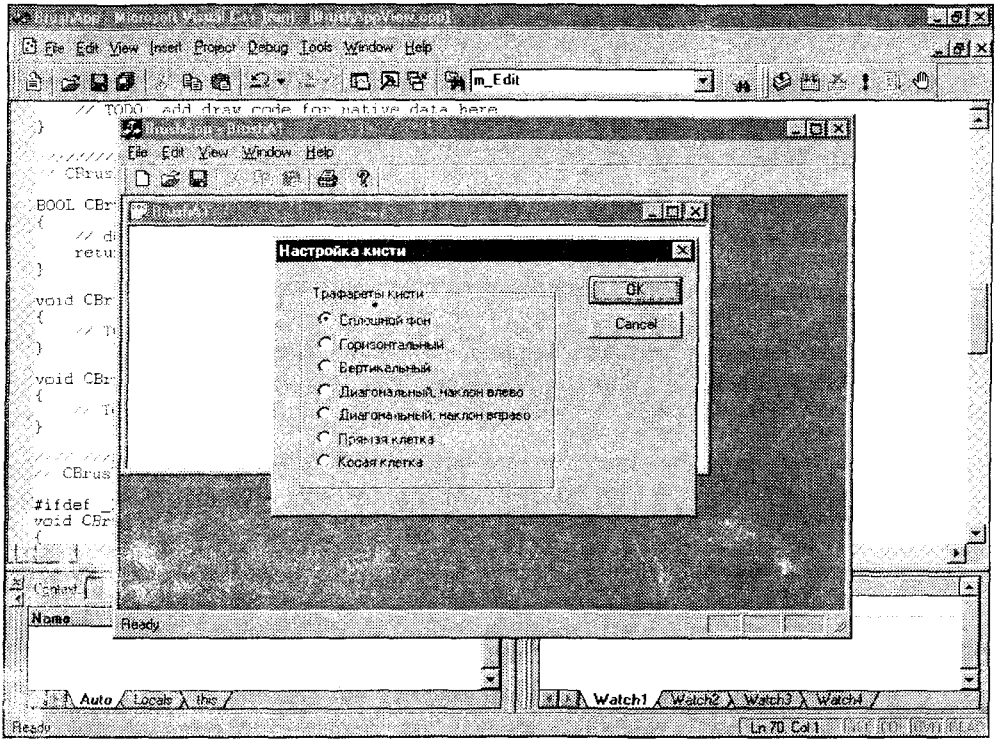


Рис. 6.9. Диалоговое окно **Настройка кисти**

Работа с битовыми образами

Термин *битовый образ* иногда неправильно трактуется некоторыми программистами. Многие из них понимают под ним любое изображение, которое может выводиться на экран. Хотя это определение имеет право на существование, необходимо помнить, что существуют две разновидности битовых образов, из которых только один подходит под данное определение. Первая разновидность называется *аппаратно-независимыми битовыми образами* (Device-Independent Bitmaps, DIB). Эти битовые образы хранятся в файлах с расширением .bmp и именно эту разновидность имеют в виду большинство пользователей, когда говорят о битовых образах. Второй разновидностью битовых образов являются *аппаратно-зависимые битовые об-*

разы (Device-Dependent Bitmap, DDB). Эти битовые образы существуют только в памяти компьютера и представляют собой изображение, создаваемое приложением Windows для вывода на экран.

Аппаратно-зависимые битовые образы

Аппаратно-зависимые битовые образы не содержат таблиц цветов и поэтому привязаны к конкретному устройству. Эти битовые образы создаются в памяти компьютера непосредственно перед выводом изображения на экран и удаляются из памяти вместе с создавшим их приложением.

Этот тип битовых образов обычно используется для выполнения чисто утилитарных задач. Например, для хранения образца текстуры, используемой для заполнения некоторого пространства. Другой достаточно распространенной областью применения аппаратно-зависимых битовых образов является копирование части экрана с целью переноса его в новое место. При этом выделенная часть экрана копируется в память в формате битового образа, а затем этот битовый образ выводится в указанном месте экрана.

Текст приложения, демонстрирующего принципы работы с аппаратно-зависимыми битовыми образами, расположен в каталоге \DDBApp дискеты, прилагаемой к данной книге. Чтобы самостоятельно создать данное приложение:

1. Создайте приложение с именем DDBApp по методике, описанной в главе 1. Оставьте все установки по умолчанию без изменения.
2. Перейдите на вкладку **ClassView** окна **Workspace**. Раскройте папку \DDBApp classes и щелкните правой кнопкой мыши на значке класса `CDDBAppView`.
3. В раскрывшемся контекстном меню выберите команду **Add Member Variable...** (Добавить переменную-член класса). Появится диалоговое окно **Add Member Variable**, изображенное на рис. 6.10.

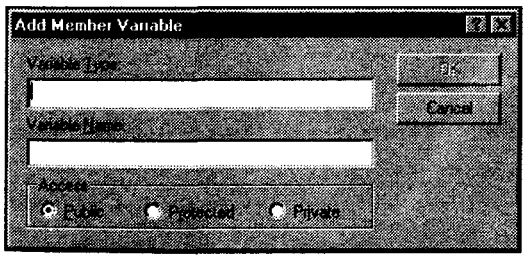


Рис. 6.10. Диалоговое окно **Add Member Variable**

4. В текстовое поле **Variable Type:** (тип переменной) введите тип "BYTE", а в текстовое поле **Variable Name:** (идентификатор переменной) введите имя "m_Buffer[32768]".

5. Оставьте без изменения положение **Public** переключателя прав доступа **Access** и нажмите кнопку **OK**.
6. Повторите пп. 3—5 для переменной `m_Rect`, являющейся объектом класса `CRect`.
7. Повторите пп. 3—5 для переменной `was_copied`, имеющей тип `bool`.
8. Перейдите на вкладку **FileView** окна **Workspace**. Раскройте папку \DDBApp files, а в ней — папку \Source Files. Дважды щелкните левой кнопкой мыши на значке **DDBAppView.cpp**. Раскроется окно редактирования соответствующего файла.
9. Из окна редактирования файла вызовите мастер ClassWizard. Появится диалоговое окно **MFC ClassWizard**.
10. Раскройте вкладку **Message Maps**.
11. В окне списка **Object IDs:** выделите имя класса `CDDBAppView`, в окне списка **Messages:** выделите идентификатор сообщения `WM_LBUTTONDBLCLK` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится новое имя функции обработки сообщения `OnLButtonDblClk`.
12. Нажмите кнопку **Edit Code**. Диалоговое окно **MFC ClassWizard** закроется и текстовый курсор будет помещен в текст функции `OnLButtonDblClk`.
13. Измените функцию `OnLButtonDblClk` в соответствии с приведенным ниже текстом:

```
void CDDBAppView::OnLButtonDblClk(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  CClientDC  pDC( this);
  CDC       memDC;
  CBitmap    m_Bitmap;

  int       wd  = m_Rect.Width() >> 2;

  m_Bitmap.CreateCompatibleBitmap( &pDC, wd, m_Rect.Height());
  memDC.CreateCompatibleDC( &pDC);

  CBitmap*  oldBitmap = memDC.SelectObject( &m_Bitmap);

  if( m_Rect.PtInRect( point)){

    int  nRect = (point.x - m_Rect.left)/wd;

    memDC.BitBlt(0, 0, wd, 40, &pDC, m_Rect.left + nRect*wd,
                                                    10, SRCCOPY);
    m_Bitmap.GetBitmapBits(32768, m_Buffer);

    was_copied = true;
  }
```

```
    else{

      CRect  rect;

      GetClientRect( rect);

      if( was_copied && (point.x < (rect.right - wd)) &&
                                   (point.y < (rect.bottom - 40))){
        m_Bitmap.SetBitmapBits(32768, m_Buffer);
        pDC.BitBlt(point.x, point.y, wd, 40, &memDC, 0, 0, SRCCOPY);
      }
    }

    memDC.SelectObject( &oldBitmap);

    CView::OnLButtonDblClk(nFlags, point);
}
```

14. Измените функцию `OnDraw` в соответствии с приведенным ниже текстом:

```
void CDDBAppView::OnDraw(CDC* pDC)
{
  CDDBAppDoc* pDoc = GetDocument();
  ASSERT_VALID(pDoc);
  // TODO: add draw code for native data here

  CRect  rect;

  GetClientRect( rect);

  if( (rect.Width() > 24) && (rect.Height() > 80)){

    m_Rect  = CRect( 10, 10, rect.right - 10, 50);

    int  wd  = m_Rect.Width() >> 2;

    rect = m_Rect;
    rect.right = rect.left + wd;

    pDC->FillRect( rect, &CBrush( RGB( 0, 0, 0)));

    rect.left  = rect.right;
    rect.right  += wd;

    pDC->FillRect( rect, &CBrush( RGB( 255, 0, 0)));

    rect.left  = rect.right;
    rect.right  += wd;
    pDC->FillRect( rect, &CBrush( RGB( 0, 255, 0)));

    rect.left  = rect.right;
    rect.right  += wd;
```

```
        pDC->FillRect( rect, &CBrush( RGB( 0, 0, 255)));
    }
    was_copied  = false;
}
```

15. Нажмите клавишу <F5> и запустите приложение на исполнение.
16. Нажмите кнопку **Развернуть** в заголовке окна приложения. Окно приложения развернется на весь экран.
17. Нажмите кнопку **Развернуть** в заголовке окна документа. Окно примет вид, изображенный на рис. 6.11.

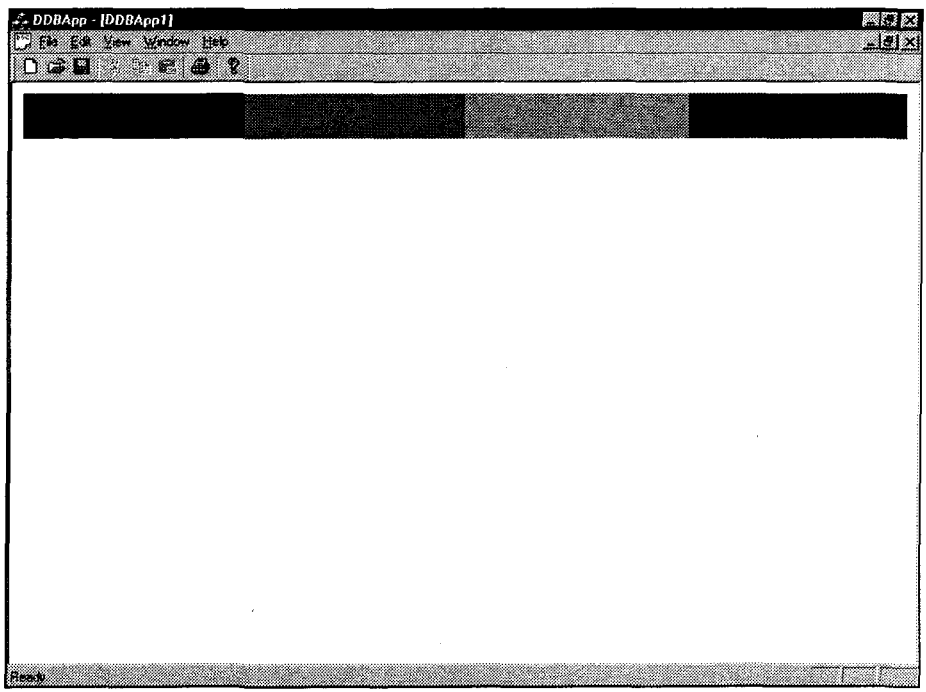


Рис. 6.11. Исходное окно приложения DDBApp

18. Дважды щелкните левой кнопкой мыши в одном из прямоугольников с основными цветами, расположенных в верхней части окна документа.
19. Переместите курсор мыши на свободное место под этими прямоугольниками и дважды щелкните левой кнопкой мыши. На указанном месте появится прямоугольник выбранного цвета.
20. Повторите эту операцию с другими цветами. Примерный результат приведен на рис. 6.12.

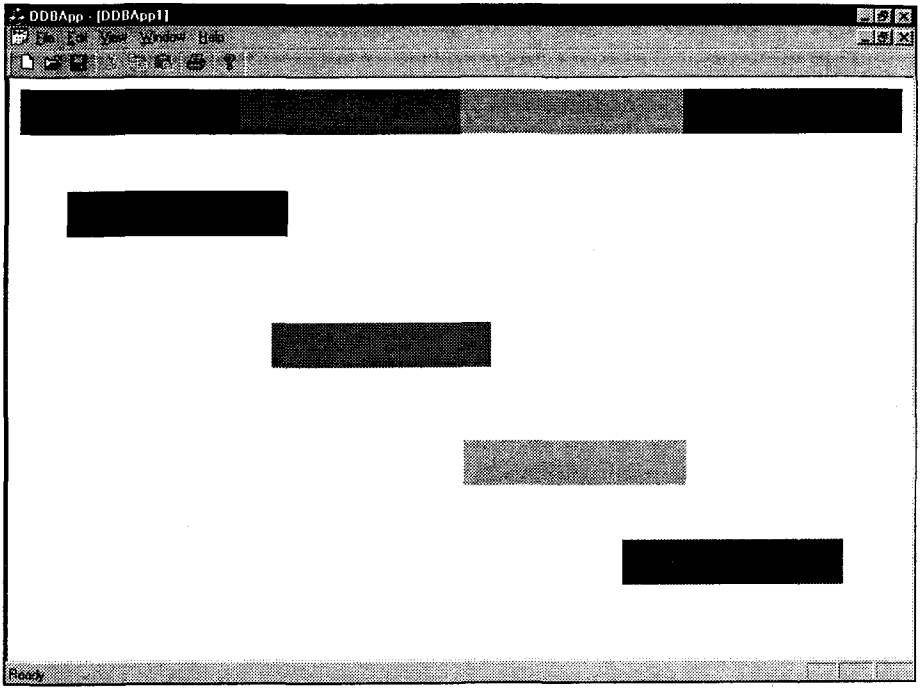


Рис. 6.12. Окно приложения после работы с ним

Для демонстрации принципов работы с аппаратно-независимыми битовыми образами в окне документа выводятся четыре прямоугольника, окрашенные в четыре основных цвета (чистый тон каждого из основных цветов и отсутствие цветов или черный цвет). Поскольку эти прямоугольники не должны выходить за пределы окна при его перерисовке, они генерируются непосредственно в функции `OnDraw`. После получения указателя на объект класса документа, связанный с данным объектом класса представления, и проверки его допустимости, вызывается функция `CWnd::GetClientRect`, копирующая пользовательские координаты рабочей области окна, связанного с объектом класса `CWnd`, в структуру, на которую указывает ее аргумент.

Если размеры окна позволяют разместить в нем четыре прямоугольника, и при этом в окне остается еще достаточно места для их копирования, в окне выводятся эти четыре прямоугольника. Для этого координаты области размещения прямоугольников вычисляются и сохраняются в переменной `m_Rect`. Кроме того, определяется значение переменной `wd`, в которой хранится ширина прямоугольников. После этого прямоугольники выводятся на экран функцией `CDC::FillRect`, заполняющей заданный прямоугольник указанной кистью.

Перед выходом из функции `OnDraw` независимо от того, были в ней выведены цветные прямоугольники или нет, переменной `was_copied` присваивается

значение false, говорящее о том, что информация, хранящаяся в буфере, не может больше использоваться для формирования битового образа.

Непосредственная работа с битовым образом производится в функции OnLButtonDblClk. Для этого в ней создаются объекты классов CDC и CBitmap. Переменная memDC, являющаяся объектом класса CDC, служит для хранения контекста устройства памяти, представляющего собой область памяти, эмулирующую виртуальный дисплей. Контекст устройства памяти обычно используется для сборки изображений перед их выводом на экран.

Созданный объект класса CBitmap инициализируется функцией CBitmap::CreateCompatibleBitmap, первым аргументом которой является указатель на объект класса контекста устройства, вторым аргументом — ширина битового образа, а третьим — его высота. Использование данной функции устанавливает для инициализируемого битового образа такой формат, который позволит выбирать его в указанный контекст устройства в качестве текущего битового образа.

Объект класса CDC инициализируется функцией CDC::CreateCompatibleDC, инициализирующей контекст устройства памяти и делающей его совместимым с контекстом устройства, объект класса которого задается в качестве ее аргумента. Инициализированные объекты классов используются для выбора в контекст устройства памяти пользовательского объекта класса CBitmap. Указатель на старый объект класса CBitmap сохраняется в специальной переменной, для чего вызывается функция CDC::SelectObject.

Затем функция CRect::PtInRect проверяет, находится ли указатель мыши в прямоугольнике с эталонными цветами. Если да, вычисляется номер цветного прямоугольника, в котором находится указатель мыши, и вызывается функция CDC::BitBlt для объекта класса контекста устройства памяти. Данная функция имеет восемь аргументов. Первые два аргумента определяют координаты левого верхнего угла области вывода. Следующие два аргумента определяют размеры этой области. Пятый аргумент является указателем на объект класса контекста устройства, из которого копируется информация. Следующие два аргумента определяют координаты левого верхнего угла копируемого изображения, а последний определяет режим копирования информации в область вывода. После копирования области экрана в текущий битовый образ контекста устройства памяти функцией CDC::BitBlt, данный битовый образ сохраняется в памяти. Для этого вызывается функция CBitmap::GetBitmapBits, копирующая битовый образ из объекта класса CBitmap в буфер, на который указывает ее второй аргумент. Первый аргумент данной функции определяет число копируемых байт.

Этот битовый образ будет впоследствии копироваться в область вывода, имеющую те же размеры и формат, что и исходная область. При этом надо, чтобы вся необходимая информация поместилась в области вывода. Исполь-

зуемое в данном случае значение удовлетворяет требованиям, предъявляемым при работе в большинстве реальных режимов отображения информации на дисплее.

Последней операцией в данной ветви условного оператора является присваивание переменной `was_copied` значения `true`, свидетельствующего о том, что буфер заполнен и его можно использовать для вывода битового образа на экран.

Если двойной щелчок левой кнопки мыши был выполнен за пределами прямоугольника с эталонными цветами, функция `CWnd::GetClientRect` определяет координаты рабочей области окна. После чего производится проверка, что буфер содержит информацию о битовом образе, и что изображение не выйдет за пределы рабочей области окна. Если эти условия выполнены, информация выводится в окно.

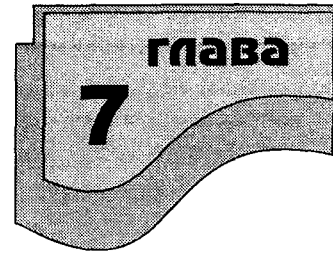
Для вывода на экран информации, хранящейся в памяти, вызывается функция `CBitmap::SetBitmapBits`, заполняющая биты в битовом образе данного объекта. Ее аргументы аналогичны аргументам функции `CBitmap::GetBitmapBits`. Поскольку данный объект класса `CBitmap` выбран в контекст устройства памяти, то информация загружается непосредственно в объект контекста устройства. Поэтому последующий вызов функции `CDC::BitBlt` для данного объекта приводит к выводу записанной информации на экран.

По завершении всех операций, независимо от того, где был произведен двойной щелчок левой кнопкой мыши, в контекст устройства памяти выбирается его старый битовый образ, освобождая при этом битовый образ, выбранный в контекст устройства памяти перед этим.

В данном случае для заполнения области памяти, содержимое которой выводится на экран, использовалась операция копирования содержимого экрана. В этом случае пользователю нет необходимости заботиться о соответствии форматов выводимой информации. Однако в некоторых случаях возникает необходимость формирования битового образа из пользовательских данных. Например, при выводе трехмерного графика, в котором третья координата задается цветом или яркостью точки.

В большинстве случаев подготовка информации для вывода на экран с использованием битовых образов не представляет особых сложностей. Однако пользователю при этом следует помнить об одной неприятной особенности формата битовых образов: битовые образы размещаются в памяти как одномерный массив, в который последовательно записываются значения цветов элементов изображения. Цвета элементов изображения записываются по горизонтальным строкам изображения. А размер строки в данном массиве должен быть кратен 16 битам или 2 байтам при том, что наиболее часто употребляемые форматы цветов используют 1 байт (256 цветов) или 3 байта (TrueColor). Если строка пользовательского битового образа состоит из чет-

ного числа элементов изображения, то при ее выводе не наблюдается никаких искажений. Если же она состоит из нечетного числа элементов изображения и пользователь не предпринял никаких мер по выравниванию размеров строк, то изображение при выводе будет искажено. Наиболее простым выходом из данной ситуации является увеличение передаваемого приложению горизонтального размера области вывода до ближайшего четного числа. Если область отсечки установлена правильно, то последний столбец элементов изображения, сформированный приложением, не будет выведен. В большинстве приложений с этим можно смириться.



Работа с файлами документов

В главе 6 было рассмотрено сохранение изображения в окне при изменении его размера. Другим очень важным вопросом является вопрос о том, как восстановить изображение в окне, существовавшее до его закрытия, при новом открытии этого окна. Сходная проблема возникает и при открытии окна в новом приложении. Данный вопрос решается в рамках концепции Документ/Представление, описанной в главе 2. Согласно ей вся информация о выводимом в окне изображении хранится в объекте класса документа, связанном с объектом класса представления, осуществляющим вывод этого изображения на экран или на принтер.

См. также главу 2

Работа с архивом

Как отмечалось в главе 2, для чтения и записи информации, содержащейся в документе при создании и уничтожении объекта класса этого документа, используется функция `CObject::Serialize`, имеющая своим аргументом объект класса `CArchive`. Данная функция осуществляет инициализацию объекта класса документа при его создании и сохранение имеющейся в нем информации при уничтожении данного объекта. Для хранения данных объекта используется файл архива на диске. Данный файл не содержит специальных записей, характеризующих его как файл архива, поэтому в данном качестве может выступать любой файл, даже созданный средствами операционной системы MS DOS.

Изменим созданное нами приложение TextApp таким образом, чтобы выводимая в окне информация сохранялась в файле архива и появлялась на экране после открытия этого файла.

Чтобы внести в приложение TextApp изменения, позволяющие сохранять информацию, выводимую в окно, в файле архива:

1. Если вы недавно работали с приложением TextApp, поместите указатель мыши на команду меню **File**, **Recent Workspaces**. При этом раскроется контекстное меню, показанное на рис. 7.1.
2. Если в раскрывшемся контекстном меню содержится имя проекта TextApp, щелкните на нем левой кнопкой мыши или нажмите на клавишу, соответствующую его номеру в меню. Проект TextApp станет активным и в окнах Visual C++ восстановится информация, содержавшаяся в них до последнего закрытия данного проекта.
3. В случае успешного завершения этой операции перейдите к п. 6.
4. В противном случае выберите команду **File**, **Open Workspace...**. Откроется диалоговое окно **Open Workspace**.
5. В данном диалоговом окне раскройте папку \TextApp, выделите значок **TextApp.dsw** и нажмите кнопку **Открыть**. Проект TextApp станет активным и в окнах Visual C++ восстановится информация, содержавшаяся в них до последнего закрытия данного проекта.

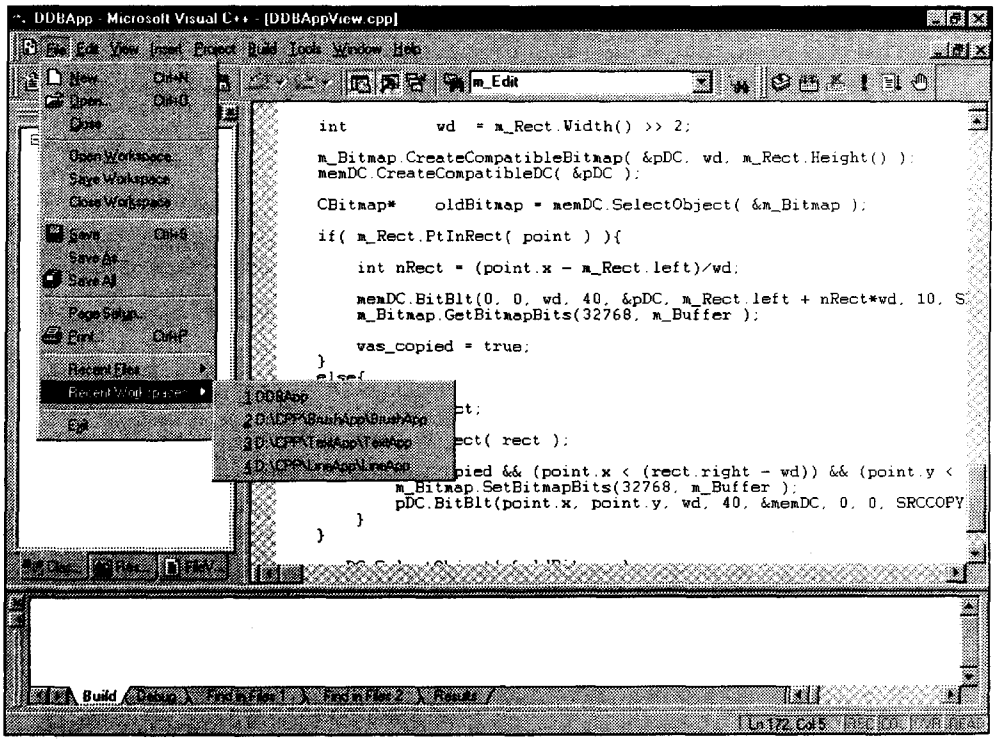


Рис. 7.1. Контекстное меню выбора проекта

6. Откройте окно редактирования файла TextAppView.cpp и вставьте в текст функции `OnLButtonDown` после строки `lpDoc-> aY.Add( point.y);` строку

```
lpDoc-> SetModifiedFlag();
```

7. Откройте окно редактирования файла TextAppDoc.cpp и отредактируйте текст функции `Serialize` в соответствии с приведенным ниже текстом.

```
void CTextAppDoc::Serialize(CArchive& ar)
{
  int    i, n;

  if (ar.IsStoring())
  {
    // TODO: add storing code here

    n = aX.GetSize();

    ar << n;

    for(i=0; i < n; i++){
      ar << aX[i];
      ar << aY[i];
    }
  }
  else
  {
    // TODO: add loading code here

    ar >> n;

    aX.SetSize( n);
    aY.SetSize( n);

    for(i=0; i < n; i++){
      ar >> aX[i];
      ar >> aY[i];
    }
  }
}
```

8. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится окно приложения TextApp.

9. Несколько раз щелкните левой кнопкой мыши в рабочей области окна. Окно примет вид, изображенный на рис. 7.2.

10. Нажмите кнопку (Закрыть) в окне документа. Появится окно сообщения **TextApp**, изображенное на рис. 7.3.

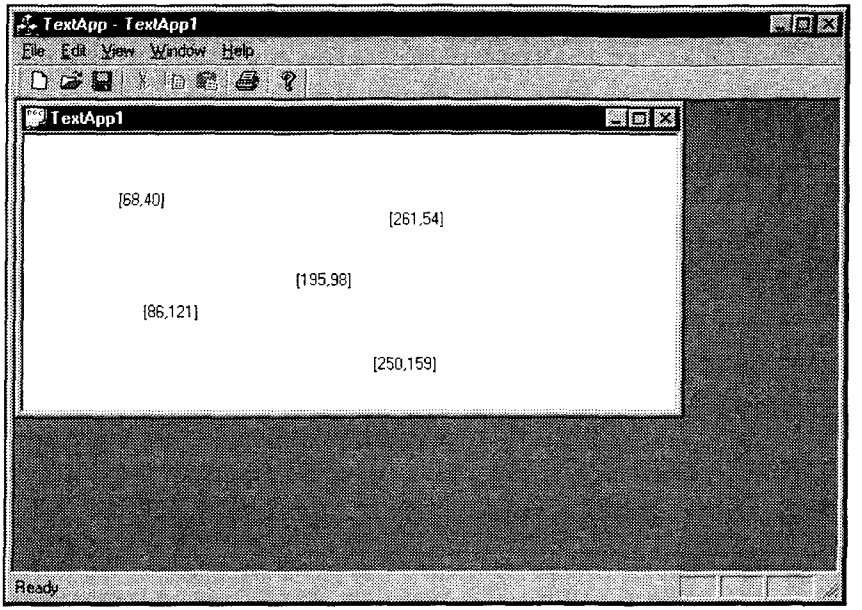


Рис. 7.2. Окно приложения TextApp

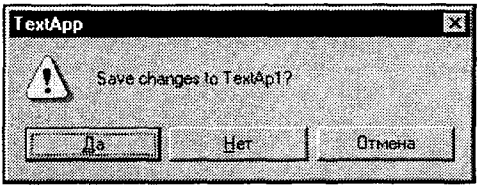


Рис. 7.3. Окно сообщения **TextApp**

11. Нажмите кнопку **Да**, подтвердив необходимость сохранения изменений, внесенных в документ TextApp1. Появится стандартное диалоговое окно **Save As** (Окно сохранения файла), изображенное на рис. 7.4.
12. Нажмите кнопку **Сохранить**. Окно документа закроется.

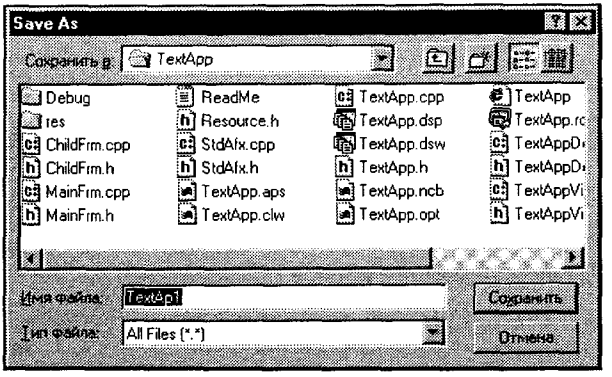


Рис. 7.4. Диалоговое окно **Save As**

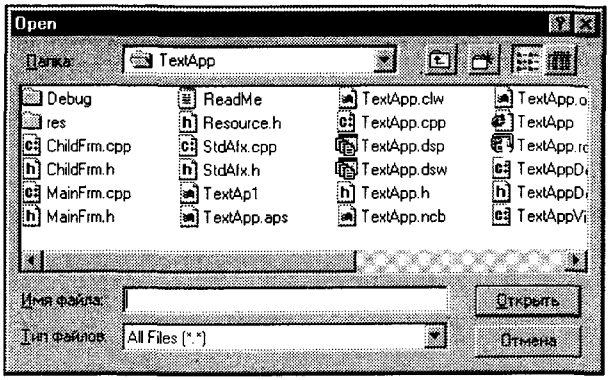


Рис. 7.5. Диалоговое окно **Open**

13. Выберите команду **File**, **Open** или нажмите кнопку (Open, открыть) в панели инструментов (на этой кнопке изображена раскрытая желтая папка). Появится стандартное диалоговое окно **Open** (окно открытия файла), изображенное на рис. 7.5.
14. Выделите в списке значок **TextApp1** и нажмите кнопку **Открыть**. Откроется окно документа, в котором будет сохранена вся информация, отображавшаяся в нем до его закрытия.

Использование в данном приложении функции `CDocument::SetModifiedFlag`, вызываемой после записи в массив очередных координат щелчка левой кнопкой мыши, позволяет приложению определить, что данный документ не является пустым и, следовательно, нуждается в сохранении.

Функция `CObject::Serialize` осуществляет непосредственную работу по инициализации и сохранению документа. Вызов функции `CArchive::IsStoring` позволяет определить, какая именно операция выполняется в данный момент. При сохранении информации, хранящейся в документе, в архив сначала заносится размер массивов координат точек, а затем сохраняются сами координаты в виде пар координат точек. Для сохранения информации в документе используется переопределенный в классе `CArchive` оператор <<. При инициализации объекта класса документа сохраненной в архиве информацией сначала считывается размер массивов координат, и на основании этой информации устанавливаются размеры массивов, в которые будут записываться координаты. А затем массивы заполняются информацией, хранящейся в архиве. Для чтения информации в документ используется переопределенный в классе `CArchive` оператор >>.

Данное приложение имеет два основных недостатка:

- Документ сохраняется в файле без расширения, что приводит к необходимости отображать в окне **Open** и в окне **Save As** все имеющиеся в данном каталоге файлы
- Файл записывается в рабочий каталог программы, что в большинстве случаев не совсем удобно

Чтобы устранить перечисленные выше недостатки:

1. Раскройте панель **ResourceView** окна **Workspace**.
2. Раскройте папку \TextApp Resources, в ней раскройте папку \String Table и дважды щелкните на значке **String Table**.
3. В окне редактирования появится таблица строковых ресурсов.
4. Найдите в ней идентификатор ресурса `IDR_TEXTAPPTYPE` и дважды щелкните на нем левой кнопкой мыши. Появится диалоговое окно **String Properties**, изображенное на рис. 7.6.

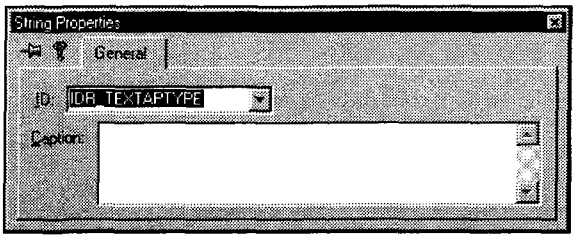


Рис. 7.6. Диалоговое окно **String Properties**

5. В текстовое поле **Caption:** введите следующий текст:

   "\nTextApp\nTextApp\nTextApp
   file (*.tpf)\n.tpf\nTextApp.Document\nTextAp Document".
6. Закройте диалоговое окно **String Properties**. Окно редактирования строковых ресурсов примет вид, изображенный на рис. 7.7.
7. Откройте окно редактирования файла TextApp.cpp и вставьте в текст функции `InitInstance` перед оператором `return TRUE;` следующий текст:

```
CString    Work_Dir;

Work_Dir = GetProfileString("CONTROL","DIR","c:\\");
SetCurrentDirectory( Work_Dir);
```

8. Откройте окно редактирования файла TextAppDoc.cpp и отредактируйте текст функции `Serialize` в соответствии с приведенным ниже текстом.

```
void CTextAppDoc::Serialize(CArchive& ar)
{
  int     i, n;
  CWinApp*  pApp = AfxGetApp();

  if (ar.IsStoring())
  {
    // TODO: add storing code here

    n = aX.GetSize();

    ar << n;
```

```
      for(i=0; i < n; i++){
        ar << aX[i];
        ar << aY[i];
      }

      char  Cur_Dir[_MAX_DIR];

      GetCurrentDirectory(_MAX_DIR, Cur_Dir);

      ASSERT(pApp-> WriteProfileString("CONTROL","DIR",Cur_Dir));
    }
    else
    {
      // TODO: add loading code here

      ar >> n;

      aX.SetSize( n);
      aY.SetSize( n);

      for(i=0; i < n; i++){
        ar >> aX[i];
        ar >>.aY[i];
      }
    }
  }
```

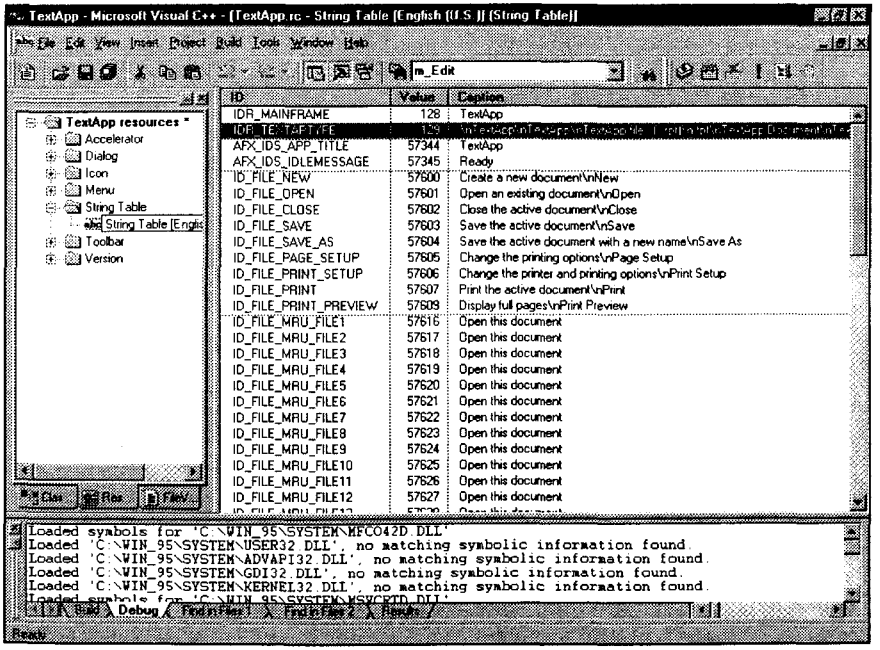


Рис. 7.7. Окно редактирования строковых ресурсов

9. Нажмите клавишу <F5> и запустите приложение на исполнение.
10. Несколько раз щелкните левой кнопкой мыши в рабочей области окна.
11. Нажмите кнопку **Закрыть** в окне документа.
12. В появившемся окне сообщения **TextApp** нажмите кнопку **Да**, подтвердив необходимость сохранения изменений, внесенных в документ TextApp1. Появившееся диалоговое окно **Save As** будет иметь вид, изображенный на рис. 7.8.
13. Раскройте папку, в которой вы собираетесь сохранить документ и нажмите кнопку **Сохранить**. Диалоговое окно **Save As** и окно документа закроются.

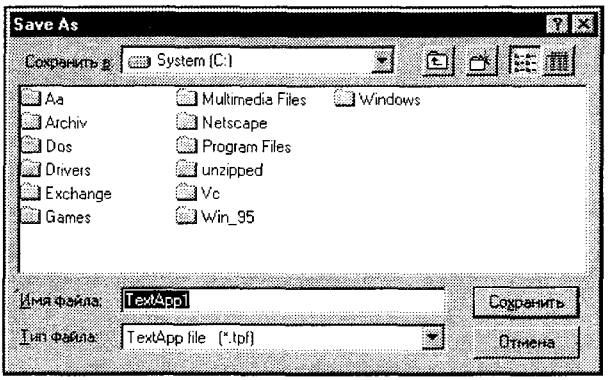


Рис. 7.8. Диалоговое окно **Save As**

14. Выберите команду **File**, **Open** или нажмите кнопку **Open** в панели инструментов. Появится стандартное диалоговое окно **Open** (Открыть), изображенное на рис. 7.9.

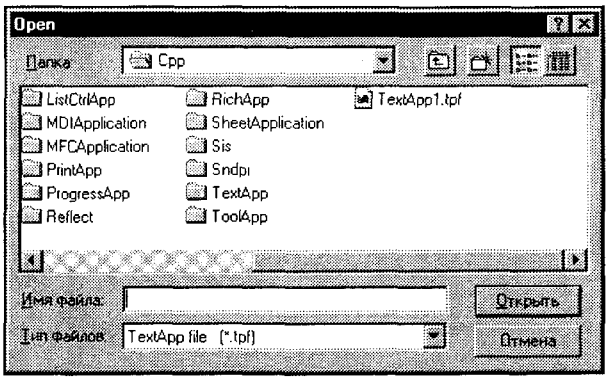


Рис. 7.9. Диалоговое окно **Open**

15. Выделите в списке значок **TextApp1.tpf** и нажмите кнопку **Открыть**. Откроется окно документа, в котором будет сохранена вся информация, отображавшаяся в нем до его закрытия.

Основным отличием диалогового окна **Save As**, приведенного на рис. 7.8, от аналогичного окна, приведенного на рис. 7.4, является то, что в текстовом поле его раскрывающегося списка **Тип файлов:** размещается строка **TextApp file (*.tpf)**, в раскрывающемся списке **Сохранить в:** размещается строка **System (C:)**, а в окне списка файлов выведены только папки, поскольку в корневом каталоге диска C: отсутствуют файлы с расширением .tpf.

Основным отличием диалогового окна **Open**, приведенного на рис. 7.9, от аналогичного окна, приведенного на рис. 7.5, является то, что в текстовом поле его раскрывающегося списка **Тип файлов:** размещается строка **TextApp file (*.tpf)**, в раскрывающемся списке **Папка:** содержится имя каталога, в котором был сохранен файл TextApp1.tpf, а в окне списка файлов содержится единственное имя файла, имеющее расширение .tpf.

Текст, введенный в текстовое поле **Caption:** диалогового окна **String Properties** в пункте 4, является строковым ресурсом, связанным с шаблоном документа, использующего данный идентификатор ресурса. Этот строковый ресурс может содержать до семи текстовых полей, разделенных символом перевода строки (`\n`) (этот символ используется в данном случае для обеспечения возможности указания числа пустых полей, — если после некоторого поля все остальные поля являются пустыми, то нет необходимости указывать для них символы перевода строки). Подробно эти поля описаны в функции `CDocTemplate::GetDocString`. Здесь отметим только, что первое поле в многооконных документах не используется, а заданные нами четвертое и пятое поля содержат строки с описанием типа документа и его расширения, используемых для поиска документов данного типа (эта строка отображается в раскрывающемся списке **Тип файлов:** диалоговых окон **Open** и **Save As**).

Чтобы приложение запоминало рабочий каталог, установленный при предыдущем запуске приложения, он сохраняется в системном реестре, представляющем собой файл, в который записываются сведения о параметрах всех приложений, зарегистрированных в операционной системе данного компьютера. Регистрация приложения является обязательной процедурой для его нормального функционирования в системе. Каждому зарегистрированному приложению в системном реестре соответствует запись, в которой хранится вся информация, необходимая приложению для нормального возобновления работы. Характер данной информации определяется исключительно самим приложением. Более подробно системный реестр будет рассмотрен в заключительном разделе данной главы.

Для записи текстовой строки в системный реестр используется функция `CWinApp::WriteProfileString`, первый аргумент которой представляет собой текстовую строку, определяющую секцию, в которую необходимо записать текстовую строку, указанную в третьем аргументе данной функции, а второй ее аргумент является текстовой строкой, содержащей идентификатор запи-

сываемой строки в секции. Данная функция вызывается из функции `CTextAppDoc::Serialize` после того, как пользователь сохранил в файле информацию о содержимом окна. Поскольку сохранение информации производится в текущем каталоге процесса, после сохранения информации в файле вызывается функция `GetCurrentDirectory`, сохраняющая в переменной `Cur_Dir` текстовую строку, содержащую полный путь в текущий каталог приложения. После этого вызывается функция `CWinApp::WriteProfileString`, сохраняющая данную строку в системном реестре. Указатель на объект класса `CWinApp`, вызывающий данную функцию, получается посредством вызова функции `AfxGetApp`.

Для чтения текстовой строки из системного реестра используется функция `CWinApp::GetProfileString`, первый аргумент которой представляет собой текстовую строку, определяющую секцию, в которой содержится текстовая строка, возвращаемая данной функцией, второй ее аргумент является текстовой строкой, содержащей идентификатор возвращаемой строки в секции, а третий аргумент — текстовой строкой, возвращаемой функцией в случае отсутствия указанной записи в системном реестре. Данная функция вызывается из функции `CTextAppApp::InitInstance`, поскольку именно в этой функции задаются основные параметры создаваемого объект класса документа. Одним из таких параметров можно считать его рабочий каталог. Поскольку функция `InitInstance` является членом класса, производного от класса `CWinApp`, в данном случае нет необходимости получать указатель на объект данного класса. Возвращаемое данной функцией значение используется в качестве аргумента функции `SetCurrentDirectory`, устанавливающей новый рабочий каталог приложения.

Непосредственное чтение и запись файлов

Использование архива позволяет достаточно просто решить вопрос сохранения на диске и чтения с диска отдельных документов. Однако основной особенностью документов Windows является то, что они рассматриваются как единое целое. Это значит, что при создании документа в него записывается вся необходимая для его работы информация, а после завершения работы с ним вся эта информация сохраняется на диске. Последовательный принцип доступа к файлу требует как чтения, так и записи всей сохраняемой информации в документе. Кроме того, в классе документа Windows не предусмотрены методы доступа к диску в процессе работы с документом.

Предположим, данное приложение работает с мультимедийными объектами, такими как файлы фонограмм или файлы изображений. Требование загрузить всю информацию, связанную с данными объектами, в оперативную память, может оказаться невыполнимым. Поэтому при работе с данным типом объектов приходится постоянно обращаться к файлу на диске.

Предположим, что в приложении используется шаблон документа, предназначенный для вывода на экран фонограммы звукового файла. Размер этого файла может достигать нескольких мегабайт и, в принципе, не ограничен. В данном случае можно использовать функцию `CDocument::Serialize` для создания промежуточного массива, хранящего достаточно информации для того, чтобы корректно отображать фонограмму при любом разрешении и любом размере окна. Казалось бы, вопрос решен, и нет необходимости в дальнейших обращениях к исходному файлу. Однако пользователь может потребовать увеличить масштаб отображения или выполнить некоторые операции редактирования. В этом случае информации, хранящейся в массиве, созданном функцией `Serialize`, может быть недостаточно для корректного отображения фонограммы, а операции редактирования требуют непосредственного доступа ко всему файлу. Данные проблемы решаются при помощи объектов класса `CFile`.

Использование объектов класса *CFile* при работе с классом *CArchive*

Класс `CFile` является базовым классом библиотеки MFC для работы с файлами. Он обеспечивает произвольный доступ к двоичному файлу. Данный класс используется совместно с классом `CArchive` для сохранения объектов классов из библиотеки MFC. Файл на диске автоматически открывается конструктором класса `CFile` и автоматически закрывается его деструктором. Подробное описание данного класса содержится в Приложении В. Здесь будут рассмотрены только некоторые аспекты работы с объектами данного класса.

В приведенном выше примере возникла необходимость работы с файлом после заполнения документа функцией `CDocument::Serialize`. Поскольку данная функция является неотъемлемой частью класса документа, и структура приложения оптимизирована под эту функцию, нет никакого смысла игнорировать эту функцию при работе с файлом в приложении. Целесообразнее использовать ее для инициализации класса документа информацией, хранящейся в файле, и сохранения дополнительной информации, необходимой для повторного открытия файла. Как видно из описания конструктора данного класса, для повторного открытия файла необходимо задать только его имя и атрибуты открытия.

Чтобы модифицировать приложение TextApp для непосредственного обращения из него к файлу в процессе работы с документом:

1. Выберите команду **File**, **Open Workspace...** и в открывшемся диалоговом окне **Open Workspace** раскройте папку \TextApp, выделите значок **TextApp.dsw** и нажмите кнопку **Открыть**. Или выберите команду **File**, **Recent Workspaces** и в открывшемся контекстном меню выберите строку, содержащую имя проекта TextApp. Проект станет активным и в окнах

Visual C++ восстановится информация, содержавшаяся в них до последнего закрытия данного проекта.

2. Откройте окно редактирования файла TextAppView.cpp и вызовите мастер ClassWizard.
3. В диалоговом окне **MFC ClassWizard** раскройте вкладку **Message Maps**.
4. В окне списка **Object IDs:** выделите имя класса `CTextAppView`, в окне списка **Messages:** выделите идентификатор сообщения `WM_RBUTTONDOWN` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится имя новой функции обработки сообщения `OnRButtonDown`.
5. Нажмите кнопку **Edit Code**. Диалоговое окно **MFC ClassWizard** закроется, откроется окно редактирования файла TextAppView.cpp и текстовый курсор будет помещен в текст функции `OnRButtonDown`.
6. Измените текст в заготовке функции `OnRButtonDown` в соответствии с приведенным ниже текстом.

```
void CTextAppView::OnRButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  CTextAppDoc*  lpDoc = GetDocument();
  CFileException  fe;

  if( lpDoc-> GetPathName() == ""){
    CView::OnRButtonDown(nFlags, point);
    return;
  }

  CFile*  lpFile = lpDoc-> GetFile( lpDoc-> GetPathName(),
CFile::modeRead | CFile::shareDenyNone, &fe);

  if( fe.m_cause){
    ::MessageBox( 0, "Ошибка при создании файла",
                     "Ошибка", MB_OK | MB_SYSTEMMODAL);
    CView::OnRButtonDown(nFlags, point);
    return;
  }

  int    i, n;

  lpFile->Read( &n, sizeof(int));

  lpDoc-> aX.SetSize( n);
  lpDoc-> aY.SetSize( n);

  for(i=0; i < n; i++){
    lpFile->Read( &lpDoc-> aX[i], sizeof(UINT));
```

```
        lpFile->Read( &lpDoc-> aY[i], sizeof(UINT));
    }

    lpDoc->ReleaseFile( lpFile, true);
    lpDoc-> SetModifiedFlag( FALSE);

    Invalidate();

    CView::OnRButtonDown(nFlags, point);
}
```

7. Нажмите клавишу <F5> и запустите приложение на исполнение.
8. Нажмите кнопку **Open** на панели инструментов, в окне списка файлов появившегося диалогового окна **Open** выделите файл с расширением .tpf и нажмите кнопку **Открыть**. (Если файл с таким расширением отсутствует, создайте его по методике, описанной в предыдущем разделе.)
9. Появится окно документа с сохраненной в файле информацией.
10. Несколько раз щелкните левой кнопкой мыши и добавьте в окно новые записи.
11. После этого щелкните правой кнопкой мыши. Все новые записи исчезнут.

Для возврата окна к сохраненному состоянию в функции `OnRButtonDown` с помощью вызова функции `CDocument::GetFile` создается указатель на объект класса `CFile`. Поскольку первым аргументом данной функции является путь к открываемому файлу, а флаги открытия файла это второй ее аргумент, можно сделать вывод, что в объекте класса `CDocument` не хранится связанный с ним объект класса `CFile`. Единственным параметром, связывающим данный документ с файлом, является хранимое в объекте данного класса полное имя файла. Оно может быть получено с помощью вызова функций `CDocument::GetPathName`. Как видно из описания данной функции, приведенного в Приложении В, возвращаемая этой функцией строка может быть пустой. При передаче пустой строки в качестве первого аргумента функции `GetFile` возникает ошибка. Чтобы избежать ее, возвращаемое функцией `GetPathName` значение проверяется на допустимость и в том случае, если оно является пустой строкой, выполнение функции `OnRButtonDown` прерывается и после вызова метода базового класса производится выход из функции.

В случае положительного исхода данной проверки с помощью функции `CDocument::GetFile` создается указатель на объект класса `CFile`. Последний аргумент данной функции содержит код возможной ошибки, при возникновении которой во время выполнения данной функции выдается соответствующее сообщение. Для этого используется функция `MessageBox`. Нулевое значение первого аргумента и флаг `MB_SYSTEMMODAL` говорят о том, что работа со всеми приложениями, запущенными в данном сеансе работы с операци-

онной системой будет невозможна, пока пользователь не нажмет кнопку **OK** в появившемся окне сообщения. Поскольку само по себе окно сообщения не предпринимает каких-либо действий, после его закрытия вызывается метод базового класса и функция прекращает свою работу.

После этого работа функции напоминает работу соответствующей секции функции `CTextAppDoc::Serialize` за тем исключением, что класс `CFile`, в отличие от класса `CArchive`, не содержит перегруженного оператора `>>`, и поэтому операция ввода реализуется функцией `CFile::Read`. Первый аргумент данной функции представляет собой указатель на буфер, в который будет записана считанная из файла информация.

В данном случае производится чтение целочисленных переменных, поэтому в качестве первого аргумента используется ссылка на соответствующую статическую переменную. Второй аргумент данной функции определяет длину буфера в байтах. Поскольку в качестве первого аргумента используется ссылка на статическую переменную, вторым аргументом является оператор `sizeof`, возвращающий размер переменных заданного типа.

После использования в данной функции объекта класса `CFile` связанный с ним файл необходимо освободить для в других приложений. Это делается при помощи функции `CDocument::ReleaseFile`. Первым аргументом данной функции является указатель на объект класса `CFile`, который необходимо освободить. Второй ее аргумент определяет функцию, которая будет использоваться при освобождении файла. Если данный аргумент имеет значение `FALSE`, то файл освобождается при помощи функции `CFile::Close`. В противном случае используется функция `CFile::Abort`. Различие между этими функциями заключается в том, что функция `CFile::Abort` в процессе своей работы не вызывает исключений.

Поскольку данная функция возвращает окно документа в его исходное состояние, то в ней вызывается функция `CDocument::SetModifiedFlag` с аргументом `FALSE`, указывающим, что данный документ не следует сохранять, поскольку в него не были внесены никакие изменения.

Вызов функции `CWnd::Invalidate` вызывает перерисовку окна после того как в документ будут внесены изменения.

Последним оператором функции `OnRButtonDown` является вызов метода базового класса.

Данный пример включает в себя основные методы работы с файлом, связанным с данным шаблоном документа. Естественно, что при необходимости промежуточного сохранения информации в данном файле следует использовать функцию `CFile::Write`, не рассматриваемую в данном примере, но работа с ней ничем принципиально не отличается от работы с функцией `CFile::Read`.

Однако при непосредственной работе с файлом необходимо всегда помнить, что он был открыт с помощью архива, и система считает, что и сохраняться он должен с помощью архива. Это обстоятельство часто забывают при работе с большими документами, которые нецелесообразно полностью записывать в память. В этом случае все изменения в документе сразу же записываются в связанный с ним файл, и операция сохранения документа теряет смысл.

Поскольку при такой работе с файлом отсутствует необходимость в операции его сохранения, некоторые пользователи считают, что секцию сохранения документа в функции `Serialize` можно не заполнять программным кодом. Но приложение об этом "не знает", и вызов команды меню **File**, **Save** инициирует процедуру сохранения документа, который, как она полагает, хранится в памяти компьютера. Не найдя операторов сохранения файла она создает файл нулевой длины, то есть уничтожает всю хранившуюся в документе информацию.

Одним из способов избежать данной ситуации является создание в классе представления функции обработки сообщения `UPDATE_COMMAND_UI` для идентификатора ресурса `ID_FILE_SAVE`, единственным оператором которой должен быть вызов функции `CCmdUI::Enable` с аргументом `FALSE`, делающим соответствующую команду недоступной. Однако, если в данном приложении используется команда **File**, **Save As...**, в секции сохранения документа в функции `Serialize` необходимо поместить операторы, позволяющие переписать содержимое файла данного документа в другой файл.

Автономное использование класса *CFile*

Создание в данном приложении объекта класса `CFile` в качестве дублера класса `CArchive` не означает, что объект данного класса не может использоваться самостоятельно. Для его создания лишь нужно указать имя файла, с которым будет работать данный класс, а это можно сделать и не прибегая к услугам класса `CArchive`.

Объекты класса `CFile`, кроме дублирования архива, обычно используются для работы с файлами, обработка которых производится вне рамок концепции Документ/Представление и для создания пользовательского буфера обмена большого объема. Каждая из этих областей применения имеет свою специфику.

См. также главу 2

Чтобы включить в приложение TextApp объекты класса `CFile`, используемые для создания пользовательского буфера обмена и работы с файлом вне рамок концепции Документ/Представление:

1. Выберите команду **File**, **Open Workspace...** и в открывшемся диалоговом окне **Open Workspace** раскройте папку \TextApp, выделите значок

TextApp.dsw и нажмите кнопку **Открыть**. Или выберите команду **File**, **Recent Workspaces** и в открывшемся контекстном меню выберите строку, содержащую имя проекта TextApp. Проект станет активным, и в окнах Visual C++ восстановится информация, содержавшаяся в них до последнего закрытия данного проекта.

2. Раскройте вкладку **ClassView** окна **Workspace**. Раскройте папку \TextApp classes и щелкните левой кнопкой мыши на имени класса `CTextAppDoc`.
3. В раскрывшемся контекстном меню выберите команду **Add Member Variable...**. Появится диалоговое окно **Add Member Variable**.
4. Введите в текстовое поле **Variable Type:** тип указателя на класс `CFile*`, в текстовое поле **Variable Name:** введите идентификатор переменной `lpClipFile` и нажмите кнопку **OK**.
5. Повторите пп. 2—4 для переменной `lpOutFile`, имеющей тип `CFile*`.
6. Откройте окно редактирования файла TextAppView.cpp и вставьте в текст функции `OnLButtonDown` после строки `lpDoc-> aY.Add( point.y);` следующий текст:

```
lpDoc-> lpClipFile-> Write( &point.x, sizeof(LONG));
lpDoc-> lpClipFile-> Write( &point.y, sizeof(LONG));
```

7. Измените текст функции `OnRButtonDown`:

```
void CTextAppView::OnRButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  int  i, n;
  CTextAppDoc*  lpDoc = GetDocument();

  CString    File_Name;
  CWinApp*  pApp = AfxGetApp();

  VERIFY( pApp-> m_pDocManager != NULL);

  if( pApp-> m_pDocManager-> DoPromptFileName(File_Name,
              AFX_IDS_SAVEFILE, OFN_HIDEREADONLY | OFN_PATHMUSTEXIST,
             FALSE, NULL)){

    char  Cur_Dir[_MAX_DIR];

    GetCurrentDirectory(_MAX_DIR, Cur_Dir);
    VERIFY(WriteProfileString("CONTROL","DIR",Cur_Dir));

    try{
      lpDoc-> lpOutFile  = new CFile( File_Name,
                  CFile::modeCreate | CFile::modeReadWrite |
                                      CFile::shareDenyNone);

    }
```

```
    catch( ...) {
      ::MessageBox( 0, "Ошибка при открытии выходного файла",
                       "Ошибка", MB_OK|MB_SYSTEMMODAL);
      exit(1);
    }

    UINT  nTemp;

    n = lpDoc-> lpClipFile-> GetLength()/(2*sizeof(UINT));

    lpDoc-> lpOutFile-> Write( &n, sizeof(int));
    lpDoc-> lpClipFile-> SeekToBegin();

    while( lpDoc-> lpClipFile-> Read( &nTemp, sizeof(LONG)))
      lpDoc-> lpOutFile-> Write( &nTemp, sizeof(LONG));
  }

  CFileException  fe;

  if( lpDoc-> GetPathName() == ""){
    CView::OnRButtonDown(nFlags, point);
    return;
  }

  CFile*  lpFile = lpDoc-> GetFile( lpDoc-> GetPathName(),
CFile::modeRead | CFile::shareDenyNone, &fe);

  if( fe.m_cause){
    ::MessageBox( 0, "Ошибка при создании файла",
                     "Ошибка", MB_OK | MB_SYSTEMMODAL);
    CView::OnRButtonDown(nFlags, point);
    return;
  }

  lpFile->Read( &n, sizeof(int));

  lpDoc-> aX.SetSize( n);
  lpDoc-> aY.SetSize( n);

  for(i=0; i < n; i++){
    lpFile->Read( &lpDoc-> aX[i], sizeof(UINT));
    lpFile->Read( &lpDoc-> aY[i], sizeof(UINT));
  }

  lpDoc->ReleaseFile( lpFile, true);
  lpDoc-> SetModifiedFlag( FALSE);

  Invalidate();

  CView::OnRButtonDown(nFlags, point);
}
```

8. Откройте окно редактирования файла TextAppDoc.cpp и измените текст конструктора и деструктора класса CTextAppDoc в соответствии с приведенным ниже текстом:

```
/////////////////////////////////////////////////////////////////////////////
// CTextAppDoc construction/destruction

CTextAppDoc::CTextAppDoc()
{
  // TODO: add one-time construction code here

  struct  _finddata_t  c_file;
  long        hFile;
  CString  File_Name = "C:\\Файл пользовательского буфера
                        обмена. Не уничтожайте этот файл";

  if( (hFile = _findfirst( File_Name, &c_file)) != -1L ){
    ::MessageBox(0, "Данное приложение не может иметь двух файлов
                  буфера обмена","Ошибка", MB_OK|MB_SYSTEMMODAL);
    exit(1);
  }

  _findclose( hFile);

  try{
    lpClipFile  = new CFile(File_Name, CFile::modeCreate |
                  CFile::modeReadWrite | CFile::shareDenyNone);

    SetFileAttributes(File_Name, FILE_ATTRIBUTE_HIDDEN);

    lpOutFile  = NULL;
  }

  catch( ...) {
    ::MessageBox(0, "Ошибка при открытии файла буфера обмена",
                  "Ошибка", MB_OK|MB_SYSTEMMODAL);
    exit(1);
  }
}

CTextAppDoc::~CTextAppDoc()
{
  CString    F_Name = lpClipFile-> GetFilePath();

  lpClipFile-> Close();

  if( lpOutFile) lpOutFile-> Close();

  delete  lpClipFile;
  delete  lpOutFile;
```

```
    CFile::Remove( F_Name);
}
```

9. Откройте окно редактирования файла TextApp.cpp и уничтожьте в функции `CTextAppApp::InitInstance` следующие строки:

```
// Dispatch commands specified on the command line
  if (!ProcessShellCommand(cmdInfo))
    return FALSE;
```

10. Нажмите клавишу <F5> и запустите приложение на исполнение.
11. Выберите команду меню **File**, **Open...** или нажмите кнопку **Open** в панели инструментов.
12. В окне списка файлов появившегося диалогового окна **Open** выделите имя файла TextApp1.tpf, созданного на предыдущем этапе рассмотрения данного приложения, и нажмите кнопку **Открыть**.
13. В приложении откроется новое окно документа, в котором будет содержаться информация, хранившаяся в файле TextApp1.tpf.
14. Несколько раз щелкните в нем левой кнопкой мыши. В окне появятся координаты новых точек.
15. Щелкните правой кнопкой мыши в окне документа. Появится диалоговое окно **Save As**, как это показано на рис. 7.10. Постарайтесь сделать короткий щелчок. В противном случае, если диалоговое окно **Save As** появится на месте щелчка, поверх него будет раскрыто контекстное меню.

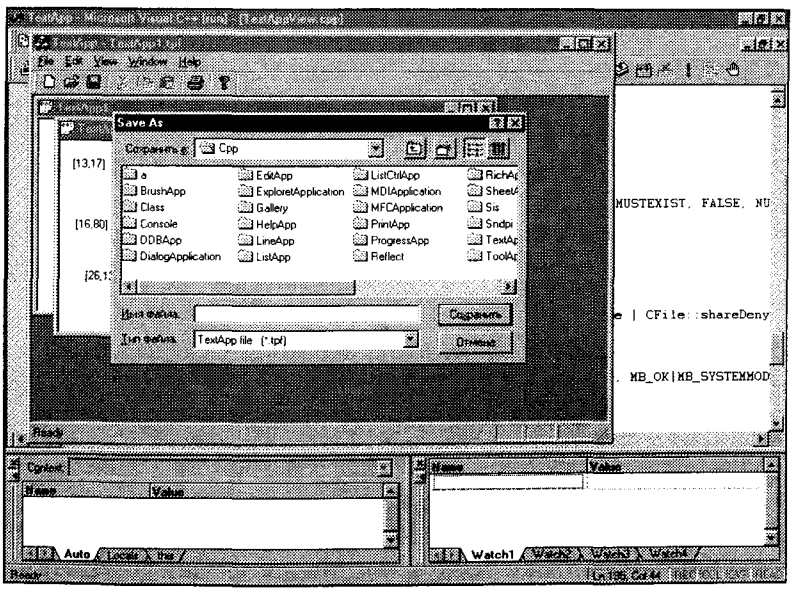


Рис. 7.10. Диалоговое окно **Save As**

16. Введите в текстовое поле **Имя файла:** "TextApp2" и нажмите кнопку **Сохранить**. Из диалогового окна исчезнет вся информация, введенная в него в последнем сеансе.
17. Закройте окно документа и снова выберите команду меню **File**, **Open...**.
18. В списке файлов диалогового окна **Open** выделите файл TextApp2.tpf и нажмите кнопку **Открыть**. В окне документа будут отображены изменения, внесенные в документ в последнем сеансе работы с ним.

Модификация функции `OnLButtonDown` заключается в том, что в нее добавлены операторы записи координат точек в файл пользовательского буфера обмена. Для этого используется функция `CFile::Write`, первый аргумент которой содержит указатель на созданный пользователем буфер, содержимое которого будет записано в файл, а второй ее аргумент содержит размер этого буфера в байтах.

В функцию `OnRButtonDown` внесены более серьезные изменения, поскольку на нее возлагается задача сохранения информации, внесенной пользователем в документ в текущем сеансе работы.

Прежде всего, данная функция выводит на экран стандартное диалоговое окно **Save As**. Для этого она вызывает функцию `CDocManager::DoPromptFileName`. Переменная `m_pDocManager`, содержащая указатель на объект класса `CDocManager`, является членом класса `CWinApp`. Для получения указателя на связанный с данным приложением объект класса `CWinApp` используется глобальная функция `AfxGetApp`. Функция `CDocManager::DoPromptFileName` не документирована. Первым ее аргументом является ссылка на объект класса `CString`, в который будет записано имя файла, выбранное в вызываемом диалоговом окне. Ее второй аргумент — идентификатор строкового ресурса заголовка диалогового окна. Заголовок диалогового окна **Open** содержится в строковом ресурсе `AFX_IDS_OPENFILE`, а заголовок диалогового окна **Save As** — в строковом ресурсе `AFX_IDS_SAVEFILE`. В третьем аргументе данной функции содержатся флаги, определяющие вид и поведение диалогового окна. Наиболее часто используемые значения этих флагов приведены ниже:

- `OFN_HIDEREADONLY` — в диалоговом окне не отображаются файлы, имеющие атрибут "только для чтения"
- `OFN_PATHMUSTEXIST` — выдает сообщение об ошибке в случае задания в диалоговом окне несуществующего каталога
- `OFN_FILEMUSTEXIST` — выдает сообщение об ошибке в случае задания в диалоговом окне несуществующего файла

Четвертый аргумент данной функции имеет тип `BOOL` и определяет тип выводимого диалогового окна. Если этот аргумент имеет значение `TRUE`, то на экран выводится диалоговое окно **Open**, в противном случае выводится диалоговое окно **Save As**. Последний, пятый аргумент данной функции пред-

ставляет собой указатель на объект класса `CDocTemplate`, который должен использоваться при открытии файлов. Если этот аргумент имеет ненулевое значение, то в текстовом поле раскрывающегося списка **Тип файла:** выводится фильтр данного шаблона. Кроме этого фильтра раскрывающийся список **Тип файла:** будет содержать только универсальный фильтр **All Files ("*.*)**. Если данный аргумент имеет нулевое значение, то в текстовое поле списка **Тип файла:** выводится фильтр первого зарегистрированного шаблона документа, а сам список данного шаблона **Тип файла:** содержит фильтры всех зарегистрированных шаблонов документов и универсальный фильтр.

См. также главу 2

Если пользователь при работе с диалоговым окном **Save As** нажал кнопку **Сохранить:**, функция `CDocManager::DoPromptFileName` возвращает ненулевое значение, а в ее первом аргументе содержится имя файла, выбранного пользователем в диалоговом окне **Save As**. Из этого имени выделяется текущий каталог и сохраняется в системном реестре, как это было описано при рассмотрении функции `Serialize`.

После внесения изменений в системный реестр функция `OnRButtonDown` создает объект класса `CFile` и запоминает указатель на этот объект в переменной `lpOutFile`. В случае возникновения ошибок при создании объекта данного класса его конструктор вызывает исключение. При обработке данного исключения вызывается глобальная функция `MessageBox`, выводящая сообщение об ошибке, и работа данного приложения прерывается вызовом функции `exit`.

В случае успешного создания объекта класса `CFile` в него записывается количество точек, содержащихся в пользовательском буфере обмена. При этом для файла пользовательского буфера обмена вызывается функция `CFile::GetLength`, позволяющая получить логическую длину файла в байтах. Эта величина делится на размерность элемента данных и на их количество в структуре.

Поскольку при записи информации в файл текущая позиция находится в конце файла, для файла пользовательского буфера обмена вызывается функция `CFile::SeekToBegin`, устанавливающая текущую позицию в начало файла.

После этого информация из файла пользовательского буфера обмена копируется в выходной файл. Для этого при помощи функции `CFile::Read` выполняются чтение из файла пользовательского буфера обмена и последующая его запись в выходной файл с использованием функции `CFile::Write`. Поскольку функция `CFile::Read` возвращает количество прочитанных ею байт, то процесс копирования будет продолжаться, пока данная функция не возвратит нулевое значение, то есть, пока не будет скопирован весь файл пользовательского буфера обмена.

После этого работа функции OnRButtonDown аналогична работе ее исходной версии.

В конструкторе класса CTextAppDoc инициализируются указатели на объекты класса CFile. Прежде всего в конструкторе данного класса проверяется, не существует ли на диске файл с именем, используемым для пользовательского буфера обмена. Если такой файл существует, то пользователю выдается соответствующее сообщение, и приложение завершает свою работу. Необходимость данной проверки связана с тем, что файл пользовательского буфера обмена создается в объекте класса CTextAppDoc, который не является уникальным для данного приложения. Наличие нескольких буферов обмена, использующих для хранения информации один и тот же файл, может привести к некорректной работе приложения. Не говоря уже о том, что операционная система не позволяет уничтожать открытые файлы.

В реальном приложении, использующем один пользовательский буфер обмена, соответствующий объект класса CFile обычно создается и уничтожается в классе CMainFrame или в классе приложения, что гарантирует его уникальность для данного приложения. При этом проверка на существование файла не обязательна. Однако имя данного файла должно быть достаточно "экстравагантным", чтобы его случайно не использовал пользователь. Для этого можно использовать длинное имя файла, включив в него знаки препинания (например, запятые). Имя должно содержать хотя бы одну точку, но в конце ставить точку нежелательно, поскольку символы, стоящие после последней точки, будут рассматриваться как расширение файла. Таким образом, данный файл будет иметь не только экстравагантное имя, но и экстравагантное расширение.

Если на диске отсутствует файл с именем, зарезервированным для пользовательского буфера обмена, приложение создает объект класса CFile с использованием данного имени и при помощи глобальной функции SetFileAttributes, устанавливает для данного файла атрибут скрытого файла. Скрытый файл для пользовательского буфера обмена повышает "дуракоустойчивость" приложения, поскольку у пользователя не возникнет вопроса, что это за файл появился в каталоге, и желания его уничтожить.

В деструкторе класса CTextAppDoc уничтожаются объекты класса CFile, созданные в конструкторе данного класса. Хотя мы сами задали имя файла пользовательского буфера обмена и можем просто присвоить его соответствующей переменной, для его получения в деструкторе используется функция CFile::GetFilePath, возвращающая полный путь к файлу. Это особенно важно, если при задании имени файла пользовательского буфера обмена использовалось только имя файла с расширением. В этом случае при изменении пользователем текущего каталога в процессе работы с приложением, при использовании функции GetFileName приложение не сможет уничтожить данный файл, поскольку будет искать его в другом каталоге.

После этого файл пользовательского буфера обмена закрывается функцией `CFile::Close`, но связанный с ним объект не уничтожается. Если для файла пользовательского буфера обмена заранее известно, что он был создан, то для выходного файла такая информация отсутствует. Если объект класса `CFile` не был создан, то любые операции с ним приводят к возникновению ошибок, поэтому перед вызовом функции `CFile::Close` для данного объекта производится проверка на существование самого объекта. После закрытия файлов соответствующие объекты класса `CFile` уничтожаются.

После закрытия файла пользовательского буфера обмена и уничтожения соответствующего ему объекта класса `CFile`, данный файл удаляется с диска. Для этого вызывается функция `CFile::Remove`, которой в качестве аргумента передается имя уничтожаемого файла.

Примечание

Смысл включения в класс функции, которая может работать только после прекращения работы с объектом данного класса и не использующей хранимой в нем информации, понятен только особо просвещенным специалистам фирмы Microsoft.

Удаление операторов из функции `CTextAppApp::InitInstance` связано с тем, что они выводили на экран пустое окно документа, создавая при этом файл пользовательского буфера обмена. Это затрудняло дальнейшую работу с приложением.

Работа с системным реестром

В 16-разрядных версиях Windows для хранения настроек приложения использовались специальные файлы инициализации, обычно имевшие расширение .ini. Работа с ними доставляла много хлопот программисту, поскольку при переносе приложения с одной машины на другую приходилось следить за тем, чтобы все связанные с ним файлы инициализации, обычно записывавшиеся в системный каталог, были полностью перенесены на новый компьютер. В 32-разрядных версиях Windows приложение хранит всю необходимую для его запуска информацию в *системном реестре*. Теперь приложение, перенесенное на другой компьютер вместо того, чтобы искать по всему компьютеру связанные с ним файлы инициализации и жаловаться на их отсутствие, при первом запуске использует информацию, заданную по умолчанию, а затем вносит соответствующие записи в системный реестр.

В прежних версиях Windows файлы инициализации представляли собой обычные текстовые файлы, в которые можно было внести изменения с помощью любого текстового редактора, что было сопряжено с определенным риском. В отличие от них файл системного реестра можно редактировать только при помощи специального редактора реестра. Исполняемому файлу

данного редактора, расположенному в главной папке Windows, присвоено имя RegEdit.exe. Для его запуска достаточно нажать кнопку **Пуск**, расположенную в панели задач, и в раскрывшемся меню выбрать команду **Выполнить...**. Появится диалоговое окно **Запуск программы**, изображенное на рис. 7.11.

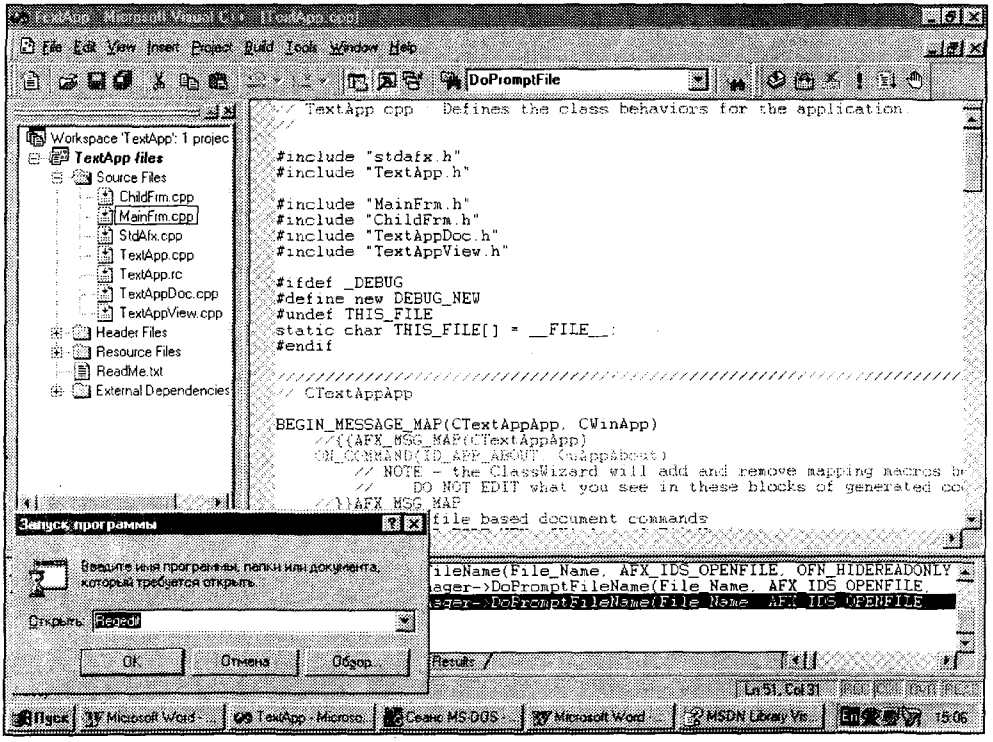


Рис. 7.11. Диалоговое окно **Запуск программы**

Введите в текстовое поле **Открыть:** имя исполняемого файла Regedit, как это показано на рис. 7.11, и нажмите кнопку **OK**. Появится окно **Редактор реестра**, изображенное на рис. 7.12.

В левой панели окна приведены все предопределенные ключи реестра. Каждый ключ имеет несколько уровней иерархии, которые при желании можно последовательно разворачивать. Если в левой панели выделен терминальный элемент дерева, то в правой выводится хранимая по этому идентификатору информация.

На рис. 7.12 приведены шесть предопределенных ключей. Каждому из них соответствует определенная информация:

- HKEY_CLASSES_ROOT — содержит информацию о типах документов и их свойствах, а также информацию о классах и различных приложениях,

установленных на компьютере. Здесь, помимо прочего, хранится информация о том, какое приложение обрабатывает файлы с каждым из зарегистрированных в данной системе расширений

- HKEY_CURRENT_USER — содержит информацию обо всех системных установках, произведенных пользователем. Здесь также хранятся все параметры настройки создаваемых пользователем приложений
- HKEY_LOCAL_MACHINE — содержит информацию о данном компьютере
- HKEY_USERS — содержит информацию обо всех пользователях, зарегистрированных в системе, и о конфигурации системы по умолчанию
- HKEY_CURRENT_CONFIG — содержит информацию о конфигурации аппаратных средств системы
- HKEY_DYN_DATA — служит для хранения динамических данных реестра, т. е. содержит быстро изменяющуюся информацию

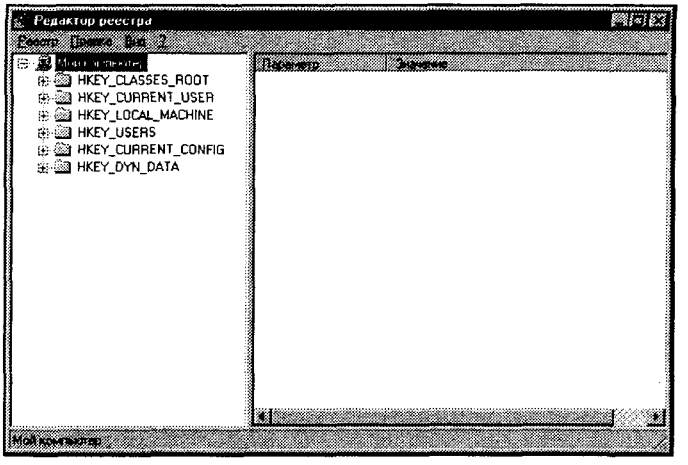


Рис. 7.12. Окно **Редактор реестра**

На рис. 7.13 приведен системный реестр с раскрытой папкой \CONTROL, в которой хранится рабочий каталог приложения.

Чтобы приложение могло получить доступ к системному реестру в функции `InitInstance` пользовательского класса, производного от класса `CWinApp`, необходимо вызвать функцию `SetRegistryKey`, в качестве аргумента которой следует указать индивидуальный ключ приложения. Этот ключ отображается в окне **Редактор реестра** в виде папки, расположенной в папке \Software, которая, в свою очередь, расположена в папке \HKEY_CURRENT_USER. В этой папке содержится вся информация, записываемая в системный реестр приложением. По умолчанию мастер AppWizard использует в качестве ключа строку "Local AppWizard-Generated Applications" (Локальное приложение, созданное мастером AppWizard). Если пользователь не изменил ключа,

установленного по умолчанию, то для каждого из приложений, использующих данный ключ, создается отдельная папка с именем данного приложения, в которую и записываются ее установки. Однако при создании серьезных приложений рекомендуется изменить устанавливаемый по умолчанию ключ и использовать вместо него, например, имя вашей фирмы.

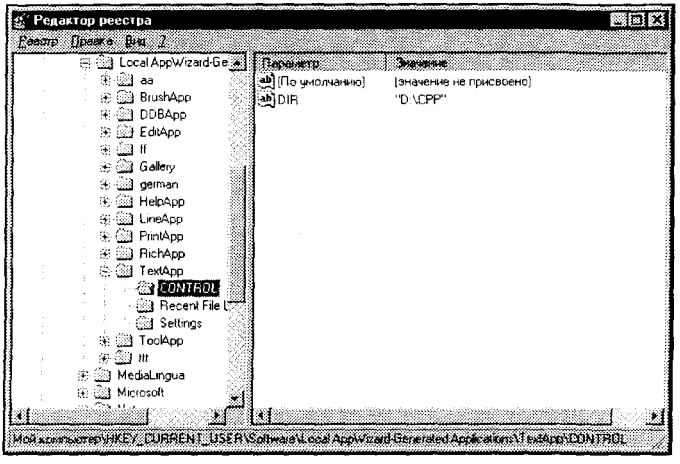


Рис. 7.13. Раскрытая папка \CONTROL

После этого следует вызвать функцию `LoadStdProfileSettings`, загружающую стандартные опции приложений, включая список последних файлов, с которыми работало данное приложение. В качестве ее аргумента указывается размерность списка последних файлов, расположенного в меню **File**, с которыми работало данное приложение. Существенным недостатком списка является то, что файлы в нем приводятся без расширений. Это очень неудобно, если приложение может работать с документами, характеризующими один и тот же объект с разных сторон. При этом естественно давать файлам, относящимся к одному объекту одно и то же имя и различать их по расширениям. В этом использование списка файлов в меню **Файл** будет похоже на игру в русскую рулетку: пользователь будет вынужден выбирать между несколькими одинаковыми именами файлов в списке и пытаться с первого раза угадать, какой из них имеет нужное ему расширение.

Для записи/чтения информации системного реестра используются специальные функции, являющиеся членами класса `CWinApp`. Функция `WriteProfileString` позволяет записать в системный реестр строку текста, а функция `WriteProfileInt` позволяет записать туда целое число. Для извлечения данной информации из реестра используются функции `GetProfileString` и `GetProfileInt` соответственно.

Работа с текстовыми документами

Как неоднократно отмечалось в этой книге, специалисты Microsoft, видимо, считают текстовые документы единственным достойным объектом своей работы. Вся операционная система разрабатывалась именно для работы с данным типом документов. Поэтому, рассматривая Visual C++, нельзя обойти молчанием эту область его применения.

Однако вследствие своей любви к написанию редакторов создатели Visual C++ самостоятельно разработали два типа редакторов, обеспечив их всеми необходимыми с их точки зрения функциями и, посчитав свои творения совершенством, практически исключили всякое вмешательство в их работу со стороны пользователя. Эти редакторы реализованы в объектах классов `CEdit` и `CRichEditView`. Первый из них представляет собой простейший редактор, используемый в текстовых полях диалоговых окон, и не предусматривает форматирования текста и использование различных шрифтов. Все эти возможности имеются в текстовом редакторе, реализованном в классе `CRichEditView`. Возможности, предоставляемые этим редактором, наглядно продемонстрированы в текстовом редакторе WordPad, являющемся примером использования данного класса, текст которого можно найти в библиотеке MSDN.

Создание простейшего текстового редактора

Текст демонстрационного приложения простейшего текстового редактора, использующего возможности класса `CEdit` расположен в каталоге \EditApp дискеты, прилагаемой к данной книге. Чтобы самостоятельно создать данное приложение:

1. Создайте приложение с именем EditApp по методике, описанной в главе 1. Оставьте все установки по умолчанию без изменения до последнего, шестого шага, включительно.
2. На шестом шаге выберите в раскрывающемся списке **Base class:** базовый класс `CEditView`. Диалоговое окно **MFC AppWizard** примет вид, изображенный на рис. 8.1.
3. Нажмите кнопку **Finish** и завершите процедуру создания приложения.

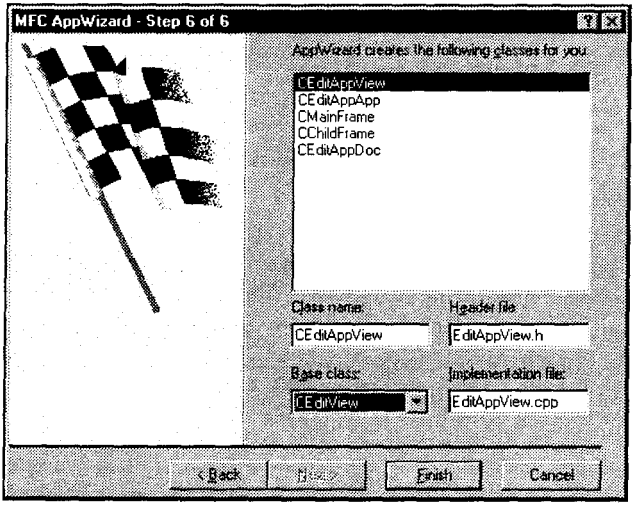


Рис. 8.1. Диалоговое окно **MFC AppWizard**

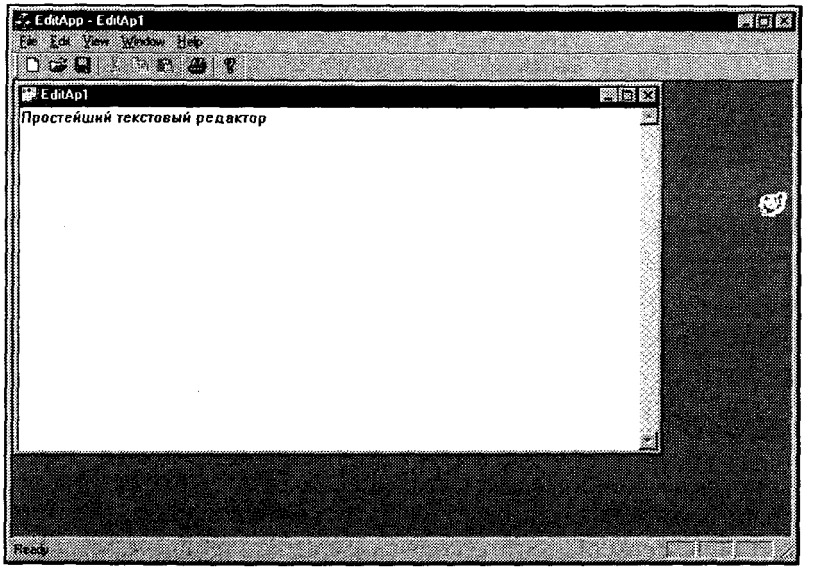


Рис. 8.2. *Простейший текстовый редактор*

4. Нажмите клавишу <F5> и запустите приложение на исполнение. Открывшееся в приложении окно будет окном текстового редактора, как это показано на рис. 8.2.

Данное приложение является законченным текстовым редактором, позволяющим вводить текст, вырезать или копировать его в буфер обмена и вставлять текст, содержащийся в буфере обмена, в любое место окна, указанное текстовым курсором. Для этого используются команды меню **Edit, Cut**; **Edit, Copy** и **Edit, Paste**, а также соответствующие им кнопки панели инструментов. В созданном окне текстового редактора автоматически появляются полосы прокрутки, причем, в отличие от обычного окна, они не исчезают в том случае, когда весь текст помещается в окно, а становятся недоступными. Кроме того, в нем реализованы возможности печати и предварительного просмотра печати.

Создание более сложного редактора

Чтобы более подробно ознакомиться с возможностями, предоставляемыми классом `CEdit`, создадим приложение, позволяющее находить слова в тексте и автоматически прокручивать текст в процессе его чтения. Чтобы внести в приложение EditApp соответствующие изменения:

1. Раскройте вкладку **ResourceView** окна **Workspace** и последовательно раскройте папки \EditApp resources и \Menu.
2. Дважды щелкните левой кнопкой мыши на значке **IDR_EDITAPTYPE**. Откроется окно редактирования ресурса меню.
3. Раскройте меню **Edit** и щелкните правой кнопкой мыши в прямоугольнике, расположенном в нижней части раскрывшегося меню. В появившемся меню выберите команду **Properties**. Появится диалоговое окно **Menu Item Properties**.
4. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна.
5. Установите флажок **Separator**. В заготовке меню появится разделитель.
6. Щелкните левой кнопкой мыши в прямоугольнике, расположенном под разделителем, и введите в текстовое поле раскрывающегося списка **ID:** появившегося диалогового окна **Menu Item Properties** идентификатор ресурса `ID_SEEK`.
7. Введите в текстовое поле **Caption:** заголовок "&Find...", а в текстовое поле **Prompt:** — текст "Finds the specified text\nFind".
8. Щелкните правой кнопкой мыши на папке \Dialog, расположенной в папке \EditApp resources вкладки **ResourceView** окна **Workspace**.
9. В появившемся контекстном меню выберите команду **Insert Dialog**. В окне редактирования ресурса появится заготовка диалогового окна, а во вклад-

ке **ResourceView** окна **Workspace** раскроется папка \Dialog, и в ней появится идентификатор ресурса `IDD_DIALOG1`.

10. Щелкните левой кнопкой мыши на значке **IDD_DIALOG1** и введите в текстовое поле раскрывающегося списка **ID:** появившегося диалогового окна **Dialog Properties** идентификатор ресурса `IDD_SEEK`.
11. Щелкните левой кнопкой мыши по свободному полю заготовки диалогового окна и введите в текстовое поле **Caption:** диалогового окна **Dialog Properties** заголовок окна "Поиск".
12. Переместите кнопки **OK** и **Cancel**, расположенные в заготовке диалогового окна, в его нижнюю часть.
13. Нажмите кнопку **Edit Box** панели инструментов **Controls** и, не отпуская кнопки, перетащите появившуюся рамку текстового поля в заготовку диалогового окна. Отредактируйте ее размеры в окне.
14. Введите в текстовое поле **ID:** диалогового окна **Edit Properties** идентификатор ресурса `IDC_SAMPLE`.
15. Нажмите кнопку **Static Text** панели инструментов **Controls** и, не отпуская кнопки, перетащите появившуюся рамку статического текста в заготовку диалогового окна.
16. Введите в текстовое поле **Caption:** диалогового окна **Text Properties** текст "Искать:". В результате этих операций заготовка диалогового окна примет вид, изображенный на рис. 8.3.
17. Закройте диалоговое окно **Text Properties** и вызовите мастер **ClassWizard** из окна редактирования ресурса диалога.
18. В появившемся диалоговом окне **Adding a class** оставьте все установки без изменения и нажмите кнопку **OK**. Появится диалоговое окно **New Class**.
19. В текстовое поле **Name:** введите имя нового класса `CSeekDialog` и нажмите кнопку **Change**. Появится диалоговое окно **Change Files**, изображенное на рис. 8.4.
20. Нажмите кнопку **Browse...**, расположенную напротив текстового поля **Header file:**. Появится диалоговое окно **Browse Files**, изображенное на рис. 8.5.
21. В списке файлов выделите имя файла EditAppView.h и нажмите кнопку **OK**.
22. Нажмите кнопку **Browse...**, расположенную напротив текстового поля **Implementation file:**. Диалоговое окно **Browse Files** примет вид, показанный на рис. 8.6.
23. В окне списка файлов выделите имя файла EditAppView.cpp и нажмите кнопку **OK**.
24. Нажмите кнопку **OK** в диалоговом окне **Change Files**.

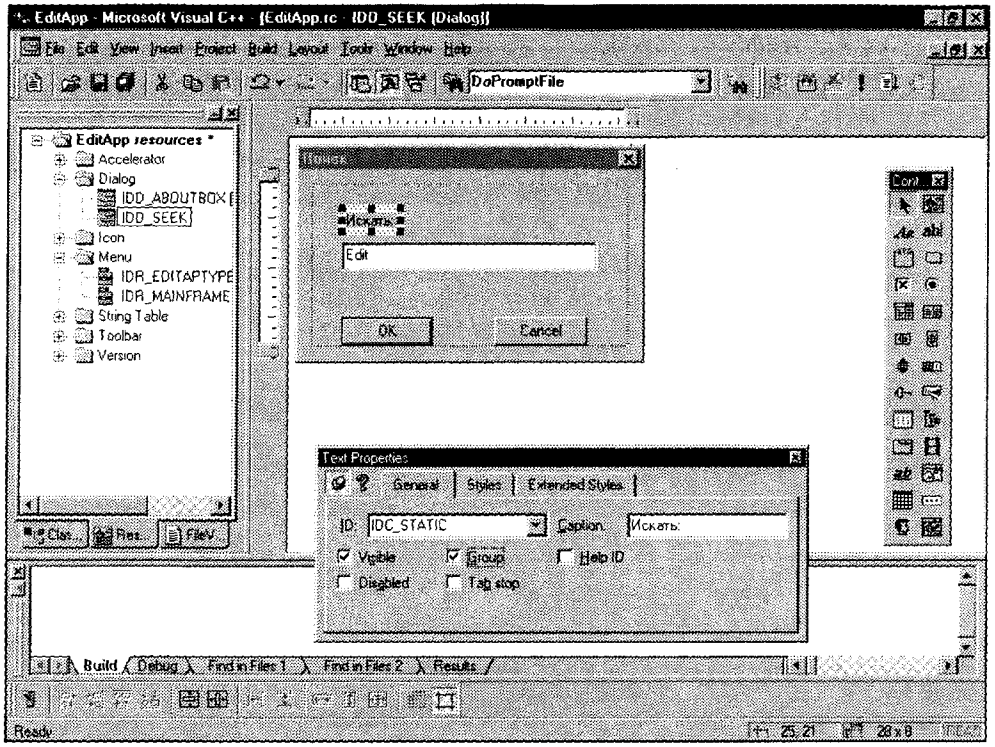


Рис. 8.3. Заготовка диалогового окна

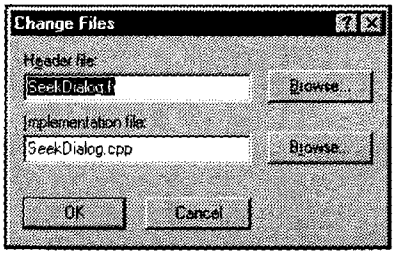


Рис. 8.4. Диалоговое окно **Change Files**

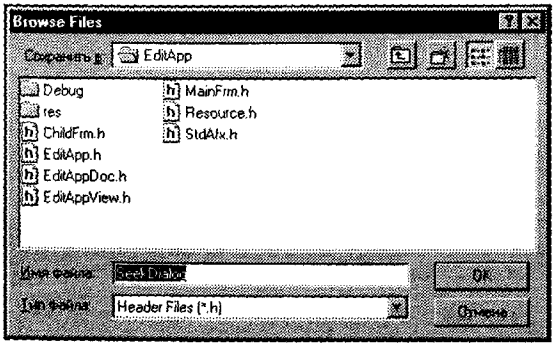


Рис. 8.5. Диалоговое окно **Browse Files**

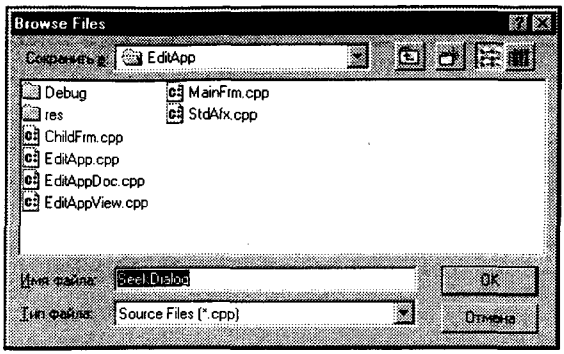


Рис. 8.6. Диалоговое окно **Browse Files**

25. Нажмите кнопку **OK** в диалоговом окне **New Class**. Диалоговое окно **MFC ClassWizard** станет доступным.
26. Раскройте вкладку **Member Variables** и выделите идентификатор ресурса `IDC_SAMPLE` в окне списка **Control IDs:**.
27. Нажмите кнопку **Add Variable**, откроется диалоговое окно **Add Member Variable**.
28. Введите в текстовое поле **Member variable name:** идентификатор переменной `m_Sample` и нажмите кнопку **OK**.
29. Раскройте вкладку **Message Maps** и в раскрывающемся списке **Class name:** выберите имя класса `CEditAppView`. Появится окно сообщения, предлагающее сохранить изменения, внесенные в класс `CSeekDialog`. Нажмите кнопку **OK**.
30. В окне списка **Object IDs:** выделите имя класса `CEditAppView`, в окне списка **Messages:** выделите сообщение `WM_LBUTTONDBLCLK` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится новое имя функции обработки сообщения `OnLButtonDblClk`.
31. В окне списка **Object IDs:** выделите идентификатор ресурса `ID_SEEK`, в окне списка **Messages:** выделите сообщение `COMMAND` и нажмите кнопку **Add Function**. Появится диалоговое окно **Add Member function**. Не изменяя его установок, нажмите кнопку **OK**. В окне списка **Member functions:** появится новое имя функции обработки сообщения `OnSeek`.
32. Нажмите кнопку **Edit Code** и закройте диалоговое окно **MFC ClassWizard**. Откроется окно редактирования файла EditAppView.cpp.
33. Перенесите заготовку функции `OnSeek` в область функций класса `CEditAppView` и измените ее в соответствии с приведенным ниже текстом:

```
void CEditAppView::OnSeek()
{
  // TODO: Add your command handler code here

  CSeekDialog    dlg(this);
```

```
    if( dlg.DoModal() == IDOK){
      FindText( dlg.m_Sample);
    }
  }
```

34. Перенесите заготовку функции `OnLButtonDblClk` в область функций класса `CEditAppView` и измените ее в соответствии с приведенным ниже текстом:

```
void CEditAppView::OnLButtonDblClk(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  CEdit&  edit = GetEditCtrl();

  UINT  n = 0xffff & edit.CharFromPos( point);

  while( n < (int) GetBufferLength()){
    n++;
    edit.SetSel( n, n);
    Sleep( 200);
  }

  CEditView::OnLButtonDblClk(nFlags, point);
}
```

35. Нажмите клавишу <F5> и запустите приложение на исполнение.
36. Введите в редактор текст с таким расчетом, чтобы в окне появилась вертикальная полоса прокрутки.
37. Перейдите в начало введенного текста и щелкните левой кнопкой мыши в первой или второй строке.
38. В месте щелчка появится текстовый курсор, который начнет перемещаться по тексту. Как только он дойдет до последней строки, окно автоматически прокрутится на одну строку. Дойдя до конца текста, курсор автоматически переместится в то место на экране, где был произведен щелчок левой кнопкой мыши. Если в этом месте на экране текст отсутствует, то курсор переместится в конец первой строки на экране.
39. Выберите команду меню **Edit**, **Find...**. Появится диалоговое окно **Поиск**.
40. Введите в текстовое поле последовательность символов, которую необходимо найти в тексте, и нажмите кнопку **OK**. Если эта последовательность имеется в тексте после текущей позиции курсора, она будет найдена и выделена блоком. В противном случае курсор останется на прежнем месте.

Данное приложение демонстрирует возможности управления текстовым курсором в редакторе. Первым оператором функции `OnLButtonDown` является

создание ссылки на объект класса CEdit, связанный с данным объектом класса приложения. Для этого используется функция CEditView::GetEditCtrl.

Затем при помощи полученной ссылки на объект класса CEdit вызывается функция CEdit::CharFromPos, позволяющая получить индекс ближайшего к данной точке символа и индекс строки, в которой находится данный символ. Формат возвращаемой данной функцией величины указывает на то, что объем текста не может превысить 65 534 символа, что явно недостаточно для любого серьезного редактора.

Поскольку интересующая нас величина находится в младшем слове, то возвращаемое значение функции CharFromPos маскируется и записывается в счетчик текущей позиции.

После инициализации процедуры перемещения курсора вызывается цикл его перемещения по экрану. Чтобы данная процедура завершилась немедленно по достижении курсором последнего символа в тексте, счетчик текущей позиции постоянно сравнивается с числом символов в тексте. Для получения этой информации вызывается функция CEditView::GetBufferLength.

Для реализации возможности поиска текста в меню была введена специальная команда, а для задания искомого шаблона создано специальное диалоговое окно. Программирование всех необходимых для работы с диалоговым окном операций взял на себя мастер ClassWizard, поэтому единственным изменением, которое требуется внести в программу пользователю, сводится к заполнению заготовки функции OnSeek соответствующим программным текстом.

Первым оператором данной функции является создание объекта класса диалогового окна, в которое будет вводиться искомый текст. После этого вызывается функция CDialog::DoModal, осуществляющая вывод диалогового окна на экран и обеспечивающая работу с ним. В том случае, если пользователь нажал кнопку **OK**, вызывается функция CEditView::FindText, осуществляющая поиск текста в буфере. Вместо нее можно было бы вызвать функцию CEditView::OnFindNext самостоятельно обрабатывающую ситуацию, когда искомый текст не найден, и выдающую при этом звуковой сигнал, но в этом случае пришлось бы указывать три аргумента вместо одного.

Форматирование документов

Как уже говорилось выше, форматирование документов и работа со шрифтами осуществляются в объект класса CRichEditView. Базовый набор его возможностей реализован в приложении WordPad, текст которого можно найти в библиотеке MSDN. Однако это довольно-таки сложное приложение, и его описание займет слишком много места. Поэтому здесь будет создано простейшее приложение, позволяющее форматировать текст.

Текст приложения, демонстрирующего возможности класса `CRichEditView` и связанных с ним классов, расположен в каталоге \RichApp дискеты, прилагаемой к данной книге. Чтобы самостоятельно создать данное приложение:

1. Создайте приложение с именем RichApp по методике, описанной в главе 1. Оставьте все установки по умолчанию без изменения до последнего, шестого шага, включительно.
2. На шестом шаге выберите в раскрывающемся списке **Base class:** базовый класс `CRichEditView`. Вид диалогового окна **MFC AppWizard** на шестом шаге мастера AppWizard приведен на рис. 8.1.
3. Нажмите кнопку **Finish**. Появится окно сообщения, предупреждающее о том, что использование класса `CRichEditView` предполагает использование контейнера OLE и предлагающее включить поддержку контейнеров OLE в проект. Это окно изображено на рис. 8.7.
4. Нажмите кнопку **OK** и завершите процедуру создания приложения.
5. Нажмите клавишу <F5> и запустите приложение на исполнение.

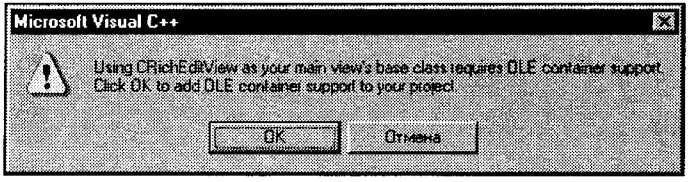


Рис. 8.7. Окно сообщения **Microsoft Visual C++**

На первый взгляд оно ничем не будет отличаться от приложения TextApp. Однако в нем появятся дополнительные команды меню. Например, не закрывая данное приложение, вызовите текстовый редактор Word, выделите в нем фрагмент текста и скопируйте его в буфер обмена. После этого перейдите в приложение RichApp и выберите команду **Edit**, **Paste Special...** (Правка, Специальная вставка). Появится диалоговое окно **Paste Special**, изображенное на рис. 8.8.

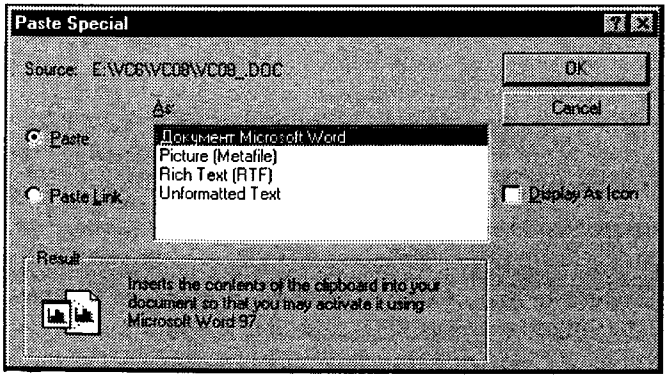


Рис. 8.8. Диалоговое окно **Paste Special**

В этом окне указан источник, из которого был взят фрагмент, расположенный в буфере обмена, и предложен формат, в котором его можно вставить в документ данного приложения. Оставьте все установки без изменения и нажмите кнопку **OK**. Вставленный текст будет обведен тонкой рамкой. Дважды щелкните левой кнопкой мыши внутри этой рамки. Результат этого действия показан на рис. 8.9.

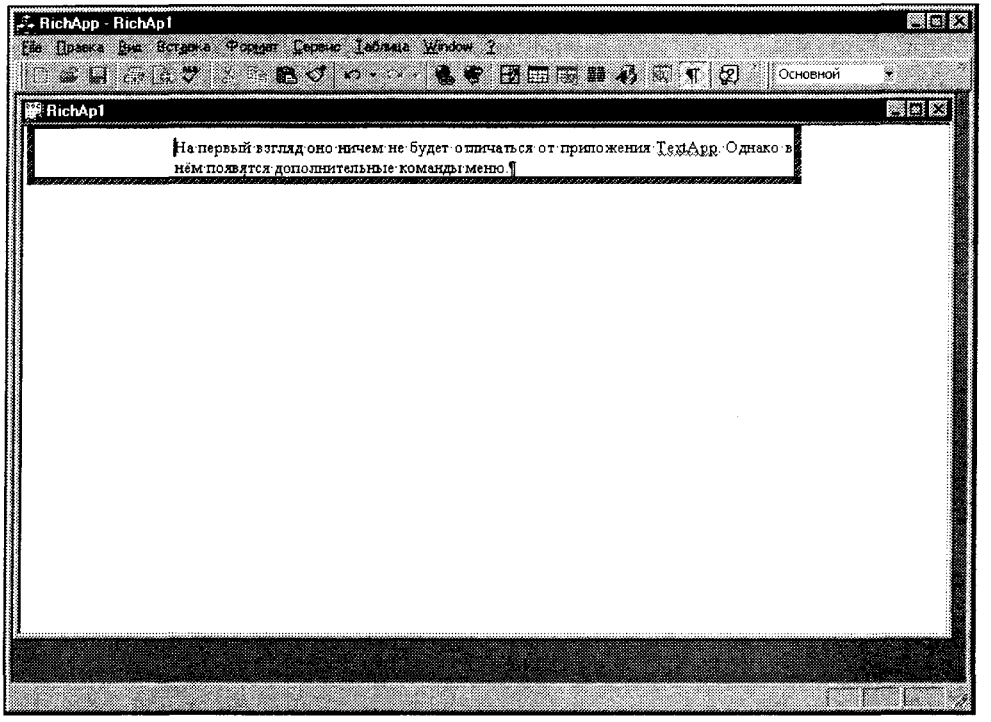


Рис. 8.9. Приложение RichApp с текстом в формате Word

Как видно из рис. 8.9, рамка вокруг текста становится толстой, а вместо меню и панели инструментов приложения появляется меню и панель инструментов Word. Таким образом, данное приложение безо всяких изменений позволяет редактировать документы в формате Word. Того же самого результата можно добиться, вызвав команду меню **Edit**, **Insert New Object...**, выделив в списке **Object Type:** диалогового окна **Insert Object**, изображенного на рис. 8.10, опцию **Документ Microsoft Word** и нажав кнопку **OK**.

В этом случае в окне документа сразу появляется пустая толстая рамка, готовая для ввода текста.

Однако цель данного раздела не в том, чтобы показать возможности контейнера OLE, а в том, чтобы продемонстрировать способы работы с классом `CRichEditView`.

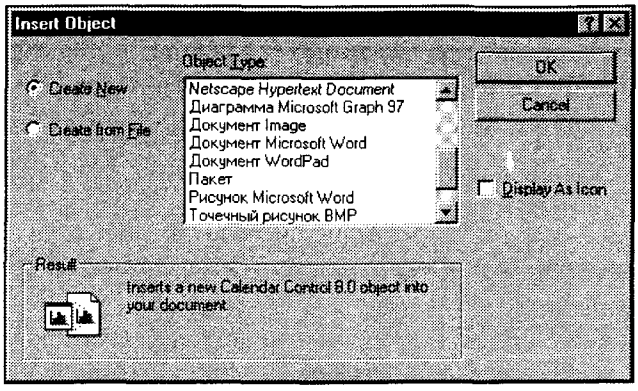


Рис. 8.10. Диалоговое окно **Insert Object**

Чтобы внести в приложение RichApp соответствующие изменения:

1. Откройте вкладку **ResourceView** окна **Workspace** и последовательно раскройте папки \RichtApp resources и \Menu.
2. Дважды щелкните левой кнопкой мыши на значке **IDR_RICHAPTYPE**. Откроется окно редактирования соответствующего ресурса меню.
3. Щелкните правой кнопкой мыши в прямоугольнике, расположенном в левой части строки меню и в появившемся меню выберите команду **Properties**. Появится диалоговое окно **Menu Item Properties**.
4. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна.
5. Переместите курсор мыши на прямоугольник, расположенный в левой части строки меню, нажмите левую кнопку мыши и, удерживая ее, переместите появившийся разделитель с двумя направленными вовнутрь стрелками между пунктами меню **View** и **Window**.
6. Пустой прямоугольник переместится в позицию между этими пунктами меню.
7. Введите в текстовое поле **Caption:** диалогового окна **Menu Item Properties** заголовок "&Format".
8. Щелкните левой кнопкой мыши в прямоугольнике, расположенном под заголовком пункта меню, и введите в текстовое поле раскрывающегося списка **ID:** появившегося диалогового окна **Menu Item Properties** идентификатор ресурса `ID_LEFT`.
9. Введите в текстовое поле **Caption:** заголовок "&Left", а в текстовое поле **Prompt:** — текст "Align left \nLeft".
10. Повторите операции, описанные в пунктах 8 и 9, установив идентификатор ресурса `ID_CENTER`, введя в текстовое поле **Caption:** заголовок "&Center", а в текстовое поле **Prompt:** — текст "Center text\nCenter".

11. Повторите операции, описанные в пунктах 8 и 9, установив идентификатор ресурса ID_RIGHT, введя в текстовое поле **Caption:** заголовок "&Right", а в текстовое поле **Prompt:** — текст "Align right\nRight".
12. Установите в диалоговом окне **Menu Item Properties** флажок **Separator**.
13. Щелкните левой кнопкой мыши в прямоугольнике, расположенном под разделителем и повторите операции, описанные в пунктах 8 и 9, установив идентификатор ресурса ID_ITALIC, введя в текстовое поле **Caption:** заголовок "&Italic", а в текстовое поле **Prompt:** — текст "Italic font\nItalic".
14. Повторите операции, описанные в пунктах 8 и 9, установив идентификатор ресурса ID_UNDERLINE, введя в текстовое поле **Caption:** заголовок "&Underline", а в текстовое поле **Prompt:** — текст "Underlined font\nUnderline".
15. Закройте диалоговое окно **Menu Item Properties** и вызовите мастер ClassWizard из окна редактирования ресурса меню.
16. Раскройте вкладку **Message Maps** диалогового окна **MFC ClassWizard**.
17. В окне списка **Object IDs:** выделите идентификатор ресурса ID_LEFT, в окне списка **Messages:** выделите сообщение COMMAND и нажмите кнопку **Add Function**. Появится диалоговое окно **Add Member function**. Не изменяя его установок нажмите кнопку **OK**. В окне списка **Member functions:** появится новое имя функции обработки сообщения OnLeft.
18. Повторите данную операцию для идентификаторов ресурса ID_CENTER, ID_RIGHT, ID_ITALIC и ID_UNDERLINE.
19. Нажмите кнопку **Edit Code** и закройте диалоговое окно **MFC ClassWizard**. Откроется окно редактирования файла RichAppView.cpp.
20. Измените текст созданных функций обработки сообщений меню в соответствии с приведенным ниже текстом:

```
////////////////////////////////////////////////////////////////////
// CRichAppView message handlers

void CRichAppView::OnLeft()
{
  // TODO: Add your command handler code here

  OnParaAlign( PFA_LEFT);
}

void CRichAppView::OnCenter()
{
  // TODO: Add your command handler code here

  OnParaAlign( PFA_CENTER);
}
```

```
void CRichAppView::OnRight()
{
  // TODO: Add your command handler code here

  OnParaAlign( PFA_RIGHT);
}

void CRichAppView::OnItalic()
{
  // TODO: Add your command handler code here

  OnCharEffect( CFM_ITALIC, CFE_ITALIC);
}

void CRichAppView::OnUndeline()
{
  // TODO: Add your command handler code here

  OnCharEffect( CFM_UNDERLINE, CFE_UNDERLINE);
}
```

21. Нажмите кнопку <F5> и запустите приложение на исполнение.
22. Выберите команду меню **Format**, **Italic** и введите текст в окно документа. Текст будет выведен курсивом.
23. Выберите команду **Format**, **Underline** и продолжите ввод текста. Текст будет выведен подчеркнутым курсивом.
24. Снова выберите команду **Format**, **Italic** и продолжите ввод текста. Текст будет выведен подчеркнутым обычным шрифтом.
25. Выберите команду **Format**, **Underline** и продолжите ввод текста. Текст будет выведен обычным шрифтом.
26. Выберите команду **Format**, **Right**. Введенный текст будет выровнен по правому краю.
27. Выберите команду **Format**, **Center**. Введенный текст будет выровнен по центру.
28. Выберите команду **Format**, **Left**. Введенный текст будет выровнен по левому краю. Окно документа примет вид, изображенный на рис. 8.11.

Как видно из текстов функций обработки, сообщение команд меню, процесс форматирования абзацев и шрифтов в данном приложении не представляет особых сложностей. Для форматирования абзацев используется функция `CRichEditView::OnParaAlign`, аргумент которой определяет требуемый режим форматирования выделенного абзаца. Для форматирования шрифтов используется функция `CRichEditView::OnCharEffect`. Ее первый аргумент является маской, определяющей изменения, которые можно внести в данный

шрифт, а второй определяет конкретные изменения, которые необходимо внести в данный шрифт.

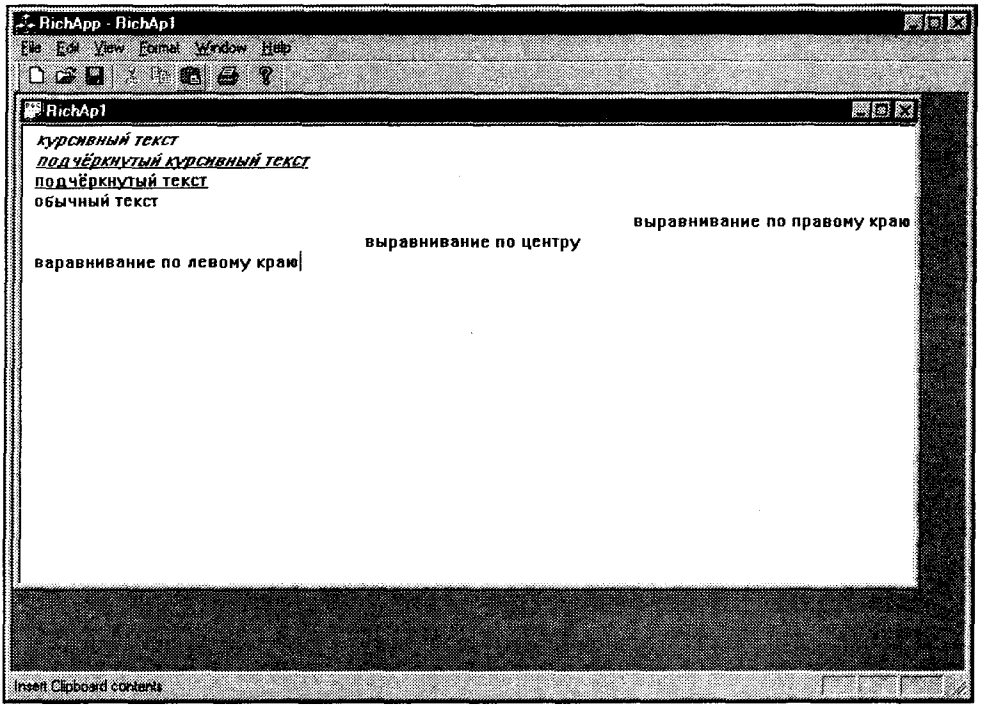


Рис. 8.11. Окно приложения RichApp

Задание пользовательского шрифта

Одним из отличий редактора, создаваемого с использованием класса `CRichEditView` по сравнению с редактором, создаваемым с использованием класса `CEdit`, является возможность работы с пользовательскими шрифтами. Чтобы продемонстрировать возможности работы класса `CRichEditView` по работе с пользовательскими шрифтами:

1. Выберите команду **File**, **Open Workspace...** (Файл, Открыть проект). В раскрывшемся диалоговом окне **Open Workspace** раскройте папку \RichApp и дважды щелкните левой кнопкой мыши на значке **RichApp.dsw**. Откроется соответствующий проект.
2. Раскройте вкладку **ResourceView** окна **Workspace** и последовательно раскройте папки \RichtApp resources и \Menu.
3. Дважды щелкните левой кнопкой мыши на значке **IDR_RICHAPTYPE**. Откроется окно редактирования ресурса меню.

4. Раскройте меню **Format** и щелкните правой кнопкой мыши в прямоугольнике, расположенном в нижней части раскрывшегося меню. В появившемся меню выберите команду **Properties**. Появится диалоговое окно **Menu Item Properties**.
5. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна.
6. Установите флажок **Separator**. В заготовке меню появится разделитель.
7. Щелкните левой кнопкой мыши в прямоугольнике, расположенном под разделителем, и введите в текстовое поле раскрывающегося списка **ID:** появившегося диалогового окна **Menu Item Properties** идентификатор ресурса `ID_FONT`.
8. Введите в текстовое поле **Caption:** заголовок "&My Font", а в текстовое поле **Prompt:** — текст "Set user font\nMy Font".
9. Закройте диалоговое окно **Menu Item Properties** и вызовите мастер ClassWizard.
10. Раскройте вкладку **Message Maps** диалогового окна **MFC ClassWizard**.
11. В окне списка **Object IDs:** выделите идентификатор ресурса `ID_FONT`, в окне списка **Messages:** выделите сообщение `COMMAND` и нажмите кнопку **Add Function**. Появится диалоговое окно **Add Member function**. Не изменяя его установок, нажмите кнопку **OK**. В окне списка **Member functions:** появится новое имя функции обработки сообщения `OnFont`.
12. Нажмите кнопку **Edit Code** и закройте диалоговое окно **MFC ClassWizard**. Откроется окно редактирования файла RichAppView.cpp.
13. Измените текст функции `OnFont` в соответствии с приведенным ниже текстом:

```
void CRichAppView::OnFont()
{
  // TODO: Add your command handler code here

  CHARFORMAT&     cf = GetCharFormatSelection();

  cf.dwMask       |= CFM_COLOR;
  cf.yHeight       = 600;
  cf.crTextColor     = RGB( 0, 0, 255);
  cf.bCharSet       = ANSI_CHARSET;
  cf.bPitchAndFamily  = VARIABLE_PITCH | FF_ROMAN;
  strcpy(cf.szFaceName, "MS Sans Serif");

  if( cf.dwEffects & CFE_AUTOCOLOR)
    cf.dwEffects = 0;

  SetCharFormat( cf);
}
```

14. Нажмите клавишу <F5> и запустите приложение на исполнение.
15. Введите какой-нибудь текст в окно документа, после чего выберите команду **Format**, **My Font**.
16. Продолжите ввод текста в окно документа. При этом текст будет иметь другое начертание, другой размер и будет иметь синий цвет, как это показано на рис. 8.12.

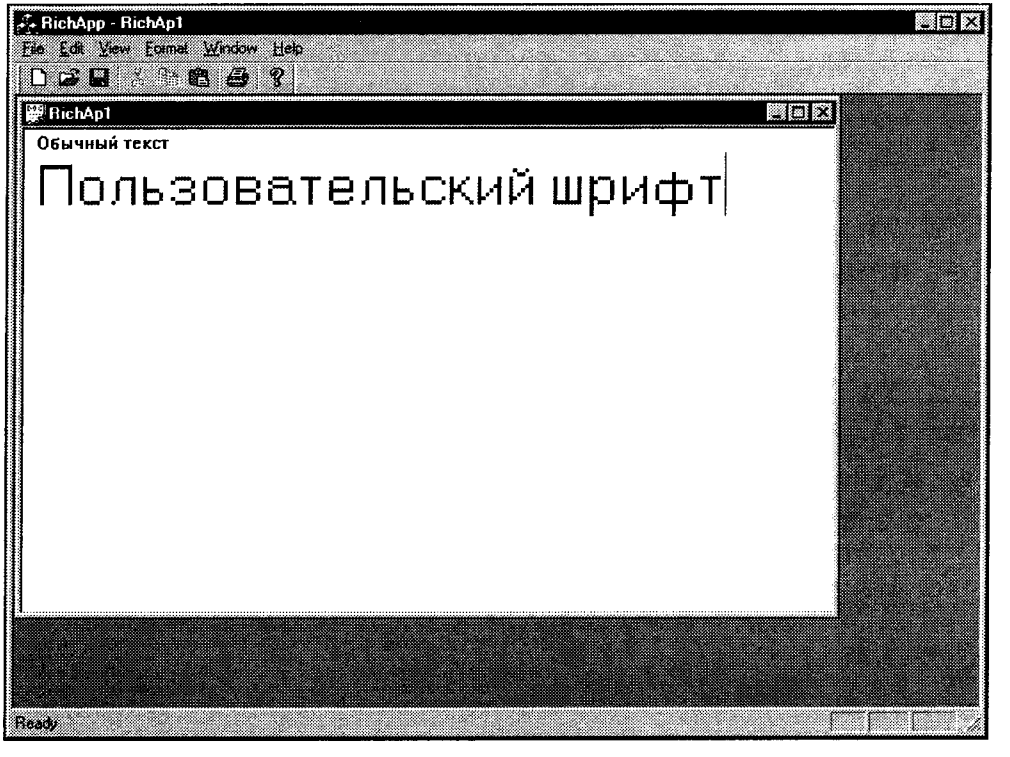


Рис. 8.12. Окно приложения RichApp с пользовательским шрифтом

Для задания параметров пользовательского шрифта использовался объект структуры `CHARFORMAT`, содержащей различные параметры форматирования шрифтов. Все операции форматирования шрифта, рассмотренные нами в разделе, посвященном форматированию текста, можно было бы осуществить с использованием объекта структуры `CHARFORMAT`, тем более, что аргументы функции `CRichEditView::OnCharEffect` являются, по сути, членами данной структуры.

Функция `OnCharEffect` поддерживает меньше операций форматирования шрифта, чем структура `CHARFORMAT`, но работать с функцией намного удобнее.

В первом операторе функции OnFont получается ссылка на объект класса CHARFORMAT, содержащий параметры текущего шрифта. Для этого используется функция CRichEditView::GetCharFormatSelection, позволяющая получить текущие атрибуты форматирования выделенного участка текста. После этого в полученный объект структуры CHARFORMAT вносятся изменения. Прежде всего в переменную dwMask данной структуры добавляется флаг CFM_COLOR, позволяющий изменять цвет текста (вообще-то, этот флаг уже установлен по умолчанию, но осторожность не помешает). В переменной yHeight устанавливается высота шрифта в пиках. В переменную crTextColor записывается значение нового цвета шрифта. Значение переменной bCharSet определяет используемый набор символов. Переменная bPitchAndFamily определяет, будет ли шрифт иметь фиксированную или переменную ширину литер, а также семейство, к которому принадлежит данный шрифт. В текстовой переменной szFaceName хранится имя шрифта. После задания параметров шрифта проверяется, установлен ли в переменной dwEffects флаг CFE_AUTOCOLOR, препятствующий использованию переменной crTextColor для задания цвета текста и, если этот флаг установлен, то он сбрасывается вместе со всеми остальными флагами данной переменной. В завершение функции OnFont вызывается функция CRichEditView::SetCharFormat, устанавливающая атрибуты форматирования нового текста в объекте класса CRichEditView. В качестве аргумента данной функции используется модифицированный объект структуры CHARFORMAT.

Панели инструментов и строка состояния

Панели инструментов представляют собой неотъемлемую часть интерфейса любой полноценной программы Windows. Основное их назначение заключается в обеспечении пользователю быстрого доступа к наиболее часто используемым командам меню. В принципе, можно создать панель инструментов, кнопки которой не будут дублироваться в меню, но это существенно усложнит работу с приложением без использования мыши.

Строка состояния располагается в нижней части главного окна программы и служит для отображения расширенной справки по командам меню и кнопкам панели инструментов. Кроме того, строка состояния содержит специальные поля для вывода дополнительной информации. По умолчанию в них отображается состояние расположенных на клавиатуре индикаторов Caps Lock (поле **Cap**), Num Lock (поле **Num**) и Scroll Lock (поле **SCRL**). Если соответствующий светодиод на клавиатуре не светится, то в данном поле ничего не выводится. В противном случае в поле выводится его идентификатор, указанный в скобках.

Наличие в приложении панели инструментов и строки состояния определяется на четвертом шаге работы мастера MFC AppWizard, окно которого изображено на рис. 9.1. Чтобы создаваемое приложение имело стандартную панель инструментов, не нужно сбрасывать установленный по умолчанию флажок **Docking toolbar** (*перемещаемая панель инструментов*). Чтобы создаваемое приложение имело строку состояния, не нужно сбрасывать установленный по умолчанию флажок **Initial status bar** (отображение строки состояния по умолчанию).

Работа с панелью инструментов

По умолчанию приложение создает стандартную панель инструментов, состоящую из четырех групп кнопок. Эта панель инструментов изображена на рис. 9.2.

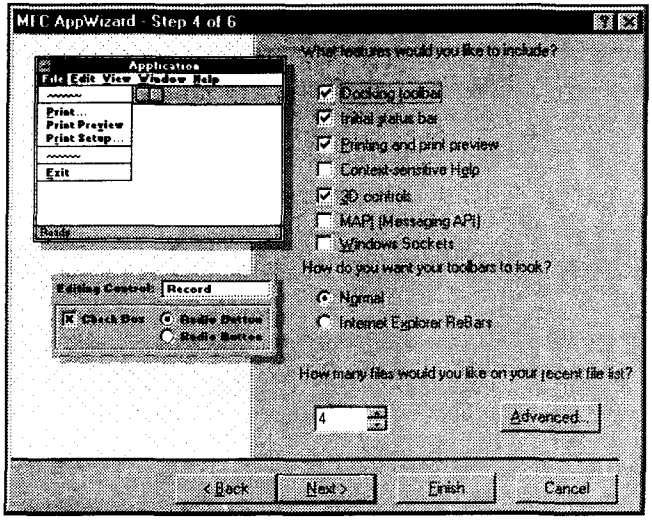


Рис. 9.1. Диалоговое окно **MFC AppWizard — Step 4 of 6**

Рис. 9.2. Стандартная панель инструментов приложения

В первую группу кнопок, выполняющих действия по созданию и сохранению документа, входят кнопки (New, открытие нового документа), (Open, открытие существующего документа) и (Save, сохранение документа). Во вторую группу, осуществляющих операции редактирования документа, входят кнопки (Cut, вырезать выделенный блок в буфер обмена), (Copy, копировать выделенный блок в буфер обмена) и (Paste, вставить блок из буфера обмена). В третью группу входит кнопка (Print, печать текущего документа), а в четвертую — (About, вызов диалогового окна, содержащего сведения о приложении).

Система программирования Visual C++ позволяет добавлять и уничтожать кнопки в стандартной панели инструментов, но в большинстве случаев для новых кнопок целесообразнее использовать новые панели инструментов, а из стандартной панели инструментов только удалять ненужные кнопки.

Удаление кнопок из панели инструментов

Предположим, что в создаваемом приложении нет необходимости в использовании буфера обмена. В этом случае необходимо или постоянно делать недоступными соответствующие кнопки панели инструментов или же просто удалить их из данной панели инструментов. Второй путь, конечно же предпочтительнее, поскольку, как говорил К. С. Станиславский, если в первом акте на стене висит ружье, то в третьем акте оно должно выстрелить. Постоянное присутствие в панели инструментов "нестреляющих" кнопок

вызывает у пользователя законный вопрос: "В каком же режиме они работают?"

Текст приложения, позволяющего продемонстрировать принципы работы с панелью инструментов, расположен в каталоге \ToolApp дискеты, прилагаемой к данной книге. Чтобы самостоятельно создать данное приложение:

1. Создайте приложение с именем ToolApp по методике, описанной в главе 1. Оставьте все установки по умолчанию без изменения.
2. Раскройте вкладку **ResourceView** окна **Workspace**. Раскройте папку \ToolApp resources, а в ней — папку \Toolbar.
3. Дважды щелкните на идентификаторе ресурса панели инструментов **IDR_MAINFRAME**. В окне редактора ресурсов появится панель инструментов, в которой будет выделена первая кнопка, а ее увеличенное изображение будет находиться в специальной области редактирования. Экран при этом примет вид, изображенный на рис. 9.3.

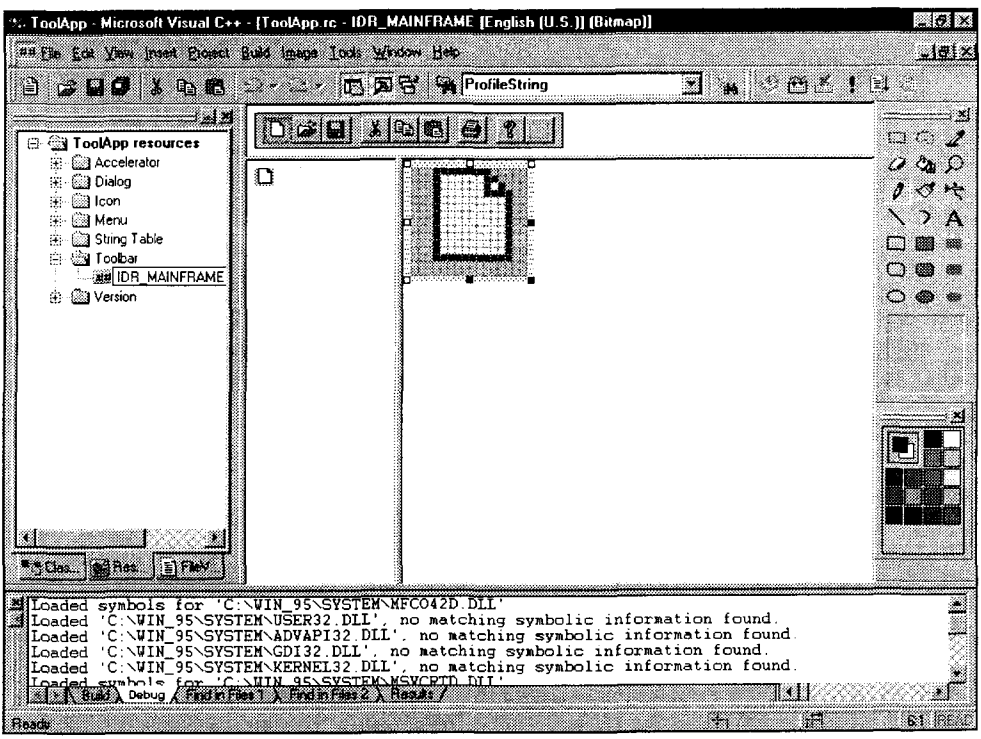


Рис. 9.3. Редактор ресурса панели инструментов

4. Поместите указатель мыши на изображение кнопки **Cut** в панели инструментов, изображенной в редакторе ресурса. Нажмите левую кнопку мыши и, не отпуская ее, перетащите данную кнопку за пределы панели

инструментов. После отпускания кнопки мыши перетаскиваемое изображение исчезнет, а панель инструментов не будет содержать кнопки **Cut**.

5. Повторите эту же операцию для кнопок **Copy** и **Paste**. Панель инструментов не будет содержать кнопок редактирования документа.
6. Нажмите клавишу <F5> и запустите приложение на исполнение. Главное окно приложения примет вид, показанный на рис. 9.4.

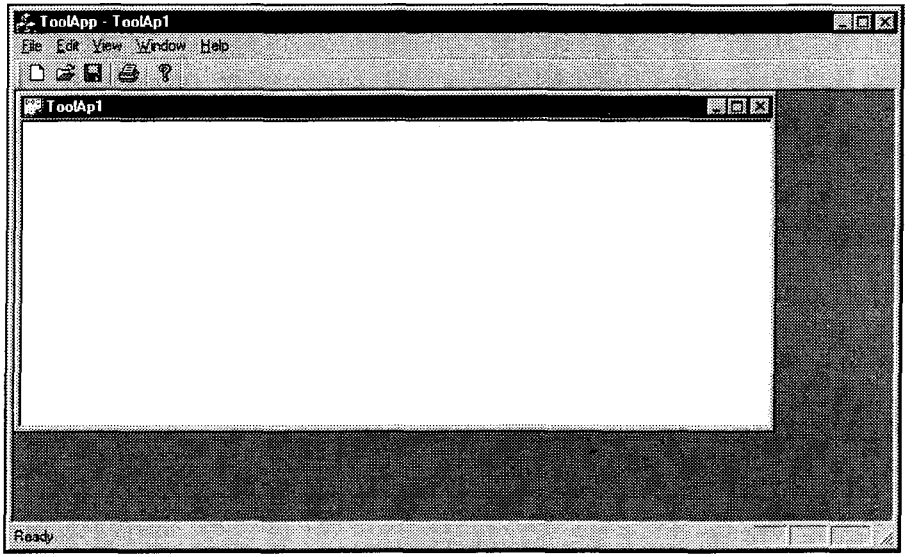


Рис. 9.4. Главное окно приложения ToolApp

Если с удаляемыми кнопками были связаны какие-либо сообщения, обрабатывавшиеся приложением, то перед удалением кнопок из панели инструментов необходимо предварительно уничтожить функции обработки этих сообщений.

Добавление кнопок в панель инструментов

Поскольку процедура добавления кнопки в стандартную панель инструментов ничем не отличается от процедуры добавления кнопки в любую другую панель инструментов, а создание пользовательской панели инструментов является далеко не тривиальной задачей, то в созданное в предыдущем разделе приложение будет добавлена еще одна панель, содержащая всего одну кнопку.

Чтобы внести соответствующие изменения в приложение ToolApp:

1. Выберите команду **File**, **Open Workspace...** и в открывшемся диалоговом окне **Open Workspace** раскройте папку \ToolApp, выделите значок **ToolApp.dsw** и нажмите кнопку **Открыть**.

2. Раскройте вкладку **ResourceView** окна **Workspace**. Щелкните правой кнопкой мыши на папке \ToolApp Resources и в появившемся меню выберите команду **Insert...**. Появится диалоговое окно **Insert Resource** (рис. 9.5).

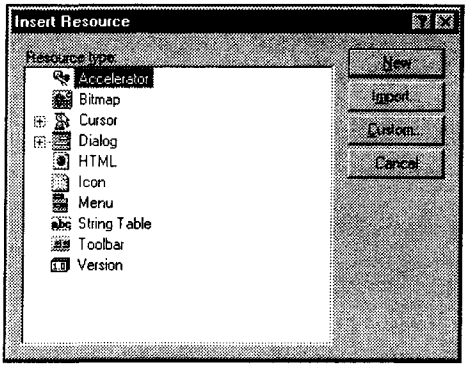


Рис. 9.5. Диалоговое окно **Insert Resource**

3. Выделите значок **ToolBar** и нажмите кнопку **New**. В окне редактирования ресурса появится новая панель инструментов, и экран примет вид, изображенный на рис. 9.6.

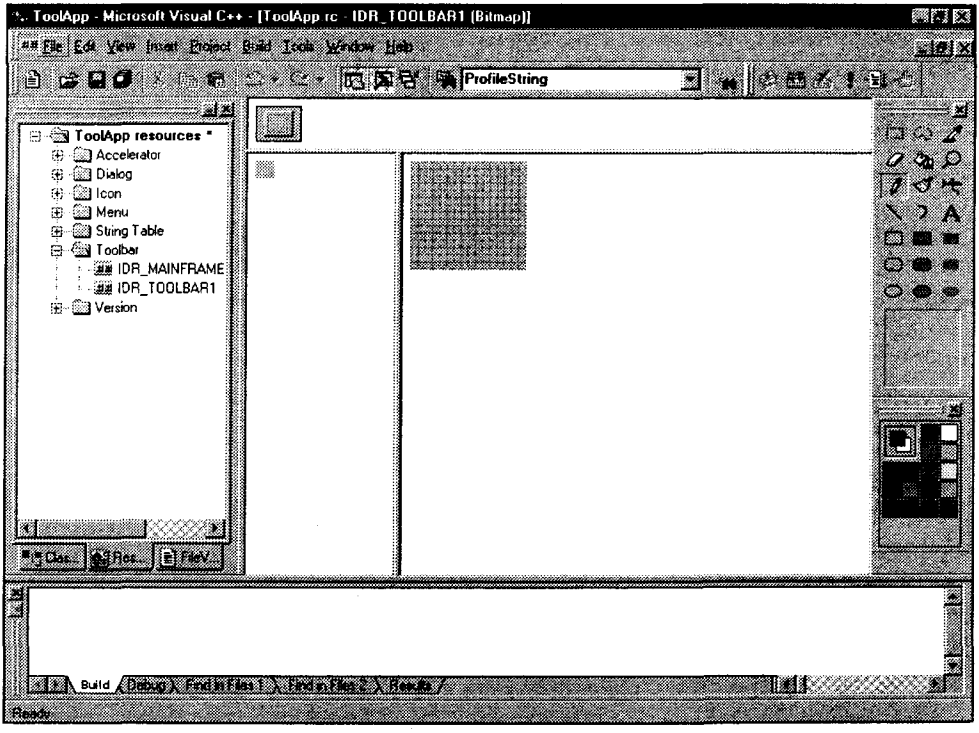


Рис. 9.6. Окно редактирования пользовательской панели инструментов

4. Щелкните правой кнопкой мыши на идентификаторе `IDR_TOOLBAR1`, расположенном в папке \Toolbar вкладки **Resource View** окна **Workspace**.
5. В раскрывшемся контекстном меню выберите команду **Properties**. Раскроется диалоговое окно **Toolbar Properties**.
6. Нажмите на кнопку **Keep Visible** в левом верхнем углу диалогового окна **Toolbar Properties**.
7. В текстовом поле раскрывающегося списка **ID:** замените идентификатор ресурса `IDR_TOOLBAR1` идентификатором ресурса `IDR_TOOLBAR`.
8. Используя панели инструментов **Graphics** и **Colors**, расположенные справа от окна редактирования ресурса, нарисуйте в новой кнопке панели инструментов красный квадрат в черной рамке.
9. Введите в текстовое поле раскрывающегося списка **ID:** диалогового окна **Toolbar Button Properties** идентификатор ресурса `ID_SQUARE`. В текстовое поле **Prompt:** введите текст "Рисование красного квадрата\nКвадрат". Окно редактирования ресурса примет вид, изображенный на рис. 9.7.
10. Закройте диалоговое окно **Toolbar Button Properties**.

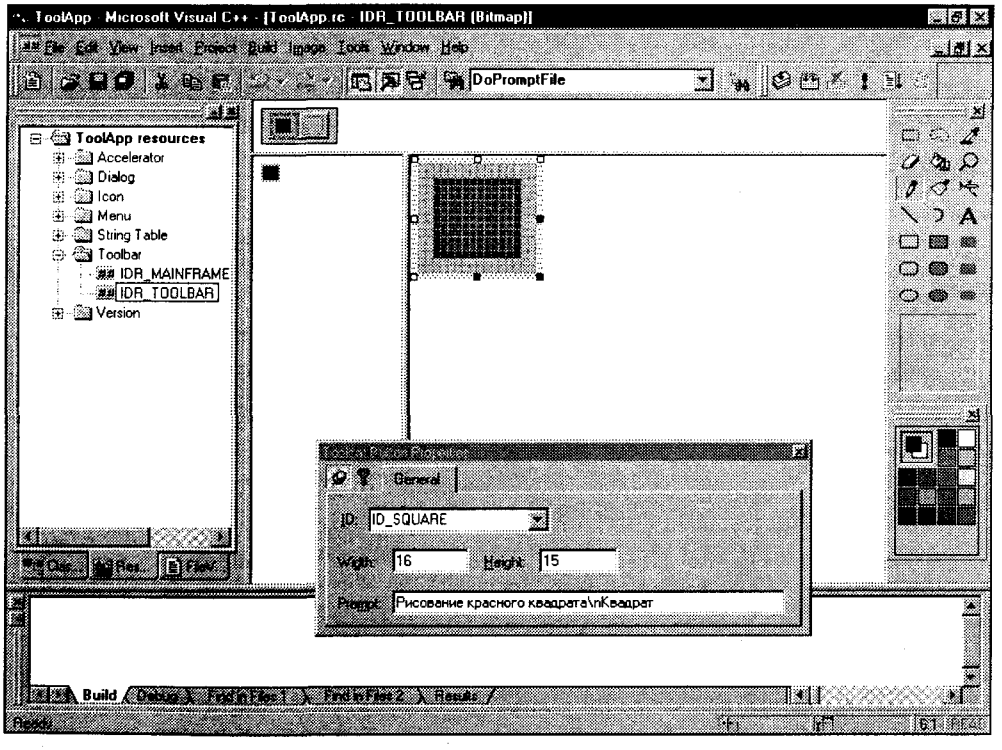


Рис. 9.7. Окно редактирования пользовательской панели инструментов

11. Выберите команду меню **View**, **ClassWizard**. Появится диалоговое окно **Adding a Class**, расположенное поверх диалогового окна **MFC ClassWizard** и предлагающее выбрать существующий класс для обработки сообщений от созданной панели инструментов.
12. Не меняя установок окна нажмите кнопку **OK**. Появится диалоговое окно **Select Class** (рис. 9.8).

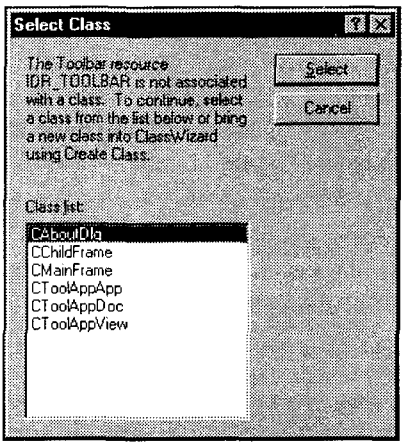


Рис. 9.8. Диалоговое окно **Select Class**

13. В окне списка **Class list:** выделите имя класса `CToolAppView` и нажмите кнопку **Select**. Появится диалоговое окно **MFC ClassWizard**.
14. Раскройте список **Class name:** и выберите в нем класс `CToolAppView`.
15. В окне списка **Object IDs:** уже выделен идентификатор ресурса `ID_SQUARE`, если он не выделен, то выделите его. В окне списка **Messages:** выделите сообщение `COMMAND` и нажмите кнопку **Add Function**. Появится диалоговое окно **Add Member Function**, предлагающее для функции обработки данного сообщения имя `OnSquare`. Не изменяйте его и нажмите кнопку **OK**. В окне списка **Member functions:** появится имя новой функции.
16. Повторите эту операцию для сообщения `UPDATE_COMMAND_UI`, функция обработки которого будет иметь имя `OnUpdateSquare`.
17. В окне списка **Object IDs:** выделите имя класса `CToolAppView`, в окне списка **Messages:** выделите сообщение `WM_LBUTTONDOWN` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится имя новой функции, `OnLButtonDown`.
18. Нажмите кнопку **OK**. Диалоговое окно **MFC ClassWizard** закроется.
19. Раскройте вкладку **ResourceView** окна **Workspace**. Раскройте папку \ToolApp Resources, а в ней папку \Menu. Дважды щелкните на значке **IDR_MAINFRAME**. Появится окно редактирования ресурса основного меню приложения.

20. В окне редактирования ресурса раскройте меню **View**.
21. Переместите курсор мыши на заштрихованный прямоугольник, расположенный в нижней части раскрывающегося меню и нажмите левую кнопку мыши.
22. Не отпуская кнопки мыши, переместите курсор мыши вверх и установите появившуюся при этом синюю линию, на концах которой расположены направленные внутрь стрелки, между пунктами меню **Toolbar** и **Status Bar**. Прямоугольник переместится в указанную позицию.
23. Щелкните в прямоугольнике правой кнопкой мыши и в раскрывшемся контекстном меню выберите команду **Properties**. Раскроется диалоговое окно **Menu Item Properties**.
24. Нажмите в нем кнопку **Keep Visible**, расположенную в левом верхнем углу.
25. В текстовое поле **ID:** введите идентификатор ресурса `ID_MY_TOOLBAR`, в текстовое поле **Caption:** введите заголовок "&My Toolbar", а в текстовое поле **Prompt:** — справочную информацию "Отображает или скрывает панель инструментов\nПанель инструментов".
26. Дважды щелкните на значке **IDR_TOOLAPTYPE** во вкладке **ResourceView** окна **Workspace**. Появится окно редактирования ресурса меню работы с документом приложения.
27. Повторите операции, описанные в пунктах 20—25, для данного ресурса меню.
28. Переместите курсор мыши на заштрихованный прямоугольник, расположенный в правой части меню и нажмите левую кнопку мыши. Переместите курсор мыши влево и установите появившуюся при этом синюю линию, на концах которой расположены направленные внутрь стрелки, между пунктами меню **View** и **Window**. Прямоугольник переместится в указанную позицию.
29. В текстовое поле **Caption:** диалогового окна **Menu Item Properties** введите заголовок "&Draw".
30. Щелкните левой кнопкой мыши в появившемся под данным заголовком раскрывающемся меню и выберите в раскрывающемся списке **ID:** идентификатор ресурса `ID_SQUARE`, в текстовое поле **Caption:** введите заголовок "&Square", а в текстовое поле **Prompt:** — справочную информацию "Рисование красного квадрата\nКвадрат".
31. Закройте диалоговое окно **Menu Item Properties**.
32. Раскройте вкладку **FileView** окна **Workspace**. Откройте окно редактирования файла ToolAppView.cpp и вызовите мастер ClassWizard.
33. В окне списка **Object IDs:** выделите идентификатор ресурса `ID_MY_TOOLBAR`, в окне списка **Messages:** выделите сообщение `UPDATE_COMMAND_UI` и на-

жмите кнопку **Add Function**. Появится диалоговое окно **Add Member Function**, предлагающее для функции обработки данного сообщения имя `OnUpdateMyToolbar`. Не изменяйте его и нажмите кнопку **OK**. В окне списка **Member functions:** появится имя новой функции.

34. В раскрывающемся списке **Class name:** выберите имя класса `CMainFrame`. Появится окно сообщения Microsoft Visial C++, предлагающее сохранить изменения, внесенные в класс `CToolAppView`.
35. Нажмите кнопку **Да** и повторите для класса `CMainFrame` операции, описанные в предыдущем пункте.
36. В окне списка **Object IDs:** выделите имя класса `CMainFrame`, в окне списка **Messages:** выделите сообщение `WM_CLOSE` и нажмите кнопку **Add Function**. Появится диалоговое окно **Add Member Function** предлагающее для функции обработки данного сообщения имя `OnClose`. Не изменяйте его и нажмите кнопку **OK**. В окне списка **Member functions:** появится имя новой функции.
37. *Нажмите кнопку* **OK** *и закройте окно* **MFC ClassWizard**.
38. Раскройте вкладку **ClassView** окна **Workspace**, раскройте в ней папку \ToolApp classes и щелкните правой кнопкой мыши на имени класса `CMainFrame`.
39. В появившемся контекстном меню выберите команду **Add Member Variable...**. Появится диалоговое окно **Add Member Variable**.
40. Введите в текстовое поле **Variable Type:** имя класса `CToolBar`, а в текстовое поле **Variable Name:** — идентификатор переменной `m_MyToolBar`.
41. Откройте окно редактирования файла MainFrm.h и в карте сообщений класса `CMainFrame` перед строкой `afx_msg void OnUpdateMyToolbar(CCmdUI* pCmdUI);` вставьте строку

```
afx_msg BOOL OnMyToolbarCheck(UINT nID);
```

42. Откройте окно редактирования файла MainFrm.cpp и в карте сообщений перед строкой `ON_UPDATE_COMMAND_UI(ID_MY_TOOLBAR, OnUpdateMyToolbar)` вставьте строку

```
ON_COMMAND_EX(ID_PROCESS_TOOLBAR, OnMyToolbarCheck)
```

43. Измените текст функции `OnCreate` в соответствии с приведенным ниже текстом:

```
int CMainFrame::OnCreate(LPCREATESTRUCT lpCreateStruct)
{
  if (CMDIFrameWnd::OnCreate(lpCreateStruct) == -1)
    return -1;

  if (!m_wndToolBar.CreateEx(this, TBSTYLE_FLAT, WS_CHILD |
                             WS_VISIBLE | CBRS_TOP | CBRS_GRIPPER |
```

```
                        CBRS_TOOLTIPS | CBRS_FLYBY | CBRS_SIZE_DYNAMIC) ||
                        !m_wndToolBar.LoadToolBar(IDR_MAINFRAME))
    {
      TRACE0("Failed to create toolbar\n");
      return -1;      // fail to create
    }

    if (!m_MyToolBar.CreateEx(this, TBSTYLE_FLAT, WS_CHILD | WS_VISIBLE |
                              CBRS_TOP | CBRS_GRIPPER | CBRS_TOOLTIPS |
                              CBRS_FLYBY | CBRS_SIZE_DYNAMIC) ||
                              !m_MyToolBar.LoadToolBar(IDR_TOOLBAR))
    {
      TRACE0("Failed to create toolbar\n");
      return -1;      // fail to create
    }

    if (!m_wndStatusBar.Create(this) ||
      !m_wndStatusBar.SetIndicators(indicators,
        sizeof(indicators)/sizeof(UINT)))
    {
      TRACE0("Failed to create status bar\n");
      return -1;      // fail to create
    }

    SetWindowLong( m_MyToolBar.m_hWnd, GWL_ID, ID_MY_TOOLBAR);

    // TODO: Delete these three lines if you don't want the toolbar
    // to be dockable
    m_wndToolBar.EnableDocking(CBRS_ALIGN_ANY);
    m_MyToolBar.EnableDocking(CBRS_ALIGN_ANY);
    EnableDocking(CBRS_ALIGN_ANY);
    DockControlBar(&m_wndToolBar);
    DockControlBar(&m_MyToolBar);
    LoadBarState( "Local AppWizard-Generated Applications");

    return 0;
}
```

44. В текст функции `OnClose` после строки `// TODO: Add your message handler code here and/or call default` вставьте строку

```
SaveBarState( "Local AppWizard-Generated Applications");
```

45. Измените текст функции `OnUpdateMyToolbar` в соответствии с приведенным ниже текстом:

```
void CMainFrame::OnUpdateMyToolbar(CCmdUI* pCmdUI)
{
  // TODO: Add your command update UI handler code here
```

```
    if (&m_MyToolBar != NULL)
    {
      pCmdUI->SetCheck((m_MyToolBar.GetStyle() & WS_VISIBLE) != 0);
      return;
    }

    pCmdUI->ContinueRouting();
}
```

46. В конец данного файла вставьте следующий текст:

```
BOOL CMainFrame::OnMyToolbarCheck(UINT nID)
{
  if (&m_MyToolBar != NULL)
  {
    ShowControlBar(&m_MyToolBar, (m_MyToolBar.GetStyle() &
WS_VISIBLE) == 0, FALSE);
    return TRUE;
  }

  return FALSE;
}
```

47. Раскройте вкладку **ClassView** окна **Workspace**, в ней раскройте папку \ToolApp classes и щелкните правой кнопкой мыши на имени класса `CToolAppView`.

48. В контекстном меню выберите команду **Add Member Variable...**. Появится диалоговое окно **Add Member Variable**.

49. Введите в текстовое поле **Variable Type:** тип переменной `bool`, а в текстовое поле **Variable Name:** — идентификатор переменной `bDrawn`.

50. Откройте окно редактирования файла ToolAppView.cpp и вставьте в конструктор класса `CToolAppView` строку

```
bDrawn  = false;
```

51. Измените функцию `OnSquare` в соответствии с приведенным ниже текстом:

```
void CToolAppView::OnSquare()
{
  // TODO: Add your command handler code here

  CClientDC  clientDC(this);
  CBrush     redBrush( RGB(255,0,0));
  CPen     pen( PS_SOLID, 0, RGB(0,0,255));
  CPen*     oldPen = dc.SelectObject( &pen);
  CBrush*     oldBrush = dc.SelectObject( redBrush);
```

```
    dc.Rectangle( 20, 20, 120, 120);
    dc.SelectObject( oldBrush);
    dc.SelectObject( oldPen);

    bDrawn  = true;

    CView::OnLButtonDown(nFlags, point);
}
```

52. Вставьте в текст функции `OnUpdateSquare` строку

```
pCmdUI-> Enable( !bDrawn);
```

53. Измените функцию `OnLButtonDown` в соответствии с приведенным ниже текстом

```
void CToolAppView::OnLButtonDown(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  bDrawn = false;

  Invalidate();

  CView::OnLButtonDown(nFlags, point);
}
```

54. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится главное окно приложения, изображенное на рис. 9.9.

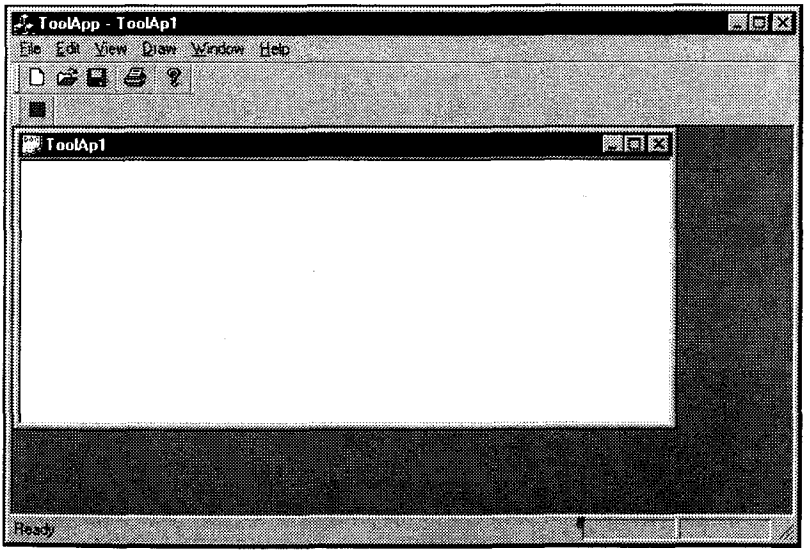


Рис. 9.9. Главное окно приложения ToolApp с пользовательской панелью инструментов

55. Нажмите кнопку (Квадрат) в пользовательской панели инструментов. В окне документа появится красный квадрат, а кнопка **Квадрат** в панели инструментов станет недоступной.
56. Щелкните левой кнопкой мыши в окне документа. Красный квадрат исчезнет, а кнопка **Квадрат** в пользовательской панели инструментов станет доступной.
57. Выберите команду меню **Draw**, **Square**. В окне документа появится красный квадрат, а кнопка **Квадрат** в панели инструментов станет недоступной.
58. Раскройте меню **Draw** и убедитесь, что команда **Square** в нем недоступна.
59. Снова щелкните левой кнопкой мыши в окне документа и закройте окно документа. Кнопка **Квадрат** в панели инструментов станет недоступной, а меню **Draw** исчезнет, как это показано на рис. 9.10.

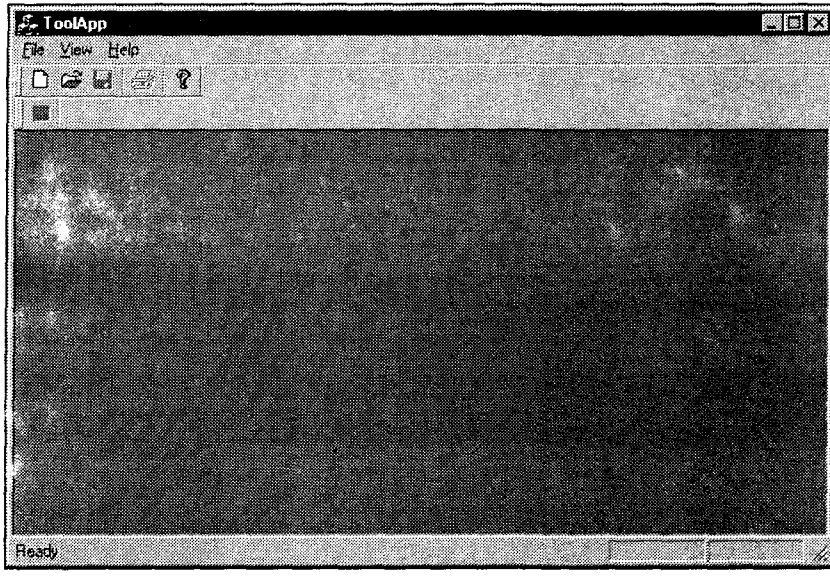


Рис. 9.10. Главное окно приложения ToolApp с закрытым окном документа

60. Откройте меню **View** и сбросьте флажок рядом с командой **My Toolbar**. Новая панель инструментов исчезнет из главного окна приложения, как это показано на рис. 9.11.
61. Снова откройте меню **View** и установите флажок команды **My Toolbar**. Новая панель инструментов появится в главном окне приложения.
62. Перетащите новую панель инструментов в один ряд со стандартной панелью, как показано на рис. 9.12.

63. Закройте приложение и снова запустите его на исполнение. Новая панель инструментов восстановит свое последнее положение.

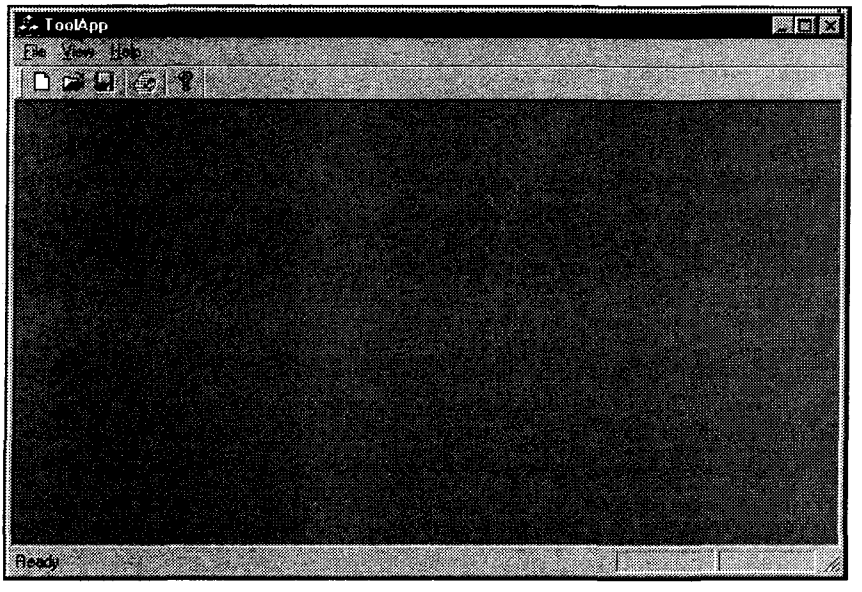


Рис. 9.11. Главное окно приложения ToolApp без пользовательской панели инструментов

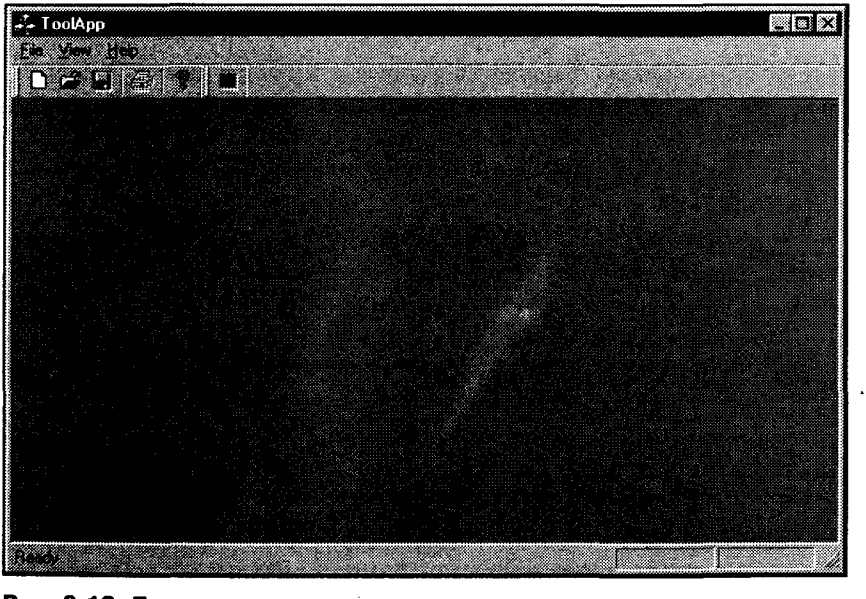


Рис. 9.12. Перемещенная пользовательская панель инструментов

Как следует из описанной выше процедуры, процесс создания пользовательской панели инструментов не полностью автоматизирован, и пользователю приходится вносить исправления там, где безраздельно должен хозяйничать мастер ClassWizard. Это заставляет предположить: разработчики Visual C++ не предполагали, что пользователям потребуется использовать несколько панелей инструментов.

Объект класса `CToolBar` включается пользователем в описание класса `CMainFrame` вручную, что не представляет особой сложности, поскольку в данном классе уже определен один объект класса `CToolBar` — стандартная панель инструментов. Если пользователь хочет иметь возможность убирать созданную им панель инструментов с экрана, он должен самостоятельно вставить в карту сообщений соответствующие макросы и объявления функций. Формат этих макросов и функций аналогичен используемым в данном приложении.

Панели инструментов создаются в функции `CMainFrame::OnCreate`, то есть при создании главного окна приложения. Создание пользовательской панели инструментов в приложении во многом аналогично созданию уже существующей стандартной панели инструментов, поэтому большинство операторов, используемых для ее создания повторяют уже содержащиеся в данной функции операторы создания стандартной панели инструментов.

Первым оператором данной функции является вызов метода базового класса. Следующим оператором является вызов функции `CToolBar::CreateEx` для объекта класса стандартной панели инструментов. Данная функция создает панель инструментов Windows (дочернее окно) и связывает ее с объектом класса `CToolBar`. После этого в том же условном операторе вызывается функция `CToolBar::LoadToolBar`, загружающая панель инструментов Windows, определяемую ее аргументом. В случае возникновения ошибки при выполнении одной из этих функций выдается сообщение об ошибке при помощи макроса `TRACE0`, не использующего строковых ресурсов, вопреки установленному разработчиками Windows правилу, согласно которому все выдаваемые системой сообщения должны храниться в строковых ресурсах для обеспечения возможности полной локализации программного обеспечения.

После выдачи сообщения об ошибке производится выход из функции с присвоением выходной величине значения −1, свидетельствующего об ошибке при создании главного окна приложения.

Следующий условный оператор выполняет те же действия для создания объекта класса пользовательской панели инструментов. Меняется только идентификатор объекта класса. В следующем условном операторе создается объект класса строки состояния. Поскольку мы еще не рассматривали данный класс, то пока оставим этот оператор без рассмотрения. Его описание будет приведено при рассмотрении класса строки состояния.

После этого следует вызов глобальной функции `SetWindowLong`, устанавливающей идентификатор окна пользовательской панели инструментов.

Примечание

Дело в том, что по непонятным причинам функции, используемые для создания панели инструментов, присваивают всем создаваемым ими панелям инструментов один и тот же идентификатор, 0x0000e800, что не позволяет приложению правильно работать с системным реестром, поскольку при записи и чтении информации из него две панели инструментов становятся неразличимыми, и их порядок появления на экране определяется не тем, как их расположил пользователь, а тем, в каком порядке они зарегистрированы в приложении.

Вызов данной функции устанавливает для пользовательской панели инструментов другой идентификатор, соответствующий идентификатору ее ресурса, что снимает проблемы, связанные с идентификацией панелей инструментов.

Функция `CControlBar::EnableDocking` определяет возможность фиксации панели управления на одной из сторон окна и определяет стороны, на которых может фиксироваться панель управления. В данном случае любая из панелей инструментов может фиксироваться на любой из сторон рабочей области главного окна приложения. После этого вызывается функция `CFrameWnd::EnableDocking`, разрешающая *фиксацию* (docking) панелей управления на любой стороне своей рабочей области. Аргумент данной функции должен совпадать с аргументом функции `CControlBar::EnableDocking`. После этого для каждой панели инструментов вызываются функции `DockControlBar`, осуществляющие фиксацию указанной ей в качестве аргумента панели инструментов. Последним оператором функции `OnCreate` является вызов функции `CFrameWnd::LoadBarState`, позволяющей восстановить установки для каждой панели управления, принадлежащей главному окну приложения. В качестве аргумента данной функции используется ключ, заданный функцией `CWinApp::SetRegistryKey`, вызываемой в функции `CToolAppApp::InitInstance`.

Для того чтобы при вызове функции `CFrameWnd::LoadBarState` панели инструментов принимали свое последнее состояние, в класс `CMainFrame` введена функция `OnClose`, являющаяся обработчиком сообщения `WM_CLOSE`, посылаемого окну или приложению для того, чтобы известить его о необходимости завершить свою работу. В данную функцию добавлен вызов функции `CFrameWnd::SaveBarState`, позволяющей сохранить информацию об установках каждой панели управления, принадлежащей данному окну.

Функция `OnUpdateMyToolbar` является функцией обработки сообщения `UPDATE_COMMAND_UI`, посылаемого приложением при открытии соответствующего меню для определения состояния его команд. Данная функция прежде всего проверяет наличие в приложении объекта класса, которым управляет данная команда меню. При наличии такого объекта класса вызывается функция `CCmdUI::SetCheck`, устанавливающая состояние команды меню или кнопки панели инструментов. В качестве аргумента данной функции используется логическое выражение, проверяющее текущее состояние панели инструмен-

тов. Для этого вызывается функция CWnd::GetStyle, возвращающая стиль окна, а в полученной величине проверяется значение бита WS_VISIBLE. Таким образом, если панель инструментов отображается на экране, то у данной команды меню устанавливается флажок. Если в данном приложении отсутствует объект класса, которым управляет данная команда меню, то функция OnUpdateMyToolbar вызывает функцию CCmdUI::ContinueRouting, передающую процедуре обработки сообщений, что обработка данного сообщения не завершена в данной функции и требуется найти обработчик сообщения, который завершит процедуру его обработки.

Прототип функции OnMyToolbarCheck не создается мастером ClassWizard, поэтому пользователю необходимо написать ее целиком. Данная функция является функцией обработки сообщения об установке и снятии флажка у команды меню, идентификатор ресурса которой передается ей в качестве аргумента и нигде в данной функции не используется. Как и в функции OnUpdateMyToolbar, здесь проверяется наличие в приложении объекта класса, управление которым осуществляется данной командой меню. Если такой объект класса присутствует, вызывается функция CFrameWnd::ShowControlBar, позволяющая выводить панель управления на экран и убирать ее с экрана. Первым ее аргументом является указатель на объект класса панели управления, которую необходимо вывести на экран или убирать с экрана. Второй аргумент определяет действие, которое необходимо произвести с данной панелью управления, а третий аргумент определяет задержку при выводе панели управления на экран. В случае благоприятного исхода функция OnMyToolbarCheck возвращает значение TRUE, в противном случае возвращается значение FALSE.

Функция CToolAppView::OnSquare является функцией обработки сообщения, поступающего от кнопки панели инструментов и от команды меню. Кнопка панели инструментов и команда меню дублируют друг друга, поскольку они имеют один и тот же идентификатор ресурса, поэтому все изменения с данными элементами управления происходят синхронно. Сама функция OnSquare является демонстрационной и не представляет из себя ничего особенного. В ней создаются объекты классов, необходимые для рисования прямоугольника, выбираются в контекст устройства, рисуется прямоугольник, а после этого контекст устройства возвращается в исходное состояние. После этого переменной bDrawn, которой при инициализации объекта данного класса было присвоено значение FALSE, присваивается значение TRUE.

Функция OnUpdateSquare определяет состояние соответствующей кнопки панели инструментов и команды меню. Вызываемая в ней функция CCmdUI::Enable в зависимости от значения переменной bDrawn делает доступными или недоступными данные элементы управления. Логика работы данной функции состоит в том, что при фиксированных координатах и размерах в окне можно нарисовать только один квадрат. Поэтому после его

рисования кнопка панели инструментов и команда меню, выполняющие данную операцию, становятся недоступными.

Для возобновления доступа к данным элементам управления в функции `OnLButtonDown`, являющейся функцией обработки сообщения о нажатии на левую кнопку мыши, переменной `bDrawn` присваивается значение `FALSE` и вызывается функция `Invalidate`, производящая очистку рабочей области окна документа, поскольку в функцию `OnDraw` не были внесены никакие изменения.

Работа со строкой состояния

Строка состояния в приложениях, созданных мастером AppWizard, является настолько стандартным элементом, что для нее даже не создается никакого ресурса, позволяющего использовать для работы с ней мастер ClassWizard.

Строка состояния разрабатывалась специалистами фирмы Microsoft для простейшего текстового редактора, поскольку только в нем имеет смысл отображать состояние функциональных клавищ, определяющих режим вывода текста на экран. В подавляющем большинстве других применений информация, выводимая в панелях строки состояния, не несет никакой полезной информации. Это обстоятельство, при отсутствии в основной среде разработки приложений, поставляемой фирмой Microsoft для своей операционной системы, средств для удобного редактирования строки состояния, еще раз подтверждает, что Microsoft считает текстовые редакторы единственно возможной формой приложения, работающего в среде Windows.

Однако широкое распространение этой "операционной системы для домохозяек", как изначально позиционировал ее на рынке Microsoft, в несвойственные ей области применения привело к необходимости редактирования строки состояния практически в любом серьезном приложении, создаваемом с использованием Visual C++.

В большинстве случаев это редактирование сводится к уничтожению стандартных панелей строки состояния и созданию на их месте новых панелей. Поскольку функции обработки сообщений от стандартных панелей строки состояния находятся в библиотеке MFC и не могут быть отредактированы пользователем, то вся операция уничтожения стандартных панелей сводится к уничтожению их идентификаторов в массиве `indicators`.

Чтобы измерить информацию, выводимую в строке состояния приложения ToolApp:

1. Выберите команду **File**, **Open Workspace...** и в открывшемся диалоговом окне **Open Workspace** раскройте папку \ToolApp, выделите значок **ToolApp.dsw** и нажмите кнопку **Открыть**.
2. Раскройте вкладку **ResourceView** окна **Workspace**.

3. Выберите команду **View**, **Resource Symbols** (Вид, Символы ресурсов). Появится диалоговое окно **Resource Symbols**, изображенное на рис. 9.13.

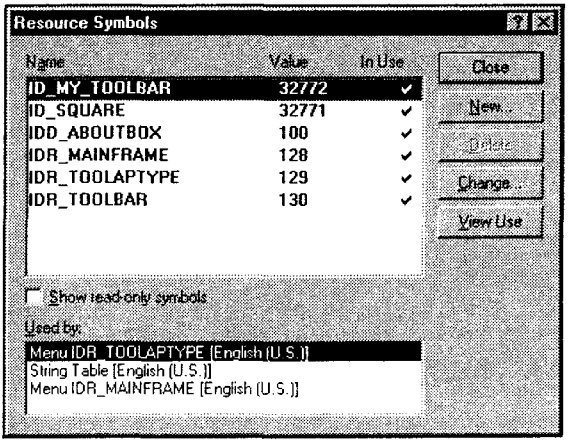


Рис. 9.13. Диалоговое окно **Resource Symbols**

4. Нажмите кнопку **New...**, появится диалоговое окно **New Symbol**, изображенное на рис. 9.14.

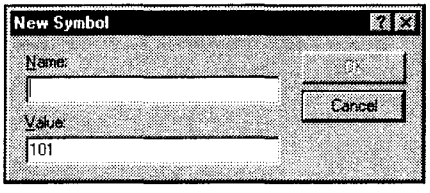


Рис. 9.14. Диалоговое окно **New Symbol**

5. В текстовое поле **Name:** введите идентификатор ресурса `ID_INDICATOR_X` и нажмите кнопку **OK**. В списке появится новый идентификатор ресурса.
6. Повторите эту операцию для идентификатора ресурса `ID_INDICATOR_Y` и нажмите кнопку **Close** в диалоговом окне **Resource Symbols**.
7. Раскройте папку \ToolApp Resources, а в ней папку \String Table. Дважды щелкните левой кнопкой мыши на значке **String Table [English (U.S.)]**. Откроется окно редактирования строковых ресурсов.
8. Щелкните правой кнопкой мыши на любом идентификаторе ресурса в данном окне.
9. В появившемся контекстном меню выберите команду **New String** (новая строка). Появится диалоговое окно **String Properties**.
10. Введите в текстовое поле раскрывающегося списка **ID:** идентификатор ресурса `ID_INDICATOR_X`, а в текстовое поле **Caption:** введите символ "X".
11. Закройте диалоговое окно **String Properties**.

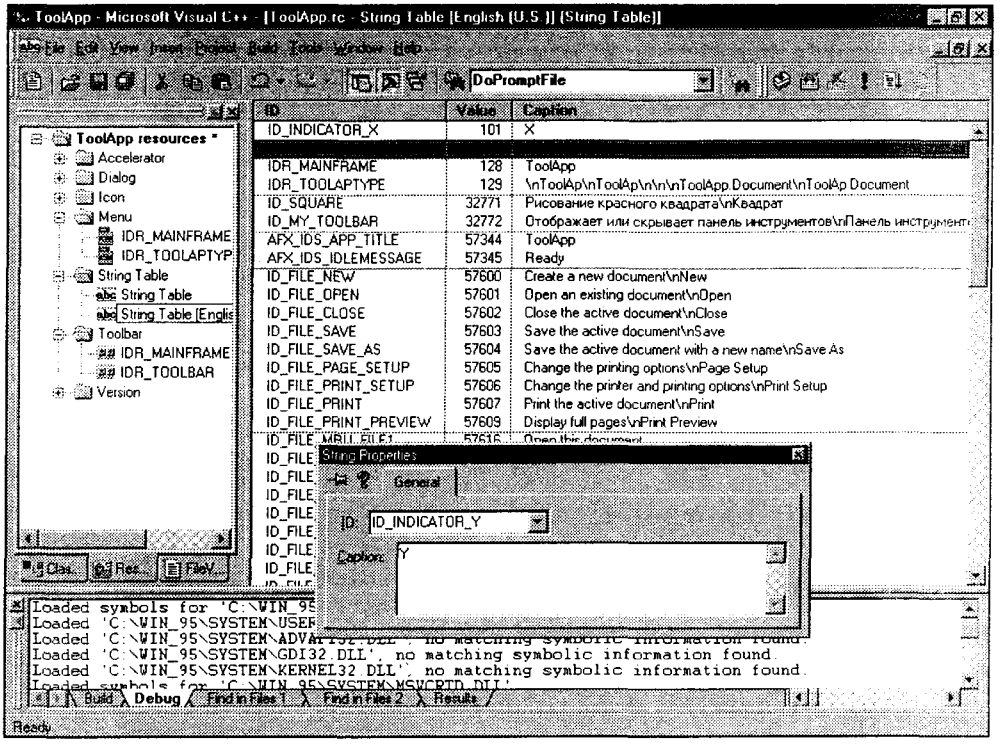


Рис. 9.15. Окно редактирования строковых ресурсов

12. Повторите пункты 8—11 для создания строкового ресурса с идентификатором `ID_INDICATOR_Y` и текстом "Y". Окно редактирования строковых ресурсов примет вид, изображенный на рис. 9.15.

13. Откройте окно редактирования файла MainFrm.h и переместите строку `CStatusBar m_wndStatusBar;` на две строки вверх, то есть из раздела `protected` в раздел `public`.

14. Откройте окно редактирования файла MainFrm.cpp и измените содержимое массива `indicators` в соответствии с приведенным ниже текстом:

```
static UINT indicators[] =
{
  ID_SEPARATOR,           // status line indicator
  ID_INDICATOR_X,
  ID_INDICATOR_Y,
};
```

15. В текст функции `OnCreate` между вызовом функции `LoadBarState` и оператором `return 0;` вставьте следующие строки:

```
m_wndStatusBar.SetPaneInfo(m_wndStatusBar.CommandToIndex(ID_INDICATOR
                                    _X), ID_INDICATOR_X, SBPS_NORMAL, 64);
m_wndStatusBar.SetPaneInfo(m_wndStatusBar.CommandToIndex(ID_INDICATOR
                                    _Y), ID_INDICATOR_Y, SBPS_NORMAL, 64);
```

16. Откройте окно редактирования файла ToolAppView.cpp и после строки `#include "ToolAppView.h"` вставьте строку

```
#include "MainFrm.h"
```

17. Вызовите мастер ClassWizard. Откроется диалоговое окно **MFC ClassWizard**.

18. В окне списка **Object IDs:** выделите имя класса `CToolAppView`, в окне списка **Messages:** выделите сообщение `WM_MOUSEMOVE` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится имя новой функции `OnMouseMove`.

19. Нажмите кнопку **Edit Code**, диалоговое окно **MFC ClassWizard** закроется, а текстовый курсор переместится в текст заготовки новой функции.

20. Измените текст заготовки функции `OnMouseMove` в соответствии с приведенным ниже текстом:

```
void CToolAppView::OnMouseMove(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  CMainFrame*  lpFrame  = (CMainFrame*) GetParentOwner();

  char  szTemp[16];

  sprintf( szTemp,"%d", point.x);

  lpFrame-> m_wndStatusBar.SetPaneText( lpFrame->
          m_wndStatusBar.CommandToIndex( ID_INDICATOR_X), szTemp);

  sprintf( szTemp,"%d", point.y);

  lpFrame-> m_wndStatusBar.SetPaneText( lpFrame->
          m_wndStatusBar.CommandToIndex( ID_INDICATOR_Y), szTemp);

  CView::OnMouseMove(nFlags, point);
}
```

21. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится главное окно приложения, в котором будет открыто окно документа.

22. Если указатель мыши не расположен в окне документа, в панелях строки состояния будут отображаться символы X и Y, как это показано на рис. 9.16.

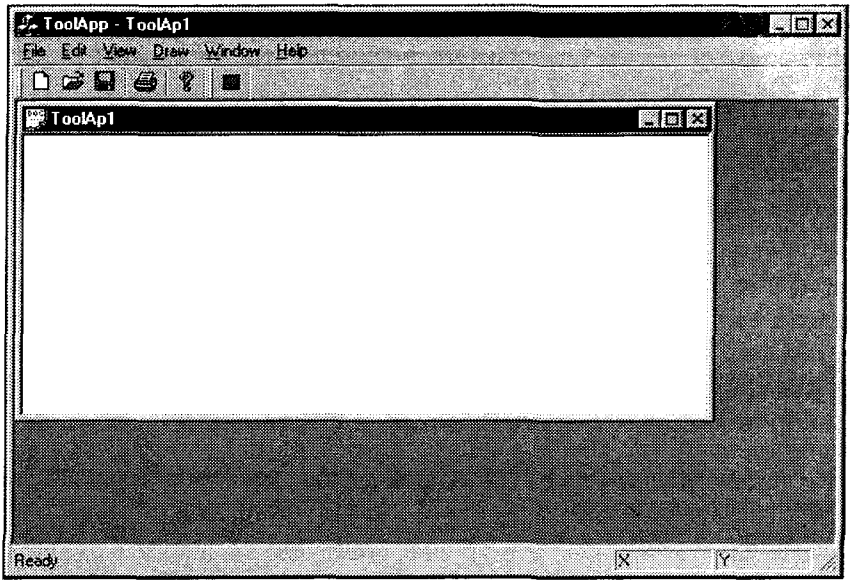


Рис. 9.16. Строка состояния открытого приложения

23. Переместите курсор мыши в окно документа. В панелях строки состояния будут постоянно отображаться текущие координаты указателя мыши, как показано на рис. 9.17.

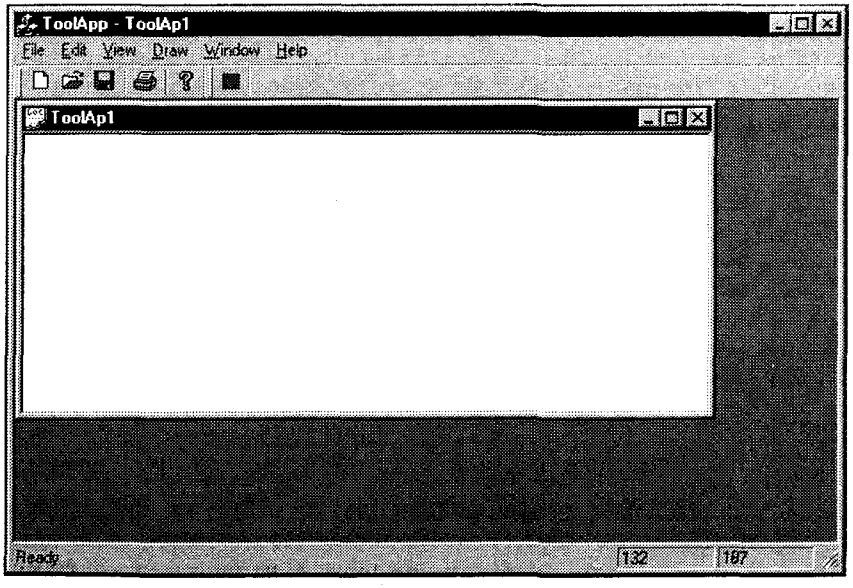


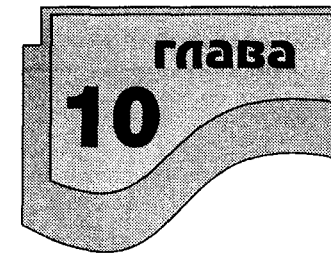
Рис. 9.17. Строка состояния, отображающая координаты курсора мыши

24. При перемещении курсора мыши за пределы окна документа в панелях строки состояния будут отображаться координаты точки выхода курсора мыши из окна документа.

В функции `CMainFrame::OnCreate` остались не рассмотренные нами операторы, создающие строку состояния в приложении, кроме того в нее были введены дополнительные операторы. Функция `CStatusBar::Create` создает строку состояния (дочернее окно) и связывает его с объектом класса `CStatusBar`. Функция `CStatusBar::SetIndicators` сопоставляет идентификатору каждого индикатора значение, указанное для него в массиве, на который указывает ее первый аргумент, загружает строковый ресурс, определенный для каждого идентификатора, и выводит его в соответствующей панели. Для вычисления размерности массива `indicators` определяется его размер в байтах, который затем делится на размер элемента массива. В случае возникновения ошибки при выполнении данных функций выдается сообщение об ошибке и функция `OnCreate` прекращает свою работу, возвращая значение -1.

Функция `CStatusBar::SetPaneInfo` устанавливает для панели индикатора новые идентификатор, стиль и ширину. В данном случае она используется для установки новой ширины для каждой из панелей. Использование фиксированной ширины панели в данном случае вызвано тем, что информация в ней будет быстро изменяться. В случае постоянной настройки ширины панели на ширину выводимого в ней текста появится неприятное для глаза мигание панели даже в том случае, когда ширина выводимого в ней текста не будет меняться. Если не предполагается часто менять выводимую в панелях индикаторов информацию, а сама выводимая информация существенно различается по ширине текста, для задания ширины панели можно использовать функцию `CDC::GetTextExtent`, возвращающую объект класса `CSize`, который содержит длину текстовой строки. Переменная `cx` данного объекта содержит требуемую ширину строки.

Функция `CToolAppView::OnMouseMove` является функцией обработки сообщения о перемещении мыши в рабочей области окна. В первом ее операторе вызывается функция `CWnd::GetParentOwner`, возвращающая указатель на ближайшее в иерархии родительское окно, которое само не является дочерним окном. В данном случае это главное окно приложения. После этого горизонтальная координата позиции курсора преобразуется в текстовую форму и используется в качестве аргумента функции `CStatusBar::SetPaneText`, обновляющей текст в панели индикаторов. В качестве первого аргумента данной функции необходимо указать индекс панели, в которую необходимо вывести текст. Для получения этой информации используется функция `CStatusBar::CommandToIndex`, позволяющая получить индекс индикатора по заданному идентификатору. После этого подобная операция производится с вертикальной координатой позиции курсора. И в завершение функции вызывается соответствующий метод базового класса.



Печать документов и организация прокрутки в окне

Печать документа в операционной системе Windows до появления библиотеки MFC представляла собой достаточно сложную задачу. Это было связано с необходимостью обеспечения аппаратной независимости создаваемых программ при достаточно скудных программных средствах. Теперь для того чтобы обеспечить возможность печати документа и его предварительного просмотра, достаточно не сбрасывать флажок **Printing and print preview** в четвертой панели мастера MFC AppWizard, приведенной на рис. 10.1. Однако не все так просто. Дело в том, что по умолчанию печать документа производится "точка в точку", то есть элементу изображения на экране соответствует точка в распечатке. Поскольку разрешение большинства принтеров намного выше, чем разрешение большинства мониторов, то на печать выводятся слишком мелкие изображения.

Кроме того, многие документы состоят из нескольких страниц, и, поскольку не существует единого принципа разбиения любого документа на страницы, то эта задача полностью ложится на плечи пользователя.

Другим вопросом, тесно связанным с разбиением документа на страницы, является организация его прокрутки в окне, поскольку разбиение на страницы можно условно представить себе как последовательную прокрутку документа в окне.

Организация прокрутки в окне

Прежде чем приступить к печати документа, его нужно создать. Многостраничный документ не помещается полностью на экране, что обуславливает необходимость *прокрутки документа в окне* для его просмотра.

Текст демонстрационного приложения, реализующего печать документа и его прокрутку в окне, расположен в каталоге \PrintApp дискеты, прилагаемой к данной книге. Чтобы самостоятельно создать данное приложение:

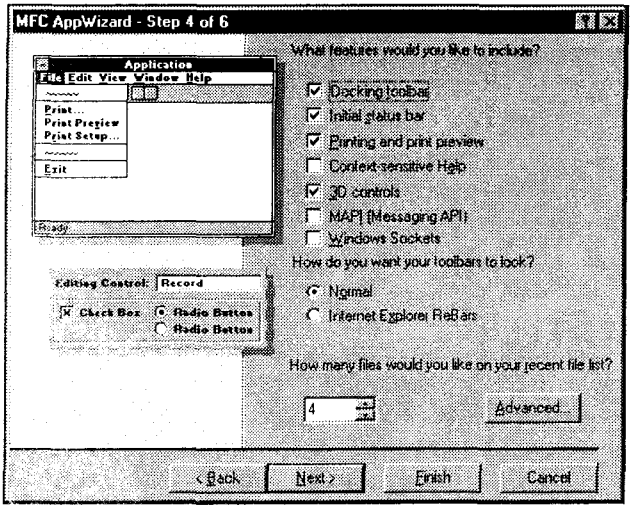


Рис. 10.1. Диалоговое окно **MFC AppWizard — Step 4 of 6**

1. Создайте многооконное приложение с именем PrintApp по методике, описанной в главе 1. Оставьте все установки по умолчанию без изменения вплоть до последнего, шестого шага.
2. На шестом шаге выберите в списке **Base class:** базовый класс `CScrollView`. Нажмите кнопку **Finish** и завершите процедуру создания приложения.
3. Раскройте вкладку **ClassView** окна **Workspace**. Раскройте папку \PrintApp classes и щелкните правой кнопкой мыши на имени класса `CPrintAppDoc`.
4. В раскрывшемся контекстном меню выберите команду **Add Member Variable...** (добавить переменную-член класса). Появится диалоговое окно **Add Member Variable**, изображенное на рис. 10.2.

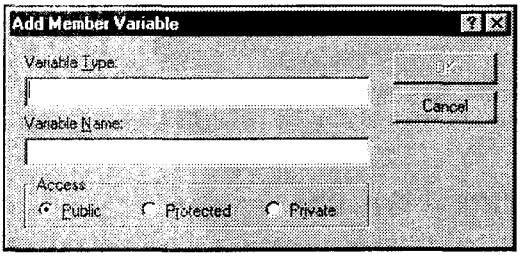


Рис. 10.2. Диалоговое окно **Add Member Variable**

5. В текстовое поле **Variable Type:** (тип переменной) введите тип переменной `int`, а в текстовое поле **Variable Name:** (идентификатор переменной) введите идентификатор переменной `m_nRects`.
6. Оставьте без изменения положение **Public** переключателя прав доступа **Access** и нажмите кнопку **OK**.

7. Раскройте вкладку **FileView** окна **Workspace**. Раскройте папку \PrintApp files, а в ней папку \Source Files. Дважды щелкните левой кнопкой мыши на имени файла PrintAppDoc.cpp. Раскроется окно редактирования соответствующего файла.
8. В текст функции `OnNewDocument` перед оператором `return TRUE;` вставьте строку

```
m_nRects = 0;
```

9. Измените текст функции `Serialize` в соответствии с приведенным ниже текстом:

```
void CPrintAppDoc::Serialize(CArchive& ar)
{
  if (ar.IsStoring())
  {
    // TODO: add storing code here
    ar << m_nRects;
  }
  else
  {
    // TODO: add loading code here
    ar >> m_nRects;
  }
}
```

10. Откройте окно редактирования файла PrintAppView.cpp и вызовите мастер ClassWizard. Появится диалоговое окно **MFC ClassWizard**.
11. В окне списка **Object IDs:** выделите имя класса `CPrintAppView`, в окне списка **Messages:** выделите идентификатор сообщения `WM_LBUTTONDBLCLK` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится новое имя функции обработки сообщения `OnLButtonDblClk`.
12. Нажмите кнопку **Edit Code** и закройте диалоговое окно **MFC ClassWizard**. Текстовый курсор переместится в текст функции `OnLButtonDblClk`.
13. Измените текст функции `OnLButtonDblClk` в соответствии с приведенным ниже текстом:

```
void CPrintAppView::OnLButtonDblClk(UINT nFlags, CPoint point)
{
  // TODO: Add your message handler code here and/or call default

  CPrintAppDoc*  lpDoc = GetDocument();
  ASSERT_VALID( lpDoc);

  lpDoc-> m_nRects++;

  Invalidate();
```

```
    CScrollView::OnLButtonDblClk(nFlags, point);
}
```

14. Измените текст функции `OnDraw` в соответствии с приведенным ниже текстом:

```
void CPrintAppView::OnDraw(CDC* pDC)
{
  CPrintAppDoc* pDoc = GetDocument();
  ASSERT_VALID(pDoc);
  // TODO: add draw code for native data here

  CPen      pen( PS_SOLID, 0, RGB(0,0,0));
  CPen*      oldPen = pDC-> SelectObject( &pen);

  for( int i=0; i < pDoc-> m_nRects; i++)
    pDC-> Rectangle( 50, 50+500*i, 250, 250+500*i);

  pDC-> SelectObject( oldPen);

  CSize  docSize( 300, 300 + 500*pDoc-> m_nRects);

  CRect  rect;
  GetClientRect( &rect);
  CSize  pageSize( rect.right, rect.bottom);

  CSize  lineSize( 0, 500);

  SetScrollSizes( MM_TEXT, docSize, pageSize, lineSize);
}
```

15. Нажмите клавишу <F5> и запустите приложение на исполнение.
16. Дважды щелкните левой кнопкой мыши в окне документа. В нем появится квадрат, вдоль правой границы которого будет расположена полоса прокрутки, как показано на рис. 10.3.
17. Повторите эту операцию. Размер бегунка полосы прокрутки уменьшится.
18. Щелкните левой кнопкой мыши по стрелке в полосе прокрутки. В рабочей области окна ничего не изменится, но бегунок полосы прокрутки переместится вниз, как показано на рис. 10.4. Сохранение изображения в окне связано с тем, что размер строки выбран равным шагу расстановки квадратов в окне, что приводит к возникновению "стробоскопического эффекта".
19. Щелкните левой кнопкой мыши в полосе прокрутки выше бегунка. Изображение в окне переместится на один экран вверх и примет вид, изображенный на рис. 10.5. В данном случае не наблюдается "стробоскопического эффекта", поскольку размер экрана не связан с шагом расстановки квадратов в окне.

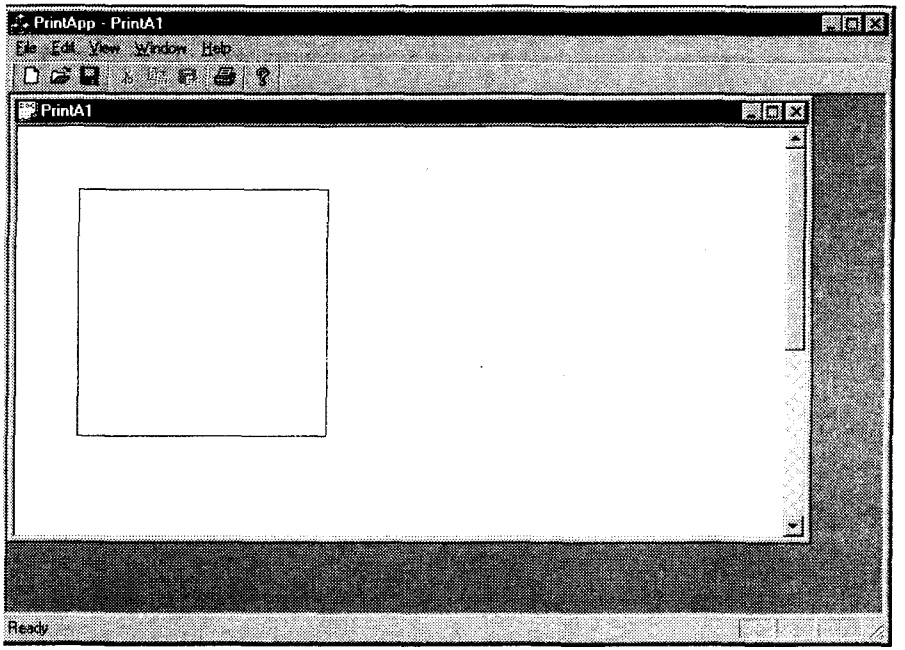


Рис. 10.3. Полоса прокрутки в окне документа

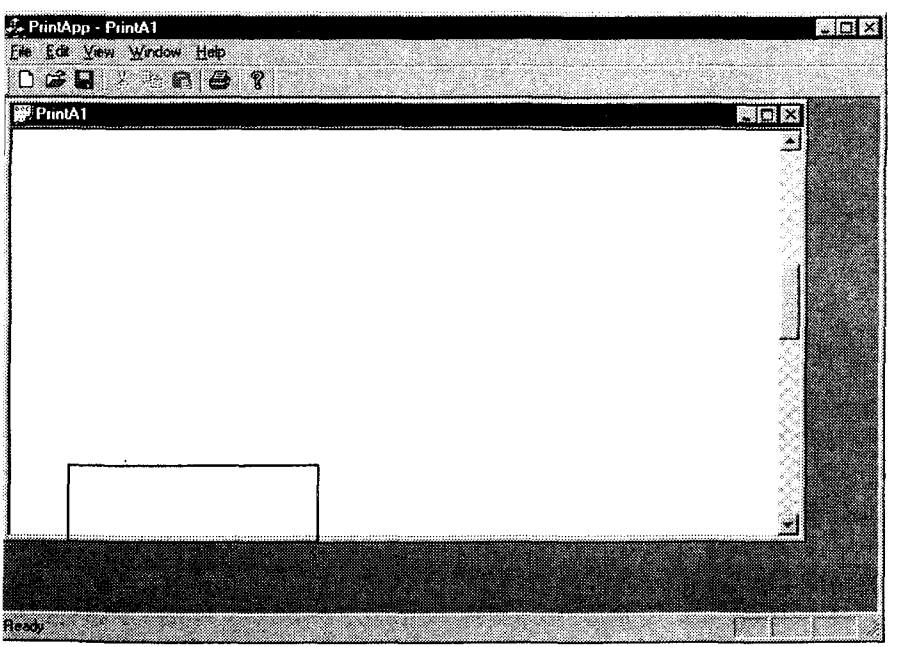


Рис. 10.4. Прокрутка документа вниз

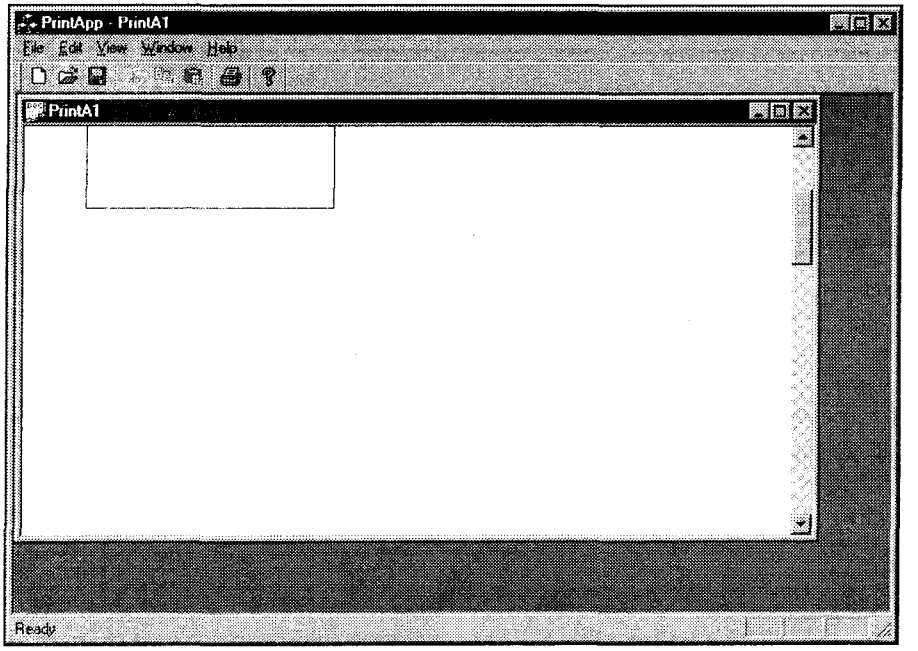


Рис. 10.5. Прокрутка документа вверх

20. Нажмите кнопку **Save** в панели инструментов. Появится диалоговое окно **Save As**.
21. Оставьте предлагаемое имя файла без изменений и нажмите кнопку **Сохранить**.
22. Закройте окно документа и нажмите кнопку **Open** в панели инструментов. Появится диалоговое окно **Open**.
23. Выделите в списке файлов имя файла сохраненного документа и нажмите кнопку **Открыть**. Откроется окно документа, имеющее вертикальную полосу прокрутки, в котором выведены квадраты, а бегунок полосы прокрутки стоит в крайнем верхнем положении.

Как видно из приведенного выше примера, практически всю работу по созданию в окне полосы прокрутки приложение берет на себя. От пользователя лишь требуется указать параметры прокрутки в окне.

Изменения, внесенные в функцию `CPrintAppDoc::Serialize`, позволяют сохранять в архиве число квадратов, выведенных в окне и не затрагивают прокрутки изображения.

Функция `CPrintAppView::OnLButtonDblClk` используется для увеличения числа квадратов в окне и тоже не затрагивает прокрутки изображения. Для выполнения этой операции прежде всего с помощью вызова функции

CView::GetDocument получается указатель на объект класса документа, связанный с данным объектом класса представления, и проверяется на допустимость. В случае отрицательного исхода проверки в отладочной версии программы выполнение данной функции будет прекращено. При помощи полученного указателя обеспечивается доступ к переменной m_nRects, хранящей число квадратов в окне. Значение этой переменной увеличивается на единицу и вызывается функция CWnd::Invalidate, приводящая к перерисовке рабочей области окна.

Перерисовка рабочей области окна выполняется функцией OnDraw. В этой функции, как и в описанной выше функции OnLButtonDblClk, получается и проверяется указатель на объект класса документа, связанный с данным объектом класса представления. После этого создается объект класса CPen и выбирается текущий контекст устройства, причем указатель на старый объект класса запоминается в переменной oldPen. Затем выполняется рисование квадратов с использованием функции CDC::Rectangle. После завершения операции рисования в контекст устройства выбирается старый объект класса CPen.

После этого вычисляются параметры прокрутки изображения в окне. Эти параметры записываются в объекты класса CSize.

В переменной docSize хранится размер всего отображаемого документа. Он определяется границами выводимых квадратов и окружающей их рамкой, имеющей ширину 50 логических единиц.

В переменной pageSize хранится размер *страницы*. Под страницей понимается шаг перемещения изображения по горизонтали или по вертикали при щелчке левой кнопкой мыши в полосе прокрутки за пределами бегунка. В данном случае он равен размеру рабочей области окна, получаемой с помощью вызова функции CWnd::GetClientRect.

В переменной lineSize хранится размер *строки*. Под строкой понимается шаг перемещения изображения по горизонтали или по вертикали при щелчке левой кнопкой мыши по стрелкам полосы прокрутки.

Эти переменные передаются в качестве аргументов в функцию CScrollView::SetScrollSizes. Кроме указанных аргументов в данной функции задается режим отображения. В качестве режима отображения в данном случае выбран MM_TEXT, при котором логическая единица соответствует элементу изображения, а ось Y направлена сверху вниз.

Режимы отображения

Практически все функции GDI требуют определения координат или размеров каких-либо графических объектов. Данные координаты задаются в так называемых *логических единицах*. Windows преобразует эти логические коор-

динаты и размеры в *физические координаты* (элементы изображения) устройства используемого для вывода информации. При этом преобразовании используются первые пять атрибутов контекста устройства, относящиеся к группе атрибутов режимов отображения: начальные координаты, направление осей координат и масштаб координат для каждой оси.

Характеристики различных режимов отображения приведены в табл. 10.1.

Таблица 10.1. *Характеристики режимов отображения*

Режим отображения	Логические единицы	Направление по оси X	Направление по оси Y
MM_TEXT	1 элемент изображения	Возрастает слева направо	Возрастает сверху вниз
MM_HIMETRIC	0.01 мм	Возрастает слева направо	Возрастает снизу вверх
MM_TWIPS	1/1440 дюйма	Возрастает слева направо	Возрастает снизу вверх
MM_HIENGLISH	0.001 дюйма	Возрастает слева направо	Возрастает снизу вверх
MM_LOMETRIC	0.1 мм	Возрастает слева направо	Возрастает снизу вверх
MM_LOENGLISH	0.01 дюйма	Возрастает слева направо	Возрастает снизу вверх
MM_ISOTROPIC	Определяется пользователем (X=Y)	Определяется пользователем	Определяется пользователем
MM_ANISOTROPIC	Определяется пользователем (X!=Y)	Определяется пользователем	Определяется пользователем

Windows использует три системы координат устройств отображения. Во всех этих системах координаты измеряются в элементах изображения, ось X направлена слева направо, а ось Y — сверху вниз.

При использовании всего экрана дисплея работа ведется в экранных координатах. При этом начало координат находится в левом верхнем углу экрана. *Экранные координаты* используются в сообщении `WM_MOVE` и в функциях `CreateWindow`, `MoveWindow`, `GetCursorPos`, `SetCursorPos`, `GetWindowRect` и `WindowFromPoint`. Если для получения контекста устройства всего экрана дисплея использовалась глобальная функция `CreateDC`, то при использовании функций GDI логические координаты преобразуются в экранные.

В оконной системе координат начало координат находится в левом верхнем углу рамки окна. Данная система координат описывает все окно, включая

меню, полосы прокрутки и заголовок. Для использования оконной системы координат следует получать контекст устройства при помощи функции `GetWindowDC`. При этом функции GDI, использующие данный контекст, преобразуют логические координаты в оконные.

Пользовательская система координат связана с рабочей областью окна, а начало координат расположено в левом верхнем углу рабочей области окна. При получении контекста устройства с помощью вызова функций `GetDC` и `BeginPaint` функции GDI, использующие данный контекст, преобразуют логические координаты в координаты рабочей области окна. Для преобразования экранных координат в пользовательские используется функция `CWnd::ScreenToClient`.

Поскольку пользователь в основном выводит информацию в рабочую область окна, то ему практически во всех случаях приходится работать в пользовательской системе координат. При выводе информации на печать эта система координат претерпевает определенные изменения, рассмотренные в следующем разделе.

Распечатка и предварительный просмотр

Как уже говорилось выше, возможности печати документов автоматически закладываются в приложение при его создании. Однако существуют задачи, связанные с печатью документа, которые должны быть решены самим пользователем.

Работа с окном предварительного просмотра печати

Одним из полезных свойств, предоставляемых библиотекой MFC при работе с распечатываемым документом, является предварительный просмотр печати. Эта возможность реализуется в виде специального окна, позволяющего оценить внешний вид каждой страницы распечатываемого документа. Чтобы вывести на экран окно предварительного просмотра в приложении PrintApp:

1. Откройте приложение PrintApp.
2. Нажмите клавишу <F5> и запустите его на исполнение.
3. Дважды щелкните левой кнопкой мыши в рабочей области окна документа для вывода в него квадрата.
4. Выполните команду меню **File**, **Print Preview** (Печать, Предварительный просмотр). Появится окно предварительного просмотра, изображенное на рис. 10.6.

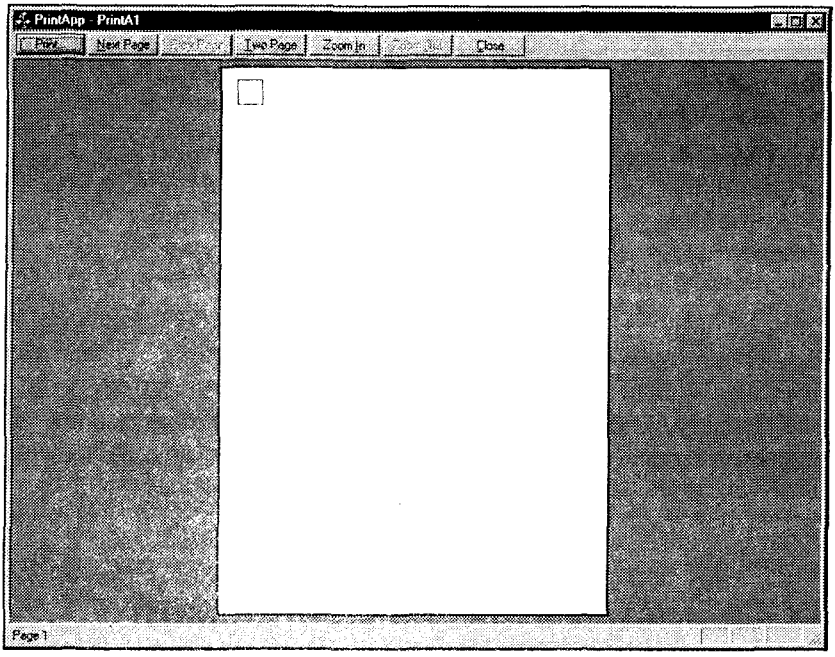


Рис. 10.6. Исходное окно предварительного просмотра

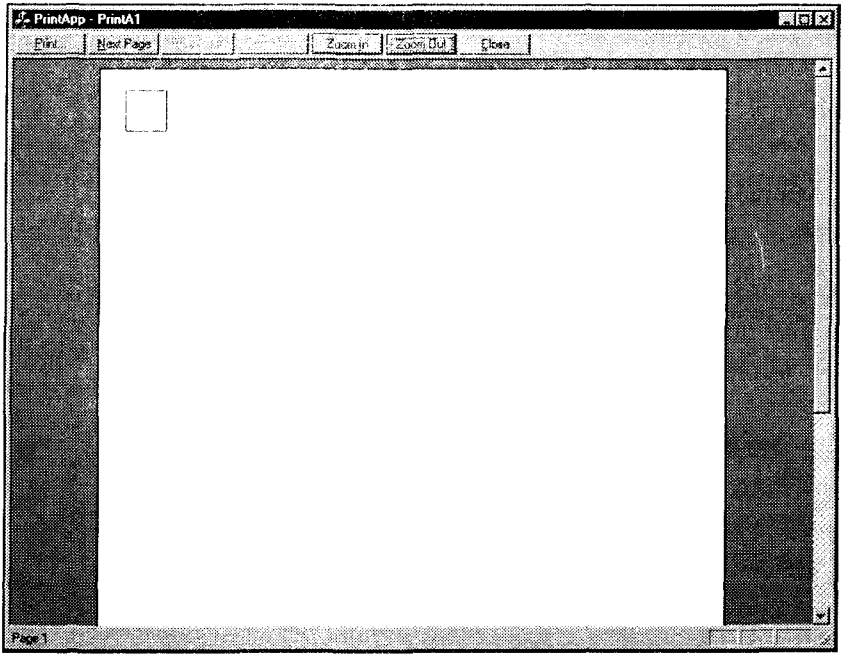


Рис. 10.7. Увеличенный масштаб окна предварительного просмотра

5. Переместите курсор мыши в область печати. Он примет вид увеличительного стекла.
6. Нажмите левую кнопку мыши. Изображение листа увеличится и в нем появится вертикальная полоса прокрутки, как показано на рис. 10.7.
7. Повторите эту операцию. Изображение листа еще увеличится и в нем появится горизонтальная полоса прокрутки, как показано на рис. 10.8.
8. Нажмите кнопку **Zoom Out** (сжатие). Размеры листа уменьшатся и окно примет вид, изображенный на рис. 10.3.
9. Еще раз нажмите кнопку **Zoom Out**, возвращая исходный масштаб изображения, и нажмите кнопку **Two Pages** (две страницы).
10. На экране появятся две страницы (рис. 10.9).
11. Нажмите кнопку **Close** и закройте окно предварительного просмотра.
12. Закройте приложение и откройте окно редактирования файла PrintAppView.cpp.
13. Измените текст функции `OnDraw` в соответствии с приведенным ниже текстом:

```
void CPrintAppView::OnDraw(CDC* pDC)
{
  CPrintAppDoc* pDoc = GetDocument();
  ASSERT_VALID(pDoc);
  // TODO: add draw code for native data here

  CPen      pen( PS_SOLID, 0, RGB(0,0,0));
  CPen*      oldPen = pDC-> SelectObject( &pen);

  for( int i=0; i < pDoc-> m_nRects; i++)
    pDC-> Rectangle( 50, - 50-500*i, 250, -250-500*i);

  pDC-> SelectObject( oldPen);

  CSize  docSize( 300, 100 + 500*pDoc-> m_nRects);

  CRect  rect;
  GetClientRect( &rect);
  CSize  pageSize( rect.right, rect.bottom);

  CSize  lineSize( 0, 500);

  SetScrollSizes( MM_LOENGLISH, docSize, pageSize, lineSize);
}
```

14. Повторите операции, описанные в пунктах 2—4. Появится окно предварительного просмотра, показанное на рис. 10.10.

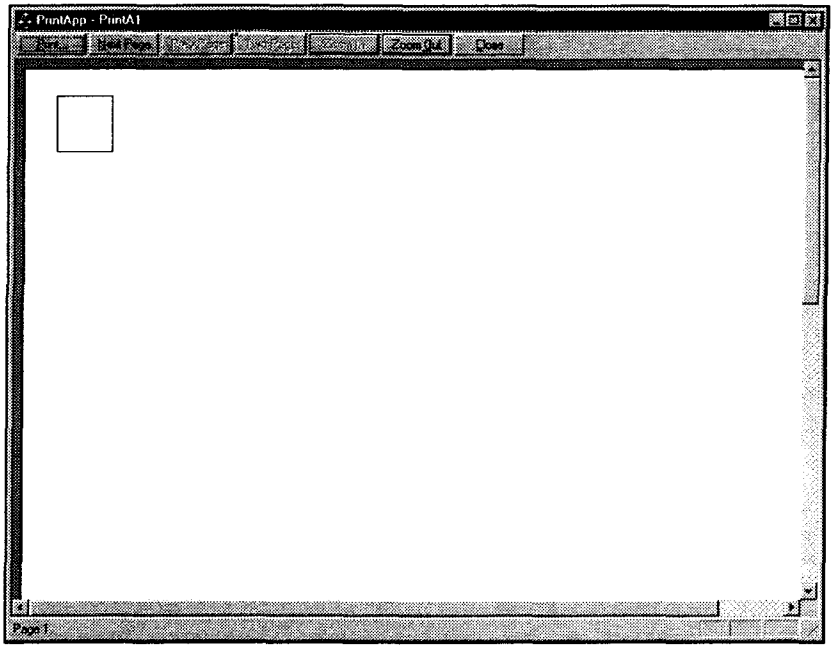


Рис. 10.8. Максимальный масштаб окна предварительного просмотра

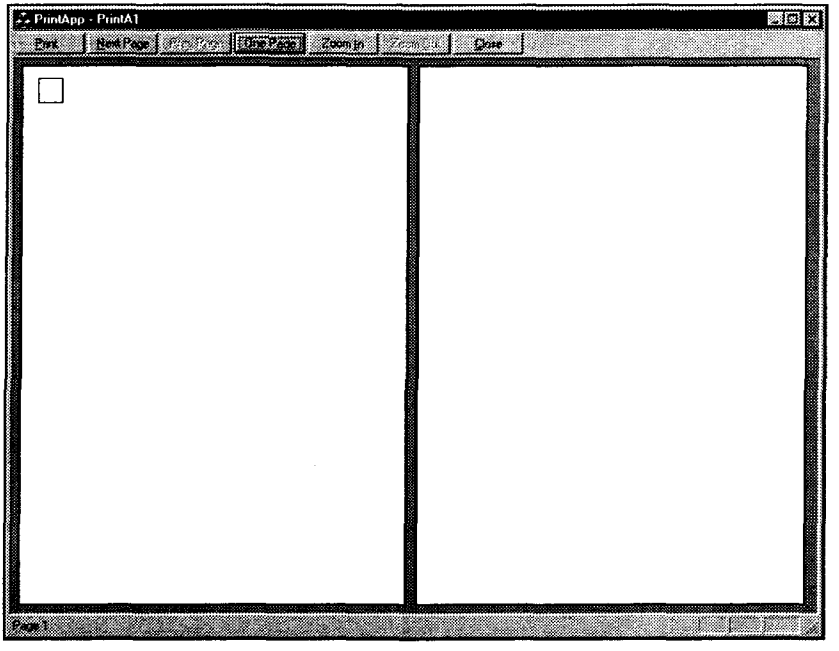


Рис. 10.9. Окно предварительного просмотра в режиме одновременного просмотра двух страниц

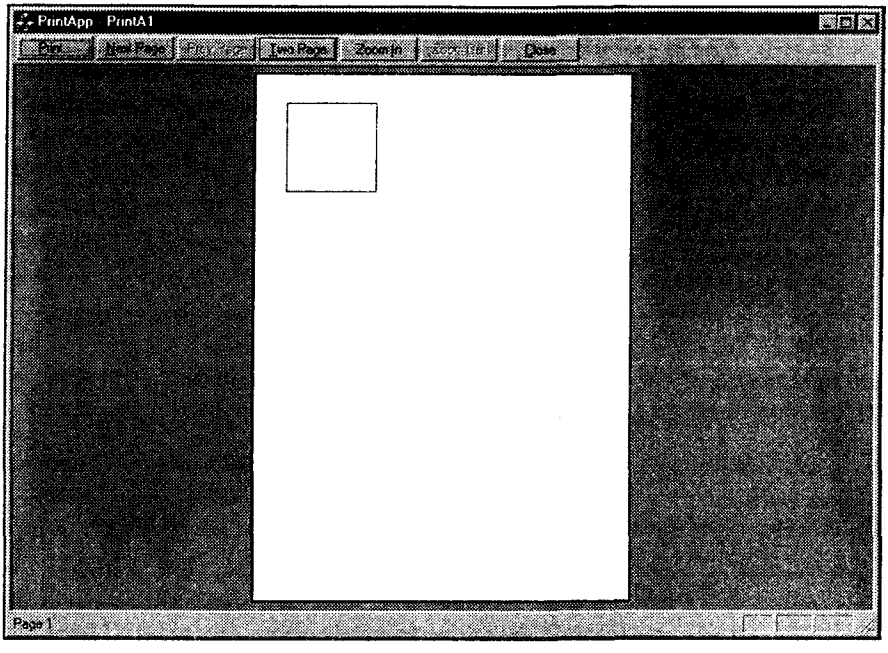


Рис. 10.10. Окно предварительного просмотра печати

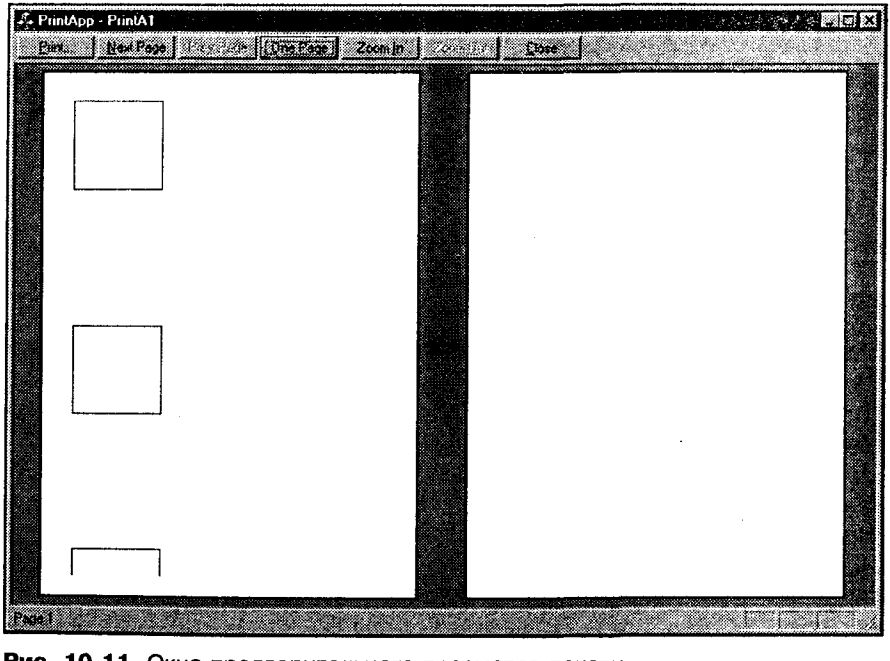


Рис. 10.11. Окно предварительного просмотра печати

15. Закройте окно предварительного просмотра печати и еще два раза дважды щелкните левой кнопкой мыши в рабочей области окна документа.
16. Откройте окно предварительного просмотра печати и нажмите кнопку **Two Pages**. Окно предварительного просмотра примет вид, изображенный на рис. 10.11.
17. Закройте окно предварительного просмотра и закройте приложение.

Как видно из сравнения рис. 10.6 и 10.10, режим отображения MM_LOENGLISH позволяет существенно увеличить размеры печатаемого изображения по сравнению с режимом MM_TEXT. Кроме того, существенным преимуществом режима MM_LOENGLISH является его независимость от разрешения принтера, на котором будет распечатываться данное изображение.

Однако гораздо больший интерес представляет рис. 10.11, из которого следует, что приложение PrintApp в его настоящем виде не позволяет распечатывать многостраничные документы, т. е. переносить остаток изображения, не распечатанного на текущей странице, на следующую страницу. Исправлению данного недостатка посвящен следующий раздел.

Распечатка многостраничного документа

Распечатка многостраничного документа в чем-то аналогична операции прокрутки изображения в окне, где каждая страница документа представляет собой окно, в которое выводится соответствующая часть документа. Поскольку разбиение документа на страницы при печати должно выполняться автоматически и при этом должны соблюдаться некоторые специфические правила, которые достаточно сложно формализовать для всех случаев жизни, данная операция практически полностью ложится на плечи программиста.

Чтобы обеспечить в приложении PrintApp возможность распечатки многостраничного документа:

1. Откройте приложение PrintApp.
2. Откройте окно редактирования файла PrintAppView.cpp.
3. Измените текст функции OnBeginPrinting в соответствии с приведенным ниже текстом:

```
void CPrintAppView::OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo)
{
  // TODO: add extra initialization before printing

  CPrintAppDoc*  lpDoc = GetDocument();
  ASSERT_VALID( lpDoc);

  pInfo-> GetMaxPage((pDC-> GetDeviceCaps( LOGPIXELY)*(1 + 5*pDoc->
                      m_nRects))/pDC-> GetDeviceCaps( VERTRES) + 1);
}
```

4. Нажмите клавишу <F5> и запустите приложение на исполнение.
5. Выведите в окно документа три квадрата, откройте окно предварительного просмотра печати и нажмите кнопку **Two Page**. Окно предварительного просмотра примет вид, изображенный на рис. 10.12.

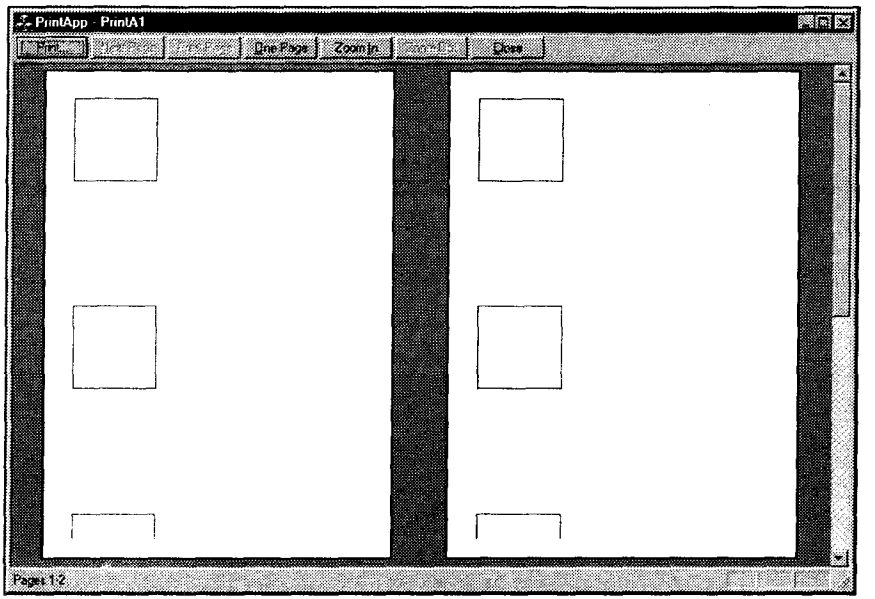


Рис. 10.12. Окно предварительного просмотра печати

6. Закройте окно предварительного просмотра печати и закройте приложение.
7. Вызовите мастер ClassWizard.
8. В окне списка **Object IDs:** выделите имя класса `CPrintAppView`, в окне списка **Messages:** — функцию обработки сообщения `OnPrepareDC` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится имя новой функции обработки сообщения.
9. Нажмите кнопку **Edit Code** и закройте диалоговое окно **MFC ClassWizard**. Откроется окно редактирования файла PrintAppView.cpp и текстовый курсор будет помещен в текст функции `OnPrepareDC`.
10. Измените текст функции `OnPrepareDC` в соответствии с приведенным ниже текстом:

```
void CPrintAppView::OnPrepareDC(CDC* pDC, CPrintInfo* pInfo)
{
  // TODO: Add your specialized code here and/or call the base class
```

```
    if( pDC-> IsPrinting()){
      ASSERT(m_nMapMode > 0);
      pDC->SetMapMode(m_nMapMode);
      pDC-> SetViewportOrg(0, -pDC-> GetDeviceCaps(VERTRES)*
                                    (pInfo->m_nCurPage - 1));
      CView::OnPrepareDC(pDC, pInfo);
    }
    else
      CScrollView::OnPrepareDC(pDC, pInfo);
}
```

11. Повторите операции, описанные в пунктах 4 и 5. Окно предварительного просмотра примет вид, изображенный на рис. 10.13.

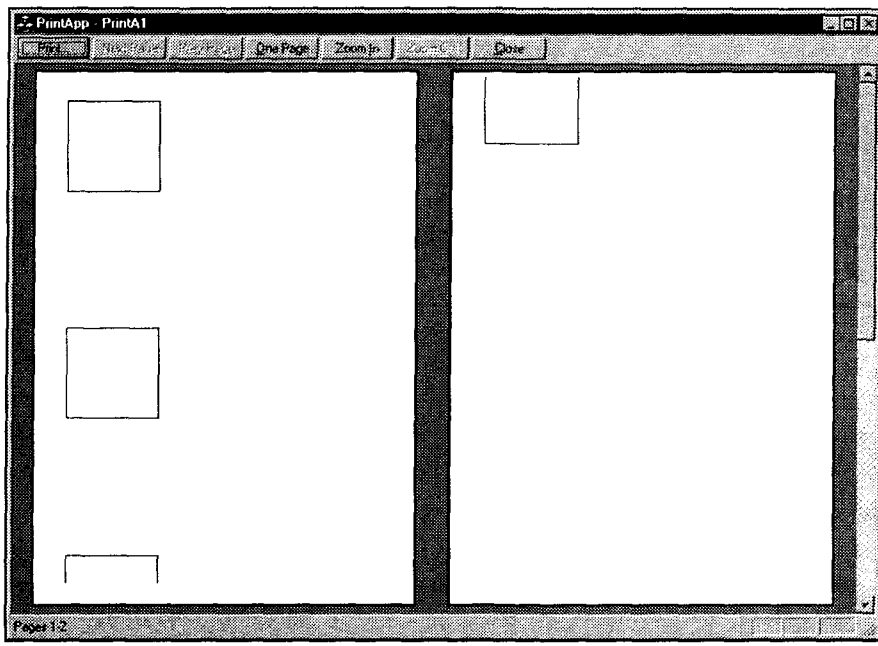


Рис. 10.13. Окно предварительного просмотра печати

12. Закройте окно предварительного просмотра печати и закройте приложение.
13. Вызовите мастер ClassWizard.
14. В окне списка **Member functions:** выделите имя функции `OnPrepareDC` и нажмите кнопку **Delete Function**. Появится окно сообщения **Microsoft Visual C++**, предупреждающее о необходимости уничтожить текст данной функции в файле реализации. Нажмите кнопку **OK**.

15. Имя функции `OnPrepareDC` исчезнет из окна списка **Member functions:**.
16. В окне списка **Object IDs:** выделите имя класса `CPrintAppView`, в окне списка **Messages:** выделите функцию обработки сообщения `OnPrint` и нажмите кнопку **Add Function**. В окне списка **Member functions:** появится имя новой функции обработки сообщения.
17. Нажмите кнопку **Edit Code** и закройте диалоговое окно **MFC ClassWizard**. Откроется окно редактирования файла PrintAppView.cpp и текстовый курсор будет помещен в текст функции `OnPrint`.
18. Измените текст функции `OnPrint` в соответствии с приведенным ниже текстом:

```
void CPrintAppView::OnPrint(CDC* pDC, CPrintInfo* pInfo)
{
  // TODO: Add your specialized code here and/or call the base class

  LOGFONT    logFont;
  CFont    font;
  CString    Out;

  logFont.lfHeight          = 24;
    logFont.lfWidth         = 0;
    logFont.lfEscapement    = 0;
    logFont.lfOrientation   = 0;
    logFont.lfWeight        = FW_NORMAL;
    logFont.lfItalic        = 0;
    logFont.lfUnderline     = 0;
    logFont.lfStrikeOut     = 0;
    logFont.lfCharSet       = ANSI_CHARSET;
    logFont.lfOutPrecision  = OUT_DEFAULT_PRECIS;
    logFont.lfClipPrecision = CLIP_DEFAULT_PRECIS;
    logFont.lfQuality       = PROOF_QUALITY;
    logFont.lfPitchAndFamily = VARIABLE_PITCH | FF_ROMAN;
  strcpy(logFont.lfFaceName, "MS Sans Serif");

  font.CreateFontIndirect( &logFont);

  CFont*  oldFont = pDC-> SelectObject( &font);

  sprintf( Out.GetBuffer(16),"Страница № %d", pInfo-> m_nCurPage);
  Out.ReleaseBuffer();

  pDC-> SetViewportOrg(0, ((pInfo-> m_rectDraw.top + 50)*
                           pDC-> GetDeviceCaps( LOGPIXELSY))/100);

  CScrollView::OnPrint(pDC, pInfo);

  CBrush  wtBrush( RGB(255, 255, 255));
  CRect  rect = pInfo-> m_rectDraw;
```

```
    rect.bottom  = pInfo-> m_rectDraw.top;
    rect.top  += 60;

    pDC-> FillRect( rect, &wtBrush);

    pDC-> TextOut( 0, pInfo-> m_rectDraw.top + 36, Out );
    pDC-> SelectObject( oldFont);
}
```

19. Уничтожьте текст функции `OnPrepareDC` в файле реализации.
20. Измените текст функции `OnBeginPrinting` в соответствии с приведенным ниже текстом:

```
void CPrintAppView::OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo)
{
  // TODO: add extra initialization before printing

  CPrintAppDoc*  lpDoc = GetDocument();
  ASSERT_VALID( lpDoc);

  pInfo-> SetMaxPage( (pDC-> GetDeviceCaps( LOGPIXELSY)*
    (1 + 5*lpDoc-> m_nRects))/(pDC-> GetDeviceCaps( VERTRES) - 50) + 1);
}
```

21. Повторите операции, описанные в пунктах 4 и 5. Окно предварительного просмотра примет вид, изображенный на рис. 10.14.
22. Включите принтер и нажмите кнопку **Print** в панели инструментов. Появится диалоговое окно **Печать**, изображенное на рис. 10.15.
23. Нажмите кнопку **OK**. Документ будет распечатан.
24. Закройте окно предварительного просмотра печати и закройте приложение.

Внесение в документ изменений, связанных с печатью многостраничного документа, было разбито на три этапа. На первом этапе были внесены изменения в функцию `OnBeginPrinting`, заготовку которой создал мастер AppWizard при генерации приложения.

Прежде всего следует отметить, что при создании заготовки данной функции ее аргументы были закомментированы, и заголовок функции выглядел следующим образом:

```
void CPrintAppView::OnBeginPrinting(CDC* /*pDC*/, CPrintInfo* /*pInfo*/)
```

Это было сделано потому, что в заготовке функции ее аргументы нигде не используются, что неизбежно приведет к появлению соответствующих сообщений при трансляции приложения, содержащего данную функцию. Указание в списке аргументов функции только типа аргумента без его иден-

тификатора означает, что этот аргумент в функции не используется, что в данном случае соответствует действительности. При включении в тело функции операторов, использующих данные аргументы, комментарии с них снимаются.

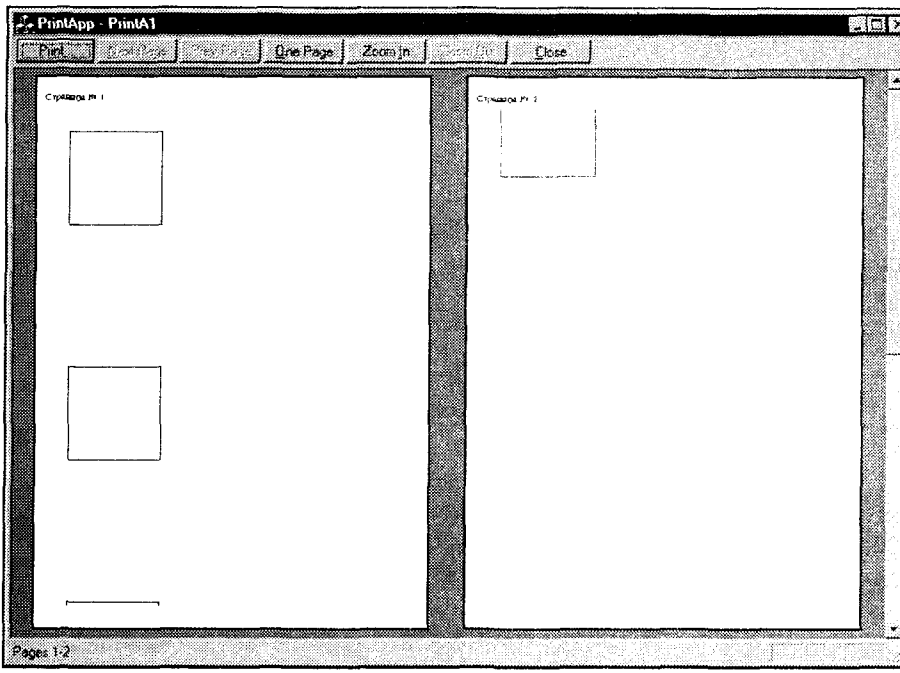


Рис. 10.14. Окно предварительного просмотра печати

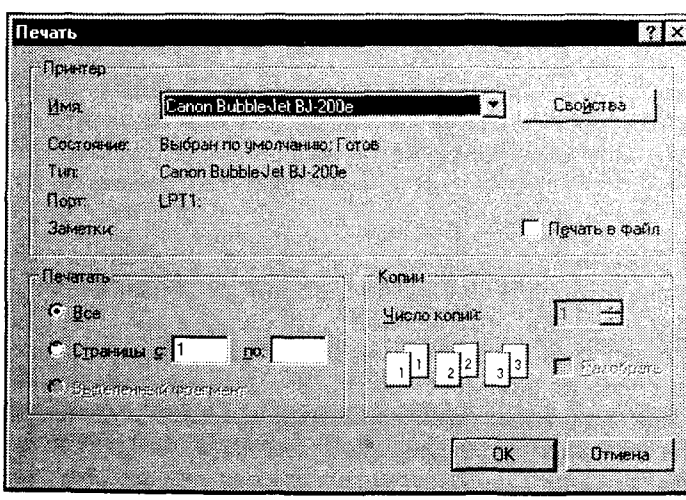


Рис. 10.15. Диалоговое окно **Печать**

Данная функция вызывается приложением при инициализации процесса печати или предварительного просмотра печати после вызова функции OnPreparePrinting. В данном приложении функция OnBeginPrinting используется для определения количества страниц в документе. Для этого вызывается функция CPrintInfo::SetMaxPage, позволяющая задать номер последней страницы в документе. Для вычисления числа страниц в документе используется функция CDC::GetDeviceCaps, позволяющая получить разнообразные сведения об устройствах отображения информации. Задание в качестве аргумента данной функции значения LOGPIXELSY позволяет получить число элементов изображения в логическом дюйме по вертикали дисплея.

Поскольку нами задан режим отображения MM_LOENGLISH, при котором один элемент изображения имеет размер 0.01 дюйма, то для определения вертикального размера выводимого изображения в элементах изображения необходимо умножить полученную величину на размер изображения в логических единицах, и разделить результат на 100. Операция деления была выполнена заранее. При задании в качестве аргумента значения VERTRES функция CDC::GetDeviceCaps возвращает размер дисплея по вертикали в элементах изображения. Поскольку в данном случае в качестве дисплея выступает печатаемая страница, то операция деления позволяет определить количество страниц, необходимое для печати всего документа. Поскольку результатом деления целых чисел является целая часть результата деления, к частному следует прибавить единицу, соответствующую неполной последней странице.

После этого приложение может правильно определить количество страниц в документе, но, как видно из рис. 10.12, оно на каждой странице начинает печать документа с начала. Чтобы на каждой странице печаталась своя информация, необходимо произвести настройку начала отсчета для каждой страницы. Эту операцию выполняет функция OnPrepareDC.

Поскольку данная функция используется как для вывода информации на экран, так и для печати документа, то в ней прежде всего выясняется цель вызова. Для этого вызывается функция CDC::IsPrinting, позволяющая определить, является ли данный объект класса CDC объектом класса контекста устройства принтера. Если нет, вызывается метод базового класса. Этот метод сам вызывает функцию CDC::IsPrinting и проверяет возвращаемое ею значение.

При печати документа функция CScrollView::OnPrepareDC по непонятной причине устанавливает начало отсчета для каждой страницы в начало документа, хотя логично предположить, что документ, использующий полосы прокрутки, скорее всего будет многостраничным, и, по крайней мере, неразумно сводить на нет все усилия пользователя по его полной распечатке. Однако, поскольку это так, при распечатке данного документа в функции CPrintAppView::OnPrepareDC производятся основные установки, выполняемые функцией CScrollView::OnPrepareDC, устанавливается начало отсчета для данной страницы документа и вызывается функция CView::OnPrepareDC.

Функция `CDC::SetMapMode` устанавливает режим отображения, задаваемый его аргументом. В качестве аргумента данной функции используется переменная `CScrollView::m_nMapMode`, в которой хранится значение текущего режима отображения.

Функция `CDC::SetViewportOrg` устанавливает начало отсчета рабочей области контекста устройства. Начало отсчета рабочей области определяет точку, заданную в системе координат устройства, в которую GDI помещает начало координат окна, т. е. точка, заданная в логической системе координат, определяемая функцией `SetWindowOrg`.

В качестве аргументов данной функции выступают координаты начала отсчета рабочей области, выраженные в координатах устройства. Горизонтальная координата начала отсчета рабочей области равна нулю, поскольку нет необходимости разбиения документа по ширине. Вертикальная координата начала отсчета каждой страницы совпадает с координатами нижней границы рабочей области предыдущей страницы. Для определения этой величины используется описанная выше функция `CDC::GetDeviceCaps` с аргументом `VERTRES`, возвращающая размер дисплея по вертикали в элементах изображения, и переменная `CPrintInfo::m_nCurPage`, содержащая имя текущей страницы. После этого вызывается функция `CView::OnPrepareDC`, являющаяся методом базового класса. Как видно из рис. 10.13, после внесения изменений в функцию `OnPrepareDC` пользовательского класса представления приложение корректно производит разбиение на страницы.

При выводе документа на экран его имя и текущее положение рабочей области в документе обычно отображаются в заголовке окна и в строке состояния. При печати документа бывает полезно вывести на печать номер текущей страницы и имя документа. Эту операцию проделывают даже те программы, которые полностью соответствуют требованиям WYSIWYG (What You See Is What You Get, "что вы видите, то вы и получите").

Обычно для вывода подобной информации используются *колонтитулы*. Поскольку данная операция выполняется только при выводе документа на печать, то ее целесообразно включить в функцию `OnPrint`. Для этого необходимо соответствующим образом изменить начало отсчета рабочей области и, возможно, масштаб координат окна. Вывод колонтитулов может привести также к увеличению числа страниц в документе.

Все изменения необходимо передать в контекст устройства перед вызовом функции `OnDraw`, чтобы исключить перекрытие областей вывода колонтитулов и самого документа. Может возникнуть необходимость внесения изменений и в саму функцию `OnDraw`.

Как утверждает библиотека MSDN, одним из путей учета областей, занимаемых колонтитулами, является использование переменной `m_rectDraw`, являющейся членом класса `CPrintInfo`. При инициализации процесса печати каждой страницы в данную переменную записывается область печати

страницы, заданная в логических координатах. После печати колонтитулов предлагается соответствующим образом изменить размеры прямоугольника, хранящегося в переменной m_rectDraw. После этого функция OnPrint может обращаться к данной переменной для определения области, остающейся для печати документа.

Печать колонтитулов или какой-либо другой информации не может производиться в функции OnPrepareDC, поскольку она вызывается до вызова функции CDC::StartPage, когда контекст устройства принтера используется для всей страницы. Печать может осуществляться только в функции OnPrint.

Для печати верхнего колонтитула в функции OnPrint создается структура логического шрифта и на ее основе с помощью функции CFont::CreateFontIndirect инициализируется объект класса CFont. Процедура вывода текста подробно описана при рассмотрении приложения TextApp в главе 6. Поэтому здесь мы на ней подробно не останавливаемся. После записи в объект класса CString содержимого верхнего колонтитула устанавливается начало отсчета для данной страницы. Для этого используется уже описанная функция CDC::SetViewportOrg. Начало координат страницы, выраженное в логических координатах, хранится в переменной top объекта класса CRect, хранящийся в переменной CPrintInfo::m_rectDraw. Поскольку для вывода колонтитула необходимо пространство, имеющее размер по вертикали, равный 50 логическим единицам, то эта величина прибавляется к текущему значению начала отсчета (имеющему отрицательное значение). После этого вызывается метод базового класса CScrollView::OnPrint, осуществляющий вывод текущей страницы документа на экран. Поскольку в данном случае производятся достаточно сложные преобразования области отсечки, лучше доверить эту операцию методу базового класса.

См. также главу 6

После этого необходимо удалить с экрана ту часть документа, которая перекрывается с областью колонтитула. Для этого в нее выводится белый прямоугольник, левая и правая границы которого совпадают с границами прямоугольника, хранящегося в переменной CPrintInfo::m_rectDraw, нижняя граница совпадает с началом отсчета рабочей области документа, а верхняя находится за верхней границей листа. Использование старого значения начала отсчета рабочей области в качестве верхней границы области колонтитулов приведет к тому, что часть документа будет отображаться над этой областью. Это связано с тем, что область печати устанавливается меньше физического размера страницы для компенсации влияния различных допусков при настройке принтера и изготовлении бумаги. После очистки содержимого области колонтитула в него выводится текст самого колонтитула.

Уничтожение функции OnPrepareDC связано с тем, что устанавливаемое в ней начало отсчета страницы нигде не используется, поскольку эту задачу решает функция OnPrint.

Использование функций библиотеки MFC при печати

При печати многостраничного документа функции библиотеки MFC обеспечивают постоянное взаимодействие между приложением и объектом пользовательского класса представления. Сначала приложение выводит диалоговое окно **Печать**, создает контекст устройства для принтера и вызывает функцию `CDC::StartDoc`. Затем для каждой страницы документа приложение вызывает функцию `CDC::StartPage` и сообщает объекту класса представления о необходимости напечатать страницу, после этого вызывается функция `CDC::EndPage`. Если необходимо изменить настройки принтера перед началом печати данной страницы, то объект класса представления должен вызвать функцию `CDC::Escape` и послать принтеру соответствующие управляющие последовательности. По завершении печати документа приложение вызывает функцию `CDC::EndDoc`.

Класс `CView` содержит различные функции, вызываемые приложением в процессе печати. Перегрузка этих функций в пользовательском классе представления позволяет настроить параметры печати в соответствии с требованиями пользователя. В табл. 10.2 перечислены эти функции и приведены причины, по которым они могут быть перегружены.

Таблица 10.2. *Функции класса* `CView`*, используемые при печати*

Имя функции	Причина перегрузки
`OnPreparePrinting`	Установить нестандартные значения величин в текстовых полях диалогового окна **Печать**
`OnBeginPrinting`	Создать шрифты и другие ресурсы GDI, используемые при печати
`OnPrepareDC`	Настроить атрибуты контекста устройства для данной страницы или произвести разбиение на страницы в процессе печати
`OnPrint`	Печатать данную страницу
`OnEndPrinting`	Освободить ресурсы GDI, используемые при печати

Параметры печати можно настраивать и в других функциях, но функции из табл. 10.2 являются непременными участниками процесса печати.

Печать происходит следующим образом:

- Вызывается функция `OnPreparePrinting` пользовательского класса представления, устанавливающая число страниц документа, если оно известно заранее, вызывается диалоговое окно **Печать**, в котором могут быть заданы нестандартные значения для содержимого его текстовых полей, затем создается объект класса контекста устройства для принтера

- Вызывается функция `OnBeginPrinting` пользовательского класса представления, определяющая число страниц документа на основе информации, содержащейся в контексте устройства, если эта величина не была определена заранее, и создаются объекты классов ресурсов GDI, используемых при печати
- Вызывается функция `CDC::StartDoc`, информирующая драйвер принтера о начале нового сеанса печати и о том, что все последующие вызовы функций `StartPage` и `EndPage` будут относиться к этому сеансу печати, пока не будет вызвана функция `EndDoc`
- Вызывается функция `OnPrepareDC` пользовательского класса представления, в которой устанавливается начало отсчета рабочей области и другие атрибуты контекста устройства. Если в данном приложении не определено количество страниц в документе, данная функция проверяет, не достигнут ли конец документа
- Вызывается функция `CDC::StartPage` для подготовки принтера к приему данных
- Вызывается функция `OnPrint` пользовательского класса представления, печатающая колонтитулы и текущую страницу документа. Если изображение выводится на экран функцией `OnDraw`, то она вызывается из функции `OnPrint`
- Вызывается функция `CDC::EndPage`, информирующая устройство о том, что приложение закончило передачу текущей страницы. После вызова этой функции приложение вызывает функцию `OnPrepareDC` для новой страницы
- После печати последней страницы вызывается функция `CDC::EndDoc`, завершающая текущий сеанс печати, инициированный вызовом функции `StartDoc`
- Вызывается функция `OnEndPrinting` пользовательского класса представления, в которой уничтожаются все объекты классов ресурсов GDI, созданных в функции `OnBeginPrinting`

Приложение хранит большую часть информации, касающейся процесса печати, в объекте класса `CPrintInfo`. Некоторые из членов данного класса связаны с разбиением документа на страницы. Эти члены класса `CPrintInfo` перечислены в табл. 10.3.

Таблица 10.3. *Члены класса* `CPrintInfo`*, связанные с разбиением на страницы*

Член класса	Назначение
`GetMinPage/SetMinPage`	Позволяют получить и установить номер первой страницы документа

Таблица 10.3 *(окончание)*

Член класса	Назначение
`GetMaxPage/SetMaxPage`	Позволяют получить и установить номер последней страницы документа
`GetFromPage`	Позволяют получить номер первой печатаемой страницы документа
`GetToPage`	Позволяют получить номер последней печатаемой страницы документа
`m_nCurPage`	Содержит номер текущей печатаемой страницы документа

Номера страниц начинаются с 1, т. е. первая страница имеет номер 1, а не 0.

При инициализации печати приложение вызывает функцию `OnPreparePrinting` пользовательского класса представления, передавая ей указатель на объект класса `CPrintInfo`. Мастер AppWizard создает при генерации приложения заготовку функции `OnPreparePrinting`, в которой вызывается функция `CView::DoPreparePrinting`. Функция `DoPreparePrinting` выводит диалоговое окно **Печать** и создает контекст устройства принтера.

На этот момент приложение не знает, сколько страниц содержится в документе. По умолчанию номер первой страницы документа устанавливается в 1, а номер последней — в 0xFFFF. Если число страниц в документе известно заранее, то в перегруженной функции `OnPreparePrinting` следует вызвать функцию `CPrintInfo::SetMaxPage` прежде, чем вызывать функцию `DoPreparePrinting`. Это позволит вывести в полях диалогового окна **Печать** истинные размеры документа.

Функция `DoPreparePrinting` выводит диалоговое окно **Печать**. После его закрытия объект класса `CPrintInfo` содержит установки данного диалогового окна. Если пользователь хочет распечатать некоторый диапазон страниц, то ему необходимо указать границы этого диапазона в соответствующих текстовых полях диалогового окна **Печать**. Приложение получает информацию об этих установках пользователя путем вызова функций `CPrintInfo::GetFromPage` и `CPrintInfo::GetToPage`. Если пользователь определил границы диапазона печати, то приложение вызывает функции `GetMinPage` и `GetMaxPage` и использует возвращенные ими значения для печати всего документа.

Для каждой печатаемой страницы документа приложение вызывает две функции пользовательского класса представления: `OnPrepareDC` и `OnPrint`. Каждая из них имеет два аргумента: указатель на объект класса `CDC` и указатель на объект класса `CPrintInfo`. Каждый раз, когда приложение вызывает функции `OnPrepareDC` и `OnPrint`, оно передает им различные значения в пе-

ременной CPrintInfo::m_nCurPage, содержащей номер текущей печатаемой страницы. Таким образом приложение сообщает объекту класса представления, какая страница должна быть напечатана.

Функция OnPrepareDC используется также для вывода документа на экран. Это позволяет вносить изменения в контекст устройства до запуска процедуры вывода изображения на экран. Функция OnPrepareDC играет сходную роль и в процессе печати, но в данном случае имеется два существенных различия: во-первых, объект класса CDC представляет собой контекст устройства принтера, а не контекст устройства дисплея, и во-вторых, в качестве второго аргумента передается указатель на объект класса CPrintInfo. (Этот аргумент имеет значение NULL при использовании функции OnPrepareDC для вывода на экран.) Перегрузка функции OnPrepareDC позволяет вносить изменения в контекст устройства в зависимости от того, какая страница печатается в данном случае. Например, в данной функции может быть установлено начало отсчета рабочей области и настроена область отсечки для вывода соответствующего фрагмента документа на печать.

Функция OnPrint используется для непосредственного вывода страницы на печать. Приложение вызывает функцию OnPrint и передает ей в качестве аргументов указатели на объекты классов CDC и CPrintInfo. Функция OnPrint передает контекст устройства функции OnDraw. Перегрузка функции OnPrint позволяет выводить информацию только в процессе печати, исключая ее вывод при отображении на экран. Например, в данной функции можно печатать колонтитулы. После вывода специфической для печати информации вызывается функция OnDraw для вывода информации, выводимой как на экран, так и на принтер.

Тот факт, что функция OnDraw выводит информацию как на экран, так и на принтер, гарантирует, что данное приложение соответствует принципам WYSIWYG. Если ваше приложение не должно решать эту задачу, то вместо функции OnDraw можно вызвать любую пользовательскую функцию, используемую только для вывода на печать.

При нумерации страниц иногда следует помнить о различии между страницами принтера и страницами документа. С точки зрения принтера страница представляет собой лист бумаги. Однако один лист бумаги не обязательно должен соответствовать одной странице документа. Например, при печати информационного бюллетеня, в котором листы должны складываться, один лист бумаги может содержать одновременно первые и последние страницы документа. А при печати электронных таблиц документ вообще не содержит страниц, а один лист бумаги может содержать строки с 1 по 20 и столбцы с 6 по 10.

Все номера страниц в структуре CPrintInfo относятся к страницам принтера. Приложение вызывает функции OnPrepareDC и OnPrint один раз для каждого листа бумаги, заложенного в принтер. При перегрузке функции

`OnPreparePrinting` для задания числа страниц в документе указываются страницы принтера. Если имеется взаимно однозначное соответствие (то есть одна страница принтера равняется одной странице документа), то все вопросы снимаются просто. Если же взаимно однозначное соответствие отсутствует, необходимо установить между ними другой тип соответствия. Например, в случае печати электронной таблицы при перегрузке функции `OnPreparePrinting` необходимо вычислить, сколько листов бумаги потребуется для распечатки всей электронной таблицы, а затем использовать эту величину при вызове функции `CPrintInfo::SetMaxPage`. Точно так же при перегрузке функции `OnPrepareDC` необходимо определить по текущему значению переменной `m_nCurPage` диапазон строк и столбцов, которые необходимо распечатать на данном листе.

В некоторых случаях пользовательский класс представления не может заранее определить размер документа. Например, эта ситуация возникает в том случае, если данное приложение не соответствует требованиям WYSIWYG, и по размеру документа на экране нельзя определить его размер при печати.

Это вызывает проблемы при перегрузке функции `OnPreparePrinting` пользовательского класса просмотра, поскольку в нем не может быть вызвана функция `CPrintInfo::SetMaxPage` для задания размера документа. Если пользователь не определяет число страниц в документе, приложение должно само определить, когда ему прекратить процесс печати. Единственным способом для этого является распечатка документа до конца и автоматическое определение момента достижения конца документа.

Приложение возлагает эту задачу на функцию `OnPrepareDC` пользовательского класса представления. После каждого вызова этой функции приложение проверяет значение переменной `CPrintInfo::m_bContinuePrinting`. Если оно равно `TRUE`, то процесс печати следует продолжить, а если `FALSE`, то — прекратить. По умолчанию функция `OnPrepareDC` присваивает переменной `m_bContinuePrinting` значение `FALSE`, если номер текущей страницы превышает 1. Это означает, что если длина документа не была определена, приложение считает, что документ состоит из одной страницы.

Особенности программирования в среде Visual C++

11. Исключения, шаблоны и новые возможности Visual C++
12. Многозадачность на основе потоков Windows
13. Справка в приложении
14. Отладка приложения

Исключения, шаблоны и новые возможности Visual C++

Специалисты Microsoft постоянно совершенствуют язык Visual C++. Новыми средствами, которые сравнительно недавно были добавлены в этот язык программирования, являются исключения, шаблоны и классы коллекций.

Исключения

Windows является операционной системой, управляемой сообщениями. Это означает, что приложение по сути разбивается на ряд практически независимых фрагментов, связь между которыми осуществляется операционной системой. В связи с этим остро встает вопрос об обработке ошибок, поскольку при возникновении ошибки в одном из этих фрагментов достаточно сложно передать ее код вызывающей данный фрагмент процедуре. В случае работы с классами, не имеющими прототипов в библиотеке MFC, эта проблема становится практически неразрешимой без использования механизма исключений.

Механизм исключений позволяет процедурам приложения обмениваться информацией о возникновении серьезных проблем в процессе их выполнения. Для реализации данного механизма используются три оператора языка Visual C++:

- В блок `try` помещается программный код, в котором могут возникнуть ошибки
- В блоке `catch`, располагающемся непосредственно после связанного с ним блока `try`, находится программный код, обрабатывающий информацию о возникшей ошибке
- Оператор `throw` прерывает исполнение программы и передат управление связанному с ним блоку `catch`. Если с данным оператором не связан

catch, то управление передается модулю библиотеки MFC, выводящему сообщение об ошибке и завершающему выполнение данного приложения

Схематически структуру программы, использующей механизм исключений, можно представить следующим образом:

```
try{
  . . . . . .
  if( . . . .)
    throw;
  . . . . . .
}
catch( ...){
  . . . . . .
}
```

В данном примере открывается блок try и в нем выполняются некоторые операции. В условном операторе if проверяется условие вызова исключения. Если оно выполняется, то оператор throw вызывает исключение и передает управление блоку catch, в котором и обрабатывается данное исключение. Если условие не выполняется, и программа доходит до конца блока try, то следующий за ним блок catch не выполняется.

Если в блоке try вызывается некоторая функция, содержащая оператор throw, не заключенный в блок try, то оператор throw действует в данном случае как оператор return, и управление передается в связанный с данным блоком try блок catch. Если оператор throw содержится в вызываемой функции, то он действует как оператор return во всей последовательности вызывающих функций, пока не будет достигнута функция, вызываемая из блока try. После чего выполняется описанная выше процедура. Если во всей иерархии функций ни одна не заключена в блок try, то приложение выдает сообщение об ошибке.

Аргументы исключений

Необходимо помнить, что при вызове исключения ему передается один аргумент, в качестве которого может выступать любая структура данных. Синтаксис блока catch предполагает передачу аргумента, указанного в скобках после ключевого слова catch. При этом каждый блок обрабатывает только те исключения, которым при их вызове был передан аргумент того же типа. Это позволяет не только передать блоку catch всю информацию, необходимую для обработки данного исключения, но и создать для каждого типа исключения свой обработчик.

Если обработка исключения представляет собой сложную операцию, а само исключение вызывается часто, то процедура обработки может быть скрыта в классе исключения, как это показано в листинге 11.1.

Листинг 11.1. Класс исключения

```
class MyException{
protected:
  UINT  nIDSError;
public:
MyException( int nErr)  { nIDSError = nErr;};
~MyException()  {};

void  Error_Put();
};

void  MyException::Error_Put(){

    CString    buf;

    VERIFY(buf.LoadString( nIDSError));

    ::MessageBox( 0, buf, "Error", MB_OK|MB_SYSTEMMODAL);
}
int  TestFunction(){
  int*  aBuffer;

  try{
    if( (aBuffer = new int[1024]) == NULL){
      MyException*  me = new MyException( ID_MEMORY);
      throw me;
    }
    else
      delete  aBuffer;
  }
  catch( MyException* me){
    me-> Error_Put();
  }
}
```

В листинге 11.1 приведено описание класса `MyException`, содержащего переменную `nIDSError`, в которую записывается идентификатор строкового ресурса с текстом сообщения об ошибке, и функцию `Error_Put`, выводящую сообщение об ошибке на экран. Переменная `nIDSError` инициализируется в конструкторе класса. В функции `Error_Put` с помощью вызова функции `CString::LoadString` по идентификатору ресурса получается связанное с ним сообщение, после чего вызывается функция `MessageBox` для вывода окна сообщения с соответствующим текстом. На этом обработка исключения завершается.

Библиотека MFC сама активно использует механизм исключений. Причем многие функции пользователя вызываются функциями библиотеки MFC,

заключенными в блок `try`. Некоторые функции библиотеки MFC содержат оператор `throw`, но не заключены в блок `try`. Например, упомянутая выше функция `CString::LoadString` посылает исключение с аргументом, представляющим собой объект класса `CMemoryException`. При этом неизвестно, находится ли пользовательская функция в блоке `try`, и какие типы исключений он обрабатывает. Эта информация получается методом проб и ошибок.

При обработке исключений необходимо помнить, что оператор `throw` действует аналогично оператору `return`, и в нем не предусмотрены операции уничтожения динамических переменных, открытых в функции, из которой было вызвано исключение, или, что еще хуже, в функции, из которой была вызвана функция, вызвавшая исключение.

Поскольку заранее часто неизвестно, какая из функций пользователя может вызвать исключение, ему следует воздержаться от применения в своих функциях динамических переменных и создавать все переменные в стеке. Если пользователь все-таки работает с динамическими переменными, следует создать для них отдельный класс, объект которого разместить в стеке. Это гарантирует, что при вызове исключения будет вызван деструктор данного класса, который уничтожит динамический объект.

Механизмы исключений Visual C++

В Visual C++ существует два механизма исключений:

- Исключения, вызываемые как операторы языка C++. Этот тип исключений используется в библиотеке MFC версии 3.0 и более поздних
- *Макросы исключений*, используемые во всех версиях библиотеки MFC

При написании новых программ, использующих библиотеку MFC, для работы с исключениями следует использовать операторы C++. Использование макросов оправдано только при добавлении дополнительных возможностей в старые приложения, активно использующие макросы.

Возможно преобразование программ, содержащих макросы исключений, в программы, использующие операторы C++. Способ данного преобразования и его преимущества будут описаны позднее.

Для использования исключений в программах выберите команду **Project**, **Settings...**. Появится диалоговое окно **Project Settings**, изображенное на рис. 11.1.

Раскройте вкладку **C/C++**, выберите в раскрывающемся списке **Category:** значение **C++ Language** и убедитесь, что в данной вкладке установлен флажок **Enable exception handling**. Если он не установлен, установите его. Диалоговое окно **Project Settings** примет вид, изображенный на рис. 11.2.

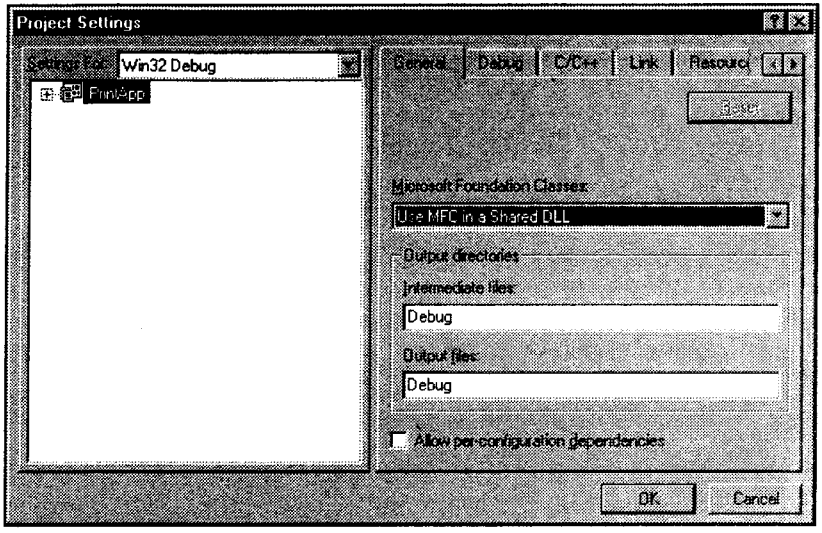


Рис. 11.1. Диалоговое окно **Project Settings**

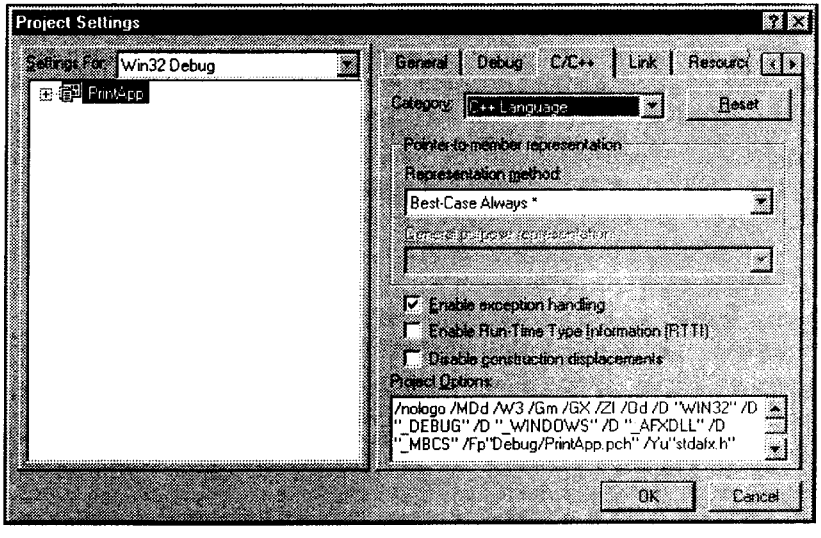


Рис. 11.2. Диалоговое окно **Project Settings**, вкладка **C/C++**

В каких случаях следует вызывать исключения

При вызове любой функции приложения возможны три исхода:

- **Нормальное завершение.** Функция может нормально завершить работу и возвратить управление вызвавшей ее функции. Некоторые функции возвращают значение, содержащее информацию о результатах выполнения.

Возвращаемые значения строго определены для каждой функции и обозначают все возможные исходы ее выполнения. Возвращаемое значение может говорить об успешном завершении работы функции или о возникновении ошибки. В некоторых случаях возвращаемое значение может говорить о конкретном виде ошибки. Например, функции, позволяющие получать сообщение о состоянии объекта, могут возвращать значение, говорящее о том, что данный объект отсутствует

- **Завершение с ошибкой.** При вызове функции возникла ошибка при передаче функции аргументов или функция была вызвана в недопустимом контексте. Для обнаружения подобных ошибок в процессе отладки используются специальные макросы, одним из которых является макрос `ASSERT`
- **Возникновение особых ситуаций при выполнении программы.** Особые ситуации при выполнении программы возникают по причинам, не зависящим от данной программы. Например, вследствие ошибок ввода/вывода или недостаточного количества оперативной памяти. Именно для работы с такими особыми ситуациями и используются исключения, что позволяет корректно завершить работу приложения практически из любой его точки

Если исключения вызываются операторами C++ или макросами библиотеки MFC, используются объекты класса `CException` или производных от него классов. Библиотека MFC содержит несколько классов исключений, предназначенных для работы в определенных ситуациях:

- `CMemoryException` — недостаточно оперативной памяти
- `CFileException` — исключения, возникающие при работе с файлами
- `CArchiveException` — исключения, связанные с функцией `Serialize` и работой с архивами
- `CNotSupportedException` — было вызвано устройство, не поддерживаемое в данной конфигурации операционной системы
- `CResourceException` — исключения, возникающие при работе с ресурсами
- `CDaoException` — исключения, связанные с базами данных (классами DAO)
- `CDBException` — исключения, связанные с базами данных (классами ODBC)
- `COleException` — исключения, связанные с объектами OLE
- `COleDispatchException` — исключения, связанные с работой серверов автоматизации
- `CUserException` — данное исключение выводит окно сообщения, после чего посылает сообщение `CException`

Библиотека MFC позволяет вызывать и обрабатывать исключения как с помощью операторов C++, так и с помощью макросов библиотеки MFC, но

не позволяет непосредственно работать со *структурированными обработчиками исключений* (Structured Exteption Handlers, SEH) Windows NT.

В качестве аргумента блока `catch` может быть использована переменная практически любого типа, но функции библиотеки MFC вызывают только исключения, являющиеся объектами классов, производных от класса `CException`. Чтобы перехватить исключение, вызванное функцией библиотеки MFC, следует написать блок `catch` и указать в качестве его аргумента указатель на объект класса `CException` (или на объект класса, производного от класса `CException`, например, `CMemoryException`). Объект каждого класса исключения содержит переменные, позволяющие установить причину возникновения исключения.

Например, объект класса `CFileException` содержит переменную `m_cause`, содержащую код причины вызова файлового исключения. В качестве кодов причины вызова исключения могут использоваться значения `CFileException::fileNotFound` и `CFileException::readOnly`. Ниже приведен пример использования данного класса исключения.

```
try
{
    // Фрагмент программы, в котором может быть
    // вызвано исключение.
}
catch( CFileException* theException)
{
    if( theException->m_cause == CFileException::fileNotFound)
        TRACE( "File not found\n");
    theException->Delete();
}
```

Перехват и уничтожение исключений

Пользовательский блок обработки исключений должен уничтожать передаваемые ему объекты, поскольку в противном случае могут возникнуть утечки памяти. Блок `catch` должен уничтожать исключения в следующих случаях:

- Когда блок `catch` вызывает новое исключение. Если исключение просто пересылается, его не следует уничтожать. Пересылка сообщения продемонстрирована ниже:

```
catch(CException* e)
{
    if (m_bThrowExceptionAgain)
        throw; // Do not delete e
}
```

- Программа прекращает выполнение данной функции в блоке `catch` и передает управление вызвавшей ее функции

Для уничтожения объекта класса `CException` следует использовать функцию `Delete` данного объекта, а не оператор `delete`, поскольку исключение может не находиться в куче.

Чтобы перехватить и уничтожить исключение, следует использовать блок `try`. Любой фрагмент программы, который может вызвать исключение, должен находиться в пределах блока `try`. Фрагмент программы, осуществляющий обработку исключения, следует поместить в блок `catch`. Этот фрагмент программы будет выполняться только в том случае, если в блоке `try` было вызвано исключение того же типа, который обрабатывается в данном блоке `catch`. Ниже приведена структура блоков `try` и `catch`, в которых уничтожается объект класса исключения.

```
// Нормальное выполнение программы
...

try
{
    // Фрагмент программы, в котором может быть вызвано исключение.
}
catch( CException* e)
{
    // Обработка исключения.
    // Аргумент "e" содержит информацию об исключении.
    e->Delete();
}

// Продолжение нормального выполнения программы
...
```

При вызове исключения управление передается первому блоку `catch`, который обрабатывает исключения того же типа, что и вызванное исключение. Последовательное расположение блоков `catch`, обрабатывающих различные типы исключений, позволяет произвести отдельную обработку каждого типа исключения. Если необходимо обработать все возможные типы исключений, вместо объявления аргумента в блоке `catch` ставятся три точки (...). Такой блок полезно разместить после всех других блоков catch для перехвата не обработанных ими типов исключений. Ниже приведен пример использования нескольких блоков `catch`.

```
try
{
    // Фрагмент программы, в котором может быть вызвано исключение.
}
```

```
catch( CMemoryException* e)
{
    // Обработка нехватки памяти.
}
catch( CFileException* e)
{
    // Обработка ошибок при работе с файлами.
}
catch( CException* e)
{
    // Обработка всех остальных системных исключений.
}
```

Уничтожение объектов в исключениях

Вызов исключения в приложении может изменить нормальный ход выполнения программы. Поэтому необходимо постоянно следить за динамически созданными объектами и корректно уничтожать их при вызове исключения. Для этого необходимо придерживаться двух основных принципов:

- Обрабатывать исключения локально, после чего уничтожать все объекты одним оператором
- Уничтожать все объекты в блоке `catch` перед вызовом исключения, которое будет обрабатываться за пределами данной функции

Следование этим принципам позволит избежать проблем, возникших в следующем примере:

```
void SomeFunc()     // Данная функция содержит ошибку
{
    CPerson* myPerson = new CPerson;

    // Фрагмент программы, в котором может быть вызвано исключение.
    myPerson->SomeFunc();

    // Уничтожение объекта перед выходом из функции.
    delete myPerson;
}
```

В приведенной выше функции переменная `myPerson` не уничтожается при вызове исключения в функции `SomeFunc`. Управление из этой функции передается непосредственно в обработчик исключения, расположенный за ее пределами, не давая функции завершить свою работу и освободить динамическую переменную. Указатель на данный объект выходит за пределы своей области действия, и область памяти, занимаемая данным объектом, не будет освобождена до завершения данного приложения. Эта ситуация называется утечкой памяти и диагностируется программной оболочкой.

Ниже приведен текст той же самой функции, в которой эта проблема решена.

```
void SomeFunc()
{
    CPerson* myPerson = new CPerson;

    try
    {
    // Фрагмент программы, в котором может быть
    // вызвано исключение.
        myPerson->SomeFunc();
    }
    catch( CException* e)
    {
        // Handle the exception locally
        e->Delete();
    }

    // Уничтожение объекта перед выходом из функции.
delete myPerson;
}
```

В данном случае используются локальные блоки обработки исключений. Это позволяет произвести все действия, необходимые для обработки данного исключения в самой функции, а затем продолжить ее нормальную работу, по завершении которой динамические переменные удаляются.

Если обработку исключения нельзя или нецелесообразно выполнить локально (например, это исключение возникает во многих точках приложения, а его обработка требует достаточно сложных операций), можно воспользоваться другим методом работы с динамическими объектами при вызове исключений. В этом случае блок `catch` просто транслирует объект класса исключения во внешний обработчик, но перед этим уничтожает все динамические переменные, созданные в данной функции. Пример подобной функции приведен ниже:

```
void SomeFunc()
{
    CPerson* myPerson = new CPerson;

    try
    {
    // Фрагмент программы, в котором может быть
    // вызвано исключение.
        myPerson->SomeFunc();
    }
    catch( CException e)
```

```
    {
        // Уничтожение объекта перед вызовом исключения.
        delete myPerson;
        // Вызов исключения для внешней обработки.
        throw;
    }

    // Уничтожение объекта перед выходом из функции.
    delete myPerson;
}
```

Механизм исключений автоматически уничтожает оконные объекты, поэтому для них не нужно вызывать деструкторы.

Вызов исключений из функций пользователя

Существует две принципиальные возможности вызова исключения в пользовательском приложении. Первый из них заключается в вызове одной из функций библиотеки MFC, например, `AfxThrowMemoryException`, вызывающих предопределенный объект соответствующего класса исключения. В приведенном ниже примере функция пытается разместить два блока памяти и вызывает исключение в случае неудачи:

```
void SomeFunc()
{
    char* p1 = (char*)malloc( SIZE_FIRST);
    if( p1 == NULL)
        AfxThrowMemoryException();
    char* p2 = (char*)malloc( SIZE_SECOND);
    if( p2 == NULL)
    {
        free( p1);
        AfxThrowMemoryException();
    }

    free( p1);
    free( p2);
}
```

В случае, если не удалось разместить первый блок памяти, можно просто вызвать исключение, если же не удалось разместить второй блок памяти, то необходимо сначала уничтожить первый блок памяти, и только потом можно вызывать исключение. Если оба блока были размещены успешно, они освобождаются перед выходом из функции.

Второй возможностью вызова исключения в пользовательском приложении является создание пользовательского класса исключения для хранения ин-

формации о причине его возникновения. В приведенном ниже примере приложение пытается проиграть звуковой файл и вызывает исключение в случае неудачи:

```
#define WAVE_ERROR -5
{
    // Используемая функция Win32 API возвращает 0,
   // если файл не может быть проигран.
   // Исключение имеет тип целочисленной константы.
    if( !sndPlaySound("SIREN.WAV", SND_ASYNC))
        throw WAVE_ERROR;
}
```

Процедура, используемая по умолчанию в библиотеке MFC, обрабатывает только исключения, являющиеся указателями на объект класса `CException` (и производных от него классов). В данном примере механизм обработки исключений библиотеки MFC не используется.

Преобразование макросов исключений в операторы C++

В библиотеке MFC версии 3.0 и более поздних версиях макросы исключений были заменены на операторы C++. В предыдущих версиях библиотеки MFC использовался макрос `CATCH`, определявший тип исключения. То есть тип исключения определялся обработчиком. В операторах C++ тип исключения определяется при его вызове. Это несоответствие может вызвать проблемы в тех редких случаях, когда тип указателя на объект исключения отличается от типа самого объекта исключения. Это продемонстрировано в приведенном ниже примере:

```
TRY
{
    THROW( (CException*) new CCustomException());
}
CATCH( CCustomException, e)
{
    TRACE( "Это исключение будет обработано в библиотеке MFC 2.x \n");
}
AND_CATCH( CException, e)
{
    TRACE( " Это исключение будет обработано в библиотеке MFC 3.0 \n");
}
END_CATCH
```

Этот фрагмент программы будет вести себя иначе в версии 3.0, поскольку управление всегда передается первому блоку `catch`, имеющему соответствующий тип аргумента. В результате работы следующего макроса:

```
THROW((CException*)new CCustomException());
```

будет вызвано исключение CException* несмотря на то, что программист хотел вызвать исключение CCustomException. Макрос CATCH в библиотеке MFC версии 2.5 и более ранних использует функцию CObject::IsKindOf для проверки типа в процессе выполнения программы. Поскольку выражение

```
e->IsKindOf( RUNTIME_CLASS( CException))
```

является истинным, первый блок CATCH перехватывает управление. В версии 3.0, использующей операторы C++ для обработки большинства исключений, управление перехватит второй блок CATCH, имеющий аргумент CException.

Такие фрагменты редко встречаются в программах. Они обычно появляются в тех случаях, когда объект исключения передается другой функции, принимающей исключение типа CException*, выполняющей некоторую обработку и транслирующей данное исключение.

Для решения этой проблемы необходимо перенести оператор throw из функции в вызывающий фрагмент программы и послать исключение соответствующего типа, известного на момент посылки исключения.

Блок catch не может вызывать тот же самый указатель на объект класса исключения, что он получил. Следующий пример не содержал ошибок в ранних версиях, но его исполнение в версии 3.0 и более поздних может привести к непредсказуемым последствиям:

```
TRY
{
    // Фрагмент программы, в котором может быть вызвано исключение.
}
CATCH( CSomeException, e)
{
    THROW( e);  // Ошибка. Необходимо использовать THROW_LAST()
}
END_TRY
```

Использование макроса THROW в блоке обработки исключения приводит к уничтожению указателя e, таким образом, внешний блок обработки исключения получит некорректный указатель. Вместо этого макроса необходимо использовать макрос THROW_LAST.

Хотя для работы со старыми программами нет необходимости преобразовывать все имеющиеся в них макросы исключений в операторы C++ (хотя бы потому, что практически все функции библиотеки MFC используют макросы, а не операторы C++), но необходимо четко понимать, какие преимущества сулит данный переход:

- Программы, использующие операторы C++ для работы с исключениями, имеют меньший размер файлов .exe и .dll

- Использование операторов C++ для работы с исключениями предоставляет дополнительные возможности, к которым относится, например, обработка исключений практически любых типов (`int`, `float`, `char` и т. д.), в то время как макросы позволяют работать только с объектами класса `CException` и производных от него классов

Основным различием между применением макросов и операторов C++ для работы с исключениями является то, что при использовании макросов перехваченное исключение автоматически удаляется, как только оно выходит за область своего действия. В случае обработки исключения операторами C++ уничтожение необходимо выполнить явным образом.

Кроме того, имеются некоторые различия в синтаксисе макросов и операторов C++. Эти различия можно разделить на три группы:

- Различия в аргументах макросов и объявлении исключений:
  - Макрос `CATCH` имеет следующий синтаксис:

    ```
    CATCH( exception_class, exception_object_pointer_name)
    ```

  - Между именем класса и указателем на объект стоит запятая
  - Заголовок блока `catch` имеет следующий синтаксис:

    ```
    catch( exception_type exception_name)
    ```

    В нем имя класса используется для определения типа имени исключения и между ними не стоит запятой.
- Различия в задании последовательности блоков обработки исключений:
  - Если применяются макросы, то для первого блока в последовательности работает макрос `CATCH` (со своими аргументами), для последующих блоков — макрос `AND_CATCH`, а для завершения последовательности используется макрос `END_CATCH`
  - При использовании операторов C++ каждый блок обработки начинается с ключевого слова `catch` (с объявлением своего аргумента). Для макроса `END_CATCH` не существует аналога. Блок `catch` завершается закрывающей скобкой
- Различия в способе трансляции исключения:
  - Для трансляции исключения используется макрос `THROW_LAST`. Вызов оператора `throw` без аргумента имеет те же последствия

Чтобы преобразовать программу, использующую макросы для работы с исключениями в программу, использующую для этого операторы C++:

1. Определите местоположение всех макросов библиотеки MFC, таких как `TRY`, `CATCH`, `AND_CATCH`, `END_CATCH`, `THROW` и `THROW_LAST`.
2. Произведите следующие замены:

- Макрос TRY замените ключевым словом try
- Макрос CATCH замените ключевым словом catch
- Макрос AND_CATCH замените ключевым словом catch
- Уничтожьте макрос END_CATCH
- Макрос THROW замените ключевым словом throw
- Макрос THROW_LAST замените ключевым словом throw

3. Внесите изменения в аргументы макросов, чтобы они приняли вид объявления объектов исключений. Например, замените выражение

```
CATCH( CException, e)
```

выражением

```
catch( CException* e)
```

4. Внесите изменения в блоки обработки исключений, чтобы в них при необходимости уничтожались объекты исключений.

Ниже приведен фрагмент программы, использующей макросы:

```
TRY
{
    // Фрагмент программы, в котором может быть вызвано исключение.
}
CATCH(CException, e)
{
    if (m_bPassExceptionsUp)
        THROW_LAST();

    if (m_bReturnFromThisFunction)
        return;

    // Нет необходимости уничтожать переменную e.
}
END_CATCH
```

Этот фрагмент, модифицированный в соответствии с пп. 1—4, приведенными выше, будет выглядеть следующим образом:

```
try
{
    // Фрагмент программы, в котором может быть вызвано исключение.
}
catch(CException* e)
{
    if (m_bPassExceptionsUp)
        throw;
```

```
    if (m_bThrowDifferentException)
    {
        e->Delete();
        throw new CMyOtherException;
    }

    if (m_bReturnFromThisFunction)
    {
        e->Delete();
        return;
    }

    e->Delete();
}
```

Совместное использование макросов и операторов C++

В одной и той же программе для работы с исключениями могут одновременно использоваться и макросы, и операторы C++. Однако недопустимо одновременное использование макросов и операторов в одном и том же блоке, поскольку макросы, в отличие от операторов C++, автоматически уничтожают объект класса исключения при выходе из блока обработки. Поэтому смешение макросов и операторов может привести или к утечке памяти, если объект класса исключения не будет уничтожен, или к нарушению работы памяти при попытке повторного уничтожения данного объекта. В приведенном ниже примере дальнейшее использование указателя на объект класса исключения невозможно:

```
TRY
{
    TRY
    {
     // Фрагмент программы, в котором может быть вызвано исключение.
}
    CATCH(CException, e)  // "Внутренний" блок catch
    {
        throw;   // Недопустимая попытка вызвать исключение и передать
                 // управление во внешний блок catch, расположенный ниже.
    }
    END_CATCH
}
CATCH(CException, e)  // "Внешний" блок catch
{
     // Указатель e нельзя использовать, поскольку
     // он был уничтожен во внутреннем блоке catch.
}
END_CATCH
```

Проблема возникает потому, что указатель e был уничтожен при выходе из "внутреннего" блока CATCH. Использование макроса THROW_LAST вместо оператора throw позволит "внешнему" блоку CATCH получить корректный указатель:

```
TRY
{
    TRY
    {
    // Фрагмент программы, в котором может быть вызвано исключение.
}
    CATCH(CException, e) // The "inner" catch block
    {
        THROW_LAST();     // Вызывается исключение и управление
                          // передается внешнему блоку catch.
    }
    END_CATCH
}
CATCH(CException, e)      // "Внешний" блок catch
{
     // Указатель e корректен, поскольку использован
     // макрос THROW_LAST().
}
END_CATCH
```

Запрещается трансляция исключения из блока try, расположенного в блоке макроса CATCH. Выполнение приведенного ниже фрагмента программы приведет к возникновению ошибки:

```
TRY
{
    // Фрагмент программы, в котором может быть вызвано исключение.
}
CATCH(CException, e)
{
    try
    {
        throw;  // Ошибка. При выполнении этой операции
                // переменная e (транслируемое исключение)
                // будет уничтожена.
    }
    catch(CException exception)
    {
    }
}
END_CATCH
```

Шаблоны

При написании программ часто приходится создавать функции или классы для выполнения некоторых действий над объектами разных типов, причем во многих случаях над объектами различных типов необходимо производить одни и те же операции. Например, необходимо разработать класс, выводящий на экран содержимое массива в виде графика. Без использования шаблонов пришлось бы создавать специальный класс для каждого возможного типа переменных, хранящихся в данном массиве, и практически полностью повторять тексты функций в каждом из классов. Использование шаблонов позволяет написать один класс, который сможет работать с массивами всех возможных типов. Эта идея была заложена в концепцию шаблонов, описанную в рабочих документах ISO WG21/ANSI X3J16.

Для задания шаблона используется ключевое слово `template`. Оператор `template`, формат которого приведен ниже, позволяет задавать абстрактные типы данных при объявлении и описании функций и классов:

```
template < [typelist] [, [ arglist ]] > declaration
```

Аргументом оператора `template` является список абстрактных типов (имеющих формат `class identifier` или `typename identifier`) или других объектов, используемых в теле шаблона. В поле `declaration` размещается объявление функции или класса.

Объявление класса при помощи шаблона ничем не отличается от объявления класса без шаблона за тем исключением, что в данном классе при объявлении типов его переменных, аргументов функций и возвращаемых значений может использоваться идентификатор абстрактного класса, объявленный в аргументе оператора `template`. Пример объявления класса с использованием шаблона приведен ниже.

```
template <class T, int i> class TestClass {
public:
   char buffer[i];
   T testFunc(T* p1);
};

template <class T, int i>
T TestClass<T,i>::testFunc(T* p1) {
    return *(p1++)
};
```

Чтобы создать объект данного класса в программе, необходимо объявить переменную, например:

```
TestClass<char, 5> ClassInst;
```

Понятие шаблона

Шаблоны представляют собой механизм создания функций и классов при помощи абстрактных типов переменных с возможностью использования вместо этих абстрактных типов любых допустимых типов переменных. Шаблон позволяет создать один класс, способный работать с переменными различных типов, вместо того, чтобы создавать множество классов, по одному на каждый возможный тип переменной.

Например, для создания функции, сохраняющей тип возвращаемого значения при определении минимальной величины из двух заданных, можно написать ряд перегруженных функций, подобных приведенным ниже.

```
// min для целых чисел
int min( int a, int b)
    return ( a < b) ? a : b;

// min для длинных целых чисел
long min( long a, long b)
    return ( a < b) ? a : b;

// min для символов
char min( char a, char b)
    return ( a < b) ? a : b;

// и т. д....
```

С использованием шаблона все эти функции могут быть заменены одной функцией, например:

```
template <class T> T min( T a, T b)
    return ( a < b) ? a : b;
```

Это позволяет уменьшить размер программы и облегчить программирование приложений при сохранении проверки корректности используемых типов данных.

Практически во всех случаях использования шаблонов можно обойтись и без них, однако шаблоны дают следующие преимущества:

- Облегчается процесс написания программ. Вместо того, чтобы писать одинаковые функции и классы для различных типов, достаточно написать одну обобщенную функцию или один обобщенный класс
- Использование шаблонов повышает читаемость программы, поскольку обеспечивает возможность использования абстрактных типов данных
- Шаблоны обеспечивают проверку типов используемых данных. Поскольку типы данных, с которыми работает данный шаблон становятся известными при компиляции программы, компилятор может проверить используемые в функции типы данных на совместимость

Во многих отношениях шаблоны аналогичны макросам условной компиляции, заменяющим шаблон переменной указанным типом. Однако имеются существенные различия между макросом, подобным приведенному ниже:

```
#define min(i, j) (((i) < (j)) ? (i) : (j))
```

и следующим шаблоном:

```
template<class T> T min (T i, T j) { return ((i < j) ? i : j) }
```

При работе с макросом возникнут следующие проблемы:

- Транслятор не может проверить совместимость типов аргументов макроса. Он выполняется без специальной проверки типов аргументов
- Переменные `i` и `j` оцениваются дважды. Если, например, любой из этих аргументов получает приращение, то данная операция будет выполнена дважды
- Поскольку макросы раскрываются препроцессором, то ошибки трансляции будут указаны для операторов раскрытого макроса, а не для самого макроса. При отладке макрос будет представлен в раскрытом виде, что затруднит сопоставление исходного текста программы с ее отладочным текстом

Многие функции, использующие указатель на переменные типа `void`, могут быть записаны при помощи шаблонов, поскольку пустые указатели часто используются, чтобы позволить функции работать с данными неизвестного типа. При использовании пустых указателей компилятор не может определить тип передаваемых аргументов и, следовательно, не может проверить совместимость типов используемых аргументов при перегрузке операторов, в конструкторах и деструкторах классов.

Применение шаблонов позволяет создавать функции и классы, работающие с типизированными данными. При написании функций они имеют абстрактные типы, которые преобразуются в конкретные типы в процессе компиляции. При этом для каждого типа аргумента, используемого при вызове данной функции или класса в файле, содержащем их описание, создается версия данной функции или класса, использующая аргументы этого типа.

Такой подход позволяет транслятору рассматривать функции и классы, использующие абстрактные типы, как прототипы функций или классов при создании их версий для конкретных типов данных, используемых в данном приложении. Использование шаблонов улучшает читаемость функций, поскольку позволяет не создавать специальных версий данных функций для сложных типов, таких как объекты классов или структур.

Visual C++ позволяет создавать классы "интеллектуальных указателей", позволяющих перегружать операции над указателями для повышения удобства работы с ними. Использование шаблонов позволяет разработать единый подход к работе с указателями практически любого типа.

Приведенный ниже пример подсчитывает количество ссылок на данный объект для определения момента, когда его необходимо удалить из памяти. Класс `Ptr<T>` представляет собой указатель на любой класс, производный от класса `RefCount`:

```
#include <stdio.h>

#define TRACE printf

class RefCount {
    int crefs;
public:
    RefCount(void) { crefs = 0; }
    ~RefCount() { TRACE("goodbye(%d)\n", crefs); }
    void upcount(void) { ++crefs; TRACE("up to %d\n", crefs);}
    void downcount(void)
     {
     if (--crefs == 0)
      {
      delete this;
      }
     else
      TRACE("downto %d\n", crefs);
     }
};

class Sample : public RefCount {
public:
    void doSomething(void) { TRACE("Did something\n");}
};

template <class T> class Ptr {
    T* p;
public:
    Ptr(T* p_) : p(p_) { p->upcount(); }
    ~Ptr(void) { p->downcount(); }
    operator T*(void) { return p; }
    T& operator*(void) { return *p; }
    T* operator->(void) { return p; }
    Ptr& operator=(Ptr<T> &p_)
        {return operator=((T *) p_);}
    Ptr& operator=(T* p_) {
        p_->downcount(); p = p_; p->upcount(); return *this;
    }
};

int main() {
    Ptr<Sample> p  = new Sample; // sample #1
```

```
    Ptr<Sample> p2 = new Sample; // sample #2
    p = p2; // Величина crefs указателя p становится равной 0,
            // следовательно он уничтожается;
            // величина crefs указателя p2 становится равной 2.
    p->doSomething();
    return 0;
    // Поскольку указатели p2 и p вышли за свою область действия,
    // вызывается их деструктор, уменьшающий значение счетчика.
    // Значение величины cref указателя p2 становится равным 0,
    // следовательно он уничтожается;
}
```

Классы `RefCount` и `Ptr<T>` совместно образуют простейший сборщик мусора для любого класса. Пример взят из библиотеки MSDN, поэтому его работоспособность не гарантирована.

Шаблоны функций

Шаблоны функций используются для замены набора функций, имеющих один и тот же текст, но работающих с аргументами, имеющими различный тип. Пример шаблона функции приведен ниже:

```
template <class T> void MySwap( T& a, T& b)
{
    T c( a);
    a = b; b = c;
}
```

Эта функция позволяет своим аргументам обмениваться значениями. Данный шаблон может использоваться для создания функций, аргументами которых могут быть не только переменные типов `int` и `long`, но и объекты созданных пользователем типов. Функция `MySwap` может иметь в качестве своих аргументов даже объекты классов, если в них определены конструктор, аргументом которого является объект данного класса, и оператор присваивания.

Кроме того, шаблон функции не позволит производить обмен значениями между объектами разных типов, поскольку компилятор имеет информацию о типе переменных `a` и `b` на момент трансляции.

Вызов функции, использующей шаблон, практически ничем не отличается от вызова нормальной функции. Например:

```
int i, j;
char k;
MySwap( i, j);      // OK
MySwap( i, k);      // Ошибка. Использованы различные типы аргументов.
```

Возможно явное задание типа аргументов в шаблоне функции. Например:

```
template<class T> void f(T) {...}
void g(char j) {
   f<int>(j);   //Создает функцию f(int)
}
```

В случае явного задания типа аргумента при необходимости производится соответствующее преобразование типов ее аргументов. В приведенном выше примере переменная `j`, имеющая тип `char`, будет преобразована к типу `int`.

В Visual C++ версии 5.0 и более поздних может использоваться новый синтаксис для явного задания типов в шаблонах функций. Например:

```
template<class T> void f(T t) {...};

//Явное задание типа аргумента char в функции f:
template<> void f<char>(char c){...}

// Другая форма явного задания типа аргумента double в функции:
template<> void f(double d) {...}
```

Может быть использован и старый синтаксис:

```
// Старая форма задания типа аргумента char в функции:
void f(char) {...}
```

При первом вызове функции, использующей шаблон, с новым типом аргумента транслятор создает копию данной функции, в которой абстрактный тип заменяется на конкретный. Если в различных модулях создается несколько идентичных версий функции с одинаковыми параметрами, в исполняемом файле останется только одна из них.

Если тип аргумента функции не задан явно в шаблоне функции, как это было описано выше, преобразование абстрактного типа аргумента к другому типу невозможно. В приведенном ниже примере:

```
template<class T> void f(T, int){ };

int i; char c;
f(i, c);
```

преобразование переменной `i` к типу `T` может вызвать ошибку, однако допустимо преобразование переменной `c` к типу `int`.

Шаблоны классов

Шаблоны классов используются для замены набора классов, имеющих один и тот же набор переменных и функций, но в котором эти переменные,

аргументы и возвращаемые значения функций имеют различный тип. Пример шаблона класса приведен ниже:

```
template <class T, int i> class TempClass
{
public:
    TempClass( void);
    ~TempClass( void);
    int MemberSet( T a, int b);
private:
    T Tarray[i];
    int arraysize;
};
```

В этом примере шаблон класса имеет два параметра, первый из которых имеет тип `T`, а второй является целочисленной переменной `i`. Параметру `T` может быть передано значение любого типа, включая структуру или класс. Параметру `i` может быть передано значение любой целочисленной константы. Поскольку переменная `i` представляет собой константу, определяемую в процессе компиляции, пользователь может создать массив с размерностью `i`, используя стандартное объявление массива.

Объявление членов класса, использующего шаблон, несколько отличается от объявления членов класса, не использующих шаблон. Так, текст функции `MemberSet`, *являющейся членом описанного выше класса*, имеет вид:

```
template <class T, int i>
    int TempClass< T, i >::MemberSet( T a, int b)
    {
        if( ( b >= 0) && (b < i))
        {
           Tarray[b++] = a;
           return sizeof( a);
        }
        else
           return -1;
    }
```

Хотя в конструкторах и деструкторах имя шаблона класса упоминается дважды, параметры шаблона должны полностью указываться только один раз:

```
template <class T, int i>
TempClass< T, i >::TempClass( void)
{
    TRACE( "TempClass created.\n");
}
```

```
template <class T, int i>
TempClass< T, i >::~TempClass( void)
{
    TRACE( "TempClass destroyed.\n");
}
```

В отличие от шаблонов функций, при создании конкретных версий классов пользователь должен в явном виде указать передаваемые аргументы шаблона класса. Для создания объекта класса `TempClass` могут быть использованы следующие операторы:

```
TempClass< float, 6 > test1;        // OK
TempClass< char, items++ > test2;   // Ошибка, второй оператор
                                    // должен быть константой.
```

Для шаблона класса (или функции) не создается никакого исполнительного кода, пока транслятор не встретит объявления данного класса с конкретными параметрами. Более того, исполнительный код для функций-членов класса создается только в том случае, если функция вызывается в данной версии класса. Шаблон класса сначала определяется, а затем для него создается необходимая версия класса. Компилятор не создает версии класса для данного типа аргументов, пока этот класс не будет вызван с этим типом аргументов.

В Visual C++ версии 5.0 и более поздних может использоваться новый синтаксис для явного задания типов в шаблонах классов. Например:

```
template<class T> class X {...};

//Явное задание параметра int в шаблоне класса X
template<> class X<int> { };
```

Может быть использован и старый синтаксис:

```
//Задание типа char для класса X
class X<char> { };
```

При работе с шаблонами классов необходимо обращать особое внимание на положение угловых скобок (<>). Необходимо таким образом разместить круглые скобки в выражениях, чтобы за угловые скобки не могли быть ошибочно приняты такие операторы, как >> и ->. Например, выражение

```
TempClass< float, a > b ? a : b > test1;
```

должно быть заменено на выражение

```
TempClass< float, (a > b ? a : b) > test1;
```

Кроме того, необходимо обратить внимание на макросы, использующие угловые скобки в качестве аргументов шаблонов.

Как уже отмечалось, чтобы в программе появилась версия класса, использующая данный тип аргументов, необходимо, чтобы транслятор встретил объявление данного класса с данным типом аргумента. Поскольку трансляция приложения производится по файлам, необходимо, чтобы текст функций данного класса и его вызов располагались в одном файле. В противном случае данная версия класса не будет создана, а при вызове ее из другого файла компоновщик выдаст соответствующее сообщение об ошибке.

Для исключения подобных ситуаций создан механизм, позволяющий указать транслятору, какие типы аргументов будут использоваться в данном классе. Например, объявление класса `MyStack` в его файле реализации создаст версию класса, работающую с целочисленными переменными и способного хранить шесть переменных:

```
template class MyStack<int, 6>;
```

Это выражение создает соответствующую версию класса, не создавая самого объекта данного класса. При этом программный код генерируется для всех функций класса. Реализация этого метода в Visial C++ не лишена некоторых недоработок. Например, если в файле реализации некоторого класса, использующего шаблон, некоторая функция или класс вызывает некоторые его функции, а пользователю необходимо обеспечить работу со всеми функциями данного класса в других файлах проекта, то применение данного механизма вызовет при трансляции приложения предупреждение (warning) о том, что данный класс уже определен. Это предупреждение приходится игнорировать.

Если необходимо создать программный код только для конструктора данного класса, используется следующее выражение:

```
template MyStack<int, 6>::MyStack( void);
```

Использование ключевого слова `extern` позволяет избежать автоматического создания всех функций-членов данного класса. Например:

```
extern template class MyStack<int, 6>;
```

Аналогично, в качестве внешних могут быть объявлены отдельные функции данного класса:

```
extern template MyStack<int, 6>::MyStack( void);
```

Ключевое слово `extern` может быть применено только к тем функциям, которые объявлены за пределами класса. Функции, объявленные внутри класса считаются встроенными и для них всегда создается программный код.

Классы коллекций

Библиотека MFC предлагает широкий выбор классов для работы с группами объектов. Эти классы могут быть условно разделены на две группы:

- Классы, созданные с использованием шаблонов
- Классы, созданные без использования шаблонов

Последняя группа классов представлена для обеспечения совместимости со старыми программами, поскольку она используется в библиотеке MFC, начиная с версии 1.0.

Виды классов коллекций

Классы коллекций различаются между собой по видам и по типу элементов. Вид класса определяет способ организации и хранения элементов в классе. В библиотеке MFC содержатся классы трех видов: списков, массивов и карты отображений (иногда называемые словарями). Это позволяет выбрать способ хранения информации, максимально соответствующий поставленной задаче.

Краткое описание каждого вида классов приведено ниже:

- *Списки.* Списки представляют собой упорядоченный, неиндексированный набор элементов, организованных в связанный список. В списке возможен непосредственный доступ к его первому, последнему и текущему элементам. Чтобы найти остальные элементы, необходимо произвести последовательный поиск, основываясь на хранящихся в каждом элементе списка указателях на предыдущий и последующий элементы. Эта структура позволяет сравнительно просто добавлять элементы в любую позицию списка
- *Массивы.* Массивы представляют собой упорядоченный набор объектов, каждому из которых соответствует целочисленный индекс. В процессе выполнения программы наборы объектов могут пополняться
- *Карты отображений* (словари). В данном наборе объектов для доступа к хранящемуся объекту используется ключевой объект

Простейшим способом создания класса коллекции, хранящего объекты любого типа, которые проверяют соответствие типа хранимых объектов, являются стандартные классы коллекций библиотеки MFC, использующие шаблоны. Список этих классов коллекций приведен в табл. 11.1.

Таблица 11.1. *Классы коллекций, использующие шаблоны*

Коллекции	Массив	Список	Карта отображений
Коллекции объектов любого типа	`CArray`	`CList`	`CMap`
Коллекции указателей на объекты любого типа	`CTypedPtrArray`	`CTypedPtrList`	`CTypedPtrMap`

Если в приложении есть классы коллекций, не использующие шаблоны, то нет необходимости в преобразовании их в новый стандарт, однако новые классы коллекций лучше создавать при помощи шаблонов. Список классов коллекций, не использующих шаблоны, приведен в табл. 11.2.

Таблица 11.2. *Классы коллекций, не использующие шаблоны*

Массив	Список	Карта отображений
CObArray	CObList	CMapPtrToWord
CByteArray	CPtrList	CMapPtrToPtr
CDWordArray	CStringList	CMapStringToOb
CPtrArray		CMapStringToPtr
CStringArray		CMapStringToString
CWordArray		CMapWordToOb
CUIntArray		CMapWordToPtr

При выборе класса коллекции для приложения необходимо сопоставить характеристики различных видов классов и другие особенности их реализации. Эти характеристики включают в себя:

- Особенности различных видов классов, такие как порядок расположения объектов, индексирование и производительность. Эти параметры приведены в табл. 11.3
- Использование шаблонов C++
- Возможность работы с архивами для сохранения объектов данных классов
- Возможность диагностики хранимых объектов
- Проверка типов при работе с данным классом

В табл. 11.3 приведены характеристики классов коллекций. Столбцы 2 и 3 описывают упорядоченность и возможность доступа по индексу к хранящемуся объекту. Под упорядоченностью понимается возможность введения понятия предыдущего и последующего объекта в наборе.

Столбцы 4 и 5 описывают скорость выполнения определенных операций с объектами данных классов. Важность каждого из этих параметров зависит от конкретного приложения.

Таблица 11.3. *Виды классов коллекций*

Вид класса	Упорядочен?	Индексируется?	Скорость вставки элемента	Скорость поиска элемента
Список	Да	Нет	Быстрая	Медленная

Таблица 11.3 *(окончание)*

Вид класса	Упорядочен?	Индексируется?	Скорость вставки элемента	Скорость поиска элемента
Массив	Да	Целыми числами	Медленная	Быстрая
Карта отображений	Да	По ключу	Быстрая	Быстрая

В табл. 11.4 приведены характеристики различных классов коллекций, содержащихся в библиотеке MFC.

Таблица 11.4. *Характеристики различных классов коллекций*

Класс	Использует шаблоны C++	Может работать с архивом	Имеет встроенную диагностику	Проверяет тип данных
CArray	Да	Да[1]	Да[1]	Нет
CByteArray	Нет	Да	Да	Да[3]
CDWordArray	Нет	Да	Да	Да[3]
CList	Да	Да[1]	Да[1]	Нет
CMap	Да	Да[1]	Да[1]	Нет
CMapPtrToPtr	Нет	Нет	Да	Нет
CMapPtrToWord	Нет	Нет	Да	Нет
CMapStringToOb	Нет	Да	Да	Нет
CMapStringToPtr	Нет	Нет	Да	Нет
CMapStringToString	Нет	Да	Да	Да[3]
CMapWordToOb	Нет	Да	Да	Нет
CMapWordToPtr	Нет	Нет	Да	Нет
CObArray	Нет	Да	Да	Нет
CObList	Нет	Да	Да	Нет
CPtrArray	Нет	Нет	Да	Нет
CPtrList	Нет	Нет	Да	Нет
CStringArray	Нет	Да	Да	Да[3]
CStringList	Нет	Да	Да	Да[3]
CTypedPtrArray	Да	Условно[2]	Да	Да

Таблица 11.4 *(окончание)*

Класс	Использует шаблоны C++	Может работать с архивом	Имеет встроенную диагностику	Проверяет тип данных
CTypedPtrList	Да	Условно[2]	Да	Да
CTypedPtrMap	Да	Условно[2]	Да	Да
CUIntArray	Нет	Нет	Да	Да[3]
CWordArray	Нет	Нет	Да	Да[3]

[1] Для использования архива необходимо непосредственно вызвать для объекта класса коллекции функцию Serialize. Для вывода диагностического сообщения необходимо непосредственно вызвать функцию Dump. Нельзя использовать выражения ar << collObj для работы с архивом или dmp << collObj для вывода диагностических сообщений.

[2] Возможность работы с архивом зависит от используемого класса коллекции. Например, если массив типизированных указателей основан на классе CObArray, то он может работать с архивом. Если он основан на классе CPtrArray, то он не может работать с архивом. Как правило, классы "Ptr" не могут работать с архивом.

[3] Значение "Да" в этом столбце означает, что класс коллекции, не использующей шаблона, проверяет только свой класс. Так, например, класс CByteArray проверяет, что он хранит байты, но при хранении в нем символов никаких дополнительных проверок не производится.

Классы коллекций, использующие шаблоны

Классы коллекций, использующие шаблоны, появились в библиотеке MFC версии 3.0 и могут применяться в последующих версиях данной библиотеки. Использование шаблонов для задания типа переменных, хранимых в классе коллекций, обеспечивает простой пользовательский интерфейс с объектами данных классов и более корректную проверку типов, чем в классах коллекций не использующих шаблонов.

Базовым классом для простых классов коллекций является класс CObject, поэтому они способны работать с архивом, могут создаваться динамически в процессе выполнения программы и обладают другими свойствами класса CObject. Классы коллекций указателей на объекты требуют от пользователя указать их базовый класс, в качестве которого может выступать любой класс коллекций указателей на объекты, не использующий шаблонов, из определенных в библиотеке MFC. В качестве базовых могут выступать такие классы, как CPtrList или CPtrArray. Новый класс коллекций наследует функции своего базового класса, однако при их вызове производится проверка используемых объектов классов на совместимость.

Для использования простых классов коллекций необходимо знать тип данных, хранимых в этих классах, и параметры, которые следует указать при объявлении данных классов.

Простые классы массивов и списков, CArray и CList, имеют два параметра: TYPE и ARG_TYPE. В этих классах могут храниться любые данные, имеющие тип, задаваемый параметром TYPE:

- Базовые типы данных C++, такие, как int, char и float
- Структуры и классы C++
- Другие, определяемые пользователем типы

Для простоты и повышения эффективности работы с данным классом пользователь может в параметре ARG_TYPE задать типы аргументов функций. Обычно в параметре ARG_TYPE задается ссылка на тип, заданный в параметре TYPE. Например:

```
CArray<int, int> myArray;
CList<CPerson, CPerson&> myList;
```

В первом примере объявлен целочисленный массив myArray, а во втором — список myList, хранящий объекты класса CPerson. Некоторые функции-члены класса коллекции имеют аргументы, тип которых определяется значением параметра ARG_TYPE. Ниже приведен пример использования функции CArray::Add.

```
CArray<CPerson, CPerson&> myArray;
CPerson person;
myArray->Add( person);
```

Простые классы карт отображений CMap имеют четыре параметра: KEY, ARG_KEY, VALUE и ARG_VALUE. Классы карт отображений позволяют хранить любые типы данных. Они представляют собой массивы, в которых доступ к элементу возможен по любому ключу, а не только по целочисленному. Параметр KEY определяет тип данных, используемых в качестве ключей для доступа к хранящимся в объекте класса данным. Если параметр KEY определяет структуру или класс, то в параметре, обычно ARG_KEY, задается ссылка на тип данных, заданный в параметре KEY. Параметр VALUE определяет тип данных, хранящихся в объекте данного класса. Если параметр VALUE определяет структуру или класс, то в параметре ARG_VALUE задается ссылка на тип данных, заданный в параметре VALUE. Например:

```
CMap< int, int, MY_STRUCT, MY_STRUCT& > myMap1;
CMap< CString, LPCSTR, CPerson, CPerson& > myMap2;
```

В первом примере в переменной myMap1 хранятся объекты структуры MY_STRUCT, доступ к которым осуществляется по целочисленным ключам. Во втором примере в переменной myMap2 хранятся объекты класса CPerson, доступ к которым осуществляется по ключам, представляющим собой объекты класса CString, причем при запросе объекта, хранящегося в объекте данного класса, возвращается ссылка на этот объект. В качестве ключа в

данном примере может выступать фамилия человека, а в качестве хранимых данных — сведения о нем.

Поскольку параметр KEY имеет значение CString, а параметр ARG_KEY — значение LPCSTR, то ключи хранятся в карте отображения как объект класса CString, но в аргументах функций, таких как SetAt, соответствующие аргументы имеют тип LPCSTR. Например:

```
CMap< CString, LPCSTR, CPerson, CPerson& > myMap2;
CPerson person;
LPCSTR lpstrName = "Jones";
myMap2->SetAt( lpstrName, person);
```

Для использования шаблонов в классах коллекций указателей на объекты, как и в случае простых классов коллекций, необходимо знать тип данных, хранимых в данных классах, и параметры, которые необходимо указать при их объявлении.

Классы массивов и списков указателей на объекты CTypedPtrArray и CTypedPtrList имеют два параметра: BASE_CLASS и TYPE. В этих классах могут храниться любые данные, имеющие тип, задаваемый параметром TYPE. Они являются производными от одного из классов коллекций, не использующих шаблоны. Базовый класс задается в параметре BASE_CLASS. Для массивов в качестве базовых используются классы CObArray или CPtrArray, а для списков — CObList или CPtrList. При этом, если в качестве базового класса используется класс CObList, новый класс не только наследует члены базового класса, но также объявляет несколько дополнительных функций или операторов, позволяющих проверять корректность используемых типов переменных при доступе к функциям и переменным базового класса. При этом допустимо любое разрешенное преобразование типов. Например:

```
CTypedPtrArray<CObArray, CPerson*> myArray;
CTypedPtrList<CPtrList, MY_STRUCT*> myList;
```

В первом примере базовым для массива myArray является класс CObArray. В массиве хранятся указатели на объекты класса CPerson (причем класс CPerson является производным от класса CObject). При работе с данным массивом можно использовать любые функции-члены класса CObArray или использовать новые функции GetAt и ElementAt, проверяющие тип своих аргументов, или оператор [], также проверяющий тип.

Во втором примере объявляется список указателей myList, базовым для которого является класс CPtrList. В списке хранятся указатели на объекты структуры MY_STRUCT. Класс, базовым для которого является класс CPtrList, используется для хранения указателей на объекты, для которых класс CObject не является базовым. Класс CTypedPtrList содержит следующие функции, проверяющие тип своих аргументов: GetHead, GetTail, RemoveHead, RemoveTail, GetNext, GetPrev и GetAt.

Класс карты отображений указателей на объекты CTypedPtrMap имеет три параметра шаблона: BASE_CLASS, KEY и VALUE. Параметр BASE_CLASS определяет базовый класс данного класса: CMapPtrToWord, CMapPtrToPtr, CMapStringToPtr, CMapWordToPtr, CMapStringToOb и т. д. Параметр KEY аналогичен параметру KEY в классе CMap: он определяет тип переменной, используемой в качестве ключа для поиска информации. Параметр VALUE аналогичен параметру VALUE в классе CMap: он определяет тип объекта, хранимого в карте отображений. Например:

```
CTypedPtrMap<CMapPtrToPtr, CString, MY_STRUCT*> myPtrMap;
CTypedPtrMap<CMapStringToOb, CString, CMyObject*> myObjectMap;
```

Первый пример представляет собой объект класса карты отображений, производного от класса CMapPtrToPtr, использующего в качестве ключей объекты класса CString и хранящего указатели на объекты структуры MY_STRUCT. Для получения указателя по ключу может быть использована функция Lookup, проверяющая тип своих аргументов. Оператор [] позволяет получить хранимый указатель и добавить его, если он не найден. При поиске в карте отображений может быть использована функция GetNextAssoc, проверяющая тип своих аргументов.

Второй пример представляет собой объект класса карты отображений, производного от класса CMapStringToOb, использующего в качестве ключей объекты класса CString и хранящего указатели на объекты класса CMyObject. При работе с данным классом могут быть использованы те же функции, что и в предыдущем примере.

Если указанный в параметре тип переменной является class или struct, а не указатель или ссылка на данный тип, этот класс или структура должны иметь копирующий конструктор.

Классы CArray, CList и CMap используют семь глобальных функций, которые можно перегрузить в производных от них классах. Три из этих функций используются для создания, удаления и сохранения элементов коллекций.

Функции ConstructElements и DestructElements вызываются членами класса для добавления или удаления элемента в коллекцию. В табл. 11.5 показано, какие функции-члены класса вызывают данные глобальные функции.

***Таблица 11.5.** Использование функций ConstructElements и DestructElements*

Глобальная функция	Вызывается непосредственно	Вызывается опосредованно
ConstructElements	CArray::SetSize CArray::InsertAt	CList::AddHead CList::AddTail CList::InsertBefore CList::InsertAfter CMap::operator [ ]

Таблица 11.5 *(окончание)*

Глобальная функция	Вызывается непосредственно	Вызывается опосредованно
DestructElements	CArray::SetSize CArray::RemoveAt CList::RemoveAll CMap::RemoveAll	CList::RemoveHead CList::RemoveTail CList::RemoveAt CMap::RemoveKey

Данные функции перегружаются в том случае, когда их поведение по умолчанию не соответствует требованиям пользовательского класса. По умолчанию функция ConstructElements обнуляет всю область памяти, выделенную для данного элемента и вызывает конструктор для каждого элемента. По умолчанию функция DestructElements вызывает деструктор для каждого элемента.

В большинстве случаев перегрузка функции ConstructElements бывает необходимой, если при сохранении элемента в классе необходимо вызвать конструктор (или другую функцию инициализации) для самого элемента или для одного из его членов. Перегрузка функции DestructElements осуществляется в том случае, когда при уничтожении объекта требуется произвести дополнительные действия, такие как освобождение памяти в куче. Пример перегрузки функции ConstructElements приведен ниже:

```
class CPerson : public CObject { . . . };
CArray< CPerson, CPerson& > personArray;

template <> void AFXAPI ConstructElements <CPerson>
                                    ( CPerson* pNewPersons, int nCount)
{
    for ( int i = 0; i < nCount; i++, pNewPersons++)
    {
        // Непосредственный вызов конструктора класса CPerson
        new( pNewPersons)CPerson;
    }
}
```

Перегруженная функция проходит по всем объектам класса CPerson и для каждого из них вызывает конструктор. Вместо того, чтобы создавать новый объект класса CPerson в куче, для его создания используется специальный оператор new.

Классы CArray, CList и CMap вызывают функцию SerializeElements для записи хранящихся в них элементов на диск и чтения их с диска. По умолчанию функция SerializeElements выполняет побитовую запись объектов в архив и побитовое чтение их из архива.

Если класс коллекций является производным от класса CObject и при реализации класса используется макрос IMPLEMENT_SERIAL, то процесс программирования чтения и записи информации существенно упрощается:

```
class CPerson : public CObject { . . . };
CArray< CPerson, CPerson& > personArray;

template <> void AFXAPI SerializeElements <CPerson>
                        ( CArchive& ar, CPerson* pNewPersons, int nCount)
{
    for ( int i = 0; i < nCount; i++, pNewPersons++)
    {
        // Запись и чтение каждого объекта класса CPerson
        pNewPersons->Serialize( ar);
    }
}
```

Классы коллекций, не использующие шаблоны

Библиотека MFC содержит также классы коллекций, не использующие шаблоны, эти классы содержались еще в первой версии данной библиотеки. Они используются для хранения данных, имеющих типы CObject*, UINT, DWORD и CString. Кроме того, могут быть использованы классы коллекций объектов классов, производных от класса CObject (такие как CObList). Кроме того, библиотека MFC предоставляет классы коллекций для хранения таких простейших типов, как UINT и пустые указатели (void*).

Существует два пути использования классов коллекций, не использующих шаблоны для надежного хранения данных:

- Использование в случае необходимости преобразования типов
- Создание производных классов на их основе

Чтобы использовать преобразование типов при работе с классами коллекций, не использующими шаблонов:

- Используйте данные классы, такие как CWordArray, непосредственно. Например, класс CWordArray позволяет хранить любые 32-разрядные значения. Они могут быть преобразованы к любому нужному типу. Класс CObList позволяет хранить указатели на объекты класса CObject. После извлечения указателя на объект из списка он может быть преобразован к любому совместимому типу. Например, если в объекте класса CObList хранится указатель на объект класса CPerson, то полученный из списка указатель необходимо преобразовать к типу CPerson*. Пример данного преобразования приведен ниже:

```
class CPerson : public CObject {...};

CPerson* p1 = new CPerson(...);
CObList myList;

myList.AddHead( p1);   // No cast needed
CPerson* p2 = ( CPerson*)myList.GetHead();
```

Такое преобразование позволяет добиться удовлетворительного результата во многих практических случаях. Если необходимо добавить в класс дополнительные функции или обеспечить более жесткую проверку типов используемых переменных, задайте шаблоны при помощи типов или в соответствии с описанной ниже процедурой.

- Создайте собственный класс коллекций на основе одного из существующих классов, не использующих шаблоны. При создании собственного класса коллекций пользователь может добавить в него функции проверки типов данных при доступе к функциям базового класса. Например, при создании на базе класса `CObList` пользовательского класса, хранящего указатели на объект класса `CPerson`, в него могут быть добавлены функции `AddHeadPerson` и `GetHeadPerson`, как это показано ниже:

```
class CPersonList : public CObList
{
public:
    void AddHeadPerson( CPerson* person)
        {AddHead( person);}

    const CPerson* GetHeadPerson()
        {return (CPerson*)GetHead();}
};
```

Эти функции обеспечивают проверку типа аргумента и преобразование возвращаемого значения к типу `CPerson`.

В данный класс могут быть добавлены и новые функции, расширяющие возможности базового класса.

Доступ к элементам классов коллекций

Классы массивов, определенные в библиотеке MFC, как основанные на шаблонах, так и нет, используют целочисленные индексы для доступа к своим элементам. Классы списков и карт отображений используют индикатор типа `POSITION` для указания позиции внутри коллекции. Для доступа к одному или нескольким элементам данной коллекции необходимо сначала установить позицию индикатора, а затем несколько раз передать это значение объекту класса коллекции и запросить следующий элемент. Объекты класса коллекций не отслеживают информацию о ходе поиска. Эта инфор-

мация хранится в индикаторе позиции. Но она хранит информацию об элементах, соседних с указанным.

Для поиска информации в массиве используется функция GetAt. Например:

```
CTypedPtrArray<CObArray, CPerson*> myArray;

for( int i = 0; i < myArray.GetSize();i++)
{
    CPerson* thePerson = myArray.GetAt( i);
    ...
}
```

В этом примере определен массив указателей на объект класса CPerson. Этот массив является производным от класса CObArray, не использующего шаблоны. Функция GetAt возвращает указатель на объект класса CPerson. Класс CTypedPtrArray содержит также перегруженный оператор [], позволяющий использовать привычный синтаксис работы с массивами. При помощи этого оператора выражение в цикле можно записать следующим образом:

```
CPerson* thePerson = myArray[ i ];
```

Этот оператор существует как в const, так и в обычной версиях. Оператор версии const, используемой в массивах констант, может использоваться только в правой части оператора присваивания.

Для поиска информации в списке используются функции GetHeadPosition и GetNext. Пример их использования приведен ниже:

```
CTypedPtrList<CObList, CPerson*> myList;

POSITION pos = myList.GetHeadPosition();
while( pos != NULL)
{
    CPerson* thePerson = myList.GetNext( pos);
    ...
}
```

В этом примере определен список указателей на объект класса CPerson. Объявление списка во многом аналогично объявлению массива, рассмотренному выше, но в качестве базового используется класс CObList. Функция GetNext возвращает указатель на объект класса CPerson.

Для поиска информации в карте отображений при неизвестном значении ключа для начальной установки индикатора используется функция GetStartPosition, а функция GetNextAssoc — для последовательного доступа к элементам. Например:

```
CMap<CString, LPCTSTR, CPerson*, CPerson*> myMap;

POSITION pos = myMap.GetStartPosition();
```

```
while( pos != NULL)
{
    CPerson* pPerson;
    CString string;
    // Получаем значение ключа ( string) и элемента ( pPerson)
    myMap.GetNextAssoc( pos, string, pPerson);
    // Используем полученные элемент и ключ
}
```

В этом примере рассмотрен простой шаблон класса карты отображений (вместо класса типизированных указателей на объекты), использующий в качестве ключей объекты класса `CString` и хранящий объекты класса `CPerson`. При использовании функций доступа, таких как `GetNextAssoc`, объект класса возвращает указатели на объекты класса `CPerson`. Если при этом класс карты отображений не использует шаблона, то возвращаемый данной функцией указатель на объект класса `CObject` должен быть преобразован в указатель на объект класса `CPerson`. Пример использования подобного класса приведен ниже. Следует обратить внимание, что второй аргумент функции `GetNextAssoc` преобразуется к типу `CObject*&`.

```
CMapStringToOb myMap;     // Класс коллекции, не использующий шаблоны.

POSITION pos = myMap.GetStartPosition();
while( pos != NULL)
{
    CPerson* pPerson;
    CString string;
    // Получаем значение ключа ( string) и элемента ( pPerson)
myMap.GetNextAssoc( pos, string, (CObject*&)pPerson);
    ASSERT( pPerson->IsKindOf( RUNTIME_CLASS( CPerson)));
    // Используем полученные элемент и ключ
}
```

Чтобы получить информацию по соответствующему ключу, используется функция `Lookup`, первый аргумент которой представляет собой ключ, по которому необходимо получить информацию, а второй — ссылку на соответствующий элемент в карте отображения. Для этой цели может использоваться и перегруженный оператор `[ ]`.

Удаление элементов классов коллекций

Чтобы удалить все элементы в классе коллекции, хранящей объект класса `CObject` (или объекты классов производных от класса `CObject`), можно получить доступ к каждому элементу данного класса и уничтожить его. Однако элементы в классе коллекций могут разделяться в том случае, когда коллекция хранит указатели на объекты. Если другие элементы коллекции или

другие переменные программы хранят указатели на те же самые объекты, то их удаление без соответствующей проверки может привести к непредсказуемым результатам.

Чтобы уничтожить все элементы списка:

1. Используйте функции `GetHeadPosition` и `GetNext` для последовательного получения доступа ко всем элементам списка.
2. Используйте оператор `delete` для уничтожения объекта, связанного с каждым из элементов списка.
3. Вызовите функцию `RemoveAll` для удаления всех элементов списка после того, как будут уничтожены все связанные с ними объекты.

Ниже приведен пример уничтожения всех объектов класса `CPerson` из списка. Список содержит указатели на объекты класса `CPerson`, размещенные в куче:

```
CTypedPtrList<CObList, CPerson*> myList;
POSITION pos = myList.GetHeadPosition();

while( pos != NULL)
{
    delete myList.GetNext( pos);
}
myList.RemoveAll();
```

Функция `RemoveAll` удаляет все элементы из списка. Функция `RemoveAt` удаляет отдельный элемент.

Обратите внимание на различие между удалением объекта, связанного с элементом списка, и удалением самого элемента. В первом случае удаляется только содержащийся в списке указатель на данный объект, сам же объект продолжает существовать. При уничтожении объекта он прекращает свое существование и освобождает занимаемую им память. Поэтому после удаления объекта необходимо немедленно удалить все указатели на него для предотвращения их использования.

Чтобы удалить все элементы в массиве:

1. Используйте функцию `GetSize` и целочисленный индикатор для получения доступа ко всем элементам массива.
2. Используйте оператор `delete` для уничтожения объекта, связанного с каждым из элементов массива.
3. Вызовите функцию `RemoveAll` для удаления всех элементов массива после того, как будут уничтожены все связанные с ними объекты.

Ниже приведен пример уничтожения всех объектов класса `CPerson` в массиве. Массив содержит указатели на объекты класса `CPerson`, размещенные в куче:

```
CArray<CPerson*, CPerson*> myArray;

int i = 0;
while (i < myArray.GetSize())
{
    delete myArray.GetAt( i++);
}

myArray.RemoveAll();
```

Как и при работе со списком, функция `RemoveAll` удаляет все элементы массива, а функция `RemoveAt` — отдельный элемент.

Чтобы удалить все элементы в карте отображений:

1. Используйте функции `GetStartPosition` и `GetNextAssoc` для получения доступа ко всем элементам карты отображений.
2. Используйте оператор `delete` для уничтожения ключа и/или объекта, связанного с каждым из элементов карты отображений.
3. Вызовите функцию `RemoveAll` для удаления всех элементов карты отображений после того, как будут уничтожены все связанные с ними объекты.

Ниже приведен пример уничтожения всех элементов в объекте класса `CMap`. Каждый элемент карты отображений имеет строковый ключ и хранит объект класса `CPerson` (производного от класса `CObject`):

```
CMap<CString, LPCSTR, CPerson*, CPerson*> myMap;
// ... Добавьте в карту отображений несколько элементов ...
// Теперь уничтожьте элементы
POSITION pos = myMap.GetStartPosition();
while( pos != NULL)
{
    CPerson* pPerson;
    CString string;
    // Получение значения ключа (string) и объекта (pPerson)
    myMap.GetNextAssoc( pos, string, pPerson);
    delete pPerson;
}
// Функция RemoveAll уничтожает все ключи
myMap.RemoveAll();
```

Функция `RemoveAll` удаляет все элементы в карте отображения, а функция `RemoveKey` — отдельный элемент по ключу.

Использование классов коллекций

Классы коллекций могут использоваться для создания различных полезных структур данных. Ниже будут приведены примеры использования класса `CList` для создания стеков и очередей.

Поскольку стандартный список имеет голову, то этот класс может быть легко использован для создания *стека*. Стек иногда называют магазинной памятью по аналогии с магазином стрелкового оружия, где последний вставленный патрон используется первым. Для добавления элементов в стек может использоваться функция AddHead, а для удаления элементов из стека — функция RemoveHead.

Чтобы создать класс стека, создайте новый класс, производный от существующего класса списка библиотеки MFC, и добавьте в него дополнительные функции, отвечающие за функционирование стека. Ниже приведен пример такого класса:

```
class CBullet : public CObject { ... };

class CStack : public CTypedPtrList< CObList, CBullet* >
{
public:
    // Добавление элемента в вершину стека
    void Push( CBullet* newBullet)
        { AddHead( newBullet); }

    // Чтение элемента из стека
    CBullet* Peek()
        { return IsEmpty() ? NULL : GetHead(); }

    // Удаление элемента из стека
    CBullet* Pop()
        { return RemoveHead(); }
};
```

В данном примере использованы функции-члены класса CObList, доступ к которым имеется у пользователя независимо от того, имеют они отношение к стеку или нет.

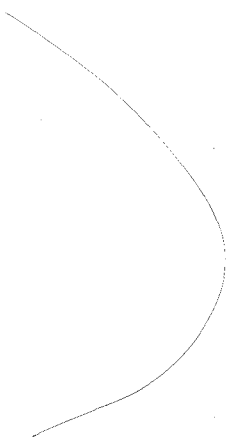
Поскольку стандартный список имеет как голову, так и хвост, то этот класс может быть легко использован для создания очереди. Принцип работы очереди очевиден: первым пришел — первым обслужили. Новые объекты добавляются в конец очереди, поэтому для добавления элементов в очередь может использоваться функция AddTail, а для удаления элементов из очереди — функция RemoveHead.

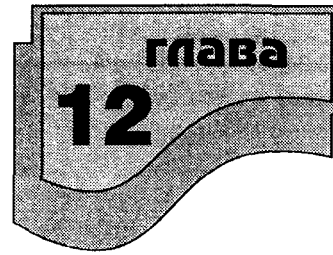
Чтобы создать класс очереди, создайте новый класс, производный от существующего класса списка библиотеки MFC и добавьте в него дополнительные функции, отвечающие за функционирование очереди. Ниже приведен пример такого класса:

```
class CPerson : public CObject { ... };

class CQueue : public CTypedPtrList< CObList, CPerson* >
```

```
{
public:
    // Переход в конец списка
    void AddToEnd( CPerson* newPerson)
        { AddTail( newPerson); }          // Конец очереди

    // Первый элемент в очереди
    CPerson* GetFromFront()
        { return IsEmpty() ? NULL : RemoveHead(); }
};
```

В данном случае новые элементы добавляются в хвост списка, а чтение элементов производится из его начала с одновременным удалением читаемого элемента.





Многозадачность на основе потоков Windows

Графическая оболочка Windows разрабатывалась как среда с *кооперативной многозадачностью* (cooperative multiasking) без разделения времени. Это означало, что некоторая функция, получив управление, не отдавала его до тех пор, пока не завершала свою работу, или ее работа не прерывалась извне. Если функция осуществляет достаточно сложные вычисления над большим объемом данных, подобный подход может привести к практическому зависанию системы. Эта особенность сохранилась и после превращения графической оболочки в операционную систему.

Выходом из этой ситуации является использование потоков. *Потоки* (threads) обладают свойством *вытесняющей многозадачности* (preemptive multitasking). Это означает, что каждому потоку в соответствии с его приоритетом выделяется квант времени для его выполнения. По истечении данного кванта времени выполнение процесса прерывается и вызывается процедура распределения квантов времени, которая при отсутствии других, более приоритетных претендентов, может опять выделить квант времени тому же потоку.

Приложение Win32 состоит из одного или более процессов. Под *процессом* (process) можно понимать некоторую исполняемую программу. В соответствии с другим определением под процессом понимают исполняемое приложение. Например, при двойном щелчке на значке **Блокнот** запускается процесс, в котором исполняется приложение Блокнот. В одном процессе могут быть запущены один или несколько потоков.

Поток представляет собой основную программную единицу, которой операционная система предоставляет процессорное время. В потоке может выполняться любой программный код, являющийся составной частью данного процесса, включая программный код, который в настоящее время выполняется другим процессом. Под потоком можно понимать ветвь исполнения

процесса. При запуске приложения Блокнот операционная система создает процесс и запускает на исполнение основной поток данного процесса. При завершении работы данного потока завершается и процесс. Основной поток приложения запускается операционной системой и передается ей в качестве адреса функции, запускающей данный поток. Обычно это адрес функции `main` или `WinMain`.

Пользователь может создавать в приложении дополнительные потоки, если в этом возникает необходимость. Например, требуется выделить фоновые процессы, выполнение которых необходимо для правильной работы приложения, но так, чтобы пользователь не дожидался их окончания для продолжения работы с приложением. Все потоки в приложениях, использующих библиотеку MFC, используют объекты класса `CWinThread` или производных от него классов. В большинстве случаев пользователю нет необходимости самому создавать эти объекты. Достаточно вызвать функцию `AfxBeginThread`, которая сама создаст необходимый объект и запустит поток на исполнение.

Библиотека MFC позволяет работать с двумя видами потоков: *интерфейсными* и *рабочими*. Интерфейсные потоки обычно используются для обработки команд и сообщений пользователя. Рабочие потоки — для выполнения задач, не требующих вмешательства пользователя, например, для фоновых вычислений. API Win32 не различает эти виды потоков. Ему надо только сообщить начальный адрес потока, по которому следует произвести его запуск на исполнение. Интерфейсный поток содержит собственный *цикл обработки сообщений*, позволяющий ему обрабатывать сообщения и команды пользователя. Примером класса интерфейсного потока может служить класс `CWinApp`, производный от класса `CWinThread` и осуществляющий обработку событий и сообщений, поступающих в приложение.

Особое внимание следует уделить ситуации, когда несколько потоков имеют доступ к одному и тому же объекту. Для этого необходимо произвести их синхронизацию. Методы синхронизации будут описаны ниже в данной главе. При написании и отладке приложений, в которых используется несколько потоков, необходимо убедиться, что ни к одному из имеющихся объектов приложения не может быть осуществлен одновременный доступ из нескольких потоков.

Работа с одним потоком

Создание рабочего потока

Одной из областей применения потоков является создание рабочих потоков. Они обычно используются для выполнения фоновых задач, выполнение которых занимает достаточно много времени, и в процессе выполнения которых у пользователя может появиться потребность продолжить работу

с данным приложением. К таким задачам относятся различные вычисления и фоновая печать документов.

Создание рабочего потока является довольно-таки простой задачей. Необходимо выполнить всего две операции: создать исполняющую функцию потока и запустить поток. При этом нет необходимости создавать класс, производный от класса `CWinThread`. В большинстве случаев этот класс может использоваться без перегрузки его функций, однако в некоторых случаях такая перегрузка необходима.

Для создания рабочего потока необходимо вызвать функцию `AfxBeginThread` и передать ей в качестве аргументов следующую информацию:

- Адрес исполняющей функции потока
- Значение аргумента исполняющей функции потока

Кроме того, данная функция имеет еще четыре необязательных аргумента:

- Приоритет потока. По умолчанию используется нормальный приоритет. Возможные значения приоритета перечислены в описании функции `::SetThreadPriority`
- Предполагаемый размер стека, используемого потоком. По умолчанию для создаваемого потока создается стек того же размера, что и для вызывающего
- Значение `CREATE_SUSPENDED`, если поток создается в приостановленном состоянии. По умолчанию в этом аргументе передается нулевое значение, означающее нормальное начало работы потока
- Атрибуты безопасности потока. По умолчанию для потока выбираются те же атрибуты безопасности, что и для вызывающего потока

Функция `AfxBeginThread` создает и инициализирует объект класса `CWinThread`, запускает его на исполнение и возвращает его адрес пользователю, чтобы он мог на него ссылаться. Если в процессе выполнения функции `AfxBeginThread` возникает ошибка, то при выходе из функции необходимо проверить, что все объекты, созданные во время выполнения, будут соответствующим образом уничтожены.

В *исполняющей функции потока* Поток:функция выполняются все действия, предусмотренные для данного потока. При вызове исполняющей функции поток начинает работу, а при выходе из нее — завершает. Эта функция имеет следующий формат:

```
UINT MyControllingFunction( LPVOID pParam);
```

Аргументом данной функции является 32-разрядная величина. Значение, передаваемое функции в этом аргументе, совпадает со значением второго аргумента функции `AfxBeginThread`, используемой для создания данного потока. Исполняющая функция может интерпретировать это значение любым

образом. Оно может быть интерпретировано как скалярная величина, как указатель на объект структуры, содержащей несколько параметров, или может вообще игнорироваться. Если аргумент функции представляет собой указатель на объект структуры, то данный объект может использоваться не только для передачи данных в процедуру, но и для передачи обработанных данных приложению.

Завершая свою работу, функция должна возвратить значение типа UINT, указывающее на причину завершения функции. Обычно возврат нулевого значения говорит о нормальном завершении функции, а все остальные значения — о возможных ошибках. Эти значения определяются пользователем при написании исполняющей функции. Некоторые потоки могут подсчитывать и возвращать число вызовов объектов определенных классов.

Ниже приведен пример исполняющей функции потока и ее вызова из приложения:

```
UINT MyThreadProc( LPVOID pParam)
{
    CMyObject* pObject = (CMyObject*)pParam;

    if (pObject == NULL ||
        !pObject->IsKindOf(RUNTIME_CLASS(CMyObject)))
    return 1;// Если pObject не указывает на объект

    // Работа с объектом

    return 0;    // Успешное завершение потока
}

// Вызов в одной из функций приложения
   .
   .
pNewObject = new CMyObject;
AfxBeginThread(MyThreadProc, pNewObject);
```

Создание интерфейсных потоков

Поток для реализации пользовательского интерфейса обычно используется при организации ввода информации независимо от выполнения основного потока приложения. Основной поток приложения (производный от класса CWinApp) запускается вместе с приложением и завершается вместе с ним.

Для создания интерфейсного потока необходимо создать пользовательский класс, производный от класса CWinThread. При объявлении и при реализации данного класса следует использовать макросы DECLARE_DYNACREATE и IMPLEMENT_DYNACREATE. В этом классе необходимо перегрузить несколько функций, описание которых приведено в табл. 12.1.

Таблица 12.1. *Перегружаемые функции-члены класса* `CWinThread`

Имя функции	Назначение
`ExitInstance`	Осуществляет уничтожение объектов после завершения потока. Обычно перегружается
`InitInstance`	Производит инициализацию экземпляра потока. Должна перегружаться
`OnIdle`	Производит обработку во время ожидания процесса. Обычно не перегружается
`PreTranslateMessage`	Производит фильтрацию сообщений перед их передачей функциям `TranslateMessage` и `DispatchMessage`. Обычно не перегружается
`ProcessWndProcException`	Перехватывает необработанные исключения, вызванные сообщениями и обработчиками команд потока. Обычно не перегружается
`Run`	Исполняющая функция потока. Содержит цикл обработки сообщений. Перегружается очень редко

Для создания интерфейсного потока необходимо вызвать функцию `AfxBeginThread` и передать в нее в качестве аргумента имя класса, производного от класса `CWinThread`. Кроме того, данная функция имеет еще четыре необязательных аргумента:

- Приоритет потока. По умолчанию используется нормальный приоритет. Возможные значения приоритета перечислены в описании функции `::SetThreadPriority`
- Предполагаемый размер стека, используемого потоком. По умолчанию для создаваемого потока создается стек того же размера, что и для вызывающего
- Значение `CREATE_SUSPENDED`, если поток создается в приостановленном состоянии. По умолчанию в этом аргументе передается нулевое значение, означающее нормальное начало работы потока
- Атрибуты безопасности потока. По умолчанию для потока выбираются те же атрибуты безопасности, что и для вызывающего потока

Функция `AfxBeginThread` выполняет основную работу по созданию потока. Она создает новый объект пользовательского класса, инициализирует его, используя информацию, предоставленную пользователем, и вызывает функцию `CWinThread::CreateThread`. Если в процессе выполнения функции `AfxBeginThread` возникает ошибка, при выходе из нее необходимо проверить, что все объекты, созданные во время выполнения, будут соответствующим образом уничтожены.

Прекращение работы потока

Поток может прекратить свою работу по двум причинам: завершение исполняющей функции, или прекращение извне. Если текстовый процессор использует поток для осуществления фоновой печати, то поток может завершить свою работу, распечатав весь имеющийся текст, или может быть прерван пользователем по той или иной причине.

Для рабочего потока оба случая ничем не отличаются друг от друга, поскольку любой выход из исполнительной функции означает прекращение его работы. Различие состоит лишь в том, что во втором случае поток должен получить некоторую информацию извне, а для нормального завершения никакой дополнительной информации не надо. Обычно о нормальном завершении потока говорит нулевое возвращаемое значение его исполняющей функции. Для прекращения работы рабочего потока может быть использован оператор `return` или функция `AfxEndThread`, вызываемая в исполняемой функции.

Для нормального завершения интерфейсного потока в одной из его функций вызывается функция `::PostQuitMessage`, единственным аргументом которой является код завершения потока. Как и в случае рабочего потока, нулевое возвращаемое значение обычно означает нормальное завершение работы потока. Для прерывания работы интерфейсного потока необходимо вызвать функцию `AfxEndThread` в одной из функций класса потока. В качестве аргумента данной функции передается код завершения потока. Эта функция прекращает работу потока, освобождает стек и все библиотеки динамической компоновки, связанные с потоком, и уничтожает объект класса потока.

Функция `AfxEndThread` должна вызываться из завершаемого потока. Если необходимо завершить данный поток из другого потока, ему должно быть передано соответствующее сообщение, в результате обработки которого поток завершает свою работу.

Для получения кода завершения любого потока необходимо вызвать функцию `::GetExitCodeThread`, первым аргументом которой является дескриптор потока (хранящийся в переменной `m_hThread` объекта класса `CWinThread`), а вторым — указатель на 32-разрядную величину, в которой хранится код завершения потока. Если указанный поток еще не завершил свою работу, то во втором коде завершения имеет значение `STILL_ACTIVE`.

Получение кода завершения потока требует дополнительных операций. По умолчанию при завершении потока соответствующий объект класса `CWinThread` уничтожается. Это означает, что доступ к переменной `m_hThread` объекта класса `CWinThread` становится невозможен, поскольку сам объект больше не существует. Чтобы избежать этой ситуации:

- Присвойте переменной `m_bAutoDelete` значение `FALSE`. Это позволит избежать автоматического уничтожения объекта класса `CWinThread` по за-

вершении соответствующего потока, а значение переменной `m_hThread` можно будет использовать после окончания работы потока. Когда данная переменная станет не нужна, необходимо самостоятельно уничтожить объект класса `CWinThread`. Использование этого метода предпочтительнее, чем метода, описанного ниже

- Сохраните дескриптор окна в отдельной переменной. После создания потока с использованием функции `::DuplicateHandle` скопируйте переменную `m_hThread` в другую переменную и используйте ее для вызова функции `GetExitCodeThread`. Это позволит автоматически уничтожить объект класса потока и узнать причину его завершения. Необходимо убедиться, что при копировании дескриптора процесс еще не завершил свою работу, для чего надежнее всего использовать флаг `CREATE_SUSPENDED` при вызове функции `AfxBeginThread`, сохранить дескриптор, а затем возобновить его исполнение функцией `ResumeThread`

Любой из этих методов позволит определить причину завершения работы потока.

Взаимодействие между потоками

Одновременная работа нескольких потоков в одном приложении требует более внимательного отношения к проблеме обеспечения безопасности данных, чем работа с одним потоком. Поскольку потоки могут независимо обрабатывать одни и те же данные, необходимо при доступе к данным убедиться в их корректности. Например, при изменении содержимого записи в базе данных необходимо внести изменения во все ее поля.

В соответствии с требованиями уменьшения размера и повышения быстродействия программ объекты библиотеки MFC располагают средствами для безопасной работы с потоками. Этими средствами располагают только классы. Это означает, что двум различным потокам не разрешается работать с одним и тем же объектом класса `CString`. Каждый из них должен работать с собственным объектом данного класса. Если же работа двух потоков с одним объектом безусловно необходима, следует воспользоваться механизмом синхронизации Win32, например, критическими секциями. Библиотека классов использует критические секции для защиты глобальных структур данных, используемых, например, при выделении памяти во время отладки.

Потоки, созданные каким-либо другим образом, кроме наследования от класса `CWinThread`, не могут получить доступ к объектам библиотеки MFC, расположенным в другом потоке. Другими словами, чтобы получить доступ к объектам библиотеки MFC, расположенным в другом потоке, необходимо создать данный поток с использованием средств библиотеки MFC.

Взаимодействие между потоком и приложением

Как правило, поток имеет доступ только к тому объекту библиотеки MFC, в котором он был создан. Это происходит потому, что временные и постоянные карты дескрипторов Windows хранятся в локальной памяти потока для защиты от одновременного доступа нескольких потоков. Например, рабочий поток не может произвести вычисления и вызвать функцию `UpdateAllViews` в соответствующем документе, чтобы отобразить все внесенные в него изменения во всех связанных с ним окнах.

Эта операция не вызовет никаких изменений, поскольку карта преобразования объектов класса `CWnd` в объекты дескрипторов `HWND` является локальной для первичного потока. Это означает, что один поток может содержать преобразование дескриптора Windows в объект C++, но в другом потоке тот же самый дескриптор будет соответствовать другому объекту C++. Изменения, произведенные в одном потоке, не будут отображаться в другом.

Существует несколько путей решения этой проблемы. Первым из них является передача рабочему потоку индивидуальных дескрипторов (таких, как `HWND`), а не объектов C++. После этого рабочий поток добавляет данный объект в свою временную карту вызовом функции `FromHandle`. Можно добавить этот объект и в постоянную карту потока, вызвав функцию `Attach`, однако это можно делать только в том случае, если есть гарантия, что данный объект просуществует дольше, чем сам поток.

Другим способом решения данной проблемы является создание новых пользовательских сообщений, соответствующих определенным ситуациям, о которых рабочий поток должен извещать вызвавшее его приложение, и передача их в главное окно при помощи функции `::PostMessage`. Этот метод аналогичен способу передачи сообщений между двумя приложениями за исключением того, что в данном случае оба потока используют одно и то же адресное пространство.

Использование классов синхронизации

При работе с несколькими потоками необходимо решить проблему синхронизации доступа потоков к общему ресурсу. Одновременный доступ двух и более потоков к одним и тем же данным может привести к нежелательным и непредсказуемым последствиям. Например, один поток может производить обновления содержимого объекта структуры, а другой — считывать содержимое данного объекта. В этом случае при чтении обновляемой структуры поток может получить новые данные, старые данные или их смесь.

Библиотека MFC содержит несколько классов синхронизации доступа к данным, позволяющим решить эту проблему.

В типичном приложении с несколькими потоками для хранения разделяемого ими ресурса используется специальным образом организованный

класс, который позволяет обойтись без разработки специальных функций синхронизации при написании приложения. Все необходимые функции включены и скрыты в классе, и пользователь может сосредоточиться на решении поставленной задачи, а не на обеспечении безопасности работы с данными.

Возьмем в качестве примера приложение, работающее с тремя окнами связного списка счетов. Это приложение позволяет вывести на экран три окна, в которые выводится один и тот же счет, но работа может вестись только с одним окном одновременно. По завершении работы со счетом он сохраняется в архиве.

В этом примере используются три класса синхронизации. Поскольку приложение работает не более чем с тремя окнами счетов, для ограничения числа объектов применяется класс `CSemaphore`, задающий максимальное число объектов в приложении. При попытке открыть четвертое окно счета приложение или будет ждать, пока не будет закрыто одно из открытых окон счетов, или аварийно завершит свою работу.

Для ограничения доступа к объекту при внесении в него изменений используется класс `CCriticalSection`, позволяющий в данный момент времени вносить изменения только в одном из окон счетов.

После внесения изменений в счет используется объект класса `CEvent` для запуска остальных потоков, находившихся до этого в режиме ожидания. Затем данные посылаются в архив.

Чтобы создать класс, полностью скрывающий все функции синхронизации, необходимо добавить в него соответствующие объекты классов синхронизации. В предыдущем примере в класс представления должен быть добавлен объект класса `CSemaphore`, в класс связного списка должен быть добавлен объект класса `CCriticalSection`, а в класс документа — объект класса `CEvent`.

После этого необходимо вызвать соответствующие функции синхронизации в каждом из этих классов. Это означает, что все функции-члены классов, изменяющие данные, хранимые в классе, или осуществляющие доступ к этим данным, должны создать объекты классов `CSingleLock` или `CMultiLock` и вызывать для этих объектов функцию `Lock`.

Если заблокированный объект выходит из области своего действия или уничтожается, то деструктор объекта вызывает функцию `Unlock` и освобождает ресурс. Пользователь может самостоятельно вызывать функцию `Unlock`, если в этом возникнет необходимость.

Создание подобных классов позволяет использовать их в приложениях, работающих с несколькими потоками, так же просто и надежно, как и в приложениях, имеющих единственный поток. Недостатком данного подхода является некоторое замедление работы с этими классами по сравнению с их версиями, в которые не добавлены объекты синхронизации. Кроме того, если имеется вероятность того, что один и тот же объект может быть унич-

тожен различными потоками, этот подход может привести к нежелательным результатам. В этом случае лучше использовать отдельные объекты синхронизации.

Шесть классов, используемых в библиотеке MFC для работы с несколькими потоками, могут быть разделены на две категории: классы синхронизации работы (`CSyncObject`, `CSemaphore`, `CMutex`, `CCriticalSection` и `CEvent`) и классы синхронизации доступа (`CMultiLock` и `CSingleLock`).

Классы синхронизации работы используются в том случае, когда доступ к ресурсу должен контролироваться для обеспечения целостности данного ресурса. *Классы синхронизации доступа* к объекту используются для получения доступа к контролируемому ресурсу.

Чтобы определить, какой из классов синхронизации следует применить в конкретном случае, необходимо ответить на несколько вопросов:

- Должно ли приложение ждать возникновения некоторого события перед тем, как получить доступ к данному ресурсу?

  (Например, получения данных из коммуникационного порта перед тем, как записать их в файл.) Если да, то используйте класс `CEvent`

- Могут ли несколько потоков одного приложения получить одновременный доступ к данному ресурсу?

  (Например, приложение позволяет открыть до пяти окон, связанных с одним документом.) Если да, то используйте класс `CSemaphore`

- Будет ли данный ресурс использован несколькими приложениями?

  (Например, если ресурс расположен в библиотеке динамической компоновки.) Если да, то используйте класс `CMutex`. Если нет, используйте класс `CCriticalSection`

Объекты класса `CSyncObject` нельзя использовать непосредственно. Он является базовым классом для четырех других классов синхронизации.

Выбор класса синхронизации доступа, используемого в данном приложении, не представляет особой проблемы. Если приложение должно контролировать доступ только к одному разделяемому ресурсу, используется объект класса `CSingleLock`, если же необходимо контролировать доступ к нескольким разделяемым ресурсам, используется класс `CMultiLock`.

Примеры приложений, работающих с потоками

Работа с потоками представляет собой достаточно сложный раздел программирования с использованием Visual C++, поэтому в качестве примеров работы с потоками будут рассмотрены не простейшие приложения, создаваемые пользователем, а примеры из библиотеки MSDN.

Приложение MTGDI

Приложение MTGDI демонстрирует возможности разделения ресурсов GDI между несколькими потоками, запущенными в однооконном приложении. Его меню **Thread** позволяет запустить одновременно 1, 10 или 25 новых потоков. Каждый из потоков выводит на экран перемещающиеся шары, прямоугольники или линии, цвет которых выбирается случайным образом.

Приложение MTGDI использует несколько классов, производных от класса `CWinThread`. Описание этих классов содержится в файле Threads.cpp. Созданный в данном примере виртуальный базовый класс `CGDIThread` используется для создания каждого из классов потоков GDI (по одному классу для вывода на экран шаров, прямоугольников и линий). Поскольку ресурсы GDI не могут разделяться между потоками, как объекты библиотеки MFC, в данном примере также продемонстрирован метод преобразования стандартных ресурсов GDI в объекты библиотеки MFC и обратно.

Приложение создает указанное количество потоков в ответ на выбор пользователем соответствующей команды в меню **Thread**. Каждый отображаемый объект управляется собственным потоком. К имеющимся потокам может быть добавлено любое количество новых. Количество действующих потоков выводится в заголовке окна.

Все потоки прекращают свою работу после выбора пользователем соответствующей команды в меню **Thread**. Однако в данном меню имеются две команды: одна для быстрого, а вторая для медленного завершения работы потоков. Если число потоков не превышает 100, то различие в скорости выполнения данных команд практически не заметно.

Процедура копирования примера с CD-ROM на диск компьютера подробно описана в главе 4 при описании класса просмотрового окна списка, поэтому здесь данная процедура будет описана кратко. Чтобы переписать текст проекта с примером на диск вашего компьютера:

См. также главу 4

1. Выберите команду **Help**, **Index...** (Справка, Индекс). Появится окно **MSDN Library Visual Studio 6.0**, показанное на рис. 12.1.
2. В текстовое поле **Введите ключевое слово для поиска:** введите текст "MTGDI sample (MFC)" и нажмите клавишу <Enter>. В окне появится текст справки по приложению, как показано на рис. 12.2.
3. Щелкните левой кнопкой мыши на синем тексте **Click to open or copy the MTGDI project files** (Щелкните левой кнопкой мыши, чтобы открыть или скопировать файлы приложения MTGDI). Если библиотека MSDN не установлена на диске, появится диалоговое окно **Справка**, изображенное на рис. 12.3, с требованием вставить компакт-диск, помеченный "MSDN VS 6.0 CD1" в привод CD-ROM.

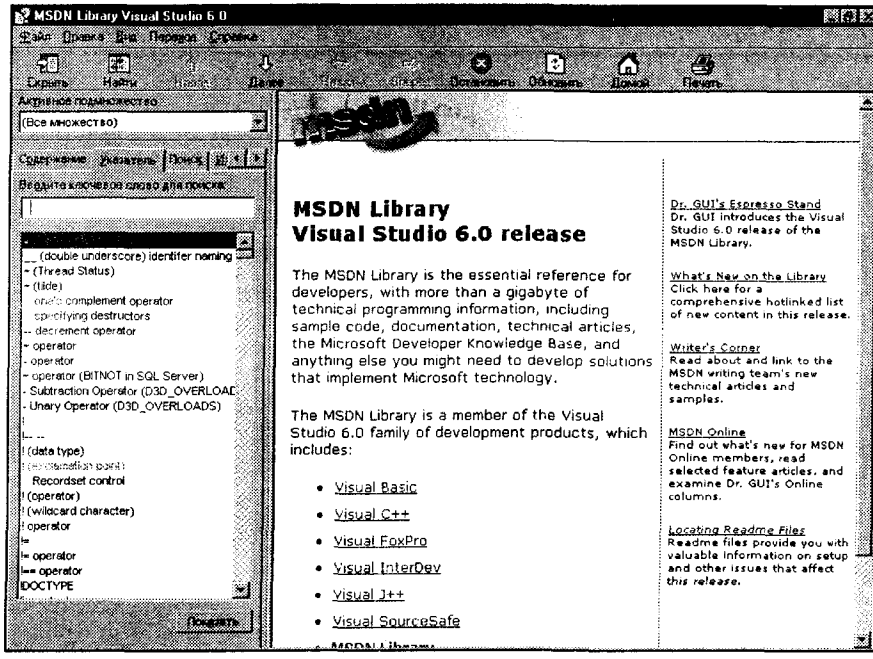


Рис. 12.1. Окно **MSDN Library Visual Studio 6.0**

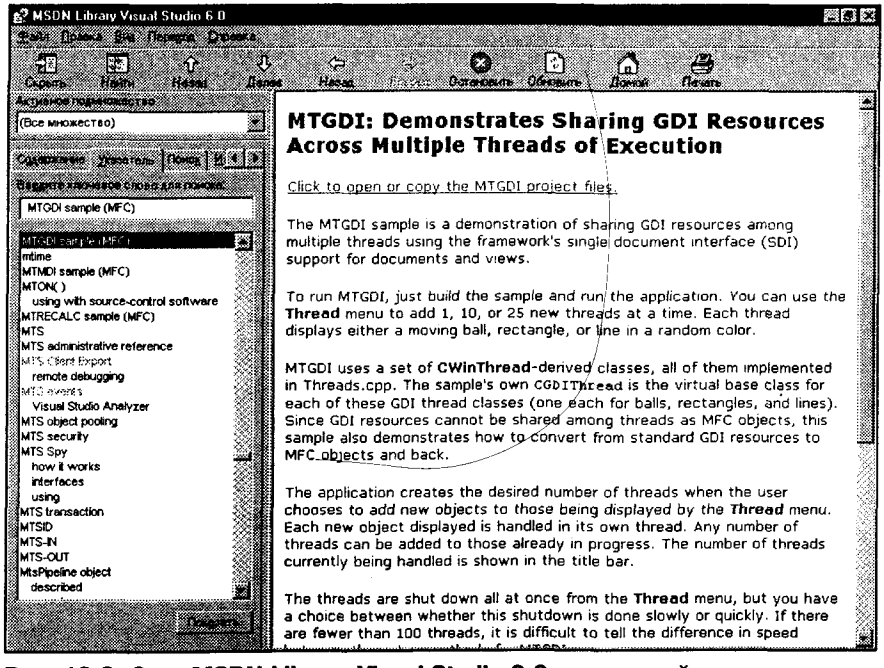


Рис. 12.2. Окно **MSDN Library Visual Studio 6.0** со справкой по приложению

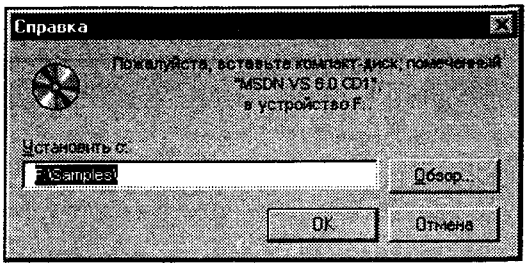


Рис. 12.3. Диалоговое окно **Справка**

4. Вставьте компакт-диск, помеченный "MSDN VS 6.0 CD1" и нажмите кнопку **OK**. Появится диалоговое окно **Visual C++ Samples** (Примеры Visual C++), показанное на рис. 12.4.

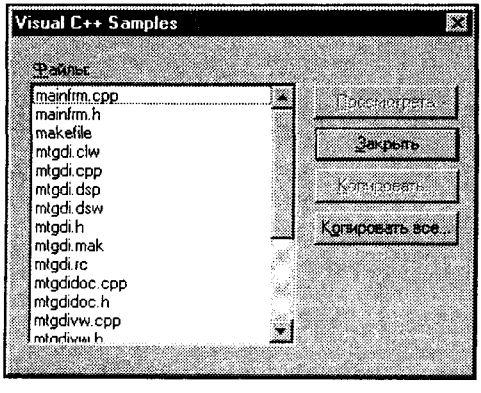


Рис. 12.4. Диалоговое окно **Visual C++ Samples**

5. Нажмите кнопку **Копировать все...**. Появится диалоговое окно **Копирование**, изображенное на рис. 12.5.

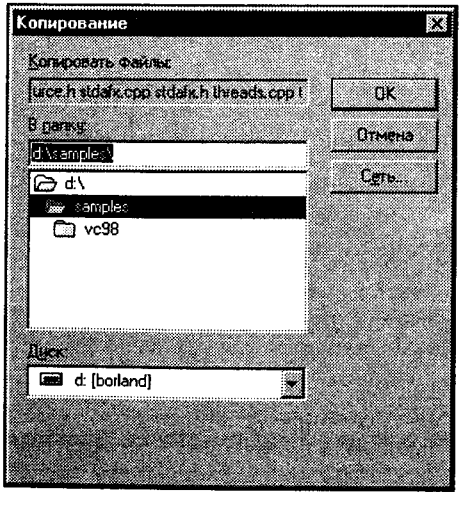


Рис. 12.5. Диалоговое окно **Копирование**

6. Выберите каталог, в который будет производиться копирование, и нажмите кнопку **OK**.
7. Подтвердите создание нового каталога, чтобы скопировать в него файлы проекта, и нажмите кнопку **OK** после того, как появится окно сообщения об успешном завершении копирования.
8. Нажмите кнопку **Закрыть** в диалоговом окне **Visual C++ Samples**.
9. Закройте окно **MSDN Library Visual Studio 6.0**.
10. Если библиотека Unicode не установлена, выберите команду **Build**, **Set Active Configuration...**. Появится диалоговое окно **Set Active Project Configuration** (Установка активной конфигурации проекта), изображенное на рис. 12.6.

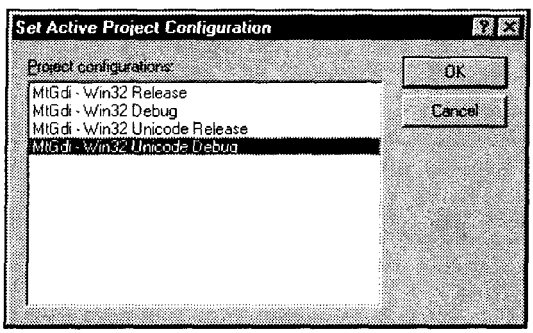


Рис. 12.6. *Диалоговое окно* **Set Active Project Configuration**

11. В окне списка **Project configurations**: выделите строку **MtGdi — Win32 Debug** и нажмите кнопку **OK**.
12. Нажмите клавишу <F5> и запустите приложение на исполнение.
13. Выберите команду меню **Thread**, **New Ball**.
14. Выберите команду меню **Thread**, **New Rectangle**. Результат выполнения этих команд показан на рис. 12.7.

Как уже говорилось, описание всех потоков содержится в файле Threads.cpp. В самом начале данного файла объявляются две статические переменные базового класса `CGDIThread`:

```
CRITICAL_SECTION CGDIThread::m_csGDILock;
HANDLE CGDIThread::m_hAnotherDead = CreateEvent(NULL, FALSE, FALSE, NULL);
```

Критическая секция `m_csGDILock` используется для защиты контекста устройства во время рисования, а дескриптор `m_hAnotherDead` используется для сообщения о том, что один или несколько потоков завершили свою работу. Данный дескриптор объекта события, созданный функцией `CreateEvent`, не может наследоваться, автоматически сбрасывается системой, создается неотмеченным и неименованным.

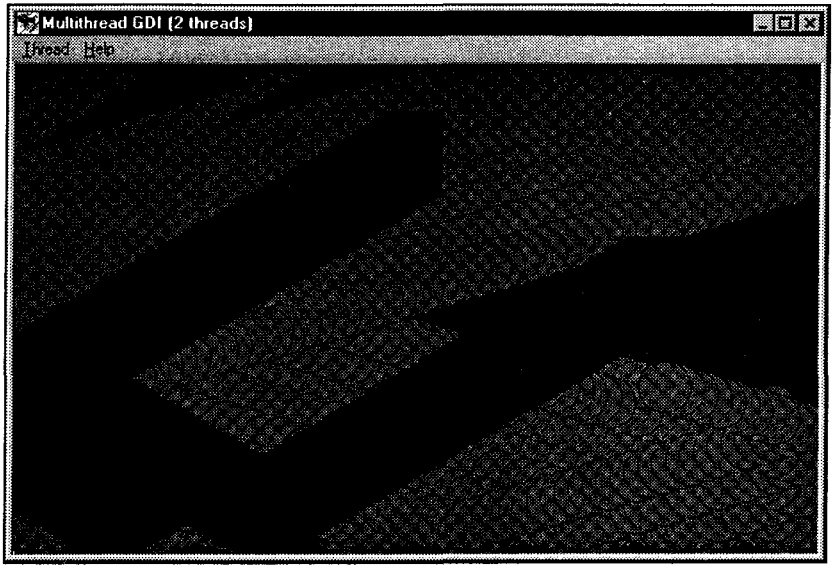


Рис. 12.7. Приложение MTGDI в работе

Конструктор базового класса `CGDIThread` имеет следующий вид:

```
CGDIThread::CGDIThread(CWnd* pWnd, HDC hDC)
{
  m_bAutoDelete = FALSE;
  m_pMainWnd = pWnd;
  m_pMainWnd->GetClientRect(&m_rectBorder);
  m_hDC = hDC;

  // kill event starts out in the signaled state
  m_hEventKill = CreateEvent(NULL, TRUE, FALSE, NULL);
  m_hEventDead = CreateEvent(NULL, TRUE, FALSE, NULL);
}
```

В данном конструкторе сохраняется указатель на объект родительского окна и дескриптор контекста устройства, передаваемые конструктору в качестве аргументов. В переменной `m_rectBorder` сохраняются координаты рабочей области окна, полученные с помощью функции `CWnd::GetClientRect`, и создаются два дескриптора объектов события в отмеченном состоянии.

Функция `InitInstance` используется для обхода стандартного цикла обработки сообщений.

```
BOOL CGDIThread::InitInstance()
{
  // thread setup
  m_dc.Attach(m_hDC);
```

```
  // loop but check for kill notification
  while (WaitForSingleObject(m_hEventKill, 0) == WAIT_TIMEOUT)
    SingleStep();

  // thread cleanup
  m_dc.Detach();

  // avoid entering standard message loop by returning FALSE
  return FALSE;
}
```

Причина, по которой в данном потоке используется собственный контекст устройства, кроется в том, что объекты GDI, определенные в библиотеке MFC, не могут передаваться из одного потока в другой. Поэтому вместо того, чтобы передавать сами объекты, приходится передавать их дескрипторы и на их основе создавать свои объекты контекста устройств. Это связано с тем, что для каждого потока библиотека MFC создает свой список объектов GDI. Поэтому при простой передаче объекта GDI в другой поток он окажется не в том списке, и библиотека MFC выдаст сообщение об ошибке.

Для связи объекта класса контекста устройств с дескриптором контекста устройства используется функция `CDC::Attach`. После этого создается цикл ожидания с использованием объекта события `m_hEventKill`, извещающего поток о необходимости прекратить свою работу. Для этого в цикле вызывается функция `WaitForSingleObject`, проверяющая состояние объекта `m_hEventKill`. Если этот объект находится в неотмеченном состоянии, вызывается виртуальная функция `CGDIThread::SingleStep`. В противном случае происходит выход из цикла и вызывается функция `CDC::Detach`, освобождающая контекст устройств вывода из объекта класса `CDC`. Чтобы после выхода из функции `InitInstance` не был вызван стандартный цикл обработки сообщений, данная функция возвращает значение `FALSE`.

Данный класс содержит также функции, уничтожающие поток.

```
void CGDIThread::Delete()
{
  // calling the base here won't do anything but it is a good habit
  CWinThread::Delete();

  // acknowledge receipt of kill notification
  VERIFY(SetEvent(m_hEventDead));
  VERIFY(SetEvent(m_hAnotherDead));
}

CGDIThread::~CGDIThread()
{
  CloseHandle(m_hEventKill);
  CloseHandle(m_hEventDead);
}
```

```
void CGDIThread::KillThread()
{
  // Note: this function is called in the context of other threads,
  // not the thread itself.

  // reset the m_hEventKill which signals the thread to shutdown
  VERIFY(SetEvent(m_hEventKill));

  // allow thread to run at higher priority during kill process
  SetThreadPriority(THREAD_PRIORITY_ABOVE_NORMAL);
  WaitForSingleObject(m_hEventDead, INFINITE);
  WaitForSingleObject(m_hThread, INFINITE);

  // now delete CWinThread object since no longer necessary
  delete this;
}
```

Функция `Delete` вызывает метод базового класса и извещает приложение о получении сообщения о прекращении работы. В деструкторе класса уничтожаются дескрипторы объектов событий, использованных в данном классе.

В функции `KillThread`, вызываемой другими потоками для уничтожения данного, отмечается объект события `m_hEventKill`, что приводит к выходу из цикла в функции `InitInstance`. После этого вызывается функция `CWinThread::SetThreadPriority`, повышающая уровень приоритета данного потока для скорейшего прекращения его работы. Затем вызываются функции `WaitForSingleObject` для объектов событий `m_hEventDead` и `m_hThread`, неограниченное время ожидающие их отметки. После этого уничтожается данный объект класса протока.

Функция `UpdateBorder` вызывается при изменении размеров рабочей области окна.

```
void CGDIThread::UpdateBorder()
{
  // Since another thread can access this variable, we block them when
  // we write it. In this case, it is overkill since the other thread
  // would just animate in the wrong direction while reading intermediate
  // values and finally adjust itself when the new value is valid.
  // It is a good idea to avoid such anomalies, however.

  EnterCriticalSection(&CGDIThread::m_csGDILock);
  {
    m_pMainWnd->GetClientRect(&m_rectBorder);
  }
  LeaveCriticalSection(&CGDIThread::m_csGDILock);
}
```

Первым оператором данной функции является вызов функции EnterCriticalSection, препятствующий другому потоку получить доступ к некорректным данным во время их изменения. Затем вызывается функция CWnd::GetClientRect, записывающая в переменную m_rectBorder новые координаты рабочей области окна, и функция LeaveCriticalSection, позволяющая другим потокам получить доступ к обновленным данным.

Рассмотрим работу класса CBallThread, поскольку другие классы построены аналогично. Различия заключаются только в типе выводимых объектов.

Текст конструктора класса CBallThread приведен ниже:

```
CBallThread::CBallThread(CWnd* pWnd, HDC hDC, CPoint ptPos, CPoint ptVel,
  CSize Size, COLORREF color)
  : CGDIThread(pWnd,hDC)
{
  m_rectPosition.SetRect(ptPos.x,ptPos.y,ptPos.x+Size.cx,ptPos.y+Size.cx);
  m_ptVelocity = ptVel;

  CBrush brush(color);
  m_hBrush = (HBRUSH)brush.Detach();
}
```

В конструктор данного класса кроме аргументов конструктора базового класса передаются начальная позиция прямоугольника, описывающего выводимый шар, скорость его перемещения по экрану для каждой из координат, размер прямоугольника и цвет шара.

На основании полученных аргументов создается объект класса прямоугольника с заданными размерами и положением, параметры скорости сохраняются в переменной-члене класса и создается объект класса кисти с заданным цветом. После этого дескриптор объекта кисти сохраняется в переменной-члене класса, для чего используется функция CDC::Detach. Причина, по которой вместо объекта класса кисти используется дескриптор объекта GDI, объяснена выше при описании базового класса.

Текст функции CBallThread::InitInstance приведен ниже:

```
BOOL CBallThread::InitInstance()
{
  m_brush.Attach(m_hBrush);

  return CGDIThread::InitInstance();
}
```

В функции InitInstance дескриптор объекта GDI связывается с соответствующим объектом класса кисти и вызывается соответствующий метод базового класса.

Функция SingleStep является реализацией виртуальной функции, определенной в базовом классе, и используется для вывода информации на экран.

```
void CBallThread::SingleStep()
{
  m_rectPosition.OffsetRect(m_ptVelocity);

  if (m_rectPosition.top<m_rectBorder.top)
    m_ptVelocity.y = (m_ptVelocity.y>0) ? m_ptVelocity.y : -m_ptVelocity.y;

  if (m_rectPosition.bottom>m_rectBorder.bottom)
    m_ptVelocity.y = (m_ptVelocity.y>0) ? -m_ptVelocity.y : m_ptVelocity.y;

  if (m_rectPosition.left<m_rectBorder.left)
    m_ptVelocity.x = (m_ptVelocity.x>0) ? m_ptVelocity.x : -m_ptVelocity.x;

  if (m_rectPosition.right>m_rectBorder.right)
    m_ptVelocity.x = (m_ptVelocity.x>0) ? -m_ptVelocity.x : m_ptVelocity.x;

  // Since all threads share the same HDC it is necessary
  // to block all other threads while we render in the HDC
  EnterCriticalSection(&CGDIThread::m_csGDILock);
  {
    CBrush* oldbrush;

    oldbrush = m_dc.SelectObject(&m_brush);
    m_dc.Ellipse(m_rectPosition);
    m_dc.SelectObject(oldbrush);

    // Win32 optimizes GDI calls by collecting them in a batch
    // and then thunking the whole batch at once on a per
    // thread basis. Since we share an HDC with multiple
    // threads, we must flush the batch before yielding to
    // other threads that will adjust the HDC.  To see what
    // I mean, comment out the GdiFlush() function call and
    // watch the results.
    GdiFlush();
  }
  LeaveCriticalSection(&CGDIThread::m_csGDILock);
}
```

В первом операторе данной функции определяется новое положение выводимого шара. Для этого при помощи функции CRect::OffsetRect, перемещающей прямоугольник, описывающий данный шар, в направлении, заданном аргументом данной функции. После перемещения прямоугольника проверяется, не достиг ли он одной из границ рабочей области окна и в том случае, если какая-либо граница достигнута, соответствующая переменная

объекта класса CPoint, задающего скорость перемещения прямоугольника, меняет знак. Это приводит к изменению направления движения по данной координате.

Поскольку все потоки разделяют один и тот же контекст устройства Windows, необходимо заблокировать выполнение всех потоков на время работы с данным контекстом устройства. Для этого используется функция EnterCriticalSection, ожидающая возможности получить в безраздельную собственность указанный объект класса критической секции. Данная функция прекращает ожидание, когда вызывающему потоку предоставляется право собственности на данную секцию.

После этого выполняется защищенный фрагмент программы. В этом фрагменте в контекст устройства выбирается кисть, значение старой кисти сохраняется, с использованием выбранной кисти на экране рисуется эллипс, вписанный в прямоугольник, перемещение которого производилось ранее, и в контекст устройства выбирается старая кисть.

Поскольку Win32 использует пакетный режим при вызове функций GDI, перед передачей права собственности на контекст устройства Windows необходимо запустить пакетное выполнение вызванных ранее графических функций. Для этого вызывается функция GdiFlush.

По завершении работы с разделяемым ресурсом поток отказывается от права собственности на него, вызывая функцию LeaveCriticalSection.

В классе CThreadApp, производном от класса CWinApp, в функцию InitInstance до определения шаблона класса был добавлен вызов функции

```
InitializeCriticalSection(&CGDIThread::m_csGDILock);
```

инициализирующей объект критической секции. Описание класса CThreadApp содержится в файле mtgdi.cpp.

Следует рассмотреть также функцию ExitInstance, вызываемую для завершения работы данного приложения:

```
int CThreadApp::ExitInstance()
{
  DeleteCriticalSection(&CGDIThread::m_csGDILock);
  CloseHandle(CGDIThread::m_hAnotherDead);

  return CWinApp::ExitInstance();
}
```

В данной функции вызывается функция DeleteCriticalSection, освобождающая все ресурсы, используемые указанным объектом критической секции CGDIThread::m_csGDILock. Затем при помощи функции CloseHandle уничтожается дескриптор CGDIThread::m_hAnotherDead. После этого вызывается соответствующий метод базового класса.

Управление потоками производится в классе CThreadView, описание которого расположено в файле mtgdivw.cpp.

Функция UpdateTitle выводит в заголовок окна число активных потоков:

```
void CThreadView::UpdateTitle(int nThreads)
{
  if (nThreads == -1)
    nThreads = m_threadList.GetCount();

  CString strCaption;
  if (nThreads != 0)
  {
    CString strFormat; strFormat.LoadString(IDS_CAPTION_FORMAT);
    strCaption.Format(strFormat, nThreads);
  }
  else
    strCaption.LoadString(IDS_CAPTION_NOTHREADS);

  GetParentFrame()->SetWindowText(strCaption);
}
```

В качестве аргумента данной функции передается число активных потоков. Если аргумент имеет значение −1, то для определения числа активных потоков используется функция CTypedPtrList::GetCount. Если число активных потоков больше нуля, то функция CString::LoadString считывает значение соответствующего строкового ресурса, а затем с помощью функции CString::Format строка форматирования и целочисленная переменная преобразуются в строку символов. Если число активных потоков равно нулю, то в строку загружается соответствующий символьный ресурс. Последним оператором данной функции является вывод полученной строки в заголовок окна. Для этого вызывается функция CWnd::SetWindowText, устанавливающая новый заголовок окна. Для получения указателя на объект класса окна используется функция CWnd::GetParentFrame, возвращающая указатель на главное окно приложения.

Функция StartThread служит для запуска новых потоков:

```
void CThreadView::StartThread(int ThreadID)
{
  CGDIThread* pThread;
  CPoint Pos(rand()%100,rand()%100);
  CPoint Vel1((rand()%10)+2,(rand()%10)+2);
  CPoint Vel2((rand()%5)+2,(rand()%5)+2);
  CSize Size(((rand()%60)+20),((rand()%60)+20));
  COLORREF Color;
```

```
//BLOCK: calculate random color
{
  int r,g,b;

  do
  {
    r = (rand()%3);
    r = (r!=0) ? (r==2) ? 255 : 127 : 0;

    g = (rand()%3);
    g = (g!=0) ? (g==2) ? 255 : 127 : 0;

    b = (rand()%3);
    b = (b!=0) ? (b==2) ? 255 : 127 : 0;
  }
  while ((r==g) && (g==b));
  // No white(255,255,255), gray(127,127,127), or black(0,0,0) allowed

  Color = RGB(r,g,b);
}

if (rand()%1) Vel1.x*=-1;
if (rand()%1) Vel1.y*=-1;
if (rand()%1) Vel2.x*=-1;
if (rand()%1) Vel2.y*=-1;

switch(ThreadID)
{
case 1:
  pThread = new CBallThread(this,
    m_pDC->GetSafeHdc(), Pos, Vel1, Size, Color);
  break;
case 2:
  pThread = new CRectThread(this,
    m_pDC->GetSafeHdc(), Pos, Vel1, Size, Color);
  break;
case 3:
  pThread = new CLineThread(this,
    m_pDC->GetSafeHdc(), Pos, Vel1, Vel2, Size, Color);
  break;
}

if (pThread == NULL)
  return;

ASSERT_VALID(pThread);
pThread->m_pThreadParams = NULL;
```

```
  // Create Thread in a suspended state so we can set the Priority
  // before it starts getting away from us
  if (!pThread->CreateThread(CREATE_SUSPENDED))
  {
    delete pThread;
    return;
  }

  // since everything is successful, add the thread to our list
  m_threadList.AddTail(pThread);

  // If you want to make the sample more sprightly, set the thread
  // priority here a little higher. It has been set at idle priority
  // to keep from bogging down other apps that may also be running.
  VERIFY(pThread->SetThreadPriority(THREAD_PRIORITY_IDLE));
  // Now the thread can run wild
  pThread->ResumeThread();
}
```

Аргумент данной функции определяет, какого типа поток следует запустить. В начале данной функции случайным образом выбираются начальное положение, два значения вектора скорости и размер. Далее запускается цикл случайного выбора цвета, в котором каждый из основных цветов может иметь одно из трех значений: 0, 127 или 255, но не допускается равенство значений всех основных цветов. После этого случайным образом выбирается знак скорости по каждой координате. После определения значений всех аргументов, необходимых для запуска потока любого типа, в зависимости от значения аргумента данной функции создается поток одного из трех типов: шаров, прямоугольников или линий. Для получения значения дескриптора контекста устройства используется функция `CDC::GetSafeHdc`.

Если полученный указатель на объект класса потока равен нулю, то функция завершает свою работу. В противном случае выполняется проверка на допустимость данного указателя и создается объект класса потока в приостановленном состоянии. Если в процессе создания данного объекта возникла ошибка, то указатель на него уничтожается, и функция прекращает свою работу. В случае успешного завершения процедуры создания объекта класса потока, функция `CTypedPtrList::AddTail` добавляет указатель на него в хвост списка активных потоков, функция `CWinThread::SetThreadPriority` устанавливает для него низкий уровень приоритета, чтобы он не мешал исполнению других приложений, и данный поток запускается на исполнение функцией `CWinThread::ResumeThread`.

В качестве примера функции обработки сообщения о запуске потока рассмотрим функцию `On25newballs`, запускающую 25 потоков, выводящих на экран шары:

```
void CThreadView::On25newballs()
{
  for (int i = 0; i < 25; i++)
  {
    StartThread(1);
    UpdateTitle();
  }
}
```

В данной функции организован цикл, в котором 25 раз вызывается функция `StartThread` с аргументом 1, а затем функция `UpdateTitle` с установленным по умолчанию значением аргумента −1 обновляет заголовок окна.

В приложении используются два метода уничтожения потоков. В первом из них всем потокам одновременно посылается сообщение о необходимости прекращения работы, а затем ожидается сообщение о завершении работы последнего потока. Второй метод предполагает последовательное закрытие каждого потока.

Первый метод реализован в функции `OnKillThreads`:

```
void CThreadView::OnKillThreads()
{
  // tell all threads to shutdown
  for (POSITION pos = m_threadList.GetHeadPosition(); pos != NULL;)
  {
    CGDIThread* pThread = m_threadList.GetNext(pos);
    VERIFY(SetEvent(pThread->m_hEventKill));
  }

  // wait for all threads to finish shutdown
  for (int nThreadsLeft = m_threadList.GetCount(); nThreadsLeft != 0;)
  {
    WaitForSingleObject(CGDIThread::m_hAnotherDead, INFINITE);
    Sleep(nThreadsLeft*2);// 200ms for every 100 threads
    nThreadsLeft = 0;
    for (pos = m_threadList.GetHeadPosition(); pos != NULL;)
    {
      CGDIThread* pThread = m_threadList.GetNext(pos);
      if (WaitForSingleObject(pThread->m_hEventDead, 0) == WAIT_TIMEOUT)
        ++nThreadsLeft;
    }
    UpdateTitle(nThreadsLeft);
  }

  // delete all thread objects
  while (!m_threadList.IsEmpty())
```

```
    {
      CGDIThread* pThread = m_threadList.RemoveHead();
      VERIFY(WaitForSingleObject(pThread->m_hThread, INFINITE) ==
                                                        WAIT_OBJECT_0);
      delete pThread;
    }
    UpdateTitle();

    // invalidate the window since all threads are now gone
    Invalidate();
}
```

Для извещения всех потоков о необходимости завершения работы организуется цикл по переменной типа `POSITION`, которой в начале работы цикла функцией `CTypedPtrList::GetHeadPosition` присваивается позиция первого элемента в списке. В теле цикла при помощи функции `CTypedPtrList::GetNext` получается указатель на объект класса потока, а функция `SetEvent` отмечает объект класса события `m_hEventKill`, являющийся членом данного класса. Цикл прекращает свою работу, когда переменная типа `POSITION`, используемая для просмотра списка, принимает значение `NULL`.

После этого открывается цикл ожидания завершения работы всех потоков. Переменной `nThreadsLeft` присваивается количество активных потоков, затем функция `WaitForSingleObject` ожидает уничтожения хотя бы одного потока, и вызывается функция `Sleep`, дающая на уничтожение каждого потока 2 мс. После этого переменная `nThreadsLeft` обнуляется и просматривается список, причем для каждого объекта класса потока поверяется состояние переменной `m_hEventDead`. Если оно не отмечено, то поток продолжает свою работу и переменная `nThreadsLeft` получает приращение. После этого функция `UpdateTitle` обновляет заголовок окна на основе полученных данных, и начинается новый проход цикла, пока значение переменной `nThreadsLeft` не станет нулевым.

Затем уничтожаются все объекты классов потока: пока список потоков не пуст, для него вызывается функция `RemoveHead`, уничтожающая соответствующий объект и указатель на него, после чего уничтожение элементов списка продолжается.

Для вывода соответствующего заголовка окна вызывается функция `UpdateTitle`, а для перерисовки окна — функция `Invalidate`.

Второй метод уничтожения потоков реализован в функции `OnThreadKillThreadsSlow`:

```
void CThreadView::OnThreadKillThreadsSlow()
{
  // Iterate through the threads killing each
  // KillThread doesn't return until the Thread is dead
```

```
  while (!m_threadList.IsEmpty())
  {
    m_threadList.RemoveHead()->KillThread();
    UpdateTitle();
  }

  // Invalidate the window so it blanks the window
  Invalidate();
}
```

В этой функции, пока список активных потоков не пуст, уничтожается указатель, расположенный в голове данного списка, а для объекта, на который он указывает, вызывается функция `KillThread`, описанная выше. Функция `UpdateTitle` обновляет заголовок окна, и процесс продолжается для нового указателя, расположенного в голове списка. По завершении работы цикла вызывается функция `Invalidate` для перерисовки окна.

Функция `OnDestroy` вызывается при закрытии приложения:

```
void CThreadView::OnDestroy()
{
  OnKillThreads();

  delete m_pDC;
  m_pDC = NULL;

  CView::OnDestroy();
}
```

В данной функции используется быстрый метод прекращения работы потоков, уничтожается объект класса контекста устройства и вызывается метод базового класса.

Функция `OnCreate` вызывается при создании окна:

```
int CThreadView::OnCreate(LPCREATESTRUCT lpCreateStruct)
{
  if (CView::OnCreate(lpCreateStruct) == -1)
    return -1;

  // m_pDC must be initialized here instead of the constructor
  // because the HWND isn't created until Create is called.
  m_pDC = new CClientDC(this);
  UpdateTitle();

  return 0;
}
```

В ней вызывается метод базового класса, создается объект класса контекста устройства и вызывается функция `UpdateTitle` для вывода заголовка окна.

Функция OnSize вызывается при изменении размеров рабочей области окна:

```
void CThreadView::OnSize(UINT nType, int cx, int cy)
{
  CView::OnSize(nType, cx, cy);

  // Instead of getting the client rect every time we draw, we only
  // update it when it changes.
  for (POSITION pos = m_threadList.GetHeadPosition(); pos != NULL;)
    m_threadList.GetNext(pos)->UpdateBorder();
}
```

В данной функции вызывается метод базового класса, а для каждого объекта класса потока — функция UpdateBorder, изменяющая размеры рабочей области окна.

Приложение MUTEXES

Приложение MUTEXES демонстрирует использование объектов класса синхронизации CMutex. Приложение MUTEXES представляет собой диалоговое приложение, в котором создаются два объекта класса CWinThread, используемые для выполнения простейшей задачи под управлением пользователя.

При запуске приложения его окно списка заполняется одним из созданных приложением потоков, который преобразует хранящуюся в приложении целочисленную величину в строку и добавляет эту величину к списку. Другой поток приложения в это время постоянно инкрементирует эту величину.

Пользователь может независимо установить приоритет каждого из потоков при помощи соответствующих раскрывающихся списков. Кроме того, можно изменить класс приоритета процесса, что отразится сразу на обоих потоках и на всех задачах, запущенных в системе. Для этого используется раскрывающийся список **Process priority class:**.

При установке флажка **Synchronize** процессы используют объекты классов CMutex и CSingleLock, чтобы исключить одновременный доступ обоих процессов к строке или целочисленной переменной. Если флажок **Synchronize** сброшен, наблюдается нарушение порядка строк в списке. При установленном флажке **Synchronize** значения в списке плавно нарастают при добавлении в него новых элементов.

По умолчанию флажок **Pause** сброшен. Если он установлен, то выполнение обоих потоков приостанавливается, и содержимое окна списка может быть считано пользователем.

При установленном флажке **Show Counter Thread** поток счетчика выводит информацию о своей работе в окно списка.

Чтобы переписать текст проекта с примером на жесткий диск:

1. Выберите команду **Help**, **Index...** (Справка, Индекс). Появится окно **MSDN Library Visual Studio 6.0**.
2. Раскройте вкладку **Указатель**. В текстовое поле **Введите ключевое слово для поиска:** введите текст "MUTEXES sample (MFC)" и нажмите клавишу <Enter>. В окне появится текст справки по приложению.
3. Щелкните левой кнопкой мыши на синем тексте **Click to open or copy the MUTEXES project files** (Щелкните левой кнопкой мыши, чтобы открыть или скопировать файлы приложения MUTEXES).
4. Если библиотека MSDN не установлена на диске, появится диалоговое окно **Справка** с требованием вставить компакт-диск, помеченный "MSDN VS 6.0 CD1" в привод CD-ROM.
5. Вставьте компакт-диск, помеченный "MSDN VS 6.0 CD1" в привод CD-ROM и нажмите кнопку **OK**. Появится диалоговое окно **Visual C++ Samples** (Примеры Visual C++).
6. Нажмите кнопку **Копировать все...**. Появится диалоговое окно **Копирование**.
7. Выберите каталог, в который будет производиться копирование, и нажмите кнопку **OK**.
8. Согласитесь с созданием нового каталога, если вы копируете файлы проекта в новый каталог, и нажмите кнопку **OK** после того, как появится окно сообщения об успешном завершении копирования.
9. Нажмите кнопку **Закрыть** в диалоговом окне **Visual C++ Samples**.
10. Закройте окно **MSDN Library Visual Studio 6.0**.
11. Если библиотека Unicode не установлена, выполните команду **Build**, **Set Active Configuration...**. Появится диалоговое окно **Set Active Project Configuration** (Установка активной конфигурации проекта). В окне списка **Project configurations:** выделите строку **mutexes — Win32 Debug** и нажмите кнопку **OK**.
12. Нажмите клавишу <F5> и запустите приложение на исполнение. Появится диалоговое окно **Mutexes Sample**, изображенное на рис. 12.8.
13. Установите флажок **Synchronize**. Диалоговое окно **Mutexes Sample** примет вид, изображенный на рис. 12.9.
14. Установите флажок **Show counter thread**. Диалоговое окно **Mutexes Sample** примет вид, изображенный на рис. 12.10.
15. Установите флажок **Pause**. Потоки прекратят свою работу и вы сможете просмотреть содержимое окна списка.

Начнем рассмотрение данного приложения с файла threads.cpp, в котором содержится описание двух потоков, используемых в данном приложении.

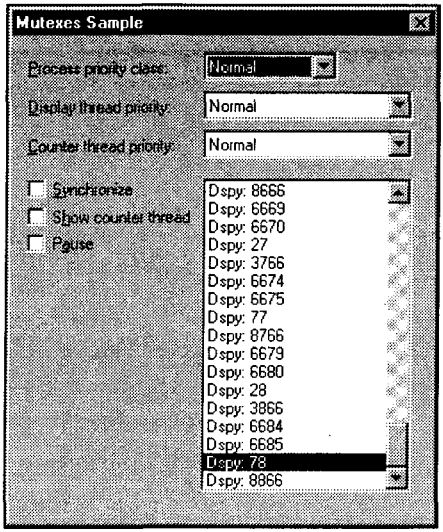


Рис. 12.8. Диалоговое окно **Mutexes Sample**

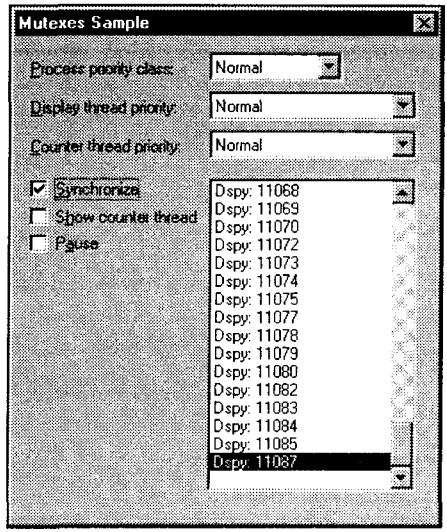


Рис. 12.9. Диалоговое окно **Mutexes Sample**, флажок **Synchronize** установлен

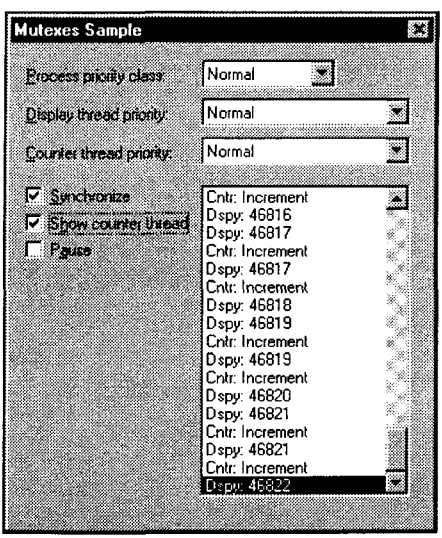


Рис. 12.10. Диалоговое окно **Mutexes Sample**, флажок **Show counter thread** установлен

Вообще-то в данном файле определяются три класса потоков, один из которых является базовым для двух других. В качестве базового используется класс `CExampleThread`, описание которого из заголовочного файла приведено ниже:

```
class CExampleThread : public CWinThread
{
  DECLARE_DYNCREATE(CExampleThread)
```

```
protected:
  CExampleThread();

public:
  CMutexesDlg* m_pOwner;
  BOOL m_bDone;
  void SetOwner(CMutexesDlg* pOwner) { m_pOwner = pOwner; };
};
```

Его защищенный конструктор реализован следующим образом:

```
CExampleThread::CExampleThread()
{
  m_bDone = FALSE;
  m_pOwner = NULL;
  m_bAutoDelete = FALSE;
}
```

За ним в файле реализации расположено описание класса потока счетчика CCounterThread, исполняющая функция которого имеет вид:

```
int CCounterThread::Run()
{
  BOOL fSyncChecked;
  unsigned int nNumber;

  if (m_pOwner == NULL)
    return -1;

  CSingleLock sLock(&(m_pOwner->m_mutex));

  while (!m_bDone)
  {
    // Get the status of the Synchronize
    // check box and save it.

    fSyncChecked = m_pOwner->IsDlgButtonChecked(IDC_SYNCHRONIZE);

    // If the user wants us synchronized, do it!
    if (fSyncChecked)
      sLock.Lock();

    _stscanf((LPCTSTR) m_pOwner->m_strNumber, _T("%d"), &nNumber);
    nNumber++;

    m_pOwner->m_strNumber.Empty();
    while (nNumber != 0)
    {
      m_pOwner->m_strNumber += (TCHAR) ('0' + nNumber%10);
```

```
      // A call to Sleep() here tells the system that we want
      // to relinquis the remainder of our time slice to
      // another thread.  this call is needed for
      // single-CPU systems so that the results of the
      // synchronization or lack thereof are obvious.
      // Normally, your programs would NOT call Sleep() here.

      Sleep(0);

      // get ready to get the next digit.
      nNumber /= 10;
    }

    // Characters were generated in reverse order,
    // reverse the string
    m_pOwner->m_strNumber.MakeReverse();

    if (fSyncChecked)
    {
    // If the user wants synchronization, do it.
      sLock.Unlock();
    }

    // If the user wants to display something
    // after each iteration, do it.

    if (m_pOwner->IsDlgButtonChecked(IDC_SHOWCNTRTHRD))
      m_pOwner->AddToListBox(_T("Cntr: Increment"));
  }

  m_pOwner->PostMessage(WM_CLOSE, 0, 0L);
  return 0;
}
```

Первый оператор данной функции проверяет, вызван ли данный поток главным окном приложения, и если это не так, то функция прекращает свою работу и возвращает значение, свидетельствующее о возникновении ошибки.

Затем создается объект класса `CSingleLock` с использованием объекта класса `CMutex`, хранящегося в классе главного окна приложения. После необходимой подготовки запускается рабочий цикл.

В данном цикле для определения состояния флажка **Synchronize** вызывается функция `CWnd::IsDlgButtonChecked`. Если этот флажок установлен, исполняющая функция потока запрашивает монопольный доступ к разделяемому ресурсу при помощи функции `CSingleLock::Lock`. Функция `_stscanf` преобразует разделяемый объект класса `CString` в целочисленное значение, сохраняе-

мое в переменной nNumber, которая сразу после этого инкрементируется. Затем разделяемый объект класса CString очищается с функцией CString::Empty.

Далее следует цикл преобразования переменной nNumber в строку. Для этого, пока переменная nNumber имеет значение большее нуля, ее младший разряд преобразуется в символ и добавляется к строке. Вызов функции Sleep(0) необходим, чтобы показать достоинства синхронизации на однопроцессорной машине. Эта функция передает управление другому процессу в самый неподходящий момент. В следующем кванте времени значение переменной nNumber уменьшается в десять раз, чтобы получить значение следующего разряда, и цикл повторяется. После этого порядок следования символов в строке инвертируется функцией CString::MakeReverse, что позволяет получить корректное преобразование целого числа в строку.

Если был запрошен монопольный доступ к разделяемому ресурсу, этот ресурс освобождается. Если в диалоговом окне установлен флажок **Show counter thread**, в список с помощью функции CMutexesDlg::AddToListBox добавляется строка, говорящая о том, что поток счетчика завершил свою работу.

Цикл завершает свою работу, когда переменная m_bDone принимает значение TRUE. После этого исполняющая функция потока посылает сообщение родительскому окну о его закрытии и возвращает нулевое значение.

Исполняющая функция класса потока CDisplayThread имеет следующий вид:

```
int CDisplayThread::Run()
{
  BOOL fSyncChecked;
  CString strBuffer;

  ASSERT(m_pOwner != NULL);
  if (m_pOwner == NULL)
    return -1;

  CSingleLock sLock(&(m_pOwner->m_mutex));

  while (!m_bDone)
  {
    // Determine whether the user wants the threads
    // to be synchronized.

    fSyncChecked = m_pOwner->IsDlgButtonChecked(IDC_SYNCHRONIZE);

    if (fSyncChecked)
      sLock.Lock();

    // Construct a string with the string form of the number.

    strBuffer = _T("Dspy: ");
    strBuffer += m_pOwner->m_strNumber;
```

```
    if (fSyncChecked)
      sLock.Unlock();

    // Add the string form of the number to the list box.
    m_pOwner->AddToListBox(strBuffer);
  }

  m_pOwner->PostMessage(WM_CLOSE, 0, 0L);
  return 0;
}
```

Она имеет структуру, аналогичную исполняющей функции класса потока CCounterThread, но при этом разделяемый ресурс просто добавляется к выводимой данным потоком строке, вывод которой осуществляется по освобождении данного ресурса.

В файле mutexesDlg.cpp содержится описание главного окна приложения. Реализация конструктора класса диалогового окна приведена ниже:

```
CMutexesDlg::CMutexesDlg(CWnd* pParent /*=NULL*/)
  : CDialog(CMutexesDlg::IDD, pParent),
   m_mutex(FALSE, NULL),
   m_strNumber(_T("0"))
{
  //{{AFX_DATA_INIT(CMutexesDlg)
    // NOTE: the ClassWizard will add member initialization here
  //}}AFX_DATA_INIT
  // Note that LoadIcon does not require a subsequent DestroyIcon in Win32
  m_hIcon = AfxGetApp()->LoadIcon(IDR_MAINFRAME);

  m_pCounterThread = NULL;
  m_pDisplayThread = NULL;
}
```

Функция OnInitDialog обрабатывает сообщение WM_INITDIALOG, посылаемое диалоговому окну функциями Create, CreateIndirect или DoModal, вызываемых для вывода диалогового окна на экран. В данном случае функция DoModal, посылающая это сообщение, располагается в функции CMutexesApp::InitInstance:

```
BOOL CMutexesDlg::OnInitDialog()
{
  CDialog::OnInitDialog();

  // Add "About..." menu item to system menu.

  // IDM_ABOUTBOX must be in the system command range.
  ASSERT((IDM_ABOUTBOX & 0xFFF0) == IDM_ABOUTBOX);
  ASSERT(IDM_ABOUTBOX < 0xF000);
```

```
CMenu* pSysMenu = GetSystemMenu(FALSE);
CString strAboutMenu;
strAboutMenu.LoadString(IDS_ABOUTBOX);
if (!strAboutMenu.IsEmpty())
{
  pSysMenu->AppendMenu(MF_SEPARATOR);
  pSysMenu->AppendMenu(MF_STRING, IDM_ABOUTBOX, strAboutMenu);
}

// Fill the Process Priority Class combo box and select
// Normal.

CComboBox* pBox;
pBox = (CComboBox*) GetDlgItem(IDC_PRIORITYCLASS);

ASSERT(pBox != NULL);
if (pBox != NULL)
{
  pBox->AddString(_T("Idle"));
  pBox->AddString(_T("Normal"));
  pBox->AddString(_T("High"));
  pBox->AddString(_T("Realtime"));
  pBox->SetCurSel(1); // Normal
}

// Fill the Display Thread Priority
// combo box and select Normal.

pBox = (CComboBox*) GetDlgItem(IDC_DSPYTHRDPRIORITY);
ASSERT(pBox != NULL);
if (pBox != NULL)
{
  pBox->AddString(_T("Idle"));
  pBox->AddString(_T("Lowest"));
  pBox->AddString(_T("Below normal"));
  pBox->AddString(_T("Normal"));
  pBox->AddString(_T("Above normal"));
  pBox->AddString(_T("Highest"));
  pBox->AddString(_T("Timecritical"));
  pBox->SetCurSel(3);  // Normal
}

// Fill the Counter Thread Priority
// combo box and select Normal.

pBox = (CComboBox*) GetDlgItem(IDC_CNTRTHRDPRIORITY);
ASSERT(pBox != NULL);
if (pBox != NULL)
```

```
  {
    pBox->AddString(_T("Idle"));
    pBox->AddString(_T("Lowest"));
    pBox->AddString(_T("Below normal"));
    pBox->AddString(_T("Normal"));
    pBox->AddString(_T("Above normal"));
    pBox->AddString(_T("Highest"));
    pBox->AddString(_T("Timecritical"));
    pBox->SetCurSel(3);  // Normal
  }

  // initialize the threads and let them start running!

  m_pDisplayThread = (CDisplayThread*)
    AfxBeginThread(RUNTIME_CLASS(CDisplayThread), THREAD_PRIORITY_NORMAL,
      0, CREATE_SUSPENDED);
  m_pDisplayThread->SetOwner(this);
  m_pDisplayThread->ResumeThread();

  m_pCounterThread = (CCounterThread*)
    AfxBeginThread(RUNTIME_CLASS(CCounterThread), THREAD_PRIORITY_NORMAL,
      0, CREATE_SUSPENDED);
  m_pCounterThread->SetOwner(this);
  m_pCounterThread->ResumeThread();

  return TRUE;  // return TRUE  unless you set the focus to a control
}
```

После вызова метода базового класса выполняется проверка, что значение идентификатора диалогового окна **About** находится в диапазоне системных команд. Затем с помощью функции `CWnd::GetSystemMenu` получается дескриптор копии текущего меню `Control`, с помощью функции `CString::LoadString` извлекается строковый ресурс и с помощью функции `CString::IsEmpty` проверяется, не является ли полученная строка пустой.

Если строка не является пустой, функция `CMenu::AppendMenu` добавляет горизонтальную разделительную линию в меню **Control**, а затем та же функция добавляет в меню команду отображения диалогового окна **About**.

Затем с помощью функции `CWnd::GetDlgItem` последовательно получаются указатели на три объекта класса `CComboBox`. Если эти указатели являются допустимыми, функция `CComboBox::AddString` заполняет раскрывающиеся списки, а с помощью функции `CComboBox::SetCurSel` выбирается значение, используемое по умолчанию.

Примечание

Почему для этого не использован редактор ресурсов, остается загадкой.

В завершение работы функции для каждого из потоков вызывается функция AfxBeginThread, создающая этот поток в приостановленном состоянии, вызывается функция SetOwner, определяющая владельца потока, и функция ResumeThread, запускающая данный поток на исполнение.

Функция OnSysCommand используется для вывода диалогового окна **About** и не имеет никакого отношения к теме данной главы.

Функция AddToListBox используется для добавления строки в окно списка:

```
void CMutexesDlg::AddToListBox(LPCTSTR szBuffer)
{
  CListBox* pBox = (CListBox*) GetDlgItem(IDC_DATABOX);
  ASSERT(pBox != NULL);

  if (pBox != NULL)
  {
    int x = pBox->AddString(szBuffer);
    pBox->SetCurSel(x);

    if (pBox->GetCount() > 100)
      pBox->DeleteString(0);
  }
}
```

В данной функции получается указатель на объект класса CListBox, проверяется его корректность, при помощи функции CListBox::AddString в окно списка добавляется новая строка, которая выбирается в качестве текущей. Если количество строк в списке превышает 100, то первая строка в списке уничтожается.

Функция OnClose вызывается при закрытии окна:

```
void CMutexesDlg::OnClose()
{
  int nCount = 0;
  DWORD   dwStatus;

  CButton* pCheck = (CButton*) GetDlgItem(IDC_PAUSE);
  BOOL bPaused = ((pCheck->GetState() & 0x003) != 0);

  if (bPaused == TRUE)
  {
    pCheck->SetCheck(0);
    m_pCounterThread->ResumeThread();
    m_pDisplayThread->ResumeThread();
  }
```

```
  if (m_pCounterThread != NULL)
  {
    VERIFY(::GetExitCodeThread(m_pCounterThread->m_hThread, &dwStatus));
    if (dwStatus == STILL_ACTIVE)
    {
      nCount++;
      m_pCounterThread->m_bDone = TRUE;
    }
    else
    {
      delete m_pCounterThread;
      m_pCounterThread = NULL;
    }
  }

  if (m_pDisplayThread != NULL)
  {
    VERIFY(::GetExitCodeThread(m_pDisplayThread->m_hThread, &dwStatus));
    if (dwStatus == STILL_ACTIVE)
    {
      nCount++;
      m_pDisplayThread->m_bDone = TRUE;
    }
    else
    {
      delete m_pDisplayThread;
      m_pDisplayThread = NULL;
    }
  }

  if (nCount == 0)
    CDialog::OnClose();
  else
    PostMessage(WM_CLOSE, 0, 0);
}
```

Здесь функция `CWnd::GetDlgItem` возвращает указатель на флажок **Pause**, а функция `CButton::GetState` проверяет его состояние. Если этот флажок установлен, то функция `CButton::SetCheck` сбрасывает его, а функция `CWinThread::ResumeThread` запускает оба потока на исполнение.

Если поток счетчика существует, функция `GetExitCodeThread` позволяет получить его статус завершения. Если поток продолжает свою работу, то счетчик `nCount` инкрементируется, а переменной `m_bDone`, являющейся членом класса потока, присваивается значение `TRUE`, извещающее поток о необходимости завершить свою работу. В противном случае объект класса потока уничтожается, а его указателю присваивается нулевое значение. Подобная процедура выполняется и для объекта класса потока отображения.

Если счетчик nCount имеет нулевое значение, то есть потоки остановлены и их классы уничтожены, то вызывается метод базового класса. В противном случае посылается сообщение WM_CLOSE, снова вызывающее данную функцию.

Функция OnPriorityClassChange изменяет класс приоритета текущего процесса:

```
void CMutexesDlg::OnPriorityClassChange()
{
  DWORD dw;
  CComboBox* pBox = (CComboBox*) GetDlgItem(IDC_PRIORITYCLASS);
  int nCurSel = pBox->GetCurSel();

  switch (nCurSel)
  {
  case 0:
    dw = IDLE_PRIORITY_CLASS;
    break;

  case 1:
  default:
    dw = NORMAL_PRIORITY_CLASS;
    break;

  case 2:
    dw = HIGH_PRIORITY_CLASS;
    break;

  case 3:
    dw = REALTIME_PRIORITY_CLASS;
    break;
  }

  SetPriorityClass(GetCurrentProcess(), dw);
}
```

Здесь после определения текущего содержимого текстового поля раскрывающегося списка **Process priority class:** в зависимости от полученного значения выбирается значение класса приоритета, которое и используется для установки класса приоритета процесса при помощи функции SetPriorityClass. Для получения дескриптора текущего процесса вызывается функция GetCurrentProcess.

Функция OnPriorityChange устанавливает приоритет потока, идентификатор раскрывающегося списка которого передается ей в качестве аргумента:

```
void CMutexesDlg::OnPriorityChange(UINT nID)
{
  ASSERT(nID == IDC_CNTRTHRDPRIORITY || nID == IDC_DSPYTHRDPRIORITY);
```

```
    DWORD dw;
    CComboBox* pBox = (CComboBox*) GetDlgItem(nID);
    int nCurSel = pBox->GetCurSel();

    switch (nCurSel)
    {
    case 0:
      dw = (DWORD) THREAD_PRIORITY_IDLE;
      break;

    case 1:
      dw = (DWORD) THREAD_PRIORITY_LOWEST;
      break;

    case 2:
      dw = (DWORD) THREAD_PRIORITY_BELOW_NORMAL;
      break;

    case 3:
    default:
      dw = (DWORD) THREAD_PRIORITY_NORMAL;
      break;

    case 4:
      dw = (DWORD) THREAD_PRIORITY_ABOVE_NORMAL;
      break;

    case 5:
      dw = (DWORD) THREAD_PRIORITY_HIGHEST;
      break;

    case 6:
      dw = (DWORD) THREAD_PRIORITY_TIME_CRITICAL;
      break;
    }

    if (nID == IDC_CNTRTHRDPRIORITY)
      m_pCounterThread->SetThreadPriority(dw);
    else
      m_pDisplayThread->SetThreadPriority(dw);
}
```

Данная функция проверяет передаваемый ей идентификатор ресурса, и, если этот идентификатор не является идентификатором одного из раскрывающихся списков потока, отладочная версия данной программы выдает сообщение об ошибке.

Далее структура этой функции аналогична структуре функции `OnPriorityClassChange` за тем исключением, что в ней устанавливается при-

оритет только того потока, идентификатор раскрывающегося списка которого передан ей в качестве аргумента.

Приоритет потока устанавливается функцией `CWinThread::SetThreadPriority`.

Функция `OnPause` вызывается при нажатии кнопки **Pause**, то есть при установке или снятии соответствующего флажка:

```
void CMutexesDlg::OnPause()
{
  CButton* pCheck = (CButton*) GetDlgItem(IDC_PAUSE);
  BOOL bPaused = ((pCheck->GetState() & 0x003) != 0);

  if (bPaused)
  {
    m_pCounterThread->SuspendThread();
    m_pDisplayThread->SuspendThread();
  }
  else
  {
    m_pCounterThread->ResumeThread();
    m_pDisplayThread->ResumeThread();
  }
}
```

Здесь функция `CWnd::GetDlgItem` возвращает указатель на объект класса `CButton`, который является аргументом функции `CButton::GetState`, позволяющей определить текущее состояние данной кнопки. Если данный флаг установлен, то для каждого из потоков вызывается функция `CWinThread::SuspendThread`, приостанавливающая выполнение данного потока, а если флаг сброшен, то вызывается функция `CWinThread::ResumeThread`, возобновляющая выполнение.

Справка в приложении

Наличие хорошо организованной справочной системы является одной из основных отличительных особенностей профессионального приложения. Отсутствие справки допустимо только в приложениях, создаваемых пользователем для решения текущих задач, и только в том случае, если он не собирается использовать, а тем более модифицировать это приложение в дальнейшем. В противном случае ему постоянно придется вспоминать, какие функции заложены в данном приложении, и как ими можно воспользоваться. В некоторых случаях вместо справочной системы можно использовать комментарии в тексте программы. Но этот способ применим только в программах, написанных для "внутреннего употребления". В остальных случаях комментарии в тексте программы должны не заменять, а дополнять справочную систему.

Несмотря на особую роль справочной системы в оценке приложения потребителем многие программисты откладывают ее создание до полного завершения работы по отладке проекта. Хотя во многих случаях они не могут создать ее раньше вследствие некорректно выданного задания, что приводит к необходимости вносить кардинальные изменения в практически готовый проект. В этом случае создание справочной системы на более ранних стадиях приведет к необходимости написания нескольких практически независимых друг от друга справочных систем для каждой из промежуточных версий приложения.

Описание справочной системы приложения

Microsoft Windows предоставляет специальные функции и сообщения, позволяющие приложению, работающему под управлением данной операционной системы, создать свою справочную систему. Большинство подобных

приложений используют различные методы доступа к справочной системе. Для создания каждой конкретной справки необходимо ответить на следующие вопросы:

- Каким образом она будет вызываться?
- В какое окно будет выводиться справочная информация?
- В какой форме она будет представлена?
- Каким образом справочная система будет включена в текст программы?

Ниже приведены возможные ответы на каждый из поставленных вопросов. Какой из них выбрать в каждом конкретном случае, решает программист.

Способы доступа к справочной системе

Различные виды справочной информации подразумевают различные способы доступа к ней. Например, для получения справки о назначении данной кнопки панели инструментов достаточно просто поместить на нее указатель мыши, поскольку в данном случае ясно, какая информация может понадобиться пользователю. Но чтобы предоставить пользователю более полную информацию, например, по ключевому слову, не обойтись без вызова специального диалогового окна. Основными способами выдачи запроса на получение справочной информации являются:

- Вызов команды меню

  Многие приложения предоставляют пользователю доступ к справочной системе приложения через соответствующее меню в главном окне программы. Когда пользователь выбирает команду меню, соответствующая процедура окна получает сообщение `WM_COMMAND`, определяющее соответствующий запрос к справочной системе. В ответ на этот запрос приложение выводит соответствующую информацию, например, список разделов, индекс или вводную информацию к разделу

- Вызов с клавиатуры

  После нажатия пользователем клавиши <F1> приложение получает соответствующее сообщение Windows. При этом система посылает сообщение `WM_HELP` окну, имеющему в данный момент фокус ввода. Если это окно является дочерним (например, элементом управления диалогового окна), функция `DefWindowProc` передает это сообщение родительскому окну. Если в момент нажатия клавиши <F1> фокус ввода принадлежал меню, система посылает сообщение окну, связанному с данным меню. В ответ на это сообщение приложение выводит информацию, связанную с данным окном, элементом управления или меню, которому принадлежал фокус ввода. Например, при выделении элемента управления в диалоговом окне и нажатии клавиши <F1> приложение выводит информацию о данном элементе управления

Аргумент lParam сообщения формата WM_HELP представляет собой указатель на объект структуры HELPINFO, содержащей детальную информацию об объекте, которому в данный момент принадлежит фокус ввода. Эта информация используется для определения характера выводимой информации. Кроме того, в объекте структуры HELPINFO содержатся координаты указателя мыши в момент нажатия клавиши <F1>. Эта информация также может использоваться для определения характера выводимой информации

- Вызов с помощью мыши

  Чтобы получить справочную информацию с помощью мыши, необходимо щелкнуть ее правой кнопкой в окне или щелкнуть левой кнопкой на кнопке в окне или элементе управления. Приложение выведет информацию, связанную с данным окном или элементом управления.

  При щелчке правой кнопкой мыши система посылает сообщение WM_CONTEXTMENU окну, в котором находился в это время указатель мыши. Если это окно является дочерним, то функция DefWindowProc передает сообщение родительскому окну. В сообщении WM_CONTEXTMENU содержатся координаты указателя мыши. Его горизонтальная координата содержится в младшем слове аргумента lParam, а вертикальная — в старшем. Если пользователь щелкнул на элементе управления, то в аргументе wParam передается дескриптор этого элемента управления.

  При щелчке левой кнопкой мыши на кнопке , расположенной в заголовке окна, система посылает сообщение WM_HELP. Чтобы добавить такую кнопку в заголовок окна, необходимо задать стиль WS_EX_CONTEXTHELP в функции CreateWindowEx при создании окна. В аргументе lParam сообщения WM_HELP содержится указатель на объект структуры HELPINFO, содержащей детальную информацию об объекте, которому в данный момент принадлежит фокус ввода, включая координаты указателя мыши в момент щелчка.

  Кнопку со знаком вопроса рекомендуется использовать только в диалоговых окнах

Способы представления справочной информации

Получив запрос на вывод справочной информации, приложение должно вывести ее на экран. При выводе справочной информации в пользовательских приложениях рекомендуется использовать стандартный интерфейс базовых приложений Windows, для доступа к которому используется функция WinHelp. Этой функции следует передать выводимую информацию и указать форму ее вывода.

Справочную информацию можно выводить в окна различных типов:

- Окно **Справочная система**

  В этом диалоговом окне, показанном на рис. 13.1, пользователь может выполнить поиск по ключевому слову, просмотреть оглавление или найти в справочных текстах определенное слово

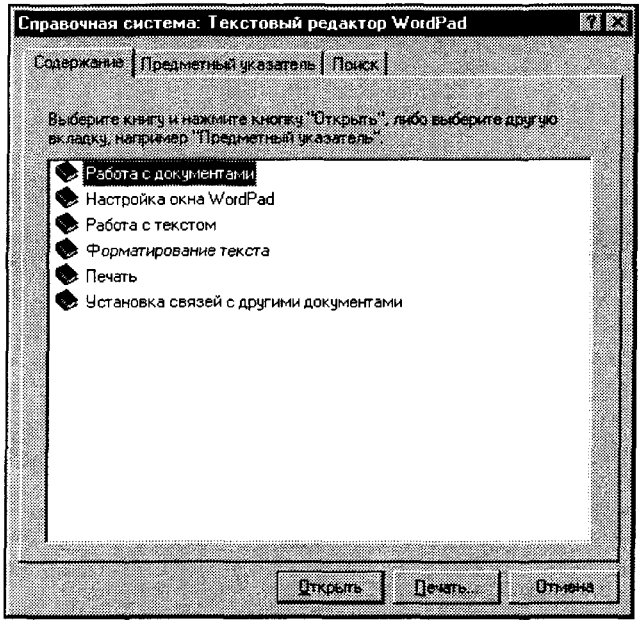


Рис. 13.1. Диалоговое окно **Справочная система**

- Обычное окно справки

  В окне (рис. 13.2) выводится текст конкретной справки. Это окно можно свернуть в пиктограмму, развернуть, изменить его размер или закрыть. Обычно окно справки всегда располагается поверх других окон, что позволяет использовать его в качестве подсказки при выполнении некоторых действий, описанных в данном окне. Иногда это окно справки называют вторичным

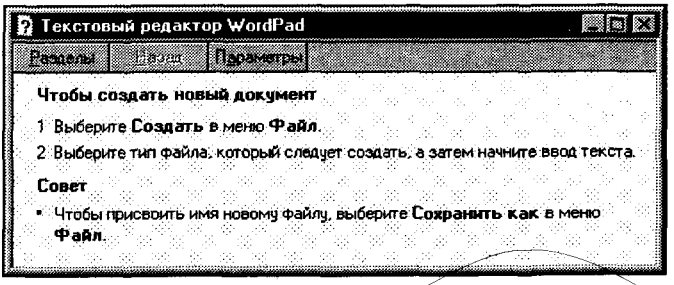


Рис. 13.2. Окно справки

- Всплывающее окно

  Окно (рис. 13.3) используется для вывода справочной информации небольшого объема или кратких пояснений. Это окно не имеет элементов управления и закрывается при щелчке мышью за его пределами. Его нельзя минимизировать или переместить

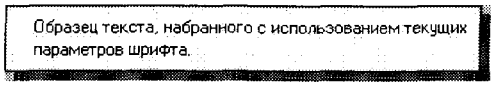


Рис. 13.3. Всплывающее окно справки

Особой формой справки является поиск слов или комбинаций слов непосредственно в тексте справки.

При первой попытке перейти на вкладку **Поиск** окна **Справочная система** запускается процесс настройки поиска. При этом в справочной системе приложения создается база данных ключевых слов по файлам справочной системы.

Чтобы настроить справочную систему приложения на поиск информации по ключевому слову:

1. Откройте окно **Справочная система** и перейдите в нем на вкладку **Поиск**. На экране появится диалоговое окно мастера Настройка поиска, изображенное на рис. 13.4.

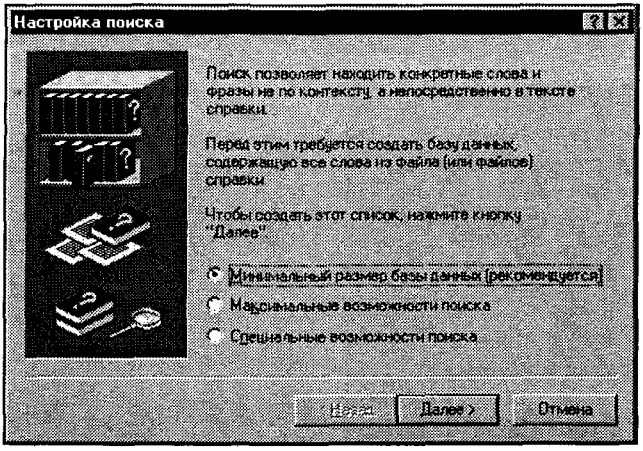


Рис. 13.4. Диалоговое окно мастера Настройка поиска

2. Установите переключатель в требуемое положение и нажмите кнопку **Далее>**. Появится последнее окно мастера Настройка поиска (рис. 13.5).
3. Нажмите кнопку **Готово**. Появится диалоговое окно **Справочная система**, раскрытое на вкладке **Поиск**, как показано на рис. 13.6.
4. Введите искомое слово или набор слов в текстовое поле **1. Введите искомое слово или слова** или выберите его в окне списка 2. **При желании вы-**

берите нужное слово из списка или в окне списка **3. Выберите нужный раздел и нажмите кнопку "Вывести"**. Появится список разделов, связанных с данным ключевым словом или набором слов.

5. В окне списка **3. Выберите нужный раздел и нажмите кнопку "Вывести"** выделите заголовок раздела и нажмите кнопку **Вывести**.
6. На экране появится окно справки, показанное на рис. 13.7.

При повторном открытии вкладки **Поиск** в окне **Справочная система** процесс настройки пропускается и работа начинается с п. 4. Чтобы запустить процесс настройки поиска из вкладки **Поиск**, нажмите кнопку **Перестроить...**.

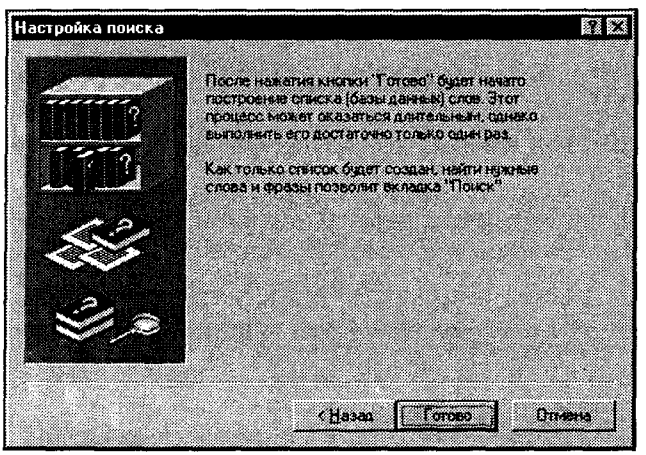


Рис. 13.5. Последнее окно мастера Настройка поиска

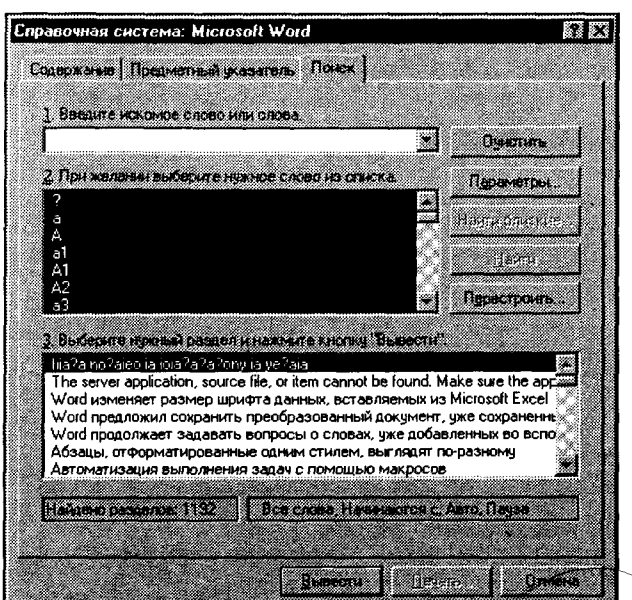


Рис. 13.6. Окно **Справочная система**, вкладка **Поиск**

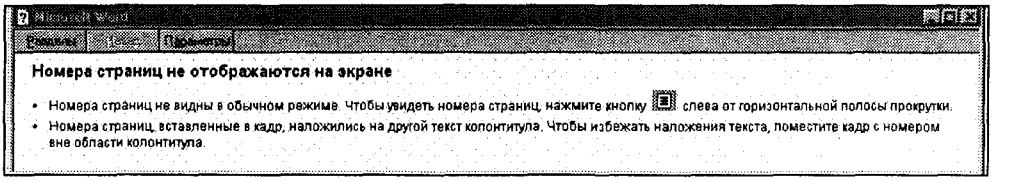


Рис. 13.7. Окно справки

Как видно из рис. 13.6, в окне списка списка **3. Выберите нужный раздел и нажмите кнопку "Вывести"** присутствует псевдографика. Поскольку это окно принадлежит русифицированному редактору Word, запущенному в русской версии операционной системы Windows 95, можно сделать вывод, что присутствие псевдографики в справочной системе Windows является органичным, и пользователям просто нужно привыкнуть к ее использованию. Соответствующая работа, как видно, уже ведется.

Для вывода справочной информации необходимо создать файл, содержащий эту информацию, и передать его имя в качестве аргумента функции `WinHelp`. Файл справки должен иметь расширение .hlp и соответствующий формат.

Данный файл содержит описание одного или нескольких разделов. Под разделом понимается самостоятельная единица информации, содержащая описание некоторого вопроса, последовательность действий, описание оглавления и т. п. Каждый раздел должен быть однозначно идентифицирован таким образом, чтобы справочная система Windows могла найти его по запросу. Для этого приложение обычно использует контекстные идентификаторы (уникальные целочисленные значения).

Создатель файла справки должен в явном виде указать контекстный идентификатор раздела в секции `[MAP]` файла проекта, используемого для компиляции файла справки.

Если в имени файла справки в функции `WinHelp` не задан путь к файлу, то функция ищет этот файл в каталоге \Help или в рабочем каталоге приложения. Кроме того, функция `WinHelp` может использовать имя файла справки, указанное в системном реестре по адресу:

```
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\Help
```

Для использования системного реестра необходимо создать переменную с тем же именем, что и файл справки. Значением этой переменной должен быть каталог, в котором расположен файл справки.

Если функция `WinHelp` не может найти файл справки, она выводит окно сообщения **Справочная система Windows**, изображенное на рис. 13.8.

Данное окно сообщает о том, что указанный в приложении файл справки отсутствует. При нажатии кнопки **Да** появляется диалоговое окно **Открытие файла**, изображенное на рис. 13.9.

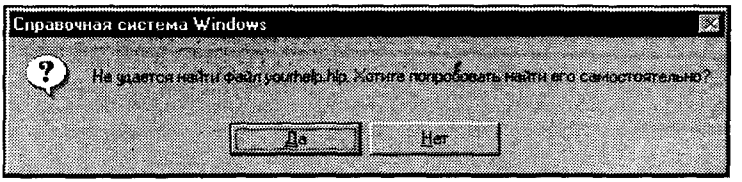


Рис. 13.8. Окно сообщения **Справочная система Windows**

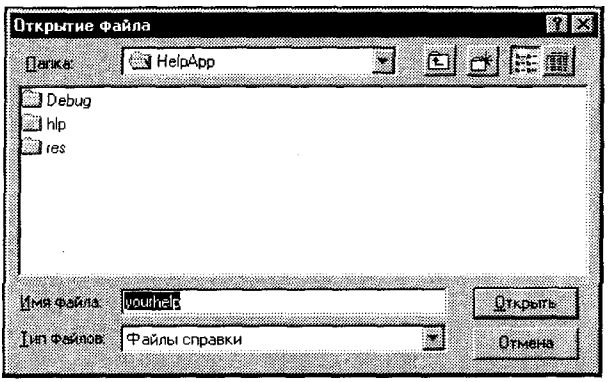


Рис. 13.9. Диалоговое окно **Открытие файла**

В этом диалоговом окне пользователь может указать операционной системе путь к файлу справки. Поскольку функция `WinHelp` сохраняет переданную ей информацию в системном реестре, при последующем запуске приложения она не будет спрашивать о местоположении данного файла справки.

Вот пример использования функции `WinHelp` в Windows NT:

```
HWND hwnd;      // Дескриптор главного окна приложения
BOOL bResult    // Используется для проверки результата

bResult = WinHelp(hWnd, "WINNT.HLP", HELP_CONTENTS, 0L);
```

Ниже приведен пример открытия файла справки в Windows NT и поиск темы в нем по ключевой строке.

```
HWND hwnd;      // Дескриптор главного окна приложения
BOOL bResult    // Используется для проверки результата

bResult = WinHelp(hWnd, "WINNT.HLP", HELP_KEY,
                  (DWORD) "finding topics");
```

Для вывода диалогового окна **Справочная система** при вызове функции `WinHelp` используется команда `HELP_FINDER`. Как уже говорилось, в диалоговом окне **Справочная система** пользователь может выбирать разделы для получения справочной информации с использованием заголовков разделов, ключевых слов, связанных с данными темами, или слов и фраз, встречающихся в тексте данных разделов.

Это диалоговое окно обычно выводится, если пользователь выбирает соответствующую команду меню. Кроме того, приложение может выводить данное диалоговое окно при нажатии пользователем клавиши <F1>, если в данный момент времени ни одно из окон, элементов управления или меню данного приложения не имеют фокуса ввода или не являются активными.

Старые приложения используют команду HELP_CONTENTS в функции WinHelp для раскрытия панели **Содержание** в диалоговом окне **Справочная система** и команду HELP_INDEX для раскрытия в нем панели **Предметный указатель**. Теперь использовать эти команды не рекомендуется. Вместо них следует использовать команду HELP_FINDER.

Для вывода информации по конкретной теме в функции WinHelp используется команда HELP_CONTEXT и указывается контекстный идентификатор данного раздела. Обычно команда HELP_CONTEXT вызывается в ответ на выбор пользователем конкретного раздела в диалоговом окне **Справочная система** и редко применяется для предоставления информации по конкретному элементу управления или команде меню. При помощи данной команды пользователь может продолжить поиск информации в справочной системе, не возвращаясь в приложение.

Команда HELP_CONTEXT вызывает справочную систему Windows, в которой пользователь может продолжить поиск по другим темам. Информация по этому сообщению обычно выводится в окне справки, имеющем заголовок, системное меню, кнопки минимизации и максимизации, главное меню, дополнительную панель навигации, рамку изменяемого размера и рабочую область. Текст справки размещается в рабочей области окна. Для перемещения по файлу справки могут быть использованы расположенные в тексте ссылки и кнопки навигации в главном окне.

Для вывода всплывающего окна справки в качестве аргумента функции WinHelp необходимо указать команду HELP_WM_HELP или команду HELP_CONTEXTMENU. Чтобы вывод данной справки не препятствовал дальнейшей работе пользователя с приложением, всплывающее окно справки удаляется с экрана после первого же нажатия пользователем любой клавиши или первого же щелчка левой кнопкой мыши.

Команда HELP_WM_HELP используется для обработки сообщения WM_HELP в окне элемента управления. Поскольку большинство элементов управления передают сообщение WM_HELP функции DefWindowProc, то оно обрабатывается функцией соответствующего класса диалогового окна (или функцией класса родительского окна).

Вместо того чтобы использовать отдельный контекстный идентификатор, функция обработки данного сообщения в классе диалогового окна должна передать массив, содержащий пары элементов управления и соответствующих им контекстных идентификаторов, функции WinHelp вместе с дескриптором элемента управления, содержащегося в переменной hItemHandle,

являющейся членом структуры HELPINFO, передаваемой в качестве аргумента сообщения WM_HELP. Функция определяет идентификатор элемента управления, для которого посылается сообщение WM_HELP, и использует соответствующий ему контекстный идентификатор для вывода на экран справочной информации.

Команда HELP_CONTEXTMENU используется для обработки сообщения WM_CONTEXTMENU. Поскольку большинство элементов управления передают сообщение WM_CONTEXTMENU функции DefWindowProc, то данное сообщение обрабатывается функцией соответствующего класса диалогового окна (или функцией класса родительского окна).

В функции обработки данного сообщения создается массив, содержащий пары элементов управления и соответствующих им контекстных идентификаторов. При вызове функции WinHelp в аргументе wParam передается дескриптор элемента управления. Таким образом, функция может определить контекстный идентификатор, соответствующий данному дескриптору, и вывести на экран соответствующую справочную информацию. В отличие от команды HELP_WM_HELP, команда HELP_CONTEXTMENU сначала раскрывает контекстное меню с единственной командой **What's This?** (Что это такое?). Если пользователь выбирает эту команду, функция WinHelp выводит информацию по данной теме. В противном случае запрос на получение справочной информации отменяется.

Для получения справки в всплывающем окне можно использовать команду HELP_CONTEXTPOPUP, указав ей контекстный идентификатор раздела справки. Эта команда работает аналогично команде HELP_CONTEXT, но использует для вывода информации всплывающее окно. Приложение может использовать эту команду, получив сообщение WM_HELP при выводе справочной информации по командам меню и для окон, не являющихся элементами управления диалогового окна. Для более эффективного использования данной команды приложение должно назначить контекстные идентификаторы этим командам меню и окнам.

Контекстный идентификатор может быть назначен любому окну и любой команде меню в приложении. При посылке сообщения WM_HELP родительскому окну система включает в объект структуры HELPINFO контекстный идентификатор. Родительское окно передает этот контекстный идентификатор функции WinHelp для вывода соответствующего раздела справки.

Функция SetWindowContextHelpId позволяет назначить контекстный идентификатор окну или элементу управления, а функция SetMenuContextHelpId — команде меню. Контекстный идентификатор окна или команды меню может быть получен при помощи функций GetWindowContextHelpId и GetMenuContextHelpId, соответственно.

Справочная система Windows поддерживает поиск по ключевому слову в файле справки. Ключевое слово представляет собой строку, с которой связаны один или несколько разделов в файле справки.

Для поиска по ключевому слову приложение использует команды HELP_KEY, HELP_PARTIALKEY или HELP_MULTIKEY в функции WinHelp. При вызове функции WinHelp необходимо указать команду, ключевое слово, имя файла справки и дескриптор родительского окна. В любом случае, если в файле справки обнаружено только одно совпадение ключевого слова, функция WinHelp выводит справочную информацию по этой теме.

Поиск по ключевому слову ведется из вкладки **Предметный указатель** окна **Справочная система**, изображенного на рис. 13.10.

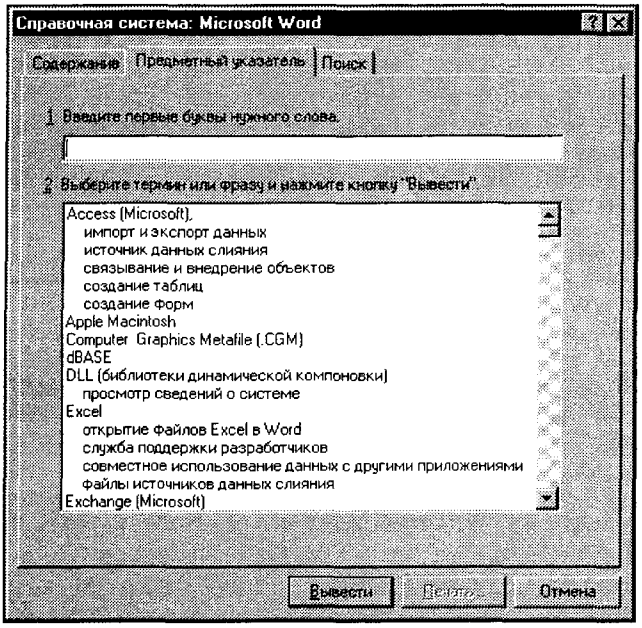


Рис. 13.10. Вкладка **Предметный указатель** окна **Справочная система**

В текстовое поле **1. Введите первые буквы нужного слова** вводится искомое ключевое слово. По мере его ввода содержимое в окне списка **2. Выберите термин или фразу и нажмите кнопку "Вывести"** прокручивается, при этом в нем выделено первое ключевое слово, совпадающее с введенным пользователем. Как только в окне списка **2. Выберите термин или фразу и нажмите кнопку "Вывести"** будет выделено требуемое ключевое слово, пользователь может нажать кнопку **Вывести**.

Если найдено несколько совпадений, функция выводит диалоговое окно **Найденные разделы**, изображенное на рис. 13.11. В этом диалоговом окне пользователь может выбрать раздел, по которому он хочет получить справку.

Если в файле справки не содержатся разделы, связанные с данным ключевым словом, то функция WinHelp или продолжает выводить предметный указатель (для команд HELP_KEY и HELP_PARTIALKEY) или выдает сообщение об ошибке (для команды HELP_MULTIKEY).

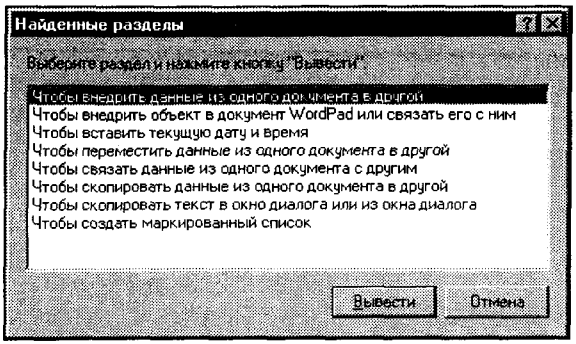


Рис. 13.11. Диалоговое окно **Найденные разделы**

Приложение может производить поиск по нескольким ключевым словам при однократном вызове функции `WinHelp`. При этом различные ключевые слова разделяются точкой с запятой.

Примечание

Поиск по нескольким ключам невозможен в файлах справки, созданных для приложений Windows версии 3.x. Если с файлом справки, указанным в качестве основного, связан файл контекста (с расширением .cnt), в котором содержатся команды `:Index` или `:Link`, то возможен поиск по нескольким файлам справки.

Команда `HELP_KEY` в функции `WinHelp` обеспечивает поиск ключевого слова во всех файлах, указанных в данной команде. Использование в данной функции команд `HELP_MULTIKEY` и `HELP_PARTIALKEY` обеспечивает поиск во всех файлах, кроме указанных в командах `:Link`.

По умолчанию справочная система Windows использует только таблицу ключевых слов, отмеченную сноской "K" в исходном файле справки. Пользователь может указать справочной системе Windows на необходимость создания дополнительных таблиц ключевых слов, добавляя при определении ключевого слова в исходном файле справки другие сноски, отличные от "K". (При этом следует помнить, что сноска "A" зарезервирована для других целей.) Любая дополнительная таблица ключевых слов должна быть определена ключевым словом `MULTIKEY` в разделе `[OPTIONS]` файла проекта при компиляции файла справки.

Для вывода в панели **Предметный указатель** таблицы ключевых слов, отличной от таблицы **K**, приложение может использовать команду `HELP_SETINDEX` в функции `WinHelp`.

Для проведения поиска по ключам, содержащимся в альтернативных таблицах ключевых слов необходимо использовать команду `HELP_MULTIKEY`. Ключевое слово и таблица ключевых слов определяются с объекта структуры `MULTIHELP`, передаваемом функции `WinHelp`.

Для вывода справочной информации функция `WinHelp` использует окно, определяемое сноской ">" данного раздела, окно, определяемое командой

:Base в контекстном файле, или главное окно справочной системы. Если главное окно справочной системы в момент вызова функции WinHelp уже открыто для другого файла справки, функция скрывает главное окно на время поиска. В этом случае последовательное закрытие окон **Найденные разделы** и **Справочная система** закрывает главное окно справочной системы.

Главное окно справочной системы Windows иногда называют также первичным окном. Справочная система Windows может выводить информацию и во вторичные окна. В отличие от первичного окна, вторичное не содержит контекстного меню. Вторичное окно может содержать панель навигации, в которую могут быть добавлены дополнительные кнопки. Кроме того, для этого окна может быть выбран режим, в котором его высота и ширина автоматически устанавливаются справочной системой Windows исходя из объема выводимой информации.

Вторичные окна справки должны определяться в разделе [WINDOWS] файла проекта справочной системы. При этом необходимо указать имя и, возможно, начальный размер и положение каждого окна. Чтобы сообщить справочной системе Windows о необходимости вывести данную тему во вторичном окне, надо добавить к имени файла справки угловую скобку (>) и имя вторичного окна. Полученная строка передается в качестве аргумента функции WinHelp.

Для изменения размера и положения первичного или вторичного окна приложение должно внести соответствующие изменения в объект структуры HELPWININFO и передать команду HELP_SETWINPOS функции WinHelp. Структура HELPWININFO содержит имя окна, его размер и положение на экране.

Для обучения пользователя методам решения некоторых задач, встающих перед ним в процессе работы с приложением, используется последовательность окон, каждое из которых содержит описание одного из шагов выполнения данной задачи. Эти окна выводятся в последовательности, определяемой описанными в них шагами. Эти окна, кроме текста, содержат кнопки, связанные с макросами TCard, позволяющие пользователю выбирать следующее окно справки в зависимости от результатов своих действий. Данные последовательности окон могут состоять только из вторичных окон, и в них не могут содержаться ссылки на другие разделы, содержащиеся в файле справки.

Вывод последовательности окон инициируется функцией WinHelp, в которой наряду с другими командами, такими как HELP_CONTEXT, указана команда HELP_TCARD. При нажатии пользователем кнопок окна справки, вызове макросов TCard или закрытии окна справочная система Windows посылает приложению сообщение WM_TCARD. Аргумент wParam данного сообщения определяет нажатую кнопку или предпринятое пользователем действие, а аргумент lParam содержит дополнительную информацию, структура которой определяется значением аргумента wParam.

Для закрытия справочной системы Windows из приложения должна поступить явная команда, чтобы освободить все ресурсы, используемые для поиска информации в файле справки. Приложение может в любой момент закрыть справочную систему, вызвав функцию `WinHelp` и передав ей в качестве аргумента команду `HELP_QUIT`. Этот метод неприменим к всплывающим окнам справки, которые не могут закрываться приложением.

Если любое приложение вызвало функцию `WinHelp`, то оно должно завершить работу со справочной системой до закрытия своего главного окна (например, в ответ на сообщение `WM_DESTROY` в главном окне приложения). Для прекращения работы со справочной системой достаточно всего один раз вызвать функцию `WinHelp` независимо от того, сколько файлов справки было открыто на данный момент. Справочная система продолжает работу до тех пор, пока все приложения или библиотеки динамической компоновки (dll) не завершили с ней работу.

Чтобы закрыть последовательность окон в справочной системе Windows, необходимо одновременно передать функции `WinHelp` команды `HELP_TCARD` и `HELP_QUIT`. Приложение не должно закрывать последовательность окон в справочной системе, если до этого ее закрыл пользователь. Справочная система Windows извещает приложение о закрытии пользователем последовательности окон, посылая сообщение `WM_TCARD`, в котором аргумент `wParam` имеет значение `IDCLOSE`.

Формы представления справочной информации

Форма представления справочной информации в основном определяется причинами, по которым эта информация вызывается. Справочная информация может быть представлена в форме:

- *Контекстной справки*

  Данная форма служит для ответа на вопросы типа "Для чего предназначена данная кнопка?" или "Что нужно ввести в это текстовое поле?"

- *Справки о методике решения задачи*

  Данная форма служит для получения описания алгоритма действий при решении конкретной задачи, решаемой данным приложением. Например, о том, как распечатать документ

- *Справки по ключевому слову*

  Данная форма служит для предоставления развернутой информации по конкретному вопросу

В приложениях Windows используются два основных типа контекстной справки. Первый из них, вызываемый при нажатии клавиши <F1>, вызывает систему `WinHelp`, используя контекстный идентификатор, соответствующий активному объекту.

Второй из них вызывается при нажатии комбинации клавиш <Shift>+<F1>. В этом режиме указатель мыши приобретает форму курсора справки (комбинация стрелки и знака вопроса), и для получения справки по конкретному элементу управления пользователю нужно установить на нем курсор справки и щелкнуть левой кнопкой мыши.

Нажатие клавиши <F1> позволяет получить справку по активному окну или команде меню. При этом приложение не переводится в специальный режим вывода справочной информации. Эта информация исчезает при первом же активном действии пользователя (нажатии клавиши или щелчке мыши).

Справка по методике решения задачи и справка по ключевому слову иногда объединяются под общим наименованием *командной справки*, поскольку в большинстве случаев для их вызова используется команда меню.

Программирование справочной системы

Вопрос о способах программирования справочной системы приложения является наиболее важным для программиста, поскольку именно эта задача полностью возлагается на его плечи. Именно поэтому ответ на него вынесен в отдельный раздел.

Для обеспечения возможности работы со справочной системой в приложении необходимо при его создании с использованием мастера AppWizard на четвертом шаге его работы установить флажок **Context-sensitive Help**, как показано на рис. 13.12.

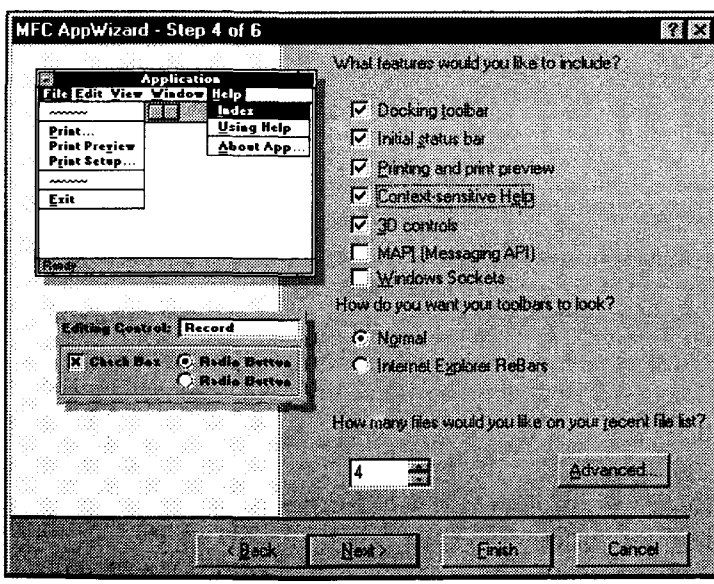


Рис. 13.12. Установка флажка **Context-sensitive Help**

Компоненты справочной системы

При создании справочной системы используется большое количество различных файлов. Большинство из них являются промежуточными для создания файла справки, имеющего расширение .hlp. Ниже приведены расширения файлов, создаваемых мастером AppWizard и компилятором справки:

- .h — файл заголовка (header file) содержит определения идентификаторов ресурсов и идентификаторы разделов справки, которые будут использоваться в программах
- .hm — файл адресации справок (help mapping file) содержит идентификаторы разделов справки. Файл с именем приложения и данным расширением генерируется при каждой компиляции приложения
- .rtf — файл расширенного текстового формата (rich text format). Содержит тексты справок по каждому разделу
- .cnt — файл таблицы содержания, используемый для подготовки вкладки **Предметный указатель** (**Contents**) в диалоговом окне **Справочная система** (**Help Topics**). (Файл с именем приложения и данным расширением, необходимо поставлять вместе с файлом .hlp в составе приложения)
- .hpj — файл проекта справочной системы (help project file), с именем приложения и данным расширением, объединяет файлы с расширениями .hm и .rtf, используемые при создании файла .hlp

В процессе использования приложения справочная система создает и другие типы файлов. Их имена совпадают с именем приложения, а расширения перечислены ниже:

- .gid — файл конфигурации, как правило, скрытый
- .fts — файл поиска по всему тексту, который создается при поиске раздела в текстах справки
- .ftg — файл группового списка поиска по всему тексту, который также создается при поиске раздела в текстах справки

Идентификаторы разделов используются для связи текста справки со справочной системой.

Кроме этого, мастер AppWizard при создании многооконного приложения (без поддержки баз данных и технологии OLE) при установке флажка **Context-sensitive Help** внесет в остальные файлы проекта следующие изменения:

- В карту сообщений класса `CMainFrame` будут добавлены макросы обработки команд `ID_HELP_FINDER`, `ID_HELP`, `ID_CONTEXT_HELP` и `ID_DEFAULT_HELP`. Для обработки сообщений в этих макросах будут использованы функции-члены класса `CMDIFrameWnd`
- В панель инструментов будет добавлена кнопка **Help**

- В обе версии меню, создаваемые мастером AppWizard (в меню, появляющееся при отсутствии в приложении открытых окон документа, и в меню, открываемое при наличии в приложении открытого окна документа), добавляется команда **Help Topics**
- Добавляются команды обработки нажатия клавиши <F1> (ID_HELP) и на комбинацию клавиш <Shift>+<F1> (ID_CONTEXT_HELP)
- Помещаемое по умолчанию в строку состояния сообщение **Ready** (Готово) заменяется на сообщение **For Help, press F1** (Для получения справки нажмите клавишу <F1>)
- В таблицу строковых ресурсов, расположенную в файле с именем приложения и расширением .rc, добавляется строковый ресурс AFX_IDS_HELPMODEMESSAGE, содержащий текст "Select an object on which to get Help" (Укажите объект, по которому хотите получить справку). Этот текст выводится в строку состояния при посылке команды ID_CONTEXT_HELP
- В таблицу строковых ресурсов будут добавлены сообщения, выводимые в строку состояния при выделении команд меню **Help**
- В проект включается файл afxcore.rtf, содержащий текст справочной информации по стандартным командам меню
- Если на четвертом шаге мастера AppWizard пользователь не сбросил флажок **Printing and print preview** (Печать и предварительный просмотр печати), в проект включается файл afxprintf.rtf, содержащий текст справочной информации по процессу печати и предварительного просмотра документа
- В проект будут включены двадцать два файла графических изображений в формате .bmp, используемые в качестве иллюстраций в различных разделах справки

Обработка сообщений справочной системы

Когда пользователь выбирает команду в меню **Help** (**?**) или нажимает кнопку **Help** в диалоговом окне, система, как и всегда в подобном случае, получает сообщение WM_COMMAND. Это сообщение обрабатывается соответствующей функцией обработки сообщения, вызывающей систему WinHelp. Таким образом вызывается справка по ключевому слову или по методике решения задачи.

При щелчке правой кнопкой мыши в окне ему посылается сообщение WM_CONTEXTMENU. После этого на экране должно появиться контекстное меню, в котором пользователь должен указать действие, которое он желает произвести. При работе со справочной системой это меню, как правило, содержит только одну команду **Что это такое?**, при выборе которой выводится контекстная справка по указанному элементу управления.

При нажатии клавиши <F1> активному окну или команде меню посылается сообщение WM_HELP, после чего приложение выводит справочную информацию об активном окне, элементе управления или команде меню.

Если требуется получить справку по элементу управления диалогового окна, то класс элемента управления пересылает сообщение классу родительского диалогового окна. Класс диалогового окна должен содержать функции обработки сообщений WM_HELP и WM_CONTEXTMENU.

В аргументе lParam сообщения WM_HELP передается указатель на объект структуры HELPINFO. В переменной hItemHandle данного объекта хранится дескриптор элемента управления, на котором был произведен щелчок правой кнопкой мыши. Этот дескриптор необходимо передать функции WinHelp вместе с командой HELP_WM_HELP, именем файла справки и указателем на массив идентификаторов.

Функция WinHelp осуществляет поиск в массиве переданного ей идентификатора элемента управления и по нему определяет контекстный идентификатор раздела в файле справки. Затем функция передает полученный контекстный идентификатор справочной системе Windows, которая отыскивает связанную с ним тему и выводит ее текст в всплывающее окно. Если элемент управления, на котором был выполнен щелчок левой кнопкой мыши, имеет идентификатор −1, то система ищет следующий элемент управления (тот, к которому перейдет фокус ввода при нажатии клавиши табуляции, если в текущий момент он принадлежит данному элементу управления) и использует его идентификатор для поиска контекстного идентификатора. Поэтому статический текст следует располагать в файле ресурсов перед идентификатором элемента управления, к которому он относится. То есть, создавать в редакторе ресурсов статический текст до создания соответствующего элемента управления.

Нажатие клавиши <F1> при наличии в приложении активного меню или диалогового окна обычно приводит к посылке команды с идентификатором ID_HELP. Связь между данной клавишей и командой устанавливается в таблице акселераторов. Команда ID_HELP может также посылаться кнопкой диалогового окна, имеющей данный идентификатор.

Команда ID_HELP обрабатывается как обычная команда, пока она не вызовет свою функцию обработки. Если приложение имеет справочную систему, команда ID_HELP обрабатывается функцией CWinApp::OnHelp. Поскольку предусмотренная по умолчанию обработка команд не позволяет определить необходимый для получения справочной информации контекст, данная команда всегда направляется в объект приложения, в котором вызывается обычная процедура поиска запрошенной справочной информации.

Функция CWinApp::OnHelp выполняет следующие действия:

- Проверяет наличие активного вызова функции AfxMessageBox, в котором указан идентификатор справки. Если в данный момент имеется активное

окно сообщения, вызывается система WinHelp с контекстом, связанным с данным окном сообщения

- Если в приложении отсутствует активное окно сообщения, функция CWinApp::OnHelp посылает сообщение WM_COMMANDHELP активному окну. Сообщение WM_COMMANDHELP является внутренним сообщением библиотеки MFC, принимаемым активным окном при запросе справочной информации. Получив это сообщение, приложение может вызвать функцию CWinApp::WinHelp с контекстом, соответствующим внутреннему состоянию окна. Если это окно не вызовет систему WinHelp, то же самое сообщение посылается родительскому окну активного окна и эта процедура продолжается до тех пор, пока данное сообщение не будет обработано или не будет достигнуто окно верхнего уровня в иерархии
- Если сообщение остается необработанным, вызывается справочная информация, предусмотренная по умолчанию. Для этого главному окну программы посылается сообщение ID_DEFAULT_HELP. Эта команда обычно обрабатывается функцией CWinApp::OnHelpIndex

Использование команды HELP_CONTEXTMENU приводит к тому, что справочная система Windows выводит контекстное меню перед выводом справочной информации. Это меню определено в системе, и с его помощью пользователь может выводить справочную информацию об отдельном элементе управления или вызывать диалоговое окно **Справочная система**.

Действия функции WinHelp при обработке сообщений WM_CONTEXTMENU и WM_HELP аналогичны за двумя исключениями:

- В сообщении WM_CONTEXTMENU используется аргумент wParam, в котором передается дескриптор элемента управления, посылающего сообщение
- В качестве аргумента функции используется команда HELP_CONTEXTMENU, а не HELP_WM_HELP

Ниже приведен пример создания контекстной справки в диалоговом окне:

```
LRESULT CALLBACK EditDlgProc(HWND hwndDlg, UINT uMsg, WPARAM wParam,
                                                      LPARAM lParam)
{

// Создание массива идентификаторов элементов управления
// и контекстных идентификаторов.

static DWORD aIds[] = {
    ID_SAVE,    IDH_SAVE,
    ID_DELETE,  IDH_DELETE,
    ID_COPY,    IDH_COPY,
    ID_PASTE,   IDH_PASTE,
    0,0
};
```

```
switch (uMsg) {
    case WM_HELP:
        WinHelp(((LPHELPINFO) lParam)->hItemHandle, "helpfile.hlp",
            HELP_WM_HELP, (DWORD) (LPSTR) aIds);
        break;

    case WM_CONTEXTMENU:
        WinHelp((HWND) wParam, "helpfile.hlp", HELP_CONTEXTMENU,
            (DWORD) (LPVOID) aIds);
        break;
    .
    . // Обработка других сообщений.
    .
    }

return FALSE;

}
```

Приложение может задать размер, позицию и режим отображения вторичного окна справки при использовании в функции `WinHelp` команды `HELP_SETWINPOS` и внесении соответствующих изменений в значения переменных передаваемого при этом объекта структуры `HELPWININFO`. Переменные данной структуры позволяют задать имя окна, его размер, положение и режим отображения.

Ниже приведен пример изменения параметров вторичного окна с именем "wnd_menu". Это имя должно быть определено в разделе `[WINDOWS]` файла проекта справочной системы:

```
BOOL DoWindowSize(VOID)
{
    HANDLE hhwi;
    LPHELPWININFO lphwi;
    WORD wSize;
    char *szWndName = "wnd_menu";

    wSize = sizeof(HELPWININFO) + lstrlen(szWndName);
    hhwi = GlobalAlloc(GHND, wSize);
    lphwi = (LPHELPWININFO) GlobalLock(hhwi);

    lphwi->wStructSize = wSize;
    lphwi->x     = 256;       // горизонтальная позиция
    lphwi->y     = 256;       // вертикальная позиция
    lphwi->dx    = 767;       // ширина
    lphwi->dy    = 512;       // высота
    lphwi->wMax = SW_SHOW;  // режим отображения
    lstrcpy(lphwi->rgchMember, szWndName);  // имя окна
```

```
    WinHelp(hwnd, "myhelp.hlp", HELP_SETWINPOS, (DWORD) lphwi);
    GlobalUnlock(hhwi);
    GlobalFree(hhwi);

return;
}
```

При вызове контекстной справки нажатием клавиш <SHIFT>+<F1>, кнопок панели инструментов или соответствующими командами меню вызывается команда `ID_CONTEXT_HELP`. При наличии в приложении активного меню или модального диалогового окна для обработки данной команды не используется стандартный фильтр сообщений, поэтому данную команду можно использовать только во время работы главного цикла обработки сообщений приложения (`CWinApp::Run`).

После вызова подобной справки указатель мыши приобретает форму курсора справки, отображаемого над всеми областями приложения, даже если при отображении в данной области указатель мыши обычно имеет особую форму (например, над толстой рамкой окна). Пользователь может выбрать команду при помощи мыши или клавиатуры. При этом команда не выполняется, а выводится справка по данному элементу управления.

Кроме того, пользователь может щелкнуть по любому видимому объекту на экране, например, нажать кнопку панели инструментов и получить справку по данному объекту. В этом режиме используется функция `CWinApp::OnContextHelp`. При этом все клавиши на клавиатуре, не связанные с клавишами, обеспечивающими доступ к меню, блокируются. Передача команд производится через функцию `PreTranslateMessage`, что позволяет использовать предопределенные комбинации клавиш и получать справку по данной команде.

Если в данном режиме работы предусмотрена особая реакция или особая трактовка команд в функции `PreTranslateMessage`, то перед выполнением этих операций следует проверить значение переменной `m_bHelpMode` объекта класса `CWinApp`. При использовании функции `PreTranslateMessage` в объектах класса `CDialog` она, например, вызывает функцию `IsDialogMessage`, что препятствует использованию клавиш перемещения по элементам управления диалогового окна в данном режиме получения контекстной справки. Кроме того, в цикле обработки сообщения вызывается функция `CWinApp::OnIdle`.

При выборе пользователем команды меню выводится справка по данной команде (при помощи описанной ниже команды `WM_COMMANDHELP`). Если пользователь щелкнет левой кнопкой мыши в области окна приложения, то функция `OnContextHelp` посылает сообщение `WM_HELPHITTEST` окну, в котором был произведен щелчок. Если данное окно возвращает ненулевое значение, то это значение используется в качестве контекста для справки. Если возвращается нулевое значение, то функция `OnContextHelp` посылает это сообщение родительскому окну (и так далее, пока не будет возвращен контекст

справки или не будет достигнуто верхнее окно в иерархии). Если контекст справки не может быть определен, главному окну приложения посылается сообщение `ID_DEFAULT_HELP`, обычно обрабатываемое функцией `CWinApp::OnHelpIndex`.

Сообщение `WM_HELPHITTEST` представляет собой внутреннее сообщение библиотеки MFC, посылаемое активному окну, в котором был выполнен щелчок мыши в режиме <Shift>+<F1>. В ответ на данное сообщение окно возвращает значение типа `DWORD`, содержащее идентификатор справки, используемый в `WinHelp`.

Русификация файла ресурсов

При создании Visual C++ был определен достаточно узкий список языков, которые предполагалось использовать при создании приложений с использованием данной системы программирования. Этот список расположен в первой вкладке мастера AppWizard и включает в себя немецкий, испанский, французский, итальянский и английский языки.

Примечание

Кроме того, в дистрибутив Visual C++ включены ресурсы, необходимые для создания приложений на японском, корейском и китайском языках. Однако, по непонятной причине, соответствующие файлы ресурсов игнорируются при компиляции приложения и в результате в приложении используются три языка: английский, выбранный язык приложения и язык версии Windows, в которой запускается данное приложение.

Для профессиональных программистов, имеющих опыт работы с продуктами фирмы Microsoft, подобная ситуация привычна и они ее не замечают, тем более что в качестве языка приложения они обычно выбирают английский, и, таким образом, приложение использует только два языка. Однако у неподготовленных пользователей подобная ситуация может вызвать нарекания. Поэтому желательно использовать в приложении только русский язык.

Если предположить, что создаваемое приложение будет работать под управлением русской версии Windows, то задачей программиста является исключение из него английских текстов. Часть этих текстов можно найти в файле ресурсов приложения, другие же содержатся в системных файлах ресурсов, которые должны включаться, но не включаются в файл ресурсов приложения (то есть оператор включения данных файлов присутствует, а результаты включения отсутствуют).

Изменения в файл ресурсов приложения могут быть внесены по отдельности для каждого из ресурсов с использованием редактора соответствующего ресурса. Однако все эти изменения проще внести непосредственно в файл ресурсов.

В файл ресурсов многооконного приложения, позволяющего печатать документы и не работающего с базами данных и компонентами OLE, должны

быть включены файлы afxres.rc и afxprint.rc, содержащие строковые ресурсы и описание диалоговых окон на выбранном языке приложения. Первый из них содержит заголовки и тексты для стандартных диалоговых окон открытия и закрытия файлов, всевозможные сообщения об ошибках, описание диалогового окна создания нового документа и описание некоторых ресурсов, не связанных с локализацией (курсоров и битовых образов).

Как выяснилось, независимо от используемого системного файла ресурсов, приложение использует для вывода сообщений об ошибках ресурсы операционной системы, и, в случае использования русской версии Windows, нет необходимости вносить в них какие-либо изменения.

Второй файл (afxprint.rc) содержит описание окон печати и предварительного просмотра печати, строковые ресурсы для данных окон и таблицу акселераторов.

Чтобы приложение использовало ресурс, содержащийся в системном файле ресурсов, этот ресурс должен быть включен непосредственно в файл ресурса приложения. Кроме того, в сам файл ресурса приложения необходимо внести некоторые изменения, связанные с его локализацией. Ниже будет приведен локализованный файл ресурса приложения HelpApp (листинг 13.1), который можно с минимальными изменениями использовать для локализации любого многооконного приложения, позволяющего распечатывать документы и не работающего с базами данных и объектами OLE.

Чтобы открыть этот файл для редактирования в текстовом режиме, выберите команду **File**, **Open...** или нажмите соответствующую кнопку панели инструментов. В раскрывающемся списке **Open as:** появившегося диалогового окна **Open** выберите опцию **Text**, как показано на рис. 13.13, и дважды щелкните левой кнопкой мыши на значке соответствующего файла ресурсов.

Текст открывшегося файла ресурсов должен быть изменен в соответствии с листингом 13.1.

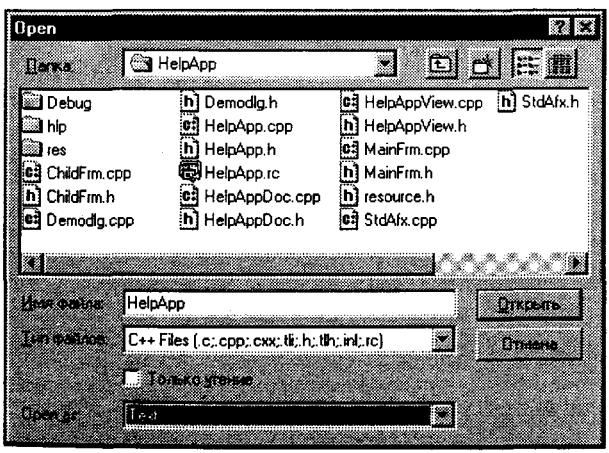


Рис. 13.13. Диалоговое окно **Open**

Листинг 13.1. Русифицированный файл ресурсов

```
//Файл ресурсов, созданный Microsoft Developer Studio.
//
#include "resource.h"

#define APSTUDIO_READONLY_SYMBOLS
////////////////////////////////////////////////////////////////////////////
//
// Создан из ресурса TEXTINCLUDE 2.
//
#include "afxres.h"

////////////////////////////////////////////////////////////////////////////
#undef APSTUDIO_READONLY_SYMBOLS

////////////////////////////////////////////////////////////////////////////
// Ресурсы на русском языке

#if !defined(AFX_RESOURCE_DLL) || defined(AFX_TARG_RUS)
#ifdef _WIN32
LANGUAGE LANG_RUSSIAN, SUBLANG_DEFAULT
#pragma code_page(1251)
#endif //_WIN32

#ifdef APSTUDIO_INVOKED
////////////////////////////////////////////////////////////////////////////
//
// TEXTINCLUDE
//

1 TEXTINCLUDE DISCARDABLE
BEGIN
    "resource.h\0"
END

2 TEXTINCLUDE DISCARDABLE
BEGIN
    "#include ""afxres.h""\r\n"
    "\0"
END

3 TEXTINCLUDE DISCARDABLE
BEGIN
    "#define _AFX_NO_SPLITTER_RESOURCES\r\n"
    "#define _AFX_NO_OLE_RESOURCES\r\n"
    "#define _AFX_NO_TRACKER_RESOURCES\r\n"
    "#define _AFX_NO_PROPERTY_RESOURCES\r\n"
```

```
    "\r\n"
    "#if !defined(AFX_RESOURCE_DLL) || defined(AFX_TARG_ENU)\r\n"
    "#ifdef _WIN32\r\n"
    "LANGUAGE 9, 1\r\n"
    "#pragma code_page(1252)\r\n"
    "#endif //_WIN32\r\n"
    "#include ""res\\HelpApp.rc2""  // non-Microsoft Visual C++ edited
                                   // resources\r\n"
    "#endif\r\n"
    "\0"
END

#endif    // APSTUDIO_INVOKED

////////////////////////////////////////////////////////////////////////
//
// DESIGNINFO
//

#ifdef APSTUDIO_INVOKED
GUIDELINES DESIGNINFO DISCARDABLE
BEGIN
    IDD_DEMO, DIALOG
    BEGIN
        LEFTMARGIN, 7
        RIGHTMARGIN, 179
        TOPMARGIN, 7
        BOTTOMMARGIN, 88
    END
END
#endif    // APSTUDIO_INVOKED

////////////////////////////////////////////////////////////////////////
//
// Icon
//

// Значок, имеющий наименьшее значение идентификатора, располагается первым,
// чтобы обеспечить использование значка приложения во всех системах
IDR_MAINFRAME           ICON    DISCARDABLE     "res\\HelpApp.ico"
IDR_HELPAPTYPE          ICON    DISCARDABLE     "res\\HelpAppDoc.ico"

////////////////////////////////////////////////////////////////////////
//
// Bitmap
//

IDR_MAINFRAME           BITMAP  MOVEABLE PURE   "res\\Toolbar.bmp"
```

```
////////////////////////////////////////////////////////////////////////
// CMiniFrameWnd Bitmap

AFX_IDB_MINIFRAME_MENU      BITMAP  DISCARDABLE     "res\\minifwnd.bmp"

////////////////////////////////////////////////////////////////////////
// CCheckListBox Bitmaps

AFX_IDB_CHECKLISTBOX_NT     BITMAP  DISCARDABLE     "res\\ntcheck.bmp"
AFX_IDB_CHECKLISTBOX_95     BITMAP  DISCARDABLE     "res\\95check.bmp"

////////////////////////////////////////////////////////////////////////
//
// Toolbar
//

IDR_MAINFRAME TOOLBAR DISCARDABLE  16, 15
BEGIN
    BUTTON      ID_FILE_NEW
    BUTTON      ID_FILE_OPEN
    BUTTON      ID_FILE_SAVE
    SEPARATOR
    BUTTON      ID_EDIT_CUT
    BUTTON      ID_EDIT_COPY
    BUTTON      ID_EDIT_PASTE
    SEPARATOR
    BUTTON      ID_FILE_PRINT
    SEPARATOR
    BUTTON      ID_APP_ABOUT
    BUTTON      ID_CONTEXT_HELP
END

////////////////////////////////////////////////////////////////////////
//
// Menu
//

IDR_MAINFRAME MENU PRELOAD DISCARDABLE
BEGIN
    POPUP "&Файл"
    BEGIN
        MENUITEM "Созд&ать...\tCtrl+N",          ID_FILE_NEW
        MENUITEM "&Открыть...\tCtrl+O",          ID_FILE_OPEN
        MENUITEM SEPARATOR
        MENUITEM "&Настройка принтера...",       ID_FILE_PRINT_SETUP
        MENUITEM SEPARATOR
        MENUITEM "&Текущий документ",            ID_FILE_MRU_FILE1, GRAYED
```

```
        MENUITEM SEPARATOR
        MENUITEM "В&ыход",                      ID_APP_EXIT
    END
    POPUP "&Вид"
    BEGIN
        MENUITEM "Панель &инструментов",        ID_VIEW_TOOLBAR
        MENUITEM "Панель &состояния",           ID_VIEW_STATUS_BAR
    END
    POPUP "&?"
    BEGIN
        MENUITEM "&Вызов справки",              ID_HELP_FINDER
        MENUITEM SEPARATOR
        MENUITEM "&О программе",                ID_APP_ABOUT
    END
END

IDR_HELPAPTYPE MENU PRELOAD DISCARDABLE
BEGIN
    POPUP "&Файл"
    BEGIN
        MENUITEM "Созд&ать...\tCtrl+N",         ID_FILE_NEW
        MENUITEM "&Открыть...\tCtrl+O",         ID_FILE_OPEN
        MENUITEM "&Закрыть",                    ID_FILE_CLOSE
        MENUITEM SEPARATOR
        MENUITEM "&Сохранить\tCtrl+S",          ID_FILE_SAVE
        MENUITEM "Сохранить &как...",           ID_FILE_SAVE_AS
        MENUITEM SEPARATOR
        MENUITEM "&Печать...\tCtrl+P",          ID_FILE_PRINT
        MENUITEM "Пред&варительный просмотр",   ID_FILE_PRINT_PREVIEW
        MENUITEM "&Настройка принтера...",      ID_FILE_PRINT_SETUP
        MENUITEM SEPARATOR
        MENUITEM "&Текущий документ",           ID_FILE_MRU_FILE1, GRAYED
        MENUITEM SEPARATOR
        MENUITEM "В&ыход",                      ID_APP_EXIT
    END
    POPUP "&Правка"
    BEGIN
        MENUITEM "&Отменить\tCtrl+Z",           ID_EDIT_UNDO
        MENUITEM SEPARATOR
        MENUITEM "&Вырезать\tCtrl+X",           ID_EDIT_CUT
        MENUITEM "&Копировать\tCtrl+C",         ID_EDIT_COPY
        MENUITEM "Вст&авить\tCtrl+V",           ID_EDIT_PASTE
    END
    POPUP "&Вид"
    BEGIN
        MENUITEM "Панель &инструментов",        ID_VIEW_TOOLBAR
```

```
        MENUITEM "Панель &состояния",           ID_VIEW_STATUS_BAR
    END
    POPUP "&Окно"
    BEGIN
        MENUITEM "&Новое",                      ID_WINDOW_NEW
        MENUITEM "&Каскадировать",              ID_WINDOW_CASCADE
        MENUITEM "&Разделить",                  ID_WINDOW_TILE_HORZ
        MENUITEM "&Упорядочить",                ID_WINDOW_ARRANGE
    END
    POPUP "&?"
    BEGIN
        MENUITEM "&Вызов справки",              ID_HELP_FINDER
        MENUITEM SEPARATOR
        MENUITEM "&О программе",                ID_APP_ABOUT
    END
END

////////////////////////////////////////////////////////////////////////////
//
// Accelerator
//

IDR_MAINFRAME ACCELERATORS PRELOAD MOVEABLE PURE
BEGIN
    "N",            ID_FILE_NEW,            VIRTKEY, CONTROL
    "O",            ID_FILE_OPEN,           VIRTKEY, CONTROL
    "S",            ID_FILE_SAVE,           VIRTKEY, CONTROL
    "P",            ID_FILE_PRINT,          VIRTKEY, CONTROL
    "Z",            ID_EDIT_UNDO,           VIRTKEY, CONTROL
    "X",            ID_EDIT_CUT,            VIRTKEY, CONTROL
    "C",            ID_EDIT_COPY,           VIRTKEY, CONTROL
    "V",            ID_EDIT_PASTE,          VIRTKEY, CONTROL
    VK_BACK,        ID_EDIT_UNDO,           VIRTKEY, ALT
    VK_DELETE,      ID_EDIT_CUT,            VIRTKEY, SHIFT
    VK_INSERT,      ID_EDIT_COPY,           VIRTKEY, CONTROL
    VK_INSERT,      ID_EDIT_PASTE,          VIRTKEY, SHIFT
    VK_F6,          ID_NEXT_PANE,           VIRTKEY
    VK_F6,          ID_PREV_PANE,           VIRTKEY, SHIFT
    VK_F1,          ID_CONTEXT_HELP,        VIRTKEY, SHIFT
    VK_F1,          ID_HELP,                VIRTKEY
END

// Таблица акселераторов предварительного просмотра печати
AFX_IDR_PREVIEW_ACCEL ACCELERATORS LOADONCALL MOVEABLE
BEGIN
  VK_NEXT,    AFX_ID_PREVIEW_NEXT,    VIRTKEY, NOINVERT
  VK_PRIOR,   AFX_ID_PREVIEW_PREV,    VIRTKEY, NOINVERT
```

```
  VK_ESCAPE,  AFX_ID_PREVIEW_CLOSE,   VIRTKEY, NOINVERT
END

/////////////////////////////////////////////////////////////////////////////
// Cursors

AFX_IDC_CONTEXTHELP     CURSOR  DISCARDABLE     "res\\help.cur"

#ifndef _AFX_NO_SPLITTER_RESOURCES
AFX_IDC_SMALLARROWS     CURSOR  DISCARDABLE     "res\\sarrows.cur"
AFX_IDC_HSPLITBAR       CURSOR  DISCARDABLE     "res\\splith.cur"
AFX_IDC_VSPLITBAR       CURSOR  DISCARDABLE     "res\\splitv.cur"
AFX_IDC_NODROPCRSR      CURSOR  DISCARDABLE     "res\\nodrop.cur"
#endif //!_AFX_NO_SPLITTER_RESOURCES

#ifndef _AFX_NO_TRACKER_RESOURCES
AFX_IDC_TRACKNWSE       CURSOR  DISCARDABLE     "res\\trcknwse.cur"
AFX_IDC_TRACKNESW       CURSOR  DISCARDABLE     "res\\trcknesw.cur"
AFX_IDC_TRACKNS         CURSOR  DISCARDABLE     "res\\trckns.cur"
AFX_IDC_TRACKWE         CURSOR  DISCARDABLE     "res\\trckwe.cur"
AFX_IDC_TRACK4WAY       CURSOR  DISCARDABLE     "res\\trck4way.cur"
AFX_IDC_MOVE4WAY        CURSOR  DISCARDABLE     "res\\move4way.cur"
#endif //!_AFX_NO_TRACKER_RESOURCES

#ifndef _AFX_NO_CTL3D_RESOURCES
26567 BITMAP "res\\3dcheck.bmp" // Примечание: идентификатор 26567
                                // определен в файле CTL3D.H
#endif

/////////////////////////////////////////////////////////////////////////////
//
// Dialog
//

IDD_ABOUTBOX DIALOG DISCARDABLE  0, 0, 235, 55
STYLE DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
CAPTION "О программе"
FONT 8, "MS Sans Serif"
BEGIN
    ICON            IDR_MAINFRAME,IDC_STATIC,11,17,20,20
    LTEXT           "HelpApp версия
1.0",IDC_STATIC,40,10,119,8,SS_NOPREFIX
    LTEXT           "Copyright (C) 1999",IDC_STATIC,40,25,119,8
    DEFPUSHBUTTON   "OK",IDOK,178,7,50,14,WS_GROUP
END

/////////////////////////////////////////////////////////////////////////////
// Standard Dialogs

#ifndef _AFX_NO_NEWTYPEDLG_RESOURCES
AFX_IDD_NEWTYPEDLG DIALOG DISCARDABLE  9, 26, 183, 70
```

```
STYLE DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU | 0x04
CAPTION "Создание документа"
FONT 8, "MS Shell Dlg"
BEGIN
  LTEXT          "&Тип создаваемого документа",IDC_STATIC,6,5,123,8,
                                                          NOT WS_GROUP
  LISTBOX         AFX_IDC_LISTBOX,6,15,125,49,WS_VSCROLL | WS_TABSTOP
  DEFPUSHBUTTON   "OK",IDOK,137,6,40,14
  PUSHBUTTON      "&Отмена",IDCANCEL,137,23,40,14
  PUSHBUTTON      "&Справка ...",ID_HELP,137,43,40,14
END
#endif // !_AFX_NO_NEWTYPEDLG_RESOURCES

AFX_IDD_PRINTDLG DIALOG DISCARDABLE  6, 18, 133, 95
STYLE DS_MODALFRAME | DS_3DLOOK | WS_POPUP | WS_CAPTION
FONT 8, "MS Sans Serif"
BEGIN
    CTEXT           "Печать",IDC_STATIC,0,9,133,8
    CTEXT           "",AFX_IDC_PRINT_DOCNAME,0,19,133,8
    CTEXT           "на",IDC_STATIC,0,29,133,8
    CTEXT           "",AFX_IDC_PRINT_PRINTERNAME,0,39,133,8
    CTEXT           "",AFX_IDC_PRINT_PORTNAME,0,49,133,8
    CTEXT           "",AFX_IDC_PRINT_PAGENUM,0,59,133,8
    PUSHBUTTON      "Отмена",IDCANCEL,46,74,40,14
END

// Диалоговое окно панели инструментов предварительного просмотра печати
AFX_IDD_PREVIEW_TOOLBAR DIALOG PRELOAD DISCARDABLE  0, 0, 330, 16
STYLE WS_CHILD | 0x04
FONT 8, "MS Shell Dlg"
BEGIN
    PUSHBUTTON      "&Печать...",AFX_ID_PREVIEW_PRINT,2,2,44,12
    PUSHBUTTON      "С&ледующая",AFX_ID_PREVIEW_NEXT,47,2,44,12
    PUSHBUTTON      "Пр&едыдущая",AFX_ID_PREVIEW_PREV,92,2,50,12
    PUSHBUTTON      "",AFX_ID_PREVIEW_NUMPAGE,143,2,44,12
    PUSHBUTTON      "&Крупнее",AFX_ID_PREVIEW_ZOOMIN,188,2,44,12
    PUSHBUTTON      "&Мельче",AFX_ID_PREVIEW_ZOOMOUT,233,2,44,12
    PUSHBUTTON      "&Закрыть",AFX_ID_PREVIEW_CLOSE,278,2,44,12
END

#ifndef _MAC

////////////////////////////////////////////////////////////////////////
//
// Version
//

VS_VERSION_INFO VERSIONINFO
 FILEVERSION 1,0,0,1
```

```
 PRODUCTVERSION 1,0,0,1
 FILEFLAGSMASK 0x3fL
#ifdef _DEBUG
 FILEFLAGS 0x1L
#else
 FILEFLAGS 0x0L
#endif
 FILEOS 0x4L
 FILETYPE 0x1L
 FILESUBTYPE 0x0L
BEGIN
    BLOCK "StringFileInfo"
    BEGIN
        BLOCK "040904B0"
        BEGIN
            VALUE "CompanyName", "\0"
            VALUE "FileDescription", "HelpApp MFC Application\0"
            VALUE "FileVersion", "1, 0, 0, 1\0"
            VALUE "InternalName", "HelpApp\0"
            VALUE "LegalCopyright", "Copyright (C) 1999\0"
            VALUE "LegalTrademarks", "\0"
            VALUE "OriginalFilename", "HelpApp.EXE\0"
            VALUE "ProductName", "HelpApp Application\0"
            VALUE "ProductVersion", "1, 0, 0, 1\0"
        END
    END
    BLOCK "VarFileInfo"
    BEGIN
        VALUE "Translation", 0x409, 1200
    END
END

#endif    // !_MAC

/////////////////////////////////////////////////////////////////////////////
//
// DESIGNINFO
//

#ifdef APSTUDIO_INVOKED
GUIDELINES DESIGNINFO DISCARDABLE
BEGIN
    IDD_ABOUTBOX, DIALOG
    BEGIN
        LEFTMARGIN, 7
        RIGHTMARGIN, 228
        TOPMARGIN, 7
```

```
        BOTTOMMARGIN, 48
    END
END
#endif    // APSTUDIO_INVOKED

////////////////////////////////////////////////////////////////////////////
//
// String Table
//

STRINGTABLE DISCARDABLE
BEGIN
#ifndef _AFX_NO_APPMENU_RESOURCES
    AFX_IDS_OPENFILE        "Открытие документа"
    AFX_IDS_SAVEFILE        "Сохранение документа"
    AFX_IDS_ALLFILTER       "Все файлы (*.*)
    AFX_IDS_UNTITLED        "Новый"
       AFX_IDS_HIDE,        "&Скрыть"
#endif // !_AFX_NO_APPMENUSTRING_RESOURCES
END

// Строковые ресурсы, используемые при печати
STRINGTABLE DISCARDABLE
BEGIN
    // используются в диалоговых окнах
    AFX_IDS_PRINTONPORT       "на %1"
    AFX_IDS_ONEPAGE           "&Одна стр." // для кнопки окна
                                           // предварительного просмотра
                                           // печати
    AFX_IDS_TWOPAGE          "&Две стр."  // для кнопки окна
                                          // предварительного просмотра
                                          // печати
    AFX_IDS_PRINTPAGENUM     "Стр. %u"
    AFX_IDS_PREVIEWPAGEDESC  "Страница %u\nСтраницы %u-%u\n"

  // используются при печати в файл
    AFX_IDS_PRINTDEFAULTEXT  "prn"           // расширение, используемое
                                             // по умолчанию
    AFX_IDS_PRINTDEFAULT     "Output.prn"  // имя файла, используемое
                                           // по умолчанию
    AFX_IDS_PRINTFILTER      "Файлы печати (*.prn)|*.prn|Все файлы
                                                    (*.*)|*.*||"
    AFX_IDS_PRINTCAPTION      "Печать в файл"
    AFX_IDS_PRINTTOFILE       "файл %1"
END

STRINGTABLE PRELOAD DISCARDABLE
BEGIN
    IDR_MAINFRAME           "HelpApp"
```

```
    IDR_HELPAPTYPE          "\nHelpAp\nHelpAp\n\n\nHelpApp.Document
                             \nHelpAp Document"
END

STRINGTABLE PRELOAD DISCARDABLE
BEGIN
    AFX_IDS_APP_TITLE       "HelpApp"
    AFX_IDS_IDLEMESSAGE     "Для вывода справки нажмите клавишу F1"
    AFX_IDS_HELPMODEMESSAGE "Выберите объект, по которому хотели бы
                             получить информацию"
END

STRINGTABLE DISCARDABLE
BEGIN
    ID_INDICATOR_EXT        "EXT"
    ID_INDICATOR_CAPS       "CAP"
    ID_INDICATOR_NUM        "NUM"
    ID_INDICATOR_SCRL       "SCRL"
    ID_INDICATOR_OVR        "OVR"
    ID_INDICATOR_REC        "REC"
END

STRINGTABLE DISCARDABLE
BEGIN
    ID_FILE_NEW             "Создание нового окна\nСоздать"
    ID_FILE_OPEN            "Открытие существующего документа\nОткрыть"
    ID_FILE_CLOSE           "Закрытие активного окна\nЗакрыть"
    ID_FILE_SAVE            "Сохранение активного документа\nСохранить"
    ID_FILE_SAVE_AS         "Сохранение активного документа под другим
                             именем\nСохранить как"
    ID_FILE_PAGE_SETUP      "Изменение параметров страницы\nПараметры
                             страницы"
    ID_FILE_PRINT_SETUP     "Смена принтера и изменение его
                             параметров\nНастройка принтера"
    ID_FILE_PRINT           "Печать активного документа\nПечать"
    ID_FILE_PRINT_PREVIEW   "Просмотр печатаемого материала
                             в полностраничном режиме\nПредварительный
                             просмотр"
END

STRINGTABLE DISCARDABLE
BEGIN
    ID_APP_ABOUT            "Вывод сведений о программе, ее версии
                             и авторских правах\nО программе"
    ID_APP_EXIT             "Завершение работы с приложением; сохранение
                             документов\nВыход"
    ID_HELP_INDEX           "Вывод содержимого файловой системы\nВывод
                             справки"
    ID_HELP_FINDER          "Поиск по образцу\nВывод справки"
```

```
    ID_HELP_USING           "Выводит руководство по пользованию
                             справочной системой\nСправка"
    ID_CONTEXT_HELP         "Выводит информацию о выделенных кнопках,
                             пунктах меню и окнах\nСправка"
    ID_HELP                 "Выводит справку по текущей задаче или
                             команде\nСправка"
END

STRINGTABLE DISCARDABLE
BEGIN
    ID_FILE_MRU_FILE1       "Открыть этот документ"
    ID_FILE_MRU_FILE2       "Открыть этот документ"
    ID_FILE_MRU_FILE3       "Открыть этот документ"
    ID_FILE_MRU_FILE4       "Открыть этот документ"
    ID_FILE_MRU_FILE5       "Открыть этот документ"
    ID_FILE_MRU_FILE6       "Открыть этот документ"
    ID_FILE_MRU_FILE7       "Открыть этот документ"
    ID_FILE_MRU_FILE8       "Открыть этот документ"
    ID_FILE_MRU_FILE9       "Открыть этот документ"
    ID_FILE_MRU_FILE10      "Открыть этот документ"
    ID_FILE_MRU_FILE11      "Открыть этот документ"
    ID_FILE_MRU_FILE12      "Открыть этот документ"
    ID_FILE_MRU_FILE13      "Открыть этот документ"
    ID_FILE_MRU_FILE14      "Открыть этот документ"
    ID_FILE_MRU_FILE15      "Открыть этот документ"
    ID_FILE_MRU_FILE16      "Открыть этот документ"
END

STRINGTABLE DISCARDABLE
BEGIN
    ID_NEXT_PANE            "Переход к следующей панели окна\nСледующая
                             панель"
    ID_PREV_PANE            "Переход к предыдущей панели окна\nПредыдущая
                             панель"
END

STRINGTABLE DISCARDABLE
BEGIN
    ID_WINDOW_NEW           "Открыть другое окно\nНовое окно"
    ID_WINDOW_ARRANGE       "Упорядочить значки в нижней части
                             окна\nУпорядочить значки"
    ID_WINDOW_CASCADE       "Упорядочить окна таким образом, чтобы они
                             перекрывались\nКаскадировать окна"
    ID_WINDOW_TILE_HORZ     "Упорядочить окна по горизонтали
                             без перекрытия\nУпорядочить по горизонтали"
    ID_WINDOW_TILE_VERT     "Упорядочить окна по горизонтали
                             без перекрытия\nУпорядочить по горизонтали"
    ID_WINDOW_SPLIT         "Разбить активное окно на панели\nРазбить"
END
```

```
STRINGTABLE DISCARDABLE
BEGIN
    ID_EDIT_CLEAR          "Удаление выделенного фрагмента\nУдалить"
    ID_EDIT_CLEAR_ALL      "Удалить все\nУдалить все"
    ID_EDIT_COPY           "Копирование выделенного фрагмента в буфер
                            обмена\nКопировать в буфер"
    ID_EDIT_CUT            "Удаление выделенного фрагмента в буфер
                            обмена\nУдалить в буфер"
    ID_EDIT_FIND           "Поиск заданного текста\nНайти"
    ID_EDIT_PASTE          "Вставка содержимого буфера обмена\nВставить"
    ID_EDIT_REPEAT         "Повторить последнее действие\nПовторить"
    ID_EDIT_REPLACE        "Замена одного текста другим\nЗаменить"
    ID_EDIT_SELECT_ALL     "Выделение всего документа целиком\nВыделить все"
    ID_EDIT_UNDO           "Отмена последнего выполненного
                            действия\nОтменить"
    ID_EDIT_REDO           "Вернуть результат последнего отмененного
                            действия\nВернуть"
END

STRINGTABLE DISCARDABLE
BEGIN
    ID_VIEW_TOOLBAR        "Вывод и скрытие панели
                            инструментов\nПанель инструментов"
    ID_VIEW_STATUS_BAR     "Вывод и скрытие строки состояния\nСтрока
                            состояния"
END

STRINGTABLE DISCARDABLE
BEGIN
    AFX_IDS_SCSIZE         "Изменение размеров окна"
    AFX_IDS_SCMOVE         "Изменение положения окна"
    AFX_IDS_SCMINIMIZE     "Минимизация окна"
    AFX_IDS_SCMAXIMIZE     "Максимизация окна"
    AFX_IDS_SCNEXTWINDOW   "Переход к следующему окну"
    AFX_IDS_SCPREVWINDOW   "Переход к предыдущему окну"
    AFX_IDS_SCCLOSE        "Закрытие активного окна и сохранение документа"
END

STRINGTABLE DISCARDABLE
BEGIN
    AFX_IDS_SCRESTORE      "Восстановление размеров окна"
    AFX_IDS_SCTASKLIST     "Активизация списка задач"
    AFX_IDS_MDICHILD       "Активизация данного окна"
END

STRINGTABLE DISCARDABLE
BEGIN
    AFX_IDS_PREVIEW_CLOSE   "Выйти из режима предварительного просмотра
                             печати\nПрекратить просмотр"
END
```

```
#endif    // Ресурсы на русском языке
/////////////////////////////////////////////////////////////////////////////

#ifndef APSTUDIO_INVOKED
/////////////////////////////////////////////////////////////////////////////
//
// Создан из ресурса TEXTINCLUDE 3.
//
#define _AFX_NO_SPLITTER_RESOURCES
#define _AFX_NO_OLE_RESOURCES
#define _AFX_NO_TRACKER_RESOURCES
#define _AFX_NO_PROPERTY_RESOURCES

#if !defined(AFX_RESOURCE_DLL) || defined(AFX_TARG_ENU)
#ifdef _WIN32
LANGUAGE 9, 1
#pragma code_page(1252)
#endif //_WIN32
#include "res\\HelpApp.rc2"  // редактируемые ресурсы, не принадлежащие
                             // Microsoft Visual C++
#endif

/////////////////////////////////////////////////////////////////////////////
#endif    // not APSTUDIO_INVOKED
```

В исходный файл были внесены следующие изменения:

- Все комментарии, кроме удаляемых, были переведены на русский язык
- В разделе `3 TEXTINCLUDE DISCARDABLE` были уничтожены строки:

```
"#include ""afxres.rc""      // Standard components\r\n"
"#include ""afxprint.rc""    // printing/print preview resources\r\n"
```

- Перед разделом `Icon` были уничтожены строки:

```
#endif    // Russian resources
/////////////////////////////////////////////////////////////////////////////

/////////////////////////////////////////////////////////////////////////////
// English (U.S.) resources

#if !defined(AFX_RESOURCE_DLL) || defined(AFX_TARG_ENU)
#ifdef _WIN32
LANGUAGE LANG_ENGLISH, SUBLANG_ENGLISH_US
#pragma code_page(1252)
#endif //_WIN32
```

- В раздел `Bitmap` из файла afxres.rc было добавлено объявление битовых образов окна `CMiniFrameWnd`
- В разделе `Menu` были переведены на русский язык все команды меню
- В раздел `Accelerator` из файла afxprint.rc была перенесена таблица акселераторов предварительного просмотра печати
- После раздела `Accelerator` был включен раздел `Cursors`, составленный из соответствующего раздела файла afxres.rc и ресурса курсора из файла afxprint.rc
- В разделе `Dialog` было русифицировано окно **О программе** и добавлены русифицированное диалоговое окно **Создание документа** из файла afxres.rc и окна **Предварительный просмотр** и **Печать** из файла afxprint.rc
- В начало раздела `String Table` были помещены таблица строковых ресурсов из файла afxres.rc, содержащая строковые ресурсы для создания диалоговых окон **Создание документа** и **Сохранение документа**, а также таблицы строковых ресурсов из файла afxprint.rc со строковыми ресурсами для работы с окнами **Предварительный просмотр** и **Печать**
- Все строковые ресурсы в разделе `String Table` были переведены на русский язык

Если предполагается использование приложения в версии Windows, использующей язык, отличный от русского, для русификации сообщений об ошибках в таблицу строковых ресурсов следует поместить следующий текст:

```
STRINGTABLE DISCARDABLE
BEGIN
    AFX_IDS_MEMORY_EXCEPTION          "Недостаточно памяти."
    AFX_IDS_NOT_SUPPORTED_EXCEPTION   "Данная операция не поддерживается
                                       системой."
    AFX_IDS_RESOURCE_EXCEPTION        "Требуемый ресурс недоступен."
    AFX_IDS_USER_EXCEPTION            "Возникла неизвестная ошибка."

  // Основные сообщения об ошибках
#ifndef _AFX_NO_FILECMD_RESOURCES
    AFX_IDP_INVALID_FILENAME       "Недопустимое имя файла."
    AFX_IDP_FAILED_TO_OPEN_DOC     "Ошибка при открытии документа."
    AFX_IDP_FAILED_TO_SAVE_DOC     "Ошибка при закрытии документа."
    AFX_IDP_ASK_TO_SAVE            "Сохранить изменения в %1?"
    AFX_IDP_FAILED_TO_CREATE_DOC   "Ошибка при создании пустого
                                    документа."
    AFX_IDP_FILE_TOO_LARGE         "Невозможно открыть файл такого
                                    размера."
    AFX_IDP_FAILED_TO_START_PRINT "Невозможно инициировать процесс
                                    печати."
#endif // !_AFX_NO_FILECMD_RESOURCES
```

```
    AFX_IDP_FAILED_TO_LAUNCH_HELP  "Ошибка при обращении к справочной
                                    информации."
    AFX_IDP_INTERNAL_FAILURE       "Внутренняя ошибка приложения."
    AFX_IDP_COMMAND_FAILURE        "Команда не выполнена."
    AFX_IDP_FAILED_MEMORY_ALLOC    "Недостаточно памяти для данной
                                    операции."
  AFX_IDP_GET_NOT_SUPPORTED,       "Чтение невозможно. Файл открыт только
                                    для записи."
  AFX_IDP_SET_NOT_SUPPORTED,       "Запись невозможна. Файл открыт только
                                    для чтения."
    AFX_IDP_UNREG_DONE       "Записи удалены из системного реестра,
                              а файл INI (если он существовал) уничтожен."
    AFX_IDP_UNREG_FAILURE    "Не все записи удалены из системного
                              реестра (или файла INI file)."
    AFX_IDP_DLL_LOAD_FAILED "Для работы данной программы требуется
                             использование файла %s, который отсутствует
                             в данной системе."
    AFX_IDP_DLL_BAD_VERSION "Данная программа связана с отсутствующим
                             объектом %s в файле %s. На данном компьютере
                             установлена несовместимая версия %s."
  // CFile/CArchive error strings for user failure
#ifndef _AFX_NO_CFILE_RESOURCES
    AFX_IDP_FAILED_INVALID_FORMAT "Неизвестный формат файла."
    AFX_IDP_FAILED_INVALID_PATH   "%1\nДанный файл отсутствует.
                                   \nПроверьте правильность задания пути
                                   и имени файла."
    AFX_IDP_FAILED_DISK_FULL      "На указанном диске отсутствует
                                   необходимое свободное пространство."
    AFX_IDP_FAILED_ACCESS_READ    "Невозможно чтение файла %1, он открыт
                                   другим приложением."
    AFX_IDP_FAILED_ACCESS_WRITE   "Невозможна запись в файл %1, он
                                   открыт только для чтения или открыт
                                   другим приложением."
    AFX_IDP_FAILED_IO_ERROR_READ  "Неизвестная ошибка при чтении
                                   файла %1."
    AFX_IDP_FAILED_IO_ERROR_WRITE  "Неизвестная ошибка при записи
                                    файла %1."
#endif // !_AFX_NO_CFILE_RESOURCES

  // Сообщения об ошибках ввода
    AFX_IDP_PARSE_INT           "Введите целое число."
    AFX_IDP_PARSE_REAL          "Введите число."
    AFX_IDP_PARSE_INT_RANGE     "Введите целое число в диапазоне
                                 от %1 до %2."
    AFX_IDP_PARSE_REAL_RANGE    "Введите число в диапазоне от %1 до %2."
    AFX_IDP_PARSE_STRING_SIZE   "Введите не более  %1 символов."
    AFX_IDP_PARSE_RADIO_BUTTON "Выберите кнопку."
    AFX_IDP_PARSE_BYTE          "Введите целое число в диапазоне
                                 от 0 до 255."
    AFX_IDP_PARSE_UINT          "Введите допустимое целое число."
```

```
    AFX_IDP_PARSE_DATETIME      "Введите дату и/или время."
    AFX_IDP_PARSE_CURRENCY      "Введите величину в денежном формате."

#ifndef _AFX_NO_OLE_RESOURCES
  // Строки, используемые в серверах и контейнерах OLE
    AFX_IDS_PASTELINKEDTYPE  "Связано с %s"
    AFX_IDS_UNKNOWNTYPE      "Неизвестный тип"
  AFX_IDP_FAILED_TO_NOTIFY   "%1\nНевозможно зарегистрировать
                              документ.\nДокумент, возможно, уже открыт."
  AFX_IDS_NOT_DOCOBJECT      "Файл не может быть обработан сервером
                              документов."
#endif //!_AFX_NO_OLE_RESOURCES

    AFX_IDP_NO_ERROR_AVAILABLE    "Сообщения об ошибках отсутствуют."

#ifndef _AFX_NO_CFILE_RESOURCES
    AFX_IDP_FILE_NONE             "Ошибки отсутствуют."
    AFX_IDP_FILE_GENERIC          "Возникла неизвестная ошибка при доступе
                                   к %1."
    AFX_IDP_FILE_NOT_FOUND        "Файл %1 не найден."
    AFX_IDP_FILE_BAD_PATH         "Файл %1 имеет некорректный путь."
    AFX_IDP_FILE_TOO_MANY_OPEN    "Файл %1 не может быть открыт, поскольку
                                   в системе открыто слишком много файлов."
    AFX_IDP_FILE_ACCESS_DENIED    "Отказ в доступе к файлу %1."
    AFX_IDP_FILE_INVALID_FILE     "Файл %1 имеет недопустимый дескриптор."
    AFX_IDP_FILE_REMOVE_CURRENT   "Файл %1 не может быть удален, поскольку
                                   является текущим директорием."
    AFX_IDP_FILE_DIR_FULL         "Файл %1 не может быть создан, поскольку
                                   текущий директорий переполнен."
    AFX_IDP_FILE_BAD_SEEK         "На %1 файл отсутствует"
    AFX_IDP_FILE_HARD_IO          "При доступе к файлу %1 возникла
                                   аппаратная ошибка ввода/вывода."
    AFX_IDP_FILE_SHARING          "При доступе к файлу %1 возникла ошибка
                                   при разделении ресурсов."
    AFX_IDP_FILE_LOCKING          "При доступе к файлу %1 произошло
                                   нарушение защиты."
    AFX_IDP_FILE_DISKFULL         "При доступе к файлу %1 произошло
                                   переполнение диска."
    AFX_IDP_FILE_EOF              "Произошла попытка доступа к записи
                                   в файле %1, расположенной после его
                                   конца."
  AFX_IDS_UNNAMED_FILE            "Неименованный файл"
#endif // !_AFX_NO_CFILE_RESOURCES

    AFX_IDP_ARCH_NONE             "Ошибки отсутствуют."
    AFX_IDP_ARCH_GENERIC          "Возникла неизвестная ошибка при доступе
                                   к %1."
    AFX_IDP_ARCH_READONLY         "Произошла попытка записи в файл %1,
                                   открытый только для чтения."
```

```
    AFX_IDP_ARCH_ENDOFFILE   "Произошла попытка доступа к записи
                              в файле %1, расположенной после его конца."
    AFX_IDP_ARCH_WRITEONLY   "Произошла попытка чтения из файла %1,
                              открытого только для записи."
    AFX_IDP_ARCH_BADINDEX    "Файл %1 имеет недопустимый формат."
    AFX_IDP_ARCH_BADCLASS    "Файл %1 содержит недопустимый объект."
    AFX_IDP_ARCH_BADSCHEMA   "Файл %1 содержит неправильную схему."

#ifndef _AFX_NO_MAPI_RESOURCES
  // MAPI strings / error messages
    AFX_IDP_FAILED_MAPI_LOAD    "Невозможно связаться системой поддержки
                                 электронной почты."
    AFX_IDP_INVALID_MAPI_DLL    "Некорректная версия системной DLL
                                 электронной почты."
    AFX_IDP_FAILED_MAPI_SEND    "Приложение Send Mail не смогло отправить
                                 сообщение."
#endif //!_AFX_NO_MAPI_RESOURCES

#ifndef _AFX_NO_OCC_RESOURCES
  AFX_IDS_OCC_SCALEUNITS_PIXELS "Элементы изображения"
#endif //!_AFX_NO_OCC_RESOURCES
END
```

При переводе английских наименований команд меню и соответствующих им сообщений учитывалась сложившаяся в большинстве стандартных приложений практика, однако перевод сообщений об ошибках в большинстве случаев не поверялся, поэтому приведенные выше варианты перевода могут корректироваться.

Для полной русификации приложения необходимо перевести также строки, включаемые в поле **Тип файлов:** диалоговых окон **Открытие документа** и **Сохранение документа**. Эти строки располагаются в четвертом поле текстового ресурса, соответствующего идентификатору шаблона документа. Если приложение HelpApp будет обрабатывать текстовые документы, то соответствующей строковый ресурс будет иметь следующий вид:

```
IDR_HELPAPTYPE          "\nHelpAp\nHelpAp\nТекстовые файлы
(.txt)\n.txt\nHelpApp.Document\nHelpAp Document"
```

Однако даже полная русификация строковых ресурсов не гарантирует полной русификации приложения, поскольку разработчики библиотеки MFC, по крайней мере в ее отладочной версии, в некоторых случаях выводят диагностические сообщения на английском языке, не прибегая к использованию таблицы строковых ресурсов.

Создание системы командной справки

Мастер AppWizard произвел практически все необходимые действия для создания командной справки и, в большинстве случаев, этого достаточно.

Но если пользователю надо добавить новый пункт в меню **Help**, то все необходимые для этого действия он должен осуществить сам.

Чтобы продемонстрировать действия, которые необходимо выполнить в приложении при добавлении новой команды в меню **Help**, следует создать тестовое приложение, которое, строго говоря, не должно выполнять какие-либо полезные действия.

Текст демонстрационного приложения, имеющего справочную систему, содержится в каталоге \HelpApp на дискете, прилагаемой к данной книге. Чтобы самостоятельно создать это приложение:

1. По методике, описанной в главе 1, создайте многооконное приложение с именем HelpApp, установив на четвертом шаге мастера AppWizard флажок **Context-sensitive Help**.
2. Выберите команду **File**, **Open...** или нажмите соответствующую кнопку панели инструментов. Появится диалоговое окно **Open**.
3. В раскрывающемся списке **Open as:** выберите опцию **Text**, и дважды щелкните левой кнопкой мыши на значке файла ресурсов **HelpApp.rc**.
4. В открывшийся файл ресурсов внесите изменения в соответствии с листингом 13.1.
5. Раскройте вкладку **ResourceView** окна **Workspace**.
6. Раскройте папку \HelpApp Resources, в ней раскройте папку \Menu и дважды щелкните левой кнопкой мыши на значке **IDR_HELPAPTYPE**. В окне редактирования ресурса появится соответствующее меню.
7. Раскройте меню **?**. Поместите указатель мыши на пунктирный прямоугольник, расположенный в нижней части данного меню. Нажмите левую кнопку мыши и, не отпуская ее, переместите появившуюся на месте прямоугольника горизонтальную линию между командой меню **Вызов справки** и разделителем.
8. Отпустите левую кнопку мыши и щелкните в прямоугольнике правой кнопкой мыши.
9. В появившемся контекстном меню выберите команду **Properties**. Появится диалоговое окно **Menu Item Properties**.
10. Введите в текстовое поле раскрывающегося списка **ID:** идентификатор команды `ID_NEW_HELP`, в текстовое поле **Caption:** введите заголовок "&Новая справка", а в текстовое поле **Prompt:** — текст "Новая справка\nНовая".
11. Закройте диалоговое окно **Menu Item Properties** и вызовите мастер ClassWizard.
12. В окне списка **Object IDs:** выделите идентификатор команды `ID_NEW_HELP`, в окне списка **Messages:** выберите опцию **COMMAND** и нажмите кнопку

Add Function. В появившемся окне **Add Member Function** нажмите кнопку **OK**. В списке **Member functions:** появится имя новой функции `OnNewHelp`.

13. Нажмите кнопку **Edit Code**. Окно **MFC ClassWizard** закроется, откроется окно редактирования файла HelpAppView.cpp и текстовый курсор установится в тексте функции `OnNewHelp`.

14. Измените функцию `OnNewHelp` в соответствии с приведенным ниже текстом:

```
void CHelpAppView::OnNewHelp()
{
  // TODO: Add your command handler code here

  WinHelp( HID_NEW_HELP);
}
```

15. Откройте для редактирования файл HelpAppView.h и после оператора `#endif // _MSC_VER > 1000` вставьте директиву

```
#define    HID_NEW_HELP   0x01
```

16. Выберите команду **File**, **New...**. Раскроется диалоговое окно **New**, изображенное на рис. 13.14.

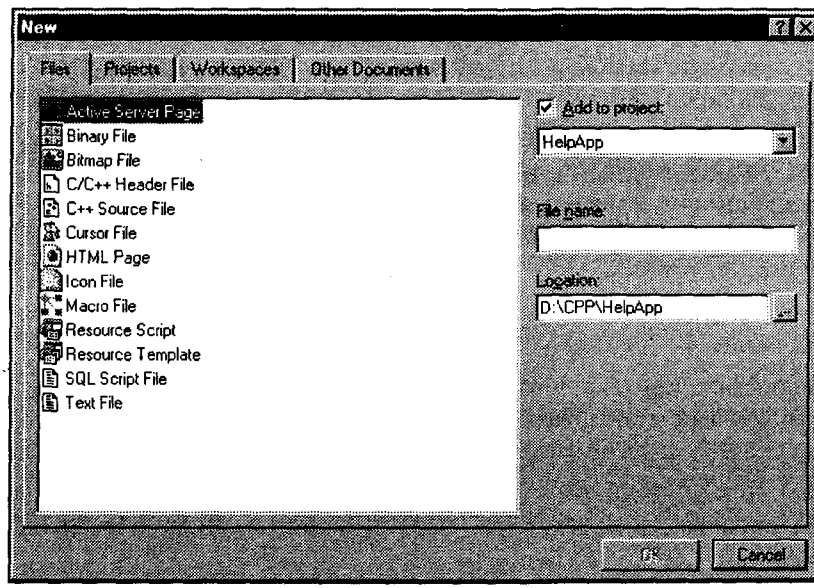


Рис. 13.14. Диалоговое окно **New**

17. Перейдите на вкладку **Files** и выделите значок **Text File**. В текстовое поле **File name:** (имя файла) введите имя файла "NewHelp.hm".

18. В текстовое поле **Location:** (Расположение) введите путь **D:\CPP\HelpApp \hlp** (или другой путь в подкаталог \hlp каталога вашего приложения) и нажмите кнопку **OK**.
19. В окне редактирования появится новый чистый текстовый файл. Введите в него строку

```
HID_NEW_HELP  0x01
```

20. Раскройте вкладку **FileView** окна **Workspace**.
21. Раскройте папку \HelpApp Files, в ней — папку \Source и дважды щелкните левой кнопкой мыши на значке **HelpApp.hpj**. Откроется окно редактирования данного файла.
22. В конце этого файла вставьте строку

```
#include <NewHelp.hm>
```

Создание команды меню и функции ее обработки ничем не отличаются от подобных операций для других команд меню. В функции обработки сообщения `ID_NEW_HELP` с использованием функции `CWinApp::WinHelp` вызывается справочная система по контекстному идентификатору `HID_NEW_HELP`.

Контекстный идентификатор используется для связи программы со справочной системой, поэтому его необходимо объявить одновременно в двух файлах: в файле заголовка программы и в файле адресации справок. Поэтому определение данного идентификатора включается в файлы HelpAppView.h и NewHelp.hm. Вообще-то, определение контекстного идентификатора может быть помещено в уже существующий файл HelpApp.hm, и файл NewHelp.hm создан исключительно для демонстрации возможности его создания. Файл NewHelp.hm должен быть зарегистрирован в файле HelpApp.hpj.

Описанных действий недостаточно для создания командной справки, поскольку с указанным контекстным идентификатором не связан никакой раздел справки, однако, этот вопрос будет рассмотрен позднее при описании процедуры редактирования исходного текстового файла справки.

Создание системы контекстной справки

Система контекстной справки обычно используется при работе с диалоговыми окнами и командами меню. Поэтому, прежде всего, создадим демонстрационное диалоговое окно и поместим в него кнопку, для которой будет создана контекстная справка.

Чтобы внести в приложение HelpApp изменения, необходимые для демонстрации процедуры создания контекстной и других видов справки:

1. Откройте приложение HelpApp и перейдите на вкладку **ResourceView** окна **Workspace**.
2. Раскройте папку \HelpApp Resources, а в ней — папку \Menu.

3. Дважды щелкните левой кнопкой мыши на значке **IDR_HELPAPTYPE**. Откроется окно редактирования ресурса меню.
4. Поместите указатель мыши на штриховой прямоугольник, расположенный в конце панели меню, нажмите левую кнопку мыши и переместите появившийся разделитель между командами меню **Вид** и **Окно**.
5. Отпустите левую кнопку мыши и щелкните в штриховом прямоугольнике правой кнопкой мыши.
6. В появившемся контекстном меню выберите команду **Properties**. Появится диалоговое окно **Menu Item Properties**.
7. Нажмите кнопку **Keep Visible**, расположенную в левом верхнем углу диалогового окна **Menu Item Properties**.
8. Введите в текстовое поле **Caption:** заголовок "&Демо".
9. Щелкните левой кнопкой мыши в пунктирном прямоугольнике, расположенном в раскрывшемся меню.
10. Введите в текстовое поле раскрывающегося списка **ID:** идентификатор команды "ID_DEMO_DIALOG", в текстовое поле **Caption:** введите заголовок "&Вызов", а в текстовое поле **Prompt:** — текст "Вызов диалогового окна\nВызов".
11. В окне **Workspace** щелкните правой кнопкой мыши на папке \Dialog.
12. В появившемся контекстном меню выберите команду **Insert Dialog** (Создать новый ресурс диалогового окна).
13. В окне редактирования ресурсов появится заготовка нового диалогового окна.
14. Дважды щелкните левой кнопкой мыши на идентификаторе нового диалогового окна `IDD_DIALOG1`. Появится диалоговое окно **Dialog Properties**.
15. В текстовое поле раскрывающегося списка **ID:** введите идентификатор ресурса "IDD_DEMO", а в текстовое поле **Caption:** — заголовок "Контекстная справка".
16. Раскройте вкладку **Extended Styles** диалогового окна **Dialog Properties**.
17. Установите флажок **Context help**, как показано на рис. 13.15.

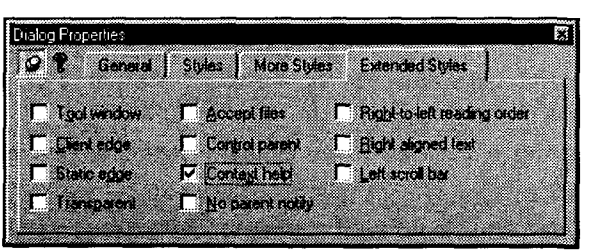


Рис. 13.15. Вкладка **Extended Styles** диалогового окна **Dialog Properties**

18. Поместите указатель мыши на кнопку **Button** панели инструментов **Controls** и нажмите левую кнопку мыши.
19. Не отпуская кнопку мыши, перетащите изображение кнопки в заготовку диалогового окна.
20. В текстовом поле раскрывающегося списка **ID:** введите идентификатор "IDC_CONTEXT", а в текстовое поле **Caption:** — заголовок "&Демо".
21. Нажмите кнопку **Cancel** заготовки диалогового окна и введите в текстовое поле **Caption:** заголовок "&Отмена".
22. Закройте диалоговое окно **Menu Item Properties** и вызовите мастер ClassWizard.
23. Появится диалоговое окно **Adding a Class**. Нажмите в нем кнопку **OK**, согласившись создать новый класс для данного диалогового окна.
24. Появится диалоговое окно **New Class**. Введите в текстовое поле **Name:** имя класса "CDemoDlg" и нажмите кнопку **OK**.
25. Появится окно **MFC ClassWizard**.
26. В раскрывающемся списке **Class name:** выберите имя класса `CHelpAppView`, в окне списка **Object IDs:** выделите идентификатор команды `ID_DEMO_DIALOG`, в окне списка **Messages:** выберите сообщение `COMMAND` и нажмите кнопку **Add Function**. В появившемся окне **Add Member Function** нажмите кнопку **OK**. В списке **Member functions:** появится имя новой функции `OnDemoDialog`.
27. Нажмите кнопку **Edit Code**, окно **MFC ClassWizard** закроется, откроется окно редактирования файла HelpAppView.cpp и текстовый курсор установится в тексте функции `OnDemoDialog`.
28. Измените функцию `OnNewHelp` в соответствии с приведенным ниже текстом:

```
void CHelpAppView::OnDemoDialog()
{
  // TODO: Add your command handler code here

  CDemoDlg dlg( this);

  dlg.DoModal();
}
```

29. После строки `#include "HelpAppView.h"` вставьте строку

```
#include "DemoDlg.h"
```

30. Откройте для редактирования файл DemoDlg.h и измените карту сообщений класса `CDemoDlg` в соответствии с приведенным ниже текстом:

```
// Generated message map functions
//{{AFX_MSG(CDemoDlg)
  // NOTE: the ClassWizard will add member functions here
//}}AFX_MSG
afx_msg    BOOL OnHelpInfo( HELPINFO*);
afx_msg    void OnContextMenu( CWnd*, CPoint);
DECLARE_MESSAGE_MAP()
```

31. После строки `// CDemoDlg dialog` вставьте следующий текст:

```
#define HIDD_CONTEXT          2
#define HIDD_CONTEXT_OK       3
#define HIDD_CONTEXT_CANCEL   4
```

32. Откройте на редактирование файл DemoDlg.cpp и измените карту сообщений класса `CDemoDlg` в соответствии с приведенным ниже текстом:

```
BEGIN_MESSAGE_MAP(CDemoDlg, CDialog)
  //{{AFX_MSG_MAP(CDemoDlg)
    // NOTE: the ClassWizard will add message map macros here
  //}}AFX_MSG_MAP
  ON_WM_HELPINFO()
  ON_WM_CONTEXTMENU()
END_MESSAGE_MAP()
```

33. После строки `// CDemoDlg dialog` поместите следующий текст:

```
static  DWORD  aHelpIDs[] =
{
  IDC_CONTEXT, HIDD_CONTEXT,
  IDOK, HIDD_CONTEXT_OK,
  IDCANCEL, HIDD_CONTEXT_CANCEL,
  0, 0
};
```

34. После строки `// CDemoDlg message handlers` поместите следующий текст:

```
BOOL CDemoDlg::OnHelpInfo( HELPINFO* lpHelpInfo)
{
  if( lpHelpInfo->iContextType == HELPINFO_WINDOW)
    ::WinHelp( (HWND) lpHelpInfo->hItemHandle,
    AfxGetApp()->m_pszHelpFilePath, HELP_WM_HELP, (DWORD) aHelpIDs);
  return TRUE;
}

void CDemoDlg::OnContextMenu( CWnd* pWnd, CPoint /*point*/)
{
  ::WinHelp( (HWND) *pWnd, AfxGetApp()->m_pszHelpFilePath,
  HELP_CONTEXTMENU, (DWORD) aHelpIDs);
}
```

35. Откройте на редактирование файл NewHelp.hm и вставьте в его конец строки:

```
HIDD_CONTEXT     0x02
HIDD_CONTEXT_OK     0x03
HIDD_CONTEXT_CANCEL  0x04
```

При создании контекстной справки в карту сообщений добавляются объявления функций обработки сообщений контекстной справки. Поскольку эти объявления добавляются пользователем, а не мастером ClassWizard, они находятся за пределами соответствующих скобок макросов.

В файл DemoDlg.cpp добавляется массив `aHelpIDs`, определяющий соответствие идентификаторов элементов управления контекстным идентификаторам, которые должны быть объявлены в файлах заголовка и адресации справок. Для хранения этих идентификаторов используется созданный ранее файл адресации справок NewHelp.hm. Как и в предыдущем случае, вместо него мог быть использован файл HelpApp.hm.

При создании контекстной справки во вкладке **General** диалогового окна **Push Button Properties** не устанавливался флажок **Help ID**, позволяющий автоматически формировать контекстные идентификаторы, поскольку при установке данного флажка массив соответствия идентификаторов элементов управления их контекстным идентификаторам не формируется автоматически, и не происходит автоматического включения контекстных идентификаторов в файл заголовка соответствующего класса диалогового окна.

Введенные в класс диалогового окна функции обработки сообщений используются для вызова приложения WinHelp. Функция `CWnd::OnHelpInfo` вызывается приложением, если пользователь нажимает клавишу <F1>. Она проверяет, что в данном случае справка вызывается для окна, а не для команды меню, и в случае успешного завершения проверки вызывается глобальная функция `WinHelp`.

Необходимость вызова глобальной версии функции `WinHelp` обусловлена тем, что функция `CWinApp::WinHelp` не может принимать в качестве аргумента дескриптор окна. Функция `CWnd::OnContextMenu` вызывается приложением при щелчке правой кнопки мыши в окне. В ней нет необходимости производить какие-либо проверки, поэтому она сразу вызывает глобальную функцию `WinHelp`.

Подготовка справочных текстов

Создатели языка Visual C++ понимали под подготовкой справочных текстов добавление или исключение разделов справки в исходных файлах справочной системы, имеющих формат .rtf, однако для русского пользователя также важен вопрос перевода уже содержащихся в них текстов на русский язык.

Русификация справочной системы приложения

Чтобы редактировать исходный файл справки:

1. Откройте приложение Microsoft Word, используя значок на рабочем столе или кнопку **Пуск** в **Панели задач**.
2. Выберите команду меню **Файл**, **Открыть...** (**File**, **Open...**) или нажмите соответствующую кнопку в панели инструментов.
3. В появившемся диалоговом окне **Открытие документа** в раскрывающемся списке **Тип файлов:** выберите **Текст в формате RTF**, раскройте папку \hlp в каталоге вашего проекта, как показано на рис. 13.16, и откройте нужный исходный файл справки.

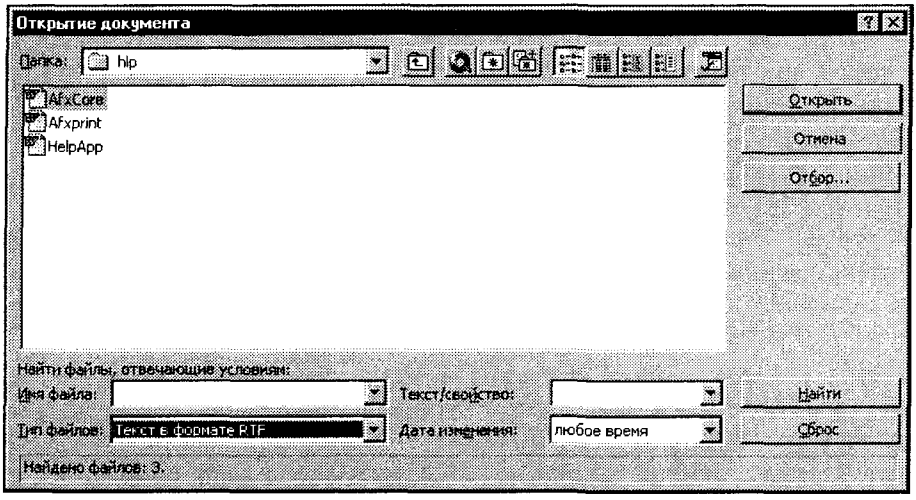


Рис. 13.16. Диалоговое окно **Открытие документа**

4. Выберите команду **Вид**, **Сноски** (**View**, **Footnotes**). В окне редактирования текста появится панель сносок, как показано на рис. 13.17.
5. Выберите команду **Сервис**, **Параметры...** (**Tools**, **Options**). Появится диалоговое окно **Параметры**.
6. Раскройте вкладку **Вид** (**View**) и установите флажок **скрытый текст** (**Hidden text**) или флажок **все**, как показано на рис. 13.18.

Скрытым текстом выводятся связи между разделами справочной системы. Каждый раздел начинается с новой страницы.

Справочная система использует восемь типов сносок, которые перечислены ниже:

- `#` – идентификатор раздела справки. Используется глобальной функцией `WinHelp` при поиске данного раздела справки

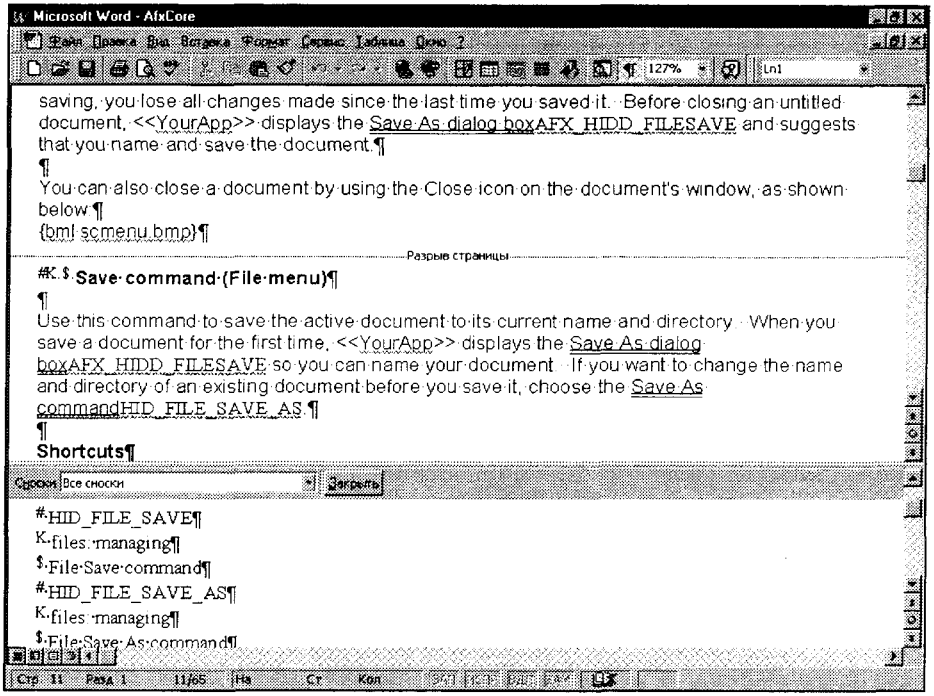


Рис. 13.17. Исходный файл справки с панелью сносок

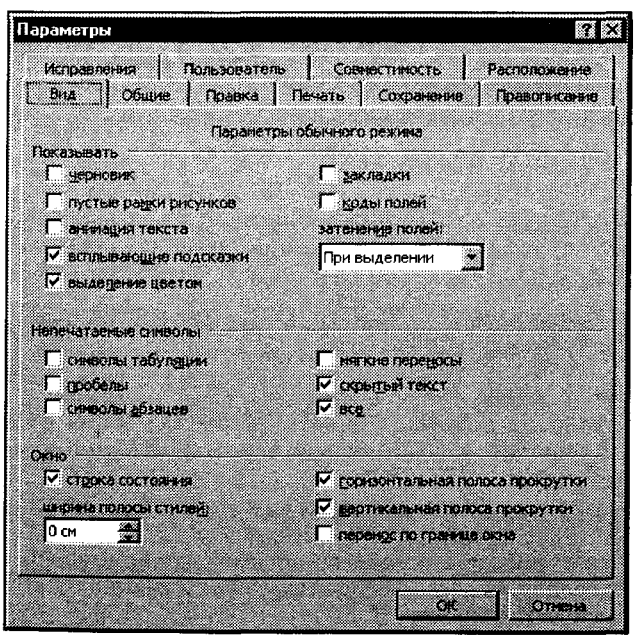


Рис. 13.18. Диалоговое окно **Параметры**

- `$` – `заголовок раздела`. Выводится на экран при завершении поиска
- `K` – `ключевые слова`. Выводятся во вкладку **Предметный указатель** диалогового окна **Справочная система**
- `A` – `ключевое слово типа A`. Эти ключевые слова могут вызывать переход к следующей теме, но не выводятся во вкладку **Предметный указатель** диалогового окна **Справочная система**
- `+` – `код просмотра`. Определяет положение данного раздела в последовательности разделов
- `!` – `вход макроса`. Определяет макрос раздела, запускаемый при запросе данного раздела пользователем
- `*` – `тег построения`. Используется для включения определенных тегов в файл справочной системы специального формата
- `>` – `тип окна`. Определяет тип окна, используемого для вывода данного раздела

Текст, подчеркнутый двойной линией, за которым следует скрытый текст, указывает на возможность перехода к другому разделу справочной системы. Если пользователь щелкнет левой кнопкой мыши на данном тексте, отображаемом в окне справки, то вместо текущей темы в данном окне будет отображена новая тема, определяемая скрытым текстом.

Если текст перед скрытым текстом подчеркнут одной линией, то справка, определяемая скрытым текстом, появится в всплывающем окне поверх текущего окна справки. В обоих случаях скрытый текст содержит идентификатор раздела справочной системы, на который будет сделан переход или который будет отображаться в всплывающем окне справки.

Создаваемые мастером AppWizard исходные файлы справочной системы содержат достаточно большой объем информации, не относящейся к создаваемой заготовке приложения, но эти темы могут стать актуальными, если заготовка будет превращена в рабочее приложение. Кроме того, в нем предусмотрены строки подсказок, указывающие, какого рода информацию следует подставить вместо них.

Русификация приложения подразумевает, конечно, и русификацию справочной системы. Однако мастер AppWizard не позволяет создать исходный файл справки на русском языке. Ниже будет приведен вариант исходного файла справки на русском языке, созданный путем замены соответствующих текстов в исходном файле справки, созданном мастером AppWizard.

Внимание!

Исходный файл справки имеет достаточно сложный формат, который необходимо максимально точно передать при его воспроизведении в книге, поэтому в него были введены следующие изменения:

Начало новой страницы отмечается строкой **<<Новая страница>>**

Комбинация заголовка раздела, подчеркнутого двойной линией, и идентификатора раздела, выведенного скрытым текстом, представляется в виде слитного написания заголовка и идентификатора раздела, в котором заголовок подчеркнут одной линией

Местодержатель (placeholder) **<<YourApp>>**, используемый для обозначения имени приложения, остается без изменения

Удалена ссылка на заголовок раздела **Menu Record**, для которой отсутствует раздел справочной системы

Файл afxcore.rtf после его русификации с учетом приведенных выше соглашений по изменению формата имеет следующий вид:

```
# <<YourApp>> Предметный указатель

Что это такое ...
  <<добавьте сюда разделы контекстной справки вашего приложения>>

Команды меню
  Меню Файлmenu_file
  Меню Правкаmenu_edit
  Меню Видmenu_view
  Меню Окнаmenu_window
  Меню Справкаmenu_help

<<Новая страница>>
#K $ Команды меню Файл

Меню Файл содержит следующие команды:

СоздатьHID_FILE_NEW              Создание нового документа.
ОткрытьHID_FILE_OPEN             Открытие существующего документа.
ЗакрытьHID_FILE_CLOSE            Закрывает текущий документ.
СохранитьHID_FILE_SAVE           Сохраняет текущий документ, используя
                                 то же имя файла.
Сохранить какHID_FILE_SAVE_AS    Сохраняет текущий документ в файле
                                 с указанным именем.
ПечатьHID_FILE_PRINT             Печать текущего документа.
Предварительный просмотрHID_FILE_PRINT_PREVIEW      Выводит окно
                                 предварительного просмотра печати.
Настройка принтераHID_FILE_PRINT_SETUP  Выбор принтера и настройка его
                                        параметров.
Отправить...HID_FILE_SEND_MAIL   Посылает активный документ по электронной
                                 почте.
ВыходHID_APP_EXIT                Завершает работу приложения <<YourApp>>.

<<Новая страница>>
```

#**Команды меню Правка**

Меню Правка содержит следующие команды:

ОтменитьHID_EDIT_UNDO	Отменяет предыдущую операцию редактирования.
ВырезатьHID_EDIT_CUT	Уничтожает выделенный фрагмент в документе и помещает его в буфер обмена.
КопироватьHID_EDIT_COPY	Копирует в буфер обмена выделенный фрагмент в документе.
ВставитьHID_EDIT_PASTE	Копирует данные из буфера обмена в активный документ.
Специальная вставкаHID_EDIT_PASTE_LINK	Копирует из буфера обмена связь с данными другого приложения.
Вставка объектаHID_OLE_INSERT_NEW	Вставляет в документ или связывает с ним объект такой, как диаграмма или формула.
СвязиHID_OLE_EDIT_LINKS	Выводит и позволяет редактировать связи с внедренными документами.

<<Новая страница>>

#**Команды меню Вид**

Меню Вид содержит следующие команды:

Панель инструментовHID_VIEW_TOOLBAR	Отображает или скрывает панель инструментов.
Панель состояния HID_VIEW_STATUS_BAR	Отображает или скрывает панель состояния.

<<Новая страница>>

#**Команды меню Окно**

Меню Окно содержит следующие команды, позволяющие пользователю организовать несколько представлений нескольких документов в многооконном приложении:

НовоеHID_WINDOW_NEW	Создает новое окно для отображения текущего документа.
КаскадироватьHID_WINDOW_CASCADE	Выводит перекрывающиеся окна.
УпорядочитьHID_WINDOW_TILE	Выводит неперекрывающиеся окна.
Упорядочить значкиHID_WINDOW_ARRANGE	Упорядочивает значки минимизированных окон.
РазделитьHID_WINDOW_SPLIT	Разделяет активное окно на панели.
Окно 1, 2, ...HID_WINDOW_ALL	Осуществляет переход к указанному окну.

<<Новая страница>>

#**Команды меню Справка**

Меню Справка содержит следующие команды, позволяющие получить справочную информацию по данному приложению:

Вызов справкиHID_HELP_INDEX	Выводит предметный указатель по разделам справочной системы.
О программеHID_APP_ABOUT	Выводит номер версии данного приложения.

<<Новая страница>>

#K $ **Команда Создать (Меню Файл)**

Данная команда меню используется для создания нового документа в приложении <<YourApp>>. Тип нового документа определяется в диалоговом окне Создание документаAFX_HIDD_NEWTYPEDLG. << Удалите предыдущее предложение, если ваше приложение работает только с одним типом документа. >>

Для открытия существующего документа может быть использована команда меню ОткрытьHID_FILE_OPEN.

Другие способы вызова

Панель инструментов: {bmc filenew.bmp}
Комбинация клавиш: CTRL+N

<<Новая страница>>

#**Диалоговое окно Создание документа**

<< Удалите этот раздел, если ваше приложение работает только с одним типом документа. >>

Определяет типы документов, создаваемые в приложении:
<< Перечислите типы документов, создаваемых в приложении >>

<<Новая страница>>

#K $ **Команда Открыть (Меню Файл)**

Данная команда меню используется для открытия существующего документа в новом окне. В приложении могут быть одновременно открыты несколько документов. Для перемещения между окнами документов используйте меню Окно. См. See команду меню Окно 1, 2, ... HID_WINDOW_ALL.

Для создания нового документа используйте команду меню СоздатьHID_FILE_NEW.

Другие способы вызова

Панель инструментов: {bmc fileopen.bmp}
Комбинация клавиш: CTRL+O

<<Новая страница>>

#**Диалоговое окно Открытие документа**

Перечисленные ниже элементы управления диалогового окна Открытие документа позволяют задать открываемый файл:

Имя файла

Текстовое поле, в которое нужно ввести имя открываемого файла. Имя файла можно выделить в расположенном выше окне списка и открыть его двойным щелчком левой кнопки мыши.

Тип файлов

Определяет тип и расширение файлов, отображаемых в окне списка. В данном приложении могут быть открыты файлы следующих типов:

<< Перечислите типы файлов, используемых в приложении. >>

Папка

Раскрывающийся список, позволяющий открыть любую папку на любом диске компьютера.

<<Новая страница>>

#K $ **Команда Закрыть (Меню Файл)**

Данная команда меню используется для закрытия всех окон, содержащих текущий документ. Если данный документ не был сохранен перед данной операцией, приложение <<YourApp>> выводит окно сообщения о необходимости сохранить изменения, внесенные в данный документ. Если документ не имел имени, то автоматически открывается диалоговое окно Сохранение документа AFX_HIDD_FILESAVE, в котором предлагается выбрать имя файла сохраняемого документа.

<< Все описанные выше действия должны быть реализованы в данном приложении. >>

Документ может быть закрыт с использованием приведенного ниже значка Закрыть в окне документа:

{bml scmenu.bmp}

<<Новая страница>>

#K $ **Команда Сохранить (Меню Файл)**

Данная команда меню используется для сохранения активного документа с использованием его текущего имени и каталога. При сохранении документа без имени приложение <<YourApp>> выводит диалоговое окно Сохранение документа AFX_HIDD_FILESAVE, позволяющее выбрать имя файла сохраняемого документа. Если необходимо изменить имя или каталог существующего документа перед его сохранением, выберите команду меню Сохранить как HID_FILE_SAVE_AS.

Другие способы вызова

Панель инструментов:	{bmc filesave.bmp}
Комбинация клавиш:	CTRL+S

<<Новая страница>>

#K $ **Команда Сохранить как (Меню Файл)**

Данная команда меню используется для сохранения активного документа в новом каталоге или под новым именем. Данная команда меню выводит диалоговое окно Сохранение документа AFX_HIDD_FILESAVE.

Чтобы сохранить документ под его текущим именем в своем каталоге, выберите команду меню Сохранить HID_FILE_SAVE.

<<Новая страница>>

#**Команда Отправить (Меню Файл)**

Данная команда меню используется для пересылки активного документа по электронной почте. Эта команда выводит окно электронной почты, в котором содержится активный документ. Пользователь должен заполнить поля адреса, темы и другие поля, необходимые для пересылки сообщения и нажать кнопку Отправить.

<<Новая страница>>

#**Диалоговое окно Сохранение документа**

Перечисленные ниже элементы управления диалогового окна Сохранение документа позволяют определить имя файла, в котором будет сохранен документ:

Имя файла

Текстовое поле, в которое нужно ввести имя файла, в котором будет сохранен документ. Если предполагается сохранить документ в одном из существующих файлов, выделите его имя в расположенном выше окне списка и дважды щелкните левой кнопки мыши. После этого появится диалоговое окно с вопросом о необходимости замены существующего файла.

Тип файлов

Определяет тип и расширение файлов, отображаемых в окне списка. В данном приложении могут сохраняться файлы следующих типов:

<< Перечислите типы файлов, используемых в приложении. >>

Папка

Раскрывающийся список, позволяющий открыть любую папку на любом диске компьютера.

<< Опишите другие элементы управления диалогового окна Сохранение документа, набор которых определяется флагами OFN_ в структуре OPENFILENAME, используемой в конструкторе класса CFileDialog. >>

<<Новая страница>>

K $ **Команды 1, 2, 3, 4 (Меню Файл)**

Номера и имена файлов, перечисленных в нижней части меню Файл позволяют открыть файлы документов, которые были закрыты последними. Для вызова файла по номеру в списке одновременно нажмите клавишу Shift и соответствующий номер.

<<Новая страница>>

#K $ **Команда Выход (Меню Файл)**

Данная команда меню закрывает текущую копию приложения <<YourApp>>. При наличии в приложении <<YourApp>> несохраненных документов, оно выводит сообщение, предупреждающее о необходимости их сохранить.

Другие способы вызова

Мышь: Нажать кнопку Закрыть, расположенную в заголовке главного окна приложения.

{bmc appexit.bmp}

Комбинация клавиш: ALT+F4

<<Новая страница>>

#**Команда Отменить (Меню Правка)**

<< Способ использования данной команды в приложении может отличаться от описанного ниже. Если это так, измените текст данного раздела. >>

Данная команда используется для отмены, если это возможно, последней операции редактирования.

Другие способы вызова

Панель инструментов: {bmc editundo.bmp}

Комбинация клавиш: CTRL+Z или
ALT-BACKSPACE

<<Новая страница>>

#**Команда Повторить (Меню Правка)**

<< Содержимое данного раздела зависит от приложения. >>

<<Новая страница>>

#**Команда Вырезать (Меню Правка)**

Данная команда меню уничтожает выделенный фрагмент в документе и помещает его в буфер обмена. Команда недоступна, если в приложении отсутствует выделенный фрагмент.

Помещенные в буфер обмена данные замещают прежнее его содержимое.

Другие способы вызова

Панель инструментов: {bmc editcut.bmp}

Комбинация клавиш: CTRL+X

<<Новая страница>>

#**Команда Копировать (Меню Правка)**

Данная команда меню копирует в буфер обмена выделенный фрагмент в документе. Команда недоступна, если в приложении отсутствует выделенный фрагмент.

Помещенные в буфер обмена данные замещают прежнее его содержимое.

Другие способы вызова

Панель инструментов: {bmc editcopy.bmp}

Комбинация клавиш: CTRL+C

<<Новая страница>>

#**Команда Вставить (Меню Правка)**

Данная команда меню копирует данные из буфера обмена в активный документ. Команда недоступна, если в буфере обмена не содержится информации.

Другие способы вызова

Панель инструментов: {bmc editpast.bmp}
Комбинация клавиш: CTRL+V

<<Новая страница>>

#K $ **Команда Панель инструментов (Меню Вид)**

Данная команда меню используется для вывода на экран и скрытия панели инструментов, содержащей кнопки наиболее часто используемых команд приложения <<YourApp>>, таких как Открыть. Для вывода на экран панели инструментов необходимо установить флажок у данного пункта меню.

См. справочную информацию по теме Панель инструментовAFX_HIDW_TOOLBAR.

<<Новая страница>>

#K $ **Панель инструментов**

{bml hlptbar.bmp}

Панель инструментов изначально выводится в верхней части главного окна приложения под панелью меню. Она может быть установлена вдоль любой из сторон главного окна приложения или выводиться в виде отдельного окна. Панель инструментов позволяет получить быстрый доступ к основным командам приложения <<YourApp>>.

Чтобы вывести или скрыть панель инструментов выберите команду Вид, Панель инструментов.

<< Откорректируйте приведенный ниже список кнопок панели инструментов *вашего приложения*. >>

Кнопка	**Описание**
{bmc filenew.bmp}	Создает новый документ.
{bmc fileopen.bmp}	Открывает существующий документ. При этом приложение <<YourApp>> выводит диалоговое окно Открытие документа, позволяющее выбрать нужный файл.
{bmc filesave.bmp}	Сохраняет активный документ без изменения его имени. Если активный документ не имеет имени, приложение <<YourApp>> выводит диалоговое окно Сохранение документа.
{bmc fileprnt.bmp}	Печатает активный документ.
{bmc editcut.bmp}	Уничтожает выделенный фрагмент в документе и помещает его в буфер обмена.

{bmc editcopy.bmp}	Копирует в буфер обмена выделенный фрагмент в документе.
{bmc editpast.bmp}	Копирует данные из буфера обмена в активный документ.
{bmc editundo.bmp}	Отменяет последнюю операцию редактирования. Примечание: Некоторые операции не могут быть отменены.
{bmc recfirst.bmp}	Переход к первой записи в выделенном фрагменте.
{bmc recprev.bmp}	Переход к предыдущей записи в выделенном фрагменте.
{bmc recnext.bmp}	Переход к следующей записи в выделенном фрагменте.
{bmc reclast.bmp}	Переход к последней записи в выделенном фрагменте.

<<Новая страница>>

#K $ **Команда Строка состояния (Меню Вид)**

Данная команда меню используется для вывода на экран или скрытия строки состояния, в которую выводится краткая справка по выделенной команде меню или кнопке панели инструментов, а также состояние некоторых управляющих клавиш. Для вывода на экран строки состояния необходимо установить флажок у данного пункта меню.

См. справочную информацию по теме Строка состоянияAFX_HIDW_STATUS_BAR.

<<Новая страница>>

#K $**Строка состояния**

{bml hlpsbar.bmp}

Строка состояния выводится в нижней части главного окна приложения <<YourApp>>. Чтобы вывести на экран строку состояния выберите команду Вид, Строка состояния.

В левой части строки состояния выводится справочная информация по выделенным командам меню. Здесь же выводится справочная информация по кнопкам панели инструментов, на которых располагается указатель мыши.

Правая часть строки состояния содержит панели, в которых отображается состояние следующих индикаторов клавиатуры:

Панель	**Описание**
CAP	Горит индикатор Caps Lock.
NUM	Горит индикатор Num Lock.
SCRL	Горит индикатор Scroll Lock.

<<Новая страница>>

Команда Новое (Меню Окно)

Данная команда меню создает новое окно для вывода активного документа. Пользователь может открыть несколько окон для отображения различных

фрагментов одного документа. При изменении содержимого одного окна все окна, в которых отображается данный документ, отразят это изменение. При открытии нового окна оно становится активным и выводится поверх всех других открытых окон.

<<Новая страница>>

#**Команда Каскад (Меню Окно)**

Данная команда меню выводит перекрывающиеся окна.

<<Новая страница>>

#**Команда Упорядочить (Меню Окно)**

Данная команда меню выводит неперекрывающиеся окна.

<<Новая страница>>

#**Команда Расположить по горизонтали (Меню Окно)**

Данная команда меню выводит неперекрывающиеся окна, расположенные одно над другим (если количество окон не превышает трех).

<<Новая страница>>

#**Команда Расположить по вертикали (Меню Окно)**

Данная команда меню выводит неперекрывающиеся окна, расположенные одно рядом с другим (если количество окон не превышает трех).

<<Новая страница>>

#**Команда Упорядочить значки (Меню Окно)**

Данная команда упорядочивает значки минимизированных окон в нижней части главного окна приложения. Если в нижней части главного окна приложения располагается открытое окно, то некоторые или все значки могут быть не видны, поскольку будут располагаться под этим окном.

<<Новая страница>>

Команда Разделить (Меню Окно)

Данная команда меню позволяет разделить активное окно на панели. Для установки разделительной полосы между панелями можно использовать мышь или клавиши управления курсором. По завершении настройки щелкните кнопкой мыши или нажмите клавишу Enter. Нажатие клавиши Esc возвращает разделительную полосу в ее первоначальное положение.

<< В однооконном приложении эта команда расположена в меню Вид. >>

<<Новая страница>>

#**Команды 1,2,... (Меню Окно)**

Приложение <<YourApp>> выводит список открытых окон документов в нижней части меню Окно. Перед именем активного окна ставится флажок. Выбор документа из списка делает его окно активным.

<<Новая страница>>

#**Команда Вывод справки (Меню Справка)**

Данная команда меню выводит диалоговое окно Справочная система и раскрывает в нем вкладку Предметный указатель. Эта вкладка позволяет производить в справочной системе приложения поиск по ключевому слову.

Для возврата в приложение необходимо закрыть диалоговое окно Справочная система.

<<Новая страница>>

#**Команда Использование справки (Меню Справка)**

Данная команда меню позволяет получить информацию по пользованию справочной системой приложения.

<<Новая страница>>

#**Команда О программе (Меню Справка)**

Данная команда меню позволяет вывести информацию о правах собственности и версии приложения <<YourApp>>.

<<Новая страница>>

#$**Команда Справка**

{bml curhelp.bmp}

Команда Справка позволяет получить справочную информацию о различных элементах управления приложения <<YourApp>>. При нажатии кнопки Справка в панели инструментов указатель мыши приобретает вид стрелки и знака вопроса. При щелчке таким указателем на элементах управления приложение <<YourApp>> вызывает контекстную справку по данному элементу управления window.

Другие способы вызова

Комбинация клавиш: SHIFT+F1

<<Новая страница>>

#**Заголовок окна**

<< Поместите здесь заголовок вашего приложения. >>

Заголовок окна располагается в его верхней части. Он содержит имя приложения и документа.

Для перемещения окна следует перетащить его заголовок.
Примечание: Для перемещения диалоговых окон следует перетащить их заголовок.

Заголовок может содержать следующие элементы:

{bmc bullet.bmp} Системное меню приложения
{bmc bullet.bmp} Меню документа

{bmc bullet.bmp} Кнопка Развернуть окно
{bmc bullet.bmp} Кнопка Свернуть окно
{bmc bullet.bmp} Имя приложения
{bmc bullet.bmp} Имя документа
{bmc bullet.bmp} Кнопка Восстановить окно

<<Новая страница>>

Полосы прокрутки

Отображаются у правой и нижней границ окна документа. Бегунки, расположенные в полосах прокрутки, показывают вертикальное и горизонтальное положение окна в документе. Для прокрутки документа может быть использована мышь.

<< Опишите действия, которые можно производить с полосой прокрутки в вашем приложении. >>

<<Новая страница>>

Команда Размер (Меню документа)

Эта команда отображает на экране крестик, образованный из четырех стрелок, направленных наружу. Этот крестик может использоваться для изменения размеров активного окна с использованием клавиш управления курсором.

{bml curarw4.bmp}

После того как курсор принял форму крестика из стрелок:

1. Нажмите одну из клавиш управления курсором (влево, вправо, вверх или вниз) и переместите курсор на ту сторону рамки окна, которую необходимо переместить.
2. Используйте клавиши управления курсором для перемещения стороны рамки.
3. Нажмите клавишу ENTER, когда окно примет желаемый размер.

Примечание: Эта команда недоступна, если окно расширено до максимального размера.

Другие способы вызова

Мышь: Поместите указатель мыши на рамку окна, нажмите левую кнопку мыши и, не отпуская ее, измените размер окна.

<<Новая страница>>

Команда Переместить (Меню документа)

Эта команда отображает на экране крестик, образованный из четырех стрелок, направленных наружу. Этот крестик может использоваться для перемещения активного окна или диалогового окна с использованием клавиш управления курсором.

{bmc curarw4.bmp}

Примечание: Эта команда недоступна, если окно расширено до максимального размера.

Другие способы вызова

Комбинация клавиш: CTRL+F7

<<Новая страница>>

#$**Команда Свернуть (Системное меню)**

Данная команда меню позволяет свернуть приложение <<YourApp>> в значок.

Другие способы вызова

Мышь: Нажмите кнопку Свернуть окно {bmc scmin.bmp} в заголовке окна.

Комбинация клавиш: ALT+F9

<<Новая страница>>

#**Команда Развернуть (Меню документа)**

Данная команда меню позволяет расширить размеры окна таким образом, чтобы оно занимало все доступное пространство.

Другие способы вызова Команда Следующее (Меню документа)

Мышь: Нажмите кнопку Развернуть {bmc scmax.bmp} в заголовке окна или дважды щелкните в заголовке окна.

Комбинация клавиш: CTRL+F10 для окон документов.

<<Новая страница>>

#**Команда Следующее (Меню документа)**

Данная команда меню используется для перехода к следующему окну документа. Приложение <<YourApp>> определяет следующее окно исходя из порядка их открытия.

Другие способы вызова

Комбинация клавиш: CTRL+F6

<<Новая страница>>

#**Команда Следующее (Меню документа)**

Данная команда меню используется для перехода к следующему окну документа. Приложение <<YourApp>> определяет следующее окно исходя из порядка их открытия.

Другие способы вызова

Комбинация клавиш: SHIFT+CTRL+F6

<<Новая страница>>

#**Команда Закрыть (Меню документа)**

Данная команда меню используется для закрытия активного окна или диалогового окна.

Вместо выбора этой команды можно нажать кнопку Закрыть окно в заголовке активного окна.

{bml appexit.bmp}

Примечание: Если для одного документа открыто несколько окон, то для одновременного закрытия всех окон данного документа следует выбрать команду Файл, Закрыть.

Другие способы вызова

Комбинация клавиш: CTRL+F4 закрывает окно документа
ALT+F4 закрывает приложение <<YourType>> или диалоговое окно

<<Новая страница>>

#**Команда Восстановить (Меню документа)**

Данная команда позволяет возвратить размер и позицию активного окна до вызова для него команд Развернуть или Свернуть.

<<Новая страница>>

#**Список задач (Меню приложения)**

Данная команда позволяет открыть меню Пуск операционной системы Windows.

Другие способы вызова

Комбинация клавиш: CTRL+ESC

<<Новая страница>>

#$**Команда Линейка (Меню Вид)**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$ **Диалоговое окно Шрифт**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$ **Диалоговое окно Установка цвета**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$**Команда Найти (Меню Правка)**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$**Диалоговое окно Найти**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$**Команда Заменить command (Меню Правка)**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$**Диалоговое окно Найти и заменить**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$**Команда Повторить (Меню Правка)**

Данная команда позволяет повторить последнюю команду редактирования.

Другие способы вызова

Комбинация клавиш: F4

<<Новая страница>>

#$**Команда Очистить (Меню Правка)**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$**Команда Очистить все (Меню Правка)**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$**Следующая панель**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

#$**Предыдущая панель**

<< Содержание данной темы зависит от приложения. >>

<<Новая страница>>

Внесение изменений в документ

<< Содержание данной темы зависит от того, какие типы документов используются в приложении и какие изменения в них можно вносить из приложения. >>

<<Новая страница>>

#**Справочная информация отсутствует**

По данной области окна справочная информация отсутствует.

```
<<Новая страница>>
# Справочная информация отсутствует

По данному окну сообщения справочная информация отсутствует.

<< Если необходимо создать справочную информацию, связанную с каждым
окном сообщения, удалите значения AFX_HIDP_xxx из раздела [ALIAS] файла
.HPJ, и создайте раздел справки для каждого значения AFX_HIDP_xxx.
Например, AFX_HIDP_INVALID_FILENAME представляет собой идентификатор
справки для окна сообщения о недопустимом имени файла. >>
```

Для полной русификации данного файла необходимо изменить список его сносок в соответствии с приведенным ниже текстом:

```
# main_index
# menu_file
K Работа с файлами
$ Команды меню Файл
# menu_edit
# menu_view
# menu_window
# menu_help
# HID_FILE_NEW
K Работа с файлами
$ Команда меню Файл, Создать
# AFX_HIDD_NEWTYPEDLG
# HID_FILE_OPEN
K Работа с файлами
$ Команда меню Файл, Открыть
# AFX_HIDD_FILEOPEN
# HID_FILE_CLOSE
K Работа с файлами
$ Команда меню Файл, Закрыть
# HID_FILE_SAVE
K Работа с файлами
$ Команда меню Файл, Сохранить
# HID_FILE_SAVE_AS
K Работа с файлами
$ Команда меню Файл, Сохранить как
# HID_FILE_SEND_MAIL
# AFX_HIDD_FILESAVE
# HID_FILE_MRU_FILE1
```

```
K Работа с файлами
$ Команда меню Файл, 1, 2, 3, 4
# HID_APP_EXIT
K Выход из приложения
$ Команда меню Файл, Выход
# HID_EDIT_UNDO
# HID_EDIT_REDO
# HID_EDIT_CUT
# HID_EDIT_COPY
# HID_EDIT_PASTE
# HID_VIEW_TOOLBAR
K Панель инструментов
$ Команда меню Вид, Панель инструментов
# AFX_HIDW_TOOLBAR
K Панель инструментов
$ Панель инструментов
# HID_VIEW_STATUS_BAR
K Строка состояния
$ Команда меню Вид, Строка состояния
# AFX_HIDW_STATUS_BAR
K Строка состояния
$ Строка состояния
# HID_WINDOW_NEW
# HID_WINDOW_CASCADE
# HID_WINDOW_TILE
# HID_WINDOW_TILE_HORZ
# HID_WINDOW_TILE_VERT
# HID_WINDOW_ARRANGE
# HID_WINDOW_SPLIT
# HID_WINDOW_ALL
# HID_HELP_INDEX
# HID_HELP_USING
# HID_APP_ABOUT
# HID_CONTEXT_HELP
$ Команда Справка
# HID_HT_CAPTION
# Полосы прокрутки
# HID_SC_SIZE
# HID_SC_MOVE
# HID_SC_MINIMIZE
$ Команда меню Свернуть
# HID_SC_MAXIMIZE
```

```
# HID_SC_NEXTWINDOW
# HID_SC_PREVWINDOW
# HID_SC_CLOSE
# HID_SC_RESTORE
# HID_SC_TASKLIST
# HID_VIEW_RULER
$ Команда меню Вид, Линейка
# AFX_HIDD_FONT
$ Диалоговое окно Шрифт
# AFX_HIDD_COLOR
$ Диалоговое окно Установка цвета
# HID_EDIT_FIND
$ Команда меню Правка, Найти
# AFX_HIDD_FIND
$ Диалоговое окно Найти
# HID_EDIT_REPLACE
$ Команда меню Правка, заменить
# AFX_HIDD_REPLACE
$ Диалоговое окно Найти и заменить
# HID_EDIT_REPEAT
$ Команда меню Правка, Заменить
# HID_EDIT_CLEAR
$ Команда меню Правка, Очистить
# HID_EDIT_CLEAR_ALL
$ Команда меню Правка, Очистить все
# HID_NEXT_PANE
$ Команда Следующая панель
# HID_PREV_PANE
$ Команда предыдущая панель
# HIDR_DOC1TYPE
# HID_HT_NOWHERE
# Справка в окне сообщения
```

Аналогичные изменения должны быть внесены и в текст файла afxprint.rtf, текст которого приведен ниже:

```
<<Новая страница>>
#K $ Команда меню Печать (Меню Файл)

Данная команда меню используется для печати документа. Эта команда
вызывает диалоговое окно Печать AFX_HIDD_PRINT, в котором пользователь
может задать диапазон печатаемых страниц, число копий, принтер и другие
параметры печати.
```

Другие способы вызова

Панель инструментов: {bmc fileprnt.bmp}

Комбинация клавиш: CTRL+P

<<Новая страница>>

Диалоговое окно Печать

Диалоговое окно Печать содержит следующие группы элементов управления, позволяющих задать параметры печати документа:

Принтер

Позволяет выбрать принтер и установить его параметры. Принтер выбирается из раскрывающегося списка Имя. Для изменения параметров принтера нажмите кнопку Свойства.

Кнопка Свойства

Раскрывает диалоговое окно Свойства AFX_HIDD_PRINTSETUP, позволяющее установить параметры принтера.

Печатать

Определяет диапазон печатаемых страниц в документе:

Все	Печатает весь документ.
Выделенный фрагмент	Печатает выделенный фрагмент текста.
Страницы	Печатает страницы, определяемые текстовыми полями "с" и "по".

Копии

Определяет количество печатаемых копий страниц диапазона печати.

Разобрать

Установка этого флажка приводит к тому, что документ печатается в порядке возрастания номеров страниц, а не делает по несколько копий одной страницы.

<<Новая страница>>

Диалоговое окно Процесс печати

Диалоговое окно Процесс печати выводится приложением <<YourApp>> в процессе подготовки документа для печати. В нем указывается число обработанных страниц, имя используемого принтера и порт, через который он подключен.

Для прекращения печати нажмите кнопку Отмена (если успеете).

<<Новая страница>>

#K $ **Предварительный просмотр (Меню Файл)**

Данная команда меню используется для отображения активного документа в специальном окне, позволяющем оценить вид документа после его печати. При выборе данной команды активное окно замещается окном предварительного просмотра печати, в который выводятся одна или две страницы документа в том виде, в котором они будут напечатаны. В верхней части окна располагается панель инструментов предварительного просмотра печати AFX_HIDW_PREVIEW_BAR содержит кнопки, позволяющие одновременно просматривать одну или две страницы документа, перемещаться по страницам

документа, увеличивать или уменьшать масштаб отображения и инициировать процесс печати.

<<Новая страница>>

#K $ **Панель инструментов предварительного просмотра печати**

Панель инструментов предварительного просмотра печати содержит следующие кнопки:

Печать

Выводит диалоговое окно Печать для инициализации процесса печати.

Следующая

Предварительный просмотр следующей печатаемой страницы.

Предыдущая

Предварительный просмотр предыдущей печатаемой страницы.

Одна страница / Две страницы

Переход к одновременному просмотру указанного числа страниц.

Крупнее

Увеличивает масштаб отображения в окне.

Мельче

Уменьшает масштаб отображения в окне.

Закрыть

Закрывает окно предварительного просмотра печати и возвращается к отображению активного окна.

<<Новая страница>>

#K $ **Команда Настройка принтера (Меню Файл)**

Данная команда меню позволяет установить параметры используемого принтера. Она вызывает диалоговое окно Настройка принтера AFX_HIDD_PRINTSETUP.

<<Новая страница>>

Диалоговое окно Настройка принтера

Диалоговое окно Настройка принтера имеет следующие элементы управления:

Печать

Группа элементов управления Печать позволяет выбрать принтер, на котором будет распечатываться документ, и установить его параметры.

Ориентация

Переключатель Ориентация позволяет установить ориентацию листа при печати (Книжная или Альбомная).

Размер

Раскрывающийся список Размер группы Бумага позволяет выбрать формат листа, на который будет производиться печать.

Подача

Раскрывающийся список Подача группы Бумага позволяет установить способ подачи листов в принтере (Автоматическая подача, Ручная подача или С перфорацией).

Свойства

Кнопка Свойства группы Печать вызывает диалоговое окно Свойства, позволяющее установить параметры, специфичные для каждого принтера.

```
<<Новая страница>>
#$Команда Параметры страницы (Меню Файл)

<< Содержимое данного раздела зависит от приложения.>>
```

Список сносок данного файла должен иметь следующий вид:

```
# HID_FILE_PRINT
K Печать и предварительный просмотр
$ Команда меню Файл, Печать
# AFX_HIDD_PRINT
# AFX_HIDD_PRINTDLG
# HID_FILE_PRINT_PREVIEW
K Печать и предварительный просмотр
$ Команда меню Файл, Предварительный просмотр
# AFX_HIDW_PREVIEW_BAR
K Печать и предварительный просмотр
$ Панель инструментов предварительного просмотра печати
# HID_FILE_PRINT_SETUP
K Печать и предварительный просмотр
$ Команда меню Файл, Настройка принтера
# AFX_HIDD_PRINTSETUP
# HID_FILE_PAGE_SETUP
$ Команда меню Файл, Параметры страницы
```

Для полной русификации справочной системы приложения остается внести изменения в файлы с расширением .cnt и .hpj. В данном случае имеется только по одному файлу с каждым из расширений, имена которых совпадают с именем приложения. Файл HelpApp.cnt, расположенный в подкаталоге \hlp каталога приложения, должен содержать следующий текст:

```
:Base HelpApp.hlp
1 Меню
2 Меню Файл=menu_file
2 Меню Правка=menu_edit
2 Меню Вид=menu_view
2 Меню Окно=menu_window
2 Меню Справка=menu_help
1 <<Вставьте сюда заголовки разделов справки вашего приложения>>
2 <<Вставьте сюда переходы к отдельным разделам справки>>=main_index
```

В файле HelpApp.hpj, расположенном в том же каталоге, необходимо заменить строку `TITLE=HELPAPP Application Help` на строку

```
TITLE=Справка по приложению HELPAPP
```

После внесения этих изменений справочная система приложения будет полностью русифицирована, однако она будет содержать большое количество ненужной информации и указаний на то, какую информацию в нее следует вставить или удалить.

Чтобы удалить раздел справочной системы, необходимо удалить всю страницу, содержащую его описание (вместе с одним из символов разрыва страницы, расположенным до или после нее). После редактирования текст справочной системы не должен содержать строк-местодержателей (placeholders), т. е. строк, заключенных в угловые скобки << >>, и предназначенных для хранения указаний пользователю.

Кроме строк-местодержателей приложение содержит местодержатель <<YourApp>>, используемый в качестве имени текущего приложения. Этот местодержатель во всех исходных текстах справочной системы должен быть заменен на имя приложения.

Примечание

Остается непонятным, почему эта операция не выполняется при создании приложения мастером AppWizard. Наверное, это очень сложная операция для специалистов Microsoft. Поэтому она поручается создателю приложения. В данном случае эта операция не производится, чтобы текст данной справки мог быть скопирован в любое приложение.

Добавление новых тем

При добавлении новых тем не рекомендуется вносить их в файлы, подготовленные мастером AppWizard. Для этого лучше создать отдельные файлы. Каждый новый файл должен быть зарегистрирован в секции `[FILES]` файла проекта (с расширением .hpj).

Чтобы включить в справочную систему приложения разделы, связанные со спецификой данного приложения:

1. Откройте на редактирование приложение HelpApp.
2. Перейдите на вкладку **FileView** окна **Workspace**. Раскройте в нем папку \HelpApp Files, а в ней папку \Source Files.
3. Щелкните на значке **HelpApp.hpj**. Откроется окно редактирования файла проекта справки.
4. В конец раздела `[FILES]` файла HelpApp.hpj добавьте строку

   ```
   helpapp.rtf
   ```

5. Не закрывая Visual C++, вызовите MS Word.
6. Откройте файл afxcore.rtf и сохраните его под именем helpapp.rtf.
7. По методике, описанной в начале предыдущего раздела, установите режим отображения скрытого текста и раскройте панель сносок.
8. Используя форматирование разделов справки файла afxcore.rtf, поместите в файл helpapp.rtf следующий текст:

```
#K $ Команды меню Демо

Меню Демо содержит следующие команды:

ВызовHID_DEMO_DIALOG      Вызов диалогового окна

<<Новая страница>>
#K $ Команда Вызов (Меню Демо)

Данная команда меню используется для вызова диалогового окна,
демонстрирующего принципы программирования контекстной справки.

<<Новая страница>>
#Кнопка Демо (Диалоговое окно Контекстная справка)

Данная кнопка существует исключительно для того, чтобы можно было
вывести этот текст.

<<Новая страница>>
#Кнопка ОК (Диалоговое окно Контекстная справка)

Данная кнопка служит для выхода из диалогового окна Контекстная
справка.

<<Новая страница>>
#Кнопка Отмена (Диалоговое окно Контекстная справка)

Данная кнопка служит для выхода из диалогового окна Контекстная
справка.

<<Новая страница>>
#K $ Дополнительные возможности приложения

Данное приложение является чисто демонстрационным. Данный раздел
демонстрирует возможности создания командной справки. Для демонстрации
возможности создания контекстной справки используется диалоговое окно
Контекстная справкаHIDD_DEMO.

<<Новая страница>>
#Диалоговое окно Контекстная справка

Данное диалоговое окно служит для демонстрации возможностей
программирования контекстной справки.
```

9. Измените список сносок файла helpapp.rtf в соответствии со следующим текстом.

```
# menu_demo
  K Демонстрационное меню
  $ Команды меню Демо
# HID_DEMO_DIALOG
  K Демонстрационное меню
  $ Команда меню Демо, Вызов
# HIDD_CONTEXT
# HIDD_CONTEXT_OK
# HIDD_CONTEXT_CANCEL
# HID_NEW_HELP
  K Дополнительно
  $ Новая справка
# HIDD_DEMO
```

10. Нажмите клавишу <F5> и запустите на исполнение приложение HelpApp.

11. Выберите команду **Демо**, **Вызов**. Появится диалоговое окно **Контекстная справка**.

12. Нажмите кнопку контекстной справки (кнопка со знаком вопроса, расположенная в заголовке диалогового окна). Курсор мыши примет вид курсора контекстной справки.

13. Нажмите кнопку **Демо**. Появится контекстная справка, изображенная на рис. 13.19.

14. Закройте диалоговое окно **Контекстная справка** и выберите команду меню **?**, **Новая справка**. Появится окно **Справка по приложению HELPAPP**, изображенное на рис. 13.20.

15. Закройте окно справки и закройте приложение.

Обычно для введения новых разделов в справочную систему достаточно воспользоваться копией уже имеющихся разделов. Однако, если у вас нет желания выискивать соответствующим образом отформатированные фрагменты текста, вы можете самостоятельно выполнить соответствующее форматирование в MS Word.

❐ Чтобы вставить сноску:

- Введите текст, к которому будет относиться сноска, и установите текстовый курсор на то место, где должна появиться сноска
- Выберите команду **Вставка**, **Сноска...**

- В появившемся диалоговом окне **Сноски**, изображенном на рис. 13.21, введите в текстовое поле требуемый символ сноски (при этом переключатель **Нумерация** автоматически установится в положение **другая:**) и нажмите кнопку **OK**
- В панели **Сноски** введите текст данной сноски (зависит от типа сноски)
- Выделите введенный символ сноски без стоящей за ним точки и выберите из раскрывающегося списка **Стиль** панели инструментов **Форматирование** стиль **footnote reference**
- Повторите п. 5 для знака ссылки в основном тексте файла справки

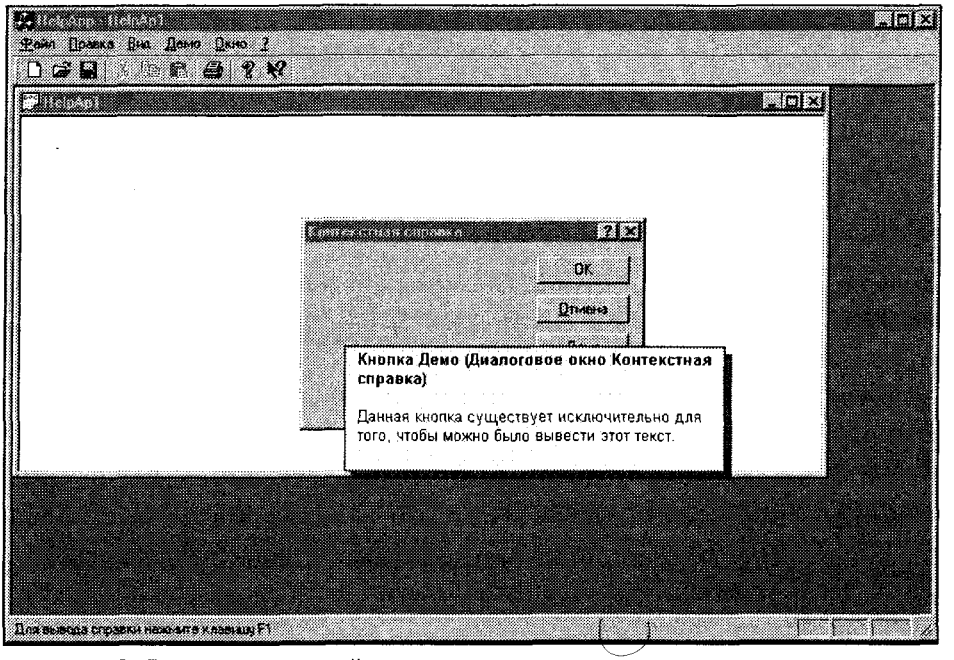


Рис. 13.19. Вывод контекстной справки

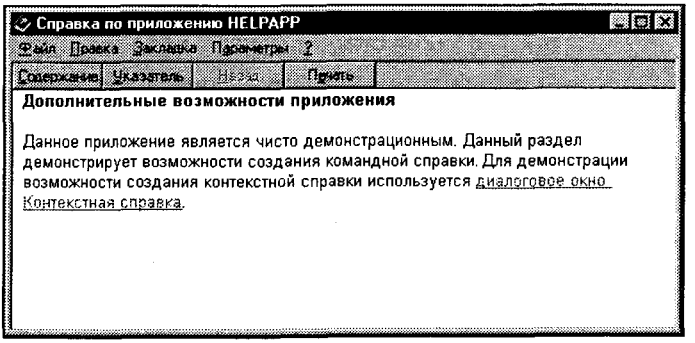


Рис. 13.20. Окно **Справка по приложению HELPAPP**

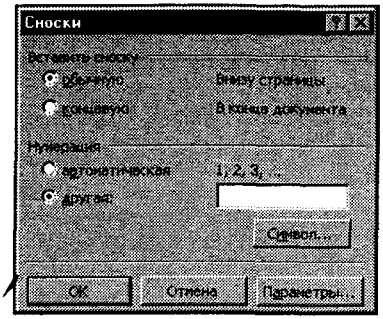


Рис. 13.21. Диалоговое окно **Сноски**

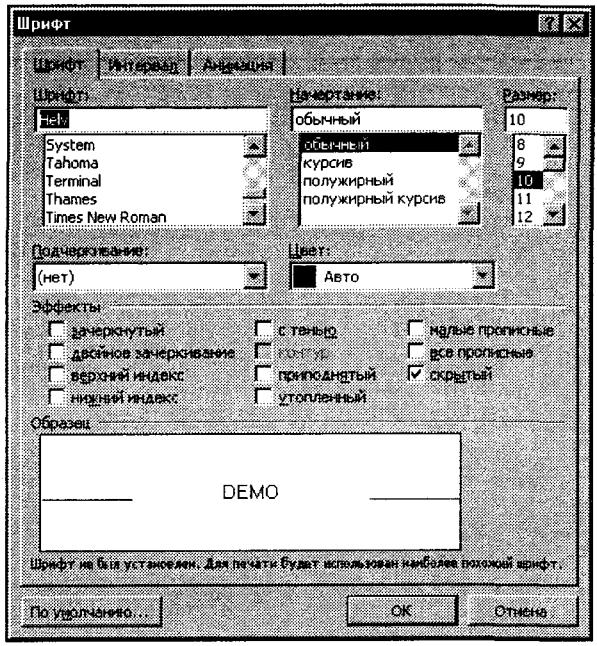


Рис. 13.22. Диалоговое окно **Шрифт**

- Чтобы дважды подчеркнуть текст, нажмите комбинацию клавиш <Ctrl>+<Shift>+<D>
- Чтобы сделать текст скрытым:
  - Выделите его и выберите команду **Формат**, **Шрифт...**
  - В появившемся диалоговом окне **Шрифт**, изображенном на рис. 13.22, установите флажок **скрытый**

Внесение изменений в оглавление справочной системы

Для внесения изменений в оглавление справочной системы лучше воспользоваться приложением Help Workshop.

Чтобы внести изменения в оглавление справочной системы с использованием приложения Help Workshop:

1. В панели задач выберите команду **Пуск**, **Программы**, **Microsoft Visual Studio 6.0**, **Microsoft Visual Studio 6.0 Tools**, **Help Workshop**. Откроется приложение Help Workshop.
2. Выберите команду **File**, **Open...** или нажмите кнопку **Open** панели инструментов.

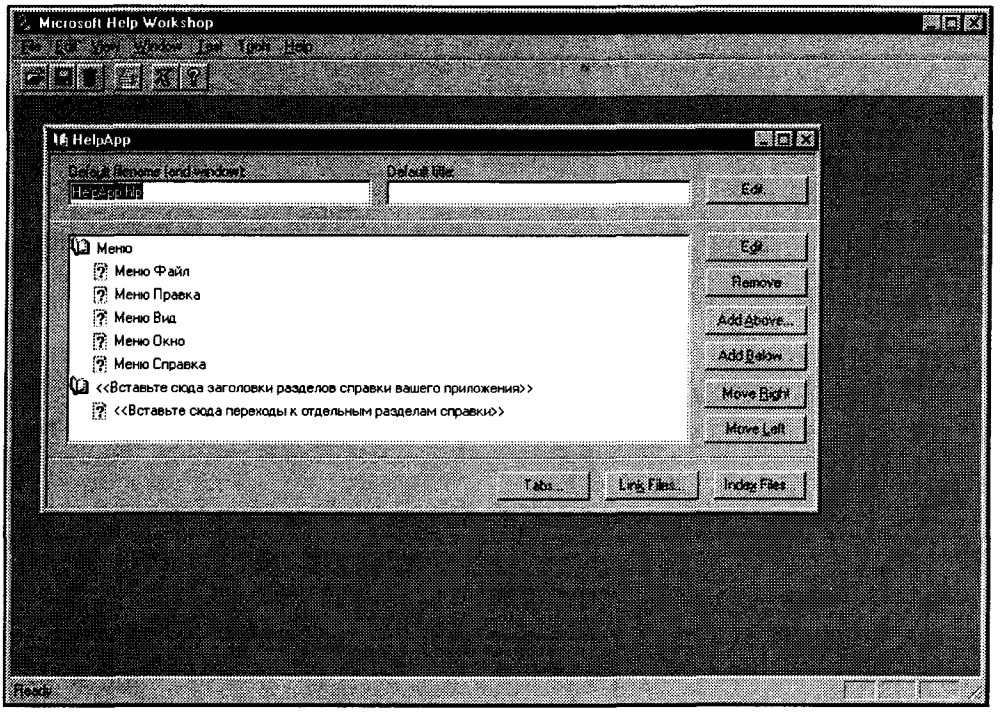


Рис. 13.23. Окно редактирования файла контекста

3. В появившемся диалоговом окне раскройте папку \hlp в каталоге вашего приложения, в раскрывающемся списке **Тип файлов:** выберите опцию **Help Contents (*.cnt)** и дважды щелкните на имени файла HelpApp.

4. Откроется окно редактирования файла контекста, изображенное на рис. 13.23.

5. Выделите строку **Меню Окно** и нажмите кнопку **Add Above...** (Вставить выше) (или выделите строку **Меню Вид** и нажмите кнопку **Add Below...** (Вставить ниже)). Появится диалоговое окно **Edit Contents Tab Entry** (Добавление новых пунктов в оглавление справочной системы), изображенное на рис. 13.24.

6. Введите в текстовое поле **Title:** заголовок "Меню Демо", а в текстовое поле **Topic ID:** — идентификатор справки "`menu_demo`". Нажмите кнопку **OK**.

7. Выделите строку **Меню Справка** и нажмите кнопку **Add Below...**.

8. В диалоговом окне **Edit Contents Tab Entry** установите переключатель в положение **Heading** (Заголовок) и введите в текстовое поле **Title:** заголовок "Дополнительные возможности" (остальные поля при таком положении переключателя становятся недоступными). Нажмите кнопку **OK**.

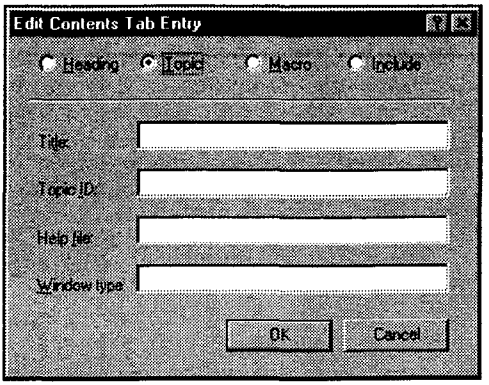


Рис. 13.24. Диалоговое окно **Edit Contents Tab Entry**

9. Переместитесь на одну строку вниз, нажмите кнопку **Add Below...** и в появившемся диалоговом окне **Edit Contents Tab Entry** введите в текстовое поле **Title:** заголовок "Новая справка", а в текстовое поле **Topic ID:** — идентификатор справки "HID_NEW_HELP". Нажмите кнопку **OK**.
10. Переместитесь на одну строку вниз и нажмите кнопку **Remove**. Заголовок со строкой-местодержателем исчезнет.
11. Повторите эту операцию еще раз.
12. Закройте окно редактирования файла контекста. Появится окно сообщения **Microsoft Help Workshop** с вопросом о необходимости сохранения внесенных изменений.
13. Нажмите кнопку **OK** в окне сообщения **Microsoft Help Workshop** и выйдите из приложения Help Workshop.
14. Вызовите Visual C++ и запустите на исполнение приложение HelpApp.
15. Убедитесь, что все внесенные изменения нашли отражение в справочной системе.

Диалоговое окно *О программе*

При создании приложения с использованием мастера AppWizard на втором шаге создания диалогового приложения по умолчанию установлен флажок **About Box** (диалоговое окно **О программе**), позволяющее включить в приложение возможность вывода диалогового окна, содержащего основные сведения о разработчике программы, ее версии и правах собственности на данную программу. Окно мастера AppWizard на втором шаге создания диалогового приложения приведено на рис. 13.25.

При создании многооконного или однооконного приложения вопрос о необходимости включения подобного окна в приложение даже не ставится: оно включается в приложение автоматически.

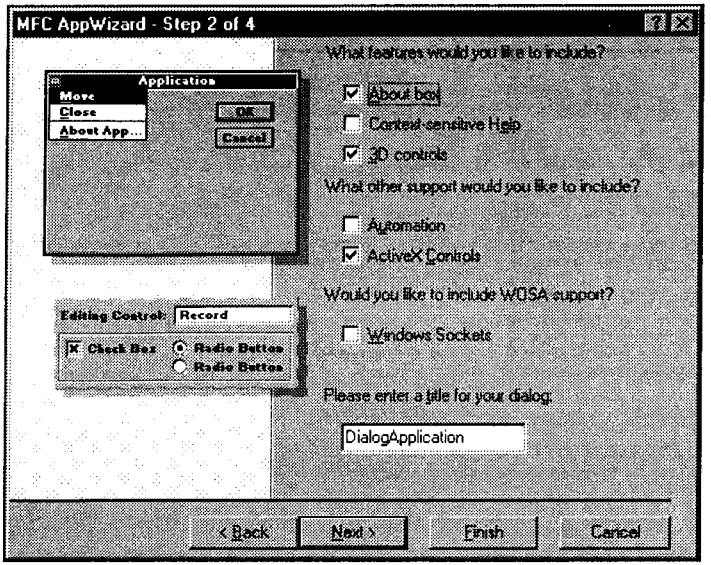


Рис. 13.25. Диалоговое окно **MFC AppWizard — Step 2 of 4**

По умолчанию диалоговое окно **About** имеет вид, приведенный на рис. 13.26.

После русификации файла ресурсов приложения HelpApp его диалоговое окно **О программе** приняло вид, приведенный на рис. 13.27.

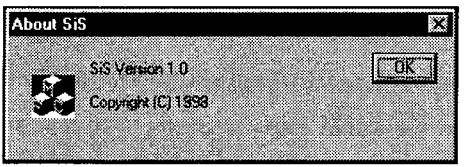


Рис. 13.26. Диалоговое окно **About**

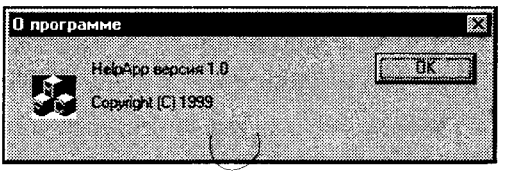


Рис. 13.27. Диалоговое окно **О программе**

Конечно, в его настоящем виде диалоговое окно **О программе** несет мало полезной информации, однако, как и всякое другое диалоговое окно, его можно отредактировать в соответствующем редакторе ресурса и, с использованием статического текста, внести в него всю информацию, которую создатель программы желает о себе сообщить. При этом в него могут быть помещены любые битовые образы вместо используемого по умолчанию значка библиотеки MFC.

Отладка приложения

Отладка является существенной частью процесса программирования, особенно в среде Windows, где приложения управляются сообщениями и невозможно заранее определить, в какой последовательности будут выполняться его отдельные фрагменты.

В данной главе рассмотрены компоненты интерфейса пользователя Visual C++, связанные с процессом отладки. Остальные компоненты интерфейса рассматриваются в Приложении D.

Средства отладки, предоставляемые интерфейсом пользователя

Очевидно, что при отладке программ ключевым понятием является термин *точка останова*. Под точкой останова понимается место, в котором требуется приостановить выполнение программы и предоставить пользователю возможность воспользоваться средствами отладки, предоставляемыми средой программирования Visual C++. Получив необходимую информацию, пользователь может:

- Возобновить выполнение программы до следующей точки останова или до конца программы, если при ее дальнейшем выполнении не встретятся точки останова. Для этого можно нажать клавишу <F5> или выбрать команду меню **Debug**, **Go**. (Меню **Debug**, изображенное на рис. 14.1, появляется в панели меню после запуска приложения в режиме отладки.)
- Перезапустить исполняемое приложение. Для этого нажать комбинацию клавиш <Ctrl>+<Shift>+<F5> или выбрать команду меню **Debug**, **Restart**
- Завершить работу с приложением. Для этого нажать комбинацию клавиш <Shift>+<F5> или выбрать команду меню **Debug**, **Stop Debugging**

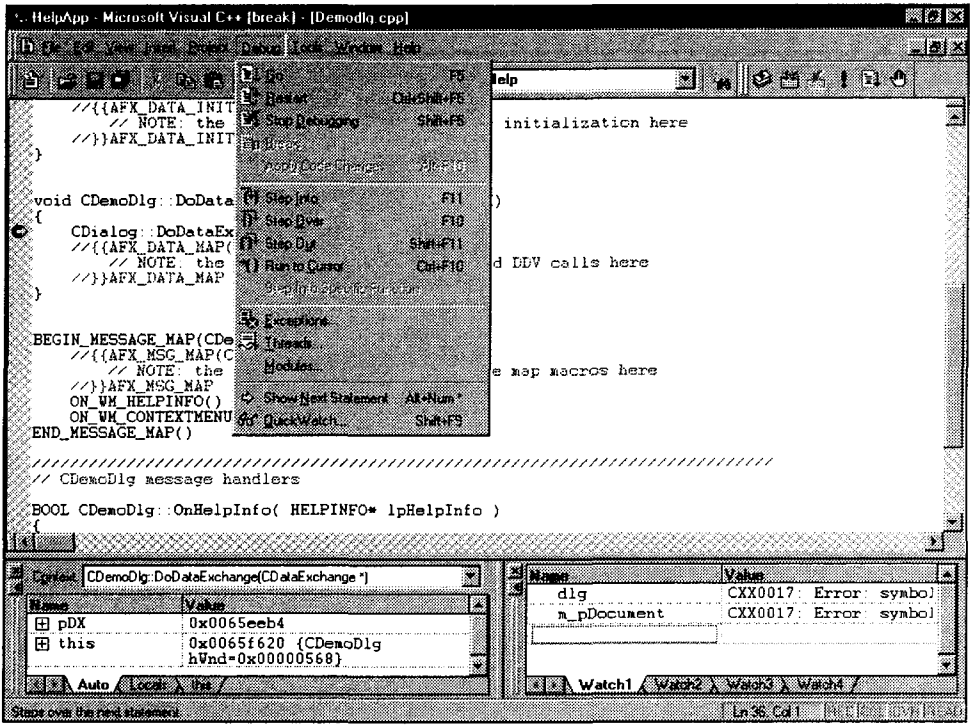


Рис. 14.1. Раскрытое меню **Debug**

- Выполнить один оператор программы и снова остановиться. Для этого нажать клавишу <F10> или выбрать команду меню **Debug**, **Step Over**
- Если текущий оператор является функцией, доступной для отладки, пользователь может войти в эту функцию и остановиться на первом ее операторе (заголовке). Для этого надо нажать клавишу <F11> или выбрать команду меню **Debug**, **Step Into**. Если текущий оператор не является функцией, данная команда работает аналогично предыдущей. Если текст данной функции не содержится в каталогах, указанных пользователем при настройке системы, система выводит диалоговое окно, предлагая указать путь к данному файлу. Если функция содержится в системной библиотеке и недоступна для отладки, система выводит окно сообщения с соответствующим текстом
- При нажатии клавиши <Shift>+<F11> или выборе команды меню **Debug**, **Step Out** приложение завершает работу данной функции и останавливается на следующем после вызова данной функции операторе
- Пользователь может продолжить выполнение программы и остановиться на операторе, на котором в данное время расположен текстовый курсор. Для этого он должен нажать комбинацию клавиш <Ctrl>+<F10> или выбрать команду меню **Debug**, **Run to Cursor**

Точки останова

Как следует из вышесказанного, одним из способов установки точки останова является выбор команды меню **Debug, Run to Cursor**. Это наиболее простой и распространенный метод установки точки останова. Однако этот метод имеет свои недостатки:

- Таким образом может быть установлена только одна точка останова
- Данным методом невозможно воспользоваться, если приложение находится в состоянии ожидания команды пользователя, а это состояние очень удобно для установки точек останова в процессе выполнения программы
- Если данная команда используется для запуска приложения на исполнение, то ошибки в процессе компиляции отвлекают пользователя, и он может после их устранения автоматически запустить приложение нормальным образом, не установив точку останова, и после этого долго удивляться, почему приложение не останавливается в данной точке

Более надежным способом установки точек останова является использование клавиши <F9> или соответствующей ей кнопки **Insert/Remove Breakpoint** панели инструментов **Build** или панели инструментов **Build MiniBar**.

Чтобы установить или удалить точку останова, достаточно установить текстовый курсор на соответствующую строку текста программы и нажать клавишу <F9>. Точка останова индицируется красной точкой, расположенной слева от строки текста, перед выполнением которой приложение будет остановлено.

Для редактирования точек останова выберите команду **Edit, Breakpoints...**. Появится диалоговое окно **Breakpoints**, изображенное на рис. 14.2.

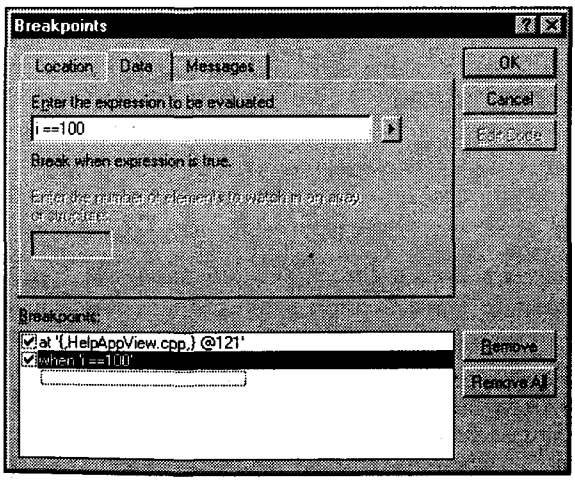


Рис. 14.2. Диалоговое окно **Breakpoints**

В данном окне содержатся сведения о двух точках останова, одна из которых является безусловной и расположена на 121 строке файла HelpAppView.cpp, а вторая — условной и расположена в той точке, где переменная `i` принимает значение 100. К сожалению, отсутствует возможность объединения этих типов точек останова в один (прервать выполнение приложения, если в данной точке программы значение некоторого выражения удовлетворяет определенным условиям). В настоящем виде использование условной точки останова оправдано в крайне ограниченном числе случаев, не говоря уже о том, что ее использование существенно замедляет выполнение программы.

Использование окна списка **Breakpoints** и кнопки **Edit Code** позволяет быстро перейти к любой точке останова в любом файле проекта. Кнопки **Remove** и **Remove All** позволяют быстро и надежно убрать все точки останова после устранения возникшей проблемы.

Анализ значений переменных

Обычно точки останова ставятся, чтобы обеспечить программисту возможность анализа значений переменных в данной точке для оценки правильности выполнения программы и обнаружения источников ошибок, возникающих в процессе ее выполнения.

Среда программирования Visual C++ в процессе отладки приложения показана на рис. 14.3. Активная точка останова (на которой было прервано выполнение приложения) отмечается желтой стрелкой, расположенной поверх красной точки. В нижней части экрана располагаются окна **Watch** и **Variables**. Они имеют практически одинаковую структуру и различаются тем, что окно **Variables** заполняет Visual C++ исходя из одному ему ведомого критерия актуальности значений отдельных переменных в данной точке программы.

Как правило, в окне **Variables** отображаются значения переменных, используемых в текущем и предыдущем операторах программы. Если значение переменной было изменено с момента прошлого его отображения в данном окне, оно отображается красным цветом. Если около имени переменной стоит знак плюс, то эта переменная представляет собой объект структуры или класса. Чтобы просмотреть значения переменных, достаточно щелкнуть левой кнопкой мыши на этом крестике.

Содержимое окна **Watch** определяется пользователем и сохраняется даже после закрытия приложения. Чтобы поместить идентификатор переменной в поле **Name** окна **Watch**, достаточно щелкнуть на нем левой кнопкой мыши и ввести это имя в расширившееся поле или выделить его в тексте программы и перетащить в окно. В последнем случае в окне выделяется последняя занятая строка, за которой и вставляется имя выделенной переменной после отпускания левой кнопки мыши.

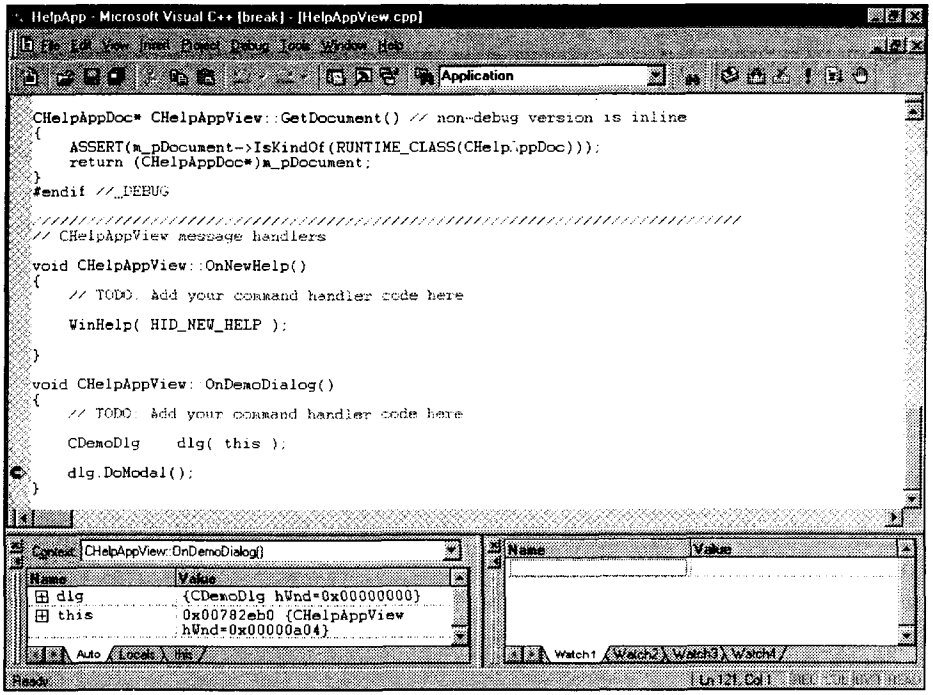


Рис. 14.3. Среда Visual C++ в процессе отладки приложения

Другим способом помещения идентификатора переменной в окно **Watch** является использование команды меню **Debug**, **QuickWatch**, комбинации клавиш <Shift>+<F9> или нажатие кнопки **QuickWatch** панели инструментов **Debug**. Все эти действия приведут к появлению диалогового окна **QuickWatch**, изображенного на рис. 14.4. Нажатие кнопки **Add Watch** в данном окне добавит соответствующую переменную в окно **Watch**.

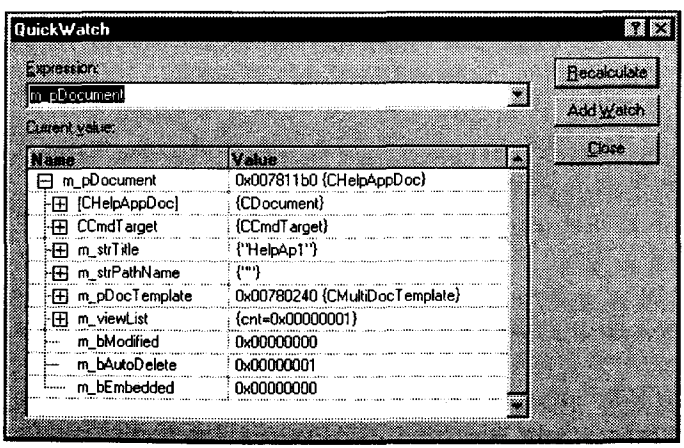


Рис. 14.4. Диалоговое окно **QuickWatch**

Окно **Variables** содержит раскрывающийся список **Context:**, в текстовом поле которого отображается полное имя функции, значения переменных которой оцениваются в настоящее время. Данный список содержит имена всех функций, вызовы которых привели программу в ее текущее состояние. В корне этого списка обычно находятся системные функции, не допускающие отладки.

Если в вашем приложении используется пользовательская функция для вывода информации обо всех ошибках приложения, то имеет смысл поместить в ней точку останова. В этом случае, при возникновении любой ошибки приложение будет останавливаться, оставаясь в рабочем состоянии. Последовательный просмотр функций в раскрывающемся списке **Context:**, как правило, позволяет быстро установить конкретную причину возникновения ошибки. Анализ конкретной причины возникновения ошибки позволит найти и нарушение в логике программы.

Как уже говорилось, в окне **Variables**, как правило, отображаются только переменные, используемые в данном и в предыдущем операторах программы. Кроме того, это окно может иметь полосу прокрутки, позволяющую просмотреть значения интересующей вас переменной. Следует учесть, что в большинстве случаев после выполнения очередного оператора программы список прокручивается в начало, и для просмотра значения переменной его нужно снова прокручивать. Поэтому во многих случаях для оценки значения переменной достаточно поместить на нее указатель мыши. При этом появляется окно **Data Tip**, в котором содержится ее значение. Если оцениваемая переменная является членом структуры или класса, то можно выделить эту переменную вместе с именем объекта ее структуры или класса и воспользоваться окном **Data Tip**. Однако этот метод срабатывает не во всех случаях.

Настройка уровня предупреждений транслятора

При отладке приложения часто приходится рассматривать возможность влияния на программу таких факторов, которые обычно не рассматриваются, и вывод которых в процессе трансляции считается излишним.

Чтобы повысить уровень выводимых предупреждений транслятора:

1. Выберите команду **Project**, **Settings** (Проект, Установки). Раскроется диалоговое окно **Project Settings**, изображенное на рис. 14.5.
2. В раскрывающемся списке **Settings For:** (Установки для) выберите **Win32 Debug**.
3. Раскройте панель **C/C++** и выберите в раскрывающемся списке **Warning level:** (Уровень предупреждений транслятора) **Level 4**, как это показано на рис. 14.6.

При создании окончательной версии приложения будет использоваться установленный по умолчанию третий уровень предупреждений транслятора.

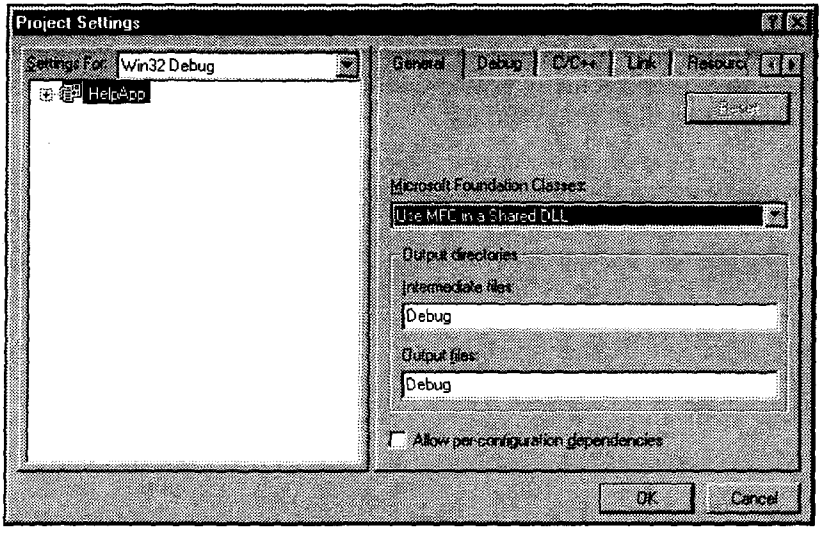


Рис. 14.5. Диалоговое окно **Project Settings**

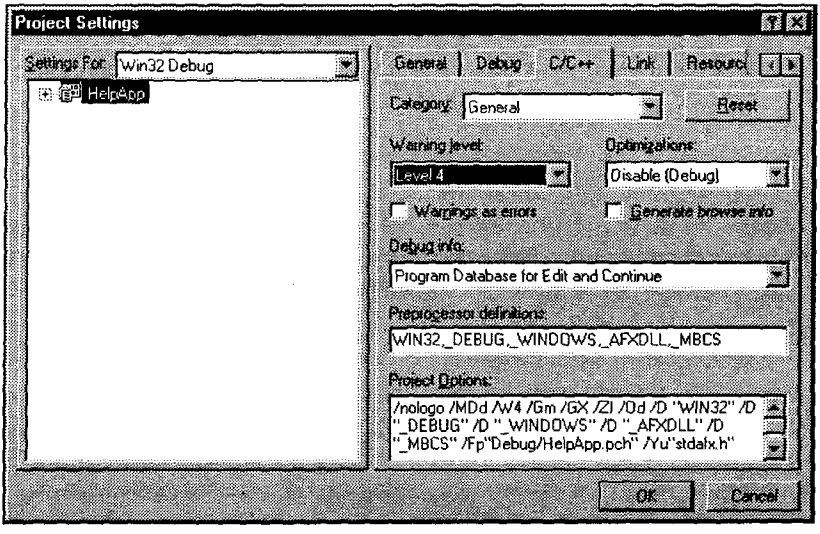


Рис. 14.6. Диалоговое окно **Project Settings**, вкладка **C/C++**

Программные средства отладки

Программируя приложение, в него необходимо включить определенные операторы, используемые только во время отладки. Это замедляет выполнение программы и чревато выдачей пользователю сообщений, понятных

только программисту. Поэтому при создании окончательной версии приложения эти операторы следует удалить. Однако для качественной отладки приложения количество отладочных операторов должно быть велико и они должны находиться в большинстве функций приложения.

Ручное удаление этих операторов представляет собой достаточно сложную задачу и не гарантирует удаление всех отладочных операторов. Кроме того, никто не может поручиться, что вместе с ними не будут удалены и рабочие операторы, а окончательную версию не придется отлаживать заново. Например, при внесении в нее изменений "по просьбам трудящихся". Поэтому в Visual C++ предусмотрено два режима трансляции приложения Debug (отладочный) и Release (окончательный) и созданы специальные функции и макросы, работающие только в отладочном режиме и игнорируемые в окончательном приложении.

В отладочной версии приложения Visual C++ с помощью директивы `#define` определяется константа `_DEBUG`, используемая в директивах условной компиляции и в макросах отладки для исключения отладочной информации в окончательной версии программы.

Макросы *ASSERT* и *TRACE*

Макрос `ASSERT` (подтверждение) используется для проверки некоторых логических условий, которые должны выполняться в данной точке программы. Предположим, что в некоторую функцию приложения передается указатель на некоторый объект `lpObject`. Предполагается, что этот указатель имеет ненулевое значение, однако для предотвращения возможных ошибок это следует проверить. С этой целью перед использованием данного указателя следует вызвать макрос `ASSERT`, как показано ниже:

```
ASSERT( lpObject)
```

Всякий раз, когда логическое выражение, переданное в качестве аргумента макросу `ASSERT`, будет принимать значение `FALSE`, выполнение приложения будет останавливаться и на экран будет выводиться окно сообщения, аналогичное приведенному на рис. 14.7. В данном окне указывается имя файла и номер строки, в которой произошла ошибка. Нажатие кнопки **Повтор** позволяет перейти в текст программы для ее отладки. Причем текущая точка останова устанавливается на строку соответствующего макроса `ASSERT`.

Макрос `TRACE` служит для вывода диагностических сообщений. Программист обычно хорошо знает, где в приложении может возникнуть ошибка. Например, многие функции возвращают значение, говорящее о том, успешно ли завершилась их работа и, возможно, являющееся кодом возникшей ошибки. Это значение можно игнорировать, считая, что ничего плохого с этим приложением случиться не может, можно прекратить выполнение функции, в которой возникла ошибка, а можно и послать при этом сообщение пользователю.

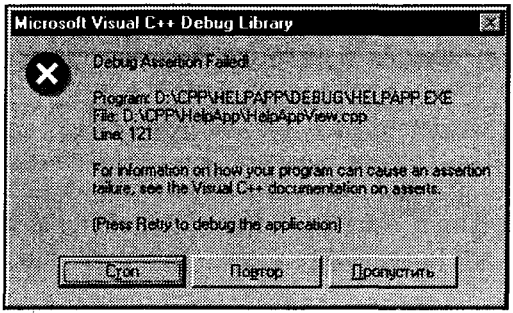


Рис. 14.7. Окно сообщения *об ошибке*

Для посылки сообщения могут быть использованы и обычные функции вывода, однако они будут работать и в окончательной версии, что нежелательно в случае простых информационных сообщений или возникновения ошибок, позволяющих сравнительно безболезненно продолжить выполнение приложения.

Для подобного вывода диагностической информации следует использовать макрос `TRACE`, под которым понимается целое семейство макросов, включающее в себя макросы `TRACE`, `TRACE0`, `TRACE1`, `TRACE2` и `TRACE3`. Число в имени макроса указывает на количество параметрических аргументов в данном макросе. Под параметрическим аргументом понимается идентификатор переменной, значение которой должно быть преобразовано в текстовую строку в соответствии с указанным форматом. Синтаксис макроса `TRACE` аналогичен синтаксису функции `printf`. Все макросы `TRACE` посылают свои сообщения в поток `afxDump`.

Макрос `TRACE` позволяет выводить сообщения с любым числом параметрических аргументов. Макросы `TRACE0`, `TRACE1`, `TRACE2` и `TRACE3` созданы исключительно с целью экономии места в сегменте данных. Ниже приведен пример использования макроса `TRACE0` в функции `CMainFrame::OnCreate`.

```
if (!m_wndStatusBar.Create(this) ||
  !m_wndStatusBar.SetIndicators(indicators,
    sizeof(indicators)/sizeof(UINT)))
{
  TRACE0("Failed to create status bar\n");
  return -1;      // fail to create
}
```

Следующий пример демонстрирует вывод сообщения о возникновении ошибки в файле:

```
TRACE2("Ошибка номер %d в файле %s\n", nError, szFileName);
```

В окончательной версии приложения, в которой константа `_DEBUG` не определена, макросы `ASSERT` и `TRACE` не выполняют никаких действий. Это по-

зволяет оставлять их в тексте программы без опасения, что они замедлят выполнение окончательной версии приложения или будут выдавать конечному пользователю непонятные сообщения.

С другой стороны, такая практика затрудняет отладку окончательной версии приложения, которая зачастую работает совсем не так, как отладочная. Здесь нельзя посоветовать ничего другого, кроме вставки в эту версию условных операторов для проверки определенных условий. Данные операторы в дальнейшем не удаляются, поскольку это может привести к возникновению устраненной проблемы.

Отладочные функции

Пользователь может создавать собственные отладочные функции, которые будут исключаться из окончательной версии приложения. Примером такой функции являются функции `CObject::AssertValid` и `CObject::Dump`, наследуемые практически всеми классами библиотеки MFC от класса `CObject`.

Функция `AssertValid` служит для проверки допустимости значений данного объекта. В случае возникновения ошибки она вызывает исключение. Функция `Dump` вызывается приложением при возникновении ошибки для передачи содержимого пользовательского объекта в объект класса `CDumpContext`. Данные функции имеют следующий формат:

```
virtual void AssertValid() const;
virtual void Dump(CDumpContext& dc) const;
```

Ключевое слово `virtual` говорит о том, что данный метод должен быть перегружен в производных классах, а ключевое слово `const` указывает, что данная функция не будет модифицировать состояние объекта.

Подобно отладочным макросам, функции `AssertValid` и `Dump` будут отсутствовать в окончательной версии приложения. Для этого объявление данных функций в заголовочном файле заключается в операторы условной трансляции, как показано ниже:

```
#ifdef _DEBUG
  virtual void AssertValid() const;
  virtual void Dump(CDumpContext& dc) const;
#endif
```

По умолчанию данные функции просто вызывают методы базового класса. Однако в файл реализации каждого класса, производного от класса `CObject`, должна быть включена реализация этих функций, заключенная в операторы условной трансляции. Как правило, эту задачу выполняет мастер AppWizard. Ниже приведен соответствующий пример для файла реализации класса `CMainFrame`:

```
////////////////////////////////////////////////////////////////////////
// CMainFrame diagnostics

#ifdef _DEBUG
void CMainFrame::AssertValid() const
{
  CMDIFrameWnd::AssertValid();
}

void CMainFrame::Dump(CDumpContext& dc) const
{
  CMDIFrameWnd::Dump(dc);
}

#endif //_DEBUG
```

Примеры перегрузки этих функций приведены в их описаниях в Приложении В.

Устранение утечки памяти

Утечку памяти можно считать одной из наиболее распространенных и трудно локализуемых ошибок. Небольшие утечки памяти могут не проявлять себя до тех пор, пока приложение не будет запущено на машине с малым объемом оперативной памяти или параллельно с большим количеством других задач.

Поскольку программисты обычно работают на машинах с большим объемом оперативной памяти и при проверке функционирования своего приложения стараются не запускать других приложений (кроме среды программирования), то эти ошибки могут остаться незамеченными и перейти в окончательную версию. Последствия утечек памяти могут быть достаточно серьезными и приводить даже к краху операционной системы с необходимостью ее переинсталляции.

Основные причины утечек памяти

В принципе, все утечки памяти имеют одну причину: где-то в приложении под некоторый объект выделяется область оперативной памяти, которая не освобождается после использования данного объекта. Другой вопрос, что эта единая причина имеет несколько конкретных проявлений, на которые программисту следует обратить внимание.

Чтобы проиллюстрировать вышесказанное, приведем основные причины утечки памяти в приложении:

1. Указатель на объект создан оператором `new` и не освобожден оператором `delete`.

2. Указатель на объект создан оператором `new[]`, а освобожден оператором `delete`. (Необходимо использовать оператор `delete[]`.)
3. Переменной, хранящей указатель на некоторый объект, присвоено новое значение, а объект не освобожден.
4. При вызове исключения область памяти, выделенная из кучи переменным данной функции, не освобождена.

Эти причины имеют свои разновидности, связанные с работой с классами. Например, разновидностью второго случая является использование в классе переменной, являющейся указателем на объект, при отсутствии в классе конструктора копирования, переопределенного оператора присваивания или деструктора. Ниже приведен листинг класса, в котором содержатся эти ошибки, и программы, вызывающей на их основе утечки памяти:

```
class SomeClass
{
  public:
    int*  p;
    SomeClass()  { p = new int[10];};
};
SomeClass SomeFunction()
{
  SomeClass  A;
  SomeClass  B;
  B = A;
  return  B;
}
void OtherFunction()
{
  SomeClass  A = SomeFunction();
}
```

При вызове функции `OtherFunction` вызывается функция `SomeFunction`, в которой создаются два объекта класса `SomeClass`, в каждом из них создаваемой переменной `p` присваивается значение указателя на некоторую область памяти, отведенную для хранения десяти целых чисел.

Поскольку в данном классе отсутствует перегруженный оператор присваивания, то операция присваивания одного объекта данного класса другому трактуется как побитовое присваивание значений соответствующих переменных одного объекта переменным другого объекта. При этом переменные `p` обоих классов будут указывать на одну и ту же область памяти, зарезервированную для объекта `A`. Указатель на другую область памяти будет потерян.

Оператор `return` вызывает конструктор копирования класса, который по умолчанию выполняет операцию побитового копирования переменных ис-

ходного объекта класса в создаваемый объект. Этот же конструктор используется и при создании объекта A в функции OtherFunction.

Включение в данный класс деструктора, уничтожающего указатель на целочисленный массив, только осложнит ситуацию и приведет к появлению явных ошибок, поскольку при закрытии функции SomeFunction два деструктора попытаются освободить одну и ту же область памяти, а если эта ошибка не приведет к остановке программы, при инициализации объекта A в функции OtherFunction будет использован некорректный указатель на область памяти.

Для устранения этих ошибок класс SomeClass должен быть переписан следующим образом:

```
class SomeClass
{
  public:
    int*  p;
    SomeClass()         { p = new int[10];};
    SomeClass( SomeClass&);
    ~SomeClass()        { if( p) delete p; };

    void  operator =( SomeClass&);
};

SomeClass:: SomeClass( SomeClass& Input)
{
  p = new int[10];

  for(int i=0; i < 10; i++)
    p[i] = Input.p[i];
}

void SomeClass:: operator =( SomeClass& Input)
{
  if( this == &Input) return;

  delete p;
  p = Input.p;
  Input.p = NULL;
}
```

В деструкторе данного класса использован оператор delete, хотя по теории необходимо использовать оператор delete[]. Различия в этих операторах при уничтожении массивов трудно уловимы и в некоторых случаях действительно необходимо использовать оператор delete[], но в других случаях его использование приводит к возникновению ошибок. Правильный выбор оператора определяется "методом научного тыка".

Проверка значения указателя перед его уничтожением является простой формальностью, поскольку освобождение нулевого указателя не приводит ни к каким действиям, однако использование данного оператора является своего рода "правилом хорошего тона".

Проверка значения указателя перед его уничтожением может потребоваться, если этот указатель содержит адрес объекта класса, имеющего непустой деструктор, который может быть создан, а может и не быть создан в процессе работы с исходным классом. Вызов деструктора для объекта класса, который не был создан, может привести к непредсказуемым последствиям. Поэтому в конструкторе пользовательского класса необходимо присвоить нулевое значение всем указателям на содержащиеся в нем объекты классов, а в деструкторе проверять значение указателя на объект перед его уничтожением. Если в процессе работы с пользовательским классом уничтожается указатель на один из содержащихся в нем объектов, то данному указателю следует присвоить нулевое значение.

В копирующем конструкторе данного класса создается указатель на новую область памяти, в который копируется содержимое массива исходного объекта класса. Это позволяет впоследствии использовать оба объекта класса. Для разнообразия оператор присваивания выполнен по другому принципу. После него нельзя использовать исходный объект класса. Данный метод обычно используется совместно с методом, примененным в копирующем конструкторе, если в классе содержится индикатор временного объекта. Использование этого индикатора позволяет существенно сократить время выполнения промежуточных операций копирования, если в данном классе имеются перегруженные операторы арифметических действий, а сами объекты имеют большой размер.

Условный оператор, расположенный в начале функции оператора присваивания, имеет очень большое значение, особенно при используемом методе реализации данного оператора. Он обеспечивает возможность корректной обработки операции присваивания объекта самому себе.

Для предотвращения возможности возникновения утечек памяти:

- Старайтесь не использовать память в куче (не пользоваться оператором `new`), если размещаемый в ней объект используется только в данной функции
- Если использование памяти из кучи неизбежно, создайте класс для работы с этой памятью, в который включите копирующий конструктор, деструктор, освобождающий всю используемую данным классом память, и оператор присваивания для основных типов данных, которые будут использоваться с данным классом
- Если функция, запрашивающая память, возвращает значение, позволяющее получить доступ к данной памяти, то это значение не должно быть ссылкой, поскольку освободить память по ссылке невозможно

- Никогда не изменяйте значения указателя на область памяти, пока эта область не будет освобождена
- Не используйте операцию приращения для указателя, полученного при помощи оператора `new`
- Используйте для хранения информации классы коллекций, описанные в главе 11

См. также главу 11

Отладочные версии операторов *new* и *delete*

Как говорилось выше, утечка памяти представляет собой довольно коварную ошибку, которая может остаться невыявленной после тщательного тестирования приложения (не говоря уже о "щадящем" тестировании, которое устраивают многие программисты для своего детища). Поэтому особенно ценна возможность автоматического обнаружения утечек памяти, предоставляемая библиотекой MFC. При создании отладочной версии приложения вместо операторов `new` и `delete` используются их специальные отладочные версии, отслеживающие имя файла и номер строки, в которых выделяется данный фрагмент памяти.

По завершении работы приложения все операторы `delete` сопоставляются с сопряженными с ними операторами `new` и, в случае обнаружения несоответствия, в окно **Output** выводится сообщение, аналогичное показанному на рис. 14.8.

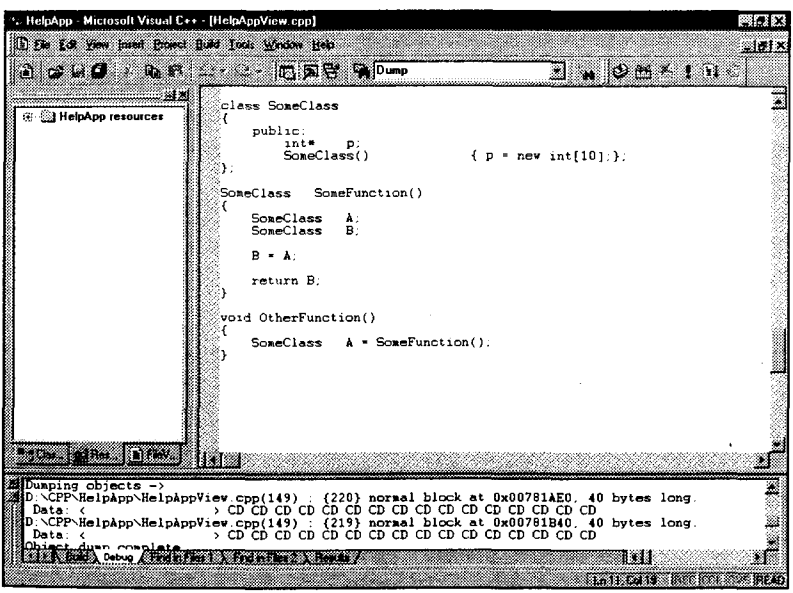


Рис. 14.8. Окно **Output** с сообщением об утечке памяти

Как видно из диагностического сообщения, источник обеих ошибок указан один: конструктор класса. Однако даже эта информация значительно сужает круг поиска возможных причин возникновения утечки памяти.

Особенности отладки приложений, использующих шаблоны

Шаблоны функций и классов, описанные в главе 11, позволяют существенно упростить процесс программирования, уменьшить объемы исходных текстов программ и снизить возможное количество ошибок в них. Однако при их отладке следует помнить об одной неприятной особенности Visual C++: при выборе команды меню **Debug**, **Run to Cursor** (или нажатии соответствующей этой команде комбинации клавиш <Ctrl>+<F10>) в функциях классов, использующих шаблоны, очень часто появляется диалоговое окно **Resolve Source Line Ambiguity** (Разрешение неоднозначности в строках текста), *изображенное на рис. 14.9.*

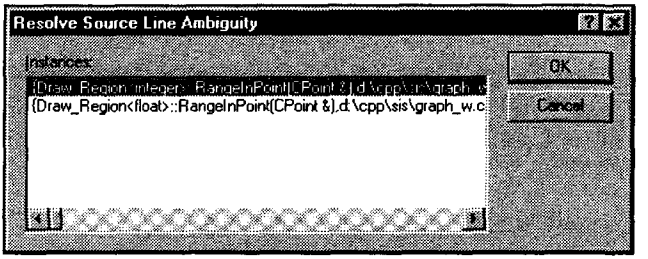


Рис. 14.9. Диалоговое окно **Resolve Source Line Ambiguity**

Появление данного диалогового окна заставляет пользователя судорожно вспоминать, какой из типов данных используется в данной функции, выделять его в окне списка **Instances:** и нажимать кнопку **OK** для продолжения отладки. Особенно "приятно" работать с этим окном, если команда меню **Debug**, **Run to Cursor** используется для обхода циклов при отслеживании работы функции. В этом случае для прохода функции придется по несколько раз совершать описанные выше действия.

Интересно, что при установке точки останова никакой неопределенности не возникает, и программа останавливает свою работу в указанной точке без вывода каких-либо диалоговых окон. Однако использование точек останова для просмотра работы функции требует их удаления по завершении просмотра для обеспечения возможности перехода к оценке работы других функций.

Приложения

A. Объектно-ориентированное программирование и классы

B. Описание классов и функций Visual C++

C. Средства для работы с классами Visual C++

D. Интерфейс пользователя Visual C++

Объектно-ориентированное программирование и классы

Объектно-ориентированный подход лежит в основе программирования в среде Windows. Однако многие программисты, работавшие в DOS и даже использовавшие библиотеки классов, например, библиотеку Turbo Vision, разработанную фирмой Borland для своей системы программирования Borland C++, имеют довольно-таки смутное представление о классах и пользуются при программировании заготовками, предоставляемыми системой программирования, внося в них минимальные изменения. Такой подход возможен и в среде Visual C++, однако для использования всех предоставляемых ею возможностей необходимо четко понимать, что такое классы, что они позволяют, а что нет. В данном приложении дано краткое описание объектно-ориентированного подхода к программированию и описаны основные свойства классов. Данное изложение не претендует на полноту. Оно призвано помочь читателю сформировать целостное представление по данному вопросу, а технические детали достаточно подробно описаны в справочной системе среды программирования.

Обзор объектно-ориентированных методов программирования

Чтобы перейти к написанию собственных классов, необходимо, прежде всего, подробнее ознакомиться с концепцией объектно-ориентированного программирования. Эта концепция позволяет рассматривать программу как набор модулей, каждый из которых выполняет отведенную ему функцию. Например, чтобы отправить письмо, вы не идете на вокзал, не отыскиваете человека, едущего в тот же город и не просите его передать письмо по определенному адресу. Вы опускаете письмо в почтовый ящик и надеетесь, что оно вовремя дойдет до адресата. Точно так же, если необходимо вызвать на

экран окно, то совсем не обязательно разбираться, каким образом отображается его рамка, а тем более, каким образом производится обработка поступающих в него и отправляемых им сообщений. К тому же, если вы попытаетесь это сделать, то что-нибудь упустите (а это естественно при отсутствии полного описания системы) и, скорее всего, вызовете крах всей системы.

Однако концепция объектно-ориентированного программирования (ООП) не ограничивается предоставлением пользователю удобных библиотек для решения сложных задач. Эти вопросы могут быть решены, например, в рамках структурного программирования. Программные объекты были разработаны для облегчения процесса программирования и приближения структуры программ к привычным для человека формам.

Можно смело сказать, что именно способность обобщать, классифицировать и генерировать абстракции явилась причиной превращения обезьяны в человека. Без этой способности невозможно было бы создание языка общения, поскольку каждое имя существительное является по сути своей классом объектов, обладающих некоторым набором атрибутов или отличительных черт. С другой стороны, каждый подобный класс является базовым для других классов, формируемых, например, путем добавления к этому существительному конкретизирующего его прилагательного. Например, "Белый дом".

В основе концепции объектно-ориентированного программирования лежат три основные принципа:

- *Инкапсуляция*

  Совмещение структур данных с функциями (методами), предназначенными для манипулирования этими данными. Инкапсуляция достигается путем введения класса нового механизма структурирования и типизации данных

- *Наследование*

  Создание новых, производных классов, которые наследуют данные и функции от одного или нескольких ранее определенных базовых классов. При этом возможно переопределение или добавление новых данных и методов. В результате создается иерархия классов

- *Полиморфизм*

  Присвоение методу единого имени или идентификатора в рамках иерархии классов таким образом, чтобы любой класс в иерархии имел возможность по-своему выполнять связанные с этим методом действия

Одновременно с появлением и детализацией концепции появились и основанные на ней языки программирования. Одним из первых явился алгоритмический язык Modula 2. Язык программирования Turbo Pascal, разработанный фирмой Borland, начиная с версии 5.5 стал объектно-ориентированным. Но наиболее последовательное воплощение концепция объектно-ориенти-

рованного программирования нашла в алгоритмическом языке C++, разработанном сотрудником Bell Laboratories AT&T Брайаном Страуструпом на базе языка программирования C как средство моделирования больших телефонных коммутационных систем.

Рассмотрим подробнее основополагающие принципы ООП.

Инкапсуляция

Одним из главных отличий стандартного структурного программирования от объектно-ориентированного является *инкапсуляция*. Совмещение структур данных с методами, предназначенными для манипулирования этими данными позволяет сосредоточить в одном месте всю информацию, касающуюся определенных данных и методов их обработки и, тем самым, облегчить доступ к этим методам и данным из любой точки программы.

Действительно, представьте себе, что все данные в программе имеют глобальную область видимости (что неизбежно при тесном взаимодействии всех частей программы). При этом для всех данных и для каждого метода придется использовать уникальный идентификатор, который должен быть информативным, то есть длинным. Программы с такими идентификаторами станут практически нечитаемыми, особенно на экранах дисплеев. Вам придется постоянно следить, чтобы новые имена не пересекались со старыми, чтобы в каждой функции использовались именно те данные, для которых она была написана. Кроме того, придется отказаться от таких элегантных возможностей, как перегрузка операций и создание собственных типов данных, позволяющих автоматически проверять корректность выполняемых над ними операций.

Кроме того, инкапсуляция позволяет скрыть данные и функции, используемые исключительно членами данного класса или членами производного от него класса, что упрощает описание класса для его использования внешними функциями и повышает надежность его работы, поскольку критически важные данные могут быть таким образом защищены от несанкционированного доступа.

Хорошим тоном считается скрытие всех данных, используемых классом, и обеспечение доступа к ним только посредством соответствующих методов класса. Это, прежде всего, устраняет все вопросы, связанные с областью видимости переменных, а в каждом классе можно использовать простые и информативные идентификаторы, не заботясь о том, что подобные идентификаторы уже есть в других классах.

Обеспечение доступа к данным исключительно через методы класса позволяет избегать потенциально очень опасных ситуаций. Предположим, что в процессе разработки какого-либо проекта одному из разработчиков была поставлена задача написать обработчик для некоторого массива, содержащего указатели на символьные строки. На каком-то этапе разработчик ре-

шил заменить этот массив связанным списком и соответствующим образом переписал методы обращения к нему.

Если он оформил этот массив в виде класса и обеспечил доступ к массиву исключительно через методы данного класса, то другие разработчики ни при каких обстоятельствах не должны почувствовать последствия этой замены (разве что она может отразиться на скорости выполнения приложения). Но если к этому массиву был обеспечен прямой доступ, то никто не может поручиться, что другой программист, работающий с теми же данными, не захочет воспользоваться собственными, более эффективными с его точки зрения, методами работы с массивом, которого уже нет.

Наследование

Принцип наследования в объектно-ориентированном программировании позволяет создать класс, включающий в себя все данные и методы некоторого другого класса, и, кроме того, содержащий свои, присущие только ему данные и методы обработки. Такой подход позволяет строить целые *иерархии классов*, каждый следующий уровень которых отличается все большей детализацией по сравнению с предшествующими уровнями. Достоинства такого подхода наиболее полно проявляются при программировании сложных объектов, к которым по праву относится сама операционная система Windows.

При всех различиях между окном представления и диалоговым окном, они, являясь окнами Windows, имеют много общего. Представьте себе, во что превратилось бы описание класса диалогового окна, если бы в нем непосредственно были описаны все обрабатываемые им данные и все методы обработки, учитывая тот факт, что каждое текстовое поле и каждый переключатель по сути является самостоятельным и полноправным окном Windows. Не говоря уже о том, сколько места займет описание каждого класса, и сколько в них будет повторяющихся и практически аналогичных по тексту функций.

Наследование позволяет описать сначала общий класс окна Windows, на его основе описать общий класс диалогового окна Windows, а уже на основе этого класса сформировать класс конкретного диалогового окна, используемого для реализации конкретного пользовательского интерфейса.

Полиморфизм

Последней отличительной чертой объектно-ориентированного программирования является *полиморфизм*. Использование полиморфизма позволяет существенно расширить набор функций в базовом классе, что, в свою очередь, приводит к существенному сокращению текста программы и улучшению ее читаемости.

Греческое слово полиморфизм было заимствовано из биологии, где оно означает наличие в пределах одного вида резко отличающихся по облику

особей. В C++ полиморфизм реализуется с помощью так называемых виртуальных функций, которые позволяют в пределах иерархии классов использовать одно имя метода для выполнения одной и той же операции над различными типами данных. Решение о том, какая именно версия данного метода будет использована в каждом конкретном случае, принимается на этапе исполнения программы и носит название позднего связывания.

Суть полиморфизма может быть наглядно проиллюстрирована на следующем примере. Предположим, вы решили создать иерархию классов для манипулирования отображаемыми на экране графическими объектами. Вы создали базовый класс, в который включили, среди прочих, функцию перемещения графических объектов. Казалось бы, данную функцию очень просто реализовать, необходимо только определить координаты нового положения объекта, уничтожить его изображение на старом месте и нарисовать на новом.

Однако при ближайшем рассмотрении перед вами встает неразрешимая на первый взгляд задача: функции отображения конкретных графических объектов задаются в производных классах, а обращение к ним предполагается производить в базовом классе. Конечно, у данной проблемы существует простое решение. Надо перенести функцию перемещения объекта из базового в производный класс и для каждого класса написать свой вариант функции. Однако, если следовать такой логике, в базовом классе практически не останется методов обработки, и все преимущества наследования будут практически сведены на нет.

Поэтому, как упоминалось выше, в рамках ООП был разработан аппарат *виртуальных функций*. Если в производном и базовом классах одна и та же функция объявлена виртуальной (для этого функции должны иметь не только одно и то же имя, но и один набор формальных параметров и одинаковый тип возвращаемой величины), то при обращении к этой функции из любого метода, унаследованного данным классом от своих базовых классов, будет вызываться функция, определенная в данном классе, а не функция соответствующего базового класса, как это имело бы место в противном случае.

Если виртуальные функции имеют различный набор формальных параметров, то они рассматриваются как различные функции. Различный тип возвращаемой величины у виртуальных функций с одинаковым именем и набором формальных параметров рассматривается как ошибка.

Классы

Классы как типы данных

В сущности, классы являются просто определяемыми пользователем типами данных. Аналогично всем другим типам данных они позволяют создавать столько своих копий, сколько требуется для решения конкретной задачи.

Возьмем, к примеру, стандартный тип данных `int`. Было бы странно, если бы в программе было разрешено использовать только одну переменную данного типа. Конечно, можно объявить столько целочисленных переменных, сколько понадобится.

То же самое относится и к классам. Описав новый класс, можно создавать столько переменных, имеющих тип данного класса, сколько нужно. Каждая переменная, называемая *объектом* данного класса, имеет полный доступ к функциям класса и свою копию хранящихся в нем данных. Поскольку каждый объект имеет свою копию данных, то изменения, внесенные в данные одного объекта, никак не отражаются на данных другого объекта.

Файлы заголовков и файлы реализации

Весь текст относительно простой программы может быть расположен в одном файле. Но такой подход становится недопустимым, когда размеры программы превышают определенные границы.

Во-первых, с таким файлом неудобно работать, поскольку часто приходится практически полностью пролистать весь файл, чтобы найти описание нужной функции. Понятно, что пролистать один из десяти файлов намного проще, чем один файл, включающий в себя содержимое этих десяти.

Во-вторых, не следует забывать, что объектно-ориентированное программирование развивалось в те времена, когда IBM PC имела объем памяти 64 Кб, внешним запоминающим устройством для нее служил кассетный магнитофон и никому, даже в страшном сне, не могло привидеться, что кому-то может не хватить 640 Кб оперативной памяти. Из вышесказанного очевидно, что в те времена очень остро стоял вопрос о размерах файлов, загружаемых в оперативную память. Поэтому при разработке алгоритмических языков особое внимание уделялось возможности компоновки программ из нескольких исходных файлов.

Поскольку классы представляли собой пользовательские типы данных, то можно было ожидать, что любой конкретный класс будет использоваться в любом из исходных программных файлов. Имевшиеся в то время средства обеспечения видимости переменных требовали, чтобы все обращения к любой переменной происходили, как минимум, в пределах одного файла. Чтобы обойти это ограничение была предпринята попытка разделить описание классов и программную реализацию используемых в них методов обработки. В результате появились *файлы заголовков* (header files) или *заголовочные файлы* и *файлы реализации* (implementation files). Файлы заголовков обычно имеют расширение .h и включают в себя краткое описание классов и других типов данных для включения их в файлы реализации, обычно имеющие расширение .cpp, в которых предполагается использование классов и переменных, определенных в данном файле заголовка.

Рассмотрим типичный файл заголовка, сгенерированный мастером AppWizard в Visual C++ версии 6.0 для приложения Class, в котором все опции были выбраны по умолчанию.

Листинг А.1. Файл заголовка класса

```
// ClassDoc.h : interface of the CClassDoc class
//////////////////////////////////////////////////////////////////////////

#if !defined(AFX_CLASSDOC_H__6791646D_7E54_11D2_
                                                    9C0A_B3CF58734270__INCLUDED_)
#define AFX_CLASSDOC_H__6791646D_7E54_11D2_9C0A_B3CF58734270__INCLUDED_

#if _MSC_VER > 1000
#pragma once
#endif // _MSC_VER > 1000

class CClassDoc : public CDocument
{
protected: // create from serialization only
  CClassDoc();
  DECLARE_DYNCREATE(CClassDoc)

// Attributes
public:

// Operations
public:

// Overrides
  // ClassWizard generated virtual function overrides
  //{{AFX_VIRTUAL(CClassDoc)
  public:
  virtual BOOL OnNewDocument();
  virtual void Serialize(CArchive& ar);
  //}}AFX_VIRTUAL

// Implementation
public:
  virtual ~CClassDoc();
#ifdef _DEBUG
  virtual void AssertValid() const;
  virtual void Dump(CDumpContext& dc) const;
#endif

protected:

// Generated message map functions
protected:
```

```
    //{{AFX_MSG(CClassDoc)
      // NOTE - the ClassWizard will add and remove member functions here.
      // DO NOT EDIT what you see in these blocks of generated code !
    //}}AFX_MSG
    DECLARE_MESSAGE_MAP()
};

/////////////////////////////////////////////////////////////////////////////

//{{AFX_INSERT_LOCATION}}
// Microsoft Visual C++ will insert additional declarations immediately
before the previous line.

#endif // !defined(AFX_CLASSDOC_H__6791646D_7E54_11D2_9C0A_
                                                    B3CF58734270__INCLUDED_)
```

Первые строки листинга А.1 являются комментарием, в котором указано имя файла заголовка и дано его краткое описание: интерфейс класса `CClassDoc`. За комментарием следует оператор условной компиляции `#if !defined(AFX_CLASSDOC_H__6791646D_7E54_11D2_9C0A_B3CF58734270__INCLUDED_)`, включенный для того, чтобы следующий за ним текст был скомпилирован только однажды. Дело в том, что файлы заголовка включаются в программу оператором `#include`, выполняющим непосредственную подстановку вместо себя содержимого указанного за ним файла. Оператор `#include` может быть включен и в файлы заголовков, если в них используются классы, определенные в других файлах заголовков. Таким образом, определение одного и того же класса может оказаться несколько раз включенным в файл реализации, что недопустимо.

Чтобы избежать этого, используются *операторы условной трансляции*, которые проверяют, был ли уже определен некоторый идентификатор, уникальный для данного файла заголовка. В данном случае это `AFX_CLASSDOC_H__6791646D_7E54_11D2_9C0A_B3CF58734270__INCLUDED_`. Если он уже был определен, то участок файла от данного оператора условной трансляции до оператора `#endif`, расположенного в конце данного файла заголовка, пропускается. То есть текст этого файла повторно не включается в текст файла реализации. Если этот идентификатор не был определен, то компиляция продолжается со следующего оператора, в данном случае — `#define AFX_CLASSDOC_H__6791646D_7E54_11D2_9C0A_B3CF58734270__INCLUDED_`. Этот оператор определяет указанный уникальный идентификатор и предотвращает повторное включение данного файла заголовка в файл реализации. Затем следует проверка используемой версии библиотеки MFC. И только после этого расположено описание класса.

Строка `class CClassDoc : public CDocument` указывает на то, что последующий блок будет содержать описание класса `CClassDoc`, базовым классом для которого является класс `CDocument`. Ключевое слово `public` перед именем

базового класса означает, что образуемый класс будет иметь максимальные права доступа к данным и методам базового класса.

Блок описания класса представляет собой специальным образом организованный список элементов класса, к которым относятся хранимые в нем данные и методы их обработки. Здесь блок описания класса открывается *атрибутом прав доступа.* В данном случае это `protected`. Об атрибутах прав доступа следует сказать подробнее.

Как уже говорилось, одним из преимуществ концепции объектно-ориентированного программирования является скрытие данных, то есть запрет на доступ к некоторым данным класса со стороны внешних функций и классов. Различают три уровня ограничения доступа к компонентам класса.

- Наиболее строго ограничен доступ к частным компонентам класса. К ним могут обращаться только методы данного класса. По умолчанию все компоненты класса являются частными. Чтобы установить подобный режим доступа для компонентов, описание которых следует за компонентами, имеющими другой режим доступа, необходимо перед их описанием поставить атрибут права доступа `private`, а после него поставить двоеточие. Естественно, что классом, в котором имеются только частные компоненты, невозможно воспользоваться, поскольку в нем недоступны не только данные, но и методы их обработки

- Наиболее свободным режимом доступа является глобальный режим. К компонентам с глобальным режимом доступа могут обращаться любые внешние функции и классы. Для установки этого режима доступа для компонентов необходимо перед их описанием поставить атрибут права доступа `public` с последующим двоеточием

- Защищенный режим доступа занимает промежуточное положение между частным и глобальным режимами. К защищенным компонентам класса могут обращаться только члены данного и производного от него классов. Чтобы установить этот режим доступа для компонентов, необходимо перед их описанием поставить атрибут права доступа `protected` с последующим двоеточием

Использование режимов доступа для каждого конкретного компонента класса определяется исключительно требованиями к надежности хранения и манипулирования хранящимися в классе данными, а также удобством работы с данным классом.

Атрибуты права доступа делят список элементов класса на секции, все элементы которых имеют указанный при открытии секции режим доступа. Единственным исключением может являться первая секция, для которой можно не указывать атрибут права доступа. В этом случае все перечисленные в ней элементы класса будут иметь режим доступа по умолчанию, то есть являться частными элементами класса.

Обычно первыми методами, перечисляемыми при описании класса, являются его конструкторы. *Конструктор класса* представляет собой метод, вызываемый при создании объекта данного класса и используемый для установки начальных величин хранящихся в нем данных, если имеется необходимость в такой установке. Если данные класса не нуждаются в начальной установке, то конструктор все равно описывается, но в него не включаются никакие операторы. Такой конструктор называется пустым. Класс может иметь неограниченное количество различных конструкторов, различающихся набором формальных параметров. В языке C++ конструктор имеет то же имя, что и класс. В данном случае конструктору присвоено имя `CClassDoc`.

Другим обязательным методом любого класса является его *деструктор*, представляющий собой метод, вызываемый при уничтожении объекта данного класса. Класс может иметь только один деструктор, который не должен иметь списка формальных параметров. Обычно деструктор используется для уничтожения всех динамических переменных, созданных в процессе работы методов класса. Деструктор имеет то же имя, что и класс, но ему предшествует символ ~. В данном случае деструктор является виртуальной функцией с именем `~CClassDoc`.

Непосредственно за конструктором класса следует макрос `DECLARE_DYNCREATE (CClassDoc)`, имеющий синтаксис `DECLARE_DYNCREATE (class_name)`. Его единственным аргументом является переменная `class_name`, представляющая собой имя данного класса (не заключенное в кавычки). Данный макрос обеспечивает динамическое создание объектов, произведенных от класса `CObject`, в ходе выполнения программы. Операционная система использует эту возможность для динамического создания новых объектов, например, при чтении объекта с диска, что необходимо при работе с документами и представлениями.

Данный пример содержит описание двух генерируемых мастером ClassWizard виртуальных функций: `OnNewDocument`, возвращающей величину логического типа, и `Serialize`, не возвращающую значение и имеющую своим аргументом ссылку на объект класса `CArchive`. В него также включены отладочные функции `AssertValid` и `Dump`.

Рассмотрим теперь файл реализации для данного класса.

Листинг А.2. Файл реализации класса

```
// ClassDoc.cpp : implementation of the CClassDoc class
//

#include "stdafx.h"
#include "Class.h"

#include "ClassDoc.h"
```

```
#ifdef _DEBUG
#define new DEBUG_NEW
#undef THIS_FILE
static char THIS_FILE[] = __FILE__;
#endif

/////////////////////////////////////////////////////////////////////////////
// CClassDoc

IMPLEMENT_DYNCREATE(CClassDoc, CDocument)

BEGIN_MESSAGE_MAP(CClassDoc, CDocument)
  //{{AFX_MSG_MAP(CClassDoc)
    // NOTE - the ClassWizard will add and remove mapping macros here.
    //    DO NOT EDIT what you see in these blocks of generated code!
  //}}AFX_MSG_MAP
END_MESSAGE_MAP()

/////////////////////////////////////////////////////////////////////////////
// CClassDoc construction/destruction

CClassDoc::CClassDoc()
{
  // TODO: add one-time construction code here

}

CClassDoc::~CClassDoc()
{
}

BOOL CClassDoc::OnNewDocument()
{
  if (!CDocument::OnNewDocument())
    return FALSE;

  // TODO: add reinitialization code here
  // (SDI documents will reuse this document)

  return TRUE;
}


/////////////////////////////////////////////////////////////////////////////
// CClassDoc serialization

void CClassDoc::Serialize(CArchive& ar)
{
  if (ar.IsStoring())
```

```
    {
      // TODO: add storing code here
    }
    else
    {
      // TODO: add loading code here
    }
}

/////////////////////////////////////////////////////////////////////////////
// CClassDoc diagnostics

#ifdef _DEBUG
void CClassDoc::AssertValid() const
{
  CDocument::AssertValid();
}

void CClassDoc::Dump(CDumpContext& dc) const
{
  CDocument::Dump(dc);
}
#endif //_DEBUG

/////////////////////////////////////////////////////////////////////////////
// CClassDoc commands
```

Файл реализации, как и файл заголовка, начинается с комментария, в котором указывается, что это за файл. Затем следуют операторы `#include` для всех файлов заголовка, которые необходимо включить в данную программу. Далее следует непосредственное описание реализации методов данного класса.

Описание начинается с макроса `IMPLEMENT_DYNCREATE(CClassDoc, CDocument)`, имеющего синтаксис `IMPLEMENT_DYNCREATE(class_name,base_class_name)`, где аргумент `class_name` соответствует имени данного класса, а аргумент `base_class_name` — имени базового класса. Этот макрос является парой для макроса `DECLARE_DYNACREATE`, описанного выше, и может использоваться исключительно в паре с ним. Ни один из этих макросов не может работать поодиночке.

Сразу за этим макросом находится описание конструктора и деструктора данного класса. Данный пример является не совсем удачным, поскольку в нем конструктор базового класса не имеет формальных параметров, которые должны быть для него определены в производном классе. В противном случае заголовок описания метода конструктора выглядел бы не `CClassDoc::`

`CClassDoc()`, как в данном примере, а, например, `CClassDoc::CClassDoc(int par):CDocument(par)`, хотя в действительности класс `CDocument` не имеет конструктора с формальными параметрами.

Такой способ передачи параметров конструкторов является одним из наиболее существенных недостатков языка программирования C++, поскольку существенно затрудняет преобразование параметров при их передаче конструктору базового класса. Например, в языке программирования Turbo Pascal конструкторы и деструкторы имели предопределенные имена и могли вызываться в любой точке конструктора производного класса.

Наибольшие сложности при передаче параметров конструктору базового класса способом, принятым в C++, возникают при необходимости существенных преобразований параметров данного класса предшествующих передаче их в базовый класс. Такие преобразования могут, например, потребовать использования кусочно-линейной аппроксимации, параметры которой зависят от некоторой величины перечислимого типа, или проведения достаточно сложных математических вычислений, например, вычисления логарифма по основанию два и преобразования полученной величины к наименьшему целому, превышающему полученную величину. Эти примеры не выдуманы автором, а взяты из конкретной жизни.

Другим неприятным следствием такого метода передачи данных является невозможность вызова одного конструктора класса из другого. Предположим, что вам необходимо создать два конструктора, отличающихся лишь необходимостью задания исходной величины одного единственного параметра, а решить этот вопрос с использованием значений по умолчанию не позволяет наличие уже имеющегося списка формальных параметров с заданными начальными значениями, не обеспечивающего минимальное число задаваемых параметров для большинства практических случаев. Казалось бы, существует простое решение: задать в конструкторе начальное значение параметра, а потом вызвать другой конструктор для инициализации остальных величин. Однако такой фокус не проходит, и придется создать два полноценных конструктора с практически идентичным текстом.

Описание деструктора и остальных функций ничем не отличается от описания обычных "неклассовых" функций. Единственным исключением является необходимость указывать имя этого класса перед именем каждой функции, реализующей тот или иной метод класса. Такой способ позволяет помещать описания методов класса в любой файл проекта. Иногда такое разнесение описания класса по нескольким файлам оказывается очень удобным. При описании методов класса следует обращать особое внимание на права доступа к переменным, поскольку в самый неподходящий момент может оказаться, что вы не имеете права доступа к переменной, которую собираетесь использовать в данном классе.

Когда следует использовать классы

Объектно-ориентированный подход поднимает процесс программирования на новую ступень. Осознав возможности, предоставляемые ООП, программист начинает использовать классы где надо и где не надо. Даже работая с библиотекой классов MFC, программист обязан трезво оценить глубину иерархии создаваемых классов. Не следует забывать, что C++ является не только объектно-ориентированным, но и процедурным языком. Другими словами, программисты могут воспользоваться лучшим, что имеется в каждом из этих подходов к программированию и что наиболее соответствует сущности решаемой ими задачи.

Применение классов наиболее целесообразно для обработки разнородной информации, имеющей в своей основе общие черты. Например, имеет смысл создать базовый класс для работы со строками, а потом на его основе создавать классы для работы с конкретными строками в текстовом редакторе. Это позволит достаточно просто определить множество разнообразных классов, обрабатывающих символьную информацию на основе единых принципов и методов обработки.

Однако за все приходится платить. При обращении к методу класса приходится использовать более сложный синтаксис, чем при обращении к обычной функции. При таком обращении необходимо указать не только имя самой функции, но и имя содержащего ее объекта. Кроме того, создание классов требует определенных затрат на формирование его структуры и разработку его концепции. Поэтому всегда следует оценивать дополнительные затраты и получаемые преимущества от применения ООП.

Одним из преимуществ классов является возможность создания их библиотек. Программисты, работающие в какой-либо определенной области, вынуждены постоянно пользоваться для решения своих задач одним и тем же набором функций, которые целесообразно объединить в классы и организовать таким образом, чтобы программирование каждой новой задачи сводилось к использованию нескольких объектов определенных заранее классов, в каждом из которых вызывалось бы ограниченное количество методов обработки. Такой подход не только позволит сократить количество ошибок при программировании задачи и ускорит сам процесс программирования, но и повысит читаемость программы.

Перегрузка функций и операторов

Одним из основных отличий языка программирования C от C++ является возможность перегрузки функций и операторов. В языке C каждая функция должна была иметь уникальное имя. Функции, выполняющие одну и ту же операцию над данными разных типов, должны были иметь различные имена. Применение в C++ *перегружаемых функций* позволило различать функции

не только по имени, но и по типу формальных параметров, т. е. использовать одно и то же имя функции для выполнения сходных операций над данными, имеющими различный тип.

Перегрузка функций является мощным инструментом программирования, но всегда следует трезво оценить, является ли перегрузка функции лучшим решением в данном конкретном случае. Предположим, нужно создать конструктор для класса, при инициализации которого следует задать два целочисленных параметра, однако в большинстве случаев второй параметр принимается равным нулю.

Данная задача может быть решена при помощи перегрузки функций, когда будет создано две версии конструкторов, например:

```
SomeClass(int);
SomeClass(int,int);
```

Другим, более красивым решением данной задачи может явиться *использование аргументов по умолчанию.* В этом случае необходимо создать только один конструктор с двумя формальными параметрами, в котором второму параметру будет присваиваться значение по умолчанию. Этот конструктор выглядит следующим образом

```
SomeClass(int,int=0)
```

При этом, создавая объекты данного класса, допустимо задавать как один, так и два параметра. Если задан только один параметр, он будет сопоставлен первому формальному параметру, а значение второго формального параметра будет принято равным нулю.

Одной из наиболее привлекательных возможностей, предоставляемых языком программирования C++, является *перегрузка операторов.* С точки зрения этого языка любой оператор является особой разновидностью функции и, как и любую другую функцию, его можно определить и перезагрузить. Например, оператор `argument1 + argument2` полностью эквивалентен функции `SomeClass argument1.operator +(argument2)`, где оба аргумента являются объектами класса `SomeClass`, в котором объявлена соответствующая функция.

Как с точки зрения простоты, так и с точки зрения читаемости программы выражение `string1 += string2` предпочтительнее выражения `strcat(string1, string2)`, хотя для реализации первого выражения требуется создать специальный класс для работы со строковыми переменными и определить в нем оператор `+=`, а второе выражение является стандартной функцией C++. Но при всей своей привлекательности, перегрузка операторов таит в себе множество потенциальных опасностей.

Набор символов стандартных операций языка C++ довольно ограничен, и все они имеют интуитивно подразумеваемый смысл. Поэтому, например, использование знака операции `+` для реализации метода объединения двух

массивов из двух элементов в один массив из четырех элементов вызовет у большинства пользователей неправильные ассоциации, что приведет к росту числа ошибок при его использовании. Хотя, с другой стороны, использование этого знака операции для объединения двух строк из двух символов в одну строку из четырех символов большинством пользователей будет воспринято адекватно.

Перегрузка префиксной или постфиксной *унарной операции* может быть выполнена при помощи нестатической функции компонента, не принимающего аргументов, либо при помощи функции, не являющейся методом, то есть не являющейся членом класса, и принимающей один аргумент. Таким образом, оператор отрицания для объекта `x`, принадлежащего к классу `SomeClass`, может быть описан как `x. operator -()` или как `operator -(x)`.

Перегрузка бинарной операции может быть выполнена при помощи объявления нестатической функции компонента, принимающей один аргумент, или при помощи функции, не являющейся методом и принимающей два аргумента. В последнем случае функция обычно объявляется *дружественной* (`friend`). Это практически единственный случай, в котором оправдано применение дружественных функций.

Дружественные функции не являются членами класса, хотя их описание располагается в блоке описания класса. От обычных функций они отличаются тем, что имеют доступ к частным и защищенным элементам данного класса. С точки зрения программиста, модификатор `friend` можно рассматривать просто как непременный атрибут описания функции, реализующей бинарную операцию и принимающей два формальных параметра. Таким образом, оператор вычитания для объекта `y` из объекта `x`, каждый из которых принадлежит к классу `SomeClass`, может быть описан как `x.operator - (y)` или как `operator - (x,y)`.

Операция присваивания = может быть перегружена только при помощи объявления нестатического метода. В отличие от прочих операций, функция операции присваивания не может наследоваться производными классами. Если эта операция не определена в классе, то по умолчанию ее результатом является последовательное присваивание элементов класса источника элементам класса приемника. Это может привести к серьезным проблемам, если хотя бы один элемент данного класса является указателем.

Использование виртуальных функций

Виртуальные функции позволяют производным классам обеспечивать различные версии функций базового класса. Как отмечалось выше, виртуальные функции позволяют вызывать в методах базового класса методы производного класса. Необходимость в этом возникает, если базовый класс обеспечивает описание некоторой базовой процедуры обработки, которая соответствующим образом детализируется в каждом из производных классов.

Необходимо отметить, что язык программирования C++ и без применения аппарата виртуальных функций допускает создание в базовом и производных классах функций, имеющих не только одно и то же имя, но и один и тот же список формальных параметров. Однако в этом случае в методах базового класса будет вызываться соответствующая функция базового класса, а в методах производного класса — функция производного класса, т. е. в процессе одной и той же обработки под одним и тем же именем могут вызываться две совершенно различные функции. Иногда это может быть полезно, но требует пристального внимания со стороны программиста.

Прежде чем решиться на использование виртуальной функции, оцените, чем отличаются классы в иерархии. Должны ли они выполнять различные действия? Или все различия сводятся к обработке различных значений? Например, при отображении различных графических объектов виртуальные функции использовались для отображения этих объектов. В каждом объекте имелся метод, ответственный за его вывод на экран, и именно он использовался в функциях базового класса. С другой стороны, нет никакой необходимости использовать виртуальные функции, например, для задания цвета объекта, поскольку реализация данного метода не зависит от конкретного объекта.

Область действия класса

Имя элемента класса имеет область действия, локальную по отношению к области действия самого класса. Имя элемента класса может использоваться:

- В методах данного класса
- В выражениях типа `X.M`, где `X` — объект данного класса, а `M` — имя элемента класса
- В выражениях типа `lpX-> M`, где `lpX` — указатель на объект данного класса
- В выражениях типа `SomeClass::M` или `DerivedClass::M`, где `SomeClass` — имя данного класса, а `DerivedClass` — имя производного от него класса
- В ссылках вперед в пределах класса, к которому принадлежит данный элемент

Классы, перечисляемые данные и имена `typedef`, определенные в пределах данного класса, либо имена функций, объявленных дружественными данному классу, не являются элементами данного класса и имеют ту же область действия, что и сам класс.

Имя может быть скрыто явным объявлением того же самого имени в окружающем блоке или классе. Доступ к скрытому элементу класса, тем не менее, возможен при помощи модификатора области действия, заданного с именем класса: `SomeClass::M`. Скрытое имя с глобальной областью действия допускает ссылку на него при помощи унарной операции `::`, например, `::M`.

Так, в любом методе класса, содержащего виртуальный или простой метод `SomeFunction(...)`, имеющий то же имя и список формальных параметров, что и метод базового класса, может быть непосредственно вызван соответствующий ему метод базового класса при помощи модификатора области действия `BaseClass::SomeFunction(...)`, где `BaseClass` — имя базового класса. С другой стороны, если имеется глобальная функция с тем же именем, что и метод класса, для ее вызова следует воспользоваться следующим синтаксисом: `::SomeFunction(...)`.

Проверка допустимости имени в конкретной области действия производится в следующем порядке:

1. Сначала само имя проверяется на наличие неоднозначностей. Если в пределах области действия неоднозначности отсутствуют, то инициализируется последовательность доступа.
2. Если ошибок управления доступа не обнаружено, то проверяется тип объекта, класса и т. д.
3. Если имя используется вне функции или класса, либо имеет префикс унарной операции доступа к области действия `::`, если оно не уточняется бинарной операцией `::` или операциями выбора элемента класса или `->`, то это имя должно быть именем глобального объекта, функции или нумератора.
4. Если идентификатор `SomeIdent` встречается в одном из следующих контекстов: `SomeClass:: SomeIdent`, `SomeObj.SomeIdent` или `SomePtr-> SomeIdent`, где `SomeClass` — имя некоторого класса, `SomeObj` — имя объекта некоторого класса, а `SomePtr` — указатель на объект некоторого класса, то этот идентификатор `SomeIdent` является именем элемента данного класса или именем элемента производного от него класса.
5. Любое нерассмотренное имя, используемое в качестве статического или нестатического метода, должно быть объявлено в блоке, в котором оно встречается, либо в окружающем блоке, либо являться глобальным методом. Объявление в блоке идентификатора с именем, совпадающим с именем идентификатора, определенного в окружающем блоке, или именем глобальной переменной, скрывает имена идентификаторов, внешних по отношению к данному блоку. Имена в различных областях действия не могут быть перегружены.
6. Имя аргумента функции в определении функции находится в области действия самого внешнего блока данной функции. Аргумент по умолчанию вычисляется в каждой точке вызова.
7. Инициализатор конструктора вычисляется в области действия самого внешнего блока конструктора и поэтому разрешает ссылки на имена аргументов конструктора.

Описание классов и функций Visual C++

В данной книге при описании реализации различных процедур обработки и вывода информации часто встречаются ссылки на различные классы и их методы. Часто одни и те же классы используются для решения задач, поставленных в различных главах книги. Чтобы научиться программировать в среде Visual C++, необходимо досконально разбираться в методах основных классов и функций, используемых в данной среде программирования. Это могут быть классы библиотеки MFC, классы Windows или глобальные функции.

В настоящее время далеко не все программисты настолько хорошо владеют английским языком, чтобы черпать всю необходимую информацию о классах из библиотеки MSDN. Тем более что более или менее полная версия данной библиотеки, позволяющая получить ответ на большинство вопросов без обращения к CD, занимает около 150 Мб.

В связи с этим существует настоятельная потребность в подробном справочнике по классам и функциям Visual C++. Поскольку нельзя объять необъятное, в этот справочник вошли описания только тех классов и функций, которые рассматривались при решении задач, поставленных в данной книге.

Начнем с описания глобальных функций Windows, разбитых по областям их применения, а затем следует описание классов, расположенных в алфавитном порядке.

Глобальные функции Windows

AddFontResource

```
int AddFontResource(LPCTSTR lpszFilename);
```

Возвращаемое значение

В случае успешного завершения функции возвращается количество добавленных шрифтов, в противном случае возвращается нулевое значение.

В Windows NT более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

lpszFilename — указатель на заканчивающуюся нулем текстовую строку, содержащую корректное имя файла шрифта. Имя файла шрифта имеет расширение .fon для файлов ресурса шрифта, расширение .fnt для файлов шрифта, содержащих битовые образы символов, расширение .ttf для файлов шрифтов TrueType и расширение .fot для файлов ресурсов шрифтов TrueType.

Описание

Функция AddFontResource позволяет добавить ресурсы шрифта из указанного файла в системную таблицу шрифтов. После этого данный шрифт может быть использован для вывода текста любым приложением Win32.

Любые приложения, добавляющие или удаляющие шрифты из системной таблицы шрифтов извещают об этом другие приложения, посылая сообщение WM_FONTCHANGE всем окнам верхнего уровня в операционной системе. Для посылки этого сообщения приложение должно использовать функцию SendMessage, в аргументе hWnd которой должно стоять значение HWND_BROADCAST.

Как только приложение перестает использовать ресурс шрифта, загруженный функцией AddFontResource, оно должно удалить его функцией RemoveFontResource.

AfxBeginThread

```
CWinThread* AfxBeginThread(AFX_THREADPROC pfnThreadProc, LPVOID pParam,
  int nPriority = THREAD_PRIORITY_NORMAL, UINT nStackSize = 0,
  DWORD dwCreateFlags = 0, LPSECURITY_ATTRIBUTES lpSecurityAttrs = NULL);

CWinThread* AfxBeginThread(CRuntimeClass* pThreadClass,
  int nPriority = THREAD_PRIORITY_NORMAL, UINT nStackSize = 0,
  DWORD dwCreateFlags = 0, LPSECURITY_ATTRIBUTES lpSecurityAttrs = NULL);
```

Возвращаемое значение

Указатель на созданный данной функцией объект класса потока.

Аргументы

pfnThreadProc — указатель на исполняющую функцию рабочего потока. Не может иметь нулевого значения. Исполняющая функция потока имеет формат UINT MyControllingFunction(LPVOID pParam).

pThreadClass — указатель на структуру CRuntimeClass для пользовательского класса, производного от класса CWinThread.

pParam — аргумент, передаваемый исполняющей функции потока, как показано в описании аргумента pfnThreadProc.

nPriority — приоритет потока. Если эта величина равно нулю, то у создаваемого потока устанавливается такой же приоритет, как и у вызывающего потока. Полный список значений приоритетов приведен в описании функции SetThreadPriority.

nStackSize — определяет размер стека нового потока в байтах. Если эта величина равно нулю, то у создаваемого потока создается стек того же размера, что и у вызывающего потока.

dwCreateFlags — определяет дополнительный флаг, управляющий процессом создания потока. Этот флаг может иметь два значения:

- CREATE_SUSPENDED — создает поток, в котором счетчик остановки установлен в единицу. Этот поток не будет исполняться, пока не будет вызвана функция ResumeThread
- 0 — начать работу потока непосредственно после его создания

lpSecurityAttrs — указатель на объект структуры SECURITY_ATTRIBUTES, определяющий атрибуты безопасности данного потока. Если эта величина равна нулю, то создаваемый поток имеет те же атрибуты безопасности, что и вызывающий поток.

Описание

Данная функция позволяет создать новый поток. Ее первая версия создает рабочий поток, вторая — интерфейсный поток.

Функция AfxBeginThread создает новый объект класса CWinThread, вызывает функцию CreateThread для начала исполнения потока, а также возвращает указатель на созданный объект класса потока. В процессе выполнения данной функции производятся все проверки, необходимые для того, чтобы гарантировать, что в случае возникновения ошибки в процессе выполнения данной функции все созданные до этого объекты будут соответствующим образом уничтожены. Для завершения работы потока следует вызвать функцию AfxEndThread из данного потока или выйти из исполняющей функции потока.

AfxEnableControlContainer

```
void AfxEnableControlContainer();
```

Описание

Данная функция вызывается функцией CWinApp::InitInstance, чтобы обеспечить в приложении поддержку элементов управления OLE.

AfxEndThread

```
void AfxEndThread(UINT nExitCode);
```

Аргументы

`nExitCode` — определяет код завершения потока.

Описание

Данная функция используется для завершения текущего потока. Должна вызываться из потока, который необходимо завершить.

AfxGetApp

```
CWinApp* AfxGetApp();
```

Возвращаемое значение

Указатель на единственный объект класса `CWinApp` данного приложения.

Описание

Указатель, возвращаемый данной функцией, может быть использован для доступа к информации о данном приложении. Например, о главной процедуре обработки сообщения или об объекте главного окна программы.

AfxGetInstanceHandle

```
HINSTANCE AfxGetInstanceHandle();
```

Возвращаемое значение

Дескриптор `HINSTANCE` текущей копии приложения. Если вызов данной функции произошел из библиотеки динамической компоновки, связанной с версией USRDLL библиотеки MFC, возвращается дескриптор `HINSTANCE` данной библиотеки динамической компоновки (DLL).

Описание

Данная функция позволяет получить дескриптор текущей копии приложения. Функция `AfxGetInstanceHandle` всегда возвращает объект `HINSTANCE` исполняемого в настоящее время файла (.exe), кроме случая, когда она вызывается из библиотеки динамической компоновки, связанной с версией USRDLL библиотеки MFC. В этом случае она возвращает дескриптор `HINSTANCE` данной библиотеки динамической компоновки.

AfxGetResourceHandle

```
HINSTANCE AfxGetResourceHandle();
```

Возвращаемое значение

Дескриптор HINSTANCE ресурсов, принадлежащих данному приложению по умолчанию.

Описание

Возвращаемый данной функцией дескриптор может быть использован для непосредственного доступа к ресурсам приложения, например, при вызове функции Windows FindResource.

AfxGetAppName

```
LPCTSTR AfxGetAppName();
```

Возвращаемое значение

Указатель на строку, кончающуюся нулем, содержащую имя приложения.

Описание

Возвращаемая данной функцией строка может быть использована для диагностических сообщений или как основа для имен временных строк.

AfxRegisterWndClass

```
LPCTSTR AFXAPI AfxRegisterWndClass(UINT nClassStyle,
  HCURSOR hCursor = 0, HBRUSH hbrBackground = 0, HICON hIcon = 0);
```

Возвращаемое значение

Символьная строка, заканчивающаяся нулем и содержащая имя класса. Это имя может быть использовано в функции CWnd::Create для создания объекта класса окна. Имя класса генерируется библиотекой MFC. Возвращаемое значение является указателем на статический буфер. Для его сохранения можно воспользоваться объектом класса CString.

Аргументы

nClassStyle — определяет стиль окна Windows или комбинацию стилей, созданных с использованием оператора поразрядного ИЛИ (|). Список стилей окна приведен в описании структуры WNDCLASS. Если данный аргумент равен нулю, то используется следующая комбинация стилей:

- Стиль мыши устанавливается в CS_DBLCLKS, что означает посылку процедуре окна сообщения о двойном щелчке
- Устанавливается стандартный для Windows стиль указателя мыши IDC_ARROW

- Стиль фоновой кисти окна устанавливается в ноль, что препятствует уничтожению окном своего фона
- В качестве значка окна выбирается "рассыпающийся" флаг Windows

`hCursor` — определяет дескриптор ресурса курсора, устанавливаемого в каждое окно, созданное данным классом окна. Если значение данного аргумента не задано или равно нулю, используется стандартный курсор `IDC_ARROW`.

`hbrBackground` — определяет дескриптор ресурса фоновой кисти, устанавливаемого в каждое окно, созданное данным классом окна. Если значение данного аргумента не задано или равно нулю, для рисования фона используется нулевая кисть, и окно не стирает свой фон при обработке сообщения `WM_ERASEBKGND`.

`hIcon` — определяет дескриптор ресурса значка, устанавливаемого в каждое окно, созданное данным классом окна. Если значение этого аргумента не задано или равно нулю, в качестве значка окна выбирается флаг Windows.

Описание

Библиотека MFC автоматически регистрирует некоторые стандартные классы окон, поэтому данная функция вызывается для регистрации нестандартных классов окон. Имя, которое будет сопоставлено классу функцией `AfxRegisterWndClass`, определяется исключительно переданными ей аргументами. Если данная функция будет несколько раз вызвана с одними и теми же значениями аргументов, то регистрация класса произойдет только при ее первом вызове. При последующих вызовах данная функция возвратит имя уже зарегистрированного ею класса.

При вызове функции `AfxRegisterWndClass` для нескольких классов, производных от класса `CWnd`, с идентичными значениями аргументов, вместо создания для каждого из этих классов своего класса окна, все они будут использовать один и тот же класс окна. Это может вызвать проблемы при использовании стиля `CS_CLASSDC`, поскольку все окна, использующие данный класс окна будут разделять один и тот же контекст устройства. В связи с этим рекомендуется использовать для регистрации класса функцию `AfxRegisterClass`.

AfxWinInit

```
BOOL AFXAPI AfxWinInit(HINSTANCE hInstance, HINSTANCE hPrevInstance,
  LPTSTR lpCmdLine, int nCmdShow)
```

Аргументы

`hInstance` — дескриптор текущего выполняющегося модуля.

`hPrevInstance` — дескриптор предыдущей копии данного приложения. Для приложений Win32 этот аргумент всегда имеет нулевое значение.

`lpCmdLine` — указатель на завершающуюся нулем строку, содержащую командную строку приложения.

`nCmdShow` — определяет, сколько окон приложения, использующего GUI, должно отображаться.

Описание

Вызывается функцией `WinMain` библиотеки MFC как часть процесса инициализации класса `CWinApp` для приложения, использующего GUI, отвечающая за его связь с библиотекой MFC. В консольных приложениях, не имеющих функции `WinMain` библиотеки MFC, для поддержки функций библиотеки MFC необходимо непосредственно вызвать функцию `AfxWinInit`.

При непосредственном вызове функции `AfxWinInit` необходимо создать объект класса, производного от класса `CWinApp`. При вызове данной функции в консольных приложениях пользователь может не создавать собственного класса, производного от `CWinApp`, а непосредственно использовать объект класса `CWinApp`. Это оправдано в том случае, когда пользователь не модифицирует стандартное поведение функций библиотеки MFC.

Приведенный ниже пример демонстрирует использование функции `AfxWinInit` в консольном приложении. Для компиляции данного примера необходимо установить параметры /GX и /MT. Например, следующим образом: cl /GX /MT thisfile.cpp

Пример

```
#include <afx.h>
#include <afxdb.h>
#include <iostream.h>

int main()
{
   // Попытка инициализации библиотеки MFC

   if (!AfxWinInit(::GetModuleHandle(NULL), NULL, ::GetCommandLine(), 0))
   {
      cerr << "Не удалось инициализировать библиотеку MFC" << endl;
      return 1;
   }

   // Попытка связаться с базой данных ODBC, которая не существует
   // (без инициализации библиотеки MFC эта операция вообще невозможна)

   CDatabase db;
   try
   {
      db.Open("База данных отсутствует");
```

```
      // При правильном выполнении программы в эту точку
      // попасть невозможно, поскольку в предыдущей строке
      // будет вызвано исключение

      cout << "Операция завершена успешно!" << endl;
      cout << "Закрытие ... ";
      db.Close();
      cout << "Закрыто!" << endl;
   }
   catch (CDBException* pEx)
   {
      // Вызвано исключение!
      // Необходимо напечатать сообщение об ошибке
      // (Используемые ниже функции не будут работать
      // без библиотеки MFC)

      char sz[1024];

      cout << "Ошибка: ";
      if (pEx->GetErrorMessage(sz, 1024))
         cout << sz;
      else
         cout << "Для данного исключения не предусмотрено сообщение
                  об ошибке ";
      cout << endl;

      pEx->Delete();
      return 1;
   }

   return 0;
}
```

CFHookProc

```
UINT CALLBACK CFHookProc(HWND hdlg, UINT uiMsg, WPARAM wParam,
  LPARAM lParam);
```

Возвращаемое значение

Ноль, если данное сообщение должна обработать стандартная функция его обработки в классе диалогового окна. Если функция `CFHookProc` возвращает ненулевое значение, стандартная функция обработки игнорирует данное сообщение.

Аргументы

`hdlg` — дескриптор диалогового окна **Шрифт**, сообщения которого обрабатывает данная функция обратного вызова.

uiMsg — идентификатор обрабатываемого сообщения.

wParam — содержит дополнительную информацию о сообщении. Трактовка данного аргумента зависит от значения аргумента uiMsg.

lParam — содержит дополнительную информацию о сообщении. Трактовка данного аргумента зависит от значения аргумента uiMsg. Если аргумент uiMsg имеет значение WM_INITDIALOG, то аргумент lParam содержит указатель на объект структуры CHOSEFONT, содержащий информацию, необходимую для инициализации диалогового окна.

Описание

Функция CFHookProc представляет собой созданную в приложении или библиотечную функцию обратного вызова, используемую в функции ChooseFont. Данная функция получает сообщение или извещение, предназначенное стандартному диалоговому окну **Шрифт**.

Тип LPCFHOOKPROC представляет собой указатель на функцию обратного вызова. Имя функции CFHookProc служит для замены имени функции, определенной в приложении.

Создавая диалоговое окно **Шрифт** с помощью функции ChooseFont, пользователь может передать ему указатель на функцию обратного вызова CFHookProc, которая будет обрабатывать сообщения, направленные данному диалоговому окну. Для задания функции обратного вызова используется объект структуры CHOOSEFONT, передаваемый диалоговому окну при его создании. Адрес функции обратного вызова помещается в переменную lpfnHook данной структуры и в переменной Flags устанавливается флаг CF_ENABLEHOOK.

Собственная процедура обработки сообщений диалогового окна обрабатывает сообщение WM_INITDIALOG до его передачи функции обратного вызова. Все другие сообщения сначала обрабатываются функцией обратного вызова и, в зависимости от значения, возвращенного ею, это сообщение обрабатывается или игнорируется собственной функцией обработки сообщения диалогового окна.

Если функция обратного вызова обрабатывает сообщение WM_CTLCOLORDLG, она должна возвращать корректный дескриптор кисти для заполнения фона диалогового окна. Вообще, если функция обратного вызова обрабатывает любое сообщение формата WM_CTLCOLOR*, она должна возвращать корректный дескриптор кисти для заполнения фона соответствующего элемента управления.

В функции обратного вызова не следует вызывать функцию EndDialog. Вместо этого надо при помощи функции PostMessage послать диалоговому окну сообщение WM_COMMAND с аргументом IDABORT. Это приведет к закрытию диалогового окна и возврату его функцией значения FALSE. Чтобы получить ин-

формацию о том, почему функция обратного вызова закрыла диалоговое окно, пользователю необходимо самому позаботиться о посылке соответствующих сообщений в приложение и их последующей обработке.

ChooseFont

```
BOOL ChooseFont(LPCHOOSEFONT lpcf);
```

Возвращаемое значение

Если пользователь нажимает кнопку **OK** в диалоговом окне **Шрифт**, возвращаемое значение не равно нулю. Выбор пользователя фиксируется в объекте структуры CHOOSEFONT.

Если пользователь нажимает кнопку **Cancel** или закрывает диалоговое окно **Шрифт (Font)**, данная функция возвращает нулевое значение.

Для получения дополнительной информации используется вызов функции CommDlgExtendedError, возвращающей следующие значения:

CDERR_FINDRESFAILURE, CDERR_INITIALIZATION, CDERR_LOCKRESFAILURE, CDERR_LOADRESFAILURE, CDERR_LOADSTRFAILURE, CDERR_MEMALLOCFAILURE, CDERR_MEMLOCKFAILURE, CDERR_NOHINSTANCE, CDERR_NOHOOK, CDERR_NOTEMPLATE, CDERR_STRUCTSIZE, CFERR_MAXLESSTHANMIN и CFERR_NOFONTS.

Аргументы

lpcf — указатель на объект структуры CHOOSEFONT, содержащий информацию об установках элементов управления в диалоговом окне.

Описание

Функция ChooseFont открывает стандартное диалоговое окно **Шрифт (Font)**, позволяющее пользователю установить атрибуты логического шрифта. Эти атрибуты включают в себя начертание, стиль (жирный, курсив или обычный), размер шрифта, эффекты (подчеркивание, зачеркивание или цвет) и набор символов шрифта.

В диалоговом окне **Шрифт** для обработки сообщений, посылаемых данному окну, может быть использован объект класса CFHookProc. Для использования данного объекта установите флаг CF_ENABLEHOOK в переменной Flags, являющейся членом структуры CHOOSEFONT, и поместите указатель на соответствующую функцию обратного вызова в переменную lpfnHook объект структуры CHOOSEFONT.

Функция обратного вызова может посылать диалоговому окну сообщения WM_CHOOSEFONT_GETLOGFONT, WM_CHOOSEFONT_SETFLAGS и WM_CHOOSEFONT_SETLOGFONT для получения информации о текущих установках его элементов управления.

CloseHandle

```
BOOL CloseHandle(HANDLE hObject);
```

Возвращаемое значение

Ненулевое, если функция успешно завершает свою работу, и нулевое — в противном случае. Расширенную информацию об ошибке можно получить, вызвав функцию `GetLastError`.

Аргументы

`hObject` — дескриптор открытого объекта.

Описание

Функция `CloseHandle` уничтожает дескриптор открытого объекта, уменьшает счетчик дескрипторов объекта и проверяет этот счетчик. Как только значение счетчика становится равным нулю, объект удаляется из системы.

Уничтожение дескриптора потока не приводит к завершению связанного с ним потока. Для уничтожения объекта потока необходимо сначала завершить выполнение потока, а затем уничтожить все его дескрипторы.

Функция `CloseHandle` используется для уничтожения дескрипторов, полученных при вызове функции `CreateFile`. Для уничтожения дескрипторов, возвращаемых функцией `FindFirstFile`, следует вызвать функцию `FindClose`.

Уничтожение недопустимого дескриптора вызывает исключение. Это происходит, например, при повторном уничтожении дескриптора.

CreateEvent

```
HANDLE CreateEvent(LPSECURITY_ATTRIBUTES lpEventAttributes,
  BOOL bManualReset, BOOL bInitialState, LPCTSTR lpName);
```

Возвращаемое значение

В случае успешного завершения функции возвращается дескриптор объекта события. Если именованный объект события существовал до вызова данной функции, то функция возвращает дескриптор существующего объекта, а функция `GetLastError` — значение `ERROR_ALREADY_EXISTS`.

Если в процессе выполнения данной функции возникает ошибка, то возвращается ноль. Расширенную информацию по ошибке можно получить, вызвав функцию `GetLastError`.

Аргументы

`lpEventAttributes` — указатель на объект структуры `SECURITY_ATTRIBUTES`, определяющую, может ли возвращаемый дескриптор наследоваться дочерним процессом. Если данный аргумент имеет нулевое значение, то дескриптор не может наследоваться. В Windows NT переменная

lpSecurityDescriptor, являющаяся членом структуры, определяет дескриптор безопасности нового объекта события. Если эта переменная равна нулю, то объект события использует дескриптор безопасности, заданный по умолчанию.

bManualReset — определяет ручной или автоматический сброс объекта события при его создании. Если этот аргумент имеет значение TRUE, то для сброса данного объекта события в неотмеченное состояние необходимо использовать функцию ResetEvent. В противном случае система автоматически сбрасывает состояние данного объекта события в неотмеченное после освобождения единственного ожидающего потока.

bInitialState — определяет начальное состояние объекта события. Если данный аргумент имеет значение TRUE, то данный объект события изначально отмечен.

lpName — указатель на заканчивающуюся нулем строку, содержащую имя объекта события. Длина данного имени ограничена величиной MAX_PATH и может содержать любые символы, кроме символа "\", используемого для разделения полей при задании пути к файлу. При сравнении имен учитывается регистр символов.

Если аргумент lpName совпадает с именем существующего именованного объекта, функция запрашивает доступ к данному объекту в режиме EVENT_ALL_ACCESS. В этом случае значения аргументов bManualReset и bInitialState игнорируются. Если аргумент lpEventAttributes имеет ненулевое значение, он определяет возможность наследования дескриптора, но его дескриптор безопасности игнорируется. Если аргумент lpName имеет значение NULL, то создается неименованный объект события.

Если аргумент lpName совпадает с именем семафора, объекта mutex, таймера ожидания, задания или объекта карты файла (file-mapping object), функция прекращает свою работу, а функция GetLastError возвращает значение ERROR_INVALID_HANDLE. Дело в том, что эти объекты разделяют одно и то же пространство имен.

Описание

Функция CreateEvent позволяет создавать именованные и неименованные объекты событий. Дескриптор, возвращаемый данной функцией, имеет право доступа EVENT_ALL_ACCESS к новому объекту события и может быть использован любой функцией, аргументом которой является дескриптор данного объекта.

Любой поток вызывающего процесса может использовать дескриптор объекта события при вызове любой из функций ожидания. Функции ожидания, использующие один объект, возвращают значение после того, как будет отмечен соответствующий объект события. Функции ожидания, использующие несколько объектов, могут быть настроены так, чтобы возвращать зна-

чение после того, как будет отмечен любой из связанных с нею объектов событий, или когда будут отмечены все объекты. После возвращения значения функцией ожидания ожидающий поток может продолжить свою работу.

Начальное состояние объекта события определяется аргументом `bInitialState`. Для установки объекта события в отмеченное состояние используется функция `SetEvent`, а для сброса отмеченного состояния — функция `ResetEvent`. Отмеченное состояние объекта события, созданного в режиме ручного сброса, сохраняется до вызова пользователем функции `ResetEvent`. Пока состояние объекта события остается отмеченным, может быть возобновлена работа любого количества ожидающих потоков.

Отмеченное состояние объекта события, созданного в режиме автоматического сброса, сохраняется до возобновления работы одного ожидающего потока. После этого система автоматически сбрасывает отмеченное состояние объекта события. Если ожидающие потоки отсутствуют, состояние объекта события остается отмеченным.

Несколько процессов могут использовать дескрипторы одного объекта события, позволяющего этим процессам осуществлять взаимную синхронизацию. Возможно использование следующих механизмов разделения объектов:

- Функция `CreateProcess` создает дочерний процесс, который наследует дескриптор объекта события, если значение аргумента `lpEventAttributes` в функции `CreateEvent` допускает это наследование
- Процесс может передать дескриптор объекта события в аргументе функции `DuplicateHandle`, создающей дубликат дескриптора, который может использоваться другим процессом
- Процесс может передать имя объекта события в аргументе функций `OpenEvent` или `CreateEvent`

Для уничтожения дескриптора используется функция `CloseHandle`. Система автоматически уничтожает дескриптор при завершении процесса. Объект события уничтожается при уничтожении его последнего дескриптора.

CreateFontIndirect

```
HFONT CreateFontIndirect(CONST LOGFONT *lplf);
```

Возвращаемое значение

В случае успешного завершения функции — дескриптор логического шрифта, и ноль — в противном случае. В Windows NT более подробную информацию об ошибке можно получить, вызвав функцию `GetLastError`.

Аргументы

`lplf` — указатель на объект структуры `LOGFONT`, содержащий описание логического шрифта.

Описание

Функция `CreateFontIndirect` позволяет создать логический шрифт по его параметрам, заданным в объекте структуры `LOGFONT`. После этого созданный шрифт может быть выбран в любой контекст устройства в качестве текущего шрифта.

При выборе созданного шрифта в контекст устройства функцией `SelectObject` интерфейс графических устройств Windows старается найти среди физических шрифтов такой, который бы максимально соответствовал параметрам заданного логического шрифта. Если ему не удается обеспечить полное соответствие, он использует шрифт, параметры которого максимально соответствуют требуемым.

После того как приложение закончило работу с данным логическим шрифтом, необходимо вызвать функцию `DeleteObject` для его уничтожения.

DeleteCriticalSection

```
VOID DeleteCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

Аргументы

`lpCriticalSection` — указатель на объект класса критической секции.

Описание

Функция `DeleteCriticalSection` освобождает все ресурсы, используемые указанным объектом критической секции, на который никто не должен иметь права собственности. После уничтожения объект класса критической секции не может использоваться в качестве аргумента функций `EnterCriticalSection`, `TryEnterCriticalSection` или `LeaveCriticalSection`.

EnterCriticalSection

```
VOID EnterCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

Аргументы

`lpCriticalSection` — указатель на объект класса критической секции.

Описание

Функция `EnterCriticalSection` ожидает возможности получить в безраздельную собственность указанный объект класса критической секции. Функция прекращает ожидание, когда вызывающему потоку предоставляется право собственности на данную секцию.

Потоки, принадлежащие одному процессу, могут использовать объект класса критической секции для синхронизации доступа к некоторому ресурсу. Этот процесс отвечает за выделение памяти, используемой объектом класса

критической секции. Для этого в процессе необходимо объявить переменную типа `CRITICAL_SECTION`. Перед использованием критических секций некоторые из потоков данного процесса должны вызвать функции `InitializeCriticalSection` или `InitializeCriticalSectionAndSpinCount` для инициализации объекта.

Для обеспечения исключительного права доступа к разделяемому ресурсу каждый поток должен вызвать функцию `EnterCriticalSection` или `TryEnterCriticalSection` для запроса на право собственности на критическую секцию до того, как приступит к исполнению любого фрагмента данной секции, осуществляющего доступ к разделяемому ресурсу.

Различие между этими функциями заключается в том, что функция `TryEnterCriticalSection` немедленно возвращает управление независимо от того, получила ли она право собственности на указанную критическую секцию, в то время как функция `EnterCriticalSection` прекращает выполнение потока до тех пор, пока не получит право собственности на критическую секцию. После завершения работы с критической секцией необходимо вызвать функцию `LeaveCriticalSection` для отказа от права собственности, что позволяет другим потокам получить право собственности над разделяемым ресурсом. Поток должен вызывать функцию `LeaveCriticalSection` каждый раз после того, как функция `EnterCriticalSection` или функция `TryEnterCriticalSection` предоставят ему право собственности на критическую секцию.

Получив право собственности на критическую секцию, поток может несколько раз вызвать функции `EnterCriticalSection` или `TryEnterCriticalSection` без блокировки своей работы. Это позволяет избежать зависания программы при ожидании потоком права собственности на критическую секцию, которой он уже обладает.

В любом из потоков данного процесса может быть вызвана функция `DeleteCriticalSection`, освобождающая системные ресурсы, предоставленные критической секции при ее инициализации. После вызова данной функции соответствующая критическая секция не может быть использована для синхронизации.

Если поток завершает свою работу, обладая правом собственности на критическую секцию, состояние критической секции становится неопределенным.

EnumFontFamilies

```
int EnumFontFamilies(HDC hdc, LPCTSTR lpszFamily,
  FONTENUMPROC lpEnumFontFamProc, LPARAM lParam);
```

Возвращаемое значение

Значение, возвращенное последней функцией обратного вызова. Трактовка этого значения зависит от контекста, в котором была вызвана данная функция.

Аргументы

hdc — дескриптор контекста устройства.

lpszFamily — указатель на заканчивающуюся нулем текстовую строку, определяющую имя семейства запрошенных шрифтов. Если аргумент lpszFamily имеет нулевое значение, то функция EnumFontFamilies случайным образом выбирает и нумерует по одному шрифту для каждого доступного семейства типов шрифтов.

lpEnumFontFamProc — определяет адрес экземпляра процедуры, определенной в приложении функцией обратного вызова. Функции обратного вызова описаны при рассмотрении функции EnumFontFamProc.

lParam — указатель на блок данных. Структура блока данных определяется приложением. Эти данные передаются функции обратного вызова вместе с информацией о шрифте.

Описание

Функция EnumFontFamilies нумерует шрифты в указанном семействе шрифтов, доступном на указанном устройстве.

Функции EnumFontFamilies и EnumFontFamProc оставлены для обеспечения совместимости с 16-разрядными версиями Windows. Приложения Win32 должны использовать функцию EnumFontFamiliesEx.

Функция EnumFontFamilies позволяет получить информацию по каждому шрифту, семейство которого указано в аргументе lpszFamily, и передать эту информацию функции, указанной в аргументе lpEnumFontFamProc. Определенная в приложении функция обратного вызова может производить любую обработку полученной информации о шрифте. Нумерация шрифтов продолжается до тех пор, пока не останется необработанных шрифтов, или пока функция обратного вызова не возвратит нулевое значение.

EnumFontFamiliesEx

```
int EnumFontFamiliesEx(HDC hdc, LPLOGFONT lpLogfont,
  FONTENUMPROC lpEnumFontFamExProc, LPARAM lParam, DWORD dwFlags);
```

Возвращаемое значение

Значение, возвращенное последней функцией обратного вызова. Трактовка этого значения зависит от того, какие семейства шрифтов присутствуют в данном устройстве.

Аргументы

hdc — дескриптор контекста устройства.

lpLogfont — указатель на объект структуры LOGFONT, содержащий информацию о шрифтах, которые следует пронумеровать. Данная функция проверяет значения следующих переменных данного объекта структуры:

- lfCharset — если данная переменная имеет значение DEFAULT_CHARSET, функция EnumFontFamiliesEx нумерует все шрифты, относящиеся ко всем семействам. Если данная переменная указывает на определенный набор символов, функция нумерует только шрифты данного набора символов
- lfFaceName — если эта переменная содержит пустую строку, функция нумерует только один шрифт для каждого начертания. Если строка, содержащаяся в данной переменной, является именем начертания, то функция EnumFontFamiliesEx нумерует все шрифты, относящиеся к данному начертанию
- lfPitchAndFamily — данная переменная должна содержать нулевое значение для всех языковых версий операционной системы, кроме еврейской и арабской. Для этих языков переменная IfPitchAndFamily должна содержать значение MONO_FONT, если необходимо нумеровать только те шрифты, которые содержат все коды символов данного языка

lpEnumFontFamProc — определяет адрес экземпляра процедуры, определенной в приложении функцией обратного вызова. Функции обратного вызова описаны при рассмотрении функции EnumFontFamProc.

lParam — указатель на блок данных. Структура блока данных определяется приложением. Эти данные передаются функции обратного вызова вместе с информацией о шрифте.

dwFlags — зарезервирован и должен содержать нулевое значение.

Описание

Функция EnumFontFamProc позволяет пронумеровать все шрифты в системе, соответствующие характеристикам шрифта, определенным в объекте структуры LOGFONT. Данная функция позволяет пронумеровать шрифты на основании имени начертания, набора символов или обоих этих признаков.

Функция EnumFontFamiliesEx не использует "ярлыки" для идентификации наборов символов. Вместо этого она всегда передает функции обратного вызова корректное имя начертания и отдельный номер набора символов. Функция выполняет нумерацию шрифтов, основываясь на значениях переменных lfCharset и lfFacename, являющихся членами структуры LOGFONT.

Если переменная lfCharset имеет значение DEFAULT_CHARSET, а переменная lfFaceName содержит пустую строку, функция нумерует по одному шрифту в каждом наборе символов каждого начертания. Если переменная lfFaceName содержит непустую строку, функция нумерует каждый шрифт указанного начертания независимо от используемого в нем набора символов.

Если переменная lfCharset содержит номер существующего набора символов, а переменная lfFaceName — пустую строку, функция нумерует все

шрифты в данном наборе символов. Если переменная lfFaceName содержит непустую строку, функция нумерует каждый шрифт указанного начертания, использующий указанный набор символов.

EnumFontFamProc

```
int CALLBACK EnumFontFamProc(ENUMLOGFONT FAR *lpelf,
  NEWTEXTMETRIC FAR *lpntm, int FontType, LPARAM lParam);
```

Возвращаемое значение

Для продолжения процедуры нумерации шрифтов данная функция должна возвращать ненулевое значение. Нумерация шрифтов останавливается, если функция возвращает ноль.

Аргументы

lpelf — указатель на объект структуры ENUMLOGFONT, содержащий информацию о логических атрибутах шрифта. Данная структура является локальной.

lpntm — указатель на объект структуры NEWTEXTMETRIC, содержащий информацию о физических атрибутах шрифта, если данный шрифт является шрифтом TrueType. Если шрифт не является шрифтом TrueType, данный аргумент является указателем на объект структуры TEXTMETRIC.

FontType — определяет тип шрифта. Этот аргумент может являться комбинацией следующих флагов:

- DEVICE_FONTTYPE
- RASTER_FONTTYPE
- TRUETYPE_FONTTYPE

lParam — указатель на блок данных, переданный функцией EnumFontFamilies. Структура данного блока определяется приложением.

Описание

Функция EnumFontFamProc представляет собой определяемую приложением функцию обратного вызова, используемую в функции EnumFontFamilies. Данная функция получает информацию об имеющихся в системе шрифтах. Указатель на нее имеет тип FONTENUMPROC. Имя EnumFontFamProc используется вместо определенного в приложении имени функции обратного вызова.

Функции EnumFontFamilies и EnumFontFamProc оставлены для обеспечения совместимости с 16-разрядными версиями Windows. Приложения Win32 должны использовать функцию EnumFontFamiliesEx.

Приложение должно зарегистрировать свою функцию обратного вызова, передав ее адрес функции EnumFontFamilies.

При определении типа шрифта для объединения флагов RASTER_FONTTYPE, DEVICE_FONTTYPE и TRUETYPE_FONTTYPE используется оператор поразрядного И (&).

Если установлен флаг RASTER_FONTTYPE, то данный шрифт является растровым шрифтом. Если установлен флаг TRUETYPE_FONTTYPE, то шрифт является шрифтом TrueType. Если не установлен ни один флаг, то шрифт является векторным шрифтом.

Флаг DEVICE_FONTTYPE устанавливается в том случае, когда устройство (например, лазерный принтер) позволяет загружать шрифты TrueType, или если приложение использует внутренние шрифты устройства. Этот флаг сбрасывается, если устройство является адаптером дисплея, матричным принтером или другим растровым устройством.

Приложение может установить флаг DEVICE_FONTTYPE, чтобы определить, поддерживает ли графический интерфейс устройства загружаемые графическим интерфейсом Windows (GDI) растровые шрифты. Графический интерфейс Windows позволяет эмулировать полужирные, наклоненные (курсивные), подчеркнутые и зачеркнутые растровые шрифты. Однако он не может этого сделать с внутренними шрифтами устройства.

GdiFlush

```
BOOL GdiFlush(VOID)
```

Возвращаемое значение

Если все функции в пакете успешно завершили свою работу, то функция возвращает ненулевое значение. В противном случае функция возвращает ноль.

Описание

Функция GdiFlush вызывает на исполнение пакет функций вызывающего потока.

Использование пакетной обработки позволяет повысить производительность приложения за счет уменьшения времени, необходимого для вызова функций GDI и оценки возвращаемых ими значений. Система готовит аргументы, необходимые для вызова этих функций, а затем вызывает группу функций для их пакетной обработки. Пакетная обработка начинается в следующих случаях:

- При вызове функции GdiFlush
- По достижении или превышении размера пакета, определенного функцией GdiSetBatchLimit
- При заполнении пакетных буферов
- При вызове любой функции GDI, не возвращающей логическое значение

Возвращаемое функцией GdiFlush значение относится только к тем функциям, которые присутствовали в пакете при вызове данной функции. Ошибки, возникающие при других способах запуска на пакетную обработку, никогда не выводятся.

Размер пакета определяется для каждого потока отдельно. Для полного отключения пакетной обработки вызовите функцию GdiSetBatchLimit(1) в процессе инициализации каждого потока.

Приложение должно вызывать функцию GdiFlush перед своим завершением, если имеется вероятность, что в пакете остались неисполненные графические функции, поскольку система не очищает пакет потока при его закрытии.

При использовании нескольких потоков в приложении, совместно использующих объекты GDI с объектом mutex, необходимо вызывать функцию GdiFlush всякий раз, когда данный поток уступает право собственности на объект GDI.

GetCommandLine

```
LPTSTR GetCommandLine(VOID)
```

Возвращаемое значение

Указатель на строку, содержащую командную строку текущего процесса.

Описание

Консольные процессы, написанные на языке C в соответствии со стандартом ANSI, могут использовать в функции main аргументы argc и argv, позволяющие получить доступ к аргументам командной строки. Приложения ANSI GUI могут использовать аргумент lpCmdLine функции WinMain для доступа к любым аргументам командной строки, за исключением имени программы. Функции main и WinMain не могут возвращать строки в формате Unicode, поскольку аргументы argc, argv и lpCmdLine имеют тип LPSTR, а не LPTSTR, позволяющий работать с текстами в формате Unicode. Поэтому для доступа к строкам в формате Unicode в данных функциях используется функция GetCommandLine, возвращающая значение типа LPTSTR.

GetCurrentDirectory

```
DWORD GetCurrentDirectory(DWORD nBufferLength, LPTSTR lpBuffer);
```

Возвращаемое значение

Если функция завершается успешно, возвращается количество символов, записанное в буфер, исключая завершающий строку нулевой символ. В противном случае возвращается нулевое значение. Расширенную информацию об ошибке можно получить, вызвав функцию GetLastError.

Если текстовый буфер, на который указывает аргумент lpBuffer, имеет недостаточный размер, возвращаемое значение определяет требуемый размер буфера, включая байты, необходимые для размещения завершающего строку нулевого символа.

Аргументы

nBufferLength — определяет размер буфера для хранения строки, содержащей текущий каталог. Размер буфера должен учитывать необходимость записи в него завершающего строку нулевого символа.

lpBuffer — указатель на буфер для хранения строки, содержащей текущий каталог. В данный буфер будет записана текстовая строка, завершающаяся нулевым символом, и содержащая абсолютный путь к текущему каталогу.

Описание

Функция GetCurrentDirectory позволяет получить путь к текущему каталогу данного процесса.

GetCurrentProcess

```
HANDLE GetCurrentProcess(VOID)
```

Возвращаемое значение

Псевдодескриптор текущего процесса.

Описание

Функция GetCurrentProcess возвращает псевдодескриптор текущего процесса, представляющий собой специальную константу, интерпретируемую как дескриптор текущего процесса. Вызывающий процесс может использовать возвращаемое данной функцией значение для указания дескриптора процесса в аргументах функций. Псевдодескриптор не может наследоваться дочерним процессом.

Этот дескриптор обеспечивает максимально возможный доступ к объекту процесса. Для операционных систем, использующих дескрипторы безопасности, уровень доступа определяется значением дескриптора безопасности. Для остальных систем это уровень доступа PROCESS_ALL_ACCESS.

Процесс может создать "настоящий" свой дескриптор, который может быть передан другим процессам или наследоваться другими процессами. Для этого необходимо использовать псевдодескриптор в качестве аргумента функции DuplicateHandle. Для той же цели процесс может вызвать функцию OpenProcess.

Псевдодескриптор не нужно уничтожать, после того как в нем отпала необходимость. Вызов функции CloseHandle для псевдодескриптора не приводит

ни к каким результатам. Если псевдодескриптор использовался в качестве аргумента функции DuplicateHandle, то полученный дубликат должен быть уничтожен, когда он будет не нужен.

GetDlgItem

```
HWND GetDlgItem(HWND hDlg, int nIDDlgItem);
```

Возвращаемое значение

В случае успешного завершения функции возвращается дескриптор окна указанного элемента управления. В противном случае возвращается нулевое значение, указывающее, что в качестве аргументов данной функции был использован дескриптор несуществующего диалогового окна или идентификатор несуществующего элемента управления. Расширенную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

hDlg — дескриптор диалогового окна, содержащего данный элемент управления.

nIDDlgItem — идентификатор элемента управления, дескриптор которого требуется получить.

Описание

Функция GetDlgItem позволяет получить дескриптор окна элемента управления в указанном диалоговом окне.

Данная функция может использоваться для любой пары родительского и диалогового окна, а не только в диалоговых окнах. Если аргумент hDlg определяет родительское окно, а дочернее окно имеет уникальный идентификатор (определяемый аргументом hMenu в функциях CreateWindow или CreateWindowEx при создании дочернего окна), функция GetDlgItem возвращает корректный дескриптор соответствующего дочернего окна.

GetExitCodeThread

```
BOOL GetExitCodeThread(HANDLE hThread, LPDWORD lpExitCode);
```

Возвращаемое значение

Ненулевое, если функция успешно завершает свою работу, и нулевое — в противном случае. Расширенную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

hThread — дескриптор потока. В Windows NT дескриптор должен иметь уровень доступа THREAD_QUERY_INFORMATION.

lpExitCode — указатель на 32-разрядную величину, в которую будет записан код завершения потока.

Описание

Функция GetExitCodeThread позволяет получить код завершения указанного потока.

Если указанный поток еще не завершил работу, код завершения имеет значение STILL_ACTIVE, а если завершил, то код завершения может принимать следующие значения:

- Код завершения, указанный в качестве аргумента функции ExitThread или TerminateThread
- Возвращаемое значение исполняющей функции потока
- Код завершения процесса потока

GetLastError

```
DWORD GetLastError(VOID)
```

Возвращаемое значение

Функция возвращает значение кода последней ошибки, возникшей в вызывающем потоке. Функция устанавливает это значение с помощью вызова функции SetLastError.

Поскольку функция SetLastError является 32-разрядной, функции Win32, представляющие собой скрытый вызов 16-разрядных функций, не устанавливают код последней ошибки. В этих функциях необходимо игнорировать данную величину. К подобным функциям относятся функции работы с окнами, функции GDI и функции работы с устройствами мультимедиа.

Описание

Функция GetLastError возвращает значение кода последней ошибки, возникшей в вызывающем потоке. Каждый поток имеет свой код последней ошибки.

Вызов функции GetLastError должен следовать немедленно после возврата любой функцией значения, свидетельствующего о том, что при ее выполнении произошла ошибка. Дело в том, что некоторые функции вызывают функцию SetLastError(0) в случае своего успешного завершения, а это приводит к уничтожению кода ошибки, установленного предыдущей функцией.

Большинство функций Win32 API устанавливают код последней ошибки при возникновении ошибки, но некоторые устанавливают его в случае своего нормального завершения. Ошибка при выполнении функции индицируется обычно следующими кодами ошибки: FALSE, NULL, 0xFFFFFFFF или -1.

Все случаи, когда функция вызывает функцию SetLastError при своем нормальном завершении, отмечены в справке по этой функции.

Коды ошибок представляют собой 32-разрядные величины (бит 31 является старшим битом). Бит 29 зарезервирован для кодов ошибок, определяемых пользователем, поэтому при определении собственного кода ошибки пользователь должен установить этот бит в единицу.

Чтобы получить строку, описывающую ошибку, по ее коду, нужно вызвать функцию FormatMessage.

GetMenuContextHelpId

```
DWORD GetMenuContextHelpId(HMENU hmenu);
```

Возвращаемое значение

Возвращает контекстный идентификатор справки, если такой идентификатор связан с данным меню, и ноль — в противном случае.

Аргументы

hmenu — дескриптор меню, для которого необходимо получить контекстный идентификатор справки.

Описание

Позволяет получить контекстный идентификатор справки для указанного меню.

GetMessage

```
BOOL GetMessage(LPMSG lpMsg, HWND hWnd, UINT wMsgFilterMin,
  UINT wMsgFilterMax);
```

Возвращаемое значение

Если данная функция извлекает сообщение, отличное от WM_QUIT, то возвращается ненулевая величина. В противном случае возвращается ноль. Если в процессе выполнения функции произошла ошибка, то возвращаемая величина равна −1. Например, ошибка может возникнуть, если аргумент hWnd не является дескриптором окна, или аргумент lpMsg не указывает на объект структуры MSG. Расширенную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

lpMsg — указатель на объект структуры MSG, в который будет записана информация из очереди сообщений потока.

hWnd — дескриптор окна, сообщения для которого следует получить. Этот аргумент может принимать нулевое значение. Это значение свидетельст-

вует о том, что функция GetMessage должна получать сообщения для любого окна, принадлежащего данному потоку, и сообщения от потока, посланные функцией PostThreadMessage.

wMsgFilterMin — определяет нижнюю границу диапазона отыскиваемых сообщений.

wMsgFilterMax — определяет верхнюю границу диапазона отыскиваемых сообщений.

Описание

Функция GetMessage получает сообщения из очереди сообщений вызвавшего ее потока и помещает их в объект специальной структуры. Данная функция позволяет получать сообщения для любого окна, принадлежащего данному потоку, и сообщения от потока, посланные функцией PostThreadMessage. При поиске сообщений выполняется проверка на нахождение их в определенном диапазоне значений. Функция GetMessage не может отыскивать сообщения, направленные другим окнам или приложениям.

Приложение использует возвращаемое данной функцией значение, чтобы определить, нет ли необходимости выйти из главного цикла сообщений приложения и завершить его работу.

Функция GetMessage позволяет получить сообщения, связанные с окном, определенным в аргументе hWnd, или с любым его дочерним окном, как это определено в функции IsChild. Получаемые сообщения должны находиться в диапазоне, заданном аргументами wMsgFilterMin и wMsgFilterMax. Если аргумент hWnd равен нулю, то функция GetMessage должна получать сообщения для любого окна, принадлежащего данному потоку, и сообщения от потока, посланные функцией PostThreadMessage. Даже если аргумент hWnd равен нулю, функция GetMessage не позволяет получать сообщения, предназначенные для окон, принадлежащих другим потокам, или от других потоков, кроме вызывающего. Сообщение от потока, направляемое функцией PostThreadMessage, имеет нулевое значение параметра hWnd. Если оба аргумента wMsgFilterMin и wMsgFilterMax равны нулю, то функция GetMessage позволяет получить все возможные сообщения (фильтрация сообщений отсутствует).

В качестве границ диапазона могут использоваться значения WM_KEYFIRST и WM_KEYLAST, позволяющие выделить все сообщения, связанные с клавиатурой, и значения WM_MOUSEFIRST и WM_MOUSELAST, отбирающие все сообщения, связанные с мышью.

Функция GetMessage не удаляет сообщение WM_PAINT из очереди сообщений. Оно остается там, пока не будет обработано. Поскольку данная функция может возвращать положительные, нулевые и отрицательные значения, то при написании программы следует избегать таких выражений, как

`while (GetMessage(lpMsg, hWnd, 0, 0)) ...`, поскольку данное выражение не реагирует на величину −1, свидетельствующую о возникновении фатальной ошибки в приложении.

GetModuleHandle

```
HMODULE GetModuleHandle(LPCTSTR lpModuleName);
```

Возвращаемое значение

В случае успешного завершения работы функции возвращается дескриптор указанного модуля. В противном случае возвращается ноль. Расширенную информацию об ошибке можно получить, вызвав функцию `GetLastError`.

Аргументы

`lpModuleName` — указатель на завершающуюся нулем строку, содержащую имя модуля (файла .dll или .exe). Если при указании имени файла опущено его расширение, то этот файл рассматривается как файл библиотеки динамической компоновки и к его имени добавляется расширение .dll. Чтобы указать, что имя файла данного модуля не имеет расширения, необходимо после него поставить точку (.). Данная строка может не содержать пути к данному файлу. При указании пути к файлу для разделения имен каталогов и подкаталогов следует использовать символ "\". Это имя сравнивается с именами модулей, размещенных в адресном пространстве текущего процесса. При сравнении не учитывается регистр используемых символов.

Если данный аргумент равен нулю, то функция `GetModuleHandle` возвращает дескриптор файла, использованного для создания текущего процесса.

Описание

Функция `GetModuleHandle` возвращает дескриптор модуля, если дескриптор файла данного модуля расположен в адресном пространстве вызывающего процесса.

Возвращаемый дескриптор не является глобальным, наследуемым или копируемым и не может использоваться другими процессами.

Дескрипторы, возвращаемые функциями `GetModuleHandle` и `LoadLibrary`, могут использоваться в функциях `GetProcAddress`, `FreeLibrary` и `LoadResource`. Различие между этими функциями состоит в том, что функция `LoadLibrary` может, в случае необходимости, добавить модуль в адресное пространство данного процесса и увеличить значение счетчика ссылок данного модуля. Функция `GetModuleHandle` позволяет только получить дескриптор уже размещенного в оперативной памяти модуля без увеличения счетчика ссылок.

Счетчик ссылок используется функцией FreeLibrary, чтобы определить, нужно ли освобождать область памяти, занимаемую данным модулем после его завершения. Поэтому необходимо с особой осторожностью использовать дескриптор, полученный с использованием функции GetModuleHandle в качестве аргумента функции FreeLibrary, поскольку это может привести к преждевременному удалению из памяти библиотеки динамической компоновки.

Функцию GetModuleHandle следует с осторожностью вызывать в приложениях, использующих несколько параллельно работающих потоков, поскольку никто не может гарантировать, что возвращенный ею дескриптор модуля останется корректным по истечении того времени, которое пройдет до его использования в другой функции. Например, поток может, вызвав данную функцию, получить дескриптор модуля, но перед тем как данный процесс успеет использовать полученную величину в другой функции, другой процесс может освободить этот модуль и загрузить другой, присвоив ему дескриптор освобожденного модуля. В этом случае первый поток будет работать с дескриптором совсем другого модуля, чем ожидалось. Последствия такой подмены модулей непредсказуемы.

GetWindowContextHelpId

```
DWORD GetWindowContextHelpId(HWND hwnd);
```

Возвращаемое значение

Возвращает контекстный идентификатор справки, если такой идентификатор связан с данным окном, и ноль — в противном случае.

Аргументы

hwnd — дескриптор окна, для которого необходимо получить контекстный идентификатор справки.

Описание

Позволяет получить контекстный идентификатор справки для указанного окна, если с данным окном связан контекстный идентификатор справки.

GetWindowLong

```
LONG GetWindowLong(HWND hWnd, int nIndex);
```

Возвращаемое значение

В случае успешного завершения операции возвращается искомая 32-разрядная величина. В противном случае возвращается ноль. Расширенную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

hWnd — дескриптор окна и, косвенно, класса, связанного с данным окном.

nIndex — определяет смещение величины, которую следует получить. Величина смещения может принимать значение в диапазоне от нуля до размера дополнительной памяти окна, выраженной в байтах, минус четыре. Например, при задании 12 или более байт в качестве дополнительной памяти, величина 8 будет являться индексом третьего 32-разрядного целого числа. Для доступа к величинам, характеризующим некоторые параметры окна, существуют предопределенные величины. Ниже приведен их список и возвращаемые функцией значения:

- GWL_EXSTYLE — расширенный стиль окна
- GWL_STYLE — стиль окна
- GWL_WNDPROC — указатель на процедуру обработки окна или дескриптор, позволяющий получить доступ к этому указателю
- GWL_HINSTANCE — дескриптор экземпляра (instance) данного приложения
- GWL_HWNDPARENT — дескриптор родительского окна, если таковое имеется
- GWL_ID —идентификатор окна
- GWL_USERDATA — 32-разрядная величина, связанная с окном. Эта величина предназначена для использования приложением, связанным с окном

Если аргумент hWnd является дескриптором диалогового окна, второй параметр может дополнительно принимать следующие значения:

- DWL_DLGPROC — возвращает указатель на процедуру обработки диалогового окна или дескриптор, позволяющий получить доступ к указателю на эту процедуру
- DWL_MSGRESULT — возвращает значение процедуры обработки диалогового окна
- DWL_USER — возвращает дополнительную информацию, содержание которой специфично для каждого приложения. В качестве такой информации могут выступать указатели и дескрипторы

Описание

Функция GetWindowLong позволяет получать информацию об указанном окне. Эта информация представляет собой 32-разрядные величины, хранящиеся в дополнительной памяти данного окна.

Для резервирования дополнительной памяти окна необходимо задать ненулевое значение величины cbWndExtra, являющейся членом структуры WNDCLASSEX, используемой функцией RegisterClassEx.

GetWindowRect

```
BOOL GetWindowRect(HWND hWnd, LPRECT lpRect);
```

Возвращаемое значение

Ненулевое, если функция завершается успешно, и ноль — в противном случае. Расширенную информацию об ошибке можно получить при помощи функции `GetLastError`.

Аргументы

`hWnd` — дескриптор окна.

`lpRect` — указатель на объект структуры `RECT`, содержащий экранные координаты верхнего левого и нижнего правого углов окна.

Описание

Функция `GetWindowRect` позволяет получить координаты прямоугольника, описывающего указанное окно. Координаты окна измеряются в экранных координатах, нулевая точка которых расположена в левом верхнем углу экрана.

InitializeCriticalSection

```
VOID InitializeCriticalSection(LPCRITICAL_SECTION);
```

Аргументы

`lpCriticalSection` — указатель на объект класса критической секции.

Описание

Функция `InitializeCriticalSection` инициализирует объект класса критической секции. Потоки, принадлежащие одному процессу, могут использовать объект класса критической секции для синхронизации доступа к некоторому ресурсу. Хотя невозможно предсказать порядок обретения отдельными потоками прав собственности на критическую секцию, система старается быть объективной при распределении прав собственности.

Этот процесс отвечает за выделение памяти, используемой объектом класса критической секции. Для этого в процессе необходимо объявить переменную типа `CRITICAL_SECTION`. Перед использованием критических секций некоторые из потоков данного процесса должны вызвать функцию `InitializeCriticalSection` или `InitializeCriticalSectionAndSpinCount` для инициализации объекта.

После инициализации объекта критической секции потоки процесса могут использовать данный объект в качестве аргумента функций `EnterCriticalSection`, `TryEnterCriticalSection` или `LeaveCriticalSection` для

обеспечения взаимного доступа к разделяемому ресурсу в режиме исключительного права собственности на данный ресурс. Для подобной синхронизации потоков, принадлежащих к различным процессам, используется объект mutex.

Объект класса критической секции не может перемещаться или копироваться. Процесс не должен также вносить изменения в данный объект, а должен рассматривать его как "черный ящик". Для работы с критическими секциями следует использовать только функции, предоставляемые для этого Win32 API.

В случае недостаточного объема памяти для выполнения функции `InitializeCriticalSection` вызывается исключение `STATUS_NO_MEMORY`.

LeaveCriticalSection

```
VOID LeaveCriticalSection(LPCRITICAL_SECTION lpCriticalSection);
```

Аргументы

`lpCriticalSection` — указатель на объект класса критической секции.

Описание

Функция `LeaveCriticalSection` отказывает данному потоку в праве собственности на указанный объект класса критической секции.

Потоки, принадлежащие одному процессу, могут использовать объект класса критической секции для синхронизации доступа к некоторому ресурсу. Этот процесс отвечает за выделение памяти, используемой объектом класса критической секции. Для этого в процессе необходимо объявить переменную типа `CRITICAL_SECTION`. Перед использованием критических секций некоторые из потоков данного процесса должны вызвать функцию `InitializeCriticalSection` или `InitializeCriticalSectionAndSpinCount` для инициализации объекта.

Для обеспечения исключительного права доступа к разделяемому ресурсу каждый поток должен вызвать функцию `EnterCriticalSection` или `TryEnterCriticalSection` для запроса на право собственности на критическую секцию. Поток должен вызывать функцию `LeaveCriticalSection` каждый раз после того, как функция `EnterCriticalSection` или функция `TryEnterCriticalSection` предоставят ему право собственности на критическую секцию.

Если поток вызывает функцию `LeaveCriticalSection` для объекта класса критической секции, к которой у него нет доступа, возникает ошибка, которая может привести к тому, что другой поток будет бесконечно ждать выхода из функции `EnterCriticalSection`.

В любом из потоков данного процесса может быть вызвана функция `DeleteCriticalSection`, освобождающая системные ресурсы, предоставлен-

ные критической секции при ее инициализации. После вызова данной функции соответствующая критическая секция не может быть использована для синхронизации.

MessageBox

```
int MessageBox(HWND hWnd, LPCTSTR lpText, LPCTSTR lpCaption,
  UINT uType);
```

Возвращаемое значение

Ноль, если в системе недостаточно оперативной памяти для создания окна сообщения. В случае успешного завершения функции возвращаемая данной функцией величина может принимать одно из перечисленных ниже значений:

- IDABORT — нажата кнопка **Abort**
- IDCANCEL — нажата кнопка **Cancel**
- IDIGNORE — нажата кнопка **Ignore**
- IDNO — нажата кнопка **No**
- IDOK — нажата кнопка **OK**
- IDRETRY — нажата кнопка **Retry**
- IDYES — нажата кнопка **Yes**

Если в окне сообщения есть кнопка **Cancel**, функция возвращает значение IDCANCEL независимо от того, была ли нажата кнопка **Cancel** или клавиша <Esc>. Если окно сообщения не имеет кнопки **Cancel**, то нажатие клавиши <Esc> не приводит ни к каким результатам.

Аргументы

hWnd — определяет окно, являющееся собственником создаваемого окна сообщения. Если этот аргумент имеет значение NULL, то создаваемое окно сообщения не имеет собственника.

lpText — указатель на текстовую строку, завершающуюся нулевым символом, содержащую выводимое сообщение.

lpCaption — указатель на текстовую строку, завершающуюся нулевым символом, используемую в качестве заголовка диалогового окна. Если этот аргумент имеет значение NULL, то создаваемое окно содержит стандартный заголовок "Error".

uType — задает комбинацию битовых флагов, определяющих содержимое и поведение диалогового окна. Этот аргумент может представлять собой комбинацию флагов из приведенных ниже групп (не более одного из каждой группы).

Флаги, определяющие состав кнопок окна сообщения:

- MB_ABORTRETRYIGNORE — окно сообщения содержит три кнопки: **Abort**, **Retry** и **Ignore**
- MB_OK — окно сообщения содержит одну кнопку **OK**. Это установка по умолчанию
- MB_OKCANCEL — окно сообщения содержит две кнопки: **OK** и **Cancel**
- MB_RETRYCANCEL — окно сообщения содержит две кнопки: **Retry** и **Cancel**
- MB_YESNO — окно сообщения содержит две кнопки: **Yes** и **No**
- MB_YESNOCANCEL — окно сообщения содержит три кнопки: **Yes**, **No** и **Cancel**

Флаги, определяющие значок, выводимый в окне сообщения:

- MB_ICONEXCLAMATION, MB_ICONWARNING — в окне сообщения появляется восклицательный знак
- MB_ICONINFORMATION, MB_ICONASTERISK — в окне сообщения появляется буква "*i*", помещенная в кружок
- MB_ICONQUESTION — в окне сообщения появляется вопросительный знак
- MB_ICONSTOP, MB_ICONERROR, MB_ICONHAND — в окне сообщения появляется сигнал остановки

Флаги, задающие кнопку, используемую по умолчанию:

- MB_DEFBUTTON1 — по умолчанию используется первая кнопка. Если в данном аргументе не установлен флаг этой группы, то кнопкой по умолчанию является первая кнопка
- MB_DEFBUTTON2 — по умолчанию используется вторая кнопка
- MB_DEFBUTTON3 — третья кнопка
- MB_DEFBUTTON4 — четвертая кнопка

Флаги, определяющие модальность диалогового окна:

- MB_APPLMODAL — пользователь должен закончить работу с окном сообщения, прежде чем он получит возможность продолжить работу с окном, определенным в аргументе hWnd. В зависимости от иерархии окон в приложении пользователь сможет иметь возможность продолжить работу с другими окнами данного потока. Все дочерние окна, принадлежащие родительскому окну сообщения, становятся недоступными, но может быть продолжена работа со вспомогательными окнами. Этот режим устанавливается в том случае, если в данном аргументе не установлен флаг этой группы
- MB_SYSTEMMODAL — действует аналогично флагу MB_APPLMODAL за тем исключением, что окно сообщения получает стиль WS_EX_TOPMOST. Дан-

ный флаг используется для извещения пользователя о серьезных ошибках, требующих его немедленной реакции (например, выход за границы памяти). Этот флаг не оказывает никакого влияния на возможности пользователя по работе с окнами, не связанными с окном, определенным в аргументе hWnd

- MB_TASKMODAL — действует аналогично флагу MB_APPLMODAL за тем исключением, что если аргумент hWnd равен нулю, то пользователь теряет возможность доступа ко всем основным и дочерним окнам, принадлежащим данному потоку. Данный флаг используется, если приложение или библиотека не имеют дескриптора окна, но должны прекратить доступ ко всем окнам данного потока, не препятствуя работе других потоков

Кроме перечисленных выше флагов, объединенных в группы, программист может использовать следующие флаги:

- MB_DEFAULT_DESKTOP_ONLY — рабочий стол, получивший фокус ввода, должен быть рабочим столом, выбираемым по умолчанию, т. е. сразу после загрузки системы. В противном случае функция завершается с ошибкой
- MB_HELP — добавляет в окно сообщения кнопку **Help**. Нажатие кнопки **Help** или клавиши <F1> отмечает соответствующее событие
- MB_RIGHT — текст в окне сообщения выравнивается по правому краю
- MB_RTLREADING — выводит текст сообщения и заголовка справа налево, как это принято в иврите и арабском языке
- MB_SETFOREGROUND — делает активным поток, создающий окно сообщения, и устанавливает в него фокус ввода. Для этого система вызывает для окна сообщения функцию SetForegroundWindow
- MB_TOPMOST — окно сообщения создается с флагом WS_EX_TOPMOST
- MB_SERVICE_NOTIFICATION — флаг, специфический для Windows NT, означающий, что данное окно вызывает служба, извещая пользователя о событии. Данная функция отображает окно сообщения на текущем активном рабочем столе, даже если на данном компьютере не зарегистрирован ни один пользователь. Если установлен данный флаг, то аргумент hWnd должен иметь нулевое значение. Это означает, что окно сообщения может появиться на любом рабочем столе, а не обязательно на том, которому принадлежит окно, определяемое аргументом hWnd

  В Windows NT 4.0 значение MB_SERVICE_NOTIFICATION изменено. Новые и старые значения описаны в файле WinUser.h. Операционная система Windows NT 4.0 обеспечивает совместимость снизу вверх для существующих служб, преобразуя старые значения в новые при работе с функциями MessageBox и MessageBoxEx. Это преобразование выпол-

няется только для тех исполнительных файлов, которые имеют номер версии, установленный компоновщиком, меньший, чем 4.0

- MB_SERVICE_NOTIFICATION_NT3X — флаг, специфический для Windows NT, имеет значение MB_SERVICE_NOTIFICATION, используемое в Windows NT версии 3.51

Описание

Функция MessageBox создает, отображает и поддерживает интерфейс с окном сообщения. Окно сообщения состоит из определяемого приложением текста, заголовка, предопределенного значка и набора кнопок.

При использовании модального системного окна сообщения для вывода информации о том, что в системе не хватает памяти, значения аргументов lpText и lpCaption нельзя загружать из файла ресурсов (что, кстати сказать, разрушает всю концепцию использования строковых ресурсов), поскольку в данном случае попытка загрузить ресурс может закончиться безрезультатно.

Если приложение вызывает функцию MessageBox и указывает в аргументе uType комбинацию флагов MB_ICONHAND и MB_SYSTEMMODAL, операционная система отображает окно сообщения вне зависимости от того, имеется в системе свободная память или нет. При данной комбинации флагов операционная система ограничивает размер сообщения тремя строками. Система не выполняет автоматического разбиения текста на строки. Для этого необходимо использовать управляющие символы перевода строки.

Если окно сообщения создается в классе диалогового окна, то в качестве аргумента hWnd должен выступать дескриптор диалогового окна. В качестве данного аргумента нельзя использовать дескриптор дочернего окна, например, дескриптор окна элемента управления.

В Windows 95 максимальное число дескрипторов окон составляет 16 384.

OpenEvent

```
HANDLE OpenEvent(DWORD dwDesiredAccess, BOOL bInheritHandle,
  LPCTSTR lpName);
```

Возвращаемое значение

В случае успешного завершения функция возвращает дескриптор объекта события. Если в процессе работы функции возникла ошибка, возвращается нулевое значение. Более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

dwDesiredAccess — определяет режим доступа к объекту события. В системах, обеспечивающих безопасность объектов, данная функция аварийно

завершает свою работу, если дескриптор указанного объекта не допускает использование указанного режима доступа для вызывающего процесса. Этот аргумент может принимать одно из следующих значений:

- EVENT_ALL_ACCESS — устанавливает все возможные флаги доступа к объекту события
- EVENT_MODIFY_STATE — обеспечивает возможность использования дескриптора объекта события в функциях SetEvent и ResetEvent, изменяющих состояние объекта события
- SYNCHRONIZE — специфичен для Windows NT и разрешает использовать дескриптор объекта события в любой из функций ожидания для задания состояния объекта

bInheritHandle — определяет возможность наследования возвращаемого дескриптора. Если этот аргумент имеет значение TRUE, то процесс, созданный функцией CreateProcess, может наследовать дескриптор. В противном случае дескриптор не может наследоваться.

lpName — указатель на заканчивающуюся нулем строку, содержащую имя открываемого объекта события. При сравнении имен учитывается регистр используемых символов.

Описание

Функция OpenEvent возвращает дескриптор существующего именованного объекта события. Позволяет нескольким процессам открывать дескрипторы одного и того же объекта события. Функция нормально завершает свою работу, только если уже существует объект события с таким именем, созданный другим процессом с помощью функции CreateEvent. Вызывающий процесс может передать возвращаемый данной функцией дескриптор в качестве аргумента любой функции, использующей дескриптор объекта события, при условии соблюдения ограничений, налагаемых значением аргумента dwDesiredAccess.

Дескриптор может быть дублирован при помощи функции DuplicateHandle. Для уничтожения дескриптора вызывается функция CloseHandle. Система автоматически уничтожает дескриптор при завершении процесса. Объект события уничтожается вместе с его последним дескриптором.

PulseEvent

```
BOOL PulseEvent(HANDLE hEvent);
```

Возвращаемое значение

Ненулевое, если работа функции завершилась успешно. Если в процессе работы функции возникла ошибка, возвращается ноль. Более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

hEvent — дескриптор объекта события, возвращенный функцией CreateEvent или OpenEvent. В Windows NT этот аргумент должен иметь уровень доступа EVENT_MODIFY_STATE.

Описание

Функция PulseEvent служит для установки в отмеченное состояние указанного объекта события и последующего сброса его в неотмеченное состояние после запуска определенного числа ожидающих потоков.

Для объектов событий, созданных в режиме ручного сброса, все ожидающие потоки, которые могут быть немедленно запущены, запускаются на исполнение. После этого функция сбрасывает объект события в неотмеченное состояние и завершает свою работу.

Для объектов событий, созданных в режиме автоматического сброса, функция запускает один ожидающий поток, даже если в режиме ожидания находится несколько потоков, сбрасывает объект события в неотмеченное состояние и завершает свою работу.

Если в приложении отсутствуют ожидающие потоки, функция PulseEvent просто сбрасывает объект события в неотмеченное состояние и завершает свою работу.

В случае применения функций ожидания, работающих с несколькими объектами события и настроенными на срабатывание по установке всех этих объектов в отмеченное состояние, функция PulseEvent сбросит отмеченное состояние объекта события независимо от того, сработала данная функция или нет.

RegisterClass

```
ATOM RegisterClass(CONST WNDCLASS *lpWndClass);
```

Возвращаемое значение

При успешном завершении процедуры регистрации данная функция возвращает атом, однозначно идентифицирующий зарегистрированный класс. В противном случае функция возвращает ноль. Подробную информацию об ошибке можно получить с помощью вызова функции GetLastError.

Аргументы

lpWndClass — указатель на объект структуры WNDCLASS. Перед использованием объекта данной структуры в функции его необходимо заполнить информацией о соответствующих атрибутах класса.

Описание

Функция RegisterClass позволяет зарегистрировать класс окна для его дальнейшего использования при вызове функций CreateWindow или CreateWindowEx.

На смену данной функции пришла функция RegisterClassEx, однако, если нет необходимости задавать для класса маленький значок, можно продолжать пользоваться функцией RegisterClass.

Если класс окна регистрируется функцией RegisterClassA, приложение сообщает системе, что создаваемый класс окна ожидает сообщений с текстовыми параметрами, использующими набор символов ANSI. Если класс окна регистрируется функцией RegisterClassW, приложение сообщает системе, что создаваемый класс окна ожидает сообщений с текстовыми параметрами, использующими набор символов Unicode. Функция IsWindowUnicode позволяет определить набор символов, используемых в каждом окне.

ReleaseCapture

```
BOOL ReleaseCapture(VOID)
```

Возвращаемое значение

Ненулевое, если сохранение прошло успешно. Если в процессе сохранения возникла ошибка, данная функция возвращает ноль. Более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Описание

Функция ReleaseCapture восстанавливает нормальную процедуру обработки сообщений, поступающих от мыши. Окно, захватившее мышь, получает все сообщения, поступающие от нее, независимо от того, где в данный момент находится курсор мыши. Единственным сообщением мыши, обрабатываемым по обычной процедуре, является щелчок в окне, принадлежащем другому потоку.

Приложение вызывает данную функцию после вызова функции SetCapture. В операционной системе Windows 95 вызов функции ReleaseCapture приводит к тому, что окно, освобождающее мышь, получает сообщение WM_CAPTURECHANGED.

RemoveFontResource

```
BOOL RemoveFontResource(LPCTSTR lpFileName);
```

Возвращаемое значение

В случае успешного завершения функции возвращается ненулевое значение. В противном случае возвращается ноль. В Windows NT более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

lpFileName — указатель на заканчивающуюся нулем строку, содержащую имя файла ресурса шрифта.

Описание

Функция RemoveFontResource удаляет ресурс из системной таблицы шрифтов, записывая его в указанный файл.

Любые приложения, добавляющие или удаляющие шрифты из системной таблицы шрифтов, извещают об этом другие приложения, посылая сообщение WM_FONTCHANGE всем окнам верхнего уровня в операционной системе. Для этого приложение должно вызвать функцию SendMessage, в аргументе hWnd которой должно стоять значение HWND_BROADCAST.

Если на данный ресурс шрифта имеются ссылки из других приложений, то этот ресурс не будет выгружен, пока не останется использующих его контекстов устройств.

ResetEvent

```
BOOL ResetEvent(HANDLE hEvent);
```

Возвращаемое значение

Ненулевое, если работа функции завершилась успешно. Если в процессе работы функции возникла ошибка, возвращается ноль. Более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

hEvent — дескриптор объекта события, возвращенный функцией CreateEvent или OpenEvent. В Windows NT этот аргумент должен иметь уровень доступа EVENT_MODIFY_STATE.

Описание

Функция ResetEvent сбрасывает отмеченное состояние объекта события. Состояние объекта события после этого остается неотмеченным до следующего вызова функции SetEvent или PulseEvent. Это неотмеченное состояние блокирует выполнение любых потоков, указавших данный объект события в аргументе функций ожидания.

Данная функция используется преимущественно для ручного сброса состояния объектов. Объекты состояния, для которых определен автоматический сброс, сбрасываются после запуска первого же ожидающего потока.

SendMessage

```
LRESULT SendMessage(HWND hWnd, UINT Msg, WPARAM wParam, LPARAM lParam);
```

Возвращаемое значение

Возвращаемое значение определяет результат посылки сообщения, и его трактовка зависит от посланного сообщения.

Аргументы

hWnd — дескриптор окна, процедура окна которого должна получить данное сообщение. Если аргумент имеет значение HWND_BROADCAST, посылаемое сообщение будет послано всем окнам верхнего уровня в системе, включая недоступные или невидимые окна без владельцев, перекрывающиеся и всплывающие окна. Но данное сообщение не получат дочерние окна.

Msg — определяет посылаемое сообщение.

wParam — определяет дополнительную информацию, посылаемую с сообщением.

lParam — определяет дополнительную информацию, посылаемую с сообщением.

Описание

Функция SendMessage посылает указанное сообщение окну или окнам. Она вызывает оконную процедуру для указанного окна и не возвращает управление, пока данное сообщение не будет обработано. Ее поведение отличается от поведения функции PostMessage, помещающей сообщение в очередь сообщений потока и немедленно возвращающей управление.

Приложение, использующее при посылке сообщения значение HWND_BROADCAST, должно вызвать функцию RegisterWindowMessage, чтобы получить уникальное сообщение для обмена внутри приложения.

Если указанное окно было создано вызывающим потоком, процедура окна вызывается немедленно как подпрограмма. Если указанное окно было создано другим потоком, система переключается на данный поток и вызывает оконную процедуру. Сообщения, посылаемые между потоками, обрабатываются, если получающий поток вызовет функцию приема сообщения. Выполнение посылающего потока при этом приостанавливается, и он возобновляет свою работу только после того, как принимающий поток обработает сообщение.

SetCurrentDirectory

```
BOOL SetCurrentDirectory(LPCTSTR lpPathName);
```

Возвращаемое значение

Ненулевое, если сохранение прошло успешно. Если в процессе сохранения возникла ошибка, данная функция возвращает ноль. Более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

lpPathName — указатель на заканчивающуюся нулем текстовую строку, содержащую путь в новый рабочий каталог. Этот параметр может быть как

относительным, так и полным путем. В любом случае по данному аргументу полный путь в каталог определяется и запоминается как текущий.

Описание

Функция `SetCurrentDirectory` изменяет рабочий каталог для текущего процесса.

Каждый процесс имеет единственный рабочий каталог, имя которого состоит из двух частей:

- Имени диска, представляющего собой букву, соответствующую данному диску, за которой стоит символ двоеточия, или имя сервера и разделяемое имя (\\servername\sharename)
- Имени каталога на этом диске

SetEvent

```
BOOL SetEvent(HANDLE hEvent);
```

Возвращаемое значение

Ненулевое, если работа функции завершилась успешно. Если в процессе работы функции возникла ошибка, возвращается ноль. Более подробную информацию об ошибке можно получить, вызвав функцию `GetLastError`.

Аргументы

`hEvent` — дескриптор объекта события, возвращенный функцией `CreateEvent` или `OpenEvent`. В Windows NT этот аргумент должен иметь уровень доступа `EVENT_MODIFY_STATE`.

Описание

Функция `SetEvent` устанавливает отмеченное состояние объекта события. Состояние объекта события, открытого в режиме ручного сброса, после этого остается отмеченным до следующего вызова функции `ResetEvent`. Пока состояние объекта события остается отмеченным, может быть возобновлена работа любого количества ожидающих потоков или потоков, которые последовательно запускали операцию ожидания с использованием данного объекта события.

Состояние объекта события, открытого в режиме автоматического сброса, остается отмеченным до запуска первого же ожидающего потока. После этого система автоматически устанавливает неотмеченное состояние данного объекта. Если ожидающие потоки отсутствуют, состояние объекта остается отмеченным.

SetMenuContextHelpId

```
BOOL SetMenuContextHelpId(HMENU hmenu, DWORD dwContextHelpId);
```

Возвращаемое значение

Ненулевое, если функция успешно завершила свою работу, и ноль — в противном случае. Более подробную информацию об ошибке можно получить, вызвав функцию `GetLastError`.

Аргументы

`hmenu` — дескриптор меню, связанного с данным контекстным идентификатором.

`dwContextHelpId` — контекстный идентификатор справки.

Описание

Связывает контекстный идентификатор справки с указанным меню. Все команды меню разделяют этот идентификатор. Контекстный идентификатор справки не может быть назначен отдельной команде меню.

SetPriorityClass

```
BOOL SetPriorityClass(HANDLE hProcess, DWORD dwPriorityClass);
```

Возвращаемое значение

Ненулевое, если функция успешно завершила свою работу, и ноль — в противном случае. Более подробную информацию об ошибке можно получить, вызвав функцию `GetLastError`.

Аргументы

`hProcess` — дескриптор процесса. В Windows NT данный дескриптор должен иметь уровень доступа `PROCESS_SET_INFORMATION`.

`dwPriorityClass` — определяет класс приоритета процесса. Данный аргумент может принимать одно из следующих значений:

- `HIGH_PRIORITY_CLASS` — данный класс приоритета используется для критичных ко времени процессов, которые должны исполняться незамедлительно. Потоки данного процесса имеют преимущество перед потоками с нормальным и фоновым классами приоритета. Примером такого процесса является список задач, который должен появляться сразу после запроса пользователя независимо от загрузки операционной системы. Этот класс приоритета нужно использовать с осторожностью, поскольку его потоки могут занять практически все время центрального процессора

- IDLE_PRIORITY_CLASS — данный класс приоритета используется для процессов, выполняемых в фоновом режиме. Потоки данного процесса имеют низший приоритет по сравнению с потоками любого другого класса приоритета. Примером такого процесса может быть заставка экрана (screen saver). Наследуется дочерними процессами
- NORMAL_PRIORITY_CLASS — используется для процессов, не предъявляющих никаких специальных требований к своему приоритету
- REALTIME_PRIORITY_CLASS — для процессов, имеющих наивысший уровень приоритета. Потоки данного класса имеют приоритет над потоками любого другого процесса, включая процессы операционной системы. Выполнение процесса реального времени может привести к нарушению процесса хеширования диска или прекращению работы с мышью

Описание

Функция SetPriorityClass устанавливает класс приоритета указанного процесса. Это значение вместе с уровнем приоритета потока определяет базовый уровень приоритета потока.

Каждый поток имеет базовый уровень приоритета, определяемый значением приоритета потока и классом приоритета его процесса. Система использует базовый уровень приоритета для всех исполняемых потоков, чтобы определить, какому из них необходимо выделить следующий квант времени центрального процессора. Функция SetThreadPriority позволяет установить базовый уровень приоритета потока относительно класса приоритета процесса.

SetThreadPriority

```
BOOL SetThreadPriority(HANDLE hThread, int nPriority);
```

Возвращаемое значение

Ненулевое, если функция успешно завершила свою работу, и ноль — в противном случае. Более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

hThread — дескриптор потока, приоритет которого требуется установить. В Windows NT этот дескриптор должен иметь уровень доступа THREAD_SET_INFORMATION.

nPriority — определяет значение приоритета указанного потока. Может принимать одно из следующих значений:

- THREAD_PRIORITY_ABOVE_NORMAL — устанавливает значение приоритета на 1 выше нормального приоритета для приоритетного класса

- THREAD_PRIORITY_BELOW_NORMAL — устанавливает значение приоритета на 1 ниже нормального приоритета для приоритетного класса
- THREAD_PRIORITY_HIGHEST — устанавливает значение приоритета на 2 выше нормального приоритета для приоритетного класса
- THREAD_PRIORITY_IDLE — устанавливает базовый приоритет, равный 1, для процессов IDLE_PRIORITY_CLASS, NORMAL_PRIORITY_CLASS или HIGH_PRIORITY_CLASS, и базовый приоритет, равный 16, — для процессов REALTIME_PRIORITY_CLASS
- THREAD_PRIORITY_LOWEST — устанавливает значение приоритета на 2 ниже нормального приоритета для приоритетного класса
- THREAD_PRIORITY_NORMAL — устанавливает нормальное значение приоритета для приоритетного класса
- THREAD_PRIORITY_TIME_CRITICAL — устанавливает базовый приоритет, равный 15, для процессов IDLE_PRIORITY_CLASS, NORMAL_PRIORITY_CLASS или HIGH_PRIORITY_CLASS, и базовый приоритет, равный 31, — для процессов REALTIME_PRIORITY_CLASS

Описание

Функция SetThreadPriority устанавливает значение приоритета для указанного процесса. Это значение наряду с приоритетом класса процесса потока определяет базовый уровень приоритета потока.

Каждый поток имеет базовый уровень приоритета, определяемый значением приоритета потока и классом приоритета его процесса. Система использует базовый уровень приоритета для всех исполняемых потоков, чтобы определить, какому из них необходимо выделить следующий квант времени центрального процессора. Квант времени выделяется по очереди всем потокам, имеющим одинаковый уровень приоритета, и только в том случае, если нет ни одного запроса на исполнение от потоков высшего уровня приоритета, рассматриваются запросы на обслуживание от потоков более низкого уровня приоритета.

Функция SetThreadPriority позволяет установить базовый уровень приоритета относительно класса приоритета его процесса. Например, задание уровня приоритета THREAD_PRIORITY_HIGHEST в функции SetThreadPriority для процесса, имеющего класс приоритета IDLE_PRIORITY_CLASS, устанавливает базовый уровень приоритета равным 6.

Для процессов, имеющих классы приоритетов IDLE_PRIORITY_CLASS, NORMAL_PRIORITY_CLASS и HIGH_PRIORITY_CLASS, система динамически повышает базовый уровень приоритета, если происходит событие, важное для данного потока. Для процессов, имеющих класс приоритета REALTIME_PRIORITY_CLASS, динамическое изменение базового уровня невозможно.

Все потоки запускаются со значением приоритета THREAD_PRIORITY_NORMAL. Для получения и установки класса приоритета используются функции GetPriorityClass и SetPriorityClass. Для получения значения приоритета потока вызывается функция GetThreadPriority.

Классы приоритета процесса позволяют разделять приложения, работающие в реальном времени, на обычные и фоновые приложения. Значения приоритетов потоков позволяют определить важность выполнения отдельных потоков в приложении. Например, поток, выводящий окно на экран, должен иметь более высокий приоритет, чем поток, выполняющий интенсивные вычисления.

Устанавливая приоритеты, нужно следить, чтобы потоки, имеющие высокий приоритет, не занимали все процессорное время. Потоки, имеющие базовый уровень приоритета выше 11, включаются в нормальную работу операционной системы. Использование класса приоритета процесса REALTIME_PRIORITY_CLASS может нарушить процесс хеширования диска, прекратить работу с мышью и вызвать другие подобные проблемы.

SetWindowContextHelpId

```
BOOL SetWindowContextHelpId(HWND hwnd, DWORD dwContextHelpId);
```

Возвращаемое значение

Ненулевое, если функция успешно завершила свою работу, и ноль — в противном случае. Более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

hwnd — дескриптор окна, связанного с данным контекстным идентификатором.

dwContextHelpId — контекстный идентификатор справки.

Описание

Связывает контекстный идентификатор справки с указанным окном. Если дочернее окно не имеет контекстного идентификатора справки, то оно наследует его от родительского окна. Аналогично, если окно принадлежит другому окну и не имеет контекстного идентификатора справки, то оно наследует его от того окна, которому оно принадлежит. Это наследование контекстного идентификатора справки позволяет приложению использовать один контекстный идентификатор справки для диалогового окна и всех его элементов управления.

SetWindowLong

```
LONG SetWindowLong(HWND hWnd, int nIndex, LONG dwNewLong);
```

Возвращаемое значение

В случае нормального завершения функции — предыдущее значение заменяемой 32-разрядной величины. В противном случае — ноль. Расширенную информацию об ошибке можно получить, вызвав функцию `GetLastError`.

Если предыдущее значение заменяемой 32-разрядной величины было нулевым, то в случае нормального завершения функции возвращается нулевая величина, но функция не сбрасывает информацию о предыдущей ошибке, что существенно затрудняет вопрос о том, как завершилась работа данной функции. Чтобы избежать неопределенности, необходимо вызвать функцию `SetLastError(0)` перед вызовом функции `SetWindowLong`. В этом случае об ошибке при выполнении данной функции будет говорить ноль, возвращаемый данной функцией, и ненулевое значение, возвращаемое функцией `GetLastError`.

Аргументы

`hWnd` — дескриптор окна и, косвенно, класса, связанного с данным окном.

`nIndex` — определяет смещение величины, которую следует получить. Величина смещения может принимать значение в диапазоне от нуля до размера дополнительной памяти окна, выраженной в байтах, минус четыре. Например, при задании 12 или более байт в качестве дополнительной памяти, величина 8 будет являться индексом третьего 32-разрядного целого числа. Для доступа к величинам, характеризующим некоторые параметры окна, существуют предопределенные величины. Ниже приведен их список:

- `GWL_EXSTYLE` — устанавливает величину расширенного стиля окна
- `GWL_STYLE` — устанавливает величину расширенного стиля окна
- `GWL_WNDPROC` — устанавливает указатель на процедуру обработки окна или дескриптор, позволяющий получить доступ к этому указателю
- `GWL_HINSTANCE` — устанавливает дескриптор экземпляра (instance) данного приложения
- `GWL_HWNDPARENT` — устанавливает дескриптор родительского окна, если таковое имеется
- `GWL_ID` — устанавливает идентификатор окна
- `GWL_USERDATA` — устанавливает 32-разрядную величину, связанную с окном. Эта величина предназначена для использования приложением, связанным с окном

В случае, если аргумент hWnd является дескриптором диалогового окна, второй параметр может дополнительно принимать следующие значения:

- DWL_DLGPROC — устанавливает указатель на процедуру обработки диалогового окна или дескриптор, позволяющий получить доступ к указателю на эту процедуру
- DWL_MSGRESULT — устанавливает возвращаемое значение процедуры обработки диалогового окна
- DWL_USER — устанавливает дополнительную информацию, содержание которой специфично для каждого приложения. В качестве такой информации могут выступать указатели и дескрипторы

dwNewLong — новое значение устанавливаемой величины

Описание

Функция SetWindowLong изменяет атрибуты указанного окна. Она записывает 32-разрядную величину в расширенную память окна по заданному смещению. Функция SetWindowLong возвращает ошибку, если окно, заданное аргументом hWnd, не принадлежит тому же процессу, что и вызывающий поток. Некоторые окна кэшируют свою память, поэтому изменения, произведенные функцией SetWindowLong, вступят в силу только после очередного вызова функции SetWindowPos.

Если в функции SetWindowPos используется индекс GWL_WNDPROC для установки новой процедуры обработки окна, то новая процедура обработки окна должна соответствовать формату функции WindowProc.

Если в функции SetWindowPos используется индекс DWL_MSGRESULT для установки возвращаемого значения процедуры обработки окна, то после этого необходимо непосредственно возвратить значение TRUE. В противном случае при вызове любой функции, посылающей сообщение диалоговому окну, его собственное сообщение может переписать возвращаемое значение, установленное индексом DWL_MSGRESULT.

Вызов функции SetWindowLong с индексом GWL_WNDPROC приводит к созданию подкласса класса окна, использованного для создания данного окна. Приложение может создавать подклассы системных классов, но не может создавать подклассы для классов окон, созданных другими процессами. Функция SetWindowLong создает подкласс окна путем изменения процедуры обработки окна, связанной с конкретным классом окна, заставляя приложение вызывать новую процедуру обработки окна вместо старой. Приложение должно передавать все сообщения, не обработанные новой процедурой окна, старой процедуре посредством вызова функции CallWindowProc. Это позволяет приложению создавать цепочки процедур обработки окна.

Для резервирования дополнительной памяти окна необходимо задать ненулевое значение величины cbWndExtra, являющейся членом структуры WNDCLASSEX, используемой функцией RegisterClassEx.

Нельзя вызывать функцию SetWindowLong с индексом GWL_HWNDPARENT для замены родительского окна у дочернего окна. Вместо этого следует использовать функцию SetParent.

Sleep

```
VOID Sleep(DWORD dwMilliseconds);
```

Аргументы

dwMilliseconds — определяет промежуток времени, измеряемый в миллисекундах, на который следует приостановить исполнение данного потока. Нулевое значение данного аргумента заставляет поток передать остаток своего кванта времени любому другому потоку, имеющему одинаковый с ним приоритет и готовому приступить к работе. Если такой поток отсутствует, то функция немедленно завершает свою работу и передает управление потоку. Значение INFINITE означает бесконечную задержку.

Описание

Функция Sleep приостанавливает исполнение данного потока на указанный интервал времени. Поток может уступить остаток своего кванта времени при вызове данной функции с нулевым временем ожидания.

Необходимо соблюдать известную осторожность при использовании функции Sleep в программах, прямо или косвенно создающих окна. Любое окно, создаваемое потоком, должно обрабатывать сообщения. Сообщения передаются всем окнам в системе. При вызове функции Sleep с бесконечным временем ожидания система может зависнуть. Поэтому в тех случаях, когда поток создает окна, вместо функции Sleep следует использовать функции MsgWaitForMultipleObjects или MsgWaitForMultipleObjectsEx.

WaitForSingleObject

```
DWORD WaitForSingleObject(HANDLE hHandle, DWORD dwMilliseconds);
```

Возвращаемое значение

Если функция успешно завершает свою работу, то возвращаемое значение указывает на причину завершения работы функции:

- WAIT_ABANDONED — указанный объект является объектом mutex, который не был освобожден потоком, которому он принадлежит, перед завершением данного потока. Принадлежность объекта mutex вызывающему потоку гарантируется. Поэтому данный объект остался неотмеченным
- WAIT_OBJECT_0 — указанный объект находится в отмеченном состоянии
- WAIT_TIMEOUT — истек период ожидания, а объект остался неотмеченным

В случае аварийного завершения функция возвращает значение WAIT_FAILED. Более подробную информацию об ошибке можно получить, вызвав функцию GetLastError.

Аргументы

hHandle — дескриптор объекта. Список типов объектов, дескрипторы которых могут использоваться в качестве данного аргумента, приведен ниже в описании. В Windows NT дескриптор должен иметь уровень доступа SYNCHRONIZE.

dwMilliseconds — определяет интервал времени, измеряемый в миллисекундах. По истечении этого интервала времени, если указанный объект остается неотмеченным, функция завершает свою работу. Если данный аргумент равен нулю, функция проверяет состояние объекта и немедленно прекращает свою работу. Если аргумент dwMilliseconds имеет значение INFINITE, то функция ждет отметки объекта неограниченное время.

Описание

Функция WaitForSingleObject возвращает значение в двух случаях:

- Когда указанный объект устанавливается в отмеченное состояние
- Когда истекает время ожидания

Данная функция проверяет текущее состояние указанного объекта. Если данный объект находится в неотмеченном состоянии, выполнение потока приостанавливается. В процессе ожидания поток практически не использует процессорное время.

Перед завершением своей работы функция изменяет состояние некоторых объектов синхронизации. Изменения происходят только в том случае, если изменение состояния объекта привело к выходу из функции. Например, счетчик семафора уменьшается на единицу.

Функция WaitForSingleObject может использоваться со следующими объектами:

- Извещениями об изменениях
- Вводом с системной консоли
- Объектами событий
- Заданиями
- Объектами mutex
- Процессами
- Семафорами
- Потоками
- Таймерами ожидания

Необходимо соблюдать известную осторожность при вызове функций ожидания из программ, прямо или косвенно создающих объекты окон. Любое окно, создаваемое потоком, должно обрабатывать сообщения. Сообщения посылаются всем окнам системы. Поток, использующий функцию ожидания без интервала ожидания, может подвесить систему.

WindowProc

```
LRESULT CALLBACK WindowProc(HWND hwnd, UINT uMsg,
  WPARAM wParam, LPARAM lParam);
```

Возвращаемое значение

Представляет собой результат обработки сообщения и зависит от посланного сообщения.

Аргументы

`hwnd` — дескриптор окна.

`uMsg` — определяет сообщение.

`wParam` — определяет дополнительную информацию сообщения. Интерпретация данного аргумента зависит от значения аргумента `uMsg`.

`lParam` — определяет дополнительную информацию сообщения. Интерпретация данного аргумента зависит от значения аргумента `uMsg`.

Описание

Функция `WindowProc` обрабатывает сообщения, посылаемые в окно, и полностью определяется приложением. Тип `WNDPROC` представляет собой указатель на эту функцию обратного вызова. Имя `WindowProc` по умолчанию используется для определяемой приложением процедуры обработки сообщений, посылаемых в окно.

WinHelp

```
BOOL WinHelp(HWND hWndMain, LPCTSTR lpszHelp, UINT uCommand,
  DWORD dwData);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае. Расширенную информацию об ошибке можно получить, вызвав функцию `GetLastError`.

Аргументы

`hWndMain` — дескриптор окна, из которого вызывается справка. Функция `WinHelp` использует данный дескриптор, чтобы определить, какое из приложений запросило справочную информацию. Если аргумент `uCommand` имеет значение `HELP_CONTEXTMENU` или `HELP_WM_HELP`, то данный аргумент определяет элемент управления, по которому нужно получить справку.

`lpszHelp` — указатель на заканчивающуюся нулем текстовую строку, содержащую имя и, если это необходимо, путь к файлу справки, текст которой данная функция должна выводить на экран. После имени файла может стоять угловая скобка (>), за которой указывается имя вторичного окна,

если информация выводится во вторичное, а не в первичное окно справки. Имя вторичного окна справки должно быть определено в разделе [WINDOWS] файла проекта справки (.hpj).

uCommand — определяет тип запрашиваемой справочной информации.

dwData — дополнительная информация. Структура данного аргумента определяется значением аргумента uCommand.

Описание

Вызывает справочную систему Windows (Winhelp.exe) и передает ей дополнительные данные, определяющие характер запрашиваемой справочной информации.

Прежде чем закрыть окно, запросившее справочную информацию, необходимо вызвать функцию WinHelp и передать в аргументе uCommand команду HELP_QUIT. Пока все приложения не выполнят эту операцию, справочная система Windows не может быть закрыта. В этой операции нет необходимости, если справка была вызвана командой HELP_CONTEXTPOPUP.

В табл. В.1 указаны возможные значения аргумента uCommand, предпринимаемые при этом действия и соответствующий ему формат аргумента dwData.

***Таблица В.1.** Соответствие значений аргументов uCommand и dwData*

Значения аргумента uCommand	Предпринимаемые действия	Формат аргумента dwData
HELP_COMMAND	Выполняет макрос справки или строку макроса	Указатель на строку, содержащую имя выполняемого макроса справки. Если в строке указано несколько имен макросов, они должны разделяться точкой с запятой. Для некоторых макросов необходимо использовать короткую форму имени, поскольку справочная система Windows не позволяет работать с длинными именами
HELP_CONTENTS	Выводит содержимое раздела справки, определенное ключевым словом Contents в разделе [OPTIONS] файла .hpj. Эта команда используется для обеспечения совместимости с предыдущими версиями. Новые приложения должны использовать файл .cnt и команду HELP_FINDER	Игнорируется и устанавливается в 0

***Таблица В.1** (продолжение)*

Значения аргумента `uCommand`	Предпринимаемые действия	Формат аргумента `dwData`
`HELP_CONTEXT`	Выводит тему справки, заданную контекстным идентификатором, определенным в разделе `[MAP]` файла .hpj	Целое число без знака, содержащее контекстный идентификатор раздела
`HELP_CONTEXTMENU`	Выводит меню **Help** для указанного окна. Затем выводит справочную информацию по выделенному элементу управления в всплывающем окне	Указатель на массив пар идентификаторов. Первое двойное слово в каждой паре представляет собой идентификатор элемента управления, а второе слово — контекстный идентификатор раздела
`HELP_CONTEXTPOPUP`	Выводит в всплывающем окне тему справки, заданную контекстным идентификатором, определенным в разделе `[MAP]` файла .hpj	Целое число без знака, содержащее контекстный идентификатор раздела
`HELP_FINDER`	Выводит диалоговое окно **Справочная система**	Игнорируется и устанавливается в 0
`HELP_FORCEFILE`	Обеспечивает вывод требуемого файла справки справочной системой. Если выводится другой файл справки, справочная система выводит нужный. В противном случае не выполняет никаких действий	Игнорируется и устанавливается в 0
`HELP_HELPONHELP`	Если доступен файл WINHLP32.HLP, выводит справочную информацию по пользованию справочной системой	Игнорируется и устанавливается в 0
`HELP_INDEX`	Выводит содержимое раздела справки, определенное ключевым словом `Contents` в разделе `[OPTIONS]` файла .hpj. Эта команда используется для обеспечения совместимости с предыдущими версиями. Новые приложения должны использовать файл .cnt и команду `HELP_FINDER`	Игнорируется и устанавливается в 0

***Таблица В.1** (продолжение)*

Значения аргумента `uCommand`	**Предпринимаемые действия**	**Формат аргумента** `dwData`
`HELP_KEY`	Выводит тему справки по ключевому слову, содержащемуся в таблице ключевых слов, в том случае, если обеспечено полное совпадение. Если найдено более одного раздела, соответствующего данному ключевому слову, выводит диалоговое окно **Найденные разделы**	Указатель на строку, содержащую ключевое слово. Несколько ключевых слов должны разделяться точкой с запятой
`HELP_MULTIKEY`	Выводит тему справки по ключевому слову, *содержащемуся в альтернативной* таблице ключевых слов	Указатель на объект структуры `MULTIKEYHELP`, *определяющей символ нижнего индекса таблицы и ключевое слово*
`HELP_PARTIALKEY`	Выводит тему справки по ключевому слову, содержащемуся в таблице ключевых слов, в том случае, если обеспечено полное совпадение. Если найдено более одного раздела, выводит диалоговое окно **Найденные разделы**	Указатель на строку, содержащую ключевое слово. Несколько ключевых слов должны разделяться точкой с запятой. Для вывода содержания без указания ключевого слова необходимо передать указатель на пустую строку
`HELP_QUIT`	Сообщает справочной системе Windows о прекращении работы с ней. Если со справочной системой не работают другие приложения, закрывает справочную систему Windows	Игнорируется и устанавливается в 0
`HELP_SETCONTENTS`	Определяет содержимое раздела `[Contents]`. Справочная система Windows выводит эту тему, если файл справки не имеет связанного с ним файла .cnt	Целое число без знака, содержащее контекстный идентификатор раздела `[Contents]`
`HELP_SETPOPUP_POS`	Устанавливает позицию всплывающего окна. Позиция всплывающего окна устанавливается таким образом, как будто бы указатель мыши располагался в указанной точке при вызове данного окна	Указатель на объект структуры `POINT`

Таблица В.1 (окончание)

Значения аргумента uCommand	Предпринимаемые действия	Формат аргумента dwData
HELP_SETWINPOS	Выводит окно справки, если оно было минимизировано или располагалось в памяти	Указатель на объект структуры HELPWININFO, определяющей размер и положение первичного или вторичного окна справки
HELP_TCARD	Указывает на то, что данная команда относится к последовательности вторичных окон. Эта команда комбинируется с другими командами с использованием операции логического ИЛИ	Зависит от команды, с которой объединена данная команда
HELP_WM_HELP	Выводит во всплывающее окно справку об элементе управления, указанном в аргументе hWndMain	Указатель на массив пар идентификаторов. Первое двойное слово в каждой паре представляет собой идентификатор элемента управления, а второе слово — контекстный идентификатор раздела

Класс *CArchive*

Класс CArchive позволяет сохранять на диске достаточно сложные объекты, а также загружать их в память по мере необходимости. Объекты хранятся в файлах с последовательным доступом. Данный тип файла можно представить себе как двоичный поток данных. Подобно потокам, используемым в операциях ввода/вывода, архив использует буферизованную запись и чтение данных в память и из памяти. Поток ввода/вывода обрабатывает последовательности символов ASCII, но архив обрабатывает двоичные данные объектов в более эффективном формате.

Прежде чем использовать объект класса архива, следует создать соответствующий ему объект класса CFile. При этом необходимо удостовериться, что атрибуты доступа к данным в этих объектах соответствуют друг другу. Одному объекту класса CFile может соответствовать не более одного активного объекта класса CArchive.

Создаваемый объект класса CArchive присоединяется к объекту класса CFile (или производного от него класса), соответствующего открытому файлу.

При этом следует также определить режим работы с архивом: будет ли он использоваться для загрузки или сохранения данных. Объект класса CArchive может работать не только с простейшими типами данных, но и с объектами классов, производных от класса CObject, позволяющих сохранять данные в последовательном формате. Класс, допускающий сохранение своих данных в последовательном формате, обычно имеет функцию Serialize, а при его описании и реализации используются макросы DECLARE_SERIAL и IMPLEMENT_SERIAL.

Перегруженные операторы чтения данных из архива (>>) и записи данных в архив (<<) обеспечивают удобный программный интерфейс работы с архивом. Аргументом данных операторов могут быть как переменные простейших типов, так и объекты классов, производных от класса CObject.

Описание данного класса содержится в заголовочном файле afx.h.

CArchive::operator >>

```
friend CArchive& operator >>(CArchive& ar, CObject *& pOb);
throw(CArchiveException, CFileException, CMemoryException);
friend CArchive& operator >>(CArchive& ar, const CObject *& pOb);
throw(CArchiveException, CFileException, CMemoryException);
CArchive& operator >>(BYTE& by);
throw(CArchiveException, CFileException);
CArchive& operator >>(WORD& w);
throw(CArchiveException, CFileException);
CArchive& operator >>(int& i);
throw(CArchiveException, CFileException);
CArchive& operator >>(LONG& l);
throw(CArchiveException, CFileException);
CArchive& operator >>(DWORD& dw);
throw(CArchiveException, CFileException);
CArchive& operator >>(float& f);
throw(CArchiveException, CFileException);
CArchive& operator >>(double& d);
throw(CArchiveException, CFileException);
```

Возвращаемое значение

Ссылка на объект класса CArchive, позволяющая разместить в одной строке программы несколько подобных операторов.

Описание

Читает указанный объект или переменную простейшего типа из архива.

Если в файле реализации класса использовался макрос IMPLEMENT_SERIAL, то данный оператор вызывает защищенную функцию ReadObject с ненулевым

указателем на объект класса, производного от класса CObject, который, в свою очередь, вызывает функцию Serialize данного класса.

CArchive::operator <<

```
friend CArchive& operator <<(CArchive& ar, const CObject* pOb);
throw(CArchiveException, CFileException);
CArchive& operator <<(BYTE by);
throw(CArchiveException, CFileException);
CArchive& operator <<(WORD w);
throw(CArchiveException, CFileException);
CArchive& operator <<(int i);
throw(CArchiveException, CFileException);
CArchive& operator <<(LONG l);
throw(CArchiveException, CFileException);
CArchive& operator <<(DWORD dw);
throw(CArchiveException, CFileException);
CArchive& operator <<(float f);
throw(CArchiveException, CFileException);
CArchive& operator <<(double d);
throw(CArchiveException, CFileException);
```

Возвращаемое значение

Ссылка на объект класса CArchive, позволяющая разместить в одной строке программы несколько подобных операторов.

Описание

Записывает указанный объект или переменную простейшего типа в архив.

Если в файле реализации класса использовался макрос IMPLEMENT_SERIAL, то данный оператор вызывает защищенную функцию WriteObject с ненулевым указателем на объект класса, производного от класса CObject, который, в свою очередь, вызывает функцию Serialize данного класса.

CArchive::Flush

```
void Flush();
throw(CFileException);
```

Описание

Переписывает данные, хранящиеся в буфере данного архива, в файл. Вызов этой функции гарантирует, что все данные, переданные в архив, будут записаны в файл. Для завершения записи информации на диск необходимо вызвать функцию CFile::Close.

CArchive::GetFile

```
CFile* GetFile() const;
```

Возвращаемое значение

Указатель на объект класса `CFile`, связанный с данным архивом.

Описание

Позволяет получить указатель на объект класса `CFile`, связанный с данным архивом. Перед вызовом данной функции необходимо очистить буфер данного архива с помощью функции `Flush`.

CArchive::IsLoading

```
BOOL IsLoading() const;
```

Возвращаемое значение

Ненулевое, если архив в настоящее время используется для чтения из него информации, и ноль — в противном случае.

Описание

Определяет режим работы с архивом. Данная функция вызывается в функции `Serialize` классов, поддерживающих работу с архивом.

CArchive::IsStoring

```
BOOL IsStoring() const;
```

Возвращаемое значение

Ненулевое, если архив в настоящее время используется для записи информации, и ноль — в противном случае.

Описание

Определяет режим работы с архивом. Данная функция вызывается в функции `Serialize` классов, поддерживающих работу с архивом. Если функция `IsStoring` возвращает ненулевое значение, то функция `IsLoading` возвращает нулевое значение. И наоборот.

CArchive::Read

```
UINT Read(void* lpBuf, UINT nMax);
throw(CFileException);
```

Возвращаемое значение

Целое без знака, содержащее количество прочитанных данной функцией байт информации. Если возвращаемая величина меньше запрашиваемого

объема, то достигнут конец файла. В этом случае не посылается никаких исключений.

Аргументы

`lpBuf` — указатель на предоставленный пользователем буфер, в который будут записаны данные, считанные из архива.

`nMax` — целое число без знака, содержащее количество байт, которые нужно прочесть из архива.

Описание

Читает указанное количество байт из архива. Архив никак не интерпретирует передаваемую информацию. Данная функция может использоваться в функции `Serialize` для чтения структур данных фиксированного размера.

CArchive::Write

```
void Write(const void* lpBuf, UINT nMax);
throw(CFileException);
```

Аргументы

`lpBuf` — указатель на предоставленный пользователем буфер, из которого данные будут считываться в архив.

`nMax` — целое число без знака, содержащее количество байт, которые нужно записать в архив.

Описание

Переписывает указанное количество байт из буфера в архив. Архив не форматирует записываемую в него информацию. Данная функция может использоваться в функции `Serialize` для записи структур данных фиксированного размера.

Класс *CArray*

```
template< class TYPE, class ARG_TYPE > class CArray : public CObject
```

Аргументы

`TYPE` — параметр шаблона, определяющий тип объектов, хранящихся в массиве. Объекты данного типа возвращаются функциями-членами класса `CArray`.

`ARG_TYPE` — параметр шаблона, определяющий тип, используемый для доступа к объектам, хранящимся в массиве. Часто он представляет собой

ссылку на значение параметра TYPE. Объекты данного типа используются в качестве аргументов функций-членов класса CArray.

Описание

Класс CArray служит для хранения информации в массивах, аналогичных массивам языка C, но допускает динамическое изменение их размера.

Индекс первого элемента массива всегда равен 0. Пользователь может зафиксировать верхнюю границу массива или позволить ему расти при добавлении новых элементов, если их индекс превышает его верхнюю границу. Для данного массива всегда выделяется единая область памяти, даже если отдельные его элементы не используются.

Как и в случае массивов языка C, время доступа к любому элементу массива CArray постоянно и не зависит от размера массива.

Прежде чем использовать массив, необходимо вызвать функцию SetSize для установки его размера и резервирования необходимой памяти. Если при создании массива не использовалась функция SetSize, то при добавлении в него элементов часто будет производиться перераспределение памяти и копирование массива, что может привести к замедлению работы программы и к фрагментации памяти.

При необходимости вывести диагностическую информацию об отдельном элементе массива, следует указать глубину объекта CDumpContext равной 1 или большей величине.

Описание данного класса содержится в файле заголовка afxtempl.h.

CArray::Add

```
int Add(ARG_TYPE newElement);
throw(CMemoryException);
```

Возвращаемое значение

Индекс вставленного элемента.

Аргументы

ARG_TYPE — параметр шаблона, определяющий тип аргументов, используемых для ссылки на элементы массва.

newElement — элемент, добавляемый в массив.

Описание

Добавляет новый элемент в конец массива, увеличивая его размер на 1. Если в функции SetSize аргумент nGrowBy больше 1, и увеличение размера массива на 1 привело к выходу за пределы отведенной ему памяти, то для массива выделяется дополнительная память, в которую могут быть записаны новые

элементы без новой операции выделения памяти, размер которой определяется аргументом nGrowBy функции SetSize. Однако величина максимального индекса элемента в массиве возрастает только на 1.

Пример

```
CArray<CPoint,CPoint> ptArray;
CPoint pt(10,20);
ptArray.Add(pt);              // Элемент 0
ptArray.Add(CPoint(30,40));   // Элемент 1
```

CArray::GetSize

```
int GetSize() const;
```

Возвращаемое значение

Размер массива.

Описание

Возвращает размер массива. Поскольку первый элемент массива имеет нулевой индекс, размер массива всегда на 1 превышает максимальный индекс элемента массива.

CArray::GetUpperBound

```
int GetUpperBound() const;
```

Возвращаемое значение

Максимальный индекс элемента массива. Если возвращаемое значение равно −1, то данный массив не содержит элементов.

Описание

Возвращает максимальный индекс элемента массива. Поскольку первый элемент массива имеет нулевой индекс, максимальный индекс элемента массива всегда на 1 меньше, чем размер массива.

CArray::operator [ ]

```
TYPE& operator [](int nIndex);
TYPE operator [](int nIndex) const;
```

Аргументы

TYPE — параметр шаблона, определяющий тип объектов, хранящихся в массиве.

nIndex — индекс элемента, к которому необходимо получить доступ.

Описание

Эти операторы могут применяться вместо функций `SetAt` и `GetAt`. Первый оператор используется для обычных массивов и может вызываться как с левой, так и справой стороны от оператора присваивания, а второй — для массивов констант и может вызываться только с правой стороны оператора присваивания.

В отладочной версии библиотеки используемый индекс массива проверяется на попадание в разрешенный диапазон значений.

CArray::RemoveAll

```
void RemoveAll();
```

Описание

Удаляет все указатели из массива. Если массив уже пуст, функция все равно работает.

CArray::SetSize

```
void SetSize(int nNewSize, int nGrowBy = -1);
throw(CMemoryException);
```

Аргументы

`nNewSize` — новый размер массива (количество элементов). Должен иметь неотрицательное значение.

`nGrowBy` — минимальное количество элементов, на которое будет увеличен размер массива при необходимости.

Описание

Устанавливает размер пустого или существующего массива. Выделяет память в случае необходимости. Если размер массива при этом уменьшается, то массив усекается и вся неиспользуемая память освобождается. Для повышения производительности приложений, использующих массив, необходимо вызвать функцию `SetSize` до начала работы с ним. Правильный выбор ее аргументов позволит исключить или существенно уменьшить объем операций по перераспределению памяти и копирования массива при добавлении к нему элементов.

Аргумент `nGrowBy` влияет только на распределение памяти при увеличении размеров массива. Его значение никак не отражается на возвращаемых значениях функций `GetSize` и `GetUpperBound`. Если используется значение, выбранное по умолчанию, библиотека MFC выделяет память таким образом, чтобы избежать фрагментации памяти и оптимизировать работу с массивом для большинства практических случаев.

Класс *CBitmap*

Объекты класса `CBitmap` содержат функции для работы с битовыми образами интерфейса графических устройств Windows. Объект класса `CBitmap` необходимо создать с помощью конструктора, а затем связать с ним дескриптор битового образа с помощью одной из функций инициализации. После этого можно вызывать функции данного класса.

Описание данного класса содержится в файле afxwin.h.

CBitmap::CreateCompatibleBitmap

```
BOOL CreateCompatibleBitmap(CDC* pDC, int nWidth, int nHeight);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

`pDC` — указатель на объект класса контекста устройства.

`nWidth` — определяет ширину битового образа (в элементах изображения).

`nHeight` — определяет высоту битового образа (в элементах изображения).

Описание

Инициализирует битовый образ, делая его совместимым с контекстом устройства, определяемого аргументом `pDC`. Битовый образ имеет то же число цветовых битовых плоскостей или то же число битов, используемых для кодирования цвета каждого из элементов изображения, что и указанный контекст устройства. После этой операции данный битовый образ может быть выбран в качестве текущего битового образа контекста устройства, на который указывает аргумент `pDC`.

Если аргумент `pDC` указывает на контекст устройства памяти, инициализированный битовый образ имеет тот же формат, что и текущий битовый образ указанного контекста устройства. Контекст устройства памяти представляет собой область памяти, отображающую экран дисплея. Он может использоваться для подготовки изображений перед выводом их на экран. При создании контекста устройства памяти GDI автоматически выбирает в него монохромный битовый образ.

Поскольку в контекст устройства памяти могут быть выбраны как монохромные, так и цветные битовые образы, формат битового образа, возвращаемый функцией `CreateCompatibleBitmap`, использующей данный контекст устройства, может не совпадать при различных вызовах. Однако во всех ос-

тальных случаях битовый образ имеет формат, определяемый только возможностями устройства.

По завершении работы с объектом класса `CBitmap`, инициализированным функцией `CreateCompatibleBitmap`, необходимо сначала удалить битовый образ из контекста устройства, а затем уничтожить сам объект класса `CBitmap`.

CBitmap::GetBitmapBits

```
DWORD GetBitmapBits(DWORD dwCount, LPVOID lpBits) const;
```

Возвращаемое значение

Ноль, в случае возникновения ошибки, и скопированное количество байт в битовом образе — в противном случае.

Аргументы

`dwCount` — определяет количество байт, которые необходимо скопировать.

`lpBits` — указатель на буфер, в который будет записан битовый образ. Битовый образ представляет собой байтовый массив. Информация хранится по линиям горизонтальной развертки, для чего используются 16-битные структуры. Поэтому, если битовый образ содержит нечетное количество столбцов, и для хранения информации о цвете каждого из элементов изображения требуется нечетное количество байт, объем памяти, необходимый для хранения одной строки горизонтальной развертки, будет на один байт превышать ее действительный размер.

Описание

Копирует битовый образ из объекта класса `CBitmap` в буфер, на который указывает аргумент `lpBits`. Аргумент `dwCount` определяет количество байт, которое необходимо скопировать в данный буфер. Для определения корректного значения аргумента `dwCount` следует вызывать функцию `CGdiObject::GetObject` для данного объекта класса битового образа.

CBitmap::SetBitmapBits

```
DWORD SetBitmapBits(DWORD dwCount, const void* lpBits);
```

Возвращаемое значение

Нулевое значение, в случае возникновения ошибки, и скопированное количество байт в битовом образе — в противном случае.

Аргументы

`dwCount` — определяет количество байт, которые необходимо скопировать.

`lpBits` — указатель на область в оперативной памяти, из которой следует скопировать битовый образ в объект класса `CBitmap`.

Описание

Присваивает битам в битовом образе значения, содержащиеся в массиве `lpBits`.

Класс *CButton*

Класс `CButton` содержит функции для работы с кнопками Windows. Кнопка представляет собой маленькое прямоугольное дочернее окно, которое может находиться в двух состояниях, переключаемых мышью или клавишами клавиатуры. Данный элемент управления может использоваться самостоятельно или в группах. В зависимости от типа кнопки связанный с ней текст может располагаться внутри данного элемента управления или рядом с ним. Изменение состояния кнопки обычно отражается при ее отображении на экране.

Типичным примером кнопок являются флажки, переключатели и простые кнопки. Тип кнопки, к которому будет относиться объект класса `CButton`, определяется ее стилем, задаваемым в функции `Create` при его создании.

Класс `CBitmapButton`, производный от класса `Cbutton`, позволяет получать кнопки, в которых вместо текста помещается растровое изображение, а нажатой, отжатой, получившей фокус ввода и недоступной кнопкам соответствуют различные битовые образы.

Объект данного класса может быть создан на основе шаблона диалога или непосредственно пользователем. В обоих случаях сначала вызывается конструктор класса `CButton`, создающий объект данного класса, а затем вызывается функция `Create`, создающая кнопку Windows и связывающая ее с объектом класса `CButton`.

Объекты класса, производного от класса `Cbutton`, можно создавать за один шаг. Для этого в конструкторе данного класса нужно вызвать функцию `Create`.

Для обработки сообщений Windows, посылаемых объектом класса кнопки объекту класса родительского окна (обычно это объект класса, производного от класса `CDialog`), необходимо добавить в карту сообщений родительского класса соответствующие макросы, а в сам класс включить соответствующие функции обработки сообщений.

Макрос карты сообщений имеет следующий формат:

```
ON_Notification(id, memberFxn)
```

Параметр `id` определяет идентификатор дочернего окна элемента управления, а `memberFxn` — имя функции обработки данного сообщения.

Функция обработки сообщения объявляется следующим образом:

```
afx_msg void memberFxn();
```

Наиболее часто в качестве макросов обработки сообщений от кнопок используются:

- ON_BN_CLICKED — пользователь нажал кнопку
- ON_BN_DOUBLECLICKED — пользователь дважды нажал кнопку

Объект класса CButton, созданный из ресурса диалога, автоматически уничтожается при закрытии диалогового окна. Если данный объект создавался в куче оператором new, то при закрытии родительского окна он должен быть уничтожен с использованием оператора delete. Если же он создавался в стеке или был внедрен в объект класса родительского окна диалога, то он уничтожается автоматически.

Описание данного класса содержится в заголовочном файле afxwin.h.

CButton::GetState

```
UINT GetState() const;
```

Возвращаемое значение

Определяет текущее состояние кнопки. Для выделения конкретной информации о состоянии кнопки могут использоваться следующие маски:

- 0x0003 — определяет состояние флажков и переключателей. Значение 0 говорит о том, что флажок сброшен, 1 — установлен, а 2 — что кнопка находится в неопределенном состоянии
- 0x0004 — определяет выделение кнопки. Ненулевое значение означает, что данная кнопка выделена, то есть пользователь нажал и удерживает на ней левую кнопку мыши
- 0x0008 — определяет фокус ввода. Ненулевое значение означает, что данная кнопка имеет фокус ввода

Описание

Позволяет определить состояние кнопки.

CButton::SetCheck

```
void SetCheck(int nCheck);
```

Аргументы

nCheck — определяет состояние кнопки. Этот аргумент может принимать одно из следующих значений:

- 0 — кнопка сброшена
- 1 — кнопка установлена

- 2 — состояние кнопки не определено. Это значение состояния кнопки может быть установлено, только если она имеет стиль BS_3STATE или стиль BS_AUTO3STATE

Описание

Устанавливает или сбрасывает переключатели или флажки. Эта функция не оказывает никакого действия на простые кнопки.

Класс *CCmdUI*

Класс CCmdUI используется в функциях обработки сообщения ON_UPDATE_COMMAND_UI в классах, производных от класса CCmdTarget.

Когда пользователь раскрывает меню в своем приложении, приложение должно определить состояние каждой его команды, каждая их которых может быть доступной или недоступной. Эта информация предоставляется функциями обработки сообщения ON_UPDATE_COMMAND_UI. Прототипы этих функций и соответствующие им макросы карты сообщений могут быть созданы при помощи мастера ClassWizard.

При раскрытии меню приложение ищет и вызывает функции обработки сообщения ON_UPDATE_COMMAND_UI, передавая им в качестве аргумента объект класса CCmdUI. Пользователь вызывает для данного объекта такие функции, как Enable или Check, позволяющие приложению правильно установить состояние для каждой команды меню.

Функции команд меню могут выполнять кнопки панели инструментов или другие объекты пользовательского интерфейса. При этом все они будут вызывать функцию обработки сообщения ON_UPDATE_COMMAND_UI.

В табл. В.2 содержатся сведения о реакции отдельных элементов управления на вызов функций-членов класса CCmdUI.

Таблица В.2. *Воздействие функций класса CCmdUI на отдельные элементы управления*

Элемент управления	Enable	SetCheck	SetRadio	SetText
Команда меню	Делает доступной или недоступной	Устанавливает или снимает флажок (v)	Устанавливает или снимает переключатель (.)	Устанавливает текст элемента управления
Кнопка панели инструментов	Делает доступной или недоступной	Выделяет, снимает выделение или переводит в неопределенное состояние	Работает аналогично функции SetCheck	Не используется

***Таблица B.2** (окончание)*

Элемент управления	Enable	SetCheck	SetRadio	SetText
Панель строки состояния	Делает текст видимым или невидимым	Устанавливает выступающую или нормальную рамку	Работает аналогично функции SetCheck	Устанавливает текст в панели
Обычная кнопка в CDialogBar	Делает доступной или недоступной	Устанавливает или снимает флажок	Работает аналогично функции SetCheck	Устанавливает текст в кнопке
Обычный элемент управления в CDialogBar	Делает доступной или недоступной	Не используется	Не используется	Устанавливает текст окна

Класс CCmdUI не имеет базового класса, а его описание содержится в файле заголовка afxwin.h.

CCmdUI::ContinueRouting

```
void ContinueRouting();
```

Описание

Сообщает процедуре обработки сообщений, что обработка данного сообщения не завершена и требуется найти обработчик, который завершит обработку сообщения.

Функцию ContinueRouting необходимо вызывать в функциях обработки сообщения ON_COMMAND_EX, если они возвращают значение FALSE.

CCmdUI::Enable

```
virtual void Enable(BOOL bOn = TRUE);
```

Аргументы

bOn — если данный аргумент имеет значение TRUE, то соответствующий элемент управления становится доступным, и наоборот.

Описание

Данная функция делает элемент управления доступным или недоступным.

CCmdUI::SetCheck

```
virtual void SetCheck(int nCheck = 1);
```

Аргументы

nCheck — определяет состояние флажка. Если данный аргумент равен 0, то флажок сбрасывается, если — 1, то флажок устанавливается, а если значение данного аргумента равно 2, то состояние флажка не определено.

Описание

Функция позволяет установить состояние команды меню или кнопки панели инструментов. В неопределенное состояние может быть установлена только кнопка панели инструментов.

Класс CCommandLineInfo

Класс CCommandLineInfo позволяет передавать приложению параметры из командной строки. Приложение, использующее библиотеку MFC, обычно создает объект данного класса в функции CWinApp::InitInstance, который затем передается в функцию CWinApp::ParseCommandLine. Эта функция заполняет данный объект класса CCommandLineInfo путем нескольких последовательных вызовов функции ParseParam. После этого объект класса CCommandLineInfo передается в качестве аргумента функции CWinApp::ProcessShellCommand, интерпретирующей параметры командной строки в команды приложения.

Описание данного класса содержится в файле afxwin.h.

CCommandLineInfo::ParseParam

```
virtual void ParseParam(LPCTSTR lpszParam, BOOL bFlag, BOOL bLast);
```

Аргументы

lpszParam — команда или флаг командной строки.

bFlag — определяет, является ли первый аргумент командой или флагом.

bLast — определяет, является ли данная команда или флаг последними в командной строке.

Описание

Приложение вызывает данную функцию для выделения и интерпретации отдельных параметров командной строки. Данная функция обычно вызывается функцией CWinApp::ParseCommandLine для каждой команды или флага в командной строке. Если первым символом параметра командной строки

является '-' или '/', то функция ParseCommandLine удаляет этот символ при передаче параметра в первый аргумент данной функции и присваивает ее второму аргументу значение TRUE. Если передаваемый параметр является последним в командной строке, то аргументу bLast присваивается значение TRUE.

Команды и флаги командной строки, обрабатываемые функцией ParseParam, а также соответствующие им действия приведены в табл. В.3.

Таблица В.3. *Команды и флаги командной строки*

Параметр командной строки	Выполняемые действия
<без параметров>	Открывается новый файл
<имя файла>	Открывается существующий файл
/p <имя файла>	Печатает файл на принтере, указанном по умолчанию
/pt <имя файла> <принтер> <драйвер> <порт>	Печатает файл на указанном принтере
/dde	Запускает приложение на исполнение и ожидает команды динамического обмена данными
/Automation	Запускает приложение как сервер автоматизации OLE
/Embedding	Запускает приложение на редактирование внедренного объекта OLE

Полученная из командной строки информация хранится в переменных m_bRunAutomated, m_bRunEmbedded и m_nShellCommand. По умолчанию первый параметр, не являющийся флагом, помещается в переменную m_strFileName. Если установлен флаг /pt, то по умолчанию второй, третий и четвертый параметры, не являющиеся флагами, записываются в переменные m_strPrinterName, m_strDriverName и m_StrPortName, соответственно. Кроме того, по умолчанию переменной m_bShowSplash присваивается значение TRUE только при создании нового файла.

Перегрузка данной функции в созданных пользователем классах позволяет обрабатывать нестандартные параметры командной строки.

Класс *CControlBar*

Класс CControlBar является базовым для таких классов, как CStatusBar, CToolBar, CDialogBar, CReBar и COleResizeBar. Панель управления представляет собой окно, обычно выровненное по левой или правой стороне основ-

ного окна приложения. Оно может содержать дочерние окна, которые могут быть как элементами управления, имеющими свой дескриптор окна HWND, то есть обычными окнами Windows, способными посылать и принимать сообщения, или другими элементами управления, не являющимися окнами и управляемыми функциями класса приложения или класса главного окна приложения. Примером элементов управления, имеющих свой дескриптор окна, могут служить просмотровые окна списка и окна редактора. Примером элементов управления, не имеющих своего дескриптора окна, являются панели строки состояния или кнопки панели инструментов.

Окна панели управления обычно являются дочерними окнами главного окна приложения и используются наравне с рабочей областью однооконного приложения или окнами документов многооконного приложения. Для позиционирования объекта класса CControlBar используется информация о размерах рабочей области родительского окна. После этого объект информирует родительское окно о том, сколько места осталось в его рабочей области.

Описание данного класса содержится в файле afxext.h.

CControlBar::EnableDocking

```
void EnableDocking(DWORD dwStyle);
```

Аргументы

dwStyle — разрешает фиксацию панели управления на одной из сторон окна и определяет стороны, на которых может фиксироваться панель управления. Флаги, определяющие место фиксации, приведены ниже:

- CBRS_ALIGN_TOP — на верхней стороне рабочей области окна
- CBRS_ALIGN_BOTTOM — на нижней стороне рабочей области окна
- CBRS_ALIGN_LEFT — на левой стороне рабочей области окна
- CBRS_ALIGN_RIGHT — на правой стороне рабочей области окна
- CBRS_ALIGN_ANY — на любой стороне рабочей области окна
- CBRS_FLOAT_MULTI — разрешает использование нескольких плавающих панелей управления в одном окне

Если данный аргумент равен 0 (то есть не указан ни один флаг), панель управления не фиксируется.

Описание

Функция разрешает фиксацию панелей управления. Указанные положения фиксации должны соответствовать положениям, определенным в главном окне приложения, в противном случае панель управления не может быть зафиксирована в данном окне.

Структура *CCreateContext*

Приложение использует структуру `CCreateContext` при создании класса окна и класса представления, связанных с документом. При создании окна данная структура содержит информацию, используемую для установления соответствия между объектами классов документа и представления. С помощью данной структуры пользователь может вносить изменения в стандартный процесс создания окна документа.

Структура `CCreateContext` содержит указатели на объекты классов документа, окна, представления и шаблона документа. Она также содержит указатель на объект класса `CRuntimeClass`, определяющего тип класса представления, объект которого необходимо создать. Информация о классе, предоставляемая во время исполнения приложения, и указатель на текущий документ используются для динамического создания нового объекта класса представления. Основными элементами структуры `CCreateContext` являются:

- `m_pNewViewClass` — указатель на объект класса `CRuntimeClass`, содержащий объект класса представления
- `m_pCurrentDoc` — указатель на объект класса документа, связанного с новым объектом класса представления
- `m_pNewDocTemplate` — указатель на объект класса шаблона документа, связанного с новым окном многооконного приложения
- `m_pLastView` — указатель на изначальный объект класса представления, на основании которого могут быть созданы дополнительные объекты этого класса, например, для отображения разделенного окна или для создания второго объекта класса представления для данного документа
- `m_pCurrentFrame` — указатель на изначальный объект класса окна, на основании которого может быть создано второе окно для отображения того же документа

Создавая объект класса документа и объекты связанных с ним классов, объект класса шаблона документа проверяет правильность информации, хранящейся в структуре `CCreateContext`. Например, что существующему объекту класса представления соответствует конкретный объект класса документа.

Все указатели в данной структуре могут принимать нулевое значение, если соответствующая им информация отсутствует или не определена.

Основные принципы работы с объектами данной структуры:

- При использовании объекта данной структуры в качестве аргумента функций `CWnd::Create`, `CFrameWnd::Create` и `CFrameWnd::LoadFrame` содержащаяся в нем информация определяет связи объекта данного окна с объектами других классов. Для большинства окон наличие этого аргумента не обязательно и вместо него может быть передан нулевой указатель

- В перегруженных функциях, например, в `CFrameWnd::OnCreateClient`, объект класса `CCreateContext` может не задаваться
- При создании новых объектов класса представления, например, для первого объекта класса представления в разделенном окне, необходимо предоставить информацию о классе представления и связанном с ним объекте класса документа

Описание данной структуры содержится в заголовочном файле afxext.h.

Класс *CDC*

Представляет собой класс контекста устройств. Объект класса `CDC` содержит методы для работы с контекстом таких устройств, как дисплей или принтер, а также позволяет работать с контекстом дисплея, связанным с рабочей областью окна.

Все операции вывода на дисплей и принтер должны выполняться только при помощи методов, предоставляемых объектом класса `CDC`. Данный класс содержит методы для работы с контекстом устройств вывода графической информации, методы выбора объектов интерфейса графических устройств (GDI), учитывающие тип выбираемых объектов, и методы для работы с цветами и палитрами. Он также обеспечивает методы для получения и установки атрибутов отображения графической информации, масштабирования, работы с областями просмотра, работы с расширениями окон, преобразования координат, работы с областями, отсечения неотображаемых фрагментов, рисования линий, простых форм, эллипсов и многоугольников. Данный класс содержит также методы, позволяющие выводить текст, работать со шрифтами, использовать управляющие последовательности принтера, прокрутку окон и проигрывание метафайлов.

Чтобы использовать объект класса `CDC`, необходимо его создать, а затем использовать его функции, соответствующие функциям Windows, предназначенным для работы с контекстами устройств. В Windows 95 экранные координаты хранятся в 16-разрядных целых числах со знаком. Поэтому все величины типа `int`, используемые в качестве аргументов функций класса `CDC`, должны находиться в диапазоне от −32768 до 32767.

В библиотеке MFC предусмотрены классы, производные от класса `CDC`. Класс `CPaintDC` включает в себя вызовы функций `BeginPaint` и `EndPaint`. Класс `CClientDC` позволяет работать с контекстом дисплея, связанным с рабочей областью окна. Класс `CWindowDC` позволяет работать с контекстом дисплея, связанным со всем окном, включая его рамку и элементы управления. Класс `CMetaFileDC` связывает контекст устройства с метафайлом.

Класс `CDC` содержит две переменные для хранения контекста устройства: `m_hDC` и `m_hAttribDC`, которые при создании объекта класса `CDC` относятся

к одному и тому же устройству. Класс CDC направляет все запросы на вывод информации с использованием GDI к переменной m_hDC, а большинство обращений к атрибутам GDI задействуют переменную m_hAttribDC. (Примером обращения к атрибутам может служить функция GetTextColor, а функция SetTextColor — примером запроса на вывод информации.)

Например, с помощью этих контекстов устройства приложение, которое работает с объектом класса CMetaFileDC, записывающего выходную информацию в метафайл, использует при этом атрибуты, считанные из контекста физического устройства.

Пользователю может потребоваться одна и та же информация, полученная из обоих контекстов устройств. Для этого вызываются пары функций, приведенные в табл. В.4.

Таблица В.4. *Функции, выполняющие аналогичные действия с различными контекстами устройств*

Использует m_hAttribDC	**Использует m_hDC**
GetTextExtent	GetOutputTextExtent
GetTabbedTextExtent	GetOutputTabbedTextExtent
GetTextMetrics	GetOutputTextMetrics
GetCharWidth	GetOutputCharWidth

Описание данного класса содержится в файле afxwin.h.

:Attach

```
BOOL Attach(HDC hDC);
```

Возвращаемое значение

Ненулевое, если функция завершилась успешно, и ноль в противном случае.

Аргументы

hDC — контекст устройства Windows.

Описание

Данная функция связывает контекст устройства Windows с объектом класса CDC. Контекст устройства Windows хранится в переменной m_hDC контекста устройств вывода и в переменной m_hAttribDC атрибутов контекста устройств.

:BitBlt

```
BOOL BitBlt(int x, int y, int nWidth, int nHeight, CDC* pSrcDC,
  int xSrc, int ySrc, DWORD dwRop);
```

Возвращаемое значение

Ненулевое, если функция завершилась успешно, и ноль — в противном случае.

Аргументы

`x` — определяет горизонтальную координату верхнего левого угла прямоугольника, в который будет копироваться битовый образ (области вывода).

`y` — определяет вертикальную координату верхнего левого угла прямоугольника, в который будет копироваться битовый образ.

`nWidth` — определяет ширину (в логических единицах) прямоугольника, в который будет копироваться битовый образ.

`nHeight` — определяет высоту (в логических единицах) прямоугольника, в который будет копироваться битовый образ.

`pSrcDC` — указатель на объект класса `CDC`, определяющий контекст устройства, из которого копируется битовый образ. Если значение аргумента `dwRop` определяет растровую операцию, для которой не требуется источника, этот аргумент должен иметь значение `NULL`.

`xSrc` — определяет горизонтальную координату верхнего левого угла прямоугольника, из которого будет копироваться битовый образ.

`ySrc` — определяет вертикальную координату верхнего левого угла прямоугольника, из которого будет копироваться битовый образ.

`dwRop` — определяет растровую операцию, которую необходимо выполнить. Коды растровых операций определяют, каким образом GDI должен комбинировать цвета при выполнении операции, включающей в себя текущую кисть, возможно, исходный битовый образ и результирующий битовый образ. Данный аргумент может принимать следующие значения:

- `BLACKNESS` — закрашивает всю область вывода в черный цвет
- `DSTINVERT` — инвертирует битовый образ, расположенный в области вывода
- `MERGECOPY` — комбинирует образец и исходный битовый образ при помощи операции логического И
- `MERGEPAINT` — комбинирует инвертированный битовый образ, расположенный в области вывода, и исходный битовый образ при помощи операции логического ИЛИ
- `NOTSRCCOPY` — копирует инвертированный исходный битовый образ в область вывода

- NOTSRCERASE — инвертирует результат логического ИЛИ между инвертированным битовым образом, расположенным в области вывода, и исходным битовым образом
- PATCOPY — заполняет область вывода с использованием образца
- PATINVERT — комбинирует битовый образ, расположенный в области вывода, и исходный битовый образ при помощи ИСКЛЮЧАЮЩЕГО ИЛИ
- PATPAINT — комбинирует образец и инвертированный исходный битовый образ с использованием операции логического ИЛИ. Результат этой операции комбинируется с битовым образом, расположенным в области вывода, операцией логического ИЛИ
- SRCAND — комбинирует элементы изображения битового образа, расположенного в области вывода, и исходного битового образа при помощи логического И
- SRCCOPY — копирует исходный битовый образ в область вывода
- SRCERASE — инвертирует битовый образ, расположенный в области вывода, и комбинирует результат с исходным битовым образом операцией логического И
- SRCINVERT — комбинирует битовый образ, расположенный в области вывода, и исходный битовый образ операцией ИСКЛЮЧАЮЩЕГО ИЛИ
- SRCPAINT — комбинирует элементы изображения битового образа, расположенного в области вывода, и исходного битового образа операцией логического ИЛИ
- WHITENESS — закрашивает всю область вывода в белый цвет

Описание

Копирует битовый образ из исходного контекста устройства в текущий контекст устройства.

Приложение может выравнивать окна или их рабочие области по границе байта (для функции BitBlt). Это достигается установкой флага CS_BYTEALIGNWINDOW или CS_BYTEALIGNCLIENT при регистрации класса окна.

Работа функции BitBlt с прямоугольниками, выровненными по границам байт, происходит намного быстрее, чем с не выровненными прямоугольниками. При необходимости задания стилей класса, обеспечивающих работу с прямоугольниками, выровненными по границам байт, в пользовательском контексте устройства, следует самостоятельно зарегистрировать класс окна, не полагаясь на библиотеку MFC. Для этого используется глобальная функция AfxRegisterClass.

Интерфейс графических устройств Windows преобразует аргументы nWidth и nHeight, используя контекст устройства, в который копируется битовый образ. Если метрики устройств не совпадают, GDI вызывает функцию Windows StretchBlt для сжатия или растяжения исходного битового образа.

Если битовый образ в области вывода, исходный битовый образ и образец имеют различный формат цвета, функция BitBlt преобразует форматы исходного битового образа и образца к формату устройства вывода. Для преобразования используются как фоновый, так и основной битовые образы.

Во время преобразования монохромного битового образа в цветной функция BitBlt присваивает белому биту (1) цвет фона, а черному биту (0) — основной цвет. При этом используются основной цвет (foreground color) и цвет фона (background color). Во время преобразования цветного битового образа в монохромный функция BitBlt преобразует элементы изображения, имеющие цвет фона, в белые, а все остальные — в черные элементы.

Не все устройства могут работать с функцией BitBlt. Чтобы проверить возможность работы конкретного устройства с функцией BitBlt, вызовите функцию GetDeviceCaps с аргументом RASTERCAPS.

:CreateCompatibleDC

```
virtual BOOL CreateCompatibleDC(CDC* pDC);
```

Возвращаемое значение

Ненулевое, если функция завершилась успешно, и ноль — в противном случае.

Аргументы

pDC — указатель на объект класса контекста устройства. Если аргумент pDC имеет значение NULL, функция создает контекст устройства памяти, совместимый с системным дисплеем.

Описание

Создает контекст устройства памяти, совместимый с устройством, определяемым аргументом pDC. Контекст устройства памяти представляет собой блок памяти, соответствующий поверхности экрана. Он может использоваться для создания в памяти изображений перед копированием их на совместимое устройство.

При создании контекста устройства памяти GDI автоматически выбирает в него монохромный битовый образ размером 1×1 элемент изображения. Функции вывода информации интерфейса графических устройств могут работать с контекстом устройства памяти, только если в него был выбран объект класса битового образа.

Эта функция позволяет создавать совместимые контексты устройств только для устройств, обеспечивающих возможность вывода растровых изображений. Способ передачи информации битовыми блоками описан при рассмотрении функции CDC::BitBlt. Чтобы определить, способно ли конкретное устройство выводить растровые изображения, необходимо вызвать функцию CDC::GetDeviceCaps с аргументом RC_BITBLT.

:Detach

```
HDC Detach();
```

Возвращаемое значение

Контекст устройства Windows.

Описание

Данная функция вызывается для освобождения m_hDC (контекста устройств вывода) из объекта класса CDC и присвоения его переменным m_hDC и m_hAttribDC значения NULL.

:EndDoc

```
int EndDoc();
```

Возвращаемое значение

Неотрицательное, если функция завершилась успешно, в противном случае возвращается одно из следующих значений ошибки:

- SP_ERROR — неизвестная ошибка
- SP_OUTOFDISK — недостаточно дискового пространства, выделенного для спулинга, и оно не может быть увеличено
- SP_OUTOFMEMORY — недостаточно оперативной памяти для спулинга
- SP_USERABORT — пользователь прервал печать через Диспетчер печати

Описание

Завершает текущий сеанс печати, открытый вызовом функции StartDoc. Эта функция служит для замены управляющей последовательности ENDDOC, посылаемой принтеру, и должна вызываться непосредственно после успешного завершения печати.

Если в сеансе печати возникла ошибка или печать была прервана пользователем, для завершения печати не следует вызывать функции EndDoc или AbortDoc. GDI автоматически завершит процесс, прежде чем он выдаст сообщение об ошибке.

Эта функция не должна использоваться в метафайлах.

:EndPage

```
int EndPage();
```

Возвращаемое значение

Неотрицательное, если функция завершилась успешно, в противном случае возвращается одно из следующих значений:

- SP_ERROR — неизвестная ошибка
- SP_APPABORT — печать прекращена, потому что приложение завершило свою работу
- SP_USERABORT — пользователь прервал печать через Диспетчер печати
- SP_OUTOFDISK — недостаточно дискового пространства, выделенного для спулинга, и оно не может быть увеличено
- SP_OUTOFMEMORY — недостаточно оперативной памяти для спулинга

Описание

Информирует устройство, что приложение закончило передачу текущей страницы. Эта функция обычно указывает драйверу устройства на необходимость перехода к печати следующей страницы. Эта функция заменяет управляющую последовательность NEWFRAME, посылаемую принтеру. В отличие от управляющей последовательности NEWFRAME данная функция всегда вызывается после печати страницы.

:FillRect

```
void FillRect(LPCRECT lpRect, CBrush* pBrush);
```

Аргументы

lpRect — указатель на объект структуры RECT, содержащий логические координаты заполняемого прямоугольника. В качестве данного аргумента может использоваться объект класса CRect.

pBrush — определяет кисть, используемую для заполнения прямоугольника.

Описание

Заполняет прямоугольник указанной кистью, включая его левую и верхнюю границы, но исключая его правую и нижнюю границы.

Кисть должна быть создана функциями CBrush::CreateHatchBrush, CBrush::CreatePatternBrush или CBrush::CreateSolidBrush или функцией Windows ::GetStockObject.

Функция FillRect проверяет значения величин, присваеваемых переменным top, bottom, left и right объекта структуры RECT, передаваемого ей в качест-

ве аргумента. Если величина bottom меньше либо равна величине top, или величина right меньше либо равна величине left, то прямоугольник не вычерчивается.

Функция FillRect аналогична функции CDC::FillSolidRect за тем исключением, что в функции FillRect указывается кисть, что в свою очередь означает, что данный прямоугольник может быть заполнен одним цветом, определенным в трафарете или заданном образце. Функция FillSolidRect может заполнить данный прямоугольник только одним цветом (указанным в аргументе COLORREF). Функция FillRect обычно работает медленнее, чем функция FillSolidRect.

:GetDeviceCaps

```
int GetDeviceCaps(int nIndex) const;
```

Возвращаемое значение

В случае успешного завершения функции возвращается значение запрашиваемого параметра.

Аргументы

nIndex — определяет тип возвращаемой информации. Может принимать одно из следующих значений:

- DRIVERVERSION — номер версии. Например, для версии 1.0 возвращается величина 0x100
- TECHNOLOGY — технология устройства. Возможные значения:

  DT_PLOTTER — векторный плоттер

  DT_RASDISPLAY — растровый дисплей

  DT_RASPRINTER — растровый принтер

  DT_RASCAMERA — растровая камера

  DT_CHARSTREAM — текстовый поток

  DT_METAFILE — метафайл

  DT_DISPFILE — файл дисплея
- HORZSIZE — физическая ширина дисплея (в миллиметрах)
- VERTSIZE — физическая высота дисплея (в миллиметрах)
- HORZRES — размер дисплея по горизонтали (в элементах изображения)
- VERTRES — размер дисплея по вертикали (в строках растра)
- LOGPIXELSX — число элементов изображения в логическом дюйме по горизонтали дисплея
- LOGPIXELSY — число элементов изображения в логическом дюйме по вертикали дисплея

- BITSPIXEL — количество битов, используемых для кодирования цвета каждого элемента изображения
- PLANES — количество битовых плоскостей
- NUMBRUSHES — количество кистей в устройстве
- NUMPENS — количество перьев в устройстве
- NUMFONTS — количество шрифтов в устройстве
- NUMCOLORS — количество цветов в таблице цветов устройства
- ASPECTX — относительная ширина элемента изображения устройства, используемая при рисовании линий
- ASPECTY — относительная высота элемента изображения устройства, задаваемый при рисовании линий
- ASPECTXY — размер по диагонали элемента изображения устройства, задаваемый при рисовании линий
- PDEVICESIZE — размер внутренней структуры PDEVICE
- CLIPCAPS — возможность задания областей отсечки. Возможные значения:

  CP_NONE — выводимое изображение не ограничивается

  CP_RECTANGLE — прямоугольная область отсечки

  CP_REGION — произвольная область отсечки
- SIZEPALETTE — количество цветов в системной палитре. Аргумент nIndex может принимать данное значение, только если драйвер устройства установил бит RC_PALETTE в индексе RASTERCAPS
- NUMRESERVED — количество зарезервированных цветов в системной палитре. Аргумент nIndex может принимать данное значение, только если драйвер устройства установил бит RC_PALETTE в индексе RASTERCAPS
- COLORRES — текущее цветовое разрешение устройства, измеряемое в битах на элемент изображения. Аргумент nIndex может принимать данное значение, только если драйвер устройства установил бит RC_PALETTE в индексе RASTERCAPS
- RASTERCAPS — величина, указывающая на возможности растеризации устройства. Возможные значения:

  RC_BANDING — требует поддержки возможностей объединения

  RC_BIGFONT — устройство способно работать со шрифтами размером более 64 Кб

  RC_BITBLT — обладает возможностью передачи битовых образов

  RC_BITMAP64 — устройство способно работать с битовыми образами шрифтов размером более 64 Кб

RC_DEVBITS — устройство способно работать с аппаратно зависимыми битовыми образами

RC_DI_BITMAP — устройство способно работать с функциями Windows SetDIBits и GetDIBits

RC_DIBTODEV — устройство способно работать с функцией Windows SetDIBitsToDevice

RC_FLOODFILL — устройство способно осуществлять потоковое заполнение

RC_GDI20_OUTPUT — устройство способно работать с функциями Windows 2.0

RC_GDI20_STATE — включает блок состояния в контекст устройства

RC_NONE — не поддерживает растровые операции

RC_OP_DX_OUTPUT — устройство способно работать с массивом DX и выводить прозрачные изображения

RC_PALETTE — устройство способно работать с палитрами

RC_SAVEBITMAP — устройство способно локально хранить битовые образы

RC_SCALING — устройство поддерживает масштабирование изображения

RC_STRETCHBLT — устройство способно работать с функцией StretchBlt

RC_STRETCHDIB — устройство может работать с функцией Windows StretchDIBits

- CURVECAPS — определяет способность устройства вычерчивать кривые. Возможные значения:

  CC_NONE — устройство не может вычерчивать кривые

  CC_CIRCLES — устройство способно вычерчивать окружности

  CC_PIE — устройство может вычерчивать секторные диаграммы

  CC_CHORD — устройство вычерчивает хорды

  CC_ELLIPSES — устройство вычерчивает эллипсы

  CC_WIDE — устройство способно вычерчивать широкие рамки

  CC_STYLED — устройство может вычерчивать рамки с использованием стилей

  CC_WIDESTYLED — устройство вычерчивает широкие рамки с использованием стилей

  CC_INTERIORS — устройство способно заполнять внутреннее пространство

  CC_ROUNDRECT — устройство способно вычерчивать прямоугольники с закругленными углами

- LINECAPS — определяет способность устройства вычерчивать линии. Возможные значения:

  LC_NONE — устройство не вычерчивает линии

  LC_POLYLINE — устройство может одновременно вычерчивать несколько связанных линий

LC_MARKER — устройство способно выводить маркеры

LC_POLYMARKER — устройство может выводить множественные маркеры

LC_WIDE — устройство вычерчивает широкие линии

LC_STYLED — устройство вычерчивает линии с использованием стилей

LC_WIDESTYLED — устройство вычерчивает широкие линии с использованием стилей

LC_INTERIORS — устройство способно заполнять внутреннее пространство

- POLYGONALCAPS — определяет способность устройства вычерчивать многоугольники. Возможные значения:

PC_NONE — устройство не способно вычерчивать многоугольники

PC_POLYGON — устройство способно вычерчивать многоугольники с различным заполнением

PC_RECTANGLE — устройство вычерчивает прямоугольники

PC_WINDPOLYGON — устройство вычерчивает многоугольники со спиральным заполнением

PC_SCANLINE — устройство может использовать построчную развертку

PC_WIDE — устройство способно вычерчивать широкие рамки

PC_STYLED — устройство способно вычерчивать границы с использованием стилей

PC_WIDESTYLED — устройство может вычерчивать широкие рамки с использованием стилей

PC_INTERIORS — устройство способно заполнять внутреннее пространство

- TEXTCAPS — определяет возможности устройства при работе с текстами. Возможные значения:

TC_OP_CHARACTER — определяет разрешение при выводе текстов, позволяющее устройству поместить свой шрифт в любой элемент изображения. Это является необходимым условием для работы с устройствами, имеющими собственные шрифты

TC_OP_STROKE — определяет разрешение для вывода штрихов, указывая, что устройство может пропустить любой штрих в своем шрифте

TC_CP_STROKE — определяет точность отсечки штриха, указывая, что устройство может обрезать шрифты по границам элементов изображения

TC_CR_90 — устройство способно поворачивать символы на 90 градусов

TC_CR_ANY — устройство способно поворачивать символы на любой угол

TC_SF_X_YINDEP — устройство поддерживает независимое масштабирование шрифта по вертикальной и горизонтальной осям

TC_SA_DOUBLE — устройство может удваивать размеры шрифтов

TC_SA_INTEGER — устройство может увеличивать размеры своих шрифтов в любое целое число раз

TC_SA_CONTIN — устройство может произвольно изменять размеры своих шрифтов, но при этом сохраняет соотношение вертикального и горизонтального размеров

TC_EA_DOUBLE — если этот параметр установлен для принтера, GDI старается создать жирный шрифт в устройстве за счет его двойной печати

TC_IA_ABLE — устройство позволяет использовать курсивные шрифты. Если этот бит не установлен, GDI полагает, что данное устройство не может работать с курсивными шрифтами

TC_UA_ABLE — устройство позволяет использовать подчеркивание текста. Если этот бит не установлен, GDI генерирует подчеркивание для шрифта устройства

TC_SO_ABLE — устройство позволяет использовать зачеркивание текста. Если этот бит не установлен, GDI генерирует зачеркивание для шрифта устройства

TC_RA_ABLE — устройство позволяет использовать растровые шрифты. Это означает, что GDI должен пронумеровать все растровые шрифты или шрифты TrueType, имеющиеся в устройстве. Для этого вызывается функция Windows `EnumFonts` или `EnumFontFamilies`. Если этот бит не установлен, то при вызове данных функций растровые шрифты или шрифты TrueType не нумеруются

TC_VA_ABLE — устройство позволяет использовать векторные шрифты. Это означает, что GDI должен пронумеровать все имеющиеся в устройстве векторные шрифты. Для этого вызывается функция Windows `EnumFonts` или `EnumFontFamilies`. Это имеет значение только для векторных устройств (например, плоттеров). Драйверы дисплея (который должен использовать растровые шрифты) и растровых принтеров всегда нумеруют векторные шрифты, поскольку GDI растеризует векторные шрифты перед их передачей драйверу

TC_RESERVED — зарезервировано и должно быть равно нулю

Описание

Возвращает информацию об устройствах отображения информации.

:GetSafeHdc

```
HDC GetSafeHdc() const;
```

Возвращаемое значение

Дескриптор контекста устройства.

Описание

Данная функция позволяет получить контекст устройств вывода `m_hDC`. Может работать с нулевыми указателями.

:GetTextExtent

```
CSize GetTextExtent(LPCTSTR lpszString, int nCount) const;
CSize GetTextExtent(const CString& str) const;
```

Возвращаемое значение

Размер строки (в логических единицах), записанный в объект класса `CSize`.

Аргументы

`lpszString` — указатель на символьную строку. В качестве данного аргумента может также выступать объект класса `CString`.

`nCount` — определяет количество символов в строке.

`str` — объект класса `Cstring`, содержащий измеряемую строку.

Описание

Определяет ширину и высоту строки символов текущего шрифта. Информация извлекается из переменной `m_hAttribDC` данного контекста устройства. Установки текущей области отсечки не влияют на возвращаемые функцией `GetTextExtent` значения.

Поскольку некоторые устройства не помещают символы строки в последовательный массив ячеек (уменьшают апрош[1] в характерных сочетаниях пар знаков), сумма размеров отдельных символов в строке может не совпадать с размером всей строки.

:IsPrinting

```
BOOL IsPrinting() const;
```

Возвращаемое значение

Ненулевое, если данный объект класса `CDC` является объектом класса контекста устройства принтера, и ноль — в противном случае.

:Rectangle

```
BOOL Rectangle(int x1, int y1, int x2, int y2);
BOOL Rectangle(LPCRECT lpRect);
```

---

[1] *апрош* — межбуквенный пробел (от англ. approach — приближение, подход).

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

`x1` — определяет горизонтальную координату левого верхнего угла прямоугольника (в логических единицах).

`y1` — определяет вертикальную координату левого верхнего угла прямоугольника (в логических единицах).

`x2` — определяет горизонтальную координату правого нижнего угла прямоугольника (в логических единицах).

`y2` — определяет вертикальную координату правого нижнего угла прямоугольника (в логических единицах).

`lpRect` — определяет прямоугольник в логических единицах. В качестве данного аргумента может выступать как объект класса `CRect`, так и указатель на объект структуры `RECT`.

Описание

Рисует прямоугольник, используя текущее перо. Внутренняя поверхность прямоугольника заполняется текущей кистью.

Высота прямоугольника составляет `y2 - y1`, а ширина — `x2 - x1`. Как высота, так и ширина прямоугольника должны находиться в интервале от 2 до 32767 логических единиц.

:SelectClipRgn

```
virtual int SelectClipRgn(CRgn* pRgn);
int SelectClipRgn(CRgn* pRgn, int nMode);
```

Возвращаемое значение

Тип области. Может принимать одно из следующих значений:

- `COMPLEXREGION` — новая область отсечки имеет перекрывающиеся границы
- `ERROR` — используется недопустимый контекст устройства или недопустимая область
- `NULLREGION` — создана пустая область отсечки
- `SIMPLEREGION` — новая область отсечки не имеет перекрывающихся границ

Аргументы

`pRgn` — указатель на выбранную область

- В первой версия функции SelectClipRgn нулевое значение данного аргумента означает, что выбирается вся рабочая область окна, а область отсечки совпадает с границами окна
- Во второй версии функции SelectClipRgn данный аргумент может принимать нулевое значение только в режиме RGN_COPY

nMode — определяет выполняемую операцию. Может принимать одно из следующих значений:

- RGN_AND — новая область отсечки представляет собой пересечение текущей области отсечки и области, определенной в аргументе pRgn
- RGN_COPY — новая область отсечки полностью совпадает с областью, заданной в аргументе pRgn. В этом случае данная функция идентична своей первой версии, однако, если аргумент pRgn в данной версии равен нулю, то в качестве новой области отсечки принимается область отсечки по умолчанию (нулевая область)
- RGN_DIFF — новая область отсечки представляет собой текущую область отсечки, из которой исключены области, перекрывающиеся с областью, заданной в аргументе pRgn
- RGN_OR — новая область отсечки представляет собой объединение текущей области отсечки и области, заданной в аргументе pRgn
- RGN_XOR — новая область отсечки представляет собой объединение текущей области отсечки и области, заданной в аргументе pRgn, за исключением областей их пересечения

Описание

Выбирает указанную область как новую область отсечки для данного контекста устройства. Объекты работают с копией указанной области. Сама область может использоваться в неограниченном количестве объектов класса контекста устройств или может быть удалена.

Функция предполагает, что координаты указанной области заданы в единицах устройства. Некоторые принтеры поддерживают в текстовом режиме более высокое разрешение, чем в графическом. Эти устройства, получив запрос о разрешении, возвращают величины, используемые в режиме высокого разрешения, то есть в текстовом режиме. После этого в данных устройствах производится перерасчет координат графических объектов таким образом, чтобы несколько точек, заданных с высоким разрешением, отображались в одной точке, выводимой с графическим разрешением. При использовании режима вывода текста необходимо всегда вызывать функцию SelectClipRgn.

Приложения, выполняющие масштабирование графических объектов, которые выводятся функциями GDI, должны вызывать функцию GETSCALINGFACTOR,

позволяющую определить параметры масштабирования данного принтера. Задание неправильного параметра масштабирования может привести к потере информации. Используя область для задания отсечки графических объектов, GDI делит полученные координаты на величину параметра масштабирования. Если задается область отсечки текста, такое преобразование координат не выполняется. Параметр масштабирования, равный 1, приводит к делению координат на 2, а параметр масштабирования, равный 2, приводит к делению координат на 4, и т. д.

:SelectObject

```
CPen* SelectObject(CPen* pPen);
CBrush* SelectObject(CBrush* pBrush);
virtual CFont* SelectObject(CFont* pFont);
CBitmap* SelectObject(CBitmap* pBitmap);
int SelectObject(CRgn* pRgn);
```

Возвращаемое значение

Указатель на замещаемый объект одного из классов, производных от класса `CGdiObject`, например, на объект класса `CPen`. Тип возвращаемого значения зависит от версии используемой функции. В случае ошибки возвращается нулевое значение. Эта функция может возвращать указатель на временный объект. Это означает, что указатель на данный объект можно использовать только в пределах функции обработки одного сообщения Windows. Более подробную информацию можно найти в описании функции `CGdiObject::FromHandle`.

Версия функции, аргументом которой является указатель на объект класса `CRgn`, выполняет те же действия, что и функция `SelectClipRgn`. Возвращаемая величина может принимать следующие значения:

- `COMPLEXREGION` — новая область отсечки имеет пересекающиеся границы
- `ERROR` — ошибка при задании контекста устройства или недопустимая область отсечки
- `NULLREGION` — новая область отсечки представляет собой пустую область
- `SIMPLEREGION` — новая область отсечки не имеет пересекающихся границ

Аргументы

`pPen` — указатель на объект класса `CPen`

`pBrush` — указатель на объект класса `CBrush`

`pFont` — указатель на объект класса `CFont`

`pBitmap` — указатель на объект класса `CBitmap`

`pRgn` — указатель на объект класса `CRgn`

Описание

Выбирает объект класса в контекст устройства. Класс CDC содержит пять версий данной функции, отличающихся типом аргумента и возвращаемого значения. В качестве аргументов данной функции могут выступать объекты классов пера, кисти, шрифта, битового образа и области. Выбранный в контекст объект замещает соответствующий объект в контексте устройства. Например, если аргумент pObject обобщенной версии данной функции указывает на объект класса CPen, то данная функция замещает в данном контексте устройства текущее перо на перо, указанное в аргументе pObject.

Приложение может выбирать битовый образ только в контекст устройства памяти, и этот битовый образ не может быть выбран одновременно в два и более контекстов устройства памяти. Формат битового образа должен быть монохромным или совместимым с контекстом данного устройства. В противном случае функция SelectObject возвращает ошибку.

В версии Windows 3.1 и более поздних функция SelectObject возвращает ту же величину независимо от того, используется ли она в метафайлах или нет. В предыдущих версиях данная функция при вызове ее в метафайлах возвращала ненулевое значение в случае своего успешного завершения, и ноль — в противном случае.

:SetMapMode

```
virtual int SetMapMode(int nMapMode);
```

Возвращаемое значение

Предыдущий режим отображения.

Аргументы

nMapMode — определяет новый режим отображения. Может принимать одно из следующих значений:

- MM_ANISOTROPIC — логические единицы преобразуются в произвольные единицы с произвольным направлением осей координат. Установка режима отображения MM_ANISOTROPIC не изменяет установок в текущем окне и в текущей рабочей области окна. Для изменения единиц измерения, ориентации и масштаба вызываются функции SetWindowExt и SetViewportExt
- MM_HIENGLISH — каждая логическая единица преобразуется в 0.001 дюйма. Рост величин по горизонтальной оси приводит к перемещению вправо, а рост величин по вертикальной оси приводит к перемещению вверх
- MM_HIMETRIC — каждая логическая единица преобразуется в 0.01 миллиметра. Рост величин по горизонтальной оси приводит к перемеще-

нию вправо, а рост величин по вертикальной оси приводит к перемещению вверх

- MM_ISOTROPIC — логические единицы преобразуются в произвольные единицы с произвольным направлением осей координат, но с одинаковым масштабом по осям. Для изменения единиц измерения, ориентации и масштаба вызываются функции SetWindowExt и SetViewportExt. Для обеспечения одинакового масштаба отображения по осям GDI выполняет необходимые настройки
- MM_LOENGLISH — каждая логическая единица преобразуется в 0.01 дюйма. Рост величин по горизонтальной оси приводит к перемещению вправо, а рост по вертикали — к перемещению вверх.
- MM_LOMETRIC — каждая логическая единица преобразуется в 0.1 миллиметра. Рост величин по горизонтальной оси приводит к перемещению вправо, а рост по вертикали — к перемещению вверх
- MM_TEXT — каждая логическая единица преобразуется в 1 элемент изображения. Рост величин по горизонтальной оси приводит к перемещению вправо, а рост по вертикали — к перемещению вниз
- MM_TWIPS — каждая логическая единица преобразуется в 1/20 пики (поскольку пика составляет 1/72 дюйма, то данная величина составляет 1/1440 дюйма). Рост величин по горизонтальной оси приводит к перемещению вправо, а рост по вертикали — к перемещению вверх

Описание

Устанавливает режим отображения. Режим отображения определяет единицы измерения, используемые в преобразовании логических единиц в единицы устройства. Кроме того, режим отображения определяет направление осей координат. Режим отображения MM_TEXT позволяет приложению непосредственно работать с элементами изображения устройства. В этом случае одна логическая единица соответствует одному элементу изображения устройства. Физический размер элементов изображения зависит от конкретного устройства.

Режимы отображения MM_HIENGLISH, MM_HIMETRIC, MM_LOENGLISH, MM_LOMETRIC и MM_TWIPS используются при выводе изображений, размеры которых не зависят от устройства, на котором они выводятся. Режим отображения MM_ISOTROPIC обеспечивает соотношение масштабов горизонтальной и вертикальной оси, равное 1:1. В режиме отображения MM_ANISOTROPIC горизонтальная и вертикальная оси имеют независимый масштаб.

:SetViewportOrg

```
virtual CPoint SetViewportOrg(int x, int y);
virtual CPoint SetViewportOrg(POINT point);
```

Возвращаемое значение

Предыдущее значение начала отсчета рабочей области (в координатах устройства) как объект класса `CPoint`.

Аргументы

`x` — определяет горизонтальную координату начала отсчета рабочей области, выраженную в координатах устройства. Значение данного аргумента должно лежать в пределах системы координат устройства.

`y` — определяет вертикальную координату начала отсчета рабочей области, выраженную в координатах устройства. Значение данного аргумента должно лежать в пределах системы координат устройства.

`point` — определяет координаты начала отсчета рабочей области. Значение данного аргумента должно лежать в пределах системы координат устройства. В качестве аргумента может выступать как объект структуры `POINT`, так и объект класса `CPoint`.

Описание

Устанавливает начало отсчета рабочей области контекста устройства. Рабочая область, наряду с окном контекста устройства, определяет, каким образом GDI преобразует координаты, указанные в логической системе координат, в систему координат физического устройства.

Начало отсчета рабочей области определяет точку, заданную в системе координат устройства, в которую GDI помещает начало координат окна, под которым понимается точка, заданная в логической системе координат, определяемая функцией `SetWindowOrg`. GDI преобразует все остальные точки изображения, следуя той же процедуре, которая использовалась для преобразования начала координат окна в начало отсчета рабочей области. Например, все точки круга, описанного вокруг точки начала координат окна, преобразуются в точки круга, описанного вокруг начала отсчета рабочей области. Аналогично, все точки линии, проходящей через начало координат окна, преобразуются в точки линии, проходящей через начало отсчета рабочей области.

:StartDoc

```
int StartDoc(LPDOCINFO lpDocInfo);
```

Возвращаемое значение

Положительная величина, в случае успешного завершения функции, в случае возникновения ошибки, такой как недостаток памяти или неправильно указанный порт, возвращается значение −1.

Аргументы

lpDocInfo — указатель на объект структуры DOCINFO, содержащей имя файла документа и имя выходного файла.

Описание

Информирует драйвер устройства о начале нового сеанса печати и о том, что все последующие вызовы функций StartPage и EndPage будут относиться к этому сеансу печати, пока не будет вызвана функция EndDoc. Это гарантирует, что при печати многостраничного документа процесс печати не будет прерываться другими сеансами.

В Windows версии 3.1 и последующих версиях данная функция заменяет управляющую последовательность STARTDOC, посылаемую принтеру.

Функция StartDoc не должна использоваться в метафайлах.

:StartPage

```
int StartPage();
```

Описание

Данная функция вызывается для подготовки принтера к приему данных. Функция StartPage служит для замены управляющих последовательностей NEWFRAME и BANDINFO, посылаемых принтеру. Система делает недоступным вызов функции ResetDC между вызовами функций StartPage и EndPage.

Класс *CDialog*

Класс CDialog является родительским классом для пользовательских классов, используемый для отображения диалоговых окон на экране. Диалоговые окна могут быть модальными и немодальными. Модальное диалоговое окно должно быть закрыто пользователем, только после этого приложение сможет продолжить работу. Немодальное диалоговое окно допускает работу пользователя с приложением до своего закрытия и удаления с экрана.

Диалоговый объект является комбинацией шаблона диалога и одного из пользовательских классов, производных от класса CDialog. Для создания шаблона диалога и сохранения его в файле ресурсов обычно используется редактор диалога. Запуск мастера ClassWizard из окна редактора диалога позволяет создать соответствующий данному диалоговому окну класс, производный от класса CDialog.

Диалоговое окно, подобное любому другому окну, получает сообщения от Windows. Основным назначением объекта пользовательского класса диалога

является обработка сообщений, поступающих от объектов классов элементов управления данного диалогового окна, при взаимодействии пользователя с соответствующими элементами управления. Мастер ClassWizard просматривает потенциальные сообщения, которые может послать каждый элемент управления данного диалогового окна, и предоставляет пользователю возможность выбора тех сообщений, которые он действительно собирается обрабатывать. После выбора пользователем обрабатываемых сообщений мастер ClassWizard добавляет соответствующие макросы в карту сообщения и создает заготовки функций обработки для данных сообщений в пользовательском классе диалогового окна.

Все классы диалоговых окон, кроме самых простых, содержат переменные для хранения информации, необходимой для работы с элементами управления диалогового окна. Мастер ClassWizard просматривает все классы элементов управления созданного пользователем диалогового окна и определяет, какие переменные могут понадобиться для работы с ними. Пользователь сам выбирает тип переменной и допустимый диапазон значений для каждой из создаваемых мастером переменных. После этого мастер ClassWizard добавляет описание соответствующих переменных в пользовательский класс диалога.

После закрытия своего диалогового окна мастер ClassWizard составляет карту данных, обеспечивающую автоматический обмен данными между переменными объекта класса диалогового окна и объектами классов элементов управления данного окна. Карта данных содержит функции, инициализирующие элементы управления диалогового окна соответствующими значениями, получающие информацию, содержащуюся в этих объектах, и проверяющие правильность передаваемой информации.

Чтобы создавать модальное диалоговое окно, необходимо создать объект на стеке, используя конструктор пользовательского диалогового класса, присвоить его переменным исходные значения, а затем вызвать функцию `DoModal`, создающую окно диалога и его элементы управления. Чтобы создать немодальный диалог, вызовите в конструкторе диалогового класса функцию `Create`.

Одним из способов создания диалогового окна является создание в памяти объекта структуры `DLGTEMPLATE`. Для этого, создав указатель на объект класса `CDialog` вызовите функцию `CreateIndirect` для создания немодального диалогового окна или функции `InitModalIndirect` и `DoModal` для создания модального диалогового окна.

Мастер ClassWizard помещает функции обмена и проверки правильности информации в функцию `CWnd::DoDataExchange`, которую он же и включает в класс пользовательского диалогового окна. Вызов данной функции обычно осуществляется в функции `CWnd::UpdateData`.

Приложение вызывает функцию UpdateData, когда пользователь нажимает кнопку **OK**, чтобы закрыть модальное диалоговое окно. (Данные не обновляются при нажатии кнопки **Cancel**.)

Заданная по умолчанию реализация функции OnInitDialog вызывает функцию UpdateData, чтобы установить начальные значения переменных в объектах классов элементов управления. Обычно пользователь перегружает функцию OnInitDialog для проведения нестандартных операций по инициализации объектов классов элементов управления. Функция OnInitDialog вызывается после создания всех объектов классов элементов управления в диалоговом окне и непосредственно перед отображением диалогового окна на экране.

Пользователь может вызывать функцию CWnd::UpdateData в любой функции класса модального или немодального диалогового окна.

Для установки цвета фона диалогового окна в приложении используется функция CWinApp::SetDialogBkColor.

Модальное диалоговое окно закрывается автоматически, когда пользователь нажимает кнопки **OK** или **Cancel**, или когда программа непосредственно вызывает функцию EndDialog.

При создании класса немодального диалогового окна необходимо всегда перегружать функцию OnCancel и вызывать в ней функцию DestroyWindow. При этом не следует вызывать функцию базового класса CDialog::OnCancel, поскольку она вызывает функцию EndDialog, которая сделает данное диалоговое окно невидимым, но не уничтожит его класс. Кроме того, при создании классов немодальных диалоговых окон необходимо перегрузить функцию PostNcDestroy, добавив в нее оператор delete this, поскольку немодальные диалоговые окна обычно создаются оператором new. Объекты классов модальных диалоговых окон обычно являются локальными объектами функций и не нуждаются в самоликвидации в перегруженной функции PostNcDestroy.

Описание данного класса содержится в файле заголовка afxwin.h.

CDialog::CreateIndirect

```
BOOL CreateIndirect(LPCDLGTEMPLATE lpDialogTemplate,
  CWnd* pParentWnd = NULL);
BOOL CreateIndirect(HGLOBAL hDialogTemplate, CWnd* pParentWnd = NULL);
```

Возвращаемое значение

Ненулевое, если создание и инициализация диалогового окна прошли успешно, и ноль — в противном случае.

Аргументы

lpDialogTemplate — указатель на шаблон диалогового окна, содержащий объект структуры DLGTEMPLATE и область дополнительной памяти для каждого элемента управления данного диалогового окна.

pParentWnd — указатель на объект класса родительского окна (производного от класса CWnd). Если этот аргумент равен нулю, то родительским окном объекта класса диалога является главное окно приложения.

hDialogTemplate — дескриптор глобальной памяти шаблона диалогового окна, содержащего объект структуры DLGTEMPLATE и область дополнительной памяти для каждого элемента управления данного диалогового окна.

Описание

Функция создает немодальные диалоговые окна на основе шаблона диалогового окна. Функция CreateIndirect возвращает управление сразу же после создания ею диалогового окна.

Если диалоговое окно должно отображаться при создании родительского окна, используйте стиль WS_VISIBLE. В противном случае для вывода диалогового окна на экран необходимо использовать функцию ShowWindow. Для уничтожения окон, созданных данной функцией, необходимо использовать функцию CWnd::DestroyWindow.

CDialog::InitModalIndirect

```
BOOL InitModalIndirect(LPCDLGTEMPLATE lpDialogTemplate,
  CWnd* pParentWnd = NULL);
BOOL InitModalIndirect(HGLOBAL hDialogTemplate,
  CWnd* pParentWnd = NULL);
```

Возвращаемое значение

Ненулевое, если создание и инициализация диалогового окна прошли успешно, и ноль — в противном случае.

Аргументы

lpDialogTemplate — указатель на шаблон диалогового окна, содержащий объект структуры DLGTEMPLATE и область дополнительной памяти для каждого элемента управления данного диалогового окна.

hDialogTemplate — дескриптор глобальной памяти шаблона диалогового окна, содержащего объект структуры DLGTEMPLATE и область дополнительной памяти для каждого элемента управления данного диалогового окна.

pParentWnd — указатель на объект класса родительского окна (производного от класса CWnd). Если этот аргумент равен нулю, то родительским окном объекта класса диалога является главное окно приложения.

Описание

Данная функция вызывается для инициализации объекта класса модального диалогового окна с использованием шаблона диалогового окна, создаваемого пользователем в оперативной памяти.

Чтобы создать диалоговое окно по шаблону, необходимо сначала выделить глобальный блок в оперативной памяти и записать в него информацию о шаблоне диалогового окна. После этого нужно создать объект класса `CDialog` при помощи конструктора данного класса, не принимающего аргументов. После этого следует вызов функции `InitModalIndirect`, сохраняющей дескриптор содержащегося в памяти шаблона диалогового окна. Само диалоговое окно создается и отображается позднее: после вызова функции `DoModal`.

CDialog::DoModal

```
virtual int DoModal();
```

Возвращаемое значение

Целочисленное значение, соответствующее значению аргумента `nResult` функции `CDialog::EndDialog`, которая использовалась для закрытия данного диалогового окна. Возвращаемое значение может быть −1, если функции не удалось создать данное диалоговое окно, или `IDABORT`, если при его создании возникла какая-либо другая ошибка.

Описание

Функция `DoModal` используется для отображения модального диалогового окна и возвращения результатов после его закрытия. Данная функция поддерживает интерфейс с пользователем при отображении диалогового окна. Она делает открытое ею диалоговое окно модальным, т. е. перехватывает все обращения пользователя к приложению и реагирует только на обращения к элементам управления данного диалогового окна. Все остальные обращения пользователя, кроме немаскируемого прерывания, игнорируются.

Если пользователь нажимает в диалоговом окне кнопку **OK** или кнопку **Cancel**, то вызываются соответствующие им функции обработки сообщений `OnOK` или `OnCancel`, закрывающие данное диалоговое окно. По умолчанию функция `OnOK` обновляет содержимое переменных в объекте класса диалогового окна, проверяет их правильность и закрывает диалоговое окно, возвращая величину `IDOK`, а функция `OnCancel` закрывает диалоговое окно без обновления значений переменных в данном объекте и возвращает величину `IDCANCEL`. Пользователь может перегружать эти функции.

В настоящее время в стандартный процесс создания модального диалогового окна включена функция `PreTranslateMessage`.

CDialog::OnInitDialog

```
virtual BOOL OnInitDialog();
```

Возвращаемое значение

Определяет, установило ли приложение фокус ввода на одном из элементов управления в диалоговом окне. Если функция `OnInitDialog` возвращает ненулевое значение, Windows устанавливает фокус ввода на первом элементе управления диалогового окна. Приложение возвращает ноль, только если оно само явно установило фокус ввода на один из элементов управления в диалоговом окне.

Описание

Эта функция обрабатывает сообщение `WM_INITDIALOG`, посылаемое диалоговому окну функциями `Create`, `CreateIndirect` или `DoModal`, вызываемых для вывода диалогового окна на экран.

Перегрузка данной функции позволяет выполнить необходимую обработку данных в объекте класса диалогового окна перед его выводом на экран. Первым оператором перегруженной функции должен быть вызов метода базового класса, однако, возвращаемое им значение нужно игнорировать и в большинстве случаев возвращать значение `TRUE`.

Windows вызывает функцию `OnInitDialog`, используя стандартную глобальную процедуру диалогового окна, общую для всех диалоговых окон, созданных с использованием библиотеки MFC, а не карту сообщений класса диалогового окна, поэтому макрос для данной функции не включается в карту сообщений.

Класс CDocTemplate

Это абстрактный класс, который определяет базисные функциональные возможности шаблонов документа. В функции `CWinApp::InitInstance` обычно создается один или несколько шаблонов документа, определяющих взаимодействие трех типов классов:

- Класса документа, являющегося потомком класса `CDocument`
- Класса представления, отображающего данные, хранящиеся в связанном с ним классе документа. Этот класс может быть потомком класса `CView`, `CScrollView`, `CFormView` или `CEditView`. (Класс `CEditView` может непосредственно использоваться в шаблонах документов.)
- Класса окна, связанного с классом представления. Для однооконного приложения (SDI) базовым является класс `CFrameWnd`, а для многооконного (MDI) — класс `CMDIChildWnd`. Если в приложении применяются нестандартные методы работы с окном, то в шаблоне документа может не-

посредственно использоваться класс CFrameWnd или CMDIChildWnd. В противном случае пользователю придется создать собственный класс, производный от одного из вышеперечисленных

Приложение должно создавать по одному шаблону документа для каждого типа документа. Например, если приложение работает с электронными таблицами и текстовыми документами, оно должно иметь два объекта класса шаблона документа. Каждый шаблон документа отвечает за создание и управление всеми документами типа.

Шаблон документа сохраняет указатели на объекты класса CRuntimeClass, соответствующие классам документа, представления и окна. Эти объекты класса CRuntimeClass создаются при создании шаблона документа.

Шаблон документа содержит идентификаторы ресурсов, используемых совместно с данным типом документа (например, меню, значки или таблицы акселераторов). Шаблон документа также имеет строки, содержащие дополнительную информацию о типе документа. Они включают имя типа документа (например, "Рабочий лист") и расширение файла (например, .xls). Кроме того, шаблон документа может содержать другие строки, содержащие информацию, используемую интерфейсом пользователя приложения и диспетчером файлов Windows.

Если приложение является контейнером объектов OLE, шаблон документа содержит также идентификатор меню, созданного для работы с данным объектом. Если приложение является сервером OLE, шаблон документа, кроме идентификатора меню, содержит идентификатор инструментальной панели. Для определения этих дополнительных ресурсов OLE вызываются функции SetContainerInfo и SetServerInfo.

Поскольку класс CDocTemplate является абстрактным классом, то с его объектами нельзя работать непосредственно. Типичное приложение использует один из двух производных от него классов, определенных в библиотеке MFC: CSingleDocTemplate для однооконного приложения, и CMultiDocTemplate для многооконного.

Если создаваемое пользователем приложение реализует интерфейс пользователя, существенно отличающейся от SDI или MDI, необходимо создать собственный класс, производный от CDocTemplate.

Описание данного класса содержится в файле заголовка afxwin.h.

CDocTemplate::GetDocString

```
virtual BOOL GetDocString(CString& rString,
  enum DocStringIndex index) const;
```

Возвращаемое значение

Ненулевое, если найдена указанная строка, и ноль — в противном случае.

Аргументы

`rString` — ссылка на объект класса `CString`, в который будет помещена искомая строка после окончания работы функции.

`index` — индекс тестового поля, которое следует извлечь из строкового ресурса, связанного с данным объектом класса шаблона документа. Этот аргумент может принимать одно из следующих значений:

- `CDocTemplate::windowTitle` — текст, который появляется в заголовке окна приложения (например, "Microsoft Excel"). Данное поле используется только в однооконном приложении
- `CDocTemplate::docName` — корневой элемент для имени документа по умолчанию (например, "Sheet"). Это имя в сочетании с порядковым номером составляет имя нового документа данного типа при его создании в результате выбора команды меню **File**, **New** (например, "Sheet1" или "Sheet2"). Если это поле пусто, то в качестве корневого имени используется "Untitled"
- `CDocTemplate::fileNewName` — имя данного типа документа. Если приложение работает более чем с одним типом документа, то это имя выводится в диалоговом окне **New** (например, "Worksheet"). Если это имя не задано, то новый документ данного типа не может быть создан, если приложение использует несколько типов документов
- `CDocTemplate::filterName` — описание типа документа и его расширения, используемого для поиска документов данного типа. Эта строка отображается в раскрывающемся списке **Тип Файлов** диалоговых окон **Open** и **Save As** (например, "Worksheets (*.xls)). Если это текстовое поле пусто, то файлы данного типа отображаются только в режиме просмотра всех файлов каталога
- `CDocTemplate::filterExt` — расширение для документов данного типа (например, ".xls"). Если это текстовое поле пусто, то этот тип документа не имеет расширения, подставляемого по умолчанию к его имени
- `CDocTemplate::regFileTypeId` — идентификатор типа документа, сохраняемый в базе данных реестра, поддерживаемого Windows. Эта строка используется исключительно для системных целей (например, "ExcelWorksheet"). Если это текстовое поле пусто, то данный тип документа не может быть зарегистрирован в диспетчере файлов Windows
- `CDocTemplate::regFileTypeName` — имя типа документа, хранимое в базе данных реестра, поддерживаемого Windows. Эта строка может отображаться в диалоговых окнах приложений, имеющих доступ к базе данных реестра (например, "Microsoft Excel Worksheet")

Описание

Функция вызывается для получения текстовых строк, описывающих тип документа. Строка, содержащая эти строки в виде полей, хранится в объекте

класса шаблона документа и передается туда из строкового ресурса, хранящегося в файле ресурсов данного приложения. Приложение вызывает эту функцию, если необходимо получить текстовую информацию для организации интерфейса пользователя. Если для файлов, хранящих документы данного типа, определено особое расширение, то приложение вызывает эту функцию для включения соответствующей записи в базу данных реестра Windows, что позволяет открывать документы этого типа средствами диспетчера файлов Windows. Эта функция может вызываться только в классах, производных от класса `CDocTemplate`.

Класс *CDocument*

Класс `CDocument` обеспечивает основные функциональные возможности создаваемого пользователем класса документа. Документ представляет собой модуль данных, используемых в приложении, и обычно открывается командой **File**, **Open** и сохраняется командой **File**, **Save**.

Класс `CDocument` содержит методы, обеспечивающие создание документа, его загрузку и сохранение. Приложение может поддерживать больше чем один тип документа. Например, оно может работать как с электронными таблицами, так и с текстовыми документами. Каждый тип документа должен быть включен в один из шаблонов. Шаблон документа определяет ресурсы (например, меню, значок или таблица акселераторов), используемые данным типом документа. Каждый документ содержит указатель на связанный объект `CDocTemplate`.

Пользователи взаимодействуют с документом через связанный с ним объект класса `CView`. Класс представления выполняет отображение документа в окне и преобразует вводимую пользователем информацию в соответствующие изменения в документе. Один класс документа может быть связан с несколькими классами представления. Когда пользователь открывает окно документа, приложение создает объект соответствующего класса представления и присоединяет его к документу. Шаблон документа определяет класс представления и класс окна, соответствующие данному типу документа.

Классы документов включены в стандартную процедуру обработки сообщений в приложении и, следовательно, могут получать команды от стандартных компонентов интерфейса пользователя, например, от команды меню **File**, **Save**. Документ получает команды, посланные активным классом представления. Если документ не обрабатывает данную команду, то она передается шаблону документа для дальнейшей обработки.

Когда в документ вносятся изменения, каждый из связанных с ним объектов класса представления должен отразить эти изменения. Класс `CDocument` имеет функцию-член класса `UpdateAllViews`, позволяющую приложению со-

общить всем связанным с данным документом объектам класса представления, что в документ внесены изменения, и данные объекты должны соответствующим образом изменить выводимую ими информацию. Перед закрытием документа приложение запрашивает пользователя о необходимости сохранить внесенные изменения.

Чтобы создать собственный класс документа:

1. Создайте класс, производный от класса `CDocument`.
2. Включите в него переменные для хранения пользовательских данных.
3. Создайте методы для чтения и внесения изменений в эти данные. В основном этими методами будут пользоваться связанные с документом объекты класса представления.
4. Перегрузите функцию `CObject::Serialize`, позволяющую читать и записывать документы на диск.

Описание данного класса содержится в файле заголовка afxwin.h.

CDocument::GetFile

```
virtual CFile* GetFile(LPCTSTR lpszFileName, UINT nOpenFlags,
  CFileException* pError);
```

Возвращаемое значение

Указатель на объект класса `CFile`.

Аргументы

`lpszFileName` — строка, содержащая путь к открываемому файлу. Путь может быть как относительным, так и абсолютным.

`nOpenFlags` — флаги, определяющие режим доступа к файлу, определяющие действия, которые необходимо предпринять при открытии файла. Эти флаги могут принимать те же значения, что и соответствующие флаги конструктора класса `CFile`, и они также объединяются оператором поразрядного ИЛИ (|). Необходимо указать как минимум один флаг доступа и один флаг разделения ресурсов. Флаги `modeCreate` и `modeNoInherit` не являются обязательными.

`pError` — указатель на существующий объект класса исключений файла, в который будут помещены результаты открытия файла.

Описание

Данная функция используется для получения указателя на объект класса `CFile`.

CDocument::GetPathName

```
const CString& GetPathName() const;
```

Возвращаемое значение

Полный путь к файлу документа. Эта строка является пустой, если документ еще не был сохранен или не связан с дисковым файлом.

Описание

Данная функция позволяет получить полный путь к файлу документа на диске.

CDocument::OnNewDocument

```
virtual BOOL OnNewDocument();
```

Возвращаемое значение

Отличное от нуля, если документ был успешно инициализирован, в противном случае — ноль.

Описание

Вызывается приложением как часть обработки команды меню **File**, **New**. По умолчанию данная функция вызывает функцию `DeleteContents`, гарантирующую, что данный документ пуст, и отмечает данный документ как пустой. В данную функцию следует помещать инициализацию всех структур данных нового документа, причем все операторы инициализации должны располагаться после вызова функции базового класса.

Если данная функция вызывается в однооконном приложении, то она уничтожает содержимое существующего документа. В случае многооконного приложения при выборе команды **File**, **New** происходит создание нового документа и уже для него вызывается данная функция. Поскольку в однооконном приложении класс документа создается с помощью конструктора при инициализации приложения, а новые документы создаются путем уничтожения текущего документа при запуске функции `OnNewDocument`, то все процедуры инициализации документа в однооконном приложении должны располагаться в данной функции, а не в конструкторе.

CDocument::ReleaseFile

```
virtual void ReleaseFile(CFile* pFile, BOOL bAbort);
```

Аргументы

`pFile` — указатель на объект класса `CFile`, который необходимо освободить.

bAbort — определяет функцию, которая будет вызвана при освобождении файла. Если данный аргумент имеет значение FALSE, то файл освобождается функцией CFile::Close, в противном случае используется функция CFile::Abort.

Описание

Данная функция вызывается приложением для освобождения файла, делая его доступным для других приложений. Если аргумент bAbort принимает значение TRUE, то функция ReleaseFile вызывает функцию CFile::Abort, и файл освобождается. Функция CFile::Abort не вызывает исключения. Если аргумент bAbort принимает значение FALSE, то функция ReleaseFile вызывает функцию CFile::Close, и файл освобождается.

Данная функция перегружается в том случае, если пользователь должен выполнить определенные действия перед закрытием файла.

CDocument::SetModifiedFlag

```
void SetModifiedFlag(BOOL bModified = TRUE);
```

Аргументы

bModified — флаг, указывающий, вносились ли в документ изменения.

Описание

Данная функция вызывается после внесения любых изменений в документ. Это гарантирует, что при закрытии данного документа приложение выведет диалоговое окно с запросом о том, нужно ли сохранять внесенные в документ изменения. Обычно в качестве аргумента данной функции используется установленное по умолчанию значение TRUE. Чтобы пометить данный документ как чистый (неизмененный), вызовите данную функцию с аргументом bModified, имеющим значение FALSE.

CDocument::UpdateAllViews

```
void UpdateAllViews(CView* pSender, LPARAM lHint = 0L,
  CObject* pHint = NULL);
```

Аргументы

pSender — указатель на объект класса представления, вносящий изменения в данный документ, или NULL, если необходимо внести изменения во все объекты класса представления, связанные с данным документом.

lHint — содержит информацию о вносимых изменениях.

pHint — указатель на объект, содержащий информацию о вносимых изменениях.

Описание

Данная функция вызывается после внесения изменений в документ. Ее вызов должен располагаться после вызова функции `SetModifiedFlag`. Функция `UpdateAllViews` извещает все объекты класса представления, связанные с данным документом, кроме объекта класса представления, на который указывает аргумент `pSender`, о том, что в данный объект класса документа были внесены изменения. Обычно данная функция вызывается, после того как пользователь через один из объектов класса представления внес изменения в данный документ.

Функция `SetModifiedFlag` вызывает функцию `CView::OnUpdate` для каждого объекта класса представления, кроме объекта, через который было внесено изменение. При этом каждому из объектов класса представления передаются значения аргументов `pHint` и `lHint`. Эти аргументы используются для передачи информации объектам класса представления о характере изменений, внесенных в документ. Эта информация может передаваться в аргументе `lHint`. Кроме того, для хранения этой информации может быть создан объект класса, производного от класса `CObject`, указатель на который должен передаваться в аргументе `pHint`.

Перегрузка функции `CView::OnUpdate` в пользовательском классе представления позволяет оптимизировать процедуру обновления содержимого окна на основе переданной этой функции информации.

Класс *CEdit*

Объект класса `CEdit` обеспечивает функционирование элемента управления текстового поля Windows. Текстовое поле Windows представляет собой прямоугольное дочернее окно, в которое пользователь может вводить текст.

Данный элемент управления может создаваться в шаблоне диалога или непосредственно в программе пользователя. В обоих случаях сначала вызывается конструктор класса `CEdit`, создающий объект данного класса, а затем — функция `Create`, создающая текстовое поле Windows и связывающая его с объектом класса `CEdit`.

Объект класса, производного от класса `CEdit`, может быть создан за один шаг, если в его конструкторе будет вызвана функция `Create`. Объект класса `CEdit` наследует многие возможности объекта класса `CWnd`. Чтобы записать текст в объект класса `CEdit`, используется функция `CWnd::SetWindowText`, а чтобы считать его оттуда, вызывается функция `CWnd::GetWindowText`, которая позволяет записать или считать весь текст, содержащийся в данном элементе управления, даже если он является многострочным. В том случае, если данный элемент управления является многострочным, в нем могут использоваться следующие функции для работы с частью содержащегося в нем текста: `CEdit::GetLine`, `CEdit::SetSel`, `CEdit::GetSel` и `CEdit::ReplaceSel`.

Чтобы обработать сообщение, посылаемое текстовым полем своему родительскому окну (обычно это объект класса, производного от `CDialog`), добавьте соответствующий макрос в карту сообщений и создайте функцию для обработки данного сообщения. Макрос карты сообщений для данного типа сообщений имеет следующий формат:

```
ON_Notification(id, memberFxn)
```

где `id` — идентификатор дочернего окна элемента управления, посылающего сообщение, а `memberFxn` — имя функции обработки данного сообщения в классе родительского окна. Прототип функции обработки сообщения имеет следующий формат:

```
afx_msg void memberFxn();
```

Ниже приведен список макросов карты сообщений, которые могут использоваться для обработки сообщений, посылаемых данным элементом управления:

- `ON_EN_CHANGE` — пользователь произвел действие, которое может привести к изменению текста, содержащегося в текстовом поле. В отличие от сообщения `EN_UPDATE` данное сообщение посылается после того, как Windows произведет обновление экрана
- `ON_EN_ERRSPACE` — для данного текстового поля не может быть выделен необходимый объем памяти
- `ON_EN_HSCROLL` — пользователь щелкнул мышью на горизонтальной полосе прокрутки данного элемента управления. Сообщение посылается родительскому окну до обновления экрана
- `ON_EN_KILLFOCUS` — данное текстовое поле потеряло фокус ввода
- `ON_EN_MAXTEXT` — текущая вставка привела к превышению определенного в данном объекте максимального числа символов, что привело к ее урезанию. Данное сообщение посылается также в том случае, если текстовое поле не имеет стиля `ES_AUTOHSCROLL`, а количество символов в текущей строке превышает ширину текстового поля. Также данное сообщение посылается, если текстовое поле не имеет стиля `ES_AUTOHSCROLL`, количество строк в нем превышает его высоту, а количество символов в текущей строке превышает ширину текстового поля
- `ON_EN_SETFOCUS` — данное текстовое поле получило фокус ввода
- `ON_EN_UPDATE` — в текстовом поле будет выводиться измененный текст. Посылается после того, как элемент управления отформатирует текст, но до того как этот текст будет выведен в текстовое поле, что позволяет изменить размеры текстового поля в случае необходимости
- `ON_EN_VSCROLL` — пользователь щелкнул мышью на вертикальной полосе прокрутки данного элемента

Этот объект автоматически уничтожается при закрытии диалогового окна, в котором был создан объект класса CEdit. То же самое происходит, если объект класса CEdit создается в шаблоне диалога.

Если объекта класса CEdit создается в окне, то от пользователя может потребоваться его уничтожить. Если же объект класса CEdit создается в стеке, то он уничтожается автоматически. Объект класса CEdit, создаваемый в куче оператором new, необходимо уничтожить оператором delete по завершении работы пользователя с элементом управления Windows. Если в объект класса, производного от CEdit, была распределена какая-либо память, то необходимо перегрузить деструктор данного класса таким образом, чтобы он освобождал эту память.

Описание данного класса содержится в файле заголовка afxwin.h.

CEdit::CharFromPos

```
int CharFromPos(CPoint pt) const;
```

Возвращаемое значение

В младшем слове, имеющем формат WORD, хранится индекс символа. В старшем слове, также имеющем формат WORD, хранится индекс строки.

Аргументы

pt — координаты точки в рабочей области элемента управления, связанного с данным объектом класса CEdit.

Описание

Данная функция позволяет получить индекс ближайшего к данной точке символа и индекс строки, в которой находится данный символ. Первая строка и первый символ имеют нулевой индекс.

Данная функция может использоваться только в программах, работающих под управлением Windows 95 и Windows NT 4.0.

Класс CEditView

Объект класса CEditView, подобно объекту класса CEdit, обеспечивает функционирование элемента управления текстового поля Windows и позволяет создать простейший текстовый редактор. Класс CEditView поддерживает следующие дополнительные возможности:

- Печать документа
- Поиск и замена

Поскольку класс `CEditView` является производным от класса `CView`, его объекты могут работать с документами и использоваться в шаблонах документов. Для текста, содержащегося в данном объекте элемента управления, выделяется собственная область глобальной памяти. Количество объектов не ограничено.

Использование объектов класса `CEditView` позволяет создавать простейшие оконные редакторы, имеющие перечисленные выше свойства. Объект класса `CEditView` может занимать всю рабочую область окна. Чтобы дополнить данный редактор пользовательскими свойствами, необходимо создать пользовательский класс, производный от класса `CEditView`, и добавить в него соответствующие функции.

По умолчанию класс `CEditView` обрабатывает следующие сообщения: `ID_EDIT_SELECT_ALL`, `ID_EDIT_FIND`, `ID_EDIT_REPLACE`, `ID_EDIT_REPEAT` и `ID_FILE_PRINT`. На объекты класса `CEditView` (или на объекты классов, производных от класса `CEditView`) накладываются следующие ограничения:

- Объект класса `CEditView` не обладает свойствами WYSIWYG. Если требования соответствия изображения на экране и печатаемой копии приводят к снижению качества изображения на экране, объект класса `CEditView` старается улучшить изображение на экране за счет потери соответствия с печатаемой копией
- Объект класса `CEditView` использует только один шрифт для вывода всей информации. При этом не предусматривается никакого специального форматирования текста. Эти возможности реализованы в классе `CRichEditView`
- Объем текста, выводимого объектом класса `CEditView`, ограничен. Эти ограничения совпадают с ограничениями на объект класса `CEdit`

Описание данного класса содержится в файле заголовка afxext.h.

CEditView::FindText

```
BOOL FindText(LPCTSTR lpszFind, BOOL bNext = TRUE, BOOL bCase = TRUE);
```

Возвращаемое значение

Ненулевое, если найден искомый текст, и ноль — в противном случае.

Аргументы

`lpszFind` — указатель на текстовую строку, содержащую искомый текст.

`bNext` — определяет направление поиска. Если данный аргумент имеет значение `TRUE`, то поиск производится по направлению к концу буфера. В противном случае — по направлению к началу буфера.

bCase — определяет необходимость учета регистра символов при сравнении. Если данный аргумент имеет значение TRUE, то регистр учитывается. В противном случае — нет.

Описание

Функция FindText выполняет поиск указанного текста в текстовом буфере объекта класса CEditView. Поиск текста, содержащегося в аргументе lpszFind, начинается с текущей позиции текстового курсора в направлении, определяемом аргументом bNext.

Если аргумент bCase равен TRUE, то при поиске учитывается регистр символов. Если текст найден, то он выделяется, и функция возвращает ненулевую величину. В противном случае возвращается ноль.

CEditView::GetBufferLength

```
UINT GetBufferLength() const;
```

Возвращаемое значение

Длина строки в буфере.

Описание

Данная функция возвращает количество символов, содержащихся в настоящее время в буфере текстового поля, исключая завершающий нулевой символ.

CEditView::GetEditCtrl

```
CEdit& GetEditCtrl() const;
```

Возвращаемое значение

Ссылка на объект класса CEdit.

Описание

Функция GetEditCtrl возвращает ссылку на объект класса текстового поля, используемый в данном объекте класса представления. Поскольку возвращается ссылка на объект класса CEdit, то возвращаемая величина позволяет получить доступ к элементу управления путем вызова функций данного класса.

Использование объекта класса CEdit может изменить состояние связанного с ним элемента управления Windows. Например, при установке позиций табуляции функцией CEdit::SetTabStops позиции табуляции изменятся как в самом элементе управления, так и при его распечатке. Чтобы изменить позиции табуляции только в самом элементе управления, следует вызвать функцию CEditView::SetTabStops.

CEditView::OnFindNext

```
virtual void OnFindNext(LPCTSRT lpszFind, BOOL bNext, BOOL bCase);
```

Аргументы

`lpszFind` — указатель на текстовую строку, содержащую искомый текст.

`bNext` — определяет направление поиска. Если данный аргумент имеет значение `TRUE`, то поиск производится по направлению к концу буфера. В противном случае — по направлению к началу буфера.

`bCase` — определяет необходимость учета регистра символов при сравнении. Если данный аргумент имеет значение `TRUE`, то регистр символов учитывается. В противном случае — нет.

Описание

Функция `OnFindNext` выполняет поиск указанного текста в текстовом буфере объекта класса `CEditView`. Поиск текста, содержащегося в аргументе `lpszFind`, начинается с текущей позиции текстового курсора в направлении, определяемом аргументом `bNext`. Если аргумент `bCase` равен `TRUE`, то регистр символов учитывается.

Для поиска строки данная функция вызывает функцию `FindText`, передавая ей все свои аргументы. По умолчанию, если искомый текст не найден, функция `OnFindNext` вызывает функцию `OnTextNotFound`.

CEditView::OnTextNotFound

```
virtual void OnTextNotFound(LPCTSTR lpszFind);
```

Аргументы

`lpszFind` — указатель на текстовую строку, содержащую искомый текст.

Описание

По умолчанию данная функция вызывает функцию Windows `MessageBeep`.

CEditView::SerializeRaw

```
void SerializeRaw(CArchive& ar);
```

Аргументы

`ar` — ссылка на объект класса `CArchive`, в котором хранится текст документа.

Описание

Функция `SerializeRaw` позволяет использовать объект класса `CArchive` для чтения или записи текстового файла, отображаемого в объекте класса

CEditView. Функция SerializeRaw отличается от функции CEditView::Serialize тем, что она читает только текст без предшествующей ему информации, описывающей данный объект.

Класс *CFile*

Это базовый класс библиотеки MFC для работы с файлами. Он обеспечивает небуферированный двоичный доступ к файлу. Производные от него классы поддерживают работу с текстовыми файлами и файлами в оперативной памяти. Класс CFile используется совместно с классом CArchive для сохранения объектов классов из библиотеки MFC.

Класс CFile, как базовый, обеспечивает интерфейс для всех классов, работающих с файлами. Поэтому, например, работа с файлом, расположенным в памяти, практически не отличается от работы с файлом на диске.

Класс CFile и производные от него классы используются для обычной работы с диском. Для записи на диск форматированного текста используется класс ofstream и другие подобные классы Microsoft.

Обычно файл на диске автоматически открывается конструктором класса CFile и автоматически закрывается его деструктором. Статические функции данного класса позволяют определять статус файла, не открывая его.

Описание данного класса содержится в файле заголовка afx.h.

CFile::CFile

```
CFile();
CFile(int hFile);
CFile(LPCTSTR lpszFileName, UINT nOpenFlags);
throw(CFileException);
```

Аргументы

hFile — дескриптор уже открытого файла.

lpszFileName — строка, содержащая путь к открываемому файлу. Путь может быть как относительным, так и абсолютным.

nOpenFlags — флаги доступа к файлу. Определяют действия, которые необходимо предпринять при открытии файла. Приведенные ниже флаги могут объединяться оператором побитового ИЛИ (|). Необходимо указать как минимум один флаг доступа и один флаг разделения ресурсов. Флаги modeCreate и modeNoInherit не являются обязательными. В данном аргументе могут быть указаны следующие флаги:

- `CFile::modeCreate` — создается новый файл. Если файл с данным именем уже существовал, то для него устанавливается нулевая длина
- `CFile::modeNoTruncate` — если файл с данным именем уже существовал, то для него сохраняется прежняя длина. Этот флаг устанавливается совместно с флагом `modeCreate`. Это позволяет использовать один конструктор для создания новых файлов и открытия уже существующих. Этот флаг используется при создании объектов класса `CStdioFile`
- `CFile::modeRead` — открывает файл только для чтения
- `CFile::modeReadWrite` — открывает файл как для чтения, так и для записи
- `CFile::modeWrite` — открывает файл только для записи
- `CFile::modeNoInherit` — запрещает дочернему процессу использовать данный файл
- `CFile::shareDenyNone` — позволяет другому процессу читать и записывать информацию в данный файл. При открытии данного файла возникает ошибка, если он был открыт каким-либо другим процессом в режиме совместимости
- `CFile::shareDenyRead` — не позволяет другому процессу читать информацию из данного файла. При открытии файла возникает ошибка, если он был открыт в режиме совместимости или для чтения каким-либо другим процессом
- `CFile::shareDenyWrite` — не позволяет другому процессу записывать информацию в данный файл. При открытии данного файла возникает ошибка, если он был открыт в режиме совместимости или для записи каким-либо другим процессом
- `CFile::shareExclusive` — открывает данный файл в режиме безраздельного пользования, запрещая любому другому процессу читать или записывать в него информацию. При открытии файла возникает ошибка, если он был открыт в режиме чтения или записи каким-либо другим процессом или даже тем же самым процессом
- `CFile::shareCompat` — этот флаг отсутствует в 32-разрядной библиотеке MFC. Он соответствует флагу `CFile::shareExclusive` в функции `CFile::Open`
- `CFile::typeText` — устанавливает текстовый режим обработки файла, в котором предусмотрены специальные процедуры обработки пар символов "возврат каретки/перевод строки" (CR/LF, используется только в производных классах)
- `CFile::typeBinary` — устанавливает двоичный режим обработки файла (используется только в производных классах)

Описание

Используемый по умолчанию конструктор данного класса (не имеющий аргументов) не открывает файл, а присваивает величине `m_hFile` значение `CFile::hFileNull`. Поскольку данный класс не вызывает исключения, нет никакого смысла помещать его в структуры `TRY/CATCH`. Вместо этого следует непосредственно проверять условия возникновения исключений при помощи функции `Open`.

Вторая версия конструктора данного класса создает объект класса `CFile`, соответствующий файлу, открытому операционной системой и идентифицируемому дескриптором `hFile`. При этом не осуществляется никаких проверок режима доступа к файлу и его типа. При уничтожении объекта класса `CFile`, созданного подобным конструктором, файл операционной системы не закрывается. Пользователь может уничтожить его самостоятельно.

Третья версия конструктора данного класса создает объект класса `CFile` и открывает файл операционной системы, путь к которому указан в аргументе `lpszFileName`. Этот конструктор можно рассматривать как объединение первой версии конструктора с функцией `Open`. В случае возникновения ошибок при открытии файла данный конструктор вызывает исключения. В большинстве случаев это означает возникновение фатальной ошибки, о которой должно быть сообщено пользователю.

Пример

```
char* pFileName = "test.dat";
TRY
{
   CFile f(pFileName, CFile::modeCreate | CFile::modeWrite);
}
CATCH(CFileException, e)
{
   #ifdef _DEBUG
      afxDump << "File could not be opened " << e->m_cause << "\n";
   #endif
}
END_CATCH
```

CFile::Close

```
virtual void Close();
throw(CFileException);
```

Описание

Закрывает файл, связанный с объектом данного класса, и делает его недоступным для операций чтения и записи. Если файл не был закрыт до уничтожения объекта данного класса, эта операция выполняется его деструктором.

Если объект класса CFile создавался в куче оператором new, его необходимо уничтожить после закрытия файла. Функция Close присваивает переменной m_hFile значение CFile::hFileNull.

CFile::GetFilePath

```
virtual CString GetFilePath() const;
```

Возвращаемое значение

Полный путь к файлу, связанному с данным объектом класса.

Описание

Функция возвращает полный путь к файлу, связанному с данным объектом класса. Например, для файла c:\windows\write\myfile.wri функция GetFilePath возвратит строку "c:\windows\write\myfile.wri".

Чтобы получить имя файла с расширением (myfile.wri), вызовите функцию GetFileName. Только имя файла без расширения (myfile) возвращает функция GetFileTitle.

CFile::GetLength

```
virtual DWORD GetLength() const;
throw(CFileException);
```

Возвращаемое значение

Длина файла в байтах.

Описание

Возвращает текущую логическую длину файла в байтах.

CFile::Read

```
virtual UINT Read(void* lpBuf, UINT nCount);
throw(CFileException);
```

Возвращаемое значение

Количество байт, записанных в буфер. Для всех классов, производных от класса CFile возвращаемое значение может быть меньше, чем значение аргумента nCount, если достигнут конец файла.

Аргументы

lpBuf — указатель на созданный пользователем буфер, в который будут записаны данные из файла.

nCount — количество байт, которые нужно считать из файла. Для текстовых файлов пара управляющих символов "возврат каретки/перевод строки" (CR/LF) считается за один символ.

Описание

Читает данные из файла, связанного с объектом класса CFile, в буфер.

Пример

```
extern CFile cfile;
char pbuf[100];
UINT nBytesRead = cfile.Read(pbuf, 100);
```

CDocument::ReleaseFile

```
virtual void ReleaseFile(CFile* pFile, BOOL bAbort);
```

Аргументы

pFile — указатель на объект класса CFile, который необходимо освободить.

bAbort — определяет функцию, вызываемую при освобождении файла. Если данный аргумент имеет значение FALSE, то будет вызвана функция CFile::Close, а иначе — функция CFile::Abort.

Описание

Функция вызывается приложением для освобождения файла, чтобы сделать его доступным для других приложений. Если аргумент bAbort имеет значение TRUE, функция ReleaseFile для освобождения файла вызывает функцию CFile::Abort. Функция CFile::Abort при своей работе не вызывает исключений.

Если аргумент bAbort имеет значение FALSE, функция ReleaseFile при освобождении файла вызывает функцию CFile::Close.

Перегрузка данной функции позволяет произвести с файлом некоторые операции перед его закрытием.

CFile::Seek

```
virtual LONG Seek(LONG lOff, UINT nFrom);
throw(CFileException);
```

Возвращаемое значение

Если указанная позиция расположена в файле, функция Seek возвращает новую текущую позицию в файле, равную смещению от начала файла, выраженному в байтах. В противном случае возвращаемое значение не определено, и функция вызывает исключение CFileException.

Аргументы

`lOff` — количество байт, на которые необходимо переместить текущую позицию.

`nFrom` — флаг начала отсчета. Может принимать следующие значения:

- `CFile::begin` — аргумент `lOff` определяет смещение от начала файла
- `CFile::current` — аргумент `lOff` определяет смещение от текущей позиции файла
- `CFile::end` — аргумент `lOff` определяет смещение от конца файла. В этом случае величина смещения должна быть отрицательной. При положительной величине смещения поиск будет выполняться за концом файла

Описание

Перемещает текущую позицию в предварительно открытом файле. Функция `Seek` обеспечивает произвольный доступ к содержимому файла. При перемещении текущей позиции чтение информации не производится. При открытии файла текущее смещение указателя от начала файла равно нулю.

Пример

```
extern CFile cfile;
LONG lOffset = 1000, lActual;
lActual = cfile.Seek(lOffset, CFile::begin);
```

CFile::SeekToBegin

```
void SeekToBegin();
throw(CFileException);
```

Описание

Устанавливает текущую позицию на начало файла. Функция `SeekToBegin()` эквивалентна функции `Seek(0L, CFile::begin)`.

Пример

```
extern CFile cfile;
cfile.SeekToBegin();
```

CFile::Write

```
virtual void Write(const void* lpBuf, UINT nCount);
throw(CFileException);
```

Аргументы

`lpBuf` — указатель на созданный пользователем буфер, содержащий данные, которые будут записаны в файл.

nCount — количество байт, которые нужно записать в файл. Для текстовых файлов пара управляющих символов "возврат каретки/перевод строки" считается за один символ.

Описание

Записывает данные из буфера в файл, связанный с объектом класса CFile. В некоторых случаях, включая переполнение диска, функция Write вызывает исключение.

Пример

```
extern CFile cfile;
char pbuf[100];
cfile.Write(pbuf, 100);
```

Класс *CFont*

Объект класса CFont предназначен для работы со шрифтами, созданными интерфейсом графических устройств Windows (GDI). Прежде чем использовать объект класса CFont его необходимо создать и связать с ним шрифт Windows при помощи функций CreateFont, CreateFontIndirect, CreatePointFont или CreatePointFontIndirect. После этого можно вызывать функции-члены данного класса для работы со шрифтом.

Работа с функциями CreatePointFont или CreatePointFontIndirect во многих случаях проще, чем с функциями CreateFont или CreateFontIndirect, поскольку они выполняют автоматический перевод высоты шрифта, измеренной в пиках, в высоту шрифта, измеренную в логических единицах.

Описание данного класса содержится в файле заголовка afxwin.h.

CFont::CreateFontIndirect

```
BOOL CreateFontIndirect(const LOGFONT* lpLogFont);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

lpLogFont — указатель на объект структуры LOGFONT, содержащий параметры логического шрифта.

Описание

Инициализирует объект класса CFont с использованием переменных объекта структуры LOGFONT, на который указывает аргумент lpLogFont. После этого

объект данного класса может выбираться в качестве текущего шрифта в контекст устройства.

Этот шрифт имеет характеристики, заданные структурой LOGFONT. Для выбора шрифта в контекст устройства вызывается функция CDC::SelectObject. При этом программа масштабирования шрифтов графического интерфейса пытается найти среди имеющихся физических шрифтов такой шрифт, который бы максимально соответствовал указанному логическому шрифту. Если не удается достичь полного соответствия, то загружается ближайший по параметрам шрифт.

По завершении работы с объектом класса CFont, созданным функцией CreateFontIndirect, его необходимо сначала удалить из контекста устройства, а затем уничтожить объект данного класса.

Класс *CFrameWnd*

Класс CFrameWnd обеспечивает основные функциональные возможности перекрывающихся и всплывающих окон однооконного приложения Windows (SDI), а также функции для работы с элементами управления окна.

Чтобы создать рабочее окно приложения, необходимо создать класс окна, производный от класса из CFrameWnd, и добавить в него переменные для хранения данных, специфичных для создаваемого приложения. Затем создать карту сообщений и функции для обработки сообщений, поступающих в данное окно.

Имеются три способа создать окно:

- Функцией Create
- Функцией LoadFrame
- При помощи шаблона документа

Прежде чем использовать функции Create или LoadFrame для непосредственного создания окна, необходимо создать в стеке объект класса CFrameWnd при помощи стандартного оператора C++ new. Перед вызовом функции Create данный класс можно зарегистрировать глобальной функцией AfxRegisterWndClass, сопоставляющей данному окну значок и устанавливающей стили окна.

Функция Create позволяет непосредственно задавать параметры создания окна, а функция LoadFrame упрощает этот процесс за счет использования меньшего числа аргументов и установки большинства параметров окна по умолчанию на основе информации, содержащейся в соответствующих ресурсах, включая заголовок окна, значок, таблицу акселераторов и меню. Для того чтобы содержащаяся в них информация могла быть использована в

функции LoadFrame, все эти ресурсы должны иметь один и тот же идентификатор ресурса (например, IDR_MAINFRAME).

Связанные с объектом класса CFrameWnd объекты класса представления и класса документа создаются самим приложением, а не программистом. Объект класса CDocTemplate обеспечивает одновременное создание связанных объектов классов окна, просмотра и документа. Параметры конструктора класса CDocTemplate определяют характеристики объектов класса CRuntimeClass, создаваемых для обращения к трем связанным классам (документа, окна и представления). Классы, производные от CRuntimeClass, позволяют приложению создавать свои объекты в процессе выполнения приложения (например, при обработке команды меню **File**, **New** или команды меню многооконного приложения **Window**, **New Window**).

При создании класса окна, производного от класса CFrameWnd, должен использоваться макрос DECLARE_DYNCREATE, обеспечивающий правильную работу описанного выше механизма RUNTIME_CLASS.

Объект класса CFrameWnd содержит заданные по умолчанию реализации следующих функций основного окна типичного приложения Windows:

- Он следит за объектом класса просмотра, соответствующего неактивному окну Windows или не имеющего на данный момент фокуса ввода. При активизации данного окна объект класса CFrameWnd вызывает функцию CView::OnActivateView
- Он транслирует в активный объект класса представления основные сообщения, включая сообщения, обрабатываемые функциями OnSetFocus, OnHScroll и OnVScroll класса CWnd
- Активный объект класса представления (или активное окно многооконного приложения) может установить текущий заголовок окна. Эта возможность может быть заблокирована сбросом бита FWS_ADDTOTITLE в стиле окна
- Объект класса CFrameWnd управляет позиционированием элементов управления, областей просмотра и других дочерних окон внутри рабочей области окна. Объект данного класса также обновляет состояние кнопок панели инструментов и других элементов управления в окне при отсутствии других исполняемых задач. Кроме того, объект класса CFrameWnd содержит стандартные процедуры отображения и исключения с экрана панелей инструментов и строки состояния
- Объект данного класса управляет отображением главного меню приложения. При раскрытии подменю объект класса окна использует макрос UPDATE_COMMAND_UI, чтобы определить, использование каких пунктов меню следует разрешить, какие пункты меню следует заблокировать, а какие из них нужно отметить. Когда пользователь выбирает команду меню, окно изменяет состояние элементов управления в соответствии с выбранной командой

- Объект класса `CFrameWnd` использует таблицу акселераторов для преобразования нажатий пользователем определенных комбинаций клавиш в конкретные действия
- Объект класса `CFrameWnd` может содержать набор идентификаторов, используемых функцией `LoadFrame` для создания контекстно-зависимой справки. Объект данного класса также организует работу с полумодальными окнами, используемыми для отображения контекстно-зависимой справки (<Shift>+<F1>) и в режиме предварительного просмотра печати
- Объект данного класса позволяет открыть файл, перемещенный из Диспетчера файлов в данное окно. Если расширение файла зарегистрировано и связано с приложением, данный объект может ответить на запрос о динамическом обмене данными (DDE), пришедший из Диспетчера файлов при открытии пользователем соответствующего файла или при вызове функции Windows `ShellExecute`
- Если объект данного класса соответствует главному окну приложения, то при закрытии пользователем данного окна ему выдается запрос о необходимости сохранить все изменения, внесенные им в документы в данном сеансе работы с приложением. Для этого вызываются функции `OnClose` и `OnQueryEndSession`
- Если объект данного класса соответствует главному окну приложения, то это окно предоставляет контекст устройства при вызове справочной системы. Закрытие окна вызовет прекращение исполнения файла Winhelp.exe, если в данном приложении была вызвана справка

Объект данного класса уничтожается при вызове функции `CFrameWnd::PostNcDestroy`. При закрытии окна пользователем функция `OnClose` вызывает функцию `DestroyWindow`.

Описание данного класса содержится в файле заголовка afxwin.h.

CFrameWnd::Create

```
BOOL Create(LPCTSTR lpszClassName, LPCTSTR lpszWindowName,
  DWORD dwStyle = WS_OVERLAPPEDWINDOW, const RECT& rect = rectDefault,
  CWnd* pParentWnd = NULL, LPCTSTR lpszMenuName = NULL,
  DWORD dwExStyle = 0, CCreateContext* pContext = NULL);
```

Возвращаемое значение

Ненулевое, если инициализация прошла успешно, и ноль — в противном случае.

Аргументы

`lpszClassName` — указатель на завершающуюся нулем символьную строку, содержащую имя класса Windows. Имя класса может представлять собой

любое имя, зарегистрированное в глобальной функции AfxRegisterWndClass или в функции Windows RegisterClass. Если этот параметр равен нулю, то используются атрибуты, установленные по умолчанию для класса CFrameWnd.

lpszWindowName — указатель на заканчивающуюся нулем символьную строку, содержащую имя окна. Используется для вывода заголовка окна.

dwStyle — определяет атрибуты стиля окна. Чтобы иметь возможность автоматически выводить в заголовок окна имя отображаемого в нем документа, установите стиль FWS_ADDTOTITLE.

rect — определяет размер и положение окна на экране. Использование переменной rectDefault позволяет Windows определить размер и положение нового окна.

pParentWnd — определяет родительское окно. Для главного окна программы этот аргумент равен нулю.

lpszMenuName — определяет имя ресурса меню, используемого совместно с данным окном. Целочисленный идентификатор ресурса используется в макросе MAKEINTRESOURCE. Данный аргумент может принимать нулевое значение.

dwExStyle — определяет атрибуты расширенного стиля окна.

pContext — указатель на объект структуры CCreateContext. Данный аргумент может принимать нулевое значение.

Описание

Создание объекта класса, производного от CFrameWnd выполняется в два этапа. Сначала вызывается конструктор, создающий объект класса CFrameWnd, а затем вызывается функция Create. Она загружает главное окно приложения и связанные с ним ресурсы и сопоставляет загруженное окно объекту, производному от класса CMainFrame. Данная функция позволяет определить имя класса окна, заголовок окна, параметры стиля окна, его родительское окно и связанный с ним ресурс меню.

Функция LoadFrame загружает окно, используя файл ресурсов вместо непосредственного задания соответствующих аргументов.

CFrameWnd::DockControlBar

```
void DockControlBar(CControlBar * pBar, UINT nDockBarID = 0,
  LPCRECT lpRect = NULL);
```

Аргументы

pBar — указатель на объект класса CControlBar, панель которого необходимо фиксировать.

nDockBarID — определяет сторону рабочей области окна, на которой будет произведена фиксация. Данный аргумент может принимать одно из перечисленных ниже значений:

- AFX_IDW_DOCKBAR_TOP — фиксирует панель управления на верхней стороне главного окна приложения
- AFX_IDW_DOCKBAR_BOTTOM — фиксирует панель управления на нижней стороне главного окна приложения
- AFX_IDW_DOCKBAR_LEFT — фиксирует панель управления на левой стороне главного окна приложения
- AFX_IDW_DOCKBAR_RIGHT — фиксирует панель управления на правой стороне главного окна приложения
- 0 — панель управления может быть зафиксирована на любой стороне, разрешенной для фиксации в данном окне

lpRect — определяет в экранных координатах положение, в котором панель управления будет зафиксирована в служебной области данного окна.

Описание

Фиксирует указанную панель управления в главном окне приложения. Панель управления может быть зафиксирована на любой стороне главного окна приложения, определенной для этой цели в соответствующих функциях CControlBar::EnableDocking и CFrameWnd::EnableDocking. Сторона, на которой производится фиксация, определяется аргументом nDockBarID.

CFrameWnd::EnableDocking

```
void EnableDocking(DWORD dwDockStyle);
```

Аргументы

dwDockStyle — определяет, какая из сторон окна может служить для фиксации панели управления. Использует следующие флаги:

- CBRS_ALIGN_TOP — разрешает фиксацию на верхней стороне рабочей области окна
- CBRS_ALIGN_BOTTOM — разрешает фиксацию на нижней стороне рабочей области окна
- CBRS_ALIGN_LEFT — разрешает фиксацию на левой стороне рабочей области окна
- CBRS_ALIGN_RIGHT — разрешает фиксацию на правой стороне рабочей области окна
- CBRS_ALIGN_ANY — разрешает фиксацию на любой стороне рабочей области окна

Описание

Данная функция управляет фиксацией панелей управления в окне. По умолчанию панели управления фиксируются в окне в следующем порядке: сверху, снизу, слева, справа.

CFrameWnd::LoadBarState

```
void LoadBarState(LPCTSTR lpszProfileName);
```

Аргументы

`lpszProfileName` — имя секции в файле инициализации или указатель на строку, содержащую имя ключа в системном регистре, по которому будет получена информация.

Описание

Функция позволяет восстановить установки для каждой панели управления, принадлежащей главному окну приложения. Эта информация (о видимости, горизонтальной или вертикальной ориентации, фиксации и расположения панели управления) сохраняется функцией `SaveBarState`.

CFrameWnd::LoadFrame

```
virtual BOOL LoadFrame(UINT nIDResource,
  DWORD dwDefaultStyle = WS_OVERLAPPEDWINDOW | FWS_ADDTOTITLE,
  CWnd* pParentWnd = NULL, CCreateContext* pContext = NULL);
```

Аргументы

`nIDResource` — идентификатор разделяемого ресурса, связанного с данным окном.

`dwDefaultStyle` — стиль окна. Чтобы иметь возможность автоматически выводить в заголовок окна имя отображаемого в нем документа, установите стиль `FWS_ADDTOTITLE`.

`pParentWnd` — указатель на класс родительского окна.

`pContext` — указатель на объект структуры `CCreateContext`. Может равняться нулю.

Описание

Создание объекта класса, производного от класса `CFrameWnd` выполняется в два этапа. Сначала вызывается конструктор, создающий объект класса `CFrameWnd`, а затем вызывается функция `LoadFrame`, загружающая главное окно приложения и связанные с ним ресурсы и ассоциирует загруженное окно с объектом, производным от класса `CMainFrame`. В переменной `nIDResource`,

передаваемой функции в качестве параметра, определены меню, таблица назначения клавиш, значки и строковый ресурс заголовка окна.

Для задания всех параметров создания окна вместо функции `LoadFrame` следует использовать функцию `Create`. Приложение вызывает функцию `LoadFrame` для создания главного окна программы на основе объекта класса шаблона документа.

Аргумент `pContext` данной функции задает объекты классов, связанные с данным классом окна, включая все объекты класса представления.

CFrameWnd::SaveBarState

```
void SaveBarState(LPCTSTR lpszProfileName) const;
```

Аргументы

`lpszProfileName` — имя секции в файле инициализации или указатель на строку, содержащую имя ключа в системном регистре, по которому будет записана информация.

Описание

Данная функция вызывается для сохранения информации об установках каждой панели управления, принадлежащей данному окну. Эта информация может быть считана из системного реестра функцией `LoadBarState`. Записываемая информация касается видимости, горизонтальной или вертикальной ориентации, фиксации и расположения панели управления.

CFrameWnd::ShowControlBar

```
void ShowControlBar(CControlBar* pBar, BOOL bShow, BOOL bDelay);
```

Аргументы

`pBar` — указатель на панель управления, которую необходимо вывести на экран или убрать с экрана.

`bShow` — если данный аргумент имеет значение `TRUE`, то панель управления следует вывести на экран. В противном случае ее необходимо убрать с экрана.

`bDelay` — если данный аргумент имеет значение `TRUE`, то панель управления следует вывести на экран с некоторой задержкой. В противном случае ее необходимо вывести немедленно.

Описание

Данная функция позволяет выводить панель управления на экран и убирать ее с экрана.

Класс *CGdiObject*

Это базовый класс для классов объектов интерфейса графических устройств Windows, таких как битовые образы, области окон, кисти, перья, палитры и шрифты. Объекты данного класса никогда не используются непосредственно. Вместо них создаются объекты производного класса, например класса `CPen` или `CBrush`.

Описание данного класса содержится в файле заголовка afxwin.h.

CGdiObject::DeleteObject

```
BOOL DeleteObject();
```

Возвращаемое значение

Ненулевое при успешном удалении объекта класса GDI, в противном случае — ноль.

Описание

Удаляет связанный с объектом данного класса объект Windows GDI из памяти, освобождая при этом все связанные с ним буфера. Данная функция не затрагивает память, занимаемую самим объектом класса `CGdiObject`. Приложение не должно вызывать функцию `DeleteObject` для объекта класса `CGdiObject`, выбранного в настоящее время в контекст устройства.

При уничтожении кисти, использующей образец, связанный с ней объект битового образа не удаляется. Его необходимо удалить после удаления связанной с ним кисти.

Класс *CHeaderCtrl*

Элемент управления заголовком представляет собой окно, обычно располагающееся над столбцами, содержащими текстовую или числовую информацию. Он содержит заголовок каждого столбца и может быть разделен на секции. Пользователь может перемещать границы секций, изменяя таким образом ширину каждого столбца.

Класс `CHeaderCtrl` содержит функции для работы со стандартным элементом управления заголовка Windows. Этот элемент управления (а следовательно и класс `CHeaderCtrl`) может использоваться только в программах, работающих под управлением операционных систем Windows 95/98 и Windows NT версии 3.51 и более поздних версий.

Internet Explorer 4.0 добавил следующие свойства в стандартные элементы управления:

- Возможность упорядочения заголовков элементов данных по установленному пользователем критерию
- Возможность перетаскивания заголовков элементов данных с целью их упорядочения. Для этого необходимо задать стиль HDS_DRAGDROP при создании объекта класса CHeaderCtrl
- Текст заголовка столбца остается видимым в процессе изменения размера столбцов. Для этого необходимо задать стиль HDS_FULLDRAG при создании объекта класса CHeaderCtrl
- Выделение заголовка, если указатель мыши задержится на нем дольше определенного времени. Необходимо задать стиль HDS_HOTTRACK при создании объекта класса CHeaderCtrl
- Поддержка списка изображений. Заголовки могут содержать не только текст, но и значки, сохраняемые в объектах класса CImageList

Описание данного класса содержится в файле заголовка afxcmn.h.

CHeaderCtrl::GetItem

```
BOOL GetItem(int nPos, HDITEM* pHeaderItem) const;
```

Возвращаемое значение

Ненулевое в случае успешного завершения функции и ноль в противном случае.

Аргументы

nPos — индекс элемента, информацию о котором следует получить.

pHeaderItem — указатель на объект структуры HDITEM, в который будет записана информация о заголовке столбца. Этот объект структуры используется в функциях InsertItem и SetItem. Любой флаг, установленный в переменной mask данной структуры, гарантирует, что значения соответствующих элементов будут корректными после завершения работы данной функции. Если в переменной mask не установлен какой-либо флаг, то значение соответствующей ему переменной не несет в себе никакой информации.

Описание

Позволяет получить информацию о заголовке соответствующего столбца.

CHeaderCtrl::SetImageList

```
CImageList* SetImageList(CImageList* pImageList);
```

Возвращаемое значение

Указатель на объект класса CImageList, который был до этого связан с данным элементом управления.

Аргументы

`pImageList` — указатель на объект класса `CImageList`, содержащий список изображений, который должен быть связан с данным элементом управления.

Описание

Связывает список изображений с существующим элементом управления. Объект класса `CImageList`, указатель на который возвращается данной функцией, является временным и уничтожается при первом удобном случае.

CHeaderCtrl::SetItem

```
BOOL SetItem(int nPos, HDITEM* pHeaderItem);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

`nPos` — индекс заголовка столбца, в который необходимо внести изменения. Индекс первого заголовка столбца равен нулю.

`pHeaderItem` — указатель на объект структуры `HDITEM`, содержащий информацию об изменениях, которые необходимо внести в заголовок столбца.

Описание

Устанавливает атрибуты указанного заголовка столбца.

Структура *CHOOSEFONT*

```
typedef struct {
    DWORD        lStructSize;
    HWND         hwndOwner;
    HDC          hDC;
    LPLOGFONT    lpLogFont;
    INT          iPointSize;
    DWORD        Flags;
    DWORD        rgbColors;
    LPARAM       lCustData;
    LPCFHOOKPROC lpfnHook;
    LPCTSTR      lpTemplateName;
    HINSTANCE    hInstance;
    LPTSTR       lpszStyle;
    WORD         nFontType;
```

```
    WORD          __MISSING_ALIGNMENT__;
    INT           nSizeMin;
    INT           nSizeMax;
} CHOOSEFONT;
```

Содержит информацию, используемую функцией `ChooseFont` для инициализации стандартного диалогового окна **Шрифт**. После того как пользователь закроет это диалоговое окно, в данную структуру будет помещена информация об установках элементов управления данного диалогового окна.

`lStructSize` — определяет размер объекта данной структуры в байтах.

`hwndOwner` — дескриптор окна, которому принадлежит данное диалоговое окно. В этой переменной может размещаться любой допустимый дескриптор окна или значение `NULL`, если диалоговое окно не имеет владельца.

`hDC` — дескриптор контекста устройства (или информационного контекста) принтера, шрифт которого будет отображаться в диалоговом окне. Значение этой переменной учитывается только в том случае, когда в переменной `Flags` установлен флаг `CF_PRINTERFONTS` или `CF_BOTH`. В противном случае значение данной переменной игнорируется.

`lpLogFont` — указатель на объект структуры `LOGFONT`. Если в переменной `Flags` установлен флаг `CF_INITTOLOGFONTSTRUCT`, функция `ChooseFont` использует значения переменных объекта структуры `LOGFONT` для инициализации диалогового окна. При этом выбираются параметры шрифта наиболее близкого к заданным параметрам. Если пользователь нажал кнопку **OK**, функция `ChooseFont` устанавливает значения переменных в объекте структуры `LOGFONT` исходя из установок элементов управления диалогового окна.

`iPointSize` — устанавливает размер выделенного шрифта в единицах, равных 1/10 пики. Функция `ChooseFont` устанавливает значение данной переменной после закрытия диалогового окна.

`Flags` — набор битовых флагов, используемых для инициализации диалогового окна **Шрифт**. После закрытия этого окна флаги содержат информацию об установках элементов управления диалогового окна. Эта переменная может включать в себя следующие флаги:

- `CF_APPLY` — диалоговое окно содержит кнопку **Применить**. При установке данного флажка пользователь должен написать функцию обратного вызова для обработки сообщения `WM_COMMAND`, посылаемого кнопкой **Применить**. Функция обратного вызова может посылать диалоговому окну сообщение `WM_CHOOSEFONT_GETLOGFONT` для получения указателя на объект структуры `LOGFONT`, содержащего текущие параметры выделенного шрифта
- `CF_ANSIONLY` — данный флаг является устаревшим. Чтобы ограничить выбор шрифтов всеми рукописными шрифтами, кроме использующих

набор символов OEM или Symbol, используйте флаг CF_SCRIPTSONLY. Чтобы эмулировать поведение флажка CF_ANSIONLY из Windows 3.1, установите флажок CF_SELECTSCRIPT и присвойте значение ANSI_CHARSET переменной lfCharSet объекта структуры LOGFONT, на который указывает переменная lpLogFont

- CF_BOTH — выводит в диалоговом окне все имеющиеся шрифты принтеров и дисплея. Переменная hDC данного объекта содержит в этом случае контекст устройства (или информационный контекст), связанный с принтером. Этот флажок представляет собой комбинацию флагов CF_SCREENFONTS и CF_PRINTERFONTS
- CF_TTONLY — определяет, что функция ChooseFont должна пронумеровать и предоставить выбор исключительно из шрифтов TrueType
- CF_EFFECTS — выводит в диалоговом окне элементы управления, с помощью которых пользователь задает подчеркивание, зачеркивание и цвет шрифта. Если этот флажок установлен, то переменная rgbColors задает изначальный цвет шрифта. Пользователь может также использовать переменные lfStrikeOut и lfUnderline объекта структуры LOGFONT, на который указывает переменная lpLogFont, для установки или сброса флажков подчеркивания и зачеркивания шрифта. Функция ChooseFont сохраняет в них состояние этих флажков
- CF_ENABLEHOOK — разрешает использовать функцию обратного вызова, указатель на которую располагается в переменной lpfnHook
- CF_ENABLETEMPLATE — указывает, что переменные hInstance и lpTemplateName содержат шаблон диалогового окна, который следует использовать вместо стандартного шаблона
- CF_ENABLETEMPLATEHANDLE — указывает на то, что переменная hInstance определяет блок данных, в котором содержится шаблон диалогового окна. В этом случае система игнорирует значение переменной lpTemplateName
- CF_FIXEDPITCHONLY — указывает на то, что функция ChooseFont должна выделять только шрифты фиксированного размера (не пропорциональные)
- CF_FORCEFONTEXIST — указывает, что функция ChooseFont должна сообщать о возникновении ошибки, если пользователь пытается выбрать несуществующий шрифт или стиль
- CF_INITTOLOGFONTSTRUCT — указывает, что функция ChooseFont должна использовать объект структуры LOGFONT, на который указывает переменная lpLogFont при инициализации элементов управления диалогового окна
- CF_LIMITSIZE — указывает, что функция ChooseFont должна выделять шрифты, размеры которых находятся в диапазоне, заданном переменными nSizeMin и nSizeMax

- CF_NOOEMFONTS — аналогичен флажку CF_NOVECTORFONTS
- CF_NOFACESEL — при использовании объекта структуры LOGFONT для инициализации элементов управления диалогового окна данный флаг предотвращает преждевременный вывод имени шрифта в соответствующей комбинированный список. Эта установка бывает полезной, если в выделенном фрагменте текста присутствуют различные шрифты
- CF_NOSCRIPTSEL — делает недоступным раскрывающийся список **Набор символов:**. Если данный флаг установлен, то переменной lfCharSet объекта структуры LOGFONT при закрытии диалогового окна присваивается значение DEFAULT_CHARSET. Этот флаг используется только при инициализации диалогового окна
- CF_NOSTYLESEL — при использовании объекта структуры LOGFONT для инициализации элементов управления диалогового окна данный флаг предотвращает преждевременный вывод стиля шрифта в соответствующей комбинированный список. Эта установка бывает полезной, если в выделенном фрагменте текста присутствуют различные шрифты
- CF_NOSIZESEL — при использовании объекта структуры LOGFONT для инициализации элементов управления диалогового окна данный флаг предотвращает преждевременный вывод размера шрифта в соответствующей комбинированный список. Эта установка бывает полезной, если в выделенном фрагменте текста присутствуют различные шрифты
- CF_NOSIMULATIONS — определяет, что функция ChooseFont не должна допускать эмуляции шрифтов интерфейсом графических устройств Windows (GDI)
- CF_NOVECTORFONTS — определяет, что функция ChooseFont не должна допускать выделения векторных шрифтов
- CF_NOVERTFONTS — указывает диалоговому окну **Шрифт** на необходимость вывода шрифтов только с горизонтальной ориентацией символов
- CF_PRINTERFONTS — указывает диалоговому окну на необходимость вывода только тех шрифтов, которые поддерживаются принтером, связанным с контекстом устройства (или информационным контекстом), определяемым переменной hDC
- CF_SCALABLEONLY — определяет, что функция ChooseFont должна допускать выделение только масштабируемых шрифтов. (К масштабируемым относятся векторные шрифты, масштабируемые шрифты принтеров, шрифты TrueType и шрифты, масштабируемые с использованием других технологий.)
- CF_SCREENFONTS — указывает диалоговому окну на необходимость вывода только экранных шрифтов, поддерживаемых системой
- CF_SCRIPTSONLY — определяет, что функция ChooseFont должна допускать выделение шрифтов, использующих набор символов Symbol и

ANSI OEM, и не допускать выбора шрифтов, использующих набор символов OEM. Этот флаг заменяет флаг CF_ANSIONLY

- CF_SELECTSCRIPT — при инициализации диалогового окна этот флажок указывает на возможность вывода только шрифтов, использующих набор символов, определенный в переменной lfCharSet объекта структуры LOGFONT. При этом пользователю запрещается вносить изменения в набор символов, определенный в раскрывающемся списке **Набор символов:**
- CF_SHOWHELP — в диалоговом окне будет выведена кнопка **Справка**. В переменной hwndOwner должно быть определено окно, которое будет получать сообщение HELPMSGSTRING, посылаемое диалоговым окном при нажатии этой кнопки
- CF_USESTYLE — указывает, что в переменной lpszStyle содержится указатель на буфер, содержащий данные о стиле, которые функция ChooseFont должна использовать при инициализации комбинированного списка **Начертание:**. При закрытии диалогового окна функция ChooseFont копирует данные из этого раскрывающегося списка в указанный буфер
- CF_WYSIWYG — определяет, что функция ChooseFont должна допускать выделение только тех шрифтов, которые присутствуют как на принтере, так и на дисплее. Если установлен данный флажок, то одновременно должны быть установлены флажки CF_BOTH и CF_SCALABLEONLY

rgbColors — если установлен флаг CF_EFFECTS, переменная rgbColors содержит исходный цвет текста. После успешного завершения работы функции ChooseFont эта переменная содержит значения RGB цвета выделенного пользователем текста.

lCustData — указатель на блок данных, передаваемых системой функции обратного вызова, указатель на которую содержится в переменной lpfnHook. Когда система посылает сообщение WM_INITDIALOG функции обратного вызова, в аргументе lParam данного сообщения содержится указатель на объект структуры CHOOSEFONT, определенный при создании данного диалогового окна. Функция обратного вызова может использовать этот указатель для получения доступа к переменной lCustData.

lpfnHook — указатель на функцию обратного вызова CFHookProc, обрабатывающую сообщения диалогового окна. Значение данной переменной игнорируется, если не установлен флаг CF_ENABLEHOOK в переменной Flags.

lpTemplateName — указатель на заканчивающуюся нулем текстовую строку, содержащую имя шаблона ресурса диалогового окна, определенного в переменной hInstance. Этот шаблон используется вместо стандартного шаблона диалогового окна. Для перечислимых ресурсов диалогового окна переменная lpTemplateName может хранить значение, возвращаемое мак-

росом MAKEINTRESOURCE. Значение данной переменной игнорируется, если не установлен флаг CF_ENABLETEMPLATE в переменной Flags.

hInstance — если установлен флаг CF_ENABLETEMPLATEHANDLE в переменной Flags, то переменная hInstance содержит дескриптор объекта, расположенного в оперативной памяти и содержащего шаблон диалогового окна. Если установлен флаг CF_ENABLETEMPLATE, переменная hInstance определяет модуль, содержащий шаблон диалогового окна, имя которого содержится в переменной lpTemplateName. Если ни один из указанных выше флажков не установлен, значение данной переменной игнорируется.

lpszStyle — указатель на буфер, содержащий информацию о стиле. Если установлен флаг CF_USESTYLE, функция ChooseFont использует эту информацию при инициализации диалогового окна. При закрытии диалогового окна функция ChooseFont копирует в этот буфер информацию о стилях.

nFontType — определяет тип выделенного шрифта при завершении работы функции ChooseFont. Эта переменная может содержать следующие флаги:

- BOLD_FONTTYPE — шрифт является полужирным. Эта информация дублируется в переменной lfWeight объекта структуры LOGFONT и эквивалентна флагу FW_BOLD
- ITALIC_FONTTYPE — установлен атрибут курсива. Эта информация дублируется в переменной lfItalic объекта структуры LOGFONT
- PRINTER_FONTTYPE — шрифт является внутренним шрифтом принтера
- REGULAR_FONTTYPE — шрифт является нормальным. Эта информация дублируется в переменной lfWeight объекта структуры LOGFONT и эквивалентна флагу FW_REGULAR
- SCREEN_FONTTYPE — шрифт является экранным шрифтом
- SIMULATED_FONTTYPE — шрифт эмулируется интерфейсом графических устройств Windows (GDI)

nSizeMin — определяет минимальный размер выделяемого пользователем шрифта. Функция ChooseFont использует значение этой переменной только при установленном флаге CF_LIMITSIZE.

nSizeMax — определяет максимальный размер выделяемого пользователем шрифта. Функция ChooseFont использует значение этой переменной только при установленном флаге CF_LIMITSIZE.

Класс *CImageList*

Список изображений представляет собой набор изображений одинакового размера, к которым можно обращаться по индексу. Первое изображение имеет нулевой индекс. Списки изображений увеличивают эффективность

работы с большими наборами значков и битовых образов. Все изображения в списке содержатся в одном большом битовом образе, хранящемся в экранном формате. Список изображения может включать в себя монохромный битовый образ, содержащий маски для рисования прозрачных изображений (стиль значков). Интерфейс Win32 API предоставляет пользователю функции для работы со списком изображений, позволяющие выводить изображения на экран, а также создавать и уничтожать списки изображений, добавлять, удалять, заменять, объединять и перемещать изображения, включенные в списки.

Класс `CImageList` содержит функции для работы со списком изображений Windows. Этот элемент управления (а, следовательно, и класс `CImageList`) доступен только в программах, выполняющихся под управлением операционных систем Windows 95/98/NT (3.51 и старше).

Описание данного класса содержится в файле заголовка afxcmn.h.

CImageList::Add

```
int Add(CBitmap* pbmImage, CBitmap* pbmMask);
int Add(CBitmap* pbmImage, COLORREF crMask);
int Add(HICON hIcon);
```

Возвращаемое значение

В случае успешного завершения функции — индекс нового изображения, в противном случае −1. Индекс первого изображения в списке равен нулю.

Аргументы

`pbmImage` — указатель на битовый образ, содержащий изображение или изображения. Количество изображений определяется шириной битового образа.

`pbmMask` — указатель на битовый образ, содержащий маску. Если список изображений не использует маску, значение данного аргумента игнорируется.

`crMask` — цвет, используемый для создания маски. Каждый элемент изображения, имеющий данный цвет в соответствующем битовом образе, становится черным, а соответствующий бит маски устанавливается в единицу.

`hIcon` — дескриптор значка, содержащий битовый образ и маску нового изображения.

Описание

Добавляет значок или одно или несколько изображений в список изображений.

CImageList::BeginDrag

```
BOOL BeginDrag(int nImage, CPoint ptHotSpot);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

`nImage` — индекс перетаскиваемого изображения. Индекс первого элемента в списке равен нулю.

`ptHotSpot` — исходные координаты перетаскиваемого изображения (обычно, позиция курсора). Координаты отсчитываются от левого верхнего угла изображения.

Описание

Инициализирует процесс перетаскивания изображения. Она создает временный список изображений, используемый при перетаскивании. При перетаскивании изображение представляет собой комбинацию самого изображения, его маски и текущего курсора. После этого, в ответ на периодически поступающие сообщения `WM_MOUSEMOVE`, функция обработки данного сообщения `DragMove` перемещает изображение на новую позицию. Для завершения процесса перетаскивания вызывается функция `EndDrag`.

CImageList::Create

```
BOOL Create(int cx, int cy, UINT nFlags, int nInitial, int nGrow);
BOOL Create(UINT nBitmapID, int cx, int nGrow, COLORREF crMask);
BOOL Create(LPCTSTR lpszBitmapID, int cx, int nGrow, COLORREF crMask);
BOOL Create(CImageList& imagelist1, int nImage1,
  CImageList& imagelist2, int nImage2, int dx, int dy);
BOOL Create(CImageList* pImageList);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

`cx` — горизонтальный размер изображения в элементах изображения.

`cy` — вертикальный размер изображения в элементах изображения.

`nFlags` — определяет тип создаваемого изображения. Этот аргумент может представлять собой комбинацию следующих флагов:

- ILC_COLOR — указывает на необходимость использования значения по умолчанию. Обычно в качестве такого значения задается IDC_COLOR4, однако для старых драйверов это значение IDC_COLORDDB. Не может комбинироваться с другими флагами
- ILC_COLOR4 — включает в список изображения в формате 4-битовых (16 цветов) аппаратно-независимых битовых образов (DIB)
- ILC_COLOR8 — включает в список изображения в формате 8-битовых (256 цветов) аппаратно-независимых битовых образов
- ILC_COLOR16 — включает в список изображения в формате 16-битовых (32 K/64 K цветов) аппаратно-независимых битовых образов
- ILC_COLOR24 — включает в список изображения в формате 24-битовых аппаратно-независимых битовых образов
- ILC_COLOR32 — включает в список изображения в формате 32-битовых аппаратно-независимых битовых образов
- ILC_COLORDDB — включает в список изображения в формате аппаратно-зависимых битовых образов (DDB)
- ILC_MASK — использует маску. Список изображений включает в себя два битовых образа, один из которых является монохромным битовым образом и используется как маска. Это значение может сочетаться с любым значением ILC_COLOR*

nInitial — число изображений, которое изначально содержит данный список.

nGrow — определяет приращение списка изображений системой для помещения в него новых изображений. Этот аргумент определяет число новых изображений, которые можно добавить в список после увеличения его размера.

nBitmapID — идентификатор ресурса битового образа, связанного с данным списком изображений.

crMask — цвет, используемый для создания маски. Каждый элемент изображения, имеющий данный цвет в соответствующем битовом образе становится черным, а соответствующий бит в маске устанавливается в единицу.

lpszBitmapID — строка, содержащая идентификаторы ресурсов изображений.

imagelist1 — ссылка на объект класса CImageList.

nImage1 — индекс первого изображения.

imagelist2 — ссылка на объект класса CImageList.

nImage2 — индекс второго изображения.

dx — смещение второго изображения по оси x относительно первого изображения в элементах изображения.

dy — смещение по оси y второго изображения относительно первого изображения в элементах изображения.

pImageList — указатель на объект класса CImageList.

Описание

Объект класса CImageList создается в два этапа: сначала вызывается конструктор для создания объекта данного класса, а затем для данного объекта вызывается функция Create, создающая список изображений и связывающая его с объектом класса CImageList.

CImageList::DragEnter

```
static BOOL DragEnter(CWnd* pWndLock, CPoint point);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

pWndLock — указатель на окно, которому принадлежит перетаскиваемое изображение.

point — позиция, в которую нужно вывести перетаскиваемое изображение. Задаваемые в данном аргументе координаты отсчитываются от верхнего левого угла окна, а не его рабочей области.

Описание

Данная функция запрещает на время операции перетаскивания любое обновление окна, определенного аргументом pWndLock, и отображает перетаскиваемое изображение в позиции, определенной аргументом point.

Поскольку координаты отсчитываются от верхнего левого угла окна, необходимо при их расчете учесть размеры элементов окна, таких как рамка, заголовок окна и панель меню.

Если аргумент pWndLock равен NULL, данная функция выводит изображение с использованием контекста, связанного с рабочей областью окна, а его координаты исчисляются относительно левого верхнего угла экрана.

Функция DragEnter запрещает любые изменения в указанном окне в течение всей операции перетаскивания. Если пользователю необходимо выполнить любую операцию рисования в окне, например, выделить позицию, в которую производится перетаскивание, ему нужно временно скрыть перетаскиваемое изображение, при помощи функции CImageList::DragLeave.

CImageList::DragLeave

```
static BOOL DragLeave(CWnd* pWndLock);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

`pWndLock` — указатель на окно, которому принадлежит перетаскиваемое изображение.

Описание

Разблокирует окно, определенное в аргументе `pWndLock`, и скрывает перетаскиваемое изображение, позволяя внести в окно изменения.

CImageList::DragMove

```
static BOOL DragMove(CPoint pt);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

`pt` — новое положение перемещаемого изображения.

Описание

Данная функция перемещает перетаскиваемое изображение. Обычно используется для обработки сообщения `WM_MOUSEMOVE`. Для инициализации операции перетаскивания вызывается функция `BeginDrag`.

CImageList::DragShowNolock

```
static BOOL DragShowNolock(BOOL bShow);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

`bShow` — определяет необходимость отображения перетаскиваемого элемента данных.

Описание

Отображает или скрывает перетаскиваемое изображение без блокировки окна. Если ее аргумент имеет значение FALSE, то перетаскиваемое изображение скрывается. Это делается для ускорения операций на медленных компьютерах. Функция CImageList::DragEnter запрещает внесение изменений в окно на время операции перетаскивания. Однако данная функция не блокирует окно.

CImageList::EndDrag

```
static void EndDrag();
```

Описание

Вызывается для завершения операции перетаскивания. Для инициализации перетаскивания вызывается функция BeginDrag.

CImageList::SetDragCursorImage

```
BOOL SetDragCursorImage(int nDrag, CPoint ptHotSpot);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

nDrag — индекс нового элемента данных, значок которого нужно объединить с перемещаемым изображением.

ptHotSpot — позиция базовой точки внутри нового изображения.

Описание

Создает новое перемещаемое изображение, состоящее из заданного изображения (обычно указателя мыши) и перетаскиваемого изображения. После вызова данной функции следует вызвать стандартную функцию Windows ShowCursor, чтобы скрыть используемый до этого указатель мыши. В противном случае во время перетаскивания на экране могут появиться два курсора.

Класс CList

```
template< class TYPE, class ARG_TYPE > class CList : public CObject
```

Аргументы

TYPE — тип объектов, хранящихся в списке.

ARG_TYPE — тип, используемый для ссылки на объекты, хранящиеся в списке.

Описание

Класс CList служит для работы с упорядоченными списками объектов, доступных последовательно или по значению. Для этого используются двойные связные списки.

Доступом к элементу списка управляет переменная типа POSITION. Эта переменная может использоваться для последовательного поиска в списке или для формирования в нем закладок. Однако позицию нельзя рассматривать как индекс.

Наиболее быстро осуществляется вставка элемента в голову или в хвост списка, а также по известному значению переменной POSITION. Для нахождения элемента по содержимому или по индексу необходим последовательный поиск, который занимает довольно много времени.

Чтобы получить диагностическую информацию об отдельном элементе списка, необходимо указать глубину объекта CDumpContext не меньше 1.

Описание данного класса содержится в файле заголовка afxtempl.h.

CList::AddHead

```
POSITION AddHead(ARG_TYPE newElement);
void AddHead(CList* pNewList);
```

Возвращаемое значение

Первая версия данной функции возвращает значение переменной типа POSITION для вставленного элемента.

Аргументы

ARG_TYPE — тип, используемый для ссылки на объекты, хранящиеся в списке.

newElement — новый элемент, вставляемый в начало данного списка.

pNewList — указатель на другой объект класса CList, в который будет вставлен в начало данного списка.

Описание

Добавляет в начало списка новый элемент или новый список элементов. Список может быть пуст до начала этой операции.

CList::AddTail

```
POSITION AddTail(ARG_TYPE newElement);
void AddTail(CList* pNewList);
```

Возвращаемое значение

Первая версия данной функции возвращает значение переменной типа POSITION для вставленного элемента.

Аргументы

ARG_TYPE — тип, используемый для ссылки на объекты, хранящиеся в списке.

newElement — новый элемент, вставляемый в конец данного списка.

pNewList — указатель на другой объект класса CList, который будет вставлен в конец данного списка.

Описание

Добавляет в конец списка новый элемент или новый список элементов. Список может быть пуст до начала этой операции.

CList::GetAt

```
TYPE& GetAt(POSITION position);
TYPE GetAt(POSITION position) const;
```

Возвращаемое значение

Если элементы списка являются константами, то функция GetAt возвращает копию элемента, расположенного в данной позиции. Это позволяет использовать данную функцию только в правой части оператора присваивания и не позволяет вносить изменения в содержимое списка.

Если элементы списка не являются константами, то функция GetAt возвращает ссылку на элемент списка, расположенный в данной позиции. Это позволяет использовать данную функцию как в левой, так и в правой частях оператора присваивания и позволяет вносить изменения в содержимое списка.

Аргументы

TYPE — параметр шаблона, определяющий тип объектов, хранящихся в списке.

position — величина типа POSITION, используемая для доступа к данному элементу.

Описание

Переменная типа POSITION используется в качестве ключа для доступа к элементу списка. Эту переменную нельзя интерпретировать как индекс, поскольку пользователь не может ее модифицировать. Функция GetAt возвращает элемент, расположенный в данной позиции (или ссылку на него).

При задании в качестве аргумента данной функции недопустимого значения позиции отладочная версия библиотеки MFC выдает сообщение об ошибке.

CList::GetHead

```
TYPE& GetHead();
TYPE GetHead() const;
```

Возвращаемое значение

Если элементы списка являются константами, то функция GetHead возвращает копию элемента, расположенного в голове списка. Это позволяет размещать данную функцию только в правой части оператора присваивания и не позволяет вносить изменения в содержимое списка.

Если элементы списка не являются константами, то функция GetHead возвращает ссылку на элемент списка, расположенный в его голове. Это позволяет размещать данную функцию как в левой, так и в правой частях оператора присваивания и позволяет вносить изменения в содержимое списка.

Аргументы

TYPE — параметр шаблона, определяющий тип объектов, хранящихся в списке.

Описание

Возвращает элемент списка, расположенный в его голове (или ссылку на него). Перед тем как вызывать функцию GetHead, необходимо убедиться, что список не пуст. В противном случае отладочная версия библиотеки MFC выдаст сообщение об ошибке. Для проверки можно вызвать функцию IsEmpty.

CList::GetHeadPosition

```
POSITION GetHeadPosition() const;
```

Возвращаемое значение

Величина типа POSITION, которая может использоваться для инициализации поиска элемента в списке, или NULL, если список пуст.

Описание

Позволяет получить позицию первого элемента в списке.

CList::GetNext

```
TYPE& GetNext(POSITION& rPosition);
TYPE GetNext(POSITION& rPosition) const;
```

Возвращаемое значение

Если элементы списка являются константами, то функция GetNext возвращает копию элемента, расположенного в позиции, на которую указывает аргумент rPosition. Это позволяет размещать данную функцию только в правой части оператора присваивания и не позволяет вносить изменения в содержимое списка.

Если элементы списка не являются константами, то функция GetNext возвращает ссылку на элемент списка, расположенный в позиции, на которую

указывает аргумент rPosition. Это позволяет размещать данную функцию как в левой, так и в правой частях оператора присваивания и позволяет вносить изменения в содержимое списка.

Аргументы

TYPE — параметр шаблона, определяющий тип объектов, хранящихся в списке.

rPosition — ссылка на переменную типа POSITION, полученную при предыдущем вызове функции GetNext или GetHeadPosition.

Описание

Позволяет получить элемент, расположенный в позиции, определяемой значением аргумента rPosition. После этого в аргумент rPosition заносится позиция следующего элемента в списке. С помощью функции GetNext можно просматривать список, если ее начальное значение устанавливается функцией GetHeadPosition или Find.

При задании в качестве аргумента данной функции недопустимого значения позиции отладочная версия библиотеки MFC выдает сообщение об ошибке. Если получаемый элемент является последним в списке, то аргумент rPosition принимает значение NULL.

CList::GetPrev

```
TYPE& GetPrev(POSITION& rPosition);
TYPE GetPrev(POSITION& rPosition) const;
```

Возвращаемое значение

Если элементы списка являются константами, то функция GetPrev возвращает копию элемента, расположенного в позиции, на которую указывает аргумент rPosition. Это позволяет размещать данную функцию только в правой части оператора присваивания и не позволяет вносить изменения в содержимое списка.

Если элементы списка не являются константами, то функция GetPrev возвращает ссылку на элемент списка, расположенный в позиции, на которую указывает аргумент rPosition. Это позволяет размещать данную функцию как в левой, так и в правой частях оператора присваивания и позволяет вносить изменения в содержимое списка.

Аргументы

TYPE — параметр шаблона, определяющий тип объектов, хранящихся в списке.

rPosition — ссылка на переменную типа POSITION, полученную при предыдущем вызове функции GetNext или других функций.

Описание

Возвращает элемент, расположенный в позиции, определяемой значением аргумента `rPosition`. После этого в аргумент `rPosition` заносится позиция предыдущего элемента в списке. Функция `GetPrev` может использоваться для просмотра списка, если ее начальное значение устанавливается функцией `GetTailPosition` или `Find`.

При задании в качестве аргумента данной функции недопустимого значения позиции отладочная версия библиотеки MFC выдает сообщение об ошибке. Если получаемый элемент является первым в списке, то аргумент `rPosition` принимает значение `NULL`.

CList::GetTail

```
TYPE& GetTail();
TYPE GetTail() const;
```

Возвращаемое значение

Если элементы списка являются константами, то функция `GetHead` возвращает копию элемента, расположенного в хвосте списка. Это позволяет размещать данную функцию только в правой части оператора присваивания и не позволяет вносить изменения в содержимое списка.

Если элементы списка не являются константами, то функция `GetHead` возвращает ссылку на элемент списка, расположенный в его хвосте. Это позволяет размещать данную функцию как в левой, так и в правой частях оператора присваивания и позволяет вносить изменения в содержимое списка.

Аргументы

`TYPE` — параметр шаблона, определяющий тип объектов, хранящихся в списке.

Описание

Возвращает элемент списка, расположенный в его хвосте (или ссылку на него). Перед тем как вызывать функцию `GetTail`, необходимо убедиться, что список не пуст. В противном случае отладочная версия библиотеки MFC выдаст сообщение об ошибке. Для проверки можно использовать функцию `IsEmpty`.

CList::GetTailPosition

```
POSITION GetTailPosition() const;
```

Возвращаемое значение

Величина типа `POSITION`, которая может использоваться для инициализации поиска элемента в списке, или `NULL`, если список пуст.

Описание

Позволяет получить позицию последнего элемента в списке.

CList::IsEmpty

```
BOOL IsEmpty() const;
```

Возвращаемое значение

Ненулевое, если список не содержит элементов, и ноль — в противном случае.

Описание

Указывает на то, что список не содержит элементов.

CList::RemoveHead

```
TYPE RemoveHead();
```

Возвращаемое значение

Элемент, находившийся до этого в голове списка.

Аргументы

`TYPE` — параметр шаблона, определяющий тип объектов, хранящихся в списке.

Описание

Удаляет первый элемент в списке и возвращает указатель на него. Перед тем как вызывать функцию `RemoveHead`, необходимо убедиться, что список не пуст. В противном случае отладочная версия библиотеки MFC выдаст сообщение об ошибке. Для проверки можно использовать функцию `IsEmpty`.

CList::RemoveTail

```
TYPE RemoveTail();
```

Возвращаемое значение

Элемент, находившийся до этого в хвосте списка.

Аргументы

`TYPE` — параметр шаблона, определяющий тип объектов, хранящихся в списке.

Описание

Удаляет последний элемент в списке и возвращает указатель на него. Перед вызовом функции `RemoveTail` необходимо убедиться, что список не пуст. В противном случае отладочная версия библиотеки MFC выдаст сообщение об ошибке. Для проверки можно использовать функцию `IsEmpty`.

CList::RemoveAll

```
void RemoveAll();
```

Описание

Удаляет все элементы из списка и освобождает память, использовавшуюся для их хранения. Не выдает сообщение об ошибке, если список уже пуст.

CList::RemoveAt

```
void RemoveAt(POSITION position);
```

Аргументы

`position` — позиция элемента, который должен быть удален из списка.

Описание

Удаляет указанный элемент из списка. Если в качестве аргумента данной функции задано недопустимое значение позиции, отладочная версия библиотеки MFC выдает сообщение об ошибке.

CList::SetAt

```
void SetAt(POSITION pos, ARG_TYPE newElement);
```

Аргументы

`pos` — величина типа `POSITION`, определяющая элемент, в который необходимо произвести запись.

`ARG_TYPE` — тип, используемый для ссылки на объекты, хранящиеся в списке.

`newElement` — новый элемент, вставляемый в указанный элемент списка.

Описание

Переменная типа `POSITION` используется в качестве ключа для доступа к элементу списка. Эту переменную нельзя интерпретировать как индекс, поскольку пользователь не может ее модифицировать. Функция `SetAt` записывает информацию в элемент, расположенный в данной позиции.

Если в качестве аргумента данной функции задано недопустимое значение позиции, отладочная версия библиотеки MFC выдает сообщение об ошибке.

Класс CListBox

Класс `CListBox` обеспечивает функционирование окна списка Windows. В окне списка могут отображаться различные элементы, например имена файлов, которые пользователь может просматривать и выделять.

Существуют окна списков, допускающих выделение только одного элемента, и окна списков, допускающие выделение нескольких элементов. Элемент, выбранный пользователем, выделяется, и окно списка посылает сообщение родительскому окну.

Объект класса окна списка может быть создан на основе шаблона диалогового окна или непосредственно включен пользователем в программу. Чтобы непосредственно создать объект класса `CListBox`, необходимо сначала вызвать конструктор класса, а затем — функцию `Create`, создающую соответствующий элемент управления Windows и связывающую его с объектом класса `CListBox`. Для использования объекта данного класса в шаблоне диалогового окна необходимо объявить в классе диалогового окна переменную соответствующего типа и вызвать функцию `DDX_Control` из функции `DoDataExchange` класса диалогового окна, выполняющей обмен данными между объектами класса диалогового окна и класса элемента управления. (Эта операция выполняется автоматически мастером ClassWizard при добавлении соответствующей переменной в класс диалогового окна.)

В классах, производных от данного, объект может быть создан за один шаг. Для этого необходимо в конструкторе класса вызвать функцию `Create`.

Чтобы обеспечить обработку сообщения Windows, посылаемого окном списка своему родительскому окну (обычно в качестве родительского окна выступает объект класса, производного от класса `CDialog`), добавьте в карту сообщений класса родительского окна соответствующий макрос, а в сам класс — функцию обработки данного сообщения.

Каждый макрос карты сообщений имеет вид:

```
ON_Notification(id, memberFxn)
```

где `id` определяет идентификатор дочернего окна, соответствующего объекту класса окна списка, посылающего данное сообщение, а параметр `memberFxn` определяет имя функции обработки данного сообщения в классе родительского окна.

Прототипы функций обработки сообщения имеют вид:

```
afx_msg void memberFxn();
```

Ниже приведен список макросов карты сообщений, обрабатывающих сообщения класса окна списка и краткое описание случаев, в которых они посылаются родительскому окну:

- `ON_LBN_DBLCLK` — пользователь дважды щелкнул на строке в окне списка. Данное сообщение может послать только диалоговое окно, имеющее стиль `LBS_NOTIFY`
- `ON_LBN_ERRSPACE` — объекту класса окна списка не выделена необходимая память
- `ON_LBN_KILLFOCUS` — окно списка потеряло фокус ввода

- ON_LBN_SELCANCEL — закончилось выделение элемента в данном окне списка. Это сообщение может послать только диалоговое окно, имеющее стиль LBS_NOTIFY
- ON_LBN_SELCHANGE — намечается изменение элемента, выделенного в данном окне списка. Сообщение посылается в том случае, если это изменение производится функцией CListBox::SetCurSel. Данное сообщение может послать только диалоговое окно, имеющее стиль LBS_NOTIFY. Сообщение LBN_SELCHANGE посылается окном списка, допускающим выделение нескольких элементов, при нажатии пользователем клавиши управления курсором даже в том случае, когда выделение элементов не изменяется
- ON_LBN_SETFOCUS — окно списка получает фокус ввода
- ON_WM_CHARTOITEM — определенное пользователем окно списка, не содержащее строк, получает сообщение WM_CHAR
- ON_WM_VKEYTOITEM — окно списка, имеющее стиль LBS_WANTKEYBOARDINPUT, получает сообщение WM_KEYDOWN

Если объекты класса CListBox создаются в диалоговых окнах (с помощью ресурсов диалогового окна), то они автоматически уничтожаются при закрытии пользователем этого диалогового окна. Если объекты класса CListBox созданы в других окнах, пользователь должен сам удалять эти объекты. Если объекты класса CListBox создавались в стеке, то они будут автоматически уничтожены. Если же объекты класса CListBox создавались в куче оператором new, пользователь должен уничтожить их оператором delete при закрытии родительского окна.

Описание данного класса содержится в файле заголовка afxwin.h.

CListBox::AddString

```
int AddString(LPCTSTR lpszItem);
```

Возвращаемое значение

Индекс строки в окне списка, причем первая строка имеет нулевой индекс. Если в процессе работы функции возникла ошибка, то возвращается величина LB_ERR, если эта ошибка была связана с нехваткой памяти для хранения новой строки, то возвращается величина LB_ERRSPACE.

Аргументы

lpszItem — указатель на строку символов, заканчивающуюся нулем, которую необходимо добавить в список.

Описание

Данная функция вызывается для добавления строки в окно списка. Если данное окно списка не имеет стиля LBS_SORT, то строка добавляется в конец

списка. В противном случае строка добавляется в список, который после этого сортируется. Если окно списка имеет стиль LBS_SORT, но не имеет стиля LBS_HASSTRINGS, приложение сортирует список при помощи функции CompareItem.

Чтобы добавить строку в определенное место в списке, используйте функцию InsertString.

CListBox::DeleteString

```
int DeleteString(UINT nIndex);
```

Возвращаемое значение

Число строк, оставшихся в списке. Если величина аргумента nIndex превышает число строк в окне списка, то возвращается величина LB_ERR.

Аргументы

nIndex — определяет индекс уничтожаемой строки. Индекс первой строки равен нулю.

Описание

Уничтожает элемент в окне списка.

CListBox::GetCurSel

```
int GetCurSel() const;
```

Возвращаемое значение

Индекс строки, выделенной в окне списка, или код LB_ERR, если в окне списка нет выделенной строки, или окно списка допускает одновременное выделение нескольких элементов. Первая строка в окне списка имеет нулевой индекс.

Описание

Позволяет получить индекс выделенной в окне списка строки, если таковая имеется.

Функция GetCurSel не должна применяться в окнах списка, допускающих одновременное выделение нескольких элементов.

CListBox::GetText

```
int GetText(int nIndex, LPTSTR lpszBuffer) const;
void GetText(int nIndex, CString& rString) const;
```

Возвращаемое значение

Длина (в байтах) строки за исключением завершающего нулевого символа. Если аргумент `nIndex` имеет недопустимое значение, возвращается значение `LB_ERR`.

Аргументы

`nIndex` — определяет индекс строки, которую необходимо получить. Индекс первой строки равен нулю.

`lpszBuffer` — указатель на текстовый буфер, в который будет записана полученная строка. Буфер должен иметь достаточный размер для размещения самой строки и завершающего ее нулевого символа. Размер строки может быть определен заранее функцией `GetTextLen`.

`rString` — ссылка на объект класса `CString`.

Описание

Возвращает указанную строку из окна списка. Второй вариант данной функции использует для хранения полученной строки объект класса `CString`.

CListBox::InsertString

```
int InsertString(int nIndex, LPCTSTR lpszItem);
```

Возвращаемое значение

Индекс вставленной строки в окне списка, причем первая строка имеет нулевой индекс. Если в процессе работы функции возникла ошибка, то возвращается величина `LB_ERR`, если эта ошибка связана с нехваткой памяти для хранения новой строки, то возвращается величина `LB_ERRSPACE`.

Аргументы

`nIndex` — определяет индекс позиции, в которую нужно вставить данную строку. Если этот аргумент равен нулю, то строка вставляется в конец списка.

`lpszItem` — указатель на строку символов, заканчивающуюся нулем, которую необходимо добавить в список.

Описание

Помещает строку в окно списка. В отличие от функции `AddString`, вызов данной функции не приводит к сортировке списков, имеющих стиль `LBS_SORT`.

CListBox::SetCurSel

```
int SetCurSel(int nSelect);
```

Возвращаемое значение

LB_ERR, если при выполнении функции возникла ошибка, и ноль — в противном случае.

Аргументы

nSelect — определяет индекс выделяемой строки. Первая строка имеет нулевой индекс. Если аргумент nSelect равен −1, то в окне списка снимаются все выделения.

Описание

Выделяет строку и перемещает ее в рабочую область окна списка, если это необходимо. При выделении новой строки окно списка снимает выделение с предыдущей выделенной строки.

Функция работает только в окнах списка, допускающих выделение одного элемента. Она не может быть использована для установки или снятия выделения с нескольких элементов списка.

CListBox::SetSel

```
int SetSel(int nIndex, BOOL bSelect = TRUE);
```

Возвращаемое значение

LB_ERR, если при выполнении функции возникла ошибка, и ноль — в противном случае.

Аргументы

nIndex — индекс выделяемой строки, причем первая строка имеет нулевой индекс. Если этот аргумент равен −1, то, в зависимости от значения аргумента bSelect, выделяются все строки или снимается выделение со всех строк.

bSelect — определяет режим выделения. Если данный аргумент имеет значение TRUE, то строка выделяется, если FALSE, то со строки снимается выделение. По умолчанию строка выделяется.

Описание

Выделяет строку в окне списка, допускающем выделение нескольких элементов. Эта функция работает только в окнах списков, допускающих выделение нескольких элементов.

Класс *CListCtrl*

Класс `CListCtrl` и связанное с ним просмотровое окно списка позволяют отображать набор элементов данных, каждый из которых состоит из значка (хранящегося в списке изображений) и текстовой метки. Помимо значка и метки каждый элемент данных может содержать информацию, которая будет отображаться в столбцах, расположенных справа от значка и метки. Этот элемент управления (а, следовательно, и сам класс `CListCtrl`) доступен только в программах, выполняемых под управлением Windows 95/98/NT.

Просмотровый список использует четыре *способа отображения* (views) информации:

- *Отображение значков* (Icon view). В этом случае каждому элементу данных соответствует полноразмерный значок (32×32 элемента изображения), под которым располагается метка. Пользователь может перетащить элемент данных в любую позицию в пределах окна просмотрового списка
- *Отображение маленьких значков* (Small icon view). В этом случае каждому элементу данных соответствует маленький значок (16×16 элементов изображения), справа от которого располагается метка. Пользователь может перетащить элемент данных в любую позицию в пределах окна просмотрового списка
- *Отображение списка* (List View). Каждый элемент данных отображается как маленький значок, справа от которого располагается метка. Элементы данных организованы в столбцы и не могут быть перемещены на произвольную позицию в пределах окна просмотрового списка
- *Отображение отчета* (Report view). Каждому элементу данных соответствует отдельная строка, в которой помещается дополнительная информация, расположенная в столбцах справа от метки. В столбце слева от метки расположен маленький значок. Содержимое столбцов справа от метки специфично для каждого приложения. Созданием этих столбцов занимается класс `CHeaderCtrl`

В классе `CListCtrl` определены специальные константы стилей, которые могут быть использованы при создании экземпляра класса:

- `LVS_ALIGNLEFT` — размещает элементы списка у левой кромки окна при выводе в режимах отображения значков и маленьких значков
- `LVS_ALIGNTOP` — размещает элементы списка у верхней кромки окна при выводе в режимах отображения значков и маленьких значков
- `LVS_AUTOARRANGE` — автоматически размещает элементы списка при выводе в режимах отображения значков и маленьких значков
- `LVS_EDITLABELS` — разрешает непосредственное редактирование текста элемента данных. В случае использования данной константы стиля роди-

тельское окно (для класса просмотрового окна списка) должно обрабатывать сообщение LVN_ENDLABELEDIT

- LVS_ICON — устанавливает вывод в режиме отображения значков
- LVS_LIST — устанавливает вывод в режиме отображения списка
- LVS_NOCOLUMNHEADER — запрещает вывод заголовков столбцов в режиме отображения отчета. По умолчанию столбцы в режиме отображения отчета имеют заголовки колонок
- LVS_NOLABELWRAP — отображает текст в режиме отображения значков в одну строку. По умолчанию текст элемента данных может размещаться в нескольких строках
- LVS_NOSCROLL — запрещает протяжку изображения в окне. Все элементы данных должны располагаться в рабочей области элемента управления
- LVS_NOSORTHEADER — запрещает использование заголовков столбцов в качестве кнопок. Это позволяет исключить операции, связанные с щелчком мыши по заголовку. Например, сортировку
- LVS_OWNERDRAWFIXED — разрешает пользовательские значки в режиме отображения отчета. Для отрисовки каждого элемента данных просмотровое окно списка посылает сообщение WM_DRAWITEM. При этом не посылаются дополнительные сообщения каждому элементу данных более низкого уровня. Информация, необходимая для отображения элемента данных, хранится в структуре DRAWITEMSTRUCT в ее переменной itemData
- LVS_REPORT — устанавливает вывод в режиме отображения отчета
- LVS_SHAREIMAGELISTS — определяет, что данный элемент управления не является полноправным собственником включенного в него списка изображений (то есть не может уничтожать список после того, как в нем отпала необходимость). Этот стиль позволяет нескольким просмотровым окнам списка использовать один и тот же список изображений
- LVS_SHOWSELALWAYS — устанавливает режим, в котором выделение элемента отображается, даже если элемент управления потерял фокус ввода
- LVS_SINGLESEL — запрещает множественный выбор элементов списка, устанавливаемый по умолчанию
- LVS_SMALLICON — устанавливает вывод в режиме отображения маленьких значков
- LVS_SORTASCENDING — сортирует элементы данных по возрастанию, основываясь на тексте меток
- LVS_SORTDESCENDING — сортирует элементы данных по убыванию

Наряду со стандартными стилями отображения, класс CListCtrl поддерживает обширный набор дополнительных стилей, существенно расширяющих

область применения данного элемента управления. Ниже приведены некоторые из них:

- *Выделение "на лету"* (Hover selection). Если элемент управления реализует эту возможность, то элемент данных выделяется, если указатель мыши находится над ним дольше некоторого заранее установленного промежутка времени
- *Отображение виртуальных списков* (Virtual list views). Если элемент управления использует эту возможность, то количество элементов данных в списке ограничено максимальной величиной, принимаемой переменной формата `DWORD`. Это достигается передачей приложению большинства функций по работе с элементами данных (всех, кроме выделения элемента данных и обработки информации о фокусе ввода)
- *Активизация элемента одиночным или двойным щелчком мыши* (One- and two- click activation). Текст элемента данных автоматически выделяется при помещении на него курсора мыши, а одиночный или двойной щелчок мышью приводит к активизации данного элемента
- *Перетаскивание столбцов* (Drag and drop column ordering). Порядок следования столбцов может быть изменен простым перетаскиванием. Эта возможность доступна только при отображении отчета

Каждый элемент данных просмотрового окна списка состоит из значка, метки, сведений о текущем состоянии и некоторой величины, назначение которой зависит от приложения. С каждым элементом данных могут быть связаны один или более других элементов данных более низкого уровня, представляющих собой строки, которые отображаются справа от метки основного элемента данных в режиме отображения отчета. Все элементы данных одного просмотрового окна списка должны иметь одинаковое количество элементов данных более низкого уровня.

По умолчанию класс просмотрового окна списка должен хранить только информацию о значке элемента данных и о его текстовые атрибуты. Однако кроме элементов данных, имеющих подобную структуру, класс `CListCtrl` поддерживает *элементы данных обратного вызова* (callback items). Под элементом данных обратного вызова понимается такой элемент данных, в котором значок или текстовые атрибуты или и то и другое вместе хранятся не в элементе управления, а в самом приложении. При вызове таких элементов данных используется *маска обратного вызова*, позволяющая определить, какой из атрибутов данного элемента (текст и/или значок) хранятся в приложении. Если приложение использует элементы данных обратного вызова, то на него ложится ответственность за предоставление всей необходимой информации по запросу элемента управления. Использование этого типа элементов данных целесообразно, если в приложении уже хранится вся необходимая для элемента управления информация.

Информация о значках элемента данных, значках заголовков столбцов и значках индикации состояния хранится в нескольких списках изображений (являющихся объектами класса `CImageList`). Все эти значки создаются пользователем и им же включаются в соответствующие списки изображений. Каждое просмотровое окно списка может включать в себя не более четырех различных списков изображений:

- *Список больших значков* (Large icon). Для режима отображения значков
- *Список маленьких значков* (Small icon). Для режима отображения маленьких значков, списка и отчета. В него включаются маленькие значки
- *Список значков индикации состояния* (Application-defined state). Содержит значки индикации состояния, отображаемые рядом со значком элемента данных для указания его состояния, определяемого приложением
- *Список значков заголовков столбцов* (Header item). Эти маленькие значки используются в заголовке каждого из столбцов элемента управления

По умолчанию элемент управления окна просмотрового списка уничтожает все используемые в нем объекты списков изображений после своего завершения, однако разработчик может уничтожить каждый из этих объектов самостоятельно

Описание данного класса содержится в файле заголовка afxcmn.h.

CListCtrl::ApproximateViewRect

```
CSize ApproximateViewRect(CSize sz = CSize(-1, -1),
  int iCount = -1) const;
```

Возвращаемое значение

Объект класса `CSize`, содержащий ширину и высоту окна в элементах изображения.

Аргументы

`sz` — предлагаемые размеры элемента управления в элементах изображения. Если данный аргумент не определен, приложение использует текущую ширину и высоту элемента управления.

`iCount` — число элементов данных, которые следует вывести в элемент управления. Если этот аргумент равен −1, то приложение считает, что необходимо вывести все элементы данных, содержащиеся в элементе управления.

Описание

Вычисляет ширину и высоту окна в элементах изображения.

CListCtrl::Create

```
BOOL Create(DWORD dwStyle, const RECT& rect, CWnd* pParentWnd,
  UINT nID);
```

Возвращаемое значение

Ненулевое, если функция завершилась успешно, и ноль — в противном случае.

Аргументы

`dwStyle` — определяет стиль просмотрового окна списка. Данный аргумент является комбинацией следующих стилей:

- `LVS_ALIGNLEFT` — задает выравнивание по левому краю при отображении в режимах значков и маленьких значков
- `LVS_ALIGNTOP` — задает выравнивание по верхнему краю при отображении в режимах значков и маленьких значков
- `LVS_AUTOARRANGE` — задает автоматическое выравнивание значков при отображении в режимах значков и маленьких значков
- `LVS_EDITLABELS` — разрешает непосредственное редактирование текста элемента данных. Родительское окно должно при этом обрабатывать сообщение `LVN_ENDLABELEDIT`
- `LVS_ICON` — задает режим отображения значков
- `LVS_LIST` — задает режим отображения списка
- `LVS_NOCOLUMNHEADER` — запрещает вывод заголовков столбцов в режиме отображения отчета
- `LVS_NOLABELWRAP` — в режиме отображения значков выводит текст элемента данных в одну строку. По умолчанию текст элемента данных может выводиться в несколько строк
- `LVS_NOSCROLL` — запрещает прокрутку изображения в окне. Все элементы данных должны располагаться внутри рабочей области окна
- `LVS_NOSORTHEADER` — задает режим, в котором заголовки столбцов не могут использоваться в качестве кнопок. Этот режим применяется, если щелчок на заголовке столбца не должен вызывать какие-либо действия, например, сортировку
- `LVS_OWNERDRAWFIXED` — разрешает использование пользовательских значков в режиме отображения отчета. Для отрисовки каждого элемента данных просмотровое окно списка посылает сообщение `WM_DRAWITEM`. При этом не посылаются дополнительные сообщения каждому элементу данных более низкого уровня. Необходимая для отображения элемента данных информация хранится в структуре `DRAWITEMSTRUCT` в ее переменной `itemData`

- LVS_REPORT — устанавливает вывод в режиме отображения отчета
- LVS_SHAREIMAGELISTS — определяет, что данный элемент управления не является полноправным собственником включенного в него списка изображений (т. е. не может его уничтожать). Этот стиль позволяет нескольким просмотровым окнам списка использовать один и тот же список изображений
- LVS_SHOWSELALWAYS — устанавливает режим, в котором выделение элемента отображается, даже если элемент управления потерял фокус ввода
- LVS_SINGLESEL — запрещает множественный выбор элементов списка. По умолчанию установлен множественный режим выбора
- LVS_SMALLICON — устанавливает вывод в режиме отображения маленьких значков
- LVS_SORTASCENDING — сортирует элементы данных по возрастанию, основываясь на тексте метки
- LVS_SORTDESCENDING — сортирует элементы данных по убыванию, основываясь на тексте метки

rect — определяет положение и размеры элемента управления. В качестве данного аргумента могут выступать объект класса CRect или объект структуры RECT.

pParentWnd — определяет родительское окно просмотрового окна списка. Обычно это объект класса CDialog. Данный аргумент не может принимать нулевое значение.

nID — определяет идентификатор ресурса просмотрового окна списка.

Описание

Объект класса CListCtrl создается в два этапа: на первом этапе при помощи конструктора создается объект класса, а на втором этапе вызывается функция Create, создающая просмотровое окно списка и связывающая его с объектом класса CListCtrl.

CListCtrl::CreateDragImage

```
CImageList* CreateDragImage(int nItem, LPPOINT lpPoint);
```

Возвращаемое значение

В случае спешного завершения функции — указатель на список перемещения значка объекта данных, в противном случае — NULL.

Аргументы

nItem — индекс элемента данных, для которого создается список перемещения значка.

lpPoint — указатель на объект структуры POINT, в который будет записана начальная позиция левого верхнего угла данного элемента данных в координатах рабочей области просмотрового окна списка.

Описание

Данная функция позволяет создать список перемещения значка для элемента данных, определяемого аргументом nItem. Объект класса CImageList является постоянным, и его необходимо удалить по завершении операции перетаскивания.

Пример

```
CImageList* pImageList = MyListCtrl.CreateDragImage(nItem, &point);
   ...
   ...
   delete pImageList;
```

CListCtrl::GetHeaderCtrl

```
CHeaderCtrl* GetHeaderCtrl();
```

Возвращаемое значение

Указатель на заголовок, используемый просмотровым окном списка.

Описание

Возвращает указатель на заголовок, используемый просмотровым окном списка.

CListCtrl::GetHotCursor

```
HCURSOR GetHotCursor()
```

Возвращаемое значение

Дескриптор текущего ресурса курсора выделения, используемого в просмотровом окне списка.

Описание

Данная функция позволяет получить дескриптор текущего ресурса курсора выделения. Курсор выделения отображается, только если разрешен режим выделения "на лету". Для этого необходимо установить расширенный стиль LVS_EX_TRACKSELECT.

CListCtrl::GetItemPosition

```
BOOL GetItemPosition(int nItem, LPPOINT lpPoint) const;
```

Возвращаемое значение

Ненулевое, если функция завершилась успешно, и ноль — в противном случае.

Аргументы

`nItem` — индекс элемента данных, позицию которого следует определить.

`lpPoint` — указатель на объект структуры `POINT`, в который будет записана позиция левого верхнего угла данного элемента данных в координатах рабочей области просмотрового окна списка.

Описание

Возвращает позицию элемента данных в окне просмотрового окна списка.

CListCtrl::GetItemText

```
int GetItemText(int nItem, int nSubItem, LPTSTR lpszText, int nLen) const;
CString GetItemText(int nItem, int nSubItem) const;
```

Возвращаемое значение

Первая версия данной функции возвращает длину полученной строки. Вторая версия возвращает объект класса `CString`, содержащий полученную строку.

Аргументы

`nItem` — индекс элемента данных, текст из которого требуется получить.

`nSubItem` — определяет уровень в иерархии для элемента данных, текст из которого требуется получить.

`lpszText` — указатель на строку, в которую будет записан текст из элемента данных.

`nLen` — длина буфера, на который указывает аргумент `lpszText`.

Описание

Возвращает текст, содержащийся в элементе данных любого уровня иерархии. Если аргумент `nSubItem` равен нулю, то данная функция позволяет получить метку элемента данных, а если нет — то текст, содержащийся в элементе данных более низкого уровня.

CListCtrl::GetOrigin

```
BOOL GetOrigin(LPPOINT lpPoint) const;
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае. Однако, если просмотровое окно списка находится в режиме отображения отчета, возвращаемая величина всегда равна нулю.

Аргументы

`lpPoint` — указатель на объект структуры `POINT`, в который будут записаны координаты нулевой точки рабочей области окна данного элемента управления.

Описание

Позволяет получить координаты нулевой точки рабочей области просмотрового окна списка.

CListCtrl::HitTest

```
int HitTest(LVHITTESTINFO* pHitTestInfo) const;
int HitTest(CPoint pt, UINT* pFlags = NULL) const;
```

Возвращаемое значение

Индекс элемента данных, расположенного в позиции, определенной в аргументе `pHitTestInfo`, если таковой имеется, и −1 в противном случае.

Аргументы

`pHitTestInfo` — указатель на объект структуры `LVHITTESTINFO`, содержащий позицию проверяемого элемента данных, в который данная функция заносит результаты проверки.

`pt` — *проверяемая позиция.*

`pFlags` — указатель на целочисленную величину, в которую будет занесена информация о результатах проверки. Значения флагов определены в описании структуры `LVHITTESTINFO` и задают позицию проверяемого элемента данных:

- `LVHT_ABOVE` — выше рабочей области элемента управления
- `LVHT_BELOW` — ниже рабочей области элемента управления
- `LVHT_NOWHERE` — в рабочей области элемента управления, но не над элементом данных списка
- `LVHT_ONITEMICON` — над значком элемента данных просмотрового окна списка

- LVHT_ONITEMLABEL — над текстом элемента данных просмотрового окна списка
- LVHT_ONITEMSTATEICON — над индикатором состояния элемента данных просмотрового окна списка
- LVHT_TOLEFT — слева от рабочей области элемента управления
- LVHT_TORIGHT — справа от рабочей области элемента управления

Описание

Определяет, какой элемент данных просмотрового окна списка, если таковой имеется, расположен в данной позиции. Флаги LVHT_ABOVE, LVHT_BELOW, LVHT_TOLEFT и LVHT_TORIGHT определяют необходимость прокрутки содержимого просмотрового окна списка для отображения нужного элемента данных. Можно использовать комбинацию из двух флагов, например если позиция расположена слева и выше рабочей области окна. Флаг LVHT_ONITEM позволяет определить, не находится ли данная позиция над элементом данных просмотрового окна списка. Этот флаг представляет собой конъюнкцию флагов LVHT_ONITEMICON, LVHT_ONITEMLABEL и LVHT_ONITEMSTATEICON данной структуры.

CListCtrl::InsertColumn

```
int InsertColumn(int nCol, const LVCOLUMN* pColumn);
int InsertColumn(int nCol, LPCTSTR lpszColumnHeading,
  int nFormat = LVCFMT_LEFT, int nWidth = -1, int nSubItem = -1);
```

Возвращаемое значение

В случае успешного завершения функции — индекс нового столбца. В противном случае возвращает −1.

Аргументы

nCol — индекс нового столбца.

pColumn — указатель на объект структуры LVCOLUMN, содержащей атрибуты нового столбца.

lpszColumnHeading — указатель на строку, содержащую заголовок столбца.

nFormat — целочисленная величина, определяющая выравнивание столбца. Может принимать значения: LVCFMT_LEFT, LVCFMT_RIGHT или LVCFMT_CENTER.

nWidth — ширина столбца в элементах изображения. Если этот аргумент равен −1, то ширина столбца не фиксируется.

nSubItem — индекс элемента данных нижнего уровня, связанного со столбцом. Если этот аргумент равен −1, то со столбцом не связан ни один элемент данных нижнего уровня.

Описание

Создает новый столбец в просмотровом окне списка. Структура LVCOLUMN содержит атрибуты нового столбца, выводимого в режиме отображения отчета. Объект данной структуры может также хранить информацию о столбце.

CListCtrl::InsertItem

```
int InsertItem(const LVITEM* pItem);
int InsertItem(int nItem, LPCTSTR lpszItem);
int InsertItem(int nItem, LPCTSTR lpszItem, int nImage);
int InsertItem(UINT nMask, int nItem, LPCTSTR lpszItem, UINT nState,
  UINT nStateMask, int nImage, LPARAM lParam);
```

Возвращаемое значение

В случае успешного завершения функции — индекс нового элемента данных. В противном случае возвращает −1.

Аргументы

pItem — указатель на объект структуры LVITEM, определяющей атрибуты элемента данных.

nItem — индекс нового элемента данных.

lpszItem — указатель на строку, содержащую метку элемента данных или значение LPSTR_TEXTCALLBACK, если это элемент данных обратного вызова.

nImage — индекс значка элемента данных или значение I_IMAGECALLBACK, если это элемент данных обратного вызова.

nMask — определяет, какие из передаваемых атрибутов следует использовать. В качестве этого аргумента может использоваться одно или несколько из возможных значений переменной mask структуры LVITEM. Возможные значения объединяются оператором побитового ИЛИ.

nState — определяет состояние элемента данных, значок состояния и значок перекрытия.

nStateMask — определяет, какие биты переменной состояния необходимо получить или изменить.

nImage — индекс значка элемента данных в списке изображений.

lParam — 32-разрядная величина, назначение которой определяется приложением. Если используется данный аргумент, то в аргументе nMask необходимо установить флаг LVIF_PARAM.

Описание

Включает новый элемент данных в просмотровое окно списка.

CListCtrl::SetExtendedStyle

```
DWORD SetExtendedStyle(DWORD dwNewStyle);
```

Возвращаемое значение

Предыдущая комбинация расширенных стилей, использовавшаяся просмотровым окном списка.

Аргументы

`dwNewStyle` — новая комбинация расширенных стилей просмотрового окна списка.

Описание

Устанавливает расширенные стили просмотрового окна списка.

CListCtrl::SetItem

```
BOOL SetItem(const LVITEM* pItem);
BOOL SetItem(int nItem, int nSubItem, UINT nMask, LPCTSTR lpszItem,
  int nImage, UINT nState, UINT nStateMask, LPARAM lParam);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

`pItem` — указатель на объект структуры `LVITEM`, содержащей новые атрибуты элемента данных. Элементы этой структуры `iItem` и `iSubItem` определяют элемент данных и его уровень в структуре данных, а элемент `mask` определяет устанавливаемый атрибут.

`nItem` — индекс элемента данных, для которого устанавливаются атрибуты.

`nSubItem` — уровень в иерархии элементов, занимаемый элементом данных, для которого устанавливаются атрибуты.

`nMask` — определяет устанавливаемый атрибут.

`lpszItem` — указатель на текстовую строку, заканчивающуюся нулем, определяющую метку элемента данных.

`nImage` — индекс значка элемента данных в списке изображений.

`nState` — устанавливает значения для изменяемых состояний.

`nStateMask` — определяет изменяемое состояние.

`lParam` — 32-разрядная величина, назначение которой определяется приложением.

Описание

Устанавливает некоторые или все атрибуты элемента данных в просмотровом окне списка. Переменные `iItem` и `iSubItem`, являющиеся членами структуры `LVITEM`, и аргументы `nItem` и `nSubItem` во второй версии данной функции определяют элемент данных, атрибуты которого требуется установить, и его уровень в структуре данных.

Переменная `mask`, являющаяся членом структуры `LVITEM`, и аргумент `nMask` во второй версии данной функции определяют атрибут элемента данных, который требуется установить. Эта переменная может принимать следующие значения:

- `LVIF_TEXT` — переменная `pszText` или аргумент `lpszItem` представляют собой указатель на текстовую строку, заканчивающуюся нулем. Значение переменной `cchTextMax` игнорируется
- `LVIF_STATE` — переменная `stateMask` или аргумент `nStateMask` определяют, какое из состояний элемента данных необходимо изменить, а переменная `state` или аргумент `nState` содержат величины для данных состояний

CListCtrl::SetItemPosition

```
BOOL SetItemPosition(int nItem, POINT pt);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

`nItem` — индекс элемента данных, для которого устанавливается новая позиция.

`pt` — объект структуры `POINT`, содержащий новую позицию левого верхнего угла элемента данных, задаваемую в координатах рабочей области просмотрового окна списка.

Описание

Перемещает элемент данных в указанную позицию в просмотровом окне списка. Данное окно должно находиться в режиме отображения значков или маленьких значков.

Если просмотровое окно списка имеет стиль `LVS_AUTOARRANGE`, то значки в нем автоматически упорядочиваются после перемещения.

CListCtrl::SetItemText

```
BOOL SetItemText(int nItem, int nSubItem, LPTSTR lpszText);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

`nItem` — индекс элемента данных, для которого устанавливаются атрибуты.

`nSubItem` — уровень в иерархии элементов, занимаемый элементом данных, для которого устанавливаются атрибуты.

`lpszText` — указатель на строку, содержащую новый текст элемента данных.

Описание

Изменяет текст, содержащийся в элементе данных любого уровня иерархии.

CListCtrl::SetWorkAreas

```
void SetWorkAreas(int nWorkAreas, LPRECT lpRect);
```

Аргументы

`nWorkAreas` — количество объектов структуры `RECT` (или объектов класса `CRect`) в массиве, на который указывает аргумент `lpRect`.

`lpRect` — указатель на массив объектов структуры `RECT` (или объектов класса `CRect`), определяющих новые рабочие области в просмотровом окне списка. Эти области задаются в координатах рабочей области окна. Если этот аргумент равен нулю, то в качестве рабочей выбирается рабочая область окна элемента управления.

Описание

Устанавливает рабочие области в просмотровом окне списка.

Класс CMap

```
template< class KEY, class ARG_KEY, class VALUE,
  class ARG_VALUE >class CMap : public CObject
```

Аргументы

`KEY` — класс объектов, используемых в качестве ключей в данной карте отображений.

`ARG_KEY` — тип данных, используемых в качестве аргументов `KEY`. Обычно это ссылка на `KEY`.

VALUE — класс объектов, хранящихся в данной карте отображений.

ARG_VALUE — тип данных, используемых в качестве аргументов VALUE. Обычно это ссылка на VALUE.

Описание

Класс CMap представляет собой класс словарей, в котором для доступа к данным используются уникальные ключи. Если в карту отображений добавить пару, состоящую из ключа и элемента, то доступ к ней возможен по значению ключа. Кроме того, поддерживается последовательный просмотр всего содержимого карты отображений. Для этого используется переменная типа POSITION. Эта переменная позволяет "запомнить" текущую позицию в карте и просмотра всей карты. Последовательность просмотра карты отображений никак не связана с последовательностью ключей.

Класс CMap включает в себя макрос IMPLEMENT_SERIAL, позволяющий работать с архивом и выводить диагностические сообщения. Независимо от того, используется ли для записи в архив перегруженный оператор << или функция Serialize, в архиве сохраняется каждый элемент карты отображений.

При необходимости вывести диагностическую информацию об отдельном элементе карты отображений, следует указать глубину объекта CDumpContext равной 1 или большей величине.

При уничтожении объекта класса CMap или при удалении из него элементов уничтожаются как ключи, так и связанные с ними объекты.

Описание данного класса содержится в файле заголовка afxtempl.h.

CMap::GetNextAssoc

```
void GetNextAssoc(POSITION& rNextPosition, KEY& rKey, VALUE& rValue)
  const;
```

Аргументы

rNextPosition — ссылка на величину типа POSITION, возвращенную предыдущим вызовом функции GetNextAssoc или GetStartPosition.

KEY — параметр шаблона, определяющий тип значения ключа.

rkey — определяет ключ, по которому осуществляется доступ к элементу.

VALUE — определяет тип искомой переменной.

rValue — содержит значение искомой переменной.

Описание

Позволяет получить ключ и элемент карты отображений, расположенные в позиции, определяемой аргументом rNextPosition. После этого обновляется

значение rNextPosition, которое теперь указывает на следующий элемент в карте отображений. При этом последовательность перебора элементов карты отображений не имеет ничего общего с последовательностью ключей.

Если получено значение последнего элемента в карте отображений, величина rNextPosition принимает значение NULL.

CMap::GetStartPosition

```
POSITION GetStartPosition() const;
```

Возвращаемое значение

Значение типа POSITION, определяющее начальную позицию для поиска по карте отображений, или NULL, если карта пуста.

Описание

Инициирует процесс поиска по карте отображений, возвращая значение типа POSITION, которое может быть использовано при вызове функции GetNextAssoc. Последовательность просмотра карты отображений непредсказуема, поэтому первый элемент карты ничем особым не отличается от других.

CMap::IsEmpty

```
BOOL IsEmpty() const;
```

Возвращаемое значение

Ненулевое, если карта отображений не содержит элементов, и ноль — в противном случае.

Описание

Позволяет определить, не пуста ли карта отображений.

CMap::Lookup

```
BOOL Lookup(ARG_KEY key, VALUE& rValue) const;
```

Возвращаемое значение

Ненулевое, если элемент был найден, и ноль — в противном случае.

Аргументы

ARG_KEY — параметр шаблона, определяющий тип значения ключа.

key — определяет ключ, по которому осуществляется доступ к элементу.

VALUE — определяет тип искомой переменной.

rValue — содержит значение искомой переменной.

Описание

Функция Lookup использует хеширование для быстрого поиска элемента карты отображений по заданному ключу.

CMap::operator[]

```
VALUE& operator[](ARG_KEY key);
```

Аргументы

VALUE — определяет тип элемента карты отображений.

ARG_KEY — параметр шаблона, определяющий тип значения ключа.

key — определяет ключ, по которому осуществляется доступ к элементу.

Описание

Этот оператор может использоваться вместо функции SetAt. Он может находиться только в левой части оператора присваивания. Если в карте отображений не существует элемента, соответствующего данному ключу, то создается новый элемент.

Поскольку нет никакой гарантии, что указанному ключу соответствует элемент в карте отображений, данный оператор не может стоять в правой части оператора присваивания. Для получения элемента по ключу следует вызвать функцию Lookup.

CMap::RemoveAll

```
void RemoveAll();
```

Описание

Функция уничтожает все элементы карты отображений, вызывая глобальную функцию DestructElements. Работает корректно, если карта отображений уже пуста.

Класс CMenu

Класс CMenu предназначен для работы с дескрипторами HMENU Windows. Содержит функции для создания, отслеживания, обновления и уничтожения меню.

Объект класса CMenu следует локально создавать в стеке. После этого необходимо вызвать функцию CWnd::SetMenu для замены меню в окне и сразу же вызвать функцию CMenu::Detach. Функция CWnd::SetMenu устанавливает в окне новое меню, вызывая его перерисовку для отображения нового меню, а также передает окну права собственности на меню. Функция Detach осво-

бождает дескриптор HMENU из объекта класса CMenu, так что при уничтожении данного объекта класса CMenu деструктор не попытается уничтожить меню, на которое он уже не имеет права собственности. Само меню уничтожается деструктором окна при закрытии.

Функция LoadMenuIndirect создает меню по шаблону, хранящемуся в памяти, однако намного проще работать с меню, созданным функцией LoadMenu с использованием ресурсов, а ресурсы меню могут создаваться и изменяться в редакторе меню.

Описание данного класса содержится в файле заголовка afxwin.h.

CMenu::AppendMenu

```
BOOL AppendMenu(UINT nFlags, UINT nIDNewItem = 0,
  LPCTSTR lpszNewItem = NULL);
BOOL AppendMenu(UINT nFlags, UINT nIDNewItem, const CBitmap* pBmp);
```

Возвращаемое значение

Ненулевое, если функция успешно завершает свою работу, и ноль — в противном случае.

Аргументы

nFlags — определяет состояние новой команды меню при ее добавлении к меню. Данный аргумент может представлять собой набор следующих флагов:

- MF_CHECKED — устанавливает флаг у новой команды меню. Если для отметки команды применяются битовые образы, то выводится соответствующий битовый образ
- MF_UNCHECKED — сбрасывает флаг у новой команды меню
- MF_DISABLED — делает новую команду меню недоступной, но не отображает этого
- MF_ENABLED — делает новую команду меню доступной, и отображает это
- MF_GRAYED — делает новую команду меню недоступной, и отображает это
- MF_MENUBARBREAK — помещает команду в новую строку статического меню или в новый столбец всплывающего меню. Новый столбец всплывающего меню отделяется от старого столбца вертикальной линией
- MF_MENUBREAK — помещает команду в новую строку статического меню или в новый столбец всплывающего меню. Между столбцами отсутствуют разделительные линии
- MF_OWNERDRAW — определяет пользовательский дизайн команды меню. При первом выводе меню на экран окно, которому принадлежит дан-

ное меню, получает сообщение WM_MEASUREITEM, которое получает высоту и ширину команды меню. Сообщение WM_DRAWITEM посылается, если пользователь должен обновить изображение команды меню. Этот параметр не может применяться для команд меню верхнего уровня

- MF_POPUP — определяет, что с данной командой меню связано всплывающее меню. Идентификатор определяет дескриптор всплывающего меню, связанного с данной командой. Флаг используется для добавления всплывающих меню верхнего уровня или для создания иерархии всплывающих меню
- MF_SEPARATOR — рисует горизонтальную разделительную линию. Если установлен этот флаг, значения других флагов игнорируются
- MF_STRING — определяет, что команда меню представляет собой строку текста

Ниже приведены группы взаимоисключающих флагов, которые не могут использоваться вместе:

- MF_DISABLED, MF_ENABLED и MF_GRAYED
- MF_STRING, MF_OWNERDRAW, MF_SEPARATOR и битовые образы
- MF_MENUBARBREAK и MF_MENUBREAK
- MF_CHECKED и MF_UNCHECKED

nIDNewItem — определяет идентификатор новой команды меню или, если аргумент nFlags имеет значение MF_POPUP, дескриптор (HMENU) всплывающего меню. Если аргумент nFlags имеет значение MF_SEPARATOR, то значение данного аргумента игнорируется.

lpszNewItem — определяет содержимое новой команды меню. Влияние аргумента nFlags на интерпретацию данного аргумента проиллюстрировано в табл. В.5.

Таблица В.5. *Интерпретация аргумента* `lpszNewItem` *в зависимости от значения аргумента* `nFlags`

Значение аргумента nFlags	Интерпретация значения аргумента lpszNewItem
MF_OWNERDRAW	Содержит 32-разрядное значение, которое приложение может использовать для передачи дополнительной информации, связанной с командой меню. Приложение может использовать это значение при обработке сообщений WM_MEASUREITEM и WM_DRAWITEM. Это значение сохраняется в переменной itemData структуры, передаваемой с данными сообщениями

Таблица В.5 (окончание)

Значение аргумента nFlags	Интерпретация значения аргумента lpszNewItem
MF_STRING	Содержит указатель на строку, завершающуюся нулем. Эта интерпретация используется по умолчанию
MF_SEPARATOR	Значение переменной lpszNewItem игнорируется (не используется)

pBmp — указатель на объект класса CBitmap, используемый в качестве данной команды меню.

Описание

Добавляет в меню новую команду. Приложение может определить начальное состояние добавляемой команды, задавая соответствующие значения в аргументе nFlags. Всплывающее меню, определенное аргументом nIDNewItem, становится составной частью связанного с ним меню. Если основное меню уничтожается, то уничтожается и добавленное. Во избежание конфликта добавленное меню должно освобождаться из объекта класса CMenu. Флаги MF_STRING и MF_OWNERDRAW не могут использоваться совместно с битовыми образами.

При внесении изменений в меню окна (независимо от того, отображается окно на экране или нет) приложение должно вызывать функцию CWnd::DrawMenuBar.

Класс *CMutex*

Объект класса CMutex представляет собой объект синхронизации mutex, позволяющий потоку временно получить исключительное право собственности на разделяемый с другими потоками ресурс. Объект синхронизации mutex обычно используется, если один из потоков вносит изменения в хранящиеся в разделяемом ресурсе данные, что приводит к их временной некорректности. Например, новый узел может добавляться в связный список только одним из процессов, имеющих доступ к данному списку.

Чтобы использовать объект класса CMutex, достаточно вызвать его конструктор, в котором следует указать его имя и право собственности на него создающего потока. По завершении всех операций с ресурсом, которые необходимо было произвести в монопольном режиме, вызовите функцию CSyncObject::Unlock и освободите данный ресурс для доступа других потоков.

Другим способом применения объекта класса CMutex является включение в пользовательский класс переменной, являющейся объектом данного класса.

При этом в контструкторе пользовательского класса необходимо вызвать конструктор класса CMutex для данной переменной, указав при этом изначальную принадлежность создаваемого объекта mutex, его имя (если это имя будет использоваться в данном процессе) и его атрибуты безопасности.

Для доступа к ресурсу, контролируемому объектом класса CMutex, необходимо создать объект класса CSingleLock или CMultiLock в функции-члене класса, осуществляющей доступ к данному ресурсу. Затем для данного объекта необходимо вызвать функцию Lock. При этом вызывающий поток или получит доступ к ресурсу, или будет ожидать доступа к ресурсу и наконец получит его, или, прождав указанный промежуток времени и не получив доступа к ресурсу, получит об этом сообщение. В любом случае к указанному ресурсу в каждый момент времени будет иметь доступ только один поток. Для освобождения ресурса следует вызвать для данного объекта функцию Unlock или выйти из области действия данного объекта.

Описание данного класса содержится в файле заголовка afxmt.h.

CMutex::CMutex

```
CMutex(BOOL bInitiallyOwn = FALSE, LPCTSTR lpszName = NULL,
  LPSECURITY_ATTRIBUTES lpsaAttribute = NULL);
```

Аргументы

bInitiallyOwn — определяет необходимость передачи права собственности на объект класса CMutex создающему его потоку.

lpszName — имя объекта класса CMutex, необходимое для организации взаимодействия между потоками процесса. Если этот аргумент имеет значение NULL, то объект является неименованным. Если существует другой объект класса CMutex с тем же именем, конструктор создает новый объект класса CMutex, ссылающийся на этот другой объект. Если значение данного аргумента совпадает с именем другого объекта синхронизации, не являющегося объектом mutex, то данный конструктор аварийно завершает свою работу.

lpsaAttribute — определяет атрибуты безопасности создаваемого объекта.

Описание

Создает именованный или неименованный объект класса CMutex. Для доступа или освобождения объекта класса CMutex необходимо создать объект класса CSingleLock или CMultiLock и вызвать их функции Lock и Unlock. Если объект класса CMutex используется отдельно, для его освобождения необходимо вызвать его функцию Unlock.

Класс CObArray

Класс CObArray представляет собой класс массива указателей на объект класса CObject. Этот массив объектов аналогичен массивам языка C, но способен динамически изменять свой размер.

Индекс первого элемента массива всегда равен 0. Пользователь может зафиксировать верхнюю границу массива или позволить ему расти при добавлении новых элементов, если их индекс превышает его верхнюю границу. Для данного массива всегда выделяется единая область памяти, даже если отдельные его элементы не используются.

В ОС Windows 95/98 размер объекта класса CObArray определяется только доступной областью памяти. Как и в случае массивов языка C, время доступа к любому элементу массива CObArray постоянно и не зависит от размера массива.

Класс CObArray включает в себя макрос IMPLEMENT_SERIAL, позволяющий работать с архивом и выводить диагностические сообщения. Если массив указателей на объекты класса CObject сохраняется в архиве, независимо от того, используется ли для этого перегруженный оператор << или функция Serialize, в архиве сохраняется каждый объект класса CObject, для чего вызывается его собственная функция Serialize.

Чтобы вывести диагностическую информацию об отдельном объекте класса CObject, хранящемся в массиве, необходимо задать глубину объекта CDumpContext равной 1 или большей величине.

При уничтожении объекта класса CObArray или при удалении из него элементов уничтожаются только указатели на объекты класса CObject, а не сами объекты, на которые ссылаются эти указатели.

Прежде чем использовать массив, необходимо вызвать функцию SetSize для установки его размера и резервирования необходимой памяти. Если при создании массива не использовалась функция SetSize, то при добавлении в него элементов часто будут производиться перераспределение памяти и копирование массива, что может привести к замедлению работы программы и к фрагментации памяти.

Работу с архивом обеспечивает макрос IMPLEMENT_SERIAL для данного класса.

Описание данного класса содержится в файле заголовка afxcoll.h.

CObArray::Add

```
int Add(CObject* newElement);
throw(CMemoryException);
```

Возвращаемое значение

Индекс добавленного в массив элемента.

Аргументы

`newElement` — указатель на объект класса `CObject`, добавляемый в массив.

Описание

Добавляет новый элемент в конец массива, увеличивая его размер на 1. Если в функции `SetSize` аргумент `nGrowBy` больше 1, и увеличение размера массива на 1 привело к выходу за пределы отведенной ему памяти, то для массива выделяется дополнительная память, в которую могут быть записаны новые элементы без новой операции выделения памяти, размер которой определяется аргументом `nGrowBy` функции `SetSize`.

Соответствующие функции, отличающиеся только типом своих аргументов, имеются в классах `CByteArray`, `CDWordArray`, `CPtrArray`, `CStringArray`, `CUIntArray` и `CWordArray`.

Пример

```
CObArray array;

    array.Add(new CAge(21)); // Элемент 0
    array.Add(new CAge(40)); // Элемент 1
#ifdef _DEBUG
    afxDump.SetDepth(1);
    afxDump << "Add example: " << &array << "\n";
#endif
```

Результат выполнения функции:

```
Add example: A CObArray with 2 elements
    [0] = a CAge at $442A 21
    [1] = a CAge at $4468 40
```

CObArray::GetSize

```
int GetSize() const;
```

Возвращаемое значение

Размер массива.

Описание

Возвращает размер массива. Поскольку индекс первого элемента массива равен нулю, размер массива всегда на 1 превышает максимальный индекс элемента массива.

Идентичная функция имеется в классах `CByteArray`, `CDWordArray`, `CPtrArray`, `CStringArray`, `CUIntArray` и `CWordArray`.

CObArray::GetUpperBound

```
int GetUpperBound() const;
```

Возвращаемое значение

Максимальный индекс элемента массива. Если возвращаемое значение равно −1, то данный массив не содержит элементов.

Описание

Возвращает максимальный индекс элемента массива. Поскольку индекс первого элемента массива равен нулю, максимальный индекс элемента массива всегда на 1 меньше, чем размер массива.

Идентичная функция имеется в классах `CByteArray`, `CDWordArray`, `CPtrArray`, `CStringArray`, `CUIntArray` и `CWordArray`.

Пример

```
CObArray array;

array.Add(new CAge(21)); // Элемент 0
array.Add(new CAge(40)); // Элемент 1
ASSERT(array.GetUpperBound() == 1); // Максимальный индекс
```

CObArray::operator [ ]

```
CObject*& operator [](int nIndex);
CObject* operator [](int nIndex) const;
```

Описание

Эти операторы могут использоваться вместо функций `SetAt` и `GetAt`. Первый оператор предназначен для обычных массивов и может вызываться как с левой, так и с правой стороны от оператора присваивания. Второй — для массивов констант и может вызываться только с правой стороны оператора присваивания.

В отладочной версии библиотеки проверяется, что используемый индекс массива находится в разрешенном диапазоне значений.

Соответствующие операторы, отличающиеся только типом возвращаемых значений, имеются в классах `CByteArray`, `CDWordArray`, `CPtrArray`, `CStringArray`, `CUIntArray` и `CWordArray`.

Пример

```
CObArray array;
CAge* pa;

array.Add(new CAge(21)); // Элемент 0
array.Add(new CAge(40)); // Элемент 1
```

```
pa = (CAge*)array[0];       // Получение указателя на элемент 0
ASSERT(*pa == CAge(21));    // Проверка содержимого элемента 0
array[0] = new CAge(30);    // Замена элемента 0
delete pa;
ASSERT(*(CAge*) array[0] == CAge(30));
```

CObArray::RemoveAll

```
void RemoveAll();
```

Описание

Удаляет все указатели из массива, но не уничтожает сами объекты класса `CObject`. Если массив уже пуст, функция все равно работает. Функция `RemoveAll` освобождает всю память, отведенную для хранения указателей.

Идентичная функция имеется в классах `CByteArray`, `CDWordArray`, `CPtrArray`, `CStringArray`, `CUIntArray` и `CWordArray`.

Пример

```
CObArray array;
CAge* pa1;
CAge* pa2;

array.Add(pa1 = new CAge(21)); // Элемент 0
array.Add(pa2 = new CAge(40)); // Элемент 1
ASSERT(array.GetSize() == 2);
array.RemoveAll();  // Уничтожаются указатели, а не объекты.
ASSERT(array.GetSize() == 0);
delete pa1;
delete pa2;         // Очистка памяти.
```

CObArray::SetSize

```
void SetSize(int nNewSize, int nGrowBy = -1);
throw(CMemoryException);
```

Аргументы

`nNewSize` — новый размер массива (количество элементов). Должен иметь неотрицательное значение.

`nGrowBy` — минимальное количество элементов, на которое будет увеличен размер массива при необходимости.

Описание

Устанавливает размер пустого или существующего массива. Выделяет память в случае необходимости. Если новый размер массива меньше, то массив усекается и вся неиспользуемая память освобождается. Для повышения производительности программы до начала работы с массивом необходимо

вызвать функцию SetSize. Правильный выбор ее аргументов позволит исключить или существенно уменьшить объем операций по перераспределению памяти и копирования массива при добавлении к нему элементов.

Аргумент nGrowBy влияет только на распределение памяти при увеличении размеров массива. Его значение никак не отражается на возвращаемых значениях функций GetSize и GetUpperBound.

Идентичная функция имеется в классах CByteArray, CDWordArray, CPtrArray, CStringArray, CUIntArray и CWordArray.

Класс *CObject*

Класс CObject является одним из основных классов библиотеки MFC (Microsoft Foundation Class Library). Он является базовым классом не только для классов библиотек, таких как CFile и CObList, но и для многих пользовательских классов. Класс CObject позволяет:

- Работать с архивами
- Создавать классы в процессе выполнения приложения
- Выполнять диагностику объектов
- Обеспечивать совместимость с классами коллекций

Класс CObject не поддерживает множественное наследование. Создаваемые на его основе классы могут иметь в качестве базового класса только класс CObject, и этот класс должен быть самым старшим в иерархии.

Чтобы получить доступ ко всем возможностям, предоставляемым классом CObject при объявлении и реализации производных от него классов, необходимо использовать специальные макросы.

Макросами первого уровня являются DECLARE_DYNAMIC и IMPLEMENT_DYNAMIC, обеспечивающие доступ к имени класса и его положению в иерархии в процессе выполнения приложения. Это, в свою очередь, позволяет создавать осмысленные диагностические сообщения.

Макросами второго уровня являются DECLARE_SERIAL и IMPLEMENT_SERIAL, обладающие всеми возможностями макросов первого уровня, но обеспечивающие возможность работы с архивами.

Описание данного класса содержится в файле заголовка afx.h.

CObject::AssertValid

```
virtual void AssertValid() const;
```

Описание

Функция CObject::AssertValid предназначена для отладки и проверяет объект на допустимость значений, хранимых в нем. В отладочной версии биб-

лиотеки данная функция может посредством вызова исключения прервать выполнение программы и вывести сообщение, в котором будет указано имя файла программы и номер строки, из которой было вызвано исключение.

При написании пользовательского класса следует перегрузить функцию `AssertValid` и поместить в ней необходимые проверки допустимости значений, хранящихся в объекте. Обычно первым оператором перегруженной функции `AssertValid` является вызов соответствующей функции базового класса.

Поскольку функция `AssertValid` является функцией-константой, пользователь не может изменять значения величин в проверяемом объекте. Перегруженная версия данной функции не должна сама вызывать исключения, а должна только сообщать, допустима ли данная комбинация параметров в объекте класса или нет.

Понятие допустимости зависит от самого класса. Как правило осуществляется только "поверхностная проверка". Например, если класс содержит указатели на другие объекты, то проверяется только, что эти указатели не нулевые, но не проверяется допустимость объектов, на которые они указывают.

Пример

```
void CAge::AssertValid() const
{
    CObject::AssertValid();
    ASSERT(m_years > 0);
    ASSERT(m_years < 105);
}
```

CObject::Dump

```
virtual void Dump(CDumpContext& dc) const;
```

Аргументы

`dc` — объект класса, осуществляющего потоковый вывод отладочной информации. Обычно это используемый по умолчанию объект `afxDump`.

Описание

Передает содержимое пользовательского объекта в объект класса `CDumpContext`. Создавая пользовательский класс, необходимо перегрузить функцию `Dump`, чтобы обеспечить выдачу диагностических сообщений об ошибках, произошедших в объекте данного класса. Обычно первым оператором перегруженной функции является вызов соответствующей функции базового класса. Функция `CObject::Dump` печатает имя класса, если при его создании вызывались макросы `IMPLEMENT_DYNAMIC` или `IMPLEMENT_SERIAL`.

При выводе пользовательской информации перегруженная функция `Dump` не должна использовать символ перевода каретки в конце своей работы. Дан-

ная функция работает только в отладочной версии библиотеки MFC, и ее описание должно располагаться между операторами условной трансляции `#ifdef _DEBUG` и `#endif`.

Поскольку функция `Dump` является функцией-константой, пользователь не может изменять значения величин в проверяемом объекте. Функция `Dump` вызывается перегруженным оператором записи объекта `CDumpContext::operator <<` при записи указателя на объект класса `CObject`. Функция `Dump` не предполагает рекуррентности, то есть пользователь может распечатать список объектов, но если один из объектов сам является списком, то его содержимое не будет распечатано.

Пример

```
void CAge::Dump(CDumpContext &dc) const
 {
 CObject::Dump(dc);
 dc << "Age = " << m_years;
 }
```

CObject::Serialize

```
virtual void Serialize(CArchive& ar);
throw(CMemoryException);
throw(CArchiveException);
throw(CFileException);
```

Аргументы

`ar` — объект класса `Carchive`, осуществляющий чтение и запись объекта на диске.

Описание

Читает объект с диска и записывает его на диск с использованием объекта класса архива.

Данная функция должна быть перегружена в каждом производном классе, предполагающем работу с архивом. Первым оператором перегруженной функции должен быть вызов функции `Serialize` базового класса. Чтобы иметь возможность вызвать данную функцию в своем классе, при его описании в файле заголовка следует использовать макрос `DECLARE_SERIAL`, а в его файле реализации — макрос `IMPLEMENT_SERIAL`. При помощи функций `CArchive::IsLoading` и `CArchive::IsStoring` можно определить, какая операция выполняется в данном случае.

Функция `Serialize` вызывается функциями `CArchive::ReadObject` и `CArchive::WriteObject`, которые, в свою очередь, вызываются переопределенными операторами `CArchive::operator <<` и `CArchive::operator >>`, пер-

вый из которых записывает объект в архив, а второй извлекает объект из архива.

Класс *CPoint*

Класс `CPoint` во многом аналогичен структуре Windows `POINT`. Этот класс содержит функции для работы с элементами класса `CPoint` и структуры `POINT`.

Объект класса `CPoint` может использоваться во всех случаях, где предполагается применение объекта структуры `POINT`. Все операторы данного класса, работающие с объектом структуры `SIZE`, могут использовать объект класса `CSize` и наоборот.

Данный класс является производным от структуры `tagPOINT`. (Имя `tagPOINT` является менее распространенным именем структуры `POINT`.) Это означает, что переменные `x` и `y` структуры `POINT` доступны как члены класса `CPoint`.

Описание данного класса содержится в файле заголовка afxwin.h.

Класс *CPrintInfo*

Класс `CPrintInfo` предназначен для хранения информации о текущем задании принтера. Приложение создает объект класса `CPrintInfo` при выборе команд меню **File**, **Print** или **File**, **Print Preview** и уничтожает его, завершая выполнение этих команд.

Объект класса `CPrintInfo` содержит информацию, касающуюся как всего процесса печати, например, диапазон печатаемых страниц, так и статуса текущего задания, например, номер печатаемой страницы. Некоторая информация, касающаяся процесса печати, хранится также в связанном с данным объектом класса `CPrintInfo` объекте класса `CPrintDialog`, хранящем информацию об установках пользователя в диалоговом окне **Печать**.

Объект класса `CPrintInfo` передается приложением в объект пользовательского класса представления и служит для обмена информацией между этими классами. Например, приложение передает объекту класса представления номер текущей печатаемой страницы в переменной данного класса `m_nCurPage`. Объект класса представления с помощью данной величины определяет, какую страницу следует печатать.

Другим примером использования объекта класса `CPrintInfo` является случай, когда размер документа не известен заранее и должен быть определен в процессе печати. В этом случае по достижении конца документа объект класса представления присваивает переменной `m_bContinuePrinting`, являющейся членом класса `CPrintInfo`, значение `FALSE`, тем самым информируя приложение о завершении процесса печати.

Класс CPrintInfo не имеет базового класса.

Описание данного класса содержится в файле заголовка afxext.h.

CPrintInfo::SetMaxPage

```
void SetMaxPage(UINT nMaxPage);
```

Аргументы

nMaxPage — номер последней страницы в документе.

Описание

Данная функция позволяет задать номер последней страницы в документе. Это значение будет храниться в переменной m_pPD класса CPrintDialog. Если размер документа известен до начала процесса печати, то эта функция вызывается в функции CView::OnPreparePrinting. Если же размер документа зависит от установок пользователя, сделанных им в диалоговом окне **Печать**, то эта функция вызывается из функции CView::OnBeginPrinting. Если же размер документа не может быть определен до завершения процесса печати, то используется переменная m_bContinuePrinting.

Класс CProgressCtrl

Линейный индикатор представляет собой окно, в котором приложение может отображать степень завершения длинной операции. Данный элемент управления представляет собой прямоугольник, который заполняется системным цветом в направлении слева направо по мере выполнения операции.

Класс CProgressCtrl обеспечивает функционирование линейного индикатора в операционной системе Windows. Этот элемент управления (а, следовательно, и класс CProgressCtrl) может быть использован только в программах, работающих под управлением операционных систем Windows 95 и Windows NT версии 3.51 и более поздних версий данных операционных систем.

Линейный индикатор характеризуется диапазоном отображаемых величин и своей текущей позицией. Диапазон отображаемых величин соответствует всему объему работы, выполняемому данной операцией, а текущая позиция характеризует ту часть работы, которую данная операция проделала на данный момент. Процедура окна линейного индикатора использует значения диапазона и текущей позиции, чтобы определить, какую часть окна необходимо заполнить системным цветом и какой текст вывести в окне (если в данном окне предусмотрен вывод текста). Поскольку для задания диапазона и текущей позиции используются целые числа со знаком, нижняя граница диапазона не может быть меньше −217 483 648, а верхняя граница диапазона не может превышать число 217 483 647.

Описание данного класса содержится в файле заголовка afxcmn.h.

CProgressCtrl::Create

```
BOOL Create(DWORD dwStyle, const RECT& rect, CWnd* pParentWnd,
  UINT nID);
```

Возвращаемое значение

TRUE, если был создан объект класса CProgressCtrl, в противном случае — FALSE.

Аргументы

dwStyle — определяет стиль линейного индикатора. Окно линейного индикатора всегда создается как дочернее окно, родительским окном которого обычно является диалоговое окно. Данный аргумент может представлять собой любую допустимую комбинацию стандартных стилей окна, однако все стили, кроме перечисленных ниже, будут игнорироваться:

- WS_BORDER — создает рамку вокруг окна
- WS_CHILD — создает дочернее окно (обязательный стиль для объекта класса CProgressCtrl)
- WS_CLIPCHILDREN — исключает области, занимаемые дочерними окнами из области перерисовки родительского окна. Задается при создании объекта класса родительского окна
- WS_CLIPSIBLINGS — исключает области, занимаемые другими дочерними окнами из области перерисовки данного дочернего окна
- WS_DISABLED — создает неактивное окно
- WS_VISIBLE — создает окно, которое сразу же отображается на экране
- WS_TABSTOP — включает данное окно в последовательность окон, по которой можно перемещаться путем нажатия клавиши <Tab>

Кроме данных стилей, линейный индикатор использует два дополнительных стиля:

- PBS_VERTICAL — ориентирует полосу линейного индикатора в вертикальном направлении, и она заполняется сверху вниз. Если этот флаг не установлен, полоса линейного индикатора располагается в горизонтальном направлении, и она заполняется слева направо
- PBS_SMOOTH — отображает сплошную полосу линейного индикатора. По умолчанию полоса линейного индикатора разделена на блоки

rect — определяет размер и положение линейного индикатора. Этот аргумент может представлять собой объект класса CRect или объект структуры RECT. Поскольку данный элемент управления представляет собой дочернее окно, определяемые в данном аргументе координаты являются

относительными и определяются системой координат, установленных в родительском окне.

`pParentWnd` — указатель на объект класса родительского окна, обычно это объект класса `CDialog`. Данный аргумент не может быть равен нулю.

`nID` — идентификатор ресурса линейного индикатора.

Описание

Объект класса `CProgressCtrl` создается в два этапа: на первом этапе вызывается конструктор, создающий объект класса `CProgressCtrl`, а на втором — функция `Create`, создающая связанное с ним окно.

CProgressCtrl::SetPos

```
int SetPos(int nPos);
```

Возвращаемое значение

Старое значение позиции линейного индикатора.

Аргументы

`nPos` — новая позиция линейного индикатора.

Описание

Устанавливает текущую позицию линейного индикатора, определяемую аргументом `nPos`, и перерисовывает индикатор, чтобы отразить внесенные в него изменения. Позиция линейного индикатора не соответствует его положению на экране, а определяет его положение относительно верхней и нижней границ диапазона, определенных функцией `SetRange`.

CProgressCtrl::SetRange

```
void SetRange(short nLower, short nUpper);
void SetRange32(int nLower, int nUpper);
```

Аргументы

`nLower` — определяет нижний предел диапазона (по умолчанию равен нулю).

`nUpper` — определяет верхний предел диапазона (по умолчанию равен 100).

Описание

Устанавливает верхний и нижний пределы диапазона линейного индикатора и перерисовывает индикатор, чтобы отразить внесенные в него изменения. Функция `SetRange32` устанавливает 32-разрядные границы диапазона линейного индикатора.

Класс CPropertyPage

Объект класса CPropertyPage представляет собой отдельную страницу окна свойств, иначе называемую вкладкой диалогового окна. Как и для стандартного диалогового окна, для каждой его вкладки необходимо создать отдельный класс, производный от класса CPropertyPage. Чтобы использовать объекты данного класса, надо сначала создать объект класса CPropertySheet, а затем включить в него объекты классов всех составляющих его вкладок. Чтобы добавить в диалоговое окно новую вкладку, необходимо вызвать для объекта ее класса функцию CPropertySheet::AddPage, а затем вывести это диалоговое окно на экран, вызвав функцию CPropertySheet::DoModal для модального диалогового окна или CPropertySheet::Create — для немодального диалогового окна.

Данное диалоговое окно может быть оформлено в стиле мастера, в котором диалоговое окно представляет собой последовательность вкладок, перемещение по которым происходит в определенной последовательности и управляется специальными кнопками в диалоговом окне. Такой тип окна часто применяется для установки программ и оборудования.

Описание данного класса содержится в файле заголовка afxdlgs.h.

CPropertyPage::OnSetActive

```
virtual BOOL OnSetActive();
```

Возвращаемое значение

Ненулевое, если вкладка успешно активизировалась, и ноль — в противном случае.

Описание

Данная функция вызывается приложением при выборе данной вкладки пользователем. Перегрузка этой функции позволяет произвести инициализацию содержащихся в ней объектов классов элементов управления. Первым оператором перегруженной функции должен быть вызов соответствующего метода базового класса.

По умолчанию функция создает окно для данной вкладки, если оно не было создано ранее, и делает эту вкладку активной.

CPropertyPage::OnWizardBack

```
virtual LRESULT OnWizardBack();
```

Возвращаемое значение

Нулевое, для успешного перехода к предыдущей странице, и −1 — для предотвращения перехода. Чтобы перейти к произвольной странице, возвратите идентификатор диалогового окна, которое необходимо вывести на экран.

Описание

Данная функция вызывается приложением, пользователь нажимает кнопку **<Назад** в окне мастера. Перегрузка данной функции позволяет определить действия, которые необходимо предпринять при возврате к предыдущей странице.

CPropertyPage::OnWizardFinish

```
virtual BOOL OnWizardFinish();
```

Возвращаемое значение

Ненулевое, если окно мастера было уничтожено при завершении его работы, и нулевое — в противном случае.

Описание

Данная функция вызывается приложением, если пользователь нажимает кнопку **Готово** в окне мастера. Перегрузка данной функции позволяет определить действия, которые необходимо предпринять при нажатии кнопки **Готово**.

Если данная функция возвращает значение `TRUE`, то диалоговое окно с вкладками уничтожается, и мастер прекращает свою работу. В противном случае диалоговое окно с вкладками не уничтожается.

CPropertyPage::OnWizardNext

```
virtual LRESULT OnWizardNext();
```

Возвращаемое значение

Нулевое, для успешного перехода к следующей странице, и −1 — для предотвращения перехода. Чтобы перейти к произвольной странице, возвратите идентификатор диалогового окна, которое необходимо вывести на экран.

Описание

Данная функция вызывается приложением, если пользователь нажал кнопку **Далее>** в окне мастера. Перегрузка данной функции позволяет определить действия, которые необходимо предпринять при переходе к следующей странице.

Класс CPropertySheet

Объекты класса `CPropertySheet` представляют собой диалоговые окна с вкладками. Данный объект обычно включает в себя один или несколько объектов класса `CPropertyPage`.

Хотя класс CPropertySheet и не является производным от класса CDialog, многие функции данного класса совпадают с функциями класса CDialog. Например, объект данного класса также создается в два этапа: сначала вызывается конструктор класса, а затем — функция DoModal для создания модального диалогового окна с вкладками или Create для создания немодального диалогового окна. Класс CPropertySheet имеет два типа конструкторов: CPropertySheet::Construct и CPropertySheet::CPropertySheet.

Обмен данными между объектом класса CPropertySheet и некоторым внешним объектом осуществляется аналогично обмену данными с объектом класса CDialog. Основным отличием является то, что инициализация объектов элементов управления вкладки выполняется с помощью переменных-членов класса CPropertyPage, а не переменных класса CPropertySheet.

Класс CPropertySheet позволяет создать диалоговое окно, оформленное в стиле мастера, представляющее собой последовательность вкладок, перемещение по которым происходит в определенной последовательности и управляется специальными кнопками в диалоговом окне. В диалоговом окне мастера вкладки не имеют ярлыков, и на экране отображается только одна вкладка. Вместо кнопок **ОК** и **Применить**, в диалоговом окне мастера используются кнопки **<Назад**, **Далее>** или **Готово**. В нем также используются кнопки **Отмена** и **Справка**.

Чтобы создать диалоговое окно мастера, необходимо сначала создать простое диалоговое окно с вкладками, а затем (перед вызовом функции DoModal) вызвать функцию SetWizardButtons. Чтобы после завершения работы пользователя с окном в нем появилась кнопка **Готово**, вызовите функцию SetFinishText.

Описание данного класса содержится в файле заголовка afxdlgs.h.

CPropertySheet::AddPage

```
void AddPage(CPropertyPage *pPage);
```

Аргументы

pPage — указатель на объект класса CPropertyPage, добавляемый в данное диалоговое окно. Не может быть равен нулю.

Описание

Данная функция добавляет вкладку в диалоговое окно и помещает ее ярлык в крайнюю правую позицию в окне. Поэтому вкладки располагаются в диалоговом окне слева направо в порядке их включения в объект класса диалогового окна.

Функция AddPage добавляет вкладку в окно, но не отображает ее на экране. Вкладка отображается только после выбора соответствующего ярлыка вкладки.

Диалоговое окно является родительским окном для вкладки, поэтому доступ к ней из объекта вкладки возможен при помощи функции `CWnd::GetParent`.

Функция `AddPage` может быть вызвана до функций `DoModal` или `Create`. При вызове данной функции в открытом диалоговом окне в ряду ярлыков немедленно произойдут соответствующие изменения.

CPropertySheet::Construct

```
void Construct(UINT nIDCaption, CWnd* pParentWnd = NULL,
  UINT iSelectPage = 0);
void Construct(LPCTSTR pszCaption, CWnd* pParentWnd = NULL,
  UINT iSelectPage = 0);
```

Аргументы

`nIDCaption` — идентификатор заголовка диалогового окна с вкладками.

`pParentWnd` — указатель на родительское окно диалогового окна с вкладками. Если этот аргумент равен нулю, то родительским окном для данного окна будет главное окно приложения.

`iSelectPage` — индекс вкладки, которая должна быть открыта при открытии окна. По умолчанию раскрывается первая вкладка. Нумерация вкладок определяется порядком их включения в диалоговое окно.

`pszCaption` — указатель на строку, содержащую заголовок диалогового окна с вкладками. Данный аргумент не может быть равен нулю.

Описание

Данная функция создает объекты класса `CPropertySheet`, например, при создании массива объектов данного класса. В этом случае функция `Construct` вызывается для каждого элемента массива.

Диалоговое окно с вкладками выводится на экран функциями `DoModal` и `Create`. Строка, определяемая первым аргументом данной функции, помещается в заголовок диалогового окна.

Пример

```
int i;
CPropertySheet   grpropsheet[4];
CPropertySheet   someSheet;   // no need to call Construct for this one

UINT rgID[4] = {IDD_SHEET1, IDD_SHEET2, IDD_SHEET3, IDD_SHEET4};

for (i = 0; i < 4; i++)
    grpropsheet[i].Construct(rgID[i]);
```

CPropertySheet::CPropertySheet

```
CPropertySheet();
CPropertySheet(UINT nIDCaption, CWnd *pParentWnd = NULL,
  UINT iSelectPage = 0);
CPropertySheet(LPCTSTR pszCaption, CWnd *pParentWnd = NULL,
  UINT iSelectPage = 0);
```

Аргументы

`nIDCaption` — идентификатор заголовка диалогового окна с вкладками.

`pParentWnd` — указатель на родительское окно диалогового окна с вкладками. Если этот аргумент равен нулю, то родительским окном для данного окна будет главное окно приложения.

`iSelectPage` — индекс вкладки, раскрываемой вместе с окном. По умолчанию раскрывается первая вкладка. Нумерация вкладок определяется порядком включения их в диалоговое окно.

`pszCaption` — указатель на строку, содержащую заголовок диалогового окна с вкладками. Данный аргумент не может быть равен нулю.

Описание

Данная функция создает объекты класса `CPropertySheet`. Диалоговое окно с вкладками выводится на экран функциями `DoModal` и `Create`. Строка, определяемая первым аргументом данной функции, помещается в заголовок диалогового окна.

CPropertySheet::DoModal

```
virtual int DoModal();
```

Возвращаемое значение

`IDOK` или `IDCANCEL`, если работа с диалоговым окном завершилась без ошибок. В противном случае — 0 или −1. Если диалоговое окно было вызвано в режиме мастера, данная функция возвращает `ID_WIZFINISH` или `IDCANCEL`.

Описание

Функция вызывается для отображения диалогового окна с вкладками. Возвращаемое значение определяется идентификатором элемента управления, с помощью которого было закрыто диалоговое окно. После выхода из функции объект класса окна уничтожается, но сам объект диалогового окна продолжает существовать. Обычно информация из переменных объектов класса `CPropertyPage` извлекается после того, как функция `DoModal` возвратит значение `IDOK`.

Немодальное окно отображает функция `Create`.

При первом создании вкладки диалогового окна из соответствующего ей ресурса диалогового окна может быть вызвано исключение. Это связано с тем, что объект класса `CPropertyPage` вносит изменения в стиль ресурса диалогового окна до создания самой вкладки. Поскольку обычно объекты ресурсов имеют атрибут "только для чтения", это вызывает исключение, которое обрабатывается программой, в результате чего в системе автоматически появляется модифицированная копия данного ресурса. Таким образом, вызванное исключение игнорируется.

Поскольку данное исключение должно обрабатываться операционной системой, не помещайте вызов функции `CPropertySheet::DoModal` в блок `try/catch`, обрабатывающий все исключения, например, при помощи оператора `catch (...)`. Данный оператор возьмет на себя обработку исключения, предназначенного для операционной системы, что может привести к непредсказуемым последствиям.

CPropertySheet::SetFinishText

```
void SetFinishText(LPCTSTR lpszText);
```

Аргументы

`lpszText` — указатель на строку, отображаемую в кнопке **Готово**.

Описание

Данная функция вызывается для задания текста, отображаемого в кнопке **Готово** по завершении работы пользователя с окном мастера. Кроме отображения заданного текста эта функция убирает из диалогового окна кнопку **<Назад**.

CPropertySheet::SetWizardButtons

```
void SetWizardButtons(DWORD dwFlags);
```

Аргументы

`dwFlags` — набор флагов, определяющих набор кнопок, используемых в мастере, и их доступность. Этот аргумент может представлять собой комбинацию следующих значений:

- `PSWIZB_BACK` — доступна кнопка **<Назад**
- `PSWIZB_NEXT` — доступна кнопка **Далее>**
- `PSWIZB_FINISH` — присутствует кнопка **Готово**
- `PSWIZB_DISABLEDFINISH` — кнопка **Готово** недоступна

Описание

Функция управляет доступностью кнопок **<Назад**, **Далее>** и **Готово** в диалоговом окне мастера. Функцию `SetWizardButtons` можно вызывать только в

открытом диалоговом окне (обычно из функции CPropertyPage::OnSetActive до функции DoModal).

Чтобы изменить текст в кнопке **Готово** и скрыть кнопки **Далее>** и **<Назад**, вызовите функцию SetFinishText. Поскольку кнопки **Готово** и **Далее>** разделяют одну и ту же программную кнопку, они не могут отображаться одновременно. Кнопка **Готово** имеет приоритет, если задано одновременное отображение обеих кнопок.

CPropertySheet::SetWizardMode

```
void SetWizardMode();
```

Описание

Данная функция устанавливает режим отображения вкладок в стиле мастера. Основными отличительными особенностями отображения вкладок в режиме мастера является применение кнопок **Далее>**, **<Назад**, **Готово** и **Отмена** вместо ярлыков вкладок для перехода от одной вкладки к другой.

Функция SetWizardMode вызывается перед функцией DoModal. После этого функция DoModal возвращает одно из значений: ID_WIZFINISH (если диалоговое окно было закрыто кнопкой **Готово**) или IDCANCEL (если диалоговое окно было закрыто кнопкой **Отмена**).

Данная функция устанавливает флаг PSF_WIZARD.

Пример

```
CPropertySheet dlg;
CPropertyPage page1, page2;

dlg.AddPage(&page1);
dlg.AddPage(&page2);
dlg.SetWizardMode();
dlg.DoModal();
```

Класс CRect

Класс CRect во многом аналогичен структуре RECT, определенной в операционной системе Windows. Класс CRect включает в себя функции для работы с объектами данного класса и структуры RECT. Объекты класса CRect могут использоваться в аргументах функций вместо объектов структуры RECT, а также вместо указателей LPCRECT или LPRECT.

Данный класс является производным от структуры tagRECT. (Имя tagRECT является менее распространенным именем для структуры RECT.) Это означает, что переменные left, top, right и bottom структуры RECT являются дос-

тупными членами класса CRect. Объект класса CRect содержит переменные для задания левого верхнего и правого нижнего углов прямоугольника.

Создавая объект класса CRect, необходимо убедиться, что задаваемые координаты прямоугольника нормализованы, то есть координата left меньше координаты right, а координата top меньше координаты bottom. Например, координаты левого верхнего угла (10,10) и координаты правого нижнего угла (20,20) задают нормализованный прямоугольник, а координаты левого верхнего угла (20,20) и координаты правого нижнего угла (10,10) задают ненормализованный прямоугольник. Если прямоугольник не нормализован, то многие функции класса CRect могут возвращать неправильные результаты. Список этих функций приведен в описании функции NormalizeRect. Перед вызовом данных функций необходимо вызвать функцию NormalizeRect, нормализующую объект своего класса.

При использовании объектов класса CRect в функциях CDC::DPtoLP и CDC::LPtoDP следует соблюдать осторожность. Если установленный режим отображения предполагает использование отрицательных вертикальных координат, как это имеет место в режиме отображения MM_LOENGLISH, то функция CDC::DPtoLP преобразует объект класса CRect таким образом, что его координата top становится больше координаты bottom. В результате функции Height и Size будут возвращать отрицательные значения высоты преобразованного объекта класса CRect, и данный прямоугольник станет ненормализованным.

При перегрузке операторов класса CRect первым операндом должен быть объект класса CRect, а вторым — объект структуры RECT или объект класса CRect.

Описание данного класса содержится в файле заголовка afxwin.h.

CRect::NormalizeRect

```
void NormalizeRect();
```

Описание

Нормализует объект класса CRect таким образом, чтобы значения его высоты и ширины были положительными. Функция NormalizeRect сравнивает значения переменных top и bottom и меняет их местами, если значение переменной top превышает значение переменной bottom. Аналогично, она сравнивает значения переменных left и right и меняет их местами, если переменная left больше переменной right. Эта функция полезна при работе с различными режимами отображения и инвертированными прямоугольниками.

Ниже приведен список функций класса CRect, требующих нормализации объекта данного класса для своей нормальной работы: Height, Width, Size,

IsRectEmpty, PtInRect, EqualRect, UnionRect, IntersectRect, SubtractRect, operator ==, operator !=, operator |, operator |=, operator & и operator &=.

CRect::OffsetRect

```
void OffsetRect(int x, int y);
void OffsetRect(POINT point);
void OffsetRect(SIZE size);
```

Аргументы

x — определяет смещение данного прямоугольника вправо. Если этот аргумент имеет отрицательное значение, то смещение происходит влево.

y — определяет смещение данного прямоугольника вниз. Если этот аргумент имеет отрицательное значение, то смещение происходит вверх.

point — содержит объект структуры POINT или объект класса CPoint, определяющий смещение по обоим направлениям.

size — содержит объект структуры SIZE или объект класса CSize, определяющий смещение по обоим направлениям.

Описание

Перемещает объект класса CRect в указанном направлении. Аргументы x и y данной функции имеют знак, поэтому прямоугольник можно перемещать в любом направлении.

CRect::PtInRect

```
BOOL PtInRect(POINT point) const;
```

Возвращаемое значение

Ненулевое, если данная точка лежит в пределах прямоугольника, описываемого объектом класса CRect, и ноль — в противном случае.

Аргументы

point — объект структуры POINT или объект класса CPoint, положение которого требуется оценить.

Описание

Определяет, лежит ли данная точка в пределах прямоугольника, описываемого объектом класса CRect. Точка считается лежащей в пределах прямоугольника, описываемого объектом класса CRect, если она лежит на его левой или верхней стороне или находится внутри него. Точка, лежащая на правой или нижней сторонах данного прямоугольника, считается находящейся за его пределами.

Для корректной работы данной функции прямоугольник должен быть нормализован. Для этого необходимо вызвать функцию `NormalizeRect` до вызова данной функции.

CRect::SetRect

```
void SetRect(int x1, int y1, int x2, int y2);
```

Аргументы

`x1` — определяет горизонтальную координату левого верхнего угла прямоугольника.

`y1` — определяет вертикальную координату левого верхнего угла прямоугольника.

`x2` — определяет горизонтальную координату правого нижнего угла прямоугольника.

`y2` — определяет вертикальную координату правого нижнего угла прямоугольника.

Описание

Устанавливает размеры объекта класса `CRect`.

Класс CRichEditView

Расширенное текстовое поле представляет собой окно, в котором пользователь может редактировать текст. Поддерживаются различные шрифты, введенный текст может форматироваться и в него могут включаться различные объекты OLE. Хотя расширенное текстовое поле и позволяет осуществлять форматирование текста, однако все необходимые для этого элементы пользовательского интерфейса должны быть созданы самим пользователем.

Класс `CRichEditView`, совместно с классами `CRichEditDoc` и `CRichEditCntrItem`, обеспечивают функционирование расширенного текстового поля в рамках заложенной в основу библиотеки MFC концепции Документ/представление. Класс `CRichEditView` содержит методы для редактирования и форматирования текста. Класс `CRichEditDoc` позволяет работать со списками клиентов OLE, используемыми в приложении. Класс `CRichEditCntrItem` обеспечивает доступ к клиентам OLE со стороны контейнера.

Использование расширенного текстового поля, а следовательно, и объектов класса `CRichEditCtrl` и связанных с ним классов возможно только под управлением операционной системы Windows 95 или Windows NT версии 3.51 или более поздних версий данных операционных систем.

Описание данного класса содержится в файле заголовка afxrich.h.

CRichEditView::OnCharEffect

```
void OnCharEffect(DWORD dwMask, DWORD dwEffect);
```

Аргументы

dwMask — определяет параметры форматирования шрифтов, подлежащие изменению. Может представлять собой комбинацию следующих значений:

- CFM_BOLD — подставляется значение флага CFE_BOLD из аргумента dwEffects
- CFM_COLOR — подставляется значение флага CFE_AUTOCOLOR из аргумента dwEffects
- CFM_ITALIC — подставляется значение флага CFE_ITALIC из аргумента dwEffects.
- CFM_PROTECTED — подставляется значение флага CFE_PROTECTED из аргумента dwEffects
- CFM_STRIKEOUT — подставляется значение флага CFE_STRIKEOUT из аргумента dwEffects
- CFM_UNDERLINE — подставляется значение флага CFE_UNDERLINE из аргумента dwEffects.

dwEffect — содержит список операций форматирования. Может представлять собой комбинацию следующих значений:

- CFE_AUTOCOLOR — цвет текста принимает значение, возвращаемое функцией GetSysColor (COLOR_WINDOWTEXT)
- CFE_BOLD — текст отображается полужирным шрифтом
- CFE_ITALIC — текст отображается курсивом
- CFE_STRIKEOUT — выполняется зачеркивание текста
- CFE_UNDERLINE — подставляется подчеркивание текста
- CFE_PROTECTED — устанавливает запрет на внесение изменений в параметры шрифта. При попытке внесения в него изменений посылается сообщение EN_PROTECTED

Описание

Данная функция позволяет изменять формат символов в выделенном тексте.

CRichEditView::GetCharFormatSelection

```
CHARFORMAT& GetCharFormatSelection();
```

Возвращаемое значение

Объект структуры CHARFORMAT, содержащий текущие атрибуты форматирования выделенного участка текста.

Описание

Данная функция позволяет получить текущие атрибуты форматирования выделенного участка текста.

CRichEditView::OnParaAlign

```
void OnParaAlign(WORD wAlign);
```

Аргументы

`wAlign` — определяет выравнивание абзаца. Может принимать одно из следующих значений:

- `PFA_LEFT` — по левому краю
- `PFA_RIGHT` — по правому краю
- `PFA_CENTER` — по центру

Описание

Данная функция форматирует выделенный абзац.

CRichEditView::SetCharFormat

```
void SetCharFormat(CHARFORMAT cf);
```

Аргументы

`cf` — объект структуры `CHARFORMAT`, содержащий новые атрибуты форматирования текста по умолчанию.

Описание

Данная функция позволяет установить атрибуты форматирования нового текста в объекте класса `CRichEditView`. В результате изменяются только те атрибуты текста, которые определены в переменной `dwMask` аргумента `cf`.

Класс CRgn

Класс `CRgn` содержит методы для работы с областями окон Windows с помощью интерфейса графических устройств. Область может иметь эллиптическую или прямоугольную форму. Для работы с областями окон совместно используются функции-члены класса `CRgn` и функции отсечения неотображаемых фрагментов класса `CDC`.

Функции класса `CRgn` позволяют создавать, изменять и получать информацию об областях окон Windows.

Описание данного класса содержится в файле заголовка afxwin.h.

CRgn::CreateRectRgnIndirect

```
BOOL CreateRectRgnIndirect(LPCRECT lpRect);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения операции, и ноль — в противном случае.

Аргументы

`lpRect` — указатель на объект структуры `RECT` или объект класса `CRect`, содержащий логические координаты левого верхнего и правого нижнего углов данной области.

Описание

Создает прямоугольную область и сохраняет ее в объекте класса `CRgn`. Размер области ограничен размером его сторон, каждая из которых не может превышать 32767 логических единиц, и объемом используемой данным объектом памяти, который не может превышать 64 Кб.

По окончании работы с объектом класса `CRgn`, созданного функцией `CreateRectRgnIndirect`, приложение должно удалить его, вызвав функцию `CGDIObject::DeleteObject`.

Класс CScrollView

Класс `CScrollView` реализует все возможности класса `CView`, дополняя их прокруткой изображения в окне.

Чтобы реализовать прокрутку изображения в любом классе, производном от класса `CView`, достаточно перегрузить в нем функции обработки сообщений `OnHScroll` и `OnVScroll`, однако класс `CScrollView` позволяет также:

- Самостоятельно обрабатывать информацию о размерах окна и его рабочей области, а также о режимах отображения
- Автоматически прокручивать изображение в ответ на сообщения, поступающие от полосы прокрутки
- Автоматически прокручивать изображение в ответ на сообщения, поступающие от клавиатуры, обычной мыши и от колесика мыши IntelliMouse

Сообщения от колесика мыши обрабатываются функциями `OnMouseWheel` и `OnRegisteredMouseWheel`. В классе `CScrollView` реализация этих функций по умолчанию обеспечивает рекомендованную реакцию на сообщение `WM_MOUSEWHEEL`.

Чтобы обеспечить в пользовательском классе просмотра возможность автоматической прокрутки окна, следует использовать в качестве его базового

класса класс `CScrollView` вместо класса `CView`. Для определения размера прокручиваемой области в функции `CView::OnInitialUpdate` вызывается функция `SetScrollSizes` (Для определения значений ее аргументов пользователю необходимо написать свой программный код.)

Функция `SetScrollSizes` устанавливает режим отображения, общий размер прокручиваемой области и шаг вертикальной и горизонтальной прокрутки. Все размеры задаются в логических единицах. Логический размер области представления обычно определяется исходя из информации, хранящейся в документе, но в некоторых случаях используются фиксированные размеры области представления.

Шаг горизонтальной и вертикальной прокрутки задается в логических единицах. По умолчанию, если пользователь щелкает левой кнопкой мыши в полосе прокрутки за пределами бегунка, то документ прокручивается на страницу. Если пользователь щелкает на стрелках прокрутки, расположенных на концах полосы прокрутки, то документ прокручивается на строку. По умолчанию страница представляет собой 1/10 от общего размера области представления, а строка — 1/10 от размера страницы. Чтобы изменить эти величины, следует вызвать функцию `SetScrollSizes`. Например, при установке параметров вертикальной прокрутки текста размер страницы может быть установлен таким образом, чтобы после прокрутки вниз первой строкой нового окна была последняя строка старого окна, а размер строки равнялся высоте текстовой строки, выведенной текущим шрифтом.

Вместо прокрутки изображения в окне объекта класса `CScrollView` может автоматически масштабировать область представления таким образом, чтобы ее размеры совпадали с размерами рабочей области окна. В этом режиме в окне отсутствуют полосы прокрутки. Чтобы использовать эту возможность, нужно вызвать функцию `CScrollView::SetScaleToFitSize`. (Можно вызвать функцию `SetScaleToFitSize` или `SetScrollSizes`, но не обе одновременно.)

Перед вызовом функции `OnDraw` пользовательского класса представления объект класса `CScrollView` автоматически устанавливает начало координат объекта класса `CPaintDC`, передаваемого в качестве аргумента данной функции. Чтобы установить начало координат в прокручиваемом окне, класс `CScrollView` перегружает функцию `CView::OnPrepareDC`. Это позволяет автоматически установить начало координат объекта класса `CPaintDC`, передаваемого классом `CScrollView` своей функции `OnDraw`. Работая с объектами класса `CClientDC` и другими подобными объектами классов контекста устройств, пользователю необходимо самому вызывать функцию `CScrollView::OnPrepareDC`. Перегрузка функции `CScrollView::OnPrepareDC` позволяет установить перо, цвет фона и другие атрибуты контекста устройств, но в этой функции обязательно должен быть вызван метод базового класса для реализации процедуры масштабирования.

Различают следующие разновидности полос прокрутки:

- Стандартные полосы прокрутки, имеющие стиль окна, и устанавливаемые стилями WS_HSCROLL и WS_VSCROLL
- Полосы прокрутки, добавляемые в главное окно приложения, содержащее данный объект класса представления. В этом случае главное окно приложения само посылает сообщения WM_HSCROLL и WM_VSCROLL активному объекту класса представления
- Главное окно приложения передает сообщения прокрутки от объекта класса элемента управления CSplitterWnd активной панели разделенного окна (представлению). Если в объекте класса CSplitterWnd реализованы разделяемые полосы прокрутки, то объект класса CScrollView использует эту полосу прокрутки, а не создает новую.

Описание данного класса содержится в файле заголовка afxwin.h.

CScrollView::SetScrollSizes

```
void SetScrollSizes(int nMapMode, SIZE sizeTotal,
  const SIZE& sizePage = sizeDefault, const SIZE& sizeLine = sizeDefault);
```

Аргументы

nMapMode — режим отображения, установленный в данном объекте класса просмотра. Значения, которые может принимать данный аргумент, перечислены в табл. В.6.

Таблица В.6. *Характеристики режимов отображения*

Режим отображения	Логические единицы	Направление по оси Y
MM_TEXT	1 элемент изображения	Вниз
MM_HIMETRIC	0.01 мм	Вверх
MM_TWIPS	1/1440 дюйма	Вверх
MM_HIENGLISH	0.001 дюйма	Вверх
MM_LOMETRIC	0.1 мм	Вверх
MM_LOENGLISH	0.01 дюйма	Вверх

Все эти режимы определены в Windows. Класс CScrollView не использует два стандартных режима: MM_ISOTROPIC и MM_ANISOTROPIC. В библиотеке MFC предусмотрена функция SetScaleToFitSize, масштабирующая область представления по размерам рабочей области окна.

sizeTotal — общий размер прокручиваемой области представления. Переменная cx данного объекта структуры SIZE содержит ее размер по гори-

зонтали, а переменная cy — ее размер по вертикали. Размеры измеряются в логических единицах. Как переменная cx, так и переменная cy должны иметь неотрицательное значение.

sizePage — шаг перемещения изображения по горизонтали или по вертикали при щелчке левой кнопкой мыши в полосе прокрутки за пределами бегунка. Переменная cx данного объекта структуры SIZE содержит шаг перемещения по горизонтали, а переменная cy — шаг перемещения по вертикали.

sizeLine — шаг перемещения изображения по горизонтали и вертикали при щелчке левой кнопкой мыши по стрелкам полосы прокрутки. Переменная cx данного объекта структуры SIZE содержит шаг перемещения по горизонтали, а переменная cy — шаг перемещения по вертикали.

Описание

Функция SetScrollSizes вызывается при обновлении окна представления. Обычно это делается в функции OnUpdate для настройки параметров прокрутки, например, при первом выводе документа на экран или при изменении его размеров.

Обычно информация, на основании которой вычисляются размеры области представления, получается из связанного с данным представлением документа. Например, если размер документа возвращается функцией GetMyDocSize, вызов данной функции будет выглядеть следующим образом:

```
SetScrollSizes(nMapMode, GetDocument()->GetMyDocSize());
```

В противном случае можно использовать фиксированный размер области представления, например:

```
SetScrollSizes(nMapMode, CSize(100, 100));
```

В данной функции может быть установлен любой режим отображения Windows, кроме MM_ISOTROPIC и MM_ANISOTROPIC. Если выбраны данные стили, то вместо функции SetScrollSizes вызывается функция SetScaleToFitSize.

Пример

```
void CScaleView::OnUpdate()
{
   // ...
   // Implement a GetDocSize() member function in
   // your document class; it returns a CSize.
   SetScrollSizes(MM_LOENGLISH, GetDocument()->GetDocSize());
   ResizeParentToFit();   // Default bShrinkOnly argument
   // ...
}
```

Класс CSemaphore

Объект класса CSemaphore представляет собой объект синхронизации "семафор", позволяющий ограниченному числу потоков одного процесса получить доступ к ресурсу. Объект класса CSemaphore содержит счетчик числа потоков, имеющих в данный момент право доступа к указанному ресурсу.

Семафоры используются для контроля доступа к разделяемому ресурсу, который могут разделять только ограниченное число пользователей. Текущее значение счетчика в объекте класса CSemaphore равно количеству пользователей, которые еще могут получить доступ к данному ресурсу. Когда значение счетчика становится равным нулю, все запросы на использование данного ресурса ставятся в системную очередь и ожидают в течение указанного промежутка времени, пока значение счетчика не превысит нулевое. Максимальное количество пользователей, которые могут получить доступ к контролируемому ресурсу, указывается в конструкторе класса CSemaphore.

Чтобы использовать объект класса CSemaphore, вызовите его конструктор, указав в нем имя семафора, и запросив право доступа к разделяемому ресурсу. После создания объекта доступ к ресурсу будет разрешен. Чтобы освободить ресурс, вызовите функцию CSyncObject::Unlock.

Объект класса CSemaphore применяется также в качестве переменной некоторого класса. Создавая объект этого класса, необходимо вызвать конструктор класса CSemaphore для данной переменной, указав при этом начальное значение счетчика доступа, максимальное значение счетчика доступа, его имя (если это имя будет использоваться в данном процессе) и его атрибуты безопасности.

Для доступа к ресурсу, подобным образом контролируемому объектом класса CSemaphore, необходимо создать объект класса CSingleLock или CMultiLock в функции-члене класса, осуществляющей доступ к данному ресурсу. После этого для данного объекта следует вызвать функцию Lock. При этом вызывающий поток или получит доступ к ресурсу, или будет ожидать доступа к ресурсу и, наконец, получит его, или, прождав указанный промежуток времени и не получив доступа к ресурсу, получит об этом сообщение. В любом случае к указанному ресурсу в каждый момент времени будет иметь доступ только один поток. Для освобождения ресурса следует вызвать для данного объекта функцию Unlock или выйти из области действия данного объекта.

Если объект класса CSemaphore создается отдельно, то пользователь может сам вызывать функции, необходимые для его работы, хотя это и сопряжено с возможностью возникновения ошибок.

CSemaphore::CSemaphore

```
CSemaphore(LONG lInitialCount = 1, LONG lMaxCount = 1,
  LPCTSTR pstrName = NULL, LPSECURITY_ATTRIBUTES lpsaAttributes = NULL);
```

Аргументы

`lInitialCount` — начальное значение счетчика доступа семафора. Может принимать любое значение в пределах от 0 до `lMaxCount`.

`lMaxCount` — максимальное значение счетчика доступа семафора. Может принимать любое положительное значение.

`pstrName` — имя объекта класса `CSemaphore`. Используется для взаимодействия между потоками процесса. Если этот аргумент имеет значение `NULL`, то данный объект является неименованным. Если существует другой объект семафора с тем же именем, конструктор создает новый объект класса `CSemaphore`, ссылающийся на существующий одноименный объект. Если значение данного аргумента совпадает с именем другого объекта синхронизации, не являющегося объектом семафора, то данный конструктор аварийно завершает свою работу.

`lpsaAttribute` — определяет атрибуты безопасности создаваемого объекта.

Описание

Создает именованный или неименованный объект класса `CSemaphore`. Для доступа или освобождения объекта класса `CSemaphore` необходимо создать объект класса `CSingleLock` или `CMultiLock` и вызвать их функции `Lock` и `Unlock`. Если объект класса `CSemaphore` используется отдельно, для его освобождения необходимо вызвать его функцию `Unlock`.

Класс *CSingleLock*

Объекты класса `CSingleLock` предназначены для контроля доступа к ресурсам в приложении, использующем несколько потоков. Для работы с классами синхронизации `CSemaphore`, `CMutex`, `CCriticalSection` и `CEvent` необходимо создать объект класса `CSingleLock` или `CMultiLock` для ожидания и освобождения объекта класса синхронизации. Объект класса `CSingleLock` позволяет работать только с одним объектом класса синхронизации. Объект класса `CMultiLock` позволяет одновременно работать с несколькими объектами классов синхронизации.

Для работы с объектом класса `CSingleLock` необходимо вызвать его конструктор в функции-члене класса контролируемого ресурса. После этого вызывается функция `IsLocked`, определяющая, доступен ли данный ресурс. Если да, то можно продолжать выполнение функции-члена класса контролируемого ресурса. Если ресурс недоступен, нужно или подождать его освобождения, или аварийно завершить данную функцию. После того как использование ресурса будет завершено, необходимо либо вызвать функцию `Unlock`, если предполагается дальнейшее использование объекта класса `CSingleLock`, либо уничтожить объект данного класса.

Класс `CSingleLock` не имеет базового класса.

Описание данного класса содержится в файле заголовка afxmt.h.

CSingleLock::IsLocked

```
BOOL IsLocked();
```

Возвращаемое значение

Ненулевое, если объект недоступен, и ноль — в противном случае.

Описание

Определяет, является ли объект, связанный с данным объектом класса `CSingleLock`, неотмеченным (недоступным).

CSingleLock::Lock

```
BOOL Lock(DWORD dwTimeOut = INFINITE);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

`dwTimeOut` — определяет интервал времени, в течение которого данная функция будет ожидать отметки объекта синхронизации (его доступности). Если этот аргумент имеет значение `INFINITE`, то функция `Lock` будет ждать до тех пор, пока объект не станет доступным.

Описание

Данная функция позволяет получить доступ к ресурсу, контролируемому объектом синхронизации, указанным в конструкторе данного класса. Если объект синхронизации устанавливается в отмеченное состояние, функция `Lock` успешно завершает свою работу, и поток, в котором она вызывается, получает право собственности на объект. Если объект синхронизации находится в неотмеченном состоянии (недоступен), функция `Lock` ожидает его отметки в течение указанного в аргументе `dwTimeOut` промежутка времени (измеряемого в миллисекундах). Если в течение этого времени объект синхронизации не отмечается, то функция `Lock` возвращает нулевое значение.

CSingleLock::CSingleLock

```
CSingleLock(CSyncObject* pObject, BOOL bInitialLock = FALSE);
```

Аргументы

`pObject` — указатель на объект класса синхронизации, к которому необходимо обеспечить доступ. Не может быть равен нулю.

bInitialLock — определяет необходимость запроса доступа к объекту класса синхронизации при создании данного объекта.

Описание

Создает объект CSingleLock. Как правило, данный объект создается в функции-члене класса, осуществляющей доступ к разделяемому ресурсу.

CSingleLock::Unlock

```
BOOL Unlock();
BOOL Unlock(LONG lCount, LPLONG lPrevCount = NULL);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения работы функции, и ноль — в противном случае.

Аргументы

lCount — количество доступов, которые необходимо освободить. Значение данного аргумента должно быть больше нуля. Если указанное количество приведет к переполнению счетчика объектов, значение счетчика не изменяется, а функция возвращает значение FALSE.

lPrevCount — указатель на переменную, в которую будет записано предыдущее значение счетчика синхронизации объектов. Если данный аргумент имеет значение NULL, то предыдущее значение счетчика не возвращается.

Описание

Освобождает объекты синхронизации, принадлежащие данному объекту класса CSingleLock. Эта функция вызывается деструктором класса CSingleLock. Чтобы освободить более одного объекта в семафоре, используйте вторую версию данной функции и укажите число освобождаемых объектов.

Класс CSize

Класс CSize аналогичен структуре Windows SIZE, хранящей относительные координаты или положение. Этот класс является производным от структуры SIZE, поэтому объекты класса CSize могут использоваться в аргументах функций, имеющих тип SIZE, а переменные структуры SIZE доступны из объекта класса CSize.

Переменные cx и cy, являющиеся членами структуры SIZE (и класса CSize), имеют модификатор прав доступа public. Класс CSize включает в себя функции для работы со структурой SIZE.

Описание данного класса содержится в файле заголовка afxwin.h.

Класс CSliderCtrl

Линейный регулятор представляет собой окно, содержащее бегунок и метки шкалы. Метки шкалы могут не отображаться. Когда пользователь перемещает бегунок мышью или клавишами управления курсором, данный элемент управления посылает сообщения о перемещении.

Этот элемент управления может использоваться для выбора дискретных или непрерывных значений из определенного диапазона. Например, для установки скорости повторения символов при нажатой клавише.

Класс `CSliderCtrl` обеспечивает функционирование линейного регулятора в ОС Windows. Этот элемент управления (а, следовательно, и класс `CSliderCtrl`) доступен только в программах, работающих под управлением Windows 95 и Windows NT версии 3.51 или более поздних версий данных операционных систем.

При создании объекта класса линейного регулятора вызываются его конструктор и функция `Create`. После создания объекта данного класса необходимо с помощью его функций задать параметры его отображения и масштаба, а именно: минимальную и максимальную позицию для бегунка, шаг меток шкалы, шаг перемещения при нажатии клавиш управления курсором и начальную позицию бегунка.

Описание данного класса содержится в файле заголовка afxcmn.h.

CSliderCtrl::ClearTics

```
void ClearTics(BOOL bRedraw = FALSE);
```

Аргументы

`bRedraw` — флаг перерисовки. Если данный аргумент принимает значение `TRUE`, то окно линейного регулятора перерисовывается после удаления меток шкалы. В противном случае окно линейного регулятора не перерисовывается.

Описание

Данная функция вызывается для удаления текущей метки шкалы из окна линейного регулятора.

CSliderCtrl::Create

```
BOOL Create(DWORD dwStyle, const RECT& rect, CWnd* pParentWnd, UINT nID);
```

Возвращаемое значение

Ненулевое, если инициализация прошла успешно, и ноль — в противном случае.

Аргументы

dwStyle — определяет стиль линейного регулятора. Может содержать любую комбинацию стилей данного элемента управления.

rect — определяет размеры и положение линейного регулятора. Может быть объектом класса CRect или структуры RECT.

pParentWnd — указатель на объект класса родительского окна линейного регулятора. Обычно это объект класса CDialog. Данный аргумент не может принимать нулевое значение.

nID — определяет идентификатор ресурса линейного регулятора.

Описание

Объект класса CSliderCtrl создается в два этапа. На первом этапе вызывается конструктор данного класса, а затем — функция Create, создающая окно линейного регулятора и связывающая его с объектом класса CSliderCtrl.

Линейный регулятор может иметь как горизонтальную, так и вертикальную ориентацию, метки шкалы с каждой из сторон, с обеих сторон или не иметь их вообще. Метки шкалы позволяют определить величину, устанавливаемую бегунком. Все эти свойства определяются стилями окна линейного регулятора, задаваемыми в первом аргументе данной функции. Линейный регулятор может иметь следующие стили:

- TBS_HORZ — задает горизонтальную ориентацию элемента управления. Этот стиль используется по умолчанию.
- TBS_VERT — задает вертикальную ориентацию элемента управления.
- TBS_AUTOTICKS — создает линейный регулятор с метками шкалы, соответствующими каждой позиции бегунка. Эти метки добавляются автоматически при вызове функции SetRange. Позицию или шаг меток шкалы можно изменить функциями SetTic или SetTicFreq. Для удаления меток шкалы вызовите функцию ClearTics.
- TBS_NOTICKS — создает линейный регулятор без меток шкалы.
- TBS_BOTTOM — помещает метки шкалы под горизонтальным линейным регулятором. Может сочетаться со стилем TBS_TOP для отображения меток шкалы с обеих сторон линейного регулятора.
- TBS_TOP — помещает метки шкалы над горизонтальным линейным регулятором. Может быть задан совместно со стилем TBS_BOTTOM для отображения меток шкалы с обеих сторон линейного регулятора.
- TBS_RIGHT — помещает метки шкалы справа от вертикального линейного регулятора. Этот стиль можно комбинировать со стилем TBS_LEFT для отображения меток шкалы с обеих сторон линейного регулятора.
- TBS_LEFT — помещает метки шкалы слева от вертикального линейного регулятора. Может использоваться совместно со стилем TBS_RIGHT для отображения меток шкалы с обеих сторон линейного регулятора.

- TBS_BOTH — помещает метки с обеих сторон линейного регулятора независимо от его ориентации.
- TBS_ENABLESELRANGE — отображает область выделения. Если линейный регулятор имеет данный стиль, то метки шкалы, соответствующие верхней и нижней границам выделенного диапазона, отображаются в виде треугольников (а не вертикальных или горизонтальных штрихов), и появляется область выделения. При помощи области выделения в простейших ежедневниках можно указывать время встречи.

CSliderCtrl::GetPos

```
int GetPos() const;
```

Возвращаемое значение

Текущее положение бегунка.

Описание

С помощью данной функции можно определить текущее положение бегунка в линейном регуляторе.

CSliderCtrl::SetLineSize

```
int SetLineSize(int nSize);
```

Возвращаемое значение

Предыдущее значение размера строки.

Аргументы

nSize — новый размер строки в линейном регуляторе.

Описание

Эта функция используется для установки нового размера строки в линейном регуляторе. Этот параметр определяет шаг перемещения бегунка при обработке сообщений TB_LINEUP и TB_LINEDOWN, которые посылаются соответствующими клавишами управления курсором.

CSliderCtrl::SetPageSize

```
int SetPageSize(int nSize);
```

Возвращаемое значение

Предыдущее значение размера страницы.

Аргументы

nSize — новый размер страницы в линейном регуляторе.

Описание

Функция устанавливает новый размер страницы в линейном регуляторе. Этот параметр определяет шаг перемещения бегунка при обработке сообщений TB_PAGEUP и TB_PAGEDOWN, посылаемых клавишами управления <PgUp> и <PgDn>.

CSliderCtrl::SetPos

```
void SetPos(int nPos);
```

Аргументы

nPos — определяет новое положение бегунка линейного регулятора.

Описание

Функция устанавливает новое положение бегунка в линейном регуляторе.

CSliderCtrl::SetRange

```
void SetRange(int nMin, int nMax, BOOL bRedraw = FALSE);
```

Аргументы

nMin — нижняя граница диапазона изменения.

nMax — верхняя граница диапазона изменения.

bRedraw — флаг перерисовки. Если этот аргумент имеет значение TRUE, то окно линейного регулятора будет перерисовано после установки новых параметров, в противном случае оно не будет перерисовано.

Описание

Задает диапазон перемещения бегунка в окне линейного регулятора.

CSliderCtrl::SetTic

```
BOOL SetTic(int nTic);
```

Возвращаемое значение

Ненулевое, если метка шкалы установлена, и ноль — в противном случае.

Аргументы

nTic — позиция метки шкалы. Этот аргумент может принимать только положительные значения.

Описание

Функция определяет позиции меток шкалы в линейном регуляторе.

CSliderCtrl::SetTicFreq

```
void SetTicFreq(int nFreq);
```

Аргументы

`nFreq` — шаг метки шкалы.

Описание

Данная функция устанавливает шаг меток шкалы, отображаемых в окне линейного регулятора. Например, если шаг равен 2, то метки шкалы отображаются для каждой второй позиции бегунка в окне линейного регулятора. По умолчанию эта величина равна единице (метка устанавливается у каждой позиции бегунка).

Чтобы иметь возможность использовать данную функцию, необходимо установить в функции `CSliderCtrl::Create` стиль `TBS_AUTOTICKS`.

Класс CStatusBar

Объект класса `CStatusBar` реализует строку состояния, состоящую из ряда окон, предназначенных для вывода сообщений и отображения различных состояний, например, для вывода справочной информации о командах меню и индикации состояния клавиш <Scroll Lock>, <Num Lock> и <Caps Lock>.

В библиотеке MFC версии 4.0 появилась функция `CStatusBar::GetStatusBarCtrl`, при помощи которой пользователь может получить непосредственный доступ к элементу управления Windows, что позволяет вносить в него изменения и использовать дополнительные возможности данного элемента управления. Функция `GetStatusBarCtrl` позволяет использовать дополнительные возможности строки состояния в Windows 95. Функция `GetStatusBarCtrl` возвращает ссылку на объект класса `CStatusBarCtrl`.

Приложение хранит информацию об индикаторе строки состояния в массиве, в котором крайнему левому индикатору соответствует нулевой индекс. Создавая объект класса строки состояния, приложение связывает массив индикаторов, с соответствующим массивом идентификаторов, которые, наряду с индексами массива индикаторов, обеспечивают доступ к индикаторам.

По умолчанию первый индикатор является "эластичным": он занимает все место, не занятое в строке состояния другими индикаторами. Таким образом, другие панели индикаторов выравниваются по правому краю.

Чтобы создать в приложении строку состояния:

1. Создайте объект класса `CStatusBar`.
2. Вызовите функцию `Create` (или `CreateEx`), создающую в окне строку состояния Windows и связывающую ее с объектом класса `CStatusBar`.

3. Вызовите функцию SetIndicators, сопоставляющую каждой панели индикатора свой идентификатор.

Существует три способа обновления текста в панели строки состояния:

- Вызов функции CWnd::SetWindowText для обновления текста нулевой панели
- Вызов функции CCmdUI::SetText в функции обработки сообщения ON_UPDATE_COMMAND_UI в классе строки состояния
- Вызов функцию SetPaneText для обновления текста в каждой панели

Для изменения стиля панели строки состояния вызывается функция SetPaneStyle.

Описание данного класса содержится в файле заголовка afxext.h.

CStatusBar::CommandToIndex

```
int CommandToIndex(UINT nIDFind) const;
```

Возвращаемое значение

Индекс панели индикатора, в случае успешного завершения функции, и −1 — в противном случае.

Аргументы

nIDFind — идентификатор панели, индекс которой требуется получить.

Описание

Позволяет получить индекс индикатора по заданному идентификатору. Первый индикатор имеет нулевой индекс.

CStatusBar::Create

```
BOOL Create(CWnd* pParentWnd, DWORD dwStyle = WS_CHILD | WS_VISIBLE |
  CBRS_BOTTOM, UINT nID = AFX_IDW_STATUS_BAR);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

pParentWnd — указатель на объект класса CWnd, окно которого является родительским окном строки состояния.

dwStyle — стиль строки состояния. В дополнение к стандартным стилям Windows в данном аргументе могут присутствовать следующие стили:

- CBRS_TOP — панель управления расположена над рабочей областью главного окна приложения
- CBRS_BOTTOM — панель управления расположена под рабочей областью главного окна приложения
- CBRS_NOALIGN — панель управления не изменяет своего положения при изменении размеров и положения родительского окна

nID — идентификатор дочернего окна строки состояния

Описание

Создает строку состояния (дочернее окно) и связывает его с объектом класса CStatusBar. Кроме того, данная функция задает исходный шрифт и устанавливает высоту строки состояния по умолчанию.

CStatusBar::SetIndicators

```
BOOL SetIndicators(const UINT* lpIDArray, int nIDCount);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

lpIDArray — указатель на массив идентификаторов.

nIDCount — размерность массива, на который указывает аргумент lpIDArray.

Описание

Сопоставляет каждому идентификатору индикатора значение, указанное для него в массиве lpIDArray, загружает строковый ресурс, определенный для каждого идентификатора, и выводит его в соответствующей панели.

CStatusBar::SetPaneInfo

```
void SetPaneInfo(int nIndex, UINT nID, UINT nStyle, int cxWidth);
```

Аргументы

nIndex — индекс панели индикатора, для которой устанавливается данный стиль.

nID — новый идентификатор для панели индикатора.

nStyle — новый стиль панели индикатора. Может принимать следующие значения:

- SBPS_NOBORDERS — панель не имеет объемной рамки

- SBPS_POPOUT — инвертированная рамка (панель при этом выглядит выпуклой)
- SBPS_DISABLED — запрещает вывод текста в панель
- SBPS_STRETCH — данная панель растягивается, заполняя все свободное место. Этот стиль может иметь только одна панель в строке состояния
- SBPS_NORMAL — панель не растягивается, не имеет рамки и не инвертирована

cxWidth — новая ширина панели индикатора.

Описание

Устанавливает для панели индикатора новые идентификатор, стиль и ширину.

CStatusBar::SetPaneText

```
BOOL SetPaneText(int nIndex, LPCTSTR lpszNewText, BOOL bUpdate = TRUE );
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

nIndex — индекс панели, в которую необходимо вывести текст.

lpszNewText — указатель на текстовую строку, содержащую выводимый текст.

bUpdate — если этот аргумент имеет значение TRUE, то содержимое панели обновляется для отображения в ней нового текста.

Описание

Выводит в панель индикаторов текст, на который указывает аргумент lpszNewText.

Класс CString

Объект класса CString хранит текстовые строки переменного размера. Функции и операции данного класса во многом аналогичны операторам языка Basic. Использование операций конкатенации и сравнения строк, а также упрощение работы с памятью существенно облегчают управление объектом класса CString по сравнению с обычным массивом символов.

Для представления символов в объекте класса CString используется тип TCHAR. Если для программы определен символ _UNICODE, то типу TCHAR соот-

ветствует тип wchar_t (16-разрядный символ), в противном случае этому типу соответствует тип char (обычный 8-разрядный символ). При использовании кодировки Unicode объекты класса CString состоят из 16-разрядных символов. Для кодировки Windows они состоят из 8-разрядных символов.

Если не определен символ _UNICODE, объект класса CString может содержать многобайтные наборы символов (MBCS известные также, как DBCS). При этом следует помнить, что объект класса CString при выполнении строковых операций рассматривает строки MBCS как набор 8-разрядных символов. Поэтому приложение должно само подготовить аргументы для данных операций и правильно интерпретировать полученные результаты.

Объекты класса CString имеют следующие особенности:

- Позволяют увеличивать размер хранимой строки для размещения результата операции слияния строк
- Представляют собой строку, а не указатель на нее
- Объект класса CString самостоятельно преобразуется к типу const char* и LPCTSTR, поэтому он может быть передан в качестве аргумента функции, имеющей данный тип
- Оператор явного преобразования типов позволяет получить доступ к содержимому объекта данного класса как к строке символов, имеющей атрибут "только для чтения"

По возможности следует избегать создания динамических объектов класса CString. Это сэкономит память и упростит доступ к данному объекту.

Класс CString позволяет незначительно уменьшить объем занятой оперативной памяти, поскольку две строки, имеющие одно и то же содержимое, разделяют одну область памяти. Однако эта копеечная экономия приводит к тому, что в случае непосредственного изменения содержимого буфера, содержащего данную строку (в обход функций библиотеки MFC), изменения будут внесены сразу в обе строки.

Класс CString включает в себя две функции, позволяющие защитить данные от подобных изменений: CString::LockBuffer и CString::UnlockBuffer. Первая (LockBuffer) создает копию данной строки, а ее счетчик обращений устанавливается в −1, что означает передачу данной копии в безраздельное пользование данному объекту класса CString. В этом режиме никакой другой объект класса CString не может получить доступ к данной строке, а данный объект класса не может получить доступ к другой строке. Этот режим гарантирует, что все изменения, внесенные в данный объект, не приведут к порче информации, хранимой в других объектах. По завершении модификации новой строки вызывается функция UnlockBuffer, устанавливающая счетчик обращений в 1.

Класс CString не имеет базового класса.

Описание данного класса содержится в файле заголовка afx.h.

CString::Empty

```
void Empty();
```

Описание

Очищает объект класса `CString` и освобождает память.

Пример

```
CString s("abc");
s.Empty();
ASSERT(s.GetLength() == 0);
```

CString::Format

```
void Format(LPCTSTR lpszFormat, ...);
void Format(UINT nFormatID, ...);
```

Аргументы

`lpszFormat` — строка форматирования.

`nFormatID` — идентификатор строкового ресурса, содержащий строку форматирования.

Описание

Данная функция позволяет записать в объект класса `CString` форматированную строку аналогично тому, как это делает функция `sprintf` для символьных массивов. В результате в объекте класса `CString` сохраняется форматированная последовательность символов и значений величин. Каждый дополнительный аргумент, если он присутствует, преобразуется и выводится в соответствии со спецификациями формата, содержащимися в аргументе `lpszFormat` или в строковом ресурсе, определяемом идентификатором `nFormatID`.

Функция аварийно завершит свою работу, если в качестве аргумента функции `Format` будет указана сама строка. Например, выполнение следующего программного кода приведет к непредсказуемым результатам:

```
CString str = "Some Data";
str.Format("%s%d", str, 123);
```

Если в качестве дополнительного аргумента передается строка символов, ее необходимо явно преобразовать к типу `LPCTSTR`. Аргументы и их назначение — такие же, как и для функции `printf`. В конец записанной строки символов добавляется нулевой символ.

CString::GetBuffer

```
LPTSTR GetBuffer(int nMinBufLength);
throw(CMemoryException);
```

Возвращаемое значение

Указатель на текстовую строку (завершающуюся нулем), хранимую в данном объекте (указатель типа LPTSTR).

Аргументы

nMinBufLength — минимальный размер текстового буфера в символах. Эта величина не включает завершающий строку нулевой символ.

Описание

Возвращает указатель на внутренний символьный буфер объекта класса CString. Этот указатель имеет тип LPTSTR и не является указателем на объект типа const, что позволяет вносить непосредственные изменения в содержимое объекта класса CString.

Если возвращенный данной функцией указатель используется для внесения изменений в текстовую строку, то перед тем как обращаться к любым другим функциям данного класса, необходимо вызвать функцию ReleaseBuffer.

Значение указателя, возвращенного функцией GetBuffer, после вызова функции ReleaseBuffer не всегда корректно, поскольку в результате вызова функций класса CString содержимое буфера может быть переписано в другое место. Этого не произойдет, пока не будет изменена длина текстовой строки в объекте класса CString.

При уничтожении объекта класса CString занимаемая им память автоматически освобождается.

Если пользователь сам устанавливает длину строки, то он не должен учитывать завершающий ее нулевой символ. Однако, освобождая буфер функцией ReleaseBuffer, необходимо указать длину строки, содержащейся в буфере. Если эта строка завершается нулевым символом, то при вызове функции ReleaseBuffer в качестве ее длины следует указать −1, что приведет к автоматическому вызову функции strlen для определения истинной длины строки.

Пример

```
CString s("abcd");
#ifdef _DEBUG
afxDump << "CString s " << s << "\n";
#endif
LPTSTR p = s.GetBuffer(10);
strcpy(p, "Hello");   // directly access CString buffer
s.ReleaseBuffer();
#ifdef _DEBUG
afxDump << "CString s " << s << "\n";
#endif
```

CString::IsEmpty

```
BOOL IsEmpty() const;
```

Возвращаемое значение

Ненулевое, если объект класса `CString` имеет нулевую длину, и ноль — в противном случае.

Описание

Проверяет, не содержит ли объект класса `CString` пустую строку.

Пример

```
CString s;
ASSERT(s.IsEmpty());
```

CString::LoadString

```
BOOL LoadString(UINT nID);
throw(CMemoryException);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

`nID` — идентификатор строкового ресурса Windows.

Описание

Считывает строковый ресурс Windows, определяемый аргументом `nID`, в существующий объект класса `CString`.

Пример

```
#define IDS_FILENOTFOUND 1
CString s;
if (! s.LoadString(IDS_FILENOTFOUND))
{
    AfxMessageBox("Error Loading String: IDS_FILENOTFOUND");
    ...
}
```

CString::MakeReverse

```
void MakeReverse();
```

Описание

Инвертирует порядок следования символов в объекте класса `CString`.

Пример

```
CString s("abc");
s.MakeReverse();
ASSERT(s == "cba");
```

Класс *CTime*

Объект класса CTime хранит информацию о дате и времени. Класс CTime включает в себя тип данных time_t и связанные с ним функции работы с временем, включая функции преобразования даты в григорианский календарь.

Значения времени, хранящиеся в объектах класса CTime, соответствуют универсальному координатному времени, которое, в свою очередь, соответствует времени по Гринвичу. Значение временного пояса хранится в переменной TZ.

Если аргумент nDST создаваемого объекта класса CTime равен нулю, значит будет использоваться стандартное время. Если этот аргумент больше нуля, то производится учет перехода на летнее время. В противном случае формат времени определяется библиотеками языка C. Если не установлено или не определено значение поля tm_isdst, то в переменной mktime возвращается непредсказуемое значение. Если переменная timeptr указывает на объект структуры tm, возвращенный функциями asctime, gmtime или localtime, то поле tm_isdst содержит корректное значение.

Сопутствующий класс CTimeSpan предназначен для хранения информации о промежутках времени, получаемых в результате операции вычитания над двумя объектами класса CTime. Оба эти класса не могут использоваться в качестве базовых для пользовательских классов. Поскольку они не имеют виртуальных функций, размер объектов классов CTime и CTimeSpan составляет 4 байта. Большинство их функций являются встраиваемыми.

Класс CTime не имеет базового класса.

Описание данного класса содержится в файле заголовка afx.h.

CTime::GetCurrentTime

```
static CTime PASCAL GetCurrentTime();
```

Описание

Возвращает объект класса CTime, хранящий текущее время.

Пример

```
CTime t = CTime::GetCurrentTime();
```

Класс CTimeSpan

Объект класса CTimeSpan используется для хранения значений промежутков времени. Класс CTimeSpan включает в себя тип данных time_t и связанные с ним функции работы с временем. Эти функции преобразуют секунды в различные комбинации дней, часов, минут и секунд.

Объекты класса CTimeSpan хранят информацию в секундах. Поскольку размер объекта класса CTimeSpan составляет 4 байта, то максимальный хранимый промежуток времени составляет около 68 лет.

Сопутствующий класс CTime предназначен для хранения информации об абсолютных значениях времени. Объект класса CTimeSpan хранит разность двух объектов класса CTime. Оба эти класса не могут использоваться в качестве базовых для пользовательских классов. Поскольку они не имеют виртуальных функций, размер объектов классов CTime и CTimeSpan составляет 4 байта. Большинство их функций является встраиваемыми.

Класс CTimeSpan не имеет базового класса.

Описание данного класса содержится в файле заголовка afx.h.

CToolBar

Объекты класса CToolBar — это панели управления с кнопками, содержащими битовые образы, между которыми могут находиться разделители. Кнопки панели инструментов могут действовать как обычные кнопки, переключатели или флажки. Обычно объекты класса CToolBar внедряются в объекты оконных классов, производных от класса CFrameWnd или от класса CMDIFrameWnd.

В библиотеке MFC версии 4.0 появилась функция CToolBar::GetToolBarCtrl, открывающая пользователю непосредственный доступ к элементу управления Windows (что позволяет вносить в него изменения), а также к некоторым дополнительным возможностям панелей инструментов в Windows 95. Функция GetToolBarCtrl возвращает ссылку на объект класса CToolBarCtrl.

Чтобы создать панель инструментов с помощью редактора ресурсов:

1. Создайте ресурс панели инструментов.
2. Создайте объект класса CToolBar.
3. Вызовите функцию Create (CreateEx) для создания панели инструментов Windows и связывания ее с объектом класса CToolBar.
4. Вызовите функцию LoadToolBar для загрузки ресурса панели инструментов.

Чтобы создать панель инструментов другим способом:

1. Создайте объект класса CToolBar.

2. Вызовите функцию `Create` (`CreateEx`) для создания панели инструментов Windows и связывания ее с объектом класса `CToolBar`.
3. Вызовите функцию `LoadBitmap` для загрузки битового образа, содержащего значки для кнопок панели инструментов.
4. Вызовите функцию `SetButtons` для установки стиля кнопок и сопоставьте каждой кнопке битовый образ.

Все значки для кнопок панели инструментов хранятся в одном битовом образе. Все значки должны иметь одинаковые размеры (по умолчанию 16 элементов изображения в ширину и 15 элементов изображения в высоту). Значки должны располагаться в битовом образе без промежутков.

Функция `SetButtons` получает указатель на массив идентификаторов элементов управления и целочисленную величину, определяющую количество элементов в массиве. Функция сопоставляет каждому идентификатору кнопки сегмент битового образа и назначает каждой кнопке индекс значка, определяющий положение значка кнопки в битовом образе. Если элемент массива имеет значение `ID_SEPARATOR`, ему не назначается никакой индекс значка.

Порядок значков в битовом образе обычно совпадает с порядком их отображения на экране, но при помощи функции `SetButtonInfo` порядок вывода значков может быть изменен.

Все кнопки в панели инструментов имеют одинаковый размер. По умолчанию их размер составляет 24×22 элемента изображения. Этот размер определен в документе *"Windows Interface Guidelines for Software Design"*. Дополнительное пространство, образовавшееся вследствие разницы размеров кнопки и значка, используется для формирования рамки вокруг значка.

Каждая кнопка имеет один значок. Все изображения на кнопке, отображающие ее состояние (нажата, отжата, недоступна, недоступна и нажата и промежуточное состояние) формируются из одного значка, связанного с кнопкой. Хотя битовый образ может иметь любой цвет, наилучшие результаты достигаются при использовании черного цвета и оттенков серого.

По умолчанию кнопки панели инструментов имитируют простые кнопки. Однако они могут имитировать также и переключатели и флажки. Флажки имеют три состояния: установлен, сброшен и неопределенное. Переключатели имеют только два состояния: установлен и сброшен.

Для установки стиля отдельной кнопки или разделителя без обращения к массиву сначала необходимо вызвать функцию `GetButtonStyle`, чтобы получить текущее состояние кнопки, а затем — функцию `SetButtonStyle` вместо функции `SetButtons`. Функцию `SetButtonStyle` целесообразно использовать при изменении стиля кнопки в процессе работы с приложением.

Чтобы кнопка содержала текст, вызовите функцию `GetButtonText`, чтобы получить требуемый текст, а затем — функцию `SetButtonText`.

Чтобы создать флажок в панели инструментов, необходимо использовать стиль TBBS_CHECKBOX или вызвать функцию CCmdUI::SetCheck при обработке сообщения ON_UPDATE_COMMAND_UI. Вызов функции SetCheck превращает обычную кнопку во флажок. Если аргумент функции SetCheck равен 0, то флажок сбрасывается, если 1, то флажок устанавливается, а значение 2 соответствует неопределенному состоянию.

Чтобы создать переключатель, необходимо вызвать функцию CCmdUI::SetRadio при обработке сообщения ON_UPDATE_COMMAND_UI. При передаче функции SetRadio нулевого аргумента переключатель сбрасывается, в противном случае он устанавливается. Для группы переключателей необходимо создать функции обработки сообщения ON_UPDATE_COMMAND_UI всех кнопок группы.

Описание данного класса содержится в файле заголовка afxext.h.

CToolBar::CreateEx

```
BOOL CreateEx(CWnd* pParentWnd, DWORD dwCtrlStyle = TBSTYLE_FLAT,
  DWORD dwStyle = WS_CHILD | WS_VISIBLE | CBRS_ALIGN_TOP,
  CRect rcBorders = CRect(0, 0, 0, 0), UINT nID = AFX_IDW_TOOLBAR);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

pParentWnd — указатель на родительское окно панели инструментов.

dwCtrlStyle — дополнительные стили для создания внедренного объекта класса CToolBarCtrl. По умолчанию этот аргумент имеет значение TBSTYLE_FLAT.

dwStyle — стили панели инструментов.

rcBorders — объект класса CRect, определяющий ширину рамки окна панели инструментов. По умолчанию этот аргумент имеет значение 0,0,0,0, поэтому окно панели инструментов не имеет рамки.

nID — идентификатор дочернего окна панели инструментов.

Описание

Данная функция создает панель инструментов Windows (дочернее окно) и связывает ее с объектом класса CToolBar. Кроме того, она устанавливает значение высоты панели инструментов по умолчанию.

Использование функции CreateEx вместо функции Create позволяет устанавливать стили при создании внедренного объекта класса CToolBarCtrl. Например, если аргумент dwCtrlStyle имеет значение TBSTYLE_FLAT|TBSTYLE_TRANSPARENT, то создается панель инструментов в стиле Internet Explorer 4.0.

CToolbar::LoadToolBar

```
BOOL LoadToolBar(LPCTSTR lpszResourceName);
BOOL LoadToolBar(UINT nIDResource);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

`lpszResourceName` — указатель на имя ресурса загружаемой панели инструментов.

`nIDResource` — идентификатор ресурса загружаемой панели инструментов.

Описание

Загружает панель инструментов (дочернее окно Windows), определяемую аргументом `lpszResourceName` или аргументом `nIDResource`.

Класс CView

Класс `CView` обеспечивает основные функциональные возможности классов представления, определяемых пользователем. Класс представления ассоциирован с классом документа и обеспечивает интерфейс между документом и пользователем: класс представления реализует отображение документа на экране или принтере, а также передает информацию о реакции пользователя в документ.

Класс представления является производным от класса `CWnd`. Одна и та же рабочая область окна может совместно использоваться несколькими классами представления. Связь между классом представления и классом окна реализована в шаблоне документа. Когда пользователь открывает новое окно или разбивает существующее, шаблон документа создает новый класс представления и присоединяет это окно к документу.

Класс представления может быть присоединен только к одному документу, но к одному документу могут быть присоединены несколько классов представления. Эта ситуация может возникнуть при отображении документа в разделенном окне или в нескольких дочерних окнах многооконного приложения. Приложение может поддерживать несколько различных классов представления, соответствующих одному классу документа. Например, программа обработки текстов могла бы выполнять просмотр как текста документа, так и его иерархической структуры (т. е. только заголовков разделов). Эти различные классы представления могут выводить информацию в различные окна или в различные панели разделенного окна, если окно открыто в этом режиме.

Класс представления может получать информацию от пользователя через различные устройства, например, с клавиатуры, через щелчки и перемещения мыши, из команд меню, инструментальных панелей или полос прокрутки. Сообщения в данный класс поступают из связанного с ним окна. Если класс представления не обрабатывает данную команду, он передает ее связанному с ним классу документа. Для сопоставления идентификатору сообщения функции его обработки класс представления использует карту сообщений.

Класс представления осуществляет только отображение и изменение данных в документе, но не хранит эти изменения. Вся отображаемая на экране информация содержится в классе документа. Класс представления может непосредственно обращаться к данным, хранящимся в документе, если в последнем предусмотрены методы доступа к этим данным.

После внесения изменений в документ класс представления, выполняющий данную операцию, обычно вызывает функцию `CDocument::UpdateAllViews`, посылающую другим классам просмотра, связанным с данным документом, сообщение о том, что в документ внесены изменения, и классам просмотра необходимо обновить выводимую ими информацию. Эти сообщения обрабатываются функцией `OnUpdate` соответствующего класса просмотра.

Обычно информация выводится на экран функцией `CView::OnDraw`. Эта же функция может выполнить предварительный просмотр печати и саму печать.

Описание данного класса содержится в файле заголовка afxwin.h.

CView::DoPreparePrinting

```
BOOL DoPreparePrinting(CPrintInfo* pInfo);
```

Возвращаемое значение

Ненулевое, если может начаться процесс печати или предварительного просмотра печати. Ноль, если выполнение операции было прервано.

Аргументы

`pInfo` — указатель на объект класса `CPrintInfo`, содержащий описание текущего задания для принтера.

Описание

Данная функция вызывается из перегруженной функции `OnPreparePrinting` для вывода диалогового окна **Печать** и создания контекста устройства принтера.

Режим работы функции (печать или предварительный просмотр), являющейся членом класса `CPrintInfo`, на объект которого указывает аргумент `pInfo`, определяется значением переменной `m_bPreview`. В режиме печати

файла данная функция выводит диалоговое окно **Печать**, используя значения переменных объекта класса `CPrintInfo`, на который указывает аргумент `pInfo`. После закрытия диалогового окна функция `DoPreparePrinting` создает объект класса контекста устройства принтера, основываясь на значениях переменных аргумента `pInfo`, хранящих установки пользователя в диалоговом окне **Печать**. Контекст устройства используется для печати документа.

В режиме предварительного просмотра функция создает контекст устройства принтера, основываясь на его текущих установках. Контекст устройства используется для имитации принтера в процессе предварительного просмотра.

CView::GetDocument

```
CDocument* GetDocument() const;
```

Возвращаемое значение

Указатель на объект класса `CDocument`, связанный с данным классом представления или `NULL`, если с данным классом не связан никакой документ.

Описание

Функция возвращает указатель на документ, связанный с данным классом представления. Это открывает данному классу доступ к методам класса документа.

CView::OnBeginPrinting

```
virtual void OnBeginPrinting(CDC* pDC, CPrintInfo* pInfo);
```

Аргументы

`pDC` — указатель на объект класса контекста устройства принтера.

`pInfo` — указатель на объект класса `CPrintInfo`, содержащий описание текущего задания для принтера.

Описание

Функция вызывается приложением при инициализации процесса печати или предварительного просмотра печати после вызова функции `OnPreparePrinting`. По умолчанию эта функция не выполняет никаких операций. Перегрузка данной функции позволяет включить в контекст устройства любые ресурсы GDI, такие как перья или шрифты, необходимые для осуществления процесса печати. Объекты GDI выбираются в контекст устройства функцией `OnPrint` отдельно для каждой страницы. Если вывод изображения на экран и его печать выполняется для одних и тех же объектов класса представления, то ресурсам GDI, используемым в каждом из этих режимов, следует сопоставить разные идентификаторы. Это позволит обновлять экран в процессе печати документа.

С помощью функции OnBeginPrinting можно также инициализировать величины, зависящие от параметров принтера. Например, количество страниц, необходимое для печати документа, может зависеть от установок пользователя в диалоговом окне **Настройка принтера** (таких как размер страницы). В этом случае размер документа не может быть определен в функции OnPreparePrinting, в которой недоступна вся необходимая для этого информация, содержащаяся в объекте класса контекста устройства принтера, создаваемого на основании установок в диалоговом окне **Настройка принтера**.

Функция OnBeginPrinting является первой перегружаемой функцией, имеющей доступ к объекту класса CDC, представляющего контекст устройства принтера. Поэтому в ней может быть определен размер документа. Если размер документа не будет определен, в окне предварительного просмотра печати не появится полоса прокрутки.

CView::OnDraw

```
virtual void OnDraw(CDC* pDC) = 0;
```

Аргументы

pDC — указатель на объект класса контекста устройства, используемого для отображения информации, хранящейся в документе.

Описание

Функция вызывается приложением для отображения информации, хранящейся в документе. С ее помощью приложение выводит изображение на экран, на печать или на предварительный просмотр печати. В любом случае передается указатель на объект класса контекста устройства, используемый в данном конкретном случае. Данная функция не имеет реализации по умолчанию.

Функция OnDraw должна перегружаться в каждом пользовательском классе представления. Объект класса CDC, передаваемый в качестве параметра, позволяет функции получить доступ к таким ресурсам GDI, как перья, шрифты и кисти. Вызов функции CDC::RectVisible сокращает объем вычислений за счет отказа от рисования невидимых участков изображения. Значение, возвращаемое функцией CDC::IsPrinting, определяет, будет ли данное изображение выводиться на принтер.

CView::OnInitialUpdate

```
virtual void OnInitialUpdate();
```

Описание

Вызывается приложением при первом связывании объекта представления с объектом документа до вывода этого объекта на экран. По умолчанию

данная функция вызывает функцию OnUpdate с нулевыми первым и вторым аргументами. Перегрузка данной функции позволяет установить некоторые неизменные соотношения между документом и представлением. Например, если приложение работает с документами фиксированного размера, перегрузка данной функции позволит установить параметры прокрутки в окне исходя из известных заранее размеров документа. Если в приложении используются документы различного размера, то эта операция должна выполняться в функции OnUpdate.

CView::OnPrepareDC

```
virtual void OnPrepareDC(CDC* pDC, CPrintInfo* pInfo = NULL);
```

Аргументы

pDC — указатель на объект класса контекста устройства, используемого для отображения документа.

pInfo — указатель на объект структуры CPrintInfo, описывающей текущее задание печати, если функция OnPrepareDC участвует в процессе печати или предварительного просмотра печати документа. Переменная m_nCurPage данной структуры содержит номер печатаемой страницы документа. Если функция OnPrepareDC вызывается для вывода на экран, данный аргумент имеет нулевое значение.

Описание

Вызывается приложением перед функцией OnDraw при выводе на экран либо перед печатью или ее предварительным просмотром каждой страницы документа функцией OnPrint. По умолчанию данная функция не выполняет никаких действий при выводе изображения на экран. Однако она может быть перегружена в производных классах, таких как CScrollView, для настройки атрибутов контекста устройства. Поэтому при ее перегрузке в пользовательских классах следует вызывать метод базового класса перед выполнением пользовательских операторов.

Если данная функция вызывается для печати документа, то по умолчанию она проверяет содержимое объекта структуры, на который указывает аргумент pInfo. Если размер документа в данном объекте структуры не задан, функция OnPrepareDC считает, что документ содержит всего одну страницу и останавливает процесс печати после распечатки первой страницы документа. Для остановки процесса печати данная функция присваивает переменной m_bContinuePrinting, являющейся членом структуры CPrintInfo, значение FALSE.

Обычно функция OnPrepareDC перегружается, чтобы:

- Установить атрибуты контекста устройства для указанной страницы при ее печати. Например, может быть установлен режим отображения

- Разбить документ на страницы в процессе печати. Обычно размер документа определяется при инициализации печати в функции `OnPreparePrinting`. Однако, если размер документа не может быть определен заранее (например, при печати записей базы данных, получаемых по запросу), перегрузка функции `OnPrepareDC` позволяет проверять признак конца документа во время печати. По достижении последней страницы документа данная функция присваивает переменной `m_bContinuePrinting`, являющейся членом структуры `CPrintInfo`, значение `FALSE`
- Посылать принтеру управляющие последовательности для его настройки на печать каждой страницы. Для посылки принтеру управляющих последовательностей вызовите функцию `CDC::Escape` для объекта класса, на который указывает аргумент `pDC`

При перегрузке данной функции первым ее оператором должен быть вызов *метода базового класса*.

CView::OnPreparePrinting

```
virtual BOOL OnPreparePrinting(CPrintInfo* pInfo);
```

Возвращаемое значение

Ненулевое для начала печати и ноль, если процесс печати был прерван.

Аргументы

`pInfo` — указатель на объект класса `CPrintInfo`, содержащий описание текущего задания для принтера.

Описание

Вызывается приложением перед печатью или предварительным просмотром документа. По умолчанию не выполняет никаких действий.

Данная функция должна быть перегружена для обеспечения возможности печати или предварительного просмотра документа. В ней необходимо вызвать функцию `DoPreparePrinting` и передать ей аргумент `pInfo`. Значение, возвращаемое функцией `DoPreparePrinting`, является возвращаемым значением данной функции. Функция `DoPreparePrinting` выводит диалоговое окно **Печать** и создает контекст устройства принтера. Если при инициализации диалогового окна **Печать** указаны значения параметров, отличные от значений по умолчанию, следует присвоить их соответствующим переменным аргумента `pInfo`. Например, при известном числе страниц в документе следует вызвать функцию `SetMaxPage`, являющуюся членом класса `CPrintInfo`, на объект которого указывает аргумент `pInfo`, перед вызовом функции `DoPreparePrinting`. Эта величина определит значения, выводимые в текстовых полях **Страницы с:** и **по:** диалогового окна **Печать**.

Функция DoPreparePrinting не выводит диалоговое окно **Печать** в режиме предварительного просмотра печати. Чтобы не выводить диалоговое окно **Печать** при печати документа, проверьте, что переменная m_bPreview, являющаяся членом класса CPrintInfo, на объект которого указывает аргумент pInfo, имеет значение FALSE, присвойте ей значение TRUE перед вызовом функции DoPreparePrinting и восстановите значение FALSE после выхода из этой функции.

Чтобы произвести инициализацию с использованием объекта класса CDC, представляющего контекст устройства принтера (например, для определения размера печатаемой страницы и количества страниц в документе), перегрузите функцию OnBeginPrinting.

Если необходимо установить значения переменных m_nNumPreviewPages или m_strPageDesc, являющихся членами класса CPrintInfo, на объект которого указывает аргумент pInfo, сделайте это после вызова функции DoPreparePrinting. Функция DoPreparePrinting присваивает переменной m_nNumPreviewPages значение из файла инициализации, а переменной m_strPageDesc — ее значение по умолчанию.

Пример

Ниже приведена заготовка функции OnPreparePrinting, генерируемая мастером AppWizard при установке соответствующего флажка в диалоге создания приложения. Эта заготовка может использоваться в рабочем приложении, если нет необходимости инициализировать диалоговое окно **Печать**.

```
BOOL CMyView::OnPreparePrinting(CPrintInfo *pInfo)
{
   return DoPreparePrinting(pInfo);
}
```

CView::OnPrint

```
virtual void OnPrint(CDC* pDC, CPrintInfo* pInfo);
```

Аргументы

pDC — указатель на объект класса контекста устройства принтера.

pInfo — указатель на объект класса CPrintInfo, содержащий описание текущего задания для принтера.

Описание

Вызывается приложением для печати или предварительного просмотра страницы документа. Для каждой печатаемой страницы приложение вызывает эту функцию сразу же после вызова функции OnPrepareDC. Номер печатаемой страницы определяется значением переменной m_nCurPage, являющейся членом класса CPrintInfo, на объект которого указывает аргумент

pInfo. По умолчанию данная функция вызывает функцию OnDraw и передает ей контекст устройства принтера.

Функция OnPrint перегружается для решения следующих задач:

- Для печати многостраничных документов. При печати документа с помощью функции OnDraw для определения фрагмента документа, который будет распечатываться на данной странице, необходимо установить начало отсчета рабочей области. (Однако данная операция обычно выполняется в функции OnPrepareDC.)
- Для изменения формы отображения документа при печати (если приложение не соответствует принципу WYSIWYG (Что вы видите, то вы и имеете). В данном случае вместо того чтобы передавать контекст устройства функции OnDraw, пользователь может вызвать собственную функцию, выводящую документ на печать и использующую для обработки изображения параметры, отличные от параметров вывода его на экран
- Для обращения к дополнительным ресурсам GDI, не используемым при выводе документа на экран, при его печати. Ресурсы GDI должны выбираться в контекст устройства перед началом печати и освобождаться после ее завершения. Эти ресурсы создаются в функции OnBeginPrinting и уничтожаются функцией OnEndPrinting
- Для вывода колонтитулов. При этом можно вызвать функцию OnDraw, ограничив для нее область печати

Переменная m_rectDraw, являющаяся членом класса CPrintInfo, на объект которого указывает аргумент pInfo, содержит размеры области печати страницы, выраженные в логических единицах. Не следует вызывать функцию OnPrepareDC при перегрузке функции OnPrint, поскольку приложение автоматически вызывает функцию OnPrepareDC перед вызовом функции OnPrint.

CView::OnUpdate

```
virtual void OnUpdate(CView* pSender, LPARAM lHint, CObject* pHint);
```

Аргументы

pSender — указатель на объект класса представления или NULL, если необходимо модифицировать все классы представления.

lHint — содержит информацию об изменениях.

pHint — указатель на объект, хранящий информацию об изменениях.

Описание

Данная функция обрабатывает сообщение, посылаемое функцией CDocument::UpdateAllViews. Функция UpdateAllViews вызывается классом представления после внесения изменений в связанный с ним документ. Функция OnUpdate вызывается функцией CView::OnInitialUpdate и по умолчанию посылает со-

общение о необходимости перерисовки всей рабочей области окна, которая будет произведена после получения очередного сообщения WM_PAINT. Чтобы перерисовывать не всю рабочую область окна, а только ее часть, необходимо перегрузить данную функцию в пользовательском классе представления. Информация об области отрисовки должна передаваться во втором или третьем параметре данной функции.

Параметр lHint данной функции обычно используется для передачи битовой маски или переменной перечислимого типа. Параметр pHint является указателем на объект класса CObject, который может содержать достаточно сложные структуры данных. Тип передаваемого объекта в перегруженной функции OnUpdate обычно определяется с помощью функции CObject::IsKindOf.

Как правило, функция OnUpdate не содержит вызовов функций отображения графической информации. Вместо этого она определяет прямоугольник, заданный в экранной системе координат, ограничивающий обновляемую область экрана, и передает указатель на него в качестве первого аргумента функции CWnd::InvalidateRec. Содержимое указанной области будет обновлено после получения очередного сообщения WM_PAINT.

Если аргумент lHint равен нулю, а аргумент pHint имеет значение NULL, то документ посылает стандартное сообщение о необходимости перерисовки рабочей области данного класса представления. Получив стандартное сообщение о необходимости перерисовки рабочей области или в том случае, когда класс представления не смог определить, какую информацию ему передали в параметрах сообщения, он обновляет всю свою рабочую область.

Класс *CWinApp*

Класс CWinApp является базовым классом для создания объектов классов приложений Windows. Объект класса представления содержит функции, позволяющие инициализировать приложение пользователя (и каждый его экземпляр), а также запускать приложение на исполнение.

Каждое приложение, использующее библиотеку MFC, может содержать только один объект класса CWinApp, создаваемый вместе с глобальными объектами, и уже существует к моменту вызова функции WinMain, содержащейся в библиотеке MFC. Объявление объектов классов, производных от класса CWinApp, также должно быть глобальным.

Функция InitInstance, принадлежащая данному классу, служит для создания объекта класса главного окна приложения.

Кроме функций класса CWinApp, библиотека MFC содержит следующие функции, позволяющие получить доступ к объектам класса CWinApp и к содержащейся в них информации:

- AfxGetApp — возвращает указатель на объект класса CWinApp
- AfxGetInstanceHandle — возвращает указатель на текущую копию объекта класса приложения
- AfxGetResourceHandle — возвращает дескриптор ресурсов приложения
- AfxGetAppName — возвращает указатель на строку, содержащую имя приложения. Другим способом получить эту информацию является использование указателя на объект класса CWinApp для доступа к переменной-члену данного класса m_pszExeName, содержащей указатель на ту же самую строку

Описание данного класса содержится в файле заголовка afxwin.h.

CWinApp::AddDocTemplate

```
void AddDocTemplate(CDocTemplate* pTemplate);
```

Аргументы

pTemplate — указатель на объект класса CDocTemplate, который необходимо добавить в данное приложение.

Описание

Данная функция добавляет объект класса шаблона документа в список доступных шаблонов документов данного приложения. Список доступных шаблонов документов должен быть заполнен до вызова функции RegisterShellFileTypes.

CWinApp::Enable3dControls

```
BOOL Enable3dControls();
BOOL Enable3dControlsStatic();
```

Возвращаемое значение

Равно TRUE, если библиотека CTL3D32.DLL загрузилась успешно, и FALSE — в противном случае. Данная функция возвращает FALSE, если операционная система поддерживает объемный дизайн элементов управления.

Примечание

Одно из двух: или эта информация, взятая из библиотеки MSDN, ошибочна, или Windows 95 OSR2 не поддерживает объемный дизайн элементов управления, поскольку в реальной программе, использующей объемный дизайн элементов управления, эта функция возвратила 1, то есть TRUE.

Описание

Вызов любой из этих функций из функции InitInstance пользовательского класса позволяет диалоговым окнам и окнам представлений данного прило-

жения использовать объемный дизайн элементов управления. Эти функции загружают библиотеку CTL3D32.DLL и регистрируют приложение с данной библиотекой динамической компоновки. Вызов функции `Enable3dControls` или `Enable3dControlsStatic` избавляет пользователя от необходимости вызывать функцию `SetDialogBkColor`, устанавливающую выбираемые по умолчанию цвета фона и текста в диалоговых окнах и окнах сообщений.

Функция `Enable3dControls` вызывается при компоновке библиотеки MFC в режиме библиотеки динамической компоновки. Функция `Enable3dControlsStatic` вызывается при компоновке библиотеки MFC в режиме статической библиотеки.

Использование статической компоновки библиотеки MFC возможно только в редакциях Professional и Enterprise.

Библиотека MFC автоматически обеспечивает объемный дизайн элементов управления для следующих оконных классов:

- `Cdialog`
- `CDialogBar`
- `CFormView`
- `CPropertyPage`
- `CPropertySheet`
- `CControlBar`
- `CToolBar`

Чтобы в окнах любого из перечисленных классов появились объемные элементы управления, достаточно вызвать функцию `Enable3dControls` или `Enable3dControlsStatic` в теле функции `InitInstance`. Чтобы реализовать объемный дизайн элементов управления в окнах, использующих другие классы, вам необходимо непосредственно вызывать методы из библиотеки CTL3D32.

CWinApp::ExitInstance

```
virtual int ExitInstance();
```

Возвращаемое значение

Ноль означает отсутствие ошибок, а положительная величина — их наличие. Это значение используется как возвращаемое значение функции `WinMain`.

Описание

Вызывается приложением из функции `Run` для выхода из данной копии приложения. Эта функция может вызываться только из функции `Run`. По умолчанию данная функция записывает параметры приложения в его файл .ini. Функция `ExitInstance` перегружается для очистки памяти при завершении приложения.

CWinApp::GetProfileString

```
CString GetProfileString(LPCTSTR lpszSection, LPCTSTR lpszEntry,
  LPCTSTR lpszDefault = NULL);
```

Возвращаемое значение

Строка, сохраняющаяся в файле приложения с расширением .ini, или строка, передаваемая в аргументе `lpszDefault`, если указанная строка не содержится в файле с расширением .ini. Максимальная длина строки, которая может быть получена подобным образом, равна величине `_MAX_PATH`. Если значение `lpszDefault` равно нулю, то возвращается пустая строка.

Аргументы

`lpszSection` — указатель на символьную строку, заканчивающуюся нулем, определяющий секцию, содержащую данную строку.

`lpszEntry` — указатель на символьную строку, заканчивающуюся нулем, определяющую идентификатор строки в записи. Этот аргумент не может принимать нулевое значение.

`lpszDefault` — указатель на символьную строку, заканчивающуюся нулем, содержащую строку, которая будет возвращаться функцией, если в соответствующем файле не будет найдена указанная строка.

Описание

Возвращает строку, связанную с идентификатором, содержащимся в указанной секции раздела системного реестра, отведенного данному приложению, или в файле с расширением .ini.

Пример

```
CString strSection      = "My Section";
CString strStringItem   = "My String Item";
CString strIntItem      = "My Int Item";

CWinApp* pApp = AfxGetApp();

pApp->WriteProfileString(strSection, strStringItem, "test");

CString strValue;
strValue = pApp->GetProfileString(strSection, strStringItem);
ASSERT(strValue == "test");

pApp->WriteProfileInt(strSection, strIntItem, 1234);
int nValue;
nValue = pApp->GetProfileInt(strSection, strIntItem, 0);
ASSERT(nValue == 1234);
```

CWinApp::InitInstance

```
virtual BOOL InitInstance();
```

Возвращаемое значение

Ненулевое, если инициализация прошла успешно, в противном случае — ноль.

Описание

Операционная система Windows допускает одновременную работу нескольких копий одного приложения. Процесс создания приложения концептуально подразделяется на два этапа: предварительное создание приложения, которое осуществляется при первом запуске программы на исполнение, и инициализация экземпляра приложения, которая происходит при запуске на исполнение любой копии программы, включая и первую. Данная функция вызывается функцией `WinMain`, принадлежащей классу главного окна.

Обычно в процессе выполнения перегруженной пользователем функции `InitInstance` создается объект главного окна программы и устанавливается значение члена класса `CWinThread::m_pMainWnd`, содержащего указатель на это окно.

CWinApp::LoadCursor

```
HCURSOR LoadCursor(LPCTSTR lpszResourceName) const;
HCURSOR LoadCursor(UINT nIDResource) const;
```

Возвращаемое значение

Дескриптор курсора, если функция завершилась успешно. В противном случае — ноль.

Аргументы

`lpszResourceName` — указатель на текстовую строку, заканчивающуюся нулем, содержащую имя ресурса курсора. В качестве данного аргумента может использоваться объект класса `CString`.

`nIDResource` — численное значение идентификатора ресурса значка.

Описание

Данная функция позволяет загрузить ресурс, определяемый аргументом `lpszResourceName` или аргументом `nIDResource` из текущего исполняемого файла. Функция `LoadCursor` загружает курсор в память, только если он еще не был загружен. В противном случае она возвращает дескриптор существующего ресурса.

Для доступа к предопределенным курсорам Windows могут быть использованы функции `LoadStandardCursor` и `LoadOEMCursor`.

Пример

```
HCURSOR hCursor;

    // Загрузка ресурса курсора, который был предварительно создан
    // в редакторе ресурсов и которому был присвоен идентификатор
    // IDC_MYCURSOR.
    hCursor = AfxGetApp()->LoadCursor(IDC_MYCURSOR);
```

CWinApp::LoadIcon

```
HICON LoadIcon(LPCTSTR lpszResourceName) const;
HICON LoadIcon(UINT nIDResource) const;
```

Возвращаемое значение

Дескриптор значка, если функция завершилась успешно. В противном случае — ноль.

Аргументы

`lpszResourceName` — указатель на текстовую строку, заканчивающуюся нулем, содержащую имя ресурса значка. В качестве данного аргумента может использоваться объект класса `CString`.

`nIDResource` — численное значение идентификатора ресурса значка.

Описание

Данная функция позволяет загрузить ресурс значка, определяемый аргументом `lpszResourceName` или аргументом `nIDResource` из исполняемого файла. Функция `LoadIcon` производит загрузку значка только в том случае, если он еще не был загружен. В противном случае она возвращает дескриптор существующего ресурса.

Для доступа к предопределенным значкам Windows могут быть использованы функции `LoadStandardIcon` и `LoadOEMIcon`.

CWinApp::LoadStdProfileSettings

```
void LoadStdProfileSettings(UINT nMaxMRU = _AFX_MRU_COUNT);
```

Аргументы

`nMaxMRU` — текущее число элементов в списке недавно использованных файлов.

Описание

Вызов данной функции осуществляется из функции `InitInstance` для загрузки списка недавно использованных файлов и последнего установленного режима предварительного просмотра. Если аргумент `nMaxMRU` равен нулю, то в данном приложении не поддерживается список недавно использо-

ванных файлов. Если же он не указан, выбирается величина, указанная при создании проекта.

CWinApp::OnHelp

```
afx_msg void OnHelp();
```

Описание

Для вызова данной функции необходимо добавить в карту сообщений пользовательского класса, производного от класса CWinApp, следующий макрос

```
ON_COMMAND(ID_HELP, OnHelp)
```

Обычно для вызова данной команды пользователь нажимает клавишу <F1>; использование данной клавиши представляет собой общепринятую практику и нигде не регламентировано.

По умолчанию данная функция обработки сообщения определяет контекст справки, соответствующий активному окну, диалоговому окну или команде меню, а затем вызывает на исполнение файл WinHelp.exe. Если для данного объекта не определен контекст, то используется контекст, определенный по умолчанию. Перегрузка данной функции позволяет задать контекст для объекта, отличного от объектов окна, диалогового окна, команды меню или кнопки панели инструментов, которым в данный момент принадлежит фокус ввода. В данной функции вызывается функция WinHelp, которой передается контекстный идентификатор справки.

CWinApp::ParseCommandLine

```
void ParseCommandLine(CCommandLineInfo& rCmdInfo);
```

Аргументы

rCmdInfo — ссылка на объект класса CCommandLineInfo.

Описание

Преобразует параметры командной строки в значения, присваиваемые элементам объекта класса CCommandLineInfo. Для заполнения каждого элемента данного объекта вызывается своя функция CCommandLineInfo::ParseParam.

Если с помощью мастера AppWizard создается новое приложение, использующее библиотеку MFC, этот мастер генерирует в функции InitInstance локальный объект класса CCommandLineInfo, а затем вызывает функции ProcessShellCommand и ParseCommandLine. Параметры командной строки обрабатываются следующим образом:

1. Будучи созданным в функции InitInstance, объект класса CCommandLineInfo передается в качестве параметра функции ParseCommandLine.

2. Функция ParseCommandLine последовательно вызывает функцию CCommandLineInfo::ParseParam для каждого параметра командной строки.
3. Функция ParseParam присваивает значения переменным в объекте класса CCommandLineInfo, который затем передается в качестве аргумента функции ProcessShellCommand.
4. Функция ProcessShellCommand выполняет действия, указанные в параметрах командной строки.

CWinApp::ProcessShellCommand

```
BOOL ProcessShellCommand(CCommandLineInfo& rCmdInfo);
```

Возвращаемое значение

Ненулевое, если переданные данной функции команды были выполнены успешно, в противном случае функция InitInstance возвращает ноль.

Аргументы

rCmdInfo — ссылка на объект класса CCommandLineInfo.

Описание

Данная функция вызывается в функции InitInstance для обработки параметров командной строки, передаваемых ей в объекте структуры CCommandLineInfo.

Генерируя новое приложение, использующее библиотеку MFC, мастер AppWizard создает в функции InitInstance локальный объект класса CCommandLineInfo, а затем вызывает функции ProcessShellCommand и ParseCommandLine. Параметры командной строки обрабатываются следующим образом:

1. Объект класса CCommandLineInfo, созданный в функции InitInstance, передается в качестве параметра функции ParseCommandLine.
2. Функция ParseCommandLine последовательно вызывает функцию CCommandLineInfo::ParseParam для каждого параметра командной строки.
3. Функция ParseParam присваивает значения переменным в объекте класса CCommandLineInfo, который затем передается в качестве аргумента функции ProcessShellCommand.
4. Функция ProcessShellCommand выполняет действия, указанные в параметрах командной строки.

Переменная CCommandLineInfo::m_nShellCommand представляет собой переменную перечислимого типа, определенную в классе CCommandLineInfo следующим образом:

```
enum{
   FileNew,
   FileOpen,
   FilePrint,
   FilePrintTo,
   FileDDE,
};
```

CWinApp::RegisterShellFileTypes

```
void RegisterShellFileTypes(BOOL bCompat = FALSE);
```

Аргументы

`bCompat` — если этот аргумент имеет значение `TRUE`, то в данном приложении регистрируются команды программной оболочки `Print` и `Print To`, с помощью которых пользователь может осуществлять печать непосредственно из программной оболочки или путем перетаскивания файла в объект печати. В этом случае в приложение добавляется ключ `DefaultIcon`. По умолчанию данный аргумент имеет значение `FALSE`, что обеспечивает приложению совместимость с предыдущими версиями.

Описание

Функция `RegisterShellFileTypes` регистрирует все типы документов, используемые в данном приложении в Диспетчере файлов Windows. Это позволяет пользователю открывать файл данных, созданный этим приложением, двойным щелчком мыши в окне Диспетчера файлов. Функция `RegisterShellFileTypes` вызывается после создания шаблонов всех документов, используемых в приложении.

Эта функция просматривает список объектов классов шаблона документов данного приложения и регистрирует каждый из них в базе данных Windows, содержащей сведения о файлах. Эта база данных используется в Диспетчере файлов для открытия файлов при двойном щелчке на их значке.

CWinApp::SetRegistryKey

```
void SetRegistryKey(LPCTSTR lpszRegistryKey);
void SetRegistryKey(UINT nIDRegistryKey);
```

Аргументы

`lpszRegistryKey` — указатель на строку, содержащую имя ключа.

`nIDRegistryKey` — идентификатор или индекс ключа в реестре.

Описание

Вызов данной функции позволяет сохранять параметры начальной установки приложения пользователя в системном реестре, а не в файлах с расши-

рением .ini, как это имело место в старых версиях Windows. В ней устанавливается ключ m_pszRegistryKey, который может затем использоваться в функциях-членах класса CWinApp, таких как GetProfileInt, GetProfileString, WriteProfileInt и WriteProfileString. Вызов данной функции приводит к сохранению в системном реестре списка недавно использованных файлов. В качестве ключа регистрации обычно выбирается название компании. В этом случае адрес конкретной переменной в системном реестре выглядит следующим образом HKEY_CURRENT_USER\Software\<название компании>\<имя приложения>\<имя секции>\<идентификатор переменной>.

CWinApp::WinHelp

```
virtual void WinHelp(DWORD dwData, UINT nCmd = HELP_CONTEXT);
```

Аргументы

dwData — определяет дополнительную информацию. Трактовка данного аргумента зависит от значения аргумента nCmd.

nCmd — определяет тип запрашиваемой справочной информации. Список возможных значений данного аргумента совпадает со списком возможных значений аргумента dwData глобальной функции WinHelp.

Описание

Вызывается для запуска приложения WinHelp. Приложение WinHelp автоматически закрывается при закрытии вызвавшего его приложения.

Пример

```
// Файл заголовка HELPIDS.H
//
// Этот файл заголовка включается в два файла:
// (1) Он включается в файл .CPP, передающий контекстный
//     идентификатор, имеющий тип DWORD, функции CWinApp::WinHelp.
// (2) Он включается в секцию [MAP] файла .HPJ для связи
//     контекстной строки справки "HID_MYTOPIC" с целочисленным
//     идентификатором справки, имеющим значение 101.
// Контекстная строка справки "HID_MYTOPIC" используется для
// идентификации раздела справки в исходном файле справки
// формата .RTF
//
// Эту операцию не следует выполнять для разделов справочной
// системы, связанных с идентификаторами команд и элементов
// управления пользовательского интерфейса, определенных в файле
// RESOURCE.H; Это делается с помощью программы MAKEHM,
// использующей стандартные правила преобразования файла
// resource.h в файл адресации справок (.HM) для данных
// идентификаторов.
```

```
#define HID_MYTOPIC 101

// Файл реализации HELPIDS.CPP
   // В функции обработки сообщения вызывается справочная
   // система для отображения раздела справки, связанного
   // с контекстной строкой "HID_MYTOPIC" в исходном файле
   // справки формата .RTF, связанного с идентификатором
   // HID_MYTOPIC в приведенном выше фрагменте файла HELPIDS.H.
   AfxGetApp()->WinHelp(HID_MYTOPIC);

// Ниже приведена строка из файла адресации справок(.HM),
// созданного программой MAKEHM из файла resource.h file.
// Программа MAKEHM считывает следующий оператор #define в файле //
RESOURCE.H данного приложения:
//    #define ID_MYCOMMAND 0x08004
// и добавляет к нему смещение 0x10000 для получения из него
// контекстного идентификатора 0x18004.

HID_MYCOMMAND  0x18004

   // Иногда пользователю приходится непосредственно вызывать
   // функцию WinHelp с использованием контекстных
   // идентификаторов команд или элемента управления
   // пользовательского интерфейса. Приложение автоматически
   // вызывает функцию WinHelp при нажатии пользователем
   // клавиши <F1>, если фокус ввода принадлежит команде меню.
   // Однако, если пользователь хочет непосредственно вызвать
   // раздел справки, связанный с командой меню My Command,
   // ему необходимо вызвать следующую функцию:

   AfxGetApp()->WinHelp(0x10000 + ID_MYCOMMAND);
```

CWinApp::WriteProfileString

```
BOOL WriteProfileString(LPCTSTR lpszSection, LPCTSTR lpszEntry,
  LPCTSTR lpszValue);
```

Возвращаемое значение

Ненулевое, если операция прошла успешно, и ноль — в противном случае.

Аргументы

`lpszSection` — указатель на символьную строку, заканчивающуюся нулем, определяющий секцию, в которой содержится данная строка. Если данная секция не существует, то она создается. Имя секции не зависит от регистра используемых в ней символов, и она может представлять собой любую комбинацию заглавных и прописных букв.

`lpszEntry` — указатель на символьную строку, заканчивающуюся нулем и определяющую идентификатор строки в записи. Если данный идентификатор в секции не существует, то он создается.

`lpszValue` — указатель на строку, которую необходимо сохранить. Если этот параметр имеет значение `NULL`, то идентификатор, указанный в аргументе `lpszEntry`, удаляется из записи.

Описание

Вызывается для сохранения строки в указанной секции реестра приложения или в файле с расширением .ini.

Пример

```
CString strSection       = "My Section";
CString strStringItem    = "My String Item";
CString strIntItem       = "My Int Item";

CWinApp* pApp = AfxGetApp();

pApp->WriteProfileString(strSection, strStringItem, "test");

CString strValue;
strValue = pApp->GetProfileString(strSection, strStringItem);
ASSERT(strValue == "test");

pApp->WriteProfileInt(strSection, strIntItem, 1234);
int nValue;
nValue = pApp->GetProfileInt(strSection, strIntItem, 0);
ASSERT(nValue == 1234);
```

Класс *CWinThread*

Объекты класса `CWinThread` представляют собой потоки, исполняемые в приложении. Основной поток приложения обычно представлен объектом класса, производного от класса `CWinApp`, который, в свою очередь, является производным от класса `CWinThread`. Дополнительные объекты класса `CWinThread` позволяют создавать несколько потоков в одном приложении.

Существует две основные разновидности потоков: рабочие потоки и интерфейсные потоки. Рабочие потоки не содержат цикла обработки сообщений. Примером такого потока может служить выполнение фоновых вычислений в приложении, использующем рабочие листы. Интерфейсные потоки содержат цикл обработки сообщений и могут обрабатывать сообщения, поступающие от системы. Класс `CWinApp` и производные от него являются примером интерфейсных потоков. Другие классы интерфейсных потоков могут происходить непосредственно от класса `CWinThread`.

Объекты класса CWinThread обычно уничтожаются при завершении потоком своей работы. Для сохранения этих объектов по завершении работы потока необходимо присвоить переменной m_bAutoDelete значение FALSE.

Вся информация, необходимая для взаимодействия потока с приложением, содержится в соответствующем объекте класса CWinThread. Поэтому любой поток, использующий библиотеку MFC, должен быть производным от данного класса. Например, поток, созданный функцией _beginthreadex, не может использоваться в приложениях, работающих с библиотекой MFC.

Поток создается функцией AfxBeginThread, которая имеет две версии, первая из которых создает рабочий поток, а вторая — интерфейсный. При создании рабочего потока первым аргументом функции AfxBeginThread является указатель на исполняющую функцию данного потока. При создании интерфейсного потока первым аргументом функции AfxBeginThread является указатель на объект структуры CRuntimeClass пользовательского класса, производного от класса CWinThread. В обеих версиях функций могут быть указаны дополнительные аргументы, определяющие приоритет потока, размер его стека и атрибуты безопасности. Функция AfxBeginThread возвращает указатель на созданный ею объект класса, производного от класса CWinThread.

Объект класса, производного от класса CWinThread, может быть создан своим конструктором (вместо вызова функции AfxBeginThread), после чего для этого объекта должна быть вызвана функция CreateThread. Этот двухэтапный метод может быть применен для последовательного создания нескольких потоков с использованием одного объекта класса CWinThread.

CWinThread::CreateThread

```
BOOL CreateThread(DWORD dwCreateFlags = 0, UINT nStackSize = 0,
  LPSECURITY_ATTRIBUTES lpSecurityAttrs = NULL);
```

Возвращаемое значение

Ненулевое, если создание потока завершилось успешно, и ноль — в противном случае.

Аргументы

dwCreateFlags — определяет дополнительный флаг, задающий режим создания потока. Этот флаг может иметь два значения:

- CREATE_SUSPENDED — счетчик остановки создаваемого потока устанавливается в единицу. Чтобы запустить поток, необходимо вызвать функцию ResumeThread
- 0 — поток запускается на исполнение немедленно после создания

nStackSize — определяет размер стека нового потока в байтах. Если эта величина равна нулю, то создаваемому потоку отводится стек того же размера, что и вызывающему.

lpSecurityAttrs — указатель на объект структуры SECURITY_ATTRIBUTES, определяющий атрибуты безопасности данного потока. Если эта величина равна нулю, то создаваемый поток имеет те же атрибуты безопасности, что и вызывающий поток.

Описание

Создает поток в адресном пространстве вызывающего процесса. Создать объект класса потока и одновременно запустить его можно с помощью функции AfxBeginThread. Функция CreateThread обычно вызывается в том случае, когда один и тот же объект класса потока используется для последовательного создания нескольких потоков.

CWinThread::ResumeThread

```
DWORD ResumeThread();
```

Возвращаемое значение

В случае успешного завершения функции возвращается предыдущее значение счетчика остановки. В противном случае возвращается значение 0xFFFFFFFF (−1). Если возвращаемое значение равно нулю, то данный поток не был остановлен. Если возвращаемое значение равно единице, то данный поток был остановлен, но теперь продолжает свою работу. Любое возвращаемое значение, большее единицы, означает, что исполнение остановленного потока не было возобновлено.

Описание

Данная функция вызывается для возобновления работы потока, приостановленного функцией SuspendThread или созданного с флагом CREATE_SUSPENDED. Функция ResumeThread уменьшает на единицу значение счетчика остановки. Если данный счетчик равен нулю, то поток продолжает свою работу. В противном случае работа потока приостановлена.

CWinThread::Run

```
virtual int Run();
```

Возвращаемое значение

Целочисленное значение, возвращаемое потоком. Это значение может быть получено с помощью функции ::GetExitCodeThread.

Описание

Содержит цикл обработки сообщений интерфейсного потока, используемый по умолчанию. Функция Run получает и распределяет сообщения Windows до тех пор, пока приложение не получит сообщение WM_QUIT. Если очередь сообщений потока пуста, то функция Run вызывает функцию OnIdle, выпол-

няющую фоновую обработку. Поступающие сообщения сначала обрабатываются функцией PreTranslateMessage, производящей их специальную обработку, а затем — функцией Windows ::TranslateMessage, обрабатывающей сообщения от клавиатуры. После этого вызывается функция Windows ::DispatchMessage.

Функция Run очень редко перегружается для обеспечения нестандартного пользовательского интерфейса и используется только в интерфейсных потоках.

CWinThread::SetThreadPriority

```
BOOL SetThreadPriority(int nPriority);
```

Возвращаемое значение

Ненулевое, если создание потока завершилось успешно, и ноль — в противном случае.

Аргументы

nPriority — определяет новое значение уровня приоритета в своем классе приоритета. Этот аргумент может принимать следующие значения, перечисленные в порядке убывания приоритета:

- THREAD_PRIORITY_TIME_CRITICAL
- THREAD_PRIORITY_HIGHEST
- THREAD_PRIORITY_ABOVE_NORMAL
- THREAD_PRIORITY_NORMAL
- THREAD_PRIORITY_BELOW_NORMAL
- THREAD_PRIORITY_LOWEST
- THREAD_PRIORITY_IDLE

Дополнительная информация об уровнях приоритета потока содержится в описании функции ::SetThreadPriority.

Описание

Функция устанавливает уровень приоритета текущего потока в своем классе приоритета. Функция SetThreadPriority может вызываться только после успешного завершения работы функции CreateThread.

CWinThread::SuspendThread

```
DWORD SuspendThread();
```

Возвращаемое значение

В случае успешного завершения функции возвращается предыдущее значение счетчика остановки. В противном случае возвращается значение 0xFFFFFFFF (−1).

Описание

Функция увеличивает на единицу счетчик остановки потока. Если значение счетчика для данного потока превышает ноль, то его выполнение приостанавливается. Исполнение потока возобновляется функцией `ResumeThread`.

Класс *CWnd*

Класс `CWnd` обеспечивает основные функциональные возможности всех классов окон в библиотеке MFC. Объекты класса `CWnd` не являются окном Windows, но тесно с ним связаны. Если объект класса `CWnd` является полноценным объектом класса, создается своим конструктором и уничтожается своим деструктором, то окно Windows является внутренней структурой данных Windows, создаваемой функцией `CWnd::Create` и уничтожаемой виртуальным деструктором класса `CWnd`. Функция `CWnd::DestroyWindow` уничтожает окно Windows, не уничтожая объект класса `CWnd`.

Класс `CWnd` и механизм карты сообщения скрывают от пользователя функцию `WndProc`. Поступающие в объект данного класса сообщения Windows автоматически направлены через карту сообщения к соответствующим функциям обработки сообщений класса `CWnd`. Чтобы обработать в пользовательском классе сообщение, поступающее от некоторого элемента управления, пользователю необходимо включить в свой класс функцию обработки данного сообщения.

С помощью класса `CWnd` пользователь может создать дочернее окно Windows для своего приложения. Для этого достаточно создать пользовательский класс, производный от класса `CWnd`, а затем добавить в него переменные для хранения данных, характерных для данного окна приложения. После чего необходимо включить в карту сообщений макросы для обработки приходящих в класс сообщений и написать тела функций обработки сообщений, объявленных в карте сообщений.

Дочернее окно создается в два этапа. Сначала вызывается конструктор класса `CWnd`, создающий объект данного класса, а затем — функции `Create`, которая создает дочернее окно и присоединяет его к объекту класса `CWnd`.

Когда пользователь закрывает дочернее окно, происходит уничтожение объекта класса `CWnd` или вызывается функция `DestroyWindow`, уничтожающая окно и связанные с ним структуры данных.

В библиотеке MFC собрано множество классов, производных от `CWnd`. Многие из этих классов, включая `CFrameWnd`, `CMDIFrameWnd`, `CMDIChildWnd`, `CView` и `CDialog`, сами используются в качестве базовых для различных пользовательских классов.

Классы элементов управления, являющиеся производными от класса CWnd, такие как класс CButton, могут использоваться непосредственно или в качестве базовых для пользовательских классов.

Описание данного класса содержится в файле заголовка afxwin.h.

CWnd::BeginPaint

```
CDC* BeginPaint(LPPAINTSTRUCT lpPaint);
```

Возвращаемое значение

Указатель на объект класса контекста устройства, связанного с объектом класса CWnd. Возвращаемый указатель на объект класса контекста устройства может быть временным и не должен уничтожаться до вызова функции EndPaint.

Аргументы

lpPaint — указатель на объект структуры PAINTSTRUCT, в которую будет записана информация, необходимая для вывода на экран.

Описание

Подготавливает объект класса CWnd к выводу графической информации и заполняет объект структуры PAINTSTRUCT.

Объект структуры PAINTSTRUCT содержит объект структуры RECT, в котором содержатся координаты минимального по размерам прямоугольника, полностью включающего в себя область обновления окна, и флаг, определяющий необходимость уничтожения фона в процессе обновления окна.

Область обновления окна устанавливается функциями Invalidate, InvalidateRect или InvalidateRgn или операционной системой при изменении ею размеров окна, его перемещении, осуществлении прокрутки в окне или проведении ею других операций, приводящих к необходимости обновления рабочей области окна. Если область обновления окна имеет флаг, указывающий на необходимость стирания отображаемой в ней информации, функция BeginPaint посылает сообщение WM_ONERASEBKGND.

Функция BeginPaint может вызываться только при обработке сообщения WM_PAINT. Каждому вызову функции BeginPaint должен соответствовать вызов функции EndPaint. Если в области обновления окна располагается текстовый курсор, функция BeginPaint автоматически его скрывает для того, чтобы он не был уничтожен.

CWnd::DestroyWindow

```
virtual BOOL DestroyWindow();
```

Возвращаемое значение

Ненулевое, в случае уничтожения окна, в противном случае — ноль.

Описание

Уничтожает окно, связанное с объектом класса CWnd. Функция DestroyWindow посылает соответствующие сообщения окну, чтобы деактивировать его и переместить с него фокус ввода. При этом также уничтожается меню окна, очищается его очередь приложений, уничтожаются внешние таймеры, отменяется его право собственности на буфер обмена данными и прекращается связь буфера обмена с объектом класса представления, если объект класса CWnd непосредственно связан с буфером обмена. Данная функция посылает окну сообщения WM_DESTROY и WM_NCDESTROY, но не уничтожает сам объект класса CWnd.

Функция DestroyWindow вызывается по умолчанию для очистки окна. Поскольку функция DestroyWindow является виртуальной, ее описание присутствует в каждом классе, производном от класса CWnd. Однако даже в том случае, если данная функция перегружается в производном классе, она не обязательно будет вызываться.

Предположим, что в пользовательском классе, производном от класса CView, перегружена функция DestroyWindow. Поскольку функции библиотеки MFC не вызывают функцию DestroyWindow для объектов классов, производных от класса CFrameWnd, функция DestroyWindow не будет вызываться, пока пользователь не вызовет ее непосредственно.

Если данное окно является родительским для какого-либо другого окна, это дочернее окно автоматически уничтожается вместе с родительским. Функция DestroyWindow сначала уничтожает все дочерние окна данного окна, а затем уничтожает и само окно. Эта функция используется для уничтожения немодальных диалоговых окон, созданных функцией CDialod::Create. Если уничтожаемое окно само является дочерним окном, и для него не установлен стиль WS_EX_NOPARENTNOTIFY, то в результате выполнения данной функции родительскому окну посылается сообщение WM_PARENTNOTIFY.

CWnd::DoDataExchange

```
virtual void DoDataExchange(CDataExchange* pDX);
```

Аргументы

pDX — указатель на объект класса CDataExchange.

Описание

Вызывается приложением для обмена данными и проверки корректности данных, передаваемых между объектами классов диалогового окна и включенных в него объектов классов элементов управления.

Эта функция не должна вызываться непосредственно. Ее вызов допустим только через вызов функции UpdateData, инициализирующей переменные

в объектах классов элементов управления и для передачи значений этих переменных обратно в класс диалогового окна.

При создании пользовательского класса, производного от класса `CDialog`, чтобы иметь возможность воспользоваться методами автоматического обмена данными между объектами классов диалогового окна и элементов управления, а также автоматической проверки передаваемых при этом данных, следует перегрузить функцию `DoDataExchange` в своем диалоговом классе. Эту работу выполняет за пользователя мастер ClassWizard, включающий в данную функцию карту обмена данными диалогового окна и вызовы глобальных функций проверки допустимости значений передаваемых данных.

Чтобы при помощи мастера ClassWizard перегрузить данную функцию в пользовательском классе диалогового окна, создайте ресурс диалога с использованием редактора диалога, а затем вызовите мастер ClassWizard и создайте соответствующий ему класс диалогового окна. Включите в него переменные для хранения информации, полученной от объектов классов элементов управления, и задайте диапазоны изменения этих величин. Мастер ClassWizard создаст перегруженную версию функции `DoDataExchange`, содержащую карту данных. Как утверждает библиотека MSDN, приведенный ниже фрагмент программы сгенерирован мастером ClassWizard:

```
void CPenWidthsDlg::DoDataExchange(CDataExchange* pDX)
{
   CDialog::DoDataExchange(pDX);
   //{{AFX_DATA_MAP(CPenWidthsDlg)

      DDX_Text(pDX, IDC_THIN_PEN_WIDTH, m_nThinWidth);

      DDV_MinMaxInt(pDX, m_nThinWidth, 1, 20);

      DDX_Text(pDX, IDC_THICK_PEN_WIDTH, m_nThickWidth);

      DDV_MinMaxInt(pDX, m_nThickWidth, 1, 20);
   //}}AFX_DATA_MAP
}
```

Однако в данной библиотеке отсутствуют какие-либо сведения о том, каким образом можно заставить мастер ClassWizard включить в этот текст вызов функции `DDV_MinMaxInt`.

Как видно из приведенного фрагмента программы, мастер ClassWizard включает в код программы специальные программные скобки `\\{{` и `\\}}`. Они не должны уничтожаться пользователем.

Описание перегруженной функции `DoDataExchange` должно предшествовать описаниям макросов в файле реализации.

CWnd::EnableWindow

```
BOOL EnableWindow(BOOL bEnable = TRUE);
```

Возвращаемое значение

Указывает состояние окна до того, как была вызвана функция EnableWindow. Функция возвращает ненулевое значение, если окно до этого было недоступным. Нулевое значение указывает, что окно до этого было доступным или на то, что произошла ошибка.

Аргументы

bEnable — определяет, нужно ли сделать данное окно доступным (или недоступным). Если этот аргумент имеет значение TRUE, то окно становится доступным.

Описание

Делает окно доступным или недоступным для работы с мышью и ввода с клавиатуры. Если данное окно недоступно, то обращенные к нему щелчки мыши и ввод с клавиатуры игнорируются. Если окно доступно, оно выполняет обработку сообщений в полном объеме.

При изменении доступности окна данная функция посылает сообщение WM_ENABLE. Для того чтобы окно могло быть активизировано, оно должно быть доступным. Например, если приложение выводит немодальное диалоговое окно и сделало недоступным свое главное окно, то прежде чем уничтожить объект диалогового окна, следует сделать доступным главное окно. В противном случае фокус ввода получит другое окно, что приведет к его активизации. Недоступное дочернее окно игнорируется при определении операционной системой Windows окна, которому предназначены сообщения мыши.

По умолчанию создаваемое окно доступно. Приложение может установить стиль WS_DISABLED в функциях создания изначально недоступного окна Create или CreateEx . После того как окно будет создано, приложение может вызывать функцию EnableWindow.

С помощью этой функции можно сделать доступным или недоступным элемент управления в диалоговом окне. Недоступный элемент управления не может получить фокус ввода, и пользователь не может его использовать.

CWnd::EndPaint

```
void EndPaint(LPPAINTSTRUCT lpPaint);
```

Аргументы

lpPaint — указатель на объект структуры PAINTSTRUCT, содержащий информацию, необходимую для вывода на экран и полученную в функции BeginPaint.

Описание

Отмечает завершение процесса рисования в текущем окне. Функция EndPaint должна вызываться для каждого вызова функции BeginPaint по окончании инициированного данной функцией процесса рисования.

Если функция BeginPaint скрыла текстовый курсор, то функция EndPaint выведет его на экран.

CWnd::GetClientRect

```
void GetClientRect(LPRECT lpRect) const;
```

Аргументы

lpRect — указатель на объект структуры RECT или объект класса CRect, в который будут записаны координаты рабочей области окна. Переменные left и top в данном объекте структуры всегда равны нулю. В переменные right и bottom записывается ширина и высота рабочей области окна.

Описание

Копирует пользовательские координаты рабочей области окна, связанного с объектом класса CWnd, в структуру, на которую указывает аргумент lpRect. Пользовательские координаты определяют левый верхний и правый нижний углы рабочей области окна. Поскольку пользовательские координаты отсчитываются от верхнего левого угла рабочей области окна, связанного с объектом класса CWnd, координаты левого верхнего угла всегда равны (0,0).

CWnd::GetDC

```
CDC* GetDC();
```

Возвращаемое значение

В случае успешного завершения работы функции возвращается указатель на объект класса контекста устройства, используемого в рабочей области окна, связанного с объектом класса CWnd. В противном случае возвращается нулевое значение. Возвращаемый указатель может быть временным и не должен сохраняться для дальнейшего использования.

Описание

Возвращает указатель на объект класса обычного, принадлежащего классу, или частного контекста устройства, в зависимости от стиля класса, определенного при создании объекта класса CWnd. При каждом вызове обычных контекстов устройств функция GetDC использует атрибуты по умолчанию. Для контекстов устройств, принадлежащих классу, или частных контекстов устройств функция GetDC оставляет без изменения установленные ранее

атрибуты. Контекст устройства может использоваться в последовательности функций графического интерфейса GDI, осуществляющих вывод в рабочую область окна.

Если контекст устройства принадлежит классу окна, то для его освобождения следует вызвать функцию ReleaseDC. Поскольку в операционной системе одновременно может находиться до пяти объектов класса обычного контекста устройств, отсутствие операции освобождения в одном приложении может привести к тому, что другие приложения не смогут получить доступ к контексту устройств.

Функция GetDC возвращает контекст устройства, принадлежащий классу CWnd, в том случае, если при регистрации данного класса в структуре WNDCLASS были установлены флаги CS_CLASSDC, CS_OWNDC или CS_PARENTDC.

CWnd::GetDlgItem

```
CWnd* GetDlgItem(int nID) const;
void CWnd::GetDlgItem(int nID, HWND* phWnd) const;
```

Возвращаемое значение

Указатель на элемент управления или дочернее окно, определяемые аргументом данной функции. Если ни один элемент управления не имеет идентификатора, указанного в аргументе nID, то возвращается нулевое значение.

Аргументы

nID — идентификатор элемента управления или дочернего окна, указатель на который следует получить.

phWnd — указатель на дочернее окно.

Описание

Позволяет получить указатель на элемент управления или дочернее окно в диалоговом или в другом окне. Возвращаемый указатель обычно приводится к типу указателя на элемент управления, которому соответствует идентификатор, определенный в аргументе nID.

CWnd::GetParent

```
CWnd* GetParent() const;
```

Возвращаемое значение

Указатель на родительское окно — при удачном завершении работы функции, или NULL, что говорит об ошибке или о том, что данное окно не имеет родительского окна. Возвращаемый указатель может быть временным, и его следует сохранить в специальной переменной для дальнейшего использования.

Описание

Возвращает указатель на объект класса родительского окна (если таковой имеется) из дочернего окна. Данная функция позволяет получить указатель на объект класса непосредственного родителя. С другой стороны, функция `GetParentOwner` возвращает указатель на первое окно в иерархии родителей, которое само не является дочерним (не имеет стиля `WS_CHILD`).

CWnd::GetParentFrame

```
CFrameWnd* GetParentFrame() const;
```

Возвращаемое значение

Указатель на главное окно приложения, если функция успешно завершает свою работу, и `NULL` в противном случае.

Описание

Возвращает указатель на родительское окно. Функция `GetParentFrame` выполняет поиск вверх по цепи родителей, пока не находит объект класса `CFrameWnd` (или производного от него класса).

CWnd::GetParentOwner

```
CWnd* GetParentOwner() const;
```

Возвращаемое значение

Указатель на объект класса `CWnd`. Если объект класса `CWnd` не имеет дескриптора, создается временный объект класса `CWnd` и ему сопоставляется дескриптор. Возвращаемый данной функцией указатель может быть временным, и его не следует сохранять для дальнейшего использования.

Описание

Возвращает указатель на родительское окно данного дочернего окна или на владельца окна. Функция `GetParentOwner` позволяет получить указатель на ближайшее в иерархии родительское окно или на владельца окна, который сам не является дочерним окном (не имеет стиля `WS_CHILD`). Текущий владелец окна может быть установлен при вызове функции `SetOwner`. По умолчанию родительское окно любого окна одновременно является и его владельцем.

В отличие от данной функции, функция `GetParent` возвращает указатель на непосредственного родителя независимо от того, является ли он дочерним окном или нет. В том случае, если родительским окном для данного окна является дочернее окно другого окна, то функции `GetParent` и `GetParentOwner` возвращают указатели на различные объекты класса `CWnd`.

CWnd::GetSystemMenu

```
CMenu* GetSystemMenu(BOOL bRevert) const;
```

Возвращаемое значение

Если аргумент bRevert имеет значение FALSE, то возвращается дескриптор копии меню **Control**. В противном случае возвращаемое значение не определено. Возвращаемый указатель является временным, и не должен сохраняться.

Аргументы

bRevert — определяет выполняемую операцию. Если данный аргумент имеет значение FALSE, то функция GetSystemMenu возвращает дескриптор копии текущего меню **Control**. Эта копия изначально идентична меню **Control**, но в нее можно вносить изменения. Если данный аргумент имеет значение TRUE, то функция GetSystemMenu возвращает меню Control в исходное состояние. Предыдущее меню **Control**, в которое могли быть внесены изменения, уничтожается. Возвращаемое значение в данном случае не определено.

Описание

Позволяет приложению получить доступ к меню **Control** для его копирования и внесения в него изменений. Любое окно, не вызывающее функции GetSystemMenu для получения собственной копии меню **Control**, использует стандартное меню **Control**.

Указатель, возвращаемый функцией GetSystemMenu, может использоваться функциями CMenu::AppendMenu, CMenu::InsertMenu и CMenu::ModifyMenu для внесения изменений в меню **Control**. Меню **Control** изначально содержит команды, имеющие такие идентификаторы, как SC_CLOSE, SC_MOVE, и SC_SIZE. Его команды посылают сообщение WM_SYSCOMMAND, и значения их идентификаторов больше 0xF000. Если приложение добавляет в меню свои идентификаторы команд, то их значения также должны быть больше 0xF000.

Windows может автоматически делать недоступными команды стандартного меню **Control**. Объект класса CWnd выполняет соответствующие действия при обработке сообщения WM_INITMENU, посылаемого перед выводом меню на экран.

CWnd::GetStyle

```
DWORD GetStyle() const;
```

Возвращаемое значение

Стиль окна.

CWnd::GetWindowRect

```
void GetWindowRect(LPRECT lpRect) const;
```

Аргументы

lpRect — указатель на объект класса CRect или структуры RECT, в который будут записаны экранные координаты левого верхнего и правого нижнего углов.

Описание

Копирует координаты прямоугольника, описывающего окно, связанное с объектом класса CWnd, в объект, определяемый аргументом lpRect. Все размеры прямоугольника задаются в экранных координатах относительно левого верхнего угла экрана. Прямоугольник, координаты которого вычисляются в данной функции, включает в себя заголовок окна, его рамку и полосы прокрутки в том случае, если данное окно имеет эти элементы.

CWnd::Invalidate

```
void Invalidate(BOOL bErase = TRUE);
```

Аргументы

bErase — определяет необходимость уничтожения фона при перерисовке.

Описание

Вызывает перерисовку всей рабочей области окна, связанного с объектом класса CWnd. Рабочая область отмечается как нуждающаяся в перерисовке при поступлении следующего сообщения WM_PAINT. Перерисовка в рабочей области окна может быть отменена, если до прихода следующего сообщения WM_PAINT будут вызваны функции ValidateRect или ValidateRgn.

Аргумент bErase определяет необходимость уничтожения фона в процессе перерисовки рабочей области окна. Если данный аргумент принимает значение TRUE, то при вызове функции BeginPaint происходит уничтожение фона, в противном случае фон остается без изменений. Если аргумент bErase принимает значение TRUE для любой части перерисовываемой области, фон уничтожается во всей этой области.

Окно посылает сообщение WM_PAINT, если в окне имеются области, нуждающиеся в перерисовке, и в очереди сообщений приложения отсутствуют другие сообщения для данного окна.

CWnd::InvalidateRect

```
void InvalidateRect(LPCRECT lpRect, BOOL bErase = TRUE);
```

Аргументы

lpRect — указатель на объект структуры RECT или объект класса CRect, содержащий координаты прямоугольника (в системе координат рабочей области окна), который необходимо добавить к области обновления окна. Если аргумент lpRect равен нулю, то к данной области добавляется вся рабочая область окна.

bErase — определяет необходимость уничтожения фона при обновлении информации в окне.

Описание

Вызывает перерисовку рабочей области окна, ограниченной заданным прямоугольником. Указанная область, наряду с другими областями, образующими область обновления окна, перерисовывается при обработке следующего сообщения WM_PAINT. Перерисовка указанной области может быть отменена, если до прихода следующего сообщения WM_PAINT будут вызваны функции ValidateRect или ValidateRgn.

Аргумент bErase определяет необходимость уничтожения фона в процессе перерисовки рабочей области окна. Если данный аргумент принимает значение TRUE, то при вызове функции BeginPaint происходит уничтожение фона, в противном случае фон остается без изменений. Если аргумент bErase принимает значение TRUE для любой части перерисовываемой области, фон уничтожается во всей этой области.

Окно посылает сообщение WM_PAINT, если в окне имеются области, нуждающиеся в перерисовке, и в очереди сообщений приложения отсутствуют другие сообщения для данного окна.

CWnd::IsDlgButtonChecked

```
UINT IsDlgButtonChecked(int nIDButton) const;
```

Возвращаемое значение

Ненулевое, если указанный элемент управления установлен, и ноль — если сброшен. Аргументами данной функции могут быть только флажки и переключатели. Для кнопок, имеющих три состояния, возвращаемое значение 2 означает неопределенное состояние. Для обычных кнопок данная функция всегда возвращает ноль.

Аргументы

nIDButton — определяет целочисленный идентификатор кнопки.

Описание

Определяет состояние кнопки. Если кнопка имеет три состояния, данная функция определяет, установлена она, сброшена или находится в неопределенном состоянии.

CWnd::ModifyStyle

```
BOOL ModifyStyle(DWORD dwRemove, DWORD dwAdd, UINT nFlags = 0);
```

Возвращаемое значение

Ненулевое, если произошло изменение стиля окна, в противном случае — ноль.

Аргументы

`dwRemove` — определяет стили, которые необходимо удалить в процессе изменения стиля окна.

`dwAdd` — определяет стили, которые необходимо добавить в процессе изменения стиля окна.

`nFlags` — флаги, передаваемые функции `SetWindowPos`, или ноль, если нет необходимости вызывать функцию `SetWindowPos`. По умолчанию данный аргумент равен нулю. В данном аргументе могут быть установлены следующие флаги:

- `SWP_NOSIZE` — сохраняется прежний размер окна
- `SWP_NOMOVE` — сохраняется прежнее положение окна
- `SWP_NOZORDER` — сохраняется прежний Z порядок
- `SWP_NOACTIVATE` — окно не активизируется

Описание

Функция вызывается для внесения изменений в расширенные стили окна. Добавляемые и удаляемые стили объединяются оператором поразрядного ИЛИ. В расширенные стили окна изменения вносятся функцией `ModifyStyleEx`.

CWnd::ModifyStyleEx

```
BOOL ModifyStyleEx(DWORD dwRemove, DWORD dwAdd, UINT nFlags = 0);
```

Возвращаемое значение

Ненулевое, если произошло изменение стиля окна, в противном случае — ноль.

Аргументы

dwRemove — определяет расширенные стили, которые необходимо удалить в процессе изменения стиля окна.

dwAdd — определяет расширенные стили, которые необходимо добавить в процессе изменения стиля окна.

nFlags — флаги, передаваемые функции SetWindowPos, или ноль, если нет необходимости вызывать функцию SetWindowPos. По умолчанию данный аргумент равен нулю. В данном аргументе могут быть установлены следующие флаги:

- SWP_NOSIZE — сохраняется прежний размер окна
- SWP_NOMOVE — сохраняется прежнее положение окна
- SWP_NOZORDER — сохраняется прежний Z порядок
- SWP_NOACTIVATE — окно не активизируется

Описание

Функция вызывается для внесения изменений в расширенные стили окна. Добавляемые и удаляемые стили объединяются оператором поразрядного ИЛИ. В обычные стили окна изменения вносятся функцией ModifyStyle.

CWnd::OnCommandHelp

```
afx_msg LRESULT OnCommandHelp(WPARAM wParam, LPARAM lParam)
```

Возвращаемое значение

Ненулевое, если данная функция вызвала функцию CWinApp::WinHelp, и ноль — в противном случае.

Аргументы

wParam — не используется и должен быть равен нулю.

lParam — содержит текущий контекст справки. Если данный аргумент равен нулю, то никакой контекст справки не определен.

Описание

Функция OnCommandHelp может использовать значение, передаваемое в аргументе lParam для определения "лучшего" контекста или для передачи его функции CWinApp::WinHelp. Если функция :WinHelp возвращает значение TRUE, то обработка данной команды в других (базовых) классах и в других окнах прекращается.

CWnd::OnContextMenu

```
afx_msg void OnContextMenu(CWnd* pWnd, CPoint pos);
```

Аргументы

pWnd — дескриптор окна, в котором пользователь щелкнул правой кнопкой мыши. Это может быть дочернее окно по отношению к окну, обрабатывающему данное сообщение.

pos — позиция указателя мыши в момент щелчка правой кнопки, заданная в экранных координатах.

Описание

Вызывается приложением при щелчке правой кнопки мыши в окне. Для обработки данного сообщения и вывода контекстного меню может быть вызвана функция TrackPopupMenu.

Если пользователь не вывел контекстное меню, он должен передать сообщение функции DefWindowProc. Если щелчок правой кнопкой мыши был произведен в дочернем окне, то функция DefWindowProc посылает сообщение родительскому окну. В противном случае, если щелчок правой кнопкой мыши был выполнен в области заголовка окна, функция DefWindowProc выводит контекстное меню, определенное по умолчанию.

CWnd::OnHelpHitTest

```
afx_msg LRESULT OnHelpHitTest(WPARAM wParam, LPARAM lParam)
```

Возвращаемое значение

Контекст справки, если с данным окном связан контекст справки, и ноль — в противном случае.

Аргументы

wParam — не используется и должно равняться нулю.

lParam — младшее слово данного аргумента содержит горизонтальную координату, измеренную в координатах устройства, позиции, в которой произошел щелчок мыши в рабочей области окна, а старшее слово содержит ее вертикальную координату.

Описание

Позволяет определить контекст справки, связанный с данным окном. Если данная функция возвращает нулевое значение, то контекст справки запрашивается в родительском окне.

CWnd::OnHelpInfo

```
afx_msg BOOL OnHelpInfo(HELPINFO* lpHelpInfo);
```

Аргументы

lpHelpInfo — указатель на объект структуры HELPINFO, содержащий информацию о команде меню, элементе управления, диалоговом окне или окне.

Описание

Вызывается приложением, пользователь нажал клавишу <F1>. Если в момент нажатия клавиши <F1> фокус ввода принадлежит меню, то сообщение WM_HELP посылается окну, связанному с этим меню. В противном случае сообщение WM_HELP посылается окну, имеющему фокус ввода. Если ни одно окно не имеет фокуса ввода, сообщение WM_HELP посылается активному окну.

CWnd::OnHScroll

```
afx_msg void OnHScroll(UINT nSBCode, UINT nPos, CScrollBar* pScrollBar);
```

Аргументы

nSBCode — определяет код полосы прокрутки, содержащий запрос пользователя на прокрутку окна. Этот аргумент может принимать одно из следующих значений:

- SB_LEFT — перемещение в крайнюю левую позицию
- SB_ENDSCROLL — завершение операции прокрутки
- SB_LINELEFT — перемещение на предыдущую строку
- SB_LINERIGHT — перемещение на следующую строку
- SB_PAGELEFT — перемещение на одну страницу влево
- SB_PAGERIGHT — перемещение на одну страницу вправо
- SB_RIGHT — перемещение в крайнюю правую позицию
- SB_THUMBPOSITION — перемещение в позицию, определяемую аргументом nPos
- SB_THUMBTRACK — перемещает бегунок в позицию, определенную аргументом nPos

nPos — определяет положение бегунка полосы прокрутки, если аргумент nSBCode принимает значение SB_THUMBPOSITION или SB_THUMBTRACK. В противном случае данный аргумент не используется.

Примечание

В зависимости от значения диапазона прокрутки данный аргумент, имеющий тип беззнакового целого, может принимать отрицательное значение. Это сле-

дует понимать так: в действительности он имеет тип целого числа со знаком, и только если он преобразован в этот формат, функция будет работать абсолютно надежно, но в силу патологической любви разработчиков Microsoft к беззнаковым целым числам этот аргумент объявлен как беззнаковое целое. Поэтому, если вы не уверены, что данная функция будет работать только с положительными границами диапазона, а лучше — во всех случаях, перед использованием данного аргумента корректно преобразуйте его к его родному типу целого со знаком. Самое интересное заключается в том, что для этого достаточно присвоить величину, имеющую тип беззнакового целого, величине, имеющей тип целого со знаком, что является нарушением всех правил преобразования типов данных.

`pScrollBar` — если обрабатываемое сообщение пришло от полосы прокрутки, этот аргумент содержит указатель на объект класса данного элемента управления. Если пользователь щелкнул кнопкой мыши в полосе прокрутки окна, этот параметр имеет значение `NULL`. Данный указатель может быть временным и не должен сохраняться для дальнейшего использования.

Описание

Вызывается приложением для обработки сообщений от полосы прокрутки. Аргументы функции `OnHScroll` соответствуют параметрам, передаваемым приложению в соответствующем сообщении. При вызове в данной функции метода базового класса ему будут переданы те же параметры, что и самой функции. Все изменения, внесенные в них пользователем, будут при этом игнорироваться.

Функция `OnHScroll` позволяет приложению обмениваться информацией с горизонтальной полосой прокрутки окна, например, для реализации обратной связи при перемещении бегунка полосы прокрутки. В этом случае первый аргумент данной функции принимает значение `SB_THUMBTRACK`.

При перемещении содержимого окна, включающего в себя данную полосу прокрутки, приложение должно установить ее бегунок в соответствующее положение. Для этого используется функция `SetScrollPos`.

CWnd::OnMouseMove

```
afx_msg void OnMouseMove(UINT nFlags, CPoint point);
```

Аргументы

`nFlags` — указывает на состояние различных виртуальных клавиш. Этот аргумент может содержать следующие флаги:

- `MK_CONTROL` — указывает, что в момент перемещения мыши была нажата клавиша <Ctrl>
- `MK_LBUTTON` — указывает, что в момент перемещения мыши была нажата ее левая кнопка

- MK_MBUTTON — указывает, что в момент перемещения мыши была нажата ее средняя кнопка
- MK_RBUTTON — указывает, что в момент перемещения мыши была нажата ее правая кнопка
- MK_SHIFT — указывает, что в момент перемещения мыши была нажата клавиша <Shift>

point — определяет горизонтальные и вертикальные координаты курсора. Эти координаты отсчитываются от левого верхнего угла окна.

Описание

Приложение вызывает данную функцию при перемещении курсора мыши. Если в данный момент мышь не захвачена каким-либо окном, то сообщение WM_MOUSEMOVE обрабатывается функцией-членом того объекта класса CWnd, на окне которого в настоящее время располагается курсор мыши. В противном случае данное сообщение обрабатывается в объекте окна, захватившего мышь.

Эта функция вызывается приложением, чтобы предоставить пользователю возможность обрабатывать сообщения Windows. Передаваемые ей аргументы содержат параметры, переданные в соответствующих сообщениях. Вызываемым методам базового класса будут переданы те же самые значения аргументов, что и данной функции. Все изменения, внесенные пользователем, будут игнорироваться.

CWnd::OnPaint

```
afx_msg void OnPaint();
```

Описание

Приложение вызывает данную функцию, когда Windows или функции самого приложения посылают запрос на перерисовку части окна приложения. Сообщение WM_PAINT посылается функциями UpdateWindow и RedrawWindow.

Объект класса окна может получать также внутренние сообщения о необходимости перерисовки в результате вызова функции RedrawWindow с установленным флагом RDW_INTERNALPAINT. В этом случае окну может не передаваться обновляемый участок окна. Для его определения приложение должно вызвать функцию GetUpdateRect. Если данная функция возвращает ноль, то приложение не должно вызывать функции BeginPaint и EndPaint.

Приложение должно само следить за необходимостью обновления рабочей области своего окна, используя внутренние структуры данных для хранения информации, необходимой для перерисовки окна после получения сообщения WM_PAINT, поскольку данное сообщение может быть вызвано как необходимостью перерисовки области окна после того, как она стала види-

мой, так и вызовом функции RedrawWindow с установленным флагом RDW_INTERNALPAINT.

Внутреннее сообщение WM_PAINT посылается Windows только один раз. После того как внутреннее сообщение WM_PAINT будет послано окну функцией UpdateWindow, не могут быть посланы никакие другие сообщения WM_PAINT, пока окно не будет перерисовано или пока не будет вызвана функция RedrawWindow с установленным флагом RDW_INTERNALPAINT.

Отображение документов рассмотрено при описании функции CView::OnDraw.

CWnd::PreCreateWindow

```
virtual BOOL PreCreateWindow(CREATESTRUCT& cs);
```

Возвращаемое значение

Ненулевое, если процесс создания окна должен быть продолжен, или ноль, если в процессе создания окна возникла ошибка.

Аргументы

cs — объект структуры CREATESTRUCT.

Описание

Данная функция вызывается приложением перед созданием окна Windows, связанного с данным объектом класса CWnd. Эта функция не должна вызываться непосредственно пользователем. По умолчанию проверяет, равняется ли нулю имя класса окна и, при положительном результате проверки, подставляет вместо него соответствующее имя по умолчанию. При перегрузке данной функции изменяются значения, присвоенные переменным-членам структуры CREATESTRUCT для изменения режима открытия окна и его параметров.

Каждый класс, производный от класса CWnd, вносит свои изменения в объект структуры CREATESTRUCT, которые нигде не документируются. Поэтому прежде чем вносить свои изменения в элементы данной структуры в перегруженной версии функции PreCreateWindow, программисту необходимо ознакомиться с исходным текстом данной функции базового класса и решить, какие изменения установок по умолчанию необходимы, и не будут ли они изменены функцией базового класса.

CWnd::RedrawWindow

```
BOOL RedrawWindow(LPCRECT lpRectUpdate = NULL, CRgn* prgnUpdate = NULL,
  UINT flags = RDW_INVALIDATE | RDW_UPDATENOW | RDW_ERASE);
```

Возвращаемое значение

Ненулевое, если перерисовка окна прошла успешно, и ноль — в противном случае.

Аргументы

lpRectUpdate — указатель на объект структуры RECT, содержащий координаты перерисовываемого прямоугольника. Этот аргумент игнорируется, если аргумент prgnUpdate содержит дескриптор региона.

prgnUpdate — определяет обновляемую область. Если первые два аргумента данной функции равны нулю, перерисовывается вся рабочая область окна.

flags — флаги, используемые для перерисовки окна. Данный аргумент может быть комбинацией флагов. Все флаги могут быть разделены на следующие группы:

Флаги, задающие параметры перерисовки окна:

- RDW_ERASE — посылает окну сообщение WM_ERASEBKGND при его перерисовке. Данный флаг требует одновременной установки флага RDW_INVALIDATE, в противном случае флаг RDW_ERASE не будет иметь силы
- RDW_FRAME — посылает каждой составляющей окна, не являющейся его рабочей областью и входящей в обновляемую область, сообщение WM_NCPAINT. Данный флаг требует одновременной установки флага RDW_INVALIDATE, в противном случае флаг RDW_FRAME не будет иметь силы
- RDW_INTERNALPAINT — посылает окну сообщение WM_PAINT независимо от того, входит ли данное окно в обновляемую область
- RDW_INVALIDATE — перерисовывает область, заданную в аргументе lpRectUpdate или в аргументе prgnUpdate (из которых только один может иметь отличное от нуля значение). Если оба этих аргумента равны нулю, то перерисовывается вся рабочая область окна

Флаги, запрещающие перерисовку окна:

- RDW_NOERASE — отменяет обработку сообщения WM_ERASEBKGND
- RDW_NOFRAME — отменяет обработку сообщения WM_NCPAINT. Данный флаг должен использоваться совместно с флагом RDW_VALIDATE и обычно работает совместно с флагом RDW_NOCHILDREN. Совместная установка флагов требует осмотрительности, поскольку может привести к тому, что часть окна не будет правильно перерисовываться
- RDW_NOINTERNALPAINT — запрещает посылку любых внутренних сообщений WM_PAINT. Этот флаг не действует на сообщения WM_PAINT, посылаемые перерисовываемыми областями
- RDW_VALIDATE — отменяет перерисовку областей, заданных в аргументах lpRectUpdate и prgnUpdate (из которых только один может иметь отличное от нуля значение). Если оба этих аргумента равны нулю, то отменяется перерисовка всей рабочей области окна. Этот флаг не влияет на внутренние сообщения WM_PAINT

Флаги, определяющие момент перерисовки окна. Функция RedrawWindow не перерисовывает окно до тех пор, пока не будет установлен один из следующих битов:

- RDW_ERASENOW — при необходимости посылает в указанное окно сообщения WM_NCPAINT и WM_ERASEBKGND (в соответствии со значениями флагов RDW_ALLCHILDREN и RDW_NOCHILDREN) перед выходом из функции. При этом задерживается сообщение WM_PAINT
- RDW_UPDATENOW — при необходимости посылает в указанное окно сообщения WM_NCPAINT, WM_ERASEBKGND и WM_PAINT (в соответствии со значениями флагов RDW_ALLCHILDREN и RDW_NOCHILDREN) перед выходом из функции

По умолчанию дочерние окна, имеющие стиль WS_CLIPCHILDREN, не обрабатываются функцией RedrawWindow. Однако окна, не имеющие данного стиля, перерисовываются или освобождаются от перерисовки до тех пор, пока не встретится окно, имеющее стиль WS_CLIPCHILDREN.

Следующие флаги определяют окна, обрабатываемые функцией RedrawWindow:

- RDW_ALLCHILDREN — перерисовываются все указанные окна
- RDW_NOCHILDREN — перерисовываются все указанные окна, кроме дочерних

Описание

Перерисовывает указанный прямоугольник или область рабочей области указанного окна. Окно, перерисовываемое функцией RedrawWindow, не получает сообщения WM_PAINT.

CWnd::ReleaseDC

```
int ReleaseDC(CDC* pDC);
```

Возвращаемое значение

Ненулевое, в случае успешного завершения функции, и ноль — в противном случае.

Аргументы

pDC — указатель на объект класса контекста устройства, который нужно освободить.

Описание

Освобождает контекст устройства, делая его доступным для других приложений. Результат выполнения функции ReleaseDC зависит от типа освобождаемого контекста устройства.

Приложение должно вызвать функцию ReleaseDC после каждого вызова функций GetWindowDC и GetDC.

CWnd::ScreenToClient

```
void ScreenToClient(LPPOINT lpPoint) const;
void ScreenToClient(LPRECT lpRect) const;
```

Аргументы

lpPoint — указатель на объект класса CPoint или объект структуры POINT, содержащий преобразуемые экранные координаты.

lpRect — указатель на объект класса CRect или объект структуры RECT, содержащий преобразуемые экранные координаты.

Описание

Преобразует экранные координаты указанной точки или прямоугольника в координаты рабочей области окна. Функция ScreenToClient замещает экранные координаты, содержащиеся в объектах lpPoint или lpRect, соответствующими координатами рабочей области окна. Новые координаты отсчитываются от верхнего левого угла рабочей области окна, связанного с данным объектом класса CWnd.

CWnd::SetDlgItemText

```
void SetDlgItemText(int nID, LPCTSTR lpszString);
```

Аргументы

nID — идентификатор элемента управления, для которого необходимо задать текст.

lpszString — указатель на объект класса CString или заканчивающуюся нулем строку символов, которую следует скопировать в элемент управления.

Описание

Устанавливает заголовок или текст в элементе управления, принадлежащем окну или диалоговому окну. Функция SetDlgItemText посылает сообщение WM_SETTEXT соответствующему элементу управления.

CWnd::SetScrollPos

```
int SetScrollPos(int nBar, int nPos, BOOL bRedraw = TRUE);
```

Возвращаемое значение

Предыдущее положение бегунка полосы прокрутки.

Аргументы

nBar — определяет полосу прокрутки, в которой необходимо изменить положение бегунка. Этот аргумент может принимать одно из следующих значений:

- SB_HORZ — необходимо переместить бегунок в горизонтальной полосе прокрутки окна
- SB_VERT — необходимо переместить бегунок в вертикальной полосе прокрутки окна

nPos — определяет новое положение бегунка полосы прокрутки. Оно должно находиться в пределах диапазона прокрутки.

bRedraw — определяет необходимость перерисовки окна для отображения внесенных в него изменений. Если этот аргумент принимает значение TRUE, то полоса прокрутки перерисовывается, если FALSE, то полоса прокрутки не перерисовывается.

Описание

Устанавливает бегунок полосы прокрутки в новое положение, если необходимо, перерисовывает при этом окно полосы прокрутки, чтобы отразить в нем внесенные изменения. Запись в аргумент bRedraw значения FALSE используется при последующей перерисовке полосы прокрутки другой функцией.

CWnd::SetWindowText

```
void SetWindowText(LPCTSTR lpszString);
```

Аргументы

lpszString — указатель на объект класса CString или заканчивающуюся нулем строку, содержащую новый заголовок окна или новый текст элемента управления.

Описание

Устанавливает новый заголовок окна. Если окно представляет собой элемент управления, текст помещается в данный элемент управления. Функция посылает сообщение WM_SETTEXT данному окну.

CWnd::ShowWindow

```
BOOL ShowWindow(int nCmdShow);
```

Возвращаемое значение

Ненулевое, если окно до этого отображалось, и ноль, если оно было до этого скрыто.

Аргументы

nCmdShow — определяет режим отображения окна, связанного с объектом класса CWnd. Он может принимать одно из следующих значений:

- SW_HIDE — скрывает текущее окно и активизирует другое
- SW_MINIMIZE — свертывает активное окно в пиктограмму и активизирует самое верхнее окно в иерархическом списке окон системы
- SW_RESTORE — активизирует и отображает окно. Если окно было до этого свернуто в пиктограмму или развернуто на весь экран, Windows восстанавливает его прежние размеры и положение
- SW_SHOW — активизирует окно и отображает его с текущими размерами и положением
- SW_SHOWMAXIMIZED — активизирует окно и отображает его развернутым на весь экран
- SW_SHOWMINIMIZED — активизирует окно и отображает его свернутым в пиктограмму
- SW_SHOWMINNOACTIVE — отображает окно свернутым в пиктограмму. При этом активное окно сохраняет свою активность
- SW_SHOWNA — отображает окно в его текущем состоянии. При этом активное окно сохраняет свою активность
- SW_SHOWNOACTIVATE — отображает окно в его последнем зафиксированном состоянии. При этом активное окно сохраняет свою активность
- SW_SHOWNORMAL — активизирует и отображает окно. Если окно было до этого свернуто в пиктограмму или развернуто на весь экран, Windows восстанавливает его прежние размеры и положение

Описание

Устанавливает режим отображения окна. С аргументом CWinApp::m_nCmdShow должна вызываться только один раз. При последующих вызовах данной функции в качестве аргумента необходимо использовать одно из перечисленных выше значений переменной.

CWnd::UpdateData

```
BOOL UpdateData(BOOL bSaveAndValidate = TRUE);
```

Возвращаемое значение

Ненулевое, если операция прошла успешно, и ноль — в противном случае. Если аргумент bSaveAndValidate имеет значение TRUE, то ненулевое возвращаемое значение означает успешный исход проверки передаваемых данных.

Аргументы

bSaveAndValidate — определяет, используется ли данная функция для инициализации объектов классов элементов управления диалогового окна (FALSE) или для получения информации, содержащейся в данных объектах (TRUE).

Описание

Данная функция вызывается для инициализации объектов классов элементов управления диалогового окна или для получения информации, содержащейся в этих объектах. Приложение автоматически вызывает функцию UpdateData с параметром bSaveAndValidate, имеющим значение FALSE, при создании модального диалогового окна функцией CDialog::OnInitDialog. Функция вызывается перед выводом диалогового окна на экран.

Функция CDialog::OnOk вызывает данную функцию с параметром bSaveAndValidate, имеющим значение TRUE, для сохранения данных, хранящихся в объектах классов элементов управления, в переменных класса диалогового окна и, в случае успешного завершения данной операции, закрытия диалогового окна. (Если в диалоговом окне нажимается кнопка **Cancel**, то при закрытии диалогового окна функция UpdateData не вызывается.)

CWnd::UpdateWindow

```
void UpdateWindow();
```

Описание

Обновляет рабочую область окна, посылая сообщение WM_PAINT, если область обновления не пуста. Функция UpdateWindow посылает данное сообщение непосредственно, минуя очередь сообщений приложения.

Структура *CHARFORMAT*

```
typedef struct _charformat {
    UINT      cbSize;
    _WPAD     _wPad1;
    DWORD     dwMask;
    DWORD     dwEffects;
    LONG      yHeight;
    LONG      yOffset;
    COLORREF  crTextColor;
    BYTE      bCharSet;
    BYTE      bPitchAndFamily;
    TCHAR     szFaceName[LF_FACESIZE];
    _WPAD     _wPad2;
} CHARFORMAT;
```

Служит для хранения информации о форматировании символов в элементе управления "расширенное текстовое поле".

`cbSize` — размер данной структуры в байтах. Должен быть определен, до того как объект данной структуры будет использован в элементе управления "расширенное текстовое поле".

`dwMask` — определяет, какие переменные данной структуры содержат достоверную информацию или должны быть установлены. Эта переменная может быть комбинацией следующих значений:

- `CFM_BOLD` — используется значение флага `CFE_BOLD` в аргументе `dwEffects`
- `CFM_CHARSET` — переменная `bCharSet` содержит достоверную информацию
- `CFM_COLOR` — используется значение флага `CFE_AUTOCOLOR` в аргументе `dwEffects`, и переменная `crTextColor` содержит достоверную информацию
- `CFM_FACE` — переменная `szFaceName` содержит достоверную информацию
- `CFM_ITALIC` — используется значение флага `CFE_ITALIC` в аргументе `dwEffects`
- `CFM_OFFSET` — переменная `yOffset` содержит достоверную информацию
- `CFM_PROTECTED` — используется значение флага `CFE_PROTECTED` в аргументе `dwEffects`
- `CFM_SIZE` — переменная `yHeight` содержит достоверную информацию
- `CFM_STRIKEOUT` — используется значение флага `CFE_STRIKEOUT` в аргументе `dwEffects`
- `CFM_UNDERLINE` — используется значение флага `CFE_UNDERLINE` в аргументе `dwEffects`

`dwEffect` — содержит список операций форматирования. Может представлять собой комбинацию следующих значений:

- `CFE_AUTOCOLOR` — цвет текста принимает значение, возвращаемое функцией `GetSysColor (COLOR_WINDOWTEXT)`
- `CFE_BOLD` — текст отображается жирным шрифтом
- `CFE_ITALIC` — текст отображается курсивом
- `CFE_STRIKEOUT` — зачеркнутый текст
- `CFE_UNDERLINE` — подчеркнутый текст
- `CFE_PROTECTED` — устанавливается запрет на внесение изменений в параметры шрифта. При попытке внесения в него изменений посылается сообщение `EN_PROTECTED`

yHeight — определяет высоту символов в пиках. Пика равняется 1/1440 дюйма или 1/20 точки принтера.

yOffset — смещение символа относительно базовой линии, заданное в пиках. Если эта величина больше нуля, то символ является верхним индексом, а если меньше — нижним индексом.

crTextColor — определяет цвет текста. Эта величина игнорируется, если определена операция форматирования CFE_AUTOCOLOR.

bCharSet — определяет используемый набор символов. Этот аргумент может принимать те же значения, что и переменная lfCharSet в структуре LOGFONT.

bPitchAndFamily — определяет, будет шрифт иметь фиксированную или переменную ширину литер, а также семейство, к которому принадлежит данный шрифт. Этот аргумент может принимать те же значения, что и переменная lfPitchAndFamily в структуре LOGFONT.

szFaceName — текстовая строка, завершающаяся нулем, в которой хранится имя шрифта.

Объекты этой структуры используются в сообщениях EM_GETCHARFORMAT и EM_SETCHARFORMAT.

Структура *ENUMLOGFONT*

```
typedef struct tagENUMLOGFONT {
    LOGFONT elfLogFont;
    BCHAR   elfFullName[LF_FULLFACESIZE];
    BCHAR   elfStyle[LF_FACESIZE];
} ENUMLOGFONT;
```

Служит для определения атрибутов шрифта, полного имени шрифта и набора стилей шрифта.

elfLogFont — определяет объект структуры LOGFONT, в котором определены атрибуты шрифта.

elfFullName — определяет уникальное имя шрифта. Например, "ABCD Font Company TrueType Bold Italic Sans Serif".

elfStyle — определяет стиль шрифта. Например, "Bold Italic".

Структура *HDITEM*

```
typedef struct _HDITEM {
    UINT    mask;
    int     cxy;
    LPTSTR  pszText;
```

```
    HBITMAP hbm;
    int     cchTextMax;
    int     fmt;
    LPARAM  lParam;
#if (_WIN32_IE >= 0x0300)
    int     iImage;
    int     iOrder;
#endif
} HDITEM, FAR * LPHDITEM;
```

Содержит информацию о заголовке столбца. Используется вместо структуры HD_ITEM.

mask — набор флагов, указывающий, какие из переменных данной структуры следует заполнять при запросе или использовать их значения при изменении состояния элемента управления. Эта переменная представляет собой комбинацию следующих флагов:

- HDI_BITMAP — подставляется значение переменной hbm
- HDI_FORMAT — подставляется значение переменной fmt
- HDI_HEIGHT — следует использовать значение переменной cxy и рассматривать его как высоту заголовка столбца
- HDI_IMAGE — следует использовать значение переменной iImage и рассматривать его как значок, который необходимо вывести в заголовке столбца. Данный параметр используется начиная с версии 4.70
- HDI_LPARAM — подставляется значение переменной lParam
- HDI_ORDER — следует подставить значение переменной iOrder и рассматривать его как порядковый номер заголовка столбца. Данный параметр используется начиная с версии 4.70
- HDI_TEXT — подставляется значение переменных pszText и cchTextMax
- HDI_WIDTH — следует использовать значение переменной cxy и рассматривать его как ширину заголовка столбца

cxy — ширина или высота заголовка столбца.

pszText — указатель на строку заголовка столбца. Если эта переменная имеет значение LPSTR_TEXTCALLBACK, то элемент управления получает текст данного заголовка столбца, посылая сообщение HDN_GETDISPINFO.

hbm — дескриптор битового образа заголовка столбца.

cchTextMax — максимальная длина строки заголовка столбца в символах.

fmt — устанавливает битовые флаги, определяющие формат заголовка столбца. Эта переменная может принимать одно из следующих значений:

- HDF_CENTER — содержимое заголовка столбца центрируется

- HDF_LEFT — содержимое заголовка столбца выравнивается по левому краю
- HDF_RIGHT — содержимое заголовка столбца выравнивается по правому краю
- HDF_RTLREADING — содержимое заголовка столбца читается справа налево, как это принято в иврите и в арабском языке

Флаги из приведенного выше списка могут комбинироваться с одной из перечисленных ниже величин:

- HDF_BITMAP — в заголовке столбца выводится битовый образ
- HDF_BITMAP_ON_RIGHT — битовый образ появляется справа от текста. Это не относится к значку, хранящемуся в списке изображений, связанному с элементом управления заголовок. Данный параметр используется начиная с версии 4.70
- HDF_IMAGE — в заголовке столбца выводится значок из списка изображений. Данный параметр используется начиная с версии 4.70
- HDF_OWNERDRAW — формат заголовка столбца определяется пользователем
- HDF_STRING — в заголовке столбца выводится строка

Для выделения из переменной fmt флагов, касающихся выравнивания текста в окне, может быть использована маска HDF_JUSTIFYMASK.

lParam — величина, определяемая приложением.

iImage — индекс значка в списке изображений. Первое изображение в списке имеет нулевой индекс. Значок, определенный данной переменной, будет выводиться в заголовке столбца параллельно с любым изображением, определенным в переменной hbm. Если переменная iImage имеет значение I_IMAGECALLBACK, то элемент управления получает индекс значка данного заголовка столбца, посылая сообщение HDN_GETDISPINFO. Данный параметр используется начиная с версии 4.70.

iOrder — определяет порядок расположения заголовков столбцов в элементе управления. Заголовки столбцов выводятся слева направо, то есть значение этой величины для заголовка крайнего левого столбца равно 0. Для следующего заголовка эта величина равна 1 и т. д. Данный параметр используется начиная с версии 4.70.

В заголовке просмотрового окна списка могут одновременно располагаться текст, значок и битовый образ. Их взаимное расположение определяется атрибутом выравнивания текста. Если он имеет значение HDF_LEFT или HDF_CENTER, то слева отображается значок, затем текст, а справа располагается битовый образ. Если атрибут выравнивания текста имеет значение HDF_RIGHT, то слева отображается битовый образ, затем текст, а справа — значок.

Структура *HELPINFO*

```
typedef  struct  tagHELPINFO {
    UINT      cbSize;
    int       iContextType
    int       iCtrlId;
    HANDLE    hItemHandle;
    DWORD     dwContextId;
    POINT     MousePos;
} HELPINFO, FAR *LPHELPINFO;
```

Содержит информацию о теме для контекстной справки:

`cbSize` — размер структуры в байтах.

`iContextType` — определяет тип контекста, для которого запрашивается справка. Данный аргумент может принимать одно из следующих значений:

- `HELPINFO_MENUITEM` — справка о команде меню
- `HELPINFO_WINDOW` — справка по элементу управления или окну

`iCtrlId` — если аргумент `iContextType` имеет значение `HELPINFO_WINDOW`, то в данном аргументе содержится идентификатор окна или элемента управления. Если аргумент `iContextType` имеет значение `HELPINFO_MENUITEM`, то в данном аргументе содержится идентификатор команды меню

`hItemHandle` — если аргумент `iContextType` имеет значение `HELPINFO_WINDOW`, то в данном аргументе содержится идентификатор дочернего окна или элемента управления. Если аргумент `iContextType` имеет значение `HELPINFO_MENUITEM`, то в данном аргументе содержится идентификатор связанного меню.

`dwContextId` — контекстный идентификатор окна или элемента управления.

`MousePos` — объект структуры `POINT`, содержащий экранные координаты курсора мыши. Эта информация используется при выводе справки на основании текущей позиции курсора мыши.

Структура *LOGFONT*

```
typedef struct tagLOGFONT { // lf
   LONG lfHeight;
   LONG lfWidth;
   LONG lfEscapement;
   LONG lfOrientation;
   LONG lfWeight;
   BYTE lfItalic;
   BYTE lfUnderline;
```

```
    BYTE lfStrikeOut;
    BYTE lfCharSet;
    BYTE lfOutPrecision;
    BYTE lfClipPrecision;
    BYTE lfQuality;
    BYTE lfPitchAndFamily;
    TCHAR lfFaceName[LF_FACESIZE];
} LOGFONT;
```

Определяет атрибуты шрифта:

`lfHeight` — определяет высоту ячейки шрифта или символа в логических единицах. Высота символа представляет собой высоту ячейки символа минус межстрочный интервал. Программа масштабирования шрифтов воспринимает возможные значения данной переменной следующим образом:

- `> 0` — программа масштабирования шрифтов преобразует эту величину в единицы устройства и сопоставляет ее с высотой ячеек имеющихся шрифтов
- `0` — программа масштабирования шрифтов использует при поиске значение, установленное по умолчанию
- `< 0` — программа масштабирования шрифтов преобразует эту величину в единицы устройства и использует ее абсолютное значение для задания высоты символов

Программа масштабирования шрифтов ищет максимальный шрифт, размеры которого не превышают заданных величин. Преобразование выполняется при первом использовании шрифта. В режиме отображения `MM_TEXT` для определения высоты шрифта по его размеру в элементах изображения может быть использована следующая формула:

```
lfHeight = -MulDiv(PointSize, GetDeviceCaps(hDC, LOGPIXELSY), 72);
```

`lfWidth` — определяет среднюю ширину символов в логических единицах. Если эта переменная равна нулю, то коэффициент сжатия устройства сопоставляется с отношением ширины знака к его высоте для имеющихся шрифтов с целью нахождения ближайшего соответствия, определяемого по абсолютной величине разности.

`lfEscapement` — определяет угол наклона, выраженный в десятках градусов, между базовой линией строки текста и горизонтальной осью устройства. В Windows NT в графическом режиме `GM_ADVANCED` имеется возможность определять угол наклона базовой линии строки текста независимо от угла ориентации символов в строке. В Windows 95 и в графическом режиме `GM_COMPATIBLE` Windows NT переменная `lfEscapement` определяет одновременно угол наклона базовой линии строки текста и ориентацию символов. Поэтому переменные `lfEscapement` и `lfOrientation` должны иметь одно и то же значение.

lfOrientation — определяет угол наклона, выраженный в десятках градусов, между базовой линией каждого символа в строке текста и горизонтальной осью устройства.

lfWeight — определяет вес шрифта, значение которого находится в диапазоне от 0 до 1000. Например, значение 400 соответствует обычному шрифту, а значение 700 — полужирному шрифту. Если данная переменная равна нулю, то используется значение веса шрифта по умолчанию. В табл. В.7 приведены предопределенные значения веса шрифта.

Таблица В.7. *Предопределенные значения веса шрифта*

Идентификатор	Значение	Идентификатор	Значение
FW_DONTCARE	0	FW_SEMIBOLD	600
FW_THIN	100	FW_DEMIBOLD	600
FW_EXTRALIGHT	200	FW_BOLD	700
FW_ULTRALIGHT	200	FW_EXTRABOLD	800
FW_LIGHT	300	FW_ULTRABOLD	800
FW_NORMAL	400	FW_HEAVY	900
FW_REGULAR	400	FW_BLACK	1000
FW_MEDIUM	500		

lfItalic — если эта величина имеет значение TRUE, то текст выводится курсивом.

lfUnderline — если эта величина имеет значение TRUE, то текст выводится с подчеркиванием.

lfStrikeOut — если эта величина имеет значение TRUE, то текст выводится с зачеркиванием.

lfCharSet — определяет используемый набор символов. Может иметь следующие значения: ANSI_CHARSET, BALTIC_CHARSET, CHINESEBIG5_CHARSET, DEFAULT_CHARSET, EASTEUROPE_CHARSET, GB2312_CHARSET, GREEK_CHARSET, HANGUL_CHARSET, MAC_CHARSET, OEM_CHARSET, RUSSIAN_CHARSET, SHIFTJIS_CHARSET, SYMBOL_CHARSET и TURKISH_CHARSET. В корейской версии Windows определено значение JOHAB_CHARSET. В ближневосточной версии Windows определены значения HEBREW_CHARSET и ARABIC_CHARSET. В таиландской версии Windows определено значение THAI_CHARSET.

Набор символов OEM_CHARSET зависит от используемой версии Windows. Набор символов DEFAULT_CHARSET при задании имени и размера шрифта полностью определяет логический шрифт. Если задаваемое имя шрифта

не существует, то может использоваться шрифт с любым набором символов. Поэтому необходимо осторожно подходить к использованию набора символов DEFAULT_CHARSET.

В операционной системе могут существовать шрифты с другими наборами символов. Если приложение использует шрифт с неизвестным набором символов, оно не пытается преобразовать или интерпретировать строку, выведенную с использованием данного шрифта.

Эта переменная используется при масштабировании символов. Для получения надежных результатов необходимо использовать стандартные наборы символов. При определении начертания шрифта в переменной lfFaceName убедитесь, что значение переменной lfCharSet соответствует набору символов, определенному для начертания, определяемого переменной lfFaceName.

lfOutPrecision — определяет точность представления символов. Точность представления символов отражает, насколько точно высота, ширина, ориентация символа, тип шрифта, наклон и наклон базовой линии выводимого символа должны соответствовать заданным. Эта переменная может принимать одно из следующих значений:

- OUT_CHARACTER_PRECIS — не используется
- OUT_DEFAULT_PRECIS — задает параметры масштабирования по умолчанию
- OUT_DEVICE_PRECIS — сообщает процедуре масштабирования шрифтов о необходимости использования шрифта устройства, если в системе определены несколько шрифтов с одним именем
- OUT_OUTLINE_PRECIS — сообщает процедуре масштабирования шрифтов о необходимости использования шрифтов TrueType и им подобных. Это значение используется только в Windows NT
- OUT_RASTER_PRECIS — сообщает процедуре масштабирования о необходимости использования растровых шрифтов, если в системе определены несколько шрифтов с одним именем
- OUT_STRING_PRECIS — не используется процедурой масштабирования шрифтов, но эта величина возвращается при нумерации растровых шрифтов
- OUT_STROKE_PRECIS — в Windows NT данная величина не используется процедурой масштабирования шрифтов, но она возвращается при нумерации шрифтов TrueType и им подобных, а также векторных шрифтов. В Windows 95 данная величина используется процедурой масштабирования шрифтов и возвращается при нумерации шрифтов TrueType и векторных шрифтов
- OUT_TT_ONLY_PRECIS — сообщает процедуре масштабирования шрифтов о необходимости использования только шрифтов TrueType. Если в

системе не определены шрифты TrueType, процедура масштабирования шрифтов работает с параметрами по умолчанию

- OUT_TT_PRECIS — сообщает процедуре масштабирования шрифтов о необходимости использования шрифтов TrueType, если в системе определены несколько шрифтов с одним именем

Приложение использует значения OUT_DEVICE_PRECIS, OUT_RASTER_PRECIS и OUT_TT_PRECIS для задания режима, в котором процедура масштабирования шрифтов выбирает шрифт при наличии в операционной системе нескольких шрифтов с одним и тем же именем. Например, если в операционной системе определены растровый и TrueType шрифты с именем "Symbol", присваивание данной переменной значения OUT_TT_PRECIS заставляет процедуру масштабирования шрифтов выбрать версию TrueType. Присваивание значения OUT_TT_ONLY_PRECIS заставляет процедуру масштабирования шрифтов выбрать версию TrueType, даже если ей придется для этого выбрать шрифт TrueType с другим именем.

lfClipPrecision — определяет точность процедуры отсечки. Под точностью процедуры отсечки определяет отображение символов, частично выходящих за область отсечки. Эта переменная может принимать одно из следующих значений:

- CLIP_DEFAULT_PRECIS — требует использовать процедуру отсечки по умолчанию
- CLIP_CHARACTER_PRECIS — не используется
- CLIP_STROKE_PRECIS — не используется процедурой масштабирования шрифтов, но возвращается при нумерации растровых, векторных или TrueType шрифтов. В Windows NT для совместимости эта величина всегда возвращается при нумерации шрифтов
- CLIP_MASK — не используется
- CLIP_EMBEDDED — этот флаг используется при работе с внедренными шрифтами, имеющими атрибут "только для чтения"
- CLIP_LH_ANGLES — определяет вращение всех шрифтов по часовой или против часовой стрелки в зависимости от направления осей системы координат. Если данная переменная не используется, то все шрифты устройств вращаются против часовой стрелки, но вращение остальных шрифтов зависит от направления осей системы координат. Дополнительную информацию о направлении осей системы координат можно получить из описания переменной nOrientation
- CLIP_TT_ALWAYS — не используется

lfQuality — определяет качество выводимых символов, т. е., с какой точностью интерфейс графических устройств (GDI) должен добиваться соответствия параметров логического шрифта параметрам используемого фи-

зического шрифта. Эта переменная может принимать одно из следующих значений:

- DEFAULT_QUALITY — качество выводимого шрифта не имеет значения
- DRAFT_QUALITY — качество шрифта может быть хуже, чем при использовании значения PROOF_QUALITY. При использовании растровых шрифтов возможно применение масштабирования, что расширяет диапазон размеров шрифтов за счет снижения их качества. При необходимости синтезируются жирные, курсивные, подчеркнутые и зачеркнутые шрифты
- PROOF_QUALITY — при выводе символов шрифта требуется обеспечить совпадение дополнительных параметров, кроме указанных в атрибутах логического шрифта. Для растровых шрифтов нельзя использовать операцию масштабирования и необходимо выбрать ближайший по размерам шрифт. Хотя при выборе размера шрифта может быть использован шрифт другого размера, значение PROOF_QUALITY обеспечивает более высокое качество отображения символов и отсутствие искажений при их выводе. При необходимости синтезируются жирные, курсивные, подчеркнутые и зачеркнутые шрифты

lfPitchAndFamily — определяет, будет ли шрифт иметь фиксированную или переменную ширину литер, а также семейство, к которому принадлежит данный шрифт. Два младших бита определяют, будет ли шрифт иметь фиксированную или переменную ширину литер, и могут принимать одно из следующих значений: DEFAULT_PITCH, FIXED_PITCH и VARIABLE_PITCH. Биты с 4 по 7 данной переменной определяют семейство, к которому принадлежит данный шрифт, и могут принимать одно из следующих значений: FF_DECORATIVE, FF_DONTCARE, FF_MODERN, FF_ROMAN, FF_SCRIPT и FF_SWISS.

Полное значение данной переменной получается как результат логического ИЛИ между константой, определяющей ширину литер, и константой, определяющей семейство.

Семейство шрифтов определяет их общее начертание. Оно используется для определения шрифтов при отсутствии более точного определения их начертания. Используемые в данном случае семейства шрифтов имеют следующие отличительные особенности:

- FF_DECORATIVE — нестандартный шрифт. Например "Old English"
- FF_DONTCARE — начертание шрифта не имеет значение или не известно
- FF_MODERN — шрифт с постоянной шириной штриха, имеющий или не имеющий засечек. К этим шрифтам относятся современные шрифты, такие как "Pica", "Elite" и "CourierNew"
- FF_ROMAN — шрифт с переменной шириной штриха (пропорциональный), имеющий засечки. Примером данного шрифта является "Serif"

- FF_SCRIPT — шрифт, подобный рукописному. Например, "Script"
- FF_SWISS — шрифт с переменной шириной штриха (пропорциональный), не имеющий засечек. Примером данного шрифта является "Sans Serif"

lfFaceName — текстовая строка, завершающаяся нулем, содержащая имя шрифта. Длина строки не должна превышать 32 символа, включая завершающий ее нулевой символ. При помощи функции EnumFontFamilies можно пронумеровать все имена доступных шрифтов. Если строка lfFaceName пуста, GDI использует первый шрифт, соответствующий остальным заданным атрибутам.

Структура *LVITEM*

```
typedef struct _LVITEM {
    UINT   mask;
    int    iItem;
    int    iSubItem;
    UINT   state;
    UINT   stateMask;
    LPTSTR  pszText;
    int    cchTextMax;
    int    iImage;
    LPARAM lParam;
#if (_WIN32_IE >= 0x0300)
    int iIndent;
#endif
} LVITEM, FAR *LPLVITEM;
```

Определяет атрибуты просмотрового окна списка или служит для их хранения. Эта структура была пересмотрена для обеспечения поддержки нового значения маски (LVIF_INDENT), позволяющего выделять абзацы. Эта структура заменяет структуру LV_ITEM:

mask — набор флагов, определяющих, какие члены данной структуры содержат данные, которые следует изменить или значения которых требуется определить. Данная переменная может быть комбинацией следующих значений:

- LVIF_TEXT — переменная pszText содержит полезную информацию или должна быть заполнена
- LVIF_IMAGE — переменная iImage содержит полезную информацию или должна быть заполнена
- LVIF_INDENT — переменная iIndent содержит полезную информацию или должна быть заполнена

- LVIF_NORECOMPUTE — элемент управления не должен посылать сообщение LVN_GETDISPINFO или принимать сообщение LVM_GETITEM. Вместо этого переменная pszText должна содержать значение LPSTR_TEXTCALLBACK
- LVIF_PARAM — переменная lParam содержит полезную информацию или должна быть заполнена
- LVIF_STATE — переменная state содержит полезную информацию или должна быть заполнена
- LVIF_DI_SETITEM — операционная система должна сохранить соответствующий раздел списка и больше его не запрашивать. Установка данного флага должна сопровождаться посылкой сообщения LVN_GETDISPINFO

iItem — индекс элемента данных, которому соответствует данная структура. Первый элемент данных имеет нулевой индекс.

iSubItem — индекс уровня элемента данных, которому соответствует данная структура, в иерархии.

state — определяет состояние элемента данных, его значок состояния и значок перекрытия. Переменная stateMask определяет значимые биты в данной переменной. Биты с 0 по 7 содержат флаги состояния элемента данных. Биты с 8 по 11 определяют индекс значка перекрытия. Минимальное значение данной величины равно единице. Значки перекрытия могут содержаться как в списке больших значков, так и в списке маленьких значков. Данный значок накладывается на основной значок элемента данных. Если все эти биты равны нулю, то элемент данных не имеет значка перекрытия. Для выделения этих битов используется маска LVIS_OVERLAYMASK. Биты с 12 по 15 определяют индекс значка состояния. Этот значок отображается рядом со значком элемента данных для индикации его состояния, определяемого приложением. Если все эти биты равны нулю, то элемент данных не имеет значка состояния. Для выделения этих битов следует установить для данного аргумента значение LVIS_STATEIMAGEMASK.

stateMask — определяет, какие биты переменной state необходимо прочитать или установить. Например, значение данного аргумента LVIS_SELECTED означает, что необходимо определить, выделен ли соответствующий элемент данных. Использование данной переменной позволяет вносить изменения в состояние различных элементов данных, не получая предварительно их текущего состояния. Например, при значении этого аргумента LVIS_SELECTED и нулевом значении аргумента state с элемента данных снимается выделение, но все остальные его установки останутся неизменными.

Примечание

В библиотеке MSDN довольно туманно сказано, что можно получить или внести изменения во все поля переменной state, передав в качестве данного аргумента значение (UINT) –1.

pszText — указатель на текстовую строку, кончающуюся нулем и содержащую текст элемента данных, если структура определяет атрибуты элемента данных. Если данный аргумент имеет значение LPSTR_TEXTCALLBACK, то соответствующий элемент данных является элементом данных обратного вызова. Переменная pszText не может принимать значения LPSTR_TEXTCALLBACK, если объект класса просмотрового окна списка имеет стиль LVS_SORTASCENDING или LVS_SORTDESCENDING. Если структура LVITEM используется для чтения атрибутов элемента данных, то в этой переменной содержится указатель на буфер, в который будет записан текст элемента данных.

cchTextMax — размер буфера, на который указывает переменная pszText, если эта структура используется для чтения атрибутов элемента данных. В противном случае значение переменной игнорируется.

iImage — индекс значка элемента данных в списке изображений элемента управления. Этот индекс относится как к списку больших значков, так и к списку маленьких значков. Если эта переменная имеет значение I_IMAGECALLBACK, то данный индекс хранится в объекте класса родительского окна. В этом случае объект класса просмотрового списка посылает родительскому окну сообщение LVN_GETDISPINFO для получения данного индекса при выводе элемента данных.

lParam — 32-разрядная величина, назначение которой определяется приложением. При использовании сообщения LVM_SORTITEMS эта величина передается в определенной в приложении функции сравнения элементов данных. При использовании сообщения LVM_FINDITEM эта величина может служить параметром поиска.

iIndent — отступ элемента данных, измеряемый в целых величинах его ширины. Единичное значение данной переменной перемещает элемент данных в следующий столбец. Этот параметр может использоваться только для элементов данных высшего уровня иерархии. В противном случае при вызове функции возникнет ошибка. Данный параметр применяется начиная с версии 4.70.

Структура LVITEM используется совместно с такими сообщениями, как: LVM_GETITEM, LVM_SETITEM, LVM_INSERTITEM и LVM_DELETEITEM.

Структура *NEWTEXTMETRIC*

```
typedef struct tagNEWTEXTMETRIC {
    LONG    tmHeight;
    LONG    tmAscent;
    LONG    tmDescent;
    LONG    tmInternalLeading;
    LONG    tmExternalLeading;
```

```
    LONG   tmAveCharWidth;
    LONG   tmMaxCharWidth;
    LONG   tmWeight;
    LONG   tmOverhang;
    LONG   tmDigitizedAspectX;
    LONG   tmDigitizedAspectY;
    BCHAR  tmFirstChar;
    BCHAR  tmLastChar;
    BCHAR  tmDefaultChar;
    BCHAR  tmBreakChar;
    BYTE   tmItalic;
    BYTE   tmUnderlined;
    BYTE   tmStruckOut;
    BYTE   tmPitchAndFamily;
    BYTE   tmCharSet;
    DWORD  ntmFlags;
    UINT   ntmSizeEM;
    UINT   ntmCellHeight;
    UINT   ntmAvgWidth;
} NEWTEXTMETRIC;
```

Содержит информацию о физическом шрифте.

`tmHeight` — определяет высоту (максимальный выступ над строкой + максимальный выступ под строкой) символов.

`tmAscent` — определяет максимальный выступ символов над строкой (в единицах над базовой линией).

`tmDescent` — определяет максимальный выступ символов под строкой (в единицах под базовой линией).

`tmInternalLeading` — определяет промежутки между границами, установленными в переменной `tmHeight`. В этом пространстве могут располагаться знаки ударения и другие диакритические символы. Разработчик может присвоить этой переменной нулевое значение.

`tmExternalLeading` — определяет дополнительные промежутки между строками, добавляемые приложением. Поскольку эта область не входит в шрифт, она не может содержать никаких знаков и не изменяется при выводе текста в режиме `OPAQUE` или `TRANSPARENT`. Разработчик может присвоить данной переменной нулевое значение.

`tmAveCharWidth` — определяет среднюю ширину символа в шрифте (обычно эта величина соответствует ширине символа "x"). Не учитывает увеличение ширины символа при его выводе полужирным или наклонным шрифтом.

`tmMaxCharWidth` — определяет ширину самого широкого символа в шрифте.

`tmWeight` — определяет вес шрифта.

`tmOverhang` — определяет дополнительную ширину строки для некоторых синтезированных шрифтов. При синтезе некоторых атрибутов шрифта, таких как полужирный или наклонный, графическому интерфейсу Windows (GDI) или устройству может понадобиться увеличить ширину строки и межстрочный интервал. Например, GDI может добиться получения полужирного начертания путем увеличения межсимвольного интервала и многократного вывода данного символа со смещением. Для получения наклонного шрифта может быть использован сдвиг строки. В любом случае последний символ строки выйдет за ее старые границы. При эмуляции полужирного шрифта выступ за границу строки будет определяться величиной смещения, а при эмуляции наклонного шрифта выступ будет определяться горизонтальным смещением верхнего правого угла ячейки символа над ее нижним правым углом.

Переменная `tmOverhang` позволяет приложению определить, насколько значение ширины отдельного символа, возвращаемое функцией `GetTextExtentPoint32`, соответствует истинной ширине символа, и как это отразится на ширине всей строки. Истинный размер символа может быть определен как его экстент минус величина перекрытия.

`tmDigitizedAspectX` — определяет горизонтальный масштаб устройства, для которого создан данный шрифт.

`tmDigitizedAspectY` — определяет вертикальный масштаб устройства, для которого создан данный шрифт. Отношение величин `tmDigitizedAspectX` и `tmDigitizedAspectY` определяет коэффициент сжатия устройства, для которого создан данный шрифт.

`tmFirstChar` — определяет значение первого символа данного шрифта.

`tmLastChar` — определяет значение последнего символа данного шрифта.

`tmDefaultChar` — определяет код символа, выводимого вместо символов, отсутствующих в данном наборе.

`tmBreakChar` — определяет код символа, используемого для разделения слов при выравнивании текста.

`tmItalic` — если переменная не равна нулю, то шрифт является наклонным.

`tmUnderlined` — если переменная не равна нулю, то устанавливается атрибут подчеркивания.

`tmStruckOut` — если переменная не равна нулю, то шрифт является зачеркнутым.

`tmPitchAndFamily` — определяет шаг и семейство выбранного шрифта. Значение младшего бита (бит 0) определяет шаг шрифта. Если этот бит равен 1, то шрифт имеет переменный шаг (или является пропорциональным шрифтом), а если 0, то шрифт имеет фиксированный шаг (или фик-

сированную ширину символа). Биты 1 и 2 данной величины определяют тип шрифта. Если оба этих бита сброшены в 0, шрифт является растровым. Если бит 1 имеет значение 1, а бит 2 — 0, то шрифт является векторным. Если бит 2 имеет значение 1, то тип данного шрифта не определен. Если бит 3 имеет значение 1, то шрифт является внутренним шрифтом устройства, в противном случае этот бит имеет значение 0.

Четыре старших бита данной переменной задают семейство шрифта. Переменная `tmPitchAndFamily` может быть объединена с помощью поразрядного И с константой 0xF0. После этого ее проверяют на равенство с именами семейств шрифтов. Дополнительная информация содержится в описании структуры `LOGFONT`.

`tmCharSet` — определяет набор символов шрифта.

`ntmFlags` — содержит флаги, указывающие, является ли данный шрифт наклонным, подчеркнутым, контурным и т. д. Ниже приведен список битов данной переменной, хранящих соответствующие флаги:

- 0 — наклонный
- 1 — подчеркнутый
- 2 — инвертированный
- 3 — контурный
- 4 — зачеркнутый
- 5 — полужирный

`ntmSizeEM` — определяет размер квадрата `em` для данного шрифта. Эта величина измеряется в "виртуальных единицах" (то есть, в тех единицах, для которых создавался этот шрифт).

`NtmCellHeight` — определяет высоту шрифта в виртуальных единицах. Это значение должно соответствовать значению переменной `ntmSizeEM`.

`ntmAvgWidth` — определяет среднюю ширину символа шрифта в виртуальных единицах. Должно соответствовать значению переменной `ntmSizeEM`.

Описание

Последние четыре переменные структуры `NEWTEXTMETRIC` отсутствуют в структуре `TEXTMETRIC`. Во всех остальных отношениях эти структуры идентичны.

Размеры, указанные в переменных структуры `NEWTEXTMETRIC`, как правило, измеряются в логических единицах. То есть они зависят от текущего режима отображения, установленного в контексте устройства.

Структура *NMHDR*

```
typedef struct tagNMHDR {
HWND hwndFrom;
UINT idFrom;
UINT code;
} NMHDR;
```

Используется в функциях обработки сообщений в формате WM_NOTIFY.

hwndFrom — дескриптор окна элемента управления, пославшего извещение. Дескриптор преобразуется в указатель на объект класса CWnd функцией CWnd::FromHandle.

idFrom — идентификатор элемента управления, пославшего извещение.

code — код извещения. Значение этой переменной может определяться типом элемента управления, например, TBN_BEGINADJUST или TTN_NEEDTEXT, или одним из стандартных извещений, перечисленных ниже:

- NM_CLICK — щелчок левой кнопкой мыши на элементе управления
- NM_DBLCLK — двойной щелчок левой кнопкой мыши на элементе управления
- NM_KILLFOCUS — элемент управления потерял фокус ввода
- NM_OUTOFMEMORY — элемент управления не может завершить операцию вследствие недостатка оперативной памяти
- NM_RCLICK — щелчок правой кнопкой мыши на элементе управления
- NM_RDBLCLK — двойной щелчок правой кнопкой мыши на элементе управления
- NM_RETURN — данному элементу управления принадлежит в настоящее время фокус ввода, и пользователь нажал клавишу <Enter>
- NM_SETFOCUS — элемент управления получил фокус ввода

Структура *OUTLINETEXTMETRIC*

```
typedef struct _OUTLINETEXTMETRIC {
    UINT    otmSize;
    TEXTMETRIC otmTextMetrics;
    BYTE    otmFiller;
    PANOSE  otmPanoseNumber;
    UINT    otmfsSelection;
    UINT    otmfsType;
    int     otmsCharSlopeRise;
    int     otmsCharSlopeRun;
    int     otmItalicAngle;
    UINT    otmEMSquare;
    int     otmAscent;
    int     otmDescent;
    UINT    otmLineGap;
    UINT    otmsCapEmHeight;
    UINT    otmsXHeight;
    RECT    otmrcFontBox;
```

```
    int     otmMacAscent;
    int     otmMacDescent;
    UINT    otmMacLineGap;
    UINT    otmusMinimumPPEM;
    POINT   otmptSubscriptSize;
    POINT   otmptSubscriptOffset;
    POINT   otmptSuperscriptSize;
    POINT   otmptSuperscriptOffset;
    UINT    otmsStrikeoutSize;
    int     otmsStrikeoutPosition;
    int     otmsUnderscoreSize;
    int     otmsUnderscorePosition;
    PSTR    otmpFamilyName;
    PSTR    otmpFaceName;
    PSTR    otmpStyleName;
    PSTR    otmpFullName;
} OUTLINETEXTMETRIC;
```

Содержит атрибуты шрифта TrueType.

`otmSize` — размер структуры `OUTLINETEXTMETRIC` в байтах.

`otmTextMetrics` — объект структуры `TEXTMETRIC`, содержащий дополнительную информацию о шрифте.

`otmFiller` — задает выравнивание структуры по границе байта.

`otmPanoseNumber` — определяет число `PANOSE` для данного шрифта.

`otmfsSelection` — содержит флаги шрифта. Разрядам данной переменной соответствуют следующие флаги:

- 0 — наклонный
- 1 — подчеркнутый
- 2 — инвертированный
- 3 — контурный
- 4 — зачеркнутый
- 5 — полужирный

`otmfsType` — определяет лицензионность шрифта. Лицензионный шрифт не должен изменяться или обмениваться. Если установлен бит 1, шрифт не может быть внедрен в документ. В противном случае шрифт может внедряться в документ. Если установлен бит 2, то разрешается внедрение данного шрифта в режиме "только для чтения".

`otmsCharSlopeRise` — определяет ориентацию текстового курсора. Если эта переменная равна 1, то текстовый курсор имеет вертикальную ориентацию. Приложение может использовать эту переменную и переменную `otmsCharSlopeRun` для создания текстовых курсоров курсивного начертания, имеющих тот же угол наклона, что и символы шрифта (угол определяется переменной `otmItalicAngle`).

otmItalicAngle — определяет угол наклона шрифта в десятых долях градуса против часовой стрелки от вертикального положения. Для регулярных шрифтов (roman) эта переменная равна нулю.

otmEMSquare — определяет горизонтальные и вертикальные размеры квадрата em данного шрифта в логических единицах.

otmAscent — определяет максимальное расстояние от базовой линии до верхней точки любого символа шрифта.

otmDescent — определяет максимальное расстояние от базовой линии до нижней точки любого символа шрифта.

otmLineGap — определяет типографский межстрочный интервал.

otmsCapEmHeight — не используется.

otmsXHeight — не используется.

otmrcFontBox — определяет прямоугольник, описывающий символы шрифта.

otmMacAscent — определяет максимальное расстояние от базовой линии до верхней точки любого символа шрифта для компьютеров Macintosh.

otmMacDescent — определяет максимальное расстояние от базовой линии до нижней точки любого символа шрифта для компьютеров Macintosh.

otmMacLineGap — определяет типографский межстрочный интервал для компьютеров Macintosh.

otmusMinimumPPEM — определяет минимальный рекомендуемый размер данного шрифта в элементах изображения на квадрат em.

otmptSubscriptSize — определяет рекомендуемые горизонтальный и вертикальный размеры подстрочных символов данного шрифта.

otmptSubscriptOffset — определяет рекомендуемое горизонтальное и вертикальное смещение подстрочных символов данного шрифта. Смещение подстрочных символов шрифта измеряется от начала координат символа до начала координат подстрочного символа.

otmptSuperscriptSize — определяет рекомендуемые горизонтальный и вертикальный размеры верхних индексов символов данного шрифта.

otmptSuperscriptOffset — определяет рекомендуемое горизонтальное и вертикальное смещение верхних индексов данного шрифта. Смещение верхнего индекса шрифта измеряется от начала координат символа до начала координат верхнего индекса.

otmsStrikeoutSize — определяет ширину символа зачеркивания для данного шрифта. Обычно эта величина равна ширине подчеркивания в шрифте.

otmsStrikeoutPosition — определяет положение символа зачеркивания относительно базовой линии шрифта. Положительные значения означают расположение выше базовой линии, а отрицательные — ниже.

otmsUnderscoreSize — определяет ширину символа подчеркивания для данного шрифта.

otmsUnderscorePosition — определяет положение символа зачеркивания данного шрифта.

otmpFamilyName — определяет смещение строки, содержащей имя семейства шрифта, от начала структуры.

otmpFaceName — определяет смещение строки, содержащей имя начертания шрифта, от начала структуры. (Это имя должно соответствовать имени, определенному в объекте структуры LOGFONT.)

otmpStyleName — определяет смещение строки, содержащей имя стиля шрифта, от начала структуры.

otmpFullName — определяет смещение строки, содержащей полное имя шрифта, от начала структуры. Это имя является уникальным для данного шрифта и часто содержит имя версии или другую идентификационную информацию.

Описание

Все размеры в структуре OUTLINETEXTMETRIC задаются в логических единицах, то есть они зависят от текущего режима масштабирования, заданного в контексте устройства.

Структура *PAINTSTRUCT*

```
typedef struct tagPAINTSTRUCT {
   HDC  hdc;
   BOOL fErase;
   RECT rcPaint;
   BOOL fRestore;
   BOOL fIncUpdate;
   BYTE rgbReserved[16];
} PAINTSTRUCT;
```

Содержит информацию для вывода графической информации в рабочую область окна.

hdc — дескриптор контекста устройства дисплея, используемый при выводе графической информации.

fErase — определяет необходимость перерисовки фона. Если эта переменная равна нулю, то приложение должно перерисовать фон. Приложение должно перерисовывать фон в том случае, если при создании класса окна Windows в нем не была задана кисть для заполнения фона.

См. описание переменной hbrBackground *в структуре* WNDCLASS *ниже в данном Приложении*

`rcPaint` — определяет левый верхний и правый нижний углы прямоугольника, в пределах которого должна выводиться графическая информация.

`fRestore` — переменная зарезервирована для внутреннего использования в операционной системе Windows.

`fIncUpdate` — зарезервирована для внутреннего использования в операционной системе Windows.

`rgbReserved[16]` — область памяти, зарезервированная для внутреннего использования в операционной системе Windows.

Структура *POINT*

```
typedef struct tagPOINT {
   LONG x;
   LONG y;
} POINT;
```

Определяет горизонтальные и вертикальные координаты точки:

`x` — горизонтальная координата точки.

`y` — вертикальная координата точки.

Структура *RECT*

```
typedef struct tagRECT {
   LONG left;
   LONG top;
   LONG right;
   LONG bottom;
} RECT;
```

Определяет координаты верхнего левого и нижнего правого углов прямоугольника.

`left` — определяет горизонтальную координату верхнего левого угла прямоугольника.

`top` — определяет вертикальную координату верхнего левого угла прямоугольника.

`right` — определяет горизонтальную координату нижнего правого угла прямоугольника.

`bottom` — определяет вертикальную координату нижнего правого угла прямоугольника.

Структура *SIZE*

```
typedef struct tagSIZE {
    int cx;
    int cy;
} SIZE;
```

Определяет ширину и высоту прямоугольника.

`cx` — определяет размер по горизонтали при выходе из функции.

`cy` — определяет размер по вертикали при выходе из функции.

Описание

Эта структура служит для хранения размеров прямоугольника, определяющего размеры рабочей области окна, размеры самого окна, размеры символов шрифта, отношения сторон и других подобных величин.

Структура *WNDCLASS*

```
typedef struct _WNDCLASS {
    UINT    style;
    WNDPROC lpfnWndProc;
    int     cbClsExtra;
    int     cbWndExtra;
    HANDLE  hInstance;
    HICON   hIcon;
    HCURSOR hCursor;
    HBRUSH  hbrBackground;
    LPCTSTR lpszMenuName;
    LPCTSTR lpszClassName;
} WNDCLASS;
```

Содержит атрибуты класса окна, регистрируемые функцией `RegisterClass`. Эта структура заменена структурой `WNDCLASSEX`, используемой функцией `RegisterClassEx`. Работа со структурой `WNDCLASS` и функцией `RegisterClass` возможна, если приложение не использует маленькие значки, связанные с классом окна.

`style` — определяет стиль (стили) класса. Стили могут объединяться поразрядным ИЛИ (|). Эта переменная может представлять собой комбинацию следующих стилей:

- `CS_BYTEALIGNCLIENT` — выравнивает рабочую область окна по границе байта (по горизонтали). Это влияет на ширину окна и его горизонтальное положение на экране

- CS_BYTEALIGNWINDOW — выравнивает окно по границе байта (по горизонтали). Это влияет на ширину окна и его горизонтальное положение на экране
- CS_CLASSDC — назначает контекст устройства, который будет разделяться всеми окнами данного класса. Поскольку классы окон зависят от процесса, существует возможность, что несколько потоков приложения создадут окна, соответствующие одному и тому же классу. Возможно также, что несколько потоков попытаются одновременно использовать контекст устройства. В этом случае операционная система позволит только одному из них успешно завершить вывод на экран
- CS_DBLCLKS — посылает процедуре окна соответствующее сообщение при двойном щелчке кнопкой мыши, если указатель мыши в данный момент располагается в пределах окна, соответствующего данному классу
- CS_GLOBALCLASS — позволяет приложению создавать окно класса независимо от значения аргумента hInstance, передаваемого функции CreateWindow или CreateWindowEx. Если этот стиль не указан, аргумент hInstance, передаваемый функции CreateWindow или CreateWindowEx, должен иметь то же значение, что и аргумент hInstance, передаваемый функции RegisterClass

Пользователь может создать глобальный класс, создав класс окна в библиотеке динамической компоновки (DLL) и указав ее имя в системном реестре с помощью следующих ключей:

```
HKEY_LOCAL_MACHINE\Software\Microsoft\Windows NT\CurrentVersion
  \Windows\AppInit_DLLs
```

При запуске процесса на исполнение операционная система загружает указанную библиотеку в контекст нового процесса перед вызовом ее первой исполняемой процедуры. Библиотека динамической компоновки должна зарегистрировать класс в ходе своей процедуры инициализации и указать при этом стиль CS_GLOBALCLASS.

- CS_HREDRAW — перерисовывает все окно, если при перемещении или изменении размеров окна изменилась ширина его рабочей области
- CS_NOCLOSE — делает недоступной команду **Закрыть** в меню окна
- CS_OWNDC — сопоставляет каждому окну данного класса уникальный контекст устройства
- CS_PARENTDC — устанавливает область отсечки дочернего окна таким образом, чтобы оно могло отображаться поверх родительского окна. Окна, в которых установлен флажок стиля CS_PARENTDC, получают контекст устройства системного кэша контекстов устройств. Этот стиль не передает дочернему окну контекст устройства родительского окна

или установки контекста устройства родительского окна. Использование стиля CS_PARENTDC позволяет повысить производительность приложения

- CS_SAVEBITS — позволяет сохранять в формате битового образа часть экрана, закрытую в настоящее время данным окном. Приложение использует сохраненный битовый образ для восстановления экрана после удаления данного окна. Приложение выводит битовый образ в его прежней позиции и не посылает сообщения WM_PAINT окнам, затененным данным окном, если область памяти, занимаемая битовым образом, не была освобождена, а в окнах, фрагменты которых входят в данный битовый образ, не были произведены изменения, затронувшие восстанавливаемую их часть. Этот стиль полезен при использовании окон небольшого размера (например, меню или диалоговые окна), отображаемых на короткий промежуток времени, и удаляемых прежде, чем приложение сможет обратиться к другим своим окнам. Применение этого стиля увеличивает время, необходимое для вывода окна, поскольку при этом необходимо выделить область памяти и сохранить в нем битовый образ
- CS_VREDRAW — перерисовывает все окно, если при перемещении или изменении размеров окна изменилась высота его рабочей области

lpfnWndProc — указатель на процедуру окна. Для вызова процедуры окна необходимо использовать функцию CallWindowProc.

cbClsExtra — определяет объем дополнительной памяти, размещаемой непосредственно за структурой класса. Объем памяти указывается в байтах. При инициализации в эту область памяти записываются нули.

cbWndExtra — определяет объем дополнительной памяти, размещаемой непосредственно за экземпляром окна. Объем памяти указывается в байтах. Если приложение использует объект структуры WNDCLASS для регистрации диалогового окна, созданного при помощи директивы CLASS в файле ресурса, эта переменная должна иметь значение DLGWINDOWEXTRA.

hInstance — дескриптор экземпляра, содержащийся в процедуре окна данного класса.

hIcon — дескриптор значка окна. Если эта переменная имеет нулевое значение, приложение должно рисовать значок в том случае, когда пользователь минимизирует окно приложения.

hCursor — дескриптор курсора класса. Если эта переменная имеет нулевое значение, приложение должно в явном виде установить форму указателя мыши, которую он принимает при перемещении по окну приложения.

hbrBackground — дескриптор кисти фона приложения. Эта переменная должна являться дескриптором физической кисти, используемой для за-

полнения фона, или может представлять собой значение цвета (к значению выбранного цвета должна быть прибавлена 1). Если данная переменная имеет значение цвета, пользователь должен преобразовать ее в один из следующих типов HBRUSH:

- COLOR_ACTIVEBORDER
- COLOR_ACTIVECAPTION
- COLOR_APPWORKSPACE
- COLOR_BACKGROUND
- COLOR_BTNFACE
- COLOR_BTNSHADOW
- COLOR_BTNTEXT
- COLOR_CAPTIONTEXT
- COLOR_GRAYTEXT
- COLOR_HIGHLIGHT
- COLOR_HIGHLIGHTTEXT
- COLOR_INACTIVEBORDER
- COLOR_INACTIVECAPTION
- COLOR_MENU
- COLOR_MENUTEXT
- COLOR_SCROLLBAR
- COLOR_WINDOW
- COLOR_WINDOWFRAME
- COLOR_WINDOWTEXT

Операционная система автоматически уничтожает фоновые кисти класса вместе с его объектами. Приложение не должно этого делать, поскольку класс может использоваться несколькими экземплярами данного приложения.

Если эта переменная имеет нулевое значение, приложение должно само заполнять фон своего окна при запросе на перерисовку его рабочей области. Приложение может определить, нужно ли перерисовывать фон, либо обработав сообщение WM_ERASEBKGND, либо проверив значение переменной fErase в объекте структуры PAINTSTRUCT, заполняемом функцией BeginPaint.

lpszMenuName — указатель на заканчивающуюся нулем текстовую строку, содержащую имя ресурса меню класса в том виде, в каком это имя содержится в файле ресурсов. Если эта переменная равна нулю, то окно, соответствующее данному классу, не имеет меню, используемого по умолчанию.

lpszClassName — указатель на заканчивающуюся нулем текстовую строку или атом. Если данная переменная является атомом, она должна представлять собой глобальный атом, созданный до этого функцией GlobalAddAtom. Атом представляет собой 16-разрядную величину, помещаемую в младшее слово переменной lpszClassName, старшее слово которой должно содержать нулевую величину. Если переменная lpszClassName является строкой, то она содержит имя оконного класса.

Средства для работы с классами Visual C++

Мастер ClassWizard позволяет автоматизировать многие рутинные процессы программирования в среде Visual C++, такие как создание новых классов, определение функций обработки сообщений, перегрузка виртуальных функций библиотеки MFC, а также получение информации из элементов управления диалоговых окон. Мастер ClassWizard может работать только с приложениями, использующими библиотеку MFC, в то время как панель **ClassView** окна **Workspace** и панель инструментов **WizardBar** позволяют работать не только с классами библиотеки MFC, но и с классами библиотеки ATL, а также с классами, определенными программистом без использования прототипов из этих библиотек. Кроме того, мастер ClassView не может работать с классами, не зарегистрированными в базе данных ClassWizard (файл с расширением .clw).

Мастер ClassWizard позволяет:

- Создавать новые классы, производные от классов библиотеки MFC
- Сопоставлять сообщениям, поступающим от окон, диалоговых окон, элементов управления, команд меню и от нажатия определенных комбинаций клавиш, предопределенные функции обработки этих сообщений
- Создавать новые функции обработки сообщений
- Уничтожать функции обработки сообщений
- Определять сообщения, которым уже сопоставлены функции их обработки и переходить в тело данных функций обработки сообщений
- Определять переменные-члены класса, автоматически их инициализировать, обновлять содержащуюся в них информацию и проверять корректность информации, поступающей из элементов управления

- Добавлять в создаваемый класс свойства, позволяющие использовать его в сервере автоматизации
- Работать с библиотеками классов

Работа с базой данных мастера ClassWizard

Обновление базы данных при изменении текста программы

В процессе разработки программы часто приходится удалять или вносить изменения в существующие классы, а также удалять или добавлять ресурсы, или же перемещать описание класса из одного исходного файла в другой. Мастер ClassWizard отслеживает изменения, внесенные пользователем в текст программы и запрашивает его о необходимой для внесения соответствующих изменений информации всякий раз, когда он вызывает в нем на редактирование соответствующий класс.

Мастер ClassWizard хранит информацию о классах проекта в своей базе данных, размещенной в файле с расширением .clw. Для поддержания соответствия между текстом программы и содержимым своей базы данных мастер ClassWizard выводит диалоговое окно **Repair Class Information** (Внесение изменений в информацию о классе) всякий раз, когда он обнаруживает, что информация в его базе данных устарела.

Диалоговое окно **Repair Class Information** имеет две основных функции:

- Удаление информации о несуществующих классах из базы данных мастера ClassWizard
- Внесение изменений в базу данных мастера ClassWizard, связанных с новым именем или с новым расположением существующих классов

Уничтожение классов

Чтобы уничтожить класс, созданный мастером ClassWizard, достаточно уничтожить его описание в файле заголовка (файл с расширением .h) и текст его функций в файле реализации (файл с расширением .cpp), где он описан вместе с другими классами, или уничтожить сами файлы заголовка и реализации. В любом из этих случаев необходимо внести изменения в базу данных мастера ClassWizard (файл с расширением .clw).

Чтобы уничтожить класс:

1. Уничтожьте все ссылки на данный класс в файлах заголовка и реализации, или уничтожьте сами файлы, содержащие его описание.
2. Вызовите команду **View**, **ClassWizard**. Если в результате этой команды открывается диалоговое окно **MFC ClassWizard**, то активный проект не содержит описания уничтоженного класса.

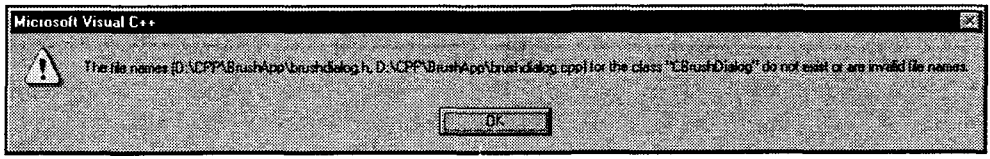


Рис. С.1. Диалоговое окно **Microsoft Visual C++**

3. Раскройте список **Project** и выберите проект, содержащий удаленный класс.
4. Если мастер ClassWizard просит закрыть любые файлы, закройте его, закройте все файлы, а затем снова запустите мастер ClassWizard. Появится диалоговое окно **Microsoft Visual C++**, изображенное на рис. С.1, сообщающее о том, что мастер ClassWizard не смог найти уничтоженный файл.
5. Нажмите кнопку **OK**. Появится окно **Repair Class Information**, изображенное на рис. С.2.
6. Нажмите кнопку **Remove**. Информация о классе будет удалена из базы данных мастера ClassWizard. Появится диалоговое окно **MFC ClassWizard**.
7. Нажмите кнопку **OK** и закройте это диалоговое окно.

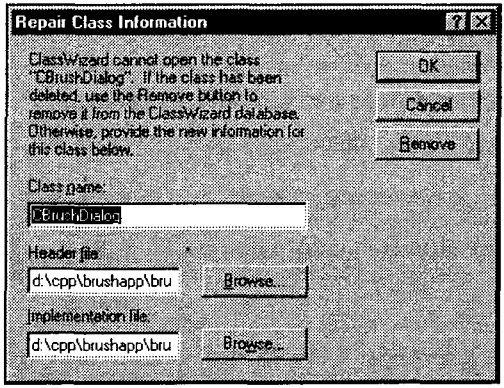


Рис. С.2. Диалоговое окно **Repair Class Information**

Переименование и перемещение классов

При переименовании класса или перемещении его описания из одного файла реализации в другой при следующем вызове мастера ClassWizard вам будет предложено внести соответствующие изменения в базу данных мастера ClassWizard.

Чтобы изменить имя файла или переместить его описание из одного файла в другой:

1. Произведите необходимые изменения в файлах проекта. Имя класса необходимо изменить во всех местах, где оно встречается в приложении.

2. Выберите команду **View**, **ClassWizard**. Если в результате этой команды открывается диалоговое окно **MFC ClassWizard**, то активный проект не содержит описания данного класса.
3. Раскройте список **Project:** и выберите проект, содержащий данный класс.
4. Если мастер ClassWizard просит закрыть какие-либо файлы, закройте его, закройте все файлы, а затем снова запустите мастер ClassWizard. Появится диалоговое окно с сообщением о том, что мастер не может найти старый класс, изображенное на рис. С.1.
5. Нажмите кнопку **OK**. Появится окно **Repair Class Information**, изображенное на рис. С.2. Внесите необходимые изменения в текстовые поля **Class name:**, **Header file:** и **Implementation file:**. В случае необходимости нажмите соответствующую кнопку **Browse...** и найдите файл в открывшемся диалоговом окне.
6. Нажмите кнопку **OK** и обновите информацию в базе данных.

Создание нового файла базы данных мастера ClassWizard

Если пользователь произвел многочисленные изменения в своем приложении или добавил в него большое количество библиотечных классов, реализующих интерфейс пользователя, может оказаться проще заново создать файл базы данных мастера ClassWizard (имеющего расширение .clw), чем вносить в нее изменения отдельно для каждого нового класса. Чтобы заново создать файл базы данных мастера ClassWizard, удалите файл с расширением .clw из проекта, и используйте мастер ClassWizard, чтобы создать его заново. Созданный заново файл базы данных мастера ClassWizard содержит информацию обо всех классах, которые имеют комментарии, соответствующие специальному формату мастера ClassWizard.

Чтобы создать заново файл базы данных мастера ClassWizard:

1. Удалите из вашего проекта файл с расширением .clw.
2. Выберите команду меню **View**, **ClassWizard**. Если в результате открывается диалоговое окно **MFC ClassWizard**, то проект, для которого удален файл с расширением .clw, в настоящее время не является активным. Если это так, раскройте список **Project:** и выберите проект, для которого нужно восстановить файл базы данных мастера ClassWizard.
3. Если мастер ClassWizard просит закрыть файлы, закройте его, закройте все файлы, а затем снова запустите мастер ClassWizard. Появится диалоговое окно **Microsoft Visual C++**, с сообщением о том, что соответствующая база данных отсутствует, и вопросом, хотите ли вы восстановить ее по вашим исходным файлам.
4. Нажмите кнопку **Да**. Появится диалоговое окно **Select Source Files**, изображенное на рис. С.3.

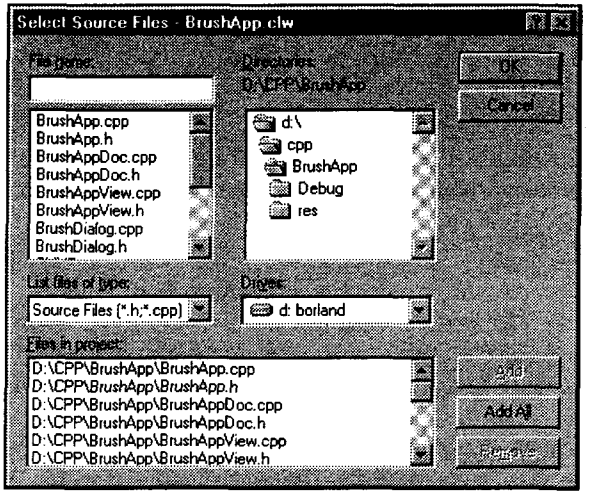


Рис. С.3. Диалоговое окно **Select Source Files**

5. Используйте кнопки **Add** и **Add All**, чтобы переместить все файлы вашего проекта, имеющие расширения .h, .cpp или .rc из списка **File name:** в список **Files in project:**.
6. Используйте кнопку **Remove**, чтобы удалить любой из файлов, имеющих одно из перечисленных выше расширений, из окна списка **Files in project:**.
7. Нажмите кнопку **OK**. Мастер ClassWizard создаст новый файл базы данных и откроет диалоговое окно **MFC ClassWizard**.
8. Нажмите кнопку **OK** и закройте это диалоговое окно.

Добавление нового класса

В Visual C++ возможны несколько путей для создания не содержащих ошибок заготовок пользовательских классов в приложениях. Это можно сделать с помощью панели **ClassView** окна **Workspace**, панели инструментов **WizardBar**, мастера ClassWizard, а также команды меню **Insert**, **New Class...**. Все эти средства используют один и тот же интерфейс (диалоговое окно **New Class**).

Если с помощью мастера ClassWizard можно создавать только классы, производные от классов, содержащихся в библиотеке MFC, то панель **ClassView** окна **Workspace**, панель инструментов **WizardBar** и команда меню **Insert**, **New Class...** позволяют создавать классы, имеющие прототипы в библиотеке MFC, в библиотеке ATL и пользовательские классы, не имеющие прототипов в данных библиотеках.

Тип добавляемого класса зависит от типа проекта, в который он добавляется. Например, по умолчанию в проектах, использующих библиотеку MFC, применяются классы, имеющие прототипы в данной библиотеке. Однако

подобные проекты допускают работу с классами, имеющими прототипы в библиотеке ATL, точно так же, как проекты ATL допускают использование в них классов, имеющих прототипы в библиотеке MFC.

В диалоговом окне **New Class** можно как создавать новые файлы заголовка и реализации, содержащие описание только данного класса, так и добавлять описание класса в уже существующие файлы заголовков и реализации, которые в этом случае будут содержать описание сразу нескольких классов. В последнем случае необходимо быть особенно осторожным при использовании мастера ClassWizard, поскольку он всегда помещает созданные им заготовки функций обработки сообщений в конец файла реализации, где может находиться описание функций другого класса. Поэтому после создания функций обработки сообщений с помощью мастера ClassWizard рекомендуется открыть файл реализации данного класса и переместить эти функции в свой класс. Другим побочным эффектом объединения описания нескольких классов в одном файле может быть расположение описания класса в файле заголовка после операторов условной трансляции, обеспечивающих однократное включение объявления класса в файл реализации. И, если размещение заготовок функций обработки сообщений в конце файла приводит только к неудобочитаемости файла и не является ошибкой, то множественное объявление класса в файле реализации ошибкой является.

Для приложений, работающих с библиотеками ATL, а также MFC и поддерживающих автоматизацию, используется интерфейс COM. Определение интерфейса не входит в процедуру создания пользовательского класса.

Создание классов, имеющих прототипы в библиотеке MFC

Добавить в проект класс, имеющий прототип в библиотеке MFC, можно при помощи любого из следующих инструментальных средств: панели **ClassView** окна **Workspace**, панели инструментов **WizardBar**, мастера ClassWizard, а также команды меню **Insert**, **New Class...**. Пользователю достаточно задать имя нового класса, выбрать для него базовый класс и определить идентификатор ресурса диалогового окна, с которым будет связан данный класс (если такая связь имеется). После этого **ClassWizard** создаст файл заголовка и файл реализации для нового класса. Следует помнить, что добавлять классы MFC к приложению ATL COM можно, только если при его создании была определена поддержка MFC.

Чтобы добавить к проекту класс, имеющий прототип в библиотеке MFC:

1. Откройте диалоговое окно **New Class**, для чего выполните одну из следующих операций:
   - Щелкните правой кнопкой мыши на значке проекта в панели **ClassView** окна **Workspace** и выберите команду **New Class...** в появившемся контекстном меню

- Раскройте контекстное меню **WizardBar Actions** в панели инструментов **WizardBar** и выберите команду **New Class...**
- Выберите команду меню **Insert**, **New Class...**
- Выполните команду **View**, **ClassWizard**, в появившемся диалоговом окне **MFC ClassWizard** нажмите кнопку **Add Class...** и в появившемся контекстном меню выберите опцию **New...**

Диалоговое окно **New Class** имеет четыре разновидности: для классов MFC и ATL, для пользовательских классов, не имеющих прототипов в библиотеках, и для классов баз данных. Кроме того, мастер ClassWizard имеет собственную версию данного окна, в которой отсутствует раскрывающийся список **Class type:**. Операционная система сама определяет тип активного приложения и открывает соответствующее диалоговое окно **New Class**. На рис. С.4 изображено соответствующее диалоговое окно **New Class** для класса MFC.

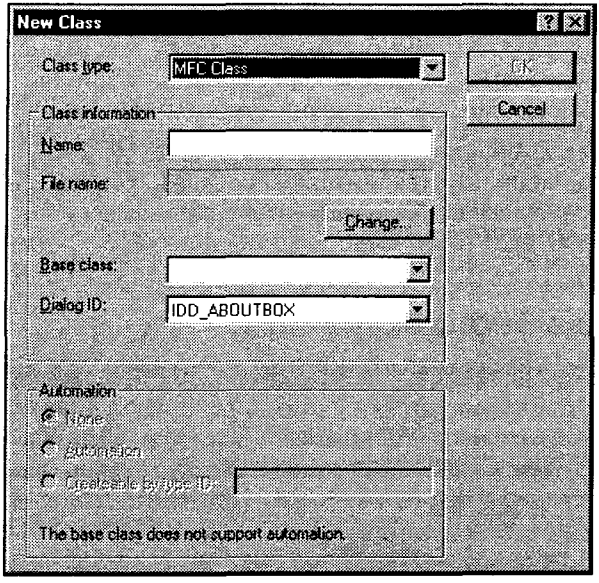


Рис. С.4. Диалоговое окно **New Class** для класса MFC

2. В раскрывающемся списке **Class type:** выберите опцию **MFC Class**.
3. В текстовое поле **Name:** введите имя нового класса. Введенное имя отображается в текстовом поле **File name:**, имеющем статус "только для чтения", причем к имени класса прибавляется расширение .cpp. В этом поле отображается имя файла реализации данного класса. По умолчанию имена файла заголовка и файла реализации совпадают с именем класса, за одним исключением: первая заглавная буква "С" не переходит из имени класса в имя файла.

- Чтобы изменить имена файлов заголовка и реализации для данного класса, нажмите кнопку **Change...** (рис. С.4) и измените имена файлов в диалоговом окне **Change Files**, приведенном на рис. С.5
- Чтобы добавить описание нового класса в существующие файлы, нажмите кнопку **Change...** (рис. С.4) и укажите в соответствующих текстовых полях имена файлов, в которых необходимо разместить описание нового класса

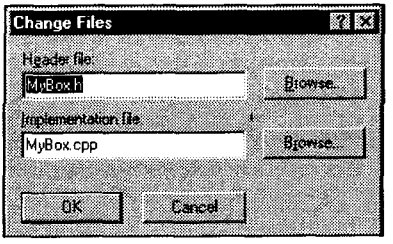


Рис. С.5. Диалоговое окно **Change Files**

4. В раскрывающемся списке **Base class:** выберите базовый класс для создаваемого в данном диалоговом окне нового класса. Имеются два вида классов MFC:
   - Типа `CButton`, не требующие указания идентификатора ресурса. Классы `CButton` или `CEdit`, имеющие базовые классы, отличные от классов `CDaoRecordView`, `CDialog`, `CFormView`, `CPropertyPage` или `CRecordView`, не используют идентификаторы ресурсов. Поэтому при создании данных классов раскрывающийся список **Dialog ID:** в диалоговом окне **New Class** недоступен
   - Типа `CDialog`, использующие идентификатор ресурса. При создании классов, требующих указания идентификатора ресурса (к таким классам относятся классы, производные от классов `CDaoRecordView`, `CDialog`, `CFormView`, `CPropertyPage` и `CRecordView`), целесообразно сначала создать ресурс диалога при помощи редактора диалога, а затем использовать мастер ClassWizard, чтобы создать класс, связанный с этим ресурсом диалога

Однако возможно и подключение нового диалогового окна к существующему классу диалогового окна. Для этого следует воспользоваться процедурой подключения ресурса к существующему классу, описанной ниже.

1. Для классов диалоговых окон выберите идентификатор ресурса из раскрывающегося списка **Dialog ID:**.
2. Установите один из переключателей группы **Automation** (поддержка Автоматизации):
   - **None** — данный класс не поддерживает Автоматизацию
   - **Automation** — данный класс поддерживает все возможности Автоматизации. Выбор данного параметра означает, что данный класс может

программироваться такими приложениями, как VBA и Excel. Это положение переключателя может быть установлено только для определенных классов

- **Createable by type ID:** — позволяет другим приложениям создавать объекты данного класса, используя Автоматизацию. Выбор данного параметра означает, что клиентское приложение может непосредственно создавать объект Автоматизации данного класса. По идентификатору типа, указанному в текстовом поле, клиентское приложение определяет создаваемый объект. Идентификатор должен быть уникальным. Это положение переключателя может быть установлено только для некоторых классов

При установке одного из двух последних переключателей группы **Automation** базовый класс должен обеспечивать доступ ко всем возможностям Автоматизации.

3. Нажмите кнопку **OK**, чтобы создать новый класс и добавить его к проекту. Новый класс немедленно появляется в панели **ClassView** окна **Workspace**, а его файлы заголовка и реализации — в панели **FileView** того же окна.

Подключение ресурса к классу

Можно подключить к существующему классу новое диалоговое окно, меню, панель инструментов или таблицу акселераторов в диалоговом окне **Select Class**. Это диалоговое окно используется, если класс создается до создания связанного с ним ресурса. Связь идентификатора ресурса и класса используется в создаваемой мастером ClassWizard карте сообщений и в панели инструментов **WizardBar**.

В большинстве случаев применяется другая процедура, по которой сначала создается ресурс, а затем запускается мастер ClassWizard для создания нового класса, связанного с этим ресурсом.

Чтобы связать идентификатор ресурса с классом:

1. Используйте редактор диалогового окна, панели инструментов или таблицы акселераторов для создания ресурса.
2. Выберите команду **View**, **ClassWizard**. Появится диалоговое окно **MFC ClassWizard**, а поверх него будет расположено диалоговое окно **Adding a Class**, изображенное на рис. С.6. Данное окно позволяет создать новый класс или использовать уже существующий.
3. Чтобы создать новый класс, установите переключатель **Create a new class**, нажмите кнопку **OK** и продолжите процедуру, описанную в предыдущем разделе.
4. В противном случае установите переключатель **Select an existing class** (Выбрать существующий класс) и нажмите кнопку **OK**. Появится диало-

говое окно **Select Class**, изображенное на рис. С.7 и содержащее перечень всех классов, включенных в проект.

5. Выделите в списке **Class list:** класс, с которым вы хотели бы связать данный ресурс, и нажмите кнопку **Select**. Появится окно сообщения **Microsoft Visual C++** с вопросом, нужно ли менять имеющийся идентификатор ресурса класса на новый.
6. Нажмите кнопку **OK**. Мастер ClassWizard сопоставит новый ресурс существующему классу.

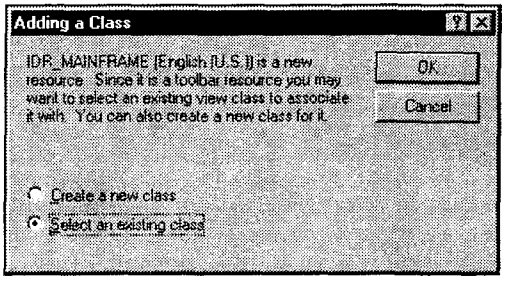


Рис. С.6. Диалоговое окно **Adding a Class**

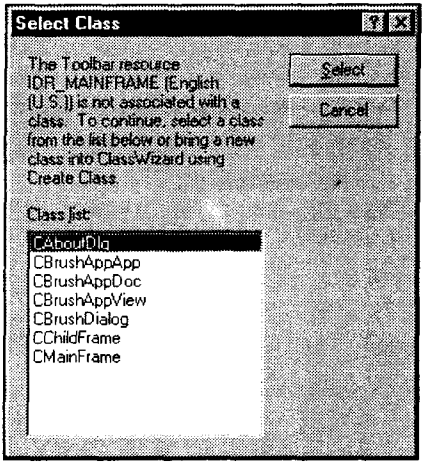


Рис. С.7. Диалоговое окно **Select Class**

Добавление поддержки библиотеки ATL в приложение MFC

Чтобы обеспечить поддержку объектов библиотеки ATL в приложении, использующем библиотеку MFC, достаточно добавить в это приложение объект одного из классов библиотеки ATL. Эта поддержка касается только простейших объектов COM, добавленных в данное приложение, и проектов, использующих библиотеки динамической компоновки. Возможно добавление в проект и других объектов COM (включая элементы управления ActiveX), но при этом не гарантируется их правильная работа в приложении.

Чтобы добавить поддержку библиотеки ATL в приложение MFC:

1. В панели **ClassView** окна **Workspace** выделите проект (папку, содержащую классы), в который необходимо добавить поддержку объектов библиотеки ATL, и щелкните правой кнопкой мыши.
2. В появившемся контекстном меню выберите опцию **New ATL Object...**. Появится окно сообщения **ATL Object Wizard**, изображенное на рис. С.8.

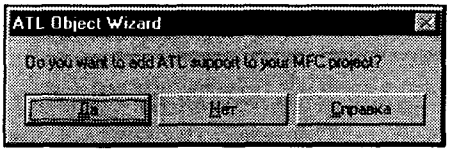


Рис. С.8. Окно сообщения **ATL Object Wizard**

3. Нажмите кнопку **Да**, признавая необходимость включения поддержки библиотеки ATL в приложении MFC. Появится диалоговое окно **ATL Object Wizard**, изображенное на рис. С.9.
4. Выделите объект, который необходимо вставить, и нажмите кнопку **Next>**, чтобы запустить мастер создания объектов ATL. Чтобы получить поддержку объектов ATL без добавления в приложение самих объектов, нажмите кнопку **Cancel**.

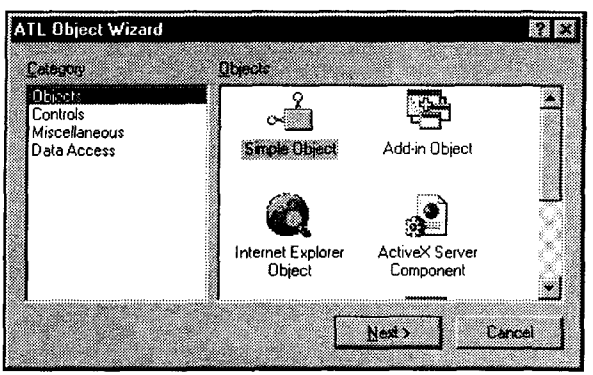


Рис. С.9. Диалоговое окно **ATL Object Wizard**

Создание пользовательского класса, не имеющего прототипов в библиотеках

Пользовательский класс, не имеющий прототипа, или его потомок любого уровня можно создать при помощи панели **ClassView** окна **Workspace**, панели инструментов **WizardBar** или команды меню **Insert**, **New Class...**.

Чтобы создать подобный класс:

1. Откройте диалоговое окно **New Class** одним из способов, описанных выше в разделе *"Создание классов, имеющих прототипы в библиотеке MFC"* данного приложения.
2. В раскрывающемся списке **Class type:** выберите опцию **Generic Class**. Диалоговое окно **New Class** примет вид, изображенный на рис. С.10.
3. В текстовое поле **Name:** введите имя нового класса. Введенное имя отображается в текстовом поле **File name:**, имеющем статус "только для чтения", причем к имени класса прибавляется расширение .cpp. В этом поле отображается имя файла реализации данного класса. По умолчанию файлы заголовка и реализации имеют то же самое имя, что и класс, за одним

исключением: первая заглавная буква "С" не переходит из имени класса в имя файла.

- Чтобы изменить имена файлов заголовка и реализации для данного класса, нажмите кнопку **Change...** и введите новые имена файлов в соответствующие поля диалогового окна **Change Files**
- Чтобы добавить описание класса в существующие файлы заголовка и реализации, нажмите кнопку **Change...**. После этого в появившемся диалоговом окне **Change Files** последовательно нажмите кнопки **Browse...**, расположенные у соответствующих текстовых полей, и выберите в появившихся диалоговых окнах имена существующих файлов

4. В столбце **Derived From** списка **Base class(es):** укажите имя базового класса, потомком которого является создаваемый класс. В столбце **As** данного списка укажите модификатор прав доступа нового класса к своему базовому классу. Модификатор прав доступа может иметь три значения: `public`, `protected` или `private`.
5. Нажмите кнопку **OK**, чтобы создать новый класс и добавить его к проекту. Новый класс немедленно появляется в панели **ClassView** окна **Workspace**, а его файлы заголовка и реализации — в панели **FileView** того же окна.

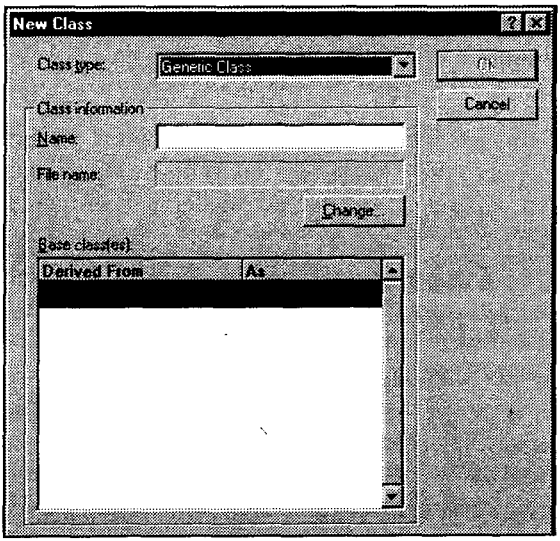


Рис. С.10. Диалоговое окно **New Class**

Создание функций обработки сообщений

Панель **ClassView** окна **Workspace**, панель инструментов **WizardBar** и мастер ClassWizard позволяют связать функции классов, производных от классов библиотеки MFC с сообщениями, посылаемыми ресурсами данного приложения. Для этого используются так называемые карты сообщений.

Когда панель **ClassView** окна **Workspace**, панель инструментов **WizardBar** или мастер ClassWizard создают новый класс, производный от одного из классов библиотеки MFC, они автоматически создают описание класса в файле заголовка (имеющего расширение .h) и помещают в файл реализации (имеющий расширение .cpp) заготовки функций, реализующих методы данного класса. Мастер ClassWizard следит за классом, обрабатываемыми им сообщениями и элементами обмена данными, что позволяет в дальнейшем вносить изменения в данный класс.

Для обработки сообщений мастер ClassWizard использует классы, производные от класса `CCmdTarget`, описанного в библиотеке MFC. Классы, не обрабатывающие сообщений, создают производный класс непосредственно в текстовом редакторе. (Исключением из этого правила является класс `CRecordset`, используемый для работы с базами данных. Производные от него классы могут создаваться с помощью мастера ClassWizard.)

Создание функций обработки сообщения с использованием панели *ClassView* окна *Workspace* или панели инструментов *WizardBar*

Чтобы создать функцию обработки сообщения при помощи панели **ClassView** окна **Workspace** или панели инструментов **WizardBar**:

1. Выполните одну из операций:
   - В панели **ClassView** окна **Workspace** щелкните правой кнопкой мыши на имени класса, в который необходимо добавить функцию обработки сообщения, и в появившемся контекстном меню выберите команду **Add Windows Message Handler...**
   - Раскройте список **WizardBar C++ Class** в панели инструментов **WizardBar** и выберите в нем имя класса, в который необходимо добавить функцию обработки сообщения, раскройте меню **WizardBar Actions** и выберите в нем команду **Add Windows Message Handler...**

   Появится диалоговое окно **New Windows Message and Event Handlers for class CDialogApplicationDlg** (Новые функции обработки сообщений и событий для класса `CDialogApplicationDlg`). Естественно, имя класса в каждом конкретном случае будет соответствовать выделенному пользователем имени класса (рис. С.11).
2. В списке **Class or object to handle:** выберите имя класса или идентификатор ресурса, функции которого будут обрабатывать данное сообщение.
   - В списке **New Windows messages/events:** перечислены сообщения, которые могут быть обработаны данным классом или объектом
   - В списке **Existing message/event handlers:** перечислены сообщения, которые уже обрабатываются данным классом или объектом

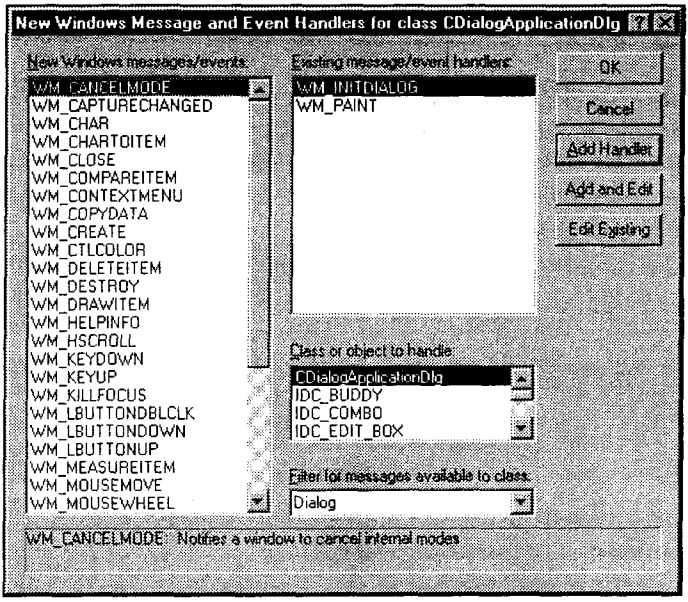


Рис. С.11. Диалоговое окно **New Windows Message and Event Handlers for class CDialogApplicationDlg**

- В текстовом поле раскрывающегося списка **Filter for messages available to class:** указан базовый класс данного класса, из списка сообщений которого необходимо выбирать обрабатываемые сообщения

Если в качестве фильтра выбран класс диалога, то список **New Windows messages/events:** будет содержать только идентификаторы сообщений класса окна, но не будет содержать идентификаторы команд меню. Чтобы включить в данный список идентификаторы команд меню, необходимо определить, объект какого класса будет обрабатывать данное сообщение (например, `CView` или `CMainFrame`). После этого следует выбрать данный класс в раскрывающемся списке **WizardBar C++ Class** панели инструментов **WizardBar**, выбрать в раскрывающемся списке **WizardBar C++ Filter** той же самой панели инструментов соответствующий идентификатор команды меню, раскрыть меню **WizardBar Actions** и выбрать команду **Add Windows Message Handler...**.

3. Выделите строку в списке **New Windows messages/events:** и нажмите кнопку **Add Handler**. Для данного сообщения будет создана функция обработки. При этом **WizardBar** добавляет в файл реализации класса заготовку данной функции. Например:

```
// CChildFrame message handlers
   void CChildFrame::OnLButtonDblClk(UINT nFlags,
         CPoint point)
   {
      // TODO: Add your message handler code here
      // and/or call default
```

```
    CMDIChildWnd::OnLButtonDblClk(nFlags, point);
}
```

После нажатия кнопки **Add Handler** диалоговое окно **New Windows Message and Event Handlers for class CDialogApplicationDlg** закрывается, открывается окно редактирования соответствующего файла реализации, а текстовый курсор помещается в тело только что созданной заготовки функции обработки сообщения.

Чтобы отредактировать функцию обработки сообщения, выделите соответствующий ей идентификатор сообщения в списке **Existing message/event handlers:** и нажмите кнопку **Edit Existing**. При этом диалоговое окно **New Windows Message and Event Handlers for class CDialogApplicationDlg** закрывается, открывается окно редактирования соответствующего файла реализации и текстовый курсор помещается в тело соответствующей функции обработки сообщения.

Быстрый способ создания функций обработки сообщения для элементов управления диалогового окна, определенных для него по умолчанию

Для диалоговых окон проекта, связанных с некоторым классом, возможно использование преимуществ работы с классами при создании функций обработки сообщений, позволяющих быстро создавать данные функции для обработки сообщений, поступающих от стандартных элементов управления, или для любого сообщения Windows, удовлетворяющего определенным требованиям.

Чтобы создать функцию обработки стандартного сообщения:

1. Откройте редактор ресурса диалогового окна, которое содержит данный элемент управления. Это можно сделать с помощью команды **Go To Dialog Editor** контекстного меню панели **ClassView** окна **Workspace**, панели инструментов **WizardBar** или двойного щелчка левой кнопкой мыши на значке ресурса диалогового окна в панели **ResourceView** окна **Workspace**.
2. Дважды щелкните левой кнопкой мыши на элементе управления, сообщение от которого следует обработать.
3. Появится диалоговое окно **Add Member Function**, изображенное на рис. С.12. Данное окно содержит заданный по умолчанию идентификатор сообщения (например, `BN_CLICKED` для кнопки) и предложенное по умолчанию имя функции обработки данного сообщения (например, `OnOk`).

   Если данное сообщение уже имеет функцию обработки, то двойной щелчок на элементе управления приведет к переходу на текст данной функции в окне редактирования файла реализации.
4. Нажмите кнопку **OK**. Будет создано описание данной функции в классе и текст заготовки для функции обработки сообщения.

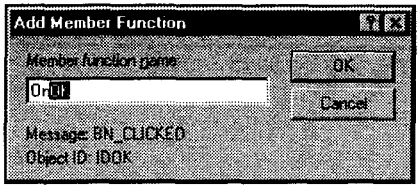


Рис. С.12. Диалоговое окно **Add Member Function**

5. Чтобы отказаться от этой возможности, выберите команду **Tools**, **Options...**. Появится диалоговое окно **Options**. Перейдите на вкладку **Compatibility** и сбросьте флажок **Double-click in dialog editor edits code (MFC only)**. Диалоговое окно **Options** примет вид, показанный на рис. С.13. При такой установке двойной щелчок на элементе управления вызовет связанное с ним диалоговое окно **Properties**.

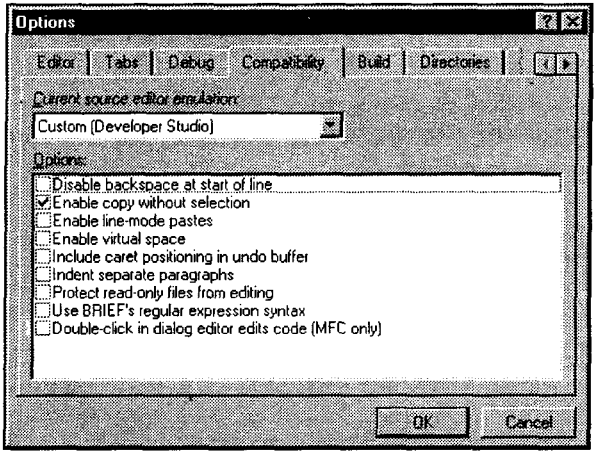


Рис. С.13. Диалоговое окно **Options**

В табл. С.1 указаны имена элементов управления и сопоставленные им по умолчанию идентификаторы сообщений.

Таблица С.1. *Соответствие имен элементов управления идентификаторам сообщений*

Имя элемента управления	Идентификатор сообщения
Animate	NM_OUTOFMEMORY
Check Box	BN_CLICKED
Combo Box	CBN_EDITCHANGE
Custom	Не определен
Date Time Picker	DTN_CLOSEUP
Edit Box	EN_CHANGE

Таблица С.1 (окончание)

Имя элемента управления	Идентификатор сообщения
Hot Key	NM_OUTOFMEMORY
IP Address	IPN_FIELDCHANGED
List	NM_CLICK
List Box	LBN_SELCHANGE
Month Calendar	MCN_GETDAYSTATE
Progress	NM_OUTOFMEMORY
Push Button	BN_CLICKED
Radio Button	BN_CLICKED
Rich Edit	NM_CLICK
Scrollbar	Не определен
Slider	NM_OUTOFMEMORY
Spin	NM_OUTOFMEMORY
Tab	TCN_SELCHANGE
Tree	NM_CLICK

Быстрый способ создания функций обработки сообщения для элементов управления диалогового окна

Чтобы создать функцию обработки сообщения для элемента управления диалогового окна:

1. Откройте редактор ресурса диалогового окна, которое содержит данный элемент управления. Это можно сделать при помощи команды **Go To Dialog Editor** контекстного меню панели **ClassView** окна **Workspace**, панели инструментов **WizardBar** или двойного щелчка левой кнопкой мыши на значке ресурса диалогового окна в панели **ResourceView** окна **Workspace**.
2. В раскрывающемся списке **WizardBar C++ Members** панели инструментов **WizardBar** выделите сообщение, для которого еще не создана функция обработки (сообщение не выделено полужирным шрифтом). Появится диалоговое окно **Add Member Function**. Если для данного сообщения уже определена функция обработки, то ее выделение приведет к переходу на текст данной функции в окне редактирования файла реализации.
3. Нажмите кнопку **OK**, чтобы принять заданное по умолчанию имя функции обработки сообщения, или измените его и только потом нажмите кнопку **OK**. **WizardBar** создаст описание данной функции в классе и текст заготовки для данной функции обработки сообщения.

Создание функции обработки сообщения при помощи мастера ClassWizard

Чтобы создать функцию обработки сообщения с использованием мастера ClassWizard:

1. Выберите команду меню **View**, **ClassWizard**. Появится диалоговое окно **MFC ClassWizard**, показанное на рис. С.14. Здесь будет отображаться информация о выделенном классе или о классе, который до этого редактировался мастером ClassWizard.

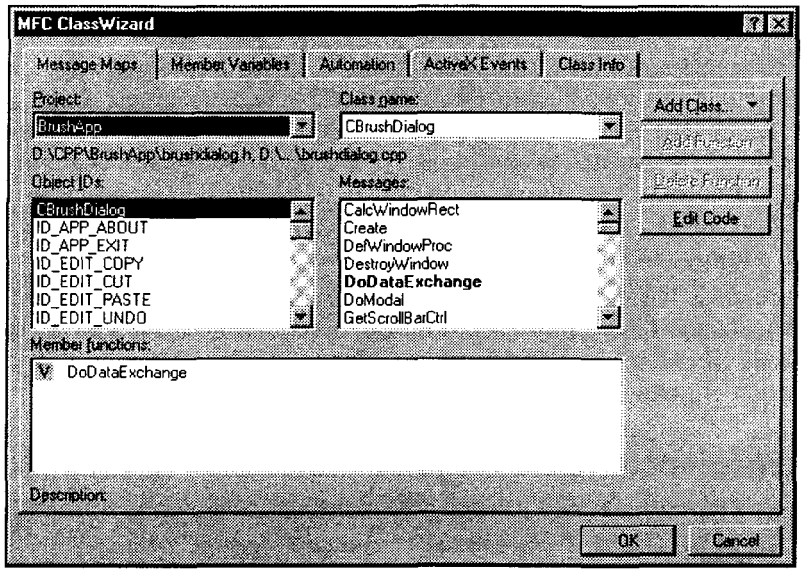


Рис. С.14. Диалоговое окно **MFC ClassWizard**

2. Перейдите на вкладку **Message Maps**.
3. В раскрывающемся списке **Class name:** выберите имя класса, функции которого будут обрабатывать данное сообщение. В списках и текстовых полях диалогового окна появится информация о выбранном классе.
4. В списке **Object IDs:** выделите имя объекта класса пользовательского интерфейса, для которого будет создаваться функция обработки сообщения.
5. В списке **Messages:** выберите идентификатор сообщения, для которого будет создаваться функция обработки. Нажмите кнопку **Add Function** (или дважды щелкните на идентификаторе сообщения). Если для данного сообщения не существует определенного по умолчанию имени функции обработки, появится диалоговое окно **Add Member Function**, изображенное на рис. С.15.

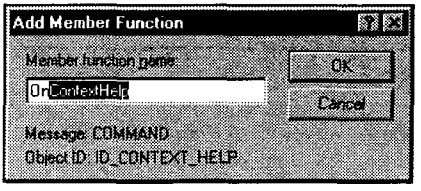


Рис. С.15. Диалоговое окно **Add Member Function**

6. Нажмите кнопку **OK** и согласитесь с предложенным по умолчанию именем функции, или измените это имя и нажмите кнопку **OK**.

7. В списке **Member functions:** появится имя новой функции обработки сообщения.

   Сообщения, для которых определены функции обработки, выделяются в списке **Messages:** полужирным шрифтом. В этом списке перечислены основные сообщения, обрабатываемые данным типом классов. Если данный класс не обрабатывает этот тип сообщений, необходимо войти в редактор ресурса, из которого поступает данное сообщение (например, ресурс диалогового окна), вызвать мастер ClassWizard и выбрать в раскрывающемся списке **Class name:** имя класса, объект которого будет обрабатывать данное сообщение.

   Другим способом изменить набор обрабатываемых сообщений является раскрытие вкладки **Class Info** диалогового окна **MFC ClassWizard**, изображенной на рис. С.16, и изменение содержимого текстового поля раскрывающегося списка **Message filter**.

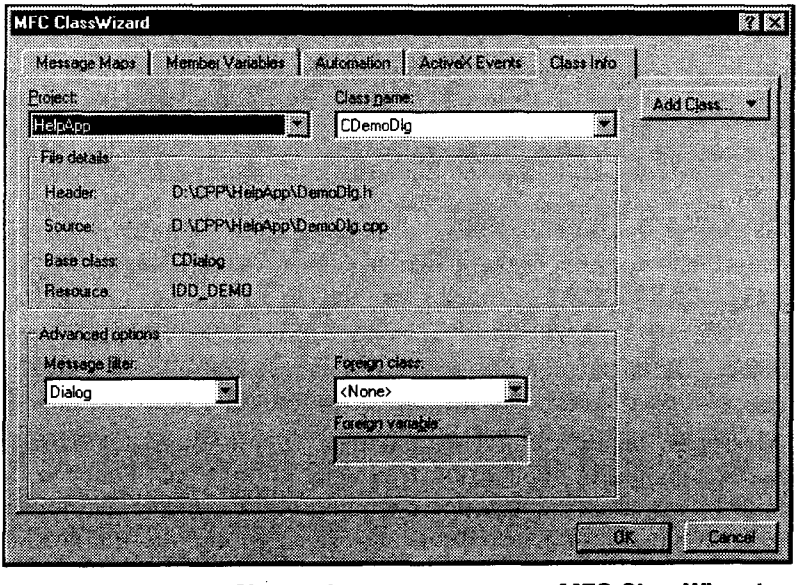


Рис. С.16. Вкладка **Class Info** диалогового окна **MFC ClassWizard**

При выделении идентификатора сообщения в списке **Messages** в нижней части диалогового окна появляется короткая справка по данному сообщению.

8. Выполните одно из следующих действий:
   - Нажмите кнопку **Cancel** и закройте диалоговое окно **MFC ClassWizard**, отказавшись от изменений, внесенных в текущем сеансе работы с мастером ClassWizard
   - Добавьте функции обработки для других сообщений
   - Нажмите кнопку **OK** и закройте диалоговое окно **MFC ClassWizard**, сохранив все изменения, внесенные в текущем сеансе работы с мастером ClassWizard
   - Нажмите кнопку **Edit Code** и перейдите к редактированию функции, выделенной в списке **Member functions:**

   Если вы нажали кнопку **OK** или кнопку **Edit Code**, мастер ClassWizard внесет в файлы заголовка и реализации следующие изменения:
   - В описание класса в файле заголовка будут добавлены объявления новых функций обработки сообщений
   - В файл реализации будут добавлены корректные и функционально полные заготовки данных функций
   - Будут внесены соответствующие изменения в карту сообщений данного класса

В некоторых случаях могут использоваться более простые процедуры создания функций обработки сообщений.

Быстрый способ создания функций обработки сообщения для кнопок диалогового окна

Чтобы создать функцию обработки сообщения для кнопки диалогового окна:

1. Выделите данную кнопку в редакторе диалогового окна.
2. Удерживая клавишу <Ctrl>, дважды щелкните на кнопке в заготовке диалогового окна. Мастер ClassWizard автоматически создаст функцию обработки в классе, связанном с данным диалоговым окном. Имя этой функции будет автоматически сформировано из идентификатора ресурса данной кнопки. После этого откроется окно редактирования соответствующего файла реализации и текстовый курсор переместится в тело заготовки новой функции.

Чтобы выполнить ту же операцию другим способом:

1. Дважды щелкните на данной кнопке в редакторе диалогового окна.
2. Если с данным ресурсом диалогового окна еще не связан класс, появится диалоговое окно **Adding a Class**, в котором пользователь может выбрать

класс, связанный с данным ресурсом. При этом имеется возможность связать данный ресурс с новым классом или выбрать уже имеющийся.

Если с ресурсом данного диалогового окна уже связан файл, но данное сообщение не имеет функции для обработки, открывается диалоговое окно **Add Member Function**, позволяющее задать имя для функции обработки сообщения. Если же сообщение уже имеет функцию обработки, то открывается окно редактирования соответствующего файла реализации и текстовый курсор помещается в тело данной функции.

Создание функций обработки для отраженного сообщения

Функции обработки отраженного сообщения позволяют классу элемента управления обрабатывать собственные сообщения до того, как они поступят в родительский класс. Для посылки сообщения из класса элемента управления в родительское окно может быть использована функция `CWnd::SendMessage`.

Отраженные сообщения позволяют, например, создать окно списка, которое само бы осуществляло свой вывод на экран, не полагаясь в этом вопросе на класс родительского окна.

Создание функций обработки для отраженного сообщения при помощи мастера ClassWizard

Чтобы создать функцию обработки отраженного сообщения, используя мастер ClassWizard:

1. Вызовите мастер ClassWizard и раскройте вкладку **Message Maps**.
2. В раскрывающемся списке **Class name:** выберите имя класса элемента управления.
3. В списке **Object IDs:** выделите имя класса элемента управления.
4. В списке **Messages:** выделите идентификатор отраженного сообщения, для которого необходимо создать функцию обработки. Обратите внимание, что все идентификаторы отраженных сообщений начинаются со знака равенства (=). Сообщения, для которых уже определены функции обработки, отображаются полужирным шрифтом.
5. Нажмите кнопку **Add Function** (или дважды щелкните на идентификаторе сообщения). Для сообщений, не имеющих предопределенного имени функции обработки, появляется диалоговое окно **Add Member Function**.
6. Нажмите кнопку **OK** и согласитесь с предложенным именем функции обработки сообщения или измените это имя и нажмите кнопку **OK**. В списке **Member functions:** появится имя новой функции обработки сообщения.

7. Выполните одно из следующих действий:
   - Нажмите кнопку **Cancel** и закройте диалоговое окно **MFC ClassWizard**, отказавшись от изменений, внесенных в текущем сеансе работы с мастером ClassWizard
   - Добавьте функции обработки для других сообщений
   - Нажмите кнопку **OK** и закройте диалоговое окно **MFC ClassWizard**, сохранив все изменения, внесенные в текущем сеансе работы с мастером ClassWizard
   - Нажмите кнопку **Edit Code** и перейдите к редактированию функции, выделенной в списке **Member functions:**

   Если вы нажали кнопку **OK** или кнопку **Edit Code**, мастер ClassWizard внесет в файлы заголовка и реализации следующие изменения:
   - В описание класса в файле заголовка будут добавлены объявления новых функций обработки сообщений
   - В файл реализации будут добавлены корректные и функционально полные заготовки данных функций
   - Будут внесены соответствующие изменения в карту сообщений данного класса

Создание функций обработки для отраженного сообщения при помощи панели инструментов *WizardBar*

Чтобы создать функцию обработки отраженного сообщения, используя панель инструментов **WizardBar**:

1. Выберите класс элемента управления в раскрывающемся списке **WizardBar C++ Class** панели инструментов **WizardBar** и нажмите кнопку **WizardBar C++ Actions**.
2. В раскрывшемся контекстном меню выберите команду **Add Windows Message Handler...**. Появится диалоговое окно **New Windows Message and Event Handlers for class...**.
3. В списке **Class or object to handle:** выберите имя класса элемента управления.
4. В списке **New Windows messages/events:** выделите идентификатор сообщения, для которого нужно создать функцию обработки. Все идентификаторы отраженных сообщений начинаются со знака равенства (=).
5. Нажмите кнопку **Add Handler** или кнопку **Add and Edit**. В файлы будут внесены следующие изменения:
   - В описание класса в файле заголовка будут добавлены объявления новых функций обработки сообщений
   - В файл реализации будут добавлены корректные и функционально полные заготовки данных функций

- Будут внесены соответствующие изменения в карту сообщений данного класса

После этого текстовый курсор будет перемещен в тело вновь созданной функции.

Уничтожение функций обработки сообщений

Функции обработки сообщений могут быть уничтожены при помощи панели **ClassView** окна **Workspace** или мастера ClassWizard. При уничтожении функций обработки сообщений с использованием панели **ClassView** окна **Workspace** описание функции удаляется из файла заголовка (имеющего расширение .h), а из карт `MSG`, `MESSAGE` или `DISPATCH` удаляются связанные с данной функцией макросы. Тело функции выделяется, что позволяет его тут же уничтожить, а каждая ее строка превращается в комментарий.

При уничтожении функций обработки сообщений мастером ClassWizard описание функции удаляется из файла заголовка, а из всех карт сообщений удаляются связанные с данной функцией макросы. Однако пользователь должен после этого удалить тело данной функции из файла реализации, а также все вызовы функции в файлах проекта. Использование мастера ClassWizard для анализа структуры классов позволяет упростить процесс поиска данных вызовов.

Примечание

В библиотеке MSDN утверждается, что существует возможность уничтожить функцию с использованием панели инструментов **WizardBar**, но ее меню **WizardBar Actions** не содержит указанной в данной библиотеке команды **Delete**.

Чтобы уничтожить функцию-член класса с использованием панели **ClassView** окна **Workspace**:

1. В панели **ClassView** окна **Workspace** откройте класс, содержащий данную функцию, и выделите ее имя.
2. Щелкните правой кнопкой мыши и в появившемся контекстном меню выберите команду **Delete**. Появится окно сообщения **Confirm Function Delete** (Подтверждение удаления функции), изображенное на рис. С.17.
3. Нажмите кнопку **Да** и уничтожьте данную функцию.

Чтобы уничтожить функцию-член класса с использованием мастера ClassWizard:

1. Вызовите мастер ClassWizard и перейдите на вкладку **Message Maps**.
2. В раскрывающемся списке **Class name:** выберите класс, содержащий функцию, которую необходимо уничтожить.

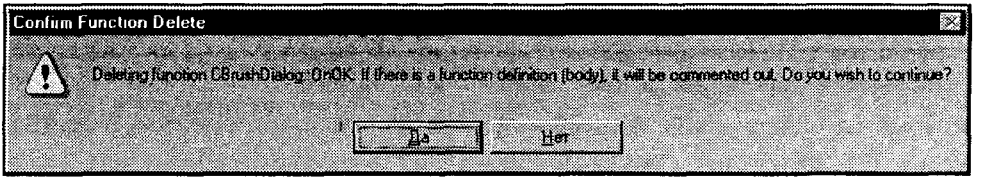


Рис. С.17. Диалоговое окно **Confirm Function Delete**

3. В списке **Member functions:** выделите имя данной функции.
4. Нажмите кнопку **Delete Function**. Появится диалоговое окно, напоминающее о необходимости уничтожить тело данной функции в файле реализации.
5. Нажмите кнопку **OK**. Функция будет удалена из файла заголовка и, из карт сообщений, расположенных как в файле заголовка, так и в файле реализации.

Редактирование функций обработки сообщений

Создав функцию обработки сообщения при помощи панели **ClassView** окна **Workspace** или панели инструментов **WizardBar**, пользователь может применить любое из этих средств для поиска тела данной функции с целью редактирования ее программного кода.

Если вы находитесь в редакторе ресурса диалогового окна, для перехода к соответствующей функции обработки сообщения достаточно дважды щелкнуть на элементе управления, посылающем данное сообщение.

Чтобы перейти к редактированию функции, определенной в мастере ClassWizard:

1. Вызовите мастер ClassWizard и раскройте вкладку **Message Maps**.
2. В раскрывающемся списке **Class name:** выберите класс, содержащий данную функцию обработки сообщений.
3. В списке **Member functions:** выделите имя функции, подлежащей редактированию.
4. Нажмите кнопку **Edit Code** или дважды щелкните на имени функции. Текстовый курсор переместится в тело функции.

Чтобы перейти к редактированию функции с использованием панели инструментов **WizardBar**:

1. Выберите в раскрывающемся списке **WizardBar C++ Class** панели инструментов **WizardBar** имя класса, содержащего редактируемую функцию.

2. В раскрывающемся списке **WizardBar C++ Members** той же панели инструментов выберите имя данной функции. Текстовый курсор будет автоматически перемещен в тело выбранной функции.

По идее, команды меню **WizardBar Actions** панели инструментов **WizardBar** должны способствовать быстрому нахождению описания функции в файле заголовка по ее реализации и наоборот, однако, описание функции в файле заголовка может быть найдено только из самого файла заголовка (когда текстовый курсор и так установлен на этом описании) и из окна редактирования, связанного с классом диалогового ресурса. Все попытки поиска описания функции в файле заголовка из файла реализации заканчивались выдачей диалогового окна с сообщением о том, что система не может найти описания данной функции.

Работа с переменными класса диалогового окна

Мастер ClassWizard предоставляет пользователю все преимущества *динамического обмена данными в диалоговом окне* (Dialog Data eXchange, DDX) и проверки корректности данных в диалоговом окне, связанные с библиотекой MFC.

Чтобы использовать DDX в приложении, необходимо определить переменные в классе диалогового окна и сопоставить их с соответствующими элементами управления диалогового окна. Приложение инициализирует эти переменные в процессе отображения диалогового окна на экране. При нажатии пользователем кнопки **OK** система сохраняет в этих переменных информацию, содержавшуюся в элементах управления закрываемого диалогового окна.

Функции, основанные на механизме динамической *проверки данных в диалоговых окнах* (Dialog Data Validation, DDV), осуществляют автоматическую проверку корректности данных, передаваемых из элементов управления в диалоговые окна. Пользователь может установить следующие критерии проверки данных:

- Максимальная длина строки, вводимой в текстовое поле
- Минимальное и максимальное значение вводимой величины

Функции для проверки длины строки включаются в текст программы мастером ClassWizard. Для проверки максимального и минимального значения вводимой величины пользователь должен сам вставить соответствующую функцию.

Кроме того, мастер ClassWizard позволяет создавать пользовательские программы проверки корректности данных в элементе управления диалогового окна.

Использование мастера ClassWizard для включения переменных в класс диалогового окна

Мастер ClassWizard позволяет включать в класс диалогового окна переменные, используемые приложением для динамического обмена данными (DDX). Чтобы инициализировать содержимое или состояние элемента управления диалогового окна или получить информацию о содержимом или состоянии этого элемента управления после закрытия диалогового окна, используйте мастер ClassWizard для включения переменной в класс диалогового окна.

В этом случае в класс диалогового окна включаются функции, обеспечивающие передачу значения этих переменных в объекты классов элементов управления диалогового окна при его создании и передачу в эти переменные информации из соответствующих элементов управления диалогового окна при его закрытии.

При необходимости передать или получить информацию из объектов классов элемента управления диалогового окна в процессе работы с ним пользователя вызовите функцию `CWnd::UpdateData`.

Чтобы включить переменную в класс диалогового окна:

1. Создайте ресурс диалогового окна, поместите в него необходимые элементы управления и задайте их свойства, используя диалоговое окно **Properties**. Создайте класс диалогового окна при помощи мастера ClassWizard.
2. В диалоговом окне **MFC ClassWizard** перейдите на вкладку **Member Variables**.
3. В списке **Control IDs:** выделите идентификатор элемента управления, которому нужно сопоставить переменную в классе диалогового окна и нажмите кнопку **Add Variable**. Появится диалоговое окно **Add Member Variable**, изображенное на рис. С.18.

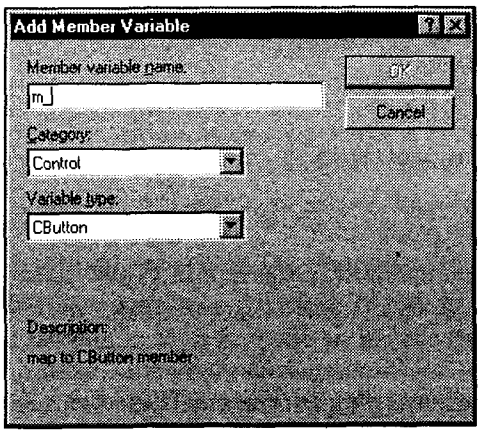


Рис. С.18. Диалоговое окно **Add Member Variable**

4. Введите в текстовое поле **Member variable name:** идентификатор новой переменной. Мастер ClassWizard добавляет к ее имени префикс `m_`, указывающий, что она является переменной-членом класса.

5. В раскрывающемся списке **Category:** определите, соответствует ли данный идентификатор переменной простого типа (**Value**) или объекту класса элемента управления (**Control**).

   Если стандартному элементу управления Windows соответствует переменная простого типа, то в ней будет храниться состояние данного элемента управления или содержащийся в нем текст. Приложение автоматически преобразует эту информацию к типу, содержащемуся в текстовом поле раскрывающегося списка **Variable type:**. Чтобы получить непосредственный доступ к объекту класса элемента управления, необходимо указать в текстовом поле раскрывающегося списка **Category:** значение **Control**. Раскрывающийся список **Variable type** позволяет выбрать тип создаваемой переменной.

6. Нажмите кнопку **OK**. Новая переменная появится в списке **Control IDs:**.

Подобная процедура включения переменной в класс диалогового окна обеспечивает ее динамическое обновление и проверку приложением.

Кроме того, она позволяет, например, сопоставить элементу управления "Полоса прокрутки" целочисленный идентификатор, в котором хранится текущее положение бегунка, или объект класса `CScrollBar`.

Мастер ClassWizard использует для работы с полосой прокрутки в функции `DoDataExchange` вызов функции `DDX_Scroll`. Если функция `DoDataExchange` содержит вызов функции `DDX_Scroll`, то в ней необходимо также задать границы полосы прокрутки. Для этого может быть использован программный код, подобный приведенному ниже:

```
void CMyDlg::DoDataExchange( CDataExchange* pDX)
{
    CScrollBar* pScrollBar = (CScrollBar*)GetDlgItem( IDC_SCROLLBAR1);
    pScrollBar->SetScrollRange( 0, 100);
    CDialog::DoDataExchange( pDX);
    //{{AFX_DATA_MAP(CMyDlg)
    DDX_Scroll(pDX, IDC_SCROLLBAR1, m_nScroll);
    //}}AFX_DATA_MAP
}
```

Начальные значения переменных, используемых мастером ClassWizard для динамического обмена данными, могут быть заданы в конструкторе класса диалогового окна. Это происходит при создании объекта класса диалогового окна. Однако во многих случаях необходимо задавать значения этих переменных в функциях классов представления, из которых вызывается данное

диалоговое окно. Для этого после создания объекта диалога и до вызова функций `CDialog::DoModal` или `CDialog::CreateDialog` нужно присвоить данным переменным соответствующие значения.

После закрытия диалогового окна содержимое переменных-членов его класса сохраняется, поэтому пользователь может прочитать его и оценить результаты вызова диалогового окна.

Функции *DDX_* и *DDV_*

Все функции по передаче и проверке данных в диалоговых окнах являются стандартными функциями языка C++. Для этого не используются никакие специальные ресурсы или особые макросы. Вместо них вызываются виртуальные функции, перегружаемые в каждом классе диалогового окна. Эти функции имеют следующий вид:

```
void CMyDialog::DoDataExchange(CDataExchange* pDX)
{
    CDialog::DoDataExchange(pDX);    // call base class

    //{{AFX_DATA_MAP(CMyDialog)
        <data_exchange_function_call>
         <data_validation_function_call>

    //}}AFX_DATA_MAP
}
```

Специальный формат комментариев AFX позволяет мастеру ClassWizard находить в функции `DoDataExchange` вызовы отдельных функций обработки информации и вносить в них изменения. Любой текст программы, не соответствующий стандарту мастера ClassWizard, должен быть помещен за пределами этих комментариев.

В приведенном выше примере функции динамического обмена данными (`<data_exchange_function_call>`) имеют вид:

```
DDX_Custom(pDX, nIDC, field);
```

а функции проверки корректности передаваемых данных (`<data_validation_function_call>`) являются не обязательными и имеют вид:

```
DDV_Custom(pDX, field, ...);
```

В функции `DoDataExchange` может содержаться неограниченное число пар функций `DDX_/DDV_`. Полный список данных функций, содержащихся в библиотеке MFC, можно найти в файле afxdd_.h.

Эти функции являются составной частью класса `CWnd` и могут использоваться не только в диалоговых окнах.

Начальные значения переменных задаются в стандартном конструкторе класса C++ и обычно располагаются между комментариями //{{AFX_DATA_INIT и //}}AFX_DATA_INIT.

Для вызова функции DoDataExchange обычно используется функция CWnd::UpdateData, выполняющая инициализацию и обработку сообщений при вызове данной функции. Функция CWnd::UpdateData может вызываться в любой функции класса окна, как только возникнет необходимость осуществить обмен данными и проверить их корректность. По умолчанию UpdateData(TRUE) вызывается в функции CDialog::OnOK, а UpdateData(FALSE) — в функции CDialog::OnInitDialog.

Вызов функции проверки корректности данных располагается непосредственно за вызовом соответствующей функции динамического обмена данными.

Принцип работы динамического обмена и проверки данных

Осуществлять динамический обмен данными и их проверку можно и не понимая лежащей в основе этого процесса процедуры. Однако ее понимание позволяет создавать пользовательские функции обмена и проверки данных.

Функция DoDataExchange во многом аналогична функции Serialize — она осуществляет передачу и прием информации в/от внешних объектов (в данном случае — объектов классов элементов управления), а также из/в переменных-членов класса. Аргумент pDX представляет собой контекст для обмена информацией, аналогичный объекту класса CArchive, являющемуся аргументом функции CObject::Serialize. Переменная pDX (являющаяся объектом класса CDataExchange) содержит флаг направления передачи, аналогичный соответствующему флагу в объекте класса CArchive.

Если флаг m_bSaveAndValidate установлен, то объект класса элемента управления инициализируется новым значением, в противном случае состояние элемента управления сохраняется в переменной класса диалогового окна. Проверка данных выполняется только при установленном флаге m_bSaveAndValidate. Состояние флага m_bSaveAndValidate определяется логическим аргументом функции CWnd::UpdateData.

Кроме флага m_bSaveAndValidate класс CDataExchange содержит следующие члены:

- m_pDlgWnd — указатель на объект класса окна (обычно диалогового), содержащего данный элемент управления. Это позволяет не использовать при вызове глобальных функций DDX/DDV аргумент this
- PrepareCtrl и PrepareEditCtrl — подготавливают элемент управления диалогового окна к обмену данными. Они сохраняют дескриптор, позволяющий установить фокус ввода в случае отрицательного результата проверки корректности данных. Функция PrepareCtrl вызывается для элементов управления, не являющихся текстовыми полями или редакторами, а функция PrepareEditCtrl вызывается для текстовых полей и редакторов

- `Fail` — вызывается после появления на экране окна сообщения об ошибке ввода. Данная процедура позволяет восстановить фокус ввода последнего элемента управления (последнее обращение к функциям `PrepareCtrl`/`PrepareEditCtrl`) и вызывает исключение. Эта функция может вызываться как в функциях `DDX_`, так и в функциях `DDV_`

Расширение пользователем возможностей данного механизма

Существует несколько способов расширить возможности механизма динамического обмена и проверки данных. Пользователь может:

- Добавлять новые типы данных

```
CTime
```

- Добавлять новые функции динамического обмена данными (`DDX_???`)

```
void PASCAL DDX_Time(CDataExchange* pDX, int nIDC, CTime& tm);
```

- Добавлять новые функции проверки корректности передаваемых данных (`DDV_???`)

```
void PASCAL DDV_TimeFuture(CDataExchange* pDX, CTime tm, BOOL
bFuture);
```

- Передавать в качестве аргументов функций проверки корректности передаваемых данных произвольные выражения

```
DDV_MinMax(pDX, age, 0, m_maxAge);
```

Эти произвольные выражения не могут редактироваться мастером ClassWizard и поэтому должны располагаться за пределами комментариев специального формата (`//{{AFX_DATA_MAP(CMyClass)`).

Функция `DoDialogExchange` может содержать условные операторы или любые другие выражения языка C++, изменяющие порядок вызова функций динамического обмена данными и проверки их корректности.

```
//{{AFX_DATA_MAP(CMyClass)
DDX_Check(pDX, IDC_SEX, m_bFemale);
DDX_Text(pDX, IDC_EDIT1, m_age);
//}}AFX_DATA_MAP
if (m_bFemale)
    DDV_MinMax(pDX, age, 0, m_maxFemaleAge);
else
    DDV_MinMax(pDX, age, 0, m_maxMaleAge);
```

Поскольку данные операторы не могут редактироваться мастером ClassWizard, они должны располагаться за пределами комментариев специального формата.

Использование мастера ClassWizard

Пользовательские процедуры динамического обмена данными и проверки их корректности можно включить в интерфейс мастера ClassWizard, хотя следует помнить, что это может привести к некоторым затруднениям при повторном использовании некоторых функций DDX и DDV в том же самом или в других проектах.

Для этого необходимо создать несколько специальных записей в файле ddx.clw (библиотека MSDN утверждает, что в Visual C++ версии 5.0 эта информация содержалась в файле apstudio.ini) или в файле с расширением .clw, расположенном в каталоге вашего проекта. Эти записи могут располагаться как в разделе `[General Info]` файла с расширением .clw, расположенного в каталоге проекта, или в разделе `[ExtraDDX]` файла ddx.clw, расположенного, по сведениям библиотеки MSDN, в каталоге \Program Files\Microsoft Visual Studio\Visual C++\bin. По всей видимости имеется в виду каталог \Program Files\Microsoft Visual Studio\VC98\bin.

Если с пользовательскими функциями `DDX_/DDV_` *предполагается работать* только в одном проекте, то соответствующие записи целесообразно поместить в раздел `[General Info]` файла с расширением .clw, расположенного в каталоге вашего проекта. В противном случае эти записи нужно поместить в раздел `[ExtraDDX]` файла ddx.clw. Если данный файл отсутствует, его необходимо создать.

Формат основных компонентов записи приведен ниже:

```
ExtraDDXCount=n
```

где `n` определяет количество строк формата `ExtraDDX?`, следующих за данной записью.

```
ExtraDDX?=<keys>;<vb-keys>; <prompt>; <type>; <initValue>;
        <DDX_Proc> [;<DDV_Proc>; <prompt1>; <arg1>; [<prompt2>; <fmt2>]]
```

где `?` представляет собой номер нового типа объекта DDX в списке. Нумерация начинается с 1.

Поля данной записи разделяются точкой с запятой. Ниже приводится описание полей этой записи.

```
<keys>
```

Данное поле состоит из знака равенства (=) и списка отдельных символов, определяющего, для каких элементов управления диалогового окна допустимо использование данного типа переменных:

- `E` — в текстовых полях и редакторах
- `C` — в переключателях, имеющих два состояния
- `c` — в переключателях, имеющих три состояния

- `R` — для первого переключателя в группе
- `L` — для несортированного списка
- `l` — для сортированного списка
- `M` — для раскрывающегося списка (с текстовым полем)
- `N` — для несортированного раскрывающегося списка (без текстового поля)
- `n` — для сортированного раскрывающегося списка (без текстового поля)
- `1` — если передаваемая в DDX величина должна помещаться в начало списка (по умолчанию она помещается в конец списка). Обычно этот ключ используется при передаче объектов классов элементов управления.

Поле `<vb-keys>` используется только в 16-разрядных версиях Windows для передачи элементов управления VBX. (32-разрядные версии Windows не используют элементов управления VBX.)

`<prompt> =`

Содержит строку, которую следует поместить в раскрывающийся список **Property:** (без кавычек)

`<type> =`

Содержит идентификатор типа, который следует использовать в файле заголовка.

`<initValue> =`

Содержит начальное значение — ноль или пробел. Если оно равно нулю, то в разделе `//{{AFX_DATA_INIT` файла реализации не будет строки для инициализации данной переменной. Это значение используется для инициализации объектов классов C++ (таких, как `CString`, `CTime` и других), имеющих конструкторы, обеспечивающие их корректную инициализацию.

`<DDX_Proc> =`

Содержит идентификатор функции `DDX_`. Имя данной функции должно начинаться с `DDX_`, но при задании этого имени в поле `<DDX_Proc>` префикс `DDX_` не должен включаться в идентификатор функции. Например, при задании в данном поле идентификатора функции `Time` мастер ClassWizard поместит в разделе `{{AFX_DATA_MAP` функции `DoDataExchange`, расположенной в файле реализации, вызов функции `DDX_Time`.

`<comment> =`

Содержит комментарий, который будет выводиться в диалоговом окне для данной переменной.

Раздел описания функции DDV не является обязательным. Не все функции DDX имеют соответствующую им функцию DDV. Часто бывает удобнее осуществлять проверку переменной в процессе ее передачи. Это приводит

к тому, что функции DDV не имеют аргументов, а мастер ClassWizard не поддерживает функции DDV без аргументов.

```
<DDV_Proc>
```

Данное поле состоит из знака равенства (=) и идентификатора функции DDV. Идентификатор функции DDV представляет собой имя данной функции без префикса `DDV_`.

За идентификатором функции DDV следует описание одного или двух аргументов данной функции:

```
<promptX> =
```

Содержит строку, указываемую над элементом управления текстового поля или редактора (с символом "&" перед подчеркиваемым символом).

```
<fmtX> =
```

Символ, определяющий формат аргумента. Может принимать одно из значений:

- `d` — целое
- `u` — целое число без знака
- `D` — длинное целое (long)
- `U` — длинное целое число без знака (DWORD)
- `f` — число с плавающей запятой
- `F` — число с плавающей запятой двойной точности
- `s` — строка

Интерфейс пользователя Visual C++

Microsoft Visual C++ является одним из компонентов среды разработки Visual Studio, все компоненты которой имеют единый интерфейс пользователя. Интерфейс Visual Studio представляет собой усовершенствованный интерфейс Developer Studio, использовавшийся в Visual C++ версии 5.0. Основным изменением, которому подвергся данный интерфейс при усовершенствовании, явилось исключение из него справочной системы, которая в среде Visual Studio представляет собой отдельное приложение, имеющее свой значок на панели задач. Сначала это несколько раздражает, но потом приходит понимание, что так даже лучше, поскольку обеспечивает простой и удобный способ переключения между окнами редактирования файлов и справочной системой.

Окно Visual Studio состоит из нескольких независимых окон, обычно отображаемых в виде панелей и панелей инструментов, являющихся, по сути, полноценными окнами. Список этих окон и панелей инструментов можно получить, щелкнув правой кнопкой мыши в любой отображаемой панели инструментов вне ее кнопок (лучше всего это сделать на вертикальной двойной линии, расположенной в левой части каждой панели инструментов, если она расположена горизонтально, или в ее верхней части, если она расположена вертикально). Этот список представляет собой контекстное меню, позволяющее выбирать окна и панели инструментов, отображаемые в данный момент на экране. Окно Visual Studio со стандартным набором окон и раскрытым всплывающим меню показано на рис. D.1.

Чтобы стандартные окна и панели инструментов перестали отображаться в виде панелей и превратились в полноценные окна, их необходимо переместить в центр окна Visual Studio. Однако в этом практически никогда не возникает необходимости.

Интерфейс пользователя Visual C++ состоит из окон, инструментальных средств, меню, панелей инструментов и других элементов, позволяющих

использовать данную среду программирования для создания и отладки приложений. Кроме того, Visual C++ позволяет работать и с документами других приложений, таких как Microsoft Excel или Microsoft Word. Для этого используется технология ActiveX.

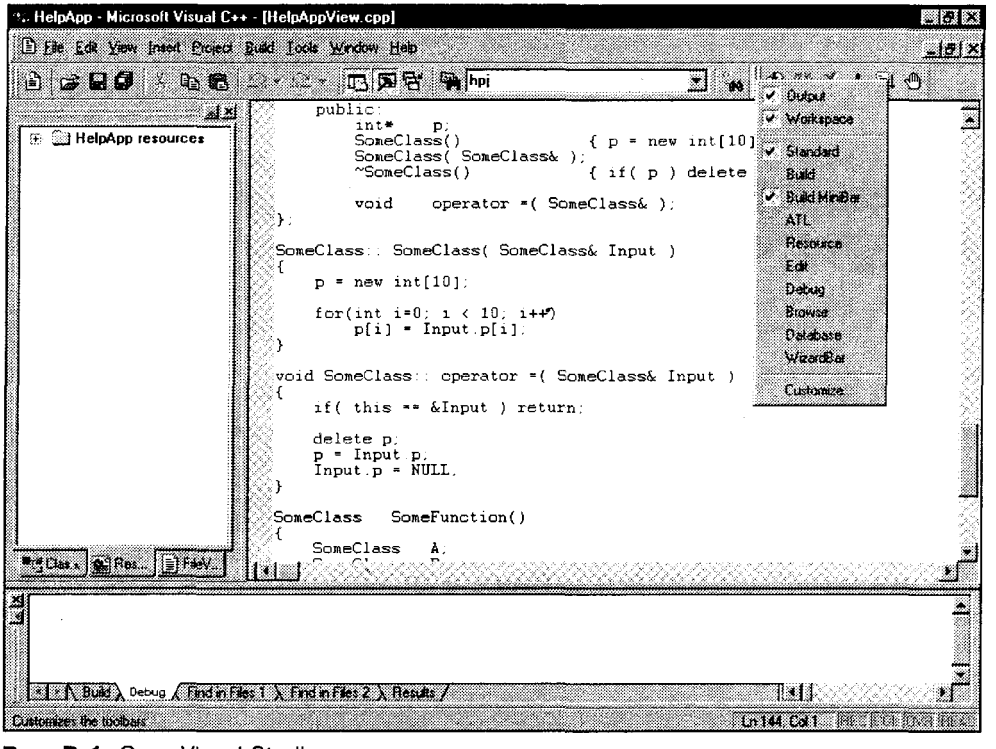


Рис. D.1. Окно Visual Studio

Интерфейс Visual C++ можно настроить в соответствии с предпочтениями пользователя. Предусмотрено создание собственных панелей инструментов, меню и комбинаций клавиш. Однако этим не стоит злоупотреблять, поскольку, привыкнув к нестандартной конфигурации, вы будете испытывать серьезные затруднения, работая со стандартными панелями на других машинах, что может существенно повредить вашей репутации программиста.

Панели инструментов

При первом запуске Visual C++ со стандартной конфигурацией панелей инструментов непосредственно под строкой меню отображается стандартная панель инструментов. Она содержит кнопки, необходимые для работы практически в любом режиме функционирования Visual C++. В редакторе ресурсов, как правило, на экран выводятся дополнительные панели инструментов.

Конкретный набор панелей инструментов, отображаемых на экране, выбирается пользователем исходя из собственных предпочтений и разрешения дисплея. Пользователь может также выбрать содержимое каждой панели инструментов, убирая или добавляя к ним любые кнопки как из стандартной панели инструментов, так и из любого другого набора.

Чтобы выбрать набор панелей инструментов, отображаемых на экране:

1. Щелкните правой кнопкой мыши в любой выведенной на экран панели инструментов за пределами ее кнопок.
2. В появившемся контекстном меню установите или сбросьте флажок у соответствующей панели инструментов.
3. Повторите пп. 1 и 2 для других панелей инструментов.

или

1. Выберите команду меню **Tools**, **Customize** (Сервис, Настройка). Появится диалоговое окно **Customize** (Настройка).
2. Перейдите на вкладку **Toolbars** (Панели инструментов). Диалоговое окно **Customize** примет вид, изображенный на рис. D.2.
3. Установите или сбросьте флажки в окне списка **Toolbars:**. При установке или сбросе каждого флажка в окне будет появляться или исчезать соответствующая панель инструментов.
4. После установки нужной конфигурации нажмите кнопку **Close**.

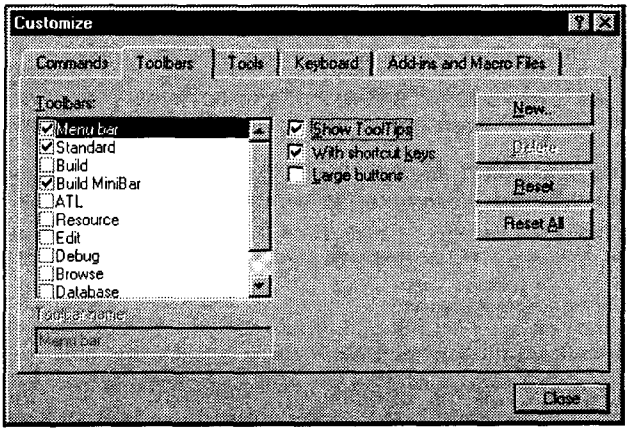


Рис. D.2. Вкладка **Toolbars** диалогового окна **Customize**

Чтобы добавить или удалить кнопку в панели инструментов:

1. Установите требуемый набор панелей инструментов, как это описано выше.
2. Выберите команду меню **Tools**, **Customize** (Сервис, Настройка). Появится диалоговое окно **Customize** (Настройка).

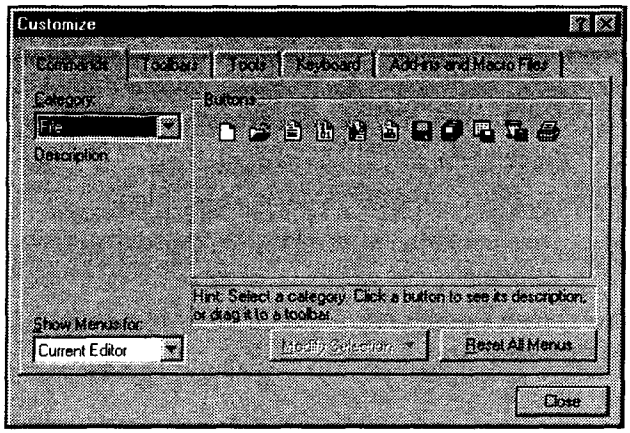


Рис. D.3. Диалоговое окно **Customize**, вкладка **Commands**

3. Перейдите на вкладку **Commands** (Команды). Диалоговое окно **Customize** примет вид, изображенный на рис. D.3.
4. Поместите указатель мыши на кнопку панели инструментов, которую необходимо удалить.
5. Мышью перетащите эту кнопку за пределы панели инструментов.
6. Отпустите левую кнопку мыши. Данная кнопка будет удалена из панели инструментов.
7. Поместите указатель мыши на кнопку панели инструментов, которую необходимо переместить в другую панель инструментов.
8. Мышью перетащите эту кнопку в другую панель инструментов.
9. Отпустите левую кнопку мыши. Данная кнопка переместится в другую панель инструментов.
10. В раскрывающемся списке **Category:** (Категория) выберите категорию, к которой относится новая кнопка, помещаемая в панель инструментов. (Категория обычно соответствует меню, в котором расположена данная команда или панели инструментов, уже содержащей данную кнопку.)
11. Нужная кнопка появится в группе **Buttons**.
12. Поместите указатель мыши на значок новой кнопки в группе **Buttons**.
13. Мышью перетащите эту кнопку в ту панель инструментов, где она должна находиться.
14. Отпустите левую кнопку мыши. Данная кнопка появится в панели инструментов.
15. Установив нужную конфигурацию, нажмите кнопку **Close**.

На рис. D.4 приведены все панели инструментов, перечисленные в соответствующем контекстном меню и в окне списка **Toolbars:** диалогового окна

Customize. Все панели инструментов выведены как отдельные окна, имеющие свой заголовок, что позволяет разобраться, какая из панелей инструментов содержит данные кнопки.

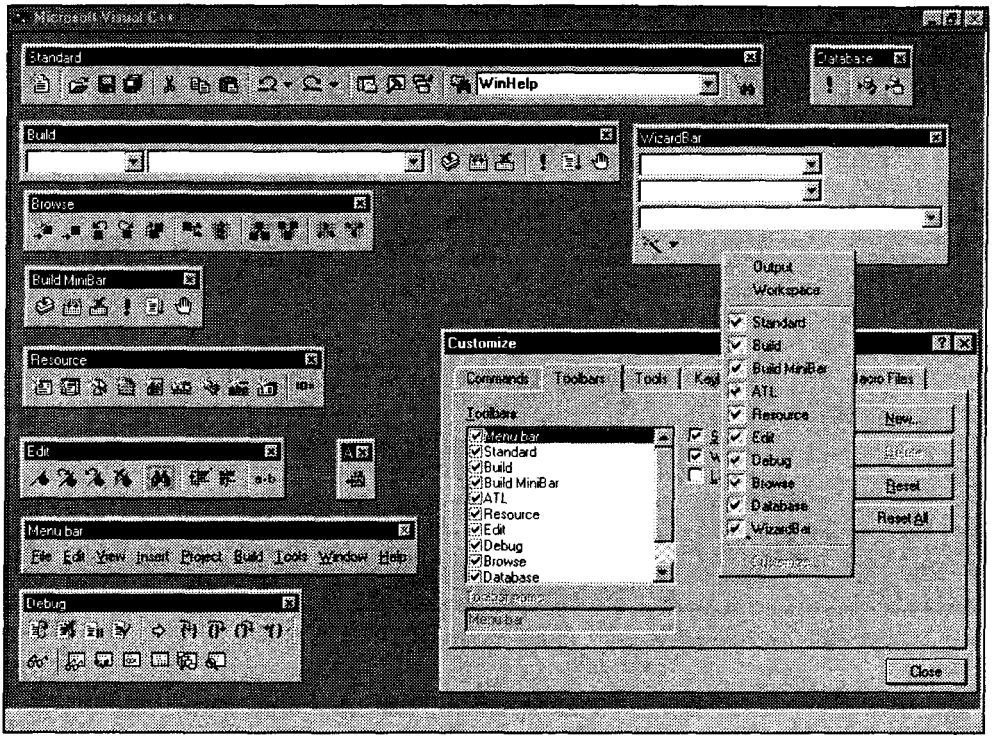


Рис. D.4. Плавающие панели инструментов

Любая из панелей инструментов может быть зафиксирована на любой из границ рабочей области, как показано на рис. D.5.

Для перемещения зафиксированной панели инструментов необходимо поместить указатель мыши на ее *бордюр* (wrinkles), представляющий собой две параллельные полоски на одном из ее краев, и, нажав левую кнопку мыши, переместить ее в новое положение. Если панель инструментов находится далеко от границ рабочей области, она становится *плавающей*, то есть превращается в обычное окно, как показано на рис. D.4. Приближая перемещаемую панель к границе рабочей области, можно наблюдать изменение формы ее рамки. Это указывает, что панель будет зафиксирована.

При установке зафиксированных панелей для устранения промежутков между ними рекомендуется "наезжать" устанавливаемой панелью инструментов на уже установленную. При этом Visual Studio сама позаботится об устранении перекрытия панелей и установит их встык.

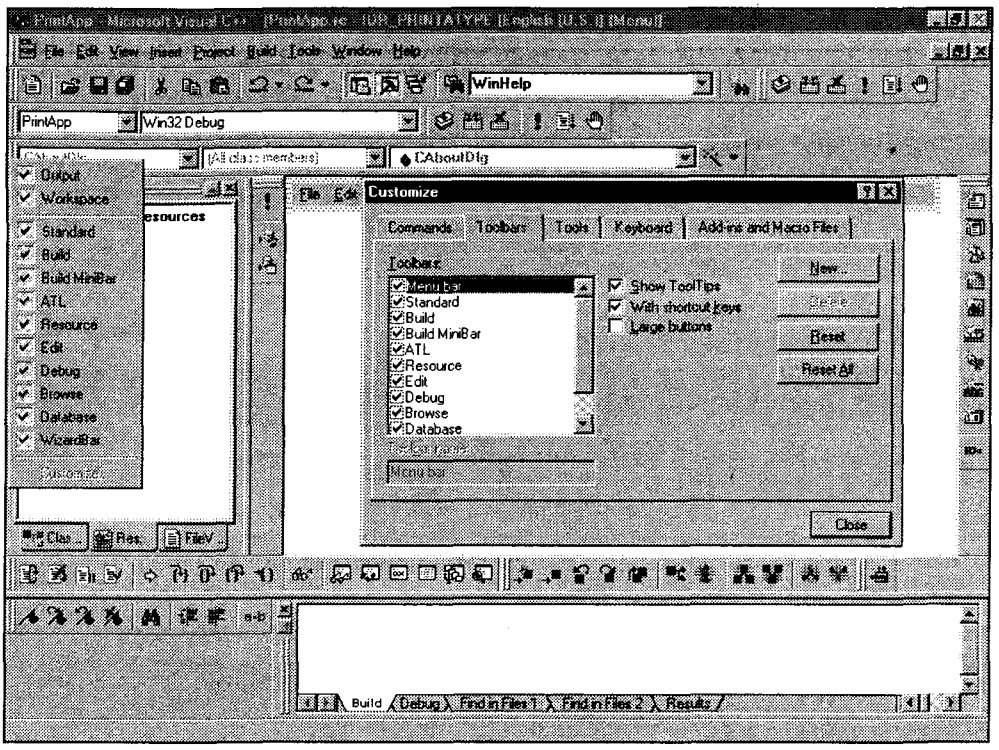


Рис. D.5. Зафиксированные панели инструментов

Наиболее часто пользователю придется работать с панелями инструментов **Standard** и **Build**.

Панель инструментов *Standard*

Из стандартной панели инструментов пользователь может вызывать, редактировать и сохранять файлы. Кнопки данной панели инструментов дублируются командами меню. В табл. D.1 приведено соответствие кнопок панели инструментов **Standard** командам меню.

Таблица D.1. *Соответствие кнопок панели инструментов* ***Standard*** *командам меню*

Кнопка панели инструментов	Команда меню
New Text File	File, New...
Open (Ctrl+O)	File, Open...
Save (Ctrl+S)	File, Save

Таблица D.1 (окончание)

Кнопка панели инструментов	Команда меню
Save All	File, Save All
Cut (Ctrl+X)	Edit, Cut
Copy (Ctrl+C)	Edit, Copy
Paste (Ctrl+V)	Edit, Paste
Undo (Ctrl+Z)	Edit, Undo
Redo (Ctrl+Y)	Edit, Redo
Workspace	View, Workspace
Output	View, Output
Window List	Window, Windows...
Find in Files	Edit, Find in Files...
Find (Ctrl+D)	Edit, Find...
Search	Help, Search...

Раскрывающийся список **Find**, строго говоря, не является кнопкой, и его соответствие команде меню **Edit**, **Find...** тоже не является абсолютным, что подтверждается и различными комбинациями клавиш, соответствующими раскрывающемуся списку и команде меню. Однако при выделении текста, расположенного в текстовом поле данного раскрывающегося списка (для чего может быть использована комбинация клавиш <Ctrl>+<D>), и нажатии клавиши <Enter> в тексте редактируемого файла выполняется поиск первого вхождения данной строки, которое выделяется блоком.

Панель инструментов *Build*

С помощью панели инструментов **Build** пользователь может компилировать активный проект. В табл. D.2 приведено соответствие кнопок панели инструментов **Build** командам меню.

*Таблица D.2. Соответствие кнопок панели инструментов **Build** командам меню*

Кнопка панели инструментов	Команда меню
Compile (Ctrl+F7)	Build, Compile
Build (F7)	Build, Build
Stop Build (Ctrl+Break)	Отсутствует

Таблица D.2 (окончание)

Кнопка панели инструментов	Команда меню
Execute Program (Ctrl+F5)	Build, Execute
Go (F5)	Отсутствует
Insert/Remove Breakpoint (F9)	Отсутствует

Приведенные выше кнопки содержатся также и в панели инструментов **Build MiniBar**, использование которой целесообразно, если пользователь ценит свободное место на экране дисплея. Отличие панели инструментов **Build** от панели инструментов **Build MiniBar** состоит в том, что первая из них содержит дополнительно раскрывающиеся списки **Set Active Project (Ctrl+Alt+P)**, позволяющий выбирать активный проект в данном приложении, и **Set Active Configuration**, позволяющего выбрать версию создаваемого приложения (отладочную или окончательную). В большинстве случаев пользователь работает с одним проектом в приложении и редко изменяет версию создаваемого приложения.

Система меню

Среда разработки Visual C++ имеет достаточно сложную систему меню. Некоторые команды находятся на третьем или даже четвертом уровне иерархии. Обычно этим командам соответствует некоторая комбинация клавиш, позволяющая вызвать их значительно быстрее, чем при помощи меню. Конечно, не каждый пользователь станет запоминать несколько десятков комбинаций клавиш, большую часть которых он будет использовать не чаще, чем раз в месяц.

Примечание

Однако разработчики Visual C++ были убеждены в обратном и установили такое количество комбинаций клавиш, что практически любое ошибочное нажатие клавиш вызывает какое-нибудь диалоговое окно, что не дает пользователям данной среды программирования забыть хорошую русскую пословицу: "Заставь дурака богу молиться, он и лоб себе разобьет".

Строка меню Visual C++ содержит девять заголовков, которые перечислены ниже:

- **File** — меню операций, связанных с файлами, включая их создание, открытие, закрытие и печать
- **Edit** — меню операций редактирования (копирование, удаление, восстановление измененной информации, поиск и перемещение по файлу)
- **View** — меню управления способом отображения информации на экране, включая управление отображением панелей инструментов и окон среды разработки **Output** и **Workspace**

- **Insert** — меню команд включения файлов или их фрагментов в активный проект
- **Project** — меню команд управления проектом в целом
- **Build** — меню команд компиляции, компоновки и отладки
- **Tools** — меню команд настройки Visual C++ и доступа к автономным утилитам
- **Window** — меню выбора окон и их автоматического упорядочения на экране
- **Help** — меню команд обращения к справочной системе Visual C++

При запуске отладочной версии приложения на исполнение вместо меню **Build** появляется меню **Debug**.

Ниже будут рассмотрены все перечисленные выше меню и описаны их основные команды.

Меню *File*

Меню **File**, приведенное на рис. D.6, содержит команды, осуществляющие операции, связанные с файлами, включая их создание, открытие, закрытие и печать.

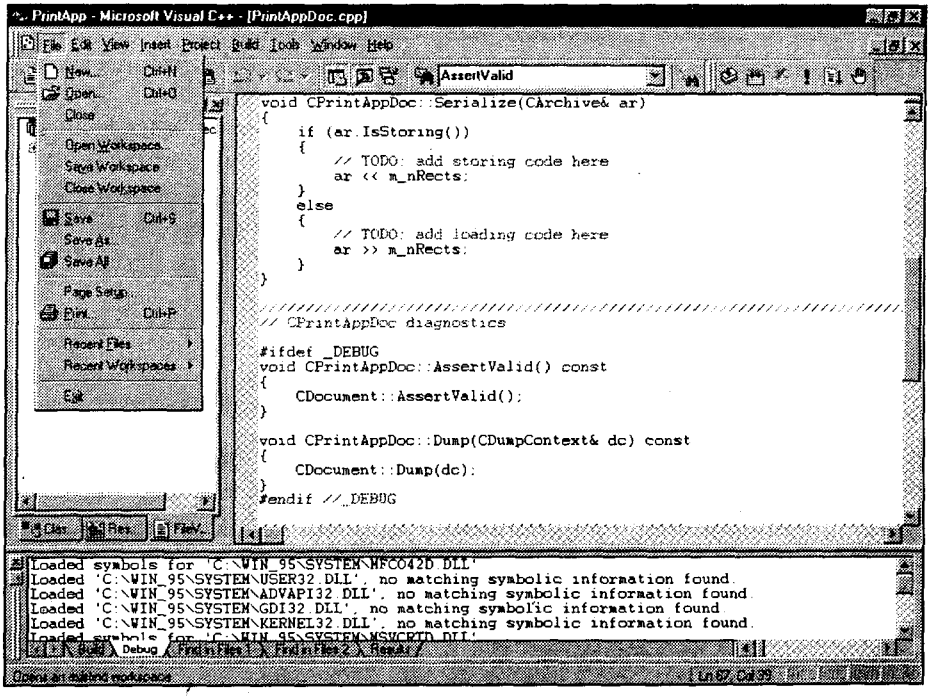


Рис. D.6. Меню **File**

Команда *File, New* (<Ctrl>+<N>)

Выбор данной команды приводит к появлению диалогового окна **New**, изображенного на рис. D.7.

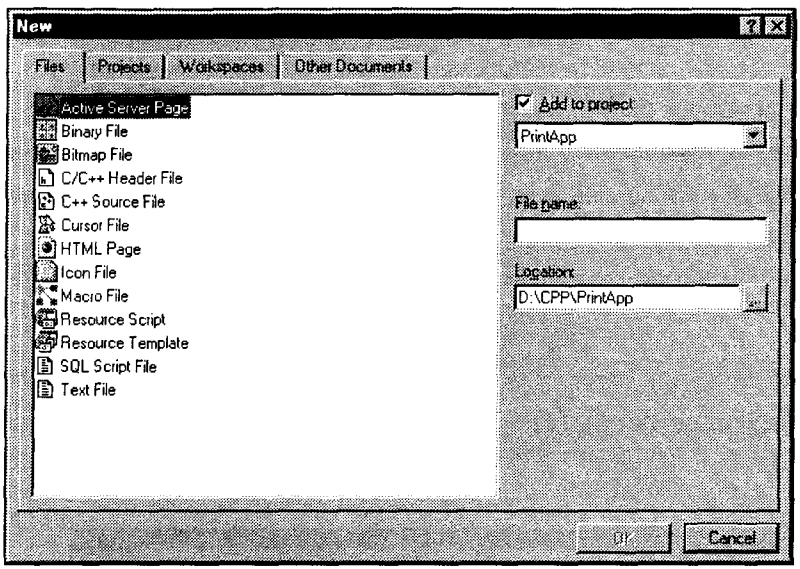


Рис. D.7. Диалоговое окно **New**

Это окно используется для создания новых файлов, проектов, описаний рабочей среды или других документов. Вкладка **Files** данного диалогового окна позволяет одновременно создать пустой файл, присвоить ему имя и включить его в активный проект.

Вкладка **Projects** данного диалогового окна используется для создания новых проектов.

См. также главу 1

Команда *File, Open* (<Ctrl>+<O>)

Выбор данной команды приводит к появлению диалогового окна **Open**, изображенного на рис. D.8.

Это стандартное диалоговое окно **Windows File Open**, используемое практически во всех приложениях Windows. В раскрывающемся списке **Тип файлов:** указана стандартная комбинация расширений файлов, отображаемых в окне списка данного диалогового окна. Чтобы открыть файл, имеющий расширение, отличное от перечисленных в этом списке, введите нужное вам расширение вручную. При этом соответствующим образом изменится содержимое данного диалогового окна.

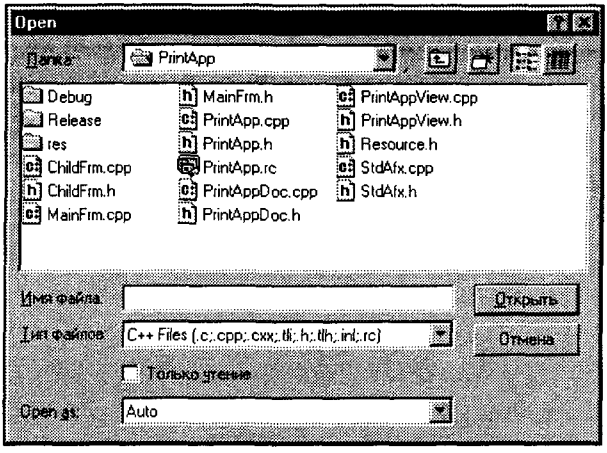


Рис. D.8. *Диалоговое окно* **Open**

Раскрывающийся список **Open as:** позволяет изменить режим открытия файла. Практически единственным случаем, когда следует воспользоваться данным раскрывающимся списком является открытие файла ресурсов в текстовом режиме. В этом случае в данном раскрывающемся списке выбирается значение **Text**.

Команда *File, Close*

Выбор данной команды меню приводит к закрытию файла, окно которого имеет на данный момент фокус ввода. Если таких файлов нет, то данная команда становится недоступной. Закрыть файл и связанное с ним окно также можно, нажав кнопку (Закрыть).

Команда *File, Open Workspace*

Выбор данной команды меню позволяет открыть существующий проект. При этом появится диалоговое окно **Open Workspace**, изображенное на рис. D.9.

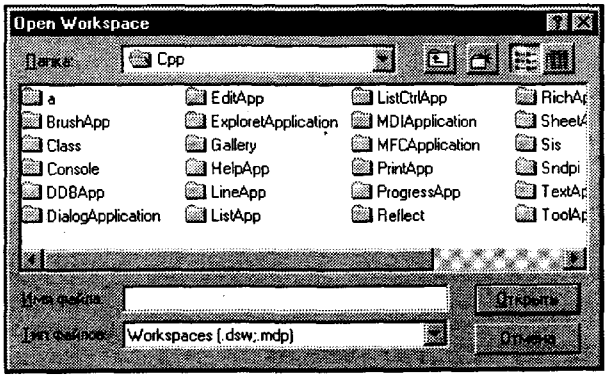


Рис. D.9. Диалоговое окно **Open Workspace**

Раскройте папку, содержащую нужный проект, выделите файл проекта и нажмите кнопку **Открыть**. Если текущий проект содержал несохраненную информацию, на дисплее появится соответствующее окно сообщения. После этого в среду программирования Visual C++ будет загружен новый проект.

Команда *File, Save Workspace*

Выбор данной команды меню позволяет сохранить изменения, внесенные в текущий проект. Сам проект при этом не закрывается.

Команда *File, Close Workspace*

Аналогична команде меню **File**, **Save Workspace**, но приводит к закрытию текущего проекта. Если текущий проект содержал несохраненную информацию, на дисплее появится соответствующее окно сообщения.

Команда *File, Save* (<Ctrl>+<S>)

Выбор данной команды меню позволяет сохранить в файл изменения, внесенные в окне, которое имеет на настоящий момент фокус ввода. Если с окном, имеющим в настоящий момент фокус ввода, не связан никакой файл, то вместо данной команды автоматически вызывается команда **File**, **Save As**. Если в этот момент ни одно из окон редактирования файлов не имеет фокус ввода, то данная команда меню и соответствующая ей кнопка панели инструментов становятся недоступными.

Команда *File, Save As*

Выбор этой команды позволяет сохранить изменения, внесенные в окно, которое имеет на настоящий момент фокус ввода, в новом файле, сохранив содержимое файла, с которым было связано данное окно, без изменений. При выборе этой команды появляется диалоговое окно **Save As**, изображенное на рис. D.10.

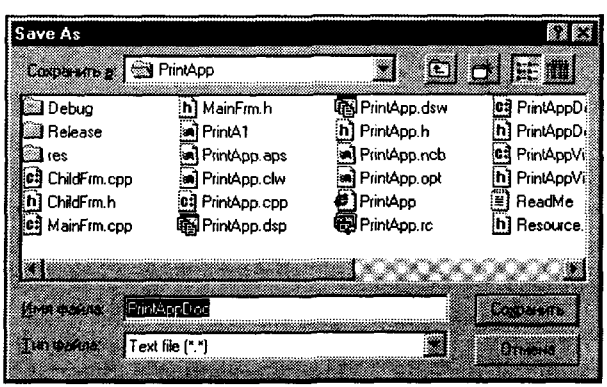


Рис. D.10. Диалоговое окно **Save As**

Если в данный момент ни одно из окон редактирования файлов не имеет фокус ввода, то данная команда меню и соответствующая ей кнопка панели инструментов становятся недоступными.

Команда *File, Save All*

Эта команда позволяет сохранить изменения, внесенные во все открытые файлы данного проекта. Рекомендуется выбирать эту команду каждые 10—15 минут, если содержимое этих файлов не сохранялось другим способом. По умолчанию все файлы проекта сохраняются при его компиляции.

Команда *File, Page Setup*

Эта команда выводит на экран диалоговое окно **Page Setup**, изображенное на рис. D.11.

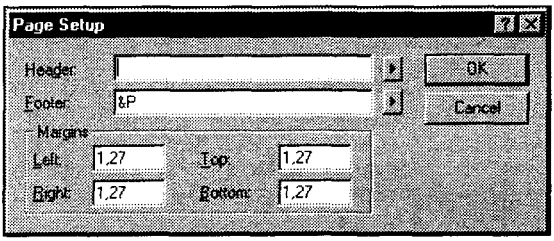


Рис. D.11. *Диалоговое окно* **Page Setup**

В текстовое поле **Header:** данного диалогового окна вводится верхний колонтитул, а в текстовое поле **Footer:** — нижний колонтитул. В группе текстовых полей **Margins** определяется размер пустых полей при печати листа. Текстовое поле **Left:** определяет пустое поле слева, текстовое поле **Right:** — пустое поле справа, текстовое поле **Top:** определяет пустое поле сверху, а текстовое поле **Bottom:** — пустое поле снизу листа.

Верхний и нижний колонтитулы могут содержать любой текст, включая специальные поля, для ввода которых нужно нажать на кнопку со стрелкой, расположенной около соответствующего текстового поля. При этом появится контекстное меню, изображенное на рис. D.12.

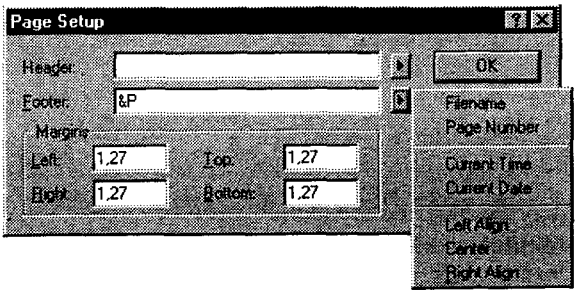


Рис. D.12. Контекстное меню вставки специальных полей

Пользователь может самостоятельно ввести коды полей. Ниже приведен список специальных полей с соответствующими им кодами:

- Имя распечатываемого файла (`&f`)
- Номер текущей страницы (`&p`)
- Время печати страницы (`&t`)
- Дата печати страницы (`&d`)
- Выравнивание по левому краю (`&l`)
- Выравнивание по правому краю (`&r`)
- Выравнивание по центру (`&c`) (выбирается по умолчанию)

Команда *File, Print* (<Ctrl+P>)

Выбор этой команды инициализирует процесс печати содержимого окна, которое на данный момент имеет фокус ввода. Если в данный момент ни одно из окон редактирования файлов не имеет фокус ввода, то данная команда становится недоступной.

Непосредственно после выбора данной команды появляется диалоговое окно **Print**, изображенное на рис. D.13.

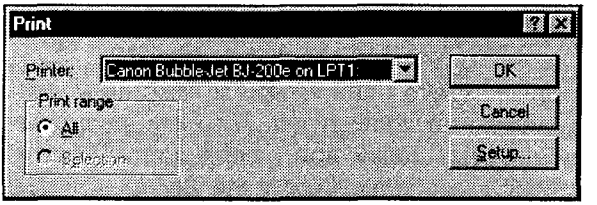


Рис. D.13. Диалоговое окно **Print**

В раскрывающемся списке **Printer:** может быть выбран принтер, на котором будет произведена распечатка.

Группа переключателей **Print range** (Диапазон печати) позволяет определить диапазон печати. Если в активном окне выделен блок и установлен переключатель **Selection**, то печатается только выделенный текст. Если установлен переключатель **All**, печатается весь файл.

Нажатие кнопки **Setup...** вызывает стандартное диалоговое окно **Свойства**, позволяющее настроить используемый для печати принтер. Для печати документа нажмите кнопку **OK**.

Команда *File, Recent Files*

Эта команда открывает контекстное меню, содержащее список последних закрытых файлов. Выбор имени файла в данном меню позволяет открыть его окно редактирования.

Команда *File, Recent Workspaces*

Эта команда раскрывает контекстное меню, содержащее список последних открытых в данной системе программирования проектов. Под номером **1** всегда стоит активный проект. Выбор имени проекта в данном меню позволяет открыть его для работы вместо проекта, активного в данный момент.

Команда *File, Exit*

Данная команда завершает работу в среде программирования Visual C++.

Меню *Edit*

Меню **Edit**, приведенное на рис. D.14, содержит команды, осуществляющие редактирование (копирование, удаление, восстановление измененной информации), поиск и перемещение по файлу.

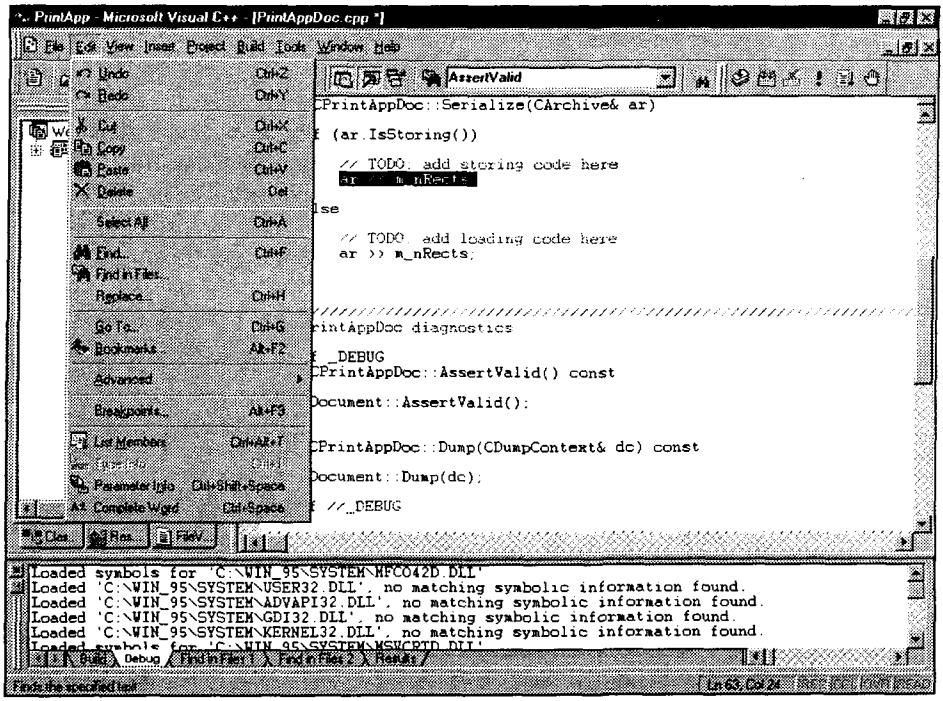


Рис. D.14. Меню **Edit**

Команда *Edit, Undo* (<Ctrl>+<Z>)

Эта команда отменяет операции редактирования, выполненные ранее. Большая часть операций, например, редактирование и удаление текста, мо-

гут отменяться. Если данная команда меню недоступна, значит, либо пользователь не произвел ни одной операции редактирования с момента последнего сохранения файла, либо все операции, которые могут быть отменены уже отменены, либо последняя операция редактирования не может быть отменена.

В панели инструментов **Standard** рядом с кнопкой (Undo) имеется кнопка , раскрывающая стек операций (список в обратном порядке от самых свежих до самых поздних операций редактирования, которые могут быть отменены). Раскрытый стек операций показан на рис. D.15. При отмене операций с использованием стека необходимо выделить последнюю отменяемую операцию в списке. Все операции, расположенные в стеке выше выделенной операции, и сама выделенная операция будут отменены.

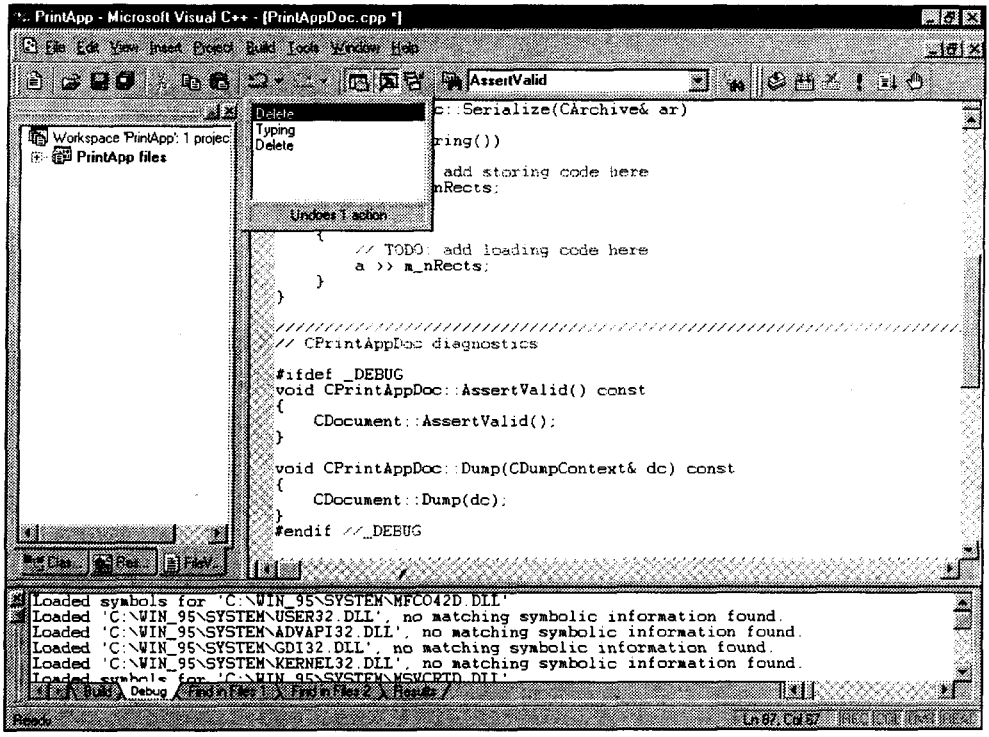


Рис. D.15. Раскрытый стек операций

Команда *Edit, Redo* (<Ctrl>+<Y>)

Как только пользователь отменил некоторую операцию с использованием команды **Undo**, становится доступной команда **Redo**. Эта команда служит для отмены действия команды **Undo**. Около кнопки (Redo) в панели ин-

струментов **Standard** также имеется кнопка , раскрывающая стек, в который записаны все операции, отмененные командой **Undo**.

Команда *Edit, Cut* (<Ctrl>+<X>)

Данная команда переписывает содержимое выделенного фрагмента в буфер обмена (Clipboard) и удаляет этот фрагмент из исходного текста. Если выделенный фрагмент в активном окне отсутствует, то данная команда меню и соответствующая ей кнопка панели инструментов становятся недоступными.

Команда *Edit, Copy* (<Ctrl>+<C>)

Эта команда копирует выделенный фрагмент в буфер обмена (Clipboard). В исходный файл не вносятся никакие изменения. Если выделенный фрагмент в активном окне отсутствует, то данная команда меню и соответствующая ей кнопка панели инструментов становятся недоступными.

Команда *Edit, Paste* (<Ctrl>+<V>)

Эта команда копирует содержимое буфера обмена (Clipboard) в окно, которому принадлежит в настоящее время фокус ввода, в позицию, определяемую текстовым курсором. Если в окне имеется выделенный фрагмент, то его содержимое заменяется на содержимое буфера обмена и выделение снимается. Если в данный момент буфер обмена не содержит никакой информации, то данная команда меню и соответствующая ей кнопка панели инструментов становятся недоступными.

Команда *Edit, Delete* (<Del>)

Команда удаляет выделенный текст или элемент. Если операция удаления восстановима, то эта операция добавляется в стек операций кнопки **Undo** в панели инструментов **Standard**.

Команда *Edit, Select All* (<Ctrl+A>)

Выделяет весь текст файла, связанного с окном, имеющим в настоящее время фокус ввода. В окне редактирования текста программы выделяется весь текст. В окне редактирования ресурса диалогового окна выделяются все элементы управления данного диалогового окна.

Для выделения элементов управления диалогового окна можно, удерживая клавишу <Ctrl>, выделить мышью требуемые элементы управления или выбрать команду меню **Edit**, **Select All** и, удерживая клавишу <Ctrl>, мышью снять выделение с элементов управления.

Команда *Edit, Find* (<Ctrl>+<F>)

Выводит на экран диалоговое окно **Find**, изображенное на рис. D.16.

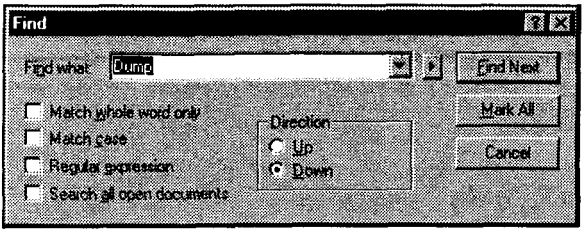


Рис. D.16. Диалоговое окно **Find**

Это диалоговое окно используется для поиска текста внутри файла, соответствующего окну, имеющему в настоящее время фокус ввода. Искомый текст помещается в текстовое поле раскрывающегося списка **Find what:**, в котором хранятся фрагменты текста, по которым производился поиск в последнее время. Если искомый фрагмент текста содержится в раскрывающемся списке, его можно выбрать, а не вводить заново.

Переключатели группы **Direction** определяют направление поиска от текущей позиции текстового курсора. Если выбран переключатель **Up**, то поиск будет производиться по направлению к началу файла. Если же выбран переключатель **Down**, то поиск будет производиться по направлению к концу файла.

В диалоговом окне **Find** могут быть установлены следующие флажки:

- **Match whole word only** — устанавливает режим, при котором текст в текстовом поле раскрывающегося списка **Find what:** рассматривается при сравнении как целое слово, и при поиске будут пропущены слова, в которых данный текст является составной частью другого слова
- **Match case** — при сравнении учитывается регистр символов, используемый при написании текста в текстовом поле раскрывающегося списка **Find what:**. Если этот флажок сброшен, то при сравнении регистр символов не учитывается, что удобно при поиске идентификаторов, содержащих заглавные буквы в середине слова
- **Regular expression** — при поиске содержимое текстового поля раскрывающегося списка **Find what:** рассматривается как регулярное выражение. Под регулярным выражением понимается некоторый текст, включающий в себя специальные символы, такие как конец строки, любое число или другие символы. Для ввода специального символа достаточно нажать на кнопку со стрелкой, расположенную справа от текстового поля раскрывающегося списка **Find what:**, и выбрать соответствующую команду в появившемся контекстном меню, изображенном на рис. D.17.
- **Search all open documents** — при установке данного флажка поиск производится во всех открытых на данный момент документах

Обычно при работе с диалоговым окном **Find** в текстовое поле раскрывающегося списка **Find what:** вводится некоторый текст и нажимается кнопка **Find Next** (Найти следующий). Диалоговое окно **Find** закрывается и в окне

редактирования файла выделяется следующее вхождение данного текста. Для поиска нужной строки в тексте надо последовательно открывать диалоговое окно **Find** и нажимать в нем кнопку **Find Next**. Эту процедуру нельзя назвать очень удобной.

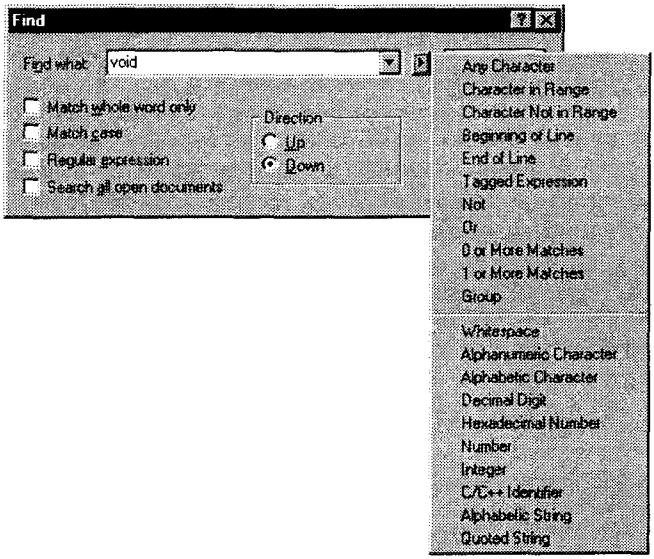


Рис. D.17. Контекстное меню ввода специальных символов

Другой возможностью поиска текста в файле является использование кнопки **Mark All**. В этом случае у каждой строки, содержащей искомый текст, будет поставлена закладка. После этого можно просмотреть все строки, нажимая клавишу <F2>.

После первого же поиска с использованием диалогового окна **Find** в текстовом поле раскрывающегося списка **Find (Ctrl+D)**, расположенного в панели инструментов **Standard**, появится текст, введенный в текстовое поле раскрывающегося списка **Find what:**. При выделении данного текста и нажатии клавиши <Enter> будет выполнен поиск в направлении к концу файла.

Однако наиболее удобным способом поиска информации является использование команд меню **Edit**, **Find in Files** и **Edit**, **Replace**, описанных ниже.

Команда *Edit, Find in Files*

Эта команда выводит на экран диалоговое окно **Find In Files**, изображенное на рис. D.18.

Это диалоговое окно используется для поиска текста во всех файлах проекта, расширения которых указаны в текстовом поле раскрывающегося списка **In files/file types:**. Как и в диалоговом окне **Find**, искомый текст помещается в текстовое поле раскрывающегося списка **Find what:**. Работа с ним аналогична работе с данным раскрывающимся списком в диалоговом окне **Find**.

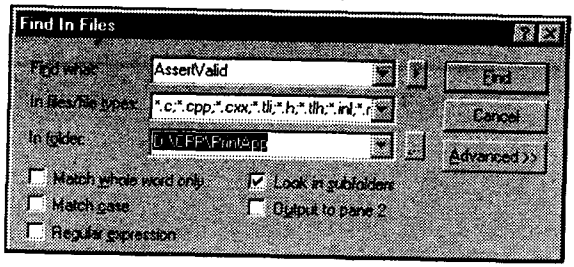


Рис. D.18. Диалоговое окно **Find In Files**

В текстовом поле раскрывающегося списка **In folder:** указывается каталог, в котором необходимо произвести поиск. Нужный каталог может быть выбран в раскрывающемся списке, введен в текстовое поле вручную или выбран в диалоговом окне **Choose Directory**, изображенном на рис. D.19. Для вывода данного диалогового окна нажмите кнопку с тремя точками, расположенную справа от текстового поля раскрывающегося списка **In folder:**.

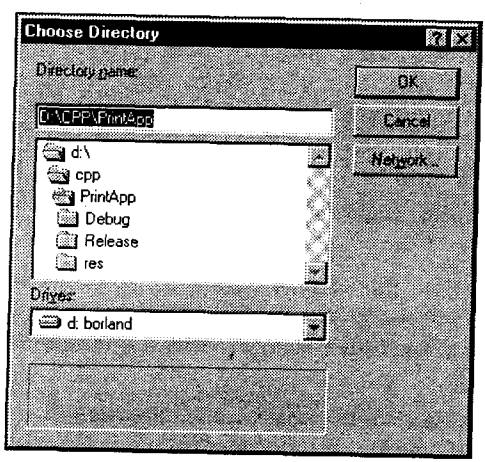


Рис. D.19. Диалоговое окно **Choose Directory**

В диалоговом окне **Find In Files** могут быть установлены следующие флажки:

- **Match whole word only** — устанавливает режим, при котором текст в текстовом поле раскрывающегося списка **Find what:** рассматривается при сравнении как целое слово, и при поиске будут пропущены слова, в которых данный текст является составной частью другого слова
- **Match case** — при сравнении учитывается регистр символов, используемый при написании текста в текстовом поле раскрывающегося списка **Find what:**. Если этот флажок сброшен, то при сравнении регистр символов не учитывается, что удобно при поиске идентификаторов, содержащих заглавные буквы в середине слова
- **Regular expression** — при поиске содержимое текстового поля раскрывающегося списка **Find what:** рассматривается как регулярное выражение. Работа с регулярным выражением рассмотрена в описании команды меню **Edit, Find**

- **Look in subfolders** — при поиске просматриваются файлы, расположенные в подкаталогах указанной папки
- **Output to pane 2** — собранная информация отображается во вкладке **Find in Files 2** окна **Output**

После ввода всей необходимой информации и нажатия кнопки **Find** среда программирования Visual C++ выполняет поиск в файлах данного проекта и выводит его результаты в окно **Output**, как показано на рис. D.20. Если окно **Output** было до этого скрыто, то оно будет выведено на экран. Если в диалоговом окне **Find In Files** не установлен флажок **Output to pane 2**, результаты поиска будут отображаться во вкладке **Find in Files 1** данного окна. В противном случае результаты поиска будут отображаться во вкладке **Find in Files 2**.

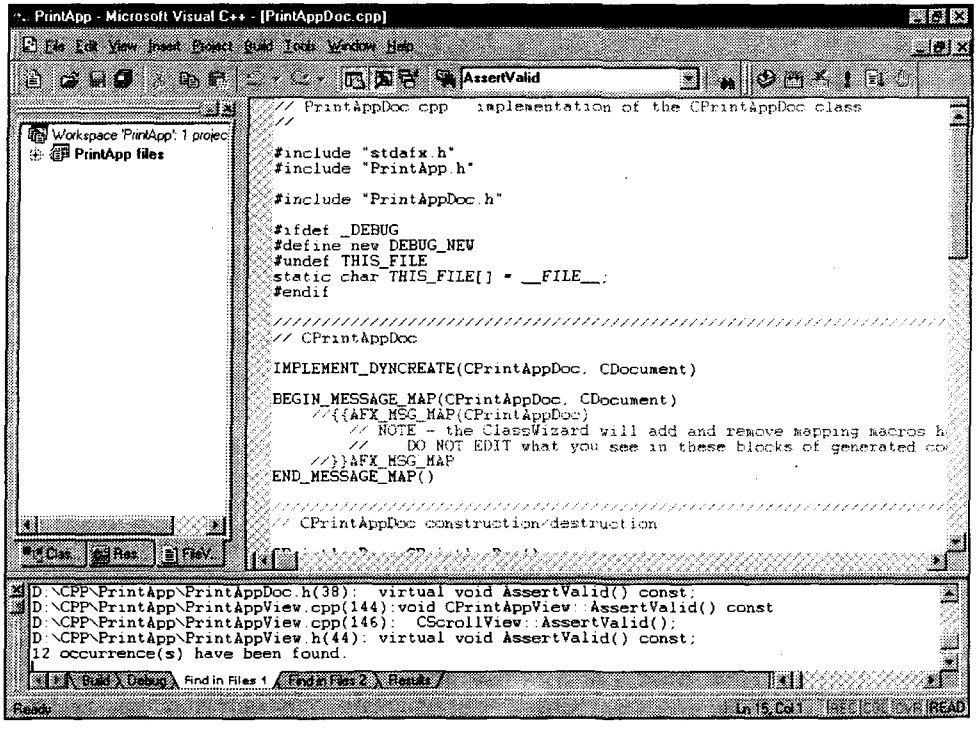


Рис. D.20. Результат поиска в файлах проекта

Чтобы перейти к найденной строке в файле, достаточно дважды щелкнуть на соответствующей ей строке в окне **Output**. При этом окно, содержащее данную строку, получит фокус ввода, и текстовый курсор переместится в начало этой строки в окне редактирования. Если соответствующее окно редактирования не было открыто, оно автоматически открывается в текстовом режиме. Поэтому поиск в файлах может использоваться для открытия файла

ресурсов в текстовом режиме. Для этого нужно произвести поиск по одному из идентификаторов ресурсов, а затем дважды щелкнуть левой кнопкой мыши на строке, расположенной в файле ресурсов.

Команда *Edit, Replace* (<Ctrl>+<H>)

Команда выводит на экран диалоговое окно **Replace**, изображенное на рис. D.21.

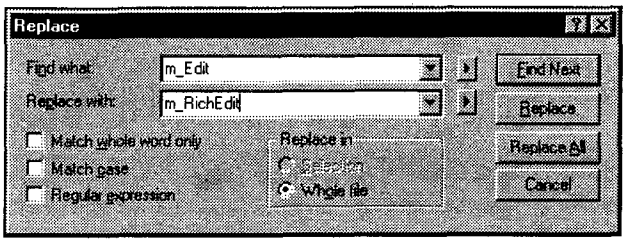


Рис. D.21. Диалоговое окно **Replace**

Это диалоговое окно, во многом аналогичное диалоговому окну **Find**, используется для замены найденного текста новым. Работа с текстовым полем раскрывающегося списка **Find what:** и флажками **Match whole word only**, **Match case** и **Regular expression** в диалоговом окне **Replace** ничем не отличается от использования этих элементов управления в диалоговых окнах **Find** и **Find In Files**. Основным преимуществом диалогового окна **Replace** перед диалоговым окном **Find** является то, что при нажатии кнопки **Find Next** окно **Replace** не закрывается, что позволяет существенно упростить процедуру поиска информации в файле.

Группа переключателей **Replace in** используется в том случае, если в окне, имеющем в настоящее время фокус ввода, имеется выделенный фрагмент. В этом случае при установке переключателя **Selection** поиск и замена текста будет производиться только в выделенном фрагменте. При установке переключателя **Whole file** поиск и замена текста будут производиться во всем файле, связанном с активным окном.

В текстовом поле раскрывающегося списка **Replace with:** помещается текст, на который следует заменить искомый текст. Как и в случае с раскрывающимся списком **Find what:**, в данном раскрывающемся списке хранятся фрагменты текста, использовавшиеся в предыдущих заменах. Если требуемый текст содержится в раскрывающемся списке, то его можно выбрать, а не вводить вручную. Текст в текстовом поле раскрывающегося списка **Replace with:** может представлять собой регулярное выражение.

После того как в тексте файла с использованием кнопки **Find Next** будет найден требуемый фрагмент текста, пользователь может нажать кнопку **Replace**. В результате найденный фрагмент текста будет заменен и будет выполнен поиск следующего фрагмента текста. Если пользователь нажал кнопку **Replace All**, то поиск и замена текста по всему файлу произойдут

автоматически без участия пользователя. Кнопку **Replace All** следует использовать с особой осторожностью, поскольку при неправильном выборе установок в окне результаты быстрой замены придется исправлять в течение нескольких часов, т. к. стек операций команды **Undo** имеет ограниченные размеры. Если вы все-таки решились использовать эту кнопку, то настоятельно рекомендуется, по крайней мере, для первых замен использовать кнопку **Replace** и убедиться, что при замене не возникает никаких проблем.

Команда *Edit, Go To* (<Ctrl>+<G>)

Выводит на экран диалоговое окно **Go To**, изображенное на рис. D.22.

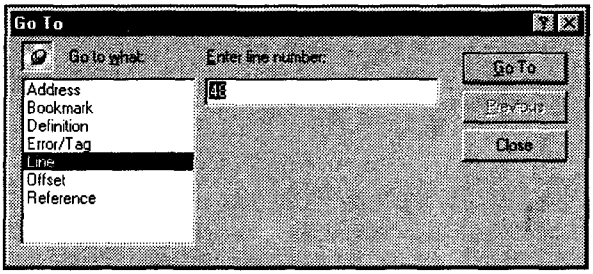


Рис. D.22. Диалоговое окно **Go To**

Это диалоговое окно позволяет перейти к строке с конкретным номером (это установка по умолчанию), выполнить переход по заданному адресу, ссылке, закладке. В зависимости от выделенной строки в окне списка **Go to what:** (Куда осуществить переход) изменяется заголовок текстового поля и появляется комментарий на чистейшем английском языке.

В окне списка **Go to what:** могут быть выделены следующие опции:

- **Address** (Адрес) — используется в окнах **Memory** (Память) и **Disassembly** (Дизассемблирование) для перехода к конкретному адресу в памяти
- **Bookmark** (Закладка) — позволяет перейти к закладке в текстовом файле. Обычно для этого используется команда меню **Edit**, **Bookmarks** или соответствующие кнопки панели инструментов
- **Definition** (Определение) — если курсор находится на имени пользовательской функции, то открывается файл реализации, содержащий описание данной функции, и текстовый курсор помещается в текст этого описания. Если курсор находится на имени переменной, то открывается файл заголовка и текстовый курсор помещается на описание данной переменной
- **Error/Tag** (Ошибки) — по завершении компиляции позволяет во вкладке **Build** окна **Output** просматривать выявленные компилятором ошибки
- **Line** (Строка) — позволяет перейти к строке, номер которой указан в текстовом поле **Enter line number:**

- **Offset** (Смещение) — выводит смещение адреса в шестнадцатеричном коде
- **Reference** (Ссылка) — позволяет перейти к строке программы, в которой объявлен введенный в текстовое поле идентификатор. Для работы требуется создание специального файла

Команда *Edit, Bookmarks* (<Alt>+<F2>)

Эта команда выводит на экран диалоговое окно **Bookmark**, изображенное на рис. D.23.

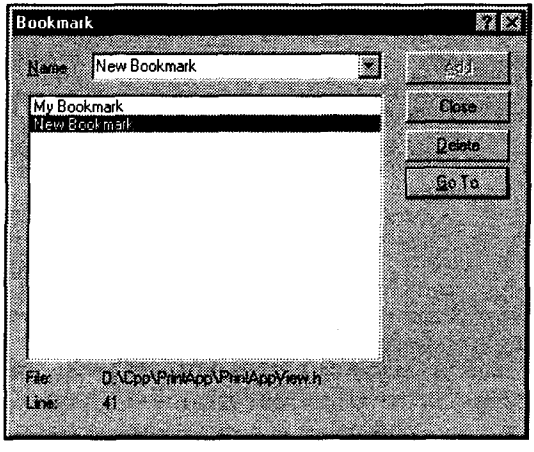


Рис. D.23. Диалоговое окно **Bookmark**

Это диалоговое окно используется для установки постоянных именованных закладок в текстовом файле. Временные закладки, устанавливаемые кнопкой окна **Find** или комбинацией клавиш <Ctrl>+<F2>, не отображаются в данном списке.

Чтобы добавить именованную закладку в приложение:

1. Откройте окно файла, в который нужно поместить закладку, и установите текстовый курсор на соответствующую строку.
2. Выберите команду меню **Edit**, **Bookmarks** или нажмите комбинацию клавиш <Alt>+<F2>. Появится диалоговое окно **Bookmark**.
3. Введите в текстовое поле раскрывающегося списка **Name** имя закладки или выберите его в раскрывающемся списке.
4. Нажмите кнопку **Add**. Имя новой закладки появится в окне списка данного диалогового окна.
5. Нажмите кнопку **Close** и закройте диалоговое окно **Bookmark**.

Чтобы перейти к именованной закладке:

1. Выберите команду меню **Edit**, **Bookmarks** или нажмите комбинацию клавиш <Alt>+<F2>. Появится диалоговое окно **Bookmark**.

2. Выделите в окне списка диалогового окна **Bookmark** имя нужной закладки. При этом под окном списка появится информация о ее местоположении.
3. Нажмите кнопку **Go To**. Диалоговое окно **Bookmark** закроется, а текстовый курсор переместится на соответствующую строку в файле.

Команда *Edit, Advanced*

Выбор данной команды меню приводит к появлению контекстного меню, показанного на рис. D.24.

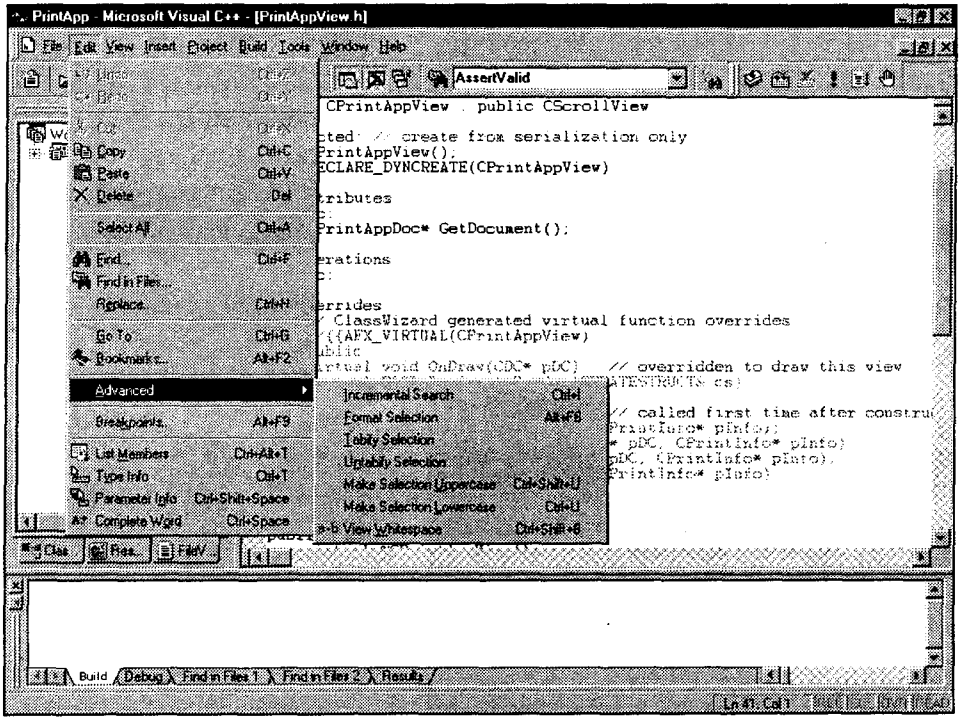


Рис. D.24. Контекстное меню **Advanced**

Данное контекстное меню содержит следующие команды:

- **Incremental Search** (Поиск по возрастанию) — эта команда меню позволяет осуществить более быстрый поиск, чем диалоговое окно **Find**. При выборе этой команды внешне ничего не происходит, однако при вводе с клавиатуры символов в тексте выделяется первое включение данной комбинации символов, начиная с текущей позиции курсора. По мере ввода новых символов выделенный фрагмент изменяется и, возможно, перемещается по тексту. Эта команда удобна для поиска первого вхождения идентификатора

- **Format Selection** (Выбор формата) — команда выполняет форматирование выделенного текста в соответствии с требованиями, предъявляемыми при написании программ
- **Tabify Selection** (Табуляция) — преобразует последовательность пробелов в выделенном фрагменте текста в символы табуляции
- **Untabify Selection** (Отмена табуляции) — преобразует в выделенном фрагменте текста символы табуляции в последовательность пробелов
- **Make Selection Uppercase** (Перевод в верхний регистр) — преобразует символы выделенного текста в прописные
- **Make Selection Lowercase** (Перевод в нижний регистр) — преобразует символы выделенного текста в строчные
- **View Whitespace** (Визуализация пробелов) — вставляет символы-заполнители ("." для пробела и ">>" для символа табуляции)

Команда Edit, Breakpoints (<Alt>+<F9>)

Команда выводит на экран диалоговое окно **Breakpoints**, изображенное на рис. D.25.

Данное диалоговое окно используется для работы с точками останова.

См. также главу 14

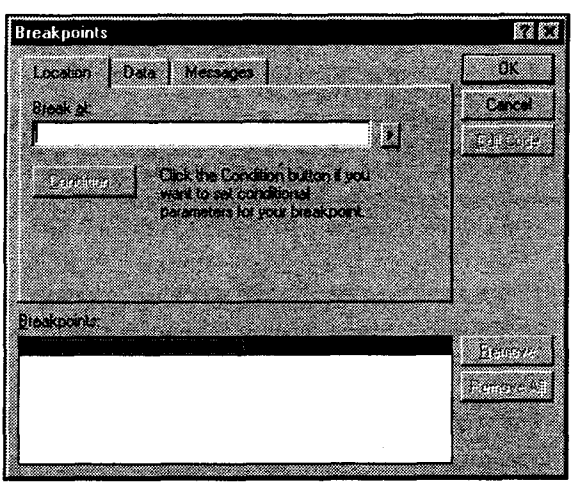


Рис. D.25. Диалоговое окно **Breakpoints**

Меню *View*

Меню **View**, приведенное на рис. D.26, содержит команды, управляющие отображением панелей инструментов и окон среды разработки **Output** и **Workspace**.

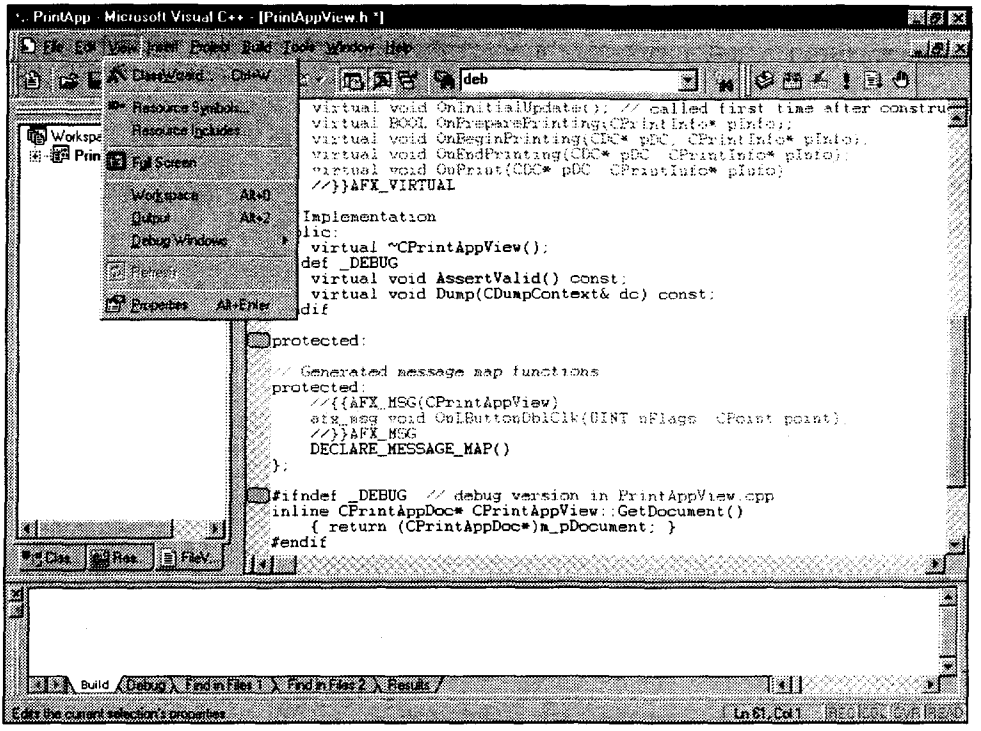


Рис. D.26. Меню **View**

Команда *View, ClassWizard* (<Ctrl>+<W>)

Выводит на экран диалоговое окно **MFC ClassWizard**, работа с которым подробно рассмотрена в Приложении С. При вызове данного диалогового окна производится автоматическое сохранение всех файлов проекта.

Команда *View, Resource Symbols*

Выводит на экран диалоговое окно **Resource Symbols**, изображенное на *рис.* D.27.

В окне списка данного диалогового окна выводятся идентификаторы ресурсов, используемые в приложении. Верхнее окно содержит имя идентификатора, его числовое значение и флажок, указывающий, что данный идентификатор ресурса используется в приложении. Список **Used by:** содержит ресурсы, в которых используется данный идентификатор (например, диалоговые окна, таблица строковых ресурсов, таблица акселераторов, меню и другие ресурсы).

Кнопка **New** данного диалогового окна выводит диалоговое окно **New Symbol**, позволяющее добавить в окно списка новый идентификатор ресур-

са. Кнопка **Delete** становится доступной только при выделении неиспользуемого идентификатора ресурса.

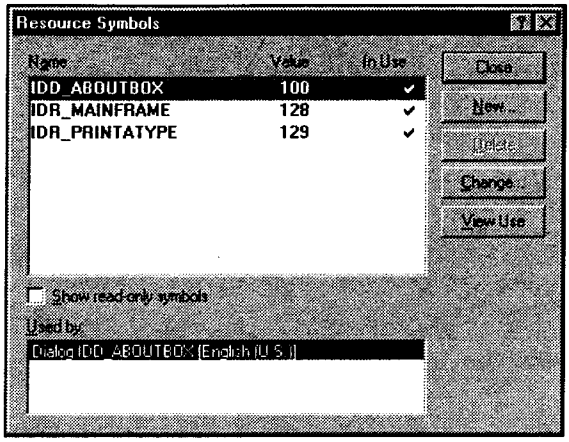


Рис. D.27. Диалоговое окно **Resource Symbols**

Чтобы изменить имя идентификатора ресурса:

1. Выделите его в окне списка диалогового окна **Resource Symbols** и нажмите кнопку **Change...**. Появится диалоговое окно **Change Symbol**, изображенное на рис. D.28.

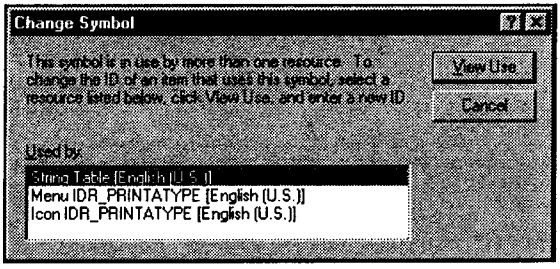


Рис. D.28. Диалоговое окно **Change Symbol**

2. В окне списка **Used by:** выделите ресурс, имя идентификатора которого нужно изменить, и нажмите кнопку **View Use**. Откроется окно редактирования соответствующего ресурса и появится диалоговое окно, в котором можно изменить идентификатор ресурса.
3. Измените имя идентификатора ресурса и закройте окно редактирования.

При нажатии кнопки **View Use** в диалоговом окне **Resource Symbols** данное диалоговое окно закрывается и открывается окно редактирования ресурса, выделенное в окне списка **Used by:**.

Команда *View, Resource Includes*

Данная команда выводит на экран диалоговое окно **Resource Includes**, изображенное на рис. D.29.

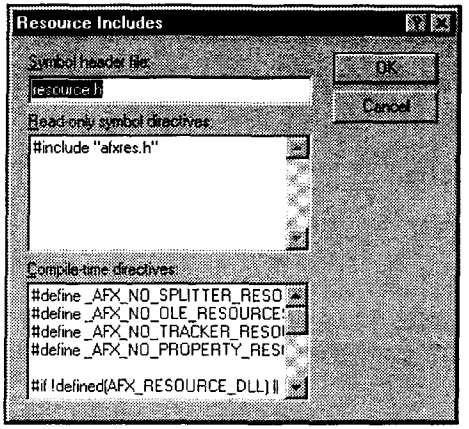


Рис. D.29. Диалоговое окно **Resource Includes**

Данное диалоговое окно позволяет добавить дополнительные строки в файл resource.h.

Команда *View, Full Screen*

Скрывает все панели инструментов, меню, окно вывода, окно **Workspace** и панель задач **Windows**, предоставляя пользователю возможность работать в полноэкранном режиме, как показано на рис. D.30.

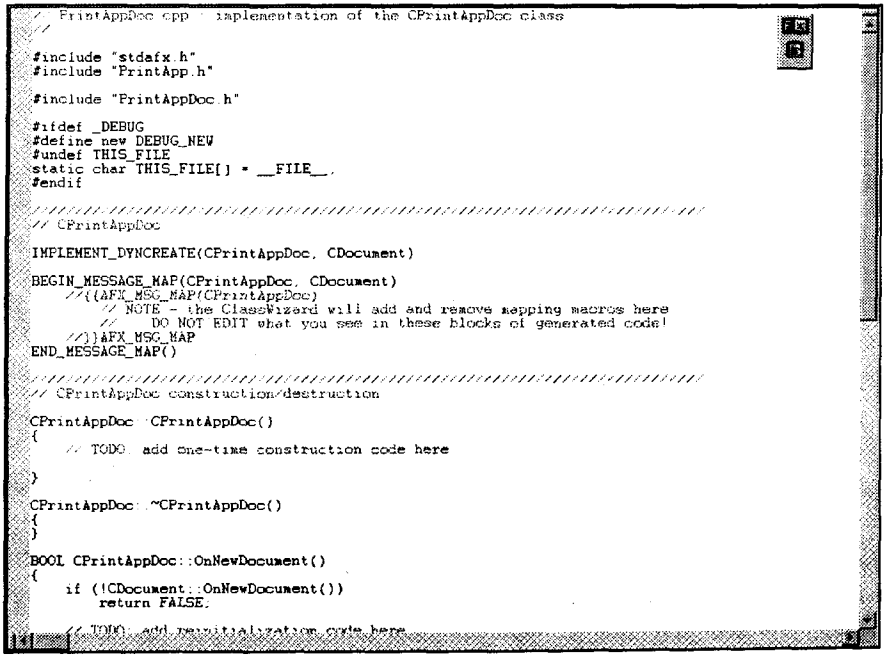


Рис. D.30. Работа в полноэкранном режиме

Для восстановления нормального режима работы в окне остается плавающая панель инструментов **Full Screen**, содержащая единственную кнопку **Toggle Full Screen**, на которую и нужно нажать для выхода из полноэкранного режима отображения.

Команда *View, Workspace* (<Alt>+<0>)

Выводит на экран окно **Workspace**, если оно до этого было скрыто. Чтобы убрать это окно, щелкните в нем правой кнопкой мыши и выберите в появившемся контекстном меню команду **Hide** или нажмите комбинацию клавиш <Shift>+<Esc>, когда данное окно имеет фокус ввода. Панель инструментов **Standard** имеет кнопку **Workspace**, позволяющую выводить на экран и скрывать данное окно.

Команда *View, Output* (<Alt>+<2>)

Команда выводит на экран окно **Output**, если оно до этого было скрыто. Чтобы убрать это окно, щелкните в нем правой кнопкой мыши и выберите в появившемся контекстном меню команду **Hide** или нажмите комбинацию клавиш <Shift>+<Esc>, когда данное окно имеет фокус ввода. Панель инструментов **Standard** имеет кнопку **Output**, позволяющую выводить на экран и скрывать данное окно. Окно **Output** автоматически появляется на экране при компиляции проекта или в результате работы диалогового окна **Find In Files**.

Команда *View, Debug Windows*

Выбор этой команды приводит к появлению контекстного меню, показанного на рис. D.31. Команды данного меню доступны только в процессе отладки приложения.

В данном контекстном меню содержатся следующие команды:

- **Watch** (Трассировка) — выводит на экран окно **Watch**, если оно было до этого скрыто, и передает ему фокус ввода
- **Call Stack** (Вызов стека) — выводит на экран окно **Call Stack**, если оно было до этого скрыто, и передает ему фокус ввода
- **Memory** (Память) — выводит на экран окно **Memory**, если оно было до этого скрыто, и передает ему фокус ввода
- **Variables** (Переменные) — выводит на экран окно **Variables**, если оно было до этого скрыто, и передает ему фокус ввода
- **Registers** (Регистры) — выводит на экран окно **Registers**, если оно было до этого скрыто, и передает ему фокус ввода
- **Disassembly** (Дизассемблирование) — выводит на экран окно **Disassembly**, если оно было до этого скрыто, и передает ему фокус ввода

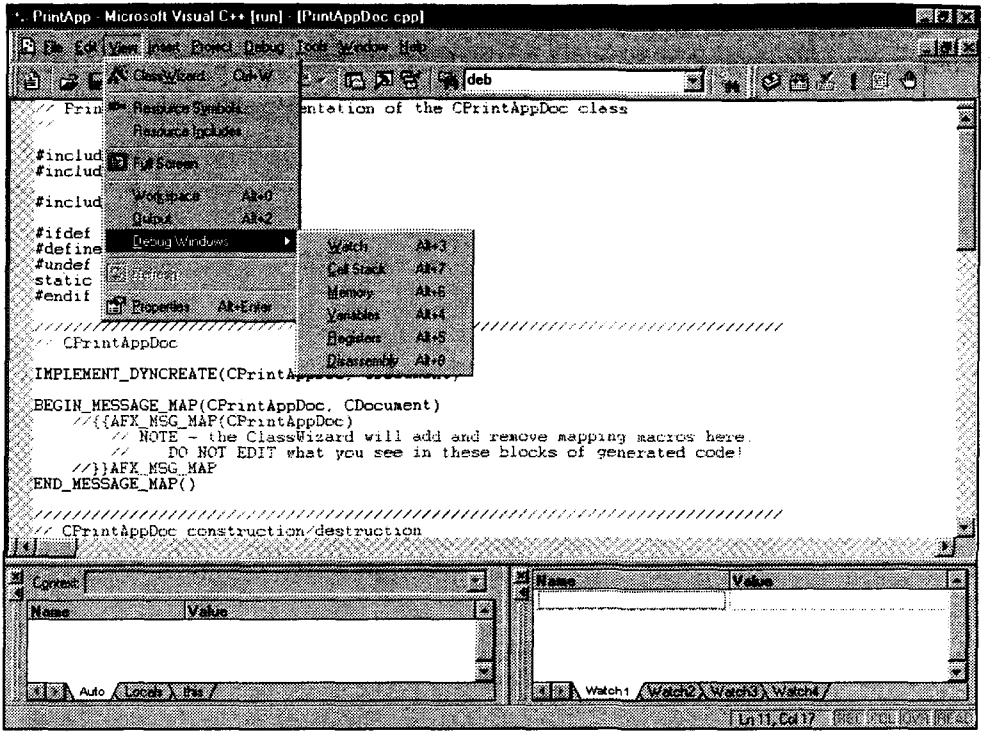


Рис. D.31. Контекстное меню **Debug Windows**

Команда *View, Properties* (<Alt>+<Enter>)

Выводит на экран различные версии диалогового окна **Properties**, обычно используемого в редакторах различных ресурсов.

См. также главы 3 и 4

Меню *Insert*

Меню **Insert**, приведенное на рис. D.32, содержит команды, осуществляющие включение файлов или их фрагментов в активный проект.

Команда *Insert, New Class*

Команда выводит на экран диалоговое окно **New Class**, изображенное на рис. D.33.

В отличие от диалогового окна **New Class**, выводимого мастером ClassWizard, данное диалоговое окно содержит раскрывающийся список **Class type:**, позволяющий создавать не только классы, имеющие прототипы в библиотеке

MFC, но и другие типы классов, включая пользовательские. Работа с данным окном подробно рассмотрена в Приложении С.

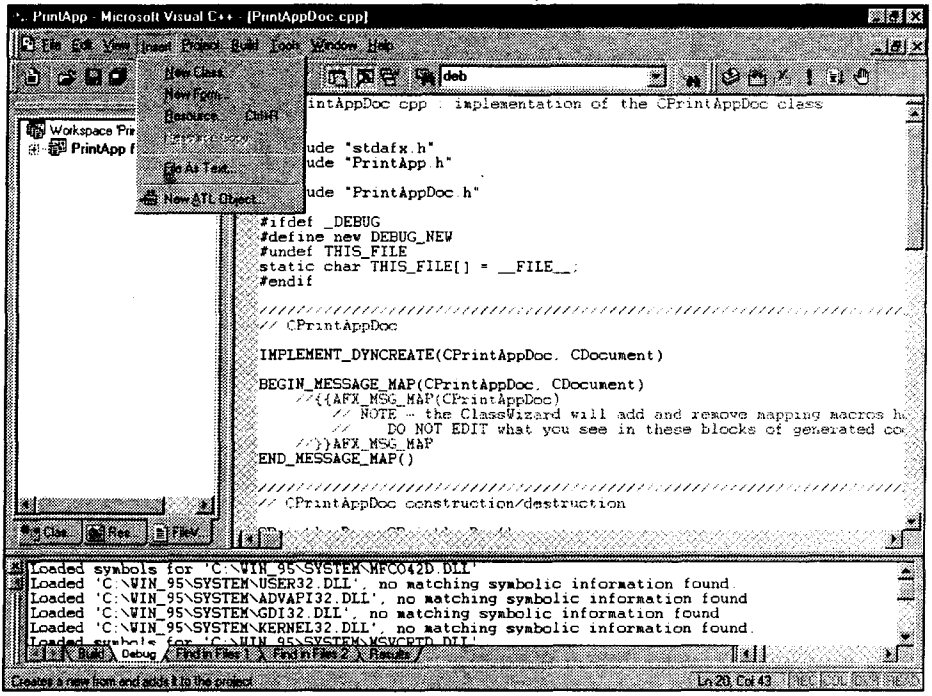


Рис. D.32. Меню **Insert**

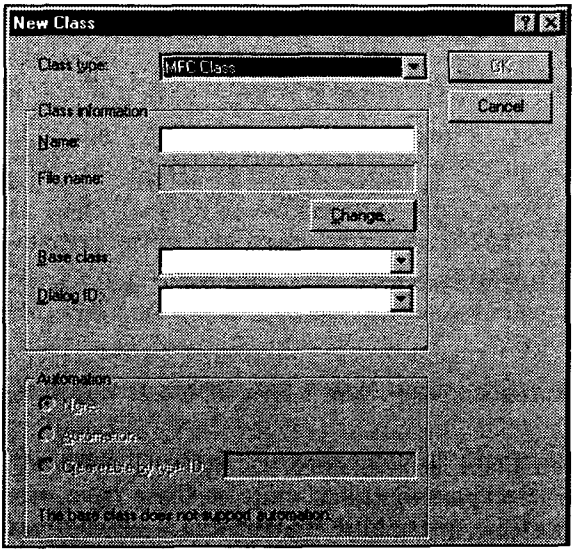


Рис. D.33. Диалоговое окно **New Class**

Команда *Insert, New Form*

Данная команда выводит на экран диалоговое окно **New Form**, изображенное на рис. D.34.

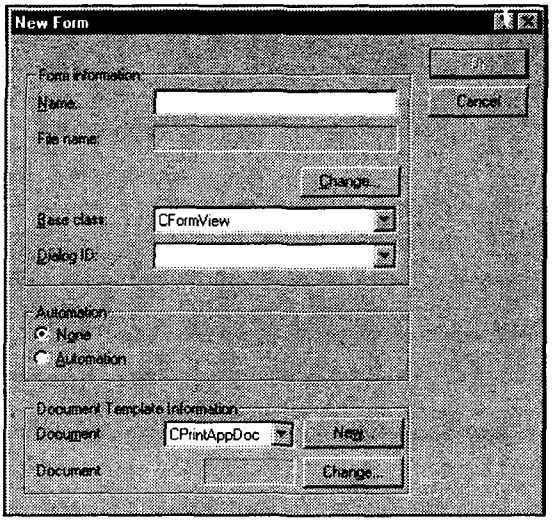


Рис. D.34. Диалоговое окно **New Form**

Это диалоговое окно позволяет создать новую форму. В текстовое поле **Name:** вводится имя добавляемого класса или формы. На основе этого имени формируются имена файлов заголовка и реализации, а также идентификатор диалогового окна, связанный с данной формой. Работа с текстовым полем **File name:**, кнопкой **Change...** и раскрывающимися списками **Base class:** и **Dialog ID:** аналогична работе с данными элементами управления в диалоговом окне **New Class**.

Группа элементов управления **Document Template Information** позволяет установить или изменить параметры документа, связанного с классом формы, производным от класса `CFormView`. Раскрывающийся список **Document** позволяет выбрать шаблон документа, который будет использоваться с данной формой или создать новый шаблон с использованием кнопки **New...**. В текстовом поле **Document** располагается автоматически генерируемое расширение, которое будет использоваться с данной формой. Оно может быть изменено при помощи кнопки **Change...**.

Команда *Insert, Resource* (<Ctrl>+<R>)

Выводит на экран диалоговое окно **Insert Resource**, изображенное на рис. D.35.

Данное диалоговое окно позволяет добавить новый ресурс в активный проект. Чтобы добавить новый ресурс в проект, выделите соответствующий тип ресурса в окне списка **Resource type:** и нажмите кнопку **New**.

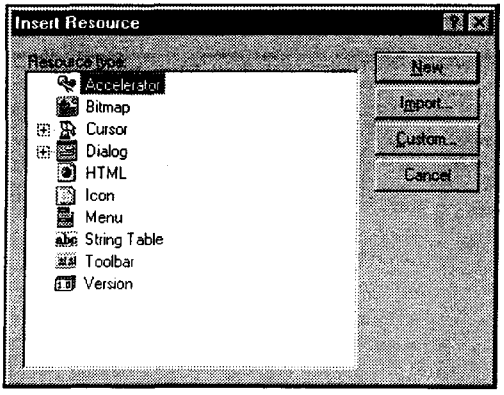


Рис. D.35. Диалоговое окно **Insert Resource**

Команда *Insert, Resource Copy*

Команда выводит на экран диалоговое окно **Insert Resource Copy**, изображенное на рис. D.36.

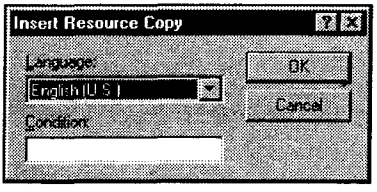


Рис. D.36. Диалоговое окно **Insert Resource Copy**

Эта команда становится доступной, когда в Visual C++ выделен какой-либо ресурс или его идентификатор. Диалоговое окно **Insert Resource Copy** позволяет скопировать данный ресурс, выбрав для него новый язык в окне **Language:**. В текстовом поле **Condition:** указывается некоторый символ. Данная версия ресурса будет включена в приложение только в том случае, если при его компиляции будет определен данный символ.

Команда *Insert, New ATL Object*

Данная команда используется для вставки объектов Active Template Library (ATL) в проект при создании элемента управления ActiveX с помощью ATL.

Меню *Project*

Меню **Project**, приведенное на рис. D.37, содержит команды, управляющие разрабатываемым проектом в целом.

Команда меню *Project, Set Active Project*

Команда позволяет выбрать активный проект из нескольких открытых проектов.

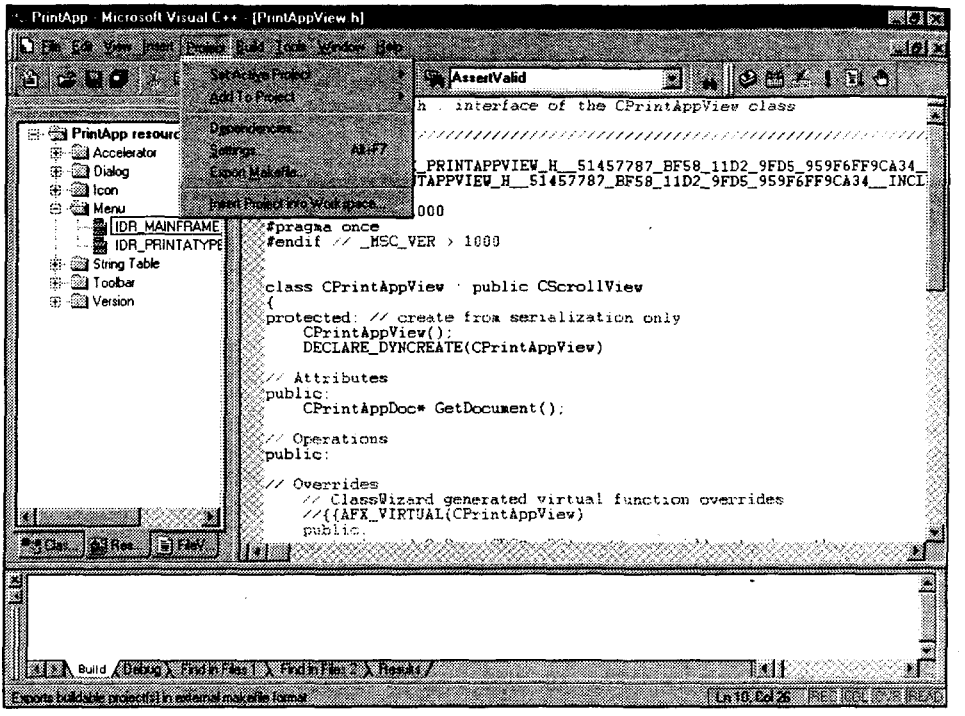


Рис. D.37. Меню **Project**

Команда меню *Project, Add to Project*

Выбор этой команды приводит к появлению контекстного меню, показанного на рис. D.38.

Данное контекстное меню содержит следующие команды:

- **New** (Создать) — работает аналогично команде **File, New**. В появляющемся диалоговом окне **New** уже установлен флажок **Add To Project** (Добавить к проекту)
- **New Folder** (Новая папка) — создает новую папку для организации классов в проекте
- **Files** (Файлы) — выводит на экран диалоговое окно **Insert Files into Project**, изображенное на рис. D.39

  Данное диалоговое окно допускает множественное выделение, что позволяет одновременно добавлять в проект несколько файлов. При добавлении файлов в проект не рекомендуется брать их из других каталогов, поскольку это может привести к проблемам при компиляции проекта
- **Data Connection** (Подключение данных) — используется для объединения проекта с источником данных

- **Components and Controls** (Компоненты и элементы управления) — выводит на экран диалоговое окно **Components and Controls Gallery**, изображенное на рис. D.40. Это диалоговое окно используется для работы с Галереей компонентов (Component Gallery).

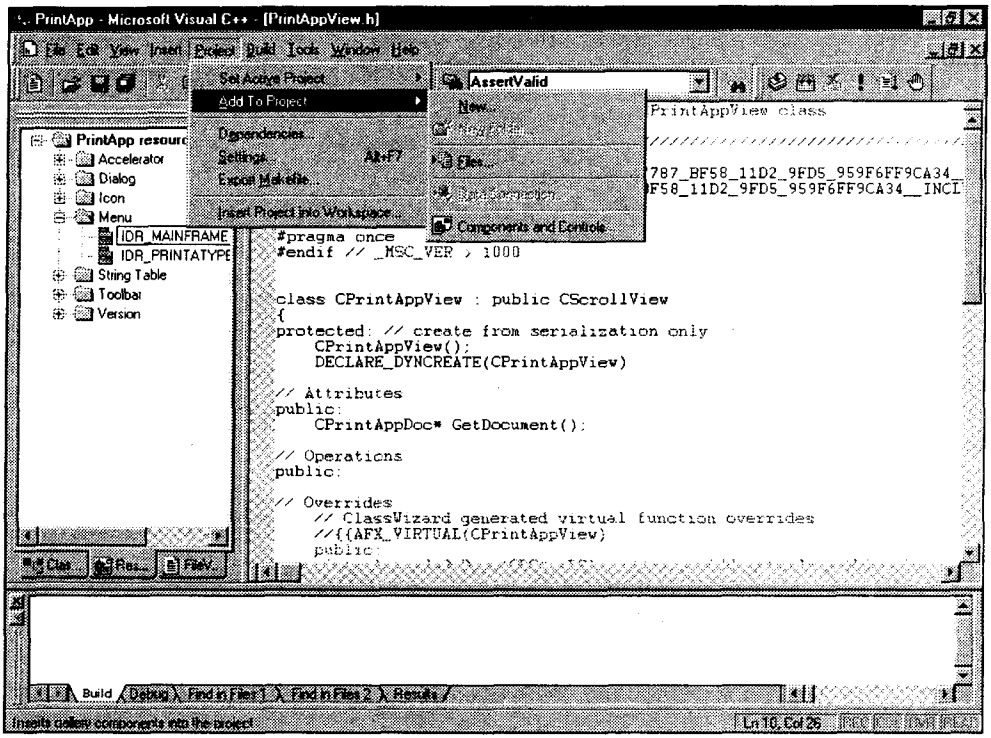


Рис. D.38. Контекстное меню **Add To Project**

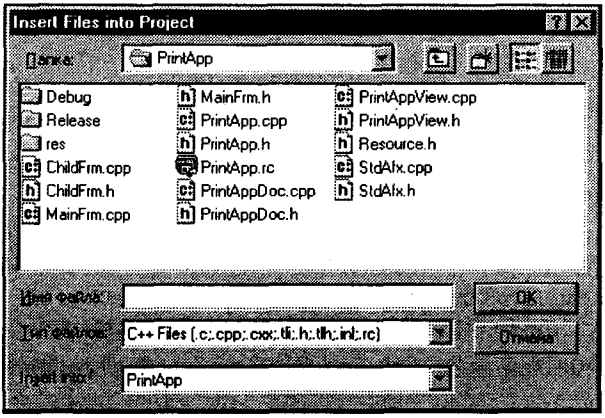


Рис. D.39. Диалоговое окно **Insert Files into Project**

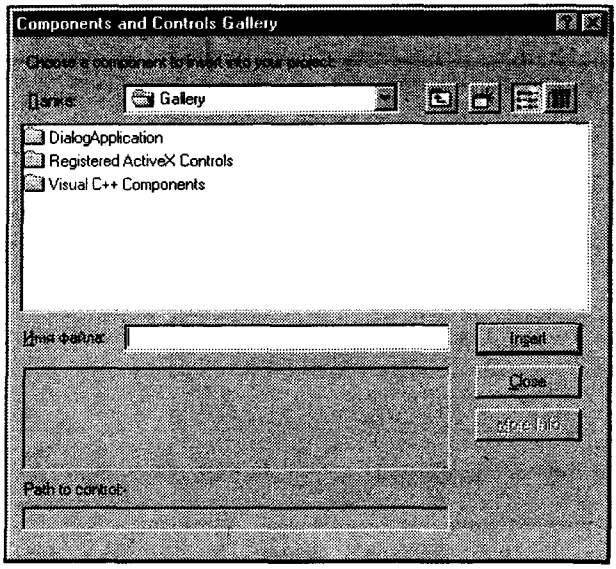


Рис. D.40. Диалоговое окно **Components and Controls Gallery**

Команда *Project, Dependencies*

Команда выводит на экран диалоговое окно **Project Dependencies**, изображенное на рис. D.41.

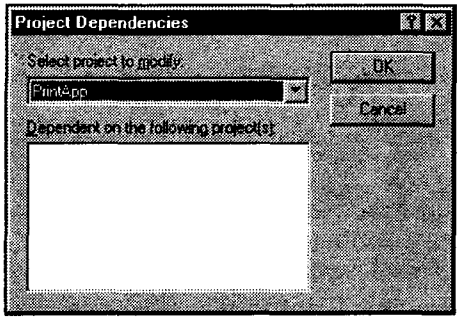


Рис. D.41. Диалоговое окно **Project Dependencies**

Это диалоговое окно позволяет установить зависимость между проектами таким образом, что при внесении изменений в один проект перекомпилируется и другой, зависящий от первого.

Команда *Project, Settings* (<Alt>+<F7>)

Выводит на экран диалоговое окно **Project Settings**. В раскрывающемся списке **Settings For:** можно выбрать конфигурацию, для которой производится данная установка, а в расположенном ниже окне списка выбрать настраиваемый проект.

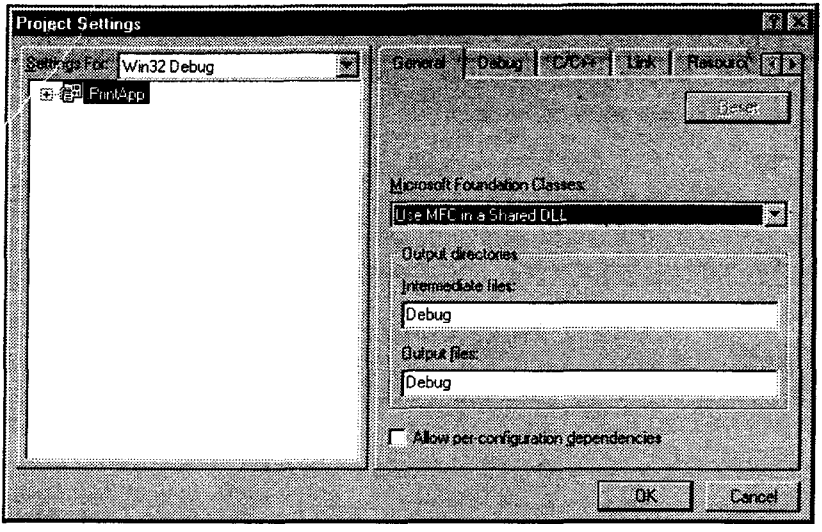


Рис. D.42. Вкладка **General** диалогового окна **Project Settings**

Данное диалоговое окно имеет следующие вкладки:

- **General** (Общие установки), изображенная на рис. D.42. В этой вкладке производится настройка способа подключения библиотек (как библиотек динамической компоновки или как статических библиотек) и определяются каталоги, в которые будут записаны промежуточные и окончательные файлы
- **Debug** (Отладка). Данная вкладка позволяет установить имя и путь к исполняемому файлу проекта, скомпилированному в режиме отладки, и задать дополнительные библиотеки, используемые в отладочной версии приложения
- **C/C++**, изображенная на рис. D.43. Позволяет задать параметры компилятора. В текстовом поле раскрывающегося списка **Category:** по умолчанию установлено значение **General** (Общие установки). Этот параметр позволяет изменять уровень предупреждений транслятора в раскрывающемся списке **Warning level:**, задать критерий оптимизации в раскрывающемся списке **Optimization:**. Возможные варианты: **Default** — по умолчанию, **Disable (Debug)** — оптимизация отключена (режим отладки), **Maximize Speed** — максимальное быстродействие, **Minimize Size** — минимальный размер кода, **Customize** — настройка пользователя
- **Link** (Компоновка). Здесь можно изменять параметры компоновщика. Необходимость в этом возникает крайне редко
- **Resources** (Ресурсы), рис. D.44. Данная вкладка используется для изменения языка текстов ресурсов. Эта вкладка позволяет заменить ресурсы, используемые в приложении, или добавить к нему новые файлы ресурсов

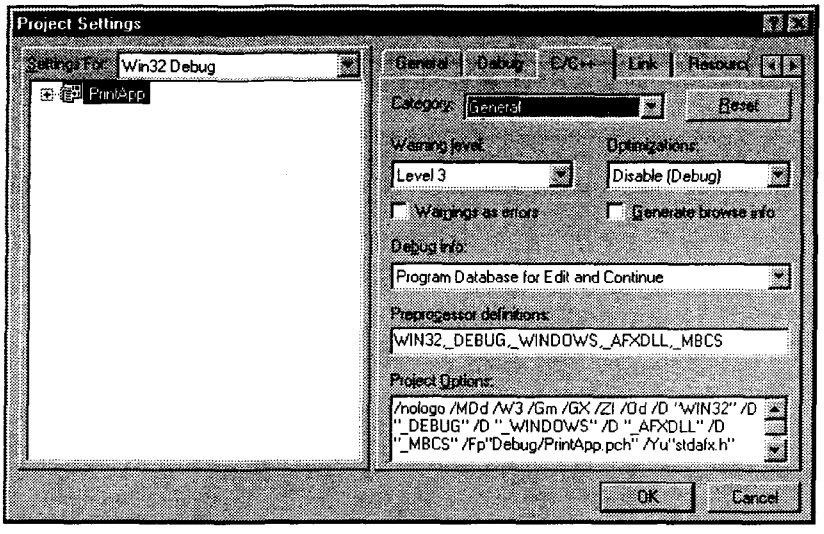


Рис. D.43. Вкладка **C/C++** диалогового окна **Project Settings**

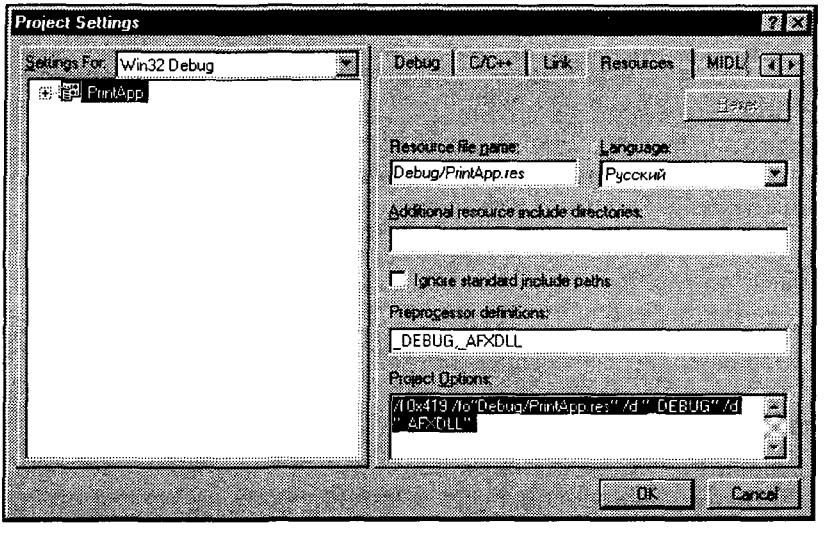


Рис. D.44. Вкладка **Resources** диалогового окна **Project Settings**

- **Browse Info** (Информация просмотра), рис. D.45. Эта вкладка управляет созданием файла Browse Info (с расширением .bsc), используемого в командах контекстного меню вкладки **ClassView** окон **Workspace Go To Definition**, **Go To Declaration** и аналогичных им. Если необходимость в использовании данных команд отсутствует, сбросьте флажок **Build browse info file**, что увеличит скорость компоновки приложения. Если существует

необходимость использования информации просмотра, то, кроме флажка **Build browse info file** в данной вкладке, необходимо установить флажок **Generate browse info** во вкладке **General** диалогового окна **Project Settings**.

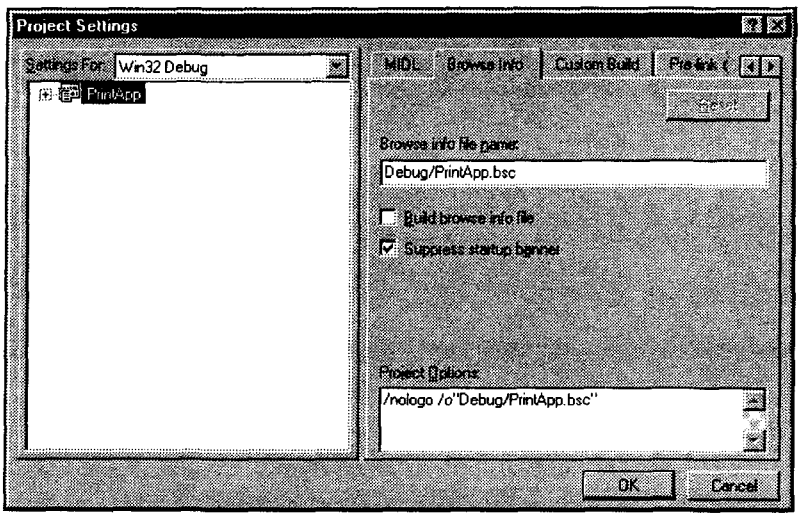


Рис. D.45. Вкладка **Browse Info** диалогового окна **Project Settings**

С остальными вкладками данного диалогового окна пользователю, как правило, не приходится работать. Для доступа к ярлыкам вкладок, которые в данный момент недоступны, используются кнопки со стрелками, расположенные на одном уровне с ярлыками вкладок.

Меню *Build*

Меню **Build**, приведенное на рис. D.46, содержит команды компиляции, компоновки и отладки.

Команда *Build, Compile* (<Alt>+<F7>)

Эта команда компилирует файл, окно которого имеет в данный момент фокус ввода. Эта команда используется для быстрой проверки корректности файла, особенно после внесения в него большого числа изменений. Выбор команд **Build**, **Rebuild All** или **Build**, **Execute**, осуществляющих компиляцию всего проекта, приведет к выдаче большого числа повторяющихся сообщений при наличии ошибки в файле заголовка. При этом компилятор может завершить работу, не дойдя до данного файла, если в предыдущих файлах встретились серьезные ошибки. (Хотя остается непонятным, почему нужно исправлять ошибки именно в файле, окно которого имеет фокус ввода, и чем он лучше других файлов.) В панелях инструментов **Build** и **Build MiniBar** имеется кнопка, соответствующая данной команде.

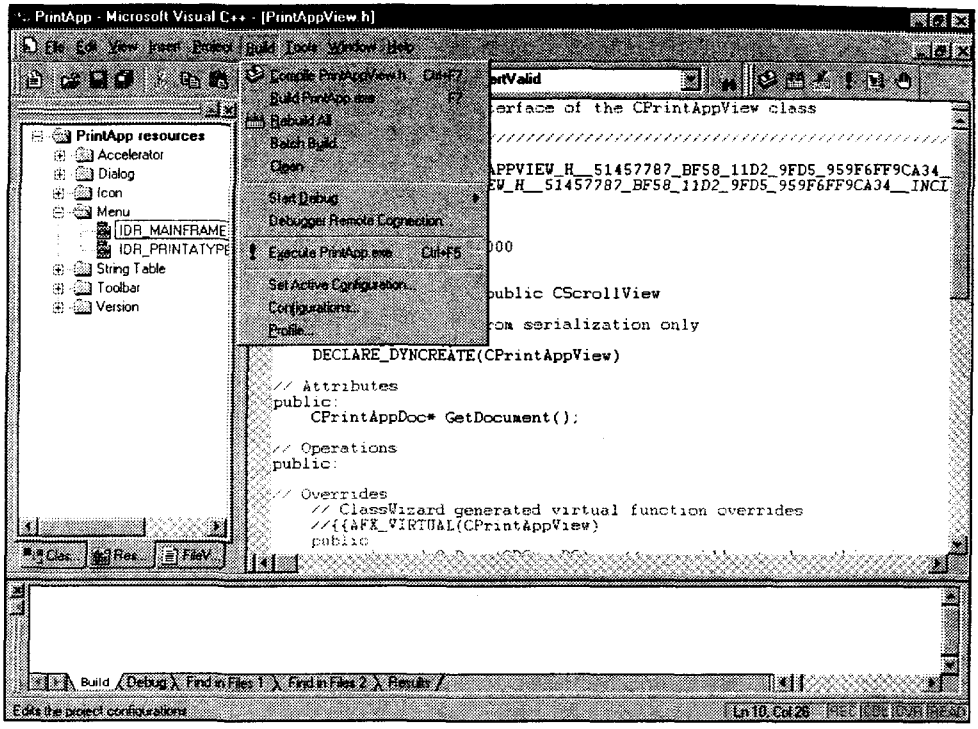


Рис. D.46. Меню **Build**

Команда *Build, Build* (<F7>)

Команда позволяет перекомпилировать все файлы, в которые были внесены изменения. На панелях инструментов **Build** и **Build MiniBar** имеется кнопка, соответствующая данной команде.

Команда *Build, Rebuild All*

Позволяет перекомпилировать все файлы проекта независимо от того, были в них внесены изменения после последней компиляции или нет. Иногда в процессе компиляции пропускается файл, который должен быть перекомпилирован. Обычно это имеет место, если в данный файл были внесены изменения в обход среды программирования Visual C++ (например, была записана старая версия одного из файлов, взятая из архива).

Примечание

К сожалению, в большинстве случаев использование этой команды не исправляет ситуацию. В таких случаях перед вызовом данной команды меню следует полностью уничтожить содержимое каталога \Debug или \Release (в зависи-

мости от того, какая версия приложения компилируется). Для этого достаточно выбрать команду **Build, Clean**. После этого, как правило, приложение начинает работать.

Команда *Build, Batch Build*

Выводит на экран диалоговое окно **Batch Build**, изображенное на рис. D.47.

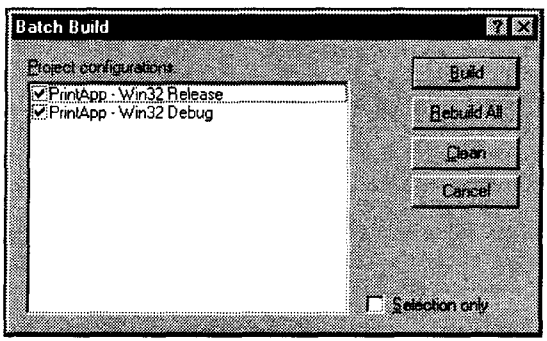


Рис. D.47. Диалоговое окно **Batch Build**

Это диалоговое окно позволяет выполнить одновременную компиляцию нескольких конфигураций приложения (например, отладочной и отладочной с использованием символов Unicode). В окне списка **Project configurations:** отмечаются компилируемые конфигурации и нажимается кнопка **Build** для компиляции измененных файлов или кнопка **Rebuild All** для компиляции всех файлов проекта. После успешной компиляции производится компоновка файлов.

Команда *Build, Clean*

Данная команда меню уничтожает все временные и окончательные файлы активной версии проекта. Это бывает необходимо, если Visual C++ отказывается перекомпилировать файл.

Команда *Build, Start Debug*

Выбор этой команды приводит к появлению контекстного меню, показанного на рис. D.48.

Контекстное меню содержит следующие команды: **Go**, **Step Into**, **Run to Cursor** и **Attach to Process...**. Первые три команды, описанные в главе 14, позволяют запустить приложение в режиме отладки. Последняя команда выводит диалоговое окно **Attach To Process**, изображенное на рис. D.49.

Это диалоговое окно позволяет подключить запускаемое приложение к одному из активных процессов в системе, что позволяет производить анализ данного приложения средствами выбранного процесса.

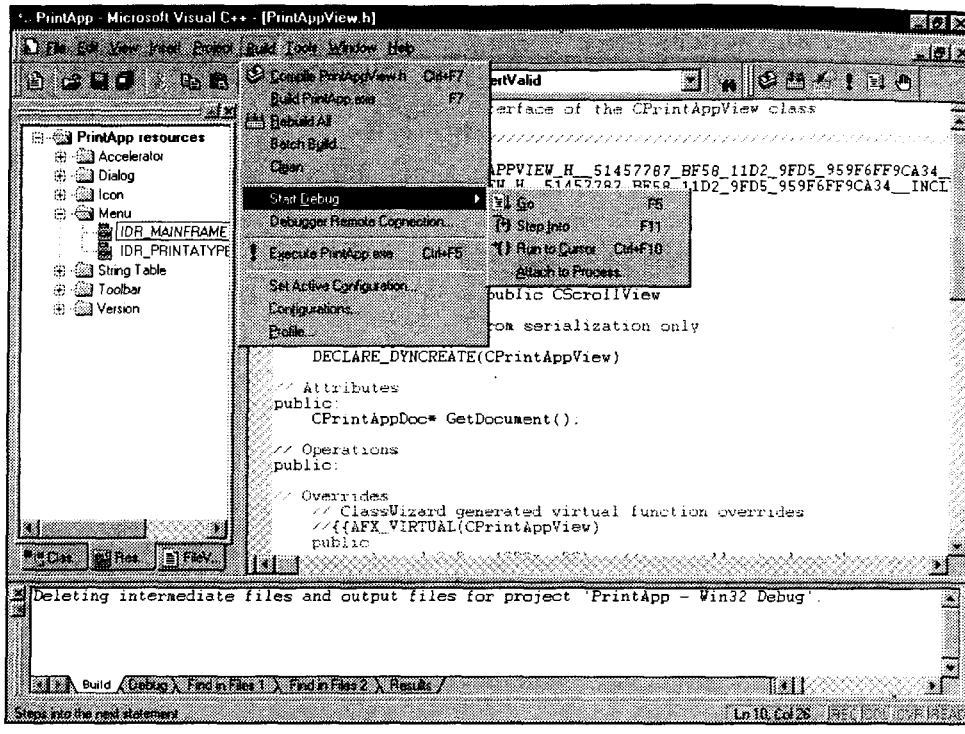


Рис. D.48. Контекстное меню **Start Debug**

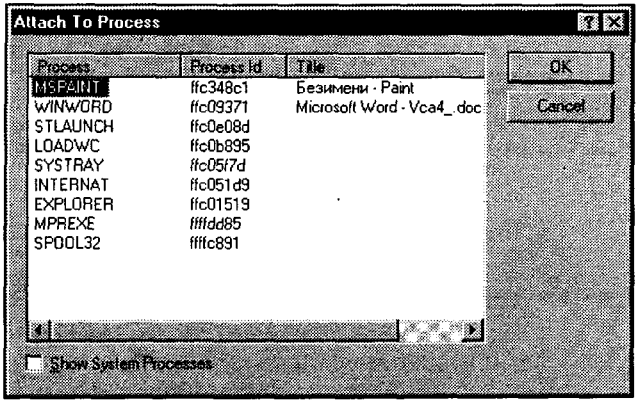


Рис. D.49. Диалоговое окно **Attach To Process**

Команда *Build, Debugger Remote Connection*

Команда используется для связи компьютеров в режиме удаленной отладки, когда программа выполняется на одном компьютере, а отладчик расположен на другом.

Команда *Build, Execute* (<Ctrl>+<F5>)

Эта команда запускает приложение на исполнение без вызова отладчика.

Команда *Build, Set Active Configuration*

Выводит на экран диалоговое окно **Set Active Project Configuration**, изображенное на рис. D.50.

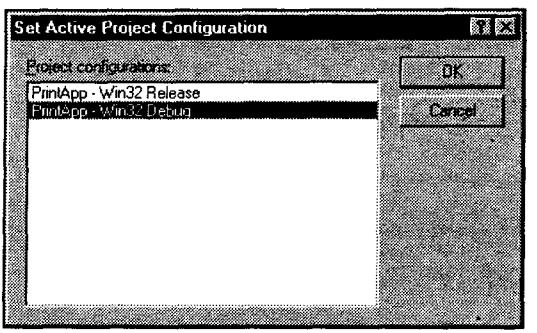


Рис. D.50. Диалоговое окно **Set Active Project Configuration**

Это диалоговое окно позволяет выбрать активную конфигурацию проекта (обычно это Debug или Release). Активная конфигурация определяет режим компиляции проекта.

Команда *Build, Configurations*

Выводит на экран диалоговое окно **Configurations**, изображенное на рис. D.51.

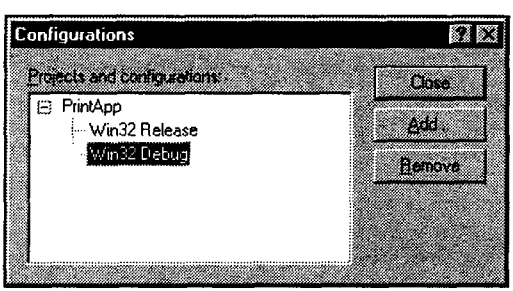


Рис. D.51. Диалоговое окно **Configurations**

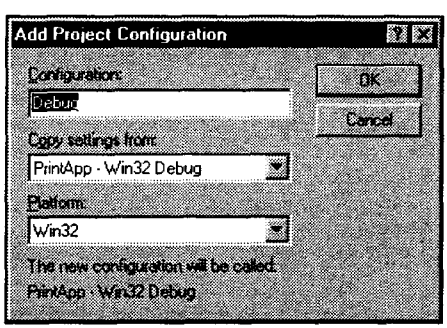


Рис. D.52. Диалоговое окно **Add Project Configuration**

Данное диалоговое окно позволяет удалить имеющуюся конфигурацию или добавить новую. Нажатие кнопки **Add...** приводит к появлению диалогового окна **Add Project Configuration**, изображенного на рис. D.52.

В текстовое поле **Configuration:** данного диалогового окна вводится имя новой конфигурации. В раскрывающемся списке **Copy settings from:** выбирает-

ся конфигурация, установки которой копируются в новую конфигурацию. В раскрывающемся списке **Platform:** выбирается платформа для новой конфигурации. Для изменения установок новой конфигурации выберите команду меню **Project**, **Settings**.

Команда *Build, Profile*

Команда используется при поиске "узких мест" в приложении.

Меню *Debug*

Меню **Debug**, приведенное на рис. D.53, появляется после запуска приложения на исполнение в режиме отладки и содержит команды, используемые при отладке приложения.

См. также главу 14

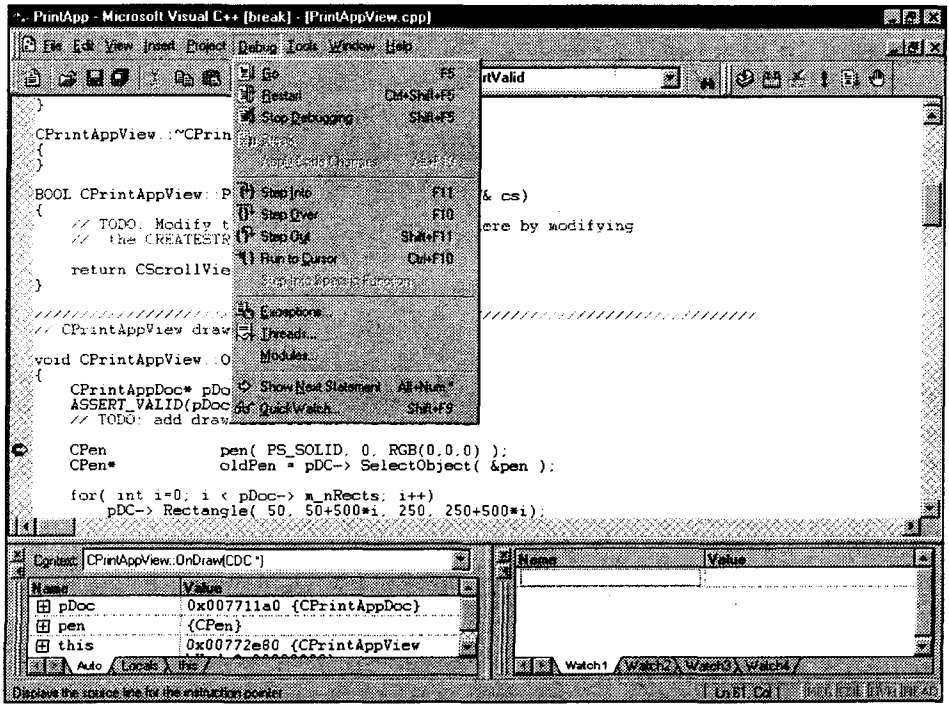


Рис. D.53. Меню **Debug**

Команда *Debug, Go* (<F5>)

Команда запускает приложение на исполнение в режиме отладки или возобновляет выполнение программы до следующей точки останова или до

конца программы, если при ее дальнейшем выполнении не встретятся точки останова.

Команда *Debug, Restart* (<Ctrl>+<Shift>+<F5>)

Перезапускает исполняемое приложение.

Команда *Debug, Stop Debugging* (<Shift>+<F5>)

Завершает процесс отладки приложения.

Команда *Debug, Step Into* (<F11>)

Команда позволяет выполнить один оператор программы и снова остановиться. Если текущий оператор является функцией, доступной для отладки, следующим оператором программы является первый оператор данной функции (ее заголовок). Если текущий оператор не является функцией, данная команда работает аналогично команде меню **Debug**, **Step Over**. Если текст данной функции не содержится в каталогах, указанных пользователем при настройке системы, система выводит диалоговое окно, предлагая указать путь к данному файлу. Если данная функция содержится в системной библиотеке и недоступна для отладки, система выводит окно сообщения с соответствующим текстом.

Команда *Debug, Step Over* (<F10>)

Позволяет выполнить один оператор программы и снова остановиться независимо от того, является ли данный оператор функцией или нет.

Команда *Debug, Step Out* (<Shift>+<F11>)

Команда позволяет выйти из текущей функции и остановиться на первом операторе программы, расположенном за вызовом данной функции.

Команда *Debug, Run to Cursor* (<Ctrl>+<F10>)

Позволяет продолжить выполнение программы и остановиться на операторе, на котором в настоящий момент расположен текстовый курсор.

Команда *Debug, Exceptions*

Выводит на экран диалоговое окно **Exceptions**, изображенное на рис. D.54.

В окне списка **Exceptions:** данного диалогового окна перечислены все обрабатываемые системой исключения. Пользователь может вносить изменения в этот список, уничтожать в нем системные исключения или добавлять в него новые исключения. Содержимое списка сохраняется в файле, имеющем имя проекта и расширение .dsw.

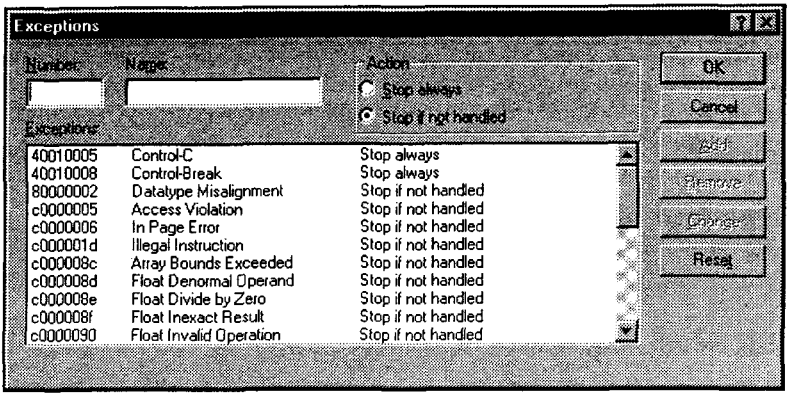


Рис. D.54. Диалоговое окно **Exceptions**

Группа переключателей **Action** позволяет определить действие, которое необходимо предпринять при вызове выделенного в окне списка **Exceptions:** исключения. Если установлен переключатель **Stop always**, то приложение всегда прекращает работу при вызове данного исключения. Если установлен переключатель **Stop if not handled**, то приложение прекращает работу, только если это исключение остается необработанным.

Чтобы добавить в окно списка **Exceptions:** новое исключение, укажите его уникальный номер в текстовом поле **Number:**. Номера системных исключений перечислены в файле заголовка WinBase.h, где им сопоставлены идентификаторы с префиксом `EXCEPTION_` (например `EXCEPTION_ACCESS_VIOLATION`). Текстовое поле **Name:**, содержащее имя, под которым данное исключение будет записано в окно списка **Exceptions:**, может не заполняться. После этого нужно установить соответствующий переключатель группы **Action** и нажать кнопку **Add**. В окне списка **Exceptions:** появится новое исключение.

Команда *Debug, Threads*

Выводит на экран диалоговое окно **Threads**, изображенное на рис. D.55.

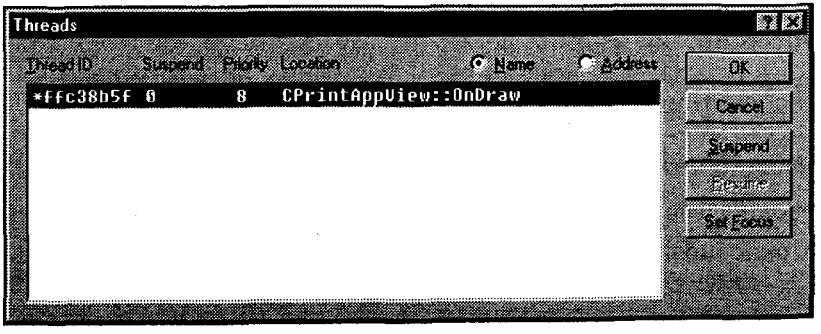


Рис. D.55. Диалоговое окно **Threads**

Это диалоговое окно позволяет просмотреть все активные на данный момент потоки, определить их состояние, приоритет и выполняемую ими функцию. При установке переключателя **Name** в поле **Location** окна списка отображается имя функции, выполняемой данным потоком. Если установлен переключатель **Address**, в поле **Location** окна списка отображается адрес команды, выполняемой данным потоком.

Нажатие кнопки **Suspend** приостановит выполнение выделенного потока, нажатие кнопки **Resume** возобновит его работу, а нажатие кнопки **Set Focus** позволит передать ему фокус ввода.

Команда *Debug, Modules*

Выводит на экран диалоговое окно **Module List**, изображенное на рис. D.56.

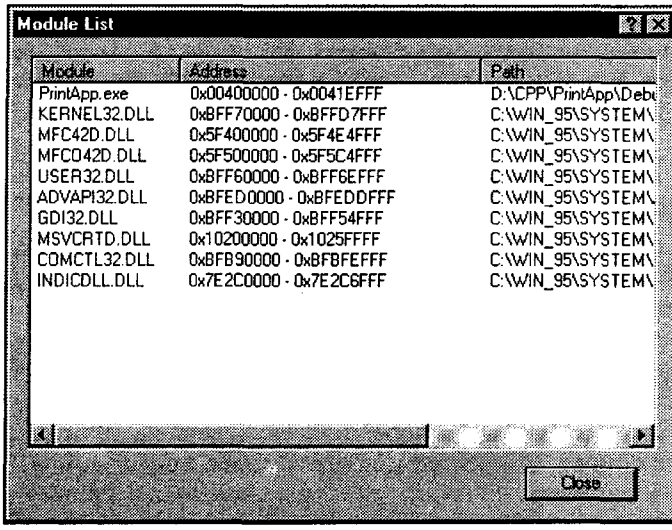


Рис. D.56. Диалоговое окно **Module List**

Данное диалоговое окно позволяет просмотреть все библиотеки динамической компоновки, загруженные в проект. Изначально они располагаются в порядке их загрузки в проект, Нажатие кнопок с текстом заголовков позволяет произвести сортировку модулей по имени, адресу, имени каталога, или порядку загрузки в проект. Для просмотра информации, не поместившейся в окно списка, используется полоса прокрутки.

Команда *Debug, QuickWatch* (<Shift>+<F9>)

Выводит на экран диалоговое окно **QuickWatch**, изображенное на рис. D.57.

Это диалоговое окно позволяет просмотреть значение выражения, которое было выделено в момент вызова данной команды меню или было введено пользователем в текстовое поле раскрывающегося списка **Expression:**. Нажатие кнопки **Add Watch** позволяет добавить это выражение в окно **Watch**.

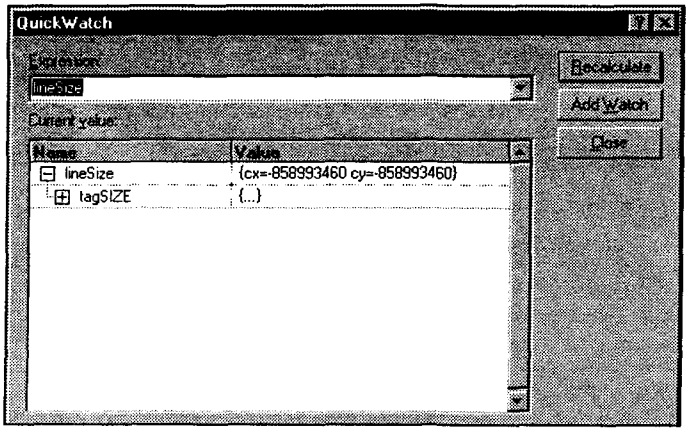


Рис. D.57. Диалоговое окно **QuickWatch**

Меню *Tools*

Меню **Tools**, приведенное на рис. D.58, содержит команды настройки Visual C++ и команды доступа к автономным утилитам, используемым в среде программирования Visual C++. Набор этих утилит изменяется командой **Tools**, **Customize**.

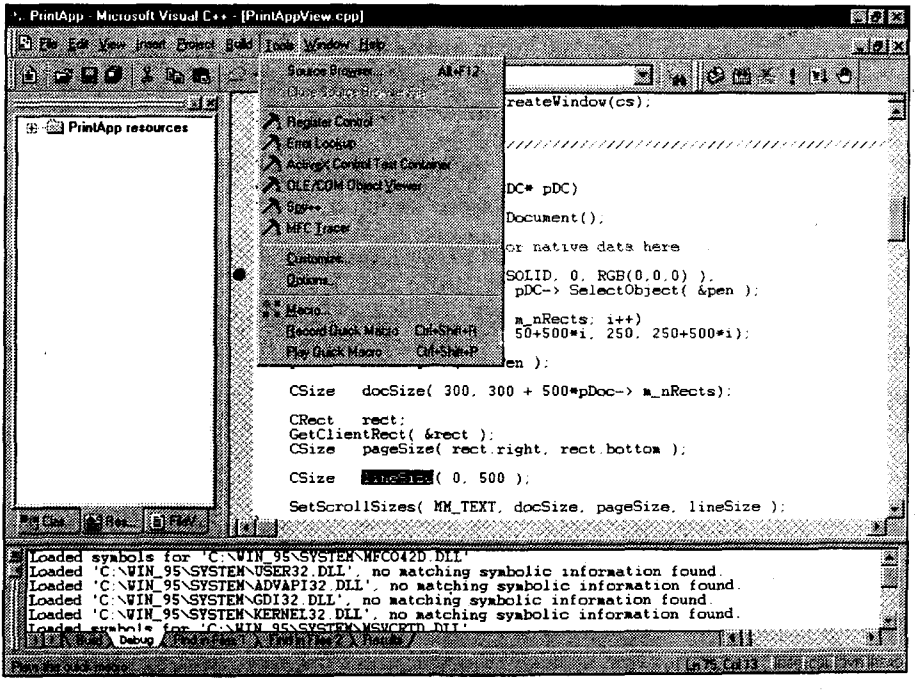


Рис. D.58. Меню **Tools**

Команда *Tools, Customize*

Эта команда выводит на экран диалоговое окно **Customize**, изображенное на рис. D.59.

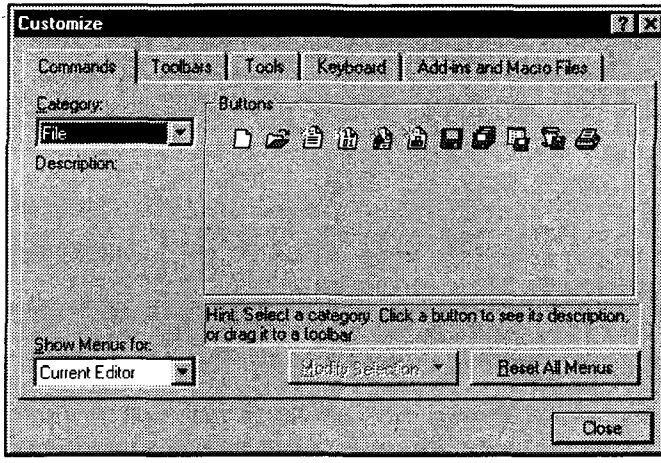


Рис. D.59. Диалоговое окно **Customize**

Данное диалоговое окно позволяет произвести настройку внешнего вида и других параметров среды программирования Visual C++. Вкладка **Commands** диалогового окна **Customize** позволяет добавить или удалить кнопки в панели инструментов. Вкладка **Toolbars** позволяет определить набор панелей инструментов, отображающихся на экране, задать их размер (крупные или нормальные кнопки), задать режим вывода подсказок к клавишам и определить, нужно ли указывать в подсказке к кнопке панели инструментов соответствующую ей комбинацию клавиш. Вкладка **Keyboard** позволяет установить новые или удалить старые комбинации клавиш.

Использование данного диалогового окна для работы с панелями инструментов описано ранее в данном приложении.

Команда *Tools, Source Browser* (<Alt>+<F12>)

Эта команда используется для расширения возможностей среды программирования Visual C++. Однако это требует создания специальной базы данных просмотра и соответствующих установок в диалоговом окне **Project Settings**.

Команда *Tools, Options*

Выводит на экран диалоговое окно **Options**, изображенное на рис. D.60.

Данное диалоговое окно имеет следующие вкладки:

- **Editor** (Редактор) — настраивает редактор среды программирования Visual C++. Устанавливает полосы прокрутки, разрешает режим перетаскивания

текста, устанавливает режимы автоматического сохранения перед запуском утилит и автоматической перегрузки файлов после их модификации в обход средств Visual C++, а также позволяет задать другие настройки редактора

- **Tabs** (Табуляция) — настраивает параметры табуляции и отступов в редакторе Visual C++
- **Debug** (Отладка) — определяет режим отладки и выводимую в нем информацию
- **Compatibility** (Совместимость) — позволяет эмулировать другой редактор (Brief, Epsilon или редактор Visual C++ версии 2.0) или создать собственный редактор на основе имеющихся свойств
- **Build** (Компиляция) — определяет параметры компиляции
- **Directories** (Директории) — определяет каталоги, в которых Visual C++ будет искать определенные типы файлов
- **Workspace** (Рабочая область) — устанавливает конфигурацию рабочей области среды программирования Visual C++
- **Data View** (Работа с базами данных) — устанавливает параметры работы с базами данных
- **Macros** (Макросы) — определяет поведение Visual C++ при внесении изменений в макросы
- **Help System** (Справочная система) — позволяет выбрать язык справочной системы (при всем богатстве выбора, другой альтернативы, кроме английского языка, нет) и источник справочной информации (одна из версий MSDN), если установлены несколько версий справочной системы.

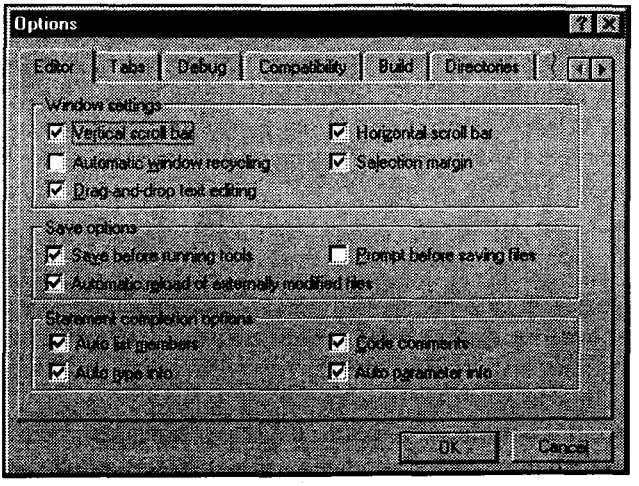


Рис. D.60. Диалоговое окно **Options**

Команда *Tools, Macro*

Выводит на экран диалоговое окно **Macro**, изображенное на рис. D.61.

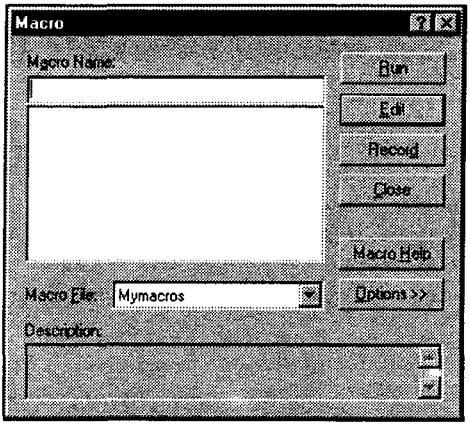


Рис. D.61. Диалоговое окно **Macro**

Это диалоговое окно позволяет записать или воспроизвести простой макрос, или редактировать записанную последовательность нажатия клавиш, добавляя в нее операторы VBScript.

Меню *Window*

Меню **Window**, приведенное на рис. D.62, содержит команды выбора окон и их автоматического упорядочения на экране.

Команда *Window, New Window*

Данная команда создает новое окно редактирования, куда помещается копия файла, которому соответствовало окно редактирования, имевшее фокус ввода в момент выбора данной команды меню. В строке заголовка к имени файла добавляется номер окна (например, ":2"). Если заголовок окна, имевшего до этого фокус ввода, не содержал его номер, то к имени файла в заголовке окна добавляется номер ":1". Изменения, произведенные в одном окне, немедленно отображаются в других окнах.

Команда *Window, Split*

Разделяет окно, имеющее в данный момент фокус ввода. При выборе данной команды в окне редактирования файла появляется штриховой крест, в центр которого помещен указатель мыши. Переместите указатель в нужную позицию и щелкните левой кнопкой мыши. Окно разобьется на четыре (или две, если поместить указатель на одной из сторон рабочей области окна) панели, как показано на рис. D.63.

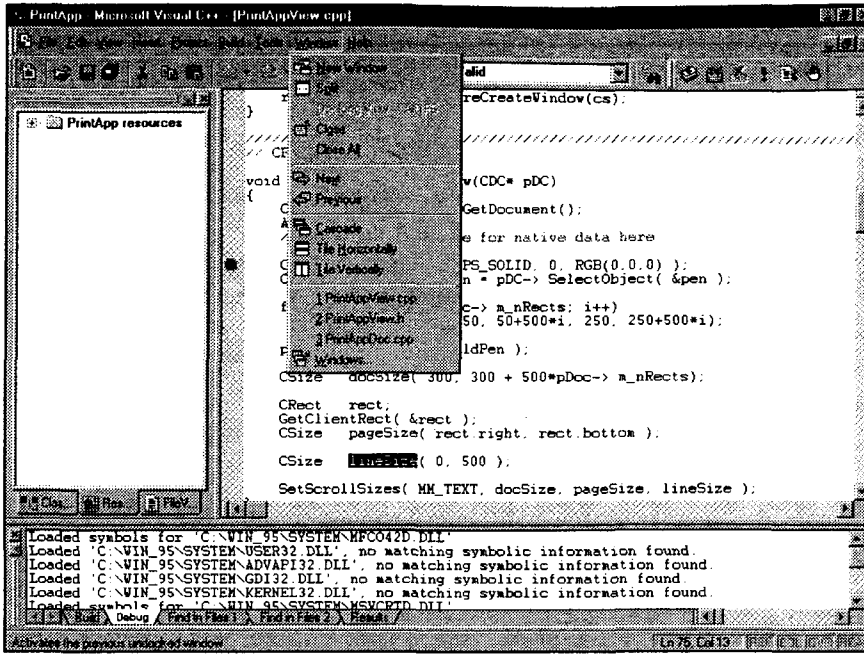


Рис. D.62. Меню **Window**

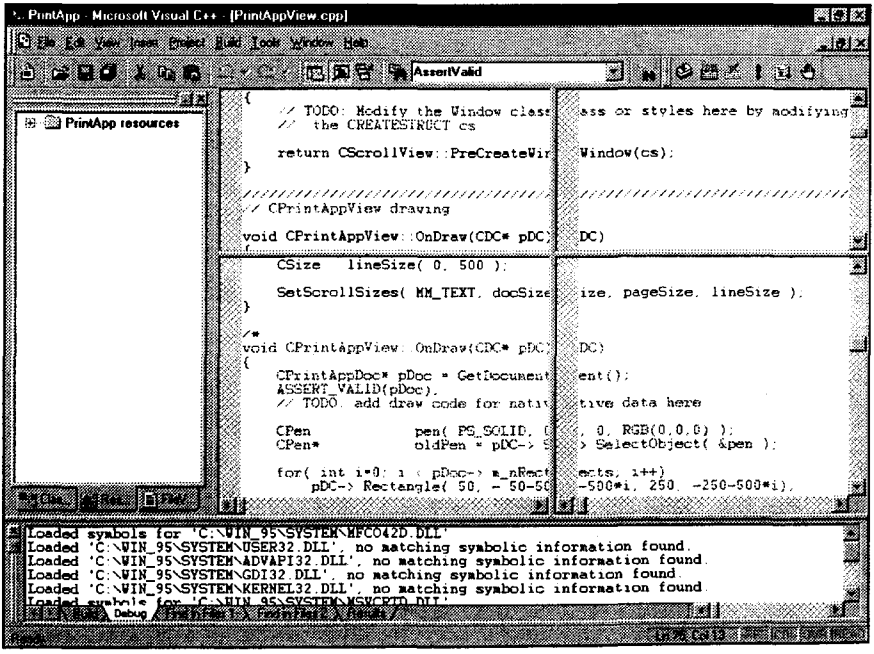


Рис. D.63. Разделенное окно

Горизонтальная и вертикальная полосы прокрутки каждой пары панелей работают независимо. Чтобы убрать разделение окна, выберите данную команду и переместите указатель в один из углов рабочей области окна.

Команда *Window, Docking View* (<Alt>+<F6>)

Данная команда управляет состоянием окон среды программирования Visual C++, таких как **Workspace**, **Output**, **Watch** и подобных им окон. Если такое окно имеет фокус ввода, и эта команда в меню отмечена флажком, то это окно является стационарным и отображается как панель главного окна Visual C++. В противном случае такое окно является обычным плавающим окном, и его имя включается в список открытых окон данного меню.

Команда *Window, Close*

Закрывает окно, имеющее в данный момент фокус ввода и связанный с ним файл. Если в файл были внесены изменения после его последнего сохранения, появляется окно сообщения с соответствующим текстом.

Команда *Window, Close All*

Эта команда закрывает окно, имеющее в данный момент фокус ввода, и связанный с ним файл. Если в файл были внесены изменения после его последнего сохранения, появляется окно сообщения с соответствующим текстом.

Команда *Window, Next*

Передает фокус ввода следующему окну. Порядок окон совпадает с их порядком в списке открытых окон данного меню.

Команда *Window, Previous*

Команда передает фокус ввода предыдущему окну. Порядок окон совпадает с их порядком в списке открытых окон данного меню.

Команда *Window, Cascade*

Упорядочивает окна, как это показано на рис. D.64.

Команда *Window, Tile Horizontally*

Команда размещает окна в рабочей области таким образом, чтобы каждое из них занимало всю ширину рабочей области, как показано на рис. D.65.

Окно, имеющее фокус ввода на момент выбора данной команды, располагается сверху.

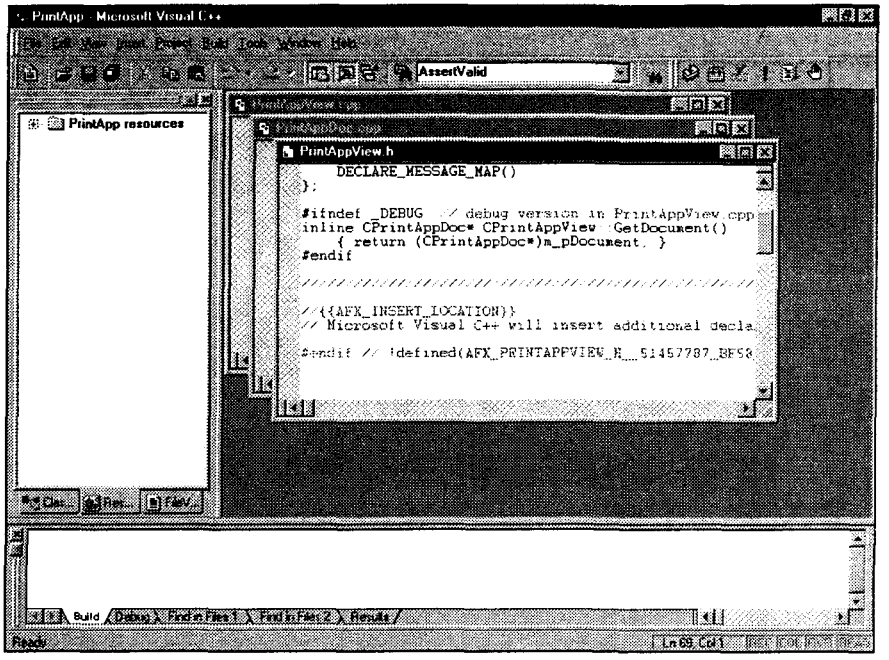


Рис. D.64. Упорядочение окон в соответствии со стандартным каскадным форматом

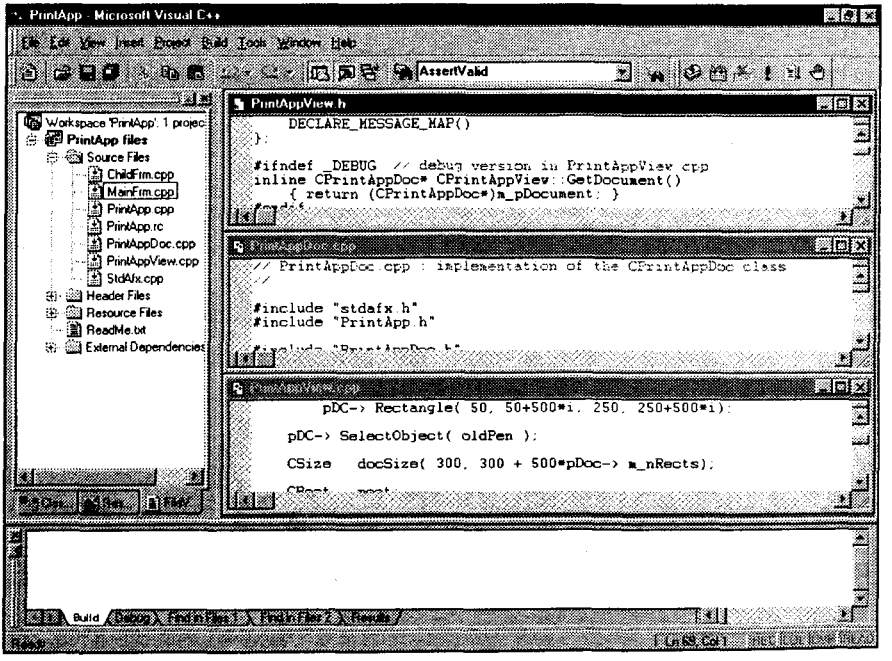


Рис. D.65. Упорядочение окон по горизонтали

Данная команда работает только в том случае, если число файлов в рабочей области не превышает трех. Если файлов больше, то она работает так же, как и команда **Window**, **Tile Vertically** (рис. D.66).

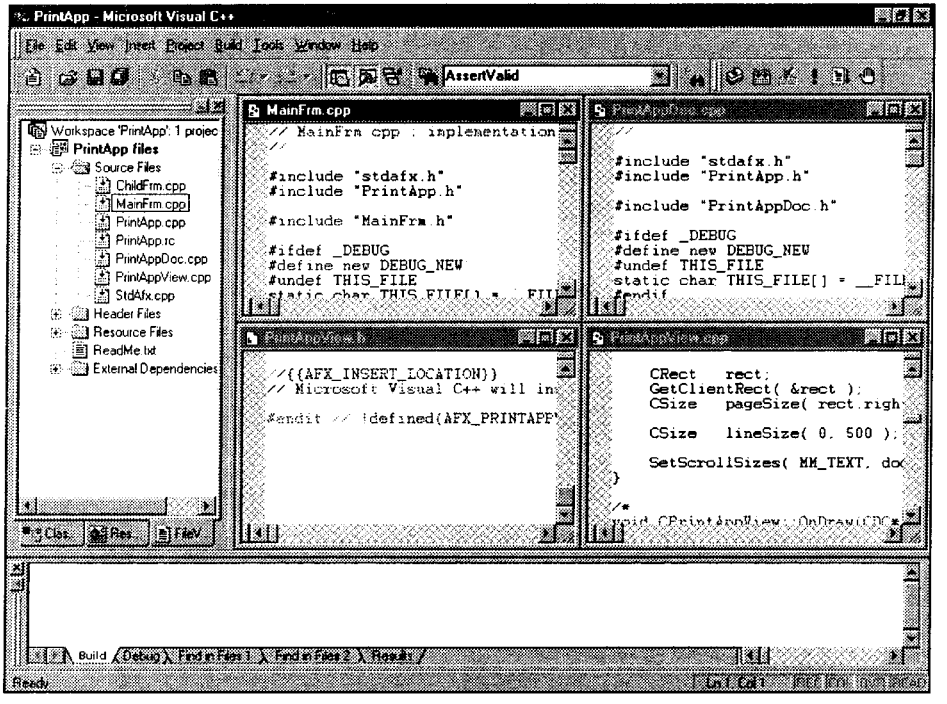


Рис. D.66. Упорядочение большого числа окон

Команда *Window, Tile Vertically*

Расставляет окна в рабочей области таким образом, чтобы каждое из них занимало всю высоту рабочей области, как показано на рис. D.67.

Окно, имеющее фокус ввода на момент выбора данной команды, располагается слева.

Список открытых окон

Нижняя часть меню **Windows** содержит список открытых окон, обеспечивающий легкое перемещение между ними. Если количество открытых окон больше десяти, то в данном списке отображаются только десять окон, расположенных в порядке убывания времени последнего обращения к ним пользователя. Для доступа к остальным раскрытым окнам следует выбрать команду меню **Window**, **Windows...**.

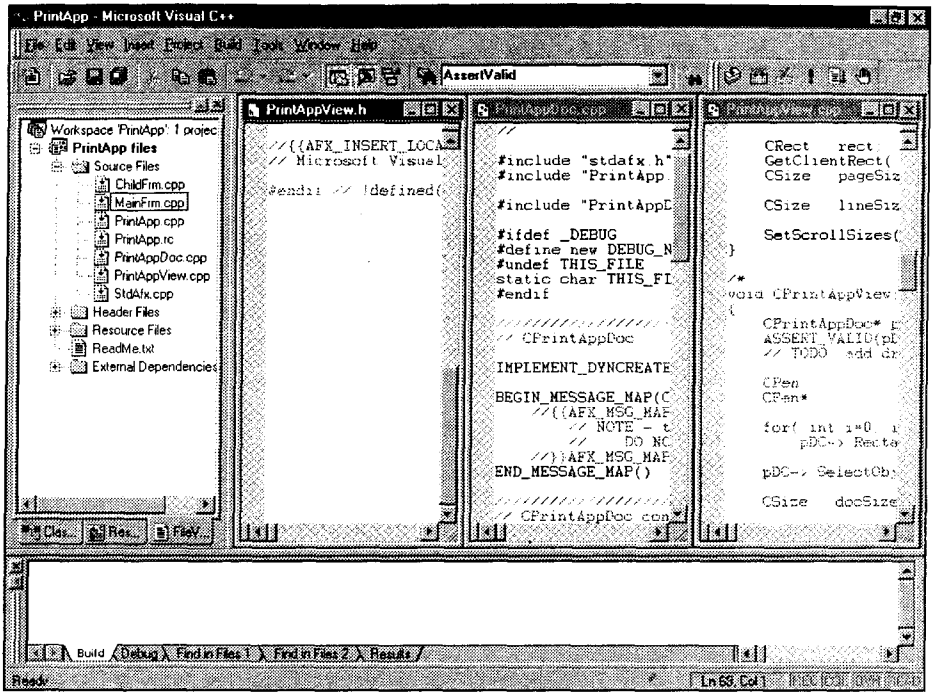


Рис. D.67. Упорядочение окон по вертикали

Команда Window, Windows

Выводит на экран диалоговое окно **Windows**, изображенное на рис. D.68.

Это диалоговое окно позволяет передавать фокус ввода окну, выделенному в окне списка **Select window:**, а также сохранять содержимое окон, выделенных в данном окне списка, и закрывать эти окна.

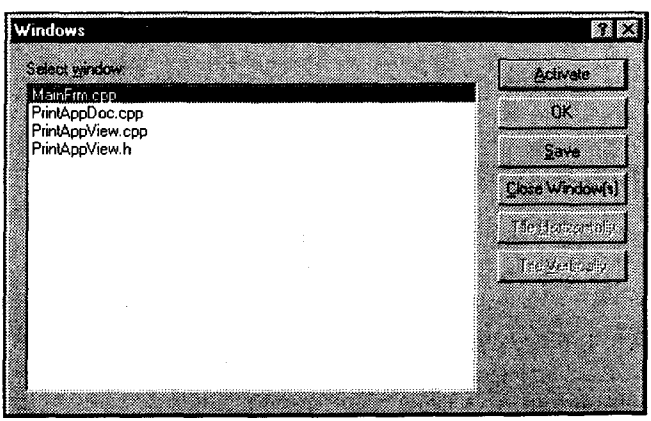


Рис. D.68. Диалоговое окно **Windows**

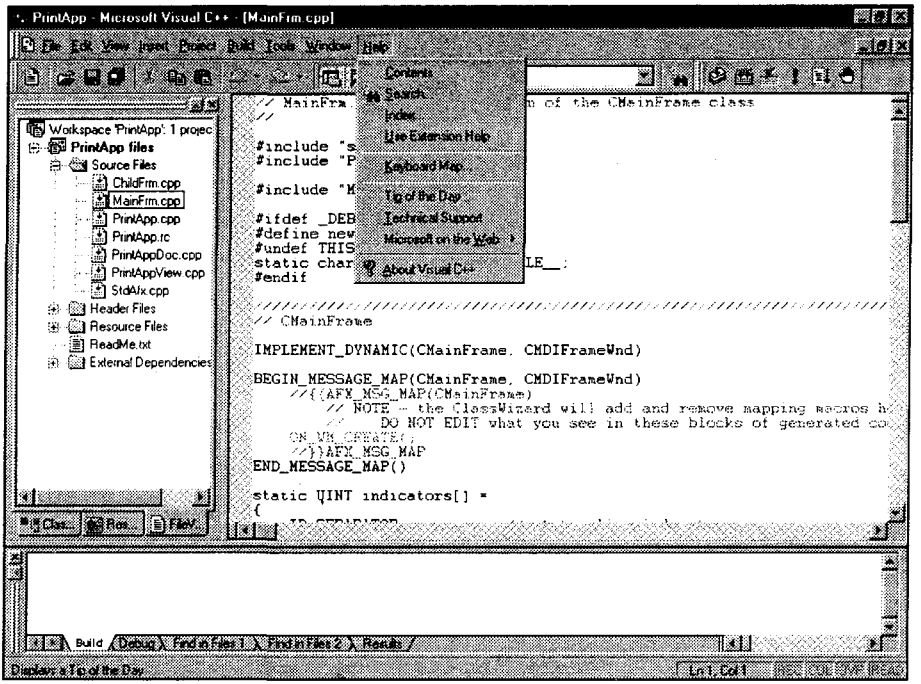


Рис. D.69. Меню **Help**

Меню *Help*

Меню **Help**, приведенное на рис. D.69, содержит команды обращения к справочной системе Visual C++.

Команда *Help, Contents*

Команда вызывает приложение MSDN Library Visual Studio 6.0, если до этого оно не было открыто, или передает ему управление. При этом в данном приложении раскрывается вкладка **Содержание**, как показано на рис. D.70.

Команда *Help, Search*

Вызывает приложение MSDN Library Visual Studio 6.0, если до этого оно не было открыто, или просто передает ему управление. При этом в данном приложении раскрывается вкладка **Поиск**, как показано на рис. D.71.

Команда *Help, Index*

Вызывает приложение MSDN Library Visual Studio 6.0, если до этого оно не было открыто, или передает ему управление. При этом в данном приложении раскрывается вкладка **Указатель**, как показано на рис. D.72.

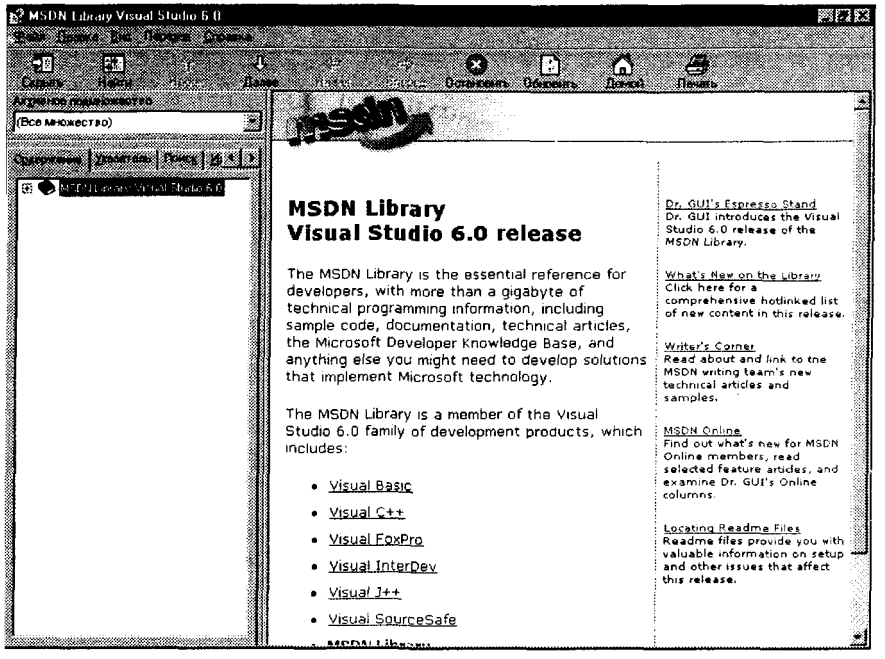


Рис. D.70. Приложение MSDN Library Visual Studio 6.0, вкладка **Содержание**

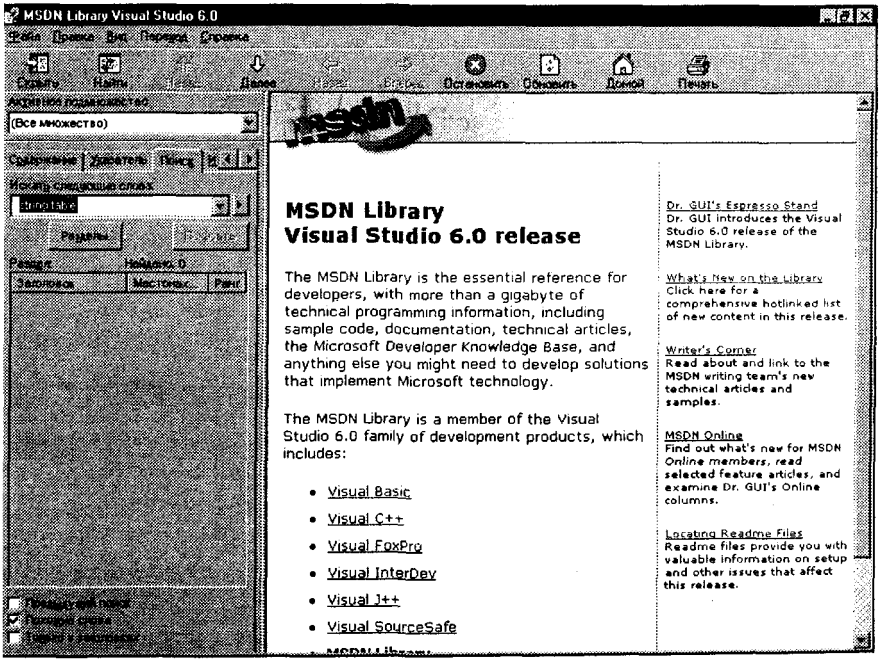


Рис. D.71. Приложение MSDN Library Visual Studio 6.0, вкладка **Поиск**

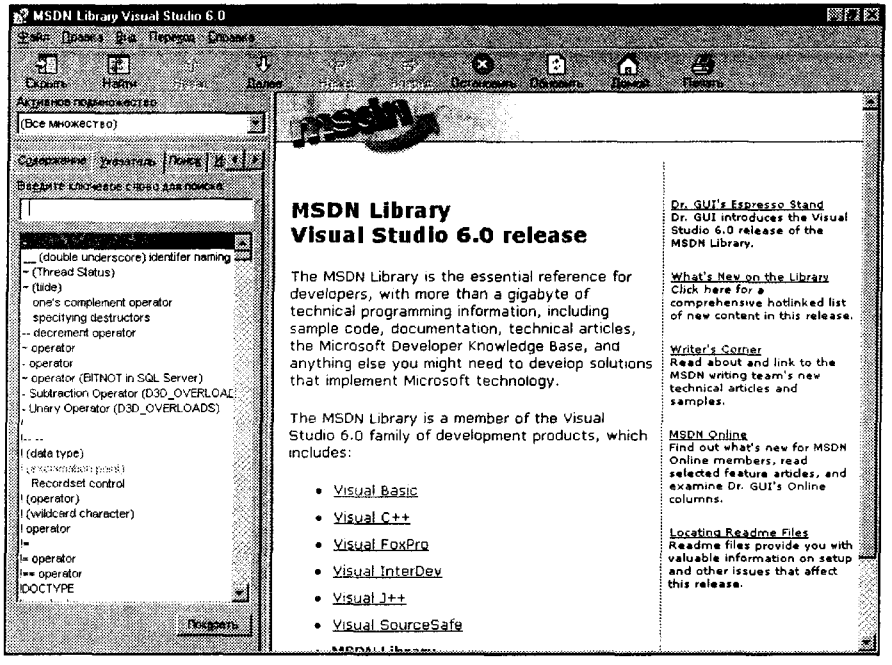
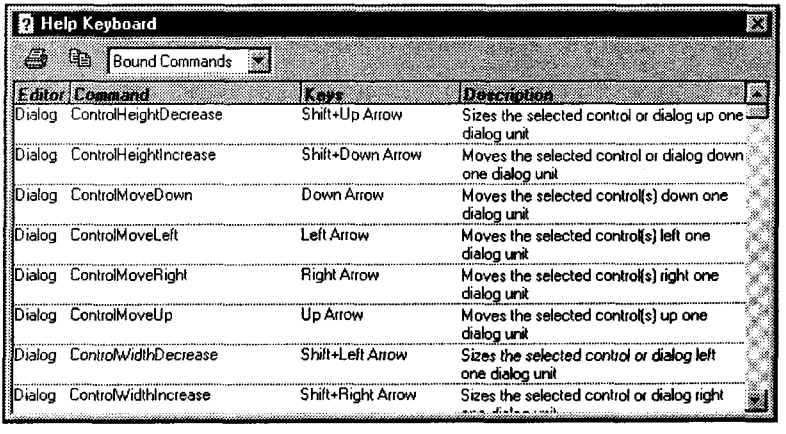


Рис. D.72. Приложение MSDN Library Visual Studio 6.0, вкладка **Указатель**

Команда *Help, Keyboard Map*

Выводит на экран диалоговое окно **Keyboard Map**, изображенное на рис. D.73.



Editor	Command	Keys	Description
Dialog	ControlHeightDecrease	Shift+Up Arrow	Sizes the selected control or dialog up one dialog unit
Dialog	ControlHeightIncrease	Shift+Down Arrow	Moves the selected control or dialog down one dialog unit
Dialog	ControlMoveDown	Down Arrow	Moves the selected control(s) down one dialog unit
Dialog	ControlMoveLeft	Left Arrow	Moves the selected control(s) left one dialog unit
Dialog	ControlMoveRight	Right Arrow	Moves the selected control(s) right one dialog unit
Dialog	ControlMoveUp	Up Arrow	Moves the selected control(s) up one dialog unit
Dialog	ControlWidthDecrease	Shift+Left Arrow	Sizes the selected control or dialog left one dialog unit
Dialog	ControlWidthIncrease	Shift+Right Arrow	Sizes the selected control or dialog right

Рис. D.73. Диалоговое окно **Keyboard Map**

В текстовом окне раскрывающегося списка, расположенном в верхней части данного диалогового окна, можно выбрать группу команд, для которой бу-

дут отображаться комбинации клавиш. Значение **Bound Commands** выводит все команды, которым соответствуют комбинации клавиш. Значение **All Commands** выводит все команды, независимо от того, соответствуют им комбинации клавиш или нет. Значения **File**, **Edit**, **View**, **Insert**, **Project**, **Build**, **Debug**, **Tools**, **Window** или **Help** выводят команды соответствующих меню. Кроме того, доступны категории **WizardBar**, **Image**, **Query** и **Diagram**.

Строки окна списка могут копироваться и распечатываться, как фрагментами, так и целиком.

Команда *Help, Tip of the Day*

Выводит на экран диалоговое окно **Tip of the Day**, изображенное на рис. D.74.

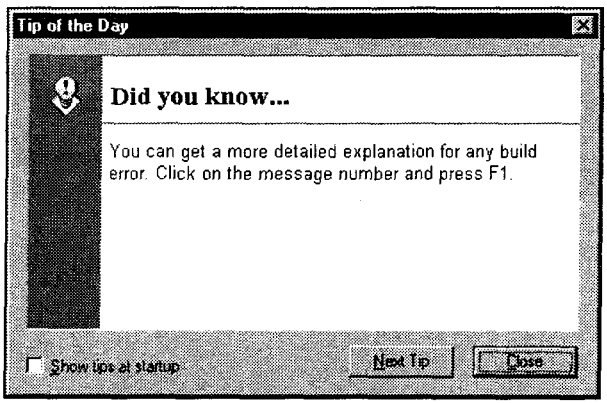


Рис. D.74. Диалоговое окно **Tip of the Day**

В данном окне предусмотрен флажок **Show tips at startup**, установив который, пользователь сможет наблюдать данное окно при каждом запуске Visual C++.

Команда *Help, Technical Support*

Эта команда вызывает приложение MSDN Library Visual Studio 6.0, если до этого оно не было открыто, или передает ему управление. При этом в данном приложении раскрывается вкладка **Getting Help from Microsoft Technical Support**, как показано на рис. D.75.

Примечание

Здесь перечислены услуги, предоставляемые фирмой Microsoft зарегистрированным пользователям, естественно, только тем, кто проживает на территории США и Канады. Остальные пользователи должны обращаться в региональные представительства. Например, в России это некое акционерное общество "Microsoft A/O", не имеющее зарегистрированного адреса и имеющее единственный контактный телефон (095) 913-99-88, что достаточно полно характеризует его надежность и компетентность работающих в нем специалистов.

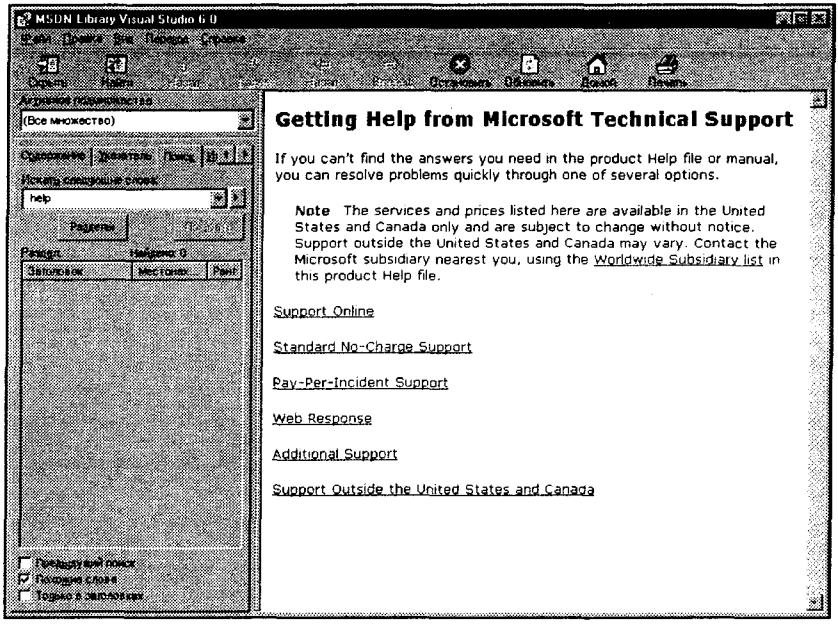


Рис. D.75. Приложение MSDN Library Visual Studio 6.0, вкладка **Getting Help from Microsoft Technical Support**

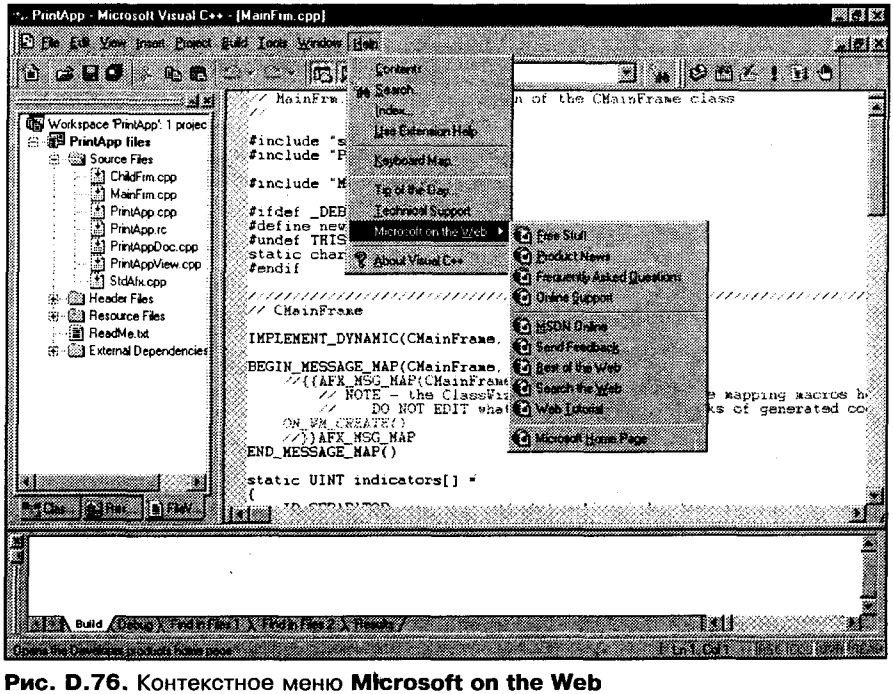


Рис. D.76. Контекстное меню **Microsoft on the Web**

Команда *Help, Microsoft on the Web*

Раскрывает контекстное меню, показанное на рис. D.76.

Выбор любой команды данного контекстного меню вызовет приложение Internet Explorer, который установит соединение с соответствующей страницей Web.

Команда *Help, About Visual C++*

Выводит на экран диалоговое окно **About Microsoft Visual C++**, содержащее информацию о пользователе данного продукта и о его регистрационном номере.

Окна Visual C++

Окно *Workspace*

Окно **Workspace** обычно располагается слева от окон редактирования в качестве панели главного окна системы программирования Visual C++. Данное окно используется для управления окнами редактирования файлов и ресурсов, а также для автоматизации разработки приложения.

Вкладка *ClassView*

Вкладка **ClassView** окна **Workspace**, изображенная на рис. D.77, используется для автоматизации работы с классами приложения.

Здесь отображаются папки проектов, содержащие список классов данного проекта. Каждый класс во вкладке **ClassView** окна **Workspace** представляет собой папку, содержащую члены данного класса, как это показано на рис. D.77. Функции класса располагаются первыми в его структуре, и рядом с ними расположен розовый значок. Переменным данного класса соответствует бирюзовый значок. Защищенные члены класса (`protected`) отмечаются ключом, а закрытые члены класса (`private`) — замком.

Щелчок правой кнопкой мыши на имени класса проекта раскрывает контекстное меню, изображенное на рис. D.78.

Данное контекстное меню содержит следующие команды:

- **Go to Definition** (Перейти к описанию) — открывает файл заголовка в том месте, где описан данный класс
- **Add Member Function** (Добавить в класс функцию) — открывает диалоговое окно **Add Member Function**, изображенное на рис. D.79. Работа с данным окном описана в Приложении C
- **Add Member Variable** (Добавить в класс переменную) — открывает диалоговое окно **Add Member Variable**, изображенное на рис. D.80

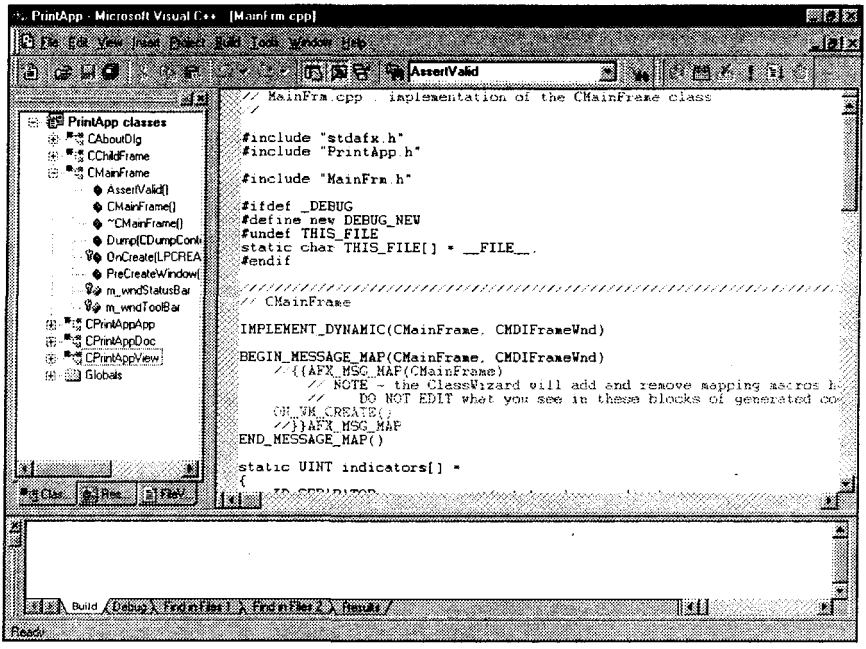


Рис. D.77. Вкладка **ClassView** окна **Workspace**

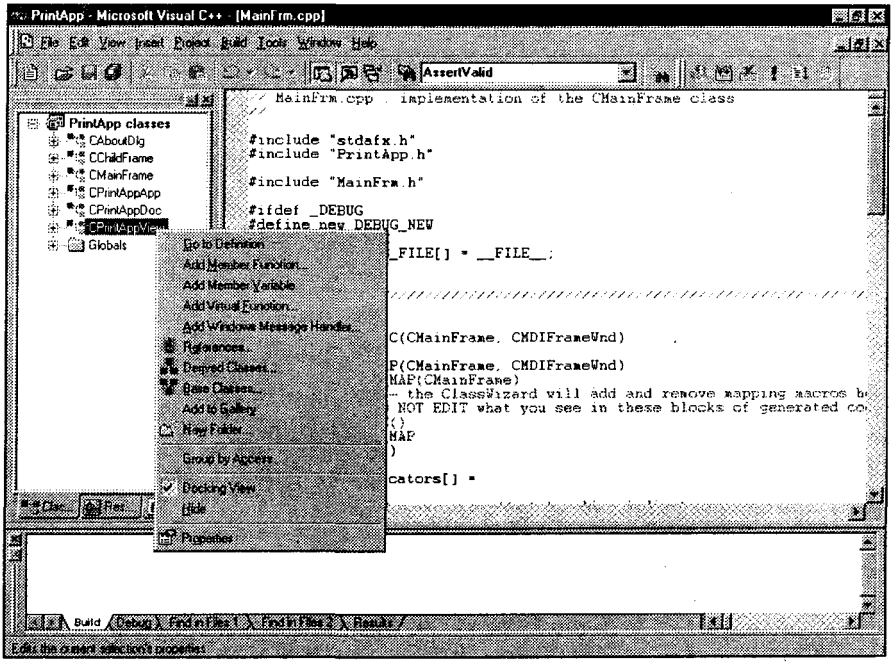


Рис. D.78. Вкладка **ClassView** окна **Workspace** с раскрытым контекстным меню

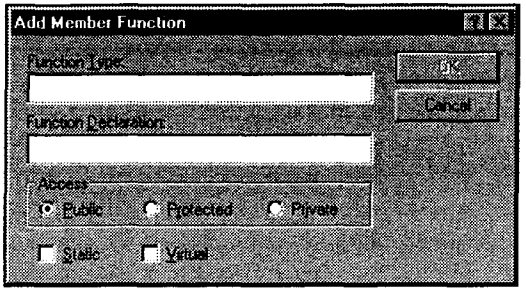


Рис. D.79. Диалоговое окно **Add Member Function**

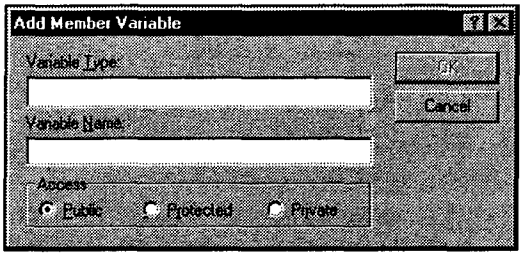


Рис. D.80. Диалоговое окно **Add Member Variable**

- **Add Virtual Function** (Добавить виртуальную функцию) — открывает диалоговое окно **New Virtual Override for class CMainFrame** (Новая перегрузка виртуальной функции для класса `CMainFrame`), изображенное на рис. D.81. (Имя класса совпадает с именем выделенного класса во вкладке **ClassView** окна **Workspace**.)

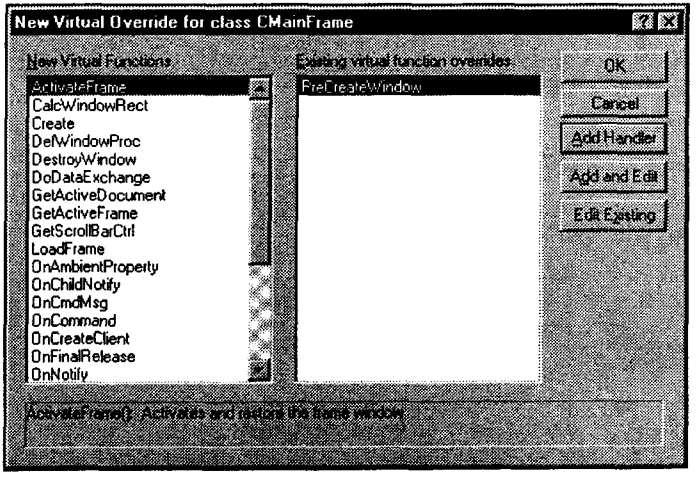


Рис. D.81. Диалоговое окно **New Virtual Override for class CMainFrame**

Чтобы добавить в класс заготовку перегруженной функции, выделите ее в окне списка **New Virtual Functions:** (Новые виртуальные функции) и на-

жмите кнопку **Add Handler** (Добавить функцию обработки). Имя функции появится в окне списка **Existing virtual function overrides** (Существующие перегруженные виртуальные функции). При нажатии кнопки **Add and Edit** диалоговое окно **New Virtual Override for class CMainFrame** будет закрыто, откроется окно редактирования соответствующего файла реализации, а текстовый курсор будет помещен в текст заготовки перегруженной виртуальной функции. То же самое произойдет и при выделении имени существующей функции в окне списка **Existing virtual function overrides** и нажатии кнопки **Edit Existing**

- **Add Windows Message Handler** (Добавить функцию обработки сообщения Windows) — открывает диалоговое окно **New Windows Message and Event Handlers for class CMainFrame** (Новая перегрузка виртуальной функции для класса `CMainFrame`), изображенное на рис. D.82. (Имя класса совпадает с именем выделенного класса во вкладке **ClassView** окна **Workspace**.)

  Работа с данным диалоговым окном описана в Приложении С.

- **References** (Ссылки) — выводит список ссылок на те строки в тексте программы, в которых упоминается имя класса в приложении. Требует создания файла базы данных

- **Derived Classes** (Производные классы) — выводит список всех членов данного класса, список классов, использующих данный класс в качестве базового, и другую сопутствующую информацию. Требует создания файла базы данных

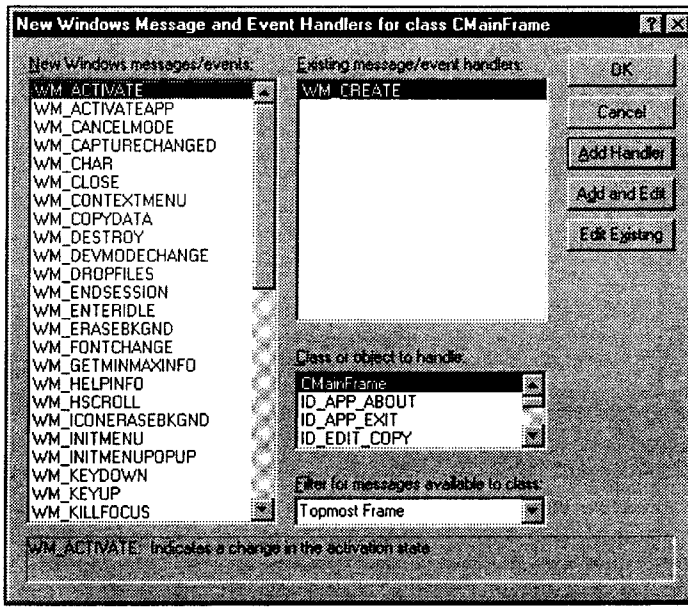


Рис. D.82. Диалоговое окно **New Windows Message and Event Handlers for class CMainFrame**

- **Base Classes** (Базовые классы) — выводит список всех членов данного класса, список классов, являющихся базовыми для данного класса, и другую сопутствующую информацию. Требует создания файла базы данных
- **Add to Gallery** (Включение в Component Gallery) — добавляет этот класс в Component Gallery

  См. также главу 3

- **New Folder** (Новая папка) — создает новую папку, в которую можно поместить файлы описания классов. Это бывает полезно в проекте с большим числом классов
- **Group by Access** (Группировка по доступу) — переупорядочивает список членов класса в соответствии с правами доступа. По умолчанию список функций и переменных класса строится в алфавитном порядке без учета права доступа к данной функции или переменной. При установке флажка у данной команды контекстного меню в списке сначала в алфавитном порядке перечисляются открытые (`public`) функции, затем в том же порядке защищенные (`protected`) функции, а затем — закрытые (`private`) функции. Далее в аналогичном порядке перечисляются переменные данного класса
- **Docking view** (Стационарная компоновка окон) — устанавливает отображение окна **Workspace** как панели главного окна среды программирования Visual C++. Если снять флажок рядом с данной командой контекстного меню, окно **Workspace** превратится в обычное плавающее окно, и его имя появится в списке открытых окон меню **Window**
- **Hide** (Скрыть) — убирает окно **Workspace** с экрана
- **Properties** (Свойства) — выводит диалоговое окно **Class Properties**, изображенное на рис. D.83

  Данное диалоговое окно содержит имя класса и имя его базового класса.

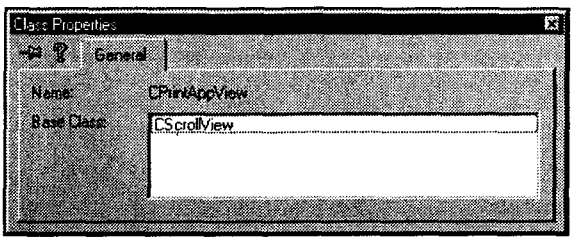


Рис. D.83. Диалоговое окно **Class Properties**

Щелчок правой кнопкой мыши на имени функции во вкладке **ClassView** окна **Workspace** приводит к появлению контекстного меню, основные команды которого приведены ниже:

- **Go to Definition** (Перейти к описанию) — открывает файл реализации в том месте, где содержится описание данной функции

- **Go to Declaration** (Перейти к объявлению) — открывает файл заголовка в том месте, где объявлена данная функция
- **Delete** (Уничтожить) — уничтожает объявление функции в файле заголовка и помещает ее описание в файле реализации в комментарий. Данная команда не может быть отменена, и ее результаты немедленно фиксируются в файлах проекта (выход без сохранения не помогает)
- **Set Breakpoint** (Установить точку останова) — устанавливает точку останова на заголовке данной функции. Данная точка останова визуализируется только в режиме отладки приложения и не позволяет произвести переход на себя из диалогового окна **Breakpoints**

Контекстное меню, появляющееся при щелчке правой кнопкой мыши на имени переменной во вкладке **ClassView** окна **Workspace**, является сокращенной версией рассмотренного выше контекстного меню.

Вкладка *ResourceView*

Вкладка **ResourceView** окна **Workspace**, изображенная на рис. D.84, используется для работы с редакторами ресурсов приложения.

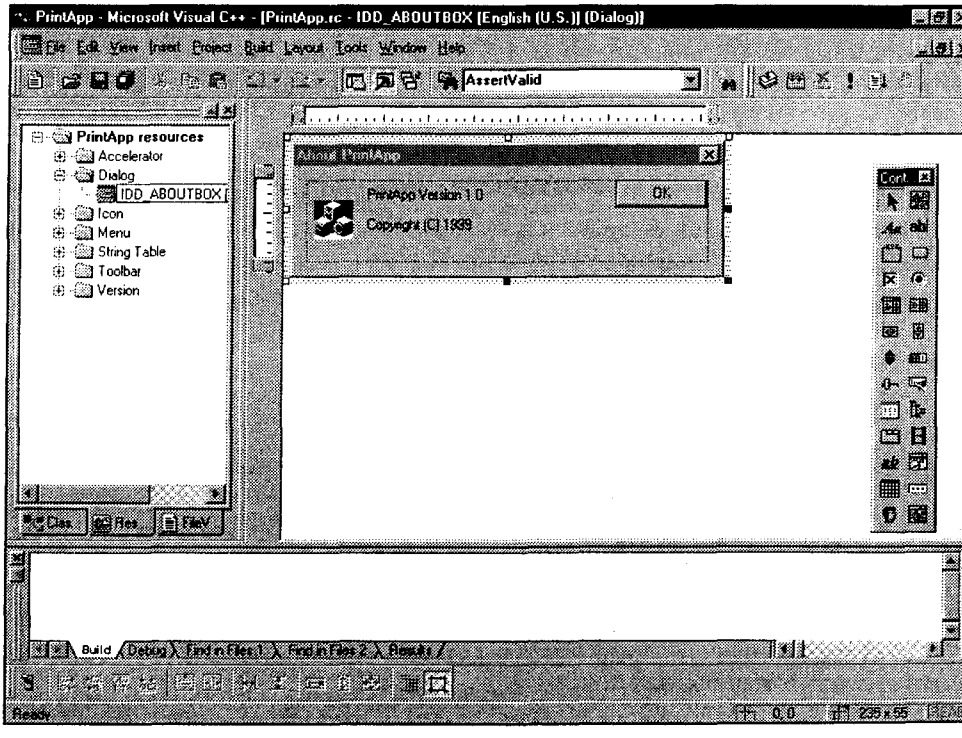


Рис. D.84. Вкладка **ResourceView** окна **Workspace**

Здесь отображаются папки проектов, содержащие папки с идентификаторами ресурсов данного проекта. Двойной щелчок левой кнопкой мыши на идентификаторе ресурса приводит к открытию окна редактирования данного ресурса. Вкладка **ResourceView** окна **Workspace** позволяет не только редактировать, но и создавать новые ресурсы. Для этого используется контекстное меню приложения. Вызов мастера ClassWizard из окна редактирования ресурса позволяет создать для него новый класс или связать его с уже существующим классом.

См. также главы 3 и 4

Вкладка *FileView*

Вкладка **FileView** окна **Workspace**, изображенная на рис. D.85, используется для работы с редакторами ресурсов приложения.

В данной вкладке отображаются папки проектов, содержащие папки с различными файлами данного проекта. Двойной щелчок левой кнопкой мыши на имени файла приводит к открытию окна редактирования данного файла. Нажатие клавиши <Delete>, когда фокус ввода принадлежит вкладке **FileView** окна **Workspace**, приводит к удалению выделенных в ней файлов из проекта.

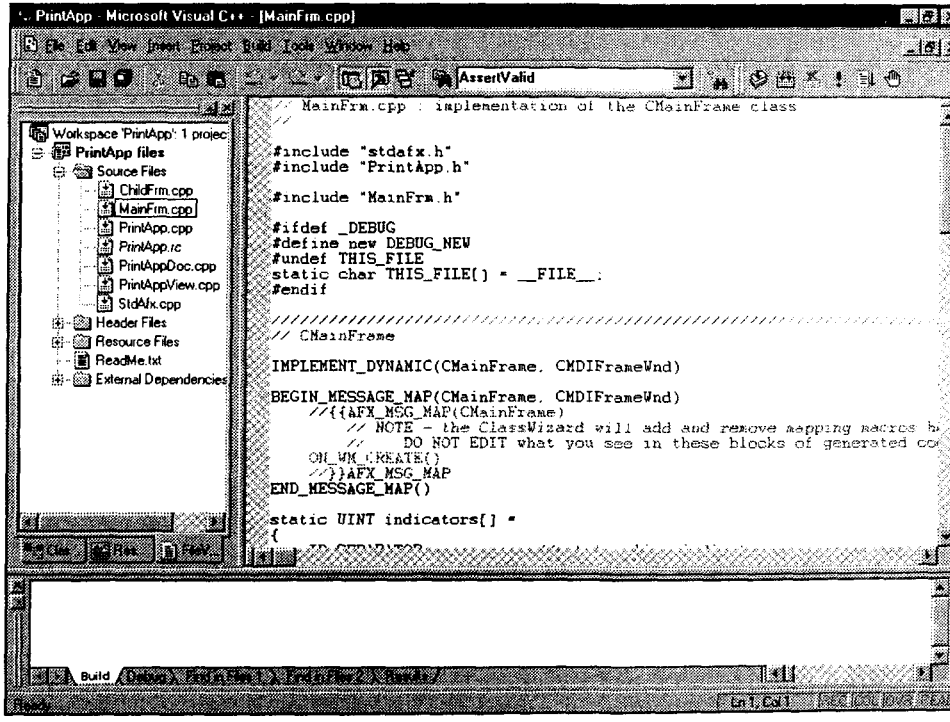


Рис. D.85. Вкладка **FileView** окна **Workspace**

Окно *Output*

Окно **Output**, изображенное на рис. D.86, обычно расположено под окнами редактирования в качестве панели главного окна среды программирования Visual C++. При запуске приложения на исполнение это окно обычно скрывается. Оно используется для вывода справочной информации средой программирования Visual C++.

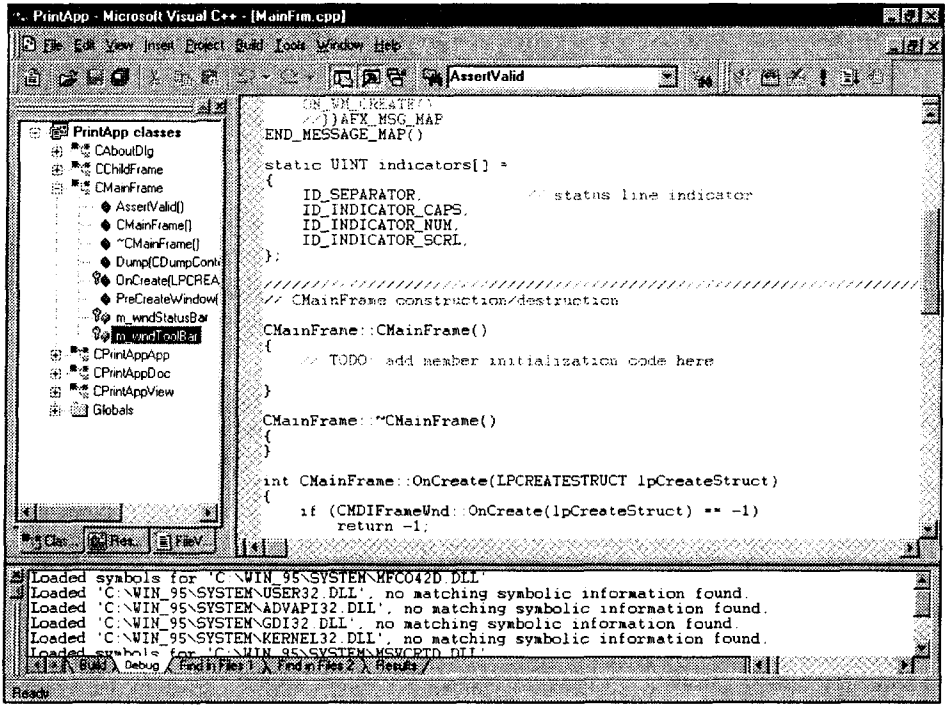


Рис. D.86. Окно **Output**

Окно **Output** имеет пять вкладок:

- **Build** (Компиляция и компоновка) — содержит результаты компиляции и компоновки
- **Debug** (Отладка) — содержит результаты отладки приложения, как показано на рис. D.86
- **Find in Files 1** (Поиск в файлах 1) — отображает результаты поиска с использованием диалогового окна **Find In Files**
- **Find in Files 2** (Поиск в файлах 2) — отображает результаты поиска с использованием диалогового окна **Find In Files**. Использование данной панели позволяет сохранить результаты предыдущего поиска

- **Results** (Результаты) — отображает результаты работы таких инструментальных средств, как отображение профиля программы

Редактирование текстов программ

Редактирование текстов программ, наряду с отладкой приложений и редактированием ресурсов, является основным режимом работы среды программирования Visual C++. Текстовый редактор Visual C++ работает аналогично многим другим текстовым редакторам, и едва ли его освоение представит для вас какую-либо сложность.

Чтобы ввести текст в файл, щелкните левой кнопкой мыши в том месте, где предполагается вставить текст, или любым другим способом переместите туда текстовый курсор. По умолчанию редактор находится в режиме вставки. Это означает, что ввод нового текста не уничтожает существующий. Для перехода в режим замены, когда при вводе каждого нового символа уничтожается расположенный в данной позиции символ существующего текста, необходимо нажать клавишу <Ins>. При этом в соответствующей панели строки состояния появится индикатор **OVR**. Для обратного перехода в режим вставки повторно нажмите клавишу <Ins>.

Для перемещения по тексту файла могут быть использованы полосы прокрутки, клавиши управления текстовым курсором, перемещающие текст на одну строку при достижении текстовым курсором верхней или нижней строки окна, или клавиши <PgUp> и <PgDn>, перемещающие текст в окне редактирования на одну страницу.

Чтобы выделить блок текста, поместите указатель мыши на одну из границ выделяемого фрагмента текста, нажмите левую кнопку мыши, и, удерживая ее, переместите указатель мыши на другую границу выделяемого фрагмента текста. Этого же эффекта можно достичь, установив текстовый курсор мыши на одну из границ выделяемого фрагмента текста и переместив его на другую границу при нажатой клавише <Shift>. Выделенный текст будет отображаться на черном фоне. Выделенный блок текста можно скопировать в буфер обмена или удалить клавишей <Del>.

При отображении текста программы в Visual C++ применяется *синтаксическая раскраска.* По умолчанию текст программы отображается черным цветом на белом фоне. Комментарии отображаются зеленым цветом. Зарезервированные слова языка C++ отображаются синим цветом. К зарезервированным относятся такие слова, как `class`, `for`, `int`, `new` и другие дескрипторы и операторы языка C++.

Наличие синтаксической раскраски помогает обнаружить грубые ошибки при написании программ. Например, если вместо `for` ввести `far`, то вывод его черным цветом укажет на допущенную ошибку. Зеленая раскраска комментариев позволяет сразу определить, что имеющийся в них код не будет выполняться.

Чтобы изменить параметры синтаксической раскраски или ввести свою синтаксическую раскраску, выберите команду меню **Tools**, **Options** и в появившемся диалоговом окне **Options** раскройте вкладку **Format**, изображенную на рис. D.87.

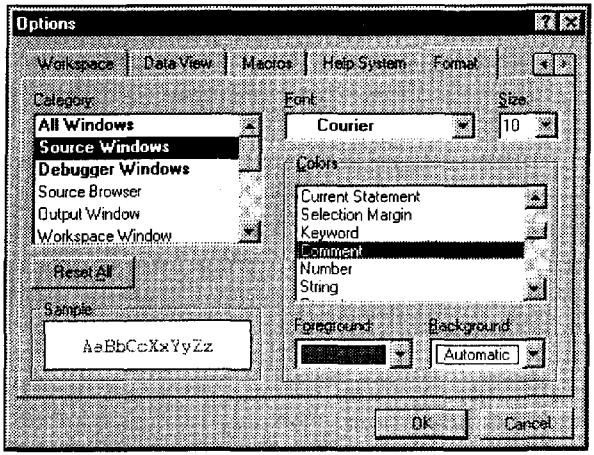


Рис. D.87. Диалоговое окно **Options**

В окне списка **Category:** выделяется категория окон, для которых будут действовать данные установки. В окне **Font:** выбирается используемый шрифт, а в окне **Size:** — его размер. В окне списка группы **Colors** выделяется элемент окна, для которого будет использоваться данная синтаксическая раскраска, в раскрывающемся списке **Foreground:** выбирается цвет текста данного элемента окна, а в раскрывающемся списке **Background:** — его фон.

Предметный указатель

А

Аргументы по умолчанию 533
Архив 245
Атрибуты
  прав доступа 527
  устройств 206

Б

Битовый образ 236, 237
Блок
  catch 341, 348, 721
  try 341
Бордюр 861

Г

Галерея компонентов 101
Глобальные функции Windows 537

Д

Демонстрационное приложение
  BrushApp 229
  Console 14
  DDBApp 237
  DialogApplication 22, 60
  EditApp 271
  LineApp 225
  Gallery 103
  ListApp 115
  MFC ClassWizard 841
  MTGDI 393
  PrintApp 311
  ProgressApp 123
  Reflect 187
  RichApp 279
  SheetApplication 85
  TextApp 220, 245
  ToolApp 290
Дескриптор HINSTANCE 540
Деструктор класса 528
Диалоговое окно
  Add Member Function 75, 841
  Add Member Variable 72, 117, 126, 190, 193, 237, 312, 849
  Adding a Class 86, 189
  Advanced Options 32
  Batch Build 898
  Bookmarks 880
  Breakpoints 503, 882
  Browse Files 274
  Change Files 274
  Check Box Properties 125
  Components and Controls Gallery 103
  Configurations 900
  Customize 906
  Dialog Properties 61, 86, 274
  Edit Properties 274
  Exceptions 902
  Find 873
  Find In Files 875, 886, 926
  Go To 879
  Group Box Properties 234
  Insert Files into Project 102
  Insert Resource 87, 292, 889
  Insert Resource Copy 890
  Keyboard Map 916
  Macro 908
  Menu Item Properties 108, 273, 281, 285
  MFC AppWizard 272, 288, 311, 499
  MFC ClassWizard 72, 196
  Module List 904
  New 14, 43, 464, 866
  New Class 190, 830, 887
  New Form 889
  New Project Information 16, 37
  New Symbol 306
  New Windows Message and Event Handlers for class CDialogApplicationDlg 200
  New Windows Message and Event Handlers for class CMyListCtrl 183
  Open 249, 252, 264, 866
  Open Workspace 102, 143, 867
  Options 906
  Page Setup 869
  Paste Special 279
  Picture Properties 125
  Print 870

Диалоговое окно
Progress Properties 124
Project Dependencies 893
Project Settings 344, 506, 893
Push Button Properties 117
QuickWatch 505, 904
Repair Class Information 825
Replace 878
Resource Includes 884
Resource Symbols 306, 883
Save As 248, 252, 264, 868
Select Class 294, 833
Select Source File 827
Set Active Configuration 900
Set Active Project Configuration 142, 396
Slider Properties 124
String Properties 250, 306
Text Properties 274
Threads 903
Tip of the Day 917
Toolbar Button Properties 293
Toolbar Properties 293
Windows 913
Запуск программы 268
Найденные разделы 433
Шрифт 213, 544—546, 661
Динамическая проверка корректности данных 848
Динамический обмен данными в диалоговом окне 848
Документы
составные 29
разбиение на страницы 324
Дюйм логический 219

И

Иерархия классов 522
Изменение положения элемента управления 63
Инкапсуляция 520, 521
Интерфейс
графических устройств 49, 205, 607
многооконный 27
однооконный 37, 652
Исключение 341
CFileException 648
STATUS_NO_MEMORY 566

К

Карта
дескрипторов 195
сообщений 78, 173
отображений 367
Кисть 229
Класс
CArchive 44, 245, 589
CArchiveException 346
CArray 371, 593
CBitmap 242, 597
CBitmapButton 599
CBrush 136
CButton 599
CByteArray 706
CClientDC 207, 607
CCmdTarget 173, 601, 836
CCmdUI 601
CComboBox 417
CCommandLineInfo 57, 603, 766
CControlBar 604
CCriticalSection 392
CDaoException 346
CDataExchange 852
CDBException 346
CDC 242, 607
CDialog 626
CDocTemplate 631, 652
CDocument 39, 634
CDumpContext 510, 594
CDWordArray 706
CEdit 271, 638
CEditView 640
CEvent 392
CFile 255, 589, 644
CFileException 346
CFont 332, 650
CFrameWnd 57, 651
CGdiObject 658
CHeaderCtrl 161, 658
CImageList 665
CList 371, 380, 671
CListBox 678
CListCtrl 138, 684
CMap 371, 380, 697
CMapPtrToPtr 373
CMapPtrToWord 373
CMapStringToOb 373
CMapStringToPtr 373
CMapWordToPtr 373
CMemoryException 346
CMenu 700
CMetaFileDC 207
CMeteFileDC 607
CMultiDocTemplate 56, 632
CMultiLock 392, 704
CMutex 392, 409, 703
CNotSupportedException 346
CObArray 372, 705
CObject 370, 709

Класс
COblList 372, 375
COleDispatchException 346
COleException 346
CPaintDC 207, 607
CPoint 402, 712
CPrintInfo 331, 712
CProgressCtrl 713
CPropertyPage 716
CPropertySheet 717
CPtrArray 370, 372, 706
CPtrList 370, 372
CRect 613, 722
CResourceException 346
CRgn 137, 727
CRichEditView 271, 725
CScrollBar 132
CScrollView 728
CSemaphore 392, 732
CSingleDocTemplate 56, 632
CSingleLock 392, 409, 704, 733
CSize 317, 619, 735
CSliderCtrl 736
CStatusBar 740
CString 743
CStringArray 706
CTime 748
CTimeSpan 749
CToolbar 302
CTypedPtrArray 372
CTypedPtrList 372
CTypedPtrMap 373
CUIntArray 706
CUserException 346
CView 39, 46, 752
CWinApp 20, 54, 195, 384, 760
CWindowDC 207, 607
CWinThread 54, 384, 539, 771
CWnd 775
CWordArray 706
приложения 50
приоритета процесса 578
синхронизации
доступа 392
работы 392
Кнопка
<Назад 718
About 289
Build 863
Compile 863
Copy 289, 863
Cut 289, 863
Execute Program 864
Find in Files 863
Finish 272
Go 864
Help 569
Insert/Remove Breakpoint 503, 864
Keep Visible 61
New 289
New Text File 862
Open 249, 289, 862
Output 863
Paste 289, 863
Print 289
Redo 863
Save 289, 862
Save All 863
Search 863
Stop Build 863
Undo 863
Window List 863
Workspace 863
Готово 718
Далее> 718
Закрыть 247
Открыть 249
Применить 718
Сохранить 248
Команда
HELP_COMMAND 586
HELP_CONTENTS 431, 586
HELP_CONTEXT 587
HELP_CONTEXTMENU 431, 587
HELP_CONTEXTPOPUP 432, 587
HELP_FINDER 430, 587
HELP_FORCEFILE 587
HELP_HELPONHELP 587
HELP_INDEX 431, 587
HELP_KEY 433, 588
HELP_MULTIKEY 433, 588
HELP_PARTIALKEY 433, 588
HELP_QUIT 436, 588
HELP_SETCONTENTS 588
HELP_SETPOPUP_POS 588
HELP_SETWINPOS 435, 442, 589
HELP_TCARD 435, 589
HELP_WM_HELP 431, 440, 589
Команда меню
Build, Batch Build 898
Build, Build 897
Build, Clean 898
Build, Compile 896
Build, Configurations 900
Build, Debugger Remote Connection 899
Build, Execute 900
Build, Profile 901
Build, Rebuild All 897
Build, Set Active Configuration 900
Debug, Exceptions 902

Команда меню
Debug, Go 901
Debug, QuickWatch 505, 904
Debug, Restart 501, 902
Debug, Run to Cursor 502, 902
Debug, Step Into 502, 902
Debug, Step Out 502, 902
Debug, Step Over 502, 902
Debug, Stop Debugging 501, 902
Debug, Threads 903
Debug,Go 501
Debug,Modules 904
Edit, Advanced 881
Edit, Bookmarks 880
Edit, Breakpoints 503, 882
Edit, Copy 873
Edit, Cut 873
Edit, Delete 873
Edit, Find 873
Edit, Find in Files 875
Edit, Go To 879
Edit, Paste 873
Edit, Redo 872
Edit, Replace 878
Edit, Select All 873
Edit, Undo 871
File, Close 867
File, Close Workspace 868
File, Exit 871
File, New 636, 866
File, Open 249, 866
File, Open Workspace 143, 246, 867
File, Page Setup 869
File, Print 870
File, Print Preview 319
File, Recent Files 870
File, Recent Workspaces 246, 871
File, Save 868
File, Save All 869
File, Save As 868
File, Save Workspace 868
Help, About Visual C++ 919
Help, Contents 914
Help, Index 139, 914
Help, Keyboard Map 916
Help, Microsoft on the Web 919
Help, Search 914
Help, Technical Support 917
Help, Tip of the Day 917
Insert Dialog 273
Insert, New ATL Object 890
Insert, New Class 829, 887
Insert, New Form 889
Insert, Resource 889
Insert, Resource Copy 890
Project, Add to Project 891
Project, Add to Project, Components and Controls 103
Project, Add To Project, Files 102
Project, Dependencies 893
Project, Set Active Project 890
Project, Settings 344, 506, 893, 901
Start Debug 898
Tools, Customize 906
Tools, Macro 908
Tools, Options 906
Tools, Source Browser 906
View, ClassWizard 883
View, Debug Windows 886
View, Full Screen 885
View, Output 886
View, Properties 887
View, Resource Includes 884
View, Resource Symbols 306, 883
View, Workspace 886
Window, Cascade 910
Window, Close 910
Window, Close All 910
Window, Docking View 910
Window, New Window 652, 908
Window, Next 910
Window, Previous 910
Window, Split 908
Window, Tile Horizontally 910
Window, Tile Vertically 912
Window, Windows 913
Консольное приложение 13
Конструктор класса 528
Контекст устройства 206, 607
атрибуты 318
Контекстная справка 436
Концепция Документ/Представление 39
Координаты
логические 319
физические 318

M

Макрос
AND_CATCH 355
ASSERT 508
BEGIN_MESSAGE_MAP 173
CATCH 352, 354, 355
DECLARE_DYNACREATE 57, 386, 652
DECLARE_DYNAMIC 709
DECLARE_DYNCREATE 528
DECLARE_MESSAGE_MAP 173
DECLARE_SERIAL 590, 709
END_CATCH 355
END_MESSAGE_MAP 173

Макрос
IMPLEMENT_DYNACREATE 57, 386
IMPLEMENT_DYNAMIC 709
IMPLEMENT_DYNCREATE 530
IMPLEMENT_SERIAL 375, 590, 698, 705, 709
MAKEINTRESOURCE 654
ON_COMMAND 174
ON_COMMAND_EX 296
ON_COMMAND_RANGE 174
ON_CONTROL 175
ON_CONTROL_RANGE 175
ON_CONTROL_REFLECT 186
ON_CONTROL_REFLECT_EX 187
ON_EDIT_CHANGE 639
ON_EN_ERRSPACE 639
ON_EN_HSCROLL 639
ON_EN_KILLFOCUS 639
ON_EN_SETFOCUS 639
ON_EN_UPDATE 639
ON_EN_VSCROLL 639
ON_ENMAXTEXT 639
ON_MESSAGE 175
ON_NOTIFY 151, 178
ON_NOTIFY_REFLECT 186
ON_NOTIFY_REFLECT_EX 187
ON_NOTIFY_REFLECTION 151
ON_UPDATE_COMMAND_UI 175, 203
ON_UPDATE_COMMAND_UI_RANGE 175
ON_UPDATE_COMMAND_UI_REFLECT 186
ON_WM_CHARTOITEM_REFLECT 187
ON_WM_COMPAREITEM_REFLECT 187
ON_WM_CTLCOLOR_REFLECT 186
ON_WM_DELETEITEM_REFLECT 186
ON_WM_DRAWITEM_REFLECT 186
ON_WM_HSCROLL_REFLECT 187
ON_WM_MEASUREITEM_REFLECT 186
ON_WM_PARENTNOTIFY_REFLECT 187
ON_WM_VKEYTOITEM_REFLECT 187
ON_WM_VSCROLL_REFLECT 187
RUNTIME_CLASS 56
TCard 435
THROW 355
THROW_LAST 355
TRACE 302, 508
TRY 355
UPDATE_COMMAND_UI 652
карты сообщений 599
исключений 344
Маска обратного вызова 686
Массивы 367
Мастер
ClassWizard 71
MFC AppWizard 13
Меню
Build 896
Debug 901
Edit 871
File 865
Help 914
Insert 887
Project 890
Tools 905
View 882
Window 908
Многозадачность
кооперативная 383
вытесняющая 383

H

Набор символов
ASCII 211
Unicode 212
Наследование 520
Начертание 209

O

Обработка сообщений 151, 179, 836, 844
Объект 524
графический 206
интерфейса графических устройств 658
Объектно-ориентированное программирование 519
Окно
MFC AppWizard 437
MSDN Library Visual Studio 6.0 139, 393
Output 886
Output 926
Variables 504
Watch 504
Workspace 886, 919
Вкладка ClassView 919
Вкладка FileView 925
Вкладка ResourceView 924
Редактор реестра 268
Справка 426
Справочная система 426
Оператор throw 341
Определение координат графических объектов 317
Отображение
значков 684
отчета 684
списка 684
Очередь сообщений 172

П

Панель инструментов 288
  Build 863
  Build MiniBar 863
  Controls 62, 116, 124
  Dialog 70
  Standard 862, 872
  WizardBar 183
  WizardBar 824, 829, 836
  плавающая 861
Перегрузка операторов и функций 532—534
Полиморфизм 520, 522
Полоса прокрутки
  Бегунок 314
  Страница 317
  Строка 317
Пользовательская система координат 319
Пользовательский линейный индикатор 133
Последовательный доступ к файлу 254, 589
Потоки 383
  прекращение работы 388
  синхронизация 384, 390
  функция исполняющая 385
Предварительный просмотр печати 319
Приложение
  Help Workshop 497
  RegEdit.exe 268
Приоритет
  потока 578
  базовый уровень 578—580
Произвольный доступ к файлу 255
Прокрутка в окне 311
Процесс 383

Р

Режим EVENT_ALL_ACCESS 548
Режим отображения 623, 730
  MM_LOENGLISH 723
  MM_TEXT 804

С

Система координат оконная 318
Скрытие данных 521
Создание
  колонтитулов 331
  очереди 381
  стека 381
Сообщение
  BN_CLICKED 75
  CBN_SELENDOK 76
  EM_GETCHARFORMAT 800
  EM_SETCHARFORMAT 800
  EN_PROTECTED 726
  EN_UPDATE 639
  HDN_GETDISPINFO 801, 802
  HELPMSGSTRING 664
  ID_EDIT_FIND 641
  ID_EDIT_PRINT 641
  ID_EDIT_REPEAT 641
  ID_EDIT_REPLACE 641
  ID_EDIT_SELECT_ALL 641
  LBN_DBLCLK 119
  LBS_SELCHANGE 680
  LVM_FINDITEM 811
  LVM_GETITEM 810
  LVM_SORTITEMS 811
  LVN_ENDLABELEDIT 685
  LVN_GETDISPINFO 810
  NM_CLICK 177
  NM_DBLCLK 177
  NM_KILLFOCUS 178
  NM_OUTOFMEMORY 178
  NM_RCLICK 177
  NM_RDBLCLK 178
  NM_RETURN 178
  NM_SETFOCUS 178
  ON_BN_CLICKED 600
  ON_BN_DOUBLECKICKED 600
  ON_COMMAND_EX 602
  ON_UPDATE_COMMAND_UI 601
  TB_LINEDOWN 738
  TB_LINEUP 738
  TB_PAGEDOWN 739
  TB_PAGEUP 739
  TranslateMessage 172
  TTN_POP 177
  TTN_SHOW 177
  UPDATE_COMMAND_UI 259, 294, 295, 303
  WM_CAPTURECHANGED 573
  WM_CHAR 680
  WM_CHOOSEFONT_GETLOGFONT 546, 661
  WM_CHOOSEFONT_SETFLAGS 546
  WM_CHOOSEFONT_SETLOGFONT 546
  WM_CLOSE 296, 303, 420
  WM_COMMAND 176, 202, 424, 439, 545, 661
  WM_COMMANDHELP 441
  WM_CONTEXTMENU 425, 439
  WM_CTLCOLORDLG 545
  WM_DESTROY 436, 777
  WM_DRAWITEM 685, 702
  WM_ENABLE 779
  WM_ERASEBKGND 542, 793

Сообщение
WM_ERASEBKGND 823
WM_FONTCHANGE 214, 538, 574
WM_HELP 425, 440, 789
WM_HELPHITTEST 443
WM_HSCROLL 132, 730
WM_INITDIALOG 545, 631, 664
WM_INITMENU 783
WM_KEYDOWN 680
WM_MEASUREITEM 702
WM_MOUSEMOVE 667, 791
WM_MOUSEWHEEL 728
WM_NCDESTROY 777
WM_NCPAINT 793
WM_NOTIFY 176, 815
WM_ONERASEBKGND 776
WM_PAINT 58, 136, 208, 561, 760, 776, 784, 791
WM_PARENTNOTIFY 777
WM_QUIT 172, 560, 773
WM_SETTEXT 795
WM_SYSCOMMAND 783
WM_TCARD 435
WM_VSCROLL 730
Списки 367
недавно использованных файлов 55
панели инструментов Find 863
Способы запроса справочной информации 424
Справка
командная 437
по ключевому слову 436
по методике решения задачи 436
Стиль 209
CS_CLASSDC 542
CS_DBLCLKS 541
FWS_ADDTOTITLE 654, 656
IDC_ARROW 541
LBS_NOTIFY 679
LBS_SORT 680
LBS_WANTKEYBOARDINPUT 680
LVS_ALIGNLEFT 684
LVS_ALIGNTOP 684
LVS_AUTOARRANGE 684, 696
LVS_EDITLABELS 684
LVS_ICON 685
LVS_LIST 685
LVS_NOCOLUMNHEADER 685
LVS_NOLABELWRAP 685
LVS_NOSCROLL 685
LVS_NOSORTHEADER 685
LVS_OWNERDRAWFIXED 685
LVS_REPORT 685
LVS_SHOWSELALWAYS 685
LVS_SINGLESEL 685
LVS_SMALLICON 685
LVS_SORTASCENDING 685
LVS_SORTDESCENDING 685
LWS_SHAREIMAGELISTS 685
PBS_SMOOTH 714
PBS_VERTICAL 714
WS_BORDER 714
WS_CHILD 714, 782
WS_CLIPCHILDREN 714, 794
WS_CLIPSIBLINGS 714
WS_DISABLED 714, 779
WS_EX_NOPARENTNOTIFY 777
WS_EX_TOPMOST 568
WS_HSCROLL 730
WS_TABSTOP 714
WS_VISIBLE 629, 714
WS_VSCROLL 730
Строка состояния 288
Строковый ресурс
редактирование 306
шаблона документа 633
Структура
CCreateContext 606, 654
CHARFORMAT 286, 726, 798
CHOOSEFONT 213, 545, 546, 660
CREATESTRUCT 792
DLGTEMPLATE 627
DOCINFO 626
ENUMLOGFONT 554, 800
HDITEM 161, 659, 800
HELPINFO 425, 789, 803
HELPWININFO 435, 442
LOGFONT 214, 549, 552, 650, 803
LVITEM 809
MSG 560
NEWTEXTMETRIC 554, 811
NMHDR 151, 176, 814
OUTLINETEXTMETRIC 215, 815
PAINTSTRUCT 776, 818
POINT 819
RECT 565, 722, 819
SECURITY_ATTRIBUTES 547
SIZE 820
TEXTMETRIC 554
WNDCLASS 572, 781, 820
WNDCLASSEX 564, 582

T

Таблица
акселераторов 653
ключевых слов 434
Типы сносок в файле текста справки 470
Точка останова 501
Трансляция условная 526

У

Уровень доступа
EVENT_MODIFY_STATE 572, 574, 576
PROCESS_ALL_ACCESS 557
PROCESS_SET_INFORMATION 577
SYNCHRONIZE 584
THREAD_QUERY_INFORMATION 558
THREAD_SET_INFORMATION 578
Утечки памяти 514

Ф

Файл
afx.h 590, 644, 709, 744, 748, 749
afxcmn.h 659, 666, 687, 713, 736
afxcoll.h 705
afxcore.rtf 439, 473
afxdlgs.h 716, 718
afxext.h 605, 607, 641, 713, 741, 751
afxmt.h 704, 733
afxprint.rc 60, 445
afxprint.rtf 439, 489
afxres.rc 60, 445
afxrich.h 725
afxtempl.h 594, 672, 698
afxwin.h 597, 600, 602, 603, 608, 628, 632, 635, 640, 650, 653, 658, 680, 701, 712, 723, 727, 730, 735, 753, 761, 776
BrushAppView.cpp 230, 235
DDBAppView.cpp 238
DemoDlg.cpp 468
DemoDlg.h 467
DialogApplication.rc 69
DialogApplicationDlg.cpp 78, 110
DialogApplicationDlg.h 76
EditAppView.cpp 276
GalleryView.cpp 110
HelpApp.cnt 492
HelpApp.hpj 465, 493
HelpApp.rc 463
helpapp.rtf 494
HelpAppView.cpp 467
ListAppDlg.cpp 120
ListAppView.cpp 226
MDIApplication.cpp 50
MDIApplicationDoc.cpp 41
MDIApplicationDoc.h 40
MDIApplicationView.cpp 46
MDIApplicationView.h 44
NewHelp.hm 464, 469
OnSetActive 716
Page1.cpp 92
PrintAppDoc.cpp 313
PrintAppView.cpp 313, 321, 324
ProgressAppDlg.cpp 128, 133
Reflectdlg.h 194
Resource.h 60, 885
RichAppView.cpp 282, 285
Sheet.h 90
SheetApplication.cpp 91
StdAfx.cpp 17
StdAfx.h 17
TextApp.cpp 250, 263
TextApp.Doc 250
TextAppDoc.cpp 222, 247, 262
TextAppView.cpp 220, 223, 247, 260
Threads.cpp 393, 396, 410
ToolAppView.cpp 298, 308
WinGdi.h 210
Winhelp.exe 653
YellowEdit.cpp 192
mlistctrl.cpp 145
Resource.h 172
адресации справок 438
группового списка поиска 438
заголовка 524
заголовка справки 438
инициализации 267
конфигурации 438
поиска по всему тексту 438
проекта справочной системы 438
реализации 524
текста справки 438
Флаг
CF_ENABLEHOOK 545, 546
CREATE_SUSPENDED 773
CS_BYTEALIGNCLIENT 610
CS_BYTEALIGNWINDOW 610
CS_CLASSDC 781
CS_OWNDC 781
CS_PARENTDC 781
DEVICE_FONTTYPE 554
HDI_IMAGE 161
PSF_WIZARD 722
PSWIZB_BACK 97
PSWIZB_DISABLEFINISH 97
PSWIZB_FINISH 97
PSWIZB_NEXT 97
RASTER_FONTTYPE 554
RDW_ALLCHILDREN 794
RDW_ERASE 793
RDW_ERASENOW 794
RDW_FRAME 793
RDW_INTERNALPAINT 793
RDW_INVALIDATE 793
RDW_NOCHILDREN 794
RDW_NOERASE 793
RDW_NOFRAME 793

Флаг
RDW_NOINTERNALPAINT 793
RDW_UPDATENOW 794
RDW_VALIDATE 793
TRUETYPE_FONTTYPE 554
HDI_FORMAT 161
Фокус ввода 569
Форматирование текста справки в MS Word 495
Функция
Abort 258, 648
Add 157, 371, 594, 666, 705
AddDocTemplate 57, 761
AddFontResource 214, 537, 538
AddHead 672
AddPage 100, 718
AddString 123, 417, 680
AddTail 381, 405, 672
AddToListBox 414
AfxBeginThread 384, 418, 538, 772
AfxEnableControlContainer 55, 539
AfxEndThread 388, 539, 540
AfxGetApp 156, 254, 540, 761
AfxGetAppName 541, 761
AfxGetInstanceHandle 540, 761
AfxGetResourceHandle 541, 761
AfxMessageBox 440
AfxRegisterWndClass 541, 651
AfxThrowMemoryException 351
AfxWinInit 20, 542
AppendMenu 417, 701
ApproximateViewRect 169, 687
AssertValid 510, 709
Attach 390, 398, 608
BeginDrag 152, 667
BeginPaint 207, 319, 776
BitBlt 242, 609
CallWindowProc 582
CDumpContext 672
CFHookProc 544, 546, 664
CharFromPos 278, 640
ChooseFont 213, 545, 546
ClearTics 736
Close 258, 267, 591, 646
CloseHandle 402, 547, 549, 557, 571
CommandToIndex 310, 741
CommDlgExtendedError 546
Construct 719
ConstructElement 373
ContinueRouting 204, 304, 602
Create 57, 157, 310, 541, 653, 667, 688, 714, 736, 741
CreateCompatibleBitmap 242, 597
CreateCompatibleDC 611
CreateDC 318
CreateDragImage 152, 689
CreateEvent 396, 547, 571
CreateEx 302, 751
CreateFile 547
CreateFont 214, 650
CreateFontIndirect 214, 549, 650
CreateIndirect 628
CreatePointFont 650
CreatePointFontIndirect 650
CreateProcess 549, 571
CreateRectRgnIndirect 137, 728
CreateScalableFontResource 214
CreateThread 387, 539, 772
CreateWindow 558, 572
CreateWindowEx 425, 558
CToolBar 749
DDX_Control 679
DefWindowProc 424
Delete 348
DeleteCriticalSection 402, 550, 551, 566
DeleteObject 550, 658
DeleteString 123, 681
DestroyWindow 163, 629, 776
DestructElement 373
Detach 398, 400, 612
DispatchMessage 173, 195
DockControlBar 303, 654
DoDataExchange 85, 777, 850
DoDialogExchange 853
DoModal 278, 630, 720
DoPreparePrinting 335, 753
DoPromptFileName 264
DPtoLP 723
DragEnter 153, 669
DragLeave 153, 670
DragMove 153, 670
DragShowNolock 152, 670
DrawText 219
Dump 510, 710
DuplicateHandle 389, 549, 557, 571
ElementAt 372
Empty 414, 745
Enable 204, 259, 304, 601, 602
Enable3dControls 55, 761
Enable3dControlsStatic 55
EnableDocking 303, 605, 655
EnableWindow 158, 779
EndDialog 545
EndDoc 333, 334, 612
EndDrag 154, 671
EndPage 333, 334, 613
EndPaint 207, 779
EnterCriticalSection 400, 550, 565
EnumFontFamilies 213, 551, 618
EnumFontFamiliesEx 552

Функция
EnumFontFamProc 552, 554
EnumFonts 618
Escape 333
ExitInstance 387, 762
ExitThread 559
ExtTextOut 219, 220
FillRect 137, 241, 613
FillSolidRect 614
FindClose 547
FindFirstFile 547
FindResource 541
FindText 278, 641
Flush 591
Format 403, 745
FormatMessage 560
FreeLibrary 562
FromHandle 390, 622
GdiFlush 402, 555
GdiSetBatchLimit 556
GetAt 372, 377, 673
GetBitmapBits 242, 598
GetBkColor 218
GetBkMode 218
GetBuffer 160, 745
GetBufferLength 278, 642
GetButtonStyle 750
GetButtonText 750
GetCharABCWidths 218
GetCharABCWidthsFloat 218
GetCharFormatSelection 287, 726
GetCharWidth32 218
GetCharWidthFloat 218
GetClientRect 241, 243, 317, 397, 780
GetCommandLine 20, 556
GetCount 403
GetCurrentDirectory 254, 556
GetCurrentProcess 420, 557
GetCurrentTime 748
GetCurSel 123, 681
GetDC 207, 319, 780
GetDeviceCaps 219, 330, 331, 611, 612, 614
GetDlgItem 137, 158, 417, 419, 558, 781
GetDocString 253, 632
GetDocument 49, 317, 754
GetEditCtrl 278, 642
GetExitCodeThread 388, 419, 558
GetFile 257, 592, 635
GetFileName 266, 647
GetFilePath 266, 647
GetFileTitle 647
GetFontData 216
GetFromPage 335
GetHead 372, 673
GetHeaderCtrl 161, 690
GetHeadPosition 377, 379, 407, 674
GetHotCursor 157, 690
GetItem 161, 659
GetItemPosition 151, 691
GetItemText 154, 691
GetLastError 538, 547—549, 556, 558—560, 562, 563, 565, 570—578, 580, 581, 584, 585
GetLength 265, 647
GetMaxPage 335
GetMenuContextHelpId 432, 560
GetMessage 172, 195, 560
GetMinPage 335
GetModuleHandle 20, 562
GetNext 372, 377, 379, 407, 674
GetNextAssoc 373, 377, 698
GetObject 598
GetOrigin 152, 692
GetOutlineTextMetrics 215, 219
GetParent 781
GetParentFrame 403, 782
GetParentOwner 310, 782
GetPathName 257, 636
GetPos 132, 738
GetPrev 372, 675
GetPriorityClass 580
GetProcAddress 562
GetProfileInt 270
GetProfileString 254, 270, 763
GetSafeHdc 405, 618
GetSize 379, 595, 706
GetStartPosition 377, 699
GetState 419, 422, 600
GetStockObject 613
GetStyle 304, 783
GetSystemMenu 417, 783
GetTabbedTextExtent 218
GetTail 372, 676
GetTailPosition 676
GetText 123, 681
GetTextAlign 217
GetTextColor 218
GetTextExtent 310, 619
GetTextExtentExPoint 218
GetTextExtentPoint32 217, 218
GetTextMetrics 218
GetThreadPriority 580
GetToPage 335
GetUpperBound 595, 707
GetWindowContextHelpId 432, 563
GetWindowDC 319
GetWindowLong 153, 163, 165, 563
GetWindowRect 137, 159, 565, 784
GetWindowText 638
HitTest 153, 692
InitializeCriticalSection 551, 565

Функция
InitializeCriticalSectionAndSpinCount 551, 565
InitInstance 54, 254, 263, 387, 397, 539, 603, 764
InitModalIndirect 629
InsertColumn 159, 693
InsertItem 160, 694
InsertString 682
Invalidate 258, 317, 784
InvalidateRect 164, 785
IsChild 561
IsDialogMessage 443
IsDlgButtonChecked 413, 785
IsEmpty 417, 677, 699, 747
IsLoading 592
IsLocked 734
IsPrinting 330, 619, 755
IsStoring 44, 249, 592
IsWindowUnicode 573
LeaveCriticalSection 400, 550, 566
LoadBarState 303, 656
LoadCursor 157, 764
LoadFrame 57, 656
LoadIcon 157, 765
LoadLibrary 562
LoadResource 562
LoadStdProfileSettings 56, 270, 765
LoadString 343, 403, 417, 747
LoadToolbar 302, 752
Lock 391, 413, 734
Lookup 373, 378, 699
LPtoDP 723
Main 20
MakeReverse 414, 747
MessageBox 257, 343, 567
ModifyStyle 164, 786
ModifyStyleEx 163, 786
MsgWaitForMultipleObjects 583
NormalizeRect 723
OffsetRect 401, 724
OnActivateView 652
OnBeginPrinting 324, 328, 334, 754
OnCancel 630
OnCharEffect 283, 726
OnClose 297, 418
OnCommandHelp 787
OnContextHelp 443
OnContextMenu 469, 788
OnCreate 296, 302, 307, 310, 408
OnDblclkList 123
OnDeltaposSpin 85
OnDraw 49, 207, 239, 314, 321, 755
OnEndPrinting 334
OnFindNext 278, 643
OnHelp 440, 441, 766
OnHelpHitTest 788
OnHelpIndex 441, 444
OnHelpInfo 469, 789
OnHScroll 132, 789
OnIdle 443
OnInitDialog 131, 415, 631
OnInitialUpdate 755
OnMouseMove 790
OnMouseWheel 728
OnMyToolbarCheck 298, 304
OnNewDocument 43, 313, 636
OnNotify 178
OnOK 630
OnPaint 136, 208, 791
OnParaAlign 283, 727
OnPrepareDC 207, 325, 330, 332, 334, 756
OnPreparePrinting 333, 757
OnPrint 327, 331, 334, 758
OnQueryEndSession 653
OnRegisteredMouseWheel 728
OnSetActive 101
OnTextNotFound 643
OnUpdate 208, 759
OnUpdateMyToolbar 297
OnWizardBack 100, 716
OnWizardFinish 100, 717
OnWizardNext 100, 717
OnWndMessage 195
OpenEvent 549, 570
OpenProcess 557
ParseCommandLine 57, 603, 766
ParseParam 603
PeekMessage 172
PostMessage 390, 545, 575
PostNcDestroy 628
PostQuitMessage 388
PostThreadMessage 172, 561
PreCreateWindow 49, 792
PreTranslateMessage 387, 443
ProcessShellCommand 57, 603, 767
ProcessWndProcException 387
PtInRect 242, 724
PulseEvent 571, 574
Read 258, 265, 592, 647
ReadObject 590
Rectangle 317, 619
RectVisible 755
RedrawWindow 792
RegisterClassEx 564, 582
RegisterClass 572
RegisterShellFileTypes 768
RegisterWindowMessage 575
ReleaseCapture 155, 573
ReleaseDC 207, 794
ReleaseFile 258, 636, 648

Функция
Remove 267
RemoveAll 379, 596, 678, 700, 708
RemoveAt 379
RemoveFontResource 214, 538, 573
RemoveHead 372, 381, 407, 677
RemoveKey 380
RemoveTail 372, 677
ResetDC 626
ResetEvent 548, 549, 574, 576
ResumeThread 389, 405, 418, 773
Run 195, 387, 773
SaveBarState 303, 657
ScreenToClient 137, 319, 795
Seek 648
SeekToBegin 265, 649
SelectClipRgn 137, 620
SelectObject 215, 221, 242, 550, 622
SendMessage 214, 538, 574, 844
Serialize 44, 247, 249, 250, 313, 316, 711
SerializeElement 374
SerializeRaw 643
SetActivePage 99, 100
SetAt 678
SetBitmapBits 243, 598
SetBkColor 217
SetBkMode 217
SetButtonStyle 750
SetButtonText 750
SetCapture 573
SetCharFormat 287, 727
SetCheck 158, 204, 303, 419, 600—602
SetCurrentDirectory 254, 575
SetCurSel 417, 683
SetDialogBkColor 628
SetDlgItemText 85, 123, 795
SetDragCursorImage 152, 671
SetEvent 407, 549, 574, 576
SetExtendedStyle 163, 167, 695
SetFileAttributes 266
SetFinishText 100, 721
SetForegroundWindow 569
SetHotCursor 167
SetImageList 161, 659
SetIndicators 310, 742
SetItem 160, 161, 660, 695
SetItemPosition 155, 170, 696
SetItemText 154, 697
SetLastError 559, 581
SetLineSize 132, 738
SetMapMode 331, 623
SetMapperFlags 215
SetMaxPage 330, 335, 713
SetMenu 700
SetMenuContextHelpId 432, 577
SetModifiedFlag 208, 249, 258, 637, 638
SetOwner 418
SetPageSize 132, 738
SetPaneInfo 310, 742
SetPaneText 310, 743
SetParent 583
SetPos 132, 715, 739
SetPriorityClass 420, 577
SetRadio 204, 601, 602
SetRange 132, 715, 739
SetRect 169, 725
SetRegistryKey 55, 269, 768
SetScaleToFitSize 729
SetScrollPos 795
SetScrollSizes 317, 730
SetSel 123, 683
SetSize 594, 596, 708
SetTabStops 642
SetText 204, 601, 602
SetTextAlign 217
SetTextCharacterExtra 217
SetTextColor 217
SetTextJustification 217
SetThreadPriority 385, 387, 399, 405, 422, 578, 774
SetTic 739
SetTickFreq 132, 740
SetViewportExt 623
SetViewPortOrg 331, 624
SetWindowContextHelpId 432, 580
SetWindowExt 623
SetWindowLong 165, 302, 581
SetWindowPos 163, 582
SetWindowText 403, 638, 796
SetWizardButtons 101, 721
SetWizardMode 722
SetWorkAreas 163, 169, 697
ShowControlBar 304, 657
ShowCursor 671
ShowWindow 58, 629, 796
Sleep 414, 583
StartDoc 333, 334, 612, 625
StartPage 332, 333, 334, 626
StretchBlt 611
SuspendThread 422, 774
TabbedTextOut 219
TerminateThread 559
TextOut 219
TranslateMessage 195
TryEnterCriticalSection 550, 565
Unlock 391, 735
UpdateAllViews 207, 390, 637
UpdateData 85, 797, 849
UpdateWindow 58, 798
WaitForSingleObject 398, 407, 583

Функция
WindowProc 195, 585
WinHelp 425, 429, 441, 585, 769
WinMain 20, 172, 384, 543
WndProc 195
Write 264, 265, 593, 649
WriteObject 591
WriteProfileInt 270
WriteProfileString 253, 270, 770
виртуальная 523, 534
встраиваемая 50
дружественная 534
обработки сообщения 171, 599
оператор [] 372, 377, 595, 700, 707
оператор << 249, 591
оператор >> 249, 590

Ц

Цикл обработки сообщений 172, 384

Ш

Шаблон
документа 39, 56, 631
класса 364
функции 362
Шрифт 209, 215

Э

Экранные координаты 318
Элемент управления
Инкрементный регулятор 63
Переключатель 67
Рамка группы 67
Раскрывающийся список 65
Статический текст 62
Текстовое поле 63
Флажок 67
Элементы данных обратного вызова 686

Описание дискеты

Все примеры сведены в самораспаковывающийся архив Samples.exe. Каждый пример хранится в отдельном каталоге и представляет собой полноценный проект, который можно загружать в среду разработки и компилировать.

Имя каждого каталога совпадает с именем содержащегося в нем проекта. Ниже приведен список проектов с указанием глав, к которым они относятся:

- BrushApp — приложение, демонстрирующее принципы работы с кистью
- Console — консольное приложение. Описывается в главе 1
- DDBApp — приложение, демонстрирующее принципы работы с битовыми образами
- DialogApplication — диалоговое приложение. Описывается в главах 1 и 3
- EditApp — простейший текстовый редактор
- Gallery — приложение, демонстрирующее принципы работы с галереей компонентов (Component Gallery)
- HelpApp — приложение, демонстрирующее принципы работы со справочной системой приложения
- LineApp — приложение, демонстрирующее принципы работы с перьями
- MDIApplication — многооконное приложение. Описывается в главах 1 и 2
- PrintApp — приложение, демонстрирующее принципы прокрутки изображения в окне и вывода информации на печатающее устройство
- ProgressApp — приложение, демонстрирующее принципы работы с линейным индикатором и линейным регулятором
- Reflect — приложение, демонстрирующее принципы работы с отраженными сообщениями
- RichApp — текстовый редактор с расширенными возможностями
- SheetApplication — приложение, демонстрирующее принципы работы с вкладками и мастерами. Описывается в главе 3
- TextApp — приложение, демонстрирующее принципы вывода текстовой информации на экран
- ToolApp — приложение, демонстрирующее принципы работы с панелью инструментов и со строкой состояния